

PathFinder

위치 기반 러닝 코스 공유 및 경쟁 앱

전체 소스코드 모음

총 파일 수: 411개 | 총 코드 라인: 41,662줄

생성일: 2026-06-10

작성자: LEE SANGJUN

목차

Backend (125개 파일)

backend/src/main/java/com/runway/RunwayApplication.java	12줄	backend/src/main/java/com/runway/course/domain/enums/CourseRecommender.java	25줄
backend/src/main/java/com/runway/attempt/controller/CourseAttemptController.java	94줄	backend/src/main/java/com/runway/course/domain/enums/CourseReportReason.java	32줄
backend/src/main/java/com/runway/attempt/domain/CourseAttempt.java	91줄	backend/src/main/java/com/runway/course/domain/enums/CourseReportStatus.java	31줄
backend/src/main/java/com/runway/attempt/domain/enums/AttemptVerificationStatus.java	44줄	backend/src/main/java/com/runway/course/domain/enums/CourseRiskLevel.java	25줄
backend/src/main/java/com/runway/attempt/domain/enums/CourseAttemptStatus.java	44줄	backend/src/main/java/com/runway/course/domain/enums/CourseSlopeLevel.java	25줄
backend/src/main/java/com/runway/attempt/dto/AbandonAttemptRequest.java	10줄	backend/src/main/java/com/runway/course/domain/enums/CourseStatus.java	44줄
backend/src/main/java/com/runway/attempt/dto/AbandonAttemptResponse.java	31줄	backend/src/main/java/com/runway/course/domain/enums/CourseSurfaceType.java	25줄
backend/src/main/java/com/runway/attempt/dto/CourseAttemptResponse.java	37줄	backend/src/main/java/com/runway/course/dto/CourseDetailResponse.java	90줄
backend/src/main/java/com/runway/attempt/dto/FinishAttemptRequest.java	26줄	backend/src/main/java/com/runway/course/dto/CoursePointResponse.java	24줄
backend/src/main/java/com/runway/attempt/dto/FinishAttemptResponse.java	62줄	backend/src/main/java/com/runway/course/dto/CoursePointsResponse.java	15줄
backend/src/main/java/com/runway/attempt/dto/LeaderboardItem.java	17줄	backend/src/main/java/com/runway/course/dto/CourseRatingRequest.java	17줄
backend/src/main/java/com/runway/attempt/dto/LeaderboardResponse.java	22줄	backend/src/main/java/com/runway/course/dto/CourseRatingResponse.java	31줄
backend/src/main/java/com/runway/attempt/dto/MyBestAttemptResponse.java	18줄	backend/src/main/java/com/runway/course/dto/CourseReportRequest.java	15줄
backend/src/main/java/com/runway/attempt/dto/StartAttemptRequest.java	10줄	backend/src/main/java/com/runway/course/dto/CourseReportResponse.java	27줄
backend/src/main/java/com/runway/attempt/dto/StartAttemptResponse.java	31줄	backend/src/main/java/com/runway/course/dto/CourseResponse.java	47줄
backend/src/main/java/com/runway/attempt/repository/CourseAttemptRepository.java	41줄	backend/src/main/java/com/runway/course/dto/CourseStatusResponse.java	22줄
backend/src/main/java/com/runway/attempt/service/CourseAttemptService.java	389줄	backend/src/main/java/com/runway/course/dto/CreateCourseFromRunRequest.java	11줄
backend/src/main/java/com/runway/auth/controller/AuthController.java	20줄	backend/src/main/java/com/runway/course/dto/GeoPoint.java	20줄
backend/src/main/java/com/runway/auth/dto/LoginRequest.java	57줄	backend/src/main/java/com/runway/course/dto/NearbyCourseItem.java	25줄
backend/src/main/java/com/runway/auth/dto/LoginResponse.java	40줄	backend/src/main/java/com/runway/course/dto/ParticipatedCourseItem.java	23줄
backend/src/main/java/com/runway/auth/dto/LogoutRequest.java	15줄	backend/src/main/java/com/runway/course/dto/PublishCourseRequest.java	30줄
backend/src/main/java/com/runway/auth/dto/ReissueRequest.java	15줄	backend/src/main/java/com/runway/course/dto/UpdateCourseRequest.java	18줄
backend/src/main/java/com/runway/auth/dto/ReissueResponse.java	21줄	backend/src/main/java/com/runway/course/repository/CourseFavoriteRepository.java	14줄
backend/src/main/java/com/runway/auth/dto/SignupRequest.java	26줄	backend/src/main/java/com/runway/course/repository/CoursePointRepository.java	14줄
backend/src/main/java/com/runway/auth/dto/SignupResponse.java	31줄	backend/src/main/java/com/runway/course/repository/CourseRatingRepository.java	19줄
backend/src/main/java/com/runway/auth/service/AuthService.java	95줄	backend/src/main/java/com/runway/course/repository/CourseReportRepository.java	11줄
backend/src/main/java/com/runway/common/config/WebConfig.java	19줄	backend/src/main/java/com/runway/course/repository/CourseRepository.java	24줄
backend/src/main/java/com/runway/common/exception/ErrorCode.java	47줄	backend/src/main/java/com/runway/course/service/CourseRatingService.java	50줄
backend/src/main/java/com/runway/common/exception/GlobalExceptionHandler.java	69줄	backend/src/main/java/com/runway/course/service/CourseReportService.java	50줄
backend/src/main/java/com/runway/common/exception/RunwayException.java	19줄	backend/src/main/java/com/runway/course/service/CourseService.java	591줄
backend/src/main/java/com/runway/common/response/ApiResponse.java	35줄	backend/src/main/java/com/runway/course/util/CoursePrivacyUtils.java	64줄
backend/src/main/java/com/runway/common/response/PageResponse.java	36줄	backend/src/main/java/com/runway/run/controller/RunningController.java	140줄
backend/src/main/java/com/runway/common/security/CustomUserDetailsService.java	28줄	backend/src/main/java/com/runway/run/domain/RunningPoint.java	53줄
backend/src/main/java/com/runway/common/security/JwtAuthenticationFilter.java	53줄	backend/src/main/java/com/runway/run/domain/RunningRecord.java	120줄
backend/src/main/java/com/runway/common/security/JwtProperties.java	17줄	backend/src/main/java/com/runway/run/domain/enums/RunningRecordStatus.java	45줄
backend/src/main/java/com/runway/common/security/JwtProvider.java	118줄	backend/src/main/java/com/runway/run/dto/FinishRunRequest.java	28줄
backend/src/main/java/com/runway/common/security/SecurityConfig.java	84줄	backend/src/main/java/com/runway/run/dto/FinishRunResponse.java	37줄
backend/src/main/java/com/runway/common/security/UserPrincipal.java	57줄	backend/src/main/java/com/runway/run/dto/PersonalRecordItemResponse.java	31줄
backend/src/main/java/com/runway/course/controller/CourseController.java	179줄	backend/src/main/java/com/runway/run/dto/PersonalRecordsResponse.java	17줄
backend/src/main/java/com/runway/course/domain/Course.java	200줄	backend/src/main/java/com/runway/run/dto/RunDetailResponse.java	39줄
backend/src/main/java/com/runway/course/domain/CourseFavorite.java	40줄	backend/src/main/java/com/runway/run/dto/RunPointResponse.java	31줄
backend/src/main/java/com/runway/course/domain/CourseFavoriteId.java	16줄	backend/src/main/java/com/runway/run/dto/RunStatusResponse.java	22줄
backend/src/main/java/com/runway/course/domain/CoursePoint.java	42줄	backend/src/main/java/com/runway/run/dto/RunSummaryResponse.java	35줄
backend/src/main/java/com/runway/course/domain/CourseRating.java	66줄	backend/src/main/java/com/runway/run/dto/RunTrimRequest.java	10줄
backend/src/main/java/com/runway/course/domain/CourseReport.java	52줄	backend/src/main/java/com/runway/run/dto/RunningStatsResponse.java	
backend/src/main/java/com/runway/course/domain/enums/CourseDifficulty.java			

backend/src/main/java/com/runway/run/dto/SavePointsRequest.java	24줄 40줄	android/app/src/main/java/com/runway/android/core/model/PageResponse.kt	14줄 16줄
backend/src/main/java/com/runway/run/dto/SavePointsResponse.java	14줄	android/app/src/main/java/com/runway/android/core/network/AuthInterceptor.kt	29줄
backend/src/main/java/com/runway/run/dto/StartRunRequest.java	12줄	android/app/src/main/java/com/runway/android/core/network/NetworkConstants.	15줄
backend/src/main/java/com/runway/run/dto/StartRunResponse.java	25줄	android/app/src/main/java/com/runway/android/core/network/TokenAuthenticator	94줄
backend/src/main/java/com/runway/run/repository/RunDateProjection.java	7줄	android/app/src/main/java/com/runway/android/core/notification/BootReceiver.kt	36줄
backend/src/main/java/com/runway/run/repository/RunSummaryProjection.java	8줄	android/app/src/main/java/com/runway/android/core/notification/NotificationChar	29줄
backend/src/main/java/com/runway/run/repository/RunningPointBatchInserter.java	40줄	android/app/src/main/java/com/runway/android/core/notification/ReminderReceiv	62줄
backend/src/main/java/com/runway/run/repository/RunningPointRepository.java	45줄	android/app/src/main/java/com/runway/android/core/notification/ReminderSched	99줄
backend/src/main/java/com/runway/run/repository/RunningRecordRepository.java	100줄	android/app/src/main/java/com/runway/android/core/posture/AnkleKalmanFilter.k	112줄
backend/src/main/java/com/runway/run/repository/RunningStatsProjection.java	10줄	android/app/src/main/java/com/runway/android/core/posture/LegSwapCorrector.	140줄
backend/src/main/java/com/runway/run/service/RunningService.java	376줄	android/app/src/main/java/com/runway/android/core/posture/PostureAngleCalcul	155줄
backend/src/main/java/com/runway/user/controller/UserController.java	92줄68줄	android/app/src/main/java/com/runway/android/core/posture/PostureAutoencode	19줄
backend/src/main/java/com/runway/user/domain/User.java	19줄	android/app/src/main/java/com/runway/android/core/posture/PostureCategoryRe	11줄
backend/src/main/java/com/runway/user/dto/AchievementItemResponse.java	13줄	android/app/src/main/java/com/runway/android/core/posture/PostureEvaluator.kt	5줄
backend/src/main/java/com/runway/user/dto/AchievementsResponse.java	16줄	android/app/src/main/java/com/runway/android/core/posture/PostureFrameAngle	33줄
backend/src/main/java/com/runway/user/dto/UpdateProfileRequest.java	31줄	android/app/src/main/java/com/runway/android/core/posture/PostureFrameInterp	44줄
backend/src/main/java/com/runway/user/dto/UserProfileResponse.java	20줄	android/app/src/main/java/com/runway/android/core/posture/PostureLandmarkC	197줄
backend/src/main/java/com/runway/user/repository/UserRepository.java	247줄	android/app/src/main/java/com/runway/android/core/posture/PostureLandmarkVt	124줄
backend/src/main/java/com/runway/user/service/UserService.java	14줄	android/app/src/main/java/com/runway/android/core/posture/PosturePoseAnalyz	249줄
backend/src/main/resources/db/migration/V10__create_course_favorites.sql	19줄	android/app/src/main/java/com/runway/android/core/posture/PostureResult.kt	25줄
backend/src/main/resources/db/migration/V11__add_course_publishing_metadata.sql	761줄	android/app/src/main/java/com/runway/android/core/posture/PostureRuleEngine.	494줄
backend/src/main/resources/db/migration/V12__seed_demo_courses.sql	51줄	android/app/src/main/java/com/runway/android/core/posture/PostureSkeletonSrr	123줄
backend/src/main/resources/db/migration/V13__densify_seeded_course_points.sql	492줄	android/app/src/main/java/com/runway/android/core/posture/PostureVideoFrame	49줄
backend/src/main/resources/db/migration/V14__replace_courses_with_real_osm_routes.sql	230줄	android/app/src/main/java/com/runway/android/core/posture/RunningFormMetric	53줄
backend/src/main/resources/db/migration/V15__seed_chungju_gps_courses.sql	20줄7줄	android/app/src/main/java/com/runway/android/core/posture/RunningFormStrike	78줄
backend/src/main/resources/db/migration/V1__init_extensions.sql	31줄	android/app/src/main/java/com/runway/android/core/posture/local/PostureAnalys	29줄
backend/src/main/resources/db/migration/V2__create_users.sql	19줄	android/app/src/main/java/com/runway/android/core/posture/local/PostureAnalys	50줄
backend/src/main/resources/db/migration/V3__create_running_records.sql	43줄	android/app/src/main/java/com/runway/android/core/posture/local/PostureDatab	36줄
backend/src/main/resources/db/migration/V4__create_running_points.sql	17줄	android/app/src/main/java/com/runway/android/core/result/NetworkResult.kt	103줄
backend/src/main/resources/db/migration/V5__create_courses.sql	44줄	android/app/src/main/java/com/runway/android/core/running/RunChartCalculator	67줄
backend/src/main/resources/db/migration/V6__create_course_points.sql	12줄	android/app/src/main/java/com/runway/android/core/running/RunChartPoint.kt	6줄
backend/src/main/resources/db/migration/V7__create_course_attempts.sql	12줄	android/app/src/main/java/com/runway/android/core/running/RunSplit.kt	8줄
backend/src/main/resources/db/migration/V8__create_course_reports.sql	12줄	android/app/src/main/java/com/runway/android/core/running/SplitCalculator.kt	99줄
backend/src/main/resources/db/migration/V9__create_course_ratings.sql	12줄	android/app/src/main/java/com/runway/android/core/share/ImageCaptureUtil.kt	203줄
Android (269개 파일)		android/app/src/main/java/com/runway/android/core/share/ImageShareUtil.kt	33줄
android/app/build.gradle.kts	153줄	android/app/src/main/java/com/runway/android/core/share/MetricPreset.kt	7줄
android/app/src/main/java/com/runway/android/MainActivity.kt	31줄	android/app/src/main/java/com/runway/android/core/share/ShareTemplate.kt	45줄
android/app/src/main/java/com/runway/android/RunwayApplication.kt	18줄	android/app/src/main/java/com/runway/android/core/share/ShareUtils.kt	63줄
android/app/src/main/java/com/runway/android/core/cadence/CadenceTracker.kt	7줄	android/app/src/main/java/com/runway/android/core/tracking/PauseReason.kt	7줄
android/app/src/main/java/com/runway/android/core/cadence/PhoneStepCadenceTracker.kt	72줄	android/app/src/main/java/com/runway/android/core/tracking/PendingPointQueu	47줄
android/app/src/main/java/com/runway/android/core/datastore/OnboardingDataStore.kt	26줄	android/app/src/main/java/com/runway/android/core/tracking/RunTrackingAction	7줄
android/app/src/main/java/com/runway/android/core/datastore/ThemeDataStore.kt	45줄	android/app/src/main/java/com/runway/android/core/tracking/RunTrackingManag	262줄
android/app/src/main/java/com/runway/android/core/datastore/TokenDataStore.kt	66줄	android/app/src/main/java/com/runway/android/core/tracking/RunTrackingMode.	3줄
android/app/src/main/java/com/runway/android/core/location/DefaultLocationTracker.kt	52줄	android/app/src/main/java/com/runway/android/core/tracking/RunTrackingNotific	78줄
android/app/src/main/java/com/runway/android/core/location/DistanceCalculator.kt	28줄	android/app/src/main/java/com/runway/android/core/tracking/RunTrackingServic	94줄
android/app/src/main/java/com/runway/android/core/location/GpsPointValidator.kt	31줄	android/app/src/main/java/com/runway/android/core/tracking/RunTrackingState.k	18줄
android/app/src/main/java/com/runway/android/core/location/GpsStatus.kt	7줄		
android/app/src/main/java/com/runway/android/core/location/LocationTracker.kt	9줄		
android/app/src/main/java/com/runway/android/core/location/RunwayLocation.kt	11줄		
android/app/src/main/java/com/runway/android/core/map/MapBoundsUtil.kt	19줄		
android/app/src/main/java/com/runway/android/core/map/MapPoint.kt	6줄		
android/app/src/main/java/com/runway/android/core/model/ApiResponse.kt			

android/app/src/main/java/com/runway/android/core/tracking/TrackingSessionSnapshot.kt	15줄
8줄	android/app/src/main/java/com/runway/android/data/course/model/CourseStatus
android/app/src/main/java/com/runway/android/core/tracking/TrackingSessionStore.kt	6줄
57줄	android/app/src/main/java/com/runway/android/data/course/model/CreateCourse
android/app/src/main/java/com/runway/android/core/tracking/local/PendingRunPointDao.kt	8줄
22줄	android/app/src/main/java/com/runway/android/data/course/model/GeoPoint.kt
android/app/src/main/java/com/runway/android/core/tracking/local/PendingRunPointEntity.kt	6줄
20줄	android/app/src/main/java/com/runway/android/data/course/model/NearbyCourse
android/app/src/main/java/com/runway/android/core/tracking/local/TrackingDatabase.kt	19줄
13줄	android/app/src/main/java/com/runway/android/data/course/model/ParticipatedCourse
android/app/src/main/java/com/runway/android/core/util/RunFormatUtils.kt	14줄
52줄	android/app/src/main/java/com/runway/android/data/course/model/PublishCourse
android/app/src/main/java/com/runway/android/core/voice/RunningVoiceGuide.kt	11줄
118줄	android/app/src/main/java/com/runway/android/data/course/remote/CourseApi.kt
android/app/src/main/java/com/runway/android/core/wear/WatchCommand.kt	114줄
72줄	android/app/src/main/java/com/runway/android/data/reminder/ReminderPrefs.kt
android/app/src/main/java/com/runway/android/core/wear/WatchCommandListenerService.kt	14줄
129줄	android/app/src/main/java/com/runway/android/data/reminder/ReminderPrefsRepository.kt
android/app/src/main/java/com/runway/android/core/wear/WatchCourseSyncCoordinator.kt	49줄
117줄	android/app/src/main/java/com/runway/android/data/running/RunningRepository.kt
android/app/src/main/java/com/runway/android/core/wear/WatchNavRepository.kt	64줄
34줄	android/app/src/main/java/com/runway/android/data/running/model/FinishRunRecord
android/app/src/main/java/com/runway/android/core/wear/WatchRunGoal.kt	10줄
29줄	android/app/src/main/java/com/runway/android/data/running/model/FinishRunRecord
android/app/src/main/java/com/runway/android/core/wear/WatchRunUploadCoordinator.kt	12줄
163줄	android/app/src/main/java/com/runway/android/data/running/model/PersonalRecord
android/app/src/main/java/com/runway/android/core/wear/WatchStateSender.kt	10줄
91줄	android/app/src/main/java/com/runway/android/data/running/model/PersonalRecord
android/app/src/main/java/com/runway/android/core/wear/WearRunSessionCoordinator.kt	11줄
354줄	android/app/src/main/java/com/runway/android/data/running/model/RunDetailRecord
android/app/src/main/java/com/runway/android/data/attempt/CourseAttemptRepositoryImpl.kt	14줄
51줄	android/app/src/main/java/com/runway/android/data/running/model/RunPointRecord
android/app/src/main/java/com/runway/android/data/attempt/model/AbandonAttemptRequest.kt	10줄
3줄	android/app/src/main/java/com/runway/android/data/running/model/RunPointRecord
android/app/src/main/java/com/runway/android/data/attempt/model/AbandonAttemptResponse.kt	10줄
8줄	android/app/src/main/java/com/runway/android/data/running/model/RunStatusRecord
android/app/src/main/java/com/runway/android/data/attempt/model/CourseAttemptResponse.kt	6줄
13줄	android/app/src/main/java/com/runway/android/data/running/model/RunSummary
android/app/src/main/java/com/runway/android/data/attempt/model/FinishAttemptRequest.kt	12줄
10줄	android/app/src/main/java/com/runway/android/data/running/model/RunningStatistics
android/app/src/main/java/com/runway/android/data/attempt/model/FinishAttemptResponse.kt	16줄
18줄	android/app/src/main/java/com/runway/android/data/running/model/SavePointsRecord
android/app/src/main/java/com/runway/android/data/attempt/model/LeaderboardItem.kt	5줄
10줄	android/app/src/main/java/com/runway/android/data/running/model/SavePointsRecord
android/app/src/main/java/com/runway/android/data/attempt/model/LeaderboardResponse.kt	6줄
8줄	android/app/src/main/java/com/runway/android/data/running/model/StartRunRecord
android/app/src/main/java/com/runway/android/data/attempt/model/MyBestAttemptResponse.kt	5줄
7줄	android/app/src/main/java/com/runway/android/data/running/model/StartRunRecord
android/app/src/main/java/com/runway/android/data/attempt/model/StartAttemptRequest.kt	7줄
3줄	android/app/src/main/java/com/runway/android/data/running/model/TrimRunRecord
android/app/src/main/java/com/runway/android/data/attempt/model/StartAttemptResponse.kt	5줄
10줄	android/app/src/main/java/com/runway/android/data/running/remote/RunningApi
android/app/src/main/java/com/runway/android/data/attempt/remote/CourseAttemptApi.kt	76줄
50줄	android/app/src/main/java/com/runway/android/data/user/UserRepositoryImpl.kt
android/app/src/main/java/com/runway/android/data/auth/AuthRepositoryImpl.kt	44줄
73줄	android/app/src/main/java/com/runway/android/data/user/model/AchievementItem
android/app/src/main/java/com/runway/android/data/auth/model/LoginRequest.kt	11줄
7줄	android/app/src/main/java/com/runway/android/data/user/model/AchievementsRecord
android/app/src/main/java/com/runway/android/data/auth/model/LoginResponse.kt	5줄
10줄	android/app/src/main/java/com/runway/android/data/user/model/UpdateProfileRequest
android/app/src/main/java/com/runway/android/data/auth/model/LogoutRequest.kt	7줄
6줄	android/app/src/main/java/com/runway/android/data/user/model/UserProfileResponse
android/app/src/main/java/com/runway/android/data/auth/model/ReissueRequest.kt	10줄
6줄	android/app/src/main/java/com/runway/android/data/user/remote/UserApi.kt
android/app/src/main/java/com/runway/android/data/auth/model/ReissueResponse.kt	29줄
9줄	android/app/src/main/java/com/runway/android/di/AppModule.kt
android/app/src/main/java/com/runway/android/data/auth/model/SignupRequest.kt	26줄
8줄	android/app/src/main/java/com/runway/android/di/DataStoreModule.kt
android/app/src/main/java/com/runway/android/data/auth/model/SignupResponse.kt	58줄
11줄	android/app/src/main/java/com/runway/android/di/DatabaseModule.kt
android/app/src/main/java/com/runway/android/data/auth/model/UserSummary.kt	33줄
9줄	android/app/src/main/java/com/runway/android/di/LocationModule.kt
android/app/src/main/java/com/runway/android/data/auth/remote/AuthApi.kt	31줄
35줄	android/app/src/main/java/com/runway/android/di/NetworkModule.kt
android/app/src/main/java/com/runway/android/data/course/CourseRepositoryImpl.kt	138줄
112줄	android/app/src/main/java/com/runway/android/di/PostureModule.kt
android/app/src/main/java/com/runway/android/data/course/model/CourseCreatorResponse.kt	44줄
7줄	android/app/src/main/java/com/runway/android/di/RepositoryModule.kt
android/app/src/main/java/com/runway/android/data/course/model/CourseDetailResponse.kt	42줄
26줄	android/app/src/main/java/com/runway/android/di/TrackingModule.kt
8줄	android/app/src/main/java/com/runway/android/domain/attempt/CourseAttemptFactory
android/app/src/main/java/com/runway/android/data/course/model/CoursePointsResponse.kt	18줄
6줄	android/app/src/main/java/com/runway/android/domain/auth/AuthRepository.kt
android/app/src/main/java/com/runway/android/data/course/model/CourseRatingRequest.kt	38줄
6줄	android/app/src/main/java/com/runway/android/domain/course/CourseRepository.kt
android/app/src/main/java/com/runway/android/data/course/model/CourseRatingResponse.kt	66줄
10줄	android/app/src/main/java/com/runway/android/domain/running/RunningRepository.kt
android/app/src/main/java/com/runway/android/data/course/model/CourseReportRequest.kt	30줄
6줄	android/app/src/main/java/com/runway/android/domain/user/UserRepository.kt
8줄	android/app/src/main/java/com/runway/android/ui/MainViewModel.kt
android/app/src/main/java/com/runway/android/data/course/model/CourseResponse.kt	47줄
android/app/src/main/java/com/runway/android/data/course/model/CourseResponse.kt	47줄
android/app/src/main/java/com/runway/android/data/course/model/CourseResponse.kt	47줄

	161줄	android/app/src/main/java/com/runway/android/ui/course/my/MyCoursesScreen.kt	
android/app/src/main/java/com/runway/android/ui/achievements/AchievementsViewModel.kt	242줄		
	57줄	android/app/src/main/java/com/runway/android/ui/course/my/MyCoursesViewMc	
android/app/src/main/java/com/runway/android/ui/attempt/CourseAttemptTrackingScreen.kt	60줄		
	729줄	android/app/src/main/java/com/runway/android/ui/discover/DiscoverCourseMap.kt	
android/app/src/main/java/com/runway/android/ui/attempt/CourseAttemptTrackingViewModel.kt	745줄		
	548줄	android/app/src/main/java/com/runway/android/ui/discover/DiscoverScreen.kt	
android/app/src/main/java/com/runway/android/ui/auth/login/LoginScreen.kt	767줄		
	153줄	android/app/src/main/java/com/runway/android/ui/discover/DiscoverViewModel.kt	
android/app/src/main/java/com/runway/android/ui/auth/login/LoginViewModel.kt	276줄		
	59줄	android/app/src/main/java/com/runway/android/ui/home/AirKoreaClient.kt	
android/app/src/main/java/com/runway/android/ui/auth/signup/SignupScreen.kt	149줄		
	152줄	android/app/src/main/java/com/runway/android/ui/home/GoalSetupSheet.kt	
android/app/src/main/java/com/runway/android/ui/auth/signup/SignupViewModel.kt	863줄		
	66줄	android/app/src/main/java/com/runway/android/ui/home/HomeScreen.kt	
android/app/src/main/java/com/runway/android/ui/components/AchievementCard.kt	277줄		
	129줄	android/app/src/main/java/com/runway/android/ui/home/HomeViewModel.kt	
android/app/src/main/java/com/runway/android/ui/components/BatteryOptimizationCard.kt	419줄		
	119줄	android/app/src/main/java/com/runway/android/ui/home/RunGoal.kt	70줄
android/app/src/main/java/com/runway/android/ui/components/ConfettiCanvas.kt		android/app/src/main/java/com/runway/android/ui/leaderboard/CourseLeaderboa	
	88줄		647줄
android/app/src/main/java/com/runway/android/ui/components/CourseRoutePreview.kt		android/app/src/main/java/com/runway/android/ui/leaderboard/CourseLeaderboa	
	83줄		127줄
android/app/src/main/java/com/runway/android/ui/components/DiscoverCourseCard.kt		android/app/src/main/java/com/runway/android/ui/leaderboard/LeaderboardScre	
	315줄		57줄
android/app/src/main/java/com/runway/android/ui/components/HomeMapView.kt		android/app/src/main/java/com/runway/android/ui/navigation/MainScaffold.kt	
	158줄		187줄
android/app/src/main/java/com/runway/android/ui/components/LocationPermissionCard.kt		android/app/src/main/java/com/runway/android/ui/navigation/MainTab.kt	
	93줄		20줄
android/app/src/main/java/com/runway/android/ui/components/PersonalRecordsSection.kt		android/app/src/main/java/com/runway/android/ui/navigation/RunwayNavGraph.kt	
	171줄		409줄
android/app/src/main/java/com/runway/android/ui/components/RecentRunCard.kt		android/app/src/main/java/com/runway/android/ui/navigation/RunwayRoutes.kt	
	92줄		59줄
android/app/src/main/java/com/runway/android/ui/components/RouteMapView.kt		android/app/src/main/java/com/runway/android/ui/onboarding/OnboardingScreer	
	347줄		223줄
android/app/src/main/java/com/runway/android/ui/components/RunDetailMetricGrid.kt		android/app/src/main/java/com/runway/android/ui/onboarding/OnboardingViewM	
	90줄		40줄
android/app/src/main/java/com/runway/android/ui/components/RunHeroSection.kt		android/app/src/main/java/com/runway/android/ui/permission/PermissionScreen.kt	
	510줄		186줄
android/app/src/main/java/com/runway/android/ui/components/RunHistoryCard.kt		android/app/src/main/java/com/runway/android/ui/posture/PostureAnalysisViewM	
	123줄		194줄
android/app/src/main/java/com/runway/android/ui/components/RunMetricCard.kt		android/app/src/main/java/com/runway/android/ui/posture/PostureAnalyzingScre	
	42줄		34줄
android/app/src/main/java/com/runway/android/ui/components/RunPaceChart.kt		android/app/src/main/java/com/runway/android/ui/posture/PostureCaptureScreer	
	187줄		373줄
android/app/src/main/java/com/runway/android/ui/components/RunRoutePreview.kt		android/app/src/main/java/com/runway/android/ui/posture/PostureHomeScreen.kt	
	100줄		480줄
android/app/src/main/java/com/runway/android/ui/components/RunSplitsCard.kt		android/app/src/main/java/com/runway/android/ui/posture/PostureResultScreen.kt	
	98줄		441줄
android/app/src/main/java/com/runway/android/ui/components/RunningControlButton.kt		android/app/src/main/java/com/runway/android/ui/posture/PostureVideoPlayerCa	
	47줄		649줄
android/app/src/main/java/com/runway/android/ui/components/RunwayBottomNavbar.kt		android/app/src/main/java/com/runway/android/ui/profile/ProfileScreen.kt	
	52줄		507줄
android/app/src/main/java/com/runway/android/ui/components/RunwayErrorText.kt		android/app/src/main/java/com/runway/android/ui/profile/ProfileViewModel.kt	
	22줄		187줄
android/app/src/main/java/com/runway/android/ui/components/RunwayLoadingButton.kt		android/app/src/main/java/com/runway/android/ui/reminder/ReminderScreen.kt	
	49줄		230줄
android/app/src/main/java/com/runway/android/ui/components/RunwayPrimaryButton.kt		android/app/src/main/java/com/runway/android/ui/reminder/ReminderViewModel.kt	
	39줄		74줄
android/app/src/main/java/com/runway/android/ui/components/RunwayTextField.kt		android/app/src/main/java/com/runway/android/ui/running/RunResultScreen.kt	
	65줄		364줄
android/app/src/main/java/com/runway/android/ui/components/SectionHeader.kt		android/app/src/main/java/com/runway/android/ui/running/RunResultViewModel.kt	
	44줄		147줄
android/app/src/main/java/com/runway/android/ui/components/StartRunCard.kt		android/app/src/main/java/com/runway/android/ui/running/RunningCountdownOn	
	79줄		189줄
android/app/src/main/java/com/runway/android/ui/components/StatsSummaryCard.kt		android/app/src/main/java/com/runway/android/ui/running/RunningTrackingScre	
	47줄		505줄
android/app/src/main/java/com/runway/android/ui/components/StreakCard.kt		android/app/src/main/java/com/runway/android/ui/running/RunningTrackingViewM	
	87줄		366줄
android/app/src/main/java/com/runway/android/ui/components/WeatherInfo.kt		android/app/src/main/java/com/runway/android/ui/running/history/MyRunsScreer	
	41줄		274줄
android/app/src/main/java/com/runway/android/ui/components/WeeklyStatsCard.kt		android/app/src/main/java/com/runway/android/ui/running/history/MyRunsViewM	
	117줄		167줄
android/app/src/main/java/com/runway/android/ui/course/CreateCourseDialog.kt		android/app/src/main/java/com/runway/android/ui/running/history/RunCalendar.kt	
	144줄		197줄
android/app/src/main/java/com/runway/android/ui/course/detail/CourseDetailScreen.kt		android/app/src/main/java/com/runway/android/ui/running/history/RunDetailScre	
	1056줄		500줄
android/app/src/main/java/com/runway/android/ui/course/detail/CourseDetailView.kt		android/app/src/main/java/com/runway/android/ui/running/history/RunDetailView	
	452줄		174줄
android/app/src/main/java/com/runway/android/ui/course/detail/CourseMapDetailScreen.kt		android/app/src/main/java/com/runway/android/ui/settings/SettingsScreen.kt	
	332줄		267줄
android/app/src/main/java/com/runway/android/ui/course/detail/CourseMapDetailView.kt		android/app/src/main/java/com/runway/android/ui/settings/SettingsViewModel.kt	
	53줄		33줄
android/app/src/main/java/com/runway/android/ui/course/detail/PublishCourseDialog.kt		android/app/src/main/java/com/runway/android/ui/share/RunShareImageScreen.kt	
	169줄		437줄
android/app/src/main/java/com/runway/android/ui/course/detail/RateCourseDialog.kt		android/app/src/main/java/com/runway/android/ui/share/RunShareImageViewMo	
	108줄		99줄
android/app/src/main/java/com/runway/android/ui/course/detail/ReportCourseDialog.kt		android/app/src/main/java/com/runway/android/ui/splash/RunwayLogoAnimation	
	117줄		120줄
android/app/src/main/java/com/runway/android/ui/course/library/CoursesLibraryScreen.kt		android/app/src/main/java/com/runway/android/ui/splash/RunwaySplashScreen.kt	
	560줄		94줄
android/app/src/main/java/com/runway/android/ui/course/library/CoursesLibraryViewModel.kt		android/app/src/main/java/com/runway/android/ui/stats/StatsScreen.kt	
	124줄		223줄

android/app/src/main/java/com/runway/android/ui/stats/StatsViewModel.kt		Wear OS (17개 파일)	
android/app/src/main/java/com/runway/android/ui/theme/Color.kt	66줄	wear-os/app/build.gradle.kts	74줄
android/app/src/main/java/com/runway/android/ui/theme/Shape.kt	43줄	wear-os/app/src/main/java/com/runway/wear/MainActivity.kt	76줄
android/app/src/main/java/com/runway/android/ui/theme/Theme.kt	14줄	wear-os/app/src/main/java/com/runway/wear/RunForegroundService.kt	
android/app/src/main/java/com/runway/android/ui/theme/Type.kt	118줄	wear-os/app/src/main/java/com/runway/wear/WatchViewModel.kt	64줄
	148줄		946줄
android/app/src/main/java/com/runway/android/ui/tracking/TrackingRecoveryDialog.kt	66줄	os/app/src/main/java/com/runway/wear/data/OfflineCourseStore.kt	103줄
android/app/src/main/java/com/runway/android/ui/tracking/TrackingRecoveryViewModel.kt	166줄	os/app/src/main/java/com/runway/wear/data/PendingWatchRunStore.kt	
android/app/src/test/java/com/runway/android/core/posture/PostureAngleCalculatorTest.kt	46줄	os/app/src/main/java/com/runway/wear/data/PhoneRunStateRepository.kt	144줄
android/app/src/test/java/com/runway/android/core/posture/PostureFrameInterpolatorTest.kt	122줄	os/app/src/main/java/com/runway/wear/data/WatchDataLayerClient.kt	128줄
android/app/src/test/java/com/runway/android/core/posture/PostureLandmarkCorrectorTest.kt	48줄	os/app/src/main/java/com/runway/wear/data/WatchSettingsStore.kt	33줄
android/app/src/test/java/com/runway/android/core/posture/PostureLandmarkValidatorTest.kt	70줄	os/app/src/main/java/com/runway/wear/health/HealthServicesManager.kt	
android/app/src/test/java/com/runway/android/core/posture/PostureRuleEngineTest.kt	211줄	wear-os/app/src/main/java/com/runway/wear/model/RunModels.kt	136줄
android/app/src/test/java/com/runway/android/core/posture/PostureSkeletonSmoothingTest.kt	93줄	os/app/src/main/java/com/runway/wear/ui/PathFinderWearApp.kt	1678줄
android/app/src/test/java/com/runway/android/core/posture/RunningFormStrikeDetectorTest.kt	30줄	os/app/src/main/java/com/runway/wear/voice/RunningVoiceGuide.kt	154줄
android/app/src/test/java/com/runway/android/core/voice/RunningVoiceTextTest.kt	14줄	wear-os/app/src/test/java/com/runway/wear/model/GoalProgressTest.kt	32줄
android/app/src/test/java/com/runway/android/core/wear/WatchCommandParserTest.kt	73줄	os/app/src/test/java/com/runway/wear/voice/RunningVoiceTextTest.kt	14줄
android/app/src/test/java/com/runway/android/ui/home/AirKoreaCoordinateConverterTest.kt	7줄	wear-os/build.gradle.kts	5줄
android/build.gradle.kts	17줄	wear-os/settings.gradle.kts	24줄
android/settings.gradle.kts	24줄		

Backend

backend/src/main/java/com/runway/RunwayApplication.java

Java | 12줄

```
1 package com.runway;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class RunwayApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(RunwayApplication.class, args);
11     }
12 }
```

```

1  package com.runway.attempt.controller;
2
3  import com.runway.attempt.dto.*;
4  import com.runway.attempt.service.CourseAttemptService;
5  import com.runway.common.response.ApiResponse;
6  import com.runway.common.response.PageResponse;
7  import com.runway.common.security.UserPrincipal;
8  import io.swagger.v3.oas.annotations.Operation;
9  import io.swagger.v3.oas.annotations.security.SecurityRequirement;
10 import io.swagger.v3.oas.annotations.tags.Tag;
11 import jakarta.validation.Valid;
12 import lombok.RequiredArgsConstructor;
13 import org.springframework.http.ResponseEntity;
14 import org.springframework.security.core.annotation.AuthenticationPrincipal;
15 import org.springframework.web.bind.annotation.*;
16
17 import java.util.UUID;
18
19 @Tag(name = "Course Attempt", description = "코스 도전 및 리더보드 API")
20 @RestController
21 @RequestMapping("/api")
22 @RequiredArgsConstructor
23 public class CourseAttemptController {
24
25     private final CourseAttemptService courseAttemptService;
26
27     @Operation(summary = "코스 도전 시작", security = @SecurityRequirement(name = "bearerAuth"))
28     @PostMapping("/courses/{courseId}/attempts/start")
29     public ResponseEntity<ApiResponse<StartAttemptResponse>> startAttempt(
30         @AuthenticationPrincipal UserPrincipal principal,
31         @PathVariable UUID courseId,
32         @RequestBody(required = false) StartAttemptRequest request) {
33         StartAttemptResponse data = courseAttemptService.startAttempt(
34             principal.getUserId(), courseId, request);
35         return ResponseEntity.ok(ApiResponse.success("코스 도전이 시작되었습니다.", data));
36     }
37
38     @Operation(summary = "코스 도전 완주", security = @SecurityRequirement(name = "bearerAuth"))
39     @PostMapping("/course-attempts/{attemptId}/finish")
40     public ResponseEntity<ApiResponse<FinishAttemptResponse>> finishAttempt(
41         @AuthenticationPrincipal UserPrincipal principal,
42         @PathVariable UUID attemptId,
43         @Valid @RequestBody FinishAttemptRequest request) {
44         FinishAttemptResponse data = courseAttemptService.finishAttempt(
45             principal.getUserId(), attemptId, request);
46         return ResponseEntity.ok(ApiResponse.success("코스 도전이 완료되었습니다.", data));
47     }
48
49     @Operation(summary = "코스 도전 포기", security = @SecurityRequirement(name = "bearerAuth"))
50     @PostMapping("/course-attempts/{attemptId}/abandon")
51     public ResponseEntity<ApiResponse<AbandonAttemptResponse>> abandonAttempt(
52         @AuthenticationPrincipal UserPrincipal principal,
53         @PathVariable UUID attemptId,
54         @RequestBody(required = false) AbandonAttemptRequest request) {
55         AbandonAttemptResponse data = courseAttemptService.abandonAttempt(
56             principal.getUserId(), attemptId, request);
57         return ResponseEntity.ok(ApiResponse.success("코스 도전이 중단되었습니다.", data));
58     }
59
60     @Operation(summary = "코스 리더보드 조회", security = @SecurityRequirement(name = "bearerAuth"))
61     @GetMapping("/courses/{courseId}/leaderboard")
62     public ResponseEntity<ApiResponse<LeaderboardResponse>> getLeaderboard(
63         @AuthenticationPrincipal UserPrincipal principal,
64         @PathVariable UUID courseId,
65         @RequestParam(defaultValue = "0") int page,
66         @RequestParam(defaultValue = "50") int size,
67         @RequestParam(defaultValue = "fastest_time") String sortBy) {
68         LeaderboardResponse data = courseAttemptService.getLeaderboard(
69             principal.getUserId(), courseId, page, size, sortBy);
70         return ResponseEntity.ok(ApiResponse.success("코스 리더보드 조회가 완료되었습니다.", data));
71     }
72
73     @Operation(summary = "내 코스 최고 기록 조회", security = @SecurityRequirement(name = "bearerAuth"))
74     @GetMapping("/courses/{courseId}/attempts/me/best")
75     public ResponseEntity<ApiResponse<MyBestAttemptResponse>> getMyBestAttempt(
76         @AuthenticationPrincipal UserPrincipal principal,
77         @PathVariable UUID courseId) {
78         MyBestAttemptResponse data = courseAttemptService.getMyBestAttempt(
79             principal.getUserId(), courseId);
80         return ResponseEntity.ok(ApiResponse.success("내 최고 기록 조회가 완료되었습니다.", data));
81     }
82
83     @Operation(summary = "내 코스 도전 기록 조회", security = @SecurityRequirement(name = "bearerAuth"))

```

```
84     @GetMapping("/courses/{courseId}/attempts/me")
85     public ResponseEntity<ApiResponse<PageResponse<CourseAttemptResponse>>> getMyAttempts(
86         @AuthenticationPrincipal UserPrincipal principal,
87         @PathVariable UUID courseId,
88         @RequestParam(defaultValue = "0") int page,
89         @RequestParam(defaultValue = "20") int size) {
90         PageResponse<CourseAttemptResponse> data = courseAttemptService.getMyAttempts(
91             principal.getUserId(), courseId, page, size);
92         return ResponseEntity.ok(ApiResponse.success("내 코스 도전 기록 조회가 완료되었습니다.", data));
93     }
94 }
```

```

1  package com.runway.attempt.domain;
2
3  import com.runway.attempt.domain.enums.AttemptVerificationStatus;
4  import com.runway.attempt.domain.enums.CourseAttemptStatus;
5  import jakarta.persistence.*;
6  import lombok.AccessLevel;
7  import lombok.Builder;
8  import lombok.Getter;
9  import lombok.NoArgsConstructor;
10
11 import java.time.Instant;
12 import java.util.UUID;
13
14 @Entity
15 @Table(name = "course_attempts")
16 @Getter
17 @NoArgsConstructor(access = AccessLevel.PROTECTED)
18 public class CourseAttempt {
19
20     @Id
21     @GeneratedValue(strategy = GenerationType.UUID)
22     @Column(columnDefinition = "uuid", updatable = false, nullable = false)
23     private UUID id;
24
25     @Column(name = "course_id", nullable = false, columnDefinition = "uuid")
26     private UUID courseId;
27
28     @Column(name = "user_id", nullable = false, columnDefinition = "uuid")
29     private UUID userId;
30
31     // running_records.id. SET NULL on run delete → nullable
32     @Column(name = "running_record_id", columnDefinition = "uuid")
33     private UUID runningRecordId;
34
35     // CourseAttemptStatus.JpaConverter autoApply=true - @Convert 불필요
36     @Column(nullable = false, length = 30)
37     private CourseAttemptStatus status;
38
39     // AttemptVerificationStatus.JpaConverter autoApply=true
40     @Column(name = "verification_status", nullable = false, length = 30)
41     private AttemptVerificationStatus verificationStatus;
42
43     @Column(name = "started_at", nullable = false)
44     private Instant startedAt;
45
46     @Column(name = "completed_at")
47     private Instant completedAt;
48
49     @Column(name = "duration_seconds")
50     private Integer durationSeconds;
51
52     @Column(name = "distance_meters")
53     private Double distanceMeters;
54
55     @Column(name = "created_at", nullable = false, updatable = false)
56     private Instant createdAt;
57
58     @Builder
59     private CourseAttempt(UUID courseId, UUID userId, UUID runningRecordId, Instant startedAt) {
60         this.courseId = courseId;
61         this.userId = userId;
62         this.runningRecordId = runningRecordId;
63         this.status = CourseAttemptStatus.IN_PROGRESS;
64         this.verificationStatus = AttemptVerificationStatus.PENDING;
65         this.startedAt = startedAt;
66     }
67
68     @PrePersist
69     private void prePersist() {
70         this.createdAt = Instant.now();
71     }
72
73     public void complete(Instant completedAt, Integer durationSeconds, Double distanceMeters,
74         AttemptVerificationStatus verificationStatus) {
75         this.status = CourseAttemptStatus.COMPLETED;
76         this.verificationStatus = verificationStatus;
77         this.completedAt = completedAt;
78         this.durationSeconds = durationSeconds;
79         this.distanceMeters = distanceMeters;
80     }
81
82     // 포기 시 verification_status 는 PENDING 유지
83     // REJECTED 는 Phase 2 GPS 검증 실패 시에만 사용

```

```

84     public void abandon() {
85         this.status = CourseAttemptStatus.ABANDONED;
86     }
87
88     public boolean isOwnedBy(UUID userId) {
89         return this.userId.equals(userId);
90     }
91 }

```

backend/src/main/java/com/runway/attempt/domain/enums/AttemptVerificationStatus.java

Java | 44줄

```

1  package com.runway.attempt.domain.enums;
2
3  import jakarta.persistence.AttributeConverter;
4  import jakarta.persistence.Converter;
5
6  public enum AttemptVerificationStatus {
7
8      PENDING("pending"),
9      VERIFIED("verified"),
10     REJECTED("rejected");
11
12     private final String dbValue;
13
14     AttemptVerificationStatus(String dbValue) {
15         this.dbValue = dbValue;
16     }
17
18     public String getDbValue() {
19         return dbValue;
20     }
21
22     public static AttemptVerificationStatus from(String value) {
23         for (AttemptVerificationStatus status : values()) {
24             if (status.dbValue.equals(value)) {
25                 return status;
26             }
27         }
28         throw new IllegalArgumentException("Unknown AttemptVerificationStatus: " + value);
29     }
30
31     @Converter(autoApply = true)
32     public static class JpaConverter implements AttributeConverter<AttemptVerificationStatus, String> {
33
34         @Override
35         public String convertToDatabaseColumn(AttemptVerificationStatus attribute) {
36             return attribute == null ? null : attribute.getDbValue();
37         }
38
39         @Override
40         public AttemptVerificationStatus convertToEntityAttribute(String dbData) {
41             return dbData == null ? null : AttemptVerificationStatus.from(dbData);
42         }
43     }
44 }

```

backend/src/main/java/com/runway/attempt/domain/enums/CourseAttemptStatus.java

Java | 44줄

```
1 package com.runway.attempt.domain.enums;
2
3 import jakarta.persistence.AttributeConverter;
4 import jakarta.persistence.Converter;
5
6 public enum CourseAttemptStatus {
7
8     IN_PROGRESS("in_progress"),
9     COMPLETED("completed"),
10    ABANDONED("abandoned");
11
12    private final String dbValue;
13
14    CourseAttemptStatus(String dbValue) {
15        this.dbValue = dbValue;
16    }
17
18    public String getDbValue() {
19        return dbValue;
20    }
21
22    public static CourseAttemptStatus from(String value) {
23        for (CourseAttemptStatus status : values()) {
24            if (status.dbValue.equals(value)) {
25                return status;
26            }
27        }
28        throw new IllegalArgumentException("Unknown CourseAttemptStatus: " + value);
29    }
30
31    @Converter(autoApply = true)
32    public static class JpaConverter implements AttributeConverter<CourseAttemptStatus, String> {
33
34        @Override
35        public String convertToDatabaseColumn(CourseAttemptStatus attribute) {
36            return attribute == null ? null : attribute.getDbValue();
37        }
38
39        @Override
40        public CourseAttemptStatus convertToEntityAttribute(String dbData) {
41            return dbData == null ? null : CourseAttemptStatus.from(dbData);
42        }
43    }
44 }
```

backend/src/main/java/com/runway/attempt/dto/AbandonAttemptRequest.java

Java | 10줄

```
1 package com.runway.attempt.dto;
2
3 import lombok.Getter;
4
5 import java.time.Instant;
6
7 @Getter
8 public class AbandonAttemptRequest {
9     private Instant endedAt;
10 }
```


backend/src/main/java/com/runway/attempt/dto/AbandonAttemptResponse.java

Java | 31줄

```
1 package com.runway.attempt.dto;
2
3 import com.runway.attempt.domain.CourseAttempt;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.UUID;
9
10 @Getter
11 @Builder
12 public class AbandonAttemptResponse {
13
14     private UUID courseAttemptId;
15     private UUID runningRecordId;
16     private UUID courseId;
17     private String status;
18     private String verificationStatus;
19     private Instant endedAt;
20
21     public static AbandonAttemptResponse from(CourseAttempt attempt, Instant endedAt) {
22         return AbandonAttemptResponse.builder()
23             .courseAttemptId(attempt.getId())
24             .runningRecordId(attempt.getRunningRecordId())
25             .courseId(attempt.getCourseId())
26             .status(attempt.getStatus().getDbValue())
27             .verificationStatus(attempt.getVerificationStatus().getDbValue())
28             .endedAt(endedAt)
29             .build();
30     }
31 }
```

backend/src/main/java/com/runway/attempt/dto/CourseAttemptResponse.java

Java | 37줄

```
1 package com.runway.attempt.dto;
2
3 import com.runway.attempt.domain.CourseAttempt;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.UUID;
9
10 @Getter
11 @Builder
12 public class CourseAttemptResponse {
13
14     private UUID courseAttemptId;
15     private UUID runningRecordId;
16     private UUID courseId;
17     private String status;
18     private String verificationStatus;
19     private Integer durationSeconds;
20     private Double distanceMeters;
21     private Instant startedAt;
22     private Instant completedAt;
23
24     public static CourseAttemptResponse from(CourseAttempt attempt) {
25         return CourseAttemptResponse.builder()
26             .courseAttemptId(attempt.getId())
27             .runningRecordId(attempt.getRunningRecordId())
28             .courseId(attempt.getCourseId())
29             .status(attempt.getStatus().getDbValue())
30             .verificationStatus(attempt.getVerificationStatus().getDbValue())
31             .durationSeconds(attempt.getDurationSeconds())
32             .distanceMeters(attempt.getDistanceMeters())
33             .startedAt(attempt.getStartedAt())
34             .completedAt(attempt.getCompletedAt())
35             .build();
36     }
37 }
```

```
1 package com.runway.attempt.dto;
2
3 import jakarta.validation.constraints.NotNull;
4 import jakarta.validation.constraints.Positive;
5 import lombok.Getter;
6
7 import java.time.Instant;
8
9 @Getter
10 public class FinishAttemptRequest {
11
12     @NotNull(message = "종료 시각은 필수입니다.")
13     private Instant endedAt;
14
15     @NotNull(message = "이동 거리는 필수입니다.")
16     @Positive(message = "이동 거리는 양수여야 합니다.")
17     private Double distanceMeters;
18
19     @NotNull(message = "소요 시간은 필수입니다.")
20     @Positive(message = "소요 시간은 양수여야 합니다.")
21     private Integer durationSeconds;
22
23     private Integer avgPaceSecondsPerKm;
24     private Integer caloriesBurned;
25     private Integer avgHeartRateBpm;
26 }
```

```
1 package com.runway.attempt.dto;
2
3 import com.runway.attempt.domain.CourseAttempt;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.UUID;
9
10 @Getter
11 @Builder
12 public class FinishAttemptResponse {
13
14     private UUID courseAttemptId;
15     private UUID runningRecordId;
16     private UUID courseId;
17     private String status;
18     private String verificationStatus;
19     private Integer durationSeconds;
20     private Double distanceMeters;
21     private Instant completedAt;
22
23     /** 개인 최고 기록(PR) 여부 */
24     private boolean isPR;
25     /** 이전 최고 기록(초). 첫 완주이면 null */
26     private Integer previousBestSeconds;
27     /** 이전 기록 대비 개선 시간(초). 양수 = 빠름, 음수 = 느림. 첫 완주이면 null */
28     private Integer improvementSeconds;
29
30     public static FinishAttemptResponse from(CourseAttempt attempt) {
31         return FinishAttemptResponse.builder()
32             .courseAttemptId(attempt.getId())
33             .runningRecordId(attempt.getRunningRecordId())
34             .courseId(attempt.getCourseId())
35             .status(attempt.getStatus().getDbValue())
36             .verificationStatus(attempt.getVerificationStatus().getDbValue())
37             .durationSeconds(attempt.getDurationSeconds())
38             .distanceMeters(attempt.getDistanceMeters())
39             .completedAt(attempt.getCompletedAt())
40             .isPR(false)
41             .build();
42     }
43
44     public static FinishAttemptResponse from(CourseAttempt attempt,
45                                             boolean isPR,
46                                             Integer previousBestSeconds,
47                                             Integer improvementSeconds) {
48         return FinishAttemptResponse.builder()
49             .courseAttemptId(attempt.getId())
50             .runningRecordId(attempt.getRunningRecordId())
51             .courseId(attempt.getCourseId())
52             .status(attempt.getStatus().getDbValue())
53             .verificationStatus(attempt.getVerificationStatus().getDbValue())
54             .durationSeconds(attempt.getDurationSeconds())
55             .distanceMeters(attempt.getDistanceMeters())
56             .completedAt(attempt.getCompletedAt())
57             .isPR(isPR)
58             .previousBestSeconds(previousBestSeconds)
59             .improvementSeconds(improvementSeconds)
60             .build();
61     }
62 }
```

backend/src/main/java/com/runway/attempt/dto/LeaderboardItem.java

Java | 17줄

```
1 package com.runway.attempt.dto;
2
3 import lombok.Builder;
4 import lombok.Getter;
5
6 import java.util.UUID;
7
8 @Getter
9 @Builder
10 public class LeaderboardItem {
11
12     private long rank;
13     private UUID userId;
14     private String nickname;
15     private Integer bestTimeSeconds;
16     private long completionCount;
17 }
```

backend/src/main/java/com/runway/attempt/dto/LeaderboardResponse.java

Java | 22줄

```
1 package com.runway.attempt.dto;
2
3 import lombok.Builder;
4 import lombok.Getter;
5
6 import java.util.List;
7 import java.util.UUID;
8
9 @Getter
10 @Builder
11 public class LeaderboardResponse {
12
13     private UUID courseId;
14     private List<LeaderboardItem> items;
15     private int page;
16     private int size;
17     private long totalElements;
18     private int totalPages;
19     private boolean hasNext;
20     private String sortBy;
21     private Long myRank;
22 }
```

backend/src/main/java/com/runway/attempt/dto/MyBestAttemptResponse.java

Java | 18줄

```
1 package com.runway.attempt.dto;
2
3 import lombok.Builder;
4 import lombok.Getter;
5
6 import java.time.Instant;
7
8 @Getter
9 @Builder
10 public class MyBestAttemptResponse {
11
12     /** 완주 횟수 (completed 기준) */
13     private long completionCount;
14     /** 최고 기록(초). 완주 기록 없으면 null */
15     private Integer bestTimeSeconds;
16     /** 가장 최근 완주 시각. 완주 기록 없으면 null */
17     private Instant lastAttemptAt;
18 }
```

backend/src/main/java/com/runway/attempt/dto/StartAttemptRequest.java

Java | 10줄

```
1 package com.runway.attempt.dto;
2
3 import lombok.Getter;
4
5 import java.time.Instant;
6
7 @Getter
8 public class StartAttemptRequest {
9     private Instant startedAt;
10 }
```

backend/src/main/java/com/runway/attempt/dto/StartAttemptResponse.java

Java | 31줄

```
1 package com.runway.attempt.dto;
2
3 import com.runway.attempt.domain.CourseAttempt;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.UUID;
9
10 @Getter
11 @Builder
12 public class StartAttemptResponse {
13
14     private UUID courseAttemptId;
15     private UUID runningRecordId;
16     private UUID courseId;
17     private String status;
18     private String verificationStatus;
19     private Instant startedAt;
20
21     public static StartAttemptResponse from(CourseAttempt attempt) {
22         return StartAttemptResponse.builder()
23             .courseAttemptId(attempt.getId())
24             .runningRecordId(attempt.getRunningRecordId())
25             .courseId(attempt.getCourseId())
26             .status(attempt.getStatus().getDbValue())
27             .verificationStatus(attempt.getVerificationStatus().getDbValue())
28             .startedAt(attempt.getStartedAt())
29             .build();
30     }
31 }
```

backend/src/main/java/com/runway/attempt/repository/CourseAttemptRepository.java

Java | 41줄

```
1 package com.runway.attempt.repository;
2
3 import com.runway.attempt.domain.CourseAttempt;
4 import com.runway.attempt.domain.enums.AttemptVerificationStatus;
5 import com.runway.attempt.domain.enums.CourseAttemptStatus;
6 import org.springframework.data.domain.Page;
7 import org.springframework.data.domain.Pageable;
8 import org.springframework.data.jpa.repository.JpaRepository;
9 import org.springframework.data.jpa.repository.Query;
10 import org.springframework.data.repository.query.Param;
11
12 import java.util.Optional;
13 import java.util.UUID;
14
15 public interface CourseAttemptRepository extends JpaRepository<CourseAttempt, UUID> {
16
17     Optional<CourseAttempt> findByIdAndUserId(UUID attemptId, UUID userId);
18
19     Page<CourseAttempt> findByCourseIdAndUserIdOrderByStartedAtDesc(
20         UUID courseId, UUID userId, Pageable pageable);
21
22     long countByUserIdAndStatus(UUID userId, CourseAttemptStatus status);
23
24     Optional<CourseAttempt> findFirstByUserIdAndStatusOrderByCompletedAtAsc(
25         UUID userId, CourseAttemptStatus status);
26
27     long countByCourseIdAndUserIdAndStatusAndVerificationStatus(
28         UUID courseId, UUID userId,
29         CourseAttemptStatus status, AttemptVerificationStatus verificationStatus);
30
31     /** 사용자의 해당 코스 완주 기록 중 최고 기록 (duration_seconds 최솟값) */
32     @Query("SELECT MIN(a.durationSeconds) FROM CourseAttempt a " +
33         "WHERE a.courseId = :courseId AND a.userId = :userId " +
34         "AND a.status = com.runway.attempt.domain.enums.CourseAttemptStatus.COMPLETED")
35     Optional<Integer> findMinDurationSecondsByCourseIdAndUserId(
36         @Param("courseId") UUID courseId, @Param("userId") UUID userId);
37
38     /** 사용자의 해당 코스 완주 횟수 */
39     long countByCourseIdAndUserIdAndStatus(
40         UUID courseId, UUID userId, CourseAttemptStatus status);
41 }
```

```

1  package com.runway.attempt.service;
2
3  import com.runway.attempt.domain.CourseAttempt;
4  import com.runway.attempt.dto.*;
5  import com.runway.attempt.repository.CourseAttemptRepository;
6  import com.runway.common.exception.ErrorCode;
7  import com.runway.common.exception.RunwayException;
8  import com.runway.common.response.PageResponse;
9  import com.runway.course.domain.Course;
10 import com.runway.course.domain.enums.CourseStatus;
11 import com.runway.course.repository.CourseRepository;
12 import com.runway.run.domain.RunningPoint;
13 import com.runway.run.domain.RunningRecord;
14 import com.runway.run.repository.RunningPointRepository;
15 import com.runway.run.repository.RunningRecordRepository;
16 import jakarta.persistence.EntityManager;
17 import lombok.RequiredArgsConstructor;
18 import lombok.extern.slf4j.Slf4j;
19 import org.locationtech.jts.geom.Coordinate;
20 import org.locationtech.jts.geom.GeometryFactory;
21 import org.locationtech.jts.geom.LineString;
22 import org.locationtech.jts.geom.PrecisionModel;
23 import org.springframework.data.domain.PageImpl;
24 import org.springframework.data.domain.PageRequest;
25 import org.springframework.stereotype.Service;
26 import org.springframework.transaction.annotation.Transactional;
27
28 import java.time.Instant;
29 import java.util.ArrayList;
30 import java.util.List;
31 import java.util.Optional;
32 import java.util.UUID;
33
34 @Slf4j
35 @Service
36 @RequiredArgsConstructor
37 public class CourseAttemptService {
38
39     private static final GeometryFactory GEOMETRY_FACTORY = new GeometryFactory(new PrecisionModel(), 4326);
40
41     private final CourseAttemptRepository courseAttemptRepository;
42     private final CourseRepository courseRepository;
43     private final RunningRecordRepository runningRecordRepository;
44     private final RunningPointRepository runningPointRepository;
45     private final EntityManager entityManager;
46
47     @Transactional
48     public StartAttemptResponse startAttempt(UUID userId, UUID courseId,
49                                             StartAttemptRequest request) {
50         Course course = courseRepository.findByIdAndDeletedAtIsNull(courseId)
51             .orElseThrow(() -> new RunwayException(ErrorCode.COURSE_NOT_FOUND));
52
53         // published 코스는 누구나 도전 가능
54         // draft 코스는 창작자 본인만 도전 가능 (연습 목적)
55         boolean canAttempt = course.getStatus() == CourseStatus.PUBLISHED
56             || (course.getStatus() == CourseStatus.DRAFT && course.getCreatorId().equals(userId));
57         if (!canAttempt) {
58             throw new RunwayException(ErrorCode.INVALID_COURSE_STATUS);
59         }
60
61         Instant startedAt = (request != null && request.getStartedAt() != null)
62             ? request.getStartedAt() : Instant.now();
63
64         // 1. running_records 먼저 생성
65         RunningRecord runningRecord = RunningRecord.builder()
66             .userId(userId)
67             .startedAt(startedAt)
68             .build();
69         runningRecordRepository.save(runningRecord);
70
71         // 2. course_attempts 생성 (running_record_id 연결)
72         CourseAttempt attempt = CourseAttempt.builder()
73             .courseId(courseId)
74             .userId(userId)
75             .runningRecordId(runningRecord.getId())
76             .startedAt(startedAt)
77             .build();
78         courseAttemptRepository.save(attempt);
79
80         // 3. courses.attempt_count 증가 (동일 트랜잭션)
81         course.incrementAttemptCount();
82
83         log.info("Attempt started: attemptId={} courseId={} runId={}",

```

```

84         attempt.getId(), courseId, runningRecord.getId());
85     return StartAttemptResponse.from(attempt);
86 }
87
88 @Transactional
89 public FinishAttemptResponse finishAttempt(UUID userId, UUID attemptId,
90     FinishAttemptRequest request) {
91     CourseAttempt attempt = courseAttemptRepository.findByIdAndUserId(attemptId, userId)
92         .orElseThrow(() -> new RunwayException(ErrorCode.COURSE_ATTEMPT_NOT_FOUND));
93
94     if (attempt.getStatus() != com.runway.attempt.domain.enums.CourseAttemptStatus.IN_PROGRESS) {
95         throw new RunwayException(ErrorCode.INVALID_ATTEMPT_STATUS);
96     }
97
98     // 12 m/s (43.2 km/h) 초과는 인간 러닝으로 물리적으로 불가능한 속도
99     if (request.getDurationSeconds() > 0
100         && (double) request.getDistanceMeters() / request.getDurationSeconds() > 12.0) {
101         throw new RunwayException(ErrorCode.IMPOSSIBLE_SPEED);
102     }
103
104     Instant endedAt = request.getEndedAt() != null ? request.getEndedAt() : Instant.now();
105
106     if (!endedAt.isAfter(attempt.getStartedAt())) {
107         throw new RunwayException(ErrorCode.INVALID_REQUEST, "종료 시각은 시작 시각 이후여야 합니다.");
108     }
109
110     // 코스 조회 (검증에도 필요하므로 먼저 fetch)
111     Course course = courseRepository.findByIdAndDeletedAtIsNull(attempt.getCourseId())
112         .orElseThrow(() -> new RunwayException(ErrorCode.COURSE_NOT_FOUND));
113
114     // running_record 완료 처리 + 경로 수집
115     List<RunningPoint> points = List.of();
116     if (attempt.getRunningRecordId() != null) {
117         RunningRecord record = runningRecordRepository.findById(attempt.getRunningRecordId())
118             .orElseThrow(() -> new RunwayException(ErrorCode.RUN_NOT_FOUND));
119
120         points = runningPointRepository
121             .findByRunningRecordIdOrderBySequenceAsc(attempt.getRunningRecordId());
122         if (points.size() >= 2) {
123             Coordinate[] coords = points.stream()
124                 .map(p -> p.getLocation().getCoordinate())
125                 .toArray(Coordinate[]::new);
126             LineString path = GEOMETRY_FACTORY.createLineString(coords);
127             record.updatePath(path);
128         }
129
130         record.finish(endedAt, request.getDistanceMeters(), request.getDurationSeconds(),
131             request.getAvgPaceSecondsPerKm(), request.getCaloriesBurned(),
132             request.getAvgHeartRateBpm());
133     }
134
135     // — 기록 무결성 검증 —
136     com.runway.attempt.domain.enums.AttemptVerificationStatus verificationStatus =
137         com.runway.attempt.domain.enums.AttemptVerificationStatus.VERIFIED;
138
139     // GPS 포인트 < 2개 → 경로 재구성 불가, 검증 보류
140     if (points.size() < 2) {
141         verificationStatus = com.runway.attempt.domain.enums.AttemptVerificationStatus.PENDING;
142         log.info("Attempt marked unverified: insufficient GPS points. count={} attemptId={}",
143             points.size(), attemptId);
144     }
145
146     // 코스 거리 대비 70% 미만 → PENDING
147     if (verificationStatus == com.runway.attempt.domain.enums.AttemptVerificationStatus.VERIFIED
148         && course.getDistanceMeters() > 0
149         && request.getDistanceMeters() < course.getDistanceMeters() * 0.7) {
150         verificationStatus = com.runway.attempt.domain.enums.AttemptVerificationStatus.PENDING;
151         log.info("Attempt marked unverified (distance coverage): attempted={}m course={}m attemptId={}",
152             request.getDistanceMeters(), course.getDistanceMeters(), attemptId);
153     }
154
155     // 시작/종료 지점 500m 이상 이탈 → PENDING
156     if (verificationStatus == com.runway.attempt.domain.enums.AttemptVerificationStatus.VERIFIED
157         && !points.isEmpty()) {
158         RunningPoint firstPt = points.get(0);
159         RunningPoint lastPt = points.get(points.size() - 1);
160         double startDist = haversineMeters(
161             course.getStartLocation().getY(), course.getStartLocation().getX(),
162             firstPt.getLocation().getY(), firstPt.getLocation().getX());
163         double endDist = haversineMeters(
164             course.getEndLocation().getY(), course.getEndLocation().getX(),
165             lastPt.getLocation().getY(), lastPt.getLocation().getX());
166         if (startDist > 500 || endDist > 500) {
167             verificationStatus = com.runway.attempt.domain.enums.AttemptVerificationStatus.PENDING;
168             log.info("Attempt marked unverified (proximity): startDist={}m endDist={}m attemptId={}",
169                 startDist, endDist, attemptId);

```

```

170     }
171 }
172
173 // — PR 계산 (complete() 호출 전 이전 기록 조회) —
174 Optional<Integer> previousBestOpt = courseAttemptRepository
175     .findMinDurationSecondsByCourseIdAndUserId(attempt.getCourseId(), userId);
176 boolean isPR;
177 Integer previousBestSeconds;
178 Integer improvementSeconds;
179 if (previousBestOpt.isPresent()) {
180     previousBestSeconds = previousBestOpt.get();
181     isPR = request.getDurationSeconds() < previousBestSeconds;
182     improvementSeconds = previousBestSeconds - request.getDurationSeconds();
183 } else {
184     // 첫 완주
185     isPR = true;
186     previousBestSeconds = null;
187     improvementSeconds = null;
188 }
189
190 // course_attempt 완료 처리
191 attempt.complete(endedAt, request.getDurationSeconds(), request.getDistanceMeters(), verificationStatus);
192
193 // courses.completion_count 증가 (동일 트랜잭션)
194 course.incrementCompletionCount();
195
196 log.info("Attempt finished: attemptId={} courseId={} durationSeconds={} isPR={}",
197     attemptId, attempt.getCourseId(), request.getDurationSeconds(), isPR);
198 return FinishAttemptResponse.from(attempt, isPR, previousBestSeconds, improvementSeconds);
199 }
200
201 @Transactional
202 public AbandonAttemptResponse abandonAttempt(UUID userId, UUID attemptId,
203     AbandonAttemptRequest request) {
204     CourseAttempt attempt = courseAttemptRepository.findByIdAndUserId(attemptId, userId)
205         .orElseThrow(() -> new RunwayException(ErrorCode.COURSE_ATTEMPT_NOT_FOUND));
206
207     if (attempt.getStatus() != com.runway.attempt.domain.enums.CourseAttemptStatus.IN_PROGRESS) {
208         throw new RunwayException(ErrorCode.INVALID_ATTEMPT_STATUS);
209     }
210
211     Instant endedAt = (request != null && request.getEndedAt() != null)
212         ? request.getEndedAt() : Instant.now();
213
214     // running_record 중단 처리
215     if (attempt.getRunningRecordId() != null) {
216         RunningRecord record = runningRecordRepository.findById(attempt.getRunningRecordId())
217             .orElseThrow(() -> new RunwayException(ErrorCode.RUN_NOT_FOUND));
218         record.abandon(endedAt);
219     }
220
221     // course_attempt 중단 처리 (verification_status 는 PENDING 유지)
222     attempt.abandon();
223
224     log.info("Attempt abandoned: attemptId={} courseId={}", attemptId, attempt.getCourseId());
225     return AbandonAttemptResponse.from(attempt, endedAt);
226 }
227
228 @Transactional(readOnly = true)
229 public LeaderboardResponse getLeaderboard(UUID userId, UUID courseId, int page, int size, String sortBy) {
230     Course course = courseRepository.findByIdAndDeletedAtIsNull(courseId)
231         .orElseThrow(() -> new RunwayException(ErrorCode.COURSE_NOT_FOUND));
232     if (!course.isVisibleTo(userId)) {
233         throw new RunwayException(ErrorCode.FORBIDDEN);
234     }
235
236     boolean byCompletions = "most_completions".equalsIgnoreCase(sortBy);
237     String rankOrder = byCompletions
238         ? "ORDER BY COUNT(*) DESC, MIN(ca.duration_seconds) ASC"
239         : "ORDER BY MIN(ca.duration_seconds) ASC";
240
241     String dataSql = String.format("""
242         WITH ranked AS (
243             SELECT
244                 u.id                AS user_id,
245                 u.nickname          AS nickname,
246                 MIN(ca.duration_seconds) AS best_time_seconds,
247                 COUNT(*)             AS completion_count,
248                 RANK() OVER (%s) AS rank
249             FROM course_attempts ca
250             JOIN users u ON ca.user_id = u.id
251             WHERE ca.course_id = ?
252                   AND ca.status = 'completed'
253                   AND ca.verification_status = 'verified'
254             GROUP BY ca.user_id, u.id, u.nickname
255         )

```



```

256         SELECT * FROM ranked
257         ORDER BY rank ASC
258         LIMIT ? OFFSET ?
259         "", rankOrder);
260
261     String countSql = ""
262         SELECT COUNT(DISTINCT ca.user_id)
263         FROM course_attempts ca
264         WHERE ca.course_id = ?
265             AND ca.status = 'completed'
266             AND ca.verification_status = 'verified'
267         "";
268
269     String myRankSql = String.format("""
270         SELECT ranked.rank FROM (
271             SELECT ca.user_id,
272                 RANK() OVER (%s) AS rank
273             FROM course_attempts ca
274             WHERE ca.course_id = ?
275                 AND ca.status = 'completed'
276                 AND ca.verification_status = 'verified'
277             GROUP BY ca.user_id
278         ) ranked
279         WHERE ranked.user_id = ?
280         "", rankOrder);
281
282     List<Object> dataParams = new ArrayList<>();
283     dataParams.add(courseId);
284     dataParams.add(size);
285     dataParams.add((long) page * size);
286
287     @SuppressWarnings("unchecked")
288     List<Object[]> rows = buildQuery(dataSql, dataParams).getResultList();
289     List<LeaderboardItem> items = rows.stream().map(this::toLeaderboardItem).toList();
290
291     long total = ((Number) buildQuery(countSql, List.of(courseId)).getSingleResult()).longValue();
292     int totalPages = size == 0 ? 0 : (int) Math.ceil((double) total / size);
293
294     @SuppressWarnings("unchecked")
295     List<Object> myRankRows = buildQuery(myRankSql, List.of(courseId, userId)).getResultList();
296     Long myRank = myRankRows.isEmpty() ? null : ((Number) myRankRows.get(0)).longValue();
297
298     return LeaderboardResponse.builder()
299         .courseId(courseId)
300         .items(items)
301         .page(page)
302         .size(size)
303         .totalElements(total)
304         .totalPages(totalPages)
305         .hasNext(page < totalPages - 1)
306         .sortBy(byCompletions ? "most_completions" : "fastest_time")
307         .myRank(myRank)
308         .build();
309 }
310
311 @Transactional(readOnly = true)
312 public MyBestAttemptResponse getMyBestAttempt(UUID userId, UUID courseId) {
313     Course course = courseRepository.findByIdAndDeletedAtIsNull(courseId)
314         .orElseThrow(() -> new RunwayException(ErrorCode.COURSE_NOT_FOUND));
315     if (!course.isVisibleTo(userId)) {
316         throw new RunwayException(ErrorCode.FORBIDDEN);
317     }
318
319     long count = courseAttemptRepository.countByCourseIdAndUserIdAndStatus(
320         courseId, userId, com.runway.attempt.domain.enums.CourseAttemptStatus.COMPLETED);
321     Optional<Integer> bestTime = courseAttemptRepository
322         .findMinDurationSecondsByCourseIdAndUserId(courseId, userId);
323     // 가장 최근 완주 시각
324     com.runway.attempt.domain.enums.CourseAttemptStatus completed =
325         com.runway.attempt.domain.enums.CourseAttemptStatus.COMPLETED;
326     java.time.Instant lastAttemptAt = courseAttemptRepository
327         .findByCourseIdAndUserIdOrderByStartedAtDesc(courseId, userId,
328             org.springframework.data.domain.PageRequest.of(0, 1))
329         .getContent().stream()
330         .filter(a -> a.getStatus() == completed)
331         .map(com.runway.attempt.domain.CourseAttempt::getCompletedAt)
332         .findFirst()
333         .orElse(null);
334
335     return MyBestAttemptResponse.builder()
336         .completionCount(count)
337         .bestTimeSeconds(bestTime.orElse(null))
338         .lastAttemptAt(lastAttemptAt)
339         .build();
340 }
341

```

```

342     @Transactional(readOnly = true)
343     public PageResponse<CourseAttemptResponse> getMyAttempts(UUID userId, UUID courseId,
344                                                             int page, int size) {
345         Course course = courseRepository.findByIdAndDeletedAtIsNull(courseId)
346             .orElseThrow(() -> new RunwayException(ErrorCode.COURSE_NOT_FOUND));
347         if (!course.isVisibleTo(userId)) {
348             throw new RunwayException(ErrorCode.FORBIDDEN);
349         }
350
351         return PageResponse.from(
352             courseAttemptRepository
353                 .findByCourseIdAndUserIdOrderByStartedAtDesc(
354                     courseId, userId, PageRequest.of(page, size))
355                 .map(CourseAttemptResponse::from));
356     }
357
358     // --- private helpers ---
359
360     private static double haversineMeters(double lat1, double lon1, double lat2, double lon2) {
361         final double R = 6_371_000.0;
362         double dLat = Math.toRadians(lat2 - lat1);
363         double dLon = Math.toRadians(lon2 - lon1);
364         double a = Math.sin(dLat / 2) * Math.sin(dLat / 2)
365             + Math.cos(Math.toRadians(lat1)) * Math.cos(Math.toRadians(lat2))
366             * Math.sin(dLon / 2) * Math.sin(dLon / 2);
367         return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
368     }
369
370     private jakarta.persistence.Query buildQuery(String sql, List<Object> params) {
371         jakarta.persistence.Query query = entityManager.createNativeQuery(sql);
372         for (int i = 0; i < params.size(); i++) {
373             query.setParameter(i + 1, params.get(i));
374         }
375         return query;
376     }
377
378     private LeaderboardItem toLeaderboardItem(Object[] row) {
379         // row[0]=user_id, row[1]=nickname, row[2]=best_time_seconds,
380         // row[3]=completion_count, row[4]=rank
381         return LeaderboardItem.builder()
382             .userId(UUID.fromString(row[0].toString()))
383             .nickname((String) row[1])
384             .bestTimeSeconds(((Number) row[2]).intValue())
385             .completionCount(((Number) row[3]).longValue())
386             .rank(((Number) row[4]).longValue())
387             .build();
388     }
389 }

```

```

1 package com.runway.auth.controller;
2
3 import com.runway.auth.dto.*;
4 import com.runway.auth.service.AuthService;
5 import com.runway.common.response.ApiResponse;
6 import com.runway.common.security.UserPrincipal;
7 import io.swagger.v3.oas.annotations.Operation;
8 import io.swagger.v3.oas.annotations.tags.Tag;
9 import jakarta.validation.Valid;
10 import lombok.RequiredArgsConstructor;
11 import org.springframework.http.HttpStatus;
12 import org.springframework.http.ResponseEntity;
13 import org.springframework.security.core.annotation.AuthenticationPrincipal;
14 import org.springframework.web.bind.annotation.*;
15
16 @Tag(name = "Auth", description = "인증 API")
17 @RestController
18 @RequestMapping("/api/auth")
19 @RequiredArgsConstructor
20 public class AuthController {
21
22     private final AuthService authService;
23
24     @Operation(summary = "회원가입")
25     @PostMapping("/signup")
26     public ResponseEntity<ApiResponse<SignupResponse>> signup(
27         @Valid @RequestBody SignupRequest request) {
28         SignupResponse response = authService.signup(request);
29         return ResponseEntity.status(HttpStatus.CREATED)
30             .body(ApiResponse.success("회원가입이 완료되었습니다.", response));
31     }
32
33     @Operation(summary = "로그인")
34     @PostMapping("/login")
35     public ResponseEntity<ApiResponse<LoginResponse>> login(
36         @Valid @RequestBody LoginRequest request) {
37         LoginResponse response = authService.login(request);
38         return ResponseEntity.ok(ApiResponse.success("로그인이 완료되었습니다.", response));
39     }
40
41     @Operation(summary = "Access Token 재발급")
42     @PostMapping("/reissue")
43     public ResponseEntity<ApiResponse<ReissueResponse>> reissue(
44         @Valid @RequestBody ReissueRequest request) {
45         ReissueResponse response = authService.reissue(request);
46         return ResponseEntity.ok(ApiResponse.success("토큰이 재발급되었습니다.", response));
47     }
48
49     @Operation(summary = "로그아웃")
50     @PostMapping("/logout")
51     public ResponseEntity<ApiResponse<Void>> logout(
52         @RequestBody(required = false) LogoutRequest request,
53         @AuthenticationPrincipal UserPrincipal principal) {
54         authService.logout(principal.getId());
55         return ResponseEntity.ok(ApiResponse.success("로그아웃이 완료되었습니다."));
56     }
57 }

```

```

1 package com.runway.auth.dto;
2
3 import jakarta.validation.constraints.Email;
4 import jakarta.validation.constraints.NotBlank;
5 import lombok.Getter;
6 import lombok.NoArgsConstructor;
7 import lombok.Setter;
8
9 @Getter
10 @Setter
11 @NoArgsConstructor
12 public class LoginRequest {
13
14     @NotBlank(message = "이메일은 필수입니다.")
15     @Email(message = "올바른 이메일 형식이 아닙니다.")
16     private String email;
17
18     @NotBlank(message = "비밀번호는 필수입니다.")
19     private String password;
20 }

```

backend/src/main/java/com/runway/auth/dto/LoginResponse.java

Java | 40줄

```
1 package com.runway.auth.dto;
2
3 import com.runway.user.domain.User;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.util.UUID;
8
9 @Getter
10 @Builder
11 public class LoginResponse {
12
13     private String accessToken;
14     private String refreshToken;
15     private String tokenType;
16     private UserSummary user;
17
18     public static LoginResponse of(String accessToken, String refreshToken, User user) {
19         return LoginResponse.builder()
20             .accessToken(accessToken)
21             .refreshToken(refreshToken)
22             .tokenType("Bearer")
23             .user(UserSummary.builder()
24                 .userId(user.getId())
25                 .email(user.getEmail())
26                 .nickname(user.getNickname())
27                 .profileImageUrl(user.getProfileImageUrl())
28                 .build()
29             ).build();
30     }
31
32     @Getter
33     @Builder
34     public static class UserSummary {
35         private UUID userId;
36         private String email;
37         private String nickname;
38         private String profileImageUrl;
39     }
40 }
```

backend/src/main/java/com/runway/auth/dto/LogoutRequest.java

Java | 15줄

```
1 package com.runway.auth.dto;
2
3 import lombok.Getter;
4 import lombok.NoArgsConstructor;
5 import lombok.Setter;
6
7 @Getter
8 @Setter
9 @NoArgsConstructor
10 public class LogoutRequest {
11
12     // Phase 1: 로그아웃은 SecurityContext의 인증 정보로 처리하므로
13     // refreshToken은 수신만 하고 사용하지 않는다 (Phase 2 token rotation 대비)
14     private String refreshToken;
15 }
```

backend/src/main/java/com/runway/auth/dto/ReissueRequest.java

Java | 15줄

```
1 package com.runway.auth.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import lombok.Getter;
5 import lombok.NoArgsConstructor;
6 import lombok.Setter;
7
8 @Getter
9 @Setter
10 @NoArgsConstructor
11 public class ReissueRequest {
12
13     @NotBlank(message = "refreshToken은 필수입니다.")
14     private String refreshToken;
15 }
```

backend/src/main/java/com/runway/auth/dto/ReissueResponse.java

Java | 21줄

```
1 package com.runway.auth.dto;
2
3 import lombok.Builder;
4 import lombok.Getter;
5
6 @Getter
7 @Builder
8 public class ReissueResponse {
9
10     private String accessToken;
11     private String refreshToken;
12     private String tokenType;
13
14     public static ReissueResponse of(String accessToken, String refreshToken) {
15         return ReissueResponse.builder()
16             .accessToken(accessToken)
17             .refreshToken(refreshToken)
18             .tokenType("Bearer")
19             .build();
20     }
21 }
```

backend/src/main/java/com/runway/auth/dto/SignupRequest.java

Java | 26줄

```
1 package com.runway.auth.dto;
2
3 import jakarta.validation.constraints.Email;
4 import jakarta.validation.constraints.NotBlank;
5 import jakarta.validation.constraints.Size;
6 import lombok.Getter;
7 import lombok.NoArgsConstructor;
8 import lombok.Setter;
9
10 @Getter
11 @Setter
12 @NoArgsConstructor
13 public class SignupRequest {
14
15     @NotBlank(message = "이메일은 필수입니다.")
16     @Email(message = "올바른 이메일 형식이 아닙니다.")
17     private String email;
18
19     @NotBlank(message = "비밀번호는 필수입니다.")
20     @Size(min = 8, message = "비밀번호는 8자 이상이어야 합니다.")
21     private String password;
22
23     @NotBlank(message = "닉네임은 필수입니다.")
24     @Size(min = 2, max = 50, message = "닉네임은 2자 이상 50자 이하여야 합니다.")
25     private String nickname;
26 }
```

```
1 package com.runway.auth.dto;
2
3 import com.runway.user.domain.User;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.UUID;
9
10 @Getter
11 @Builder
12 public class SignupResponse {
13
14     private UUID userId;
15     private String email;
16     private String nickname;
17     private String profileImageUrl;
18     private String bio;
19     private Instant createdAt;
20
21     public static SignupResponse from(User user) {
22         return SignupResponse.builder()
23             .userId(user.getId())
24             .email(user.getEmail())
25             .nickname(user.getNickname())
26             .profileImageUrl(user.getProfileImageUrl())
27             .bio(user.getBio())
28             .createdAt(user.getCreatedAt())
29             .build();
30     }
31 }
```

```

1  package com.runway.auth.service;
2
3  import com.runway.auth.dto.*;
4  import com.runway.common.exception.ErrorCode;
5  import com.runway.common.exception.RunwayException;
6  import com.runway.common.security.JwtProvider;
7  import com.runway.user.domain.User;
8  import com.runway.user.repository.UserRepository;
9  import lombok.RequiredArgsConstructor;
10 import lombok.extern.slf4j.Slf4j;
11 import org.springframework.security.crypto.password.PasswordEncoder;
12 import org.springframework.stereotype.Service;
13 import org.springframework.transaction.annotation.Transactional;
14
15 import java.util.UUID;
16
17 @Slf4j
18 @Service
19 @RequiredArgsConstructor
20 public class AuthService {
21
22     private final UserRepository userRepository;
23     private final PasswordEncoder passwordEncoder;
24     private final JwtProvider jwtProvider;
25
26     @Transactional
27     public SignupResponse signup(SignupRequest request) {
28         if (userRepository.existsByEmailAndDeletedAtIsNull(request.getEmail())) {
29             throw new RunwayException(ErrorCode.DUPLICATED_EMAIL);
30         }
31         if (userRepository.existsByNicknameAndDeletedAtIsNull(request.getNickname())) {
32             throw new RunwayException(ErrorCode.DUPLICATED_NICKNAME);
33         }
34
35         User user = User.builder()
36             .email(request.getEmail())
37             .passwordHash(passwordEncoder.encode(request.getPassword()))
38             .nickname(request.getNickname())
39             .build();
40
41         User saved = userRepository.save(user);
42         log.info("User signed up: {}", saved.getEmail());
43         return SignupResponse.from(saved);
44     }
45
46     @Transactional
47     public LoginResponse login(LoginRequest request) {
48         User user = userRepository.findByEmailAndDeletedAtIsNull(request.getEmail())
49             .orElseThrow(() -> new RunwayException(ErrorCode.INVALID_REQUEST, "이메일 또는 비밀번호가 올바르지 않습니다."));
50
51         if (!passwordEncoder.matches(request.getPassword(), user.getPasswordHash())) {
52             throw new RunwayException(ErrorCode.INVALID_REQUEST, "이메일 또는 비밀번호가 올바르지 않습니다.");
53         }
54
55         String accessToken = jwtProvider.generateAccessToken(user.getId(), user.getEmail());
56         String refreshToken = jwtProvider.generateRefreshToken(user.getId(), user.getEmail());
57
58         user.updateRefreshTokenHash(jwtProvider.hashToken(refreshToken));
59         log.info("User logged in: {}", user.getEmail());
60         return LoginResponse.of(accessToken, refreshToken, user);
61     }
62
63     @Transactional
64     public ReissueResponse reissue(ReissueRequest request) {
65         String refreshToken = request.getRefreshToken();
66
67         // 만료 → EXPIRED_TOKEN, 위조 → INVALID_TOKEN 구분
68         UUID userId = jwtProvider.validateRefreshTokenAndGetUserId(refreshToken);
69
70         User user = userRepository.findByIdAndDeletedAtIsNull(userId)
71             .orElseThrow(() -> new RunwayException(ErrorCode.USER_NOT_FOUND));
72
73         String incomingHash = jwtProvider.hashToken(refreshToken);
74         if (!incomingHash.equals(user.getRefreshTokenHash())) {
75             // DB의 해시와 불일치 → 이미 rotate되었거나 탈취된 토큰
76             throw new RunwayException(ErrorCode.INVALID_TOKEN);
77         }
78
79         String newAccessToken = jwtProvider.generateAccessToken(user.getId(), user.getEmail());
80         String newRefreshToken = jwtProvider.generateRefreshToken(user.getId(), user.getEmail());
81
82         user.updateRefreshTokenHash(jwtProvider.hashToken(newRefreshToken));
83         log.info("Token reissued for user: {}", user.getEmail());

```

```

84         return ReissueResponse.of(newAccessToken, newRefreshToken);
85     }
86
87     @Transactional
88     public void logout(UUID userId) {
89         User user = userRepository.findByIdAndDeletedAtIsNull(userId)
90             .orElseThrow(() -> new RunwayException(ErrorCode.USER_NOT_FOUND));
91
92         user.updateRefreshTokenHash(null);
93         log.info("User logged out: {}", user.getEmail());
94     }
95 }

```

backend/src/main/java/com/runway/common/config/WebConfig.java

Java | 19줄

```

1  package com.runway.common.config;
2
3  import org.springframework.beans.factory.annotation.Value;
4  import org.springframework.context.annotation.Configuration;
5  import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
6  import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
7
8  @Configuration
9  public class WebConfig implements WebMvcConfigurer {
10
11     @Value("${app.upload-dir:/opt/runway/uploads}")
12     private String uploadDir;
13
14     @Override
15     public void addResourceHandlers(ResourceHandlerRegistry registry) {
16         registry.addResourceHandler("/uploads/**")
17             .addResourceLocations("file:" + uploadDir + "/");
18     }
19 }

```



```
1 package com.runway.common.exception;
2
3 import lombok.Getter;
4 import lombok.RequiredArgsConstructor;
5 import org.springframework.http.HttpStatus;
6
7 @Getter
8 @RequiredArgsConstructor
9 public enum ErrorCode {
10
11     // 400 Bad Request
12     INVALID_REQUEST(HttpStatus.BAD_REQUEST, "요청 형식이 올바르지 않습니다."),
13     VALIDATION_ERROR(HttpStatus.BAD_REQUEST, "입력값 검증에 실패했습니다."),
14
15     // 401 Unauthorized
16     UNAUTHORIZED(HttpStatus.UNAUTHORIZED, "인증이 필요합니다."),
17     INVALID_TOKEN(HttpStatus.UNAUTHORIZED, "유효하지 않은 토큰입니다."),
18     EXPIRED_TOKEN(HttpStatus.UNAUTHORIZED, "만료된 토큰입니다."),
19
20     // 403 Forbidden
21     FORBIDDEN(HttpStatus.FORBIDDEN, "접근 권한이 없습니다."),
22
23     // 404 Not Found
24     USER_NOT_FOUND(HttpStatus.NOT_FOUND, "사용자를 찾을 수 없습니다."),
25     RUN_NOT_FOUND(HttpStatus.NOT_FOUND, "러닝 기록을 찾을 수 없습니다."),
26     COURSE_NOT_FOUND(HttpStatus.NOT_FOUND, "코스를 찾을 수 없습니다."),
27     COURSE_ATTEMPT_NOT_FOUND(HttpStatus.NOT_FOUND, "코스 시도를 찾을 수 없습니다."),
28
29     // 409 Conflict
30     DUPLICATED_EMAIL(HttpStatus.CONFLICT, "이미 사용 중인 이메일입니다."),
31     DUPLICATED_NICKNAME(HttpStatus.CONFLICT, "이미 사용 중인 닉네임입니다."),
32     INVALID_RUN_STATUS(HttpStatus.CONFLICT, "현재 러닝 상태에서는 해당 동작을 수행할 수 없습니다."),
33     INVALID_COURSE_STATUS(HttpStatus.CONFLICT, "현재 코스 상태에서는 해당 동작을 수행할 수 없습니다."),
34     INVALID_ATTEMPT_STATUS(HttpStatus.CONFLICT, "현재 코스 시도 상태에서는 해당 동작을 수행할 수 없습니다."),
35     NOT_COMPLETED_RUN(HttpStatus.CONFLICT, "완료된 러닝 기록만 코스로 생성할 수 있습니다."),
36     IMPOSSIBLE_SPEED(HttpStatus.CONFLICT, "기록된 속도가 물리적으로 불가능한 값입니다."),
37     ALREADY_REPORTED(HttpStatus.CONFLICT, "이미 신고한 코스입니다."),
38     INVALID_REPORT_REASON(HttpStatus.BAD_REQUEST, "유효하지 않은 신고 사유입니다."),
39     COURSE_PUBLISH_METADATA_REQUIRED(HttpStatus.UNPROCESSABLE_ENTITY, "공개에 필요한 메타데이터를 모두 입력해주세요."),
40     COURSE_PUBLISH_NOT_ENOUGH_COMPLETIONS(HttpStatus.UNPROCESSABLE_ENTITY, "공개하려면 이 코스를 10회 이상 완주해야 합니다."),
41
42     // 500 Internal Server Error
43     INTERNAL_SERVER_ERROR(HttpStatus.INTERNAL_SERVER_ERROR, "서버 내부 오류가 발생했습니다.");
44
45     private final HttpStatus httpStatus;
46     private final String message;
47 }
```

```

1  package com.runway.common.exception;
2
3  import com.runway.common.response.ApiResponse;
4  import lombok.extern.slf4j.Slf4j;
5  import org.springframework.http.ResponseEntity;
6  import org.springframework.http.converter.HttpMessageNotReadableException;
7  import org.springframework.validation.FieldError;
8  import org.springframework.web.bind.MethodArgumentNotValidException;
9  import org.springframework.web.bind.MissingServletRequestParameterException;
10 import org.springframework.web.bind.annotation.ExceptionHandler;
11 import org.springframework.web.bind.annotation.RestControllerAdvice;
12
13 @Slf4j
14 @RestControllerAdvice
15 public class GlobalExceptionHandler {
16
17     @ExceptionHandler(RunwayException.class)
18     public ResponseEntity<ApiResponse<?>> handleRunwayException(RunwayException e) {
19         ErrorCode code = e.getErrorCode();
20         log.warn("RunwayException [{{}: {}]", code.name(), e.getMessage());
21         return ResponseEntity
22             .status(code.getHttpStatus())
23             .body(ApiResponse.error(e.getMessage(), code.name()));
24     }
25
26     // @Valid, @Validated 검증 실패
27     @ExceptionHandler(MethodArgumentNotValidException.class)
28     public ResponseEntity<ApiResponse<?>> handleValidation(MethodArgumentNotValidException e) {
29         String message = e.getBindingResult().getFieldErrors().stream()
30             .map(FieldError::getDefaultMessage)
31             .findFirst()
32             .orElse(ErrorCode.VALIDATION_ERROR.getMessage());
33         log.warn("ValidationException: {}", message);
34         return ResponseEntity
35             .status(ErrorCode.VALIDATION_ERROR.getHttpStatus())
36             .body(ApiResponse.error(message, ErrorCode.VALIDATION_ERROR.name()));
37     }
38
39     // JSON 파싱 실패 (잘못된 형식의 요청 바디)
40     @ExceptionHandler(HttpMessageNotReadableException.class)
41     public ResponseEntity<ApiResponse<?>> handleHttpMessageNotReadable(HttpMessageNotReadableException e) {
42         log.warn("HttpMessageNotReadableException: {}", e.getMessage());
43         return ResponseEntity
44             .status(ErrorCode.INVALID_REQUEST.getHttpStatus())
45             .body(ApiResponse.error(ErrorCode.INVALID_REQUEST.getMessage(), ErrorCode.INVALID_REQUEST.name()));
46     }
47
48     // 필수 query parameter 누락
49     @ExceptionHandler(MissingServletRequestParameterException.class)
50     public ResponseEntity<ApiResponse<?>> handleMissingParam(MissingServletRequestParameterException e) {
51         String message = "필수 파라미터가 누락되었습니다: " + e.getParameterName();
52         log.warn("MissingServletRequestParameterException: {}", message);
53         return ResponseEntity
54             .status(ErrorCode.INVALID_REQUEST.getHttpStatus())
55             .body(ApiResponse.error(message, ErrorCode.INVALID_REQUEST.name()));
56     }
57
58     // 그 외 예상치 못한 예외
59     @ExceptionHandler(Exception.class)
60     public ResponseEntity<ApiResponse<?>> handleException(Exception e) {
61         log.error("Unexpected exception: {}", e.getMessage(), e);
62         return ResponseEntity
63             .status(ErrorCode.INTERNAL_SERVER_ERROR.getHttpStatus())
64             .body(ApiResponse.error(
65                 ErrorCode.INTERNAL_SERVER_ERROR.getMessage(),
66                 ErrorCode.INTERNAL_SERVER_ERROR.name()
67             ));
68     }
69 }

```

backend/src/main/java/com/runway/common/exception/RunwayException.java

Java | 19줄

```
1 package com.runway.common.exception;
2
3 import lombok.Getter;
4
5 @Getter
6 public class RunwayException extends RuntimeException {
7
8     private final ErrorCode errorCode;
9
10    public RunwayException(ErrorCode errorCode) {
11        super(errorCode.getMessage());
12        this.errorCode = errorCode;
13    }
14
15    public RunwayException(ErrorCode errorCode, String customMessage) {
16        super(customMessage);
17        this.errorCode = errorCode;
18    }
19 }
```

backend/src/main/java/com/runway/common/response/ApiResponse.java

Java | 35줄

```
1 package com.runway.common.response;
2
3 import com.fasterxml.jackson.annotation.JsonInclude;
4 import lombok.Getter;
5
6 @Getter
7 public class ApiResponse<T> {
8
9     private final boolean success;
10    private final String message;
11    private final T data;
12
13    // 에러 응답에만 포함. null이면 JSON 직렬화에서 제외
14    @JsonInclude(JsonInclude.Include.NON_NULL)
15    private final String errorCode;
16
17    private ApiResponse(boolean success, String message, T data, String errorCode) {
18        this.success = success;
19        this.message = message;
20        this.data = data;
21        this.errorCode = errorCode;
22    }
23
24    public static <T> ApiResponse<T> success(String message, T data) {
25        return new ApiResponse<>(true, message, data, null);
26    }
27
28    public static <T> ApiResponse<T> success(String message) {
29        return new ApiResponse<>(true, message, null, null);
30    }
31
32    public static <T> ApiResponse<T> error(String message, String errorCode) {
33        return new ApiResponse<>(false, message, null, errorCode);
34    }
35 }
```

backend/src/main/java/com/runway/common/response/PageResponse.java

Java | 36줄

```
1 package com.runway.common.response;
2
3 import lombok.Getter;
4 import org.springframework.data.domain.Page;
5
6 import java.util.List;
7
8 @Getter
9 public class PageResponse<T> {
10
11     private final List<T> content;
12     private final int page;
13     private final int size;
14     private final long totalElements;
15     private final int totalPages;
16     private final boolean hasNext;
17
18     private PageResponse(List<T> content, int page, int size, long totalElements, int totalPages) {
19         this.content = content;
20         this.page = page;
21         this.size = size;
22         this.totalElements = totalElements;
23         this.totalPages = totalPages;
24         this.hasNext = page < totalPages - 1;
25     }
26
27     public static <T> PageResponse<T> from(Page<T> springPage) {
28         return new PageResponse<> (
29             springPage.getContent(),
30             springPage.getNumber(),
31             springPage.getSize(),
32             springPage.getTotalElements(),
33             springPage.getTotalPages()
34         );
35     }
36 }
```

backend/src/main/java/com/runway/common/security/CustomUserDetailsService.java

Java | 28줄

```
1 package com.runway.common.security;
2
3 import com.runway.user.domain.User;
4 import com.runway.user.repository.UserRepository;
5 import lombok.RequiredArgsConstructor;
6 import org.springframework.security.core.userdetails.UserDetails;
7 import org.springframework.security.core.userdetails.UserDetailsService;
8 import org.springframework.security.core.userdetails.UsernameNotFoundException;
9 import org.springframework.stereotype.Service;
10 import org.springframework.transaction.annotation.Transactional;
11
12 import java.util.UUID;
13
14 @Service
15 @RequiredArgsConstructor
16 public class CustomUserDetailsService implements UserDetailsService {
17
18     private final UserRepository userRepository;
19
20     // username 파라미터에 userId(UUID 문자열)를 전달하는 방식으로 사용
21     @Override
22     @Transactional(readOnly = true)
23     public UserDetails loadUserByUsername(String userId) throws UsernameNotFoundException {
24         User user = userRepository.findByIdAndDeletedAtIsNull(UUID.fromString(userId))
25             .orElseThrow(() -> new UsernameNotFoundException("User not found: " + userId));
26         return new UserPrincipal(user.getId(), user.getEmail());
27     }
28 }
```

backend/src/main/java/com/runway/common/security/JwtAuthenticationFilter.java

Java | 53줄

```
1 package com.runway.common.security;
2
3 import jakarta.servlet.FilterChain;
4 import jakarta.servlet.ServletException;
5 import jakarta.servlet.http.HttpServletRequest;
6 import jakarta.servlet.http.HttpServletResponse;
7 import lombok.RequiredArgsConstructor;
8 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
9 import org.springframework.security.core.context.SecurityContextHolder;
10 import org.springframework.security.core.userdetails.UserDetails;
11 import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
12 import org.springframework.stereotype.Component;
13 import org.springframework.util.StringUtils;
14 import org.springframework.web.filter.OncePerRequestFilter;
15
16 import java.io.IOException;
17 import java.util.UUID;
18
19 @Component
20 @RequiredArgsConstructor
21 public class JwtAuthenticationFilter extends OncePerRequestFilter {
22
23     private final JwtProvider jwtProvider;
24     private final CustomUserDetailsService userDetailsService;
25
26     @Override
27     protected void doFilterInternal(HttpServletRequest request,
28                                     HttpServletResponse response,
29                                     FilterChain filterChain) throws ServletException, IOException {
30         String token = resolveToken(request);
31
32         if (token != null && jwtProvider.isValidAccessToken(token)) {
33             UUID userId = jwtProvider.getUserIdFromToken(token);
34             UserDetails userDetails = userDetailsService.loadUserByUsername(userId.toString());
35
36             UsernamePasswordAuthenticationToken authentication =
37                 new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities());
38             authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
39
40             SecurityContextHolder.getContext().setAuthentication(authentication);
41         }
42
43         filterChain.doFilter(request, response);
44     }
45
46     private String resolveToken(HttpServletRequest request) {
47         String bearer = request.getHeader("Authorization");
48         if (StringUtils.hasText(bearer) && bearer.startsWith("Bearer ")) {
49             return bearer.substring(7);
50         }
51         return null;
52     }
53 }
```

backend/src/main/java/com/runway/common/security/JwtProperties.java

Java | 17줄

```
1 package com.runway.common.security;
2
3 import lombok.Getter;
4 import lombok.Setter;
5 import org.springframework.boot.context.properties.ConfigurationProperties;
6 import org.springframework.stereotype.Component;
7
8 @Component
9 @ConfigurationProperties(prefix = "jwt")
10 @Getter
11 @Setter
12 public class JwtProperties {
13
14     private String secret;
15     private long accessTokenExpiry;
16     private long refreshTokenExpiry;
17 }
```

```

1  package com.runway.common.security;
2
3  import com.runway.common.exception.ErrorCode;
4  import com.runway.common.exception.RunwayException;
5  import io.jsonwebtoken.Claims;
6  import io.jsonwebtoken.ExpiredJwtException;
7  import io.jsonwebtoken.JwtException;
8  import io.jsonwebtoken.Jwts;
9  import io.jsonwebtoken.security.Keys;
10 import jakarta.annotation.PostConstruct;
11 import lombok.RequiredArgsConstructor;
12 import lombok.extern.slf4j.Slf4j;
13 import org.springframework.stereotype.Component;
14
15 import javax.crypto.SecretKey;
16 import java.nio.charset.StandardCharsets;
17 import java.security.MessageDigest;
18 import java.security.NoSuchAlgorithmException;
19 import java.util.Base64;
20 import java.util.Date;
21 import java.util.UUID;
22
23 @Slf4j
24 @Component
25 @RequiredArgsConstructor
26 public class JwtProvider {
27
28     private final JwtProperties jwtProperties;
29
30     private SecretKey secretKey;
31
32     @PostConstruct
33     private void init() {
34         byte[] keyBytes = jwtProperties.getSecret().getBytes(StandardCharsets.UTF_8);
35         this.secretKey = Keys.hmacShaKeyFor(keyBytes);
36     }
37
38     private static final String CLAIM_TOKEN_TYPE = "tokenType";
39     private static final String TYPE_ACCESS = "access";
40     private static final String TYPE_REFRESH = "refresh";
41
42     public String generateAccessToken(UUID userId, String email) {
43         return buildToken(userId, email, jwtProperties.getAccessTokenExpiry(), TYPE_ACCESS);
44     }
45
46     public String generateRefreshToken(UUID userId, String email) {
47         return buildToken(userId, email, jwtProperties.getRefreshTokenExpiry(), TYPE_REFRESH);
48     }
49
50     /**
51      * Bearer 토큰 전용. 서명·만료 검증 + tokenType == "access" 까지 확인한다.
52      * refresh token을 Bearer 인증에 사용하는 것을 차단한다.
53      */
54     public boolean isValidAccessToken(String token) {
55         try {
56             Claims claims = parseClaims(token);
57             return TYPE_ACCESS.equals(claims.get(CLAIM_TOKEN_TYPE, String.class));
58         } catch (JwtException | IllegalArgumentException e) {
59             return false;
60         }
61     }
62
63     public UUID getUserIdFromToken(String token) {
64         return UUID.fromString(parseClaims(token).getSubject());
65     }
66
67     // reissue 전용: 만료 vs 위조를 구분하고 tokenType == "refresh" 를 추가로 검증한다
68     public UUID validateRefreshTokenAndGetUserId(String token) {
69         try {
70             Claims claims = parseClaims(token);
71             if (!TYPE_REFRESH.equals(claims.get(CLAIM_TOKEN_TYPE, String.class))) {
72                 throw new RunwayException(ErrorCode.INVALID_TOKEN);
73             }
74             return UUID.fromString(claims.getSubject());
75         } catch (RunwayException e) {
76             throw e;
77         } catch (ExpiredJwtException e) {
78             throw new RunwayException(ErrorCode.EXPIRED_TOKEN);
79         } catch (JwtException | IllegalArgumentException e) {
80             throw new RunwayException(ErrorCode.INVALID_TOKEN);
81         }
82     }
83

```

```
84 public String getEmailFromToken(String token) {
85     return parseClaims(token).get("email", String.class);
86 }
87
88 public String hashToken(String token) {
89     try {
90         MessageDigest digest = MessageDigest.getInstance("SHA-256");
91         byte[] hash = digest.digest(token.getBytes(StandardCharsets.UTF_8));
92         return Base64.getEncoder().encodeToString(hash);
93     } catch (NoSuchAlgorithmException e) {
94         throw new RuntimeException("SHA-256 algorithm not available", e);
95     }
96 }
97
98 private String buildToken(UUID userId, String email, long expiryMs, String tokenType) {
99     Date now = new Date();
100     Date expiration = new Date(now.getTime() + expiryMs);
101     return Jwts.builder()
102         .subject(userId.toString())
103         .claim("email", email)
104         .claim(CLAIM_TOKEN_TYPE, tokenType)
105         .issuedAt(now)
106         .expiration(expiration)
107         .signWith(secretKey)
108         .compact();
109 }
110
111 private Claims parseClaims(String token) {
112     return Jwts.parser()
113         .verifyWith(secretKey)
114         .build()
115         .parseSignedClaims(token)
116         .getPayload();
117 }
118 }
```

```

1  package com.runway.common.security;
2
3  import com.fasterxml.jackson.databind.ObjectMapper;
4  import jakarta.servlet.http.HttpServletResponse;
5  import lombok.RequiredArgsConstructor;
6  import org.springframework.context.annotation.Bean;
7  import org.springframework.context.annotation.Configuration;
8  import org.springframework.http.HttpStatus;
9  import org.springframework.http.MediaType;
10 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
11 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
12 import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
13 import org.springframework.security.config.http.SessionCreationPolicy;
14 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
15 import org.springframework.security.crypto.password.PasswordEncoder;
16 import org.springframework.security.web.SecurityFilterChain;
17 import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
18
19 import java.util.Map;
20
21 @Configuration
22 @EnableWebSecurity
23 @RequiredArgsConstructor
24 public class SecurityConfig {
25
26     private final JwtAuthenticationFilter jwtAuthenticationFilter;
27     private final ObjectMapper objectMapper;
28
29     @Bean
30     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
31         http
32             .csrf(AbstractHttpConfigurer::disable)
33             .sessionManagement(session ->
34                 session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
35             .authorizeHttpRequests(auth -> auth
36                 .requestMatchers(
37                     "/api/auth/signup",
38                     "/api/auth/login",
39                     "/api/auth/reissue",
40                     "/swagger-ui/**",
41                     "/swagger-ui.html",
42                     "/v3/api-docs/**",
43                     "/uploads/**"
44                 ).permitAll()
45                 .anyRequest().authenticated()
46             )
47             .exceptionHandling(ex -> ex
48                 .authenticationEntryPoint((request, response, authException) ->
49                     writeErrorResponse(response, HttpStatus.UNAUTHORIZED,
50                         "인증이 필요합니다.", "UNAUTHORIZED"))
51                 .accessDeniedHandler((request, response, accessDeniedException) ->
52                     writeErrorResponse(response, HttpStatus.FORBIDDEN,
53                         "접근 권한이 없습니다.", "FORBIDDEN"))
54             )
55             .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
56
57         return http.build();
58     }
59
60     @Bean
61     public PasswordEncoder passwordEncoder() {
62         return new BCryptPasswordEncoder();
63     }
64
65     private void writeErrorResponse(HttpServletResponse response,
66                                     HttpStatus status,
67                                     String message,
68                                     String errorCode) {
69         try {
70             response.setStatus(status.value());
71             response.setContentType(MediaType.APPLICATION_JSON_VALUE);
72             response.setCharacterEncoding("UTF-8");
73
74             Map<String, Object> body = Map.of(
75                 "success", false,
76                 "message", message,
77                 "errorCode", errorCode
78             );
79             response.getWriter().write(objectMapper.writeValueAsString(body));
80         } catch (Exception e) {
81             response.setStatus(HttpStatus.INTERNAL_SERVER_ERROR);
82         }
83     }

```



```
83     }
84 }
```

backend/src/main/java/com/runway/common/security/UserPrincipal.java

Java | 57줄

```
1  package com.runway.common.security;
2
3  import lombok.Getter;
4  import org.springframework.security.core.GrantedAuthority;
5  import org.springframework.security.core.authority.SimpleGrantedAuthority;
6  import org.springframework.security.core.userdetails.UserDetails;
7
8  import java.util.Collection;
9  import java.util.List;
10 import java.util.UUID;
11
12 @Getter
13 public class UserPrincipal implements UserDetails {
14
15     private final UUID userId;
16     private final String email;
17
18     public UserPrincipal(UUID userId, String email) {
19         this.userId = userId;
20         this.email = email;
21     }
22
23     @Override
24     public Collection<? extends GrantedAuthority> getAuthorities() {
25         return List.of(new SimpleGrantedAuthority("ROLE_USER"));
26     }
27
28     @Override
29     public String getPassword() {
30         return null;
31     }
32
33     @Override
34     public String getUsername() {
35         return email;
36     }
37
38     @Override
39     public boolean isAccountNonExpired() {
40         return true;
41     }
42
43     @Override
44     public boolean isAccountNonLocked() {
45         return true;
46     }
47
48     @Override
49     public boolean isCredentialsNonExpired() {
50         return true;
51     }
52
53     @Override
54     public boolean isEnabled() {
55         return true;
56     }
57 }
```

```

1  package com.runway.course.controller;
2
3  import com.runway.common.response.ApiResponse;
4  import com.runway.common.response.PageResponse;
5  import com.runway.common.security.UserPrincipal;
6  import com.runway.course.dto.*;
7  import com.runway.course.service.CourseReportService;
8  import com.runway.course.service.CourseService;
9  import io.swagger.v3.oas.annotations.Operation;
10 import io.swagger.v3.oas.annotations.security.SecurityRequirement;
11 import io.swagger.v3.oas.annotations.tags.Tag;
12 import jakarta.validation.Valid;
13 import lombok.RequiredArgsConstructor;
14 import org.springframework.http.ResponseEntity;
15 import org.springframework.security.core.annotation.AuthenticationPrincipal;
16 import org.springframework.web.bind.annotation.*;
17
18 import java.util.UUID;
19 import org.springframework.http.HttpStatus;
20
21 @Tag(name = "Course", description = "코스 API")
22 @RestController
23 @RequestMapping("/api/courses")
24 @RequiredArgsConstructor
25 public class CourseController {
26
27     private final CourseService courseService;
28     private final CourseReportService courseReportService;
29     private final com.runway.course.service.CourseRatingService courseRatingService;
30
31     @Operation(summary = "러닝 기록 기반 코스 생성", security = @SecurityRequirement(name = "bearerAuth"))
32     @PostMapping("/from-run/{runId}")
33     public ResponseEntity<ApiResponse<CourseResponse>> createFromRun(
34         @AuthenticationPrincipal UserPrincipal principal,
35         @PathVariable UUID runId,
36         @Valid @RequestBody CreateCourseFromRunRequest request) {
37         CourseResponse data = courseService.createCourseFromRun(
38             principal.getUserId(), runId, request);
39         return ResponseEntity.ok(ApiResponse.success("코스가 생성되었습니다.", data));
40     }
41
42     @Operation(summary = "인근 코스 조회", security = @SecurityRequirement(name = "bearerAuth"))
43     @GetMapping("/nearby")
44     public ResponseEntity<ApiResponse<PageResponse<NearbyCourseItem>>> getNearby(
45         @AuthenticationPrincipal UserPrincipal principal,
46         @RequestParam Double latitude,
47         @RequestParam Double longitude,
48         @RequestParam(defaultValue = "3000") Double radiusMeters,
49         @RequestParam(required = false) Double minDistanceMeters,
50         @RequestParam(required = false) Double maxDistanceMeters,
51         @RequestParam(required = false) Boolean isLoop,
52         @RequestParam(required = false) String keyword,
53         @RequestParam(defaultValue = "true") boolean includeRoutePoints,
54         @RequestParam(defaultValue = "false") boolean includeExactStartPoint,
55         @RequestParam(defaultValue = "0") int page,
56         @RequestParam(defaultValue = "20") int size) {
57         PageResponse<NearbyCourseItem> data = courseService.getNearby(
58             latitude, longitude, radiusMeters,
59             minDistanceMeters, maxDistanceMeters, isLoop, keyword,
60             includeRoutePoints, includeExactStartPoint,
61             page, size);
62         return ResponseEntity.ok(ApiResponse.success("인근 코스 조회에 성공했습니다.", data));
63     }
64
65     @Operation(summary = "내가 만든 코스 목록 조회", security = @SecurityRequirement(name = "bearerAuth"))
66     @GetMapping("/me")
67     public ResponseEntity<ApiResponse<PageResponse<CourseResponse>>> getMyCourses(
68         @AuthenticationPrincipal UserPrincipal principal,
69         @RequestParam(required = false) String status,
70         @RequestParam(defaultValue = "0") int page,
71         @RequestParam(defaultValue = "20") int size) {
72         PageResponse<CourseResponse> data = courseService.getMyCourses(
73             principal.getUserId(), status, page, size);
74         return ResponseEntity.ok(ApiResponse.success("내 코스 목록 조회에 성공했습니다.", data));
75     }
76
77     @Operation(summary = "즐거찾기 코스 목록 조회", security = @SecurityRequirement(name = "bearerAuth"))
78     @GetMapping("/favorites")
79     public ResponseEntity<ApiResponse<PageResponse<CourseResponse>>> getFavoriteCourses(
80         @AuthenticationPrincipal UserPrincipal principal,
81         @RequestParam(defaultValue = "0") int page,
82         @RequestParam(defaultValue = "20") int size) {
83         PageResponse<CourseResponse> data = courseService.getFavoriteCourses(

```

```

84         principal.getUserId(), page, size);
85     return ResponseEntity.ok(ApiResponse.success("즐거찾기 코스 목록 조회에 성공했습니다.", data));
86 }
87
88 @Operation(summary = "참여한 코스 목록 조회", security = @SecurityRequirement(name = "bearerAuth"))
89 @GetMapping("/participated")
90 public ResponseEntity<ApiResponse<PageResponse<ParticipatedCourseItem>>> getParticipatedCourses(
91     @AuthenticationPrincipal UserPrincipal principal,
92     @RequestParam(defaultValue = "0") int page,
93     @RequestParam(defaultValue = "20") int size) {
94     PageResponse<ParticipatedCourseItem> data = courseService.getParticipatedCourses(
95         principal.getUserId(), page, size);
96     return ResponseEntity.ok(ApiResponse.success("참여한 코스 목록 조회에 성공했습니다.", data));
97 }
98
99 @Operation(summary = "코스 상세 조회", security = @SecurityRequirement(name = "bearerAuth"))
100 @GetMapping("/{courseId}")
101 public ResponseEntity<ApiResponse<CourseDetailResponse>> getCourseDetail(
102     @AuthenticationPrincipal UserPrincipal principal,
103     @PathVariable UUID courseId) {
104     CourseDetailResponse data = courseService.getCourseDetail(
105         principal.getUserId(), courseId);
106     return ResponseEntity.ok(ApiResponse.success("코스 상세 조회에 성공했습니다.", data));
107 }
108
109 @Operation(summary = "코스 경로 포인트 조회", security = @SecurityRequirement(name = "bearerAuth"))
110 @GetMapping("/{courseId}/points")
111 public ResponseEntity<ApiResponse<CoursePointsResponse>> getCoursePoints(
112     @AuthenticationPrincipal UserPrincipal principal,
113     @PathVariable UUID courseId) {
114     CoursePointsResponse data = courseService.getCoursePoints(
115         principal.getUserId(), courseId);
116     return ResponseEntity.ok(ApiResponse.success("코스 경로 조회에 성공했습니다.", data));
117 }
118
119 @Operation(summary = "코스 기본 정보 수정", security = @SecurityRequirement(name = "bearerAuth"))
120 @PutMapping("/{courseId}")
121 public ResponseEntity<ApiResponse<CourseResponse>> updateCourse(
122     @AuthenticationPrincipal UserPrincipal principal,
123     @PathVariable UUID courseId,
124     @Valid @RequestBody UpdateCourseRequest request) {
125     CourseResponse data = courseService.updateCourse(
126         principal.getUserId(), courseId, request);
127     return ResponseEntity.ok(ApiResponse.success("코스가 수정되었습니다.", data));
128 }
129
130 @Operation(summary = "코스 공개", security = @SecurityRequirement(name = "bearerAuth"))
131 @PatchMapping("/{courseId}/publish")
132 public ResponseEntity<ApiResponse<CourseStatusResponse>> publishCourse(
133     @AuthenticationPrincipal UserPrincipal principal,
134     @PathVariable UUID courseId,
135     @Valid @RequestBody PublishCourseRequest request) {
136     CourseStatusResponse data = courseService.publishCourse(
137         principal.getUserId(), courseId, request);
138     return ResponseEntity.ok(ApiResponse.success("코스가 공개되었습니다.", data));
139 }
140
141 @Operation(summary = "코스 보관 처리", security = @SecurityRequirement(name = "bearerAuth"))
142 @PatchMapping("/{courseId}/archive")
143 public ResponseEntity<ApiResponse<CourseStatusResponse>> archiveCourse(
144     @AuthenticationPrincipal UserPrincipal principal,
145     @PathVariable UUID courseId) {
146     CourseStatusResponse data = courseService.archiveCourse(
147         principal.getUserId(), courseId);
148     return ResponseEntity.ok(ApiResponse.success("코스가 보관 처리되었습니다.", data));
149 }
150
151 @Operation(summary = "초안 코스 삭제", security = @SecurityRequirement(name = "bearerAuth"))
152 @DeleteMapping("/{courseId}")
153 public ResponseEntity<ApiResponse<Void>> deleteDraftCourse(
154     @AuthenticationPrincipal UserPrincipal principal,
155     @PathVariable UUID courseId) {
156     courseService.deleteDraftCourse(principal.getUserId(), courseId);
157     return ResponseEntity.ok(ApiResponse.success("코스가 삭제되었습니다.", null));
158 }
159
160 @Operation(summary = "코스 신고", security = @SecurityRequirement(name = "bearerAuth"))
161 @PostMapping("/{courseId}/reports")
162 public ResponseEntity<ApiResponse<CourseReportResponse>> reportCourse(
163     @AuthenticationPrincipal UserPrincipal principal,
164     @PathVariable UUID courseId,
165     @Valid @RequestBody CourseReportRequest request) {
166     CourseReportResponse data = courseReportService.reportCourse(
167         principal.getUserId(), courseId, request);
168     return ResponseEntity.ok(ApiResponse.success("신고가 접수되었습니다.", data));
169 }

```

```

170
171     @Operation(summary = "코스 평점 등록/수정", security = @SecurityRequirement(name = "bearerAuth"))
172     @PostMapping("/{courseId}/ratings")
173     public ResponseEntity<ApiResponse<CourseRatingResponse>> rateCourse(
174         @AuthenticationPrincipal UserPrincipal principal,
175         @PathVariable UUID courseId,
176         @Valid @RequestBody CourseRatingRequest request) {
177         CourseRatingResponse data = courseRatingService.rateCourse(
178             principal.getUserId(), courseId, request);
179         return ResponseEntity.ok(ApiResponse.success("평점이 등록되었습니다.", data));
180     }
181
182     @Operation(summary = "코스 즐겨찾기 추가", security = @SecurityRequirement(name = "bearerAuth"))
183     @PostMapping("/{courseId}/favorite")
184     public ResponseEntity<ApiResponse<Void>> addFavorite(
185         @AuthenticationPrincipal UserPrincipal principal,
186         @PathVariable UUID courseId) {
187         courseService.addFavorite(principal.getUserId(), courseId);
188         return ResponseEntity.status(HttpStatus.CREATED)
189             .body(ApiResponse.success("즐거찾기에 추가되었습니다.", null));
190     }
191
192     @Operation(summary = "코스 즐겨찾기 제거", security = @SecurityRequirement(name = "bearerAuth"))
193     @DeleteMapping("/{courseId}/favorite")
194     public ResponseEntity<ApiResponse<Void>> removeFavorite(
195         @AuthenticationPrincipal UserPrincipal principal,
196         @PathVariable UUID courseId) {
197         courseService.removeFavorite(principal.getUserId(), courseId);
198         return ResponseEntity.ok(ApiResponse.success("즐거찾기에서 제거되었습니다.", null));
199     }
200 }

```

```

1  package com.runway.course.domain;
2
3  import com.runway.course.domain.enums.CourseDifficulty;
4  import com.runway.course.domain.enums.CourseRecommendedTime;
5  import com.runway.course.domain.enums.CourseRiskLevel;
6  import com.runway.course.domain.enums.CourseSlopeLevel;
7  import com.runway.course.domain.enums.CourseSurfaceType;
8  import com.runway.course.domain.enums.CourseStatus;
9  import jakarta.persistence.*;
10 import lombok.AccessLevel;
11 import lombok.Builder;
12 import lombok.Getter;
13 import lombok.NoArgsConstructor;
14 import org.locationtech.jts.geom.LineString;
15 import org.locationtech.jts.geom.Point;
16
17 import java.time.Instant;
18 import java.util.UUID;
19
20 @Entity
21 @Table(name = "courses")
22 @Getter
23 @NoArgsConstructor(access = AccessLevel.PROTECTED)
24 public class Course {
25
26     @Id
27     @GeneratedValue(strategy = GenerationType.UUID)
28     @Column(columnDefinition = "uuid", updatable = false, nullable = false)
29     private UUID id;
30
31     @Column(name = "creator_id", nullable = false, columnDefinition = "uuid")
32     private UUID creatorId;
33
34     @Column(name = "source_record_id", columnDefinition = "uuid")
35     private UUID sourceRecordId;
36
37     @Column(nullable = false, length = 100)
38     private String name;
39
40     @Column(columnDefinition = "TEXT")
41     private String description;
42
43     // CourseStatus.JpaConverter autoApply=true - @Convert 불필요
44     @Column(nullable = false, length = 30)
45     private CourseStatus status;
46
47     @Column(name = "distance_meters", nullable = false)
48     private Double distanceMeters;
49
50     @Column(name = "is_loop", nullable = false)
51     private Boolean isLoop;
52
53     @Column(name = "start_location", columnDefinition = "geography(Point,4326)", nullable = false)
54     private Point startLocation;
55
56     @Column(name = "end_location", columnDefinition = "geography(Point,4326)", nullable = false)
57     private Point endLocation;
58
59     @Column(columnDefinition = "geography(LineString,4326)")
60     private LineString path;
61
62     @Column(name = "difficulty", length = 20)
63     private CourseDifficulty difficulty;
64
65     @Column(name = "slope_level", length = 20)
66     private CourseSlopeLevel slopeLevel;
67
68     @Column(name = "risk_level", length = 20)
69     private CourseRiskLevel riskLevel;
70
71     @Column(name = "surface_type", length = 20)
72     private CourseSurfaceType surfaceType;
73
74     @Column(name = "recommended_time", length = 20)
75     private CourseRecommendedTime recommendedTime;
76
77     @Column(name = "warnings", columnDefinition = "TEXT")
78     private String warnings;
79
80     @Column(name = "attempt_count", nullable = false)
81     private Integer attemptCount;
82
83     @Column(name = "completion_count", nullable = false)

```

```

84     private Integer completionCount;
85
86     @Column(name = "created_at", nullable = false, updatable = false)
87     private Instant createdAt;
88
89     @Column(name = "updated_at", nullable = false)
90     private Instant updatedAt;
91
92     @Column(name = "deleted_at")
93     private Instant deletedAt;
94
95     @Builder
96     private Course(UUID creatorId, UUID sourceRecordId, String name, String description,
97                   CourseStatus status, Double distanceMeters, Boolean isLoop,
98                   Point startLocation, Point endLocation) {
99         this.creatorId = creatorId;
100        this.sourceRecordId = sourceRecordId;
101        this.name = name;
102        this.description = description;
103        this.status = status;
104        this.distanceMeters = distanceMeters != null ? distanceMeters : 0.0;
105        this.isLoop = Boolean.TRUE.equals(isLoop);
106        this.startLocation = startLocation;
107        this.endLocation = endLocation;
108        this.attemptCount = 0;
109        this.completionCount = 0;
110    }
111
112    @PrePersist
113    private void prePersist() {
114        Instant now = Instant.now();
115        this.createdAt = now;
116        this.updatedAt = now;
117    }
118
119    @PreUpdate
120    private void preUpdate() {
121        this.updatedAt = Instant.now();
122    }
123
124    public void updateInfo(String name, String description, Boolean isLoop) {
125        this.name = name;
126        this.description = description;
127        this.isLoop = Boolean.TRUE.equals(isLoop);
128    }
129
130    public void updateMetadata(CourseDifficulty difficulty, CourseSlopeLevel slopeLevel,
131                              CourseRiskLevel riskLevel, CourseSurfaceType surfaceType,
132                              CourseRecommendedTime recommendedTime, String warnings,
133                              String description) {
134        this.difficulty = difficulty;
135        this.slopeLevel = slopeLevel;
136        this.riskLevel = riskLevel;
137        this.surfaceType = surfaceType;
138        this.recommendedTime = recommendedTime;
139        this.warnings = warnings;
140        if (description != null) this.description = description;
141    }
142
143    public boolean hasRequiredPublishMetadata() {
144        return difficulty != null && slopeLevel != null && riskLevel != null
145            && surfaceType != null && recommendedTime != null;
146    }
147
148    public void updatePath(LineString path) {
149        this.path = path;
150    }
151
152    public void publish() {
153        this.status = CourseStatus.PUBLISHED;
154    }
155
156    public void archive() {
157        this.status = CourseStatus.ARCHIVED;
158    }
159
160    public void incrementAttemptCount() {
161        this.attemptCount++;
162    }
163
164    public void incrementCompletionCount() {
165        this.completionCount++;
166    }
167
168    public void softDelete() {
169        this.deletedAt = Instant.now();

```

```

170     }
171
172     public boolean isOwnedBy(UUID userId) {
173         return this.creatorId.equals(userId);
174     }
175
176     public boolean isVisibleTo(UUID userId) {
177         return this.status == CourseStatus.PUBLISHED || this.creatorId.equals(userId);
178     }
179 }

```

backend/src/main/java/com/runway/course/domain/CourseFavorite.java

Java | 40줄

```

1  package com.runway.course.domain;
2
3  import jakarta.persistence.*;
4  import lombok.AccessLevel;
5  import lombok.Builder;
6  import lombok.Getter;
7  import lombok.NoArgsConstructor;
8
9  import java.time.Instant;
10 import java.util.UUID;
11
12 @Entity
13 @Table(name = "course_favorites")
14 @IdClass(CourseFavoriteId.class)
15 @Getter
16 @NoArgsConstructor(access = AccessLevel.PROTECTED)
17 public class CourseFavorite {
18
19     @Id
20     @Column(name = "user_id", nullable = false, columnDefinition = "uuid")
21     private UUID userId;
22
23     @Id
24     @Column(name = "course_id", nullable = false, columnDefinition = "uuid")
25     private UUID courseId;
26
27     @Column(name = "created_at", nullable = false, updatable = false)
28     private Instant createdAt;
29
30     @Builder
31     private CourseFavorite(UUID userId, UUID courseId) {
32         this.userId = userId;
33         this.courseId = courseId;
34     }
35
36     @PrePersist
37     private void prePersist() {
38         this.createdAt = Instant.now();
39     }
40 }

```

backend/src/main/java/com/runway/course/domain/CourseFavoriteId.java

Java | 16줄

```

1  package com.runway.course.domain;
2
3  import lombok.AllArgsConstructor;
4  import lombok.EqualsAndHashCode;
5  import lombok.NoArgsConstructor;
6
7  import java.io.Serializable;
8  import java.util.UUID;
9
10 @EqualsAndHashCode
11 @AllArgsConstructor
12 @NoArgsConstructor
13 public class CourseFavoriteId implements Serializable {
14     private UUID userId;
15     private UUID courseId;
16 }

```

```
1 package com.runway.course.domain;
2
3 import jakarta.persistence.*;
4 import lombok.AccessLevel;
5 import lombok.Builder;
6 import lombok.Getter;
7 import lombok.NoArgsConstructor;
8 import org.locationtech.jts.geom.Point;
9
10 import java.util.UUID;
11
12 @Entity
13 @Table(name = "course_points")
14 @Getter
15 @NoArgsConstructor(access = AccessLevel.PROTECTED)
16 public class CoursePoint {
17
18     @Id
19     @GeneratedValue(strategy = GenerationType.UUID)
20     @Column(columnDefinition = "uuid", updatable = false, nullable = false)
21     private UUID id;
22
23     @Column(name = "course_id", nullable = false, columnDefinition = "uuid")
24     private UUID courseId;
25
26     @Column(nullable = false)
27     private Integer sequence;
28
29     @Column(columnDefinition = "geometry(Point,4326)", nullable = false)
30     private Point location;
31
32     @Column(name = "altitude_meters")
33     private Double altitudeMeters;
34
35     @Builder
36     private CoursePoint(UUID courseId, Integer sequence, Point location, Double altitudeMeters) {
37         this.courseId = courseId;
38         this.sequence = sequence;
39         this.location = location;
40         this.altitudeMeters = altitudeMeters;
41     }
42 }
```



```
1 package com.runway.course.domain;
2
3 import jakarta.persistence.*;
4 import lombok.AccessLevel;
5 import lombok.Builder;
6 import lombok.Getter;
7 import lombok.NoArgsConstructor;
8
9 import java.time.Instant;
10 import java.util.UUID;
11
12 @Entity
13 @Table(name = "course_ratings",
14       uniqueConstraints = @UniqueConstraint(columnNames = {"course_id", "user_id"}))
15 @Getter
16 @NoArgsConstructor(access = AccessLevel.PROTECTED)
17 public class CourseRating {
18
19     @Id
20     @GeneratedValue(strategy = GenerationType.UUID)
21     @Column(columnDefinition = "uuid", updatable = false, nullable = false)
22     private UUID id;
23
24     @Column(name = "course_id", nullable = false, columnDefinition = "uuid")
25     private UUID courseId;
26
27     @Column(name = "user_id", nullable = false, columnDefinition = "uuid")
28     private UUID userId;
29
30     @Column(nullable = false)
31     private Integer rating;
32
33     @Column(columnDefinition = "TEXT")
34     private String comment;
35
36     @Column(name = "created_at", nullable = false, updatable = false)
37     private Instant createdAt;
38
39     @Column(name = "updated_at", nullable = false)
40     private Instant updatedAt;
41
42     @Builder
43     private CourseRating(UUID courseId, UUID userId, Integer rating, String comment) {
44         this.courseId = courseId;
45         this.userId = userId;
46         this.rating = rating;
47         this.comment = comment;
48     }
49
50     @PrePersist
51     private void prePersist() {
52         Instant now = Instant.now();
53         this.createdAt = now;
54         this.updatedAt = now;
55     }
56
57     @PreUpdate
58     private void preUpdate() {
59         this.updatedAt = Instant.now();
60     }
61
62     public void update(Integer rating, String comment) {
63         this.rating = rating;
64         this.comment = comment;
65     }
66 }
```

```

1 package com.runway.course.domain;
2
3 import com.runway.course.domain.enums.CourseReportReason;
4 import com.runway.course.domain.enums.CourseReportStatus;
5 import jakarta.persistence.*;
6 import lombok.AccessLevel;
7 import lombok.Builder;
8 import lombok.Getter;
9 import lombok.NoArgsConstructor;
10
11 import java.time.Instant;
12 import java.util.UUID;
13
14 @Entity
15 @Table(name = "course_reports")
16 @Getter
17 @NoArgsConstructor(access = AccessLevel.PROTECTED)
18 public class CourseReport {
19
20     @Id
21     @GeneratedValue(strategy = GenerationType.UUID)
22     @Column(columnDefinition = "uuid", updatable = false, nullable = false)
23     private UUID id;
24
25     @Column(name = "course_id", nullable = false, columnDefinition = "uuid")
26     private UUID courseId;
27
28     @Column(name = "reporter_id", nullable = false, columnDefinition = "uuid")
29     private UUID reporterId;
30
31     @Column(nullable = false, length = 30)
32     private CourseReportReason reason;
33
34     @Column(columnDefinition = "TEXT")
35     private String description;
36
37     @Column(nullable = false, length = 30)
38     private CourseReportStatus status;
39
40     @Column(name = "created_at", nullable = false, updatable = false)
41     private Instant createdAt;
42
43     @Builder
44     public CourseReport(UUID courseId, UUID reporterId, CourseReportReason reason, String description) {
45         this.courseId = courseId;
46         this.reporterId = reporterId;
47         this.reason = reason;
48         this.description = description;
49         this.status = CourseReportStatus.PENDING;
50         this.createdAt = Instant.now();
51     }
52 }

```

```

1 package com.runway.course.domain.enums;
2
3 import jakarta.persistence.AttributeConverter;
4 import jakarta.persistence.Converter;
5
6 public enum CourseDifficulty {
7     EASY("easy"), NORMAL("normal"), HARD("hard");
8
9     private final String dbValue;
10
11     CourseDifficulty(String dbValue) { this.dbValue = dbValue; }
12
13     public String getDbValue() { return dbValue; }
14
15     public static CourseDifficulty from(String value) {
16         for (CourseDifficulty v : values()) if (v.dbValue.equals(value)) return v;
17         throw new IllegalArgumentException("Unknown CourseDifficulty: " + value);
18     }
19
20     @Converter(autoApply = true)
21     public static class JpaConverter implements AttributeConverter<CourseDifficulty, String> {
22         @Override public String convertToDatabaseColumn(CourseDifficulty a) { return a == null ? null : a.dbValue; }
23         @Override public CourseDifficulty convertToEntityAttribute(String d) { return d == null ? null : from(d); }
24     }
25 }

```

backend/src/main/java/com/runway/course/domain/enums/CourseRecommendedTime.java

Java | 25줄

```
1 package com.runway.course.domain.enums;
2
3 import jakarta.persistence.AttributeConverter;
4 import jakarta.persistence.Converter;
5
6 public enum CourseRecommendedTime {
7     MORNING("morning"), DAY("day"), NIGHT("night"), ANY("any");
8
9     private final String dbValue;
10
11     CourseRecommendedTime(String dbValue) { this.dbValue = dbValue; }
12
13     public String getDbValue() { return dbValue; }
14
15     public static CourseRecommendedTime from(String value) {
16         for (CourseRecommendedTime v : values()) if (v.dbValue.equals(value)) return v;
17         throw new IllegalArgumentException("Unknown CourseRecommendedTime: " + value);
18     }
19
20     @Converter(autoApply = true)
21     public static class JpaConverter implements AttributeConverter<CourseRecommendedTime, String> {
22         @Override public String convertToDatabaseColumn(CourseRecommendedTime a) { return a == null ? null : a.dbValue; }
23         @Override public CourseRecommendedTime convertToEntityAttribute(String d) { return d == null ? null : from(d); }
24     }
25 }
```

backend/src/main/java/com/runway/course/domain/enums/CourseReportReason.java

Java | 32줄

```
1 package com.runway.course.domain.enums;
2
3 import jakarta.persistence.AttributeConverter;
4 import jakarta.persistence.Converter;
5
6 public enum CourseReportReason {
7     INAPPROPRIATE_CONTENT,
8     WRONG_LOCATION,
9     SPAM,
10    OTHER;
11
12    public String toDbValue() {
13        return name().toLowerCase();
14    }
15
16    public static CourseReportReason fromDbValue(String value) {
17        return valueOf(value.toUpperCase());
18    }
19
20    @Converter(autoApply = true)
21    public static class JpaConverter implements AttributeConverter<CourseReportReason, String> {
22        @Override
23        public String convertToDatabaseColumn(CourseReportReason reason) {
24            return reason == null ? null : reason.toDbValue();
25        }
26
27        @Override
28        public CourseReportReason convertToEntityAttribute(String dbData) {
29            return dbData == null ? null : CourseReportReason.fromDbValue(dbData);
30        }
31    }
32 }
```

backend/src/main/java/com/runway/course/domain/enums/CourseReportStatus.java

Java | 31줄

```
1 package com.runway.course.domain.enums;
2
3 import jakarta.persistence.AttributeConverter;
4 import jakarta.persistence.Converter;
5
6 public enum CourseReportStatus {
7     PENDING,
8     REVIEWED,
9     DISMISSED;
10
11     public String toDbValue() {
12         return name().toLowerCase();
13     }
14
15     public static CourseReportStatus fromDbValue(String value) {
16         return valueOf(value.toUpperCase());
17     }
18
19     @Converter(autoApply = true)
20     public static class JpaConverter implements AttributeConverter<CourseReportStatus, String> {
21         @Override
22         public String convertToDatabaseColumn(CourseReportStatus status) {
23             return status == null ? null : status.toDbValue();
24         }
25
26         @Override
27         public CourseReportStatus convertToEntityAttribute(String dbData) {
28             return dbData == null ? null : CourseReportStatus.fromDbValue(dbData);
29         }
30     }
31 }
```

backend/src/main/java/com/runway/course/domain/enums/CourseRiskLevel.java

Java | 25줄

```
1 package com.runway.course.domain.enums;
2
3 import jakarta.persistence.AttributeConverter;
4 import jakarta.persistence.Converter;
5
6 public enum CourseRiskLevel {
7     LOW("low"), MEDIUM("medium"), HIGH("high");
8
9     private final String dbValue;
10
11     CourseRiskLevel(String dbValue) { this.dbValue = dbValue; }
12
13     public String getDbValue() { return dbValue; }
14
15     public static CourseRiskLevel from(String value) {
16         for (CourseRiskLevel v : values()) if (v.dbValue.equals(value)) return v;
17         throw new IllegalArgumentException("Unknown CourseRiskLevel: " + value);
18     }
19
20     @Converter(autoApply = true)
21     public static class JpaConverter implements AttributeConverter<CourseRiskLevel, String> {
22         @Override public String convertToDatabaseColumn(CourseRiskLevel a) { return a == null ? null : a.dbValue; }
23         @Override public CourseRiskLevel convertToEntityAttribute(String d) { return d == null ? null : from(d); }
24     }
25 }
```

backend/src/main/java/com/runway/course/domain/enums/CourseSlopeLevel.java

Java | 25줄

```
1 package com.runway.course.domain.enums;
2
3 import jakarta.persistence.AttributeConverter;
4 import jakarta.persistence.Converter;
5
6 public enum CourseSlopeLevel {
7     FLAT("flat"), MODERATE("moderate"), STEEP("steep");
8
9     private final String dbValue;
10
11     CourseSlopeLevel(String dbValue) { this.dbValue = dbValue; }
12
13     public String getDbValue() { return dbValue; }
14
15     public static CourseSlopeLevel from(String value) {
16         for (CourseSlopeLevel v : values()) if (v.dbValue.equals(value)) return v;
17         throw new IllegalArgumentException("Unknown CourseSlopeLevel: " + value);
18     }
19
20     @Converter(autoApply = true)
21     public static class JpaConverter implements AttributeConverter<CourseSlopeLevel, String> {
22         @Override public String convertToDatabaseColumn(CourseSlopeLevel a) { return a == null ? null : a.dbValue; }
23         @Override public CourseSlopeLevel convertToEntityAttribute(String d) { return d == null ? null : from(d); }
24     }
25 }
```

backend/src/main/java/com/runway/course/domain/enums/CourseStatus.java

Java | 44줄

```
1 package com.runway.course.domain.enums;
2
3 import jakarta.persistence.AttributeConverter;
4 import jakarta.persistence.Converter;
5
6 public enum CourseStatus {
7
8     DRAFT("draft"),
9     PUBLISHED("published"),
10    ARCHIVED("archived");
11
12    private final String dbValue;
13
14    CourseStatus(String dbValue) {
15        this.dbValue = dbValue;
16    }
17
18    public String getDbValue() {
19        return dbValue;
20    }
21
22    public static CourseStatus from(String value) {
23        for (CourseStatus status : values()) {
24            if (status.dbValue.equals(value)) {
25                return status;
26            }
27        }
28        throw new IllegalArgumentException("Unknown CourseStatus: " + value);
29    }
30
31    @Converter(autoApply = true)
32    public static class JpaConverter implements AttributeConverter<CourseStatus, String> {
33
34        @Override
35        public String convertToDatabaseColumn(CourseStatus attribute) {
36            return attribute == null ? null : attribute.getDbValue();
37        }
38
39        @Override
40        public CourseStatus convertToEntityAttribute(String dbData) {
41            return dbData == null ? null : CourseStatus.from(dbData);
42        }
43    }
44 }
```

```
1 package com.runway.course.domain.enums;
2
3 import jakarta.persistence.AttributeConverter;
4 import jakarta.persistence.Converter;
5
6 public enum CourseSurfaceType {
7     ROAD("road"), PARK("park"), TRAIL("trail"), MIXED("mixed");
8
9     private final String dbValue;
10
11     CourseSurfaceType(String dbValue) { this.dbValue = dbValue; }
12
13     public String getDbValue() { return dbValue; }
14
15     public static CourseSurfaceType from(String value) {
16         for (CourseSurfaceType v : values()) if (v.dbValue.equals(value)) return v;
17         throw new IllegalArgumentException("Unknown CourseSurfaceType: " + value);
18     }
19
20     @Converter(autoApply = true)
21     public static class JpaConverter implements AttributeConverter<CourseSurfaceType, String> {
22         @Override public String convertToDatabaseColumn(CourseSurfaceType a) { return a == null ? null : a.dbValue; }
23         @Override public CourseSurfaceType convertToEntityAttribute(String d) { return d == null ? null : from(d); }
24     }
25 }
```

```

1  package com.runway.course.dto;
2
3  import com.runway.course.domain.Course;
4  import com.runway.user.domain.User;
5  import lombok.Builder;
6  import lombok.Getter;
7
8  import java.time.Instant;
9  import java.util.UUID;
10
11  @Getter
12  @Builder
13  public class CourseDetailResponse {
14
15      private UUID courseId;
16      private String name;
17      private String description;
18      private String status;
19      private Double distanceMeters;
20      private Boolean isLoop;
21      private Integer attemptCount;
22      private Integer completionCount;
23      private Double avgRating;
24      private Long ratingCount;
25      private CreatorDto creator;
26      private GeoPoint startPoint;
27      private GeoPoint endPoint;
28      private Boolean isFavorited;
29      private String difficulty;
30      private String slopeLevel;
31      private String riskLevel;
32      private String surfaceType;
33      private String recommendedTime;
34      private String warnings;
35      private Boolean isOwner;
36      private Instant createdAt;
37      private Instant updatedAt;
38
39      @Getter
40      @Builder
41      public static class CreatorDto {
42          private UUID userId;
43          private String nickname;
44          private String profileImageUrl;
45
46          public static CreatorDto from(User user) {
47              return CreatorDto.builder()
48                  .userId(user.getId())
49                  .nickname(user.getNickname())
50                  .profileImageUrl(user.getProfileImageUrl())
51                  .build();
52          }
53      }
54
55      public static CourseDetailResponse from(Course course, User creator, boolean isOwner,
56          Double avgRating, Long ratingCount, boolean isFavorited) {
57          GeoPoint startPoint = isOwner
58              ? new GeoPoint(course.getStartLocation().getY(), course.getStartLocation().getX())
59              : null;
60          GeoPoint endPoint = isOwner
61              ? new GeoPoint(course.getEndLocation().getY(), course.getEndLocation().getX())
62              : null;
63
64          return CourseDetailResponse.builder()
65              .courseId(course.getId())
66              .name(course.getName())
67              .description(course.getDescription())
68              .status(course.getStatus().getDbValue())
69              .distanceMeters(course.getDistanceMeters())
70              .isLoop(course.getIsLoop())
71              .attemptCount(course.getAttemptCount())
72              .completionCount(course.getCompletionCount())
73              .avgRating(avgRating)
74              .ratingCount(ratingCount)
75              .isFavorited(isFavorited)
76              .difficulty(course.getDifficulty() != null ? course.getDifficulty().getDbValue() : null)
77              .slopeLevel(course.getSlopeLevel() != null ? course.getSlopeLevel().getDbValue() : null)
78              .riskLevel(course.getRiskLevel() != null ? course.getRiskLevel().getDbValue() : null)
79              .surfaceType(course.getSurfaceType() != null ? course.getSurfaceType().getDbValue() : null)
80              .recommendedTime(course.getRecommendedTime() != null ? course.getRecommendedTime().getDbValue() : null)
81              .warnings(course.getWarnings())
82              .isOwner(isOwner)
83              .creator(CreatorDto.from(creator))

```

```

84         .startPoint(startPoint)
85         .endPoint(endPoint)
86         .createdAt(course.getCreatedAt())
87         .updatedAt(course.getUpdatedAt())
88         .build();
89     }
90 }

```

backend/src/main/java/com/runway/course/dto/CoursePointResponse.java

Java | 24줄

```

1  package com.runway.course.dto;
2
3  import com.runway.course.domain.CoursePoint;
4  import lombok.Builder;
5  import lombok.Getter;
6
7  @Getter
8  @Builder
9  public class CoursePointResponse {
10
11      private Integer sequence;
12      private Double latitude;
13      private Double longitude;
14      private Double altitudeMeters;
15
16      public static CoursePointResponse from(CoursePoint point) {
17          return CoursePointResponse.builder()
18              .sequence(point.getSequence())
19              .latitude(point.getLocation().getY()) // Y = latitude
20              .longitude(point.getLocation().getX()) // X = longitude
21              .altitudeMeters(point.getAltitudeMeters())
22              .build();
23      }
24 }

```

backend/src/main/java/com/runway/course/dto/CoursePointsResponse.java

Java | 15줄

```

1  package com.runway.course.dto;
2
3  import lombok.Builder;
4  import lombok.Getter;
5
6  import java.util.List;
7  import java.util.UUID;
8
9  @Getter
10 @Builder
11 public class CoursePointsResponse {
12
13     private UUID courseId;
14     private List<CoursePointResponse> points;
15 }

```

backend/src/main/java/com/runway/course/dto/CourseRatingRequest.java

Java | 17줄

```

1  package com.runway.course.dto;
2
3  import jakarta.validation.constraints.Max;
4  import jakarta.validation.constraints.Min;
5  import jakarta.validation.constraints.NotNull;
6  import lombok.Getter;
7
8  @Getter
9  public class CourseRatingRequest {
10
11     @NotNull(message = "평점은 필수입니다.")
12     @Min(value = 1, message = "평점은 1 이상이어야 합니다.")
13     @Max(value = 5, message = "평점은 5 이하여야 합니다.")
14     private Integer rating;
15
16     private String comment;
17 }

```


backend/src/main/java/com/runway/course/dto/CourseRatingResponse.java

Java | 31줄

```
1 package com.runway.course.dto;
2
3 import com.runway.course.domain.CourseRating;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.UUID;
9
10 @Getter
11 @Builder
12 public class CourseRatingResponse {
13
14     private UUID ratingId;
15     private UUID courseId;
16     private Integer rating;
17     private String comment;
18     private Instant createdAt;
19     private Instant updatedAt;
20
21     public static CourseRatingResponse from(CourseRating r) {
22         return CourseRatingResponse.builder()
23             .ratingId(r.getId())
24             .courseId(r.getCourseId())
25             .rating(r.getRating())
26             .comment(r.getComment())
27             .createdAt(r.getCreatedAt())
28             .updatedAt(r.getUpdatedAt())
29             .build();
30     }
31 }
```

backend/src/main/java/com/runway/course/dto/CourseReportRequest.java

Java | 15줄

```
1 package com.runway.course.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import lombok.Getter;
5 import lombok.NoArgsConstructor;
6
7 @Getter
8 @NoArgsConstructor
9 public class CourseReportRequest {
10
11     @NotBlank
12     private String reason;
13
14     private String description;
15 }
```

backend/src/main/java/com/runway/course/dto/CourseReportResponse.java

Java | 27줄

```
1 package com.runway.course.dto;
2
3 import com.runway.course.domain.CourseReport;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.UUID;
9
10 @Getter
11 @Builder
12 public class CourseReportResponse {
13
14     private UUID reportId;
15     private UUID courseId;
16     private String status;
17     private Instant createdAt;
18
19     public static CourseReportResponse from(CourseReport report) {
20         return CourseReportResponse.builder()
21             .reportId(report.getId())
22             .courseId(report.getCourseId())
23             .status(report.getStatus().toDbValue())
24             .createdAt(report.getCreatedAt())
25             .build();
26     }
27 }
```

backend/src/main/java/com/runway/course/dto/CourseResponse.java

Java | 47줄

```
1 package com.runway.course.dto;
2
3 import com.runway.course.domain.Course;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.UUID;
9
10 @Getter
11 @Builder
12 public class CourseResponse {
13
14     private UUID courseId;
15     private String name;
16     private String description;
17     private String status;
18     private Double distanceMeters;
19     private Boolean isLoop;
20     private GeoPoint startPoint;
21     private GeoPoint endPoint;
22     private Integer attemptCount;
23     private Integer completionCount;
24     private Instant createdAt;
25     private Instant updatedAt;
26
27     public static CourseResponse from(Course course) {
28         return CourseResponse.builder()
29             .courseId(course.getId())
30             .name(course.getName())
31             .description(course.getDescription())
32             .status(course.getStatus().getDbValue())
33             .distanceMeters(course.getDistanceMeters())
34             .isLoop(course.getIsLoop())
35             .startPoint(new GeoPoint(
36                 course.getStartLocation().getY(), // Y = latitude
37                 course.getStartLocation().getX()) // X = longitude
38             .endPoint(new GeoPoint(
39                 course.getEndLocation().getY(),
40                 course.getEndLocation().getX())
41             .attemptCount(course.getAttemptCount())
42             .completionCount(course.getCompletionCount())
43             .createdAt(course.getCreatedAt())
44             .updatedAt(course.getUpdatedAt())
45             .build();
46     }
47 }
```

backend/src/main/java/com/runway/course/dto/CourseStatusResponse.java

Java | 22줄

```
1 package com.runway.course.dto;
2
3 import com.runway.course.domain.Course;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.util.UUID;
8
9 @Getter
10 @Builder
11 public class CourseStatusResponse {
12
13     private UUID courseId;
14     private String status;
15
16     public static CourseStatusResponse from(Course course) {
17         return CourseStatusResponse.builder()
18             .courseId(course.getId())
19             .status(course.getStatus().getDbValue())
20             .build();
21     }
22 }
```

backend/src/main/java/com/runway/course/dto/CreateCourseFromRunRequest.java

Java | 20줄

```
1 package com.runway.course.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.Size;
5 import lombok.Getter;
6
7 @Getter
8 public class CreateCourseFromRunRequest {
9
10     @NotBlank(message = "코스 이름은 필수입니다.")
11     @Size(min = 2, max = 100, message = "코스 이름은 2~100자 사이여야 합니다.")
12     private String name;
13
14     @Size(max = 1000, message = "설명은 최대 1000자까지 입력할 수 있습니다.")
15     private String description;
16
17     private Boolean isLoop;
18
19     private Boolean publish;
20 }
```

backend/src/main/java/com/runway/course/dto/GeoPoint.java

Java | 11줄

```
1 package com.runway.course.dto;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Getter;
5
6 @Getter
7 @AllArgsConstructor
8 public class GeoPoint {
9     private double latitude;
10    private double longitude;
11 }
```

backend/src/main/java/com/runway/course/dto/NearbyCourseItem.java

Java | 25줄

```
1 package com.runway.course.dto;
2
3 import lombok.Builder;
4 import lombok.Getter;
5
6 import java.util.List;
7 import java.util.UUID;
8
9 @Getter
10 @Builder(toBuilder = true)
11 public class NearbyCourseItem {
12
13     private UUID courseId;
14     private String name;
15     private String description;
16     private Double distanceMeters;
17     private Double distanceFromMeMeters;
18     private Boolean isLoop;
19     private Integer attemptCount;
20     private Integer completionCount;
21     private GeoPoint startPoint;
22     private Double avgRating;
23     private Long ratingCount;
24     private List<GeoPoint> routePoints;
25 }
```

backend/src/main/java/com/runway/course/dto/ParticipatedCourseItem.java

Java | 23줄

```
1 package com.runway.course.dto;
2
3 import lombok.Builder;
4 import lombok.Getter;
5
6 import java.time.Instant;
7 import java.util.UUID;
8
9 @Getter
10 @Builder
11 public class ParticipatedCourseItem {
12
13     private UUID courseId;
14     private String name;
15     private String description;
16     private Double distanceMeters;
17     private Boolean isLoop;
18     private String status;
19     private Integer attemptCountByMe;
20     private Integer completionCountByMe;
21     private Integer bestTimeSecondsByMe;
22     private Instant lastAttemptAt;
23 }
```

backend/src/main/java/com/runway/course/dto/PublishCourseRequest.java

Java | 30줄

```
1 package com.runway.course.dto;
2
3 import jakarta.validation.constraints.NotNull;
4 import jakarta.validation.constraints.Size;
5 import lombok.Getter;
6
7 @Getter
8 public class PublishCourseRequest {
9
10     @NotNull(message = "난이도는 필수입니다.")
11     private String difficulty;
12
13     @NotNull(message = "경사도는 필수입니다.")
14     private String slopeLevel;
15
16     @NotNull(message = "위험도는 필수입니다.")
17     private String riskLevel;
18
19     @NotNull(message = "노면 유형은 필수입니다.")
20     private String surfaceType;
21
22     @NotNull(message = "추천 시간대는 필수입니다.")
23     private String recommendedTime;
24
25     @Size(max = 1000, message = "주의사항은 최대 1000자까지 입력할 수 있습니다.")
26     private String warnings;
27
28     @Size(max = 1000, message = "설명은 최대 1000자까지 입력할 수 있습니다.")
29     private String description;
30 }
```

backend/src/main/java/com/runway/course/dto/UpdateCourseRequest.java

Java | 18줄

```
1 package com.runway.course.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.Size;
5 import lombok.Getter;
6
7 @Getter
8 public class UpdateCourseRequest {
9
10     @NotBlank(message = "코스 이름은 필수입니다.")
11     @Size(min = 2, max = 100, message = "코스 이름은 2~100자 사이여야 합니다.")
12     private String name;
13
14     @Size(max = 1000, message = "설명은 최대 1000자까지 입력할 수 있습니다.")
15     private String description;
16
17     private Boolean isLoop;
18 }
```

backend/src/main/java/com/runway/course/repository/CourseFavoriteRepository.java

Java | 14줄

```
1 package com.runway.course.repository;
2
3 import com.runway.course.domain.CourseFavorite;
4 import com.runway.course.domain.CourseFavoriteId;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import java.util.UUID;
8
9 public interface CourseFavoriteRepository extends JpaRepository<CourseFavorite, CourseFavoriteId> {
10
11     boolean existsByUserIdAndCourseId(UUID userId, UUID courseId);
12
13     void deleteByUserIdAndCourseId(UUID userId, UUID courseId);
14 }
```

backend/src/main/java/com/runway/course/repository/CoursePointRepository.java

Java | 14줄

```
1 package com.runway.course.repository;
2
3 import com.runway.course.domain.CoursePoint;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 import java.util.List;
7 import java.util.UUID;
8
9 public interface CoursePointRepository extends JpaRepository<CoursePoint, UUID> {
10
11     List<CoursePoint> findByCourseIdOrderBySequenceAsc(UUID courseId);
12
13     void deleteByCourseId(UUID courseId);
14 }
```

backend/src/main/java/com/runway/course/repository/CourseRatingRepository.java

Java | 19줄

```
1 package com.runway.course.repository;
2
3 import com.runway.course.domain.CourseRating;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.Query;
6 import org.springframework.data.repository.query.Param;
7
8 import java.util.Optional;
9 import java.util.UUID;
10
11 public interface CourseRatingRepository extends JpaRepository<CourseRating, UUID> {
12
13     Optional<CourseRating> findByCourseIdAndUserId(UUID courseId, UUID userId);
14
15     long countByCourseId(UUID courseId);
16
17     @Query("SELECT AVG(r.rating) FROM CourseRating r WHERE r.courseId = :courseId")
18     Double findAvgRatingByCourseId(@Param("courseId") UUID courseId);
19 }
```

backend/src/main/java/com/runway/course/repository/CourseReportRepository.java

Java | 11줄

```
1 package com.runway.course.repository;
2
3 import com.runway.course.domain.CourseReport;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 import java.util.UUID;
7
8 public interface CourseReportRepository extends JpaRepository<CourseReport, UUID> {
9
10     boolean existsByCourseIdAndReporterId(UUID courseId, UUID reporterId);
11 }
```

backend/src/main/java/com/runway/course/repository/CourseRepository.java

Java | 24줄

```
1 package com.runway.course.repository;
2
3 import com.runway.course.domain.Course;
4 import com.runway.course.domain.enums.CourseStatus;
5 import org.springframework.data.domain.Page;
6 import org.springframework.data.domain.Pageable;
7 import org.springframework.data.jpa.repository.JpaRepository;
8
9 import java.util.Optional;
10 import java.util.UUID;
11
12 public interface CourseRepository extends JpaRepository<Course, UUID> {
13
14     Optional<Course> findByIdAndDeletedAtIsNull(UUID id);
15
16     Page<Course> findByCreatorIdAndDeletedAtIsNullOrderByCreatedAtDesc(UUID creatorId, Pageable pageable);
17
18     Page<Course> findByCreatorIdAndStatusAndDeletedAtIsNullOrderByCreatedAtDesc(
19         UUID creatorId, CourseStatus status, Pageable pageable);
20
21     long countByCreatorIdAndDeletedAtIsNull(UUID creatorId);
22
23     Optional<Course> findFirstByCreatorIdAndDeletedAtIsNullOrderByCreatedAtAsc(UUID creatorId);
24 }
```

backend/src/main/java/com/runway/course/service/CourseRatingService.java

Java | 50줄

```
1 package com.runway.course.service;
2
3 import com.runway.common.exception.ErrorCode;
4 import com.runway.common.exception.RunwayException;
5 import com.runway.course.domain.CourseRating;
6 import com.runway.course.dto.CourseRatingRequest;
7 import com.runway.course.dto.CourseRatingResponse;
8 import com.runway.course.repository.CourseRatingRepository;
9 import com.runway.course.repository.CourseRepository;
10 import lombok.RequiredArgsConstructor;
11 import lombok.extern.slf4j.Slf4j;
12 import org.springframework.stereotype.Service;
13 import org.springframework.transaction.annotation.Transactional;
14
15 import java.util.Optional;
16 import java.util.UUID;
17
18 @Slf4j
19 @Service
20 @RequiredArgsConstructor
21 public class CourseRatingService {
22
23     private final CourseRepository courseRepository;
24     private final CourseRatingRepository courseRatingRepository;
25
26     @Transactional
27     public CourseRatingResponse rateCourse(UUID userId, UUID courseId, CourseRatingRequest request) {
28         courseRepository.findByIdAndDeletedAtIsNull(courseId)
29             .orElseThrow(() -> new RunwayException(ErrorCode.COURSE_NOT_FOUND));
30
31         Optional<CourseRating> existing = courseRatingRepository.findByCourseIdAndUserId(courseId, userId);
32         CourseRating rating;
33         if (existing.isPresent()) {
34             rating = existing.get();
35             rating.update(request.getRating(), request.getComment());
36             log.info("Course rating updated: courseId={} userId={} rating={}", courseId, userId, request.getRating());
37         } else {
38             rating = CourseRating.builder()
39                 .courseId(courseId)
40                 .userId(userId)
41                 .rating(request.getRating())
42                 .comment(request.getComment())
43                 .build();
44             courseRatingRepository.save(rating);
45             log.info("Course rating created: courseId={} userId={} rating={}", courseId, userId, request.getRating());
46         }
47
48         return CourseRatingResponse.from(rating);
49     }
50 }
```

```
1 package com.runway.course.service;
2
3 import com.runway.common.exception.ErrorCode;
4 import com.runway.common.exception.RunwayException;
5 import com.runway.course.domain.CourseReport;
6 import com.runway.course.domain.enums.CourseReportReason;
7 import com.runway.course.dto.CourseReportRequest;
8 import com.runway.course.dto.CourseReportResponse;
9 import com.runway.course.repository.CourseReportRepository;
10 import com.runway.course.repository.CourseRepository;
11 import lombok.RequiredArgsConstructor;
12 import org.springframework.stereotype.Service;
13 import org.springframework.transaction.annotation.Transactional;
14
15 import java.util.UUID;
16
17 @Service
18 @RequiredArgsConstructor
19 public class CourseReportService {
20
21     private final CourseRepository courseRepository;
22     private final CourseReportRepository courseReportRepository;
23
24     @Transactional
25     public CourseReportResponse reportCourse(UUID reporterId, UUID courseId, CourseReportRequest request) {
26         courseRepository.findByIdAndDeletedAtIsNull(courseId)
27             .orElseThrow(() -> new RunwayException(ErrorCode.COURSE_NOT_FOUND));
28
29         if (courseReportRepository.existsByCourseIdAndReporterId(courseId, reporterId)) {
30             throw new RunwayException(ErrorCode.ALREADY_REPORTED);
31         }
32
33         CourseReportReason reason;
34         try {
35             reason = CourseReportReason.fromDbValue(request.getReason());
36         } catch (IllegalArgumentException e) {
37             throw new RunwayException(ErrorCode.INVALID_REPORT_REASON);
38         }
39
40         CourseReport report = CourseReport.builder()
41             .courseId(courseId)
42             .reporterId(reporterId)
43             .reason(reason)
44             .description(request.getDescription())
45             .build();
46
47         courseReportRepository.save(report);
48         return CourseReportResponse.from(report);
49     }
50 }
```



```

1  package com.runway.course.service;
2
3  import com.runway.common.exception.ErrorCode;
4  import com.runway.common.exception.RunwayException;
5  import com.runway.common.response.PageResponse;
6  import com.runway.course.domain.Course;
7  import com.runway.attempt.domain.enums.AttemptVerificationStatus;
8  import com.runway.attempt.domain.enums.CourseAttemptStatus;
9  import com.runway.attempt.repository.CourseAttemptRepository;
10 import com.runway.course.domain.CourseFavorite;
11 import com.runway.course.domain.CourseFavoriteId;
12 import com.runway.course.domain.CoursePoint;
13 import com.runway.course.domain.enums.CourseDifficulty;
14 import com.runway.course.domain.enums.CourseRecommendedTime;
15 import com.runway.course.domain.enums.CourseRiskLevel;
16 import com.runway.course.domain.enums.CourseSlopeLevel;
17 import com.runway.course.domain.enums.CourseSurfaceType;
18 import com.runway.course.domain.enums.CourseStatus;
19 import com.runway.course.dto.*;
20 import com.runway.course.repository.CourseFavoriteRepository;
21 import com.runway.course.repository.CoursePointRepository;
22 import com.runway.course.repository.CourseRatingRepository;
23 import com.runway.course.repository.CourseRepository;
24 import com.runway.run.domain.RunningPoint;
25 import com.runway.run.domain.enums.RunningRecordStatus;
26 import com.runway.run.repository.RunningPointRepository;
27 import com.runway.run.repository.RunningRecordRepository;
28 import com.runway.user.domain.User;
29 import com.runway.user.repository.UserRepository;
30 import jakarta.persistence.EntityManager;
31 import lombok.RequiredArgsConstructor;
32 import lombok.extern.slf4j.Slf4j;
33 import org.locationtech.jts.geom.Coordinate;
34 import org.locationtech.jts.geom.GeometryFactory;
35 import org.locationtech.jts.geom.LineString;
36 import org.locationtech.jts.geom.PrecisionModel;
37 import org.springframework.data.domain.PageImpl;
38 import org.springframework.data.domain.PageRequest;
39 import org.springframework.stereotype.Service;
40 import org.springframework.transaction.annotation.Transactional;
41
42 import com.runway.course.util.CoursePrivacyUtils;
43
44 import java.time.Instant;
45 import java.util.*;
46
47 @Slf4j
48 @Service
49 @RequiredArgsConstructor
50 public class CourseService {
51
52     private static final GeometryFactory GEOMETRY_FACTORY = new GeometryFactory(new PrecisionModel(), 4326);
53     private static final int MAX_COURSE_POINTS = 200;
54     private static final int OWNER_COMPLETIONS_REQUIRED_FOR_PUBLIC = 10;
55
56     private final CourseRepository courseRepository;
57     private final CourseAttemptRepository courseAttemptRepository;
58     private final CourseFavoriteRepository courseFavoriteRepository;
59     private final CoursePointRepository coursePointRepository;
60     private final CourseRatingRepository courseRatingRepository;
61     private final RunningRecordRepository runningRecordRepository;
62     private final RunningPointRepository runningPointRepository;
63     private final UserRepository userRepository;
64     private final EntityManager entityManager;
65
66     @Transactional
67     public CourseResponse createCourseFromRun(UUID userId, UUID runId,
68                                             CreateCourseFromRunRequest request) {
69         var record = runningRecordRepository.findByIdAndUserId(runId, userId)
70             .orElseThrow(() -> new RunwayException(ErrorCode.RUN_NOT_FOUND));
71
72         if (record.getStatus() != RunningRecordStatus.COMPLETED) {
73             throw new RunwayException(ErrorCode.NOT_COMPLETED_RUN);
74         }
75
76         List<RunningPoint> runningPoints =
77             runningPointRepository.findByRunningRecordIdOrderBySequenceAsc(runId);
78
79         if (runningPoints.size() < 2) {
80             throw new RunwayException(ErrorCode.INVALID_REQUEST,
81                                     "코스 생성을 위해 최소 2개 이상의 GPS 포인트가 필요합니다.");
82         }
83

```

```

84 // 다운샘플링: MAX_COURSE_POINTS 이하로 샘플링
85 List<RunningPoint> sampled = downsample(runningPoints);
86
87 // 코스 경로 LineString 생성 (다운샘플링된 포인트 기준)
88 Coordinate[] coords = sampled.stream()
89     .map(p -> p.getLocation().getCoordinate())
90     .toArray(Coordinate[]::new);
91 LineString path = GEOMETRY_FACTORY.createLineString(coords);
92
93 CourseStatus status = Boolean.TRUE.equals(request.getPublish())
94     ? CourseStatus.PUBLISHED : CourseStatus.DRAFT;
95
96 Course course = Course.builder()
97     .creatorId(userId)
98     .sourceRecordId(runId)
99     .name(request.getName())
100     .description(request.getDescription())
101     .status(status)
102     .distanceMeters(record.getDistanceMeters())
103     .isLoop(request.getIsLoop())
104     .startLocation(runningPoints.get(0).getLocation())
105     .endLocation(runningPoints.get(runningPoints.size() - 1).getLocation())
106     .build();
107
108 courseRepository.save(course);
109 course.updatePath(path);
110
111 // course_points 저장 (courseId 확정 후)
112 List<CoursePoint> coursePoints = buildCoursePoints(course.getId(), sampled);
113 coursePointRepository.saveAll(coursePoints);
114
115 log.info("Course created: courseId={} runId={} points={} status={}",
116     course.getId(), runId, coursePoints.size(), status.getDbValue());
117 return CourseResponse.from(course);
118 }
119
120 @Transactional(readOnly = true)
121 public PageResponse<NearbyCourseItem> getNearby(
122     Double latitude, Double longitude, Double radiusMeters,
123     Double minDistanceMeters, Double maxDistanceMeters, Boolean isLoop,
124     String keyword,
125     boolean includeRoutePoints, boolean includeExactStartPoint,
126     int page, int size) {
127
128     // CTE로 현재 위치 포인트를 한 번 계산하고 재사용
129     String refPoint = "ST_SetSRID(ST_MakePoint(?, ?), 4326)::geography";
130
131     List<Object> baseParams = new ArrayList<>();
132     baseParams.add(longitude); // ?1 - ST_MakePoint(lon, lat)
133     baseParams.add(latitude); // ?2
134     baseParams.add(radiusMeters); // ?3 - ST_DWithin 반경
135
136     StringBuilder optFilter = new StringBuilder();
137     List<Object> filterParams = new ArrayList<>();
138
139     if (minDistanceMeters != null) {
140         optFilter.append(" AND c.distance_meters >= ?");
141         filterParams.add(minDistanceMeters);
142     }
143
144     if (maxDistanceMeters != null) {
145         optFilter.append(" AND c.distance_meters <= ?");
146         filterParams.add(maxDistanceMeters);
147     }
148
149     if (isLoop != null) {
150         optFilter.append(" AND c.is_loop = ?");
151         filterParams.add(isLoop);
152     }
153
154     if (keyword != null && !keyword.isBlank()) {
155         optFilter.append(" AND LOWER(c.name) LIKE ?");
156         filterParams.add("%" + keyword.toLowerCase() + "%");
157     }
158
159     String baseSql =
160         "WITH ref AS (SELECT " + refPoint + " AS pt) " +
161         "FROM courses c, ref " +
162         "WHERE ST_DWithin(c.start_location, ref.pt, ?) " +
163         " AND c.status = 'published' AND c.deleted_at IS NULL" +
164         optFilter;
165
166     String dataSql =
167         "WITH ref AS (SELECT " + refPoint + " AS pt) " +
168         "SELECT c.id, c.name, c.description, c.distance_meters, " +
169         "    ST_Distance(c.start_location, ref.pt) AS dist_from_me, " +
170         "    c.is_loop, c.attempt_count, c.completion_count, " +
171         "    ST_Y(c.start_location::geometry) AS start_lat, " +
172         "    ST_X(c.start_location::geometry) AS start_lon, " +

```

```

170         "        (SELECT ROUND(AVG(rating)::numeric, 1) FROM course_ratings WHERE course_id = c.id) AS avg_rating, " +
171         "        (SELECT COUNT(*) FROM course_ratings WHERE course_id = c.id) AS rating_count " +
172         "FROM courses c, ref " +
173         "WHERE ST_DWithin(c.start_location, ref.pt, ?) " +
174         " AND c.status = 'published' AND c.deleted_at IS NULL" +
175         optFilter +
176         " ORDER BY dist_from_me ASC" +
177         " LIMIT ? OFFSET ?";
178
179     String countSql =
180         "WITH ref AS (SELECT " + refPoint + " AS pt) " +
181         "SELECT COUNT(*) " +
182         "FROM courses c, ref " +
183         "WHERE ST_DWithin(c.start_location, ref.pt, ?) " +
184         " AND c.status = 'published' AND c.deleted_at IS NULL" +
185         optFilter;
186
187     // 데이터 쿼리 파라미터 구성
188     List<Object> dataParams = new ArrayList<>(baseParams);
189     dataParams.addAll(filterParams);
190     dataParams.add(size);
191     dataParams.add((long) page * size);
192
193     @SuppressWarnings("unchecked")
194     List<Object[]> rows = buildQuery(dataSql, dataParams).getResultList();
195     List<NearbyCourseItem> items = rows.stream()
196         .map(row -> toNearbyCourseItem(row, includeExactStartPoint))
197         .toList();
198
199     // 결과 코스들의 경로 포인트를 단일 쿼리로 일괄 조회
200     if (includeRoutePoints && !items.isEmpty()) {
201         List<String> courseIdStrs = items.stream()
202             .map(it -> it.getCourseId().toString())
203             .toList();
204         String inClause = courseIdStrs.stream()
205             .map(id -> "'" + id + "'")
206             .reduce((a, b) -> a + ", " + b).orElse("");
207         String pointsSql =
208             "SELECT course_id::text, ST_Y(location::geometry), ST_X(location::geometry) " +
209             "FROM course_points " +
210             "WHERE course_id IN (" + inClause + ") " +
211             "ORDER BY course_id, sequence";
212         @SuppressWarnings("unchecked")
213         List<Object[]> pointRows = entityManager.createNativeQuery(pointsSql).getResultList();
214
215         // courseId별로 포인트 그룹화
216         Map<String, List<GeoPoint>> pointsMap = new java.util.LinkedHashMap<>();
217         for (Object[] pr : pointRows) {
218             String cid = (String) pr[0];
219             double lat = ((Number) pr[1]).doubleValue();
220             double lon = ((Number) pr[2]).doubleValue();
221             pointsMap.computeIfAbsent(cid, k -> new java.util.ArrayList<>()).add(new GeoPoint(lat, lon));
222         }
223
224         // thin하여 최대 40포인트, NearbyCourseItem에 주입
225         items = items.stream().map(item -> {
226             List<GeoPoint> pts = pointsMap.getOrDefault(item.getCourseId().toString(), List.of());
227             List<GeoPoint> thinned = thinPoints(pts, 40);
228             return item.toBuilder().routePoints(thinned).build();
229         }).toList();
230     }
231
232     // 카운트 쿼리 파라미터 구성 (LIMIT/OFFSET 없음)
233     List<Object> countParams = new ArrayList<>(baseParams);
234     countParams.addAll(filterParams);
235     long total = ((Number) buildQuery(countSql, countParams).getSingleResult()).longValue();
236
237     return PageResponse.from(new PageImpl<>(items, PageRequest.of(page, size), total));
238 }
239
240 @Transactional(readOnly = true)
241 public PageResponse<CourseResponse> getMyCourses(UUID userId, String statusParam,
242     int page, int size) {
243     PageRequest pageable = PageRequest.of(page, size);
244
245     if (statusParam != null) {
246         CourseStatus status;
247         try {
248             status = CourseStatus.from(statusParam);
249         } catch (IllegalArgumentException e) {
250             throw new RunwayException(ErrorCode.INVALID_REQUEST,
251                 "유효하지 않은 코스 상태입니다: " + statusParam);
252         }
253         return PageResponse.from(
254             courseRepository
255                 .findByCreatorIdAndStatusAndDeletedAtIsNullOrderByCreatedAtDesc(

```

```

256         userId, status, pageable)
257         .map(CourseResponse::from));
258     }
259
260     return PageResponse.from(
261         courseRepository
262             .findByCreatorIdAndDeletedAtIsNullOrderByCreatedAtDesc(userId, pageable)
263             .map(CourseResponse::from));
264     }
265
266     @Transactional(readOnly = true)
267     public CourseDetailResponse getCourseDetail(UUID userId, UUID courseId) {
268         Course course = findVisibleCourse(courseId, userId);
269         User creator = userRepository.findById(course.getCreatorId())
270             .orElseThrow(() -> new RunwayException(ErrorCode.USER_NOT_FOUND));
271         Double avgRating = courseRatingRepository.findAvgRatingByCourseId(courseId);
272         long ratingCount = courseRatingRepository.countByCourseId(courseId);
273         boolean isFavorited = courseFavoriteRepository.existsByUserIdAndCourseId(userId, courseId);
274         return CourseDetailResponse.from(course, creator, course.isOwnedBy(userId), avgRating, ratingCount, isFavorited);
275     }
276
277     @Transactional
278     public void addFavorite(UUID userId, UUID courseId) {
279         Course course = courseRepository.findByIdAndDeletedAtIsNull(courseId)
280             .orElseThrow(() -> new RunwayException(ErrorCode.COURSE_NOT_FOUND));
281         if (!course.isVisibleTo(userId)) {
282             throw new RunwayException(ErrorCode.FORBIDDEN);
283         }
284         if (!courseFavoriteRepository.existsByUserIdAndCourseId(userId, courseId)) {
285             courseFavoriteRepository.save(CourseFavorite.builder()
286                 .userId(userId)
287                 .courseId(courseId)
288                 .build());
289         }
290     }
291
292     @Transactional
293     public void removeFavorite(UUID userId, UUID courseId) {
294         courseFavoriteRepository.deleteByUserIdAndCourseId(userId, courseId);
295     }
296
297     @Transactional(readOnly = true)
298     public PageResponse<CourseResponse> getFavoriteCourses(UUID userId, int page, int size) {
299         String dataSql =
300             "SELECT c.id, c.name, c.description, c.status, c.distance_meters, c.is_loop, " +
301             "    ST_Y(c.start_location::geometry) AS start_lat, ST_X(c.start_location::geometry) AS start_lon, " +
302             "    ST_Y(c.end_location::geometry) AS end_lat, ST_X(c.end_location::geometry) AS end_lon, " +
303             "    c.attempt_count, c.completion_count, c.created_at, c.updated_at " +
304             "FROM courses c " +
305             "JOIN course_favorites cf ON c.id = cf.course_id " +
306             "WHERE cf.user_id = ? AND c.deleted_at IS NULL " +
307             "ORDER BY cf.created_at DESC " +
308             "LIMIT ? OFFSET ?";
309
310         String countSql =
311             "SELECT COUNT(*) FROM courses c " +
312             "JOIN course_favorites cf ON c.id = cf.course_id " +
313             "WHERE cf.user_id = ? AND c.deleted_at IS NULL";
314
315         @SuppressWarnings("unchecked")
316         List<Object[]> rows = entityManager.createNativeQuery(dataSql)
317             .setParameter(1, userId)
318             .setParameter(2, size)
319             .setParameter(3, (long) page * size)
320             .getResultList();
321
322         List<CourseResponse> items = rows.stream().map(this::toCourseResponse).toList();
323
324         long total = ((Number) entityManager.createNativeQuery(countSql)
325             .setParameter(1, userId)
326             .getSingleResult()).longValue();
327
328         return PageResponse.from(new PageImpl<>(items, PageRequest.of(page, size), total));
329     }
330
331     @Transactional(readOnly = true)
332     public PageResponse<ParticipatedCourseItem> getParticipatedCourses(UUID userId, int page, int size) {
333         String dataSql =
334             "SELECT c.id, c.name, c.description, c.distance_meters, c.is_loop, c.status, " +
335             "    COUNT(ca.id) AS attempt_count_by_me, " +
336             "    COUNT(CASE WHEN ca.status = 'completed' AND ca.verification_status = 'verified' THEN 1 END) AS completion_count_by_me, " +
337             "    MIN(CASE WHEN ca.status = 'completed' AND ca.verification_status = 'verified' THEN ca.duration_seconds END) AS best_time_by_me, " +
338             "    MAX(ca.started_at) AS last_attempt_at " +
339             "FROM courses c " +

```

```

340         "JOIN course_attempts ca ON c.id = ca.course_id " +
341         "WHERE ca.user_id = ? AND c.deleted_at IS NULL " +
342         "GROUP BY c.id, c.name, c.description, c.distance_meters, c.is_loop, c.status " +
343         "ORDER BY last_attempt_at DESC " +
344         "LIMIT ? OFFSET ?";
345
346     String countSql =
347         "SELECT COUNT(DISTINCT c.id) FROM courses c " +
348         "JOIN course_attempts ca ON c.id = ca.course_id " +
349         "WHERE ca.user_id = ? AND c.deleted_at IS NULL";
350
351     @SuppressWarnings("unchecked")
352     List<Object[]> rows = entityManager.createNativeQuery(dataSql)
353         .setParameter(1, userId)
354         .setParameter(2, size)
355         .setParameter(3, (long) page * size)
356         .getResultList();
357
358     List<ParticipatedCourseItem> items = rows.stream().map(this::toParticipatedCourseItem).toList();
359
360     long total = (Number) entityManager.createNativeQuery(countSql)
361         .setParameter(1, userId)
362         .getSingleResult().longValue();
363
364     return PageResponse.from(new PageImpl<>(items, PageRequest.of(page, size), total));
365 }
366
367 @Transactional(readOnly = true)
368 public CoursePointsResponse getCoursePoints(UUID userId, UUID courseId) {
369     Course course = findVisibleCourse(courseId, userId);
370     List<CoursePointResponse> rawPoints = coursePointRepository
371         .findByCourseIdOrderBySequenceAsc(course.getId())
372         .stream()
373         .map(CoursePointResponse::from)
374         .toList();
375     List<CoursePointResponse> points = course.isOwnedBy(userId)
376         ? rawPoints
377         : CoursePrivacyUtils.maskPrivacyZone(rawPoints);
378     return CoursePointsResponse.builder()
379         .courseId(course.getId())
380         .points(points)
381         .build();
382 }
383
384 @Transactional
385 public CourseResponse updateCourse(UUID userId, UUID courseId, UpdateCourseRequest request) {
386     Course course = findOwnedCourse(courseId, userId);
387     course.updateInfo(request.getName(), request.getDescription(), request.getIsLoop());
388     log.info("Course updated: courseId={}", courseId);
389     return CourseResponse.from(course);
390 }
391
392 @Transactional
393 public CourseStatusResponse publishCourse(UUID userId, UUID courseId, PublishCourseRequest request) {
394     Course course = findOwnedCourse(courseId, userId);
395     if (course.getStatus() == CourseStatus.PUBLISHED) {
396         throw new RunwayException(ErrorCode.INVALID_COURSE_STATUS);
397     }
398
399     // 메타데이터 파싱 및 저장
400     CourseDifficulty difficulty = parseEnum(request.getDifficulty(), CourseDifficulty::from, "difficulty");
401     CourseSlopeLevel slopeLevel = parseEnum(request.getSlopeLevel(), CourseSlopeLevel::from, "slopeLevel");
402     CourseRiskLevel riskLevel = parseEnum(request.getRiskLevel(), CourseRiskLevel::from, "riskLevel");
403     CourseSurfaceType surfaceType = parseEnum(request.getSurfaceType(), CourseSurfaceType::from, "surfaceType");
404     CourseRecommendedTime recommendedTime = parseEnum(request.getRecommendedTime(), CourseRecommendedTime::from, "recommendedTime");
405
406     course.updateMetadata(difficulty, slopeLevel, riskLevel, surfaceType, recommendedTime,
407         request.getWarnings(), request.getDescription());
408
409     // 필수 메타데이터 검증
410     if (!course.hasRequiredPublishMetadata()) {
411         throw new RunwayException(ErrorCode.COURSE_PUBLISH_METADATA_REQUIRED);
412     }
413
414     // 소유자 완주 횟수 검증
415     long ownerCompletions = courseAttemptRepository
416         .countByCourseIdAndUserIdAndStatusAndVerificationStatus(
417             courseId, userId,
418             CourseAttemptStatus.COMPLETED,
419             AttemptVerificationStatus.VERIFIED);
420     if (ownerCompletions < OWNER_COMPLETIONS_REQUIRED_FOR_PUBLIC) {
421         throw new RunwayException(ErrorCode.COURSE_PUBLISH_NOT_ENOUGH_COMPLETIONS);
422     }
423
424     course.publish();

```

```

425     log.info("Course published: courseId={}, ownerCompletions={}", courseId, ownerCompletions);
426     return CourseStatusResponse.from(course);
427 }
428
429 private <T> T parseEnum(String value, java.util.function.Function<String, T> parser, String fieldName) {
430     if (value == null) return null;
431     try {
432         return parser.apply(value.toLowerCase());
433     } catch (IllegalArgumentException e) {
434         throw new RunwayException(Errorcode.INVALID_REQUEST, "유효하지 않은 " + fieldName + " 값입니다: " + value);
435     }
436 }
437
438 @Transactional
439 public CourseStatusResponse archiveCourse(UUID userId, UUID courseId) {
440     Course course = findOwnedCourse(courseId, userId);
441     if (course.getStatus() == CourseStatus.ARCHIVED) {
442         throw new RunwayException(Errorcode.INVALID_COURSE_STATUS);
443     }
444     course.archive();
445     log.info("Course archived: courseId={}", courseId);
446     return CourseStatusResponse.from(course);
447 }
448
449 @Transactional
450 public void deleteDraftCourse(UUID userId, UUID courseId) {
451     Course course = findOwnedCourse(courseId, userId);
452     if (course.getStatus() != CourseStatus.DRAFT) {
453         throw new RunwayException(Errorcode.INVALID_COURSE_STATUS,
454             "공개된 코스는 삭제할 수 없습니다.");
455     }
456     coursePointRepository.deleteByCourseId(courseId);
457     course.softDelete();
458     log.info("Draft course deleted: courseId={}", courseId);
459 }
460
461 // --- private helpers ---
462
463 private CourseResponse toCourseResponse(Object[] row) {
464     return CourseResponse.builder()
465         .courseId(UUID.fromString(row[0].toString()))
466         .name((String) row[1])
467         .description((String) row[2])
468         .status((String) row[3])
469         .distanceMeters(((Number) row[4]).doubleValue())
470         .isLoop((Boolean) row[5])
471         .startPoint(new GeoPoint(((Number) row[6]).doubleValue(), ((Number) row[7]).doubleValue()))
472         .endPoint(new GeoPoint(((Number) row[8]).doubleValue(), ((Number) row[9]).doubleValue()))
473         .attemptCount(((Number) row[10]).intValue())
474         .completionCount(((Number) row[11]).intValue())
475         .createdAt(toInstant(row[12]))
476         .updatedAt(toInstant(row[13]))
477         .build();
478 }
479
480 private ParticipatedCourseItem toParticipatedCourseItem(Object[] row) {
481     return ParticipatedCourseItem.builder()
482         .courseId(UUID.fromString(row[0].toString()))
483         .name((String) row[1])
484         .description((String) row[2])
485         .distanceMeters(((Number) row[3]).doubleValue())
486         .isLoop((Boolean) row[4])
487         .status((String) row[5])
488         .attemptCountByMe(((Number) row[6]).intValue())
489         .completionCountByMe(((Number) row[7]).intValue())
490         .bestTimeSecondsByMe(row[8] != null ? ((Number) row[8]).intValue() : null)
491         .lastAttemptAt(toInstant(row[9]))
492         .build();
493 }
494
495 private static Instant toInstant(Object val) {
496     if (val == null) return null;
497     if (val instanceof Instant i) return i;
498     if (val instanceof java.sql.Timestamp ts) return ts.toInstant();
499     if (val instanceof java.time.OffsetDateTime odt) return odt.toInstant();
500     return null;
501 }
502
503 private Course findOwnedCourse(UUID courseId, UUID userId) {
504     Course course = courseRepository.findByIdAndDeletedAtIsNull(courseId)
505         .orElseThrow(() -> new RunwayException(Errorcode.COURSE_NOT_FOUND));
506     if (!course.isOwnedBy(userId)) {
507         throw new RunwayException(Errorcode.FORBIDDEN);
508     }
509     return course;
510 }

```

```

511
512 private Course findVisibleCourse(UUID courseId, UUID userId) {
513     Course course = courseRepository.findByIdAndDeletedAtIsNull(courseId)
514         .orElseThrow(() -> new RunwayException(ErrorCode.COURSE_NOT_FOUND));
515     if (!course.isVisibleTo(userId)) {
516         throw new RunwayException(ErrorCode.FORBIDDEN);
517     }
518     return course;
519 }
520
521 private List<RunningPoint> downsample(List<RunningPoint> all) {
522     if (all.size() <= MAX_COURSE_POINTS) {
523         return new ArrayList<>(all);
524     }
525     // 간격 샘플링: 첫 점과 끝 점은 항상 포함
526     List<RunningPoint> result = new ArrayList<>();
527     int step = Math.max(1, (int) Math.ceil((double) all.size() / MAX_COURSE_POINTS));
528     for (int i = 0; i < all.size(); i++) {
529         if (i == 0 || i == all.size() - 1 || i % step == 0) {
530             result.add(all.get(i));
531         }
532     }
533     return result;
534 }
535
536 private List<CoursePoint> buildCoursePoints(UUID courseId, List<RunningPoint> sampledPoints) {
537     List<CoursePoint> result = new ArrayList<>();
538     for (int i = 0; i < sampledPoints.size(); i++) {
539         RunningPoint rp = sampledPoints.get(i);
540         result.add(CoursePoint.builder()
541             .courseId(courseId)
542             .sequence(i)
543             .location(rp.getLocation())
544             .altitudeMeters(rp.getAltitudeMeters())
545             .build());
546     }
547     return result;
548 }
549
550 private jakarta.persistence.Query buildQuery(String sql, List<Object> params) {
551     jakarta.persistence.Query query = entityManager.createNativeQuery(sql);
552     for (int i = 0; i < params.size(); i++) {
553         query.setParameter(i + 1, params.get(i));
554     }
555     return query;
556 }
557
558 private NearbyCourseItem toNearbyCourseItem(Object[] row, boolean includeExactStartPoint) {
559     double startLat = ((Number) row[8]).doubleValue();
560     double startLon = ((Number) row[9]).doubleValue();
561     if (!includeExactStartPoint) {
562         // Mask start coordinates to ~1.1 km precision for regular public lists.
563         startLat = CoursePrivacyUtils.maskCoordinate(startLat);
564         startLon = CoursePrivacyUtils.maskCoordinate(startLon);
565     }
566     return NearbyCourseItem.builder()
567         .courseId(UUID.fromString(row[0].toString()))
568         .name((String) row[1])
569         .description((String) row[2])
570         .distanceMeters(((Number) row[3]).doubleValue())
571         .distanceFromMeMeters(((Number) row[4]).doubleValue())
572         .isLoop((Boolean) row[5])
573         .attemptCount(((Number) row[6]).intValue())
574         .completionCount(((Number) row[7]).intValue())
575         .startPoint(new GeoPoint(startLat, startLon))
576         .avgRating(row[10] != null ? ((Number) row[10]).doubleValue() : null)
577         .ratingCount(row[11] != null ? ((Number) row[11]).longValue() : null)
578         .routePoints(List.of())
579         .build();
580 }
581
582 private List<GeoPoint> thinPoints(List<GeoPoint> points, int maxCount) {
583     if (points.size() <= maxCount) return points;
584     List<GeoPoint> result = new ArrayList<>(maxCount);
585     double step = (double) (points.size() - 1) / (maxCount - 1);
586     for (int i = 0; i < maxCount; i++) {
587         result.add(points.get((int) Math.round(i * step)));
588     }
589     return result;
590 }
591 }

```

```

1 package com.runway.course.util;
2
3 import com.runway.course.dto.CoursePointResponse;
4
5 import java.util.ArrayList;
6 import java.util.List;
7
8 public final class CoursePrivacyUtils {
9
10     public static final double PRIVACY_ZONE_METERS = 150.0;
11     private static final double EARTH_RADIUS_METERS = 6_371_000.0;
12
13     private CoursePrivacyUtils() {}
14
15     /**
16      * Returns a new list omitting points within PRIVACY_ZONE_METERS of the start or end.
17      * Cumulative path distance is used - not straight-line distance from endpoints.
18      * Does not mutate DB data; call only when building API responses.
19      */
20     public static List<CoursePointResponse> maskPrivacyZone(List<CoursePointResponse> points) {
21         if (points.size() < 2) return points;
22
23         double[] cumFromStart = cumulativeDistances(points);
24         double totalLength = cumFromStart[points.size() - 1];
25
26         List<CoursePointResponse> result = new ArrayList<>();
27         for (int i = 0; i < points.size(); i++) {
28             double fromStart = cumFromStart[i];
29             double fromEnd = totalLength - fromStart;
30             if (fromStart >= PRIVACY_ZONE_METERS && fromEnd >= PRIVACY_ZONE_METERS) {
31                 result.add(points.get(i));
32             }
33         }
34         return result;
35     }
36
37     /**
38      * Rounds a coordinate to ~1.1 km precision (2 decimal places).
39      * Use for start-point coordinates in public-facing responses (e.g. nearby list).
40      */
41     public static double maskCoordinate(double coord) {
42         return Math.round(coord * 100.0) / 100.0;
43     }
44
45     private static double[] cumulativeDistances(List<CoursePointResponse> points) {
46         double[] cum = new double[points.size()];
47         cum[0] = 0;
48         for (int i = 1; i < points.size(); i++) {
49             cum[i] = cum[i - 1] + haversine(
50                 points.get(i - 1).getLatitude(), points.get(i - 1).getLongitude(),
51                 points.get(i).getLatitude(), points.get(i).getLongitude());
52         }
53         return cum;
54     }
55
56     private static double haversine(double lat1, double lon1, double lat2, double lon2) {
57         double dLat = Math.toRadians(lat2 - lat1);
58         double dLon = Math.toRadians(lon2 - lon1);
59         double a = Math.sin(dLat / 2) * Math.sin(dLat / 2)
60             + Math.cos(Math.toRadians(lat1)) * Math.cos(Math.toRadians(lat2))
61             * Math.sin(dLon / 2) * Math.sin(dLon / 2);
62         return EARTH_RADIUS_METERS * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
63     }
64 }

```



```

1  package com.runway.run.controller;
2
3  import com.runway.common.response.ApiResponse;
4  import com.runway.common.response.PageResponse;
5  import com.runway.common.security.UserPrincipal;
6  import com.runway.run.dto.*;
7  import com.runway.run.service.RunningService;
8  import io.swagger.v3.oas.annotations.Operation;
9  import io.swagger.v3.oas.annotations.security.SecurityRequirement;
10 import io.swagger.v3.oas.annotations.tags.Tag;
11 import jakarta.validation.Valid;
12 import lombok.RequiredArgsConstructor;
13 import org.springframework.http.ResponseEntity;
14 import org.springframework.security.core.annotation.AuthenticationPrincipal;
15 import org.springframework.web.bind.annotation.*;
16
17 import java.util.UUID;
18
19 @Tag(name = "Running", description = "러닝 기록 API")
20 @RestController
21 @RequestMapping("/api/runs")
22 @RequiredArgsConstructor
23 public class RunningController {
24
25     private final RunningService runningService;
26
27     @Operation(summary = "러닝 시작", security = @SecurityRequirement(name = "bearerAuth"))
28     @PostMapping("/start")
29     public ResponseEntity<ApiResponse<StartRunResponse>> startRun(
30         @AuthenticationPrincipal UserPrincipal principal,
31         @RequestBody(required = false) StartRunRequest request) {
32         StartRunResponse data = runningService.startRun(
33             principal.getUserId(),
34             request != null ? request : new StartRunRequest()
35         );
36         return ResponseEntity.ok(ApiResponse.success("러닝이 시작되었습니다.", data));
37     }
38
39     @Operation(summary = "GPS 포인트 배치 저장", security = @SecurityRequirement(name = "bearerAuth"))
40     @PostMapping("/{runId}/points")
41     public ResponseEntity<ApiResponse<SavePointsResponse>> savePoints(
42         @AuthenticationPrincipal UserPrincipal principal,
43         @PathVariable UUID runId,
44         @Valid @RequestBody SavePointsRequest request) {
45         SavePointsResponse data = runningService.savePoints(principal.getUserId(), runId, request);
46         return ResponseEntity.ok(ApiResponse.success("GPS 포인트가 저장되었습니다.", data));
47     }
48
49     @Operation(summary = "러닝 일시정지", security = @SecurityRequirement(name = "bearerAuth"))
50     @PostMapping("/{runId}/pause")
51     public ResponseEntity<ApiResponse<RunStatusResponse>> pauseRun(
52         @AuthenticationPrincipal UserPrincipal principal,
53         @PathVariable UUID runId) {
54         RunStatusResponse data = runningService.pauseRun(principal.getUserId(), runId);
55         return ResponseEntity.ok(ApiResponse.success("러닝이 일시정지되었습니다.", data));
56     }
57
58     @Operation(summary = "러닝 재개", security = @SecurityRequirement(name = "bearerAuth"))
59     @PostMapping("/{runId}/resume")
60     public ResponseEntity<ApiResponse<RunStatusResponse>> resumeRun(
61         @AuthenticationPrincipal UserPrincipal principal,
62         @PathVariable UUID runId) {
63         RunStatusResponse data = runningService.resumeRun(principal.getUserId(), runId);
64         return ResponseEntity.ok(ApiResponse.success("러닝이 재개되었습니다.", data));
65     }
66
67     @Operation(summary = "러닝 완료", security = @SecurityRequirement(name = "bearerAuth"))
68     @PostMapping("/{runId}/finish")
69     public ResponseEntity<ApiResponse<FinishRunResponse>> finishRun(
70         @AuthenticationPrincipal UserPrincipal principal,
71         @PathVariable UUID runId,
72         @Valid @RequestBody FinishRunRequest request) {
73         FinishRunResponse data = runningService.finishRun(principal.getUserId(), runId, request);
74         return ResponseEntity.ok(ApiResponse.success("러닝이 완료되었습니다.", data));
75     }
76
77     @Operation(summary = "러닝 중단", security = @SecurityRequirement(name = "bearerAuth"))
78     @PostMapping("/{runId}/abandon")
79     public ResponseEntity<ApiResponse<RunStatusResponse>> abandonRun(
80         @AuthenticationPrincipal UserPrincipal principal,
81         @PathVariable UUID runId) {
82         RunStatusResponse data = runningService.abandonRun(principal.getUserId(), runId);
83         return ResponseEntity.ok(ApiResponse.success("러닝이 중단되었습니다.", data));

```

```

84     }
85
86     @Operation(summary = "내 러닝 기록 목록 조회", security = @SecurityRequirement(name = "bearerAuth"))
87     @GetMapping("/me")
88     public ResponseEntity<ApiResponse<PageResponse<RunSummaryResponse>>> getMyRuns(
89         @AuthenticationPrincipal UserPrincipal principal,
90         @RequestParam(defaultValue = "0") int page,
91         @RequestParam(defaultValue = "20") int size) {
92         PageResponse<RunSummaryResponse> data = runningService.getMyRuns(principal.getUserId(), page, size);
93         return ResponseEntity.ok(ApiResponse.success("내 러닝 기록 목록 조회가 완료되었습니다.", data));
94     }
95
96     @Operation(summary = "러닝 기록 상세 조회", security = @SecurityRequirement(name = "bearerAuth"))
97     @GetMapping("/{runId}")
98     public ResponseEntity<ApiResponse<RunDetailResponse>> getRunDetail(
99         @AuthenticationPrincipal UserPrincipal principal,
100         @PathVariable UUID runId) {
101         RunDetailResponse data = runningService.getRunDetail(principal.getUserId(), runId);
102         return ResponseEntity.ok(ApiResponse.success("러닝 기록 상세 조회가 완료되었습니다.", data));
103     }
104
105     @Operation(summary = "개인 최고 기록 조회", security = @SecurityRequirement(name = "bearerAuth"))
106     @GetMapping("/me/records")
107     public ResponseEntity<ApiResponse<Object>> getPersonalRecords(
108         @AuthenticationPrincipal UserPrincipal principal) {
109         var data = runningService.getPersonalRecords(principal.getUserId());
110         return ResponseEntity.ok(ApiResponse.success("개인 최고 기록 조회가 완료되었습니다.", data));
111     }
112
113     @Operation(summary = "러닝 통계 조회 (weekly/monthly/yearly/all)", security = @SecurityRequirement(name = "bearerAuth"))
114     @GetMapping("/me/stats")
115     public ResponseEntity<ApiResponse<com.runway.run.dto.RunningStatsResponse>> getRunningStats(
116         @AuthenticationPrincipal UserPrincipal principal,
117         @RequestParam(defaultValue = "monthly") String period) {
118         var data = runningService.getRunningStats(principal.getUserId(), period);
119         return ResponseEntity.ok(ApiResponse.success("러닝 통계 조회가 완료되었습니다.", data));
120     }
121
122     @Operation(summary = "러닝 기록 삭제", security = @SecurityRequirement(name = "bearerAuth"))
123     @DeleteMapping("/{runId}")
124     public ResponseEntity<ApiResponse<Void>> deleteRun(
125         @AuthenticationPrincipal UserPrincipal principal,
126         @PathVariable UUID runId) {
127         runningService.deleteRun(principal.getUserId(), runId);
128         return ResponseEntity.ok(ApiResponse.success("러닝 기록이 삭제되었습니다."));
129     }
130
131     @Operation(summary = "러닝 기록 거리 단축 (줄이기는 가능)", security = @SecurityRequirement(name = "bearerAuth"))
132     @PatchMapping("/{runId}/trim")
133     public ResponseEntity<ApiResponse<Void>> trimRun(
134         @AuthenticationPrincipal UserPrincipal principal,
135         @PathVariable UUID runId,
136         @RequestBody RunTrimRequest request) {
137         runningService.trimRun(principal.getUserId(), runId, request.getTargetDistanceMeters());
138         return ResponseEntity.ok(ApiResponse.success("러닝 기록이 수정되었습니다."));
139     }
140 }

```

```
1 package com.runway.run.domain;
2
3 import jakarta.persistence.*;
4 import lombok.AccessLevel;
5 import lombok.Builder;
6 import lombok.Getter;
7 import lombok.NoArgsConstructor;
8 import org.locationtech.jts.geom.Point;
9
10 import java.time.Instant;
11 import java.util.UUID;
12
13 @Entity
14 @Table(name = "running_points")
15 @Getter
16 @NoArgsConstructor(access = AccessLevel.PROTECTED)
17 public class RunningPoint {
18
19     @Id
20     @GeneratedValue(strategy = GenerationType.UUID)
21     @Column(columnDefinition = "uuid", updatable = false, nullable = false)
22     private UUID id;
23
24     @Column(name = "running_record_id", nullable = false, columnDefinition = "uuid")
25     private UUID runningRecordId;
26
27     @Column(nullable = false)
28     private Integer sequence;
29
30     // JTS Point → Hibernate Spatial 이 geography(Point,4326) 로 변환
31     @Column(columnDefinition = "geography(Point,4326)", nullable = false)
32     private Point location;
33
34     @Column(name = "altitude_meters")
35     private Double altitudeMeters;
36
37     @Column(name = "speed_mps")
38     private Double speedMps;
39
40     @Column(name = "recorded_at", nullable = false)
41     private Instant recordedAt;
42
43     @Builder
44     private RunningPoint(UUID runningRecordId, Integer sequence, Point location,
45         Double altitudeMeters, Double speedMps, Instant recordedAt) {
46         this.runningRecordId = runningRecordId;
47         this.sequence = sequence;
48         this.location = location;
49         this.altitudeMeters = altitudeMeters;
50         this.speedMps = speedMps;
51         this.recordedAt = recordedAt;
52     }
53 }
```

```

1  package com.runway.run.domain;
2
3  import com.runway.run.domain.enums.RunningRecordStatus;
4  import jakarta.persistence.*;
5  import lombok.AccessLevel;
6  import lombok.Builder;
7  import lombok.Getter;
8  import lombok.NoArgsConstructor;
9  import org.locationtech.jts.geom.LineString;
10
11 import java.time.Instant;
12 import java.util.UUID;
13
14 @Entity
15 @Table(name = "running_records")
16 @Getter
17 @NoArgsConstructor(access = AccessLevel.PROTECTED)
18 public class RunningRecord {
19
20     @Id
21     @GeneratedValue(strategy = GenerationType.UUID)
22     @Column(columnDefinition = "uuid", updatable = false, nullable = false)
23     private UUID id;
24
25     @Column(name = "user_id", nullable = false, columnDefinition = "uuid")
26     private UUID userId;
27
28     // RunningRecordStatus.JpaConverter 가 autoApply=true 이므로 @Convert 불필요
29     @Column(nullable = false, length = 30)
30     private RunningRecordStatus status;
31
32     @Column(name = "started_at", nullable = false)
33     private Instant startedAt;
34
35     @Column(name = "ended_at")
36     private Instant endedAt;
37
38     @Column(name = "distance_meters", nullable = false)
39     private Double distanceMeters;
40
41     @Column(name = "duration_seconds", nullable = false)
42     private Integer durationSeconds;
43
44     @Column(name = "avg_pace_seconds_per_km")
45     private Integer avgPaceSecondsPerKm;
46
47     @Column(name = "avg_heart_rate_bpm")
48     private Integer avgHeartRateBpm;
49
50     @Column(name = "calories_burned")
51     private Integer caloriesBurned;
52
53     @Column(columnDefinition = "geometry(LineString,4326)")
54     private LineString path;
55
56     @Column(name = "created_at", nullable = false, updatable = false)
57     private Instant createdAt;
58
59     @Builder
60     private RunningRecord(UUID userId, Instant startedAt) {
61         this.userId = userId;
62         this.startedAt = startedAt;
63         this.status = RunningRecordStatus.IN_PROGRESS;
64         this.distanceMeters = 0.0;
65         this.durationSeconds = 0;
66     }
67
68     @PrePersist
69     private void prePersist() {
70         this.createdAt = Instant.now();
71     }
72
73     public void pause() {
74         this.status = RunningRecordStatus.PAUSED;
75     }
76
77     public void resume() {
78         this.status = RunningRecordStatus.IN_PROGRESS;
79     }
80
81     public void finish(Instant endedAt, Double distanceMeters, Integer durationSeconds,
82                       Integer avgPaceSecondsPerKm, Integer caloriesBurned, Integer avgHeartRateBpm) {
83         this.status = RunningRecordStatus.COMPLETED;

```

```

84         this.endedAt = endedAt;
85         this.distanceMeters = distanceMeters;
86         this.durationSeconds = durationSeconds;
87         this.avgPaceSecondsPerKm = avgPaceSecondsPerKm;
88         this.caloriesBurned = caloriesBurned;
89         this.avgHeartRateBpm = avgHeartRateBpm;
90     }
91
92     public void abandon() {
93         abandon(null);
94     }
95
96     public void abandon(Instant endedAt) {
97         this.status = RunningRecordStatus.ABANDONED;
98         this.endedAt = endedAt != null ? endedAt : Instant.now();
99     }
100
101     public void trim(double targetDistanceMeters) {
102         double ratio = targetDistanceMeters / this.distanceMeters;
103         this.durationSeconds = (int) Math.round(this.durationSeconds * ratio);
104         this.distanceMeters = targetDistanceMeters;
105         this.avgPaceSecondsPerKm = targetDistanceMeters > 0
106             ? (int) Math.round(this.durationSeconds / (targetDistanceMeters / 1000.0))
107             : this.avgPaceSecondsPerKm;
108         if (this.caloriesBurned != null) {
109             this.caloriesBurned = (int) Math.round(this.caloriesBurned * ratio);
110         }
111     }
112
113     public void updatePath(LineString path) {
114         this.path = path;
115     }
116
117     public boolean isOwnedBy(UUID userId) {
118         return this.userId.equals(userId);
119     }
120 }

```

backend/src/main/java/com/runway/run/domain/enums/RunningRecordStatus.java

Java | 45줄

```
1 package com.runway.run.domain.enums;
2
3 import jakarta.persistence.AttributeConverter;
4 import jakarta.persistence.Converter;
5
6 public enum RunningRecordStatus {
7
8     IN_PROGRESS("in_progress"),
9     PAUSED("paused"),
10    COMPLETED("completed"),
11    ABANDONED("abandoned");
12
13    private final String dbValue;
14
15    RunningRecordStatus(String dbValue) {
16        this.dbValue = dbValue;
17    }
18
19    public String getDbValue() {
20        return dbValue;
21    }
22
23    public static RunningRecordStatus from(String value) {
24        for (RunningRecordStatus status : values()) {
25            if (status.dbValue.equals(value)) {
26                return status;
27            }
28        }
29        throw new IllegalArgumentException("Unknown RunningRecordStatus: " + value);
30    }
31
32    @Converter(autoApply = true)
33    public static class JpaConverter implements AttributeConverter<RunningRecordStatus, String> {
34
35        @Override
36        public String convertToDatabaseColumn(RunningRecordStatus attribute) {
37            return attribute == null ? null : attribute.getDbValue();
38        }
39
40        @Override
41        public RunningRecordStatus convertToEntityAttribute(String dbData) {
42            return dbData == null ? null : RunningRecordStatus.from(dbData);
43        }
44    }
45 }
```

backend/src/main/java/com/runway/run/dto/FinishRunRequest.java

Java | 28줄

```
1 package com.runway.run.dto;
2
3 import jakarta.validation.constraints.NotNull;
4 import jakarta.validation.constraints.PositiveOrZero;
5 import lombok.Getter;
6
7 import java.time.Instant;
8
9 @Getter
10 public class FinishRunRequest {
11
12     @NotNull(message = "endedAt은 필수입니다.")
13     private Instant endedAt;
14
15     @NotNull(message = "distanceMeters는 필수입니다.")
16     @PositiveOrZero(message = "distanceMeters는 0 이상이어야 합니다.")
17     private Double distanceMeters;
18
19     @NotNull(message = "durationSeconds는 필수입니다.")
20     @PositiveOrZero(message = "durationSeconds는 0 이상이어야 합니다.")
21     private Integer durationSeconds;
22
23     private Integer avgPaceSecondsPerKm;
24
25     private Integer caloriesBurned;
26
27     private Integer avgHeartRateBpm;
28 }
```

backend/src/main/java/com/runway/run/dto/FinishRunResponse.java

Java | 37줄

```
1 package com.runway.run.dto;
2
3 import com.runway.run.domain.RunningRecord;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.UUID;
9
10 @Getter
11 @Builder
12 public class FinishRunResponse {
13
14     private UUID runId;
15     private String status;
16     private Instant startedAt;
17     private Instant endedAt;
18     private Double distanceMeters;
19     private Integer durationSeconds;
20     private Integer avgPaceSecondsPerKm;
21     private Integer caloriesBurned;
22     private boolean pathCreated;
23
24     public static FinishRunResponse from(RunningRecord record, boolean pathCreated) {
25         return FinishRunResponse.builder()
26             .runId(record.getId())
27             .status(record.getStatus().getDbValue())
28             .startedAt(record.getStartedAt())
29             .endedAt(record.getEndedAt())
30             .distanceMeters(record.getDistanceMeters())
31             .durationSeconds(record.getDurationSeconds())
32             .avgPaceSecondsPerKm(record.getAvgPaceSecondsPerKm())
33             .caloriesBurned(record.getCaloriesBurned())
34             .pathCreated(pathCreated)
35             .build();
36     }
37 }
```

backend/src/main/java/com/runway/run/dto/PersonalRecordItemResponse.java

Java | 31줄

```
1 package com.runway.run.dto;
2
3 import com.runway.run.domain.RunningRecord;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.UUID;
9
10 @Getter
11 @Builder
12 public class PersonalRecordItemResponse {
13
14     private UUID runId;
15     private Double distanceMeters;
16     private Integer durationSeconds;
17     private Integer avgPaceSecondsPerKm;
18     private Integer caloriesBurned;
19     private Instant recordedAt;
20
21     public static PersonalRecordItemResponse from(RunningRecord r) {
22         return PersonalRecordItemResponse.builder()
23             .runId(r.getId())
24             .distanceMeters(r.getDistanceMeters())
25             .durationSeconds(r.getDurationSeconds())
26             .avgPaceSecondsPerKm(r.getAvgPaceSecondsPerKm())
27             .caloriesBurned(r.getCaloriesBurned())
28             .recordedAt(r.getStartedAt())
29             .build();
30     }
31 }
```

backend/src/main/java/com/runway/run/dto/PersonalRecordsResponse.java

Java | 17줄

```
1 package com.runway.run.dto;
2
3 import lombok.Builder;
4 import lombok.Getter;
5
6 @Getter
7 @Builder
8 public class PersonalRecordsResponse {
9
10     private PersonalRecordItemResponse longestRun;
11     private PersonalRecordItemResponse fastestPace;
12     private PersonalRecordItemResponse mostCalories;
13     private PersonalRecordItemResponse best5k;
14     private PersonalRecordItemResponse best10k;
15     private long totalCompletedRuns;
16     private double totalDistanceMeters;
17 }
```

backend/src/main/java/com/runway/run/dto/RunDetailResponse.java

Java | 39줄

```
1 package com.runway.run.dto;
2
3 import com.runway.run.domain.RunningPoint;
4 import com.runway.run.domain.RunningRecord;
5 import lombok.Builder;
6 import lombok.Getter;
7
8 import java.time.Instant;
9 import java.util.List;
10 import java.util.UUID;
11
12 @Getter
13 @Builder
14 public class RunDetailResponse {
15
16     private UUID runId;
17     private String status;
18     private Instant startedAt;
19     private Instant endedAt;
20     private Double distanceMeters;
21     private Integer durationSeconds;
22     private Integer avgPaceSecondsPerKm;
23     private Integer caloriesBurned;
24     private List<RunPointResponse> points;
25
26     public static RunDetailResponse from(RunningRecord record, List<RunningPoint> points) {
27         return RunDetailResponse.builder()
28             .runId(record.getId())
29             .status(record.getStatus().getDbValue())
30             .startedAt(record.getStartedAt())
31             .endedAt(record.getEndedAt())
32             .distanceMeters(record.getDistanceMeters())
33             .durationSeconds(record.getDurationSeconds())
34             .avgPaceSecondsPerKm(record.getAvgPaceSecondsPerKm())
35             .caloriesBurned(record.getCaloriesBurned())
36             .points(points.stream().map(RunPointResponse::from).toList())
37             .build();
38     }
39 }
```


backend/src/main/java/com/runway/run/dto/RunPointResponse.java

Java | 31줄

```
1 package com.runway.run.dto;
2
3 import com.runway.run.domain.RunningPoint;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8
9 @Getter
10 @Builder
11 public class RunPointResponse {
12
13     private Integer sequence;
14     private Double latitude;
15     private Double longitude;
16     private Double altitudeMeters;
17     private Double speedMps;
18     private Instant recordedAt;
19
20     public static RunPointResponse from(RunningPoint point) {
21         // JTS Coordinate: x = longitude, y = latitude
22         return RunPointResponse.builder()
23             .sequence(point.getSequence())
24             .latitude(point.getLocation().getY())
25             .longitude(point.getLocation().getX())
26             .altitudeMeters(point.getAltitudeMeters())
27             .speedMps(point.getSpeedMps())
28             .recordedAt(point.getRecordedAt())
29             .build();
30     }
31 }
```

backend/src/main/java/com/runway/run/dto/RunStatusResponse.java

Java | 22줄

```
1 package com.runway.run.dto;
2
3 import com.runway.run.domain.RunningRecord;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.util.UUID;
8
9 @Getter
10 @Builder
11 public class RunStatusResponse {
12
13     private UUID runId;
14     private String status;
15
16     public static RunStatusResponse from(RunningRecord record) {
17         return RunStatusResponse.builder()
18             .runId(record.getId())
19             .status(record.getStatus().getDbValue())
20             .build();
21     }
22 }
```

backend/src/main/java/com/runway/run/dto/RunSummaryResponse.java

Java | 35줄

```
1 package com.runway.run.dto;
2
3 import com.runway.run.domain.RunningRecord;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.UUID;
9
10 @Getter
11 @Builder
12 public class RunSummaryResponse {
13
14     private UUID runId;
15     private String status;
16     private Instant startedAt;
17     private Instant endedAt;
18     private Double distanceMeters;
19     private Integer durationSeconds;
20     private Integer avgPaceSecondsPerKm;
21     private Integer caloriesBurned;
22
23     public static RunSummaryResponse from(RunningRecord record) {
24         return RunSummaryResponse.builder()
25             .runId(record.getId())
26             .status(record.getStatus().getDbValue())
27             .startedAt(record.getStartedAt())
28             .endedAt(record.getEndedAt())
29             .distanceMeters(record.getDistanceMeters())
30             .durationSeconds(record.getDurationSeconds())
31             .avgPaceSecondsPerKm(record.getAvgPaceSecondsPerKm())
32             .caloriesBurned(record.getCaloriesBurned())
33             .build();
34     }
35 }
```

backend/src/main/java/com/runway/run/dto/RunTrimRequest.java

Java | 10줄

```
1 package com.runway.run.dto;
2
3 import lombok.Getter;
4 import lombok.NoArgsConstructor;
5
6 @Getter
7 @NoArgsConstructor
8 public class RunTrimRequest {
9     private Double targetDistanceMeters;
10 }
```

backend/src/main/java/com/runway/run/dto/RunningStatsResponse.java

Java | 24줄

```
1 package com.runway.run.dto;
2
3 import lombok.Builder;
4 import lombok.Getter;
5
6 import java.time.Instant;
7
8 @Getter
9 @Builder
10 public class RunningStatsResponse {
11
12     private String period;
13     private long totalRuns;
14     private double totalDistanceMeters;
15     private long totalDurationSeconds;
16     private long totalCaloriesBurned;
17     private int averagePaceSecondsPerKm;
18     private double longestRunMeters;
19     private int currentStreakDays;
20     private int longestStreakDays;
21     private long activeDays;
22     private Instant periodStart;
23     private Instant periodEnd;
24 }
```

backend/src/main/java/com/runway/run/dto/SavePointsRequest.java

Java | 40줄

```
1 package com.runway.run.dto;
2
3 import jakarta.validation.Valid;
4 import jakarta.validation.constraints.*;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.List;
9
10 @Getter
11 public class SavePointsRequest {
12
13     @NotEmpty(message = "포인트 목록은 비어 있을 수 없습니다.")
14     @Valid
15     private List<PointData> points;
16
17     @Getter
18     public static class PointData {
19
20         @NotNull(message = "sequence는 필수입니다.")
21         private Integer sequence;
22
23         @NotNull(message = "latitude는 필수입니다.")
24         @DecimalMin(value = "-90.0", message = "latitude는 -90 이상이어야 합니다.")
25         @DecimalMax(value = "90.0", message = "latitude는 90 이하이어야 합니다.")
26         private Double latitude;
27
28         @NotNull(message = "longitude는 필수입니다.")
29         @DecimalMin(value = "-180.0", message = "longitude는 -180 이상이어야 합니다.")
30         @DecimalMax(value = "180.0", message = "longitude는 180 이하이어야 합니다.")
31         private Double longitude;
32
33         private Double altitudeMeters;
34
35         private Double speedMps;
36
37         @NotNull(message = "recordedAt은 필수입니다.")
38         private Instant recordedAt;
39     }
40 }
```

backend/src/main/java/com/runway/run/dto/SavePointsResponse.java

Java | 14줄

```
1 package com.runway.run.dto;
2
3 import lombok.Getter;
4 import lombok.RequiredArgsConstructor;
5
6 import java.util.UUID;
7
8 @Getter
9 @RequiredArgsConstructor
10 public class SavePointsResponse {
11
12     private final UUID runId;
13     private final int savedCount;
14 }
```

backend/src/main/java/com/runway/run/dto/StartRunRequest.java

Java | 12줄

```
1 package com.runway.run.dto;
2
3 import lombok.Getter;
4
5 import java.time.Instant;
6
7 @Getter
8 public class StartRunRequest {
9
10     // null 허용 - null이면 서버가 Instant.now() 사용
11     private Instant startedAt;
12 }
```

backend/src/main/java/com/runway/run/dto/StartRunResponse.java

Java | 25줄

```
1 package com.runway.run.dto;
2
3 import com.runway.run.domain.RunningRecord;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.UUID;
9
10 @Getter
11 @Builder
12 public class StartRunResponse {
13
14     private UUID runId;
15     private String status;
16     private Instant startedAt;
17
18     public static StartRunResponse from(RunningRecord record) {
19         return StartRunResponse.builder()
20             .runId(record.getId())
21             .status(record.getStatus().getDbValue())
22             .startedAt(record.getStartedAt())
23             .build();
24     }
25 }
```

backend/src/main/java/com/runway/run/repository/RunDateProjection.java

Java | 7줄

```
1 package com.runway.run.repository;
2
3 import java.time.LocalDate;
4
5 public interface RunDateProjection {
6     LocalDate getRunDate();
7 }
```

backend/src/main/java/com/runway/run/repository/RunSummaryProjection.java

Java | 8줄

```
1 package com.runway.run.repository;
2
3 import java.time.Instant;
4
5 public interface RunSummaryProjection {
6     Double getDistanceMeters();
7     Instant getStartedAt();
8 }
```

```
1 package com.runway.run.repository;
2
3 import com.runway.run.dto.SavePointsRequest;
4 import lombok.RequiredArgsConstructor;
5 import org.springframework.jdbc.core.JdbcTemplate;
6 import org.springframework.stereotype.Component;
7
8 import java.sql.Timestamp;
9 import java.util.List;
10 import java.util.UUID;
11
12 @Component
13 @RequiredArgsConstructor
14 public class RunningPointBatchInserter {
15
16     private static final String SQL = """
17         INSERT INTO running_points
18             (id, running_record_id, sequence, location, altitude_meters, speed_mps, recorded_at)
19         VALUES
20             (gen_random_uuid(), ?, ?,
21              ST_SetSRID(ST_MakePoint(?, ?), 4326)::geography,
22              ?, ?, ?)
23         ON CONFLICT (running_record_id, sequence) DO NOTHING
24         """;
25
26     private final JdbcTemplate jdbcTemplate;
27
28     public void batchInsert(UUID runId, List<SavePointsRequest.PointData> points) {
29         jdbcTemplate.batchUpdate(SQL, points, points.size(), (ps, p) -> {
30             ps.setObject(1, runId);
31             ps.setInt(2, p.getSequence());
32             ps.setDouble(3, p.getLongitude()); // ST_MakePoint(lon, lat) order
33             ps.setDouble(4, p.getLatitude());
34             ps.setObject(5, p.getAltitudeMeters());
35             ps.setObject(6, p.getSpeedMps());
36             ps.setTimestamp(7, p.getRecordedAt() != null
37                 ? Timestamp.from(p.getRecordedAt()) : null);
38         });
39     }
40 }
```

```
1 package com.runway.run.repository;
2
3 import com.runway.run.domain.RunningPoint;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.Modifying;
6 import org.springframework.data.jpa.repository.Query;
7 import org.springframework.data.repository.query.Param;
8
9 import java.time.Instant;
10 import java.util.List;
11 import java.util.UUID;
12
13 public interface RunningPointRepository extends JpaRepository<RunningPoint, UUID> {
14
15     List<RunningPoint> findByRunningRecordIdOrderBySequenceAsc(UUID runningRecordId);
16
17     void deleteByRunningRecordId(UUID runningRecordId);
18
19     @Modifying
20     @Query("DELETE FROM RunningPoint rp WHERE rp.runningRecordId = :runId AND rp.sequence > :seq")
21     void deleteByRunningRecordIdAndSequenceGreaterThan(
22         @Param("runId") UUID runId,
23         @Param("seq") int seq
24     );
25
26     @Modifying
27     @Query(value = ""
28         INSERT INTO running_points
29         (id, running_record_id, sequence, location, altitude_meters, speed_mps, recorded_at)
30         VALUES
31         (gen_random_uuid(), :runningRecordId, :sequence,
32         ST_SetSRID(ST_MakePoint(:longitude, :latitude), 4326)::geography,
33         :altitudeMeters, :speedMps, :recordedAt)
34         ON CONFLICT (running_record_id, sequence) DO NOTHING
35         """, nativeQuery = true)
36     void insertIgnoreConflict(
37         @Param("runningRecordId") UUID runningRecordId,
38         @Param("sequence") int sequence,
39         @Param("latitude") double latitude,
40         @Param("longitude") double longitude,
41         @Param("altitudeMeters") Double altitudeMeters,
42         @Param("speedMps") Double speedMps,
43         @Param("recordedAt") Instant recordedAt
44     );
45 }
```

```

1 package com.runway.run.repository;
2
3 import com.runway.run.domain.RunningRecord;
4 import com.runway.run.domain.enums.RunningRecordStatus;
5 import org.springframework.data.domain.Page;
6 import org.springframework.data.domain.Pageable;
7 import org.springframework.data.jpa.repository.JpaRepository;
8 import org.springframework.data.jpa.repository.Query;
9 import org.springframework.data.repository.query.Param;
10
11 import java.time.Instant;
12 import java.util.List;
13 import java.util.Optional;
14 import java.util.UUID;
15
16 public interface RunningRecordRepository extends JpaRepository<RunningRecord, UUID> {
17
18     Optional<RunningRecord> findByIdAndUserId(UUID id, UUID userId);
19
20     Page<RunningRecord> findByUserIdOrderByStartedAtDesc(UUID userId, Pageable pageable);
21
22     // — Personal Records 쿼리 —
23
24     Optional<RunningRecord> findTop1ByUserIdAndStatusOrderByDistanceMetersDesc(
25         UUID userId, RunningRecordStatus status);
26
27     @Query("SELECT r FROM RunningRecord r WHERE r.userId = ?1 AND r.status = ?2 " +
28         "AND r.distanceMeters >= ?3 AND r.avgPaceSecondsPerKm IS NOT NULL " +
29         "ORDER BY r.avgPaceSecondsPerKm ASC LIMIT 1")
30     Optional<RunningRecord> findFastestPace(UUID userId, RunningRecordStatus status, double minDistance);
31
32     Optional<RunningRecord> findTop1ByUserIdAndStatusAndCaloriesBurnedNotNullOrderByCaloriesBurnedDesc(
33         UUID userId, RunningRecordStatus status);
34
35     @Query("SELECT r FROM RunningRecord r WHERE r.userId = ?1 AND r.status = ?2 " +
36         "AND r.distanceMeters >= ?3 ORDER BY r.durationSeconds ASC LIMIT 1")
37     Optional<RunningRecord> findBestTimeForDistance(UUID userId, RunningRecordStatus status, double minDistance);
38
39     long countByUserIdAndStatus(UUID userId, RunningRecordStatus status);
40
41     @Query("SELECT COALESCE(SUM(r.distanceMeters), 0.0) FROM RunningRecord r " +
42         "WHERE r.userId = ?1 AND r.status = ?2")
43     double sumDistanceMetersByUserIdAndStatus(UUID userId, RunningRecordStatus status);
44
45     // — Stats 쿼리 (period 기반) —
46
47     @Query("SELECT r FROM RunningRecord r WHERE r.userId = :userId AND r.status = :status " +
48         "AND r.startedAt >= :from AND r.startedAt < :to")
49     List<RunningRecord> findByUserIdAndStatusAndPeriod(
50         @Param("userId") UUID userId,
51         @Param("status") RunningRecordStatus status,
52         @Param("from") Instant from,
53         @Param("to") Instant to);
54
55     // streak 계산용: 완료된 런 전체를 날짜 오름차순으로 조회
56     @Query("SELECT r FROM RunningRecord r WHERE r.userId = :userId AND r.status = :status " +
57         "ORDER BY r.startedAt ASC")
58     List<RunningRecord> findAllByUserIdAndStatusOrderByStartedAt(
59         @Param("userId") UUID userId,
60         @Param("status") RunningRecordStatus status);
61
62     // — Aggregate projection 쿼리 —
63
64     @Query(value = """
65         SELECT
66             COUNT(*)                                AS totalRuns,
67             COALESCE(SUM(distance_meters), 0)        AS totalDistanceMeters,
68             COALESCE(SUM(duration_seconds), 0)       AS totalDurationSeconds,
69             COALESCE(SUM(calories_burned), 0)        AS totalCaloriesBurned,
70             COALESCE(MAX(distance_meters), 0)        AS longestRunMeters,
71             COUNT(DISTINCT DATE(started_at AT TIME ZONE 'UTC')) AS activeDays
72         FROM running_records
73         WHERE user_id = :userId
74             AND status = :status
75             AND started_at >= :from
76             AND started_at < :to
77         """, nativeQuery = true)
78     RunningStatsProjection aggregateStatsByPeriod(
79         @Param("userId") UUID userId,
80         @Param("status") String status,
81         @Param("from") Instant from,
82         @Param("to") Instant to);
83

```

```

84     @Query(value = """
85         SELECT DISTINCT DATE(started_at AT TIME ZONE 'UTC') AS runDate
86         FROM running_records
87         WHERE user_id = :userId AND status = :status
88         ORDER BY runDate ASC
89         """, nativeQuery = true)
90     List<RunDateProjection> findDistinctRunDatesByUserId(
91         @Param("userId") UUID userId,
92         @Param("status") String status);
93
94     @Query("SELECT r.distanceMeters AS distanceMeters, r.startedAt AS startedAt " +
95         "FROM RunningRecord r WHERE r.userId = :userId AND r.status = :status " +
96         "ORDER BY r.startedAt ASC")
97     List<RunSummaryProjection> findRunSummariesByUserIdAndStatus(
98         @Param("userId") UUID userId,
99         @Param("status") RunningRecordStatus status);
100 }

```

backend/src/main/java/com/runway/run/repository/RunningStatsProjection.java

Java | 10줄

```

1  package com.runway.run.repository;
2
3  public interface RunningStatsProjection {
4      long getTotalRuns();
5      double getTotalDistanceMeters();
6      long getTotalDurationSeconds();
7      long getTotalCaloriesBurned();
8      double getLongestRunMeters();
9      long getActiveDays();
10 }

```



```

1  package com.runway.run.service;
2
3  import com.runway.common.exception.ErrorCode;
4  import com.runway.common.exception.RunwayException;
5  import com.runway.common.response.PageResponse;
6  import com.runway.run.domain.RunningPoint;
7  import com.runway.run.domain.RunningRecord;
8  import com.runway.run.domain.enums.RunningRecordStatus;
9  import com.runway.run.dto.*;
10 import com.runway.run.dto.PersonalRecordItemResponse;
11 import com.runway.run.dto.PersonalRecordsResponse;
12 import com.runway.run.repository.RunDateProjection;
13 import com.runway.run.repository.RunningPointBatchInserter;
14 import com.runway.run.repository.RunningPointRepository;
15 import com.runway.run.repository.RunningRecordRepository;
16 import com.runway.run.repository.RunningStatsProjection;
17 import lombok.RequiredArgsConstructor;
18 import lombok.extern.slf4j.Slf4j;
19 import org.locationtech.jts.geom.Coordinate;
20 import org.locationtech.jts.geom.GeometryFactory;
21 import org.locationtech.jts.geom.LineString;
22 import org.locationtech.jts.geom.PrecisionModel;
23 import org.springframework.data.domain.PageRequest;
24 import org.springframework.stereotype.Service;
25 import org.springframework.transaction.annotation.Transactional;
26
27 import java.time.Instant;
28 import java.time.LocalDate;
29 import java.time.ZoneOffset;
30 import java.time.temporal.ChronoUnit;
31 import java.time.temporal.TemporalAdjusters;
32 import java.time.DayOfWeek;
33 import java.util.ArrayList;
34 import java.util.HashSet;
35 import java.util.List;
36 import java.util.Set;
37 import java.util.UUID;
38
39 @Slf4j
40 @Service
41 @RequiredArgsConstructor
42 public class RunningService {
43
44     private static final GeometryFactory GEOMETRY_FACTORY = new GeometryFactory(new PrecisionModel(), 4326);
45
46     // Stale-record policy (Phase 2): running_records with status IN_PROGRESS or PAUSED
47     // older than 24 hours should be auto-abandoned by a scheduled task. This prevents
48     // orphaned records from client crashes or network loss. Implement in Phase 2 with
49     // @Scheduled(cron = "0 0 3 * * *") and batch UPDATE ... WHERE started_at < now() - interval '24 hours'.
50
51     private final RunningRecordRepository runningRecordRepository;
52     private final RunningPointRepository runningPointRepository;
53     private final RunningPointBatchInserter runningPointBatchInserter;
54
55     @Transactional
56     public StartRunResponse startRun(UUID userId, StartRunRequest request) {
57         Instant startedAt = (request.getStartedAt() != null) ? request.getStartedAt() : Instant.now();
58         RunningRecord record = RunningRecord.builder()
59             .userId(userId)
60             .startedAt(startedAt)
61             .build();
62         RunningRecord saved = runningRecordRepository.save(record);
63         log.info("Run started: runId={} userId={}", saved.getId(), userId);
64         return StartRunResponse.from(saved);
65     }
66
67     @Transactional
68     public SavePointsResponse savePoints(UUID userId, UUID runId, SavePointsRequest request) {
69         RunningRecord record = findOwnedRecord(runId, userId);
70
71         if (record.getStatus() != RunningRecordStatus.IN_PROGRESS
72             && record.getStatus() != RunningRecordStatus.PAUSED) {
73             throw new RunwayException(ErrorCode.INVALID_RUN_STATUS);
74         }
75
76         int count = request.getPoints().size();
77         runningPointBatchInserter.batchInsert(runId, request.getPoints());
78         log.info("Points saved (upsert): runId={} count={}", runId, count);
79         return new SavePointsResponse(runId, count);
80     }
81
82     @Transactional
83     public RunStatusResponse pauseRun(UUID userId, UUID runId) {

```

```

84     RunningRecord record = findOwnedRecord(runId, userId);
85     if (record.getStatus() != RunningRecordStatus.IN_PROGRESS) {
86         throw new RunwayException(ErrorCode.INVALID_RUN_STATUS);
87     }
88     record.pause();
89     return RunStatusResponse.from(record);
90 }
91
92 @Transactional
93 public RunStatusResponse resumeRun(UUID userId, UUID runId) {
94     RunningRecord record = findOwnedRecord(runId, userId);
95     if (record.getStatus() != RunningRecordStatus.PAUSED) {
96         throw new RunwayException(ErrorCode.INVALID_RUN_STATUS);
97     }
98     record.resume();
99     return RunStatusResponse.from(record);
100 }
101
102 @Transactional
103 public FinishRunResponse finishRun(UUID userId, UUID runId, FinishRunRequest request) {
104     RunningRecord record = findOwnedRecord(runId, userId);
105     if (record.getStatus() != RunningRecordStatus.IN_PROGRESS
106         && record.getStatus() != RunningRecordStatus.PAUSED) {
107         throw new RunwayException(ErrorCode.INVALID_RUN_STATUS);
108     }
109
110     if (!request.getEndedAt().isAfter(record.getStartedAt())) {
111         throw new RunwayException(ErrorCode.INVALID_REQUEST, "종료 시각은 시작 시각 이후여야 합니다.");
112     }
113
114     // 12 m/s (43.2 km/h) 초과는 인간 러닝으로 물리적으로 불가능한 속도
115     if (request.getDistanceMeters() != null && request.getDurationSeconds() != null) {
116         boolean zeroTime = request.getDurationSeconds() == 0;
117         boolean hasDistance = request.getDistanceMeters() > 0;
118         boolean tooFast = !zeroTime && (request.getDistanceMeters() / request.getDurationSeconds() > 12.0);
119         if ((zeroTime && hasDistance) || tooFast) {
120             throw new RunwayException(ErrorCode.IMPOSSIBLE_SPEED);
121         }
122     }
123
124     // running_points로 LineString 생성 - 2개 이상 포인트가 있어야 함
125     List<RunningPoint> points = runningPointRepository.findByRunningRecordIdOrderBySequenceAsc(runId);
126     boolean pathCreated = false;
127     if (points.size() >= 2) {
128         Coordinate[] coords = points.stream()
129             .map(p -> p.getLocation().getCoordinate())
130             .toArray(Coordinate[]::new);
131         LineString path = GEOMETRY_FACTORY.createLineString(coords);
132         record.updatePath(path);
133         pathCreated = true;
134     }
135
136     record.finish(
137         request.getEndedAt(),
138         request.getDistanceMeters(),
139         request.getDurationSeconds(),
140         request.getAvgPaceSecondsPerKm(),
141         request.getCaloriesBurned(),
142         request.getAvgHeartRateBpm()
143     );
144
145     log.info("Run finished: runId={} pathCreated={}", runId, pathCreated);
146     return FinishRunResponse.from(record, pathCreated);
147 }
148
149 @Transactional
150 public RunStatusResponse abandonRun(UUID userId, UUID runId) {
151     RunningRecord record = findOwnedRecord(runId, userId);
152     if (record.getStatus() != RunningRecordStatus.IN_PROGRESS
153         && record.getStatus() != RunningRecordStatus.PAUSED) {
154         throw new RunwayException(ErrorCode.INVALID_RUN_STATUS);
155     }
156     record.abandon();
157     log.info("Run abandoned: runId={}", runId);
158     return RunStatusResponse.from(record);
159 }
160
161 @Transactional(readOnly = true)
162 public PageResponse<RunSummaryResponse> getMyRuns(UUID userId, int page, int size) {
163     return PageResponse.from(
164         runningRecordRepository
165             .findByUserIdOrderByStartedAtDesc(userId, PageRequest.of(page, size))
166             .map(RunSummaryResponse::from)
167     );
168 }
169

```

```

170     @Transactional(readOnly = true)
171     public RunDetailResponse getRunDetail(UUID userId, UUID runId) {
172         RunningRecord record = findOwnedRecord(runId, userId);
173         List<RunningPoint> points = runningPointRepository.findByRunningRecordIdOrderBySequenceAsc(runId);
174         return RunDetailResponse.from(record, points);
175     }
176
177     @Transactional(readOnly = true)
178     public PersonalRecordsResponse getPersonalRecords(UUID userId) {
179         RunningRecordStatus completed = RunningRecordStatus.COMPLETED;
180
181         PersonalRecordItemResponse longestRun = runningRecordRepository
182             .findTop1ByUserIdAndStatusOrderByDistanceMetersDesc(userId, completed)
183             .map(PersonalRecordItemResponse::from).orElse(null);
184
185         PersonalRecordItemResponse fastestPace = runningRecordRepository
186             .findFastestPace(userId, completed, 1000.0)
187             .map(PersonalRecordItemResponse::from).orElse(null);
188
189         PersonalRecordItemResponse mostCalories = runningRecordRepository
190             .findTop1ByUserIdAndStatusAndCaloriesBurnedNotNullOrderByCaloriesBurnedDesc(userId, completed)
191             .map(PersonalRecordItemResponse::from).orElse(null);
192
193         PersonalRecordItemResponse best5k = runningRecordRepository
194             .findBestTimeForDistance(userId, completed, 5000.0)
195             .map(PersonalRecordItemResponse::from).orElse(null);
196
197         PersonalRecordItemResponse best10k = runningRecordRepository
198             .findBestTimeForDistance(userId, completed, 10000.0)
199             .map(PersonalRecordItemResponse::from).orElse(null);
200
201         long totalCompletedRuns = runningRecordRepository.countByUserIdAndStatus(userId, completed);
202         double totalDistanceMeters = runningRecordRepository.sumDistanceMetersByUserIdAndStatus(userId, completed);
203
204         return PersonalRecordsResponse.builder()
205             .longestRun(longestRun)
206             .fastestPace(fastestPace)
207             .mostCalories(mostCalories)
208             .best5k(best5k)
209             .best10k(best10k)
210             .totalCompletedRuns(totalCompletedRuns)
211             .totalDistanceMeters(totalDistanceMeters)
212             .build();
213     }
214
215     @Transactional(readOnly = true)
216     public RunningStatsResponse getRunningStats(UUID userId, String period) {
217         Instant now = Instant.now();
218         LocalDate today = LocalDate.now(ZoneOffset.UTC);
219
220         Instant periodStart;
221         Instant periodEnd = today.plusDays(1).atStartOfDay(ZoneOffset.UTC).toInstant();
222
223         switch (period.toLowerCase()) {
224             case "weekly" -> {
225                 LocalDate monday = today.with(TemporalAdjusters.previousOrSame(DayOfWeek.MONDAY));
226                 periodStart = monday.atStartOfDay(ZoneOffset.UTC).toInstant();
227             }
228             case "yearly" -> {
229                 LocalDate jan1 = today.with(TemporalAdjusters.firstDayOfYear());
230                 periodStart = jan1.atStartOfDay(ZoneOffset.UTC).toInstant();
231             }
232             case "all" -> {
233                 periodStart = Instant.EPOCH;
234                 periodEnd = now;
235             }
236             default -> { // monthly
237                 LocalDate first = today.with(TemporalAdjusters.firstDayOfMonth());
238                 periodStart = first.atStartOfDay(ZoneOffset.UTC).toInstant();
239             }
240         }
241
242         RunningRecordStatus completed = RunningRecordStatus.COMPLETED;
243
244         // DB aggregate projection으로 기간 내 통계 조회
245         RunningStatsProjection stats = runningRecordRepository
246             .aggregateStatsByPeriod(userId, completed.getDbValue(), periodStart, periodEnd);
247
248         long totalRuns = stats.getTotalRuns();
249         double totalDistanceMeters = stats.getTotalDistanceMeters();
250         long totalDurationSeconds = stats.getTotalDurationSeconds();
251         long totalCaloriesBurned = stats.getTotalCaloriesBurned();
252         double longestRunMeters = stats.getLongestRunMeters();
253         long activeDays = stats.getActiveDays();
254
255         int avgPace = (totalDistanceMeters > 1.0) ? (int) (totalDurationSeconds / (totalDistanceMeters / 1000.0)) : 0;

```

```

256
257 // streak 계산 - 전체 완료 런 기준 (기간 무관)
258 List<LocalDate> runDates = runningRecordRepository
259     .findDistinctRunDatesByUserId(userId, completed.getDbValue())
260     .stream()
261     .map(RunDateProjection::getRunDate)
262     .collect(java.util.stream.Collectors.toList());
263 int[] streaks = calculateStreaksFromDates(runDates, today);
264
265 return RunningStatsResponse.builder()
266     .period(period.toLowerCase())
267     .totalRuns(totalRuns)
268     .totalDistanceMeters(totalDistanceMeters)
269     .totalDurationSeconds(totalDurationSeconds)
270     .totalCaloriesBurned(totalCaloriesBurned)
271     .averagePaceSecondsPerKm(avgPace)
272     .longestRunMeters(longestRunMeters)
273     .currentStreakDays(streaks[0])
274     .longestStreakDays(streaks[1])
275     .activeDays(activeDays)
276     .periodStart(periodStart)
277     .periodEnd(periodEnd.equals(now) ? periodEnd : periodEnd.minus(1, ChronoUnit.SECONDS))
278     .build();
279 }
280
281 // [0] = currentStreak, [1] = longestStreak
282 private int[] calculateStreaksFromDates(List<LocalDate> runDates, LocalDate today) {
283     if (runDates.isEmpty()) return new int[]{0, 0};
284
285     Set<LocalDate> runDateSet = new HashSet<>(runDates);
286
287     // 현재 streak: 오늘 또는 어제부터 연속일 수
288     int current = 0;
289     LocalDate check = today;
290     if (!runDateSet.contains(check)) {
291         check = today.minusDays(1);
292     }
293     while (runDateSet.contains(check)) {
294         current++;
295         check = check.minusDays(1);
296     }
297
298     // 최장 streak: 날짜 정렬 후 연속일 카운트
299     List<LocalDate> sorted = new ArrayList<>(runDateSet);
300     sorted.sort(null);
301     int longest = 1, cur = 1;
302     for (int i = 1; i < sorted.size(); i++) {
303         if (sorted.get(i).equals(sorted.get(i - 1).plusDays(1))) {
304             cur++;
305             if (cur > longest) longest = cur;
306         } else {
307             cur = 1;
308         }
309     }
310
311     return new int[]{current, longest};
312 }
313
314 @Transactional
315 public void deleteRun(UUID userId, UUID runId) {
316     RunningRecord record = findOwnedRecord(runId, userId);
317     runningRecordRepository.delete(record);
318     log.info("Run deleted: runId={}", runId);
319 }
320
321 @Transactional
322 public void trimRun(UUID userId, UUID runId, double targetDistanceMeters) {
323     RunningRecord record = findOwnedRecord(runId, userId);
324
325     if (record.getDistanceMeters() == null || record.getDistanceMeters() <= 0) {
326         throw new RunwayException(ErrorCode.INVALID_REQUEST, "기록된 거리가 없습니다.");
327     }
328     if (targetDistanceMeters <= 0 || targetDistanceMeters >= record.getDistanceMeters()) {
329         throw new RunwayException(ErrorCode.INVALID_REQUEST,
330             "수정 거리는 0보다 크고 현재 기록(" +
331             String.format("%.1f", record.getDistanceMeters() / 1000.0) + "km)보다 작아야 합니다.");
332     }
333
334     // 포인트 조회 후 커트 지점 계산
335     List<RunningPoint> points = runningPointRepository
336         .findByRunningRecordIdOrderBySequenceAsc(runId);
337
338     double cumulative = 0.0;
339     int cutoffSeq = -1;
340     for (int i = 1; i < points.size(); i++) {
341         RunningPoint prev = points.get(i - 1);

```

```

342         RunningPoint curr = points.get(i);
343         cumulative += haversineMeters(
344             prev.getLocation().getY(), prev.getLocation().getX(),
345             curr.getLocation().getY(), curr.getLocation().getX()
346         );
347         if (cumulative >= targetDistanceMeters) {
348             cutoffSeq = curr.getSequence();
349             break;
350         }
351     }
352
353     if (cutoffSeq > 0) {
354         runningPointRepository.deleteByRunningRecordIdAndSequenceGreaterThan(runId, cutoffSeq);
355     }
356
357     record.trim(targetDistanceMeters);
358     log.info("Run trimmed: runId={}, targetDistanceMeters={}", runId, targetDistanceMeters);
359 }
360
361 private static double haversineMeters(double lat1, double lon1, double lat2, double lon2) {
362     final double R = 6_371_000.0;
363     double phi1 = Math.toRadians(lat1);
364     double phi2 = Math.toRadians(lat2);
365     double dPhi = Math.toRadians(lat2 - lat1);
366     double dLambda = Math.toRadians(lon2 - lon1);
367     double a = Math.sin(dPhi / 2) * Math.sin(dPhi / 2)
368         + Math.cos(phi1) * Math.cos(phi2) * Math.sin(dLambda / 2) * Math.sin(dLambda / 2);
369     return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
370 }
371
372 private RunningRecord findOwnedRecord(UUID runId, UUID userId) {
373     return runningRecordRepository.findByIdAndUserId(runId, userId)
374         .orElseThrow(() -> new RunwayException(ErrorCode.RUN_NOT_FOUND));
375 }
376 }

```

```

1  package com.runway.user.controller;
2
3  import com.runway.common.response.ApiResponse;
4  import com.runway.common.security.UserPrincipal;
5  import com.runway.user.dto.UpdateProfileRequest;
6  import com.runway.user.dto.UserProfileResponse;
7  import com.runway.user.service.UserService;
8  import io.swagger.v3.oas.annotations.Operation;
9  import io.swagger.v3.oas.annotations.security.SecurityRequirement;
10 import io.swagger.v3.oas.annotations.tags.Tag;
11 import jakarta.validation.Valid;
12 import lombok.RequiredArgsConstructor;
13 import org.springframework.http.MediaType;
14 import org.springframework.http.ResponseEntity;
15 import org.springframework.security.core.annotation.AuthenticationPrincipal;
16 import org.springframework.web.bind.annotation.*;
17 import org.springframework.web.multipart.MultipartFile;
18
19 @Tag(name = "User", description = "사용자 프로필 API")
20 @RestController
21 @RequestMapping("/api/users")
22 @RequiredArgsConstructor
23 public class UserController {
24
25     private final UserService userService;
26
27     @Operation(summary = "내 프로필 조회", security = @SecurityRequirement(name = "bearerAuth"))
28     @GetMapping("/me")
29     public ResponseEntity<ApiResponse<UserProfileResponse>> getProfile(
30         @AuthenticationPrincipal UserPrincipal principal) {
31         UserProfileResponse data = userService.getProfile(principal.getUserId());
32         return ResponseEntity.ok(ApiResponse.success("내 프로필 조회가 완료되었습니다.", data));
33     }
34
35     @Operation(summary = "내 프로필 수정", security = @SecurityRequirement(name = "bearerAuth"))
36     @PutMapping("/me")
37     public ResponseEntity<ApiResponse<UserProfileResponse>> updateProfile(
38         @AuthenticationPrincipal UserPrincipal principal,
39         @Valid @RequestBody UpdateProfileRequest request) {
40         UserProfileResponse data = userService.updateProfile(principal.getUserId(), request);
41         return ResponseEntity.ok(ApiResponse.success("내 프로필 수정이 완료되었습니다.", data));
42     }
43
44     @Operation(summary = "프로필 이미지 업로드", security = @SecurityRequirement(name = "bearerAuth"))
45     @PostMapping(value = "/me/profile-image", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
46     public ResponseEntity<ApiResponse<String>> uploadProfileImage(
47         @AuthenticationPrincipal UserPrincipal principal,
48         @RequestParam("image") MultipartFile image) {
49         String imageUrl = userService.uploadProfileImage(principal.getUserId(), image);
50         return ResponseEntity.ok(ApiResponse.success("프로필 이미지가 업로드되었습니다.", imageUrl));
51     }
52
53     @Operation(summary = "회원 탈퇴", security = @SecurityRequirement(name = "bearerAuth"))
54     @DeleteMapping("/me")
55     public ResponseEntity<ApiResponse<Void>> deleteUser(
56         @AuthenticationPrincipal UserPrincipal principal) {
57         userService.deleteUser(principal.getUserId());
58         return ResponseEntity.ok(ApiResponse.success("회원 탈퇴가 완료되었습니다."));
59     }
60
61     @Operation(summary = "내 업적 조회", security = @SecurityRequirement(name = "bearerAuth"))
62     @GetMapping("/me/achievements")
63     public ResponseEntity<ApiResponse<com.runway.user.dto.AchievementsResponse>> getAchievements(
64         @AuthenticationPrincipal UserPrincipal principal) {
65         var data = userService.getAchievements(principal.getUserId());
66         return ResponseEntity.ok(ApiResponse.success("업적 조회가 완료되었습니다.", data));
67     }
68 }

```

```
1 package com.runway.user.domain;
2
3 import com.fasterxml.jackson.annotation.JsonIgnore;
4 import jakarta.persistence.*;
5 import lombok.AccessLevel;
6 import lombok.Builder;
7 import lombok.Getter;
8 import lombok.NoArgsConstructor;
9
10 import java.time.Instant;
11 import java.util.UUID;
12
13 @Entity
14 @Table(name = "users")
15 @Getter
16 @NoArgsConstructor(access = AccessLevel.PROTECTED)
17 public class User {
18
19     @Id
20     @GeneratedValue(strategy = GenerationType.UUID)
21     @Column(columnDefinition = "uuid", updatable = false, nullable = false)
22     private UUID id;
23
24     @Column(nullable = false, unique = true, length = 255)
25     private String email;
26
27     @JsonIgnore
28     @Column(name = "password_hash", nullable = false, length = 255)
29     private String passwordHash;
30
31     @Column(nullable = false, unique = true, length = 50)
32     private String nickname;
33
34     @Column(name = "profile_image_url", columnDefinition = "TEXT")
35     private String profileImageUrl;
36
37     @Column(columnDefinition = "TEXT")
38     private String bio;
39
40     @JsonIgnore
41     @Column(name = "refresh_token_hash", length = 512)
42     private String refreshTokenHash;
43
44     @Column(name = "created_at", nullable = false, updatable = false)
45     private Instant createdAt;
46
47     @Column(name = "updated_at", nullable = false)
48     private Instant updatedAt;
49
50     @Column(name = "deleted_at")
51     private Instant deletedAt;
52
53     @Builder
54     private User(String email, String passwordHash, String nickname,
55                 String profileImageUrl, String bio) {
56         this.email = email;
57         this.passwordHash = passwordHash;
58         this.nickname = nickname;
59         this.profileImageUrl = profileImageUrl;
60         this.bio = bio;
61     }
62
63     @PrePersist
64     private void prePersist() {
65         Instant now = Instant.now();
66         this.createdAt = now;
67         this.updatedAt = now;
68     }
69
70     @PreUpdate
71     private void preUpdate() {
72         this.updatedAt = Instant.now();
73     }
74
75     public void updateProfile(String nickname, String profileImageUrl, String bio) {
76         this.nickname = nickname;
77         this.profileImageUrl = profileImageUrl;
78         this.bio = bio;
79     }
80
81     public void updateRefreshTokenHash(String refreshTokenHash) {
82         this.refreshTokenHash = refreshTokenHash;
83     }
84 }
```

```

84
85     public void softDelete() {
86         this.deletedAt = Instant.now();
87     }
88
89     public boolean isDeleted() {
90         return this.deletedAt != null;
91     }
92 }

```

backend/src/main/java/com/runway/user/dto/AchievementItemResponse.java

Java | 19줄

```

1  package com.runway.user.dto;
2
3  import lombok.Builder;
4  import lombok.Getter;
5
6  import java.time.Instant;
7
8  @Getter
9  @Builder
10 public class AchievementItemResponse {
11
12     private String code;
13     private String title;
14     private String description;
15     private boolean unlocked;
16     private Instant unlockedAt;
17     private long progress;
18     private long target;
19 }

```

backend/src/main/java/com/runway/user/dto/AchievementsResponse.java

Java | 13줄

```

1  package com.runway.user.dto;
2
3  import lombok.Builder;
4  import lombok.Getter;
5
6  import java.util.List;
7
8  @Getter
9  @Builder
10 public class AchievementsResponse {
11
12     private List<AchievementItemResponse> items;
13 }

```

backend/src/main/java/com/runway/user/dto/UpdateProfileRequest.java

Java | 16줄

```

1  package com.runway.user.dto;
2
3  import jakarta.validation.constraints.Size;
4  import lombok.Getter;
5
6  @Getter
7  public class UpdateProfileRequest {
8
9     @Size(min = 2, max = 50, message = "닉네임은 2자 이상 50자 이하여야 합니다.")
10     private String nickname;
11
12     private String profileImageUrl;
13
14     @Size(max = 500, message = "자기소개는 500자 이하여야 합니다.")
15     private String bio;
16 }

```


backend/src/main/java/com/runway/user/dto/UserProfileResponse.java

Java | 31줄

```
1 package com.runway.user.dto;
2
3 import com.runway.user.domain.User;
4 import lombok.Builder;
5 import lombok.Getter;
6
7 import java.time.Instant;
8 import java.util.UUID;
9
10 @Getter
11 @Builder
12 public class UserProfileResponse {
13
14     private UUID userId;
15     private String email;
16     private String nickname;
17     private String profileImageUrl;
18     private String bio;
19     private Instant createdAt;
20
21     public static UserProfileResponse from(User user) {
22         return UserProfileResponse.builder()
23             .userId(user.getId())
24             .email(user.getEmail())
25             .nickname(user.getNickname())
26             .profileImageUrl(user.getProfileImageUrl())
27             .bio(user.getBio())
28             .createdAt(user.getCreatedAt())
29             .build();
30     }
31 }
```

backend/src/main/java/com/runway/user/repository/UserRepository.java

Java | 20줄

```
1 package com.runway.user.repository;
2
3 import com.runway.user.domain.User;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 import java.util.Optional;
7 import java.util.UUID;
8
9 public interface UserRepository extends JpaRepository<User, UUID> {
10
11     Optional<User> findByEmailAndDeletedAtIsNull(String email);
12
13     Optional<User> findByIdAndDeletedAtIsNull(UUID id);
14
15     boolean existsByEmailAndDeletedAtIsNull(String email);
16
17     boolean existsByNicknameAndDeletedAtIsNull(String nickname);
18
19     boolean existsByNicknameAndIdNotAndDeletedAtIsNull(String nickname, UUID id);
20 }
```

```

1  package com.runway.user.service;
2
3  import com.runway.attempt.domain.enums.CourseAttemptStatus;
4  import com.runway.attempt.repository.CourseAttemptRepository;
5  import com.runway.common.exception.ErrorCode;
6  import com.runway.common.exception.RunwayException;
7  import com.runway.course.repository.CourseRepository;
8  import com.runway.run.domain.enums.RunningRecordStatus;
9  import com.runway.run.repository.RunningRecordRepository;
10 import com.runway.run.repository.RunSummaryProjection;
11 import com.runway.user.domain.User;
12 import com.runway.user.dto.AchievementItemResponse;
13 import com.runway.user.dto.AchievementsResponse;
14 import com.runway.user.dto.UpdateProfileRequest;
15 import com.runway.user.dto.UserProfileResponse;
16 import com.runway.user.repository.UserRepository;
17 import lombok.RequiredArgsConstructor;
18 import lombok.extern.slf4j.Slf4j;
19 import org.springframework.stereotype.Service;
20 import org.springframework.beans.factory.annotation.Value;
21 import org.springframework.transaction.annotation.Transactional;
22 import org.springframework.web.multipart.MultipartFile;
23
24 import java.io.IOException;
25 import java.nio.file.Files;
26 import java.nio.file.Path;
27 import java.nio.file.Paths;
28 import java.nio.file.StandardCopyOption;
29 import java.time.Instant;
30 import java.time.LocalDate;
31 import java.time.ZoneOffset;
32 import java.util.ArrayList;
33 import java.util.HashSet;
34 import java.util.List;
35 import java.util.Set;
36 import java.util.UUID;
37
38 @Slf4j
39 @Service
40 @RequiredArgsConstructor
41 public class UserService {
42
43     @Value("${app.upload-dir:/opt/runway/uploads}")
44     private String uploadDir;
45
46     @Value("${app.base-url:https://pathfinder.run}")
47     private String baseUrl;
48
49     private final UserRepository userRepository;
50     private final RunningRecordRepository runningRecordRepository;
51     private final CourseAttemptRepository courseAttemptRepository;
52     private final CourseRepository courseRepository;
53
54     @Transactional(readOnly = true)
55     public UserProfileResponse getProfile(UUID userId) {
56         User user = findActiveUser(userId);
57         return UserProfileResponse.from(user);
58     }
59
60     public String uploadProfileImage(UUID userId, MultipartFile file) {
61         if (file.isEmpty()) throw new RunwayException(ErrorCode.INVALID_REQUEST, "파일이 비어 있습니다.");
62         String contentType = file.getContentType();
63         if (contentType == null || !contentType.startsWith("image/")) {
64             throw new RunwayException(ErrorCode.INVALID_REQUEST, "이미지 파일만 업로드 가능합니다.");
65         }
66         String ext = contentType.contains("png") ? ".png" : ".jpg";
67         String filename = "profile_" + userId + ext;
68         try {
69             Path dir = Paths.get(uploadDir, "profiles");
70             Files.createDirectories(dir);
71             Files.copy(file.getInputStream(), dir.resolve(filename), StandardCopyOption.REPLACE_EXISTING);
72         } catch (IOException e) {
73             throw new RunwayException(ErrorCode.INVALID_REQUEST, "이미지 저장에 실패했습니다.");
74         }
75         return baseUrl + "/uploads/profiles/" + filename;
76     }
77
78     @Transactional
79     public UserProfileResponse updateProfile(UUID userId, UpdateProfileRequest request) {
80         User user = findActiveUser(userId);
81
82         String newNickname = request.getNickname() != null ? request.getNickname() : user.getNickname();
83         String newProfileImageUrl = request.getProfileImageUrl() != null ? request.getProfileImageUrl() : user.getProfileImageUrl();

```

```

84     String newBio = request.getBio() != null ? request.getBio() : user.getBio();
85
86     // 닉네임이 변경된 경우에만 중복 확인
87     if (request.getNickname() != null && !request.getNickname().equals(user.getNickname())) {
88         if (userRepository.existsByNicknameAndIdNotAndDeletedAtIsNull(request.getNickname(), userId)) {
89             throw new RunwayException(ErrorCode.DUPLICATED_NICKNAME);
90         }
91     }
92
93     user.updateProfile(newNickname, newProfileImageUrl, newBio);
94     log.info("User profile updated: {}", user.getEmail());
95     return UserProfileResponse.from(user);
96 }
97
98 @Transactional
99 public void deleteUser(UUID userId) {
100     User user = findActiveUser(userId);
101     user.softDelete();
102     user.updateRefreshTokenHash(null);
103     log.info("User soft deleted: {}", user.getEmail());
104 }
105
106 @Transactional(readOnly = true)
107 public AchievementsResponse getAchievements(UUID userId) {
108     RunningRecordStatus completed = RunningRecordStatus.COMPLETED;
109
110     // DB aggregate 쿼리로 기본 집계
111     long completedRuns = runningRecordRepository.countByUserIdAndStatus(userId, completed);
112     double totalDistanceMeters = runningRecordRepository.sumDistanceMetersByUserIdAndStatus(userId, completed);
113
114     long completedAttempts = courseAttemptRepository.countByUserIdAndStatus(userId, CourseAttemptStatus.COMPLETED);
115     long createdCourses = courseRepository.countByCreatorIdAndDeletedAtIsNull(userId);
116
117     // 경량 summary projection으로 streak 및 unlock 날짜 계산
118     List<RunSummaryProjection> runSummaries = runningRecordRepository
119         .findRunSummariesByUserIdAndStatus(userId, completed);
120
121     // streak 계산
122     int longestStreak = computeLongestStreak(runSummaries);
123
124     List<AchievementItemResponse> items = new ArrayList<>();
125
126     // FIRST_RUN
127     Instant firstRunAt = runSummaries.isEmpty() ? null : runSummaries.get(0).getStartedAt();
128     items.add(AchievementItemResponse.builder()
129         .code("FIRST_RUN")
130         .title("첫 러닝")
131         .description("첫 러닝을 완료했습니다.")
132         .unlocked(completedRuns >= 1)
133         .unlockedAt(completedRuns >= 1 ? firstRunAt : null)
134         .progress(Math.min(completedRuns, 1))
135         .target(1)
136         .build());
137
138     // TOTAL_10K / 50K / 100K
139     items.add(buildDistanceAchievement("TOTAL_10K", "누적 10km", "총 10km를 달성했습니다.",
140         10_000, totalDistanceMeters, runSummaries));
141     items.add(buildDistanceAchievement("TOTAL_50K", "누적 50km", "총 50km를 달성했습니다.",
142         50_000, totalDistanceMeters, runSummaries));
143     items.add(buildDistanceAchievement("TOTAL_100K", "누적 100km", "총 100km를 달성했습니다.",
144         100_000, totalDistanceMeters, runSummaries));
145
146     // STREAK_3 / STREAK_7
147     items.add(buildStreakAchievement("STREAK_3", "3일 연속 러닝", "3일 연속으로 러닝을 완료했습니다.",
148         3, longestStreak));
149     items.add(buildStreakAchievement("STREAK_7", "7일 연속 러닝", "7일 연속으로 러닝을 완료했습니다.",
150         7, longestStreak));
151
152     // FIRST_COURSE_CREATED
153     Instant firstCourseAt = courseRepository.findFirstByCreatorIdAndDeletedAtIsNullOrderByCreatedAtAsc(userId)
154         .map(c -> c.getCreatedAt()).orElse(null);
155     items.add(AchievementItemResponse.builder()
156         .code("FIRST_COURSE_CREATED")
157         .title("첫 코스 등록")
158         .description("첫 번째 코스를 등록했습니다.")
159         .unlocked(createdCourses >= 1)
160         .unlockedAt(createdCourses >= 1 ? firstCourseAt : null)
161         .progress(Math.min(createdCourses, 1))
162         .target(1)
163         .build());
164
165     // FIRST_COURSE_COMPLETED
166     Instant firstAttemptAt = courseAttemptRepository
167         .findFirstByUserIdAndStatusOrderByCompletedAtAsc(userId, CourseAttemptStatus.COMPLETED)
168         .map(a -> a.getCompletedAt()).orElse(null);
169     items.add(AchievementItemResponse.builder()

```

```

170         .code("FIRST_COURSE_COMPLETED")
171         .title("첫 코스 완주")
172         .description("첫 번째 코스를 완주했습니다.")
173         .unlocked(completedAttempts >= 1)
174         .unlockedAt(completedAttempts >= 1 ? firstAttemptAt : null)
175         .progress(Math.min(completedAttempts, 1))
176         .target(1)
177         .build();
178
179     return AchievementsResponse.builder().items(items).build();
180 }
181
182 private AchievementItemResponse buildDistanceAchievement(
183     String code, String title, String description,
184     long targetMeters, double totalDistance, List<RunSummaryProjection> runs) {
185
186     boolean unlocked = totalDistance >= targetMeters;
187     Instant unlockedAt = null;
188     if (unlocked) {
189         // 누적 거리가 threshold를 넘은 시점의 런 날짜를 계산
190         double acc = 0;
191         for (RunSummaryProjection r : runs) {
192             acc += r.getDistanceMeters() != null ? r.getDistanceMeters() : 0.0;
193             if (acc >= targetMeters) {
194                 unlockedAt = r.getStartedAt();
195                 break;
196             }
197         }
198     }
199     return AchievementItemResponse.builder()
200         .code(code)
201         .title(title)
202         .description(description)
203         .unlocked(unlocked)
204         .unlockedAt(unlockedAt)
205         .progress((long) Math.min(totalDistance, targetMeters))
206         .target(targetMeters)
207         .build();
208 }
209
210 private int computeLongestStreak(List<RunSummaryProjection> runs) {
211     if (runs.isEmpty()) return 0;
212     Set<LocalDate> dates = new HashSet<>();
213     for (RunSummaryProjection r : runs) {
214         dates.add(r.getStartedAt().atZone(ZoneOffset.UTC).toLocalDate());
215     }
216     List<LocalDate> sorted = new ArrayList<>(dates);
217     sorted.sort(null);
218     int longest = 1, cur = 1;
219     for (int i = 1; i < sorted.size(); i++) {
220         if (sorted.get(i).equals(sorted.get(i - 1).plusDays(1))) {
221             cur++;
222             if (cur > longest) longest = cur;
223         } else {
224             cur = 1;
225         }
226     }
227     return longest;
228 }
229
230 private AchievementItemResponse buildStreakAchievement(
231     String code, String title, String description, int target, int longestStreak) {
232     return AchievementItemResponse.builder()
233         .code(code)
234         .title(title)
235         .description(description)
236         .unlocked(longestStreak >= target)
237         .unlockedAt(null)
238         .progress(Math.min(longestStreak, target))
239         .target(target)
240         .build();
241 }
242
243 private User findActiveUser(UUID userId) {
244     return userRepository.findByIdAndDeletedAtIsNull(userId)
245         .orElseThrow(() -> new RunwayException(ErrorCode.USER_NOT_FOUND));
246 }
247 }

```

backend/src/main/resources/db/migration/V10__create_course_favorites.sql

SQL | 14줄

```
1 CREATE TABLE course_favorites
2 (
3     user_id    UUID          NOT NULL,
4     course_id  UUID          NOT NULL,
5     created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
6
7     CONSTRAINT pk_course_favorites PRIMARY KEY (user_id, course_id),
8     CONSTRAINT fk_course_favorites_user
9         FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE,
10    CONSTRAINT fk_course_favorites_course
11        FOREIGN KEY (course_id) REFERENCES courses (id) ON DELETE CASCADE
12 );
13
14 CREATE INDEX idx_course_favorites_user_id ON course_favorites (user_id, created_at DESC);
```

backend/src/main/resources/db/migration/V11__add_course_publishing_metadata.sql

SQL | 19줄

```
1 ALTER TABLE courses
2     ADD COLUMN difficulty    VARCHAR(20),
3     ADD COLUMN slope_level   VARCHAR(20),
4     ADD COLUMN risk_level    VARCHAR(20),
5     ADD COLUMN surface_type  VARCHAR(20),
6     ADD COLUMN recommended_time VARCHAR(20),
7     ADD COLUMN warnings      TEXT;
8
9 ALTER TABLE courses
10     ADD CONSTRAINT chk_course_difficulty
11         CHECK (difficulty IS NULL OR difficulty IN ('easy', 'normal', 'hard')),
12     ADD CONSTRAINT chk_course_slope_level
13         CHECK (slope_level IS NULL OR slope_level IN ('flat', 'moderate', 'steep')),
14     ADD CONSTRAINT chk_course_risk_level
15         CHECK (risk_level IS NULL OR risk_level IN ('low', 'medium', 'high')),
16     ADD CONSTRAINT chk_course_surface_type
17         CHECK (surface_type IS NULL OR surface_type IN ('road', 'park', 'trail', 'mixed')),
18     ADD CONSTRAINT chk_course_recommended_time
19         CHECK (recommended_time IS NULL OR recommended_time IN ('morning', 'day', 'night', 'any'));
```

```

1  -- V12: 전국 유명 러닝 코스 시드 데이터 (33개)
2  -- 시스템 계정 소유, status='published'
3
4  INSERT INTO users (id, email, password_hash, nickname, bio)
5  VALUES ('00000000-0000-0000-0000-000000000001', 'system@runway.app',
6          '$2a$10$SEED_PLACEHOLDER_HASH_NOT_FOR_LOGIN_00000000000000000000',
7          'RunWay 공식', '전국 유명 러닝 코스를 엄선해 제공합니다.')
8  ON CONFLICT (id) DO NOTHING;
9
10 -- — 헬퍼: 좌표 → geography point
11 -- ST_MakePoint(경도, 위도)
12
13 -- =====
14 -- 서울 (15개)
15 -- =====
16
17 -- 1. 여의도 한강공원 순환
18 INSERT INTO courses VALUES ('c0000001-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
19 '여의도 한강공원 순환', '여의도 한강공원을 한 바퀴 도는 대표 도심 러닝 코스. 강변 뷰와 넓은 잔디밭이 매력적이며 봄 벚꽃 시즌에 특히 인기입니다. 완전 평탄 노면으로 초보자에게 추
20 'published', 6320, TRUE,
21 ST_SetSRID(ST_MakePoint(126.9230, 37.5265), 4326)::geography,
22 ST_SetSRID(ST_MakePoint(126.9230, 37.5265), 4326)::geography,
23 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.9230, 37.5265), ST_MakePoint(126.9310, 37.5275), ST_MakePoint(126.9400, 37.5282), ST_MakePoint
(126.9490, 37.5285), ST_MakePoint(126.9580, 37.5278), ST_MakePoint(126.9640, 37.5265), ST_MakePoint(126.9580, 37.5250), ST_MakePoint(126.9490, 37.524
4), ST_MakePoint(126.9400, 37.5247), ST_MakePoint(126.9310, 37.5255), ST_MakePoint(126.9230, 37.5265)]), 4326)::geography,
24 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
25
26 INSERT INTO course_points(id, course_id, sequence, location) VALUES
27 (gen_random_uuid(), 'c0000001-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(126.9230, 37.5265), 4326)::geography),
28 (gen_random_uuid(), 'c0000001-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(126.9310, 37.5275), 4326)::geography),
29 (gen_random_uuid(), 'c0000001-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(126.9400, 37.5282), 4326)::geography),
30 (gen_random_uuid(), 'c0000001-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(126.9490, 37.5285), 4326)::geography),
31 (gen_random_uuid(), 'c0000001-0000-0000-0000-000000000000', 5, ST_SetSRID(ST_MakePoint(126.9580, 37.5278), 4326)::geography),
32 (gen_random_uuid(), 'c0000001-0000-0000-0000-000000000000', 6, ST_SetSRID(ST_MakePoint(126.9640, 37.5265), 4326)::geography),
33 (gen_random_uuid(), 'c0000001-0000-0000-0000-000000000000', 7, ST_SetSRID(ST_MakePoint(126.9490, 37.5244), 4326)::geography),
34 (gen_random_uuid(), 'c0000001-0000-0000-0000-000000000000', 8, ST_SetSRID(ST_MakePoint(126.9310, 37.5255), 4326)::geography),
35 (gen_random_uuid(), 'c0000001-0000-0000-0000-000000000000', 9, ST_SetSRID(ST_MakePoint(126.9230, 37.5265), 4326)::geography)
36 ON CONFLICT DO NOTHING;
37
38 -- 2. 남산 순환
39 INSERT INTO courses VALUES ('c0000002-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
40 '서울 남산 순환', '도심 한가운데 위치한 남산을 한 바퀴 도는 코스. 오르막 내리막이 있어 체력 향상에 효과적이며 남산타워 전망이 일품입니다.',
41 'published', 7400, TRUE,
42 ST_SetSRID(ST_MakePoint(126.9872, 37.5534), 4326)::geography,
43 ST_SetSRID(ST_MakePoint(126.9872, 37.5534), 4326)::geography,
44 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.9872, 37.5534), ST_MakePoint(126.9848, 37.5495), ST_MakePoint(126.9870, 37.5455), ST_MakePoint
(126.9938, 37.5448), ST_MakePoint(127.0010, 37.5478), ST_MakePoint(127.0028, 37.5502), ST_MakePoint(126.9995, 37.5540), ST_MakePoint(126.9920, 37.554
5), ST_MakePoint(126.9872, 37.5534)]), 4326)::geography,
45 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
46
47 INSERT INTO course_points(id, course_id, sequence, location) VALUES
48 (gen_random_uuid(), 'c0000002-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(126.9872, 37.5534), 4326)::geography),
49 (gen_random_uuid(), 'c0000002-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(126.9848, 37.5495), 4326)::geography),
50 (gen_random_uuid(), 'c0000002-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(126.9870, 37.5455), 4326)::geography),
51 (gen_random_uuid(), 'c0000002-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(126.9938, 37.5448), 4326)::geography),
52 (gen_random_uuid(), 'c0000002-0000-0000-0000-000000000000', 5, ST_SetSRID(ST_MakePoint(127.0010, 37.5478), 4326)::geography),
53 (gen_random_uuid(), 'c0000002-0000-0000-0000-000000000000', 6, ST_SetSRID(ST_MakePoint(127.0028, 37.5502), 4326)::geography),
54 (gen_random_uuid(), 'c0000002-0000-0000-0000-000000000000', 7, ST_SetSRID(ST_MakePoint(126.9995, 37.5540), 4326)::geography),
55 (gen_random_uuid(), 'c0000002-0000-0000-0000-000000000000', 8, ST_SetSRID(ST_MakePoint(126.9872, 37.5534), 4326)::geography)
56 ON CONFLICT DO NOTHING;
57
58 -- 3. 청계천 코스
59 INSERT INTO courses VALUES ('c0000003-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
60 '청계천 코스', '청계광장에서 출발해 청계천을 따라 달리는 도심 속 힐링 코스. 물소리와 함께 달릴 수 있어 직장인 러너에게 인기입니다.',
61 'published', 8700, FALSE,
62 ST_SetSRID(ST_MakePoint(126.9782, 37.5696), 4326)::geography,
63 ST_SetSRID(ST_MakePoint(127.0318, 37.5642), 4326)::geography,
64 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.9782, 37.5696), ST_MakePoint(126.9900, 37.5703), ST_MakePoint(127.0020, 37.5710), ST_MakePoint
(127.0140, 37.5710), ST_MakePoint(127.0260, 37.5698), ST_MakePoint(127.0318, 37.5642)]), 4326)::geography,
65 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
66
67 INSERT INTO course_points(id, course_id, sequence, location) VALUES
68 (gen_random_uuid(), 'c0000003-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(126.9782, 37.5696), 4326)::geography),
69 (gen_random_uuid(), 'c0000003-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(126.9900, 37.5703), 4326)::geography),
70 (gen_random_uuid(), 'c0000003-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(127.0020, 37.5710), 4326)::geography),
71 (gen_random_uuid(), 'c0000003-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(127.0140, 37.5710), 4326)::geography),
72 (gen_random_uuid(), 'c0000003-0000-0000-0000-000000000000', 5, ST_SetSRID(ST_MakePoint(127.0260, 37.5698), 4326)::geography),
73 (gen_random_uuid(), 'c0000003-0000-0000-0000-000000000000', 6, ST_SetSRID(ST_MakePoint(127.0318, 37.5642), 4326)::geography)
74 ON CONFLICT DO NOTHING;
75
76 -- 4. 반포 한강공원
77 INSERT INTO courses VALUES ('c0000004-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,

```

```

78 '반포 한강공원 코스', '반포대교 인근을 달리는 코스. 달빛무지개분수와 야경이 아름다워 야간 러닝으로도 인기입니다.',
79 'published', 6000, FALSE,
80 ST_SetSRID(ST_MakePoint(126.9938, 37.5131), 4326)::geography,
81 ST_SetSRID(ST_MakePoint(127.0395, 37.5098), 4326)::geography,
82 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.9938, 37.5131), ST_MakePoint(127.0060, 37.5143), ST_MakePoint(127.0180, 37.5142), ST_MakePoint
(127.0300, 37.5122), ST_MakePoint(127.0395, 37.5098)]), 4326)::geography,
83 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
84
85 INSERT INTO course_points(id, course_id, sequence, location) VALUES
86 (gen_random_uuid(), 'c0000004-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(126.9938, 37.5131), 4326)::geography),
87 (gen_random_uuid(), 'c0000004-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(127.0060, 37.5143), 4326)::geography),
88 (gen_random_uuid(), 'c0000004-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(127.0180, 37.5142), 4326)::geography),
89 (gen_random_uuid(), 'c0000004-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(127.0300, 37.5122), 4326)::geography),
90 (gen_random_uuid(), 'c0000004-0000-0000-0000-000000000000', 5, ST_SetSRID(ST_MakePoint(127.0395, 37.5098), 4326)::geography)
91 ON CONFLICT DO NOTHING;
92
93 -- 5. 올림픽공원
94 INSERT INTO courses VALUES ('c0000005-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
95 '올림픽공원 코스', '1988 서울올림픽 유산을 간직한 공원 내부를 한 바퀴 도는 코스. 조각 공원과 잔디밭을 통과하며 달립니다.',
96 'published', 5500, TRUE,
97 ST_SetSRID(ST_MakePoint(127.1241, 37.5220), 4326)::geography,
98 ST_SetSRID(ST_MakePoint(127.1241, 37.5220), 4326)::geography,
99 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.1241, 37.5220), ST_MakePoint(127.1305, 37.5185), ST_MakePoint(127.1388, 37.5170), ST_MakePoint
(127.1445, 37.5205), ST_MakePoint(127.1440, 37.5240), ST_MakePoint(127.1388, 37.5262), ST_MakePoint(127.1315, 37.5255), ST_MakePoint(127.1241, 37.522
0)]), 4326)::geography,
100 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
101
102 INSERT INTO course_points(id, course_id, sequence, location) VALUES
103 (gen_random_uuid(), 'c0000005-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(127.1241, 37.5220), 4326)::geography),
104 (gen_random_uuid(), 'c0000005-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(127.1305, 37.5185), 4326)::geography),
105 (gen_random_uuid(), 'c0000005-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(127.1388, 37.5170), 4326)::geography),
106 (gen_random_uuid(), 'c0000005-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(127.1445, 37.5205), 4326)::geography),
107 (gen_random_uuid(), 'c0000005-0000-0000-0000-000000000000', 5, ST_SetSRID(ST_MakePoint(127.1440, 37.5240), 4326)::geography),
108 (gen_random_uuid(), 'c0000005-0000-0000-0000-000000000000', 6, ST_SetSRID(ST_MakePoint(127.1388, 37.5262), 4326)::geography),
109 (gen_random_uuid(), 'c0000005-0000-0000-0000-000000000000', 7, ST_SetSRID(ST_MakePoint(127.1241, 37.5220), 4326)::geography)
110 ON CONFLICT DO NOTHING;
111
112 -- 6. 독섬 한강공원
113 INSERT INTO courses VALUES ('c0000006-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
114 '독섬 한강공원 코스', '독섬 한강공원에서 광진교 방향까지 이어지는 강변 코스. 자전거 도로와 분리된 러닝 전용 트랙이 잘 조성되어 있습니다.',
115 'published', 5000, FALSE,
116 ST_SetSRID(ST_MakePoint(127.0658, 37.5302), 4326)::geography,
117 ST_SetSRID(ST_MakePoint(127.1050, 37.5225), 4326)::geography,
118 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.0658, 37.5302), ST_MakePoint(127.0760, 37.5280), ST_MakePoint(127.0860, 37.5260), ST_MakePoint
(127.0955, 37.5242), ST_MakePoint(127.1050, 37.5225)]), 4326)::geography,
119 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
120
121 INSERT INTO course_points(id, course_id, sequence, location) VALUES
122 (gen_random_uuid(), 'c0000006-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(127.0658, 37.5302), 4326)::geography),
123 (gen_random_uuid(), 'c0000006-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(127.0760, 37.5280), 4326)::geography),
124 (gen_random_uuid(), 'c0000006-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(127.0860, 37.5260), 4326)::geography),
125 (gen_random_uuid(), 'c0000006-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(127.0955, 37.5242), 4326)::geography),
126 (gen_random_uuid(), 'c0000006-0000-0000-0000-000000000000', 5, ST_SetSRID(ST_MakePoint(127.1050, 37.5225), 4326)::geography)
127 ON CONFLICT DO NOTHING;
128
129 -- 7. 잠실 한강공원
130 INSERT INTO courses VALUES ('c0000007-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
131 '잠실 한강공원 코스', '잠실 한강공원에서 천호대교까지 이어지는 코스. 롯데타워 전망과 함께 달릴 수 있습니다.',
132 'published', 7000, FALSE,
133 ST_SetSRID(ST_MakePoint(127.0878, 37.5212), 4326)::geography,
134 ST_SetSRID(ST_MakePoint(127.1360, 37.5172), 4326)::geography,
135 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.0878, 37.5212), ST_MakePoint(127.0988, 37.5198), ST_MakePoint(127.1095, 37.5188), ST_MakePoint
(127.1210, 37.5180), ST_MakePoint(127.1360, 37.5172)]), 4326)::geography,
136 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
137
138 INSERT INTO course_points(id, course_id, sequence, location) VALUES
139 (gen_random_uuid(), 'c0000007-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(127.0878, 37.5212), 4326)::geography),
140 (gen_random_uuid(), 'c0000007-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(127.0988, 37.5198), 4326)::geography),
141 (gen_random_uuid(), 'c0000007-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(127.1095, 37.5188), 4326)::geography),
142 (gen_random_uuid(), 'c0000007-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(127.1210, 37.5180), 4326)::geography),
143 (gen_random_uuid(), 'c0000007-0000-0000-0000-000000000000', 5, ST_SetSRID(ST_MakePoint(127.1360, 37.5172), 4326)::geography)
144 ON CONFLICT DO NOTHING;
145
146 -- 8. 망원 한강공원
147 INSERT INTO courses VALUES ('c0000008-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
148 '망원 한강공원 코스', '망원 한강공원에서 난지 방향으로 이어지는 코스. 상암 월드컵경기장 뷰와 함께 달립니다.',
149 'published', 5200, FALSE,
150 ST_SetSRID(ST_MakePoint(126.8975, 37.5537), 4326)::geography,
151 ST_SetSRID(ST_MakePoint(126.8538, 37.5672), 4326)::geography,
152 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.8975, 37.5537), ST_MakePoint(126.8878, 37.5570), ST_MakePoint(126.8778, 37.5605), ST_MakePoint
(126.8660, 37.5642), ST_MakePoint(126.8538, 37.5672)]), 4326)::geography,
153 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
154
155 INSERT INTO course_points(id, course_id, sequence, location) VALUES
156 (gen_random_uuid(), 'c0000008-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(126.8975, 37.5537), 4326)::geography),
157 (gen_random_uuid(), 'c0000008-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(126.8878, 37.5570), 4326)::geography),

```

```

158 (gen_random_uuid(),'c0000008-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(126.8778,37.5605),4326)::geography),
159 (gen_random_uuid(),'c0000008-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(126.8660,37.5642),4326)::geography),
160 (gen_random_uuid(),'c0000008-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(126.8538,37.5672),4326)::geography)
161 ON CONFLICT DO NOTHING;
162
163 -- 9. 서울숲
164 INSERT INTO courses VALUES ('c0000009-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
165 '서울숲 코스','성수동 서울숲 내부를 달리는 코스. 사슴 방목장과 자연 숲길을 통과하며 도심 속 자연을 만끽할 수 있습니다.',
166 'published',5200,TRUE,
167 ST_SetSRID(ST_MakePoint(127.0374,37.5449),4326)::geography,
168 ST_SetSRID(ST_MakePoint(127.0374,37.5449),4326)::geography,
169 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.0374,37.5449),ST_MakePoint(127.0418,37.5430),ST_MakePoint(127.0455,37.5415),ST_MakePoint
(127.0468,37.5445),ST_MakePoint(127.0452,37.5472),ST_MakePoint(127.0415,37.5480),ST_MakePoint(127.0374,37.5449)]),4326)::geography,
170 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
171
172 INSERT INTO course_points(id,course_id,sequence,location) VALUES
173 (gen_random_uuid(),'c0000009-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.0374,37.5449),4326)::geography),
174 (gen_random_uuid(),'c0000009-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.0418,37.5430),4326)::geography),
175 (gen_random_uuid(),'c0000009-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.0455,37.5415),4326)::geography),
176 (gen_random_uuid(),'c0000009-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(127.0468,37.5445),4326)::geography),
177 (gen_random_uuid(),'c0000009-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(127.0452,37.5472),4326)::geography),
178 (gen_random_uuid(),'c0000009-0000-0000-0000-000000000000',6,ST_SetSRID(ST_MakePoint(127.0374,37.5449),4326)::geography)
179 ON CONFLICT DO NOTHING;
180
181 -- 10. 양재천 코스
182 INSERT INTO courses VALUES ('c0000010-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
183 '양재천 코스','양재에서 탄천 합류 지점까지 이어지는 강변 코스. 버드나무 가로수길과 잘 정비된 산책로가 인상적입니다.',
184 'published',9500,FALSE,
185 ST_SetSRID(ST_MakePoint(127.0430,37.4796),4326)::geography,
186 ST_SetSRID(ST_MakePoint(127.1155,37.4678),4326)::geography,
187 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.0430,37.4796),ST_MakePoint(127.0600,37.4762),ST_MakePoint(127.0760,37.4732),ST_MakePoint
(127.0920,37.4705),ST_MakePoint(127.1060,37.4688),ST_MakePoint(127.1155,37.4678)]),4326)::geography,
188 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
189
190 INSERT INTO course_points(id,course_id,sequence,location) VALUES
191 (gen_random_uuid(),'c0000010-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.0430,37.4796),4326)::geography),
192 (gen_random_uuid(),'c0000010-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.0600,37.4762),4326)::geography),
193 (gen_random_uuid(),'c0000010-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.0760,37.4732),4326)::geography),
194 (gen_random_uuid(),'c0000010-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(127.0920,37.4705),4326)::geography),
195 (gen_random_uuid(),'c0000010-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(127.1155,37.4678),4326)::geography)
196 ON CONFLICT DO NOTHING;
197
198 -- 11. 인왕산 코스
199 INSERT INTO courses VALUES ('c0000011-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
200 '인왕산 코스','경복궁 서쪽 인왕산 자락을 달리는 코스. 한양도성 성곽과 함께 달리며 서울 도심 전망을 즐길 수 있습니다.',
201 'published',5300,TRUE,
202 ST_SetSRID(ST_MakePoint(126.9620,37.5820),4326)::geography,
203 ST_SetSRID(ST_MakePoint(126.9620,37.5820),4326)::geography,
204 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.9620,37.5820),ST_MakePoint(126.9645,37.5782),ST_MakePoint(126.9668,37.5748),ST_MakePoint
(126.9695,37.5778),ST_MakePoint(126.9680,37.5815),ST_MakePoint(126.9650,37.5842),ST_MakePoint(126.9620,37.5820)]),4326)::geography,
205 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
206
207 INSERT INTO course_points(id,course_id,sequence,location) VALUES
208 (gen_random_uuid(),'c0000011-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(126.9620,37.5820),4326)::geography),
209 (gen_random_uuid(),'c0000011-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(126.9645,37.5782),4326)::geography),
210 (gen_random_uuid(),'c0000011-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(126.9668,37.5748),4326)::geography),
211 (gen_random_uuid(),'c0000011-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(126.9695,37.5778),4326)::geography),
212 (gen_random_uuid(),'c0000011-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(126.9620,37.5820),4326)::geography)
213 ON CONFLICT DO NOTHING;
214
215 -- 12. 안산 순환
216 INSERT INTO courses VALUES ('c0000012-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
217 '안산(鞍山) 순환 코스','서대문 안산을 한 바퀴 도는 산악 코스. 도심 접근성이 좋고 비교적 완만한 산길이라서 출퇴근 러닝으로 인기입니다.',
218 'published',4800,TRUE,
219 ST_SetSRID(ST_MakePoint(126.9348,37.5890),4326)::geography,
220 ST_SetSRID(ST_MakePoint(126.9348,37.5890),4326)::geography,
221 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.9348,37.5890),ST_MakePoint(126.9310,37.5858),ST_MakePoint(126.9298,37.5825),ST_MakePoint
(126.9315,37.5795),ST_MakePoint(126.9355,37.5812),ST_MakePoint(126.9380,37.5848),ST_MakePoint(126.9370,37.5880),ST_MakePoint(126.9348,37.589
0)]),4326)::geography,
222 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
223
224 INSERT INTO course_points(id,course_id,sequence,location) VALUES
225 (gen_random_uuid(),'c0000012-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(126.9348,37.5890),4326)::geography),
226 (gen_random_uuid(),'c0000012-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(126.9310,37.5858),4326)::geography),
227 (gen_random_uuid(),'c0000012-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(126.9298,37.5825),4326)::geography),
228 (gen_random_uuid(),'c0000012-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(126.9355,37.5812),4326)::geography),
229 (gen_random_uuid(),'c0000012-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(126.9348,37.5890),4326)::geography)
230 ON CONFLICT DO NOTHING;
231
232 -- 13. 북서울꿈의숲
233 INSERT INTO courses VALUES ('c0000013-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
234 '북서울꿈의숲 코스','강북구 번동의 대규모 공원을 달리는 코스. 월영지 호수와 꿈의숲 아트센터 주변을 순환합니다.',
235 'published',4500,TRUE,
236 ST_SetSRID(ST_MakePoint(127.0385,37.6418),4326)::geography,
237 ST_SetSRID(ST_MakePoint(127.0385,37.6418),4326)::geography,
238 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.0385,37.6418),ST_MakePoint(127.0420,37.6398),ST_MakePoint(127.0445,37.6372),ST_MakePoint

```



```

(127.0435,37.6345),ST_MakePoint(127.0405,37.6332),ST_MakePoint(127.0372,37.6348),ST_MakePoint(127.0358,37.6378),ST_MakePoint(127.0385,37.641
8))::geography,
239 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
240
241 INSERT INTO course_points(id,course_id,sequence,location) VALUES
242 (gen_random_uuid(),'c0000013-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.0385,37.6418),4326)::geography),
243 (gen_random_uuid(),'c0000013-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.0420,37.6398),4326)::geography),
244 (gen_random_uuid(),'c0000013-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.0445,37.6372),4326)::geography),
245 (gen_random_uuid(),'c0000013-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(127.0405,37.6332),4326)::geography),
246 (gen_random_uuid(),'c0000013-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(127.0358,37.6378),4326)::geography),
247 (gen_random_uuid(),'c0000013-0000-0000-0000-000000000000',6,ST_SetSRID(ST_MakePoint(127.0385,37.6418),4326)::geography)
248 ON CONFLICT DO NOTHING;
249
250 -- 14. 중랑천 코스
251 INSERT INTO courses VALUES ('c0000014-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
252 '중랑천 코스','중랑천을 따라 달리는 장거리 강변 코스. 봄철 벚꽃 명소로 유명하며 왕복 20km까지 연장도 가능합니다.',
253 'published',10000,FALSE,
254 ST_SetSRID(ST_MakePoint(127.0728,37.5758),4326)::geography,
255 ST_SetSRID(ST_MakePoint(127.0658,37.6720),4326)::geography,
256 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.0728,37.5758),ST_MakePoint(127.0718,37.5920),ST_MakePoint(127.0705,37.6080),ST_MakePoint
(127.0688,37.6240),ST_MakePoint(127.0672,37.6400),ST_MakePoint(127.0658,37.6560),ST_MakePoint(127.0658,37.6720)]),4326)::geography,
257 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
258
259 INSERT INTO course_points(id,course_id,sequence,location) VALUES
260 (gen_random_uuid(),'c0000014-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.0728,37.5758),4326)::geography),
261 (gen_random_uuid(),'c0000014-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.0718,37.5920),4326)::geography),
262 (gen_random_uuid(),'c0000014-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.0705,37.6080),4326)::geography),
263 (gen_random_uuid(),'c0000014-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(127.0688,37.6240),4326)::geography),
264 (gen_random_uuid(),'c0000014-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(127.0658,37.6720),4326)::geography)
265 ON CONFLICT DO NOTHING;
266
267 -- 15. 경복궁-광화문 코스
268 INSERT INTO courses VALUES ('c0000015-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
269 '경복궁-광화문 코스','광화문광장에서 출발해 경복궁과 청와대 앞길을 달리는 역사 문화 코스. 국내외 러너들에게 인기 있는 대표 관광 러닝 코스입니다.',
270 'published',5500,TRUE,
271 ST_SetSRID(ST_MakePoint(126.9770,37.5796),4326)::geography,
272 ST_SetSRID(ST_MakePoint(126.9770,37.5796),4326)::geography,
273 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.9770,37.5796),ST_MakePoint(126.9745,37.5825),ST_MakePoint(126.9748,37.5892),ST_MakePoint
(126.9790,37.5924),ST_MakePoint(126.9842,37.5908),ST_MakePoint(126.9840,37.5865),ST_MakePoint(126.9810,37.5830),ST_MakePoint(126.9800,37.579
6),ST_MakePoint(126.9770,37.5796)]),4326)::geography,
274 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
275
276 INSERT INTO course_points(id,course_id,sequence,location) VALUES
277 (gen_random_uuid(),'c0000015-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(126.9770,37.5796),4326)::geography),
278 (gen_random_uuid(),'c0000015-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(126.9748,37.5892),4326)::geography),
279 (gen_random_uuid(),'c0000015-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(126.9842,37.5908),4326)::geography),
280 (gen_random_uuid(),'c0000015-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(126.9840,37.5865),4326)::geography),
281 (gen_random_uuid(),'c0000015-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(126.9770,37.5796),4326)::geography)
282 ON CONFLICT DO NOTHING;
283
284 -- =====
285 -- 부산 (4개)
286 -- =====
287
288 -- 16. 광안리 해변
289 INSERT INTO courses VALUES ('c0000016-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
290 '부산 광안리 해변 코스','광안리 해수욕장 해변을 따라 달리는 부산 대표 코스. 광안대교의 웅장한 모습과 바다 뷰가 압권입니다.',
291 'published',5800,FALSE,
292 ST_SetSRID(ST_MakePoint(129.1155,35.1538),4326)::geography,
293 ST_SetSRID(ST_MakePoint(129.1438,35.1498),4326)::geography,
294 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(129.1155,35.1538),ST_MakePoint(129.1250,35.1550),ST_MakePoint(129.1350,35.1548),ST_MakePoint
(129.1438,35.1498)]),4326)::geography,
295 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
296
297 INSERT INTO course_points(id,course_id,sequence,location) VALUES
298 (gen_random_uuid(),'c0000016-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(129.1155,35.1538),4326)::geography),
299 (gen_random_uuid(),'c0000016-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(129.1250,35.1550),4326)::geography),
300 (gen_random_uuid(),'c0000016-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(129.1350,35.1548),4326)::geography),
301 (gen_random_uuid(),'c0000016-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(129.1438,35.1498),4326)::geography)
302 ON CONFLICT DO NOTHING;
303
304 -- 17. 해운대 해변
305 INSERT INTO courses VALUES ('c0000017-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
306 '부산 해운대 해변 코스','해운대 해수욕장 백사장과 마린시티를 잇는 코스. 동백섬 일주를 포함하면 난이도가 올라갑니다.',
307 'published',3800,TRUE,
308 ST_SetSRID(ST_MakePoint(129.1604,35.1587),4326)::geography,
309 ST_SetSRID(ST_MakePoint(129.1604,35.1587),4326)::geography,
310 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(129.1604,35.1587),ST_MakePoint(129.1652,35.1575),ST_MakePoint(129.1718,35.1578),ST_MakePoint
(129.1755,35.1602),ST_MakePoint(129.1718,35.1625),ST_MakePoint(129.1652,35.1620),ST_MakePoint(129.1604,35.1587)]),4326)::geography,
311 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
312
313 INSERT INTO course_points(id,course_id,sequence,location) VALUES
314 (gen_random_uuid(),'c0000017-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(129.1604,35.1587),4326)::geography),
315 (gen_random_uuid(),'c0000017-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(129.1718,35.1578),4326)::geography),
316 (gen_random_uuid(),'c0000017-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(129.1755,35.1602),4326)::geography),
317 (gen_random_uuid(),'c0000017-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(129.1604,35.1587),4326)::geography)

```

```

318 ON CONFLICT DO NOTHING;
319
320 -- 18. 이기대 해안
321 INSERT INTO courses VALUES ('c0000018-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
322 '부산 이기대 해안 코스', '오륙도 스카이워크 인근 이기대 해안을 따라 달리는 코스. 남해 절경과 기암절벽이 러닝을 특별하게 만들어줍니다.',
323 'published', 6200, FALSE,
324 ST_SetSRID(ST_MakePoint(129.1285, 35.1237), 4326)::geography,
325 ST_SetSRID(ST_MakePoint(129.1058, 35.1362), 4326)::geography,
326 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(129.1285, 35.1237), ST_MakePoint(129.1215, 35.1268), ST_MakePoint(129.1148, 35.1308), ST_MakePoint
(129.1088, 35.1338), ST_MakePoint(129.1058, 35.1362)]), 4326)::geography,
327 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
328
329 INSERT INTO course_points(id, course_id, sequence, location) VALUES
330 (gen_random_uuid(), 'c0000018-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(129.1285, 35.1237), 4326)::geography),
331 (gen_random_uuid(), 'c0000018-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(129.1215, 35.1268), 4326)::geography),
332 (gen_random_uuid(), 'c0000018-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(129.1148, 35.1308), 4326)::geography),
333 (gen_random_uuid(), 'c0000018-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(129.1058, 35.1362), 4326)::geography)
334 ON CONFLICT DO NOTHING;
335
336 -- 19. 수영강 갈맷길
337 INSERT INTO courses VALUES ('c0000019-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
338 '부산 수영강 갈맷길', '수영강을 따라 달리는 부산 갈맷길 코스. 도심과 강변이 조화를 이루며 다양한 거리로 즐길 수 있습니다.',
339 'published', 8500, FALSE,
340 ST_SetSRID(ST_MakePoint(129.0978, 35.1733), 4326)::geography,
341 ST_SetSRID(ST_MakePoint(129.1235, 35.1238), 4326)::geography,
342 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(129.0978, 35.1733), ST_MakePoint(129.1020, 35.1640), ST_MakePoint(129.1068, 35.1545), ST_MakePoint
(129.1115, 35.1450), ST_MakePoint(129.1165, 35.1355), ST_MakePoint(129.1235, 35.1238)]), 4326)::geography,
343 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
344
345 INSERT INTO course_points(id, course_id, sequence, location) VALUES
346 (gen_random_uuid(), 'c0000019-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(129.0978, 35.1733), 4326)::geography),
347 (gen_random_uuid(), 'c0000019-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(129.1020, 35.1640), 4326)::geography),
348 (gen_random_uuid(), 'c0000019-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(129.1068, 35.1545), 4326)::geography),
349 (gen_random_uuid(), 'c0000019-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(129.1235, 35.1238), 4326)::geography)
350 ON CONFLICT DO NOTHING;
351
352 -- =====
353 -- 경기도 (5개)
354 -- =====
355
356 -- 20. 수원화성 성곽길
357 INSERT INTO courses VALUES ('c0000020-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
358 '수원화성 성곽길', '유네스코 세계문화유산 수원화성 성곽을 따라 달리는 역사 코스. 성벽 위를 달리며 수원 도심 전망을 즐기니다.',
359 'published', 5700, TRUE,
360 ST_SetSRID(ST_MakePoint(127.0154, 37.2880), 4326)::geography,
361 ST_SetSRID(ST_MakePoint(127.0154, 37.2880), 4326)::geography,
362 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.0154, 37.2880), ST_MakePoint(127.0218, 37.2855), ST_MakePoint(127.0268, 37.2812), ST_MakePoint
(127.0255, 37.2762), ST_MakePoint(127.0192, 37.2740), ST_MakePoint(127.0118, 37.2762), ST_MakePoint(127.0085, 37.2812), ST_MakePoint(127.0102, 37.285
5), ST_MakePoint(127.0154, 37.2880)]), 4326)::geography,
363 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
364
365 INSERT INTO course_points(id, course_id, sequence, location) VALUES
366 (gen_random_uuid(), 'c0000020-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(127.0154, 37.2880), 4326)::geography),
367 (gen_random_uuid(), 'c0000020-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(127.0268, 37.2812), 4326)::geography),
368 (gen_random_uuid(), 'c0000020-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(127.0192, 37.2740), 4326)::geography),
369 (gen_random_uuid(), 'c0000020-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(127.0085, 37.2812), 4326)::geography),
370 (gen_random_uuid(), 'c0000020-0000-0000-0000-000000000000', 5, ST_SetSRID(ST_MakePoint(127.0154, 37.2880), 4326)::geography)
371 ON CONFLICT DO NOTHING;
372
373 -- 21. 단천 코스 (분당)
374 INSERT INTO courses VALUES ('c0000021-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
375 '분당 단천 코스', '성남 단천을 따라 달리는 장거리 코스. 분당 신도시를 관통하며 잘 정비된 자전거·러닝 전용로가 조성되어 있습니다.',
376 'published', 10000, FALSE,
377 ST_SetSRID(ST_MakePoint(127.1242, 37.3812), 4326)::geography,
378 ST_SetSRID(ST_MakePoint(127.1125, 37.4760), 4326)::geography,
379 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.1242, 37.3812), ST_MakePoint(127.1215, 37.4000), ST_MakePoint(127.1188, 37.4180), ST_MakePoint
(127.1162, 37.4360), ST_MakePoint(127.1142, 37.4560), ST_MakePoint(127.1125, 37.4760)]), 4326)::geography,
380 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
381
382 INSERT INTO course_points(id, course_id, sequence, location) VALUES
383 (gen_random_uuid(), 'c0000021-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(127.1242, 37.3812), 4326)::geography),
384 (gen_random_uuid(), 'c0000021-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(127.1188, 37.4180), 4326)::geography),
385 (gen_random_uuid(), 'c0000021-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(127.1142, 37.4560), 4326)::geography),
386 (gen_random_uuid(), 'c0000021-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(127.1125, 37.4760), 4326)::geography)
387 ON CONFLICT DO NOTHING;
388
389 -- 22. 일산 호수공원
390 INSERT INTO courses VALUES ('c0000022-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
391 '일산 호수공원 코스', '고양 일산 호수공원을 한 바퀴 도는 코스. 국내 최대 인공 호수인 호수공원 주변을 달리며 여유로운 러닝을 즐기니다.',
392 'published', 4700, TRUE,
393 ST_SetSRID(ST_MakePoint(126.7625, 37.6752), 4326)::geography,
394 ST_SetSRID(ST_MakePoint(126.7625, 37.6752), 4326)::geography,
395 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.7625, 37.6752), ST_MakePoint(126.7685, 37.6728), ST_MakePoint(126.7728, 37.6698), ST_MakePoint
(126.7720, 37.6662), ST_MakePoint(126.7672, 37.6645), ST_MakePoint(126.7622, 37.6658), ST_MakePoint(126.7590, 37.6692), ST_MakePoint(126.7610, 37.672
8), ST_MakePoint(126.7625, 37.6752)]), 4326)::geography,
396 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;

```

```

397
398 INSERT INTO course_points(id,course_id,sequence,location) VALUES
399 (gen_random_uuid(),'c0000022-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(126.7625,37.6752),4326)::geography),
400 (gen_random_uuid(),'c0000022-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(126.7728,37.6698),4326)::geography),
401 (gen_random_uuid(),'c0000022-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(126.7672,37.6645),4326)::geography),
402 (gen_random_uuid(),'c0000022-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(126.7590,37.6692),4326)::geography),
403 (gen_random_uuid(),'c0000022-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(126.7625,37.6752),4326)::geography)
404 ON CONFLICT DO NOTHING;
405
406 -- 23. 광고호수공원
407 INSERT INTO courses VALUES ('c0000023-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
408 '광고호수공원 코스','수원 광고호수공원 원천호와 신대호 주변을 달리는 코스. 카페거리와 조화를 이루는 아름다운 호수 뷰가 인기입니다.',
409 'published',5200,TRUE,
410 ST_SetSRID(ST_MakePoint(127.0502,37.2898),4326)::geography,
411 ST_SetSRID(ST_MakePoint(127.0502,37.2898),4326)::geography,
412 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.0502,37.2898),ST_MakePoint(127.0558,37.2878),ST_MakePoint(127.0592,37.2848),ST_MakePoint
(127.0578,37.2812),ST_MakePoint(127.0532,37.2798),ST_MakePoint(127.0488,37.2815),ST_MakePoint(127.0465,37.2852),ST_MakePoint(127.0480,37.288
2),ST_MakePoint(127.0502,37.2898)]),4326)::geography,
413 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
414
415 INSERT INTO course_points(id,course_id,sequence,location) VALUES
416 (gen_random_uuid(),'c0000023-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.0502,37.2898),4326)::geography),
417 (gen_random_uuid(),'c0000023-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.0558,37.2878),4326)::geography),
418 (gen_random_uuid(),'c0000023-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.0532,37.2798),4326)::geography),
419 (gen_random_uuid(),'c0000023-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(127.0465,37.2852),4326)::geography),
420 (gen_random_uuid(),'c0000023-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(127.0502,37.2898),4326)::geography)
421 ON CONFLICT DO NOTHING;
422
423 -- 24. 인천 송도 센트럴파크
424 INSERT INTO courses VALUES ('c0000024-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
425 '인천 송도 센트럴파크 코스','인천 송도국제도시 센트럴파크를 달리는 코스. 바닷물이 흐르는 인공 수로와 현대적인 도시 경관이 특색 있습니다.',
426 'published',5000,TRUE,
427 ST_SetSRID(ST_MakePoint(126.6477,37.3929),4326)::geography,
428 ST_SetSRID(ST_MakePoint(126.6477,37.3929),4326)::geography,
429 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.6477,37.3929),ST_MakePoint(126.6535,37.3908),ST_MakePoint(126.6568,37.3878),ST_MakePoint
(126.6555,37.3848),ST_MakePoint(126.6508,37.3832),ST_MakePoint(126.6462,37.3848),ST_MakePoint(126.6445,37.3882),ST_MakePoint(126.6455,37.391
2),ST_MakePoint(126.6477,37.3929)]),4326)::geography,
430 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
431
432 INSERT INTO course_points(id,course_id,sequence,location) VALUES
433 (gen_random_uuid(),'c0000024-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(126.6477,37.3929),4326)::geography),
434 (gen_random_uuid(),'c0000024-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(126.6568,37.3878),4326)::geography),
435 (gen_random_uuid(),'c0000024-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(126.6462,37.3848),4326)::geography),
436 (gen_random_uuid(),'c0000024-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(126.6477,37.3929),4326)::geography)
437 ON CONFLICT DO NOTHING;
438
439 -- =====
440 -- 대전·충청 (3개)
441 -- =====
442
443 -- 25. 대전 갑천 코스
444 INSERT INTO courses VALUES ('c0000025-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
445 '대전 갑천 코스','대전 도심을 가로지르는 갑천을 따라 달리는 코스. 엑스포과학공원과 월평공원을 잇는 구간이 특히 인기입니다.',
446 'published',10000,FALSE,
447 ST_SetSRID(ST_MakePoint(127.3555,36.3626),4326)::geography,
448 ST_SetSRID(ST_MakePoint(127.3692,36.4558),4326)::geography,
449 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.3555,36.3626),ST_MakePoint(127.3578,36.3800),ST_MakePoint(127.3602,36.3980),ST_MakePoint
(127.3628,36.4160),ST_MakePoint(127.3655,36.4340),ST_MakePoint(127.3692,36.4558)]),4326)::geography,
450 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
451
452 INSERT INTO course_points(id,course_id,sequence,location) VALUES
453 (gen_random_uuid(),'c0000025-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.3555,36.3626),4326)::geography),
454 (gen_random_uuid(),'c0000025-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.3602,36.3980),4326)::geography),
455 (gen_random_uuid(),'c0000025-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.3655,36.4340),4326)::geography),
456 (gen_random_uuid(),'c0000025-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(127.3692,36.4558),4326)::geography)
457 ON CONFLICT DO NOTHING;
458
459 -- 26. 청주 무심천 코스
460 INSERT INTO courses VALUES ('c0000026-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
461 '청주 무심천 코스','청주 도심 무심천을 따라 달리는 강변 코스. 잘 정비된 산책로와 자전거 도로가 러너들에게 쾌적한 환경을 제공합니다.',
462 'published',10000,FALSE,
463 ST_SetSRID(ST_MakePoint(127.4865,36.6327),4326)::geography,
464 ST_SetSRID(ST_MakePoint(127.4752,36.7232),4326)::geography,
465 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.4865,36.6327),ST_MakePoint(127.4840,36.6520),ST_MakePoint(127.4815,36.6712),ST_MakePoint
(127.4788,36.6908),ST_MakePoint(127.4752,36.7232)]),4326)::geography,
466 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
467
468 INSERT INTO course_points(id,course_id,sequence,location) VALUES
469 (gen_random_uuid(),'c0000026-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.4865,36.6327),4326)::geography),
470 (gen_random_uuid(),'c0000026-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.4815,36.6712),4326)::geography),
471 (gen_random_uuid(),'c0000026-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.4752,36.7232),4326)::geography)
472 ON CONFLICT DO NOTHING;
473
474 -- 27. 공주 금강 코스
475 INSERT INTO courses VALUES ('c0000027-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
476 '공주 금강 코스','백제 고도 공주를 가로지르는 금강을 따라 달리는 코스. 공산성과 금강철교를 바라보며 달리는 역사 러닝 코스입니다.',

```

```

477 'published',8000,FALSE,
478 ST_SetSRID(ST_MakePoint(127.1243,36.4519),4326)::geography,
479 ST_SetSRID(ST_MakePoint(127.0435,36.4382),4326)::geography,
480 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.1243,36.4519),ST_MakePoint(127.1045,36.4488),ST_MakePoint(127.0845,36.4458),ST_MakePoint
(127.0640,36.4418),ST_MakePoint(127.0435,36.4382)]),4326)::geography,
481 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
482
483 INSERT INTO course_points(id,course_id,sequence,location) VALUES
484 (gen_random_uuid(),'c0000027-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.1243,36.4519),4326)::geography),
485 (gen_random_uuid(),'c0000027-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.0845,36.4458),4326)::geography),
486 (gen_random_uuid(),'c0000027-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.0435,36.4382),4326)::geography)
487 ON CONFLICT DO NOTHING;
488
489 -- =====
490 -- 광주·전라 (3개)
491 -- =====
492
493 -- 28. 광주 영산강 코스
494 INSERT INTO courses VALUES ('c0000028-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
495 '광주 영산강 코스','광주 도심 영산강 자전거길을 따라 달리는 코스. 광주호와 경양방죽 인근 자연 경관을 즐기며 달릴 수 있습니다.',
496 'published',8000,FALSE,
497 ST_SetSRID(ST_MakePoint(126.8785,35.1502),4326)::geography,
498 ST_SetSRID(ST_MakePoint(126.8072,35.1958),4326)::geography,
499 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.8785,35.1502),ST_MakePoint(126.8628,35.1618),ST_MakePoint(126.8458,35.1738),ST_MakePoint
(126.8268,35.1852),ST_MakePoint(126.8072,35.1958)]),4326)::geography,
500 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
501
502 INSERT INTO course_points(id,course_id,sequence,location) VALUES
503 (gen_random_uuid(),'c0000028-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(126.8785,35.1502),4326)::geography),
504 (gen_random_uuid(),'c0000028-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(126.8458,35.1738),4326)::geography),
505 (gen_random_uuid(),'c0000028-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(126.8072,35.1958),4326)::geography)
506 ON CONFLICT DO NOTHING;
507
508 -- 29. 순천만 국가정원 코스
509 INSERT INTO courses VALUES ('c0000029-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
510 '순천만 국가정원 코스','국내 최초 국가 정원인 순천만 국가정원을 달리는 코스. 갈대밭과 습지를 배경으로 달리는 생태 러닝 코스입니다.',
511 'published',6000,TRUE,
512 ST_SetSRID(ST_MakePoint(127.4813,34.9146),4326)::geography,
513 ST_SetSRID(ST_MakePoint(127.4813,34.9146),4326)::geography,
514 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.4813,34.9146),ST_MakePoint(127.4868,34.9118),ST_MakePoint(127.4912,34.9088),ST_MakePoint
(127.4905,34.9052),ST_MakePoint(127.4858,34.9035),ST_MakePoint(127.4808,34.9048),ST_MakePoint(127.4772,34.9082),ST_MakePoint(127.4780,34.911
8),ST_MakePoint(127.4813,34.9146)]),4326)::geography,
515 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
516
517 INSERT INTO course_points(id,course_id,sequence,location) VALUES
518 (gen_random_uuid(),'c0000029-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.4813,34.9146),4326)::geography),
519 (gen_random_uuid(),'c0000029-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.4912,34.9088),4326)::geography),
520 (gen_random_uuid(),'c0000029-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.4772,34.9082),4326)::geography),
521 (gen_random_uuid(),'c0000029-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(127.4813,34.9146),4326)::geography)
522 ON CONFLICT DO NOTHING;
523
524 -- 30. 여수 오동도 코스
525 INSERT INTO courses VALUES ('c0000030-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
526 '여수 오동도 코스','여수 오동도 섬을 한 바퀴 도는 코스. 동백꽃과 남해 절경, 여수항 아경을 배경으로 달립니다.',
527 'published',3200,TRUE,
528 ST_SetSRID(ST_MakePoint(127.7470,34.7416),4326)::geography,
529 ST_SetSRID(ST_MakePoint(127.7470,34.7416),4326)::geography,
530 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.7470,34.7416),ST_MakePoint(127.7508,34.7402),ST_MakePoint(127.7542,34.7385),ST_MakePoint
(127.7545,34.7358),ST_MakePoint(127.7520,34.7338),ST_MakePoint(127.7482,34.7340),ST_MakePoint(127.7452,34.7360),ST_MakePoint(127.7448,34.739
0),ST_MakePoint(127.7470,34.7416)]),4326)::geography,
531 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
532
533 INSERT INTO course_points(id,course_id,sequence,location) VALUES
534 (gen_random_uuid(),'c0000030-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.7470,34.7416),4326)::geography),
535 (gen_random_uuid(),'c0000030-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.7542,34.7385),4326)::geography),
536 (gen_random_uuid(),'c0000030-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.7520,34.7338),4326)::geography),
537 (gen_random_uuid(),'c0000030-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(127.7452,34.7360),4326)::geography),
538 (gen_random_uuid(),'c0000030-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(127.7470,34.7416),4326)::geography)
539 ON CONFLICT DO NOTHING;
540
541 -- =====
542 -- 대구·경상 (3개)
543 -- =====
544
545 -- 31. 대구 수성못 코스
546 INSERT INTO courses VALUES ('c0000031-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
547 '대구 수성못 코스','대구 수성구의 명소 수성못을 한 바퀴 도는 코스. 호수 주변 아름다운 산책로와 야경으로 저녁 러닝 명소입니다.',
548 'published',3500,TRUE,
549 ST_SetSRID(ST_MakePoint(128.6478,35.8453),4326)::geography,
550 ST_SetSRID(ST_MakePoint(128.6478,35.8453),4326)::geography,
551 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(128.6478,35.8453),ST_MakePoint(128.6518,35.8435),ST_MakePoint(128.6542,35.8408),ST_MakePoint
(128.6535,35.8378),ST_MakePoint(128.6498,35.8362),ST_MakePoint(128.6458,35.8370),ST_MakePoint(128.6435,35.8398),ST_MakePoint(128.6442,35.843
0),ST_MakePoint(128.6478,35.8453)]),4326)::geography,
552 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
553
554 INSERT INTO course_points(id,course_id,sequence,location) VALUES

```

```

555 (gen_random_uuid(),'c0000031-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(128.6478,35.8453),4326)::geography),
556 (gen_random_uuid(),'c0000031-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(128.6542,35.8408),4326)::geography),
557 (gen_random_uuid(),'c0000031-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(128.6435,35.8398),4326)::geography),
558 (gen_random_uuid(),'c0000031-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(128.6478,35.8453),4326)::geography)
559 ON CONFLICT DO NOTHING;
560
561 -- 32. 울산 태화강 국가정원
562 INSERT INTO courses VALUES ('c0000032-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
563 '울산 태화강 국가정원 코스','울산 태화강 국가정원을 달리는 코스. 십리대숲을 통과하며 달리는 구간이 특히 인상적입니다. 봄 유채꽃 시즌에 절경을 이룹니다.',
564 'published',8000,TRUE,
565 ST_SetSRID(ST_MakePoint(129.3164,35.5411),4326)::geography,
566 ST_SetSRID(ST_MakePoint(129.3164,35.5411),4326)::geography,
567 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(129.3164,35.5411),ST_MakePoint(129.3248,35.5388),ST_MakePoint(129.3312,35.5358),ST_MakePoint
(129.3318,35.5322),ST_MakePoint(129.3270,35.5298),ST_MakePoint(129.3192,35.5308),ST_MakePoint(129.3125,35.5338),ST_MakePoint(129.3105,35.537
2),ST_MakePoint(129.3140,35.5400),ST_MakePoint(129.3164,35.5411)]),4326)::geography,
568 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
569
570 INSERT INTO course_points(id,course_id,sequence,location) VALUES
571 (gen_random_uuid(),'c0000032-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(129.3164,35.5411),4326)::geography),
572 (gen_random_uuid(),'c0000032-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(129.3312,35.5358),4326)::geography),
573 (gen_random_uuid(),'c0000032-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(129.3192,35.5308),4326)::geography),
574 (gen_random_uuid(),'c0000032-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(129.3105,35.5372),4326)::geography),
575 (gen_random_uuid(),'c0000032-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(129.3164,35.5411),4326)::geography)
576 ON CONFLICT DO NOTHING;
577
578 -- 33. 창원 용지호수 코스
579 INSERT INTO courses VALUES ('c0000033-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
580 '창원 용지호수 코스','창원 도심 용지호수 주변을 달리는 코스. 용지공원 산책로가 잘 조성되어 있으며 계절마다 다른 꽃을 즐길 수 있습니다.',
581 'published',4000,TRUE,
582 ST_SetSRID(ST_MakePoint(128.6812,35.2285),4326)::geography,
583 ST_SetSRID(ST_MakePoint(128.6812,35.2285),4326)::geography,
584 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(128.6812,35.2285),ST_MakePoint(128.6858,35.2262),ST_MakePoint(128.6882,35.2232),ST_MakePoint
(128.6868,35.2200),ST_MakePoint(128.6825,35.2185),ST_MakePoint(128.6778,35.2198),ST_MakePoint(128.6758,35.2232),ST_MakePoint(128.6775,35.226
2),ST_MakePoint(128.6812,35.2285)]),4326)::geography,
585 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
586
587 INSERT INTO course_points(id,course_id,sequence,location) VALUES
588 (gen_random_uuid(),'c0000033-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(128.6812,35.2285),4326)::geography),
589 (gen_random_uuid(),'c0000033-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(128.6882,35.2232),4326)::geography),
590 (gen_random_uuid(),'c0000033-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(128.6758,35.2232),4326)::geography),
591 (gen_random_uuid(),'c0000033-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(128.6812,35.2285),4326)::geography)
592 ON CONFLICT DO NOTHING;
593
594 -- =====
595 -- 강원도 (3개)
596 -- =====
597
598 -- 34. 강릉 경포 코스
599 INSERT INTO courses VALUES ('c0000034-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
600 '강릉 경포 코스','경포해변과 경포호를 잇는 강릉 대표 러닝 코스. 동해 바다와 호수를 동시에 즐기며 달릴 수 있습니다.',
601 'published',5000,FALSE,
602 ST_SetSRID(ST_MakePoint(128.9041,37.7962),4326)::geography,
603 ST_SetSRID(ST_MakePoint(128.8852,37.7868),4326)::geography,
604 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(128.9041,37.7962),ST_MakePoint(128.8998,37.7952),ST_MakePoint(128.8952,37.7938),ST_MakePoint
(128.8908,37.7912),ST_MakePoint(128.8872,37.7888),ST_MakePoint(128.8852,37.7868)]),4326)::geography,
605 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
606
607 INSERT INTO course_points(id,course_id,sequence,location) VALUES
608 (gen_random_uuid(),'c0000034-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(128.9041,37.7962),4326)::geography),
609 (gen_random_uuid(),'c0000034-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(128.8952,37.7938),4326)::geography),
610 (gen_random_uuid(),'c0000034-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(128.8852,37.7868),4326)::geography)
611 ON CONFLICT DO NOTHING;
612
613 -- 35. 춘천 의암호 코스
614 INSERT INTO courses VALUES ('c0000035-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
615 '춘천 의암호 코스','춘천 의암호 주변을 달리는 코스. 드라마 촬영지로 유명한 낭만적인 호반 도시 춘천의 아름다운 뷰를 즐깁니다.',
616 'published',8000,FALSE,
617 ST_SetSRID(ST_MakePoint(127.7215,37.8862),4326)::geography,
618 ST_SetSRID(ST_MakePoint(127.6788,37.9342),4326)::geography,
619 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.7215,37.8862),ST_MakePoint(127.7105,37.8988),ST_MakePoint(127.6998,37.9115),ST_MakePoint
(127.6888,37.9228),ST_MakePoint(127.6788,37.9342)]),4326)::geography,
620 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
621
622 INSERT INTO course_points(id,course_id,sequence,location) VALUES
623 (gen_random_uuid(),'c0000035-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.7215,37.8862),4326)::geography),
624 (gen_random_uuid(),'c0000035-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.6998,37.9115),4326)::geography),
625 (gen_random_uuid(),'c0000035-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.6788,37.9342),4326)::geography)
626 ON CONFLICT DO NOTHING;
627
628 -- 36. 속초 청초호·설악 코스
629 INSERT INTO courses VALUES ('c0000036-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
630 '속초 청초호 코스','속초 청초호를 한 바퀴 도는 코스. 설악산을 배경으로 달리며 속초항과 아바이마을의 풍경을 즐깁니다.',
631 'published',4500,TRUE,
632 ST_SetSRID(ST_MakePoint(128.5912,38.2042),4326)::geography,
633 ST_SetSRID(ST_MakePoint(128.5912,38.2042),4326)::geography,
634 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(128.5912,38.2042),ST_MakePoint(128.5965,38.2018),ST_MakePoint(128.5998,38.1985),ST_MakePoint

```

```

(128.5985,38.1952),ST_MakePoint(128.5942,38.1938),ST_MakePoint(128.5895,38.1952),ST_MakePoint(128.5872,38.1988),ST_MakePoint(128.5885,38.202
2),ST_MakePoint(128.5912,38.2042) )},4326)::geography,
635 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
636
637 INSERT INTO course_points(id,course_id,sequence,location) VALUES
638 (gen_random_uuid(),'c0000036-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(128.5912,38.2042),4326)::geography),
639 (gen_random_uuid(),'c0000036-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(128.5998,38.1985),4326)::geography),
640 (gen_random_uuid(),'c0000036-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(128.5872,38.1988),4326)::geography),
641 (gen_random_uuid(),'c0000036-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(128.5912,38.2042),4326)::geography)
642 ON CONFLICT DO NOTHING;
643
644 -- =====
645 -- 제주도 (3개)
646 -- =====
647
648 -- 37. 제주 올레 1코스
649 INSERT INTO courses VALUES ('c0000037-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
650 '제주 올레 1코스','시흥초등학교에서 광치기해변까지 이어지는 제주올레 1코스. 성산일출봉과 바닷가 절경을 함께 즐기는 국내 최고 트레일런 코스입니다.',
651 'published',15000,FALSE,
652 ST_SetSRID(ST_MakePoint(126.9297,33.4627),4326)::geography,
653 ST_SetSRID(ST_MakePoint(126.9282,33.4522),4326)::geography,
654 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.9297,33.4627),ST_MakePoint(126.9315,33.4590),ST_MakePoint(126.9350,33.4560),ST_MakePoint
(126.9382,33.4542),ST_MakePoint(126.9355,33.4518),ST_MakePoint(126.9318,33.4510),ST_MakePoint(126.9282,33.4522) ]),4326)::geography,
655 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
656
657 INSERT INTO course_points(id,course_id,sequence,location) VALUES
658 (gen_random_uuid(),'c0000037-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(126.9297,33.4627),4326)::geography),
659 (gen_random_uuid(),'c0000037-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(126.9350,33.4560),4326)::geography),
660 (gen_random_uuid(),'c0000037-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(126.9382,33.4542),4326)::geography),
661 (gen_random_uuid(),'c0000037-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(126.9282,33.4522),4326)::geography)
662 ON CONFLICT DO NOTHING;
663
664 -- 38. 제주 한라산 어리목 코스
665 INSERT INTO courses VALUES ('c0000038-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
666 '제주 한라산 어리목 코스','한라산 어리목 탐방로를 달리는 코스. 해발 970m 뿔세오름까지 오르는 도전적인 트레일런 코스입니다.',
667 'published',9600,FALSE,
668 ST_SetSRID(ST_MakePoint(126.5067,33.3817),4326)::geography,
669 ST_SetSRID(ST_MakePoint(126.5285,33.3612),4326)::geography,
670 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.5067,33.3817),ST_MakePoint(126.5112,33.3775),ST_MakePoint(126.5162,33.3732),ST_MakePoint
(126.5218,33.3688),ST_MakePoint(126.5258,33.3645),ST_MakePoint(126.5285,33.3612) ]),4326)::geography,
671 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
672
673 INSERT INTO course_points(id,course_id,sequence,location) VALUES
674 (gen_random_uuid(),'c0000038-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(126.5067,33.3817),4326)::geography),
675 (gen_random_uuid(),'c0000038-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(126.5162,33.3732),4326)::geography),
676 (gen_random_uuid(),'c0000038-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(126.5258,33.3645),4326)::geography),
677 (gen_random_uuid(),'c0000038-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(126.5285,33.3612),4326)::geography)
678 ON CONFLICT DO NOTHING;
679
680 -- 39. 제주 함덕해변 코스
681 INSERT INTO courses VALUES ('c0000039-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
682 '제주 함덕해변 코스','제주 북쪽 함덕 서우봉 해변을 달리는 코스. 예메랄드빛 바다와 서우봉 능선을 배경으로 달릴 수 있습니다.',
683 'published',5500,TRUE,
684 ST_SetSRID(ST_MakePoint(126.6693,33.5437),4326)::geography,
685 ST_SetSRID(ST_MakePoint(126.6693,33.5437),4326)::geography,
686 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.6693,33.5437),ST_MakePoint(126.6748,33.5422),ST_MakePoint(126.6798,33.5405),ST_MakePoint
(126.6838,33.5378),ST_MakePoint(126.6828,33.5348),ST_MakePoint(126.6785,33.5338),ST_MakePoint(126.6735,33.5348),ST_MakePoint(126.6698,33.537
2),ST_MakePoint(126.6678,33.5405),ST_MakePoint(126.6693,33.5437) ]),4326)::geography,
687 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
688
689 INSERT INTO course_points(id,course_id,sequence,location) VALUES
690 (gen_random_uuid(),'c0000039-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(126.6693,33.5437),4326)::geography),
691 (gen_random_uuid(),'c0000039-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(126.6838,33.5378),4326)::geography),
692 (gen_random_uuid(),'c0000039-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(126.6678,33.5405),4326)::geography),
693 (gen_random_uuid(),'c0000039-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(126.6693,33.5437),4326)::geography)
694 ON CONFLICT DO NOTHING;
695
696 -- =====
697 -- 추가 (서울·경기 보완, 총 33개 초과)
698 -- =====
699
700 -- 40. 한강 마포·합정 코스
701 INSERT INTO courses VALUES ('c0000040-0000-0000-0000-000000000000','00000000-0000-0000-0000-000000000001',NULL,
702 '한강 마포·합정 코스','마포한강공원에서 합정까지 달리는 코스. 한강 다리 뷰와 서울 야경이 아름다우며 접근성이 뛰어납니다.',
703 'published',5000,FALSE,
704 ST_SetSRID(ST_MakePoint(126.9488,37.5488),4326)::geography,
705 ST_SetSRID(ST_MakePoint(126.9000,37.5512),4326)::geography,
706 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(126.9488,37.5488),ST_MakePoint(126.9378,37.5498),ST_MakePoint(126.9248,37.5505),ST_MakePoint
(126.9125,37.5510),ST_MakePoint(126.9000,37.5512) ]),4326)::geography,
707 0,0,NOW(),NOW(),NULL) ON CONFLICT (id) DO NOTHING;
708
709 INSERT INTO course_points(id,course_id,sequence,location) VALUES
710 (gen_random_uuid(),'c0000040-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(126.9488,37.5488),4326)::geography),
711 (gen_random_uuid(),'c0000040-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(126.9248,37.5505),4326)::geography),
712 (gen_random_uuid(),'c0000040-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(126.9000,37.5512),4326)::geography)
713 ON CONFLICT DO NOTHING;

```

```

714
715 -- 41. 성남 분당중앙공원 코스
716 INSERT INTO courses VALUES ('c0000041-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
717 '성남 중앙공원 코스', '성남 분당 중앙공원을 달리는 코스. 넓은 잔디밭과 호수, 분수대를 포함한 쾌적한 러닝 환경을 제공합니다.',
718 'published', 3800, TRUE,
719 ST_SetSRID(ST_MakePoint(127.1115, 37.3842), 4326)::geography,
720 ST_SetSRID(ST_MakePoint(127.1115, 37.3842), 4326)::geography,
721 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.1115, 37.3842), ST_MakePoint(127.1158, 37.3822), ST_MakePoint(127.1182, 37.3795), ST_MakePoint
(127.1170, 37.3768), ST_MakePoint(127.1130, 37.3755), ST_MakePoint(127.1090, 37.3768), ST_MakePoint(127.1072, 37.3795), ST_MakePoint(127.1082, 37.382
2), ST_MakePoint(127.1115, 37.3842)]), 4326)::geography,
722 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
723
724 INSERT INTO course_points(id, course_id, sequence, location) VALUES
725 (gen_random_uuid(), 'c0000041-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(127.1115, 37.3842), 4326)::geography),
726 (gen_random_uuid(), 'c0000041-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(127.1182, 37.3795), 4326)::geography),
727 (gen_random_uuid(), 'c0000041-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(127.1072, 37.3795), 4326)::geography),
728 (gen_random_uuid(), 'c0000041-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(127.1115, 37.3842), 4326)::geography)
729 ON CONFLICT DO NOTHING;
730
731 -- 42. 전주 전주천 코스
732 INSERT INTO courses VALUES ('c0000042-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
733 '전주 전주천 코스', '전주천을 따라 달리는 코스. 한옥마을 인근을 지나며 전통 도시 전주의 정취를 느낄 수 있습니다.',
734 'published', 8000, FALSE,
735 ST_SetSRID(ST_MakePoint(127.1478, 35.8225), 4326)::geography,
736 ST_SetSRID(ST_MakePoint(127.1248, 35.8912), 4326)::geography,
737 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.1478, 35.8225), ST_MakePoint(127.1428, 35.8392), ST_MakePoint(127.1378, 35.8558), ST_MakePoint
(127.1320, 35.8725), ST_MakePoint(127.1248, 35.8912)]), 4326)::geography,
738 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
739
740 INSERT INTO course_points(id, course_id, sequence, location) VALUES
741 (gen_random_uuid(), 'c0000042-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(127.1478, 35.8225), 4326)::geography),
742 (gen_random_uuid(), 'c0000042-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(127.1378, 35.8558), 4326)::geography),
743 (gen_random_uuid(), 'c0000042-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(127.1248, 35.8912), 4326)::geography)
744 ON CONFLICT DO NOTHING;
745
746 -- 43. 경주 대릉원·첨성대 코스
747 INSERT INTO courses VALUES ('c0000043-0000-0000-0000-000000000000', '00000000-0000-0000-0000-000000000001', NULL,
748 '경주 대릉원·첨성대 코스', '신라 천년의 역사를 품은 경주 대릉원과 첨성대, 안압지를 잇는 역사 러닝 코스. 고분군과 문화재를 곁에 두고 달립니다.',
749 'published', 6500, TRUE,
750 ST_SetSRID(ST_MakePoint(129.2195, 35.8362), 4326)::geography,
751 ST_SetSRID(ST_MakePoint(129.2195, 35.8362), 4326)::geography,
752 ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(129.2195, 35.8362), ST_MakePoint(129.2242, 35.8342), ST_MakePoint(129.2282, 35.8315), ST_MakePoint
(129.2298, 35.8282), ST_MakePoint(129.2278, 35.8255), ST_MakePoint(129.2238, 35.8242), ST_MakePoint(129.2195, 35.8255), ST_MakePoint(129.2170, 35.828
5), ST_MakePoint(129.2178, 35.8325), ST_MakePoint(129.2195, 35.8362)]), 4326)::geography,
753 0, 0, NOW(), NOW(), NULL) ON CONFLICT (id) DO NOTHING;
754
755 INSERT INTO course_points(id, course_id, sequence, location) VALUES
756 (gen_random_uuid(), 'c0000043-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(129.2195, 35.8362), 4326)::geography),
757 (gen_random_uuid(), 'c0000043-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(129.2282, 35.8315), 4326)::geography),
758 (gen_random_uuid(), 'c0000043-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(129.2238, 35.8242), 4326)::geography),
759 (gen_random_uuid(), 'c0000043-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(129.2170, 35.8285), 4326)::geography),
760 (gen_random_uuid(), 'c0000043-0000-0000-0000-000000000000', 5, ST_SetSRID(ST_MakePoint(129.2195, 35.8362), 4326)::geography)
761 ON CONFLICT DO NOTHING;

```

```
1  -- V13: 시드 코스 경로 좌표 촘촘히 보완
2  --
3  -- V12 시드 코스는 3~9개의 anchor 좌표만 갖고 있어
4  -- 지도에서 직선 다각형처럼 보인다.
5  -- ST_Segmentize로 경로를 100m 간격으로 분할해
6  -- course_points를 재생성하면 충주처럼 부드러운 경로가 표시된다.
7  --
8  -- 적용 대상: creator_id = '00000000-...-000001' (RunWay 공식 시스템 계정)
9
10 DO $$
11 DECLARE
12     rec          RECORD;
13     dense         geography;
14     geom          geometry;
15     n_pts         INT;
16     i             INT;
17 BEGIN
18     FOR rec IN
19         SELECT id FROM courses
20         WHERE creator_id = '00000000-0000-0000-0000-000000000001'
21             AND path IS NOT NULL
22     LOOP
23         -- 100 m 간격으로 중간 좌표 삽입 (geography 단위: 미터)
24         SELECT ST_Segmentize(path, 100) INTO dense
25         FROM courses WHERE id = rec.id;
26
27         IF dense IS NULL THEN
28             CONTINUE;
29         END IF;
30
31         -- courses.path 업데이트
32         UPDATE courses SET path = dense WHERE id = rec.id;
33
34         -- 기존 포인트 삭제 후 재삽입
35         DELETE FROM course_points WHERE course_id = rec.id;
36
37         geom := dense::geometry;
38         n_pts := ST_NPoints(geom);
39
40         FOR i IN 1..n_pts LOOP
41             INSERT INTO course_points (id, course_id, sequence, location)
42             VALUES (
43                 gen_random_uuid(),
44                 rec.id,
45                 i,
46                 ST_SetSRID(ST_PointN(geom, i), 4326)::geography
47             );
48         END LOOP;
49
50     END LOOP;
51 END $$;
```



```

1  -- V14: 임의 좌표 시드 코스 → 실제 OSM GPS 경로 교체
2  --
3  -- V12에서 삽입된 43개 코스는 직선 구간 3~9개로만 이루어진
4  -- 임의 좌표였다. V14에서 모두 삭제하고 OpenStreetMap 실측
5  -- 경로가 확인된 8개 코스만 삽입한다.
6  --
7  -- 출처:
8  -- 청계천 OSM Way 368276771 + 769631455
9  -- 남산 순환도로 OSM Way 357958296 + 379943267
10 -- 반포 한강공원 OSM Way 418249072
11 -- 올림픽공원 OSM Relation 17338887
12 -- 해운대 해변 OSM Way 107531972
13 -- 양재천 OSM Way 26505520
14 -- 서울숲 OSM Relation 17338887 (서울숲 boundary)
15 -- 여의도 남측 강변 OSM 한강남자전거길
16
17 -- — 1. 기존 시드 코스 삭제 —————
18 DELETE FROM course_points
19 WHERE course_id IN (
20     SELECT id FROM courses
21     WHERE creator_id = '00000000-0000-0000-0000-000000000001'
22 );
23 DELETE FROM courses
24 WHERE creator_id = '00000000-0000-0000-0000-000000000001';
25
26
27 -- — 2. 실제 OSM 경로 코스 삽입 —————
28
29 -----
30 -- 코스 A: 청계천 (OSM Way 368276771 + 769631455, 88 pts)
31 -- 청계광장 (126.9786, 37.5691) → 신답철교 방면 하류
32 -- 거리: ~8.7 km, 편도
33 -----
34 INSERT INTO courses VALUES (
35     'a0000001-0000-0000-0000-000000000000',
36     '00000000-0000-0000-0000-000000000001', NULL,
37     '서울 청계천 코스',
38     '청계광장에서 출발해 청계천을 따라 동쪽으로 달리는 도심 힐링 코스. 물소리와 함께 달릴 수 있어 직장인 러너에게 인기입니다. OSM Way 368276771 + 769631455 실측 경
39     '로.',
40     'published', 8700, FALSE,
41     ST_SetSRID(ST_MakePoint(126.9785975, 37.5691438), 4326)::geography,
42     ST_SetSRID(ST_MakePoint(127.0506623, 37.5527466), 4326)::geography,
43     ST_SetSRID(ST_MakeLine(ARRAY[
44         ST_MakePoint(126.9785975, 37.5691438), ST_MakePoint(126.9786757, 37.5691449),
45         ST_MakePoint(126.9794597, 37.5691067), ST_MakePoint(126.9801836, 37.5690719),
46         ST_MakePoint(126.9824530, 37.5688944), ST_MakePoint(126.9834423, 37.5688110),
47         ST_MakePoint(126.9852214, 37.5684594), ST_MakePoint(126.9866673, 37.5681104),
48         ST_MakePoint(126.9878120, 37.5680958), ST_MakePoint(126.9884690, 37.5680902),
49         ST_MakePoint(126.9886992, 37.5680829), ST_MakePoint(126.9901819, 37.5681529),
50         ST_MakePoint(126.9909605, 37.5682116), ST_MakePoint(126.9919388, 37.5682853),
51         ST_MakePoint(126.9925302, 37.5683299), ST_MakePoint(126.9930783, 37.5683708),
52         ST_MakePoint(126.9949392, 37.5685097), ST_MakePoint(126.9955960, 37.5685645),
53         ST_MakePoint(126.9967778, 37.5686863), ST_MakePoint(126.9977791, 37.5688006),
54         ST_MakePoint(126.9980552, 37.5688467), ST_MakePoint(127.0000376, 37.5692406),
55         ST_MakePoint(127.0015341, 37.5695268), ST_MakePoint(127.0017919, 37.5695761),
56         ST_MakePoint(127.0022487, 37.5695905), ST_MakePoint(127.0025823, 37.5695922),
57         ST_MakePoint(127.0031510, 37.5695969), ST_MakePoint(127.0044500, 37.5696019),
58         ST_MakePoint(127.0062675, 37.5696422), ST_MakePoint(127.0064916, 37.5696472),
59         ST_MakePoint(127.0084170, 37.5697478), ST_MakePoint(127.0092589, 37.5697918),
60         ST_MakePoint(127.0093071, 37.5697923), ST_MakePoint(127.0100308, 37.5697682),
61         ST_MakePoint(127.0105226, 37.5697636), ST_MakePoint(127.0123521, 37.5697464),
62         ST_MakePoint(127.0149692, 37.5697417), ST_MakePoint(127.0153425, 37.5697410),
63         ST_MakePoint(127.0155904, 37.5697406), ST_MakePoint(127.0156491, 37.5697458),
64         ST_MakePoint(127.0160978, 37.5697854), ST_MakePoint(127.0167624, 37.5699516),
65         ST_MakePoint(127.0175041, 37.5702226), ST_MakePoint(127.0188358, 37.5709441),
66         ST_MakePoint(127.0191272, 37.5710854), ST_MakePoint(127.0203224, 37.5715840),
67         ST_MakePoint(127.0211015, 37.5718150), ST_MakePoint(127.0218813, 37.5719088),
68         ST_MakePoint(127.0223862, 37.5718956), ST_MakePoint(127.0228377, 37.5718142),
69         ST_MakePoint(127.0243385, 37.5714025), ST_MakePoint(127.0252390, 37.5711508),
70         ST_MakePoint(127.0257493, 37.5710083), ST_MakePoint(127.0276164, 37.5704501),
71         ST_MakePoint(127.0296375, 37.5698946), ST_MakePoint(127.0305836, 37.5697140),
72         ST_MakePoint(127.0308563, 37.5697059), ST_MakePoint(127.0311691, 37.5697223),
73         ST_MakePoint(127.0316684, 37.5698362), ST_MakePoint(127.0319956, 37.5699425),
74         ST_MakePoint(127.0322451, 37.5700403), ST_MakePoint(127.0326045, 37.5701807),
75         ST_MakePoint(127.0336667, 37.5708695), ST_MakePoint(127.0346108, 37.5715157),
76         ST_MakePoint(127.0355345, 37.5719665), ST_MakePoint(127.0369990, 37.5723576),
77         ST_MakePoint(127.0397107, 37.5723302), ST_MakePoint(127.0405797, 37.5721941),
78         ST_MakePoint(127.0414891, 37.5719572), ST_MakePoint(127.0420925, 37.5717860),
79         ST_MakePoint(127.0437369, 37.5711469), ST_MakePoint(127.0446030, 37.5707060),
80         ST_MakePoint(127.0456793, 37.5700839), ST_MakePoint(127.0459314, 37.5698602),
81         ST_MakePoint(127.0461102, 37.5696683), ST_MakePoint(127.0463249, 37.5693912),
82         ST_MakePoint(127.0464969, 37.5690405), ST_MakePoint(127.0470598, 37.5680303),
83         ST_MakePoint(127.0519481, 37.5582118), ST_MakePoint(127.0521665, 37.5565427),

```

```

83         ST_MakePoint(127.0521311,37.5560825),ST_MakePoint(127.0520176,37.5555052),
84         ST_MakePoint(127.0517861,37.5543278),ST_MakePoint(127.0510543,37.5531646),
85         ST_MakePoint(127.0506623,37.5527466)
86     ]), 4326)::geography,
87     0, 0, NOW(), NOW(), NULL
88 ) ON CONFLICT (id) DO NOTHING;
89
90 INSERT INTO course_points(id, course_id, sequence, location) VALUES
91 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(126.9785975,37.5691438),4326)::geography),
92 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(126.9801836,37.5690719),4326)::geography),
93 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(126.9824530,37.5688944),4326)::geography),
94 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(126.9852214,37.5684594),4326)::geography),
95 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(126.9878120,37.5680958),4326)::geography),
96 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',6,ST_SetSRID(ST_MakePoint(126.9901819,37.5681529),4326)::geography),
97 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',7,ST_SetSRID(ST_MakePoint(126.9925302,37.5683299),4326)::geography),
98 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',8,ST_SetSRID(ST_MakePoint(126.9949392,37.5685097),4326)::geography),
99 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',9,ST_SetSRID(ST_MakePoint(126.9967778,37.5686863),4326)::geography),
100 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',10,ST_SetSRID(ST_MakePoint(127.0000376,37.5692406),4326)::geography),
101 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',11,ST_SetSRID(ST_MakePoint(127.0017919,37.5695761),4326)::geography),
102 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',12,ST_SetSRID(ST_MakePoint(127.0031510,37.5695969),4326)::geography),
103 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',13,ST_SetSRID(ST_MakePoint(127.0062675,37.5696422),4326)::geography),
104 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',14,ST_SetSRID(ST_MakePoint(127.0084170,37.5697478),4326)::geography),
105 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',15,ST_SetSRID(ST_MakePoint(127.0105226,37.5697636),4326)::geography),
106 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',16,ST_SetSRID(ST_MakePoint(127.0123521,37.5697464),4326)::geography),
107 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',17,ST_SetSRID(ST_MakePoint(127.0149692,37.5697417),4326)::geography),
108 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',18,ST_SetSRID(ST_MakePoint(127.0167624,37.5699516),4326)::geography),
109 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',19,ST_SetSRID(ST_MakePoint(127.0188358,37.5709441),4326)::geography),
110 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',20,ST_SetSRID(ST_MakePoint(127.0211015,37.5718150),4326)::geography),
111 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',21,ST_SetSRID(ST_MakePoint(127.0228377,37.5718142),4326)::geography),
112 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',22,ST_SetSRID(ST_MakePoint(127.0252390,37.5711508),4326)::geography),
113 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',23,ST_SetSRID(ST_MakePoint(127.0276164,37.5704501),4326)::geography),
114 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',24,ST_SetSRID(ST_MakePoint(127.0296375,37.5698946),4326)::geography),
115 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',25,ST_SetSRID(ST_MakePoint(127.0311691,37.5697223),4326)::geography),
116 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',26,ST_SetSRID(ST_MakePoint(127.0326045,37.5701807),4326)::geography),
117 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',27,ST_SetSRID(ST_MakePoint(127.0346108,37.5715157),4326)::geography),
118 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',28,ST_SetSRID(ST_MakePoint(127.0369990,37.5723576),4326)::geography),
119 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',29,ST_SetSRID(ST_MakePoint(127.0405797,37.5721941),4326)::geography),
120 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',30,ST_SetSRID(ST_MakePoint(127.0437369,37.5711469),4326)::geography),
121 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',31,ST_SetSRID(ST_MakePoint(127.0456793,37.5700839),4326)::geography),
122 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',32,ST_SetSRID(ST_MakePoint(127.0464969,37.5690405),4326)::geography),
123 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',33,ST_SetSRID(ST_MakePoint(127.0470598,37.5680303),4326)::geography),
124 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',34,ST_SetSRID(ST_MakePoint(127.0519481,37.5582118),4326)::geography),
125 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',35,ST_SetSRID(ST_MakePoint(127.0521311,37.5560825),4326)::geography),
126 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',36,ST_SetSRID(ST_MakePoint(127.0510543,37.5531646),4326)::geography),
127 (gen_random_uuid(),'a0000001-0000-0000-0000-000000000000',37,ST_SetSRID(ST_MakePoint(127.0506623,37.5527466),4326)::geography)
128 ON CONFLICT DO NOTHING;
129
130
131
132 -- 코스 B: 서울 남산 순환도로 (OSM Way 357958296 + 379943267)
133 -- 남산 동쪽 입구 → 남쪽 순환 → 서쪽 방면
134 -- 거리: ~7.5 km
135
136 INSERT INTO courses VALUES (
137     'a0000002-0000-0000-0000-000000000000',
138     '00000000-0000-0000-0000-000000000001', NULL,
139     '서울 남산 순환',
140     '남산 순환도로를 따라 달리는 코스. 남산타워 전망과 도심 스카이라인을 즐기며 완벽한 오르막 내리막을 경험합니다. OSM Way 357958296 + 379943267 실측 경로.',
141     'published', 7500, FALSE,
142     ST_SetSRID(ST_MakePoint(126.9989011, 37.5507982), 4326)::geography,
143     ST_SetSRID(ST_MakePoint(126.9821289, 37.5522744), 4326)::geography,
144     ST_SetSRID(ST_MakeLine(ARRAY[
145         ST_MakePoint(126.9989011,37.5507982),ST_MakePoint(126.9988216,37.5502447),
146         ST_MakePoint(126.9985709,37.5499927),ST_MakePoint(126.9985694,37.5497566),
147         ST_MakePoint(126.9987561,37.5494562),ST_MakePoint(126.9989519,37.5489339),
148         ST_MakePoint(126.9992149,37.5481181),ST_MakePoint(126.9993433,37.5476960),
149         ST_MakePoint(126.9993702,37.5474366),ST_MakePoint(126.9993308,37.5472553),
150         ST_MakePoint(126.9991633,37.5469907),ST_MakePoint(126.9989405,37.5468234),
151         ST_MakePoint(126.9986890,37.5461598),ST_MakePoint(126.9984822,37.5459228),
152         ST_MakePoint(126.9983524,37.5458604),ST_MakePoint(126.9982376,37.5457414),
153         ST_MakePoint(126.9980436,37.5454583),ST_MakePoint(126.9978131,37.5452664),
154         ST_MakePoint(126.9976655,37.5451587),ST_MakePoint(126.9972701,37.5447747),
155         ST_MakePoint(126.9969782,37.5445342),ST_MakePoint(126.9964029,37.5444340),
156         ST_MakePoint(126.9962850,37.5445668),ST_MakePoint(126.9960897,37.5450366),
157         ST_MakePoint(126.9957008,37.5453265),ST_MakePoint(126.9948974,37.5455369),
158         ST_MakePoint(126.9945822,37.5456638),ST_MakePoint(126.9943655,37.5457252),
159         ST_MakePoint(126.9940941,37.5457772),ST_MakePoint(126.9939038,37.5460321),
160         ST_MakePoint(126.9936940,37.5461937),ST_MakePoint(126.9933425,37.5462804),
161         ST_MakePoint(126.9932067,37.5463882),ST_MakePoint(126.9932215,37.5467648),
162         ST_MakePoint(126.9932728,37.5468915),ST_MakePoint(126.9935954,37.5473473),
163         ST_MakePoint(126.9936464,37.5474778),ST_MakePoint(126.9934623,37.5480397),
164         ST_MakePoint(126.9932892,37.5484178),ST_MakePoint(126.9931623,37.5487019),
165         ST_MakePoint(126.9931555,37.5489677),ST_MakePoint(126.9931877,37.5491952),
166         ST_MakePoint(126.9931313,37.5493441),ST_MakePoint(126.9927853,37.5497194),
167         ST_MakePoint(126.9924483,37.5500481),ST_MakePoint(126.9922497,37.5501681),
168         ST_MakePoint(126.9918502,37.5503222),ST_MakePoint(126.9915662,37.5505053),

```

```

169     ST_MakePoint(126.9912193,37.5508667),ST_MakePoint(126.9909463,37.5510556),
170     ST_MakePoint(126.9904472,37.5511710),ST_MakePoint(126.9901244,37.5512367),
171     ST_MakePoint(126.9898547,37.5512257),ST_MakePoint(126.9896460,37.5511656),
172     ST_MakePoint(126.9893689,37.5510442),ST_MakePoint(126.9889290,37.5508118),
173     ST_MakePoint(126.9886529,37.5506314),ST_MakePoint(126.9884204,37.5504291),
174     ST_MakePoint(126.9882043,37.5502746),ST_MakePoint(126.9878801,37.5500588),
175     ST_MakePoint(126.9877319,37.5499474),ST_MakePoint(126.9875476,37.5497355),
176     ST_MakePoint(126.9874676,37.5494954),ST_MakePoint(126.9874505,37.5492523),
177     ST_MakePoint(126.9874975,37.5489801),ST_MakePoint(126.9875877,37.5485925),
178     ST_MakePoint(126.9875749,37.5485073),ST_MakePoint(126.9874861,37.5483681),
179     ST_MakePoint(126.9868112,37.5482862),ST_MakePoint(126.9865521,37.5487029),
180     ST_MakePoint(126.9862979,37.5489606),ST_MakePoint(126.9856331,37.5493969),
181     ST_MakePoint(126.9854999,37.5494934),ST_MakePoint(126.9851827,37.5496590),
182     ST_MakePoint(126.9850134,37.5497515),ST_MakePoint(126.9849105,37.5498508),
183     ST_MakePoint(126.9849055,37.5500186),ST_MakePoint(126.9850689,37.5504460),
184     ST_MakePoint(126.9850168,37.5506477),ST_MakePoint(126.9847783,37.5507939),
185     ST_MakePoint(126.9843915,37.5510969),ST_MakePoint(126.9842972,37.5512409),
186     ST_MakePoint(126.9840814,37.5514363),ST_MakePoint(126.9836284,37.5515803),
187     ST_MakePoint(126.9832711,37.5517707),ST_MakePoint(126.9829934,37.5519537),
188     ST_MakePoint(126.9825696,37.5522204),ST_MakePoint(126.9824693,37.5522559),
189     ST_MakePoint(126.9823491,37.5522842),ST_MakePoint(126.9821289,37.5522744)
190     ], 4326)::geography,
191     0, 0, NOW(), NOW(), NULL
192 ) ON CONFLICT (id) DO NOTHING;
193
194 INSERT INTO course_points(id, course_id, sequence, location) VALUES
195 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 1, ST_SetSRID(ST_MakePoint(126.9989011,37.5507982), 4326)::geography),
196 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 2, ST_SetSRID(ST_MakePoint(126.9985709,37.5499927), 4326)::geography),
197 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 3, ST_SetSRID(ST_MakePoint(126.9989519,37.5489339), 4326)::geography),
198 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 4, ST_SetSRID(ST_MakePoint(126.9993433,37.5476960), 4326)::geography),
199 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 5, ST_SetSRID(ST_MakePoint(126.9991633,37.5469907), 4326)::geography),
200 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 6, ST_SetSRID(ST_MakePoint(126.9986890,37.5461598), 4326)::geography),
201 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 7, ST_SetSRID(ST_MakePoint(126.9980436,37.5454583), 4326)::geography),
202 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 8, ST_SetSRID(ST_MakePoint(126.9972701,37.5447747), 4326)::geography),
203 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 9, ST_SetSRID(ST_MakePoint(126.9964029,37.5444340), 4326)::geography),
204 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 10, ST_SetSRID(ST_MakePoint(126.9957008,37.5453265), 4326)::geography),
205 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 11, ST_SetSRID(ST_MakePoint(126.9948974,37.5455369), 4326)::geography),
206 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 12, ST_SetSRID(ST_MakePoint(126.9943655,37.5457252), 4326)::geography),
207 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 13, ST_SetSRID(ST_MakePoint(126.9939038,37.5460321), 4326)::geography),
208 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 14, ST_SetSRID(ST_MakePoint(126.9933425,37.5462804), 4326)::geography),
209 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 15, ST_SetSRID(ST_MakePoint(126.9932215,37.5467648), 4326)::geography),
210 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 16, ST_SetSRID(ST_MakePoint(126.9935954,37.5473473), 4326)::geography),
211 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 17, ST_SetSRID(ST_MakePoint(126.9934623,37.5480397), 4326)::geography),
212 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 18, ST_SetSRID(ST_MakePoint(126.9931555,37.5489677), 4326)::geography),
213 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 19, ST_SetSRID(ST_MakePoint(126.9931313,37.5493441), 4326)::geography),
214 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 20, ST_SetSRID(ST_MakePoint(126.9927853,37.5497194), 4326)::geography),
215 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 21, ST_SetSRID(ST_MakePoint(126.9922497,37.5501681), 4326)::geography),
216 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 22, ST_SetSRID(ST_MakePoint(126.9915662,37.5505053), 4326)::geography),
217 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 23, ST_SetSRID(ST_MakePoint(126.9909463,37.5510556), 4326)::geography),
218 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 24, ST_SetSRID(ST_MakePoint(126.9901244,37.5512367), 4326)::geography),
219 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 25, ST_SetSRID(ST_MakePoint(126.9893689,37.5510442), 4326)::geography),
220 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 26, ST_SetSRID(ST_MakePoint(126.9884204,37.5504291), 4326)::geography),
221 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 27, ST_SetSRID(ST_MakePoint(126.9877319,37.5499474), 4326)::geography),
222 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 28, ST_SetSRID(ST_MakePoint(126.9874505,37.5492523), 4326)::geography),
223 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 29, ST_SetSRID(ST_MakePoint(126.9874975,37.5489801), 4326)::geography),
224 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 30, ST_SetSRID(ST_MakePoint(126.9875877,37.5485925), 4326)::geography),
225 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 31, ST_SetSRID(ST_MakePoint(126.9868112,37.5482862), 4326)::geography),
226 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 32, ST_SetSRID(ST_MakePoint(126.9865521,37.5487029), 4326)::geography),
227 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 33, ST_SetSRID(ST_MakePoint(126.9856331,37.5493969), 4326)::geography),
228 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 34, ST_SetSRID(ST_MakePoint(126.9849055,37.5500186), 4326)::geography),
229 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 35, ST_SetSRID(ST_MakePoint(126.9847783,37.5507939), 4326)::geography),
230 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 36, ST_SetSRID(ST_MakePoint(126.9843915,37.5510969), 4326)::geography),
231 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 37, ST_SetSRID(ST_MakePoint(126.9836284,37.5515803), 4326)::geography),
232 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 38, ST_SetSRID(ST_MakePoint(126.9829934,37.5519537), 4326)::geography),
233 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 39, ST_SetSRID(ST_MakePoint(126.9824693,37.5522559), 4326)::geography),
234 (gen_random_uuid(), 'a0000002-0000-0000-0000-000000000000', 40, ST_SetSRID(ST_MakePoint(126.9821289,37.5522744), 4326)::geography)
235 ON CONFLICT DO NOTHING;
236
237
238
239 -- 코스 C: 반포 한강공원 (OSM Way 418249072, 109 pts, loop)
240 -- 거리: ~5.0 km
241
242 INSERT INTO courses VALUES (
243     'a0000003-0000-0000-0000-000000000000',
244     '00000000-0000-0000-0000-000000000001', NULL,
245     '서울 반포 한강공원',
246     '반포 한강공원 내부를 따라 달리는 코스. 달빛무지개분수와 잠수교를 조망하며 순환합니다. OSM Way 418249072 실측 경로.',
247     'published', 5000, TRUE,
248     ST_SetSRID(ST_MakePoint(126.9789608, 37.5048762), 4326)::geography,
249     ST_SetSRID(ST_MakePoint(126.9789608, 37.5048762), 4326)::geography,
250     ST_SetSRID(ST_MakeLine(ARRAY[
251         ST_MakePoint(126.9789608,37.5048762),ST_MakePoint(126.9786716,37.5053433),
252         ST_MakePoint(126.9789297,37.5054938),ST_MakePoint(126.9790243,37.5060853),
253         ST_MakePoint(126.9793998,37.5062502),ST_MakePoint(126.9816729,37.5063087),
254         ST_MakePoint(126.9821423,37.5062555),ST_MakePoint(126.9825634,37.5062343),

```

```

255 ST_MakePoint(126.9829953,37.5062853),ST_MakePoint(126.9834727,37.5063875),
256 ST_MakePoint(126.9838455,37.5065598),ST_MakePoint(126.9842452,37.5067407),
257 ST_MakePoint(126.9844966,37.5067944),ST_MakePoint(126.9849061,37.5068648),
258 ST_MakePoint(126.9854414,37.5068790),ST_MakePoint(126.9857070,37.5068556),
259 ST_MakePoint(126.9877964,37.5065534),ST_MakePoint(126.9880941,37.5065470),
260 ST_MakePoint(126.9884106,37.5065896),ST_MakePoint(126.9898510,37.5069343),
261 ST_MakePoint(126.9910354,37.5073235),ST_MakePoint(126.9912010,37.5073948),
262 ST_MakePoint(126.9913621,37.5075179),ST_MakePoint(126.9915773,37.5077649),
263 ST_MakePoint(126.9918348,37.5081223),ST_MakePoint(126.9922264,37.5084500),
264 ST_MakePoint(126.9933692,37.5093772),ST_MakePoint(126.9936388,37.5095960),
265 ST_MakePoint(126.9945475,37.5103609),ST_MakePoint(126.9948889,37.5106237),
266 ST_MakePoint(126.9955156,37.5110658),ST_MakePoint(126.9961622,37.5115599),
267 ST_MakePoint(126.9966799,37.5120152),ST_MakePoint(126.9969850,37.5122364),
268 ST_MakePoint(126.9973223,37.5124301),ST_MakePoint(126.9978105,37.5126237),
269 ST_MakePoint(126.9980746,37.5127205),ST_MakePoint(126.9985105,37.5128364),
270 ST_MakePoint(126.9995995,37.5130853),ST_MakePoint(127.0000045,37.5132066),
271 ST_MakePoint(127.0003223,37.5132949),ST_MakePoint(127.0007381,37.5134321),
272 ST_MakePoint(127.0010439,37.5135726),ST_MakePoint(127.0013416,37.5137896),
273 ST_MakePoint(127.0027792,37.5149767),ST_MakePoint(127.0030045,37.5151278),
274 ST_MakePoint(127.0034015,37.5152448),ST_MakePoint(127.0039447,37.5153203),
275 ST_MakePoint(127.0041847,37.5155001),ST_MakePoint(127.0057047,37.5148465),
276 ST_MakePoint(127.0034536,37.5131338),ST_MakePoint(127.0029630,37.5128055),
277 ST_MakePoint(127.0016353,37.5119589),ST_MakePoint(126.9995985,37.5109979),
278 ST_MakePoint(126.9989726,37.5107074),ST_MakePoint(126.9986137,37.5105408),
279 ST_MakePoint(126.9959318,37.5092960),ST_MakePoint(126.9937889,37.5081522),
280 ST_MakePoint(126.9919167,37.5072483),ST_MakePoint(126.9906434,37.5067249),
281 ST_MakePoint(126.9892905,37.5064744),ST_MakePoint(126.9882725,37.5063035),
282 ST_MakePoint(126.9878332,37.5061729),ST_MakePoint(126.9870975,37.5058262),
283 ST_MakePoint(126.9866834,37.5056915),ST_MakePoint(126.9841033,37.5052198),
284 ST_MakePoint(126.9825815,37.5050906),ST_MakePoint(126.9810596,37.5050777),
285 ST_MakePoint(126.9800615,37.5050788),ST_MakePoint(126.9797597,37.5047730),
286 ST_MakePoint(126.9797063,37.5043749),ST_MakePoint(126.9798096,37.5040028),
287 ST_MakePoint(126.9799210,37.5033485),ST_MakePoint(126.9801926,37.5027456),
288 ST_MakePoint(126.9798613,37.5032090),ST_MakePoint(126.9794138,37.5040797),
289 ST_MakePoint(126.9789608,37.5048762)
290 ]), 4326)::geography,
291 0, 0, NOW(), NOW(), NULL
292 ) ON CONFLICT (id) DO NOTHING;
293
294 INSERT INTO course_points(id, course_id, sequence, location) VALUES
295 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(126.9789608,37.5048762),4326)::geography),
296 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(126.9793998,37.5062502),4326)::geography),
297 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(126.9821423,37.5062555),4326)::geography),
298 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(126.9834727,37.5063875),4326)::geography),
299 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(126.9849061,37.5068648),4326)::geography),
300 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',6,ST_SetSRID(ST_MakePoint(126.9877964,37.5065534),4326)::geography),
301 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',7,ST_SetSRID(ST_MakePoint(126.9898510,37.5069343),4326)::geography),
302 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',8,ST_SetSRID(ST_MakePoint(126.9918348,37.5081223),4326)::geography),
303 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',9,ST_SetSRID(ST_MakePoint(126.9933692,37.5093772),4326)::geography),
304 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',10,ST_SetSRID(ST_MakePoint(126.9948889,37.5106237),4326)::geography),
305 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',11,ST_SetSRID(ST_MakePoint(126.9961622,37.5115599),4326)::geography),
306 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',12,ST_SetSRID(ST_MakePoint(126.9973223,37.5124301),4326)::geography),
307 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',13,ST_SetSRID(ST_MakePoint(126.9985105,37.5128364),4326)::geography),
308 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',14,ST_SetSRID(ST_MakePoint(127.0000045,37.5132066),4326)::geography),
309 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',15,ST_SetSRID(ST_MakePoint(127.0013416,37.5137896),4326)::geography),
310 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',16,ST_SetSRID(ST_MakePoint(127.0027792,37.5149767),4326)::geography),
311 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',17,ST_SetSRID(ST_MakePoint(127.0041847,37.5155001),4326)::geography),
312 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',18,ST_SetSRID(ST_MakePoint(127.0057047,37.5148465),4326)::geography),
313 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',19,ST_SetSRID(ST_MakePoint(127.0029630,37.5128055),4326)::geography),
314 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',20,ST_SetSRID(ST_MakePoint(127.0016353,37.5119589),4326)::geography),
315 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',21,ST_SetSRID(ST_MakePoint(126.9995985,37.5109979),4326)::geography),
316 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',22,ST_SetSRID(ST_MakePoint(126.9959318,37.5092960),4326)::geography),
317 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',23,ST_SetSRID(ST_MakePoint(126.9937889,37.5081522),4326)::geography),
318 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',24,ST_SetSRID(ST_MakePoint(126.9919167,37.5072483),4326)::geography),
319 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',25,ST_SetSRID(ST_MakePoint(126.9892905,37.5064744),4326)::geography),
320 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',26,ST_SetSRID(ST_MakePoint(126.9870975,37.5058262),4326)::geography),
321 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',27,ST_SetSRID(ST_MakePoint(126.9841033,37.5052198),4326)::geography),
322 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',28,ST_SetSRID(ST_MakePoint(126.9810596,37.5050777),4326)::geography),
323 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',29,ST_SetSRID(ST_MakePoint(126.9797597,37.5047730),4326)::geography),
324 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',30,ST_SetSRID(ST_MakePoint(126.9798096,37.5040028),4326)::geography),
325 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',31,ST_SetSRID(ST_MakePoint(126.9794138,37.5040797),4326)::geography),
326 (gen_random_uuid(),'a0000003-0000-0000-0000-000000000000',32,ST_SetSRID(ST_MakePoint(126.9789608,37.5048762),4326)::geography)
327 ON CONFLICT DO NOTHING;
328
329
330
331 -- 코스 D: 서울 양재천 (OSM Way 26505520, 86 pts)
332 -- 양재역 인근 ↪ 탄천 합류 방면 거리: ~9.5 km
333
334 INSERT INTO courses VALUES (
335 'a0000004-0000-0000-0000-000000000000',
336 '00000000-0000-0000-0000-000000000001', NULL,
337 '서울 양재천 코스',
338 '양재천을 따라 달리는 강변 코스. 버드나무 가로수와 잘 정비된 산책로가 인상적입니다. OSM Way 26505520 실측 경로.',
339 'published', 9500, FALSE,
340 ST_SetSRID(ST_MakePoint(127.0033660, 37.4519215), 4326)::geography,

```

```

341 ST_SetSRID(ST_MakePoint(127.0655686, 37.4900409), 4326)::geography,
342 ST_SetSRID(ST_MakeLine(ARRAY[
343 ST_MakePoint(127.0033660,37.4519215),ST_MakePoint(127.0033322,37.4522251),
344 ST_MakePoint(127.0034785,37.4525735),ST_MakePoint(127.0036529,37.4527209),
345 ST_MakePoint(127.0040355,37.4527786),ST_MakePoint(127.0042400,37.4529019),
346 ST_MakePoint(127.0046158,37.4530225),ST_MakePoint(127.0049958,37.4531202),
347 ST_MakePoint(127.0055064,37.4533325),ST_MakePoint(127.0059573,37.4535726),
348 ST_MakePoint(127.0063014,37.4537717),ST_MakePoint(127.0070058,37.4542661),
349 ST_MakePoint(127.0074164,37.4545385),ST_MakePoint(127.0078946,37.4548020),
350 ST_MakePoint(127.0083840,37.4549583),ST_MakePoint(127.0089466,37.4550922),
351 ST_MakePoint(127.0110449,37.4553557),ST_MakePoint(127.0119197,37.4555052),
352 ST_MakePoint(127.0123469,37.4555362),ST_MakePoint(127.0131545,37.4555745),
353 ST_MakePoint(127.0135924,37.4555690),ST_MakePoint(127.0141671,37.4554807),
354 ST_MakePoint(127.0154636,37.4552939),ST_MakePoint(127.0159223,37.4552798),
355 ST_MakePoint(127.0165018,37.4553468),ST_MakePoint(127.0171795,37.4555269),
356 ST_MakePoint(127.0178518,37.4556535),ST_MakePoint(127.0192655,37.4558731),
357 ST_MakePoint(127.0200636,37.4560219),ST_MakePoint(127.0210979,37.4563275),
358 ST_MakePoint(127.0223458,37.4568836),ST_MakePoint(127.0228195,37.4570525),
359 ST_MakePoint(127.0235901,37.4573513),ST_MakePoint(127.0244001,37.4576682),
360 ST_MakePoint(127.0252967,37.4580866),ST_MakePoint(127.0258240,37.4583624),
361 ST_MakePoint(127.0262414,37.4587139),ST_MakePoint(127.0269432,37.4598002),
362 ST_MakePoint(127.0275144,37.4605096),ST_MakePoint(127.0281186,37.4612860),
363 ST_MakePoint(127.0285815,37.4619134),ST_MakePoint(127.0292828,37.4628350),
364 ST_MakePoint(127.0299928,37.4637557),ST_MakePoint(127.0302198,37.4642055),
365 ST_MakePoint(127.0302676,37.4644698),ST_MakePoint(127.0290637,37.4674257),
366 ST_MakePoint(127.0290124,37.4676244),ST_MakePoint(127.0289886,37.4678529),
367 ST_MakePoint(127.0290728,37.4680292),ST_MakePoint(127.0293056,37.4682338),
368 ST_MakePoint(127.0309596,37.4703904),ST_MakePoint(127.0313842,37.4708881),
369 ST_MakePoint(127.0324246,37.4719086),ST_MakePoint(127.0331323,37.4725447),
370 ST_MakePoint(127.0335331,37.4727883),ST_MakePoint(127.0339339,37.4730116),
371 ST_MakePoint(127.0347183,37.4732823),ST_MakePoint(127.0356222,37.4734718),
372 ST_MakePoint(127.0376089,37.4738778),ST_MakePoint(127.0382402,37.4742751),
373 ST_MakePoint(127.0389960,37.4745551),ST_MakePoint(127.0394654,37.4747681),
374 ST_MakePoint(127.0399324,37.4749831),ST_MakePoint(127.0402547,37.4753759),
375 ST_MakePoint(127.0415453,37.4767184),ST_MakePoint(127.0417094,37.4769816),
376 ST_MakePoint(127.0419055,37.4772233),ST_MakePoint(127.0425784,37.4779741),
377 ST_MakePoint(127.0429664,37.4784311),ST_MakePoint(127.0435394,37.4791058),
378 ST_MakePoint(127.0439526,37.4796797),ST_MakePoint(127.0440884,37.4798264),
379 ST_MakePoint(127.0447350,37.4806060),ST_MakePoint(127.0452992,37.4810065),
380 ST_MakePoint(127.0455458,37.4811521),ST_MakePoint(127.0464238,37.4816713),
381 ST_MakePoint(127.0478631,37.4825040),ST_MakePoint(127.0497258,37.4835835),
382 ST_MakePoint(127.0515981,37.4846915),ST_MakePoint(127.0516967,37.4847518),
383 ST_MakePoint(127.0519497,37.4849066),ST_MakePoint(127.0536614,37.4859539),
384 ST_MakePoint(127.0550230,37.4867338),ST_MakePoint(127.0565044,37.4876448),
385 ST_MakePoint(127.0567652,37.4878147),ST_MakePoint(127.0576866,37.4883207),
386 ST_MakePoint(127.0580131,37.4884765),ST_MakePoint(127.0588095,37.4888073),
387 ST_MakePoint(127.0599384,37.4890531),ST_MakePoint(127.0655686,37.4900409)
388 ]), 4326)::geography,
389 0, 0, NOW(), NULL
390 ) ON CONFLICT (id) DO NOTHING;
391
392 INSERT INTO course_points(id, course_id, sequence, location) VALUES
393 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.0033660,37.4519215),4326)::geography),
394 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.0040355,37.4527786),4326)::geography),
395 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.0055064,37.4533325),4326)::geography),
396 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(127.0070058,37.4542661),4326)::geography),
397 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(127.0083840,37.4549583),4326)::geography),
398 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',6,ST_SetSRID(ST_MakePoint(127.0110449,37.4553557),4326)::geography),
399 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',7,ST_SetSRID(ST_MakePoint(127.0131545,37.4555745),4326)::geography),
400 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',8,ST_SetSRID(ST_MakePoint(127.0154636,37.4552939),4326)::geography),
401 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',9,ST_SetSRID(ST_MakePoint(127.0171795,37.4555269),4326)::geography),
402 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',10,ST_SetSRID(ST_MakePoint(127.0192655,37.4558731),4326)::geography),
403 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',11,ST_SetSRID(ST_MakePoint(127.0210979,37.4563275),4326)::geography),
404 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',12,ST_SetSRID(ST_MakePoint(127.0228195,37.4570525),4326)::geography),
405 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',13,ST_SetSRID(ST_MakePoint(127.0244001,37.4576682),4326)::geography),
406 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',14,ST_SetSRID(ST_MakePoint(127.0262414,37.4587139),4326)::geography),
407 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',15,ST_SetSRID(ST_MakePoint(127.0275144,37.4605096),4326)::geography),
408 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',16,ST_SetSRID(ST_MakePoint(127.0285815,37.4619134),4326)::geography),
409 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',17,ST_SetSRID(ST_MakePoint(127.0292828,37.4628350),4326)::geography),
410 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',18,ST_SetSRID(ST_MakePoint(127.0299928,37.4637557),4326)::geography),
411 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',19,ST_SetSRID(ST_MakePoint(127.0302676,37.4674257),4326)::geography),
412 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',20,ST_SetSRID(ST_MakePoint(127.0293056,37.4682338),4326)::geography),
413 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',21,ST_SetSRID(ST_MakePoint(127.0313842,37.4708881),4326)::geography),
414 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',22,ST_SetSRID(ST_MakePoint(127.0331323,37.4725447),4326)::geography),
415 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',23,ST_SetSRID(ST_MakePoint(127.0347183,37.4732823),4326)::geography),
416 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',24,ST_SetSRID(ST_MakePoint(127.0376089,37.4738778),4326)::geography),
417 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',25,ST_SetSRID(ST_MakePoint(127.0394654,37.4747681),4326)::geography),
418 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',26,ST_SetSRID(ST_MakePoint(127.0415453,37.4767184),4326)::geography),
419 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',27,ST_SetSRID(ST_MakePoint(127.0435394,37.4791058),4326)::geography),
420 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',28,ST_SetSRID(ST_MakePoint(127.0447350,37.4806060),4326)::geography),
421 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',29,ST_SetSRID(ST_MakePoint(127.0464238,37.4816713),4326)::geography),
422 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',30,ST_SetSRID(ST_MakePoint(127.0478631,37.4825040),4326)::geography),
423 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',31,ST_SetSRID(ST_MakePoint(127.0497258,37.4835835),4326)::geography),
424 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',32,ST_SetSRID(ST_MakePoint(127.0519497,37.4849066),4326)::geography),
425 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',33,ST_SetSRID(ST_MakePoint(127.0536614,37.4859539),4326)::geography),
426 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',34,ST_SetSRID(ST_MakePoint(127.0550230,37.4867338),4326)::geography),

```

```

427 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',35,ST_SetSRID(ST_MakePoint(127.0567652,37.4878147),4326)::geography),
428 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',36,ST_SetSRID(ST_MakePoint(127.0580131,37.4884765),4326)::geography),
429 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',37,ST_SetSRID(ST_MakePoint(127.0599384,37.4890531),4326)::geography),
430 (gen_random_uuid(),'a0000004-0000-0000-0000-000000000000',38,ST_SetSRID(ST_MakePoint(127.0655686,37.4900409),4326)::geography)
431 ON CONFLICT DO NOTHING;
432
433
434 -- _____
435 -- 코스 E: 부산 해운대 해변 (OSM Way 107531972, 40 pts, loop)
436 -- 거리: ~2.5 km
437 -- _____
438 INSERT INTO courses VALUES (
439     'a0000005-0000-0000-0000-000000000000',
440     '00000000-0000-0000-0000-000000000000', NULL,
441     '부산 해운대 해변 코스',
442     '해운대 해수욕장 해변을 따라 달리는 코스. 동백섬까지 이어지는 백사장과 마린시티 야경을 즐깁니다. OSM Way 107531972 실측 경로.',
443     'published', 2500, TRUE,
444     ST_SetSRID(ST_MakePoint(129.1622327, 35.1592391), 4326)::geography,
445     ST_SetSRID(ST_MakePoint(129.1622327, 35.1592391), 4326)::geography,
446     ST_SetSRID(ST_MakeLine(ARRAY[
447         ST_MakePoint(129.1622327,35.1592391),ST_MakePoint(129.1623125,35.1592232),
448         ST_MakePoint(129.1624084,35.1592248),ST_MakePoint(129.1625103,35.1592578),
449         ST_MakePoint(129.1626069,35.1593367),ST_MakePoint(129.1626873,35.1593937),
450         ST_MakePoint(129.1634920,35.1594990),ST_MakePoint(129.1647151,35.1596393),
451         ST_MakePoint(129.1679445,35.1595604),ST_MakePoint(129.1684263,35.1595284),
452         ST_MakePoint(129.1696193,35.1592851),ST_MakePoint(129.1698984,35.1591295),
453         ST_MakePoint(129.1692216,35.1586275),ST_MakePoint(129.1690404,35.1587919),
454         ST_MakePoint(129.1685307,35.1589040),ST_MakePoint(129.1677853,35.1588973),
455         ST_MakePoint(129.1663901,35.1588788),ST_MakePoint(129.1653382,35.1587310),
456         ST_MakePoint(129.1631374,35.1585189),ST_MakePoint(129.1610843,35.1581019),
457         ST_MakePoint(129.1590037,35.1576022),ST_MakePoint(129.1571193,35.1569709),
458         ST_MakePoint(129.1557987,35.1564239),ST_MakePoint(129.1549044,35.1558795),
459         ST_MakePoint(129.1547728,35.1556784),ST_MakePoint(129.1546597,35.1556554),
460         ST_MakePoint(129.1545308,35.1557847),ST_MakePoint(129.1543963,35.1562062),
461         ST_MakePoint(129.1544186,35.1563277),ST_MakePoint(129.1544317,35.1565279),
462         ST_MakePoint(129.1547480,35.1567272),ST_MakePoint(129.1550270,35.1570517),
463         ST_MakePoint(129.1552201,35.1572008),ST_MakePoint(129.1569151,35.1579596),
464         ST_MakePoint(129.1579452,35.1584025),ST_MakePoint(129.1586556,35.1586095),
465         ST_MakePoint(129.1593747,35.1587411),ST_MakePoint(129.1614219,35.1591353),
466         ST_MakePoint(129.1621183,35.1592743),ST_MakePoint(129.1622327,35.1592391)
467     ]), 4326)::geography,
468     0, 0, NOW(), NOW(), NULL
469 ) ON CONFLICT (id) DO NOTHING;
470
471 INSERT INTO course_points(id, course_id, sequence, location) VALUES
472 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(129.1622327,35.1592391),4326)::geography),
473 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(129.1634920,35.1594990),4326)::geography),
474 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(129.1647151,35.1596393),4326)::geography),
475 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(129.1679445,35.1595604),4326)::geography),
476 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(129.1696193,35.1592851),4326)::geography),
477 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',6,ST_SetSRID(ST_MakePoint(129.1692216,35.1586275),4326)::geography),
478 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',7,ST_SetSRID(ST_MakePoint(129.1677853,35.1588973),4326)::geography),
479 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',8,ST_SetSRID(ST_MakePoint(129.1653382,35.1587310),4326)::geography),
480 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',9,ST_SetSRID(ST_MakePoint(129.1631374,35.1585189),4326)::geography),
481 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',10,ST_SetSRID(ST_MakePoint(129.1610843,35.1581019),4326)::geography),
482 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',11,ST_SetSRID(ST_MakePoint(129.1590037,35.1576022),4326)::geography),
483 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',12,ST_SetSRID(ST_MakePoint(129.1571193,35.1569709),4326)::geography),
484 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',13,ST_SetSRID(ST_MakePoint(129.1549044,35.1558795),4326)::geography),
485 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',14,ST_SetSRID(ST_MakePoint(129.1545308,35.1557847),4326)::geography),
486 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',15,ST_SetSRID(ST_MakePoint(129.1544317,35.1565279),4326)::geography),
487 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',16,ST_SetSRID(ST_MakePoint(129.1550270,35.1570517),4326)::geography),
488 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',17,ST_SetSRID(ST_MakePoint(129.1569151,35.1579596),4326)::geography),
489 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',18,ST_SetSRID(ST_MakePoint(129.1586556,35.1586095),4326)::geography),
490 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',19,ST_SetSRID(ST_MakePoint(129.1614219,35.1591353),4326)::geography),
491 (gen_random_uuid(),'a0000005-0000-0000-0000-000000000000',20,ST_SetSRID(ST_MakePoint(129.1622327,35.1592391),4326)::geography)
492 ON CONFLICT DO NOTHING;

```

```

1  -- V15: 실제 Samsung Health GPS 기록 기반 충주 강변 코스 2개
2  -- 출처: sample/20260508_160515.gpx (10K), sample/20260420_173612.gpx (20K)
3  -- RDP 알고리즘으로 다운샘플링 (epsilon=8m)
4
5  INSERT INTO courses VALUES (
6      'b00000001-0000-0000-0000-000000000000',
7      '000000000-0000-0000-0000-000000000001', NULL,
8      '충주 강변 코스 10K',
9      '충주 남한강 강변을 따라 달리는 10km 루프 코스. 실제 GPS 기록으로 등록된 코스로 강변 자전거도로와 산책로를 따라 달립니다.',
10     'published', 9964, TRUE,
11     ST_SetSRID(ST_MakePoint(127.8676214,36.9674304),4326)::geography,
12     ST_SetSRID(ST_MakePoint(127.8679189,36.9673725),4326)::geography,
13     ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.8676214,36.9674304),ST_MakePoint(127.8680645,36.9674332),ST_MakePoint(127.8717401,36.
9667019),ST_MakePoint(127.874997,36.9658335),ST_MakePoint(127.8757984,36.9658497),ST_MakePoint(127.8765395,36.9660331),ST_MakePoint(127.87696
08,36.9662355),ST_MakePoint(127.8774718,36.966703),ST_MakePoint(127.87819,36.967761),ST_MakePoint(127.8783073,36.9685161),ST_MakePoint(127.87
74918,36.9712207),ST_MakePoint(127.8781028,36.9725987),ST_MakePoint(127.8790357,36.974144),ST_MakePoint(127.8795143,36.9744747),ST_MakePoint
(127.8802634,36.9747527),ST_MakePoint(127.8817589,36.9747893),ST_MakePoint(127.882868,36.9746792),ST_MakePoint(127.8848963,36.9748414),ST_Mak
ePoint(127.8854082,36.9749505),ST_MakePoint(127.8858125,36.97536),ST_MakePoint(127.8857353,36.9768862),ST_MakePoint(127.8858742,36.9779729),S
T_MakePoint(127.8862222,36.9789362),ST_MakePoint(127.8870634,36.9804271),ST_MakePoint(127.8876236,36.9822982),ST_MakePoint(127.8878134,36.983
415),ST_MakePoint(127.8876359,36.9840076),ST_MakePoint(127.8858138,36.9874974),ST_MakePoint(127.8862386,36.9890027),ST_MakePoint(127.8859433,
36.9900608),ST_MakePoint(127.885999,36.9907575),ST_MakePoint(127.8846457,36.9916587),ST_MakePoint(127.8827511,36.9924936),ST_MakePoint(127.88
15887,36.9932708),ST_MakePoint(127.8858358,36.9979824),ST_MakePoint(127.8800244,36.9951191),ST_MakePoint(127.8811818,36.9936555),ST_MakePoint(127.8819836,36.9929803),ST_MakePoint
(127.8827987,36.9924584),ST_MakePoint(127.8844476,36.9917584),ST_MakePoint(127.8860628,36.9906929),ST_MakePoint(127.8859552,36.9898976),ST_Ma
kePoint(127.8862022,36.9889789),ST_MakePoint(127.8857783,36.9875391),ST_MakePoint(127.8875617,36.9839991),ST_MakePoint(127.8877286,36.983301
3),ST_MakePoint(127.8877023,36.9827539),ST_MakePoint(127.8872751,36.9811695),ST_MakePoint(127.8868041,36.9799336),ST_MakePoint(127.8860827,3
6.9786981),ST_MakePoint(127.8858358,36.9779824),ST_MakePoint(127.8858359,36.9753262),ST_MakePoint(127.8855518,36.9749082),ST_MakePoint(127.88
49932,36.974766),ST_MakePoint(127.8829517,36.9746385),ST_MakePoint(127.8802655,36.9746899),ST_MakePoint(127.8796744,36.974501),ST_MakePoint(1
27.8788469,36.9738992),ST_MakePoint(127.8774965,36.9711346),ST_MakePoint(127.8782189,36.9684896),ST_MakePoint(127.8781884,36.9678261),ST_Make
Point(127.8774235,36.9666656),ST_MakePoint(127.8764351,36.9659188),ST_MakePoint(127.8758954,36.9657556),ST_MakePoint(127.8750355,36.96577),ST
_MakePoint(127.8716614,36.9666306),ST_MakePoint(127.8679189,36.9673725) ]),4326)::geography,
14     0, 0, NOW(), NOW(), NULL
15 ) ON CONFLICT (id) DO NOTHING;
16
17 INSERT INTO course_points(id,course_id,sequence,location) VALUES
18 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.8676214,36.9674304),4326)::geography),
19 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.8680645,36.9674332),4326)::geography),
20 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.8717401,36.9667019),4326)::geography),
21 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(127.874997,36.9658335),4326)::geography),
22 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(127.8757984,36.9658497),4326)::geography),
23 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',6,ST_SetSRID(ST_MakePoint(127.8765395,36.9660331),4326)::geography),
24 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',7,ST_SetSRID(ST_MakePoint(127.8769608,36.9662355),4326)::geography),
25 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',8,ST_SetSRID(ST_MakePoint(127.8774718,36.966703),4326)::geography),
26 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',9,ST_SetSRID(ST_MakePoint(127.87819,36.967761),4326)::geography),
27 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',10,ST_SetSRID(ST_MakePoint(127.8783073,36.9685161),4326)::geography),
28 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',11,ST_SetSRID(ST_MakePoint(127.8774918,36.9712207),4326)::geography),
29 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',12,ST_SetSRID(ST_MakePoint(127.8781028,36.9725987),4326)::geography),
30 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',13,ST_SetSRID(ST_MakePoint(127.8790357,36.974144),4326)::geography),
31 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',14,ST_SetSRID(ST_MakePoint(127.8795143,36.9744747),4326)::geography),
32 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',15,ST_SetSRID(ST_MakePoint(127.8802634,36.9747527),4326)::geography),
33 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',16,ST_SetSRID(ST_MakePoint(127.8817589,36.9747893),4326)::geography),
34 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',17,ST_SetSRID(ST_MakePoint(127.882868,36.9746792),4326)::geography),
35 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',18,ST_SetSRID(ST_MakePoint(127.8848963,36.9748414),4326)::geography),
36 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',19,ST_SetSRID(ST_MakePoint(127.8854082,36.9749505),4326)::geography),
37 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',20,ST_SetSRID(ST_MakePoint(127.8858125,36.97536),4326)::geography),
38 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',21,ST_SetSRID(ST_MakePoint(127.8857353,36.9768862),4326)::geography),
39 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',22,ST_SetSRID(ST_MakePoint(127.8858742,36.9779729),4326)::geography),
40 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',23,ST_SetSRID(ST_MakePoint(127.8862222,36.9789362),4326)::geography),
41 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',24,ST_SetSRID(ST_MakePoint(127.8870634,36.9804271),4326)::geography),
42 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',25,ST_SetSRID(ST_MakePoint(127.8876236,36.9822982),4326)::geography),
43 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',26,ST_SetSRID(ST_MakePoint(127.8878134,36.983415),4326)::geography),
44 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',27,ST_SetSRID(ST_MakePoint(127.8876359,36.9840076),4326)::geography),
45 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',28,ST_SetSRID(ST_MakePoint(127.8858138,36.9874974),4326)::geography),
46 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',29,ST_SetSRID(ST_MakePoint(127.8862386,36.9890027),4326)::geography),
47 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',30,ST_SetSRID(ST_MakePoint(127.8859433,36.9900608),4326)::geography),
48 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',31,ST_SetSRID(ST_MakePoint(127.885999,36.9907575),4326)::geography),
49 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',32,ST_SetSRID(ST_MakePoint(127.8846457,36.9916587),4326)::geography),
50 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',33,ST_SetSRID(ST_MakePoint(127.8827511,36.9924936),4326)::geography),
51 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',34,ST_SetSRID(ST_MakePoint(127.8815887,36.9932708),4326)::geography),
52 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',35,ST_SetSRID(ST_MakePoint(127.8800244,36.9951191),4326)::geography),
53 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',36,ST_SetSRID(ST_MakePoint(127.8811818,36.9936555),4326)::geography),
54 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',37,ST_SetSRID(ST_MakePoint(127.8819836,36.9929803),4326)::geography),
55 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',38,ST_SetSRID(ST_MakePoint(127.8827987,36.9924584),4326)::geography),
56 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',39,ST_SetSRID(ST_MakePoint(127.8844476,36.9917584),4326)::geography),
57 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',40,ST_SetSRID(ST_MakePoint(127.8860628,36.9906929),4326)::geography),
58 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',41,ST_SetSRID(ST_MakePoint(127.8859552,36.9898976),4326)::geography),
59 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',42,ST_SetSRID(ST_MakePoint(127.8862022,36.9889789),4326)::geography),
60 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',43,ST_SetSRID(ST_MakePoint(127.8857783,36.9875391),4326)::geography),
61 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',44,ST_SetSRID(ST_MakePoint(127.8875617,36.9839991),4326)::geography),
62 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',45,ST_SetSRID(ST_MakePoint(127.8877286,36.9833013),4326)::geography),
63 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',46,ST_SetSRID(ST_MakePoint(127.8877023,36.9827539),4326)::geography),
64 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',47,ST_SetSRID(ST_MakePoint(127.8872751,36.9811695),4326)::geography),
65 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',48,ST_SetSRID(ST_MakePoint(127.8868041,36.9799336),4326)::geography),
66 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',49,ST_SetSRID(ST_MakePoint(127.8860827,36.9786981),4326)::geography),

```

```

67 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',50,ST_SetSRID(ST_MakePoint(127.8858358,36.9779824),4326)::geography)
68 ON CONFLICT DO NOTHING;
69
70 INSERT INTO course_points(id,course_id,sequence,location) VALUES
71 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',51,ST_SetSRID(ST_MakePoint(127.8858359,36.9753262),4326)::geography),
72 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',52,ST_SetSRID(ST_MakePoint(127.8855518,36.9749082),4326)::geography),
73 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',53,ST_SetSRID(ST_MakePoint(127.8849932,36.974766),4326)::geography),
74 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',54,ST_SetSRID(ST_MakePoint(127.8829517,36.9746385),4326)::geography),
75 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',55,ST_SetSRID(ST_MakePoint(127.8802655,36.9746899),4326)::geography),
76 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',56,ST_SetSRID(ST_MakePoint(127.8796744,36.974501),4326)::geography),
77 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',57,ST_SetSRID(ST_MakePoint(127.8788469,36.9738992),4326)::geography),
78 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',58,ST_SetSRID(ST_MakePoint(127.8774965,36.9711346),4326)::geography),
79 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',59,ST_SetSRID(ST_MakePoint(127.8782189,36.9684896),4326)::geography),
80 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',60,ST_SetSRID(ST_MakePoint(127.8781884,36.9678261),4326)::geography),
81 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',61,ST_SetSRID(ST_MakePoint(127.8774235,36.9666656),4326)::geography),
82 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',62,ST_SetSRID(ST_MakePoint(127.8764351,36.9659188),4326)::geography),
83 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',63,ST_SetSRID(ST_MakePoint(127.8758954,36.9657556),4326)::geography),
84 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',64,ST_SetSRID(ST_MakePoint(127.8750355,36.96577),4326)::geography),
85 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',65,ST_SetSRID(ST_MakePoint(127.8716614,36.9666306),4326)::geography),
86 (gen_random_uuid(),'b0000001-0000-0000-0000-000000000000',66,ST_SetSRID(ST_MakePoint(127.8679189,36.9673725),4326)::geography)
87 ON CONFLICT DO NOTHING;
88
89 INSERT INTO courses VALUES (
90     'b0000002-0000-0000-0000-000000000000',
91     '00000000-0000-0000-0000-000000000001', NULL,
92     '충주 강변 코스 20K',
93     '충주 남한강 강변을 따라 달리는 20km 루프 코스. 강변 전 구간을 종주하는 장거리 코스로 실제 GPS 기록으로 등록됐습니다.',
94     'published', 19836, TRUE,
95     ST_SetSRID(ST_MakePoint(127.867545,36.967504),4326)::geography,
96     ST_SetSRID(ST_MakePoint(127.8677535,36.96743),4326)::geography,
97     ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(127.867545,36.967504),ST_MakePoint(127.8716634,36.9666372),ST_MakePoint(127.8751502,36.96
5741),ST_MakePoint(127.876175,36.9658045),ST_MakePoint(127.8768692,36.9660729),ST_MakePoint(127.8775169,36.9666407),ST_MakePoint(127.8782106,
36.967713),ST_MakePoint(127.8782711,36.9685163),ST_MakePoint(127.8778205,36.970105),ST_MakePoint(127.8774095,36.9710336),ST_MakePoint(127.878
4355,36.9732887),ST_MakePoint(127.8788621,36.9738946),ST_MakePoint(127.8793344,36.9742686),ST_MakePoint(127.8803285,36.9747158),ST_MakePoint
(127.8824917,36.9746238),ST_MakePoint(127.8850518,36.9747396),ST_MakePoint(127.885453,36.9748224),ST_MakePoint(127.8856461,36.9750227),ST_Mak
ePoint(127.8858025,36.9754285),ST_MakePoint(127.8857157,36.9767724),ST_MakePoint(127.8858134,36.9778386),ST_MakePoint(127.8861846,36.978923
1),ST_MakePoint(127.8869711,36.9803253),ST_MakePoint(127.88761,36.9824283),ST_MakePoint(127.8877454,36.983379),ST_MakePoint(127.8874443,36.98
42767),ST_MakePoint(127.8857476,36.9875212),ST_MakePoint(127.8861356,36.9890158),ST_MakePoint(127.8859014,36.989674),ST_MakePoint(127.885928
7,36.9907543),ST_MakePoint(127.8844968,36.9916984),ST_MakePoint(127.8826416,36.9925392),ST_MakePoint(127.8814413,36.993306),ST_MakePoint(127.
8798494,36.9951517),ST_MakePoint(127.8787385,36.9959819),ST_MakePoint(127.8783764,36.9967065),ST_MakePoint(127.8764619,36.9973596),ST_MakePoi
nt(127.8750412,36.9986237),ST_MakePoint(127.8740782,37.0007338),ST_MakePoint(127.8743279,37.0020931),ST_MakePoint(127.873983,37.003599),ST_Ma
kePoint(127.8728733,37.0048427),ST_MakePoint(127.8731446,37.0051439),ST_MakePoint(127.8729963,37.0055156),ST_MakePoint(127.8697596,37.011835
3),ST_MakePoint(127.8684887,37.0145458),ST_MakePoint(127.8677294,37.0159015),ST_MakePoint(127.8673853,37.0159918),ST_MakePoint(127.8671648,3
7.0167008),ST_MakePoint(127.8672565,37.0168914),ST_MakePoint(127.8670259,37.0174247),ST_MakePoint(127.8643912,37.0224971),ST_MakePoint(127.86
42146,37.0227067),ST_MakePoint(127.863987,37.0226807),ST_MakePoint(127.8635911,37.0217118),ST_MakePoint(127.8630571,37.0211752),ST_MakePoint
(127.8627638,37.0211629),ST_MakePoint(127.8623961,37.0219171),ST_MakePoint(127.8604924,37.0246173),ST_MakePoint(127.8585971,37.0279671),ST_Ma
kePoint(127.8579317,37.028752),ST_MakePoint(127.8578543,37.0293228),ST_MakePoint(127.8574575,37.0301383),ST_MakePoint(127.8579187,37.028747
5),ST_MakePoint(127.8586059,37.027975),ST_MakePoint(127.8601476,37.0251306),ST_MakePoint(127.8623128,37.021923),ST_MakePoint(127.8628389,37.0
209572),ST_MakePoint(127.8630271,37.0209264),ST_MakePoint(127.8634295,37.021606),ST_MakePoint(127.8636909,37.0217203),ST_MakePoint(127.864014
2,37.0226283),ST_MakePoint(127.8642121,37.0226791),ST_MakePoint(127.8643673,37.0225666),ST_MakePoint(127.8671632,37.0169033),ST_MakePoint(12
7.8673354,37.0160421),ST_MakePoint(127.8676857,37.0159325),ST_MakePoint(127.8691409,37.0128559),ST_MakePoint(127.8730462,37.0053011),ST_MakeP
oint(127.8730905,37.0051153),ST_MakePoint(127.8728744,37.0049471),ST_MakePoint(127.8728851,37.0047609),ST_MakePoint(127.8740206,37.003528),ST
_MakePoint(127.8742955,37.0021438),ST_MakePoint(127.8740748,37.0006763),ST_MakePoint(127.8747003,36.9996165),ST_MakePoint(127.8749849,36.9987
267),ST_MakePoint(127.8763977,36.9973774),ST_MakePoint(127.8783678,36.9967313),ST_MakePoint(127.8787467,36.9960157),ST_MakePoint(127.8801251,
36.9949763),ST_MakePoint(127.8814962,36.9932377),ST_MakePoint(127.8826261,36.9924802),ST_MakePoint(127.8843207,36.9917255),ST_MakePoint(127.8
860193,36.9906774),ST_MakePoint(127.8859049,36.9898973),ST_MakePoint(127.8859597,36.9893427),ST_MakePoint(127.8861468,36.9889543),ST_MakePoin
t(127.8857229,36.9874558),ST_MakePoint(127.8877276,36.9834953),ST_MakePoint(127.8877026,36.9828349),ST_MakePoint(127.8872936,36.9812638),ST_M
akePoint(127.8858157,36.9779647),ST_MakePoint(127.8857177,36.9771062),ST_MakePoint(127.8858113,36.9753628),ST_MakePoint(127.8856844,36.97505
6),ST_MakePoint(127.8854498,36.9748214),ST_MakePoint(127.8850437,36.9747132),ST_MakePoint(127.8802246,36.9746917),ST_MakePoint(127.8795023,3
6.97438),ST_MakePoint(127.8789467,36.9739885),ST_MakePoint(127.8786575,36.9735923),ST_MakePoint(127.8775636,36.9710314),ST_MakePoint(127.8782
523,36.968452),ST_MakePoint(127.8782006,36.9677661),ST_MakePoint(127.8778153,36.9670646),ST_MakePoint(127.8769038,36.9661739),ST_MakePoint(12
7.875857,36.9657468),ST_MakePoint(127.8749093,36.9657246),ST_MakePoint(127.8726919,36.9663696),ST_MakePoint(127.8677535,36.96743)]),4326)::ge
ography,
98     0, 0, NOW(), NOW(), NULL
99 ) ON CONFLICT (id) DO NOTHING;
100
101 INSERT INTO course_points(id,course_id,sequence,location) VALUES
102 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',1,ST_SetSRID(ST_MakePoint(127.867545,36.967504),4326)::geography),
103 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',2,ST_SetSRID(ST_MakePoint(127.8716634,36.9666372),4326)::geography),
104 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',3,ST_SetSRID(ST_MakePoint(127.8751502,36.965741),4326)::geography),
105 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',4,ST_SetSRID(ST_MakePoint(127.876175,36.9658045),4326)::geography),
106 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',5,ST_SetSRID(ST_MakePoint(127.8768692,36.9660729),4326)::geography),
107 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',6,ST_SetSRID(ST_MakePoint(127.8775169,36.9666407),4326)::geography),
108 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',7,ST_SetSRID(ST_MakePoint(127.8782106,36.967713),4326)::geography),
109 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',8,ST_SetSRID(ST_MakePoint(127.8782711,36.9685163),4326)::geography),
110 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',9,ST_SetSRID(ST_MakePoint(127.8778205,36.970105),4326)::geography),
111 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',10,ST_SetSRID(ST_MakePoint(127.8774095,36.9710336),4326)::geography),
112 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',11,ST_SetSRID(ST_MakePoint(127.8784355,36.9732887),4326)::geography),
113 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',12,ST_SetSRID(ST_MakePoint(127.8788621,36.9738946),4326)::geography),
114 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',13,ST_SetSRID(ST_MakePoint(127.8793344,36.9742686),4326)::geography),
115 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',14,ST_SetSRID(ST_MakePoint(127.8803285,36.9747158),4326)::geography),
116 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',15,ST_SetSRID(ST_MakePoint(127.8824917,36.9746238),4326)::geography),
117 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',16,ST_SetSRID(ST_MakePoint(127.8850518,36.9747396),4326)::geography),
118 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',17,ST_SetSRID(ST_MakePoint(127.885453,36.9748224),4326)::geography),
119 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',18,ST_SetSRID(ST_MakePoint(127.8856461,36.9750227),4326)::geography),
120 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',19,ST_SetSRID(ST_MakePoint(127.8858025,36.9754285),4326)::geography),

```



```

121 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 20, ST_SetSRID(ST_MakePoint(127.8857157, 36.9767724), 4326)::geography),
122 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 21, ST_SetSRID(ST_MakePoint(127.8858134, 36.9778386), 4326)::geography),
123 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 22, ST_SetSRID(ST_MakePoint(127.8861846, 36.9789231), 4326)::geography),
124 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 23, ST_SetSRID(ST_MakePoint(127.8869711, 36.9803253), 4326)::geography),
125 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 24, ST_SetSRID(ST_MakePoint(127.88761, 36.9824283), 4326)::geography),
126 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 25, ST_SetSRID(ST_MakePoint(127.8877454, 36.983379), 4326)::geography),
127 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 26, ST_SetSRID(ST_MakePoint(127.8874443, 36.9842767), 4326)::geography),
128 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 27, ST_SetSRID(ST_MakePoint(127.8857476, 36.9875212), 4326)::geography),
129 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 28, ST_SetSRID(ST_MakePoint(127.8861356, 36.9890158), 4326)::geography),
130 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 29, ST_SetSRID(ST_MakePoint(127.8859014, 36.989674), 4326)::geography),
131 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 30, ST_SetSRID(ST_MakePoint(127.8859287, 36.9897543), 4326)::geography),
132 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 31, ST_SetSRID(ST_MakePoint(127.8849468, 36.9916984), 4326)::geography),
133 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 32, ST_SetSRID(ST_MakePoint(127.8826416, 36.9925392), 4326)::geography),
134 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 33, ST_SetSRID(ST_MakePoint(127.8814413, 36.993306), 4326)::geography),
135 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 34, ST_SetSRID(ST_MakePoint(127.8798494, 36.9951517), 4326)::geography),
136 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 35, ST_SetSRID(ST_MakePoint(127.8787385, 36.9959819), 4326)::geography),
137 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 36, ST_SetSRID(ST_MakePoint(127.8783764, 36.9967065), 4326)::geography),
138 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 37, ST_SetSRID(ST_MakePoint(127.8764619, 36.9973596), 4326)::geography),
139 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 38, ST_SetSRID(ST_MakePoint(127.8750412, 36.9986237), 4326)::geography),
140 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 39, ST_SetSRID(ST_MakePoint(127.8740702, 37.0007338), 4326)::geography),
141 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 40, ST_SetSRID(ST_MakePoint(127.8743279, 37.0020931), 4326)::geography),
142 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 41, ST_SetSRID(ST_MakePoint(127.873983, 37.003599), 4326)::geography),
143 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 42, ST_SetSRID(ST_MakePoint(127.8728733, 37.0048427), 4326)::geography),
144 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 43, ST_SetSRID(ST_MakePoint(127.8731446, 37.0051439), 4326)::geography),
145 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 44, ST_SetSRID(ST_MakePoint(127.8729963, 37.0055156), 4326)::geography),
146 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 45, ST_SetSRID(ST_MakePoint(127.8697596, 37.0118353), 4326)::geography),
147 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 46, ST_SetSRID(ST_MakePoint(127.8684887, 37.0145458), 4326)::geography),
148 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 47, ST_SetSRID(ST_MakePoint(127.8677294, 37.0159015), 4326)::geography),
149 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 48, ST_SetSRID(ST_MakePoint(127.8673853, 37.0159918), 4326)::geography),
150 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 49, ST_SetSRID(ST_MakePoint(127.8671648, 37.0167008), 4326)::geography),
151 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 50, ST_SetSRID(ST_MakePoint(127.8672565, 37.0168914), 4326)::geography),
152 ON CONFLICT DO NOTHING;
153
154 INSERT INTO course_points(id,course_id,sequence,location) VALUES
155 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 51, ST_SetSRID(ST_MakePoint(127.8670259, 37.0174247), 4326)::geography),
156 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 52, ST_SetSRID(ST_MakePoint(127.8643912, 37.0224971), 4326)::geography),
157 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 53, ST_SetSRID(ST_MakePoint(127.8642146, 37.0227067), 4326)::geography),
158 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 54, ST_SetSRID(ST_MakePoint(127.863987, 37.0226807), 4326)::geography),
159 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 55, ST_SetSRID(ST_MakePoint(127.8635911, 37.0217118), 4326)::geography),
160 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 56, ST_SetSRID(ST_MakePoint(127.8630571, 37.0211752), 4326)::geography),
161 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 57, ST_SetSRID(ST_MakePoint(127.8627638, 37.0211629), 4326)::geography),
162 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 58, ST_SetSRID(ST_MakePoint(127.8623961, 37.0219171), 4326)::geography),
163 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 59, ST_SetSRID(ST_MakePoint(127.8604924, 37.0246173), 4326)::geography),
164 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 60, ST_SetSRID(ST_MakePoint(127.8585971, 37.0279671), 4326)::geography),
165 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 61, ST_SetSRID(ST_MakePoint(127.8579317, 37.028752), 4326)::geography),
166 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 62, ST_SetSRID(ST_MakePoint(127.8578543, 37.0293228), 4326)::geography),
167 (gen_random_uuid(), 'b0000002-0000-0000-0000-000000000000', 63, ST_SetSRID(ST_MakePoint(127.8574575,
```

```

207 INSERT INTO course_points(id,course_id,sequence,location) VALUES
208 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',101,ST_SetSRID(ST_MakePoint(127.8877026,36.9828349),4326)::geography),
209 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',102,ST_SetSRID(ST_MakePoint(127.8872936,36.9812638),4326)::geography),
210 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',103,ST_SetSRID(ST_MakePoint(127.8858157,36.9779647),4326)::geography),
211 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',104,ST_SetSRID(ST_MakePoint(127.8857177,36.9771062),4326)::geography),
212 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',105,ST_SetSRID(ST_MakePoint(127.8858113,36.9753628),4326)::geography),
213 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',106,ST_SetSRID(ST_MakePoint(127.8856844,36.975056),4326)::geography),
214 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',107,ST_SetSRID(ST_MakePoint(127.8854498,36.9748214),4326)::geography),
215 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',108,ST_SetSRID(ST_MakePoint(127.8850437,36.9747132),4326)::geography),
216 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',109,ST_SetSRID(ST_MakePoint(127.8802246,36.9746917),4326)::geography),
217 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',110,ST_SetSRID(ST_MakePoint(127.8795023,36.97438),4326)::geography),
218 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',111,ST_SetSRID(ST_MakePoint(127.8789467,36.9739885),4326)::geography),
219 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',112,ST_SetSRID(ST_MakePoint(127.8786575,36.9735923),4326)::geography),
220 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',113,ST_SetSRID(ST_MakePoint(127.8775636,36.9710314),4326)::geography),
221 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',114,ST_SetSRID(ST_MakePoint(127.8782523,36.968452),4326)::geography),
222 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',115,ST_SetSRID(ST_MakePoint(127.8782006,36.9677661),4326)::geography),
223 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',116,ST_SetSRID(ST_MakePoint(127.8778153,36.9670646),4326)::geography),
224 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',117,ST_SetSRID(ST_MakePoint(127.8769038,36.9661739),4326)::geography),
225 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',118,ST_SetSRID(ST_MakePoint(127.875857,36.9657468),4326)::geography),
226 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',119,ST_SetSRID(ST_MakePoint(127.8749093,36.9657246),4326)::geography),
227 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',120,ST_SetSRID(ST_MakePoint(127.8726919,36.9663696),4326)::geography),
228 (gen_random_uuid(),'b0000002-0000-0000-0000-000000000000',121,ST_SetSRID(ST_MakePoint(127.8677535,36.96743),4326)::geography)
229 ON CONFLICT DO NOTHING;
230

```

backend/src/main/resources/db/migration/V1__init_extensions.sql

SQL | 7줄

```

1  -- PostGIS: geography(Point), geography(LineString) 타입 및 공간 함수 활성화
2  CREATE EXTENSION IF NOT EXISTS postgis;
3
4  -- pgcrypto: gen_random_uuid(), digest() 함수 활성화
5  -- gen_random_uuid()는 PostgreSQL 13+에서 내장 함수이지만,
6  -- pgcrypto를 명시하여 암호화 함수를 추가로 활성화한다
7  CREATE EXTENSION IF NOT EXISTS pgcrypto;

```

backend/src/main/resources/db/migration/V2__create_users.sql

SQL | 20줄

```

1  CREATE TABLE users
2  (
3      id                UUID                NOT NULL DEFAULT gen_random_uuid(),
4      email             VARCHAR(255) NOT NULL,
5      password_hash     VARCHAR(255) NOT NULL,
6      nickname          VARCHAR(50)  NOT NULL,
7      profile_image_url TEXT,
8      bio               TEXT,
9      -- SHA-256 해시로 저장. 로그인 시 발급, 로그아웃 시 NULL, 재발급 시 비교 후 rotate
10     refresh_token_hash VARCHAR(512),
11     created_at         TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
12     updated_at         TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
13     deleted_at         TIMESTAMPTZ,
14
15     CONSTRAINT pk_users PRIMARY KEY (id)
16 );
17
18 CREATE UNIQUE INDEX uk_users_email    ON users (email);
19 CREATE UNIQUE INDEX uk_users_nickname ON users (nickname);
20 CREATE INDEX idx_users_deleted_at     ON users (deleted_at);

```

backend/src/main/resources/db/migration/V3__create_running_records.sql

SQL | 31줄

```
1 CREATE TABLE running_records
2 (
3     id                UUID                NOT NULL DEFAULT gen_random_uuid(),
4     user_id           UUID                NOT NULL,
5     -- PostgreSQL native enum 대신 VARCHAR(30) + CHECK 제약 사용
6     -- 이유: Spring Boot JPA @Enumerated(EnumType.STRING)과 충돌 없이 매핑 가능
7     status            VARCHAR(30) NOT NULL DEFAULT 'in_progress',
8     started_at        TIMESTAMPTZ NOT NULL,
9     ended_at          TIMESTAMPTZ,
10    distance_meters    FLOAT              NOT NULL DEFAULT 0,
11    duration_seconds   INT                NOT NULL DEFAULT 0,
12    avg_pace_seconds_per_km INT,
13    avg_heart_rate_bpm INT,
14    calories_burned    INT,
15    -- 런 완료 시 running_points를 ST_MakeLine()으로 집계하여 저장
16    path              geography(LineString, 4326),
17    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
18
19    CONSTRAINT pk_running_records PRIMARY KEY (id),
20    CONSTRAINT chk_running_record_status
21        CHECK (status IN ('in_progress', 'paused', 'completed', 'abandoned')),
22    CONSTRAINT fk_running_records_user
23        FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE
24 );
25
26 -- 유저별 최신 런 조회 (내 기록 목록 API)
27 CREATE INDEX idx_running_records_user_started ON running_records (user_id, started_at DESC);
28 -- 진행 중인 런 복구 조회 (앱 재시작 시 in_progress 런 복구)
29 CREATE INDEX idx_running_records_status ON running_records (status);
30 -- Phase 2 경로 분석 대비 (ST_DWithin, ST_Intersects 등 공간 쿼리)
31 CREATE INDEX idx_running_records_path ON running_records USING GIST (path);
```

backend/src/main/resources/db/migration/V4__create_running_points.sql

SQL | 19줄

```
1 CREATE TABLE running_points
2 (
3     id                UUID                NOT NULL DEFAULT gen_random_uuid(),
4     running_record_id UUID                NOT NULL,
5     -- 클라이언트가 GPS 샘플링 순서대로 부여하는 0-based 정수
6     sequence          INT                NOT NULL,
7     location          geography(Point, 4326) NOT NULL,
8     altitude_meters   FLOAT,
9     speed_mps         FLOAT,
10    recorded_at        TIMESTAMPTZ NOT NULL,
11
12    CONSTRAINT pk_running_points PRIMARY KEY (id),
13    CONSTRAINT fk_running_points_record
14        FOREIGN KEY (running_record_id) REFERENCES running_records (id) ON DELETE CASCADE,
15    -- 중복 sequence 방지 + B-tree 인덱스 자동 생성 (경로 순서 조회에 사용)
16    CONSTRAINT uq_running_points_record_seq UNIQUE (running_record_id, sequence)
17 );
18 -- uq_running_points_record_seq UNIQUE 제약이 (running_record_id, sequence) B-tree 인덱스를 자동 생성한다
19 -- 별도 CREATE INDEX 불필요
```

backend/src/main/resources/db/migration/V5__create_courses.sql

SQL | 43줄

```

1 CREATE TABLE courses
2 (
3     id                UUID                NOT NULL DEFAULT gen_random_uuid(),
4     creator_id        UUID                NOT NULL,
5     -- 원본 러닝 기록. SET NULL: 런 삭제 시 코스는 유지됨
6     source_record_id  UUID,
7     -- VARCHAR(100): 충분한 이름 길이 허용 (VARCHAR(10) 아님에 주의)
8     name              VARCHAR(100) NOT NULL,
9     description       TEXT,
10    status            VARCHAR(30)  NOT NULL DEFAULT 'draft',
11    distance_meters   FLOAT        NOT NULL,
12    is_loop           BOOLEAN      NOT NULL DEFAULT FALSE,
13    -- 인근 코스 탐색 (ST_DWithin) 의 기준점
14    start_location    geography(Point, 4326) NOT NULL,
15    end_location      geography(Point, 4326) NOT NULL,
16    -- course_points를 ST_MakeLine()으로 집계하여 저장
17    path              geography(LineString, 4326),
18    -- course_attempts에서 집계 시 매번 COUNT하지 않도록 denormalized 카운터 유지
19    attempt_count     INT           NOT NULL DEFAULT 0,
20    completion_count  INT           NOT NULL DEFAULT 0,
21    created_at        TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
22    updated_at        TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
23    deleted_at        TIMESTAMPTZ,
24
25    CONSTRAINT pk_courses PRIMARY KEY (id),
26    CONSTRAINT chk_course_status
27        CHECK (status IN ('draft', 'published', 'archived')),
28    CONSTRAINT fk_courses_creator
29        FOREIGN KEY (creator_id) REFERENCES users (id) ON DELETE RESTRICT,
30    CONSTRAINT fk_courses_source_record
31        FOREIGN KEY (source_record_id) REFERENCES running_records (id) ON DELETE SET NULL
32 );
33
34 -- 인근 코스 탐색 핵심 인덱스: ST_DWithin(start_location, ...) 쿼리에 사용
35 -- JPA @Index 어노테이션은 USING GIST를 지원하지 않으므로 반드시 Flyway SQL에서 생성
36 CREATE INDEX idx_courses_start_location ON courses USING GIST (start_location);
37
38 -- 내가 만든 코스 목록 조회
39 CREATE INDEX idx_courses_creator      ON courses (creator_id);
40 -- published 필터링 (인근 코스 탐색 시 WHERE status = 'published')
41 CREATE INDEX idx_courses_status      ON courses (status);
42 -- soft delete 필터링 (WHERE deleted_at IS NULL)
43 CREATE INDEX idx_courses_deleted_at ON courses (deleted_at);

```

backend/src/main/resources/db/migration/V6__create_course_points.sql

SQL | 17줄

```

1 CREATE TABLE course_points
2 (
3     id                UUID NOT NULL DEFAULT gen_random_uuid(),
4     course_id         UUID NOT NULL,
5     -- running_points를 다운샘플링하여 생성한 코스 경로 웨이포인트 순서
6     sequence          INT  NOT NULL,
7     location          geography(Point, 4326) NOT NULL,
8     altitude_meters  FLOAT,
9
10    CONSTRAINT pk_course_points PRIMARY KEY (id),
11    CONSTRAINT fk_course_points_course
12        FOREIGN KEY (course_id) REFERENCES courses (id) ON DELETE CASCADE,
13    -- 중복 sequence 방지 + B-tree 인덱스 자동 생성 (경로 순서 조회에 사용)
14    CONSTRAINT uq_course_points_course_seq UNIQUE (course_id, sequence)
15 );
16 -- uq_course_points_course_seq UNIQUE 제약이 (course_id, sequence) B-tree 인덱스를 자동 생성한다
17 -- 별도 CREATE INDEX 불필요

```

backend/src/main/resources/db/migration/V7__create_course_attempts.sql

SQL | 44줄

```
1 CREATE TABLE course_attempts
2 (
3     id                UUID          NOT NULL DEFAULT gen_random_uuid(),
4     course_id         UUID          NOT NULL,
5     user_id           UUID          NOT NULL,
6     -- 코스 시도 중 생성된 러닝 기록. SET NULL: 런 삭제 시 시도 기록은 유지됨
7     running_record_id UUID,
8     status             VARCHAR(30) NOT NULL DEFAULT 'in_progress',
9     -- Phase 1: completed 시 자동으로 verified 설정 (서비스 레이어 처리)
10    -- Phase 2: GPS 경로 매칭 검증 로직에서 pending → verified/rejected 전환
11    verification_status VARCHAR(30) NOT NULL DEFAULT 'pending',
12    started_at          TIMESTAMPTZ NOT NULL,
13    completed_at        TIMESTAMPTZ,
14    -- 완주 소요 시간 (미완주 시 NULL). 리더보드 집계 기준 컬럼
15    duration_seconds    INT,
16    distance_meters     FLOAT,
17    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
18
19    CONSTRAINT pk_course_attempts PRIMARY KEY (id),
20    CONSTRAINT chk_course_attempt_status
21        CHECK (status IN ('in_progress', 'completed', 'abandoned')),
22    CONSTRAINT chk_attempt_verification_status
23        CHECK (verification_status IN ('pending', 'verified', 'rejected')),
24    CONSTRAINT fk_course_attempts_course
25        FOREIGN KEY (course_id) REFERENCES courses (id) ON DELETE CASCADE,
26    CONSTRAINT fk_course_attempts_user
27        FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE,
28    CONSTRAINT fk_course_attempts_run
29        FOREIGN KEY (running_record_id) REFERENCES running_records (id) ON DELETE SET NULL
30 );
31
32 -- 리더보드 조회 핵심 partial index
33 -- WHERE 절로 completed + verified 기록만 인덱싱하여 크기를 최소화
34 -- SELECT MIN(duration_seconds) GROUP BY user_id ORDER BY best_time 쿼리에 사용
35 CREATE INDEX idx_course_attempts_leaderboard
36     ON course_attempts (course_id, duration_seconds)
37     WHERE status = 'completed' AND verification_status = 'verified';
38
39 -- 코스별 시도 목록 조회
40 CREATE INDEX idx_course_attempts_course_id ON course_attempts (course_id);
41 -- 유저별 시도 이력 조회
42 CREATE INDEX idx_course_attempts_user_id ON course_attempts (user_id);
43 -- 유저의 특정 코스 시도 이력 조회
44 CREATE INDEX idx_course_attempts_course_user ON course_attempts (course_id, user_id);
```

backend/src/main/resources/db/migration/V8__create_course_reports.sql

SQL | 12줄

```
1 CREATE TABLE course_reports (
2     id                UUID          NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
3     course_id         UUID          NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
4     reporter_id       UUID          NOT NULL REFERENCES users(id) ON DELETE CASCADE,
5     reason             VARCHAR(30) NOT NULL CHECK (reason IN ('inappropriate_content', 'wrong_location', 'spam', 'other')),
6     description        TEXT,
7     status             VARCHAR(30) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'reviewed', 'dismissed')),
8     created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
9 );
10
11 CREATE INDEX idx_course_reports_course_id ON course_reports (course_id);
12 CREATE INDEX idx_course_reports_reporter_id ON course_reports (reporter_id);
```

backend/src/main/resources/db/migration/V9__create_course_ratings.sql

SQL | 12줄

```
1 CREATE TABLE course_ratings (
2     id                UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
3     course_id         UUID          NOT NULL REFERENCES courses(id),
4     user_id           UUID          NOT NULL REFERENCES users(id),
5     rating            INTEGER       NOT NULL CHECK (rating BETWEEN 1 AND 5),
6     comment           TEXT,
7     created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
8     updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
9     UNIQUE (course_id, user_id)
10 );
11
12 CREATE INDEX idx_course_ratings_course_id ON course_ratings(course_id);
```



```

1  import java.util.Properties
2
3  private val roomVersion = "2.6.1"
4  private val brandName = "PathFinder"
5
6  plugins {
7      id("com.android.application")
8      id("org.jetbrains.kotlin.android")
9      id("org.jetbrains.kotlin.plugin.compose")
10     id("com.google.dagger.hilt.android")
11     id("com.google.devtools.ksp")
12 }
13
14 android {
15     namespace = "com.runway.android"
16     compileSdk = 35
17
18     defaultConfig {
19         applicationId = "com.runway.android"
20         minSdk = 26
21         targetSdk = 35
22         versionCode = 1
23         versionName = "1.0"
24         resValue("string", "app_name", brandName)
25         buildConfigField("String", "APP_NAME", "\"\$brandName\"")
26
27         testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
28
29         val localProps = Properties().also { props ->
30             val f = rootProject.file("local.properties")
31             if (f.exists()) f.inputStream().use { props.load(it) }
32         }
33         manifestPlaceholders["MAPS_API_KEY"] = localProps.getProperty("MAPS_API_KEY") ?: ""
34         buildConfigField("String", "OPENWEATHER_API_KEY", "\"\${localProps.getProperty(\"OPENWEATHER_API_KEY\") ?: \"\"}\"")
35         buildConfigField("String", "AIKOREA_API_KEY", "\"\${localProps.getProperty(\"AIKOREA_API_KEY\") ?: \"\"}\"")
36
37     }
38
39     buildTypes {
40         release {
41             isMinifyEnabled = false
42             proguardFiles(
43                 getDefaultProguardFile("proguard-android-optimize.txt"),
44                 "proguard-rules.pro",
45             )
46         }
47     }
48
49     compileOptions {
50         sourceCompatibility = JavaVersion.VERSION_17
51         targetCompatibility = JavaVersion.VERSION_17
52     }
53
54     buildFeatures {
55         compose = true
56         buildConfig = true
57     }
58
59     packaging {
60         resources {
61             excludes += "/META-INF/{AL2.0,LGPL2.1}"
62         }
63     }
64 }
65
66 kotlin {
67     compilerOptions {
68         jvmTarget.set(org.jetbrains.kotlin.gradle.dsl.JvmTarget.JVM_17)
69     }
70 }
71
72 dependencies {
73     // — Jetpack Compose BOM —
74     val composeBom = platform("androidx.compose:compose-bom:2024.10.01")
75     implementation(composeBom)
76     implementation("androidx.compose.ui:ui")
77     implementation("androidx.compose.ui:ui-graphics")
78     implementation("androidx.compose.ui:ui-tooling-preview")
79     implementation("androidx.compose.ui:ui-text-google-fonts")
80     implementation("androidx.compose.material3:material3")
81     implementation("androidx.compose.material:material-icons-core")
82     implementation("androidx.compose.material:material-icons-extended")
83

```

```

84 // — Activity + Lifecycle —
85 implementation("androidx.activity:activity-compose:1.9.3")
86 implementation("androidx.core:core-ktx:1.15.0")
87 implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.8.7")
88 implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.7")
89
90 // — Navigation —
91 implementation("androidx.navigation:navigation-compose:2.8.4")
92
93 // — Hilt —
94 implementation("com.google.dagger:hilt-android:2.52")
95 ksp("com.google.dagger:hilt-android-compiler:2.52")
96 implementation("androidx.hilt:hilt-navigation-compose:1.2.0")
97
98 // — Retrofit2 + OkHttp —
99 // JSON converter: Gson – annotation-free, zero boilerplate for Phase 1.
100 // Swap to Kotlinx Serialization later if compile-time safety is needed.
101 implementation("com.squareup.retrofit2:retrofit:2.11.0")
102 implementation("com.squareup.retrofit2:converter-gson:2.11.0")
103 implementation("com.squareup.okhttp3:okhttp:4.12.0")
104 implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")
105
106 // — DataStore —
107 implementation("androidx.datastore:datastore-preferences:1.1.1")
108
109 // — Location —
110 implementation("com.google.android.gms:play-services-location:21.3.0")
111 implementation("com.google.android.gms:play-services-wearable:19.0.0")
112 implementation("org.locationtech.proj4j:proj4j:1.4.3")
113
114 // — Google Maps —
115 implementation("com.google.android.gms:play-services-maps:19.0.0")
116 implementation("com.google.maps.android:maps-compose:4.4.1")
117
118 // — Coroutines —
119 implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.9.0")
120 implementation("org.jetbrains.kotlinx:kotlinx-coroutines-play-services:1.9.0")
121
122 // — Coil (image loading) —
123 implementation("io.coil-kt:coil-compose:2.7.0")
124
125 // — Room —
126 implementation("androidx.room:room-runtime:$roomVersion")
127 implementation("androidx.room:room-ktx:$roomVersion")
128 ksp("androidx.room:room-compiler:$roomVersion")
129
130 // — CameraX —
131 val cameraVersion = "1.4.1"
132 implementation("androidx.camera:camera-core:$cameraVersion")
133 implementation("androidx.camera:camera-camera2:$cameraVersion")
134 implementation("androidx.camera:camera-lifecycle:$cameraVersion")
135 implementation("androidx.camera:camera-video:$cameraVersion")
136 implementation("androidx.camera:camera-view:$cameraVersion")
137
138 // — MediaPipe Pose —
139 // 0.10.35+: libmediapipe_tasks_jni.so uses 16KB page alignment (was 4KB in ≤0.10.14)
140 implementation("com.google.mediarpipe:tasks-vision:0.10.35")
141
142 // — Media3 ExoPlayer (posture video replay) —
143 val media3Version = "1.5.1"
144 implementation("androidx.media3:media3-exoplayer:$media3Version")
145 implementation("androidx.media3:media3-ui:$media3Version")
146
147 // — Test —
148 testImplementation("junit:junit:4.13.2")
149
150 // — Debug —
151 debugImplementation("androidx.compose.ui:ui-tooling")
152 debugImplementation("androidx.compose.ui:ui-test-manifest")
153 }

```


android/app/src/main/java/com/runway/android/MainActivity.kt

Kotlin | 31줄

```
1 package com.runway.android
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.activity.viewModels
8 import androidx.compose.runtime.collectAsState
9 import androidx.compose.runtime.getValue
10 import com.runway.android.ui.MainViewModel
11 import com.runway.android.ui.navigation.RunwayNavGraph
12 import com.runway.android.ui.theme.RunwayTheme
13 import dagger.hilt.android.AndroidEntryPoint
14
15 @AndroidEntryPoint
16 class MainActivity : ComponentActivity() {
17
18     private val viewModel: MainViewModel by viewModels()
19
20     override fun onCreate(savedInstanceState: Bundle?) {
21         super.onCreate(savedInstanceState)
22         enableEdgeToEdge()
23         setContent {
24             val themeMode by viewModel.themeMode.collectAsState()
25             val accentColor by viewModel.accentColor.collectAsState()
26             RunwayTheme(themeMode = themeMode, accentColor = accentColor) {
27                 RunwayNavGraph()
28             }
29         }
30     }
31 }
```

android/app/src/main/java/com/runway/android/RunwayApplication.kt

Kotlin | 18줄

```
1 package com.runway.android
2
3 import android.app.Application
4 import com.runway.android.core.notification.NotificationChannels
5 import dagger.hilt.android.HiltAndroidApp
6 import javax.inject.Inject
7
8 @HiltAndroidApp
9 class RunwayApplication : Application() {
10
11     @Inject
12     lateinit var notificationChannels: NotificationChannels
13
14     override fun onCreate() {
15         super.onCreate()
16         notificationChannels.createAll()
17     }
18 }
```

android/app/src/main/java/com/runway/android/core/cadence/CadenceTracker.kt

Kotlin | 7줄

```
1 package com.runway.android.core.cadence
2
3 import kotlinx.coroutines.flow.Flow
4
5 interface CadenceTracker {
6     fun cadenceSpmFlow(): Flow<Int?>
7 }
```

```

1  package com.runway.android.core.cadence
2
3  import android.Manifest
4  import android.content.Context
5  import android.content.pm.PackageManager
6  import android.hardware.Sensor
7  import android.hardware.SensorEvent
8  import android.hardware.SensorEventListener
9  import android.hardware.SensorManager
10 import android.os.Build
11 import androidx.core.content.ContextCompat
12 import dagger.hilt.android.qualifiers.ApplicationContext
13 import kotlinx.coroutines.channels.awaitClose
14 import kotlinx.coroutines.flow.Flow
15 import kotlinx.coroutines.flow.callbackFlow
16 import java.util.ArrayDeque
17 import javax.inject.Inject
18 import javax.inject.Singleton
19
20 @Singleton
21 class PhoneStepCadenceTracker @Inject constructor(
22     @ApplicationContext private val context: Context,
23 ) : CadenceTracker {
24
25     override fun cadenceSpmFlow(): Flow<Int?> = callbackFlow {
26         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q &&
27             ContextCompat.checkSelfPermission(context, Manifest.permission.ACTIVITY_RECOGNITION) != PackageManager.PERMISSION_GRANTED
28         ) {
29             trySend(null)
30             close()
31             return@callbackFlow
32         }
33
34         val sensorManager = context.getSystemService(Context.SENSOR_SERVICE) as SensorManager
35         val stepDetector = sensorManager.getDefaultSensor(Sensor.TYPE_STEP_DETECTOR)
36         if (stepDetector == null) {
37             trySend(null)
38             close()
39             return@callbackFlow
40         }
41
42         val stepTimesMillis = ArrayDeque<Long>()
43         val listener = object : SensorEventListener {
44             override fun onSensorChanged(event: SensorEvent) {
45                 val now = System.currentTimeMillis()
46                 stepTimesMillis.addLast(now)
47
48                 val windowStart = now - CADENCE_WINDOW_MILLIS
49                 while ((stepTimesMillis.peekFirst() ?: Long.MAX_VALUE) < windowStart) {
50                     stepTimesMillis.removeFirst()
51                 }
52
53                 val cadence = (stepTimesMillis.size * 60_000.0 / CADENCE_WINDOW_MILLIS).toInt()
54                 trySend(cadence.coerceAtLeast(0))
55             }
56
57             override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) = Unit
58         }
59
60         sensorManager.registerListener(
61             listener,
62             stepDetector,
63             SensorManager.SENSOR_DELAY_NORMAL,
64         )
65
66         awaitClose { sensorManager.unregisterListener(listener) }
67     }
68
69     private companion object {
70         const val CADENCE_WINDOW_MILLIS = 20_000L
71     }
72 }

```

android/app/src/main/java/com/runway/android/core/datastore/OnboardingDataStore.kt

Kotlin | 26줄

```
1 package com.runway.android.core.datastore
2
3 import androidx.datastore.core.DataStore
4 import androidx.datastore.preferences.core.Preferences
5 import androidx.datastore.preferences.core.booleanPreferencesKey
6 import androidx.datastore.preferences.core.edit
7 import kotlinx.coroutines.flow.Flow
8 import kotlinx.coroutines.flow.map
9 import javax.inject.Inject
10 import javax.inject.Named
11 import javax.inject.Singleton
12
13 @Singleton
14 class OnboardingDataStore @Inject constructor(
15     @Named("onboardingDataStore") private val datastore: DataStore<Preferences>,
16 ) {
17     companion object {
18         private val KEY_COMPLETED = booleanPreferencesKey("onboarding_completed")
19     }
20
21     val isCompletedFlow: Flow<Boolean> = datastore.data.map { it[KEY_COMPLETED] ?: false }
22
23     suspend fun markCompleted() {
24         datastore.edit { it[KEY_COMPLETED] = true }
25     }
26 }
```

android/app/src/main/java/com/runway/android/core/datastore/ThemeDataStore.kt

Kotlin | 45줄

```
1 package com.runway.android.core.datastore
2
3 import androidx.datastore.core.DataStore
4 import androidx.datastore.preferences.core.Preferences
5 import androidx.datastore.preferences.core.edit
6 import androidx.datastore.preferences.core.stringPreferencesKey
7 import com.runway.android.ui.theme.AccentColor
8 import com.runway.android.ui.theme.ThemeMode
9 import kotlinx.coroutines.flow.Flow
10 import kotlinx.coroutines.flow.map
11 import javax.inject.Inject
12 import javax.inject.Named
13 import javax.inject.Singleton
14
15 @Singleton
16 class ThemeDataStore @Inject constructor(
17     @Named("settingsDataStore") private val datastore: DataStore<Preferences>,
18 ) {
19     companion object {
20         private val KEY_THEME_MODE = stringPreferencesKey("theme_mode")
21         private val KEY_ACCENT_COLOR = stringPreferencesKey("accent_color")
22     }
23
24     val themeModeFlow: Flow<ThemeMode> = datastore.data.map { prefs ->
25         when (prefs[KEY_THEME_MODE]) {
26             "LIGHT" -> ThemeMode.LIGHT
27             "SYSTEM" -> ThemeMode.SYSTEM
28             else -> ThemeMode.DARK
29         }
30     }
31
32     val accentColorFlow: Flow<AccentColor> = datastore.data.map { prefs ->
33         runCatching {
34             AccentColor.valueOf(prefs[KEY_ACCENT_COLOR] ?: AccentColor.GREEN.name)
35         }.getOrElse(AccentColor.GREEN)
36     }
37
38     suspend fun setThemeMode(mode: ThemeMode) {
39         datastore.edit { it[KEY_THEME_MODE] = mode.name }
40     }
41
42     suspend fun setAccentColor(color: AccentColor) {
43         datastore.edit { it[KEY_ACCENT_COLOR] = color.name }
44     }
45 }
```

```

1 package com.runway.android.core.datastore
2
3 import android.util.Base64
4 import androidx.datastore.core.DataStore
5 import androidx.datastore.preferences.core.Preferences
6 import androidx.datastore.preferences.core.edit
7 import androidx.datastore.preferences.core.stringPreferencesKey
8 import kotlinx.coroutines.flow.Flow
9 import kotlinx.coroutines.flow.first
10 import kotlinx.coroutines.flow.map
11 import kotlinx.coroutines.runBlocking
12 import org.json.JSONObject
13 import javax.inject.Inject
14 import javax.inject.Singleton
15
16 /**
17  * JWT 토큰을 DataStore에 저장하고 읽는 클래스.
18  *
19  * - saveTokens(): 로그인 / reissue 성공 시 호출
20  * - clearTokens(): 로그아웃 / token 만료 시 호출
21  * - accessTokenFlow: SplashScreen에서 자동 로그인 여부 확인에 사용
22  * - getAccessTokenBlocking(): OkHttp AuthInterceptor (suspend 불가) 에서 사용
23  */
24 @Singleton
25 class TokenDataStore @Inject constructor(
26     private val datastore: DataStore<Preferences>,
27 ) {
28     companion object {
29         private val KEY_ACCESS_TOKEN = stringPreferencesKey("access_token")
30         private val KEY_REFRESH_TOKEN = stringPreferencesKey("refresh_token")
31     }
32
33     // Flow - SplashScreen/ViewModel에서 로그인 상태 관찰 시 사용
34     val accessTokenFlow: Flow<String?> = datastore.data.map { it[KEY_ACCESS_TOKEN] }
35     val refreshTokenFlow: Flow<String?> = datastore.data.map { it[KEY_REFRESH_TOKEN] }
36     val userIdFlow: Flow<String?> = accessTokenFlow.map(::decodeUserId)
37
38     suspend fun saveTokens(accessToken: String, refreshToken: String) {
39         datastore.edit { prefs ->
40             prefs[KEY_ACCESS_TOKEN] = accessToken
41             prefs[KEY_REFRESH_TOKEN] = refreshToken
42         }
43     }
44
45     suspend fun clearTokens() {
46         datastore.edit { it.clear() }
47     }
48
49     // Blocking 버전 - OkHttp Interceptor/Authenticator (suspend 컨텍스트 없음)에서만 사용
50     // OkHttp thread pool에서 호출되므로 main thread 블로킹 위험 없음
51     fun getAccessTokenBlocking(): String? = runBlocking {
52         accessTokenFlow.first()
53     }
54
55     fun getRefreshTokenBlocking(): String? = runBlocking {
56         refreshTokenFlow.first()
57     }
58
59     suspend fun getUserId(): String? = userIdFlow.first()
60
61     private fun decodeUserId(token: String?): String? = runCatching {
62         val payload = token?.split(".").getOrNull(1) ?: return null
63         val decoded = Base64.decode(payload, Base64.URL_SAFE or Base64.NO_WRAP or Base64.NO_PADDING)
64         JSONObject(String(decoded, Charsets.UTF_8)).optString("sub").takeIf { it.isNotBlank() }
65     }.getOrNull()
66 }

```

```
1 package com.runway.android.core.location
2
3 import android.annotation.SuppressLint
4 import android.os.Looper
5 import com.google.android.gms.location.FusedLocationProviderClient
6 import com.google.android.gms.location.LocationCallback
7 import com.google.android.gms.location.LocationRequest
8 import com.google.android.gms.location.LocationResult
9 import com.google.android.gms.location.Priority
10 import kotlinx.coroutines.channels.awaitClose
11 import kotlinx.coroutines.flow.Flow
12 import kotlinx.coroutines.flow.callbackFlow
13 import java.time.Instant
14 import javax.inject.Inject
15 import javax.inject.Singleton
16
17 // NOTE: Real GPS requires a physical device or emulator with mock location input.
18 // In Android Emulator: use Extended Controls → Location to send simulated GPS points,
19 // or use the "gpx" / "kml" route playback feature for continuous movement simulation.
20 @Singleton
21 class DefaultLocationTracker @Inject constructor(
22     private val fusedLocationClient: FusedLocationProviderClient,
23 ) : LocationTracker {
24
25     // Requires ACCESS_FINE_LOCATION or ACCESS_COARSE_LOCATION – caller must verify before collecting.
26     @SuppressLint("MissingPermission")
27     override fun locationFlow(): Flow<RunwayLocation> = callbackFlow {
28         val request = LocationRequest.Builder(Priority.PRIORITY_HIGH_ACCURACY, 3_000L)
29             .setMinUpdateIntervalMillis(1_000L)
30             .build()
31
32         val callback = object : LocationCallback() {
33             override fun onLocationResult(result: LocationResult) {
34                 result.lastLocation?.let { loc ->
35                     trySend(
36                         RunwayLocation(
37                             latitude = loc.latitude,
38                             longitude = loc.longitude,
39                             altitudeMeters = if (loc.hasAltitude()) loc.altitude else null,
40                             speedMps = if (loc.hasSpeed()) loc.speed else null,
41                             recordedAt = Instant.ofEpochMilli(loc.time),
42                         )
43                     )
44                 }
45             }
46         }
47
48         fusedLocationClient.requestLocationUpdates(request, callback, Looper.getMainLooper())
49
50         awaitClose { fusedLocationClient.removeLocationUpdates(callback) }
51     }
52 }
```

android/app/src/main/java/com/runway/android/core/location/DistanceCalculator.kt

Kotlin | 28줄

```
1 package com.runway.android.core.location
2
3 import android.location.Location
4
5 object DistanceCalculator {
6
7     // Returns distance in meters between two GPS points, or 0.0 if the point should be discarded.
8     fun calculate(from: RunwayLocation, to: RunwayLocation): Double {
9         if (from.latitude == to.latitude && from.longitude == to.longitude) return 0.0
10
11         val results = FloatArray(1)
12         Location.distanceBetween(
13             from.latitude, from.longitude,
14             to.latitude, to.longitude,
15             results,
16         )
17         val meters = results[0].toDouble()
18
19         // Filter GPS teleportation: > 10 m/s implied speed is impossible for a runner.
20         // Clamp time delta to at least 1 second to avoid division by zero on same-timestamp points.
21         val timeDeltaSecs = (to.recordedAt.epochSecond - from.recordedAt.epochSecond)
22             .coerceAtLeast(1L)
23         val impliedSpeedMps = meters / timeDeltaSecs
24         if (impliedSpeedMps > 10.0) return 0.0
25
26         return meters
27     }
28 }
```

android/app/src/main/java/com/runway/android/core/location/GpsPointValidator.kt

Kotlin | 31줄

```
1 package com.runway.android.core.location
2
3 import android.location.Location
4
5 object GpsPointValidator {
6     private const val MAX_SPEED_MPS = 12.0        // ~43 km/h
7     private const val MAX_JUMP_METERS = 200.0
8     private const val MIN_DISTANCE_METERS = 1.0
9
10    fun isValid(new: RunwayLocation, prev: RunwayLocation?): Boolean {
11        if (prev == null) return true
12
13        val results = FloatArray(1)
14        Location.distanceBetween(
15            prev.latitude, prev.longitude,
16            new.latitude, new.longitude,
17            results,
18        )
19        val distance = results[0].toDouble()
20
21        if (distance < MIN_DISTANCE_METERS) return false
22        if (distance > MAX_JUMP_METERS) return false
23
24        val timeDeltaSecs = (new.recordedAt.epochSecond - prev.recordedAt.epochSecond)
25            .coerceAtLeast(1L)
26        val impliedSpeed = distance / timeDeltaSecs
27        if (impliedSpeed > MAX_SPEED_MPS) return false
28
29        return true
30    }
31 }
```

android/app/src/main/java/com/runway/android/core/location/GpsStatus.kt

Kotlin | 7줄

```
1 package com.runway.android.core.location
2
3 enum class GpsStatus {
4     PERMISSION_REQUIRED,
5     WAITING_FOR_FIX,
6     ACTIVE,
7 }
```

android/app/src/main/java/com/runway/android/core/location/LocationTracker.kt

Kotlin | 9줄

```
1 package com.runway.android.core.location
2
3 import kotlinx.coroutines.flow.Flow
4
5 interface LocationTracker {
6     // Returns a cold Flow that emits GPS locations while collected.
7     // Caller must ensure ACCESS_FINE_LOCATION or ACCESS_COARSE_LOCATION is granted before collecting.
8     fun locationFlow(): Flow<RunwayLocation>
9 }
```

android/app/src/main/java/com/runway/android/core/location/RunwayLocation.kt

Kotlin | 11줄

```
1 package com.runway.android.core.location
2
3 import java.time.Instant
4
5 data class RunwayLocation(
6     val latitude: Double,
7     val longitude: Double,
8     val altitudeMeters: Double?,
9     val speedMps: Float?,
10    val recordedAt: Instant,
11 )
```

android/app/src/main/java/com/runway/android/core/map/MapBoundsUtil.kt

Kotlin | 19줄

```
1 package com.runway.android.core.map
2
3 import com.google.android.gms.maps.model.LatLng
4 import com.google.android.gms.maps.model.LatLngBounds
5
6 fun List<MapPoint>.toLatLngBounds(paddingFraction: Double = 0.10): LatLngBounds {
7     val minLat = minOf { it.latitude }
8     val maxLat = maxOf { it.latitude }
9     val minLng = minOf { it.longitude }
10    val maxLng = maxOf { it.longitude }
11
12    val latPad = (maxLat - minLat).coerceAtLeast(0.001) * paddingFraction
13    val lngPad = (maxLng - minLng).coerceAtLeast(0.001) * paddingFraction
14
15    return LatLngBounds(
16        LatLng(minLat - latPad, minLng - lngPad),
17        LatLng(maxLat + latPad, maxLng + lngPad),
18    )
19 }
```

android/app/src/main/java/com/runway/android/core/map/MapPoint.kt

Kotlin | 6줄

```
1 package com.runway.android.core.map
2
3 data class MapPoint(
4     val latitude: Double,
5     val longitude: Double,
6 )
```

android/app/src/main/java/com/runway/android/core/model/ApiResponse.kt

Kotlin | 14줄

```
1 package com.runway.android.core.model
2
3 /**
4  * 백엔드 공통 응답 wrapper.
5  *
6  * 성공: { "success": true, "message": "...", "data": {...} }
7  * 실패: { "success": false, "message": "...", "errorCode": "SOME_CODE", "data": null }
8  */
9 data class ApiResponse<T>(
10     val success: Boolean,
11     val message: String,
12     val data: T?,
13     val errorCode: String? = null,
14 )
```

android/app/src/main/java/com/runway/android/core/model/PageResponse.kt

Kotlin | 16줄

```
1 package com.runway.android.core.model
2
3 /**
4  * 백엔드 페이지네이션 응답 wrapper.
5  * ApiResponse<PageResponse<T>> 형태로 사용된다.
6  *
7  * GET /api/runs/me?page=0&size=20 응답의 data 필드 형식.
8  */
9 data class PageResponse<T>{
10     val content: List<T>,
11     val page: Int,
12     val size: Int,
13     val totalElements: Long,
14     val totalPages: Int,
15     val hasNext: Boolean,
16 }
```

android/app/src/main/java/com/runway/android/core/network/AuthInterceptor.kt

Kotlin | 29줄

```
1 package com.runway.android.core.network
2
3 import com.runway.android.core.datastore.TokenDataStore
4 import okhttp3.Interceptor
5 import okhttp3.Response
6 import javax.inject.Inject
7
8 /**
9  * 모든 인증 필요 요청에 "Authorization: Bearer <accessToken>" header를 자동으로 추가한다.
10  *
11  * 토큰이 없으면(미로그인 상태) header를 추가하지 않는다.
12  * 401 응답 시 token 갱신은 TokenAuthenticator가 담당한다.
13  */
14 class AuthInterceptor @Inject constructor(
15     private val tokenDataStore: TokenDataStore,
16 ) : Interceptor {
17
18     override fun intercept(chain: Interceptor.Chain): Response {
19         val token = tokenDataStore.getAccessTokenBlocking()
20         val request = if (token != null) {
21             chain.request().newBuilder()
22                 .addHeader("Authorization", "Bearer $token")
23                 .build()
24         } else {
25             chain.request()
26         }
27         return chain.proceed(request)
28     }
29 }
```

android/app/src/main/java/com/runway/android/core/network/NetworkConstants.kt

Kotlin | 15줄

```
1 package com.runway.android.core.network
2
3 object NetworkConstants {
4     /**
5      * Android 에뮬레이터: 10.0.2.2 → 호스트 머신의 localhost (백엔드 포트 8080)
6      * 실기기 테스트: 호스트 머신의 실제 IP로 교체 (예: "http://192.168.x.x:8080/")
7      */
8     //const val BASE_URL = "http://10.0.2.2:8080/"           // 에뮬레이터
9     //const val BASE_URL = "http://192.168.0.101:8080/"      // 로컬 실기기
10     const val BASE_URL = "http://40.233.98.156:8080/"       // 프로덕션 서버
11
12     const val CONNECT_TIMEOUT_SECONDS = 30L
13     const val READ_TIMEOUT_SECONDS = 30L
14     const val WRITE_TIMEOUT_SECONDS = 30L
15 }
```



```

1  package com.runway.android.core.network
2
3  import com.runway.android.core.datastore.TokenDataStore
4  import com.runway.android.data.auth.model.ReissueRequest
5  import com.runway.android.data.auth.remote.AuthApi
6  import kotlinx.coroutines.runBlocking
7  import okhttp3.Authenticator
8  import okhttp3.Request
9  import okhttp3.Response
10 import okhttp3.Route
11 import retrofit2.Retrofit
12 import javax.inject.Inject
13 import javax.inject.Named
14
15 /**
16  * 401 Unauthorized 응답 수신 시 토큰을 자동으로 갱신하는 OkHttp Authenticator.
17  *
18  * 순환 의존성 방지 설계:
19  * - TokenAuthenticator는 @Named("noAuth") Retrofit (AuthInterceptor 없는 클라이언트)을 주입받는다.
20  * - 이 Retrofit으로 AuthApi를 lazy하게 생성하여 reissue API만 호출한다.
21  * - Main OkHttpClient(AuthInterceptor + TokenAuthenticator 포함)와 분리되므로 순환 없음.
22  *
23  * 재시도 루프 방지:
24  * - response.request에 Authorization header가 없으면 재시도하지 않는다.
25  *   (signup/login처럼 인증 불필요 요청이 401을 받은 경우)
26  */
27 class TokenAuthenticator @Inject constructor(
28     private val tokenDataStore: TokenDataStore,
29     @Named("noAuth") private val noAuthRetrofit: Retrofit,
30 ) : Authenticator {
31
32     private val authApi: AuthApi by lazy { noAuthRetrofit.create(AuthApi::class.java) }
33
34     override fun authenticate(route: Route?, response: Response): Request? {
35         // Authorization header가 없는 요청의 401은 재시도하지 않는다 (무한 루프 방지)
36         if (response.request.header("Authorization") == null) return null
37
38         // 같은 요청이 이미 한 번 실패했으면 포기 (refresh token도 만료)
39         if (response.responseCount() >= 2) {
40             runBlocking { tokenDataStore.clearTokens() }
41             return null
42         }
43
44         // synchronized: 동시 401 응답이 여러 스레드에서 동시에 진입하면
45         // 첫 번째 스레드만 reissue를 실행하고, 이후 스레드는 이미 갱신된 토큰을 재사용한다.
46         synchronized(this) {
47             val requestToken = response.request.header("Authorization")
48                 ?.removePrefix("Bearer ").trim()
49             val currentToken = tokenDataStore.getAccessTokenBlocking()
50
51             // 다른 스레드가 이미 토큰을 갱신한 경우 - 새 토큰으로 즉시 재시도
52             if (currentToken != null && currentToken != requestToken) {
53                 return response.request.newBuilder()
54                     .header("Authorization", "Bearer $currentToken")
55                     .build()
56             }
57
58             val refreshToken = tokenDataStore.getRefreshTokenBlocking() ?: run {
59                 runBlocking { tokenDataStore.clearTokens() }
60                 return null
61             }
62
63             val newTokens = runBlocking {
64                 runCatching {
65                     authApi.reissue(ReissueRequest(refreshToken)).data
66                 }.getOrNull()
67             }
68
69             if (newTokens == null) {
70                 runBlocking { tokenDataStore.clearTokens() }
71                 return null
72             }
73
74             runBlocking {
75                 tokenDataStore.saveTokens(newTokens.accessToken, newTokens.refreshToken)
76             }
77
78             return response.request.newBuilder()
79                 .header("Authorization", "Bearer ${newTokens.accessToken}")
80                 .build()
81         }
82     }
83 }

```

```

84 // 같은 URL에 대한 연속 재시도 횟수 계산 (OkHttp 내부 패턴)
85 private fun Response.responseCount(): Int {
86     var count = 1
87     var prior = priorResponse
88     while (prior != null) {
89         count++
90         prior = prior.priorResponse
91     }
92     return count
93 }
94 }

```

android/app/src/main/java/com/runway/android/core/notification/BootReceiver.kt

Kotlin | 36줄

```

1 package com.runway.android.core.notification
2
3 import android.content.BroadcastReceiver
4 import android.content.Context
5 import android.content.Intent
6 import com.runway.android.data.reminder.ReminderPrefsRepository
7 import dagger.hilt.android.AndroidEntryPoint
8 import kotlinx.coroutines.CoroutineScope
9 import kotlinx.coroutines.Dispatchers
10 import kotlinx.coroutines.flow.first
11 import kotlinx.coroutines.launch
12 import javax.inject.Inject
13
14 @AndroidEntryPoint
15 class BootReceiver : BroadcastReceiver() {
16
17     @Inject
18     lateinit var reminderPrefs: ReminderPrefsRepository
19
20     @Inject
21     lateinit var scheduler: ReminderScheduler
22
23     override fun onReceive(context: Context, intent: Intent) {
24         if (intent.action != Intent.ACTION_BOOT_COMPLETED) return
25         // goAsync()로 브로드캐스트 윈도우를 연장해 코루틴 완료까지 보장
26         val pendingResult = goAsync()
27         CoroutineScope(Dispatchers.IO).launch {
28             try {
29                 val prefs = reminderPrefs.getPrefs().first()
30                 if (prefs.enabled) scheduler.scheduleAll(prefs)
31             } finally {
32                 pendingResult.finish()
33             }
34         }
35     }
36 }

```

```
1 package com.runway.android.core.notification
2
3 import android.app.NotificationChannel
4 import android.app.NotificationManager
5 import android.content.Context
6 import dagger.hilt.android.qualifiers.ApplicationContext
7 import javax.inject.Inject
8 import javax.inject.Singleton
9
10 @Singleton
11 class NotificationChannels @Inject constructor(
12     @ApplicationContext private val context: Context,
13 ) {
14     companion object {
15         const val CHANNEL_REMINDER = "runway_reminder"
16         const val CHANNEL_TRACKING = "runway_tracking"
17     }
18
19     fun createAll() {
20         val manager = context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
21         manager.createNotificationChannel(
22             NotificationChannel(
23                 CHANNEL_REMINDER,
24                 "러닝 리마인더",
25                 NotificationManager.IMPORTANCE_DEFAULT,
26             ).apply { description = "예약된 러닝 리마인더 알림" },
27         )
28     }
29 }
```

```
1 package com.runway.android.core.notification
2
3 import android.app.NotificationManager
4 import android.app.PendingIntent
5 import android.content.BroadcastReceiver
6 import android.content.Context
7 import android.content.Intent
8 import androidx.core.app.NotificationCompat
9 import com.runway.android.MainActivity
10 import com.runway.android.R
11 import dagger.hilt.android.AndroidEntryPoint
12 import javax.inject.Inject
13
14 @AndroidEntryPoint
15 class ReminderReceiver : BroadcastReceiver() {
16
17     @Inject
18     lateinit var scheduler: ReminderScheduler
19
20     override fun onReceive(context: Context, intent: Intent) {
21         val dayLabel = intent.getStringExtra(EXTRA_DAY_LABEL) ?: "오늘"
22         val dayOfWeek = intent.getIntExtra(EXTRA_DAY_OF_WEEK, -1)
23         val hour = intent.getIntExtra(EXTRA_HOUR, 7)
24         val minute = intent.getIntExtra(EXTRA_MINUTE, 0)
25
26         showReminderNotification(context, dayLabel)
27
28         // Exact alarm은 one-shot - 다음 주 같은 요일로 재예약
29         if (dayOfWeek != -1) {
30             scheduler.schedule(dayOfWeek, hour, minute)
31         }
32     }
33
34     private fun showReminderNotification(context: Context, dayLabel: String) {
35         val launchIntent = Intent(context, MainActivity::class.java).apply {
36             flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
37         }
38         val pendingIntent = PendingIntent.getActivity(
39             context, 0, launchIntent,
40             PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE,
41         )
42
43         val notification = NotificationCompat.Builder(context, NotificationChannels.CHANNEL_REMINDER)
44             .setSmallIcon(R.drawable.ic_notification)
45             .setContentTitle("오늘 런 하셨나요? 🏃")
46             .setContentText("$dayLabel 러닝 목표를 달성해보세요!")
47             .setAutoCancel(true)
48             .setContentIntent(pendingIntent)
49             .build()
50
51         val manager = context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
52         manager.notify(NOTIFICATION_ID, notification)
53     }
54
55     companion object {
56         const val NOTIFICATION_ID = 1002
57         const val EXTRA_DAY_LABEL = "day_label"
58         const val EXTRA_DAY_OF_WEEK = "day_of_week"
59         const val EXTRA_HOUR = "hour"
60         const val EXTRA_MINUTE = "minute"
61     }
62 }
```

```

1 package com.runway.android.core.notification
2
3 import android.app.AlarmManager
4 import android.app.PendingIntent
5 import android.content.Context
6 import android.content.Intent
7 import android.os.Build
8 import com.runway.android.data.reminder.ReminderPrefs
9 import dagger.hilt.android.qualifiers.ApplicationContext
10 import java.util.Calendar
11 import javax.inject.Inject
12 import javax.inject.Singleton
13
14 @Singleton
15 class ReminderScheduler @Inject constructor(
16     @ApplicationContext private val context: Context,
17 ) {
18     private val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as AlarmManager
19
20     fun scheduleAll(prefs: ReminderPrefs) {
21         cancelAll()
22         if (!prefs.enabled) return
23         prefs.enabledDays.forEach { dayOfWeek ->
24             schedule(dayOfWeek, prefs.hour, prefs.minute)
25         }
26     }
27
28     fun cancelAll() {
29         for (day in 1..7) {
30             alarmManager.cancel(buildIntent(day, 0, 0))
31         }
32     }
33
34     fun schedule(dayOfWeek: Int, hour: Int, minute: Int) {
35         val calendar = Calendar.getInstance().apply {
36             set(Calendar.DAY_OF_WEEK, dayOfWeek)
37             set(Calendar.HOUR_OF_DAY, hour)
38             set(Calendar.MINUTE, minute)
39             set(Calendar.SECOND, 0)
40             set(Calendar.MILLISECOND, 0)
41             if (timeInMillis <= System.currentTimeMillis()) {
42                 add(Calendar.WEEK_OF_YEAR, 1)
43             }
44         }
45
46         val pendingIntent = buildIntent(dayOfWeek, hour, minute)
47
48         // Android 12+(API 31): 정확한 알람 권한이 없으면 비정확 알람으로 폴백
49         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S && !alarmManager.canScheduleExactAlarms()) {
50             alarmManager.setAndAllowWhileIdle(
51                 AlarmManager.RTC_WAKEUP,
52                 calendar.timeInMillis,
53                 pendingIntent,
54             )
55             return
56         }
57
58         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
59             alarmManager.setExactAndAllowWhileIdle(
60                 AlarmManager.RTC_WAKEUP,
61                 calendar.timeInMillis,
62                 pendingIntent,
63             )
64         } else {
65             alarmManager.setExact(
66                 AlarmManager.RTC_WAKEUP,
67                 calendar.timeInMillis,
68                 pendingIntent,
69             )
70         }
71     }
72
73     // ReminderReceiver에서 다음 주 재예약에 사용하기 위해 internal로 공개
74     internal fun buildIntent(dayOfWeek: Int, hour: Int, minute: Int): PendingIntent {
75         val intent = Intent(context, ReminderReceiver::class.java).apply {
76             putExtra(ReminderReceiver.EXTRA_DAY_LABEL, dayLabel(dayOfWeek))
77             putExtra(ReminderReceiver.EXTRA_DAY_OF_WEEK, dayOfWeek)
78             putExtra(ReminderReceiver.EXTRA_HOUR, hour)
79             putExtra(ReminderReceiver.EXTRA_MINUTE, minute)
80         }
81         return PendingIntent.getBroadcast(
82             context,
83             dayOfWeek,

```

```
84         intent,
85         PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE,
86     )
87 }
88
89 private fun dayLabel(dayOfWeek: Int) = when (dayOfWeek) {
90     Calendar.SUNDAY -> "일요일"
91     Calendar.MONDAY -> "월요일"
92     Calendar.TUESDAY -> "화요일"
93     Calendar.WEDNESDAY -> "수요일"
94     Calendar.THURSDAY -> "목요일"
95     Calendar.FRIDAY -> "금요일"
96     Calendar.SATURDAY -> "토요일"
97     else -> "오늘"
98 }
99 }
```

```

1 package com.runway.android.core.posture
2
3 import kotlin.math.abs
4
5 /**
6  * 2D Kalman filter for ankle position and velocity tracking.
7  *
8  * Ported from GaitKeeper (MIT) - chrismarth/GaitKeeper - lib/kalman-filter.ts
9  * Adapted from pixel coordinates to MediaPipe normalized coordinates (0-1).
10 *
11 * State vector: [x, y, vx, vy]
12 *
13 * Usage:
14 *   val filter = AnkleKalmanFilter(initialX, initialY)
15 *   filter.update(ankle.x, ankle.y, timestampMs)
16 *   if (filter.isLanding()) { /* heel strike detected */ }
17 */
18 class AnkleKalmanFilter(
19     initialX: Float,
20     initialY: Float,
21     private val processNoise: Float = 0.0005f, // Q - normalized coords are 0-1 scale
22     private val measurementNoise: Float = 0.02f, // R
23 ) {
24     // State: [x, y, vx, vy]
25     private var stateX = initialX
26     private var stateY = initialY
27     private var vx = 0f
28     private var vy = 0f
29
30     // Covariance (diagonal only, same simplification as GaitKeeper)
31     private var px = 0.1f
32     private var py = 0.1f
33     private var pvx = 0.1f
34     private var pvy = 0.1f
35
36     private var lastTimestampMs: Long = -1L
37
38     fun update(measX: Float, measY: Float, timestampMs: Long) {
39         val dt = if (lastTimestampMs < 0L) {
40             0f
41         } else {
42             ((timestampMs - lastTimestampMs) / 1000f).coerceIn(0f, 0.1f)
43         }
44         lastTimestampMs = timestampMs
45
46         // — Predict —————
47         stateX += vx * dt
48         stateY += vy * dt
49         px += processNoise * dt
50         py += processNoise * dt
51         pvx += processNoise * dt * 0.1f
52         pvy += processNoise * dt * 0.1f
53
54         // — Update —————
55         val kx = px / (px + measurementNoise)
56         val ky = py / (py + measurementNoise)
57         val innovX = measX - stateX
58         val innovY = measY - stateY
59         stateX += kx * innovX
60         stateY += ky * innovY
61
62         if (dt > 0.001f) {
63             val alpha = 0.3f // velocity blending (mirrors GaitKeeper)
64             vx = (1f - alpha) * vx + alpha * (innovX / dt)
65             vy = (1f - alpha) * vy + alpha * (innovY / dt)
66         }
67
68         px *= (1f - kx)
69         py *= (1f - ky)
70     }
71
72     val x get() = stateX
73     val y get() = stateY
74     val velocityX get() = vx
75     val velocityY get() = vy
76
77     /**
78      * Heel strike / landing detection.
79      * In normalized coordinates: ground ≈ y=0.85, threshold in normalized units.
80      * vy > 0 means moving downward (Y increases downward in image space).
81      */
82     fun isLanding(
83         groundY: Float = 0.85f,

```

```

84     posThreshold: Float = 0.05f,
85     minVy: Float = 0.05f,      // normalized units/sec moving down
86     maxVy: Float = 5.0f,
87 ): Boolean {
88     val nearGround = abs(stateY - groundY) < posThreshold
89     val movingDown = vy in minVy..maxVy
90     return nearGround && movingDown
91 }
92
93 /**
94  * Toe-off detection: ankle moving upward or already above ground.
95  */
96 fun isToeOff(
97     groundY: Float = 0.85f,
98     posThreshold: Float = 0.04f,
99     minUpVy: Float = 0.05f,    // vy < -minUpVy means moving up
100 ): Boolean {
101     val movingUp = vy < -minUpVy
102     val aboveGround = stateY < groundY - posThreshold
103     return movingUp || aboveGround
104 }
105
106 fun reset(x: Float, y: Float) {
107     stateX = x; stateY = y
108     vx = 0f; vy = 0f
109     px = 0.1f; py = 0.1f; pvx = 0.1f; pvy = 0.1f
110     lastTimestampMs = -1L
111 }
112 }

```



```

1 package com.runway.android.core.posture
2
3 import kotlin.math.sqrt
4
5 /**
6  * MediaPipe Pose Landmarker의 좌우 다리 식별 오류 (특히 다리 교차 시점) 보정 레이어.
7  *
8  * 파이프라인 위치:
9  * raw landmarks → (confidence validation) → LegSwapCorrector → temporal smoothing → angles
10 *
11 * 동작:
12 * 1. 이전 프레임의 보정된 좌/우 무릎·발목 위치를 보관한다.
13 * 2. 현재 프레임에서
14 *   keepCost = 거리(현재 L → 이전 L) + 거리(현재 R → 이전 R) // 무릎+발목 합산
15 *   swapCost = 거리(현재 L → 이전 R) + 거리(현재 R → 이전 L)
16 * 3. swapCost + HYSTERESIS_MARGIN < keepCost 가 N_CONFIRM 프레임 연속이면 식별을 토글한다.
17 * 4. 토글된 식별로 좌/우 hip/knee/ankle 을 함께 스왑한다.
18 *
19 * 안전장치:
20 * - 다리 평균 visibility 가 MIN_LEG_VISIBILITY 미만이면 swap 평가를 건너뛰다 (이전 결정 유지) .
21 * - 타임스탬프 갭이 MAX_DT_MS 를 넘으면 (seek/재시작) 상태를 리셋한다.
22 *
23 * 분석 파이프라인 1회 사용 가정. 재생 단계에서는 호출하지 않는다(분석 시점에 영구 적용) .
24 */
25 class LegSwapCorrector {
26
27     companion object {
28         /** swap 이 keep 보다 이만큼 작아야 후보로 인정 (normalized 좌표 합) . */
29         private const val HYSTERESIS_MARGIN = 0.15f
30
31         /** swap 후보가 연속 충족되어야 하는 프레임 수. */
32         private const val N_CONFIRM = 2
33
34         /** 다리 4개 landmark 평균 visibility 가 이 값 미만이면 swap 평가 보류. */
35         private const val MIN_LEG_VISIBILITY = 0.40f
36
37         /** 이 값을 넘는 타임스탬프 갭(ms) 발생 시 상태 리셋. */
38         private const val MAX_DT_MS = 500L
39     }
40
41     // 이전 프레임의 "보정된 식별 기준" 좌/우 무릎·발목 위치.
42     private var prevLeftKnee: SkeletonPoint? = null
43     private var prevRightKnee: SkeletonPoint? = null
44     private var prevLeftAnkle: SkeletonPoint? = null
45     private var prevRightAnkle: SkeletonPoint? = null
46
47     // 현재 누적된 식별 상태. true 이면 MediaPipe 의 L/R 을 스왑해서 출력해야 한다.
48     private var isSwapped = false
49     private var pendingSwapVotes = 0
50     private var lastTimestampMs: Long = -1L
51
52     /**
53      * [pts] 는 KEY_LANDMARK_INDICES 순서(13 keypoints). 길이는 R_ANKLE(12) 이상이어야 한다.
54      * [timestampMs] 는 비디오 타임라인 기준 (분석 단계에서는 timeMs).
55      */
56     fun correct(pts: List<SkeletonPoint>, timestampMs: Long): List<SkeletonPoint> {
57         if (pts.size <= SKEL_R_ANKLE) return pts
58
59         // 큰 타임스탬프 갭 → seek/재시작으로 보고 상태 리셋.
60         if (lastTimestampMs >= 0L) {
61             val gap = timestampMs - lastTimestampMs
62             if (gap < 0L || gap > MAX_DT_MS) reset()
63         }
64
65         // 현재 isSwapped 상태가 적용된 좌/우 식별값.
66         val curLK = if (!isSwapped) pts[SKEL_L_KNEE] else pts[SKEL_R_KNEE]
67         val curRK = if (!isSwapped) pts[SKEL_R_KNEE] else pts[SKEL_L_KNEE]
68         val curLA = if (!isSwapped) pts[SKEL_L_ANKLE] else pts[SKEL_R_ANKLE]
69         val curRA = if (!isSwapped) pts[SKEL_R_ANKLE] else pts[SKEL_L_ANKLE]
70
71         val pLK = prevLeftKnee; val pRK = prevRightKnee
72         val pLA = prevLeftAnkle; val pRA = prevRightAnkle
73
74         // 첫 프레임: 이전 값 초기화 후 통과.
75         if (pLK == null || pRK == null || pLA == null || pRA == null) {
76             prevLeftKnee = curLK; prevRightKnee = curRK
77             prevLeftAnkle = curLA; prevRightAnkle = curRA
78             lastTimestampMs = timestampMs
79             return if (isSwapped) swapLowerBody(pts) else pts
80         }
81
82         // Confidence validation: 다리 평균 visibility 가 낮으면 swap 평가 보류.
83         val avgVis = (pts[SKEL_L_KNEE].v + pts[SKEL_R_KNEE].v +

```

```

84         pts[SKEL_L_ANKLE].v + pts[SKEL_R_ANKLE].v) / 4f
85     if (avgVis < MIN_LEG_VISIBILITY) {
86         // 이전 상태 유지: pendingSwapVotes/prev 갱신하지 않는다.
87         lastTimestampMs = timestampMs
88         return if (isSwapped) swapLowerBody(pts) else pts
89     }
90
91     // 거리 기반 비용 비교.
92     val keepCost = dist(curLK, pLK) + dist(curLA, pLA) +
93         dist(curRK, pRK) + dist(curRA, pRA)
94     val swapCost = dist(curLK, pRK) + dist(curLA, pRA) +
95         dist(curRK, pLK) + dist(curRA, pLA)
96
97     if (swapCost + HYSTERESIS_MARGIN < keepCost) {
98         pendingSwapVotes++
99         if (pendingSwapVotes >= N_CONFIRM) {
100             isSwapped = !isSwapped
101             pendingSwapVotes = 0
102         }
103     } else {
104         pendingSwapVotes = 0
105     }
106
107     val output = if (isSwapped) swapLowerBody(pts) else pts
108
109     // 다음 프레임 비교를 위해 "출력된 식별" 기준으로 이전 값 갱신.
110     prevLeftKnee = output[SKEL_L_KNEE]
111     prevRightKnee = output[SKEL_R_KNEE]
112     prevLeftAnkle = output[SKEL_L_ANKLE]
113     prevRightAnkle = output[SKEL_R_ANKLE]
114     lastTimestampMs = timestampMs
115
116     return output
117 }
118
119 fun reset() {
120     prevLeftKnee = null; prevRightKnee = null
121     prevLeftAnkle = null; prevRightAnkle = null
122     isSwapped = false
123     pendingSwapVotes = 0
124     lastTimestampMs = -1L
125 }
126
127 /** 좌/우 hip + knee + ankle 을 한 번에 스왑. 다리 geometry 가 분리되지 않도록 함께 처리. */
128 private fun swapLowerBody(pts: List<SkeletonPoint>): List<SkeletonPoint> {
129     val out = pts.toMutableList()
130     out[SKEL_L_HIP] = pts[SKEL_R_HIP]; out[SKEL_R_HIP] = pts[SKEL_L_HIP]
131     out[SKEL_L_KNEE] = pts[SKEL_R_KNEE]; out[SKEL_R_KNEE] = pts[SKEL_L_KNEE]
132     out[SKEL_L_ANKLE] = pts[SKEL_R_ANKLE]; out[SKEL_R_ANKLE] = pts[SKEL_L_ANKLE]
133     return out
134 }
135
136 private fun dist(a: SkeletonPoint, b: SkeletonPoint): Float {
137     val dx = a.x - b.x; val dy = a.y - b.y
138     return sqrt(dx * dx + dy * dy)
139 }
140 }

```

```

1 package com.runway.android.core.posture
2
3 import kotlin.math.acos
4 import kotlin.math.atan2
5 import kotlin.math.sqrt
6
7 /**
8  * Android port of running-form-analyzer/utils/angle_calculator.py.
9  *
10 * Both body sides are calculated independently. Missing low-confidence values
11 * remain null so the stateful metrics pipeline can preserve the previous value,
12 * matching the original Python angle dictionary.
13 */
14 object PostureAngleCalculator {
15     private const val CONFIDENCE_THRESHOLD = 0.30f
16
17     fun compute(landmarks: List<SkeletonPoint>, aspectRatio: Float = 1f): PostureFrameAngles? {
18         if (landmarks.size <= SKEL_R_ANKLE) return null
19         val ar = aspectRatio.coerceAtLeast(0.1f)
20
21         fun point(index: Int): FloatArray? {
22             val landmark = landmarks[index]
23             if (landmark.v <= CONFIDENCE_THRESHOLD) return null
24             return floatArrayOf(landmark.x * ar, landmark.y)
25         }
26
27         fun angle(a: Int, vertex: Int, c: Int): Float? {
28             val pa = point(a) ?: return null
29             val pv = point(vertex) ?: return null
30             val pc = point(c) ?: return null
31             return angleBetween(
32                 floatArrayOf(pv[0] - pa[0], pv[1] - pa[1]),
33                 floatArrayOf(pv[0] - pc[0], pv[1] - pc[1]),
34             )
35         }
36
37         fun trunk(shoulderIndex: Int, hipIndex: Int): Float? {
38             val shoulder = point(shoulderIndex) ?: return null
39             val hip = point(hipIndex) ?: return null
40             return Math.toDegrees(
41                 atan2(
42                     (shoulder[0] - hip[0]).toDouble(),
43                     (hip[1] - shoulder[1]).toDouble(),
44                 ),
45             ).toFloat()
46         }
47
48         fun verticalAngle(fromIndex: Int, toIndex: Int): Float? {
49             val from = point(fromIndex) ?: return null
50             val to = point(toIndex) ?: return null
51             return angleBetween(
52                 floatArrayOf(0f, 1f),
53                 floatArrayOf(to[0] - from[0], to[1] - from[1]),
54             )
55         }
56
57         fun signedSwing(originIndex: Int, limbIndex: Int, torsoIndex: Int): Float? {
58             val origin = point(originIndex) ?: return null
59             val limb = point(limbIndex) ?: return null
60             val torso = point(torsoIndex) ?: return null
61             val limbVector = floatArrayOf(limb[0] - origin[0], limb[1] - origin[1])
62             val value = angleBetween(
63                 floatArrayOf(origin[0] - torso[0], origin[1] - torso[1]),
64                 limbVector,
65             )
66             return if (limbVector[0] < 0f) -value else value
67         }
68
69         fun signedHipSwing(shoulderIndex: Int, hipIndex: Int, kneeIndex: Int): Float? {
70             val shoulder = point(shoulderIndex) ?: return null
71             val hip = point(hipIndex) ?: return null
72             val knee = point(kneeIndex) ?: return null
73             val thighVector = floatArrayOf(knee[0] - hip[0], knee[1] - hip[1])
74             val value = angleBetween(
75                 floatArrayOf(hip[0] - shoulder[0], hip[1] - shoulder[1]),
76                 thighVector,
77             )
78             return if (thighVector[0] < 0f) -value else value
79         }
80
81         val leftKnee = angle(SKEL_L_HIP, SKEL_L_KNEE, SKEL_L_ANKLE)
82         val rightKnee = angle(SKEL_R_HIP, SKEL_R_KNEE, SKEL_R_ANKLE)
83         val leftElbow = angle(SKEL_L_SHOULDER, SKEL_L_ELLOW, SKEL_L_WRIST)

```

```

84     val rightElbow = angle(SKELETON_R_SHOULDER, SKELETON_R_ELBOW, SKELETON_R_WRIST)
85     val leftHip = signedHipSwing(SKELETON_L_SHOULDER, SKELETON_L_HIP, SKELETON_L_KNEE)
86     val rightHip = signedHipSwing(SKELETON_R_SHOULDER, SKELETON_R_HIP, SKELETON_R_KNEE)
87     val leftHipAnkle = verticalAngle(SKELETON_L_HIP, SKELETON_L_ANKLE)
88     val rightHipAnkle = verticalAngle(SKELETON_R_HIP, SKELETON_R_ANKLE)
89     val leftShank = verticalAngle(SKELETON_L_KNEE, SKELETON_L_ANKLE)
90     val rightShank = verticalAngle(SKELETON_R_KNEE, SKELETON_R_ANKLE)
91     val leftArmSwing = signedSwing(SKELETON_L_SHOULDER, SKELETON_L_ELBOW, SKELETON_L_HIP)
92     val rightArmSwing = signedSwing(SKELETON_R_SHOULDER, SKELETON_R_ELBOW, SKELETON_R_HIP)
93
94     val leftVisibility = averageVisibility(
95         landmarks,
96         SKELETON_L_SHOULDER, SKELETON_L_ELBOW, SKELETON_L_WRIST,
97         SKELETON_L_HIP, SKELETON_L_KNEE, SKELETON_L_ANKLE,
98     )
99     val rightVisibility = averageVisibility(
100         landmarks,
101         SKELETON_R_SHOULDER, SKELETON_R_ELBOW, SKELETON_R_WRIST,
102         SKELETON_R_HIP, SKELETON_R_KNEE, SKELETON_R_ANKLE,
103     )
104     if (maxOf(leftVisibility, rightVisibility) <= CONFIDENCE_THRESHOLD) return null
105
106     val useLeft = leftVisibility >= rightVisibility
107     val trunkAngle = if (useLeft) {
108         trunk(SKELETON_L_SHOULDER, SKELETON_L_HIP)
109     } else {
110         trunk(SKELETON_R_SHOULDER, SKELETON_R_HIP)
111     }
112     val hipMidY = (landmarks[SKELETON_L_HIP].y + landmarks[SKELETON_R_HIP].y) / 2f
113
114     return PostureFrameAngles(
115         kneeFlexAngle = (if (useLeft) leftKnee else rightKnee) ?: 0f,
116         trunkLeanAngle = trunkAngle ?: 0f,
117         elbowAngle = (if (useLeft) leftElbow else rightElbow) ?: 0f,
118         hipExtensionAngle = (if (useLeft) leftHip else rightHip) ?: 0f,
119         overstrideRatio = (if (useLeft) leftHipAnkle else rightHipAnkle) ?: 0f,
120         isLandingFrame = false,
121         visibility = maxOf(leftVisibility, rightVisibility),
122         hipMidY = hipMidY,
123         nearAnkleY = landmarks[(if (useLeft) SKELETON_L_ANKLE else SKELETON_R_ANKLE).y],
124         shankAngle = (if (useLeft) leftShank else rightShank) ?: 0f,
125         armSwingAngle = (if (useLeft) leftArmSwing else rightArmSwing) ?: 0f,
126         leftKneeAngle = leftKnee,
127         rightKneeAngle = rightKnee,
128         leftElbowAngle = leftElbow,
129         rightElbowAngle = rightElbow,
130         leftHipAngle = leftHip,
131         rightHipAngle = rightHip,
132         leftHipAnkleAngle = leftHipAnkle,
133         rightHipAnkleAngle = rightHipAnkle,
134         leftShankAngle = leftShank,
135         rightShankAngle = rightShank,
136         leftArmSwingAngle = leftArmSwing,
137         rightArmSwingAngle = rightArmSwing,
138     )
139 }
140
141 private fun angleBetween(first: FloatArray, second: FloatArray): Float {
142     val dot = first[0] * second[0] + first[1] * second[1]
143     val firstMagnitude = sqrt(first[0] * first[0] + first[1] * first[1])
144     val secondMagnitude = sqrt(second[0] * second[0] + second[1] * second[1])
145     if (firstMagnitude < 1e-6f || secondMagnitude < 1e-6f) return 0f
146     return Math.toDegrees(
147         acos((dot / (firstMagnitude * secondMagnitude)).coerceIn(-1f, 1f)).toDouble(),
148     ).toFloat()
149 }
150
151 private fun averageVisibility(
152     landmarks: List<SkeletonPoint>,
153     vararg indices: Int,
154 ): Float = indices.map { landmarks[it].v }.average().toFloat()
155 }

```

android/app/src/main/java/com/runway/android/core/posture/PostureAutoencoderInference.kt

Kotlin | 19줄

```
1 package com.runway.android.core.posture
2
3 // Phase P-2: replace PostureRuleEngine with this class once posture_autoencoder.tflite is trained.
4 // Training pipeline: tools/posture-model-training/train/train_autoencoder.py
5 // Swap in PostureModule.kt: bind PostureAutoencoderInference instead of PostureRuleEngine.
6 //
7 // Architecture: Dense autoencoder trained ONLY on good-posture sequences.
8 //   Encoder: Dense(32,relu) → Dense(16,relu) → latent(8)
9 //   Decoder: Dense(16,relu) → Dense(32,relu) → Dense(5,linear)
10 // Reconstruction error (MSE per dimension) → per-category score.
11 class PostureAutoencoderInference : PostureEvaluator {
12
13     override fun evaluate(frames: List<PostureFrameAngles>): PostureResult {
14         throw UnsupportedOperationException(
15             "PostureAutoencoderInference requires posture_autoencoder.tflite. " +
16             "Use PostureRuleEngine until Phase P-2 model is ready."
17         )
18     }
19 }
```

android/app/src/main/java/com/runway/android/core/posture/PostureCategoryResult.kt

Kotlin | 11줄

```
1 package com.runway.android.core.posture
2
3 data class PostureCategoryResult(
4     val score: Int,
5     val measuredValue: Float,
6     val idealMin: Float,
7     val idealMax: Float,
8     val unit: String,
9     val feedback: String,
10    val tip: String,
11 )
```

android/app/src/main/java/com/runway/android/core/posture/PostureEvaluator.kt

Kotlin | 5줄

```
1 package com.runway.android.core.posture
2
3 interface PostureEvaluator {
4     fun evaluate(frames: List<PostureFrameAngles>): PostureResult
5 }
```

android/app/src/main/java/com/runway/android/core/posture/PostureFrameAngles.kt

Kotlin | 33줄

```

1 package com.runway.android.core.posture
2
3 data class PostureFrameAngles(
4     val kneeFlexAngle: Float,
5     val trunkLeanAngle: Float,
6     val elbowAngle: Float,
7     val hipExtensionAngle: Float,
8     val overstrideRatio: Float,
9     val isLandingFrame: Boolean,
10    val visibility: Float,
11    val timestampMs: Long = 0L,
12    val landmarks: List<SkeletonPoint> = emptyList(),
13    // Bilateral signals for cadence and vertical-oscillation reference metrics
14    val hipMidY: Float = 0f, // avg of left+right hip Y; tracks vertical bounce
15    val nearAnkleY: Float = 0f, // camera-side ankle Y; peaks mark ground contacts
16    // running-form-analyzer additions
17    val shankAngle: Float = 0f, // shin angle from vertical at landing (0° = perfectly vertical)
18    val armSwingAngle: Float = 0f, // upper-arm vs torso angle
19    val leftKneeAngle: Float? = null,
20    val rightKneeAngle: Float? = null,
21    val leftElbowAngle: Float? = null,
22    val rightElbowAngle: Float? = null,
23    val leftHipAngle: Float? = null,
24    val rightHipAngle: Float? = null,
25    val leftHipAnkleAngle: Float? = null,
26    val rightHipAnkleAngle: Float? = null,
27    val leftShankAngle: Float? = null,
28    val rightShankAngle: Float? = null,
29    val leftArmSwingAngle: Float? = null,
30    val rightArmSwingAngle: Float? = null,
31    val leftFootStrike: Boolean = false,
32    val rightFootStrike: Boolean = false,
33 )

```

android/app/src/main/java/com/runway/android/core/posture/PostureFrameInterpolator.kt

Kotlin | 44줄

```

1 package com.runway.android.core.posture
2
3 /**
4  * Binary search for the first frame with timestamp strictly greater than [posMs].
5  * Mirrors Python's bisect_right. O(log n).
6  */
7 internal fun bisectRight(frames: List<PostureVideoFrame>, posMs: Long): Int {
8     var lo = 0; var hi = frames.size
9     while (lo < hi) {
10         val mid = (lo + hi) ushr 1
11         if (frames[mid].t <= posMs) lo = mid + 1 else hi = mid
12     }
13     return lo
14 }
15
16 /**
17  * Returns a [PostureVideoFrame] linearly interpolated to [posMs].
18  * [sortedFrames] must be sorted by [PostureVideoFrame.t] ascending.
19  * Returns null only when [sortedFrames] is empty.
20  */
21 internal fun interpolateFrame(sortedFrames: List<PostureVideoFrame>, posMs: Long): PostureVideoFrame? {
22     if (sortedFrames.isEmpty()) return null
23     if (sortedFrames.size == 1) return sortedFrames[0]
24
25     val idx = bisectRight(sortedFrames, posMs)
26     val beforeIdx = (idx - 1).coerceAtLeast(0)
27     val before = sortedFrames[beforeIdx]
28     val after = sortedFrames.getOrNull(idx) ?: return before
29
30     val range = (after.t - before.t).toFloat()
31     if (range <= 0f) return before
32
33     if (before.pts.size != after.pts.size) return before
34
35     val t = ((posMs - before.t) / range).coerceIn(0f, 1f)
36     val pts = before.pts.zip(after.pts).map { (a, b) ->
37         SkeletonPoint(
38             x = a.x + (b.x - a.x) * t,
39             y = a.y + (b.y - a.y) * t,
40             v = a.v + (b.v - a.v) * t,
41         )
42     }
43     return PostureVideoFrame(posMs, pts)
44 }

```

```

1  package com.runway.android.core.posture
2
3  import android.util.Log
4  import kotlin.math.abs
5  import kotlin.math.sqrt
6
7  private const val TAG = "LandmarkCorrector"
8
9  /**
10 * MediaPipe 측면 촬영 무릎·발목 전방 바이어스 보정 + 해부학적 유효성 검사.
11 *
12 * 파이프라인:
13 * 1. One Euro Filter 평활화
14 * 2. 해부학적 유효성 검사 (골반 기준 좌표 범위 위반 시 이전 유효값으로 대체)
15 * 3. 측면 프로파일 전방 오프셋 보정 (무릎·발목 x 좌표 후방 이동)
16 *
17 * 유효성 검사가 없으면, 먼 쪽(far-side) 다리가 러닝머신 기구나 배경 구조물에
18 * 감지되어도 그대로 표시된다.
19 */
20 class PostureLandmarkCorrector {
21     private val smoother = PostureSkeletonSmoother(
22         minCutoff = 1.2f,
23         beta = 4.0f,
24         lowVisFreezeThreshold = 0.5f,
25     )
26     private var facingDirection = 0f
27
28     // 해부학적으로 유효했던 마지막 무릎·발목 위치 (fallback용)
29     private var prevLKnee: SkeletonPoint? = null
30     private var prevRKnee: SkeletonPoint? = null
31     private var prevLAnkle: SkeletonPoint? = null
32     private var prevRAnkle: SkeletonPoint? = null
33
34     fun correct(raw: List<SkeletonPoint>, timestampMs: Long): List<SkeletonPoint> {
35         if (raw.size <= SKEL_R_ANKLE) return raw
36
37         val stable = smoother.smooth(raw, timestampMs)
38
39         // — 방향 추정 —————
40         val hipMidX = (stable[SKEL_L_HIP].x + stable[SKEL_R_HIP].x) / 2f
41         val hipMidY = (stable[SKEL_L_HIP].y + stable[SKEL_R_HIP].y) / 2f
42         val shoulderMidY = (stable[SKEL_L_SHOULDER].y + stable[SKEL_R_SHOULDER].y) / 2f
43         val torsoHeight = abs(hipMidY - shoulderMidY).coerceAtLeast(MIN_BODY_SCALE)
44         val noseOffset = stable[SKEL_NOSE].x - hipMidX
45         val sideProfileRatio = abs(noseOffset) / torsoHeight
46
47         if (sideProfileRatio >= FACING_UPDATE_RATIO && stable[SKEL_NOSE].v >= MIN_DIRECTION_VISIBILITY) {
48             facingDirection = if (noseOffset >= 0f) 1f else -1f
49         }
50
51         // — 해부학적 유효성 검사 —————
52         // 골반을 anchor로 사용한다. 무릎·발목이 골반에서 비정상적으로 멀면
53         // 러닝머신 기구 등 배경 오감지로 판단하고 이전 유효값으로 대체한다.
54         val lHip = stable[SKEL_L_HIP]
55         val rHip = stable[SKEL_R_HIP]
56
57         val lKneeRaw = stable[SKEL_L_KNEE]
58         val rKneeRaw = stable[SKEL_R_KNEE]
59         val lAnkleRaw = stable[SKEL_L_ANKLE]
60         val rAnkleRaw = stable[SKEL_R_ANKLE]
61
62         val lKnee = anatomicallyValid(lKneeRaw, lHip, isKnee = true, prev = prevLKnee, label = "L_KNEE")
63         val rKnee = anatomicallyValid(rKneeRaw, rHip, isKnee = true, prev = prevRKnee, label = "R_KNEE")
64         val lAnkle = anatomicallyValid(lAnkleRaw, lHip, isKnee = false, prev = prevLAnkle, label = "L_ANKLE")
65         val rAnkle = anatomicallyValid(rAnkleRaw, rHip, isKnee = false, prev = prevRAnkle, label = "R_ANKLE")
66
67         // 이번 프레임의 유효값을 다음 프레임 fallback으로 저장
68         prevLKnee = lKnee
69         prevRKnee = rKnee
70         prevLAnkle = lAnkle
71         prevRAnkle = rAnkle
72
73         val validated = stable.toMutableList().apply {
74             this[SKEL_L_KNEE] = lKnee
75             this[SKEL_R_KNEE] = rKnee
76             this[SKEL_L_ANKLE] = lAnkle
77             this[SKEL_R_ANKLE] = rAnkle
78         }
79
80         // — 측면 프로파일 전방 오프셋 보정 —————
81         if (facingDirection == 0f || sideProfileRatio <= SIDE_PROFILE_START_RATIO) {
82             return validated
83         }

```

```

84
85     val correctionStrength = (
86         (sideProfileRatio - SIDE_PROFILE_START_RATIO) /
87         (SIDE_PROFILE_FULL_RATIO - SIDE_PROFILE_START_RATIO)
88     ).coerceIn(0f, 1f)
89
90     val lFar = facingDirection > 0f
91     val lKStr = if (lFar) correctionStrength * FAR_SIDE_SCALE else correctionStrength
92     val rKStr = if (!lFar) correctionStrength * FAR_SIDE_SCALE else correctionStrength
93     val lAstr = if (lFar) correctionStrength * FAR_SIDE_SCALE else correctionStrength
94     val rAstr = if (!lFar) correctionStrength * FAR_SIDE_SCALE else correctionStrength
95
96     return validated.toMutableList().apply {
97         this[SKEL_L_KNEE] = offsetJoint(validated[SKEL_L_KNEE], lHip, validated[SKEL_L_ANKLE], lKStr, KNEE_OFFSET_RATIO, MIN_K
NEE_OFFSET, MAX_KNEE_OFFSET)
98         this[SKEL_R_KNEE] = offsetJoint(validated[SKEL_R_KNEE], rHip, validated[SKEL_R_ANKLE], rKStr, KNEE_OFFSET_RATIO, MIN_K
NEE_OFFSET, MAX_KNEE_OFFSET)
99         this[SKEL_L_ANKLE] = offsetJoint(validated[SKEL_L_ANKLE], lHip, validated[SKEL_L_ANKLE], lAstr, ANKLE_OFFSET_RATIO, MIN_A
NKLE_OFFSET, MAX_ANKLE_OFFSET)
100        this[SKEL_R_ANKLE] = offsetJoint(validated[SKEL_R_ANKLE], rHip, validated[SKEL_R_ANKLE], rAstr, ANKLE_OFFSET_RATIO, MIN_A
NKLE_OFFSET, MAX_ANKLE_OFFSET)
101    }
102 }
103
104 fun reset() {
105     smoother.reset()
106     facingDirection = 0f
107     prevLKnee = null; prevRKnee = null
108     prevLAnkle = null; prevRAnkle = null
109 }
110
111 /**
112  * [joint] 가 [hip] 으로부터 해부학적으로 가능한 범위 내에 있으면 그대로 반환하고,
113  * 벗어나면 [prev] (이전 유효값) 을 반환한다.
114  *
115  * 무릎: 골반 x 기준 ±MAX_KNEE_X, 골반 y 보다 아래여야 함.
116  * 발목: 골반 x 기준 ±MAX_ANKLE_X.
117  */
118 private fun anatomicallyValid(
119     joint: SkeletonPoint,
120     hip: SkeletonPoint,
121     isKnee: Boolean,
122     prev: SkeletonPoint?,
123     label: String,
124 ): SkeletonPoint {
125     val xDist = abs(joint.x - hip.x)
126     val maxX = if (isKnee) MAX_KNEE_X_FROM_HIP else MAX_ANKLE_X_FROM_HIP
127
128     // 무릎 y 검사:
129     // - 골반보다 MIN_KNEE_BELOW_HIP 이상 아래에 있어야 유효 (너무 높은 감지 제거)
130     // - 달리기 고강도 무릎 드라이브에서도 무릎은 항상 골반보다 아래에 위치한다
131     val yOk = if (isKnee) joint.y >= hip.y + MIN_KNEE_BELOW_HIP else true
132
133     val valid = xDist <= maxX && yOk
134
135     if (!valid) {
136         val fallback = prev?: run {
137             // 첫 프레임에 오감지된 경우: 골반 아래 추정 위치로 초기화
138             val estimatedY = if (isKnee) hip.y + KNEE_INIT_BELOW_HIP
139                             else hip.y + ANKLE_INIT_BELOW_HIP
140             hip.copy(y = estimatedY.coerceIn(0f, 1f), v = 0.3f)
141         }
142         Log.d(TAG, "[${label}] rejected: xDist=${"%0.3f".format(xDist)} maxX=$maxX " +
143             "joint.y=${joint.y} hip.y=${hip.y} yOk=$yOk → " +
144             if (prev != null) "prev(${prev.x},${prev.y})" else "estimated(${fallback.x},${fallback.y})")
145         return fallback
146     }
147     return joint
148 }
149
150 /** 측면 프로파일 기준 관절 x 좌표를 달리기 방향 반대로 이동 (전방 바이어스 제거). */
151 private fun offsetJoint(
152     joint: SkeletonPoint,
153     hip: SkeletonPoint,
154     ankle: SkeletonPoint,
155     strength: Float,
156     ratioBase: Float,
157     minOff: Float,
158     maxOff: Float,
159 ): SkeletonPoint {
160     if (minOf(hip.v, joint.v, ankle.v) < MIN_LIMB_VISIBILITY) return joint
161     val legLength = distance(hip, ankle)
162     val offset = (legLength * ratioBase).coerceIn(minOff, maxOff) * strength
163     return joint.copy(x = (joint.x - facingDirection * offset).coerceIn(0f, 1f))
164 }
165

```



```

166 private fun distance(a: SkeletonPoint, b: SkeletonPoint): Float {
167     val dx = a.x - b.x; val dy = a.y - b.y
168     return sqrt(dx * dx + dy * dy)
169 }
170
171 private companion object {
172     const val MIN_BODY_SCALE = 0.05f
173     const val MIN_DIRECTION_VISIBILITY = 0.5f
174     const val MIN_LIMB_VISIBILITY = 0.45f
175     const val FACING_UPDATE_RATIO = 0.12f
176     const val SIDE_PROFILE_START_RATIO = 0.08f
177     const val SIDE_PROFILE_FULL_RATIO = 0.28f
178
179     // 해부학적 유효성 임계값 (normalized 0-1 좌표 기준)
180     // 무릎: 골반 x ± 20% 이내, 골반 y 보다 최소 3% 아래이어야 유효
181     const val MAX_KNEE_X_FROM_HIP = 0.20f
182     const val MIN_KNEE_BELOW_HIP = 0.03f // knee.y >= hip.y + 0.03 이어야 유효
183     const val KNEE_INIT_BELOW_HIP = 0.15f // 첫 프레임 오감지 시 초기 추정값
184     // 발목: 달리기 보폭만큼 더 큰 x 범위 허용
185     const val MAX_ANKLE_X_FROM_HIP = 0.30f
186     const val ANKLE_INIT_BELOW_HIP = 0.30f // 첫 프레임 오감지 시 초기 추정값
187
188     // 전방 바이어스 오프셋 보정값
189     const val KNEE_OFFSET_RATIO = 0.06f
190     const val MIN_KNEE_OFFSET = 0.006f
191     const val MAX_KNEE_OFFSET = 0.030f
192     const val ANKLE_OFFSET_RATIO = 0.04f
193     const val MIN_ANKLE_OFFSET = 0.004f
194     const val MAX_ANKLE_OFFSET = 0.022f
195     const val FAR_SIDE_SCALE = 1.5f
196 }
197 }

```

```

1 package com.runway.android.core.posture
2
3 import kotlin.math.sqrt
4
5 /**
6  * Rejects transient lower-body detections that attach a leg to background
7  * objects. Invalid joints are hidden instead of drawing a fabricated limb.
8  */
9 class PostureLandmarkValidator {
10     private var previous: List<SkeletonPoint>? = null
11
12     fun validate(points: List<SkeletonPoint>): List<SkeletonPoint> {
13         if (points.size <= SKEL_R_ANKLE) return points
14
15         val output = points.toMutableList()
16         val torsoScale = distance(
17             midpoint(points[SKEL_L_SHOULDER], points[SKEL_R_SHOULDER]),
18             midpoint(points[SKEL_L_HIP], points[SKEL_R_HIP]),
19         ).coerceAtLeast(MIN_TORSO_SCALE)
20
21         validateMotion(output, previous, torsoScale)
22         validateLegLength(output, torsoScale)
23         validateLegLengthBalance(output)
24         previous = output
25         return output
26     }
27
28     fun reset() {
29         previous = null
30     }
31
32     private fun validateMotion(
33         points: MutableList<SkeletonPoint>,
34         old: List<SkeletonPoint>?,
35         torsoScale: Float,
36     ) {
37         if (old == null || old.size != points.size) return
38
39         rejectJump(points, old, SKEL_L_KNEE, torsoScale * MAX_KNEE_TRAVEL)
40         rejectJump(points, old, SKEL_R_KNEE, torsoScale * MAX_KNEE_TRAVEL)
41         rejectJump(points, old, SKEL_L_ANKLE, torsoScale * MAX_ANKLE_TRAVEL)
42         rejectJump(points, old, SKEL_R_ANKLE, torsoScale * MAX_ANKLE_TRAVEL)
43     }
44
45     private fun rejectJump(
46         points: MutableList<SkeletonPoint>,
47         old: List<SkeletonPoint>,
48         index: Int,
49         maxTravel: Float,
50     ) {
51         val current = points[index]
52         val previousPoint = old[index]
53         if (previousPoint.v >= MIN_VISIBILITY && distance(current, previousPoint) > maxTravel) {
54             points[index] = current.copy(v = 0f)
55         }
56     }
57
58     private fun validateLegLengthBalance(points: MutableList<SkeletonPoint>) {
59         val leftLength = legLength(points, SKEL_L_HIP, SKEL_L_KNEE, SKEL_L_ANKLE)
60         val rightLength = legLength(points, SKEL_R_HIP, SKEL_R_KNEE, SKEL_R_ANKLE)
61         if (leftLength <= 0f || rightLength <= 0f) return
62
63         val shorterRatio = minOf(leftLength, rightLength) / maxOf(leftLength, rightLength)
64         if (shorterRatio >= MIN_BILATERAL_LEG_RATIO) return
65
66         if (leftLength < rightLength) {
67             hideLeg(points, SKEL_L_KNEE, SKEL_L_ANKLE)
68         } else {
69             hideLeg(points, SKEL_R_KNEE, SKEL_R_ANKLE)
70         }
71     }
72
73     private fun validateLegLength(points: MutableList<SkeletonPoint>, torsoScale: Float) {
74         val minimumLegLength = torsoScale * MIN_LEG_TO_TORSO_RATIO
75         val leftLength = legLength(points, SKEL_L_HIP, SKEL_L_KNEE, SKEL_L_ANKLE)
76         val rightLength = legLength(points, SKEL_R_HIP, SKEL_R_KNEE, SKEL_R_ANKLE)
77
78         if (leftLength in 0f..<minimumLegLength) {
79             hideLeg(points, SKEL_L_KNEE, SKEL_L_ANKLE)
80         }
81         if (rightLength in 0f..<minimumLegLength) {
82             hideLeg(points, SKEL_R_KNEE, SKEL_R_ANKLE)
83         }
84     }

```

```

84     }
85
86     private fun legLength(
87         points: List<SkeletonPoint>,
88         hipIndex: Int,
89         kneeIndex: Int,
90         ankleIndex: Int,
91     ): Float {
92         val hip = points[hipIndex]
93         val knee = points[kneeIndex]
94         val ankle = points[ankleIndex]
95         if (minOf(hip.v, knee.v, ankle.v) < MIN_VISIBILITY) return 0f
96         return distance(hip, knee) + distance(knee, ankle)
97     }
98
99     private fun hideLeg(points: MutableList<SkeletonPoint>, kneeIndex: Int, ankleIndex: Int) {
100         points[kneeIndex] = points[kneeIndex].copy(v = 0f)
101         points[ankleIndex] = points[ankleIndex].copy(v = 0f)
102     }
103
104     private fun midpoint(first: SkeletonPoint, second: SkeletonPoint) = SkeletonPoint(
105         x = (first.x + second.x) / 2f,
106         y = (first.y + second.y) / 2f,
107         v = minOf(first.v, second.v),
108     )
109
110     private fun distance(first: SkeletonPoint, second: SkeletonPoint): Float {
111         val dx = first.x - second.x
112         val dy = first.y - second.y
113         return sqrt(dx * dx + dy * dy)
114     }
115
116     private companion object {
117         const val MIN_VISIBILITY = 0.30f
118         const val MIN_TORSO_SCALE = 0.08f
119         const val MAX_KNEE_TRAVEL = 0.45f
120         const val MAX_ANKLE_TRAVEL = 0.70f
121         const val MIN_LEG_TO_TORSO_RATIO = 1.25f
122         const val MIN_BILATERAL_LEG_RATIO = 0.78f
123     }
124 }

```

```

1  package com.runway.android.core.posture
2
3  import android.content.Context
4  import android.graphics.Bitmap
5  import android.graphics.Matrix
6  import android.media.MediaMetadataRetriever
7  import android.net.Uri
8  import android.os.Build
9  import android.os.SystemClock
10 import android.util.Log
11 import com.google.mediarpipe.framework.image.BitmapImageBuilder
12 import com.google.mediarpipe.tasks.core.BaseOptions
13 import com.google.mediarpipe.tasks.vision.core.RunningMode
14 import com.google.mediarpipe.tasks.vision.poselandmarker.PoseLandmarker
15 import kotlinx.coroutines.Dispatchers
16 import kotlinx.coroutines.withContext
17 import java.io.File
18
19 private const val TAG = "PostureAnalyzer"
20
21 data class PostureAnalysisOutput(
22     val frames: List<PostureFrameAngles>,
23     val videoFrames: List<PostureVideoFrame>,
24     val videoPath: String?,
25     val videoWidth: Int,
26     val videoHeight: Int,
27 )
28
29 class PosturePoseAnalyzer(private val context: Context) {
30
31     suspend fun extractAnalysis(videoUri: Uri, analysisId: String, targetFps: Int = 15): PostureAnalysisOutput =
32         withContext(Dispatchers.Default) {
33             val analysisStartedAt = SystemClock.elapsedRealtime()
34             val retriever = MediaMetadataRetriever()
35             val videoPath = saveVideo(videoUri, analysisId)
36             try {
37                 if (videoPath != null) {
38                     retriever.setDataSource(videoPath)
39                 } else {
40                     retriever.setDataSource(context, videoUri)
41                 }
42
43                 val durationMs = retriever.extractMetadata(MediaMetadataRetriever.METADATA_KEY_DURATION)
44                     ?.toLongOrNull() ?: return@withContext emptyOutput()
45
46                 val rawWidth = retriever.extractMetadata(MediaMetadataRetriever.METADATA_KEY_VIDEO_WIDTH)?.toIntOrNull() ?: 0
47                 val rawHeight = retriever.extractMetadata(MediaMetadataRetriever.METADATA_KEY_VIDEO_HEIGHT)?.toIntOrNull() ?: 0
48                 val rotation = retriever.extractMetadata(MediaMetadataRetriever.METADATA_KEY_VIDEO_ROTATION)?.toIntOrNull() ?: 0
49
50                 // Display dimensions: swap when the stored pixel data is 90°/270° rotated
51                 // relative to the display orientation. ExoPlayer applies the same swap
52                 // automatically, so videoWidth×videoHeight matches what the user sees.
53                 val (videoWidth, videoHeight) =
54                     if (rotation == 90 || rotation == 270) rawHeight to rawWidth else rawWidth to rawHeight
55
56                 // BUG FIX: compute analysis dimensions from DISPLAY dimensions, not raw.
57                 //
58                 // On API 27+, MediaMetadataRetriever.getScaledFrameAtTime() automatically
59                 // applies METADATA_KEY_VIDEO_ROTATION before scaling (documented behavior).
60                 // If we passed raw dimensions (e.g. 512×288 landscape for a portrait video),
61                 // the function would rotate-then-scale the frame to fit within that landscape
62                 // box, returning a tiny 162×288 portrait bitmap instead of the expected 288×512.
63                 // Then applying rotationMatrix again caused a second rotation (double-rotation),
64                 // producing a 288×162 landscape blob that MediaPipe could not locate correctly.
65                 //
66                 // Fix: pass display-oriented dimensions (videoWidth, videoHeight) so the
67                 // auto-rotated result fills the analysis frame correctly. Manual rotation is
68                 // still applied for the API 26 fallback path (getFrameAtTime does not rotate).
69                 val (analysisWidth, analysisHeight) = scaledAnalysisSize(videoWidth, videoHeight)
70                 // 각도 계산 시 x 성분을 물리 비율로 환산하기 위한 aspect ratio
71                 val aspectRatio = if (videoHeight > 0) videoWidth.toFloat() / videoHeight else 1f
72
73                 val sourceLabel = if (videoUri.scheme == "file") "app-recorded" else "gallery"
74                 Log.d(TAG, "=== Pose Analysis Start ===")
75                 Log.d(TAG, "source=$sourceLabel uri.scheme=${videoUri.scheme}")
76                 Log.d(TAG, "rawWidth=$rawWidth rawHeight=$rawHeight rotationDegrees=$rotation")
77                 Log.d(TAG, "videoWidth=$videoWidth videoHeight=$videoHeight (display dims)")
78                 Log.d(TAG, "analysisWidth=$analysisWidth analysisHeight=$analysisHeight api=${Build.VERSION.SDK_INT}")
79
80                 val safeFps = targetFps.coerceIn(1, MAX_ANALYSIS_FPS)
81                 val requestedIntervalMs = 1000L / safeFps
82                 val boundedIntervalMs = (durationMs / MAX_ANALYSIS_FRAMES)
83                     .coerceAtLeast(1L)

```

```

84         val intervalMs = maxOf(requestedIntervalMs, boundedIntervalMs)
85
86         // running-form-analyzer 기본 BlazePose 경로와 동일하게 모델의 추적
87         // 좌표를 그대로 사용한다. VIDEO 모드가 시간축 추적을 수행하므로
88         // 외부 평활화를 중복 적용하면 빠른 무릎과 발목이 뒤쳐질 수 있다.
89         val landmarker = buildLandmarker()
90         val frames = mutableListOf<PostureFrameAngles>()
91         val videoFrames = mutableListOf<PostureVideoFrame>()
92         val metricsPipeline = RunningFormMetricsPipeline()
93         val landmarkValidator = PostureLandmarkValidator()
94
95         try {
96             var timeMs = 0L
97             while (
98                 timeMs < durationMs &&
99                 SystemClock.elapsedRealtime() - analysisStartedAt < ANALYSIS_TIME_BUDGET_MS
100             ) {
101                 // analysisFrameAt handles rotation internally:
102                 // API 27+: getScaledFrameAtTime auto-applies rotation; result is
103                 // already in display orientation.
104                 // API 26: getFrameAtTime + manual rotate + scale.
105                 // Either way the returned bitmap is in display orientation - no
106                 // additional rotation step is needed here.
107                 val bitmap = retriever.analysisFrameAt(
108                     timeUs = timeMs * 1000L,
109                     displayWidth = analysisWidth,
110                     displayHeight = analysisHeight,
111                     rotation = rotation,
112                 )
113
114                 if (bitmap != null) {
115                     val mpImage = BitmapImageBuilder(bitmap).build()
116                     // detectForVideo() requires strictly increasing timestamps.
117                     val result = landmarker.detectForVideo(mpImage, timeMs)
118                     result.landmarks().firstOrNull()?.let { landmarkList ->
119                         val rawPts = KEY_LANDMARK_INDICES.map { idx ->
120                             val lm = landmarkList[idx]
121                             SkeletonPoint(lm.x(), lm.y(), lm.visibility().orElse(0f))
122                         }
123                         val validatedPts = landmarkValidator.validate(rawPts)
124                         videoFrames.add(PostureVideoFrame(timeMs, validatedPts))
125
126                         metricsPipeline.process(validatedPts, timeMs, aspectRatio)
127                             ?.let(frames::add)
128                     }
129                     bitmap.recycle()
130                 }
131                 timeMs += intervalMs
132             }
133             finally {
134                 landmarker.close()
135             }
136
137             Log.d(TAG, "=== Analysis Complete: ${frames.size} angle frames, ${videoFrames.size} video frames ===")
138             PostureAnalysisOutput(frames, videoFrames, videoPath, videoWidth, videoHeight)
139         } catch (error: Throwable) {
140             videoPath?.let { File(it).delete() }
141             throw error
142         } finally {
143             retriever.release()
144         }
145     }
146
147     /**
148     * Extracts and returns a single analysis frame in **display orientation**
149     * (i.e. after applying any rotation from the container metadata).
150     *
151     * [displayWidth] × [displayHeight]: target size in display (post-rotation) coordinates.
152     * [rotation]: degrees from [MediaMetadataRetriever.METADATA_KEY_VIDEO_ROTATION].
153     *
154     * On API 27+ ([Build.VERSION_CODES.O_MR1] and above) [getScaledFrameAtTime] applies the
155     * container rotation automatically before scaling, so the caller must NOT rotate again.
156     *
157     * On API 26 [getFrameAtTime] returns the raw stored frame; rotation is applied here
158     * manually before scaling.
159     */
160     private fun MediaMetadataRetriever.analysisFrameAt(
161         timeUs: Long,
162         displayWidth: Int,
163         displayHeight: Int,
164         rotation: Int,
165     ): Bitmap? {
166         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O_MR1 && displayWidth > 0 && displayHeight > 0) {
167             // API 27+: getScaledFrameAtTime automatically applies METADATA_KEY_VIDEO_ROTATION
168             // before scaling. Passing display-oriented dimensions means the returned bitmap
169             // is sized and oriented correctly for MediaPipe - no further rotation needed.

```

```

170         return getScaledFrameAtTime(
171             timeUs,
172             MediaMetadataRetriever.OPTION_CLOSEST,
173             displayWidth,
174             displayHeight,
175         )
176     }
177
178     // API 26 fallback: getFrameAtTime returns raw stored orientation (no rotation applied).
179     val source = getFrameAtTime(timeUs, MediaMetadataRetriever.OPTION_CLOSEST) ?: return null
180
181     // Rotate to display orientation.
182     val rotated = if (rotation != 0) {
183         val m = Matrix().apply { postRotate(rotation.toFloat()) }
184         Bitmap.createBitmap(source, 0, 0, source.width, source.height, m, true)
185             .also { if (it != source) source.recycle() }
186     } else {
187         source
188     }
189
190     // Scale to target analysis dimensions.
191     if (displayWidth <= 0 || displayHeight <= 0 ||
192         (rotated.width <= displayWidth && rotated.height <= displayHeight)
193     ) {
194         return rotated
195     }
196     return Bitmap.createScaledBitmap(rotated, displayWidth, displayHeight, true).also {
197         if (it != rotated) rotated.recycle()
198     }
199 }
200
201 private fun scaledAnalysisSize(width: Int, height: Int): Pair<Int, Int> {
202     if (width <= 0 || height <= 0) return width to height
203     val longestSide = maxOf(width, height)
204     if (longestSide <= MAX_ANALYSIS_DIMENSION) return width to height
205
206     val scale = MAX_ANALYSIS_DIMENSION.toFloat() / longestSide
207     return (width * scale).toInt().coerceAtLeast(1) to
208         (height * scale).toInt().coerceAtLeast(1)
209 }
210
211 private fun saveVideo(videoUri: Uri, id: String): String? = runCatching {
212     val dir = File(context.filesDir, "posture").also { it.mkdirs() }
213     val dest = File(dir, "$id.mp4")
214     if (videoUri.scheme == "file") {
215         File(videoUri.path!!).copyTo(dest, overwrite = true)
216     } else {
217         // error() throws inside runCatching, causing getOrNull() to return null
218         // if the content resolver cannot open the stream.
219         context.contentResolver.openInputStream(videoUri)?.use { input ->
220             dest.outputStream().use { output -> input.copyTo(output) }
221         } ?: error("Could not open input stream for $videoUri")
222     }
223     dest.absolutePath
224 }.getOrNull()
225
226 private fun buildLandmarker(): PoseLandmarker {
227     val baseOptions = BaseOptions.builder()
228         .setModelAssetPath("pose_landmarker_full.task")
229         .build()
230     val options = PoseLandmarker.PoseLandmarkerOptions.builder()
231         .setBaseOptions(baseOptions)
232         .setRunningMode(RunningMode.VIDEO) // enables inter-frame Kalman tracking
233         .setNumPoses(1)
234         .setMinPoseDetectionConfidence(0.5f)
235         .setMinPosePresenceConfidence(0.5f)
236         .setMinTrackingConfidence(0.5f)
237         .build()
238     return PoseLandmarker.createFromOptions(context, options)
239 }
240
241 private fun emptyOutput() = PostureAnalysisOutput(emptyList(), emptyList(), null, 0, 0)
242
243 private companion object {
244     const val MAX_ANALYSIS_FPS = 15
245     const val MAX_ANALYSIS_FRAMES = 300
246     const val MAX_ANALYSIS_DIMENSION = 640
247     const val ANALYSIS_TIME_BUDGET_MS = 50_000L
248 }
249 }

```

```
1 package com.runway.android.core.posture
2
3 data class PostureResult(
4     val overallScore: Int,
5     val grade: String,
6     val overallFeedback: String,
7     val knee: PostureCategoryResult,
8     val trunk: PostureCategoryResult,
9     val elbow: PostureCategoryResult,
10    val hip: PostureCategoryResult,
11    val overstride: PostureCategoryResult,
12    // Reference metrics: computed from bilateral signals, not included in overall score
13    val cadence: PostureCategoryResult,
14    val verticalOscillation: PostureCategoryResult,
15 ) {
16     companion object {
17         fun gradeFrom(score: Int): String = when {
18             score >= 90 -> "S"
19             score >= 75 -> "A"
20             score >= 60 -> "B"
21             score >= 45 -> "C"
22             else -> "D"
23         }
24     }
25 }
```

```

1 package com.runway.android.core.posture
2
3 import javax.inject.Inject
4 import kotlin.math.abs
5 import kotlin.math.roundToInt
6
7 /**
8  * Aggregates the metrics produced by the running-form-analyzer port.
9  *
10  * The upstream project returns Good / Need Improvement / Bad rather than a
11  * numeric overall score. Pathfinder converts those states into a continuous
12  * score so values within the ideal range receive 90-100 points.
13  */
14 class PostureRuleEngine @Inject constructor() : PostureEvaluator {
15
16     override fun evaluate(frames: List<PostureFrameAngles>): PostureResult {
17         if (frames.isEmpty()) return emptyResult()
18
19         val strikeKnees = buildList {
20             frames.forEach { frame ->
21                 if (frame.leftFootStrike) frame.leftKneeAngle?.let(::add)
22                 if (frame.rightFootStrike) frame.rightKneeAngle?.let(::add)
23                 if (frame.isLandingFrame && !frame.leftFootStrike && !frame.rightFootStrike) {
24                     add(frame.kneeFlexAngle)
25                 }
26             }
27         }
28         val strikeHipAnkle = buildList {
29             frames.forEach { frame ->
30                 if (frame.leftFootStrike) frame.leftHipAnkleAngle?.let(::add)
31                 if (frame.rightFootStrike) frame.rightHipAnkleAngle?.let(::add)
32                 if (frame.isLandingFrame && !frame.leftFootStrike && !frame.rightFootStrike) {
33                     add(frame.overstrideRatio)
34                 }
35             }
36         }
37         val strikeShanks = buildList {
38             frames.forEach { frame ->
39                 if (frame.leftFootStrike) frame.leftShankAngle?.let(::add)
40                 if (frame.rightFootStrike) frame.rightShankAngle?.let(::add)
41                 if (frame.isLandingFrame && !frame.leftFootStrike && !frame.rightFootStrike) {
42                     add(frame.shankAngle)
43                 }
44             }
45         }
46
47         val kneeValue = strikeKnees.lastOrNull()
48             ?: frames.map { it.kneeFlexAngle }.median()
49         val trunkValue = frames.map { abs(it.trunkLeanAngle) }.median()
50         val elbowValue = frames.map { it.elbowAngle }.median()
51         val hipSwingValue = maxCompletedSwing(frames)
52         val hipAnkleValue = strikeHipAnkle.lastOrNull()
53             ?: frames.map { it.overstrideRatio }.median()
54         val shankValue = strikeShanks.lastOrNull() ?: 0f
55
56         val knee = assessKnee(kneeValue)
57         val trunk = assessTrunk(trunkValue)
58         val elbow = assessElbow(elbowValue)
59         val hip = assessHipSwing(hipSwingValue)
60         val footStrike = assessHipAnkle(hipAnkleValue, shankValue, strikeHipAnkle.isNotEmpty())
61         val cadence = assessCadence(frames)
62         val verticalOscillation = assessVerticalOscillation(frames)
63
64         val scored = listOf(knee, trunk, elbow, hip)
65         val overall = scored.map { it.score }.average().roundToInt()
66         val grade = PostureResult.gradeFrom(overall)
67         val weakest = listOf(
68             "무릎 각도" to knee,
69             "상체 기울기" to trunk,
70             "팔꿈치 각도" to elbow,
71             "고관절 가동범위" to hip,
72         ).minBy { it.second.score }
73
74         return PostureResult(
75             overallScore = overall,
76             grade = grade,
77             overallFeedback = when {
78                 overall >= 90 -> "원본 분석 기준에서 전반적으로 좋은 자세입니다."
79                 overall >= 60 -> "${weakest.first} 항목을 우선 개선해보세요."
80                 else -> "${weakest.first} 항목에 지속적인 교정이 필요합니다."
81             },
82             knee = knee,
83             trunk = trunk,

```



```

84         elbow = elbow,
85         hip = hip,
86         overstride = footStrike,
87         cadence = cadence,
88         verticalOscillation = verticalOscillation,
89     )
90 }
91
92 private fun assessKnee(value: Float): PostureCategoryResult = category(
93     value = value,
94     idealMin = 135f,
95     idealMax = 180f,
96     unit = "°",
97     assessment = when {
98         value > 135f -> Assessment.GOOD
99         value >= 126f -> Assessment.NEEDS_IMPROVEMENT
100        else -> Assessment.BAD
101    },
102    good = "착지 순간 무릎 각도가 좋습니다.",
103    needsImprovement = "착지 순간 무릎 각도가 조금 작습니다.",
104    bad = "착지 순간 무릎이 과도하게 굽혀집니다.",
105 )
106
107 private fun assessTrunk(value: Float): PostureCategoryResult = category(
108     value = value,
109     idealMin = 5f,
110     idealMax = 15f,
111     unit = "°",
112     assessment = when {
113         value in 5f..15f -> Assessment.GOOD
114         value in 1f..<5f || value in 15f..17f -> Assessment.NEEDS_IMPROVEMENT
115         else -> Assessment.BAD
116     },
117     good = "상체 기울기가 좋습니다.",
118     needsImprovement = "상체 기울기를 5~15도로 조정해보세요.",
119     bad = "상체 기울기가 권장 범위를 크게 벗어났습니다.",
120 )
121
122 private fun assessElbow(value: Float): PostureCategoryResult = category(
123     value = value,
124     idealMin = 60f,
125     idealMax = 90f,
126     unit = "°",
127     assessment = when {
128         value in 60f..90f -> Assessment.GOOD
129         value in 53f..<60f || value in 90f..95f -> Assessment.NEEDS_IMPROVEMENT
130         else -> Assessment.BAD
131     },
132     good = "팔꿈치 각도가 좋습니다.",
133     needsImprovement = "팔꿈치 각도를 60~90도로 조정해보세요.",
134     bad = "팔꿈치 각도가 권장 범위를 크게 벗어났습니다.",
135 )
136
137 private fun assessHipSwing(value: Float): PostureCategoryResult = category(
138     value = value,
139     idealMin = 29f,
140     idealMax = 41f,
141     unit = "°",
142     assessment = when {
143         value in 29f..<41f -> Assessment.GOOD
144         value >= 41f -> Assessment.NEEDS_IMPROVEMENT
145         else -> Assessment.BAD
146     },
147     good = "고관절 스윙 범위가 좋습니다.",
148     needsImprovement = "고관절 스윙 범위가 다소 큼니다.",
149     bad = "고관절 스윙 범위가 부족합니다.",
150 )
151
152 private fun assessHipAnkle(
153     value: Float,
154     shankValue: Float,
155     hasStrike: Boolean,
156 ): PostureCategoryResult {
157     if (!hasStrike) {
158         return PostureCategoryResult(
159             score = 20,
160             measuredValue = value,
161             idealMin = 0f,
162             idealMax = 15f,
163             unit = "°",
164             feedback = "원본 방식의 착지 이벤트를 감지하지 못했습니다.",
165             tip = "전신이 보이는 측면 영상을 10초 이상 촬영하세요.",
166         )
167     }
168     val hipAnkleAssessment = when {
169         value in 0f..15f -> Assessment.GOOD

```

```

170         value <= 20f -> Assessment.NEEDS_IMPROVEMENT
171         else -> Assessment.BAD
172     }
173     val shankAssessment = when {
174         shankValue in 0f..10f -> Assessment.GOOD
175         shankValue <= 15f -> Assessment.NEEDS_IMPROVEMENT
176         else -> Assessment.BAD
177     }
178     val assessment = minOf(hipAnkleAssessment, shankAssessment)
179     return category(
180         value = value,
181         idealMin = 0f,
182         idealMax = 15f,
183         unit = "°",
184         assessment = assessment,
185         good = "착지 순간 발이 골반 아래에 가깝고 정강이 각도가 좋습니다.",
186         needsImprovement = "착지 위치나 정강이 각도를 조금 조정해 보세요.",
187         bad = "착지 위치가 골반에서 멀거나 정강이가 과도하게 기울어져 있습니다.",
188     )
189 }
190
191 private fun assessCadence(frames: List<PostureFrameAngles>): PostureCategoryResult {
192     val spm = cadenceFromSingleLegCycle(frames)
193         ?: cadenceFromAlternatingAnkles(frames)
194         ?: cadenceFromStrikeEvents(frames)
195     if (spm == null) {
196         return PostureCategoryResult(
197             0, 0f, 170f, 180f, "spm",
198             "케이던스 측정을 위한 좌우 다리 주기가 부족합니다.", "",
199         )
200     }
201     val assessment = when {
202         spm in 170f..180f -> Assessment.GOOD
203         spm in 160f..<170f || spm in 180f..190f -> Assessment.NEEDS_IMPROVEMENT
204         else -> Assessment.BAD
205     }
206     return category(
207         spm, 170f, 180f, "spm", assessment,
208         "케이던스가 좋습니다.",
209         "케이던스를 170~180spm에 가깝게 조정해 보세요.",
210         "케이던스가 권장 범위에서 크게 벗어났습니다.",
211     )
212 }
213
214 private fun cadenceFromSingleLegCycle(frames: List<PostureFrameAngles>): Float? {
215     val left = relativeAnkleSignal(frames, SKEL_L_HIP, SKEL_L_ANKLE)
216     val right = relativeAnkleSignal(frames, SKEL_R_HIP, SKEL_R_ANKLE)
217     val signal = listOf(left, right)
218         .filter { it.size >= MIN_CADENCE_SIGNAL_FRAMES }
219         .maxByOrNull { it.size } ?: return null
220
221     val smoothed = signal.indices.map { index ->
222         val from = (index - 1).coerceAtLeast(0)
223         val to = (index + 1).coerceAtMost(signal.lastIndex)
224         signal[index].first to (from..to).map { signal[it].second }.average().toFloat()
225     }
226     val values = smoothed.map { it.second }
227     val range = (values.maxOrNull() ?: 0f) - (values.minOrNull() ?: 0f)
228     if (range < MIN_ANKLE_SWING_RANGE) return null
229     val peakThreshold = (values.maxOrNull() ?: 0f) - range * 0.35f
230
231     val peaks = mutableListOf<Long>()
232     for (index in 1 until smoothed.lastIndex) {
233         val previous = smoothed[index - 1].second
234         val current = smoothed[index].second
235         val next = smoothed[index + 1].second
236         val timestamp = smoothed[index].first
237         if (current >= peakThreshold && current > previous && current >= next) {
238             if (peaks.isEmpty() || timestamp - peaks.last() >= MIN_STRIDE_INTERVAL_MS) {
239                 peaks += timestamp
240             }
241         }
242     }
243
244     val strideIntervals = peaks.zipWithNext { first, second -> second - first }
245         .filter { it in MIN_STRIDE_INTERVAL_MS..MAX_STRIDE_INTERVAL_MS }
246     if (strideIntervals.size < 2) return null
247     return 120_000f / strideIntervals.medianLong()
248 }
249
250 private fun relativeAnkleSignal(
251     frames: List<PostureFrameAngles>,
252     hipIndex: Int,
253     ankleIndex: Int,
254 ): List<Pair<Long, Float>> = frames.mapNotNull { frame ->
255     val hip = frame.landmarks.getOrNull(hipIndex)

```

```

256     val ankle = frame.landmarks.getOrNull(ankleIndex)
257     if (hip == null || ankle == null || minOf(hip.v, ankle.v) < 0.3f) {
258         null
259     } else {
260         frame.timestampMs to (ankle.x - hip.x)
261     }
262 }
263
264 /**
265  * Each sign change of leftAnkleY - rightAnkleY represents the next leg
266  * taking over in the gait cycle. Median transition spacing is robust to a
267  * missed frame and does not depend on how much of the video was analyzed.
268  */
269 private fun cadenceFromAlternatingAnkles(frames: List<PostureFrameAngles>): Float? {
270     val signals = frames.mapNotNull { frame ->
271         val left = frame.landmarks.getOrNull(SKEL_L_ANKLE)
272         val right = frame.landmarks.getOrNull(SKEL_R_ANKLE)
273         if (left == null || right == null || left.v < 0.3f || right.v < 0.3f) {
274             null
275         } else {
276             frame.timestampMs to (left.y - right.y)
277         }
278     }
279     if (signals.size < 6) return null
280
281     val smoothed = signals.indices.map { index ->
282         val from = (index - 1).coerceAtLeast(0)
283         val to = (index + 1).coerceAtMost(signals.lastIndex)
284         val value = (from..to).map { signals[it].second }.average().toFloat()
285         signals[index].first to value
286     }
287
288     val range = (smoothed.maxOf { it.second } - smoothed.minOf { it.second })
289     if (range < 0.03f) return null
290     val hysteresis = (range * 0.12f).coerceIn(0.008f, 0.04f)
291     val transitions = mutableList<Long>()
292     var phase = 0
293
294     smoothed.forEach { (timestamp, value) ->
295         when {
296             value >= hysteresis && phase <= 0 -> {
297                 if (phase < 0) transitions += timestamp
298                 phase = 1
299             }
300             value <= -hysteresis && phase >= 0 -> {
301                 if (phase > 0) transitions += timestamp
302                 phase = -1
303             }
304         }
305     }
306
307     val stepIntervals = transitions.zipWithNext { first, second -> second - first }
308         .filter { it in MIN_STEP_INTERVAL_MS..MAX_STEP_INTERVAL_MS }
309     if (stepIntervals.size < 2) return null
310     return 60_000f / stepIntervals.medianLong()
311 }
312
313 private fun cadenceFromStrikeEvents(frames: List<PostureFrameAngles>): Float? {
314     val timestamps = buildList {
315         frames.forEach { frame ->
316             if (frame.leftFootStrike) add(frame.timestampMs)
317             if (frame.rightFootStrike) add(frame.timestampMs)
318         }
319     }.sorted()
320     val intervals = timestamps.zipWithNext { first, second -> second - first }
321         .filter { it in MIN_STEP_INTERVAL_MS..MAX_STEP_INTERVAL_MS }
322     if (intervals.size < 2) return null
323     return 60_000f / intervals.medianLong()
324 }
325
326 private fun assessVerticalOscillation(frames: List<PostureFrameAngles>): PostureCategoryResult {
327     if (frames.size < 10) {
328         return PostureCategoryResult(
329             0, 0f, 0f, 6.5f, "%",
330             "수직진폭 측정을 위한 프레임이 부족합니다.", "",
331         )
332     }
333     val hipWindow = frames.map { it.hipMidY }
334     val movingAverages = hipWindow.windowed(5).map { it.average().toFloat() }
335     val oscillation = ((movingAverages.maxOrNull() ?: 0f) -
336         (movingAverages.minOrNull() ?: 0f)) / 2f * 100f
337     val assessment = when {
338         oscillation < 6.5f -> Assessment.GOOD
339         oscillation < 8f -> Assessment.NEEDS_IMPROVEMENT
340         else -> Assessment.BAD
341     }

```

```

342     return category(
343         oscillation, 0f, 6.5f, "%", assessment,
344         "수직진폭이 좋습니다.",
345         "수직진폭을 조금 줄어보세요.",
346         "수직진폭이 커서 에너지 손실이 발생할 수 있습니다.",
347     )
348 }
349
350 private fun maxCompletedSwing(frames: List<PostureFrameAngles>): Float {
351     val leftAngles = frames.mapNotNull { it.leftHipAngle }
352     val rightAngles = frames.mapNotNull { it.rightHipAngle }
353     if (leftAngles.isNotEmpty() || rightAngles.isNotEmpty()) {
354         return maxOf(
355             completedSwing(leftAngles),
356             completedSwing(rightAngles),
357         )
358     }
359     return frames.maxOrNull { abs(it.hipExtensionAngle) } ?: 0f
360 }
361
362 private fun completedSwing(values: List<Float>): Float {
363     var previous = 0f
364     var direction = 0
365     var currentForward = 0f
366     var currentBackward = 0f
367     var completedForward = 0f
368     var completedBackward = 0f
369
370     values.forEach { value ->
371         when {
372             value > previous -> {
373                 if (direction == -1) {
374                     completedBackward = currentBackward
375                     currentBackward = 0f
376                 }
377                 direction = 1
378                 currentForward = maxOf(currentForward, value)
379             }
380             value < previous -> {
381                 if (direction == 1) {
382                     completedForward = currentForward
383                     currentForward = 0f
384                 }
385                 direction = -1
386                 currentBackward = maxOf(currentBackward, abs(value))
387             }
388         }
389         previous = value
390     }
391     return maxOf(completedForward, completedBackward)
392 }
393
394 private fun category(
395     value: Float,
396     idealMin: Float,
397     idealMax: Float,
398     unit: String,
399     assessment: Assessment,
400     good: String,
401     needsImprovement: String,
402     bad: String,
403 ) = PostureCategoryResult(
404     score = categoryScore(value, idealMin, idealMax, assessment),
405     measuredValue = value,
406     idealMin = idealMin,
407     idealMax = idealMax,
408     unit = unit,
409     feedback = when (assessment) {
410         Assessment.GOOD -> good
411         Assessment.NEEDS_IMPROVEMENT -> needsImprovement
412         Assessment.BAD -> bad
413     },
414     tip = "",
415 )
416
417 private fun categoryScore(
418     value: Float,
419     idealMin: Float,
420     idealMax: Float,
421     assessment: Assessment,
422 ): Int {
423     val range = (idealMax - idealMin).coerceAtLeast(1f)
424     return when (assessment) {
425         Assessment.GOOD -> {
426             val center = (idealMin + idealMax) / 2f
427             val halfRange = range / 2f

```

```

428         val closeness = (1f - abs(value - center) / halfRange).coerceIn(0f, 1f)
429         (90f + closeness * 10f).roundToInt()
430     }
431     Assessment.NEEDS_IMPROVEMENT -> {
432         val distance = when {
433             value < idealMin -> idealMin - value
434             value > idealMax -> value - idealMax
435             else -> 0f
436         }
437         (89f - distance / range * 24f).roundToInt().coerceIn(65, 89)
438     }
439     Assessment.BAD -> {
440         val distance = minOf(abs(value - idealMin), abs(value - idealMax))
441         (64f - distance / range * 34f).roundToInt().coerceIn(30, 64)
442     }
443 }
444 }
445
446 private fun List<Float>.median(): Float {
447     if (isEmpty()) return 0f
448     val sorted = sorted()
449     val middle = sorted.size / 2
450     return if (sorted.size % 2 == 0) {
451         (sorted[middle - 1] + sorted[middle]) / 2f
452     } else {
453         sorted[middle]
454     }
455 }
456
457 private fun List<Long>.medianLong(): Float {
458     val sorted = sorted()
459     val middle = sorted.size / 2
460     return if (sorted.size % 2 == 1) {
461         sorted[middle].toFloat()
462     } else {
463         (sorted[middle - 1] + sorted[middle]) / 2f
464     }
465 }
466
467 private fun emptyResult() = PostureResult(
468     overallScore = 0,
469     grade = "D",
470     overallFeedback = "분석 가능한 프레임이 없습니다. 다시 촬영해주세요.",
471     knee = PostureCategoryResult(0, 0f, 135f, 180f, "°", "분석 불가", ""),
472     trunk = PostureCategoryResult(0, 0f, 5f, 15f, "°", "분석 불가", ""),
473     elbow = PostureCategoryResult(0, 0f, 60f, 90f, "°", "분석 불가", ""),
474     hip = PostureCategoryResult(0, 0f, 29f, 41f, "°", "분석 불가", ""),
475     overstride = PostureCategoryResult(0, 0f, 0f, 15f, "°", "분석 불가", ""),
476     cadence = PostureCategoryResult(0, 0f, 170f, 180f, "spm", "분석 불가", ""),
477     verticalOscillation = PostureCategoryResult(0, 0f, 0f, 6.5f, "%", "분석 불가", "")
478 )
479
480 private enum class Assessment {
481     BAD,
482     NEEDS_IMPROVEMENT,
483     GOOD,
484 }
485
486 private companion object {
487     const val MIN_STEP_INTERVAL_MS = 250L
488     const val MAX_STEP_INTERVAL_MS = 1_000L
489     const val MIN_STRIDE_INTERVAL_MS = 450L
490     const val MAX_STRIDE_INTERVAL_MS = 1_500L
491     const val MIN_CADENCE_SIGNAL_FRAMES = 12
492     const val MIN_ANKLE_SWING_RANGE = 0.05f
493 }
494 }

```

```

1 package com.runway.android.core.posture
2
3 import kotlin.math.PI
4 import kotlin.math.abs
5
6 /**
7  * One Euro Filter smoothing for skeleton landmark positions during video replay.
8  *
9  * Adapts its cutoff frequency based on movement speed:
10 * - Slow / stationary landmarks → low cutoff → strong smoothing, no jitter
11 * - Fast-moving landmarks (swing phase) → high cutoff → responsive, no lag
12 *
13 * Low-visibility landmarks are frozen at their last known position so occluded
14 * joints don't drift toward bad detections.
15 *
16 * Reference: Casiez et al. "1E Filter: A Simple Speed-based Low-pass Filter
17 * for Noisy Input in Interactive Systems." CHI 2012.
18 *
19 * Usage:
20 *   val smoother = PostureSkeletonSmoother()
21 *   smoother.reset()           // call on seek or new video
22 *   val stable = smoother.smooth(raw, timestampMs)
23 */
24 class PostureSkeletonSmoother(
25     private val minCutoff: Float = 1.0f,    // Hz - smoothing strength at rest
26     private val beta: Float = 3.0f,        // speed coefficient - higher = more responsive to fast motion
27     private val dCutoff: Float = 1.0f,     // Hz - cutoff for the derivative low-pass filter
28     private val lowVisFreezeThreshold: Float = 0.45f,
29 ) {
30
31     private inner class OneDEuroFilter {
32         private var initialized = false
33         private var xPrev = 0f
34         private var dxFiltered = 0f
35         var lastOutput = 0f
36         private set
37
38         fun filter(x: Float, dt: Float): Float {
39             if (!initialized) {
40                 initialized = true
41                 xPrev = x
42                 lastOutput = x
43                 return x
44             }
45             // Derivative, smoothed to prevent noisy cutoff jumps
46             val dx = (x - xPrev) / dt
47             val aDeriv = alpha(dCutoff, dt)
48             dxFiltered = aDeriv * dx + (1f - aDeriv) * dxFiltered
49
50             // Adaptive cutoff: fast movement raises the cutoff → more responsive
51             val cutoff = minCutoff + beta * abs(dxFiltered)
52             val a = alpha(cutoff, dt)
53             lastOutput = a * x + (1f - a) * lastOutput
54             xPrev = x
55             return lastOutput
56         }
57
58         fun reset() {
59             initialized = false
60             xPrev = 0f
61             dxFiltered = 0f
62             lastOutput = 0f
63         }
64
65         val isInitialized get() = initialized
66
67         private fun alpha(cutoff: Float, dt: Float): Float {
68             val r = 2f * PI.toFloat() * cutoff * dt
69             return r / (r + 1f)
70         }
71     }
72
73     private var filtersX: Array<OneDEuroFilter>? = null
74     private var filtersY: Array<OneDEuroFilter>? = null
75     private var visEma: FloatArray? = null
76     private var lastTimestampMs: Long = -1L
77
78     /**
79      * @param raw          Raw landmark list from MediaPipe (or interpolated frame).
80      * @param timestampMs  Video timestamp in ms. Pass [PostureVideoFrame.t] for accurate dt.
81      *                    Omit (or pass -1) to fall back to a fixed 30 fps assumption.
82      */
83     fun smooth(raw: List<SkeletonPoint>, timestampMs: Long = -1L): List<SkeletonPoint> {

```

```

84     val n = raw.size
85     ensureFilters(n)
86
87     val dt = computeDt(timestampMs)
88     if (timestampMs >= 0L) lastTimestampMs = timestampMs
89
90     val fx = filtersX!!
91     val fy = filtersY!!
92     val ve = visEma!!
93
94     return raw.mapIndexed { i, pt ->
95         val wasInitialized = fx[i].isInitialized
96         val visibleEnough = pt.v >= lowVisFreezeThreshold || !wasInitialized
97         val x = if (visibleEnough) fx[i].filter(pt.x, dt) else fx[i].lastOutput
98         val y = if (visibleEnough) fy[i].filter(pt.y, dt) else fy[i].lastOutput
99         // Visibility: simple EMA (no freeze needed - we want it to track confidence changes)
100         ve[i] = if (wasInitialized) ve[i] + 0.4f * (pt.v - ve[i]) else pt.v
101         SkeletonPoint(x = x, y = y, v = ve[i])
102     }
103 }
104
105 fun reset() {
106     filtersX?.forEach { it.reset() }
107     filtersY?.forEach { it.reset() }
108     visEma?.fill(0f)
109     lastTimestampMs = -1L
110 }
111
112 private fun ensureFilters(n: Int) {
113     if (filtersX?.size == n) return
114     filtersX = Array(n) { OneDEuroFilter() }
115     filtersY = Array(n) { OneDEuroFilter() }
116     visEma = FloatArray(n)
117 }
118
119 private fun computeDt(timestampMs: Long): Float {
120     if (timestampMs < 0L || lastTimestampMs < 0L) return 1f / 30f
121     return ((timestampMs - lastTimestampMs) / 1000f).coerceIn(0.001f, 0.5f)
122 }
123 }

```

```

1 package com.runway.android.core.posture
2
3 /** Normalized (0-1) landmark point extracted from one video frame. */
4 data class SkeletonPoint(val x: Float, val y: Float, val v: Float) // v = visibility
5
6 /** Landmark snapshot for one frame, used to drive the skeleton replay overlay. */
7 data class PostureVideoFrame(
8     val t: Long, // timestamp from video start (ms)
9     val pts: List<SkeletonPoint>, // 13 key landmarks in KEY_LANDMARK_INDICES order
10 )
11
12 /**
13  * MediaPipe landmark indices we store for the skeleton overlay.
14  * Order: nose, lShoulder, rShoulder, lElbow, rElbow, lWrist, rWrist,
15  *         lHip, rHip, lKnee, rKnee, lAnkle, rAnkle
16  */
17 val KEY_LANDMARK_INDICES = intArrayOf(0, 11, 12, 13, 14, 15, 16, 23, 24, 25, 26, 27, 28)
18
19 // Local indices (position in pts list) for skeleton edge drawing
20 const val SKEL_NOSE = 0
21 const val SKEL_L_SHOULDER = 1; const val SKEL_R_SHOULDER = 2
22 const val SKEL_L_ELLOW = 3; const val SKEL_R_ELLOW = 4
23 const val SKEL_L_WRIST = 5; const val SKEL_R_WRIST = 6
24 const val SKEL_L_HIP = 7; const val SKEL_R_HIP = 8
25 const val SKEL_L_KNEE = 9; const val SKEL_R_KNEE = 10
26 const val SKEL_L_ANKLE = 11; const val SKEL_R_ANKLE = 12
27
28 // One-sided limb edges (arm chain + torso + leg chain)
29 val SKELETON_EDGES_LEFT: List<Pair<Int, Int>> = listOf(
30     SKEL_L_SHOULDER to SKEL_L_ELLOW, SKEL_L_ELLOW to SKEL_L_WRIST,
31     SKEL_L_SHOULDER to SKEL_L_HIP,
32     SKEL_L_HIP to SKEL_L_KNEE, SKEL_L_KNEE to SKEL_L_ANKLE,
33 )
34 val SKELETON_EDGES_RIGHT: List<Pair<Int, Int>> = listOf(
35     SKEL_R_SHOULDER to SKEL_R_ELLOW, SKEL_R_ELLOW to SKEL_R_WRIST,
36     SKEL_R_SHOULDER to SKEL_R_HIP,
37     SKEL_R_HIP to SKEL_R_KNEE, SKEL_R_KNEE to SKEL_R_ANKLE,
38 )
39 // Bilateral connectors (drawn only when both endpoints are visible enough)
40 val SKELETON_EDGES_BILATERAL: List<Pair<Int, Int>> = listOf(
41     SKEL_L_SHOULDER to SKEL_R_SHOULDER,
42     SKEL_L_HIP to SKEL_R_HIP,
43 )
44
45 /** All edges combined - kept for backward compatibility / tests. */
46 val SKELETON_EDGES: List<Pair<Int, Int>> =
47     SKELETON_EDGES_BILATERAL + SKELETON_EDGES_LEFT + SKELETON_EDGES_RIGHT
48
49 val SKELETON_JOINT_INDICES: List<Int> = (0..12).toList()

```



```
1 package com.runway.android.core.posture
2
3 /**
4  * Stateful port of running-form-analyzer/metrics/angle_metrics.py and step_metrics.py.
5  */
6 class RunningFormMetricsPipeline {
7     private val leftStrikeDetector = RunningFormStrikeDetector()
8     private val rightStrikeDetector = RunningFormStrikeDetector()
9     private var previous: PostureFrameAngles? = null
10
11     fun process(
12         landmarks: List<SkeletonPoint>,
13         timestampMs: Long,
14         aspectRatio: Float,
15     ): PostureFrameAngles? {
16         val current = PostureAngleCalculator.compute(landmarks, aspectRatio) ?: return null
17         val carried = carryMissingAngles(current, previous)
18
19         val leftStrike = landmarks.getOrNull(SKEL_L_ANKLE)
20             ?.let { leftStrikeDetector.update(it, timestampMs) } == true
21         val rightStrike = landmarks.getOrNull(SKEL_R_ANKLE)
22             ?.let { rightStrikeDetector.update(it, timestampMs) } == true
23
24         return carried.copy(
25             timestampMs = timestampMs,
26             landmarks = landmarks,
27             leftFootStrike = leftStrike,
28             rightFootStrike = rightStrike,
29             isLandingFrame = leftStrike || rightStrike,
30         ).also { previous = it }
31     }
32
33     private fun carryMissingAngles(
34         current: PostureFrameAngles,
35         old: PostureFrameAngles?,
36     ): PostureFrameAngles {
37         if (old == null) return current
38         return current.copy(
39             leftKneeAngle = current.leftKneeAngle ?: old.leftKneeAngle,
40             rightKneeAngle = current.rightKneeAngle ?: old.rightKneeAngle,
41             leftElbowAngle = current.leftElbowAngle ?: old.leftElbowAngle,
42             rightElbowAngle = current.rightElbowAngle ?: old.rightElbowAngle,
43             leftHipAngle = current.leftHipAngle ?: old.leftHipAngle,
44             rightHipAngle = current.rightHipAngle ?: old.rightHipAngle,
45             leftHipAnkleAngle = current.leftHipAnkleAngle ?: old.leftHipAnkleAngle,
46             rightHipAnkleAngle = current.rightHipAnkleAngle ?: old.rightHipAnkleAngle,
47             leftShankAngle = current.leftShankAngle ?: old.leftShankAngle,
48             rightShankAngle = current.rightShankAngle ?: old.rightShankAngle,
49             leftArmSwingAngle = current.leftArmSwingAngle ?: old.leftArmSwingAngle,
50             rightArmSwingAngle = current.rightArmSwingAngle ?: old.rightArmSwingAngle,
51         )
52     }
53 }
```

```

1 package com.runway.android.core.posture
2
3 /**
4  * Foot strike detector - exact port of running-form-analyzer
5  * (henryczup/running-form-analyzer, AngleMetrics.kt line:
6  *   StepMetrics(filter_type='temporal', detection_axis='y')).
7  *
8  * Uses Y-axis local MINIMUM detection:
9  *   local min of Y (in image coords, Y=0 is top) = ankle at its highest
10  *   screen position = peak of swing phase. Filters with a simple moving
11  *   average (temporal filter, window=10) - matching the original's
12  *   filter_type='temporal'. Adaptive threshold clamped to [0.005f, 0.1f].
13  */
14 class RunningFormStrikeDetector(
15     private val windowSize: Int = 10,
16     private val adaptiveWindowSize: Int = 50,
17     private val minIntervalMs: Long = 400L,
18 ) {
19     // Temporal filter (moving average) - matches filter_type='temporal'
20     private val filterBuffer = ArrayDeque<Float>()
21
22     private val positions = ArrayDeque<Float>()
23     private val motionHistory = ArrayDeque<Float>()
24     private var threshold = 0.02f
25     private var lastStrikeMs = -1L
26
27     /** Returns true on the frame a step event is detected. */
28     fun update(ankle: SkeletonPoint, timestampMs: Long): Boolean {
29         if (ankle.v < 0.3f) return false
30
31         val filtered = temporalFilter(ankle.y)
32
33         if (positions.size >= windowSize) positions.removeFirst()
34         positions.addLast(filtered)
35         if (motionHistory.size >= adaptiveWindowSize) motionHistory.removeFirst()
36         motionHistory.addLast(filtered)
37
38         updateThreshold()
39         if (positions.size < 3) return false
40
41         val p0 = positions[positions.size - 3]
42         val p1 = positions[positions.size - 2] // candidate
43         val p2 = positions[positions.size - 1]
44
45         // Local minimum: ankle was descending in Y (rising in screen), bottomed, now ascending
46         // Matches original: positions[-2] < positions[-1] and positions[-2] < positions[-3]
47         // and positions[-1] - positions[-2] > threshold
48         val localMin = p1 < p0 && p1 < p2 && (p2 - p1) > threshold
49         if (!localMin) return false
50
51         val tooSoon = lastStrikeMs >= 0 && (timestampMs - lastStrikeMs) < minIntervalMs
52         if (tooSoon) return false
53
54         lastStrikeMs = timestampMs
55         return true
56     }
57
58     private fun updateThreshold() {
59         if (motionHistory.size < adaptiveWindowSize) return
60         val range = (motionHistory.maxOrNull() ?: 0f) - (motionHistory.minOrNull() ?: 0f)
61         threshold = (0.7f * threshold + 0.3f * range * 0.1f).coerceIn(0.005f, 0.1f)
62     }
63
64     private fun temporalFilter(value: Float): Float {
65         if (filterBuffer.size >= windowSize) filterBuffer.removeFirst()
66         filterBuffer.addLast(value)
67         return if (filterBuffer.size < windowSize) value
68         else filterBuffer.sum() / filterBuffer.size
69     }
70
71     fun reset() {
72         positions.clear()
73         motionHistory.clear()
74         filterBuffer.clear()
75         lastStrikeMs = -1L
76         threshold = 0.02f
77     }
78 }

```

```
1 package com.runway.android.core.posture.local
2
3 import androidx.room.Dao
4 import androidx.room.Insert
5 import androidx.room.OnConflictStrategy
6 import androidx.room.Query
7 import kotlinx.coroutines.flow.Flow
8
9 @Dao
10 interface PostureAnalysisDao {
11
12     @Insert(onConflict = OnConflictStrategy.REPLACE)
13     suspend fun insert(entity: PostureAnalysisEntity)
14
15     @Query("SELECT * FROM posture_analyses WHERE ownerId = :ownerId ORDER BY createdAt DESC")
16     fun observeAll(ownerId: String): Flow<List<PostureAnalysisEntity>>
17
18     @Query("SELECT * FROM posture_analyses WHERE id = :id AND ownerId = :ownerId")
19     suspend fun findById(id: String, ownerId: String): PostureAnalysisEntity?
20
21     @Query("SELECT COUNT(*) FROM posture_analyses WHERE ownerId = :ownerId")
22     suspend fun count(ownerId: String): Int
23
24     @Query("DELETE FROM posture_analyses WHERE id = :id AND ownerId = :ownerId")
25     suspend fun deleteById(id: String, ownerId: String)
26
27     @Query("UPDATE posture_analyses SET ownerId = :ownerId WHERE ownerId = ''")
28     suspend fun assignUnowned(ownerId: String)
29 }
```

```
1 package com.runway.android.core.posture.local
2
3 import androidx.room.Entity
4 import androidx.room.PrimaryKey
5
6 const val MAX_POSTURE_ANALYSIS_HISTORY = 10
7
8 @Entity(tableName = "posture_analyses")
9 data class PostureAnalysisEntity(
10     @PrimaryKey val id: String,
11     val ownerId: String = "",
12     val createdAt: Long,
13     val overallScore: Int,
14     val grade: String,
15     val overallFeedback: String,
16     val kneeScore: Int,
17     val kneeMeasuredAngle: Float,
18     val kneeFeedback: String,
19     val kneeTip: String,
20     val trunkScore: Int,
21     val trunkMeasuredAngle: Float,
22     val trunkFeedback: String,
23     val trunkTip: String,
24     val elbowScore: Int,
25     val elbowMeasuredAngle: Float,
26     val elbowFeedback: String,
27     val elbowTip: String,
28     val hipScore: Int,
29     val hipMeasuredAngle: Float,
30     val hipFeedback: String,
31     val hipTip: String,
32     val overstrideScore: Int,
33     val overstrideRatio: Float,
34     val overstrideFeedback: String,
35     val overstrideTip: String,
36     // Reference metrics (v3+; default 0 for older rows)
37     val cadenceScore: Int = 0,
38     val cadenceSpm: Float = 0f,
39     val cadenceFeedback: String = "",
40     val cadenceTip: String = "",
41     val verticalOscScore: Int = 0,
42     val verticalOscPercent: Float = 0f,
43     val verticalOscFeedback: String = "",
44     val verticalOscTip: String = "",
45     // Video replay fields (nullable: older analyses may not have a saved video)
46     val videoPath: String? = null,
47     val videoFramesJson: String? = null,
48     val videoWidth: Int = 0,
49     val videoHeight: Int = 0,
50 )
```

```
1 package com.runway.android.core.posture.local
2
3 import androidx.room.Database
4 import androidx.room.RoomDatabase
5 import androidx.room.migration.Migration
6 import androidx.sqlite.db.SupportSQLiteDatabase
7
8 @Database(
9     entities = [PostureAnalysisEntity::class],
10    version = 4,
11    exportSchema = false,
12 )
13 abstract class PostureDatabase : RoomDatabase() {
14     abstract fun postureAnalysisDao(): PostureAnalysisDao
15
16     companion object {
17         val MIGRATION_2_3 = object : Migration(2, 3) {
18             override fun migrate(db: SupportSQLiteDatabase) {
19                 db.execSQL("ALTER TABLE posture_analyses ADD COLUMN cadenceScore INTEGER NOT NULL DEFAULT 0")
20                 db.execSQL("ALTER TABLE posture_analyses ADD COLUMN cadenceSpm REAL NOT NULL DEFAULT 0")
21                 db.execSQL("ALTER TABLE posture_analyses ADD COLUMN cadenceFeedback TEXT NOT NULL DEFAULT ''")
22                 db.execSQL("ALTER TABLE posture_analyses ADD COLUMN cadenceTip TEXT NOT NULL DEFAULT ''")
23                 db.execSQL("ALTER TABLE posture_analyses ADD COLUMN verticalOscScore INTEGER NOT NULL DEFAULT 0")
24                 db.execSQL("ALTER TABLE posture_analyses ADD COLUMN verticalOscPercent REAL NOT NULL DEFAULT 0")
25                 db.execSQL("ALTER TABLE posture_analyses ADD COLUMN verticalOscFeedback TEXT NOT NULL DEFAULT ''")
26                 db.execSQL("ALTER TABLE posture_analyses ADD COLUMN verticalOscTip TEXT NOT NULL DEFAULT ''")
27             }
28         }
29
30         val MIGRATION_3_4 = object : Migration(3, 4) {
31             override fun migrate(db: SupportSQLiteDatabase) {
32                 db.execSQL("ALTER TABLE posture_analyses ADD COLUMN ownerId TEXT NOT NULL DEFAULT ''")
33             }
34         }
35     }
36 }
```

```

1 package com.runway.android.core.result
2
3 import com.google.gson.JsonObject
4 import com.google.gson.JsonParser
5 import com.runway.android.core.model.ApiResponse
6 import kotlinx.coroutines.CancellationException
7 import retrofit2.HttpException
8
9 /**
10  * 모든 네트워크 호출 결과를 표현하는 sealed class.
11  *
12  * ViewModel에서 다음과 같이 처리한다:
13  * when (result) {
14  *     is NetworkResult.Success -> // result.data 사용
15  *     is NetworkResult.ApiError -> // result.errorCode로 UI 메시지 결정
16  *     is NetworkResult.NetworkError -> // 연결 오류 처리
17  * }
18  */
19 sealed class NetworkResult<out T> {
20     data class Success<T>(val data: T) : NetworkResult<T>()
21
22     data class ApiError(
23         val statusCode: Int,
24         val errorCode: String?,
25         val message: String,
26     ) : NetworkResult<Nothing>()
27
28     data class NetworkError(val throwable: Throwable) : NetworkResult<Nothing>()
29 }
30
31 /**
32  * Retrofit suspend API 호출을 NetworkResult로 변환하는 helper.
33  *
34  * - 성공(2xx + success=true): NetworkResult.Success
35  * - 서버 에러(4xx/5xx): 응답 body에서 errorCode 파싱 → NetworkResult.ApiError
36  * - 네트워크 오류(IOException 등): NetworkResult.NetworkError
37  *
38  * Retrofit이 4xx/5xx에서 HttpException을 throw하므로,
39  * errorCode를 얻으려면 error body를 직접 파싱한다.
40  */
41 suspend fun <T : Any> safeApiCall(call: suspend () -> ApiResponse<T>): NetworkResult<T> {
42     return try {
43         val response = call()
44         if (response.success && response.data != null) {
45             NetworkResult.Success(response.data)
46         } else {
47             NetworkResult.ApiError(
48                 statusCode = 0,
49                 errorCode = response.errorCode,
50                 message = response.message,
51             )
52         }
53     } catch (e: HttpException) {
54         parseHttpError(e)
55     } catch (e: CancellationException) {
56         throw e // coroutine 취소 신호는 삼키지 않고 전파
57     } catch (e: Exception) {
58         NetworkResult.NetworkError(e)
59     }
60 }
61
62 /**
63  * ApiResponse<Unit> 엔드포인트(DELETE 등)용 helper.
64  * 백엔드가 data 필드를 null로 반환하므로 success=true 이면 성공으로 처리.
65  */
66 suspend fun safeApiCallUnit(call: suspend () -> ApiResponse<Unit>): NetworkResult<Unit> {
67     return try {
68         val response = call()
69         if (response.success) {
70             NetworkResult.Success(Unit)
71         } else {
72             NetworkResult.ApiError(
73                 statusCode = 0,
74                 errorCode = response.errorCode,
75                 message = response.message,
76             )
77         }
78     } catch (e: HttpException) {
79         parseHttpError(e)
80     } catch (e: CancellationException) {
81         throw e
82     } catch (e: Exception) {
83         NetworkResult.NetworkError(e)
84     }
85 }

```

```
84     }
85 }
86
87 private fun parseHttpError(e: HttpException): NetworkResult.ApiError {
88     val errorBody = e.response()?.errorBody()?.string()
89     return if (errorBody != null) {
90         runCatching {
91             val json: JsonObject = JsonParser.parseString(errorBody).asJsonObject
92             NetworkResult.ApiError(
93                 statusCode = e.code(),
94                 errorCode = json.get("errorCode")?.takeIf { !it.isJsonNull }?.asString,
95                 message = json.get("message")?.asString ?: "HTTP ${e.code()}",
96             )
97         }.getOrNull {
98             NetworkResult.ApiError(e.code(), null, "HTTP ${e.code()}")
99         }
100     } else {
101         NetworkResult.ApiError(e.code(), null, "HTTP ${e.code()}")
102     }
103 }
```

```

1 package com.runway.android.core.running
2
3 import android.location.Location
4 import com.runway.android.data.running.model.RunPointResponse
5 import java.time.Instant
6
7 object RunChartCalculator {
8
9     private const val MIN_POINTS = 5
10    private const val MAX_PACE = 1800    // 30 min/km - filter out stopped/paused noise
11    private const val SMOOTH_WINDOW = 5
12
13    fun calculate(points: List<RunPointResponse>): List<RunChartPoint> {
14        if (points.size < MIN_POINTS) return emptyList()
15
16        val sorted = points.sortedWith(compareBy({ it.sequence }, { it.recordedAt }))
17        val startSecs = parseEpochSecs(sorted.first().recordedAt) ?: return emptyList()
18
19        val raw = mutableListOf<RunChartPoint>()
20
21        for (i in 1 until sorted.size) {
22            val prev = sorted[i - 1]
23            val curr = sorted[i]
24
25            val currSecs = parseEpochSecs(curr.recordedAt) ?: continue
26            val elapsed = (currSecs - startSecs).toInt()
27            if (elapsed <= 0) continue
28
29            val pace: Int = if (curr.speedMps != null && curr.speedMps > 0.1f) {
30                (1000f / curr.speedMps).toInt()
31            } else {
32                val dist = distanceBetween(prev.latitude, prev.longitude, curr.latitude, curr.longitude)
33                val dt = (currSecs - (parseEpochSecs(prev.recordedAt) ?: currSecs)).toInt()
34                if (dist < 1.0 || dt <= 0) continue
35                (dt / (dist / 1000.0)).toInt()
36            }
37
38            if (pace in 1..MAX_PACE) {
39                raw.add(RunChartPoint(elapsedSeconds = elapsed, paceSecondsPerKm = pace))
40            }
41        }
42
43        if (raw.isEmpty()) return emptyList()
44        return smooth(raw)
45    }
46
47    private fun smooth(points: List<RunChartPoint>): List<RunChartPoint> {
48        val half = SMOOTH_WINDOW / 2
49        return points.mapIndexed { i, p ->
50            val window = points.subList(
51                maxOf(0, i - half),
52                minOf(points.size, i + half + 1),
53            )
54            p.copy(paceSecondsPerKm = window.map { it.paceSecondsPerKm }.average().toInt())
55        }
56    }
57
58    private fun distanceBetween(lat1: Double, lon1: Double, lat2: Double, lon2: Double): Double {
59        if (lat1 == lat2 && lon1 == lon2) return 0.0
60        val result = FloatArray(1)
61        Location.distanceBetween(lat1, lon1, lat2, lon2, result)
62        return result[0].toDouble()
63    }
64
65    private fun parseEpochSecs(isoString: String?): Long? =
66        runCatching { Instant.parse(isoString).epochSecond }.getOrNull()
67 }

```

```

1 package com.runway.android.core.running
2
3 data class RunChartPoint(
4     val elapsedSeconds: Int,
5     val paceSecondsPerKm: Int,
6 )

```



```
1 package com.runway.android.core.running
2
3 data class RunSplit(
4     val splitNumber: Int,          // 1-based sequence
5     val distanceKm: Double,        // 1.0 for full km splits; < 1.0 for the partial last split
6     val durationSeconds: Int,
7     val paceSecondsPerKm: Int,
8 )
```

```

1 package com.runway.android.core.running
2
3 import android.location.Location
4 import com.runway.android.data.running.model.RunPointResponse
5 import java.time.Instant
6
7 object SplitCalculator {
8
9     private const val SPLIT_DISTANCE_METERS = 1000.0
10    private const val MIN_PARTIAL_METERS = 10.0 // ignore sub-10m trailing segment
11
12    /**
13     * Calculates 1km splits from an ordered list of GPS points.
14     *
15     * Returns an empty list when:
16     * - fewer than 2 points provided
17     * - total distance < 10m
18     * - timestamps cannot be parsed
19     *
20     * The last entry may be a partial split (distanceKm < 1.0).
21     */
22    fun calculate(points: List<RunPointResponse>): List<RunSplit> {
23        if (points.size < 2) return emptyList()
24
25        // Sort defensively; backend should return ordered by sequence
26        val sorted = points.sortedWith(compareBy({ it.sequence }, { it.recordedAt }))
27
28        val first = sorted.first()
29        var splitStartSecs = parseEpochSecs(first.recordedAt) ?: return emptyList()
30
31        val splits = mutableListOf<RunSplit>()
32        var splitNumber = 1
33        var splitDistanceMeters = 0.0
34
35        for (i in 1 until sorted.size) {
36            val prev = sorted[i - 1]
37            val curr = sorted[i]
38
39            val currSecs = parseEpochSecs(curr.recordedAt) ?: continue
40            if (currSecs < splitStartSecs) continue // skip out-of-order timestamps
41
42            val segment = distanceBetweenMeters(
43                prev.latitude, prev.longitude,
44                curr.latitude, curr.longitude,
45            )
46            splitDistanceMeters += segment
47
48            if (splitDistanceMeters >= SPLIT_DISTANCE_METERS) {
49                val durationSecs = (currSecs - splitStartSecs).toInt().coerceAtLeast(1)
50                splits.add(
51                    RunSplit(
52                        splitNumber = splitNumber++,
53                        distanceKm = splitDistanceMeters / 1000.0,
54                        durationSeconds = durationSecs,
55                        paceSecondsPerKm = paceSecsPerKm(durationSecs, splitDistanceMeters),
56                    )
57                )
58                splitStartSecs = currSecs
59                splitDistanceMeters = 0.0
60            }
61        }
62
63        // Partial last split
64        if (splitDistanceMeters >= MIN_PARTIAL_METERS) {
65            val endSecs = parseEpochSecs(sorted.last().recordedAt)
66            if (endSecs != null && endSecs > splitStartSecs) {
67                val durationSecs = (endSecs - splitStartSecs).toInt().coerceAtLeast(1)
68                splits.add(
69                    RunSplit(
70                        splitNumber = splitNumber,
71                        distanceKm = splitDistanceMeters / 1000.0,
72                        durationSeconds = durationSecs,
73                        paceSecondsPerKm = paceSecsPerKm(durationSecs, splitDistanceMeters),
74                    )
75                )
76            }
77        }
78
79        return splits
80    }
81
82    private fun paceSecsPerKm(durationSecs: Int, distanceMeters: Double): Int {
83        if (distanceMeters <= 0) return 0

```

```
84         return (durationSecs / (distanceMeters / 1000.0)).toInt()
85     }
86
87     private fun distanceBetweenMeters(
88         lat1: Double, lon1: Double,
89         lat2: Double, lon2: Double,
90     ): Double {
91         if (lat1 == lat2 && lon1 == lon2) return 0.0
92         val result = FloatArray(1)
93         Location.distanceBetween(lat1, lon1, lat2, lon2, result)
94         return result[0].toDouble()
95     }
96
97     private fun parseEpochSecs(isoString: String?): Long? =
98         runCatching { Instant.parse(isoString).epochSecond }.getOrNull()
99 }
```

```
1 package com.runway.android.core.share
2
3 import android.graphics.Bitmap
4 import android.graphics.Canvas
5 import android.graphics.Paint
6 import android.graphics.Path
7 import android.graphics.RectF
8 import android.graphics.Typeface
9 import com.runway.android.BuildConfig
10 import com.runway.android.data.running.model.RunDetailResponse
11 import com.runway.android.data.running.model.RunPointResponse
12 import java.time.Instant
13 import java.time.ZoneId
14 import java.time.format.DateTimeFormatter
15
16 object ImageCaptureUtil {
17
18     private const val SIZE = 1080
19
20     fun draw(detail: RunDetailResponse, template: ShareTemplate, preset: MetricPreset = MetricPreset.FULL_STATS): Bitmap {
21         val bitmap = Bitmap.createBitmap(SIZE, SIZE, Bitmap.Config.ARGB_8888)
22         val canvas = Canvas(bitmap)
23         drawCard(canvas, detail, template, preset)
24         return bitmap
25     }
26
27     private fun drawCard(canvas: Canvas, detail: RunDetailResponse, template: ShareTemplate, preset: MetricPreset) {
28         val W = SIZE.toFloat()
29         val H = SIZE.toFloat()
30
31         // Background
32         canvas.drawColor(template.bgColor.toInt())
33
34         // Top accent bar
35         val accentPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = template.accentColor.toInt() }
36         canvas.drawRect(0f, 0f, W, 10f, accentPaint)
37
38         // App wordmark
39         val brandPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
40             color = template.textPrimary.toInt()
41             textSize = 64f
42             typeface = Typeface.create(Typeface.DEFAULT, Typeface.BOLD)
43             letterSpacing = 0.25f
44         }
45         val wordmark = BuildConfig.APP_NAME.uppercase()
46         canvas.drawText(wordmark, 80f, 120f, brandPaint)
47
48         // Accent dot after wordmark
49         canvas.drawCircle(80f + brandPaint.measureText(wordmark) + 18f, 107f, 8f, accentPaint)
50
51         // Date
52         val datePaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
53             color = template.textSecondary.toInt()
54             textSize = 38f
55         }
56         canvas.drawText(formatDate(detail.startedAt), 80f, 176f, datePaint)
57
58         // Divider
59         val dividerPaint = Paint().apply {
60             color = template.accentColor.toInt()
61             strokeWidth = 2f
62             alpha = 100
63         }
64         canvas.drawLine(80f, 210f, W - 80f, 210f, dividerPaint)
65
66         // Distance (hero number)
67         val distKm = (detail.distanceMeters ?: 0.0) / 1000.0
68         val distPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
69             color = template.textPrimary.toInt()
70             textSize = 210f
71             typeface = Typeface.create(Typeface.DEFAULT, Typeface.BOLD)
72         }
73         canvas.drawText("%.2f".format(distKm), 70f, 460f, distPaint)
74
75         val unitPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
76             color = template.textSecondary.toInt()
77             textSize = 56f
78         }
79         canvas.drawText("km", 80f, 530f, unitPaint)
80
81         // Metrics row (preset-dependent)
82         val labelPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
83             color = template.textSecondary.toInt()
```

```

84         textSize = 34f
85     }
86     val valuePaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
87         color = template.textPrimary.toInt()
88         textSize = 52f
89         typeface = Typeface.create(Typeface.DEFAULT, Typeface.BOLD)
90     }
91     val col1 = 80f
92     val col2 = W / 2 - 80f
93     val col3 = W - 280f
94
95     when (preset) {
96         MetricPreset.FULL_STATS -> {
97             canvas.drawText("시간", col1, 640f, labelPaint)
98             canvas.drawText(formatDuration(detail.durationSeconds), col1, 700f, valuePaint)
99             canvas.drawText("평균 페이스", col2, 640f, labelPaint)
100            canvas.drawText(formatPace(detail.avgPaceSecondsPerKm), col2, 700f, valuePaint)
101            if (detail.caloriesBurned != null) {
102                canvas.drawText("칼로리", col3, 640f, labelPaint)
103                canvas.drawText("${detail.caloriesBurned}", col3, 700f, valuePaint)
104                canvas.drawText("kcal", col3, 740f, labelPaint)
105            }
106        }
107        MetricPreset.DISTANCE_FOCUS -> {
108            canvas.drawText("소요 시간", col1, 640f, labelPaint)
109            canvas.drawText(formatDuration(detail.durationSeconds), col1, 700f, valuePaint)
110        }
111        MetricPreset.PACE_FOCUS -> {
112            val paceBigPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
113                color = template.accentColor.toInt()
114                textSize = 100f
115                typeface = Typeface.create(Typeface.DEFAULT, Typeface.BOLD)
116            }
117            canvas.drawText("평균 페이스", col1, 600f, labelPaint)
118            canvas.drawText(formatPace(detail.avgPaceSecondsPerKm), col1, 710f, paceBigPaint)
119            canvas.drawText("소요 시간", col1, 780f, labelPaint)
120            canvas.drawText(formatDuration(detail.durationSeconds), col1, 840f, valuePaint)
121        }
122    }
123
124    // Second divider
125    val dividerY = if (preset == MetricPreset.PACE_FOCUS) 880f else 760f
126    canvas.drawLine(80f, dividerY, W - 80f, dividerY, dividerPaint)
127
128    // Route preview (if enough points)
129    val hasRoute = detail.points.size >= 2
130    val routeTop = if (preset == MetricPreset.PACE_FOCUS) 900f else 785f
131    val routeBounds = if (hasRoute) RectF(80f, routeTop, W - 80f, H - 80f) else null
132    if (routeBounds != null) {
133        drawRoute(canvas, detail.points, template, routeBounds)
134    }
135
136    // Motivational phrase
137    val phrasePaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
138        color = template.accentColor.toInt()
139        textSize = 34f
140        letterSpacing = 0.12f
141    }
142    val phrase = if (routeBounds == null) "Run your way." else ""
143    if (phrase.isNotEmpty()) {
144        canvas.drawText(phrase, 80f, H - 40f, phrasePaint)
145    }
146 }
147
148 private fun drawRoute(
149     canvas: Canvas,
150     points: List<RunPointResponse>,
151     template: ShareTemplate,
152     bounds: RectF,
153 ) {
154     val minLat = points.minOf { it.latitude }
155     val maxLat = points.maxOf { it.latitude }
156     val minLon = points.minOf { it.longitude }
157     val maxLon = points.maxOf { it.longitude }
158     val latRange = (maxLat - minLat).coerceAtLeast(0.0001)
159     val lonRange = (maxLon - minLon).coerceAtLeast(0.0001)
160
161     val margin = 20f
162     val innerW = bounds.width() - margin * 2
163     val innerH = bounds.height() - margin * 2
164
165     fun toX(lon: Double) = bounds.left + margin + ((lon - minLon) / lonRange * innerW).toFloat()
166     fun toY(lat: Double) = bounds.bottom - margin - ((lat - minLat) / latRange * innerH).toFloat()
167
168     val routePaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
169         color = template.accentColor.toInt()

```

```

170         strokeWidth = 5f
171         style = Paint.Style.STROKE
172     }
173     val path = Path()
174     path.moveTo(toX(points[0].longitude), toY(points[0].latitude))
175     points.drop(1).forEach { p -> path.lineTo(toX(p.longitude), toY(p.latitude)) }
176     canvas.drawPath(path, routePaint)
177
178     // Start and end dots
179     val dotPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = template.accentColor.toInt() }
180     canvas.drawCircle(toX(points.first().longitude), toY(points.first().latitude), 14f, dotPaint)
181     canvas.drawCircle(toX(points.last().longitude), toY(points.last().latitude), 14f, dotPaint)
182 }
183
184 private fun formatDate(isoDate: String?): String {
185     if (isoDate == null) return ""
186     return runCatching {
187         val instant = Instant.parse(isoDate)
188         val formatter = DateTimeFormatter.ofPattern("yyyy.MM.dd")
189             .withZone(ZoneId.systemDefault())
190         formatter.format(instant)
191     }.getOrElse("")
192 }
193
194 private fun formatDuration(seconds: Int?): String {
195     if (seconds == null) return "--:--"
196     return "%d:%02d".format(seconds / 60, seconds % 60)
197 }
198
199 private fun formatPace(secsPerKm: Int?): String {
200     if (secsPerKm == null || secsPerKm <= 0) return "--'--\"
201     return "%d'%02d\"".format(secsPerKm / 60, secsPerKm % 60)
202 }
203 }

```

android/app/src/main/java/com/runway/android/core/share/ImageShareUtil.kt

Kotlin | 33줄

```

1 package com.runway.android.core.share
2
3 import android.content.Context
4 import android.content.Intent
5 import android.graphics.Bitmap
6 import android.net.Uri
7 import androidx.core.content.FileProvider
8 import java.io.File
9
10 object ImageShareUtil {
11
12     private const val CACHE_DIR = "share_images"
13     private const val FILE_NAME = "runway_run.png"
14
15     fun saveToCache(context: Context, bitmap: Bitmap): Uri? = runCatching {
16         val dir = File(context.cacheDir, CACHE_DIR).also { it.mkdirs() }
17         val file = File(dir, FILE_NAME)
18         file.outputStream().use { out ->
19             bitmap.compress(Bitmap.CompressFormat.PNG, 95, out)
20         }
21         FileProvider.getUriForFile(context, "${context.packageName}.fileprovider", file)
22     }.getOrNull()
23
24     fun buildShareIntent(uri: Uri): Intent =
25         Intent.createChooser(
26             Intent(Intent.ACTION_SEND).apply {
27                 type = "image/png"
28                 putExtra(Intent.EXTRA_STREAM, uri)
29                 addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
30             },
31             null,
32         )
33 }

```

android/app/src/main/java/com/runway/android/core/share/MetricPreset.kt

Kotlin | 7줄

```

1 package com.runway.android.core.share
2
3 enum class MetricPreset(val displayName: String) {
4     FULL_STATS("전체"),
5     DISTANCE_FOCUS("거리"),
6     PACE_FOCUS("페이스"),
7 }

```

```
1 package com.runway.android.core.share
2
3 enum class ShareTemplate(
4     val displayName: String,
5     val bgColor: Long,
6     val accentColor: Long,
7     val textPrimary: Long,
8     val textSecondary: Long,
9 ) {
10     DARK_SPORT(
11         displayName = "Dark Sport",
12         bgColor = 0xFF0D0D0D,
13         accentColor = 0xFF39FF14,
14         textPrimary = 0xFFFFFFFF,
15         textSecondary = 0xFFAAAAAA,
16     ),
17     MINIMAL_LIGHT(
18         displayName = "Minimal",
19         bgColor = 0xFFFF5F5F5,
20         accentColor = 0xFF222222,
21         textPrimary = 0xFF111111,
22         textSecondary = 0xFF666666,
23     ),
24     ROUTE_FOCUS(
25         displayName = "Route Focus",
26         bgColor = 0xFF11827,
27         accentColor = 0xFF6366F1,
28         textPrimary = 0xFFFFFFFF,
29         textSecondary = 0xFF9CA3AF,
30     ),
31     SUNRISE_RUN(
32         displayName = "Sunrise",
33         bgColor = 0xFF1A0A00,
34         accentColor = 0xFFFF6B35,
35         textPrimary = 0xFFFFF5E4,
36         textSecondary = 0xFFBB8866,
37     ),
38     OCEAN_PULSE(
39         displayName = "Ocean",
40         bgColor = 0xFF001B2E,
41         accentColor = 0xFF00D4FF,
42         textPrimary = 0xFFE0F7FF,
43         textSecondary = 0xFF6CA3B8,
44     ),
45 }
```

android/app/src/main/java/com/runway/android/core/share/ShareUtils.kt

Kotlin | 63줄

```

1 package com.runway.android.core.share
2
3 import android.app.Activity
4 import android.content.Context
5 import android.content.Intent
6 import com.runway.android.BuildConfig
7
8 object ShareUtils {
9     fun shareText(context: Context, subject: String, text: String) {
10         val shareIntent = Intent(Intent.ACTION_SEND).apply {
11             type = "text/plain"
12             putExtra(Intent.EXTRA_SUBJECT, subject)
13             putExtra(Intent.EXTRA_TEXT, text)
14         }
15         val chooser = Intent.createChooser(shareIntent, subject).apply {
16             // Activity context가 아닌 경우(Application, Service 등) 새 태스크로 시작
17             if (context !is Activity) {
18                 addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
19             }
20         }
21         context.startActivity(chooser)
22     }
23
24     fun shareCourse(
25         context: Context,
26         name: String,
27         distanceKm: String,
28         completionCount: Int,
29         isLoop: Boolean,
30     ) {
31         val loopTag = if (isLoop) " (루프 코스)" else ""
32         val text = buildString {
33             appendLine("${BuildConfig.APP_NAME} 코스 공유 🏃.")
34             appendLine()
35             appendLine("📍 $name$loopTag")
36             appendLine("📏 거리: $distanceKm")
37             appendLine("🏁 완주 횟수: ${completionCount}회")
38             appendLine()
39             append("${BuildConfig.APP_NAME} 앱에서 이 코스에 도전해보세요!")
40         }
41         shareText(context, "코스 공유 - $name", text)
42     }
43
44     fun shareRunSummary(
45         context: Context,
46         distanceFormatted: String,
47         duration: String,
48         pace: String,
49         date: String,
50     ) {
51         val text = buildString {
52             appendLine("${BuildConfig.APP_NAME} 러닝 기록 🏃.")
53             appendLine()
54             appendLine("📅 $date")
55             appendLine("📏 거리: $distanceFormatted")
56             appendLine("⌚ 시간: $duration")
57             appendLine("🏃 페이스: $pace")
58             appendLine()
59             append("${BuildConfig.APP_NAME}에서 기록하세요!")
60         }
61         shareText(context, "러닝 기록 공유", text)
62     }
63 }

```

android/app/src/main/java/com/runway/android/core/tracking/PauseReason.kt

Kotlin | 7줄

```

1 package com.runway.android.core.tracking
2
3 enum class PauseReason {
4     NONE, // running normally
5     MANUAL, // user pressed pause
6     AUTO, // auto-paused due to low speed
7 }

```


android/app/src/main/java/com/runway/android/core/tracking/PendingPointQueue.kt

Kotlin | 47줄

```
1 package com.runway.android.core.tracking
2
3 import com.runway.android.core.tracking.local.PendingRunPointDao
4 import com.runway.android.core.tracking.local.PendingRunPointEntity
5 import com.runway.android.data.running.model.RunPointRequest
6 import javax.inject.Inject
7 import javax.inject.Singleton
8
9 @Singleton
10 class PendingPointQueue @Inject constructor(
11     private val dao: PendingRunPointDao,
12 ) {
13     suspend fun add(runningRecordId: String, points: List<RunPointRequest>) {
14         if (points.isEmpty()) return
15         dao.insertAll(points.map { p ->
16             PendingRunPointEntity(
17                 runningRecordId = runningRecordId,
18                 sequence = p.sequence,
19                 latitude = p.latitude,
20                 longitude = p.longitude,
21                 altitudeMeters = p.altitudeMeters,
22                 speedMps = p.speedMps,
23                 recordedAt = p.recordedAt,
24             )
25         })
26     }
27
28     suspend fun dequeue(runningRecordId: String, batchSize: Int = 20): List<PendingRunPointEntity> =
29         dao.dequeue(runningRecordId, batchSize)
30
31     suspend fun deleteByIds(ids: List<Long>) {
32         if (ids.isEmpty()) return
33         dao.deleteByIds(ids)
34     }
35
36     suspend fun deleteByRunningRecordId(runningRecordId: String) =
37         dao.deleteByRunningRecordId(runningRecordId)
38 }
39
40 fun PendingRunPointEntity.toRunPointRequest() = RunPointRequest(
41     sequence = sequence,
42     latitude = latitude,
43     longitude = longitude,
44     altitudeMeters = altitudeMeters,
45     speedMps = speedMps,
46     recordedAt = recordedAt,
47 )
```

android/app/src/main/java/com/runway/android/core/tracking/RunTrackingAction.kt

Kotlin | 7줄

```
1 package com.runway.android.core.tracking
2
3 object RunTrackingAction {
4     const val ACTION_START = "com.runway.android.tracking.START"
5     const val ACTION_STOP  = "com.runway.android.tracking.STOP"
6     const val EXTRA_MODE   = "extra_mode"
7 }
```

```

1  package com.runway.android.core.tracking
2
3  import com.runway.android.core.cadence.CadenceTracker
4  import com.runway.android.core.location.DistanceCalculator
5  import com.runway.android.core.location.GpsPointValidator
6  import com.runway.android.core.location.LocationTracker
7  import com.runway.android.data.running.model.RunPointRequest
8  import kotlinx.coroutines.CoroutineScope
9  import kotlinx.coroutines.Job
10 import kotlinx.coroutines.delay
11 import kotlinx.coroutines.flow.MutableSharedFlow
12 import kotlinx.coroutines.flow.MutableStateFlow
13 import kotlinx.coroutines.flow.SharedFlow
14 import kotlinx.coroutines.flow.StateFlow
15 import kotlinx.coroutines.flow.asSharedFlow
16 import kotlinx.coroutines.flow.asStateFlow
17 import kotlinx.coroutines.flow.update
18 import kotlinx.coroutines.launch
19 import kotlinx.coroutines.sync.Mutex
20 import kotlinx.coroutines.sync.withLock
21 import java.time.Instant
22 import java.util.concurrent.atomic.AtomicInteger
23 import javax.inject.Inject
24 import javax.inject.Named
25 import javax.inject.Singleton
26
27 /**
28  * 앱 전체 생명주기를 따르는 추적 상태 관리자.
29  * GPS 수집과 타이머는 appScope(Dispatchers.IO)에서 실행되므로
30  * ForegroundService가 살아있는 동안 앱이 백그라운드에 있어도 계속 동작한다.
31  *
32  * Auto-pause: speed < AUTO_PAUSE_SPEED_MPS 가 AUTO_PAUSE_TICKS 번 연속으로 감지되면
33  * 자동으로 일시정지. speed > AUTO_RESUME_SPEED_MPS 가 AUTO_RESUME_TICKS 번 연속이면
34  * 자동 재개.
35  */
36 @Singleton
37 class RunTrackingManager @Inject constructor(
38     private val locationTracker: LocationTracker,
39     private val cadenceTracker: CadenceTracker,
40     @Named("appScope") private val appScope: CoroutineScope,
41 ) {
42     companion object {
43         private const val AUTO_PAUSE_SPEED_MPS = 0.5f // 이하 속도 = 정지 판정
44         private const val AUTO_RESUME_SPEED_MPS = 1.0f // 초과 속도 = 움직임 재개
45         private const val AUTO_PAUSE_TICKS = 10 // 10회 연속 → auto-pause
46         private const val AUTO_RESUME_TICKS = 3 // 3회 연속 → auto-resume
47     }
48
49     private val _state = MutableStateFlow(RunTrackingState())
50     val state: StateFlow<RunTrackingState> = _state.asStateFlow()
51
52     /** Emits the completed kilometer count (1, 2, 3 ...) each time a milestone is crossed. */
53     private val _milestoneFlow = MutableSharedFlow<Int>(extraBufferCapacity = 10)
54     val milestoneFlow: SharedFlow<Int> = _milestoneFlow.asSharedFlow()
55
56     private val pointsMutex = Mutex()
57     private val _pendingPoints = mutableListOf<RunPointRequest>()
58     private val pointSequence = AtomicInteger(0)
59
60     private var locationJob: Job? = null
61     private var timerJob: Job? = null
62     private var cadenceJob: Job? = null
63
64     @Volatile private var lowSpeedTicks = 0
65     @Volatile private var highSpeedTicks = 0
66     @Volatile private var lastMilestoneKm = 0
67
68     fun start(mode: RunTrackingMode) {
69         pointSequence.set(0)
70         lowSpeedTicks = 0
71         highSpeedTicks = 0
72         lastMilestoneKm = 0
73         _state.value = RunTrackingState(mode = mode, isTracking = true)
74         startTimer()
75         startCadenceTracking()
76         startLocationTracking()
77     }
78
79     fun pause() {
80         lowSpeedTicks = 0
81         highSpeedTicks = 0
82         _state.update { it.copy(isPaused = true, pauseReason = PauseReason.MANUAL) }
83     }

```

```

84
85 fun resume() {
86     lowSpeedTicks = 0
87     highSpeedTicks = 0
88     _state.update { it.copy(isPaused = false, pauseReason = PauseReason.NONE) }
89 }
90
91 fun stop() {
92     timerJob?.cancel()
93     locationJob?.cancel()
94     cadenceJob?.cancel()
95     timerJob = null
96     locationJob = null
97     cadenceJob = null
98     lowSpeedTicks = 0
99     highSpeedTicks = 0
100    lastMilestoneKm = 0
101    _state.value = RunTrackingState()
102    appScope.launch { pointsMutex.withLock { _pendingPoints.clear() } }
103    pointSequence.set(0)
104 }
105
106 private fun startCadenceTracking() {
107     cadenceJob?.cancel()
108     cadenceJob = appScope.launch {
109         cadenceTracker.cadenceSpmFlow().collect { cadence ->
110             _state.update { it.copy(cadenceSpm = cadence) }
111         }
112     }
113 }
114
115 /** Thread-safe consume: returns all pending points and clears the queue. */
116 suspend fun consumePoints(): List<RunPointRequest> = pointsMutex.withLock {
117     _pendingPoints.toList().also { _pendingPoints.clear() }
118 }
119
120 /** Re-enqueues points at the front (called on upload failure to preserve order). */
121 suspend fun requeuePoints(points: List<RunPointRequest>) = pointsMutex.withLock {
122     _pendingPoints.addAll(0, points)
123 }
124
125 private fun startTimer() {
126     timerJob?.cancel()
127     timerJob = appScope.launch {
128         while (true) {
129             delay(1_000L)
130             if (!_state.value.isPaused) {
131                 _state.update { it.copy(elapsedSeconds = it.elapsedSeconds + 1) }
132             }
133         }
134     }
135 }
136
137 private fun startLocationTracking() {
138     locationJob?.cancel()
139     locationJob = appScope.launch {
140         locationTracker.locationFlow().collect { location ->
141             val currentState = _state.value
142             val speed = location.speedMps ?: 0f
143
144             when {
145                 !currentState.isPaused -> {
146                     // Track low-speed ticks for auto-pause detection
147                     if (speed < AUTO_PAUSE_SPEED_MPS) {
148                         lowSpeedTicks++
149                         highSpeedTicks = 0
150                     } else {
151                         lowSpeedTicks = 0
152                         highSpeedTicks = 0
153                     }
154
155                     if (lowSpeedTicks >= AUTO_PAUSE_TICKS) {
156                         // Trigger auto-pause - don't add the current point
157                         lowSpeedTicks = 0
158                         _state.update {
159                             it.copy(
160                                 isPaused = true,
161                                 pauseReason = PauseReason.AUTO,
162                                 hasFirstFix = true,
163                                 currentSpeedMps = location.speedMps,
164                                 lastLocation = location,
165                             )
166                         }
167                         return@collect
168                     }
169

```

```

170         // Normal GPS point processing
171         val prev = currentState.lastLocation
172         if (!GpsPointValidator.isValid(location, prev)) {
173             _state.update {
174                 it.copy(
175                     hasFirstFix = true,
176                     currentSpeedMps = location.speedMps,
177                     lastLocation = location,
178                 )
179             }
180             return@collect
181         }
182
183         val deltaMeters = if (prev != null) {
184             DistanceCalculator.calculate(prev, location)
185         } else {
186             0.0
187         }
188
189         val point = RunPointRequest(
190             sequence = pointSequence.getAndIncrement(),
191             latitude = location.latitude,
192             longitude = location.longitude,
193             altitudeMeters = location.altitudeMeters ?: 0.0,
194             speedMps = location.speedMps?.toDouble() ?: 0.0,
195             recordedAt = location.recordedAt.toString(),
196         )
197         pointsMutex.withLock { _pendingPoints.add(point) }
198
199         val newDistance = currentState.distanceMeters + deltaMeters
200         _state.update {
201             it.copy(
202                 hasFirstFix = true,
203                 distanceMeters = newDistance,
204                 currentSpeedMps = location.speedMps,
205                 lastLocation = location,
206             )
207         }
208
209         // Emit km milestone event
210         val newKm = (newDistance / 1000.0).toInt()
211         if (newKm > lastMilestoneKm) {
212             lastMilestoneKm = newKm
213             _milestoneFlow.tryEmit(newKm)
214         }
215     }
216
217     currentState.pauseReason == PauseReason.AUTO -> {
218         // Auto-paused: monitor for auto-resume
219         if (speed > AUTO_RESUME_SPEED_MPS) {
220             highSpeedTicks++
221             if (highSpeedTicks >= AUTO_RESUME_TICKS) {
222                 highSpeedTicks = 0
223                 lowSpeedTicks = 0
224                 _state.update {
225                     it.copy(
226                         isPaused = false,
227                         pauseReason = PauseReason.NONE,
228                         hasFirstFix = true,
229                         currentSpeedMps = location.speedMps,
230                         lastLocation = location,
231                     )
232                 }
233                 return@collect
234             }
235         } else {
236             highSpeedTicks = 0
237         }
238         // Update position/speed without accumulating distance
239         _state.update {
240             it.copy(
241                 hasFirstFix = true,
242                 currentSpeedMps = location.speedMps,
243                 lastLocation = location,
244             )
245         }
246     }
247
248     else -> {
249         // Manual pause: update position without accumulating distance
250         _state.update {
251             it.copy(
252                 hasFirstFix = true,
253                 currentSpeedMps = location.speedMps,
254                 lastLocation = location,
255             )

```

```
256         }
257     }
258 }
259 }
260 }
261 }
262 }
```

android/app/src/main/java/com/runway/android/core/tracking/RunTrackingMode.kt

Kotlin | 3줄

```
1 package com.runway.android.core.tracking
2
3 enum class RunTrackingMode { FREE_RUN, COURSE_ATTEMPT }
```

```

1  package com.runway.android.core.tracking
2
3  import android.app.Notification
4  import android.app.NotificationChannel
5  import android.app.NotificationManager
6  import android.app.PendingIntent
7  import android.content.Context
8  import android.content.Intent
9  import androidx.core.app.NotificationCompat
10 import com.runway.android.BuildConfig
11 import com.runway.android.MainActivity
12 import com.runway.android.R
13 import dagger.hilt.android.qualifiers.ApplicationContext
14 import javax.inject.Inject
15 import javax.inject.Singleton
16
17 @Singleton
18 class RunTrackingNotification @Inject constructor(
19     @ApplicationContext private val context: Context,
20 ) {
21     companion object {
22         const val NOTIFICATION_ID = 1001
23         const val CHANNEL_ID = "runway_tracking"
24     }
25
26     private val notificationManager =
27         context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
28
29     init {
30         val channel = NotificationChannel(
31             CHANNEL_ID,
32             "${BuildConfig.APP_NAME} Tracking",
33             NotificationManager.IMPORTANCE_LOW, // LOW = no sound, no vibration
34         ).apply {
35             description = "Shown while running or course attempt tracking is active"
36         }
37         notificationManager.createNotificationChannel(channel)
38     }
39
40     fun build(state: RunTrackingState): Notification {
41         val tapIntent = PendingIntent.getActivity(
42             context,
43             0,
44             Intent(context, MainActivity::class.java).apply {
45                 flags = Intent.FLAG_ACTIVITY_SINGLE_TOP
46             },
47             PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE,
48         )
49
50         val modeLabel = when (state.mode) {
51             RunTrackingMode.FREE_RUN -> "Running"
52             RunTrackingMode.COURSE_ATTEMPT -> "Course attempt"
53         }
54         val timeStr = formatElapsed(state.elapsedSeconds)
55         val distStr = "%.2f km".format(state.distanceMeters / 1000.0)
56
57         return NotificationCompat.Builder(context, CHANNEL_ID)
58             .setSmallIcon(R.drawable.ic_notification)
59             .setContentTitle("${BuildConfig.APP_NAME} tracking")
60             .setContentText("$modeLabel · $timeStr · $distStr")
61             .setContentIntent(tapIntent)
62             .setOngoing(true)
63             .setOnlyAlertOnce(true)
64             .setSilent(true)
65             .build()
66     }
67
68     fun update(state: RunTrackingState) {
69         notificationManager.notify(NOTIFICATION_ID, build(state))
70     }
71
72     private fun formatElapsed(seconds: Int): String {
73         val h = seconds / 3600
74         val m = (seconds % 3600) / 60
75         val s = seconds % 60
76         return if (h > 0) "%d:%02d:%02d".format(h, m, s) else "%02d:%02d".format(m, s)
77     }
78 }

```

```

1  package com.runway.android.core.tracking
2
3  import android.app.Service
4  import android.content.Intent
5  import android.content.pm.ServiceInfo
6  import android.os.Build
7  import android.os.IBinder
8  import androidx.core.app.ServiceCompat
9  import dagger.hilt.android.AndroidEntryPoint
10 import kotlinx.coroutines.CoroutineScope
11 import kotlinx.coroutines.Dispatchers
12 import kotlinx.coroutines.Job
13 import kotlinx.coroutines.SupervisorJob
14 import kotlinx.coroutines.cancel
15 import kotlinx.coroutines.delay
16 import kotlinx.coroutines.launch
17 import javax.inject.Inject
18
19 /**
20  * GPS 추적이 진행되는 동안 ForegroundService를 유지한다.
21  * - 실제 GPS 수집 및 상태 관리는 RunTrackingManager(appScope)에서 처리한다.
22  * - 이 서비스는 알림 표시와 프로세스 유지만 담당한다.
23  *
24  * NOTE: 에뮬레이터에서는 Extended Controls → Location에서 mock location을 전송해야
25  *       GPS 신호가 수신된다. 물리 기기에서는 실제 GPS가 사용된다.
26  */
27 @AndroidEntryPoint
28 class RunTrackingService : Service() {
29
30     @Inject lateinit var manager: RunTrackingManager
31     @Inject lateinit var notification: RunTrackingNotification
32
33     private val serviceScope = CoroutineScope(SupervisorJob() + Dispatchers.Main.immediate)
34     private var notificationUpdateJob: Job? = null
35
36     override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
37         when (intent?.action) {
38             RunTrackingAction.ACTION_START -> handleStart(intent)
39             RunTrackingAction.ACTION_STOP  -> handleStop()
40         }
41         return START_NOT_STICKY
42     }
43
44     private fun handleStart(intent: Intent) {
45         val mode = intent.getStringExtra(RunTrackingAction.EXTRA_MODE)
46         ?.let { runCatching { RunTrackingMode.valueOf(it) }.getOrNull() }
47         ?: RunTrackingMode.FREE_RUN
48
49         val initialNotification = notification.build(RunTrackingState(mode = mode))
50
51         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
52             ServiceCompat.startForeground(
53                 this,
54                 RunTrackingNotification.NOTIFICATION_ID,
55                 initialNotification,
56                 ServiceInfo.FOREGROUND_SERVICE_TYPE_LOCATION,
57             )
58         } else {
59             startForeground(RunTrackingNotification.NOTIFICATION_ID, initialNotification)
60         }
61
62         manager.start(mode)
63         startNotificationUpdates()
64     }
65
66     private fun handleStop() {
67         notificationUpdateJob?.cancel()
68         // manager.stop() is called by the ViewModel before sending ACTION_STOP,
69         // but we call it here too as a safety net.
70         manager.stop()
71         ServiceCompat.stopForeground(this, ServiceCompat.STOP_FOREGROUND_REMOVE)
72         stopSelf()
73     }
74
75     private fun startNotificationUpdates() {
76         notificationUpdateJob?.cancel()
77         notificationUpdateJob = serviceScope.launch {
78             while (true) {
79                 delay(5_000L)
80                 val state = manager.state.value
81                 if (state.isTracking) {
82                     notification.update(state)
83                 }
84             }
85         }
86     }
87
88 }

```

```

84         }
85     }
86 }
87
88 override fun onDestroy() {
89     super.onDestroy()
90     serviceScope.cancel()
91 }
92
93 override fun onBind(intent: Intent?): IBinder? = null
94 }

```

android/app/src/main/java/com/runway/android/core/tracking/RunTrackingState.kt

Kotlin | 18줄

```

1 package com.runway.android.core.tracking
2
3 import com.runway.android.core.location.RunwayLocation
4
5 data class RunTrackingState(
6     val mode: RunTrackingMode = RunTrackingMode.FREE_RUN,
7     val isTracking: Boolean = false,
8     val isPaused: Boolean = false,
9     val pauseReason: PauseReason = PauseReason.NONE,
10    val hasFirstFix: Boolean = false,
11    val elapsedSeconds: Int = 0,
12    val distanceMeters: Double = 0.0,
13    val currentSpeedMps: Float? = null,
14    val cadenceSpm: Int? = null,
15    val lastLocation: RunwayLocation? = null,
16 ) {
17     val isAutoPaused: Boolean get() = isPaused && pauseReason == PauseReason.AUTO
18 }

```

android/app/src/main/java/com/runway/android/core/tracking/TrackingSessionSnapshot.kt

Kotlin | 8줄

```

1 package com.runway.android.core.tracking
2
3 data class TrackingSessionSnapshot(
4     val runningRecordId: String, // running_records.id - used for GPS point upload
5     val courseAttemptId: String?, // null = free run, non-null = course attempt
6     val elapsedSeconds: Int,
7     val distanceMeters: Double,
8 )

```



```
1 package com.runway.android.core.tracking
2
3 import androidx.datastore.core.DataStore
4 import androidx.datastore.preferences.core.Preferences
5 import androidx.datastore.preferences.core.edit
6 import androidx.datastore.preferences.core.floatPreferencesKey
7 import androidx.datastore.preferences.core.intPreferencesKey
8 import androidx.datastore.preferences.core.stringPreferencesKey
9 import kotlinx.coroutines.flow.Flow
10 import kotlinx.coroutines.flow.map
11 import javax.inject.Inject
12 import javax.inject.Named
13 import javax.inject.Singleton
14
15 @Singleton
16 class TrackingSessionStore @Inject constructor(
17     @Named("trackingDataStore") private val datastore: DataStore<Preferences>,
18 ) {
19     companion object {
20         private val KEY_RUNNING_RECORD_ID = stringPreferencesKey("running_record_id")
21         private val KEY_COURSE_ATTEMPT_ID = stringPreferencesKey("course_attempt_id")
22         private val KEY_ELAPSED_SECONDS = intPreferencesKey("elapsed_seconds")
23         private val KEY_DISTANCE_METERS = floatPreferencesKey("distance_meters")
24     }
25
26     val snapshot: Flow<TrackingSessionSnapshot?> = datastore.data.map { prefs ->
27         val runningRecordId = prefs[KEY_RUNNING_RECORD_ID] ?: return@map null
28         TrackingSessionSnapshot(
29             runningRecordId = runningRecordId,
30             courseAttemptId = prefs[KEY_COURSE_ATTEMPT_ID],
31             elapsedSeconds = prefs[KEY_ELAPSED_SECONDS] ?: 0,
32             distanceMeters = (prefs[KEY_DISTANCE_METERS] ?: 0f).toDouble(),
33         )
34     }
35
36     suspend fun saveSnapshot(snapshot: TrackingSessionSnapshot) {
37         datastore.edit { prefs ->
38             prefs[KEY_RUNNING_RECORD_ID] = snapshot.runningRecordId
39             if (snapshot.courseAttemptId != null) {
40                 prefs[KEY_COURSE_ATTEMPT_ID] = snapshot.courseAttemptId
41             } else {
42                 prefs.remove(KEY_COURSE_ATTEMPT_ID)
43             }
44             prefs[KEY_ELAPSED_SECONDS] = snapshot.elapsedSeconds
45             prefs[KEY_DISTANCE_METERS] = snapshot.distanceMeters.toFloat()
46         }
47     }
48
49     suspend fun clearSnapshot() {
50         datastore.edit { prefs ->
51             prefs.remove(KEY_RUNNING_RECORD_ID)
52             prefs.remove(KEY_COURSE_ATTEMPT_ID)
53             prefs.remove(KEY_ELAPSED_SECONDS)
54             prefs.remove(KEY_DISTANCE_METERS)
55         }
56     }
57 }
```

android/app/src/main/java/com/runway/android/core/tracking/local/PendingRunPointDao.kt

Kotlin | 22줄

```
1 package com.runway.android.core.tracking.local
2
3 import androidx.room.Dao
4 import androidx.room.Insert
5 import androidx.room.OnConflictStrategy
6 import androidx.room.Query
7
8 @Dao
9 interface PendingRunPointDao {
10
11     @Insert(onConflict = OnConflictStrategy.REPLACE)
12     suspend fun insertAll(points: List<PendingRunPointEntity>)
13
14     @Query("SELECT * FROM pending_run_points WHERE runningRecordId = :runningRecordId ORDER BY sequence ASC LIMIT :limit")
15     suspend fun dequeue(runningRecordId: String, limit: Int): List<PendingRunPointEntity>
16
17     @Query("DELETE FROM pending_run_points WHERE id IN (:ids)")
18     suspend fun deleteByIds(ids: List<Long>)
19
20     @Query("DELETE FROM pending_run_points WHERE runningRecordId = :runningRecordId")
21     suspend fun deleteByRunningRecordId(runningRecordId: String)
22 }
```

android/app/src/main/java/com/runway/android/core/tracking/local/PendingRunPointEntity.kt

Kotlin | 20줄

```
1 package com.runway.android.core.tracking.local
2
3 import androidx.room.Entity
4 import androidx.room.Index
5 import androidx.room.PrimaryKey
6
7 @Entity(
8     tableName = "pending_run_points",
9     indices = [Index(value = ["runningRecordId", "sequence"])],
10 )
11 data class PendingRunPointEntity(
12     @PrimaryKey(autoGenerate = true) val id: Long = 0,
13     val runningRecordId: String,
14     val sequence: Int,
15     val latitude: Double,
16     val longitude: Double,
17     val altitudeMeters: Double,
18     val speedMps: Double,
19     val recordedAt: String,
20 )
```

android/app/src/main/java/com/runway/android/core/tracking/local/TrackingDatabase.kt

Kotlin | 13줄

```
1 package com.runway.android.core.tracking.local
2
3 import androidx.room.Database
4 import androidx.room.RoomDatabase
5
6 @Database(
7     entities = [PendingRunPointEntity::class],
8     version = 2,
9     exportSchema = false,
10 )
11 abstract class TrackingDatabase : RoomDatabase() {
12     abstract fun pendingRunPointDao(): PendingRunPointDao
13 }
```

```
1 package com.runway.android.core.util
2
3 import java.time.Instant
4 import java.time.ZoneId
5 import java.time.format.TextStyle
6 import java.util.Locale
7
8 fun formatDistance(meters: Double?): String {
9     if (meters == null) return "--"
10    return if (meters < 1000.0) "${meters.toInt()} m"
11    else "%.2f km".format(meters / 1000.0)
12 }
13
14 fun formatDuration(seconds: Int?): String {
15     if (seconds == null || seconds < 0) return "---"
16     val h = seconds / 3600
17     val m = (seconds % 3600) / 60
18     val s = seconds % 60
19     return if (h > 0) "%02d:%02d:%02d".format(h, m, s)
20     else "%02d:%02d".format(m, s)
21 }
22
23 fun formatPace(secsPerKm: Int?): String {
24     if (secsPerKm == null || secsPerKm <= 0) return "---" /km"
25     return "%d'%02d\" /km".format(secsPerKm / 60, secsPerKm % 60)
26 }
27
28 fun formatRunDateShort(isoString: String?): String {
29     if (isoString == null) return ""
30     return runCatching {
31         val dt = Instant.parse(isoString).atZone(ZoneId.systemDefault())
32         val month = dt.month.getDisplayName(TextStyle.SHORT, Locale.ENGLISH)
33         "$month ${dt.dayOfMonth}, ${dt.year}"
34     }.getOrElse("")
35 }
36
37 fun formatRunDateFull(isoString: String?): String {
38     if (isoString == null) return ""
39     return runCatching {
40         val dt = Instant.parse(isoString).atZone(ZoneId.systemDefault())
41         val month = dt.month.getDisplayName(TextStyle.FULL, Locale.ENGLISH)
42         "$month ${dt.dayOfMonth}, ${dt.year}"
43     }.getOrElse("")
44 }
45
46 fun formatTime(isoString: String?): String {
47     if (isoString == null) return ""
48     return runCatching {
49         val dt = Instant.parse(isoString).atZone(ZoneId.systemDefault())
50         "%02d:%02d".format(dt.hour, dt.minute)
51     }.getOrElse("")
52 }
```

```

1 package com.runway.android.core.voice
2
3 import android.content.Context
4 import android.speech.tts.TextToSpeech
5 import android.speech.tts.Voice
6 import dagger.hilt.android.qualifiers.ApplicationContext
7 import java.util.Locale
8 import java.util.concurrent.ConcurrentLinkedQueue
9 import javax.inject.Inject
10 import javax.inject.Singleton
11
12 @Singleton
13 class RunningVoiceGuide @Inject constructor(
14     @ApplicationContext context: Context,
15 ) : TextToSpeech.OnInitListener {
16     private val pending = ConcurrentLinkedQueue<String>()
17     private val textToSpeech = TextToSpeech(
18         context.applicationContext,
19         this,
20         GOOGLE_TTS_PACKAGE,
21     )
22     @Volatile private var ready = false
23
24     override fun onInit(status: Int) {
25         if (status != TextToSpeech.SUCCESS) return
26         textToSpeech.language = Locale.KOREA
27         textToSpeech.setSpeechRate(1.08f)
28         textToSpeech.setPitch(1.08f)
29
30         selectKoreanFemaleVoice(textToSpeech.voices)?.let { textToSpeech.voice = it }
31
32         ready = true
33         while (true) {
34             val message = pending.poll() ?: break
35             speak(message)
36         }
37     }
38
39     fun speak(message: String) {
40         if (message.isBlank()) return
41         if (!ready) {
42             pending.offer(message)
43             return
44         }
45         textToSpeech.speak(
46             message,
47             TextToSpeech.QUEUE_ADD,
48             null,
49             "pathfinder-phone- $\{System.nanoTime()\}$ ",
50         )
51     }
52
53     fun start() = speak("러닝을 시작합니다. 오늘도 힘차게 달려볼까요?")
54
55     fun finish() = speak("러닝을 종료합니다. 수고하셨습니다.")
56
57     fun kilometer(kilometers: Int, elapsedSeconds: Int) {
58         speak(RunningVoiceText.kilometer(kilometers, elapsedSeconds))
59     }
60
61     fun autoPaused() = speak("움직임이 멈춰 자동으로 일시정지합니다.")
62
63     fun autoResumed() = speak("움직임이 감지되어 러닝을 다시 시작합니다.")
64
65     fun offCourse() = speak("코스를 이탈했습니다.")
66
67     fun backOnCourse() = speak("코스로 복귀했습니다.")
68
69     fun nearCourseFinish() = speak("거의 다 왔습니다.")
70
71     private fun selectKoreanFemaleVoice(voices: Set<Voice>?): Voice? {
72         val korean = voices.orEmpty().filter { it.locale.language == Locale.KOREAN.language }
73         val offline = korean.filter { !it.isNetworkConnectionRequired }
74         return offline.firstOrNull { isFemale(it.name) }
75             ?: offline.firstOrNull { !isMale(it.name) }
76             ?: offline.firstOrNull()
77             ?: korean.firstOrNull { isFemale(it.name) }
78             ?: korean.firstOrNull()
79     }
80
81     private fun isFemale(name: String) =
82         name.contains("female", ignoreCase = true) ||
83         name.contains("SMTf", ignoreCase = false) ||

```

```

84     FEMALE_PATTERN.containsMatchIn(name)
85
86     private fun isMale(name: String) =
87         name.contains("male", ignoreCase = true) ||
88         name.contains("SMTm", ignoreCase = false) ||
89         MALE_PATTERN.containsMatchIn(name)
90
91     private companion object {
92         const val GOOGLE_TTS_PACKAGE = "com.google.android.tts"
93         val FEMALE_PATTERN = Regex("[^a-zA-Z]f\\d")
94         val MALE_PATTERN = Regex("[^a-zA-Z]m\\d")
95     }
96 }
97
98 internal object RunningVoiceText {
99     fun kilometer(kilometers: Int, elapsedSeconds: Int): String {
100         val averagePace = if (kilometers > 0) elapsedSeconds / kilometers else 0
101         return "${kilometers}킬로미터 완료. 시간 ${duration(elapsedSeconds)}. " +
102             "평균 페이스 ${pace(averagePace)}."
103     }
104
105     private fun duration(totalSeconds: Int): String {
106         val hours = totalSeconds / 3600
107         val minutes = totalSeconds % 3600 / 60
108         val seconds = totalSeconds % 60
109         return buildList {
110             if (hours > 0) add("${hours}시간")
111             if (minutes > 0) add("${minutes}분")
112             if (seconds > 0 || isEmpty()) add("${seconds}초")
113         }.joinToString(" ")
114     }
115
116     private fun pace(secondsPerKilometer: Int): String =
117         "${secondsPerKilometer / 60}분 ${secondsPerKilometer % 60}초"
118 }

```

```

1  package com.runway.android.core.wear
2
3  import com.google.gson.JsonParser
4
5  sealed interface WatchCommand {
6      data object StartFreeRun : WatchCommand
7      data class StartTimeGoalRun(
8          val targetMinutes: Int,
9          val completionAction: WatchGoalCompletionAction,
10     ) : WatchCommand
11     data class StartDistanceGoalRun(
12         val targetMeters: Int,
13         val completionAction: WatchGoalCompletionAction,
14     ) : WatchCommand
15     data class StartIntervalRun(
16         val work: WatchIntervalTarget,
17         val recovery: WatchIntervalTarget,
18         val sets: Int,
19         val completionAction: WatchGoalCompletionAction,
20     ) : WatchCommand
21     data object PauseRun : WatchCommand
22     data object ResumeRun : WatchCommand
23     data object FinishRun : WatchCommand
24     data object AbandonRun : WatchCommand
25 }
26
27 object WatchCommandParser {
28     fun parse(payload: ByteArray): WatchCommand? = runCatching {
29         val json = JsonParser.parseString(payload.decodeToString()).asJsonObject
30         when (json.get("type").asString) {
31             "START_FREE_RUN" -> WatchCommand.StartFreeRun
32             "START_TIME_GOAL_RUN" -> WatchCommand.StartTimeGoalRun(
33                 targetMinutes = json.get("targetMinutes").asInt.coerceIn(1, 360),
34                 completionAction = json.completionAction(),
35             )
36             "START_DISTANCE_GOAL_RUN" -> WatchCommand.StartDistanceGoalRun(
37                 targetMeters = json.get("targetMeters").asInt.coerceIn(500, 50_000),
38                 completionAction = json.completionAction(),
39             )
40             "START_INTERVAL_RUN" -> WatchCommand.StartIntervalRun(
41                 work = json.intervalTarget("work", legacySecondsKey = "workSeconds"),
42                 recovery = json.intervalTarget("recovery", legacySecondsKey = "restSeconds"),
43                 sets = json.get("sets").asInt.coerceIn(1, 20),
44                 completionAction = json.completionAction(),
45             )
46             "PAUSE_RUN" -> WatchCommand.PauseRun
47             "RESUME_RUN" -> WatchCommand.ResumeRun
48             "FINISH_RUN" -> WatchCommand.FinishRun
49             "ABANDON_RUN" -> WatchCommand.AbandonRun
50             else -> null
51         }
52     }.getOrNull()
53
54     private fun com.google.gson.JsonObject.completionAction(): WatchGoalCompletionAction =
55         get("completionAction")?.asString
56             ?.let { runCatching { WatchGoalCompletionAction.valueOf(it) }.getOrNull() }
57             ?: WatchGoalCompletionAction.CONTINUE
58
59     private fun com.google.gson.JsonObject.intervalTarget(
60         prefix: String,
61         legacySecondsKey: String,
62     ): WatchIntervalTarget {
63         return when (get("${prefix}Type")?.asString) {
64             "DISTANCE" -> WatchIntervalTarget.Distance(
65                 get("${prefix}Meters").asInt.coerceIn(100, 50_000),
66             )
67             else -> WatchIntervalTarget.Time(
68                 (get("${prefix}Seconds") ?: get(legacySecondsKey)).asInt.coerceIn(10, 3_600),
69             )
70         }
71     }
72 }

```

```

1  package com.runway.android.core.wear
2
3  import android.content.Intent
4  import com.google.android.gms.wearable.MessageEvent
5  import com.google.android.gms.wearable.DataEvent
6  import com.google.android.gms.wearable.DataEventBuffer
7  import com.google.android.gms.wearable.DataMapItem
8  import com.google.android.gms.wearable.Wearable
9  import com.google.android.gms.wearable.WearableListenerService
10 import com.runway.android.MainActivity
11 import com.runway.android.core.datastore.TokenDataStore
12 import dagger.hilt.android.AndroidEntryPoint
13 import kotlinx.coroutines.CoroutineScope
14 import kotlinx.coroutines.Dispatchers
15 import kotlinx.coroutines.SupervisorJob
16 import kotlinx.coroutines.flow.first
17 import kotlinx.coroutines.launch
18 import kotlinx.coroutines.tasks.await
19 import org.json.JSONObject
20 import javax.inject.Inject
21
22 @AndroidEntryPoint
23 class WatchCommandListenerService : WearableListenerService() {
24     companion object {
25         const val COMMAND_PATH = "/runway/watch/command"
26         const val AUTH_REQUEST_PATH = "/runway/watch/auth/request"
27         const val AUTH_STATE_PATH = "/runway/watch/auth/state"
28         const val RUN_UPLOAD_PATH = "/runway/watch/run-upload"
29         const val RUN_UPLOAD_ACK_PATH = "/runway/watch/run-upload-ack"
30         const val COURSE_REQUEST_PATH = "/runway/watch/course-request"
31         const val COURSE_CATALOG_PATH = "/runway/watch/course-catalog"
32         const val OPEN_COURSE_PATH = "/runway/watch/open-course"
33         const val OPEN_RUN_PATH = "/runway/watch/open-run"
34         const val OPEN_APP_PATH = "/runway/watch/open-app"
35     }
36
37     @Inject lateinit var coordinator: WearRunSessionCoordinator
38     @Inject lateinit var tokenDataStore: TokenDataStore
39     @Inject lateinit var uploadCoordinator: WatchRunUploadCoordinator
40     @Inject lateinit var courseSyncCoordinator: WatchCourseSyncCoordinator
41     @Inject lateinit var watchNavRepository: WatchNavRepository
42     private val serviceScope = CoroutineScope(SupervisorJob() + Dispatchers.IO)
43
44     override fun onMessageReceived(messageEvent: MessageEvent) {
45         when (messageEvent.path) {
46             COMMAND_PATH -> {
47                 WatchCommandParser.parse(messageEvent.data)?.let { coordinator::handle }
48             }
49             AUTH_REQUEST_PATH -> sendAuthState(messageEvent.sourceNodeId)
50             COURSE_REQUEST_PATH -> {
51                 val payload = runCatching {
52                     JSONObject(messageEvent.data.decodeToString())
53                 }.getOrNull() ?: return
54                 serviceScope.launch {
55                     courseSyncCoordinator.syncNearby(
56                         latitude = payload.getDouble("latitude"),
57                         longitude = payload.getDouble("longitude"),
58                     )
59                 }
60             }
61             OPEN_COURSE_PATH -> {
62                 val courseId = messageEvent.data.decodeToString()
63                 if (courseId.isNotBlank()) {
64                     watchNavRepository.requestOpenCourse(courseId)
65                     startActivity(
66                         Intent(this, MainActivity::class.java).apply {
67                             flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_SINGLE_TOP
68                         }
69                     )
70                 }
71             }
72             OPEN_RUN_PATH -> {
73                 val runId = messageEvent.data.decodeToString()
74                 if (runId.isNotBlank()) {
75                     watchNavRepository.requestOpenRun(runId)
76                     startActivity(
77                         Intent(this, MainActivity::class.java).apply {
78                             flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_SINGLE_TOP
79                         }
80                     )
81                 }
82             }
83             OPEN_APP_PATH -> {
84                 startActivity(

```

```

84         Intent(this, MainActivity::class.java).apply {
85             flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_SINGLE_TOP
86         }
87     )
88 }
89 }
90 }
91
92 override fun onDataChange(dataEvents: DataEventBuffer) {
93     dataEvents
94         .filter { it.type == DataEvent.TYPE_CHANGED }
95         .filter { it.dataItem.uri.path?.startsWith("$RUN_UPLOAD_PATH/") == true }
96         .forEach { event ->
97             val uri = event.dataItem.uri
98             val asset = DataMapItem.fromDataItem(event.dataItem).dataMap.getAsset("run")
99             ?: return@forEach
100             serviceScope.launch {
101                 val descriptor = runCatching {
102                     Wearable.getDataClient(this@WatchCommandListenerService)
103                         .getFdForAsset(asset)
104                         .await()
105                 }.getOrNull() ?: return@launch
106                 descriptor.inputStream.use { input ->
107                     val payload = runCatching {
108                         JSONObject(input.readBytes().decodeToString())
109                     }.getOrNull() ?: return@launch
110                     uploadCoordinator.upload(payload, uri)
111                 }
112             }
113         }
114 }
115
116 private fun sendAuthState(nodeId: String) {
117     serviceScope.launch {
118         val payload = JSONObject()
119             .put("isLoggedIn", tokenDataStore.accessTokenFlow.first() != null)
120             .toString()
121             .encodeToByteArray()
122         runCatching {
123             Wearable.getMessageClient(this@WatchCommandListenerService)
124                 .sendMessage(nodeId, AUTH_STATE_PATH, payload)
125                 .await()
126         }
127     }
128 }
129 }

```



```

1 package com.runway.android.core.wear
2
3 import android.content.Context
4 import com.google.android.gms.wearable.Asset
5 import com.google.android.gms.wearable.PutDataMapRequest
6 import com.google.android.gms.wearable.Wearable
7 import com.runway.android.core.result.NetworkResult
8 import com.runway.android.domain.course.CourseRepository
9 import dagger.hilt.android.qualifiers.ApplicationContext
10 import kotlinx.coroutines.tasks.await
11 import org.json.JSONArray
12 import org.json.JSONObject
13 import javax.inject.Inject
14 import javax.inject.Singleton
15
16 @Singleton
17 class WatchCourseSyncCoordinator @Inject constructor(
18     @ApplicationContext context: Context,
19     private val courseRepository: CourseRepository,
20 ) {
21     private val dataClient = Wearable.getDataClient(context)
22
23     suspend fun syncNearby(latitude: Double, longitude: Double) {
24         // 1. 주변 공개 코스 (route points 포함)
25         val nearbyResult = courseRepository.getNearbyCourses(
26             latitude = latitude,
27             longitude = longitude,
28             radiusMeters = SEARCH_RADIUS_METERS,
29             includeRoutePoints = true,
30             includeExactStartPoint = true,
31             size = MAX_COURSES,
32         )
33         val nearbyCourses = if (nearbyResult is NetworkResult.Success) {
34             nearbyResult.data.content.filter { it.routePoints.size >= 2 }
35         } else emptyList()
36         val nearbyIds = nearbyCourses.map { it.courseId }.toSet()
37
38         // 2. 내 코스 (draft + published, route points 별도 조회)
39         val myCoursesResult = courseRepository.getMyCourses(size = MAX_MY_COURSES)
40         val myCourseItems = if (myCoursesResult is NetworkResult.Success) {
41             myCoursesResult.data.content.filter { it.courseId !in nearbyIds }
42         } else emptyList()
43
44         // 3. 내 코스 포인트 조회 (병렬 아닌 순차 - 부하 최소화)
45         val myCoursesWithPoints = myCourseItems.mapNotNull { course ->
46             val pointsResult = courseRepository.getCoursePoints(course.courseId)
47             if (pointsResult is NetworkResult.Success && pointsResult.data.points.size >= 2) {
48                 Pair(course, pointsResult.data.points)
49             } else null
50         }
51
52         // 4. 합쳐서 JSON 직렬화
53         val allCourses = JSONArray().apply {
54             // 주변 공개 코스
55             nearbyCourses.take(MAX_COURSES).forEach { course ->
56                 put(buildCourseJson(
57                     courseId = course.courseId,
58                     name = course.name,
59                     description = course.description,
60                     distanceMeters = course.distanceMeters,
61                     distanceFromMeMeters = course.distanceFromMeMeters,
62                     isLoop = course.isLoop,
63                     isPublic = true,
64                     points = course.routePoints.map { Triple(it.latitude, it.longitude, 0.0) },
65                 ))
66             }
67             // 내 코스 (개인/공개 모두)
68             myCoursesWithPoints.take(MAX_MY_COURSES).forEach { (course, points) ->
69                 put(buildCourseJson(
70                     courseId = course.courseId,
71                     name = course.name,
72                     description = course.description,
73                     distanceMeters = course.distanceMeters,
74                     distanceFromMeMeters = 0.0,
75                     isLoop = course.isLoop,
76                     isPublic = course.status == "published",
77                     points = points.map { Triple(it.latitude, it.longitude, 0.0) },
78                 ))
79             }
80         }
81
82         val request = PutDataMapRequest.create(WatchCommandListenerService.COURSE_CATALOG_PATH).apply {
83             dataMap.putLong("updatedAt", System.currentTimeMillis())

```

```

84         dataMap.putAsset("courses", Asset.createFromBytes(allCourses.toString().encodeToByteArray()))
85     }.asPutDataRequest().setUrgent()
86     dataClient.putDataItem(request).await()
87 }
88
89 private fun buildCourseJson(
90     courseId: String,
91     name: String,
92     description: String?,
93     distanceMeters: Double,
94     distanceFromMeMeters: Double,
95     isLoop: Boolean,
96     isPublic: Boolean,
97     points: List<Triple<Double, Double, Double>>,
98 ) = JSONObject()
99     .put("courseId", courseId)
100     .put("name", name)
101     .put("description", description)
102     .put("distanceMeters", distanceMeters)
103     .put("distanceFromMeMeters", distanceFromMeMeters)
104     .put("isLoop", isLoop)
105     .put("isPublic", isPublic)
106     .put("points", JSONArray().apply {
107         points.forEach { (lat, lng, alt) ->
108             put(JSONObject().put("latitude", lat).put("longitude", lng).put("altitudeMeters", alt))
109         }
110     })
111
112 private companion object {
113     const val SEARCH_RADIUS_METERS = 5_000
114     const val MAX_COURSES = 10
115     const val MAX_MY_COURSES = 5
116 }
117 }

```

android/app/src/main/java/com/runway/android/core/wear/WatchNavRepository.kt

Kotlin | 34줄

```

1  package com.runway.android.core.wear
2
3  import kotlinx.coroutines.flow.MutableStateFlow
4  import kotlinx.coroutines.flow.asStateFlow
5  import javax.inject.Inject
6  import javax.inject.Singleton
7
8  @Singleton
9  class WatchNavRepository @Inject constructor() {
10     private val _pendingCourseId = MutableStateFlow<String?>(null)
11     val pendingCourseId = _pendingCourseId.asStateFlow()
12
13     private val _pendingRunId = MutableStateFlow<String?>(null)
14     val pendingRunId = _pendingRunId.asStateFlow()
15
16     fun requestOpenCourse(courseId: String) {
17         _pendingCourseId.value = courseId
18     }
19
20     fun consumeCourse() {
21         _pendingCourseId.value = null
22     }
23
24     fun requestOpenRun(runId: String) {
25         _pendingRunId.value = runId
26     }
27
28     fun consumeRun() {
29         _pendingRunId.value = null
30     }
31
32     @Deprecated("use consumeCourse or consumeRun")
33     fun consume() = consumeCourse()
34 }

```

```
1 package com.runway.android.core.wear
2
3 enum class WatchGoalCompletionAction {
4     PAUSE,
5     CONTINUE,
6 }
7
8 sealed interface WatchIntervalTarget {
9     data class Time(val seconds: Int) : WatchIntervalTarget
10    data class Distance(val meters: Int) : WatchIntervalTarget
11 }
12
13 sealed interface WatchRunGoal {
14     data object Free : WatchRunGoal
15     data class Time(
16         val targetMinutes: Int,
17         val completionAction: WatchGoalCompletionAction,
18     ) : WatchRunGoal
19     data class Distance(
20         val targetMeters: Int,
21         val completionAction: WatchGoalCompletionAction,
22     ) : WatchRunGoal
23     data class Interval(
24         val work: WatchIntervalTarget,
25         val recovery: WatchIntervalTarget,
26         val sets: Int,
27         val completionAction: WatchGoalCompletionAction,
28     ) : WatchRunGoal
29 }
```

```

1  package com.runway.android.core.wear
2
3  import android.content.Context
4  import android.net.Uri
5  import com.google.android.gms.wearable.Wearable
6  import com.runway.android.core.datastore.TokenDataStore
7  import com.runway.android.core.result.NetworkResult
8  import com.runway.android.data.running.model.FinishRunRequest
9  import com.runway.android.data.running.model.RunPointRequest
10 import com.runway.android.data.running.model.SavePointsRequest
11 import com.runway.android.data.running.model.StartRunRequest
12 import com.runway.android.data.attempt.model.FinishAttemptRequest
13 import com.runway.android.data.attempt.model.StartAttemptRequest
14 import com.runway.android.domain.attempt.CourseAttemptRepository
15 import com.runway.android.domain.running.RunningRepository
16 import dagger.hilt.android.qualifiers.ApplicationContext
17 import kotlinx.coroutines.flow.first
18 import kotlinx.coroutines.tasks.await
19 import org.json.JSONObject
20 import javax.inject.Inject
21 import javax.inject.Singleton
22
23 @Singleton
24 class WatchRunUploadCoordinator @Inject constructor(
25     @ApplicationContext private val context: Context,
26     private val tokenDataStore: TokenDataStore,
27     private val runningRepository: RunningRepository,
28     private val courseAttemptRepository: CourseAttemptRepository,
29 ) {
30     private val preferences = context.getSharedPreferences(
31         "watch_run_uploads",
32         Context.MODE_PRIVATE,
33     )
34
35     suspend fun upload(payload: JSONObject, dataItemUri: Uri) {
36         if (tokenDataStore.accessTokenFlow.first() == null) return
37
38         val localId = payload.getString("localId")
39         if (preferences.getBoolean(completedKey(localId), false)) {
40             acknowledge(localId, null, dataItemUri)
41             return
42         }
43
44         val courseId = payload.optString("courseId").takeUnless { it.isBlank() || it == "null" }
45         val runId = preferences.getString(runIdKey(localId), null) ?: run {
46             if (courseId != null) {
47                 when (val result = courseAttemptRepository.startAttempt(
48                     courseId,
49                     StartAttemptRequest(payload.getString("startedAt")),
50                 )) {
51                     is NetworkResult.Success -> {
52                         preferences.edit()
53                             .putString(runIdKey(localId), result.data.runningRecordId)
54                             .putString(attemptIdKey(localId), result.data.courseAttemptId)
55                             .commit()
56                         result.data.runningRecordId
57                     }
58                     else -> return
59                 }
60             } else {
61                 when (val result = runningRepository.startRun(
62                     StartRunRequest(payload.getString("startedAt")),
63                 )) {
64                     is NetworkResult.Success -> result.data.runId.also {
65                         preferences.edit().putString(runIdKey(localId), it).commit()
66                     }
67                     else -> return
68                 }
69             }
70         }
71
72         val pointsJson = payload.optJSONArray("points")
73         var offset = preferences.getInt(offsetKey(localId), 0)
74         while (pointsJson != null && offset < pointsJson.length()) {
75             val end = (offset + POINT_BATCH_SIZE).coerceAtMost(pointsJson.length())
76             val points = (offset until end).map { index ->
77                 val point = pointsJson.getJSONObject(index)
78                 RunPointRequest(
79                     sequence = point.getInt("sequence"),
80                     latitude = point.getDouble("latitude"),
81                     longitude = point.getDouble("longitude"),
82                     altitudeMeters = point.getDouble("altitudeMeters"),
83                     speedMps = point.getDouble("speedMps"),

```

```

84         recordedAt = point.getString("recordedAt"),
85     )
86 }
87 if (runningRepository.savePoints(runId, SavePointsRequest(points)) !is NetworkResult.Success) {
88     return
89 }
90 offset = end
91 preferences.edit().putInt(offsetKey(localId), offset).commit()
92 }
93
94 val endedAt = payload.getString("endedAt")
95 val distanceMeters = payload.getDouble("distanceMeters")
96 val durationSeconds = payload.getInt("durationSeconds")
97 val avgPace = payload.getInt("avgPaceSecondsPerKm")
98 val calories = payload.getInt("caloriesBurned")
99 val avgHeartRate = payload.optInt("avgHeartRateBpm").takeIf {
100     payload.has("avgHeartRateBpm") && !payload.isNull("avgHeartRateBpm")
101 }
102 val finishResult = if (courseId != null) {
103     val attemptId = preferences.getString(attemptIdKey(localId), null) ?: return
104     courseAttemptRepository.finishAttempt(
105         attemptId,
106         FinishAttemptRequest(
107             endedAt = endedAt,
108             distanceMeters = distanceMeters,
109             durationSeconds = durationSeconds,
110             avgPaceSecondsPerKm = avgPace,
111             caloriesBurned = calories,
112             avgHeartRateBpm = avgHeartRate,
113         ),
114     )
115 } else {
116     runningRepository.finishRun(
117         runId,
118         FinishRunRequest(
119             endedAt = endedAt,
120             distanceMeters = distanceMeters,
121             durationSeconds = durationSeconds,
122             avgPaceSecondsPerKm = avgPace,
123             caloriesBurned = calories,
124             avgHeartRateBpm = avgHeartRate,
125         ),
126     )
127 }
128 if (finishResult !is NetworkResult.Success) return
129
130 preferences.edit()
131     .putBoolean(completedKey(localId), true)
132     .remove(runIdKey(localId))
133     .remove(offsetKey(localId))
134     .remove(attemptIdKey(localId))
135     .commit()
136 // courseAttempt run은 코스 생성 대상이 아니므로 runId만 전달
137 acknowledge(localId, if (courseId == null) runId else null, dataItemUri)
138 }
139
140 private suspend fun acknowledge(localId: String, runId: String?, dataItemUri: Uri) {
141     Wearable.getDataClient(context).deleteDataItems(dataItemUri).await()
142     val payload = JSONObject()
143         .put("localId", localId)
144         .apply { runId?.let { put("runId", it) } }
145         .toString()
146         .encodeToByteArray()
147     val nodes = Wearable.getNodeClient(context).connectedNodes.await()
148     nodes.forEach { node ->
149         Wearable.getMessageClient(context)
150             .sendMessage(node.id, WatchCommandListenerService.RUN_UPLOAD_ACK_PATH, payload)
151             .await()
152     }
153 }
154
155 private fun runIdKey(localId: String) = "run_id_$localId"
156 private fun offsetKey(localId: String) = "offset_$localId"
157 private fun attemptIdKey(localId: String) = "attempt_id_$localId"
158 private fun completedKey(localId: String) = "completed_$localId"
159
160 private companion object {
161     const val POINT_BATCH_SIZE = 50
162 }
163 }

```

```

1 package com.runway.android.core.wear
2
3 import android.content.Context
4 import com.google.android.gms.wearable.Wearable
5 import com.runway.android.core.tracking.RunTrackingState
6 import dagger.hilt.android.qualifiers.ApplicationContext
7 import kotlinx.coroutines.tasks.await
8 import org.json.JSONObject
9 import javax.inject.Inject
10 import javax.inject.Singleton
11
12 @Singleton
13 class WatchStateSender @Inject constructor(
14     @ApplicationContext context: Context,
15 ) {
16     companion object {
17         const val STATE_PATH = "/runway/watch/state"
18     }
19
20     private val nodeClient = Wearable.getNodeClient(context)
21     private val messageClient = Wearable.getMessageClient(context)
22
23     suspend fun sendState(
24         state: RunTrackingState,
25         runId: String?,
26         status: String,
27         goal: WatchRunGoal?,
28         error: String? = null,
29     ) {
30         val payload = JSONObject()
31         .put("status", status)
32         .put("runId", runId)
33         .put("elapsedSeconds", state.elapsedSeconds)
34         .put("distanceMeters", state.distanceMeters)
35         .put("paceMinPerKm", calculatePace(state))
36         .put("cadenceSpm", state.cadenceSpm)
37         .put("hasGpsFix", state.hasFirstFix)
38         .put("isPaused", state.isPaused)
39         .put("error", error)
40         .apply { putGoal(goal) }
41         .toString()
42         .encodeToByteArray()
43
44         runCatching {
45             nodeClient.connectedNodes.await().forEach { node ->
46                 messageClient.sendMessage(node.id, STATE_PATH, payload).await()
47             }
48         }
49     }
50
51     private fun calculatePace(state: RunTrackingState): Double {
52         if (state.distanceMeters < 20.0 || state.elapsedSeconds <= 0) return 0.0
53         return state.elapsedSeconds / 60.0 / (state.distanceMeters / 1000.0)
54     }
55
56     private fun JSONObject.putGoal(goal: WatchRunGoal?) {
57         when (goal) {
58             null, WatchRunGoal.Free -> put("goalType", "FREE")
59             is WatchRunGoal.Time -> {
60                 put("goalType", "TIME")
61                 put("targetMinutes", goal.targetMinutes)
62                 put("completionAction", goal.completionAction.name)
63             }
64             is WatchRunGoal.Distance -> {
65                 put("goalType", "DISTANCE")
66                 put("targetMeters", goal.targetMeters)
67                 put("completionAction", goal.completionAction.name)
68             }
69             is WatchRunGoal.Interval -> {
70                 put("goalType", "INTERVAL")
71                 put("sets", goal.sets)
72                 put("completionAction", goal.completionAction.name)
73                 putTarget("work", goal.work)
74                 putTarget("recovery", goal.recovery)
75             }
76         }
77     }
78
79     private fun JSONObject.putTarget(prefix: String, target: WatchIntervalTarget) {
80         when (target) {
81             is WatchIntervalTarget.Time -> {
82                 put("${prefix}Type", "TIME")
83                 put("${prefix}Seconds", target.seconds)

```

```
84     }
85     is WatchIntervalTarget.Distance -> {
86         put("${prefix}Type", "DISTANCE")
87         put("${prefix}Meters", target.meters)
88     }
89 }
90 }
91 }
```

```

1  package com.runway.android.core.wear
2
3  import android.Manifest
4  import android.content.Context
5  import android.content.Intent
6  import android.content.pm.PackageManager
7  import androidx.core.content.ContextCompat
8  import com.runway.android.core.result.NetworkResult
9  import com.runway.android.core.tracking.PendingPointQueue
10 import com.runway.android.core.voice.RunningVoiceGuide
11 import com.runway.android.core.tracking.RunTrackingAction
12 import com.runway.android.core.tracking.RunTrackingManager
13 import com.runway.android.core.tracking.RunTrackingMode
14 import com.runway.android.core.tracking.RunTrackingService
15 import com.runway.android.core.tracking.TrackingSessionSnapshot
16 import com.runway.android.core.tracking.TrackingSessionStore
17 import com.runway.android.core.tracking.toRunPointRequest
18 import com.runway.android.data.running.model.FinishRunRequest
19 import com.runway.android.data.running.model.SavePointsRequest
20 import com.runway.android.data.running.model.StartRunRequest
21 import com.runway.android.domain.running.RunningRepository
22 import dagger.hilt.android.qualifiers.ApplicationContext
23 import kotlinx.coroutines.CoroutineScope
24 import kotlinx.coroutines.Job
25 import kotlinx.coroutines.delay
26 import kotlinx.coroutines.flow.collectLatest
27 import kotlinx.coroutines.launch
28 import kotlinx.coroutines.sync.Mutex
29 import kotlinx.coroutines.sync.withLock
30 import java.time.Instant
31 import javax.inject.Inject
32 import javax.inject.Named
33 import javax.inject.Singleton
34
35 @Singleton
36 class WearRunSessionCoordinator @Inject constructor(
37     @ApplicationContext private val context: Context,
38     private val runningRepository: RunningRepository,
39     private val manager: RunTrackingManager,
40     private val pendingPointQueue: PendingPointQueue,
41     private val sessionStore: TrackingSessionStore,
42     private val stateSender: WatchStateSender,
43     private val voiceGuide: RunningVoiceGuide,
44     @Named("appScope") private val appScope: CoroutineScope,
45 ) {
46     private val commandMutex = Mutex()
47     private var runId: String? = null
48     private var goal: WatchRunGoal? = null
49     private var batchJob: Job? = null
50     private var stateJob: Job? = null
51     private var prevAutoPaused = false
52     private var lastAnnouncedKm = 0
53
54     fun handle(command: WatchCommand) {
55         appScope.launch {
56             commandMutex.withLock {
57                 when (command) {
58                     WatchCommand.StartFreeRun -> start(WatchRunGoal.Free)
59                     is WatchCommand.StartTimeGoalRun ->
60                         start(WatchRunGoal.Time(command.targetMinutes, command.completionAction))
61                     is WatchCommand.StartDistanceGoalRun ->
62                         start(WatchRunGoal.Distance(command.targetMeters, command.completionAction))
63                     is WatchCommand.StartIntervalRun -> start(
64                         WatchRunGoal.Interval(
65                             command.work,
66                             command.recovery,
67                             command.sets,
68                             command.completionAction,
69                         ),
70                     )
71                     WatchCommand.PauseRun -> pause()
72                     WatchCommand.ResumeRun -> resume()
73                     WatchCommand.FinishRun -> finish()
74                     WatchCommand.AbandonRun -> abandon()
75                 }
76             }
77         }
78     }
79
80     private suspend fun start(newGoal: WatchRunGoal) {
81         if (manager.state.value.isTracking || runId != null) {
82             stateSender.sendState(
83                 manager.state.value,

```



```

84         runId,
85         status = "ERROR",
86         goal = goal,
87         error = "RUN_ALREADY_ACTIVE",
88     )
89     return
90 }
91
92 if (!hasTrackingPermissions()) {
93     stateSender.sendState(
94         manager.state.value,
95         null,
96         status = "ERROR",
97         goal = newGoal,
98         error = "PHONE_PERMISSION_REQUIRED",
99     )
100    return
101 }
102
103 goal = newGoal
104 val serviceIntent = Intent(context, RunTrackingService::class.java).apply {
105     action = RunTrackingAction.ACTION_START
106     putExtra(RunTrackingAction.EXTRA_MODE, RunTrackingMode.FREE_RUN.name)
107 }
108
109 try {
110     ContextCompat.startForegroundService(context, serviceIntent)
111 } catch (_: RuntimeException) {
112     stateSender.sendState(
113         manager.state.value,
114         null,
115         status = "ERROR",
116         goal = newGoal,
117         error = "TRACKING_SERVICE_START_FAILED",
118     )
119     goal = null
120     return
121 }
122
123 observeState()
124 stateSender.sendState(manager.state.value, null, "STARTING", newGoal)
125
126 when (val result = runningRepository.startRun(
127     StartRunRequest(startedAt = Instant.now().toString()),
128 )) {
129     is NetworkResult.Success -> {
130         runId = result.data.runId
131         sessionStore.saveSnapshot(
132             TrackingSessionSnapshot(
133                 runningRecordId = result.data.runId,
134                 courseAttemptId = null,
135                 elapsedSeconds = 0,
136                 distanceMeters = 0.0,
137             ),
138         )
139         voiceGuide.start()
140         startBatchSaving()
141         stateSender.sendState(manager.state.value, runId, "RUNNING", newGoal)
142     }
143     is NetworkResult.ApiError,
144     is NetworkResult.NetworkError -> {
145         stopTrackingService()
146         stateSender.sendState(
147             manager.state.value,
148             null,
149             status = "ERROR",
150             goal = newGoal,
151             error = "RUN_CREATE_FAILED",
152         )
153         clearLocalSession()
154     }
155 }
156 }
157
158 private suspend fun pause() {
159     val currentRunId = runId ?: return sendNoActiveRun()
160     manager.pause()
161     when (runningRepository.pauseRun(currentRunId)) {
162         is NetworkResult.Success ->
163             stateSender.sendState(manager.state.value, currentRunId, "PAUSED", goal)
164         else -> {
165             manager.resume()
166             stateSender.sendState(
167                 manager.state.value,
168                 currentRunId,
169                 "ERROR",

```

```

170         goal,
171         "PAUSE_FAILED",
172     )
173     }
174 }
175 }
176
177 private suspend fun resume() {
178     val currentRunId = runId ?: return sendNoActiveRun()
179     manager.resume()
180     when (runningRepository.resumeRun(currentRunId)) {
181         is NetworkResult.Success ->
182             stateSender.sendState(manager.state.value, currentRunId, "RUNNING", goal)
183         else -> {
184             manager.pause()
185             stateSender.sendState(
186                 manager.state.value,
187                 currentRunId,
188                 "ERROR",
189                 goal,
190                 "RESUME_FAILED",
191             )
192         }
193     }
194 }
195
196 private suspend fun finish() {
197     val currentRunId = runId ?: return sendNoActiveRun()
198     val state = manager.state.value
199     batchJob?.cancel()
200     flushPendingPoints(currentRunId, drain = true)
201
202     val distanceKm = state.distanceMeters / 1000.0
203     val result = runningRepository.finishRun(
204         currentRunId,
205         FinishRunRequest(
206             endedAt = Instant.now().toString(),
207             distanceMeters = state.distanceMeters,
208             durationSeconds = state.elapsedSeconds,
209             avgPaceSecondsPerKm = if (distanceKm > 0.001) {
210                 (state.elapsedSeconds / distanceKm).toInt()
211             } else {
212                 0
213             },
214             caloriesBurned = (distanceKm * 72).toInt(),
215         ),
216     )
217
218     if (result is NetworkResult.Success) {
219         voiceGuide.finish()
220         stateSender.sendState(state, currentRunId, "FINISHED", goal)
221         sessionStore.clearSnapshot()
222         pendingPointQueue.deleteByRunningRecordId(currentRunId)
223         stopTrackingService()
224         clearLocalSession()
225     } else {
226         stateSender.sendState(state, currentRunId, "ERROR", goal, "FINISH_FAILED")
227     }
228 }
229
230 private suspend fun abandon() {
231     val currentRunId = runId
232     if (currentRunId != null) {
233         runningRepository.abandonRun(currentRunId)
234         pendingPointQueue.deleteByRunningRecordId(currentRunId)
235     }
236     sessionStore.clearSnapshot()
237     stateSender.sendState(manager.state.value, currentRunId, "ABANDONED", goal)
238     stopTrackingService()
239     clearLocalSession()
240 }
241
242 private fun observeState() {
243     stateJob?.cancel()
244     stateJob = appScope.launch {
245         manager.state.collectLatest { state ->
246             if (state.isTracking) {
247                 // 자동 일시정지 ⇄ 재개 음성 안내
248                 if (state.isAutoPaused != prevAutoPaused) {
249                     if (state.isAutoPaused) voiceGuide.autoPaused()
250                     else voiceGuide.autoResumed()
251                     prevAutoPaused = state.isAutoPaused
252                 }
253
254                 // km 마일스톤 음성 안내
255                 val currentKm = (state.distanceMeters / 1000).toInt()

```

```

256         if (currentKm > lastAnnouncedKm && currentKm > 0) {
257             voiceGuide.kilometer(currentKm, state.elapsedSeconds)
258             lastAnnouncedKm = currentKm
259         }
260
261         stateSender.sendState(
262             state,
263             runId,
264             if (state.isPaused) "PAUSED" else "RUNNING",
265             goal,
266         )
267     }
268 }
269
270 }
271
272 private fun startBatchSaving() {
273     batchJob?.cancel()
274     batchJob = appScope.launch {
275         while (true) {
276             delay(5_000)
277             val currentRunId = runId ?: continue
278             flushPendingPoints(currentRunId)
279             val state = manager.state.value
280             sessionStore.saveSnapshot(
281                 TrackingSessionSnapshot(
282                     runningRecordId = currentRunId,
283                     courseAttemptId = null,
284                     elapsedSeconds = state.elapsedSeconds,
285                     distanceMeters = state.distanceMeters,
286                 ),
287             )
288         }
289     }
290 }
291
292 private suspend fun flushPendingPoints(currentRunId: String, drain: Boolean = false) {
293     pendingPointQueue.add(currentRunId, manager.consumePoints())
294     var attempts = 0
295     do {
296         val batch = pendingPointQueue.dequeue(currentRunId, 50)
297         if (batch.isEmpty()) return
298         when (runningRepository.savePoints(
299             currentRunId,
300             SavePointsRequest(batch.map { it.toRunPointRequest() } ),
301         )) {
302             is NetworkResult.Success -> {
303                 pendingPointQueue.deleteByIds(batch.map { it.id })
304                 attempts = 0
305             }
306             else -> {
307                 attempts++
308                 if (!drain || attempts >= 3) return
309             }
310         }
311     } while (drain)
312 }
313
314 private suspend fun sendNoActiveRun() {
315     stateSender.sendState(
316         manager.state.value,
317         null,
318         status = "ERROR",
319         goal = goal,
320         error = "NO_ACTIVE_RUN",
321     )
322 }
323
324 private fun stopTrackingService() {
325     context.startService(
326         Intent(context, RunTrackingService::class.java).apply {
327             action = RunTrackingAction.ACTION_STOP
328         },
329     )
330 }
331
332 private fun clearLocalSession() {
333     batchJob?.cancel()
334     stateJob?.cancel()
335     batchJob = null
336     stateJob = null
337     runId = null
338     goal = null
339     prevAutoPaused = false
340     lastAnnouncedKm = 0
341 }

```

```

342
343     private fun hasTrackingPermissions(): Boolean {
344         val fineLocationGranted = ContextCompat.checkSelfPermission(
345             context,
346             Manifest.permission.ACCESS_FINE_LOCATION,
347         ) == PackageManager.PERMISSION_GRANTED
348         val activityRecognitionGranted = ContextCompat.checkSelfPermission(
349             context,
350             Manifest.permission.ACTIVITY_RECOGNITION,
351         ) == PackageManager.PERMISSION_GRANTED
352         return fineLocationGranted && activityRecognitionGranted
353     }
354 }

```

android/app/src/main/java/com/runway/android/data/attempt/CourseAttemptRepositoryImpl.kt

Kotlin | 51줄

```

1  package com.runway.android.data.attempt
2
3  import com.runway.android.core.result.NetworkResult
4  import com.runway.android.core.result.safeApiCall
5  import com.runway.android.data.attempt.model.AbandonAttemptRequest
6  import com.runway.android.data.attempt.model.AbandonAttemptResponse
7  import com.runway.android.data.attempt.model.FinishAttemptRequest
8  import com.runway.android.data.attempt.model.FinishAttemptResponse
9  import com.runway.android.data.attempt.model.LeaderboardResponse
10 import com.runway.android.data.attempt.model.MyBestAttemptResponse
11 import com.runway.android.data.attempt.model.StartAttemptRequest
12 import com.runway.android.data.attempt.model.StartAttemptResponse
13 import com.runway.android.data.attempt.remote.CourseAttemptApi
14 import com.runway.android.domain.attempt.CourseAttemptRepository
15 import javax.inject.Inject
16 import javax.inject.Singleton
17
18 @Singleton
19 class CourseAttemptRepositoryImpl @Inject constructor(
20     private val courseAttemptApi: CourseAttemptApi,
21 ) : CourseAttemptRepository {
22
23     override suspend fun startAttempt(
24         courseId: String,
25         request: StartAttemptRequest,
26     ): NetworkResult<StartAttemptResponse> =
27         safeApiCall { courseAttemptApi.startAttempt(courseId, request) }
28
29     override suspend fun finishAttempt(
30         attemptId: String,
31         request: FinishAttemptRequest,
32     ): NetworkResult<FinishAttemptResponse> =
33         safeApiCall { courseAttemptApi.finishAttempt(attemptId, request) }
34
35     override suspend fun abandonAttempt(
36         attemptId: String,
37         request: AbandonAttemptRequest,
38     ): NetworkResult<AbandonAttemptResponse> =
39         safeApiCall { courseAttemptApi.abandonAttempt(attemptId, request) }
40
41     override suspend fun getLeaderboard(
42         courseId: String,
43         page: Int,
44         size: Int,
45         sortBy: String,
46     ): NetworkResult<LeaderboardResponse> =
47         safeApiCall { courseAttemptApi.getLeaderboard(courseId, page, size, sortBy) }
48
49     override suspend fun getMyBestAttempt(courseId: String): NetworkResult<MyBestAttemptResponse> =
50         safeApiCall { courseAttemptApi.getMyBestAttempt(courseId) }
51 }

```

android/app/src/main/java/com/runway/android/data/attempt/model/AbandonAttemptRequest.kt

Kotlin | 3줄

```

1  package com.runway.android.data.attempt.model
2
3  data class AbandonAttemptRequest(val endedAt: String)

```

android/app/src/main/java/com/runway/android/data/attempt/model/AbandonAttemptResponse.kt

Kotlin | 8줄

```
1 package com.runway.android.data.attempt.model
2
3 data class AbandonAttemptResponse(
4     val courseAttemptId: String,
5     val runningRecordId: String,
6     val attemptStatus: String,
7     val runStatus: String,
8 )
```

android/app/src/main/java/com/runway/android/data/attempt/model/CourseAttemptResponse.kt

Kotlin | 13줄

```
1 package com.runway.android.data.attempt.model
2
3 data class CourseAttemptResponse(
4     val courseAttemptId: String,
5     val courseId: String,
6     val runningRecordId: String,
7     val attemptStatus: String,
8     val verificationStatus: String?,
9     val durationSeconds: Int?,
10    val distanceMeters: Double?,
11    val startedAt: String,
12    val completedAt: String?,
13 )
```

android/app/src/main/java/com/runway/android/data/attempt/model/FinishAttemptRequest.kt

Kotlin | 10줄

```
1 package com.runway.android.data.attempt.model
2
3 data class FinishAttemptRequest(
4     val endedAt: String,
5     val distanceMeters: Double,
6     val durationSeconds: Int,
7     val avgPaceSecondsPerKm: Int,
8     val caloriesBurned: Int,
9     val avgHeartRateBpm: Int? = null,
10 )
```

android/app/src/main/java/com/runway/android/data/attempt/model/FinishAttemptResponse.kt

Kotlin | 18줄

```
1 package com.runway.android.data.attempt.model
2
3 data class FinishAttemptResponse(
4     val courseAttemptId: String,
5     val runningRecordId: String,
6     val courseId: String,
7     val attemptStatus: String,
8     val verificationStatus: String,
9     val durationSeconds: Int,
10    val distanceMeters: Double,
11    val completedAt: String,
12    /** 개인 최고 기록(PR) 여부 */
13    val isPR: Boolean = false,
14    /** 이전 최고 기록(초). 첫 완주이면 null */
15    val previousBestSeconds: Int? = null,
16    /** 이전 기록 대비 개선 시간(초). 양수 = 빠름, 음수 = 느림. 첫 완주이면 null */
17    val improvementSeconds: Int? = null,
18 )
```

android/app/src/main/java/com/runway/android/data/attempt/model/LeaderboardItem.kt

Kotlin | 10줄

```
1 package com.runway.android.data.attempt.model
2
3 data class LeaderboardItem(
4     val rank: Int,
5     val userId: String,
6     val nickname: String,
7     val profileImageUrl: String?,
8     val bestTimeSeconds: Int,
9     val completionCount: Int,
10 )
```

android/app/src/main/java/com/runway/android/data/attempt/model/LeaderboardResponse.kt

Kotlin | 8줄

```
1 package com.runway.android.data.attempt.model
2
3 data class LeaderboardResponse(
4     val courseId: String,
5     val items: List<LeaderboardItem>,
6     val sortBy: String = "fastest_time",
7     val myRank: Long? = null,
8 )
```

android/app/src/main/java/com/runway/android/data/attempt/model/MyBestAttemptResponse.kt

Kotlin | 7줄

```
1 package com.runway.android.data.attempt.model
2
3 data class MyBestAttemptResponse(
4     val completionCount: Long,
5     val bestTimeSeconds: Int?,
6     val lastAttemptAt: String?,
7 )
```

android/app/src/main/java/com/runway/android/data/attempt/model/StartAttemptRequest.kt

Kotlin | 3줄

```
1 package com.runway.android.data.attempt.model
2
3 data class StartAttemptRequest(val startedAt: String)
```

android/app/src/main/java/com/runway/android/data/attempt/model/StartAttemptResponse.kt

Kotlin | 10줄

```
1 package com.runway.android.data.attempt.model
2
3 data class StartAttemptResponse(
4     val courseId: String,
5     val courseAttemptId: String,
6     val runningRecordId: String,
7     val status: String,
8     val verificationStatus: String,
9     val startedAt: String,
10 )
```

```
1 package com.runway.android.data.attempt.remote
2
3 import com.runway.android.core.model.ApiResponse
4 import com.runway.android.data.attempt.model.AbandonAttemptRequest
5 import com.runway.android.data.attempt.model.AbandonAttemptResponse
6 import com.runway.android.data.attempt.model.FinishAttemptRequest
7 import com.runway.android.data.attempt.model.FinishAttemptResponse
8 import com.runway.android.data.attempt.model.LeaderboardResponse
9 import com.runway.android.data.attempt.model.MyBestAttemptResponse
10 import com.runway.android.data.attempt.model.StartAttemptRequest
11 import com.runway.android.data.attempt.model.StartAttemptResponse
12 import retrofit2.http.Body
13 import retrofit2.http.GET
14 import retrofit2.http.POST
15 import retrofit2.http.Path
16 import retrofit2.http.Query
17
18 interface CourseAttemptApi {
19
20     @POST("api/courses/{courseId}/attempts/start")
21     suspend fun startAttempt(
22         @Path("courseId") courseId: String,
23         @Body request: StartAttemptRequest,
24     ): ApiResponse<StartAttemptResponse>
25
26     @POST("api/course-attempts/{attemptId}/finish")
27     suspend fun finishAttempt(
28         @Path("attemptId") attemptId: String,
29         @Body request: FinishAttemptRequest,
30     ): ApiResponse<FinishAttemptResponse>
31
32     @POST("api/course-attempts/{attemptId}/abandon")
33     suspend fun abandonAttempt(
34         @Path("attemptId") attemptId: String,
35         @Body request: AbandonAttemptRequest,
36     ): ApiResponse<AbandonAttemptResponse>
37
38     @GET("api/courses/{courseId}/attempts/me/best")
39     suspend fun getMyBestAttempt(
40         @Path("courseId") courseId: String,
41     ): ApiResponse<MyBestAttemptResponse>
42
43     @GET("api/courses/{courseId}/leaderboard")
44     suspend fun getLeaderboard(
45         @Path("courseId") courseId: String,
46         @Query("page") page: Int = 0,
47         @Query("size") size: Int = 50,
48         @Query("sortBy") sortBy: String = "fastest_time",
49     ): ApiResponse<LeaderboardResponse>
50 }
```

```
1 package com.runway.android.data.auth
2
3 import com.runway.android.core.datastore.TokenDataStore
4 import com.runway.android.core.result.NetworkResult
5 import com.runway.android.core.result.safeApiCall
6 import com.runway.android.data.auth.model.LoginRequest
7 import com.runway.android.data.auth.model.LoginResponse
8 import com.runway.android.data.auth.model.LogoutRequest
9 import com.runway.android.data.auth.model.ReissueRequest
10 import com.runway.android.data.auth.model.ReissueResponse
11 import com.runway.android.data.auth.model.SignupRequest
12 import com.runway.android.data.auth.model.SignupResponse
13 import com.runway.android.data.auth.remote.AuthApi
14 import com.runway.android.domain.auth.AuthRepository
15 import kotlinx.coroutines.flow.Flow
16 import kotlinx.coroutines.flow.map
17 import retrofit2.HttpException
18 import javax.inject.Inject
19 import javax.inject.Singleton
20
21 @Singleton
22 class AuthRepositoryImpl @Inject constructor(
23     private val authApi: AuthApi,
24     private val tokenDataStore: TokenDataStore,
25 ) : AuthRepository {
26
27     override suspend fun signup(
28         email: String,
29         password: String,
30         nickname: String,
31     ): NetworkResult<SignupResponse> {
32         return safeApiCall { authApi.signup(SignupRequest(email, password, nickname)) }
33     }
34
35     override suspend fun login(email: String, password: String): NetworkResult<LoginResponse> {
36         val result = safeApiCall { authApi.login(LoginRequest(email, password)) }
37         if (result is NetworkResult.Success) {
38             tokenDataStore.saveTokens(result.data.accessToken, result.data.refreshToken)
39         }
40         return result
41     }
42
43     override suspend fun logout(): NetworkResult<Unit> {
44         val refreshToken = tokenDataStore.getRefreshTokenBlocking()
45         ?: return NetworkResult.ApiError(401, "UNAUTHORIZED", "로그인 상태가 아닙니다.")
46         return try {
47             authApi.logout(LogoutRequest(refreshToken))
48             tokenDataStore.clearTokens()
49             NetworkResult.Success(Unit)
50         } catch (e: HttpException) {
51             // 서버 오류가 발생해도 로컬 토큰은 항상 삭제 (보안)
52             tokenDataStore.clearTokens()
53             NetworkResult.ApiError(e.code(), null, e.message() ?: "로그아웃 실패")
54         } catch (e: Exception) {
55             tokenDataStore.clearTokens()
56             NetworkResult.NetworkError(e)
57         }
58     }
59
60     override suspend fun reissue(): NetworkResult<ReissueResponse> {
61         val refreshToken = tokenDataStore.getRefreshTokenBlocking()
62         ?: return NetworkResult.ApiError(401, "UNAUTHORIZED", "로그인 상태가 아닙니다.")
63         val result = safeApiCall { authApi.reissue(ReissueRequest(refreshToken)) }
64         if (result is NetworkResult.Success) {
65             tokenDataStore.saveTokens(result.data.accessToken, result.data.refreshToken)
66         }
67         return result
68     }
69
70     override fun isLoggedInFlow(): Flow<Boolean> {
71         return tokenDataStore.accessTokenFlow.map { it != null }
72     }
73 }
```


android/app/src/main/java/com/runway/android/data/auth/model/LoginRequest.kt

Kotlin | 7줄

```
1 package com.runway.android.data.auth.model
2
3 /** POST /api/auth/login request body */
4 data class LoginRequest(
5     val email: String,
6     val password: String,
7 )
```

android/app/src/main/java/com/runway/android/data/auth/model/LoginResponse.kt

Kotlin | 10줄

```
1 package com.runway.android.data.auth.model
2
3 /** POST /api/auth/login → ApiResponse<LoginResponse>.data */
4 data class LoginResponse(
5     val accessToken: String,
6     val refreshToken: String,
7     val tokenType: String,
8     val expiresIn: Int,
9     val user: UserSummary,
10 )
```

android/app/src/main/java/com/runway/android/data/auth/model/LogoutRequest.kt

Kotlin | 6줄

```
1 package com.runway.android.data.auth.model
2
3 /** POST /api/auth/logout request body */
4 data class LogoutRequest(
5     val refreshToken: String,
6 )
```

android/app/src/main/java/com/runway/android/data/auth/model/ReissueRequest.kt

Kotlin | 6줄

```
1 package com.runway.android.data.auth.model
2
3 /** POST /api/auth/reissue request body */
4 data class ReissueRequest(
5     val refreshToken: String,
6 )
```

android/app/src/main/java/com/runway/android/data/auth/model/ReissueResponse.kt

Kotlin | 9줄

```
1 package com.runway.android.data.auth.model
2
3 /** POST /api/auth/reissue → ApiResponse<ReissueResponse>.data */
4 data class ReissueResponse(
5     val accessToken: String,
6     val refreshToken: String,
7     val tokenType: String,
8     val expiresIn: Int,
9 )
```

android/app/src/main/java/com/runway/android/data/auth/model/SignupRequest.kt

Kotlin | 8줄

```
1 package com.runway.android.data.auth.model
2
3 /** POST /api/auth/signup request body */
4 data class SignupRequest(
5     val email: String,
6     val password: String,
7     val nickname: String,
8 )
```

android/app/src/main/java/com/runway/android/data/auth/model/SignupResponse.kt

Kotlin | 11줄

```
1 package com.runway.android.data.auth.model
2
3 /** POST /api/auth/signup → ApiResponse<SignupResponse>.data */
4 data class SignupResponse(
5     val userId: String,
6     val email: String,
7     val nickname: String,
8     val profileImageUrl: String?,
9     val bio: String?,
10    val createdAt: String,
11 )
```

android/app/src/main/java/com/runway/android/data/auth/model/UserSummary.kt

Kotlin | 9줄

```
1 package com.runway.android.data.auth.model
2
3 /** 로그인 응답 내 user 필드. 프로필 조회(UserApi)의 전체 응답과는 다른 축약형이다. */
4 data class UserSummary(
5     val userId: String,
6     val email: String,
7     val nickname: String,
8     val profileImageUrl: String?,
9 )
```

android/app/src/main/java/com/runway/android/data/auth/remote/AuthApi.kt

Kotlin | 35줄

```
1 package com.runway.android.data.auth.remote
2
3 import com.runway.android.core.model.ApiResponse
4 import com.runway.android.data.auth.model.LoginRequest
5 import com.runway.android.data.auth.model.LoginResponse
6 import com.runway.android.data.auth.model.LogoutRequest
7 import com.runway.android.data.auth.model.ReissueRequest
8 import com.runway.android.data.auth.model.ReissueResponse
9 import com.runway.android.data.auth.model.SignupRequest
10 import com.runway.android.data.auth.model.SignupResponse
11 import retrofit2.http.Body
12 import retrofit2.http.POST
13
14 interface AuthApi {
15
16     @POST("api/auth/signup")
17     suspend fun signup(@Body request: SignupRequest): ApiResponse<SignupResponse>
18
19     @POST("api/auth/login")
20     suspend fun login(@Body request: LoginRequest): ApiResponse<LoginResponse>
21
22     /**
23      * Access Token 재발급.
24      * TokenAuthenticator(noAuth 클라이언트)와 AuthRepositoryImpl 양쪽에서 호출된다.
25      */
26     @POST("api/auth/reissue")
27     suspend fun reissue(@Body request: ReissueRequest): ApiResponse<ReissueResponse>
28
29     /**
30      * 로그아웃. 백엔드가 data: null을 반환하므로 Unit?로 선언한다.
31      * Gson은 null data를 Unit?으로 역직렬화한다.
32      */
33     @POST("api/auth/logout")
34     suspend fun logout(@Body request: LogoutRequest): ApiResponse<Unit?>
35 }
```

```

1  package com.runway.android.data.course
2
3  import com.runway.android.core.model.PageResponse
4  import com.runway.android.core.result.NetworkResult
5  import com.runway.android.core.result.safeApiCall
6  import com.runway.android.core.result.safeApiCallUnit
7  import com.runway.android.data.course.model.CourseDetailResponse
8  import com.runway.android.data.course.model.CoursePointsResponse
9  import com.runway.android.data.course.model.CourseRatingRequest
10 import com.runway.android.data.course.model.CourseRatingResponse
11 import com.runway.android.data.course.model.CourseReportRequest
12 import com.runway.android.data.course.model.CourseReportResponse
13 import com.runway.android.data.course.model.CourseResponse
14 import com.runway.android.data.course.model.CreateCourseFromRunRequest
15 import com.runway.android.data.course.model.NearbyCourseItem
16 import com.runway.android.data.course.model.CourseStatusResponse
17 import com.runway.android.data.course.model.ParticipatedCourseItem
18 import com.runway.android.data.course.model.PublishCourseRequest
19 import com.runway.android.data.course.remote.CourseApi
20 import com.runway.android.domain.course.CourseRepository
21 import javax.inject.Inject
22 import javax.inject.Singleton
23
24 @Singleton
25 class CourseRepositoryImpl @Inject constructor(
26     private val courseApi: CourseApi,
27 ) : CourseRepository {
28
29     override suspend fun createCourseFromRun(
30         runId: String,
31         request: CreateCourseFromRunRequest,
32     ): NetworkResult<CourseResponse> = safeApiCall { courseApi.createCourseFromRun(runId, request) }
33
34     override suspend fun getNearbyCourses(
35         latitude: Double,
36         longitude: Double,
37         radiusMeters: Int,
38         minDistanceMeters: Double?,
39         maxDistanceMeters: Double?,
40         isLoop: Boolean?,
41         keyword: String?,
42         includeRoutePoints: Boolean,
43         includeExactStartPoint: Boolean,
44         page: Int,
45         size: Int,
46     ): NetworkResult<PageResponse<NearbyCourseItem>> = safeApiCall {
47         courseApi.getNearbyCourses(
48             latitude = latitude,
49             longitude = longitude,
50             radiusMeters = radiusMeters,
51             minDistanceMeters = minDistanceMeters,
52             maxDistanceMeters = maxDistanceMeters,
53             isLoop = isLoop,
54             keyword = keyword?.takeIf { !it.isNullOrEmpty() },
55             includeRoutePoints = includeRoutePoints,
56             includeExactStartPoint = includeExactStartPoint,
57             page = page,
58             size = size,
59         )
60     }
61
62     override suspend fun getMyCourses(
63         status: String?,
64         page: Int,
65         size: Int,
66     ): NetworkResult<PageResponse<CourseResponse>> = safeApiCall {
67         courseApi.getMyCourses(status = status, page = page, size = size)
68     }
69
70     override suspend fun getCourseDetail(courseId: String): NetworkResult<CourseDetailResponse> =
71         safeApiCall { courseApi.getCourseDetail(courseId) }
72
73     override suspend fun getCoursePoints(courseId: String): NetworkResult<CoursePointsResponse> =
74         safeApiCall { courseApi.getCoursePoints(courseId) }
75
76     override suspend fun reportCourse(
77         courseId: String,
78         request: CourseReportRequest,
79     ): NetworkResult<CourseReportResponse> = safeApiCall { courseApi.reportCourse(courseId, request) }
80
81     override suspend fun rateCourse(
82         courseId: String,
83         request: CourseRatingRequest,

```

```

84     ): NetworkResult<CourseRatingResponse> = safeApiCall { courseApi.rateCourse(courseId, request) }
85
86     override suspend fun getFavoriteCourses(
87         page: Int,
88         size: Int,
89     ): NetworkResult<PageResponse<CourseResponse>> =
90         safeApiCall { courseApi.getFavoriteCourses(page, size) }
91
92     override suspend fun getParticipatedCourses(
93         page: Int,
94         size: Int,
95     ): NetworkResult<PageResponse<ParticipatedCourseItem>> =
96         safeApiCall { courseApi.getParticipatedCourses(page, size) }
97
98     override suspend fun addFavorite(courseId: String): NetworkResult<Unit> =
99         safeApiCallUnit { courseApi.addFavorite(courseId) }
100
101     override suspend fun removeFavorite(courseId: String): NetworkResult<Unit> =
102         safeApiCallUnit { courseApi.removeFavorite(courseId) }
103
104     override suspend fun publishCourse(courseId: String, request: PublishCourseRequest): NetworkResult<CourseStatusResponse> =
105         safeApiCall { courseApi.publishCourse(courseId, request) }
106
107     override suspend fun archiveCourse(courseId: String): NetworkResult<CourseStatusResponse> =
108         safeApiCall { courseApi.archiveCourse(courseId) }
109
110     override suspend fun deleteDraftCourse(courseId: String): NetworkResult<Unit> =
111         safeApiCall { courseApi.deleteDraftCourse(courseId) }
112 }

```

android/app/src/main/java/com/runway/android/data/course/model/CourseCreatorResponse.kt

Kotlin | 7줄

```

1 package com.runway.android.data.course.model
2
3 data class CourseCreatorResponse(
4     val userId: String,
5     val nickname: String,
6     val profileImageUrl: String?,
7 )

```

android/app/src/main/java/com/runway/android/data/course/model/CourseDetailResponse.kt

Kotlin | 26줄

```

1 package com.runway.android.data.course.model
2
3 data class CourseDetailResponse(
4     val courseId: String,
5     val name: String,
6     val description: String?,
7     val status: String,
8     val distanceMeters: Double,
9     val isLoop: Boolean,
10    val startPoint: GeoPoint?,
11    val endPoint: GeoPoint?,
12    val attemptCount: Int,
13    val completionCount: Int,
14    val avgRating: Double?,
15    val ratingCount: Long?,
16    val isFavorited: Boolean = false,
17    val isOwner: Boolean = false,
18    val difficulty: String? = null,
19    val slopeLevel: String? = null,
20    val riskLevel: String? = null,
21    val surfaceType: String? = null,
22    val recommendedTime: String? = null,
23    val warnings: String? = null,
24    val creator: CourseCreatorResponse,
25    val createdAt: String,
26 )

```

android/app/src/main/java/com/runway/android/data/course/model/CoursePointResponse.kt

Kotlin | 8줄

```

1 package com.runway.android.data.course.model
2
3 data class CoursePointResponse(
4     val sequence: Int,
5     val latitude: Double,
6     val longitude: Double,
7     val altitudeMeters: Double?,
8 )

```

android/app/src/main/java/com/runway/android/data/course/model/CoursePointsResponse.kt

Kotlin | 6줄

```
1 package com.runway.android.data.course.model
2
3 data class CoursePointsResponse(
4     val courseId: String,
5     val points: List<CoursePointResponse>,
6 )
```

android/app/src/main/java/com/runway/android/data/course/model/CourseRatingRequest.kt

Kotlin | 6줄

```
1 package com.runway.android.data.course.model
2
3 data class CourseRatingRequest(
4     val rating: Int,
5     val comment: String?,
6 )
```

android/app/src/main/java/com/runway/android/data/course/model/CourseRatingResponse.kt

Kotlin | 10줄

```
1 package com.runway.android.data.course.model
2
3 data class CourseRatingResponse(
4     val ratingId: String,
5     val courseId: String,
6     val rating: Int,
7     val comment: String?,
8     val createdAt: String,
9     val updatedAt: String,
10 )
```

android/app/src/main/java/com/runway/android/data/course/model/CourseReportRequest.kt

Kotlin | 6줄

```
1 package com.runway.android.data.course.model
2
3 data class CourseReportRequest(
4     val reason: String,
5     val description: String?,
6 )
```

android/app/src/main/java/com/runway/android/data/course/model/CourseReportResponse.kt

Kotlin | 8줄

```
1 package com.runway.android.data.course.model
2
3 data class CourseReportResponse(
4     val reportId: String,
5     val courseId: String,
6     val status: String,
7     val createdAt: String,
8 )
```

android/app/src/main/java/com/runway/android/data/course/model/CourseResponse.kt

Kotlin | 15줄

```
1 package com.runway.android.data.course.model
2
3 data class CourseResponse(
4     val courseId: String,
5     val name: String,
6     val description: String?,
7     val status: String,
8     val distanceMeters: Double,
9     val isLoop: Boolean,
10    val startPoint: GeoPoint,
11    val endPoint: GeoPoint,
12    val attemptCount: Int,
13    val completionCount: Int,
14    val createdAt: String,
15 )
```

android/app/src/main/java/com/runway/android/data/course/model/CourseStatusResponse.kt

Kotlin | 6줄

```
1 package com.runway.android.data.course.model
2
3 data class CourseStatusResponse(
4     val courseId: String,
5     val status: String,
6 )
```

android/app/src/main/java/com/runway/android/data/course/model/CreateCourseFromRunRequest.kt

Kotlin | 8줄

```
1 package com.runway.android.data.course.model
2
3 data class CreateCourseFromRunRequest(
4     val name: String,
5     val description: String?,
6     val isLoop: Boolean,
7     val publish: Boolean,
8 )
```

android/app/src/main/java/com/runway/android/data/course/model/GeoPoint.kt

Kotlin | 6줄

```
1 package com.runway.android.data.course.model
2
3 data class GeoPoint(
4     val latitude: Double,
5     val longitude: Double,
6 )
```

android/app/src/main/java/com/runway/android/data/course/model/NearbyCourseItem.kt

Kotlin | 19줄

```
1 package com.runway.android.data.course.model
2
3 data class NearbyCourseItem(
4     val courseId: String,
5     val name: String,
6     val description: String?,
7     val status: String,
8     val distanceMeters: Double,
9     val isLoop: Boolean,
10    val startPoint: GeoPoint,
11    val endPoint: GeoPoint?,
12    val attemptCount: Int,
13    val completionCount: Int,
14    val distanceFromMeMeters: Double,
15    val createdAt: String,
16    val avgRating: Double?,
17    val ratingCount: Long?,
18    val routePoints: List<GeoPoint> = emptyList(),
19 )
```

android/app/src/main/java/com/runway/android/data/course/model/ParticipatedCourseItem.kt

Kotlin | 14줄

```
1 package com.runway.android.data.course.model
2
3 data class ParticipatedCourseItem(
4     val courseId: String,
5     val name: String,
6     val description: String?,
7     val distanceMeters: Double,
8     val isLoop: Boolean,
9     val status: String,
10    val attemptCountByMe: Int,
11    val completionCountByMe: Int,
12    val bestTimeSecondsByMe: Int?,
13    val lastAttemptAt: String?,
14 )
```

```
1 package com.runway.android.data.course.model
2
3 data class PublishCourseRequest(
4     val difficulty: String,
5     val slopeLevel: String,
6     val riskLevel: String,
7     val surfaceType: String,
8     val recommendedTime: String,
9     val warnings: String?,
10    val description: String?,
11 )
```

```

1  package com.runway.android.data.course.remote
2
3  import com.runway.android.core.model.ApiResponse
4  import com.runway.android.core.model.PageResponse
5  import com.runway.android.data.course.model.CourseDetailResponse
6  import com.runway.android.data.course.model.CoursePointsResponse
7  import com.runway.android.data.course.model.CourseRatingRequest
8  import com.runway.android.data.course.model.CourseRatingResponse
9  import com.runway.android.data.course.model.CourseReportRequest
10 import com.runway.android.data.course.model.CourseReportResponse
11 import com.runway.android.data.course.model.CourseResponse
12 import com.runway.android.data.course.model.CreateCourseFromRunRequest
13 import com.runway.android.data.course.model.NearbyCourseItem
14 import com.runway.android.data.course.model.ParticipatedCourseItem
15 import com.runway.android.data.course.model.CourseStatusResponse
16 import com.runway.android.data.course.model.PublishCourseRequest
17 import retrofit2.http.Body
18 import retrofit2.http.DELETE
19 import retrofit2.http.GET
20 import retrofit2.http.PATCH
21 import retrofit2.http.POST
22 import retrofit2.http.Path
23 import retrofit2.http.Query
24
25 interface CourseApi {
26
27     @POST("api/courses/from-run/{runId}")
28     suspend fun createCourseFromRun(
29         @Path("runId") runId: String,
30         @Body request: CreateCourseFromRunRequest,
31     ): ApiResponse<CourseResponse>
32
33     @GET("api/courses/nearby")
34     suspend fun getNearbyCourses(
35         @Query("latitude") latitude: Double,
36         @Query("longitude") longitude: Double,
37         @Query("radiusMeters") radiusMeters: Int? = null,
38         @Query("minDistanceMeters") minDistanceMeters: Double? = null,
39         @Query("maxDistanceMeters") maxDistanceMeters: Double? = null,
40         @Query("isLoop") isLoop: Boolean? = null,
41         @Query("keyword") keyword: String? = null,
42         @Query("includeRoutePoints") includeRoutePoints: Boolean? = null,
43         @Query("includeExactStartPoint") includeExactStartPoint: Boolean? = null,
44         @Query("page") page: Int? = null,
45         @Query("size") size: Int? = null,
46     ): ApiResponse<PageResponse<NearbyCourseItem>>
47
48     @GET("api/courses/{courseId}")
49     suspend fun getCourseDetail(
50         @Path("courseId") courseId: String,
51     ): ApiResponse<CourseDetailResponse>
52
53     @GET("api/courses/me")
54     suspend fun getMyCourses(
55         @Query("status") status: String? = null,
56         @Query("page") page: Int? = null,
57         @Query("size") size: Int? = null,
58     ): ApiResponse<PageResponse<CourseResponse>>
59
60     @GET("api/courses/{courseId}/points")
61     suspend fun getCoursePoints(
62         @Path("courseId") courseId: String,
63     ): ApiResponse<CoursePointsResponse>
64
65     @POST("api/courses/{courseId}/reports")
66     suspend fun reportCourse(
67         @Path("courseId") courseId: String,
68         @Body request: CourseReportRequest,
69     ): ApiResponse<CourseReportResponse>
70
71     @POST("api/courses/{courseId}/ratings")
72     suspend fun rateCourse(
73         @Path("courseId") courseId: String,
74         @Body request: CourseRatingRequest,
75     ): ApiResponse<CourseRatingResponse>
76
77     @GET("api/courses/favorites")
78     suspend fun getFavoriteCourses(
79         @Query("page") page: Int = 0,
80         @Query("size") size: Int = 20,
81     ): ApiResponse<PageResponse<CourseResponse>>
82
83     @GET("api/courses/participated")

```



```

84     suspend fun getParticipatedCourses(
85         @Query("page") page: Int = 0,
86         @Query("size") size: Int = 20,
87     ): ApiResponse<PageResponse<ParticipatedCourseItem>>
88
89     @POST("api/courses/{courseId}/favorite")
90     suspend fun addFavorite(
91         @Path("courseId") courseId: String,
92     ): ApiResponse<Unit>
93
94     @DELETE("api/courses/{courseId}/favorite")
95     suspend fun removeFavorite(
96         @Path("courseId") courseId: String,
97     ): ApiResponse<Unit>
98
99     @PATCH("api/courses/{courseId}/publish")
100    suspend fun publishCourse(
101        @Path("courseId") courseId: String,
102        @Body request: PublishCourseRequest,
103    ): ApiResponse<CourseStatusResponse>
104
105    @PATCH("api/courses/{courseId}/archive")
106    suspend fun archiveCourse(
107        @Path("courseId") courseId: String,
108    ): ApiResponse<CourseStatusResponse>
109
110    @DELETE("api/courses/{courseId}")
111    suspend fun deleteDraftCourse(
112        @Path("courseId") courseId: String,
113    ): ApiResponse<Unit>
114 }

```

android/app/src/main/java/com/runway/android/data/reminder/ReminderPrefs.kt

Kotlin | 14줄

```

1  package com.runway.android.data.reminder
2
3  import java.util.Calendar
4
5  data class ReminderPrefs(
6      val enabled: Boolean = false,
7      val hour: Int = 7,
8      val minute: Int = 0,
9      val enabledDays: Set<Int> = setOf(
10          Calendar.MONDAY,
11          Calendar.WEDNESDAY,
12          Calendar.FRIDAY,
13      ),
14  )

```

```
1 package com.runway.android.data.reminder
2
3 import androidx.datastore.core.DataStore
4 import androidx.datastore.preferences.core.Preferences
5 import androidx.datastore.preferences.core.booleanPreferencesKey
6 import androidx.datastore.preferences.core.edit
7 import androidx.datastore.preferences.core.intPreferencesKey
8 import androidx.datastore.preferences.core.stringPreferencesKey
9 import kotlinx.coroutines.flow.Flow
10 import kotlinx.coroutines.flow.map
11 import java.util.Calendar
12 import javax.inject.Inject
13 import javax.inject.Named
14 import javax.inject.Singleton
15
16 @Singleton
17 class ReminderPrefsRepository @Inject constructor(
18     @Named("reminderDataStore") private val datastore: DataStore<Preferences>,
19 ) {
20     private object Keys {
21         val ENABLED = booleanPreferencesKey("enabled")
22         val HOUR = intPreferencesKey("hour")
23         val MINUTE = intPreferencesKey("minute")
24         val DAYS = stringPreferencesKey("days")
25     }
26
27     fun getPrefs(): Flow<ReminderPrefs> = datastore.data.map { prefs ->
28         val days = prefs[Keys.DAYS]
29             ?.split(",")
30             ?.mapNotNull { it.toIntOrNull() }
31             ?.toSet()
32             ?: setOf(Calendar.MONDAY, Calendar.WEDNESDAY, Calendar.FRIDAY)
33         ReminderPrefs(
34             enabled = prefs[Keys.ENABLED] ?: false,
35             hour = prefs[Keys.HOUR] ?: 7,
36             minute = prefs[Keys.MINUTE] ?: 0,
37             enabledDays = days,
38         )
39     }
40
41     suspend fun save(prefs: ReminderPrefs) {
42         datastore.edit { p ->
43             p[Keys.ENABLED] = prefs.enabled
44             p[Keys.HOUR] = prefs.hour
45             p[Keys.MINUTE] = prefs.minute
46             p[Keys.DAYS] = prefs.enabledDays.joinToString(",")
47         }
48     }
49 }
```

android/app/src/main/java/com/runway/android/data/running/RunningRepositoryImpl.kt

Kotlin | 64줄

```
1 package com.runway.android.data.running
2
3 import com.runway.android.core.model.PageResponse
4 import com.runway.android.core.result.NetworkResult
5 import com.runway.android.core.result.safeApiCall
6 import com.runway.android.core.result.safeApiCallUnit
7 import com.runway.android.data.running.model.FinishRunRequest
8 import com.runway.android.data.running.model.FinishRunResponse
9 import com.runway.android.data.running.model.PersonalRecordsResponse
10 import com.runway.android.data.running.model.RunDetailResponse
11 import com.runway.android.data.running.model.RunningStatsResponse
12 import com.runway.android.data.running.model.RunStatusResponse
13 import com.runway.android.data.running.model.RunSummaryResponse
14 import com.runway.android.data.running.model.SavePointsRequest
15 import com.runway.android.data.running.model.SavePointsResponse
16 import com.runway.android.data.running.model.StartRunRequest
17 import com.runway.android.data.running.model.StartRunResponse
18 import com.runway.android.data.running.model.TrimRunRequest
19 import com.runway.android.data.running.remote.RunningApi
20 import com.runway.android.domain.running.RunningRepository
21 import javax.inject.Inject
22 import javax.inject.Singleton
23
24 @Singleton
25 class RunningRepositoryImpl @Inject constructor(
26     private val runningApi: RunningApi,
27 ) : RunningRepository {
28
29     override suspend fun startRun(request: StartRunRequest): NetworkResult<StartRunResponse> =
30         safeApiCall { runningApi.startRun(request) }
31
32     override suspend fun savePoints(runId: String, request: SavePointsRequest): NetworkResult<SavePointsResponse> =
33         safeApiCall { runningApi.savePoints(runId, request) }
34
35     override suspend fun pauseRun(runId: String): NetworkResult<RunStatusResponse> =
36         safeApiCall { runningApi.pauseRun(runId) }
37
38     override suspend fun resumeRun(runId: String): NetworkResult<RunStatusResponse> =
39         safeApiCall { runningApi.resumeRun(runId) }
40
41     override suspend fun finishRun(runId: String, request: FinishRunRequest): NetworkResult<FinishRunResponse> =
42         safeApiCall { runningApi.finishRun(runId, request) }
43
44     override suspend fun abandonRun(runId: String): NetworkResult<RunStatusResponse> =
45         safeApiCall { runningApi.abandonRun(runId) }
46
47     override suspend fun getMyRuns(page: Int, size: Int): NetworkResult<PageResponse<RunSummaryResponse>> =
48         safeApiCall { runningApi.getMyRuns(page, size) }
49
50     override suspend fun getRunDetail(runId: String): NetworkResult<RunDetailResponse> =
51         safeApiCall { runningApi.getRunDetail(runId) }
52
53     override suspend fun getPersonalRecords(): NetworkResult<PersonalRecordsResponse> =
54         safeApiCall { runningApi.getPersonalRecords() }
55
56     override suspend fun getRunningStats(period: String): NetworkResult<RunningStatsResponse> =
57         safeApiCall { runningApi.getRunningStats(period) }
58
59     override suspend fun deleteRun(runId: String): NetworkResult<Unit> =
60         safeApiCallUnit { runningApi.deleteRun(runId) }
61
62     override suspend fun trimRun(runId: String, targetDistanceMeters: Double): NetworkResult<Unit> =
63         safeApiCallUnit { runningApi.trimRun(runId, TrimRunRequest(targetDistanceMeters)) }
64 }
```

android/app/src/main/java/com/runway/android/data/running/model/FinishRunRequest.kt

Kotlin | 10줄

```
1 package com.runway.android.data.running.model
2
3 data class FinishRunRequest(
4     val endedAt: String,
5     val distanceMeters: Double,
6     val durationSeconds: Int,
7     val avgPaceSecondsPerKm: Int,
8     val caloriesBurned: Int,
9     val avgHeartRateBpm: Int? = null,
10 )
```

android/app/src/main/java/com/runway/android/data/running/model/FinishRunResponse.kt

Kotlin | 12줄

```
1 package com.runway.android.data.running.model
2
3 data class FinishRunResponse(
4     val runId: String,
5     val status: String,
6     val startedAt: String,
7     val endedAt: String,
8     val distanceMeters: Double,
9     val durationSeconds: Int,
10    val avgPaceSecondsPerKm: Int,
11    val pathCreated: Boolean,
12 )
```

android/app/src/main/java/com/runway/android/data/running/model/PersonalRecordItem.kt

Kotlin | 10줄

```
1 package com.runway.android.data.running.model
2
3 data class PersonalRecordItem(
4     val runId: String,
5     val distanceMeters: Double?,
6     val durationSeconds: Int?,
7     val avgPaceSecondsPerKm: Int?,
8     val caloriesBurned: Int?,
9     val recordedAt: String,
10 )
```

android/app/src/main/java/com/runway/android/data/running/model/PersonalRecordsResponse.kt

Kotlin | 11줄

```
1 package com.runway.android.data.running.model
2
3 data class PersonalRecordsResponse(
4     val longestRun: PersonalRecordItem?,
5     val fastestPace: PersonalRecordItem?,
6     val mostCalories: PersonalRecordItem?,
7     val best5k: PersonalRecordItem?,
8     val best10k: PersonalRecordItem?,
9     val totalCompletedRuns: Long,
10    val totalDistanceMeters: Double,
11 )
```

android/app/src/main/java/com/runway/android/data/running/model/RunDetailResponse.kt

Kotlin | 14줄

```
1 package com.runway.android.data.running.model
2
3 data class RunDetailResponse(
4     val runId: String,
5     val status: String,
6     val startedAt: String,
7     val endedAt: String?,
8     val distanceMeters: Double?,
9     val durationSeconds: Int?,
10    val avgPaceSecondsPerKm: Int?,
11    val caloriesBurned: Int?,
12    val avgHeartRateBpm: Int?,
13    val points: List<RunPointResponse>,
14 )
```

android/app/src/main/java/com/runway/android/data/running/model/RunPointRequest.kt

Kotlin | 10줄

```
1 package com.runway.android.data.running.model
2
3 data class RunPointRequest(
4     val sequence: Int,
5     val latitude: Double,
6     val longitude: Double,
7     val altitudeMeters: Double,
8     val speedMps: Double,
9     val recordedAt: String,
10 )
```

android/app/src/main/java/com/runway/android/data/running/model/RunPointResponse.kt

Kotlin | 10줄

```
1 package com.runway.android.data.running.model
2
3 data class RunPointResponse(
4     val sequence: Int,
5     val latitude: Double,
6     val longitude: Double,
7     val altitudeMeters: Double?,
8     val speedMps: Float?,
9     val recordedAt: String,
10 )
```

android/app/src/main/java/com/runway/android/data/running/model/RunStatusResponse.kt

Kotlin | 6줄

```
1 package com.runway.android.data.running.model
2
3 data class RunStatusResponse(
4     val runId: String,
5     val status: String,
6 )
```

android/app/src/main/java/com/runway/android/data/running/model/RunSummaryResponse.kt

Kotlin | 12줄

```
1 package com.runway.android.data.running.model
2
3 data class RunSummaryResponse(
4     val runId: String,
5     val status: String,
6     val startedAt: String,
7     val endedAt: String?,
8     val distanceMeters: Double?,
9     val durationSeconds: Int?,
10    val avgPaceSecondsPerKm: Int?,
11    val caloriesBurned: Int?,
12 )
```

android/app/src/main/java/com/runway/android/data/running/model/RunningStatsResponse.kt

Kotlin | 16줄

```
1 package com.runway.android.data.running.model
2
3 data class RunningStatsResponse(
4     val period: String,
5     val totalRuns: Long,
6     val totalDistanceMeters: Double,
7     val totalDurationSeconds: Long,
8     val totalCaloriesBurned: Long,
9     val averagePaceSecondsPerKm: Int,
10    val longestRunMeters: Double,
11    val currentStreakDays: Int,
12    val longestStreakDays: Int,
13    val activeDays: Long,
14    val periodStart: String?,
15    val periodEnd: String?,
16 )
```

android/app/src/main/java/com/runway/android/data/running/model/SavePointsRequest.kt

Kotlin | 5줄

```
1 package com.runway.android.data.running.model
2
3 data class SavePointsRequest(
4     val points: List<RunPointRequest>,
5 )
```

android/app/src/main/java/com/runway/android/data/running/model/SavePointsResponse.kt

Kotlin | 6줄

```
1 package com.runway.android.data.running.model
2
3 data class SavePointsResponse(
4     val runId: String,
5     val savedCount: Int,
6 )
```

android/app/src/main/java/com/runway/android/data/running/model/StartRunRequest.kt

Kotlin | 5줄

```
1 package com.runway.android.data.running.model
2
3 data class StartRunRequest(
4     val startedAt: String,
5 )
```

android/app/src/main/java/com/runway/android/data/running/model/StartRunResponse.kt

Kotlin | 7줄

```
1 package com.runway.android.data.running.model
2
3 data class StartRunResponse(
4     val runId: String,
5     val status: String,
6     val startedAt: String,
7 )
```

android/app/src/main/java/com/runway/android/data/running/model/TrimRunRequest.kt

Kotlin | 5줄

```
1 package com.runway.android.data.running.model
2
3 data class TrimRunRequest(
4     val targetDistanceMeters: Double,
5 )
```

```
1 package com.runway.android.data.running.remote
2
3 import com.runway.android.core.model.ApiResponse
4 import com.runway.android.core.model.PageResponse
5 import com.runway.android.data.running.model.FinishRunRequest
6 import com.runway.android.data.running.model.FinishRunResponse
7 import com.runway.android.data.running.model.PersonalRecordsResponse
8 import com.runway.android.data.running.model.RunDetailResponse
9 import com.runway.android.data.running.model.RunningStatsResponse
10 import com.runway.android.data.running.model.RunStatusResponse
11 import com.runway.android.data.running.model.RunSummaryResponse
12 import com.runway.android.data.running.model.SavePointsRequest
13 import com.runway.android.data.running.model.SavePointsResponse
14 import com.runway.android.data.running.model.StartRunRequest
15 import com.runway.android.data.running.model.StartRunResponse
16 import com.runway.android.data.running.model.TrimRunRequest
17 import retrofit2.http.Body
18 import retrofit2.http.DELETE
19 import retrofit2.http.GET
20 import retrofit2.http.PATCH
21 import retrofit2.http.POST
22 import retrofit2.http.Path
23 import retrofit2.http.Query
24
25 interface RunningApi {
26
27     @POST("api/runs/start")
28     suspend fun startRun(@Body request: StartRunRequest): ApiResponse<StartRunResponse>
29
30     @POST("api/runs/{runId}/points")
31     suspend fun savePoints(
32         @Path("runId") runId: String,
33         @Body request: SavePointsRequest,
34     ): ApiResponse<SavePointsResponse>
35
36     @POST("api/runs/{runId}/pause")
37     suspend fun pauseRun(@Path("runId") runId: String): ApiResponse<RunStatusResponse>
38
39     @POST("api/runs/{runId}/resume")
40     suspend fun resumeRun(@Path("runId") runId: String): ApiResponse<RunStatusResponse>
41
42     @POST("api/runs/{runId}/finish")
43     suspend fun finishRun(
44         @Path("runId") runId: String,
45         @Body request: FinishRunRequest,
46     ): ApiResponse<FinishRunResponse>
47
48     @POST("api/runs/{runId}/abandon")
49     suspend fun abandonRun(@Path("runId") runId: String): ApiResponse<RunStatusResponse>
50
51     @GET("api/runs/me")
52     suspend fun getMyRuns(
53         @Query("page") page: Int = 0,
54         @Query("size") size: Int = 20,
55     ): ApiResponse<PageResponse<RunSummaryResponse>>
56
57     @GET("api/runs/{runId}")
58     suspend fun getRunDetail(@Path("runId") runId: String): ApiResponse<RunDetailResponse>
59
60     @GET("api/runs/me/records")
61     suspend fun getPersonalRecords(): ApiResponse<PersonalRecordsResponse>
62
63     @GET("api/runs/me/stats")
64     suspend fun getRunningStats(
65         @Query("period") period: String = "monthly",
66     ): ApiResponse<RunningStatsResponse>
67
68     @DELETE("api/runs/{runId}")
69     suspend fun deleteRun(@Path("runId") runId: String): ApiResponse<Unit>
70
71     @PATCH("api/runs/{runId}/trim")
72     suspend fun trimRun(
73         @Path("runId") runId: String,
74         @Body request: TrimRunRequest,
75     ): ApiResponse<Unit>
76 }
```

android/app/src/main/java/com/runway/android/data/user/UserRepositoryImpl.kt

Kotlin | 44줄

```
1 package com.runway.android.data.user
2
3 import android.content.Context
4 import android.net.Uri
5 import com.runway.android.core.result.NetworkResult
6 import com.runway.android.core.result.safeApiCall
7 import com.runway.android.data.user.model.AchievementsResponse
8 import com.runway.android.data.user.model.UpdateProfileRequest
9 import com.runway.android.data.user.model.UserProfileResponse
10 import com.runway.android.data.user.remote.UserApi
11 import com.runway.android.domain.user.UserRepository
12 import dagger.hilt.android.qualifiers.ApplicationContext
13 import okhttp3.MediaType.Companion.toMediaTypeOrNull
14 import okhttp3.MultipartBody
15 import okhttp3.RequestBody.Companion.toRequestBody
16 import javax.inject.Inject
17 import javax.inject.Singleton
18
19 @Singleton
20 class UserRepositoryImpl @Inject constructor(
21     @ApplicationContext private val context: Context,
22     private val userApi: UserApi,
23 ) : UserRepository {
24
25     override suspend fun getMe(): NetworkResult<UserProfileResponse> =
26         safeApiCall { userApi.getMe() }
27
28     override suspend fun updateMe(request: UpdateProfileRequest): NetworkResult<UserProfileResponse> =
29         safeApiCall { userApi.updateMe(request) }
30
31     override suspend fun uploadProfileImage(uri: Uri): NetworkResult<String> = safeApiCall {
32         val stream = context.contentResolver.openInputStream(uri)
33         ?: error("Cannot open image")
34         val mimeType = context.contentResolver.getType(uri) ?: "image/jpeg"
35         val bytes = stream.use { it.readBytes() }
36         val requestBody = bytes.toRequestBody(mimeType.toMediaTypeOrNull())
37         val ext = if (mimeType.contains("png")) ".png" else ".jpg"
38         val part = MultipartBody.Part.createFormData("image", "profile$ext", requestBody)
39         userApi.uploadProfileImage(part)
40     }
41
42     override suspend fun getAchievements(): NetworkResult<AchievementsResponse> =
43         safeApiCall { userApi.getAchievements() }
44 }
```

android/app/src/main/java/com/runway/android/data/user/model/AchievementItem.kt

Kotlin | 11줄

```
1 package com.runway.android.data.user.model
2
3 data class AchievementItem(
4     val code: String,
5     val title: String,
6     val description: String,
7     val unlocked: Boolean,
8     val unlockedAt: String?,
9     val progress: Long,
10    val target: Long,
11 )
```

android/app/src/main/java/com/runway/android/data/user/model/AchievementsResponse.kt

Kotlin | 5줄

```
1 package com.runway.android.data.user.model
2
3 data class AchievementsResponse(
4     val items: List<AchievementItem>,
5 )
```

android/app/src/main/java/com/runway/android/data/user/model/UpdateProfileRequest.kt

Kotlin | 7줄

```
1 package com.runway.android.data.user.model
2
3 data class UpdateProfileRequest(
4     val nickname: String,
5     val profileImageUrl: String?,
6     val bio: String?,
7 )
```


android/app/src/main/java/com/runway/android/data/user/model/UserProfileResponse.kt

Kotlin | 10줄

```
1 package com.runway.android.data.user.model
2
3 data class UserProfileResponse(
4     val userId: String,
5     val email: String,
6     val nickname: String,
7     val profileImageUrl: String?,
8     val bio: String?,
9     val createdAt: String,
10 )
```

android/app/src/main/java/com/runway/android/data/user/remote/UserApi.kt

Kotlin | 29줄

```
1 package com.runway.android.data.user.remote
2
3 import com.runway.android.core.model.ApiResponse
4 import com.runway.android.data.user.model.AchievementsResponse
5 import com.runway.android.data.user.model.UpdateProfileRequest
6 import com.runway.android.data.user.model.UserProfileResponse
7 import okhttp3.MultipartBody
8 import retrofit2.http.Body
9 import retrofit2.http.GET
10 import retrofit2.http.Multipart
11 import retrofit2.http.POST
12 import retrofit2.http.PUT
13 import retrofit2.http.Part
14
15 interface UserApi {
16
17     @GET("api/users/me")
18     suspend fun getMe(): ApiResponse<UserProfileResponse>
19
20     @PUT("api/users/me")
21     suspend fun updateMe(@Body request: UpdateProfileRequest): ApiResponse<UserProfileResponse>
22
23     @Multipart
24     @POST("api/users/me/profile-image")
25     suspend fun uploadProfileImage(@Part image: MultipartBody.Part): ApiResponse<String>
26
27     @GET("api/users/me/achievements")
28     suspend fun getAchievements(): ApiResponse<AchievementsResponse>
29 }
```

android/app/src/main/java/com/runway/android/di/AppModule.kt

Kotlin | 26줄

```
1 package com.runway.android.di
2
3 import dagger.Module
4 import dagger.Provides
5 import dagger.hilt.InstallIn
6 import dagger.hilt.components.SingletonComponent
7 import kotlinx.coroutines.CoroutineScope
8 import kotlinx.coroutines.Dispatchers
9 import kotlinx.coroutines.SupervisorJob
10 import javax.inject.Named
11 import javax.inject.Singleton
12
13 @Module
14 @InstallIn(SingletonComponent::class)
15 object AppModule {
16
17     /**
18      * 앱 생명주기와 함께하는 CoroutineScope.
19      * ViewModel의 onCleared() 이후에도 완료되어야 하는 cleanup 작업(ex. abandonRun)에 사용한다.
20      */
21     @Provides
22     @Singleton
23     @Named("appScope")
24     fun provideApplicationScope(): CoroutineScope =
25         CoroutineScope(SupervisorJob() + Dispatchers.IO)
26 }
```

```
1 package com.runway.android.di
2
3 import android.content.Context
4 import androidx.datastore.core.DataStore
5 import androidx.datastore.preferences.core.Preferences
6 import androidx.datastore.preferences.preferencesDataStore
7 import dagger.Module
8 import dagger.Provides
9 import dagger.hilt.InstallIn
10 import dagger.hilt.android.qualifiers.ApplicationContext
11 import dagger.hilt.components.SingletonComponent
12 import javax.inject.Named
13 import javax.inject.Singleton
14
15 private val Context.tokenDataStore: DataStore<Preferences> by preferencesDataStore(name = "runway_auth")
16 private val Context.trackingDataStore: DataStore<Preferences> by preferencesDataStore(name = "runway_tracking")
17 private val Context.onboardingDataStore: DataStore<Preferences> by preferencesDataStore(name = "runway_onboarding")
18 private val Context.reminderDataStore: DataStore<Preferences> by preferencesDataStore(name = "runway_reminder")
19 private val Context.settingsDataStore: DataStore<Preferences> by preferencesDataStore(name = "runway_settings")
20
21 @Module
22 @InstallIn(SingletonComponent::class)
23 object DataStoreModule {
24
25     @Provides
26     @Singleton
27     fun provideTokenDataStore(
28         @ApplicationContext context: Context,
29     ): DataStore<Preferences> = context.tokenDataStore
30
31     @Provides
32     @Singleton
33     @Named("trackingDataStore")
34     fun provideTrackingDataStore(
35         @ApplicationContext context: Context,
36     ): DataStore<Preferences> = context.trackingDataStore
37
38     @Provides
39     @Singleton
40     @Named("onboardingDataStore")
41     fun provideOnboardingDataStore(
42         @ApplicationContext context: Context,
43     ): DataStore<Preferences> = context.onboardingDataStore
44
45     @Provides
46     @Singleton
47     @Named("reminderDataStore")
48     fun provideReminderDataStore(
49         @ApplicationContext context: Context,
50     ): DataStore<Preferences> = context.reminderDataStore
51
52     @Provides
53     @Singleton
54     @Named("settingsDataStore")
55     fun provideSettingsDataStore(
56         @ApplicationContext context: Context,
57     ): DataStore<Preferences> = context.settingsDataStore
58 }
```

android/app/src/main/java/com/runway/android/di/DatabaseModule.kt

Kotlin | 33줄

```
1 package com.runway.android.di
2
3 import android.content.Context
4 import androidx.room.Room
5 import com.runway.android.core.tracking.local.PendingRunPointDao
6 import com.runway.android.core.tracking.local.TrackingDatabase
7 import dagger.Module
8 import dagger.Provides
9 import dagger.hilt.InstallIn
10 import dagger.hilt.android.qualifiers.ApplicationContext
11 import dagger.hilt.components.SingletonComponent
12 import javax.inject.Singleton
13
14 @Module
15 @InstallIn(SingletonComponent::class)
16 object DatabaseModule {
17
18     @Provides
19     @Singleton
20     fun provideTrackingDatabase(
21         @ApplicationContext context: Context,
22     ): TrackingDatabase = Room.databaseBuilder(
23         context,
24         TrackingDatabase::class.java,
25         "tracking_database",
26     ).fallbackToDestructiveMigration()
27         .build()
28
29     @Provides
30     @Singleton
31     fun providePendingRunPointDao(db: TrackingDatabase): PendingRunPointDao =
32         db.pendingRunPointDao()
33 }
```

android/app/src/main/java/com/runway/android/di/LocationModule.kt

Kotlin | 31줄

```
1 package com.runway.android.di
2
3 import android.content.Context
4 import com.google.android.gms.location.FusedLocationProviderClient
5 import com.google.android.gms.location.LocationServices
6 import com.runway.android.core.location.DefaultLocationTracker
7 import com.runway.android.core.location.LocationTracker
8 import dagger.Binds
9 import dagger.Module
10 import dagger.Provides
11 import dagger.hilt.InstallIn
12 import dagger.hilt.android.qualifiers.ApplicationContext
13 import dagger.hilt.components.SingletonComponent
14 import javax.inject.Singleton
15
16 @Module
17 @InstallIn(SingletonComponent::class)
18 abstract class LocationModule {
19
20     @Binds
21     @Singleton
22     abstract fun bindLocationTracker(impl: DefaultLocationTracker): LocationTracker
23
24     companion object {
25
26         @Provides
27         @Singleton
28         fun provideFusedLocationClient(
29             @ApplicationContext context: Context,
30         ): FusedLocationProviderClient = LocationServices.getFusedLocationProviderClient(context)
31     }
```

```

1  package com.runway.android.di
2
3  import com.runway.android.BuildConfig
4  import com.runway.android.core.datastore.TokenDataStore
5  import com.runway.android.core.network.AuthInterceptor
6  import com.runway.android.core.network.NetworkConstants
7  import com.runway.android.core.network.TokenAuthenticator
8  import com.runway.android.data.attempt.remote.CourseAttemptApi
9  import com.runway.android.data.auth.remote.AuthApi
10 import com.runway.android.data.course.remote.CourseApi
11 import com.runway.android.data.running.remote.RunningApi
12 import com.runway.android.data.user.remote.UserApi
13 import retrofit2.converter.gson.GsonConverterFactory
14 import dagger.Module
15 import dagger.Provides
16 import dagger.hilt.InstallIn
17 import dagger.hilt.components.SingletonComponent
18 import okhttp3.OkHttpClient
19 import okhttp3.logging.HttpLoggingInterceptor
20 import retrofit2.Retrofit
21 import java.util.concurrent.TimeUnit
22 import javax.inject.Named
23 import javax.inject.Singleton
24
25 /**
26  * 네트워크 레이어 DI 모듈.
27  *
28  * OkHttpClient 두 종류:
29  *   1. @Named("noAuth") - AuthInterceptor/Authenticator 없음. reissue, signup, login 전용.
30  *   2. (기본)          - AuthInterceptor + TokenAuthenticator 포함. 인증 필요 API 전용.
31  *
32  * TokenAuthenticator가 noAuth Retrofit을 주입받아 reissue를 호출하므로
33  * Main OkHttpClient ↔ AuthApi 간 순환 의존성이 발생하지 않는다.
34  */
35 @Module
36 @InstallIn(SingletonComponent::class)
37 object NetworkModule {
38
39     private fun buildLoggingInterceptor(): HttpLoggingInterceptor =
40         HttpLoggingInterceptor().apply {
41             // HEADERS: URL·상태코드·헤더만 기록. BODY는 JWT 응답 페이로드 전체가 로그에 노출됨.
42             level = if (BuildConfig.DEBUG) HttpLoggingInterceptor.Level.HEADERS
43                     else HttpLoggingInterceptor.Level.NONE
44         }
45
46     // — noAuth 클라이언트 (reissue, signup, login) —
47
48     @Provides
49     @Singleton
50     @Named("noAuth")
51     fun provideNoAuthOkHttpClient(): OkHttpClient =
52         OkHttpClient.Builder()
53             .addInterceptor(buildLoggingInterceptor())
54             .connectTimeout(NetworkConstants.CONNECT_TIMEOUT_SECONDS, TimeUnit.SECONDS)
55             .readTimeout(NetworkConstants.READ_TIMEOUT_SECONDS, TimeUnit.SECONDS)
56             .writeTimeout(NetworkConstants.WRITE_TIMEOUT_SECONDS, TimeUnit.SECONDS)
57             .build()
58
59     @Provides
60     @Singleton
61     @Named("noAuth")
62     fun provideNoAuthRetrofit(
63         @Named("noAuth") okHttpClient: OkHttpClient,
64     ): Retrofit =
65         Retrofit.Builder()
66             .baseUrl(NetworkConstants.BASE_URL)
67             .client(okHttpClient)
68             .addConverterFactory(GsonConverterFactory.create())
69             .build()
70
71     // — Interceptor / Authenticator —
72
73     @Provides
74     @Singleton
75     fun provideAuthInterceptor(tokenDataStore: TokenDataStore): AuthInterceptor =
76         AuthInterceptor(tokenDataStore)
77
78     @Provides
79     @Singleton
80     fun provideTokenAuthenticator(
81         tokenDataStore: TokenDataStore,
82         @Named("noAuth") noAuthRetrofit: Retrofit,
83     ): TokenAuthenticator =

```

```

84     TokenAuthenticator(tokenDataStore, noAuthRetrofit)
85
86     // — Main 클라이언트 (인증 필요 API) —
87
88     @Provides
89     @Singleton
90     fun provideOkHttpClient(
91         authInterceptor: AuthInterceptor,
92         tokenAuthenticator: TokenAuthenticator,
93     ): OkHttpClient =
94         OkHttpClient.Builder()
95             .addInterceptor(authInterceptor)
96             .authenticator(tokenAuthenticator)
97             .addInterceptor(buildLoggingInterceptor())
98             .connectTimeout(NetworkConstants.CONNECT_TIMEOUT_SECONDS, TimeUnit.SECONDS)
99             .readTimeout(NetworkConstants.READ_TIMEOUT_SECONDS, TimeUnit.SECONDS)
100             .writeTimeout(NetworkConstants.WRITE_TIMEOUT_SECONDS, TimeUnit.SECONDS)
101             .build()
102
103     @Provides
104     @Singleton
105     fun provideRetrofit(okHttpClient: OkHttpClient): Retrofit =
106         Retrofit.Builder()
107             .baseUrl(NetworkConstants.BASE_URL)
108             .client(okHttpClient)
109             .addConverterFactory(GsonConverterFactory.create())
110             .build()
111
112     // — API 인터페이스 —
113
114     @Provides
115     @Singleton
116     fun provideAuthApi(@Named("noAuth") retrofit: Retrofit): AuthApi =
117         retrofit.create(AuthApi::class.java)
118
119     @Provides
120     @Singleton
121     fun provideRunningApi(retrofit: Retrofit): RunningApi =
122         retrofit.create(RunningApi::class.java)
123
124     @Provides
125     @Singleton
126     fun provideCourseApi(retrofit: Retrofit): CourseApi =
127         retrofit.create(CourseApi::class.java)
128
129     @Provides
130     @Singleton
131     fun provideCourseAttemptApi(retrofit: Retrofit): CourseAttemptApi =
132         retrofit.create(CourseAttemptApi::class.java)
133
134     @Provides
135     @Singleton
136     fun provideUserApi(retrofit: Retrofit): UserApi =
137         retrofit.create(UserApi::class.java)
138 }

```

```
1 package com.runway.android.di
2
3 import android.content.Context
4 import androidx.room.Room
5 import com.runway.android.core.posture.PostureEvaluator
6 import com.runway.android.core.posture.PosturePoseAnalyzer
7 import com.runway.android.core.posture.PostureRuleEngine
8 import com.runway.android.core.posture.local.PostureAnalysisDao
9 import com.runway.android.core.posture.local.PostureDatabase
10 import com.runway.android.core.posture.local.PostureDatabase.Companion.MIGRATION_2_3
11 import com.runway.android.core.posture.local.PostureDatabase.Companion.MIGRATION_3_4
12 import dagger.Module
13 import dagger.Provides
14 import dagger.hilt.InstallIn
15 import dagger.hilt.android.qualifiers.ApplicationContext
16 import dagger.hilt.components.SingletonComponent
17 import javax.inject.Singleton
18
19 @Module
20 @InstallIn(SingletonComponent::class)
21 object PostureModule {
22
23     @Provides
24     @Singleton
25     fun providePostureDatabase(@ApplicationContext context: Context): PostureDatabase =
26         Room.databaseBuilder(context, PostureDatabase::class.java, "posture_database")
27             .addMigrations(MIGRATION_2_3, MIGRATION_3_4)
28             .fallbackToDestructiveMigration()
29             .build()
30
31     @Provides
32     @Singleton
33     fun providePostureAnalysisDao(db: PostureDatabase): PostureAnalysisDao =
34         db.postureAnalysisDao()
35
36     @Provides
37     @Singleton
38     fun providePosturePoseAnalyzer(@ApplicationContext context: Context): PosturePoseAnalyzer =
39         PosturePoseAnalyzer(context)
40
41     @Provides
42     @Singleton
43     fun providePostureEvaluator(engine: PostureRuleEngine): PostureEvaluator = engine
44 }
```

android/app/src/main/java/com/runway/android/di/RepositoryModule.kt

Kotlin | 42줄

```
1 package com.runway.android.di
2
3 import com.runway.android.data.attempt.CourseAttemptRepositoryImpl
4 import com.runway.android.data.auth.AuthRepositoryImpl
5 import com.runway.android.data.course.CourseRepositoryImpl
6 import com.runway.android.data.running.RunningRepositoryImpl
7 import com.runway.android.data.user.UserRepositoryImpl
8 import com.runway.android.domain.attempt.CourseAttemptRepository
9 import com.runway.android.domain.auth.AuthRepository
10 import com.runway.android.domain.course.CourseRepository
11 import com.runway.android.domain.running.RunningRepository
12 import com.runway.android.domain.user.UserRepository
13 import dagger.Binds
14 import dagger.Module
15 import dagger.hilt.InstallIn
16 import dagger.hilt.components.SingletonComponent
17 import javax.inject.Singleton
18
19 @Module
20 @InstallIn(SingletonComponent::class)
21 abstract class RepositoryModule {
22
23     @Binds
24     @Singleton
25     abstract fun bindAuthRepository(impl: AuthRepositoryImpl): AuthRepository
26
27     @Binds
28     @Singleton
29     abstract fun bindRunningRepository(impl: RunningRepositoryImpl): RunningRepository
30
31     @Binds
32     @Singleton
33     abstract fun bindCourseRepository(impl: CourseRepositoryImpl): CourseRepository
34
35     @Binds
36     @Singleton
37     abstract fun bindCourseAttemptRepository(impl: CourseAttemptRepositoryImpl): CourseAttemptRepository
38
39     @Binds
40     @Singleton
41     abstract fun bindUserRepository(impl: UserRepositoryImpl): UserRepository
42 }
```

android/app/src/main/java/com/runway/android/di/TrackingModule.kt

Kotlin | 18줄

```
1 package com.runway.android.di
2
3 import com.runway.android.core.cadence.CadenceTracker
4 import com.runway.android.core.cadence.PhoneStepCadenceTracker
5 import dagger.Binds
6 import dagger.Module
7 import dagger.hilt.InstallIn
8 import dagger.hilt.components.SingletonComponent
9 import javax.inject.Singleton
10
11 @Module
12 @InstallIn(SingletonComponent::class)
13 abstract class TrackingModule {
14
15     @Binds
16     @Singleton
17     abstract fun bindCadenceTracker(impl: PhoneStepCadenceTracker): CadenceTracker
18 }
```

android/app/src/main/java/com/runway/android/domain/attempt/CourseAttemptRepository.kt

Kotlin | 38줄

```
1 package com.runway.android.domain.attempt
2
3 import com.runway.android.core.result.NetworkResult
4 import com.runway.android.data.attempt.model.AbandonAttemptRequest
5 import com.runway.android.data.attempt.model.AbandonAttemptResponse
6 import com.runway.android.data.attempt.model.FinishAttemptRequest
7 import com.runway.android.data.attempt.model.FinishAttemptResponse
8 import com.runway.android.data.attempt.model.LeaderboardResponse
9 import com.runway.android.data.attempt.model.MyBestAttemptResponse
10 import com.runway.android.data.attempt.model.StartAttemptRequest
11 import com.runway.android.data.attempt.model.StartAttemptResponse
12
13 interface CourseAttemptRepository {
14
15     suspend fun startAttempt(
16         courseId: String,
17         request: StartAttemptRequest,
18     ): NetworkResult<StartAttemptResponse>
19
20     suspend fun finishAttempt(
21         attemptId: String,
22         request: FinishAttemptRequest,
23     ): NetworkResult<FinishAttemptResponse>
24
25     suspend fun abandonAttempt(
26         attemptId: String,
27         request: AbandonAttemptRequest,
28     ): NetworkResult<AbandonAttemptResponse>
29
30     suspend fun getLeaderboard(
31         courseId: String,
32         page: Int = 0,
33         size: Int = 50,
34         sortBy: String = "fastest_time",
35     ): NetworkResult<LeaderboardResponse>
36
37     suspend fun getMyBestAttempt(courseId: String): NetworkResult<MyBestAttemptResponse>
38 }
```

android/app/src/main/java/com/runway/android/domain/auth/AuthRepository.kt

Kotlin | 38줄

```
1 package com.runway.android.domain.auth
2
3 import com.runway.android.core.result.NetworkResult
4 import com.runway.android.data.auth.model.LoginResponse
5 import com.runway.android.data.auth.model.ReissueResponse
6 import com.runway.android.data.auth.model.SignupResponse
7 import kotlinx.coroutines.flow.Flow
8
9 interface AuthRepository {
10
11     /** 회원가입. 성공 시 사용자 기본 정보 반환. */
12     suspend fun signup(
13         email: String,
14         password: String,
15         nickname: String,
16     ): NetworkResult<SignupResponse>
17
18     /**
19      * 로그인. 성공 시 accessToken + refreshToken을 DataStore에 저장하고 LoginResponse를 반환.
20      * ViewModel에서 저장 여부를 신경 쓰지 않아도 된다.
21      */
22     suspend fun login(email: String, password: String): NetworkResult<LoginResponse>
23
24     /**
25      * 로그아웃. DataStore의 토큰을 삭제한다.
26      * 서버 응답이 실패해도 로컬 토큰은 항상 삭제한다.
27      */
28     suspend fun logout(): NetworkResult<Unit>
29
30     /**
31      * Access Token 재발급. 성공 시 새 토큰을 DataStore에 저장.
32      * TokenAuthenticator가 자동으로 호출하므로 일반적으로 직접 호출할 필요 없음.
33      */
34     suspend fun reissue(): NetworkResult<ReissueResponse>
35
36     /** 현재 로그인 상태를 관찰한다. SplashScreen → 자동 로그인 분기에 사용. */
37     fun isLoggedInInFlow(): Flow<Boolean>
38 }
```



```
1 package com.runway.android.domain.course
2
3 import com.runway.android.core.model.PageResponse
4 import com.runway.android.core.result.NetworkResult
5 import com.runway.android.data.course.model.CourseDetailResponse
6 import com.runway.android.data.course.model.CoursePointsResponse
7 import com.runway.android.data.course.model.CourseRatingRequest
8 import com.runway.android.data.course.model.CourseRatingResponse
9 import com.runway.android.data.course.model.CourseReportRequest
10 import com.runway.android.data.course.model.CourseReportResponse
11 import com.runway.android.data.course.model.CourseResponse
12 import com.runway.android.data.course.model.CreateCourseFromRunRequest
13 import com.runway.android.data.course.model.NearbyCourseItem
14 import com.runway.android.data.course.model.CourseStatusResponse
15 import com.runway.android.data.course.model.ParticipatedCourseItem
16 import com.runway.android.data.course.model.PublishCourseRequest
17
18 interface CourseRepository {
19
20     suspend fun createCourseFromRun(
21         runId: String,
22         request: CreateCourseFromRunRequest,
23     ): NetworkResult<CourseResponse>
24
25     suspend fun getNearbyCourses(
26         latitude: Double,
27         longitude: Double,
28         radiusMeters: Int = 3000,
29         minDistanceMeters: Double? = null,
30         maxDistanceMeters: Double? = null,
31         isLoop: Boolean? = null,
32         keyword: String? = null,
33         includeRoutePoints: Boolean = true,
34         includeExactStartPoint: Boolean = false,
35         page: Int = 0,
36         size: Int = 20,
37     ): NetworkResult<PageResponse<NearbyCourseItem>>
38
39     suspend fun getMyCourses(
40         status: String? = null,
41         page: Int = 0,
42         size: Int = 20,
43     ): NetworkResult<PageResponse<CourseResponse>>
44
45     suspend fun getCourseDetail(courseId: String): NetworkResult<CourseDetailResponse>
46
47     suspend fun getCoursePoints(courseId: String): NetworkResult<CoursePointsResponse>
48
49     suspend fun reportCourse(courseId: String, request: CourseReportRequest): NetworkResult<CourseReportResponse>
50
51     suspend fun rateCourse(courseId: String, request: CourseRatingRequest): NetworkResult<CourseRatingResponse>
52
53     suspend fun getFavoriteCourses(page: Int = 0, size: Int = 20): NetworkResult<PageResponse<CourseResponse>>
54
55     suspend fun getParticipatedCourses(page: Int = 0, size: Int = 20): NetworkResult<PageResponse<ParticipatedCourseItem>>
56
57     suspend fun addFavorite(courseId: String): NetworkResult<Unit>
58
59     suspend fun removeFavorite(courseId: String): NetworkResult<Unit>
60
61     suspend fun publishCourse(courseId: String, request: PublishCourseRequest): NetworkResult<CourseStatusResponse>
62
63     suspend fun archiveCourse(courseId: String): NetworkResult<CourseStatusResponse>
64
65     suspend fun deleteDraftCourse(courseId: String): NetworkResult<Unit>
66 }
```

android/app/src/main/java/com/runway/android/domain/running/RunningRepository.kt

Kotlin | 30行

```
1 package com.runway.android.domain.running
2
3 import com.runway.android.core.model.PageResponse
4 import com.runway.android.core.result.NetworkResult
5 import com.runway.android.data.running.model.FinishRunRequest
6 import com.runway.android.data.running.model.FinishRunResponse
7 import com.runway.android.data.running.model.PersonalRecordsResponse
8 import com.runway.android.data.running.model.RunDetailResponse
9 import com.runway.android.data.running.model.RunningStatsResponse
10 import com.runway.android.data.running.model.RunStatusResponse
11 import com.runway.android.data.running.model.RunSummaryResponse
12 import com.runway.android.data.running.model.SavePointsRequest
13 import com.runway.android.data.running.model.SavePointsResponse
14 import com.runway.android.data.running.model.StartRunRequest
15 import com.runway.android.data.running.model.StartRunResponse
16
17 interface RunningRepository {
18     suspend fun startRun(request: StartRunRequest): NetworkResult<StartRunResponse>
19     suspend fun savePoints(runId: String, request: SavePointsRequest): NetworkResult<SavePointsResponse>
20     suspend fun pauseRun(runId: String): NetworkResult<RunStatusResponse>
21     suspend fun resumeRun(runId: String): NetworkResult<RunStatusResponse>
22     suspend fun finishRun(runId: String, request: FinishRunRequest): NetworkResult<FinishRunResponse>
23     suspend fun abandonRun(runId: String): NetworkResult<RunStatusResponse>
24     suspend fun getMyRuns(page: Int = 0, size: Int = 20): NetworkResult<PageResponse<RunSummaryResponse>>
25     suspend fun getRunDetail(runId: String): NetworkResult<RunDetailResponse>
26     suspend fun getPersonalRecords(): NetworkResult<PersonalRecordsResponse>
27     suspend fun getRunningStats(period: String = "monthly"): NetworkResult<RunningStatsResponse>
28     suspend fun deleteRun(runId: String): NetworkResult<Unit>
29     suspend fun trimRun(runId: String, targetDistanceMeters: Double): NetworkResult<Unit>
30 }
```

android/app/src/main/java/com/runway/android/domain/user/UserRepository.kt

Kotlin | 14行

```
1 package com.runway.android.domain.user
2
3 import android.net.Uri
4 import com.runway.android.core.result.NetworkResult
5 import com.runway.android.data.user.model.AchievementsResponse
6 import com.runway.android.data.user.model.UpdateProfileRequest
7 import com.runway.android.data.user.model.UserProfileResponse
8
9 interface UserRepository {
10     suspend fun getMe(): NetworkResult<UserProfileResponse>
11     suspend fun updateMe(request: UpdateProfileRequest): NetworkResult<UserProfileResponse>
12     suspend fun uploadProfileImage(uri: Uri): NetworkResult<String>
13     suspend fun getAchievements(): NetworkResult<AchievementsResponse>
14 }
```

```
1 package com.runway.android.ui
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.viewModelScope
5 import com.runway.android.core.datastore.OnboardingDataStore
6 import com.runway.android.core.datastore.ThemeDataStore
7 import com.runway.android.core.wear.WatchNavRepository
8 import com.runway.android.domain.auth.AuthRepository
9 import com.runway.android.ui.theme.AccentColor
10 import com.runway.android.ui.theme.ThemeMode
11 import dagger.hilt.android.lifecycle.HiltViewModel
12 import kotlinx.coroutines.flow.SharingStarted
13 import kotlinx.coroutines.flow.StateFlow
14 import kotlinx.coroutines.flow.stateIn
15 import kotlinx.coroutines.launch
16 import javax.inject.Inject
17
18 @HiltViewModel
19 class MainViewModel @Inject constructor(
20     authRepository: AuthRepository,
21     onboardingDataStore: OnboardingDataStore,
22     private val themeDataStore: ThemeDataStore,
23     private val watchNavRepository: WatchNavRepository,
24 ) : ViewModel() {
25
26     val isLoggedIn: StateFlow<Boolean?> = authRepository.isLoggedInFlow()
27         .stateIn(viewModelScope, SharingStarted.Eagerly, null)
28
29     val isOnboardingCompleted: StateFlow<Boolean?> = onboardingDataStore.isCompletedFlow
30         .stateIn(viewModelScope, SharingStarted.Eagerly, null)
31
32     val themeMode: StateFlow<ThemeMode> = themeDataStore.themeModeFlow
33         .stateIn(viewModelScope, SharingStarted.Eagerly, ThemeMode.DARK)
34
35     val accentColor: StateFlow<AccentColor> = themeDataStore.accentColorFlow
36         .stateIn(viewModelScope, SharingStarted.Eagerly, AccentColor.GREEN)
37
38     val watchPendingCourseId = watchNavRepository.pendingCourseId
39     val watchPendingRunId = watchNavRepository.pendingRunId
40
41     fun consumeWatchPendingCourse() = watchNavRepository.consumeCourse()
42     fun consumeWatchPendingRun() = watchNavRepository.consumeRun()
43
44     fun setThemeMode(mode: ThemeMode) {
45         viewModelScope.launch { themeDataStore.setThemeMode(mode) }
46     }
47 }
```

```

1  package com.runway.android.ui.achievements
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.layout.Arrangement
5  import androidx.compose.foundation.layout.Box
6  import androidx.compose.foundation.layout.Column
7  import androidx.compose.foundation.layout.PaddingValues
8  import androidx.compose.foundation.layout.Spacer
9  import androidx.compose.foundation.layout.fillMaxSize
10 import androidx.compose.foundation.layout.height
11 import androidx.compose.foundation.layout.padding
12 import androidx.compose.foundation.layout.size
13 import androidx.compose.foundation.lazy.LazyColumn
14 import androidx.compose.foundation.lazy.items
15 import androidx.compose.material.icons.Icons
16 import androidx.compose.material.icons.automirrored.filled.ArrowBack
17 import androidx.compose.material3.CircularProgressIndicator
18 import androidx.compose.material3.ExperimentalMaterial3Api
19 import androidx.compose.material3.Icon
20 import androidx.compose.material3.IconButton
21 import androidx.compose.material3.MaterialTheme
22 import androidx.compose.material3.Text
23 import androidx.compose.material3.TextButton
24 import androidx.compose.material3.TopAppBar
25 import androidx.compose.material3.TopAppBarDefaults
26 import androidx.compose.material3.pulltorefresh.PullToRefreshBox
27 import androidx.compose.runtime.Composable
28 import androidx.compose.ui.Alignment
29 import androidx.compose.ui.Modifier
30 import androidx.compose.ui.unit.dp
31 import androidx.hilt.navigation.compose.hiltViewModel
32 import com.runway.android.ui.components.AchievementCard
33
34 @OptIn(ExperimentalMaterial3Api::class)
35 @Composable
36 fun AchievementsScreen(
37     onBack: () -> Unit,
38     viewModel: AchievementsViewModel = hiltViewModel(),
39 ) {
40     Column(
41         modifier = Modifier
42             .fillMaxSize()
43             .background(MaterialTheme.colorScheme.background),
44     ) {
45         TopAppBar(
46             title = {
47                 Text(
48                     text = "업적",
49                     style = MaterialTheme.typography.titleLarge,
50                     color = MaterialTheme.colorScheme.onBackground,
51                 )
52             },
53             navigationIcon = {
54                 IconButton(onClick = onBack) {
55                     Icon(
56                         imageVector = Icons.AutoMirrored.Filled.ArrowBack,
57                         contentDescription = "뒤로",
58                         tint = MaterialTheme.colorScheme.onBackground,
59                     )
60                 }
61             },
62             colors = TopAppBarDefaults.topAppBarColors(
63                 containerColor = MaterialTheme.colorScheme.background,
64             ),
65         )
66
67         PullToRefreshBox(
68             isRefreshing = viewModel.isRefreshing,
69             onRefresh = viewModel::refresh,
70             modifier = Modifier.fillMaxSize(),
71         ) {
72             when {
73                 viewModel.isLoading -> {
74                     Box(
75                         modifier = Modifier.fillMaxSize(),
76                         contentAlignment = Alignment.Center,
77                     ) {
78                         CircularProgressIndicator(
79                             modifier = Modifier.size(40.dp),
80                             color = MaterialTheme.colorScheme.primary,
81                         )
82                     }
83                 }

```

```

84
85     viewModel.errorMessage != null -> {
86         Box(
87             modifier = Modifier.fillMaxSize(),
88             contentAlignment = Alignment.Center,
89         ) {
90             Column(horizontalAlignment = Alignment.CenterHorizontally) {
91                 Text(
92                     text = viewModel.errorMessage!!,
93                     style = MaterialTheme.typography.bodyMedium,
94                     color = MaterialTheme.colorScheme.onSurfaceVariant,
95                 )
96                 Spacer(modifier = Modifier.height(8.dp))
97                 TextButton(onClick = viewModel::retry) {
98                     Text("다시 시도")
99                 }
100             }
101         }
102     }
103
104     viewModel.items.isEmpty() -> {
105         Box(
106             modifier = Modifier.fillMaxSize(),
107             contentAlignment = Alignment.Center,
108         ) {
109             Text(
110                 text = "러닝을 시작하면 업적을 달성할 수 있어요.",
111                 style = MaterialTheme.typography.bodyMedium,
112                 color = MaterialTheme.colorScheme.onSurfaceVariant,
113                 modifier = Modifier.padding(horizontal = 32.dp),
114             )
115         }
116     }
117
118     else -> {
119         val unlocked = viewModel.items.filter { it.unlocked }
120         val locked = viewModel.items.filter { !it.unlocked }
121
122         LazyColumn(
123             contentPadding = PaddingValues(horizontal = 16.dp, vertical = 8.dp),
124             verticalArrangement = Arrangement.spacedBy(10.dp),
125         ) {
126             if (unlocked.isNotEmpty()) {
127                 item {
128                     Text(
129                         text = "달성한 업적 (${unlocked.size})",
130                         style = MaterialTheme.typography.labelLarge,
131                         color = MaterialTheme.colorScheme.primary,
132                         modifier = Modifier.padding(vertical = 4.dp),
133                     )
134                 }
135                 items(unlocked, key = { it.code }) { item ->
136                     AchievementCard(item = item)
137                 }
138             }
139
140             if (locked.isNotEmpty()) {
141                 item {
142                     Spacer(modifier = Modifier.height(4.dp))
143                     Text(
144                         text = "미달성 업적 (${locked.size})",
145                         style = MaterialTheme.typography.labelLarge,
146                         color = MaterialTheme.colorScheme.onSurfaceVariant,
147                         modifier = Modifier.padding(vertical = 4.dp),
148                     )
149                 }
150                 items(locked, key = { it.code }) { item ->
151                     AchievementCard(item = item)
152                 }
153             }
154
155             item { Spacer(modifier = Modifier.height(16.dp)) }
156         }
157     }
158 }
159 }
160 }
161 }

```

```
1 package com.runway.android.ui.achievements
2
3 import androidx.compose.runtime.getValue
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.compose.runtime.setValue
6 import androidx.lifecycle.ViewModel
7 import androidx.lifecycle.ViewModelScope
8 import com.runway.android.core.result.NetworkResult
9 import com.runway.android.data.user.model.AchievementItem
10 import com.runway.android.domain.user.UserRepository
11 import dagger.hilt.android.lifecycle.HiltViewModel
12 import kotlinx.coroutines.launch
13 import javax.inject.Inject
14
15 @HiltViewModel
16 class AchievementsViewModel @Inject constructor(
17     private val userRepository: UserRepository,
18 ) : ViewModel() {
19
20     var items by mutableStateOf<List<AchievementItem>>(emptyList())
21     private set
22     var isLoading by mutableStateOf(false)
23     private set
24     var isRefreshing by mutableStateOf(false)
25     private set
26     var errorMessage by mutableStateOf<String?>(null)
27     private set
28
29     init {
30         load()
31     }
32
33     fun retry() = load()
34
35     fun refresh() {
36         viewModelScope.launch {
37             isRefreshing = true
38             doLoad()
39             isRefreshing = false
40         }
41     }
42
43     private fun load() {
44         viewModelScope.launch { doLoad() }
45     }
46
47     private suspend fun doLoad() {
48         isLoading = true
49         errorMessage = null
50         when (val result = userRepository.getAchievements()) {
51             is NetworkResult.Success -> items = result.data.items
52             is NetworkResult.ApiError -> errorMessage = result.message
53             is NetworkResult.NetworkError -> errorMessage = "네트워크 연결을 확인해 주세요."
54         }
55         isLoading = false
56     }
57 }
```

```
1 package com.runway.android.ui.attempt
2
3 import android.Manifest
4 import android.app.Activity
5 import android.content.pm.PackageManager
6 import android.os.Build
7 import androidx.activity.compose.BackHandler
8 import androidx.activity.compose.rememberLauncherForActivityResult
9 import androidx.activity.result.contract.ActivityResultContracts
10 import androidx.compose.animation.AnimatedVisibility
11 import androidx.compose.animation.EnterTransition
12 import androidx.compose.animation.animateColorAsState
13 import androidx.compose.animation.core.FastOutSlowInEasing
14 import androidx.compose.animation.core.LinearEasing
15 import androidx.compose.animation.core.RepeatMode
16 import androidx.compose.animation.core.animateFloat
17 import androidx.compose.animation.core.animateFloatAsState
18 import androidx.compose.animation.core.infiniteRepeatable
19 import androidx.compose.animation.core.rememberInfiniteTransition
20 import androidx.compose.animation.core.tween
21 import androidx.compose.animation.fadeOut
22 import androidx.compose.foundation.BorderStroke
23 import androidx.compose.foundation.background
24 import androidx.compose.foundation.border
25 import androidx.compose.foundation.layout.Arrangement
26 import androidx.compose.foundation.layout.Box
27 import androidx.compose.foundation.layout.Column
28 import androidx.compose.foundation.layout.IntrinsicSize
29 import androidx.compose.foundation.layout.Row
30 import androidx.compose.foundation.layout.Spacer
31 import androidx.compose.foundation.layout.fillMaxSize
32 import androidx.compose.foundation.layout.fillMaxWidth
33 import androidx.compose.foundation.layout.height
34 import androidx.compose.foundation.layout.padding
35 import androidx.compose.foundation.layout.size
36 import androidx.compose.foundation.layout.statusBarsPadding
37 import androidx.compose.foundation.layout.width
38 import androidx.compose.foundation.shape.CircleShape
39 import androidx.compose.foundation.shape.RoundedCornerShape
40 import androidx.compose.material.icons.Icons
41 import androidx.compose.material.icons.filled.Close
42 import androidx.compose.material.icons.filled.Pause
43 import androidx.compose.material.icons.filled.PlayArrow
44 import androidx.compose.material.icons.filled.Stop
45 import androidx.compose.material.icons.filled.Warning
46 import androidx.compose.material3.CircularProgressIndicator
47 import androidx.compose.material3.ExperimentalMaterial3Api
48 import androidx.compose.material3.ModalBottomSheet
49 import androidx.compose.material3.rememberModalBottomSheetState
50 import androidx.compose.material3.Icon
51 import androidx.compose.material3.LinearProgressIndicator
52 import androidx.compose.material3.MaterialTheme
53 import androidx.compose.material3.Surface
54 import androidx.compose.material3.Text
55 import androidx.compose.material3.TextButton
56 import androidx.compose.material3.VerticalDivider
57 import androidx.compose.runtime.Composable
58 import androidx.compose.runtime.LaunchedEffect
59 import androidx.compose.runtime.getValue
60 import androidx.compose.runtime.mutableStateOf
61 import androidx.compose.runtime.remember
62 import androidx.compose.runtime.setValue
63 import androidx.compose.ui.Alignment
64 import androidx.compose.ui.Modifier
65 import androidx.compose.ui.draw.clip
66 import androidx.compose.ui.graphics.Color
67 import androidx.compose.ui.platform.LocalContext
68 import androidx.compose.ui.text.font.FontWeight
69 import androidx.compose.ui.text.style.TextAlign
70 import androidx.compose.ui.unit.dp
71 import androidx.core.content.ContextCompat
72 import androidx.hilt.navigation.compose.hiltViewModel
73 import com.runway.android.core.location.GpsStatus
74 import com.runway.android.ui.components.BatteryOptimizationCard
75 import com.runway.android.ui.components.ConfettiCanvas
76 import com.runway.android.ui.running.RunningCountdownOverlay
77 import com.runway.android.ui.components.LocationPermissionCard
78 import com.runway.android.ui.components.RouteMapView
79 import com.runway.android.ui.components.RunMetricCard
80 import com.runway.android.ui.components.RunningControlButton
81 import com.runway.android.ui.components.rememberBatteryOptimizationIgnored
82
83 @OptIn(ExperimentalMaterial3Api::class)
```

```

84 @Composable
85 fun CourseAttemptTrackingScreen(
86     onNavigateToLeaderboard: (courseId: String, isPR: Boolean, previousBestSeconds: Int?, improvementSeconds: Int?) -> Unit,
87     onNavigateBack: () -> Unit,
88     viewModel: CourseAttemptTrackingViewModel = hiltViewModel(),
89 ) {
90     LaunchedEffect(Unit) {
91         viewModel.navEvent.collect { event ->
92             when (event) {
93                 is AttemptNavEvent.NavigateToLeaderboard -> onNavigateToLeaderboard(
94                     event.courseId, event.isPR, event.previousBestSeconds, event.improvementSeconds,
95                 )
96                 is AttemptNavEvent.NavigateBack -> onNavigateBack()
97             }
98         }
99     }
100
101     val context = LocalContext.current
102     var permissionDeniedPermanently by remember { mutableStateOf(false) }
103     var showStopSheet by remember { mutableStateOf(false) }
104     var showBatteryCard by remember { mutableStateOf(true) }
105     val isBatteryOptimizationIgnored = rememberBatteryOptimizationIgnored()
106
107     val permissionLauncher = rememberLauncherForActivityResult(
108         ActivityResultContracts.RequestMultiplePermissions()
109     ) { permissions ->
110         val locationGranted = permissions[Manifest.permission.ACCESS_FINE_LOCATION] == true ||
111             permissions[Manifest.permission.ACCESS_COARSE_LOCATION] == true
112         if (locationGranted) {
113             viewModel.startTracking()
114         } else {
115             val activity = context as? Activity
116             val shouldShow = activity?.shouldShowRequestPermissionRationale(
117                 Manifest.permission.ACCESS_FINE_LOCATION
118             ) ?: false
119             permissionDeniedPermanently = !shouldShow
120         }
121     }
122
123     var countdownDone by remember { mutableStateOf(false) }
124
125     LaunchedEffect(countdownDone) {
126         if (!countdownDone) return@LaunchedEffect
127         val hasFine = ContextCompat.checkSelfPermission(
128             context, Manifest.permission.ACCESS_FINE_LOCATION
129         ) == PackageManager.PERMISSION_GRANTED
130         val hasCoarse = ContextCompat.checkSelfPermission(
131             context, Manifest.permission.ACCESS_COARSE_LOCATION
132         ) == PackageManager.PERMISSION_GRANTED
133         val hasActivityRecognition = Build.VERSION.SDK_INT < Build.VERSION_CODES.Q ||
134             ContextCompat.checkSelfPermission(
135                 context,
136                 Manifest.permission.ACTIVITY_RECOGNITION,
137             ) == PackageManager.PERMISSION_GRANTED
138
139         if ((hasFine || hasCoarse) && hasActivityRecognition) {
140             viewModel.startTracking()
141         } else {
142             val permissions = buildList {
143                 add(Manifest.permission.ACCESS_FINE_LOCATION)
144                 add(Manifest.permission.ACCESS_COARSE_LOCATION)
145                 if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
146                     add(Manifest.permission.ACTIVITY_RECOGNITION)
147                 }
148                 if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
149                     add(Manifest.permission.POST_NOTIFICATIONS)
150                 }
151             }.toTypedArray()
152             permissionLauncher.launch(permissions)
153         }
154     }
155
156     BackHandler(enabled = !viewModel.isFinishing && !viewModel.isAbandoning) {
157         if (!countdownDone) onNavigateBack() else showStopSheet = true
158     }
159
160     // 완주 시 StopSheet 자동으로 열기
161     LaunchedEffect(viewModel.isCourseCompleted) {
162         if (viewModel.isCourseCompleted && !showStopSheet) {
163             kotlinx.coroutines.delay(1_500)
164             showStopSheet = true
165         }
166     }
167
168     if (showStopSheet) {
169         StopConfirmSheet(

```



```

170         progressPercent = viewModel.courseProgressPercent,
171         timerText = viewModel.timerText,
172         distanceText = viewModel.distanceText,
173         paceText = viewModel.paceText,
174         isFinishing = viewModel.isFinishing,
175         isAbandoning = viewModel.isAbandoning,
176         onFinish = { showStopSheet = false; viewModel.finish() },
177         onAbandon = { showStopSheet = false; viewModel.abandon() },
178         onContinue = { showStopSheet = false },
179     )
180 }
181
182 Box(modifier = Modifier.fillMaxSize()) {
183     Column(
184         modifier = Modifier
185             .fillMaxSize()
186             .background(MaterialTheme.colorScheme.background)
187             .statusBarsPadding(),
188     ) {
189         // — 상단: 코스 도전 레이블 —
190         Row(
191             modifier = Modifier
192                 .fillMaxWidth()
193                 .padding(horizontal = 20.dp, vertical = 12.dp),
194             horizontalArrangement = Arrangement.SpaceBetween,
195             verticalAlignment = Alignment.CenterVertically,
196         ) {
197             CoursePill()
198             Text(
199                 text = when {
200                     viewModel.gpsStatus == GpsStatus.PERMISSION_REQUIRED -> "위치 권한 필요"
201                     viewModel.gpsStatus == GpsStatus.WAITING_FOR_FIX -> "GPS 신호 수신 중..."
202                     viewModel.isAutoPaused -> "자동 일시정지 중"
203                     viewModel.isPaused -> "일시정지"
204                     else -> "GPS · Active"
205                 },
206                 style = MaterialTheme.typography.labelLarge,
207                 color = MaterialTheme.colorScheme.onSurfaceVariant,
208             )
209         }
210
211         Spacer(modifier = Modifier.height(28.dp))
212
213         // — 타이머 —
214         Column(
215             modifier = Modifier.fillMaxWidth(),
216             horizontalAlignment = Alignment.CenterHorizontally,
217         ) {
218             Text(
219                 text = "TIME",
220                 style = MaterialTheme.typography.labelMedium,
221                 color = MaterialTheme.colorScheme.onSurfaceVariant,
222             )
223             Spacer(modifier = Modifier.height(4.dp))
224             Text(
225                 text = viewModel.timerText,
226                 style = MaterialTheme.typography.displayLarge,
227                 color = MaterialTheme.colorScheme.onBackground,
228                 textAlign = TextAlign.Center,
229             )
230         }
231
232         Spacer(modifier = Modifier.height(24.dp))
233
234         // — 지표 그리드 —
235         Surface(
236             modifier = Modifier
237                 .fillMaxWidth()
238                 .padding(horizontal = 20.dp),
239             shape = MaterialTheme.shapes.extraLarge,
240             color = MaterialTheme.colorScheme.surface,
241             border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
242         ) {
243             Row(
244                 modifier = Modifier
245                     .fillMaxWidth()
246                     .height(IntrinsicSize.Min),
247             ) {
248                 RunMetricCard(
249                     label = "DISTANCE",
250                     value = viewModel.distanceText,
251                     unit = "km",
252                     modifier = Modifier.weight(1f),
253                 )
254                 VerticalDivider(color = MaterialTheme.colorScheme.outline)
255                 RunMetricCard(

```

```

256         label = "PACE",
257         value = viewModel.paceText,
258         unit = "/km",
259         modifier = Modifier.weight(1f),
260     )
261     VerticalDivider(color = MaterialTheme.colorScheme.outline)
262     RunMetricCard(
263         label = "SPEED",
264         value = viewModel.speedText,
265         unit = "km/h",
266         modifier = Modifier.weight(1f),
267     )
268 }
269 }
270
271 Spacer(modifier = Modifier.height(12.dp))
272
273 // — 코스 이탈 경고 배너 —
274 if (viewModel.showDeviationWarning) {
275     Surface(
276         modifier = Modifier
277             .fillMaxWidth()
278             .padding(horizontal = 20.dp),
279         shape = MaterialTheme.shapes.extraLarge,
280         color = MaterialTheme.colorScheme.errorContainer,
281         border = BorderStroke(1.dp, MaterialTheme.colorScheme.error.copy(alpha = 0.5f)),
282         onClick = viewModel::dismissDeviationWarning,
283     ) {
284         Row(
285             modifier = Modifier.padding(horizontal = 16.dp, vertical = 10.dp),
286             verticalAlignment = Alignment.CenterVertically,
287             horizontalArrangement = Arrangement.spacedBy(10.dp),
288         ) {
289             Icon(
290                 imageVector = Icons.Filled.Warning,
291                 contentDescription = null,
292                 tint = MaterialTheme.colorScheme.error,
293                 modifier = Modifier.size(18.dp),
294             )
295             Text(
296                 text = "코스를 이탈했습니다 · 일시정지됨. 코스로 돌아오면 자동 재개됩니다.",
297                 style = MaterialTheme.typography.bodySmall,
298                 color = MaterialTheme.colorScheme.onErrorContainer,
299                 modifier = Modifier.weight(1f),
300             )
301             Icon(
302                 imageVector = Icons.Filled.Close,
303                 contentDescription = "닫기",
304                 tint = MaterialTheme.colorScheme.onErrorContainer.copy(alpha = 0.6f),
305                 modifier = Modifier.size(14.dp),
306             )
307         }
308     }
309     Spacer(modifier = Modifier.height(8.dp))
310 }
311
312 // — Milestone banner —
313 viewModel.milestoneMessage?.let { msg ->
314     Surface(
315         modifier = Modifier
316             .fillMaxWidth()
317             .padding(horizontal = 20.dp),
318         shape = MaterialTheme.shapes.extraLarge,
319         color = MaterialTheme.colorScheme.primaryContainer,
320         border = BorderStroke(1.dp, MaterialTheme.colorScheme.primary.copy(alpha = 0.4f)),
321     ) {
322         Text(
323             text = msg,
324             style = MaterialTheme.typography.bodyMedium,
325             color = MaterialTheme.colorScheme.onPrimaryContainer,
326             textAlign = TextAlign.Center,
327             modifier = Modifier.padding(horizontal = 16.dp, vertical = 10.dp),
328         )
329     }
330     Spacer(modifier = Modifier.height(8.dp))
331 }
332
333 // — Battery optimization card —
334 if (showBatteryCard && !isBatteryOptimizationIgnored && viewModel.gpsStatus == GpsStatus.ACTIVE) {
335     BatteryOptimizationCard(
336         onDismiss = { showBatteryCard = false },
337         modifier = Modifier.padding(horizontal = 20.dp),
338     )
339     Spacer(modifier = Modifier.height(8.dp))
340 }
341

```

```

342 // ——— 경로 미리보기 / 권한 카드 ———
343 if (viewModel.gpsStatus == GpsStatus.PERMISSION_REQUIRED) {
344     Box(
345         modifier = Modifier
346             .weight(1f)
347             .fillMaxWidth(),
348         contentAlignment = Alignment.Center,
349     ) {
350         LocationPermissionCard(
351             isPermanentlyDenied = permissionDeniedPermanently,
352             onRequestPermission = {
353                 permissionLauncher.launch(
354                     arrayOf(
355                         Manifest.permission.ACCESS_FINE_LOCATION,
356                         Manifest.permission.ACCESS_COARSE_LOCATION,
357                     )
358                 )
359             },
360         )
361     }
362 } else {
363     CourseMapPanel(
364         points = viewModel.coursePoints,
365         currentLocation = viewModel.currentLocationPoint,
366         trackStatus = viewModel.trackStatus,
367         nearestDistanceMeters = viewModel.nearestCourseDistanceMeters,
368         progressPercent = viewModel.courseProgressPercent,
369         remainingDistanceText = viewModel.remainingDistanceText,
370         modifier = Modifier
371             .weight(1f)
372             .fillMaxWidth()
373             .padding(horizontal = 20.dp),
374     )
375 }
376
377 Spacer(modifier = Modifier.height(24.dp))
378
379 // ——— 컨트롤 버튼 ———
380 Row(
381     modifier = Modifier.fillMaxWidth(),
382     horizontalArrangement = Arrangement.Center,
383     verticalAlignment = Alignment.CenterVertically,
384 ) {
385     if (viewModel.isFinishing || viewModel.isAbandoning) {
386         CircularProgressIndicator(
387             modifier = Modifier.size(60.dp),
388             color = MaterialTheme.colorScheme.error,
389             strokeWidth = 3.dp,
390         )
391     } else {
392         RunningControlButton(
393             icon = Icons.Filled.Stop,
394             onClick = { showStopSheet = true },
395             size = 60.dp,
396             containerColor = MaterialTheme.colorScheme.surface,
397             contentColor = MaterialTheme.colorScheme.error,
398             border = BorderStroke(1.dp, MaterialTheme.colorScheme.error.copy(alpha = 0.5f)),
399         )
400     }
401
402     Spacer(modifier = Modifier.width(24.dp))
403
404     val isRunning = !viewModel.isPaused && !viewModel.isAutoPaused
405     if (isRunning) {
406         RunningControlButton(
407             icon = Icons.Filled.Pause,
408             onClick = viewModel::pause,
409             size = 84.dp,
410             containerColor = MaterialTheme.colorScheme.primary,
411             contentColor = MaterialTheme.colorScheme.onPrimary,
412         )
413     } else {
414         RunningControlButton(
415             icon = Icons.Filled.PlayArrow,
416             onClick = viewModel::resume,
417             size = 84.dp,
418             containerColor = MaterialTheme.colorScheme.primary,
419             contentColor = MaterialTheme.colorScheme.onPrimary,
420         )
421     }
422 }
423
424 if (viewModel.finishError != null) {
425     Spacer(modifier = Modifier.height(12.dp))
426     Text(
427         text = viewModel.finishError!!,

```

```

428         style = MaterialTheme.typography.bodySmall,
429         color = MaterialTheme.colorScheme.error,
430         textAlign = TextAlign.Center,
431         modifier = Modifier
432             .fillMaxWidth()
433             .padding(horizontal = 32.dp),
434     )
435 }
436
437     Spacer(modifier = Modifier.height(44.dp))
438 }
439
440     AnimatedVisibility(
441         visible = !countdownDone,
442         enter = EnterTransition.None,
443         exit = fadeOut(tween(350)),
444     ) {
445         RunningCountdownOverlay(onFinished = { countdownDone = true })
446     }
447
448     if (viewModel.isCourseCompleted) {
449         ConfettiCanvas(modifier = Modifier.fillMaxSize())
450         CompletionBorderOverlay()
451     }
452 } // Box
453 }
454
455 @Composable
456 private fun CompletionBorderOverlay() {
457     val primary = MaterialTheme.colorScheme.primary
458     val infiniteTransition = rememberInfiniteTransition(label = "border_pulse")
459     val alpha by infiniteTransition.animateFloat(
460         initialValue = 0.6f,
461         targetValue = 1f,
462         animationSpec = infiniteRepeatable(
463             animation = tween(800, easing = LinearEasing),
464             repeatMode = RepeatMode.Reverse,
465         ),
466         label = "pulse",
467     )
468     Box(
469         modifier = Modifier
470             .fillMaxSize()
471             .border(width = 4.dp, color = primary.copy(alpha = alpha)),
472     )
473 }
474
475 @OptIn(ExperimentalMaterial3Api::class)
476 @Composable
477 private fun StopConfirmSheet(
478     progressPercent: Int,
479     timerText: String,
480     distanceText: String,
481     paceText: String,
482     isFinishing: Boolean,
483     isAbandoning: Boolean,
484     onFinish: () -> Unit,
485     onAbandon: () -> Unit,
486     onContinue: () -> Unit,
487 ) {
488     val completed = progressPercent >= 100
489     ModalBottomSheet(
490         onDismissRequest = onContinue,
491         sheetState = rememberModalBottomSheetState(skipPartiallyExpanded = true),
492         containerColor = MaterialTheme.colorScheme.surface,
493     ) {
494         Column(
495             modifier = Modifier
496                 .fillMaxWidth()
497                 .padding(horizontal = 24.dp)
498                 .padding(bottom = 48.dp),
499             horizontalAlignment = Alignment.CenterHorizontally,
500             verticalArrangement = Arrangement.spacedBy(16.dp),
501         ) {
502             // 완주 여부 배지
503             Surface(
504                 shape = MaterialTheme.shapes.extraLarge,
505                 color = if (completed) MaterialTheme.colorScheme.primary.copy(alpha = 0.12f)
506                     else MaterialTheme.colorScheme.errorContainer,
507             ) {
508                 Text(
509                     text = if (completed) "코스 완주!" else "미완주 · ${progressPercent}% 완료",
510                     style = MaterialTheme.typography.titleMedium,
511                     fontWeight = FontWeight.ExtraBold,
512                     color = if (completed) MaterialTheme.colorScheme.primary
513                         else MaterialTheme.colorScheme.onErrorContainer,

```

```

514         modifier = Modifier.padding(horizontal = 20.dp, vertical = 10.dp),
515     )
516 }
517
518 // 진행률 바
519 LinearProgressIndicator(
520     progress = { (progressPercent / 100f).coerceIn(0f, 1f) },
521     modifier = Modifier
522         .fillMaxWidth()
523         .height(8.dp)
524         .clip(CircleShape),
525     color = if (completed) MaterialTheme.colorScheme.primary
526         else MaterialTheme.colorScheme.error,
527     trackColor = MaterialTheme.colorScheme.surfaceVariant,
528 )
529
530 // 지표
531 Row(
532     modifier = Modifier.fillMaxWidth(),
533     horizontalArrangement = Arrangement.SpaceEvenly,
534 ) {
535     StopStatBlock(label = "TIME", value = timerText)
536     StopStatBlock(label = "DIST", value = "${distanceText}km")
537     StopStatBlock(label = "PACE", value = "${paceText}/km")
538 }
539
540 Spacer(modifier = Modifier.height(4.dp))
541
542 // 저장 버튼
543 if (isFinishing) {
544     CircularProgressIndicator(modifier = Modifier.size(48.dp))
545 } else {
546     Surface(
547         onClick = onFinish,
548         shape = MaterialTheme.shapes.extraLarge,
549         color = if (completed) MaterialTheme.colorScheme.primary
550             else MaterialTheme.colorScheme.surface,
551         border = if (!completed) BorderStroke(1.dp, MaterialTheme.colorScheme.outline) else null,
552         modifier = Modifier.fillMaxWidth(),
553     ) {
554         Text(
555             text = if (completed) "리더보드 확인" else "기록 저장하고 나가기",
556             style = MaterialTheme.typography.labelLarge,
557             fontWeight = FontWeight.Bold,
558             color = if (completed) MaterialTheme.colorScheme.onPrimary
559                 else MaterialTheme.colorScheme.onSurface,
560             textAlign = TextAlign.Center,
561             modifier = Modifier.padding(vertical = 16.dp),
562         )
563     }
564 }
565
566 // 보조 버튼
567 Row(modifier = Modifier.fillMaxWidth()) {
568     TextButton(onClick = onContinue, modifier = Modifier.weight(1f)) {
569         Text("계속 달리기", color = MaterialTheme.colorScheme.onSurfaceVariant)
570     }
571     if (isAbandoning) {
572         Box(modifier = Modifier.weight(1f), contentAlignment = Alignment.Center) {
573             CircularProgressIndicator(modifier = Modifier.size(24.dp))
574         }
575     } else {
576         TextButton(onClick = onAbandon, modifier = Modifier.weight(1f)) {
577             Text("포기", color = MaterialTheme.colorScheme.error)
578         }
579     }
580 }
581 }
582 }
583 }
584
585 @Composable
586 private fun StopStatBlock(label: String, value: String) {
587     Column(horizontalAlignment = Alignment.CenterHorizontally) {
588         Text(
589             text = label,
590             style = MaterialTheme.typography.labelSmall,
591             color = MaterialTheme.colorScheme.onSurfaceVariant,
592         )
593         Spacer(modifier = Modifier.height(2.dp))
594         Text(
595             text = value,
596             style = MaterialTheme.typography.titleSmall,
597             fontWeight = FontWeight.ExtraBold,
598             color = MaterialTheme.colorScheme.onSurface,
599         )

```

```

600     }
601 }
602
603 @Composable
604 private fun CourseMapPanel(
605     points: List<com.runway.android.core.map.MapPoint>,
606     currentLocation: com.runway.android.core.map.MapPoint?,
607     trackStatus: CourseTrackStatus,
608     nearestDistanceMeters: Double?,
609     progressPercent: Int,
610     remainingDistanceText: String,
611     modifier: Modifier = Modifier,
612 ) {
613     val animatedProgress by animateFloatAsState(
614         targetValue = (progressPercent / 100f).coerceIn(0f, 1f),
615         animationSpec = tween(800, easing = FastOutSlowInEasing),
616         label = "courseProgress",
617     )
618     val progressColor by animateColorAsState(
619         targetValue = MaterialTheme.colorScheme.primary,
620         animationSpec = tween(400),
621         label = "progressColor",
622     )
623
624     Column(modifier = modifier) {
625         RouteMapView(
626             points = points,
627             currentLocation = currentLocation,
628             gesturesEnabled = true,
629             followCurrentLocation = true,
630             modifier = Modifier
631                 .weight(1f)
632                 .fillMaxWidth()
633                 .clip(RoundedCornerShape(28.dp)),
634         )
635         Spacer(modifier = Modifier.height(10.dp))
636         Surface(
637             modifier = Modifier.fillMaxWidth(),
638             shape = RoundedCornerShape(20.dp),
639             color = MaterialTheme.colorScheme.surface,
640             border = BorderStroke(1.dp, statusColor(trackStatus).copy(alpha = 0.45f)),
641         ) {
642             Column(modifier = Modifier.padding(horizontal = 16.dp, vertical = 12.dp)) {
643                 Row(
644                     modifier = Modifier.fillMaxWidth(),
645                     horizontalArrangement = Arrangement.SpaceBetween,
646                     verticalAlignment = Alignment.CenterVertically,
647                 ) {
648                     Column {
649                         Text(
650                             text = statusTitle(trackStatus),
651                             style = MaterialTheme.typography.titleSmall,
652                             fontWeight = FontWeight.ExtraBold,
653                             color = statusColor(trackStatus),
654                         )
655                         Text(
656                             text = statusDetail(trackStatus, nearestDistanceMeters),
657                             style = MaterialTheme.typography.labelMedium,
658                             color = MaterialTheme.colorScheme.onSurfaceVariant,
659                         )
660                     }
661                     Column(horizontalAlignment = Alignment.End) {
662                         Text(
663                             text = "${progressPercent}%",
664                             style = MaterialTheme.typography.titleMedium,
665                             fontWeight = FontWeight.ExtraBold,
666                             color = MaterialTheme.colorScheme.onSurface,
667                         )
668                         Text(
669                             text = "남은 ${remainingDistanceText}km",
670                             style = MaterialTheme.typography.labelSmall,
671                             color = MaterialTheme.colorScheme.onSurfaceVariant,
672                         )
673                     }
674                 }
675                 Spacer(modifier = Modifier.height(10.dp))
676                 LinearProgressIndicator(
677                     progress = { animatedProgress },
678                     modifier = Modifier
679                         .fillMaxWidth()
680                         .height(6.dp)
681                         .clip(CircleShape),
682                     color = progressColor,
683                     trackColor = MaterialTheme.colorScheme.surfaceVariant,
684                 )
685             }
686         }
687     }

```

```

686     }
687 }
688 }
689
690 @Composable
691 private fun statusColor(status: CourseTrackStatus): Color = when (status) {
692     CourseTrackStatus.UNKNOWN -> MaterialTheme.colorScheme.onSurfaceVariant
693     CourseTrackStatus.ON_COURSE -> MaterialTheme.colorScheme.primary
694     CourseTrackStatus.NEAR_COURSE -> com.runway.android.ui.theme.WarningYellow
695     CourseTrackStatus.OFF_COURSE -> MaterialTheme.colorScheme.error
696 }
697
698 private fun statusTitle(status: CourseTrackStatus): String = when (status) {
699     CourseTrackStatus.UNKNOWN -> "코스 확인 중"
700     CourseTrackStatus.ON_COURSE -> "코스 위를 달리는 중"
701     CourseTrackStatus.NEAR_COURSE -> "코스에서 조금 벗어남"
702     CourseTrackStatus.OFF_COURSE -> "코스 이탈"
703 }
704
705 private fun statusDetail(status: CourseTrackStatus, meters: Double?): String {
706     val distance = meters?.let { "%.0fm".format(it) }?: "--m"
707     return when (status) {
708         CourseTrackStatus.UNKNOWN -> "현재 위치와 코스를 맞추고 있어요"
709         CourseTrackStatus.ON_COURSE -> "코스와의 거리 $distance"
710         CourseTrackStatus.NEAR_COURSE -> "코스와의 거리 $distance · 방향을 확인하세요"
711         CourseTrackStatus.OFF_COURSE -> "코스와의 거리 $distance · 지도에서 복귀 경로를 확인하세요"
712     }
713 }
714
715 @Composable
716 private fun CoursePill() {
717     Surface(
718         shape = CircleShape,
719         color = MaterialTheme.colorScheme.surface,
720         border = BorderStroke(1.dp, MaterialTheme.colorScheme.primary.copy(alpha = 0.5f)),
721     ) {
722         Text(
723             text = "COURSE ATTEMPT",
724             style = MaterialTheme.typography.labelLarge,
725             color = MaterialTheme.colorScheme.primary,
726             modifier = Modifier.padding(horizontal = 14.dp, vertical = 6.dp),
727         )
728     }
729 }

```

```

1  package com.runway.android.ui.attempt
2
3  import android.content.Context
4  import android.content.Intent
5  import android.location.Location
6  import android.os.Build
7  import android.os.VibrationEffect
8  import android.os.Vibrator
9  import android.os.VibratorManager
10 import androidx.compose.runtime.getValue
11 import androidx.compose.runtime.mutableStateOf
12 import androidx.compose.runtime.setValue
13 import androidx.lifecycle.SavedStateHandle
14 import androidx.lifecycle.ViewModel
15 import androidx.lifecycle.viewModelScope
16 import androidx.core.content.ContextCompat
17 import com.runway.android.core.location.GpsStatus
18 import com.runway.android.core.map.MapPoint
19 import com.runway.android.core.result.NetworkResult
20 import com.runway.android.core.tracking.PendingPointQueue
21 import com.runway.android.core.tracking.RunTrackingAction
22 import com.runway.android.core.tracking.RunTrackingManager
23 import com.runway.android.core.tracking.RunTrackingMode
24 import com.runway.android.core.tracking.RunTrackingService
25 import com.runway.android.core.tracking.TrackingSessionSnapshot
26 import com.runway.android.core.tracking.TrackingSessionStore
27 import com.runway.android.core.tracking.toRunPointRequest
28 import com.runway.android.core.util.formatDuration
29 import com.runway.android.core.voice.RunningVoiceGuide
30 import com.runway.android.data.attempt.model.AbandonAttemptRequest
31 import com.runway.android.data.attempt.model.FinishAttemptRequest
32 import com.runway.android.data.running.model.SavePointsRequest
33 import com.runway.android.domain.attempt.CourseAttemptRepository
34 import com.runway.android.domain.course.CourseRepository
35 import com.runway.android.domain.running.RunningRepository
36 import dagger.hilt.android.lifecycle.HiltViewModel
37 import dagger.hilt.android.qualifiers.ApplicationContext
38 import kotlinx.coroutines.CoroutineScope
39 import kotlinx.coroutines.Job
40 import kotlinx.coroutines.delay
41 import kotlinx.coroutines.flow.MutableSharedFlow
42 import kotlinx.coroutines.flow.asSharedFlow
43 import kotlinx.coroutines.launch
44 import java.time.Instant
45 import javax.inject.Inject
46 import javax.inject.Named
47
48 sealed class AttemptNavEvent {
49     data class NavigateToLeaderboard(
50         val courseId: String,
51         val isPR: Boolean = false,
52         val previousBestSeconds: Int? = null,
53         val improvementSeconds: Int? = null,
54     ) : AttemptNavEvent()
55     object NavigateBack : AttemptNavEvent()
56 }
57
58 enum class CourseTrackStatus {
59     UNKNOWN,
60     ON_COURSE,
61     NEAR_COURSE,
62     OFF_COURSE,
63 }
64
65 @HiltViewModel
66 class CourseAttemptTrackingViewModel @Inject constructor(
67     savedStateHandle: SavedStateHandle,
68     @ApplicationContext private val context: Context,
69     private val runningRepository: RunningRepository,
70     private val courseAttemptRepository: CourseAttemptRepository,
71     private val courseRepository: CourseRepository,
72     private val manager: RunTrackingManager,
73     private val pendingPointQueue: PendingPointQueue,
74     private val sessionStore: TrackingSessionStore,
75     private val voiceGuide: RunningVoiceGuide,
76     @Named("appScope") private val appScope: CoroutineScope,
77 ) : ViewModel() {
78
79     val courseId: String = checkNotNull(savedStateHandle["courseId"])
80     private val courseAttemptId: String = checkNotNull(savedStateHandle["courseAttemptId"])
81     private val runningRecordId: String = checkNotNull(savedStateHandle["runningRecordId"])
82
83     var gpsStatus by mutableStateOf(GpsStatus.PERMISSION_REQUIRED)

```



```

84     private set
85     var elapsedSeconds by mutableStateOf(0)
86     private set
87     var distanceKm by mutableStateOf(0.0)
88     private set
89     var isFinishing by mutableStateOf(false)
90     private set
91     var isAbandoning by mutableStateOf(false)
92     private set
93     var finishError by mutableStateOf<String?>(null)
94     private set
95     var coursePoints by mutableStateOf<List<MapPoint>>(emptyList())
96     private set
97     var currentLocationPoint by mutableStateOf<MapPoint?>(null)
98     private set
99     var isAutoPaused by mutableStateOf(false)
100    private set
101    var isPaused by mutableStateOf(false)
102    private set
103    var milestoneMessage by mutableStateOf<String?>(null)
104    private set
105    var courseDistanceMeters by mutableStateOf(0.0)
106    private set
107    var nearestCourseDistanceMeters by mutableStateOf<Double?>(null)
108    private set
109    var showDeviationWarning by mutableStateOf(false)
110    private set
111
112    private var lastSpeedMps by mutableStateOf<Float?>(null)
113    private var cadenceSpm by mutableStateOf<Int?>(null)
114
115    val timerText: String
116        get() = formatDuration(elapsedSeconds)
117
118    val distanceText: String
119        get() = "%.2f".format(distanceKm)
120
121    val paceText: String
122        get() {
123        if (distanceKm < 0.001) return "--'--\\""
124        val secsPerKm = (elapsedSeconds / distanceKm).toInt()
125        return "%d'%02d\\""
126        }
127
128    val speedText: String
129        get() {
130        val gpsSpeed = lastSpeedMps
131        return if (gpsSpeed != null && gpsSpeed >= 0.1f) {
132            "%.1f".format(gpsSpeed * 3.6f)
133        } else if (elapsedSeconds < 1) {
134            "0.0"
135        } else {
136            "%.1f".format(distanceKm / (elapsedSeconds / 3600.0))
137        }
138    }
139
140    val cadenceText: String
141        get() = cadenceSpm?.takeIf { it > 0 }?.toString() ?: "--"
142
143    val courseProgressPercent: Int
144        get() = if (courseDistanceMeters > 1.0) {
145            ((distanceKm * 1000.0 / courseDistanceMeters) * 100.0).toInt().coerceIn(0, 100)
146        } else {
147            0
148        }
149
150    val remainingDistanceText: String
151        get() {
152        val remainingMeters = (courseDistanceMeters - distanceKm * 1000.0).coerceAtLeast(0.0)
153        return "%.2f".format(remainingMeters / 1000.0)
154    }
155
156    val trackStatus: CourseTrackStatus
157        get() = when (val meters = nearestCourseDistanceMeters) {
158            null -> CourseTrackStatus.UNKNOWN
159            in 0.0..30.0 -> CourseTrackStatus.ON_COURSE
160            in 30.0..80.0 -> CourseTrackStatus.NEAR_COURSE
161            else -> CourseTrackStatus.OFF_COURSE
162        }
163
164    private val _navEvent = MutableSharedFlow<AttemptNavEvent>()
165    val navEvent = _navEvent.asSharedFlow()
166
167    private var isDone = false
168    private var serviceStarted = false
169    private var lastTrackStatus: CourseTrackStatus = CourseTrackStatus.UNKNOWN

```

```

170 private var nearFinishMessageShown = false
171 private var courseCompletionHandled = false
172 var isCourseCompleted by mutableStateOf(false)
173     private set
174 private var pausedByDeviation = false
175 // true 동안에는 연속 재일시정지 중복 실행 방지 (isPaused=true 확인 후 리셋)
176 private var repausingForDeviation = false
177
178 private var stateObserveJob: Job? = null
179 private var batchJob: Job? = null
180 private var milestoneCollectJob: Job? = null
181 private var milestoneDisplayJob: Job? = null
182
183 init {
184     viewModelScope.launch {
185         sessionStore.saveSnapshot(
186             TrackingSessionSnapshot(
187                 runningRecordId = runningRecordId,
188                 courseAttemptId = courseAttemptId,
189                 elapsedSeconds = 0,
190                 distanceMeters = 0.0,
191             )
192         )
193     }
194     viewModelScope.launch {
195         when (val r = courseRepository.getCoursePoints(courseId)) {
196             is NetworkResult.Success -> {
197                 coursePoints = r.data.points.map { MapPoint(it.latitude, it.longitude) }
198                 courseDistanceMeters = calculateCourseDistance(coursePoints)
199             }
200             else -> {}
201         }
202     }
203     startBatchSaving()
204 }
205
206 fun startTracking() {
207     if (serviceStarted) return
208     serviceStarted = true
209     gpsStatus = GpsStatus.WAITING_FOR_FIX
210     voiceGuide.start()
211
212     ContextCompat.startForegroundService(
213         context,
214         Intent(context, RunTrackingService::class.java).apply {
215             action = RunTrackingAction.ACTION_START
216             putExtra(RunTrackingAction.EXTRA_MODE, RunTrackingMode.COURSE_ATTEMPT.name)
217         }
218     )
219
220     stateObserveJob = viewModelScope.launch {
221         manager.state.collect { state ->
222             if (state.hasFirstFix && gpsStatus != GpsStatus.ACTIVE) {
223                 gpsStatus = GpsStatus.ACTIVE
224             }
225             elapsedSeconds = state.elapsedSeconds
226             distanceKm = state.distanceMeters / 1000.0
227             lastSpeedMps = state.currentSpeedMps
228             cadenceSpm = state.cadenceSpm
229             isAutoPaused = state.isAutoPaused
230             isPaused = state.isPaused
231             if (isPaused) repausingForDeviation = false
232             currentLocationPoint = state.lastLocation?.let { location ->
233                 MapPoint(location.latitude, location.longitude).also { point ->
234                     nearestCourseDistanceMeters = nearestDistanceToCourse(point, coursePoints)
235                     val currentStatus = trackStatus
236                     if (currentStatus != lastTrackStatus) {
237                         if (currentStatus == CourseTrackStatus.OFF_COURSE) {
238                             showDeviationWarning = true
239                             vibrateDeviation()
240                             voiceGuide.offCourse()
241                             // 최초 이탈 시 즉시 일시정지
242                             if (!isPaused && !isAutoPaused) {
243                                 pausedByDeviation = true
244                                 repausingForDeviation = true
245                                 pause()
246                             }
247                         } else if (lastTrackStatus == CourseTrackStatus.OFF_COURSE) {
248                             showDeviationWarning = false
249                             voiceGuide.backOnCourse()
250                             if (pausedByDeviation) {
251                                 pausedByDeviation = false
252                                 repausingForDeviation = false
253                                 resume()
254                             }
255                         }
256                     }
257                 }
258             }
259         }
260     }
261 }

```

```

256         lastTrackStatus = currentStatus
257     }
258     // 이탈 중 재생 버튼으로 재개한 경우 - 진동+재일시정지 (매 틱 확인)
259     if (currentStatus == CourseTrackStatus.OFF_COURSE &&
260         !isPaused && !isAutoPaused && !reparsingForDeviation
261     ) {
262         reparsingForDeviation = true
263         pausedByDeviation = true
264         vibrate()
265         pause()
266     }
267     // 90% 도달 시 1회만 동기부여 메시지 표시
268     if (!nearFinishMessageShown && courseProgressPercent >= 90 && courseProgressPercent < 100) {
269         nearFinishMessageShown = true
270         vibrate()
271         voiceGuide.nearCourseFinish()
272         milestoneDisplayJob?.cancel()
273         milestoneMessage = "거의 다 왔어요! 조금만 더 달려요!"
274         milestoneDisplayJob = viewModelScope.launch {
275             delay(4_000)
276             milestoneMessage = null
277         }
278     }
279     // 100% 완주 감지 - 자동 일시정지 + 음성
280     if (!courseCompletionHandled && courseProgressPercent >= 100) {
281         courseCompletionHandled = true
282         isCourseCompleted = true
283         vibrateCompletion()
284         voiceGuide.speak("코스를 완주했습니다! 수고하셨습니다.")
285         milestoneMessage = "코스 완주! 🏁"
286         if (!isPaused && !isAutoPaused) pause()
287     }
288 }
289 }
290 }
291 }
292
293 milestoneCollectJob = viewModelScope.launch {
294     manager.milestoneFlow.collect { km -> triggerMilestone(km) }
295 }
296
297 private fun triggerMilestone(km: Int) {
298     vibrate()
299     voiceGuide.kilometer(km, elapsedSeconds)
300     milestoneDisplayJob?.cancel()
301     milestoneMessage = "${km}km 완료 · 현재 페이스 $spaceText /km"
302     milestoneDisplayJob = viewModelScope.launch {
303         delay(4_000)
304         milestoneMessage = null
305     }
306 }
307
308
309 fun pause() {
310     manager.pause()
311     viewModelScope.launch { runningRepository.pauseRun(runningRecordId) }
312 }
313
314 fun resume() {
315     val wasAutoPaused = manager.state.value.isAutoPaused
316     manager.resume()
317     if (!wasAutoPaused) {
318         viewModelScope.launch { runningRepository.resumeRun(runningRecordId) }
319     }
320 }
321
322 fun dismissDeviationWarning() {
323     showDeviationWarning = false
324 }
325
326 @Suppress("DEPRECATION")
327 private fun vibrateDeviation() {
328     val vibrator = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
329         (context.getSystemService(Context.VIBRATOR_MANAGER_SERVICE) as VibratorManager).defaultVibrator
330     } else {
331         context.getSystemService(Context.VIBRATOR_SERVICE) as Vibrator
332     }
333     vibrator.vibrate(
334         VibrationEffect.createWaveform(longArrayOf(0, 300, 100, 300, 100, 300), -1)
335     )
336 }
337
338 @Suppress("DEPRECATION")
339 private fun vibrate() {
340     val vibrator = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
341         (context.getSystemService(Context.VIBRATOR_MANAGER_SERVICE) as VibratorManager).defaultVibrator

```

```

342     } else {
343         context.getSystemService(Context.VIBRATOR_SERVICE) as Vibrator
344     }
345     vibrator.vibrate(
346         VibrationEffect.createWaveform(longArrayOf(0, 200, 100, 200), -1)
347     )
348 }
349
350 @Suppress("DEPRECATION")
351 private fun vibrateCompletion() {
352     val vibrator = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
353         (context.getSystemService(Context.VIBRATOR_MANAGER_SERVICE) as VibratorManager).defaultVibrator
354     } else {
355         context.getSystemService(Context.VIBRATOR_SERVICE) as Vibrator
356     }
357     // 완주: 강한 진동 3회
358     vibrator.vibrate(
359         VibrationEffect.createWaveform(longArrayOf(0, 200, 80, 200, 80, 400), -1)
360     )
361 }
362
363 private fun startBatchSaving() {
364     batchJob = viewModelScope.launch {
365         while (true) {
366             delay(5_000L)
367             flushPendingPoints()
368             updateSnapshot()
369         }
370     }
371 }
372
373 private suspend fun flushPendingPoints() {
374     val inMemory = manager.consumePoints()
375     pendingPointQueue.add(runningRecordId, inMemory)
376
377     val batch = pendingPointQueue.dequeue(runningRecordId, 50)
378     if (batch.isEmpty()) return
379
380     val result = runningRepository.savePoints(runningRecordId, SavePointsRequest(batch.map { it.toRunPointRequest() }))
381     if (result is NetworkResult.Success) {
382         pendingPointQueue.deleteByIds(batch.map { it.id })
383     }
384 }
385
386 private suspend fun updateSnapshot() {
387     sessionStore.saveSnapshot(
388         TrackingSessionSnapshot(
389             runningRecordId = runningRecordId,
390             courseAttemptId = courseAttemptId,
391             elapsedSeconds = elapsedSeconds,
392             distanceMeters = distanceKm * 1000.0,
393         )
394     )
395 }
396
397 fun finish() {
398     if (isFinishing || isAbandoning) return
399     isFinishing = true
400     finishError = null
401
402     val seconds = elapsedSeconds
403     val distance = distanceKm
404
405     viewModelScope.launch {
406         pendingPointQueue.add(runningRecordId, manager.consumePoints())
407         // 큐가 빌 때까지 업로드. 연속 실패 10회 시 중단 (서버/네트워크 문제).
408         var failures = 0
409         while (failures < 10) {
410             val batch = pendingPointQueue.dequeue(runningRecordId, 50)
411             if (batch.isEmpty()) break
412             val r = runningRepository.savePoints(
413                 runningRecordId,
414                 SavePointsRequest(batch.map { it.toRunPointRequest() }),
415             )
416             if (r is NetworkResult.Success) {
417                 pendingPointQueue.deleteByIds(batch.map { it.id })
418                 failures = 0
419             } else {
420                 failures++
421             }
422         }
423
424         val finishResult = courseAttemptRepository.finishAttempt(
425             attemptId = courseAttemptId,
426             request = FinishAttemptRequest(
427                 endedAt = Instant.now().toString(),

```

```

428         distanceMeters = distance * 1000.0,
429         durationSeconds = seconds,
430         avgPaceSecondsPerKm = if (distance > 0.001) (seconds / distance).toInt() else 0,
431         caloriesBurned = (distance * 72).toInt(),
432     ),
433 )
434
435 when (finishResult) {
436     is NetworkResult.ApiError,
437     is NetworkResult.NetworkError -> {
438         isFinishing = false
439         finishError = "완주 처리에 실패했습니다. 다시 시도해 주세요."
440         return@launch
441     }
442     is NetworkResult.Success -> {
443         voiceGuide.finish()
444         val response = finishResult.data
445         sessionStore.clearSnapshot()
446         pendingPointQueue.deleteByRunningRecordId(runningRecordId)
447         stopServiceAndJobs()
448         isDone = true
449         _navEvent.emit(
450             AttemptNavEvent.NavigateToLeaderboard(
451                 courseId = courseId,
452                 isPR = response.isPR,
453                 previousBestSeconds = response.previousBestSeconds,
454                 improvementSeconds = response.improvementSeconds,
455             )
456         )
457     }
458 }
459 }
460 }
461
462 fun abandon() {
463     if (isFinishing || isAbandoning) return
464     isAbandoning = true
465
466     viewModelScope.launch {
467         courseAttemptRepository.abandonAttempt(
468             attemptId = courseAttemptId,
469             request = AbandonAttemptRequest(endedAt = Instant.now().toString()),
470         )
471         sessionStore.clearSnapshot()
472         pendingPointQueue.deleteByRunningRecordId(runningRecordId)
473         stopServiceAndJobs()
474         isDone = true
475         _navEvent.emit(AttemptNavEvent.NavigateBack)
476     }
477 }
478
479 private fun stopServiceAndJobs() {
480     stateObserveJob?.cancel()
481     batchJob?.cancel()
482     milestoneCollectJob?.cancel()
483     milestoneDisplayJob?.cancel()
484     manager.stop()
485     context.startService(
486         Intent(context, RunTrackingService::class.java).apply {
487             action = RunTrackingAction.ACTION_STOP
488         }
489     )
490 }
491
492 override fun onCleared() {
493     super.onCleared()
494     if (!isDone) {
495         stopServiceAndJobs()
496         appScope.launch {
497             courseAttemptRepository.abandonAttempt(
498                 attemptId = courseAttemptId,
499                 request = AbandonAttemptRequest(endedAt = Instant.now().toString()),
500             )
501             sessionStore.clearSnapshot()
502             pendingPointQueue.deleteByRunningRecordId(runningRecordId)
503         }
504     }
505 }
506 }
507
508 private fun calculateCourseDistance(points: List<MapPoint>): Double {
509     if (points.size < 2) return 0.0
510     return points.zipWithNext().sumOf { (from, to) ->
511         distanceMeters(from, to)
512     }
513 }

```

```

514
515 private fun nearestDistanceToCourse(current: MapPoint, course: List<MapPoint>): Double? {
516     if (course.isEmpty()) return null
517     if (course.size == 1) return distanceMeters(current, course[0])
518     // Use segment-based distance to avoid false off-course on long straight segments
519     return course.zipWithNext().minOf { (a, b) -> distanceToSegmentMeters(current, a, b) }
520 }
521
522 private fun distanceToSegmentMeters(p: MapPoint, a: MapPoint, b: MapPoint): Double {
523     val segLenSq = distanceMeters(a, b).let { it * it }
524     if (segLenSq < 0.01) return distanceMeters(p, a) // degenerate segment
525     // Project p onto segment ab (in lat/lon space - approximate but fine for small distances)
526     val t = ((p.latitude - a.latitude) * (b.latitude - a.latitude) +
527         (p.longitude - a.longitude) * (b.longitude - a.longitude)) /
528         ((b.latitude - a.latitude) * (b.latitude - a.latitude) +
529             (b.longitude - a.longitude) * (b.longitude - a.longitude))
530     val tClamped = t.coerceIn(0.0, 1.0)
531     val proj = MapPoint(
532         latitude = a.latitude + tClamped * (b.latitude - a.latitude),
533         longitude = a.longitude + tClamped * (b.longitude - a.longitude),
534     )
535     return distanceMeters(p, proj)
536 }
537
538 private fun distanceMeters(from: MapPoint, to: MapPoint): Double {
539     val result = FloatArray(1)
540     Location.distanceBetween(
541         from.latitude,
542         from.longitude,
543         to.latitude,
544         to.longitude,
545         result,
546     )
547     return result[0].toDouble()
548 }

```

```

1  package com.runway.android.ui.auth.login
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.layout.Box
5  import androidx.compose.foundation.layout.Column
6  import androidx.compose.foundation.layout.Spacer
7  import androidx.compose.foundation.layout.fillMaxSize
8  import androidx.compose.foundation.layout.fillMaxWidth
9  import androidx.compose.foundation.layout.height
10 import androidx.compose.foundation.layout.padding
11 import androidx.compose.foundation.layout.size
12 import androidx.compose.foundation.layout.systemBarsPadding
13 import androidx.compose.foundation.rememberScrollState
14 import androidx.compose.foundation.text.KeyboardOptions
15 import androidx.compose.foundation.verticalScroll
16 import androidx.compose.material3.MaterialTheme
17 import androidx.compose.material3.Text
18 import androidx.compose.material3.TextButton
19 import androidx.compose.runtime.Composable
20 import androidx.compose.runtime.LaunchedEffect
21 import androidx.compose.ui.Alignment
22 import androidx.compose.ui.Modifier
23 import androidx.compose.ui.text.SpanStyle
24 import androidx.compose.ui.text.buildAnnotatedString
25 import androidx.compose.ui.text.input.KeyboardType
26 import androidx.compose.ui.text.input.PasswordVisualTransformation
27 import androidx.compose.ui.text.withStyle
28 import androidx.compose.ui.unit.dp
29 import androidx.hilt.navigation.compose.hiltViewModel
30 import com.runway.android.ui.components.RunwayErrorText
31 import com.runway.android.ui.components.RunwayLoadingButton
32 import com.runway.android.ui.components.RunwayTextField
33
34 @Composable
35 fun LoginScreen(
36     onNavigateToSignup: () -> Unit,
37     onLoginSuccess: () -> Unit,
38     viewModel: LoginViewModel = hiltViewModel(),
39 ) {
40     LaunchedEffect(Unit) {
41         viewModel.navigateToHome.collect { onLoginSuccess() }
42     }
43
44     Column(
45         modifier = Modifier
46             .fillMaxSize()
47             .background(MaterialTheme.colorScheme.background)
48             .systemBarsPadding()
49             .verticalScroll(rememberScrollState()),
50     ) {
51         // — Header —
52         Column(
53             modifier = Modifier.padding(horizontal = 28.dp, vertical = 40.dp),
54         ) {
55             Box(
56                 modifier = Modifier
57                     .size(40.dp)
58                     .background(
59                         color = MaterialTheme.colorScheme.primary,
60                         shape = MaterialTheme.shapes.small,
61                     ),
62                 contentAlignment = Alignment.Center,
63             ) {
64                 Text(
65                     text = "R",
66                     style = MaterialTheme.typography.titleMedium,
67                     color = MaterialTheme.colorScheme.onPrimary,
68                 )
69             }
70
71             Spacer(modifier = Modifier.height(24.dp))
72
73             Text(
74                 text = "다시 만났네요.",
75                 style = MaterialTheme.typography.headlineLarge,
76                 color = MaterialTheme.colorScheme.onBackground,
77             )
78
79             Spacer(modifier = Modifier.height(8.dp))
80
81             Text(
82                 text = "이어서 달려볼까요?",
83                 style = MaterialTheme.typography.bodyMedium,

```

```

84         color = MaterialTheme.colorScheme.onSurfaceVariant,
85     )
86 }
87
88 // — Fields —
89 Column(modifier = Modifier.padding(horizontal = 28.dp)) {
90     RunwayTextField(
91         value = viewModel.email,
92         onValueChange = viewModel::onEmailChange,
93         label = "이메일",
94         placeholder = "your@email.com",
95         keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Email),
96         isError = viewModel.error != null,
97     )
98
99     Spacer(modifier = Modifier.height(16.dp))
100
101     RunwayTextField(
102         value = viewModel.password,
103         onValueChange = viewModel::onPasswordChange,
104         label = "비밀번호",
105         placeholder = ".....",
106         visualTransformation = PasswordVisualTransformation(),
107         keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Password),
108         isError = viewModel.error != null,
109     )
110
111 }
112
113 // — Error —
114 if (viewModel.error != null) {
115     Spacer(modifier = Modifier.height(4.dp))
116     RunwayErrorText(
117         message = viewModel.error!!,
118         modifier = Modifier.padding(horizontal = 28.dp),
119     )
120 }
121
122 // — Actions —
123 Column(modifier = Modifier.padding(horizontal = 28.dp)) {
124     Spacer(modifier = Modifier.height(24.dp))
125
126     RunwayLoadingButton(
127         text = "로그인",
128         onClick = viewModel::login,
129         isLoading = viewModel.isLoading,
130     )
131
132     Spacer(modifier = Modifier.height(16.dp))
133
134     TextButton(
135         onClick = onNavigateToSignup,
136         modifier = Modifier.fillMaxWidth(),
137     ) {
138         Text(
139             text = buildAnnotatedString {
140                 append("처음이신가요? ")
141                 withStyle(SpanStyle(color = MaterialTheme.colorScheme.primary)) {
142                     append("회원가입")
143                 }
144             },
145             style = MaterialTheme.typography.bodyMedium,
146             color = MaterialTheme.colorScheme.onSurfaceVariant,
147         )
148     }
149
150     Spacer(modifier = Modifier.height(32.dp))
151 }
152 }
153 }

```



```
1 package com.runway.android.ui.auth.login
2
3 import androidx.compose.runtime.getValue
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.compose.runtime.setValue
6 import androidx.lifecycle.ViewModel
7 import androidx.lifecycle.viewModelScope
8 import com.runway.android.core.result.NetworkResult
9 import com.runway.android.domain.auth.AuthRepository
10 import dagger.hilt.android.lifecycle.HiltViewModel
11 import kotlinx.coroutines.flow.MutableSharedFlow
12 import kotlinx.coroutines.flow.asSharedFlow
13 import kotlinx.coroutines.launch
14 import javax.inject.Inject
15
16 @HiltViewModel
17 class LoginViewModel @Inject constructor(
18     private val authRepository: AuthRepository,
19 ) : ViewModel() {
20
21     var email by mutableStateOf("")
22     private set
23     var password by mutableStateOf("")
24     private set
25     var isLoading by mutableStateOf(false)
26     private set
27     var error by mutableStateOf<String?>(null)
28     private set
29
30     private val _navigateToHome = MutableSharedFlow<Unit>()
31     val navigateToHome = _navigateToHome.asSharedFlow()
32
33     fun onEmailChange(value: String) {
34         email = value
35         error = null
36     }
37
38     fun onPasswordChange(value: String) {
39         password = value
40         error = null
41     }
42
43     fun login() {
44         if (email.isBlank() || password.isBlank()) {
45             error = "이메일과 비밀번호를 입력해 주세요."
46             return
47         }
48         viewModelScope.launch {
49             isLoading = true
50             error = null
51             when (val result = authRepository.login(email.trim(), password)) {
52                 is NetworkResult.Success -> _navigateToHome.emit(Unit)
53                 is NetworkResult.ApiError -> error = result.message
54                 is NetworkResult.NetworkError -> error = "네트워크 오류가 발생했습니다."
55             }
56             isLoading = false
57         }
58     }
59 }
```

```

1  package com.runway.android.ui.auth.signup
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.layout.Column
5  import androidx.compose.foundation.layout.Row
6  import androidx.compose.foundation.layout.Spacer
7  import androidx.compose.foundation.layout.fillMaxSize
8  import androidx.compose.foundation.layout.fillMaxWidth
9  import androidx.compose.foundation.layout.height
10 import androidx.compose.foundation.layout.padding
11 import androidx.compose.foundation.layout.size
12 import androidx.compose.foundation.layout.systemBarsPadding
13 import androidx.compose.foundation.rememberScrollState
14 import androidx.compose.foundation.text.KeyboardOptions
15 import androidx.compose.foundation.verticalScroll
16 import androidx.compose.material.icons.Icons
17 import androidx.compose.material.icons.automirrored.filled.ArrowBack
18 import androidx.compose.material3.Icon
19 import androidx.compose.material3.IconButton
20 import androidx.compose.material3.MaterialTheme
21 import androidx.compose.material3.Text
22 import androidx.compose.runtime.Composable
23 import androidx.compose.runtime.LaunchedEffect
24 import androidx.compose.ui.Alignment
25 import androidx.compose.ui.Modifier
26 import androidx.compose.ui.text.input.KeyboardType
27 import androidx.compose.ui.text.input.PasswordVisualTransformation
28 import androidx.compose.ui.unit.dp
29 import androidx.hilt.navigation.compose.hiltViewModel
30 import com.runway.android.ui.components.RunwayErrorText
31 import com.runway.android.ui.components.RunwayLoadingButton
32 import com.runway.android.ui.components.RunwayTextField
33
34 @Composable
35 fun SignupScreen(
36     onNavigateBack: () -> Unit,
37     onSignupSuccess: () -> Unit,
38     viewModel: SignupViewModel = hiltViewModel(),
39 ) {
40     LaunchedEffect(Unit) {
41         viewModel.navigateToLogin.collect { onSignupSuccess() }
42     }
43
44     Column(
45         modifier = Modifier
46             .fillMaxSize()
47             .background(MaterialTheme.colorScheme.background)
48             .systemBarsPadding()
49             .verticalScroll(rememberScrollState()),
50     ) {
51         // — Top bar —
52         Row(
53             modifier = Modifier
54                 .fillMaxWidth()
55                 .padding(horizontal = 16.dp, vertical = 8.dp),
56             verticalAlignment = Alignment.CenterVertically,
57         ) {
58             IconButton(onClick = onNavigateBack) {
59                 Icon(
60                     imageVector = Icons.AutoMirrored.Filled.ArrowBack,
61                     contentDescription = "Back",
62                     tint = MaterialTheme.colorScheme.onBackground,
63                     modifier = Modifier.size(24.dp),
64                 )
65             }
66         }
67
68         // — Header —
69         Column(modifier = Modifier.padding(horizontal = 28.dp, vertical = 16.dp)) {
70             Text(
71                 text = "함께 달려요.",
72                 style = MaterialTheme.typography.headlineLarge,
73                 color = MaterialTheme.colorScheme.onBackground,
74             )
75
76             Spacer(modifier = Modifier.height(8.dp))
77
78             Text(
79                 text = "계정을 만들고 코스를 탐색해보세요.",
80                 style = MaterialTheme.typography.bodyMedium,
81                 color = MaterialTheme.colorScheme.onSurfaceVariant,
82             )
83         }

```

```

84
85 // — Fields —
86 Column(modifier = Modifier.padding(horizontal = 28.dp)) {
87     Spacer(modifier = Modifier.height(8.dp))
88
89     RunwayTextField(
90         value = viewModel.email,
91         onValueChange = viewModel::onEmailChange,
92         label = "이메일",
93         placeholder = "your@email.com",
94         keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Email),
95         isError = viewModel.error != null,
96     )
97
98     Spacer(modifier = Modifier.height(16.dp))
99
100    RunwayTextField(
101        value = viewModel.password,
102        onValueChange = viewModel::onPasswordChange,
103        label = "비밀번호",
104        placeholder = "*****",
105        visualTransformation = PasswordVisualTransformation(),
106        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Password),
107        isError = viewModel.error != null,
108    )
109
110    Spacer(modifier = Modifier.height(16.dp))
111
112    RunwayTextField(
113        value = viewModel.nickname,
114        onValueChange = viewModel::onNicknameChange,
115        label = "닉네임",
116        placeholder = "러너 이름을 입력하세요",
117        isError = viewModel.error != null,
118    )
119 }
120
121 // — Error —
122 if (viewModel.error != null) {
123     Spacer(modifier = Modifier.height(8.dp))
124     RunwayErrorText(
125         message = viewModel.error!!,
126         modifier = Modifier.padding(horizontal = 28.dp),
127     )
128 }
129
130 // — Actions —
131 Column(modifier = Modifier.padding(horizontal = 28.dp)) {
132     Spacer(modifier = Modifier.height(32.dp))
133
134     RunwayLoadingButton(
135         text = "회원가입",
136         onClick = viewModel::signup,
137         isLoading = viewModel.isLoading,
138     )
139
140     Spacer(modifier = Modifier.height(20.dp))
141
142     Text(
143         text = "가입하면 서비스 이용약관 및 개인정보처리방침에 동의하게 됩니다.",
144         style = MaterialTheme.typography.bodySmall,
145         color = MaterialTheme.colorScheme.onSurfaceVariant,
146         modifier = Modifier.fillMaxWidth(),
147     )
148
149     Spacer(modifier = Modifier.height(32.dp))
150 }
151 }
152 }

```

```
1 package com.runway.android.ui.auth.signup
2
3 import androidx.compose.runtime.getValue
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.compose.runtime.setValue
6 import androidx.lifecycle.ViewModel
7 import androidx.lifecycle.ViewModelScope
8 import com.runway.android.core.result.NetworkResult
9 import com.runway.android.domain.auth.AuthRepository
10 import dagger.hilt.android.lifecycle.HiltViewModel
11 import kotlinx.coroutines.flow.MutableSharedFlow
12 import kotlinx.coroutines.flow.asSharedFlow
13 import kotlinx.coroutines.launch
14 import javax.inject.Inject
15
16 @HiltViewModel
17 class SignupViewModel @Inject constructor(
18     private val authRepository: AuthRepository,
19 ) : ViewModel() {
20
21     var email by mutableStateOf("")
22     private set
23     var password by mutableStateOf("")
24     private set
25     var nickname by mutableStateOf("")
26     private set
27     var isLoading by mutableStateOf(false)
28     private set
29     var error by mutableStateOf<String?>(null)
30     private set
31
32     private val _navigateToLogin = MutableSharedFlow<Unit>()
33     val navigateToLogin = _navigateToLogin.asSharedFlow()
34
35     fun onEmailChange(value: String) {
36         email = value
37         error = null
38     }
39
40     fun onPasswordChange(value: String) {
41         password = value
42         error = null
43     }
44
45     fun onNicknameChange(value: String) {
46         nickname = value
47         error = null
48     }
49
50     fun signup() {
51         if (email.isBlank() || password.isBlank() || nickname.isBlank()) {
52             error = "모든 항목을 입력해 주세요."
53             return
54         }
55         viewModelScope.launch {
56             isLoading = true
57             error = null
58             when (val result = authRepository.signup(email.trim(), password, nickname.trim())) {
59                 is NetworkResult.Success -> _navigateToLogin.emit(Unit)
60                 is NetworkResult.ApiError -> error = result.message
61                 is NetworkResult.NetworkError -> error = "네트워크 오류가 발생했습니다."
62             }
63             isLoading = false
64         }
65     }
66 }
```

```

1  package com.runway.android.ui.components
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.layout.Arrangement
5  import androidx.compose.foundation.layout.Box
6  import androidx.compose.foundation.layout.Column
7  import androidx.compose.foundation.layout.Row
8  import androidx.compose.foundation.layout.Spacer
9  import androidx.compose.foundation.layout.fillMaxWidth
10 import androidx.compose.foundation.layout.height
11 import androidx.compose.foundation.layout.padding
12 import androidx.compose.foundation.layout.size
13 import androidx.compose.foundation.layout.width
14 import androidx.compose.material3.LinearProgressIndicator
15 import androidx.compose.material3.MaterialTheme
16 import androidx.compose.material3.Surface
17 import androidx.compose.material3.Text
18 import androidx.compose.runtime.Composable
19 import androidx.compose.ui.Alignment
20 import androidx.compose.ui.Modifier
21 import androidx.compose.ui.graphics.Color
22 import androidx.compose.ui.text.font.FontWeight
23 import androidx.compose.ui.unit.dp
24 import com.runway.android.data.user.model.AchievementItem
25
26 @Composable
27 fun AchievementCard(
28     item: AchievementItem,
29     modifier: Modifier = Modifier,
30 ) {
31     val unlocked = item.unlocked
32     Surface(
33         modifier = modifier.fillMaxWidth(),
34         shape = MaterialTheme.shapes.large,
35         color = if (unlocked) MaterialTheme.colorScheme.primary.copy(alpha = 0.12f)
36             else MaterialTheme.colorScheme.surface,
37         border = BorderStroke(
38             width = if (unlocked) 1.5.dp else 1.dp,
39             color = if (unlocked) MaterialTheme.colorScheme.primary
40                 else MaterialTheme.colorScheme.outline,
41         ),
42     ) {
43         Row(
44             modifier = Modifier
45                 .fillMaxWidth()
46                 .padding(16.dp),
47             verticalAlignment = Alignment.CenterVertically,
48         ) {
49             // 배지 아이콘 (텍스트 기반 이모지)
50             Box(
51                 modifier = Modifier.size(48.dp),
52                 contentAlignment = Alignment.Center,
53             ) {
54                 Text(
55                     text = achievementEmoji(item.code),
56                     style = MaterialTheme.typography.headlineMedium,
57                 )
58             }
59
60             Spacer(modifier = Modifier.width(12.dp))
61
62             Column(modifier = Modifier.weight(1f)) {
63                 Row(
64                     verticalAlignment = Alignment.CenterVertically,
65                     horizontalArrangement = Arrangement.spacedBy(6.dp),
66                 ) {
67                     Text(
68                         text = item.title,
69                         style = MaterialTheme.typography.titleSmall.copy(fontWeight = FontWeight.Bold),
70                         color = if (unlocked) MaterialTheme.colorScheme.primary
71                             else MaterialTheme.colorScheme.onSurface,
72                     )
73                     if (unlocked) {
74                         Text(
75                             text = "달성",
76                             style = MaterialTheme.typography.labelSmall,
77                             color = MaterialTheme.colorScheme.primary,
78                         )
79                     }
80                 }
81                 Spacer(modifier = Modifier.height(2.dp))
82                 Text(
83                     text = item.description,

```

```

84         style = MaterialTheme.typography.bodySmall,
85         color = MaterialTheme.colorScheme.onSurfaceVariant,
86     )
87     if (!unlocked && item.target > 1) {
88         Spacer(modifier = Modifier.height(6.dp))
89         LinearProgressIndicator(
90             progress = { (item.progress.toFloat() / item.target.toFloat()).coerceIn(0f, 1f) },
91             modifier = Modifier.fillMaxWidth(),
92             color = MaterialTheme.colorScheme.primary,
93             trackColor = MaterialTheme.colorScheme.outline,
94         )
95         Spacer(modifier = Modifier.height(2.dp))
96         Text(
97             text = formatProgress(item),
98             style = MaterialTheme.typography.labelSmall,
99             color = MaterialTheme.colorScheme.onSurfaceVariant,
100         )
101     }
102 }
103 }
104 }
105 }
106
107 private fun achievementEmoji(code: String): String = when (code) {
108     "FIRST_RUN" -> "🏃"
109     "TOTAL_10K" -> "🏃"
110     "TOTAL_50K" -> "🏃"
111     "TOTAL_100K" -> "🏃"
112     "STREAK_3" -> "🔥"
113     "STREAK_7" -> "📅"
114     "FIRST_COURSE_CREATED" -> "📅"
115     "FIRST_COURSE_COMPLETED" -> "📅"
116     else -> "🏃"
117 }
118
119 private fun formatProgress(item: AchievementItem): String {
120     return when {
121         item.code.startsWith("TOTAL_") -> {
122             val progressKm = "%.1f".format(item.progress / 1000.0)
123             val targetKm = "%.0f".format(item.target / 1000.0)
124             "$progressKm / $targetKm km"
125         }
126         item.code.startsWith("STREAK_") -> "${item.progress} / ${item.target} days"
127         else -> "${item.progress} / ${item.target}"
128     }
129 }

```

```

1  package com.runway.android.ui.components
2
3  import android.content.Context
4  import android.content.Intent
5  import android.os.PowerManager
6  import android.provider.Settings
7  import androidx.compose.foundation.BorderStroke
8  import androidx.compose.foundation.layout.Column
9  import androidx.compose.foundation.layout.Row
10 import androidx.compose.foundation.layout.Spacer
11 import androidx.compose.foundation.layout.fillMaxWidth
12 import androidx.compose.foundation.layout.padding
13 import androidx.compose.foundation.layout.size
14 import androidx.compose.foundation.layout.width
15 import androidx.compose.material.icons.Icons
16 import androidx.compose.material.icons.filled.BatteryAlert
17 import androidx.compose.material.icons.filled.Close
18 import androidx.compose.material3.Icon
19 import androidx.compose.material3.IconButton
20 import androidx.compose.material3.MaterialTheme
21 import androidx.compose.material3.Surface
22 import androidx.compose.material3.Text
23 import androidx.compose.material3.TextButton
24 import androidx.compose.runtime.Composable
25 import androidx.compose.runtime.DisposableEffect
26 import androidx.compose.runtime.getValue
27 import androidx.compose.runtime.mutableStateOf
28 import androidx.compose.runtime.remember
29 import androidx.compose.runtime.setValue
30 import androidx.compose.ui.Alignment
31 import androidx.compose.ui.Modifier
32 import androidx.compose.ui.platform.LocalContext
33 import androidx.compose.ui.unit.dp
34 import androidx.lifecycle.Lifecycle
35 import androidx.lifecycle.LifecycleEventObserver
36 import androidx.lifecycle.compose.LocalLifecycleOwner
37
38 @Composable
39 fun rememberBatteryOptimizationIgnored(): Boolean {
40     val context = LocalContext.current
41     val lifecycleOwner = LocalLifecycleOwner.current
42     var ignored by remember {
43         mutableStateOf(context.isBatteryOptimizationIgnored())
44     }
45
46     DisposableEffect(context, lifecycleOwner) {
47         val observer = LifecycleEventObserver { _, event ->
48             if (event == Lifecycle.Event.ON_RESUME) {
49                 ignored = context.isBatteryOptimizationIgnored()
50             }
51         }
52         lifecycleOwner.lifecycle.addObserver(observer)
53         onDispose {
54             lifecycleOwner.lifecycle.removeObserver(observer)
55         }
56     }
57
58     return ignored
59 }
60
61 @Composable
62 fun BatteryOptimizationCard(
63     onDismiss: () -> Unit,
64     modifier: Modifier = Modifier,
65 ) {
66     val context = LocalContext.current
67
68     Surface(
69         modifier = modifier.fillMaxWidth(),
70         shape = MaterialTheme.shapes.extraLarge,
71         color = MaterialTheme.colorScheme.surface,
72         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
73     ) {
74         Column(modifier = Modifier.padding(horizontal = 16.dp, vertical = 12.dp)) {
75             Row(verticalAlignment = Alignment.CenterVertically) {
76                 Icon(
77                     imageVector = Icons.Filled.BatteryAlert,
78                     contentDescription = null,
79                     tint = MaterialTheme.colorScheme.onSurfaceVariant,
80                     modifier = Modifier.size(18.dp),
81                 )
82                 Spacer(modifier = Modifier.width(8.dp))
83                 Text(

```

```

84         text = "백그라운드 추적 안정성을 위해 배터리 최적화를 해제하세요.",
85         style = MaterialTheme.typography.bodySmall,
86         color = MaterialTheme.colorScheme.onSurfaceVariant,
87         modifier = Modifier.weight(1f),
88     )
89     IconButton(onClick = onDismiss, modifier = Modifier.size(32.dp)) {
90         Icon(
91             imageVector = Icons.Filled.Close,
92             contentDescription = "닫기",
93             tint = MaterialTheme.colorScheme.onSurfaceVariant,
94             modifier = Modifier.size(16.dp),
95         )
96     }
97 }
98 TextButton(
99     onClick = {
100         context.startActivity(
101             Intent(Settings.ACTION_IGNORE_BATTERY_OPTIMIZATIONS_SETTINGS)
102         )
103     },
104     modifier = Modifier.align(Alignment.End),
105 ) {
106     Text(
107         text = "설정 확인",
108         style = MaterialTheme.typography.labelMedium,
109         color = MaterialTheme.colorScheme.primary,
110     )
111 }
112 }
113 }
114 }
115
116 private fun Context.isBatteryOptimizationIgnored(): Boolean {
117     val powerManager = getSystemService(Context.POWER_SERVICE) as PowerManager
118     return powerManager.isIgnoringBatteryOptimizations(packageName)
119 }

```



```

1 package com.runway.android.ui.components
2
3 import androidx.compose.foundation.Canvas
4 import androidx.compose.foundation.background
5 import androidx.compose.material3.MaterialTheme
6 import androidx.compose.runtime.Composable
7 import androidx.compose.ui.Modifier
8 import androidx.compose.ui.geometry.Offset
9 import androidx.compose.ui.graphics.Color
10 import androidx.compose.ui.graphics.Path
11 import androidx.compose.ui.graphics.StrokeCap
12 import androidx.compose.ui.graphics.StrokeJoin
13 import androidx.compose.ui.graphics.drawscope.Stroke
14 import androidx.compose.ui.unit.dp
15 import com.runway.android.data.course.model.CoursePointResponse
16
17 /**
18  * Renders actual GPS course points onto a Canvas.
19  * Falls back to a decorative placeholder when [points] is empty.
20  */
21 @Composable
22 fun CourseRoutePreview(
23     points: List<CoursePointResponse>,
24     modifier: Modifier = Modifier,
25 ) {
26     val primaryColor = MaterialTheme.colorScheme.primary
27     val bgColor = MaterialTheme.colorScheme.surfaceVariant
28
29     Canvas(modifier = modifier.background(bgColor)) {
30         val w = size.width
31         val h = size.height
32         val paddingPx = w * 0.10f
33
34         val gridColor = primaryColor.copy(alpha = 0.10f)
35         val gridStep = 24.dp.toPx()
36         var xi = 0f
37         while (xi <= w) { drawLine(gridColor, Offset(xi, 0f), Offset(xi, h), 1f); xi += gridStep }
38         var yi = 0f
39         while (yi <= h) { drawLine(gridColor, Offset(0f, yi), Offset(w, yi), 1f); yi += gridStep }
40
41         val strokePx = 2.5.dp.toPx()
42         val dotPx = 5.dp.toPx()
43
44         if (points.size >= 2) {
45             val minLat = points.minOf { it.latitude }
46             val maxLat = points.maxOf { it.latitude }
47             val minLon = points.minOf { it.longitude }
48             val maxLon = points.maxOf { it.longitude }
49             val latRange = (maxLat - minLat).coerceAtLeast(0.0001)
50             val lonRange = (maxLon - minLon).coerceAtLeast(0.0001)
51
52             val drawW = w - 2 * paddingPx
53             val drawH = h - 2 * paddingPx
54
55             fun toOffset(p: CoursePointResponse) = Offset(
56                 x = paddingPx + ((p.longitude - minLon) / lonRange * drawW).toFloat(),
57                 y = paddingPx + ((1.0 - (p.latitude - minLat) / latRange) * drawH).toFloat(),
58             )
59
60             val offsets = points.map { toOffset(it) }
61             val path = Path().apply {
62                 moveTo(offsets[0].x, offsets[0].y)
63                 offsets.drop(1).forEach { lineTo(it.x, it.y) }
64             }
65
66             drawPath(path, primaryColor, style = Stroke(strokePx, cap = StrokeCap.Round, join = StrokeJoin.Round))
67             drawCircle(primaryColor, dotPx, offsets.first())
68             drawCircle(Color.White, dotPx, offsets.last())
69             drawCircle(primaryColor, dotPx - 2.dp.toPx(), offsets.last(), style = Stroke(2.dp.toPx()))
70         } else {
71             // Placeholder path when no points available
72             val startPt = Offset(w * 0.10f, h * 0.72f)
73             val endPt = Offset(w * 0.88f, h * 0.28f)
74             val path = Path().apply {
75                 moveTo(startPt.x, startPt.y)
76                 cubicTo(w * 0.30f, h * 0.15f, w * 0.62f, h * 0.92f, endPt.x, endPt.y)
77             }
78             drawPath(path, primaryColor.copy(alpha = 0.4f), style = Stroke(strokePx, cap = StrokeCap.Round, join = StrokeJoin.Round))
79             drawCircle(primaryColor.copy(alpha = 0.4f), dotPx, startPt)
80             drawCircle(primaryColor.copy(alpha = 0.4f), dotPx, endPt, style = Stroke(2.dp.toPx()))
81         }
82     }
83 }

```



```

1  package com.runway.android.ui.components
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.Canvas
5  import androidx.compose.foundation.background
6  import androidx.compose.foundation.layout.Arrangement
7  import androidx.compose.foundation.layout.Box
8  import androidx.compose.foundation.layout.Column
9  import androidx.compose.foundation.layout.Row
10 import androidx.compose.foundation.layout.Spacer
11 import androidx.compose.foundation.layout.fillMaxSize
12 import androidx.compose.foundation.layout.fillMaxWidth
13 import androidx.compose.foundation.layout.height
14 import androidx.compose.foundation.layout.padding
15 import androidx.compose.foundation.layout.size
16 import androidx.compose.foundation.layout.width
17 import androidx.compose.foundation.shape.RoundedCornerShape
18 import androidx.compose.material.icons.Icons
19 import androidx.compose.material.icons.filled.LocationOn
20 import androidx.compose.material.icons.filled.Loop
21 import androidx.compose.material.icons.filled.Star
22 import androidx.compose.material3.Icon
23 import androidx.compose.material3.MaterialTheme
24 import androidx.compose.material3.Surface
25 import androidx.compose.material3.Text
26 import androidx.compose.runtime.Composable
27 import androidx.compose.ui.Alignment
28 import androidx.compose.ui.Modifier
29 import androidx.compose.ui.draw.clip
30 import androidx.compose.ui.geometry.Offset
31 import androidx.compose.ui.graphics.Brush
32 import androidx.compose.ui.graphics.Color
33 import androidx.compose.ui.graphics.Path
34 import androidx.compose.ui.graphics.StrokeCap
35 import androidx.compose.ui.graphics.StrokeJoin
36 import androidx.compose.ui.graphics.drawscope.Stroke
37 import androidx.compose.ui.text.font.FontWeight
38 import androidx.compose.ui.unit.dp
39 import androidx.compose.ui.unit.sp
40 import com.runway.android.data.course.model.GeoPoint
41 import com.runway.android.data.course.model.NearbyCourseItem
42 import com.runway.android.ui.discover.DiscoverViewModel
43 import com.runway.android.ui.theme.LocalIsDarkTheme
44
45 private val DarkBgTop = Color(0xFF0E1117)
46 private val DarkBgBottom = Color(0xFF161B26)
47 private val LightBgTop = Color(0xFFE4EAF2)
48 private val LightBgBottom = Color(0xFFEDF2F8)
49 private val StartGreen = Color(0xFF4ADE80)
50 private val EndOrange = Color(0xFFFFB923C)
51
52 @Composable
53 fun DiscoverCourseCard(
54     course: NearbyCourseItem,
55     modifier: Modifier = Modifier,
56     onClick: () -> Unit = {},
57 ) {
58     val isPopular = DiscoverViewModel.isPopular(course)
59     val isNew = DiscoverViewModel.isNew(course)
60     val completionRate = DiscoverViewModel.completionRate(course)
61     val isDark = LocalIsDarkTheme.current
62
63     Surface(
64         modifier = modifier.fillMaxWidth(),
65         shape = MaterialTheme.shapes.extraLarge,
66         color = MaterialTheme.colorScheme.surface,
67         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
68         onClick = onClick,
69     ) {
70         Column {
71             Box {
72                 SportyCourseCanvas(
73                     routePoints = course.routePoints,
74                     courseId = course.courseId,
75                     accentColor = MaterialTheme.colorScheme.primary,
76                     isDark = isDark,
77                     modifier = modifier
78                         .fillMaxWidth()
79                         .height(108.dp)
80                         .clip(MaterialTheme.shapes.extraLarge),
81                 )
82                 // 거리 배지 - 좌상단
83                 Surface(

```

```

84         modifier = Modifier
85             .align(Alignment.TopStart)
86             .padding(10.dp),
87         shape = RoundedCornerShape(6.dp),
88         color = if (isDark) Color.White.copy(alpha = 0.10f)
89             else Color.Black.copy(alpha = 0.07f),
90     ) {
91         Text(
92             text = formatDistance(course.distanceMeters),
93             style = MaterialTheme.typography.labelSmall.copy(
94                 fontWeight = FontWeight.Bold,
95             ),
96             color = if (isDark) Color.White.copy(alpha = 0.90f)
97                 else Color.Black.copy(alpha = 0.65f),
98             modifier = Modifier.padding(horizontal = 7.dp, vertical = 3.dp),
99         )
100     }
101     // 품질 배지 - 우상단
102     if (isPopular || isNew) {
103         Surface(
104             modifier = Modifier
105                 .align(Alignment.TopEnd)
106                 .padding(10.dp),
107             shape = RoundedCornerShape(6.dp),
108             color = if (isPopular) MaterialTheme.colorScheme.tertiary
109                 else MaterialTheme.colorScheme.secondary,
110         ) {
111             Text(
112                 text = if (isPopular) "🔥 인기" else "📖 신규",
113                 style = MaterialTheme.typography.labelSmall,
114                 color = if (isPopular) MaterialTheme.colorScheme.onTertiary
115                     else MaterialTheme.colorScheme.onSecondary,
116                 modifier = Modifier.padding(horizontal = 8.dp, vertical = 3.dp),
117             )
118         }
119     }
120 }
121
122 Column(modifier = Modifier.padding(14.dp)) {
123     Row(
124         modifier = Modifier.fillMaxWidth(),
125         horizontalArrangement = Arrangement.SpaceBetween,
126         verticalAlignment = Alignment.CenterVertically,
127     ) {
128         Text(
129             text = course.name,
130             style = MaterialTheme.typography.titleMedium,
131             color = MaterialTheme.colorScheme.onSurface,
132             modifier = Modifier.weight(1f),
133         )
134         if (course.isLoop) {
135             Spacer(modifier = Modifier.width(8.dp))
136             Surface(
137                 shape = MaterialTheme.shapes.small,
138                 color = MaterialTheme.colorScheme.primaryContainer,
139             ) {
140                 Row(
141                     modifier = Modifier.padding(horizontal = 8.dp, vertical = 3.dp),
142                     verticalAlignment = Alignment.CenterVertically,
143                 ) {
144                     Icon(
145                         imageVector = Icons.Filled.Loop,
146                         contentDescription = null,
147                         tint = MaterialTheme.colorScheme.onPrimaryContainer,
148                         modifier = Modifier
149                             .padding(end = 2.dp)
150                             .height(12.dp),
151                     )
152                     Text(
153                         text = "루프",
154                         style = MaterialTheme.typography.labelSmall,
155                         color = MaterialTheme.colorScheme.onPrimaryContainer,
156                     )
157                 }
158             }
159         }
160     }
161
162     Spacer(modifier = Modifier.height(4.dp))
163
164     Row(verticalAlignment = Alignment.CenterVertically) {
165         Icon(
166             imageVector = Icons.Filled.LocationOn,
167             contentDescription = null,
168             tint = MaterialTheme.colorScheme.onSurfaceVariant,
169             modifier = Modifier.height(13.dp),

```

```

170         )
171         Spacer(modifier = Modifier.width(2.dp))
172         Text(
173             text = "${formatDistance(course.distanceFromMeMeters)} 거리 · ${formatDistance(course.distanceMeters)}",
174             style = MaterialTheme.typography.bodyMedium,
175             color = MaterialTheme.colorScheme.onSurfaceVariant,
176         )
177     }
178
179     Spacer(modifier = Modifier.height(2.dp))
180
181     Row(
182         modifier = Modifier.fillMaxWidth(),
183         horizontalArrangement = Arrangement.SpaceBetween,
184         verticalAlignment = Alignment.CenterVertically,
185     ) {
186         val rateText = if (course.attemptCount > 0)
187             " · 완주율 ${((completionRate * 100).toInt())}%"
188         else ""
189         Text(
190             text = "도전 ${course.attemptCount}회 · 완주 ${course.completionCount}회$rateText",
191             style = MaterialTheme.typography.bodySmall,
192             color = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.7f),
193             modifier = Modifier.weight(1f),
194         )
195         if (course.avgRating != null && course.ratingCount != null && course.ratingCount > 0) {
196             Spacer(modifier = Modifier.width(8.dp))
197             Row(verticalAlignment = Alignment.CenterVertically) {
198                 Icon(
199                     imageVector = Icons.Filled.Star,
200                     contentDescription = null,
201                     tint = MaterialTheme.colorScheme.primary,
202                     modifier = Modifier.size(12.dp),
203                 )
204                 Spacer(modifier = Modifier.width(2.dp))
205                 Text(
206                     text = "%.1f".format(course.avgRating),
207                     style = MaterialTheme.typography.labelSmall,
208                     color = MaterialTheme.colorScheme.onSurface,
209                 )
210                 Spacer(modifier = Modifier.width(2.dp))
211                 Text(
212                     text = "(${course.ratingCount})",
213                     style = MaterialTheme.typography.labelSmall,
214                     color = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.6f),
215                 )
216             }
217         }
218     }
219 }
220
221 }
222
223
224 @Composable
225 private fun SportyCourseCanvas(
226     routePoints: List<GeoPoint>,
227     courseId: String,
228     accentColor: Color,
229     isDark: Boolean,
230     modifier: Modifier = Modifier,
231 ) {
232     val bgColors = if (isDark) listOf(DarkBgTop, DarkBgBottom)
233     else listOf(LightBgTop, LightBgBottom)
234     Box(modifier = modifier.background(Brush.verticalGradient(bgColors))) {
235         Canvas(modifier = Modifier.fillMaxSize()) {
236             val w = size.width
237             val h = size.height
238             val pad = 16.dp.toPx()
239
240             // 도트 그리드
241             val dotStep = 22.dp.toPx()
242             val dotR = 1.dp.toPx()
243             val dotColor = if (isDark) Color.White.copy(alpha = 0.055f)
244             else Color.Black.copy(alpha = 0.06f)
245             var xi = dotStep
246             while (xi < w) {
247                 var yi = dotStep
248                 while (yi < h) {
249                     drawCircle(dotColor, dotR, Offset(xi, yi))
250                     yi += dotStep
251                 }
252                 xi += dotStep
253             }
254
255             if (routePoints.size < 2) return@Canvas

```

```

256
257 // GPS 좌표 - 캔버스 좌표 투영 (종횡비 유지, 중앙 정렬)
258 val minLat = routePoints.minOf { it.latitude }
259 val maxLat = routePoints.maxOf { it.latitude }
260 val minLon = routePoints.minOf { it.longitude }
261 val maxLon = routePoints.maxOf { it.longitude }
262 val latSpan = (maxLat - minLat).coerceAtLeast(0.0001)
263 val lonSpan = (maxLon - minLon).coerceAtLeast(0.0001)
264
265 val drawW = w - pad * 2
266 val drawH = h - pad * 2
267 val scaleByLat = drawH / latSpan
268 val scaleByLon = drawW / lonSpan
269 val scale = minOf(scaleByLat, scaleByLon)
270
271 val projW = lonSpan * scale
272 val projH = latSpan * scale
273 val offsetX = pad + (drawW - projW) / 2f
274 val offsetY = pad + (drawH - projH) / 2f
275
276 fun project(pt: com.runway.android.data.course.model.GeoPoint) = Offset(
277     x = (offsetX + (pt.longitude - minLon) * scale).toFloat(),
278     y = (offsetY + (maxLat - pt.latitude) * scale).toFloat(),
279 )
280
281 val offsets = routePoints.map { project(it) }
282
283 val path = Path().apply {
284     moveTo(offsets.first().x, offsets.first().y)
285     offsets.drop(1).forEach { lineTo(it.x, it.y) }
286 }
287
288 // 글로우 레이어
289 drawPath(path, accentColor.copy(alpha = 0.07f), style = Stroke(28.dp.toPx(), cap = StrokeCap.Round, join = StrokeJoin.Round
d))
290 drawPath(path, accentColor.copy(alpha = 0.18f), style = Stroke(12.dp.toPx(), cap = StrokeCap.Round, join = StrokeJoin.Round
d))
291 drawPath(path, accentColor.copy(alpha = 0.40f), style = Stroke(5.dp.toPx(), cap = StrokeCap.Round, join = StrokeJoin.Round
d))
292 // 메인 라인
293 drawPath(path, accentColor.copy(alpha = 0.95f), style = Stroke(2.dp.toPx(), cap = StrokeCap.Round, join = StrokeJoin.Round
d))
294
295 // 출발점 (초록)
296 val start = offsets.first()
297 drawCircle(StartGreen.copy(alpha = 0.25f), 9.dp.toPx(), start)
298 drawCircle(StartGreen.copy(alpha = 0.60f), 5.dp.toPx(), start)
299 drawCircle(StartGreen, 3.dp.toPx(), start)
300
301 // 도착점 (주황)
302 val end = offsets.last()
303 if (end != start) {
304     drawCircle(EndOrange.copy(alpha = 0.25f), 9.dp.toPx(), end)
305     drawCircle(EndOrange.copy(alpha = 0.60f), 5.dp.toPx(), end)
306     drawCircle(EndOrange, 3.dp.toPx(), end)
307 }
308 }
309 }
310 }
311
312 private fun formatDistance(meters: Double): String = when {
313     meters >= 1000 -> "%.1f km".format(meters / 1000)
314     else -> "${meters.toInt()} m"
315 }

```



```

1  package com.runway.android.ui.components
2
3  import android.annotation.SuppressLint
4  import androidx.compose.animation.core.EaseOut
5  import androidx.compose.animation.core.RepeatMode
6  import androidx.compose.animation.core.animateFloat
7  import androidx.compose.animation.core.infiniteRepeatable
8  import androidx.compose.animation.core.rememberInfiniteTransition
9  import androidx.compose.animation.core.tween
10 import androidx.compose.foundation.Canvas
11 import com.runway.android.ui.theme.LocalIsDarkTheme
12 import androidx.compose.foundation.layout.Box
13 import androidx.compose.foundation.layout.fillMaxSize
14 import androidx.compose.foundation.layout.size
15 import androidx.compose.material3.MaterialTheme
16 import androidx.compose.runtime.Composable
17 import androidx.compose.runtime.LaunchedEffect
18 import androidx.compose.runtime.getValue
19 import androidx.compose.runtime.remember
20 import androidx.compose.ui.Alignment
21 import androidx.compose.ui.Modifier
22 import androidx.compose.ui.geometry.Offset
23 import androidx.compose.ui.graphics.Color
24 import androidx.compose.ui.graphics.drawscope.Stroke
25 import androidx.compose.ui.platform.LocalContext
26 import androidx.compose.ui.unit.dp
27 import com.google.android.gms.maps.CameraUpdateFactory
28 import com.google.android.gms.maps.model.CameraPosition
29 import com.google.android.gms.maps.model.LatLng
30 import com.google.android.gms.maps.model.MapStyleOptions
31 import com.google.maps.android.compose.GoogleMap
32 import com.google.maps.android.compose.MapProperties
33 import com.google.maps.android.compose.MapUiSettings
34 import com.google.maps.android.compose.rememberCameraPositionState
35 import com.runway.android.R
36 import com.runway.android.core.map.MapPoint
37
38 @SuppressLint("MissingPermission")
39 @Composable
40 fun HomeMapView(
41     currentLocation: MapPoint?,
42     hasLocationPermission: Boolean,
43     modifier: Modifier = Modifier,
44 ) {
45     val context = LocalContext.current
46
47     val cameraPositionState = rememberCameraPositionState {
48         position = CameraPosition.fromLatLngZoom(LatLng(37.5665, 126.9780), 13f)
49     }
50
51     val isDark = LocalIsDarkTheme.current
52     val mapStyleOptions = remember(isDark) {
53         val styleRes = if (isDark) R.raw.map_style_dark else R.raw.map_style_light
54         runCatching { MapStyleOptions.loadRawResourceStyle(context, styleRes) }.getOrNull()
55     }
56
57     LaunchedEffect(currentLocation) {
58         currentLocation?.let { loc ->
59             cameraPositionState.animate(
60                 CameraUpdateFactory.newLatLngZoom(LatLng(loc.latitude, loc.longitude), 15.5f),
61                 durationMs = 900,
62             )
63         }
64     }
65
66     Box(modifier = modifier) {
67         GoogleMap(
68             modifier = Modifier.fillMaxSize(),
69             cameraPositionState = cameraPositionState,
70             properties = MapProperties(
71                 isMyLocationEnabled = false,
72                 mapStyleOptions = mapStyleOptions,
73             ),
74             uiSettings = MapUiSettings(
75                 zoomControlsEnabled = false,
76                 scrollGesturesEnabled = false,
77                 zoomGesturesEnabled = false,
78                 rotationGesturesEnabled = false,
79                 tiltGesturesEnabled = false,
80                 compassEnabled = false,
81                 mapToolbarEnabled = false,
82                 myLocationButtonEnabled = false,
83             ),

```

```

84     )
85
86     if (currentLocation != null && hasLocationPermission) {
87         PulsingLocationDot(
88             color = MaterialTheme.colorScheme.primary,
89             modifier = Modifier.align(Alignment.Center),
90         )
91     }
92 }
93 }
94
95 @Composable
96 private fun PulsingLocationDot(
97     color: Color,
98     modifier: Modifier = Modifier,
99 ) {
100     val transition = rememberInfiniteTransition(label = "gps_pulse")
101
102     val ring1 by transition.animateFloat(
103         initialValue = 0f,
104         targetValue = 1f,
105         animationSpec = infiniteRepeatable(
106             animation = tween(2000, easing = EaseOut),
107             repeatMode = RepeatMode.Restart,
108         ),
109         label = "ring1",
110     )
111     val ring2 by transition.animateFloat(
112         initialValue = 0f,
113         targetValue = 1f,
114         animationSpec = infiniteRepeatable(
115             animation = tween(2000, easing = EaseOut, delayMillis = 700),
116             repeatMode = RepeatMode.Restart,
117         ),
118         label = "ring2",
119     )
120     val ring3 by transition.animateFloat(
121         initialValue = 0f,
122         targetValue = 1f,
123         animationSpec = infiniteRepeatable(
124             animation = tween(2000, easing = EaseOut, delayMillis = 1400),
125             repeatMode = RepeatMode.Restart,
126         ),
127         label = "ring3",
128     )
129
130     Canvas(modifier = modifier.size(88.dp)) {
131         val center = Offset(size.width / 2f, size.height / 2f)
132         val maxRadius = size.width / 2f
133
134         // 파동 링 3개 (순차적으로 퍼짐)
135         for ((progress, delay) in listOf(ring1 to 0, ring2 to 700, ring3 to 1400)) {
136             if (progress > 0f) {
137                 drawCircle(
138                     color = color.copy(alpha = (1f - progress) * 0.35f),
139                     radius = maxRadius * progress,
140                     center = center,
141                 )
142                 drawCircle(
143                     color = color.copy(alpha = (1f - progress) * 0.55f),
144                     radius = maxRadius * progress,
145                     center = center,
146                     style = Stroke(1.5.dp.toPx()),
147                 )
148             }
149         }
150
151         // 흰 테두리 링 (GPS 수신 표시)
152         drawCircle(Color.White, 10.dp.toPx(), center)
153         drawCircle(Color.White.copy(alpha = 0.4f), 13.dp.toPx(), center, style = Stroke(1.dp.toPx()))
154
155         // 컨셉 색 중앙 dot
156         drawCircle(color, 7.dp.toPx(), center)
157     }
158 }

```

```

1  package com.runway.android.ui.components
2
3  import android.content.Intent
4  import android.net.Uri
5  import android.provider.Settings
6  import androidx.compose.foundation.layout.Column
7  import androidx.compose.foundation.layout.Spacer
8  import androidx.compose.foundation.layout.fillMaxWidth
9  import androidx.compose.foundation.layout.height
10 import androidx.compose.foundation.layout.padding
11 import androidx.compose.foundation.layout.size
12 import androidx.compose.material.icons.Icons
13 import androidx.compose.material.icons.filled.LocationOff
14 import androidx.compose.material3.Icon
15 import androidx.compose.material3.MaterialTheme
16 import androidx.compose.material3.Surface
17 import androidx.compose.material3.Text
18 import androidx.compose.runtime.Composable
19 import androidx.compose.ui.Alignment
20 import androidx.compose.ui.Modifier
21 import androidx.compose.ui.platform.LocalContext
22 import androidx.compose.ui.text.style.TextAlign
23 import androidx.compose.ui.unit.dp
24
25 @Composable
26 fun LocationPermissionCard(
27     isPermanentlyDenied: Boolean,
28     onRequestPermission: () -> Unit,
29     modifier: Modifier = Modifier,
30 ) {
31     val context = LocalContext.current
32
33     Surface(
34         modifier = modifier
35             .fillMaxWidth()
36             .padding(horizontal = 20.dp),
37         shape = MaterialTheme.shapes.extraLarge,
38         color = MaterialTheme.colorScheme.surface,
39     ) {
40         Column(
41             modifier = Modifier.padding(24.dp),
42             horizontalAlignment = Alignment.CenterHorizontally,
43         ) {
44             Icon(
45                 imageVector = Icons.Filled.LocationOff,
46                 contentDescription = null,
47                 tint = MaterialTheme.colorScheme.onSurfaceVariant,
48                 modifier = Modifier.size(40.dp),
49             )
50             Spacer(modifier = Modifier.height(12.dp))
51             Text(
52                 text = "위치 권한이 필요합니다",
53                 style = MaterialTheme.typography.titleMedium,
54                 color = MaterialTheme.colorScheme.onSurface,
55                 textAlign = TextAlign.Center,
56             )
57             Spacer(modifier = Modifier.height(8.dp))
58             Text(
59                 text = if (isPermanentlyDenied) {
60                     "위치 권한이 거부되었습니다. 설정에서 권한을 허용해 주세요."
61                 } else {
62                     "GPS 트래킹을 위해 정밀 위치 권한이 필요합니다."
63                 },
64                 style = MaterialTheme.typography.bodyMedium,
65                 color = MaterialTheme.colorScheme.onSurfaceVariant,
66                 textAlign = TextAlign.Center,
67             )
68             Spacer(modifier = Modifier.height(20.dp))
69             Surface(
70                 onClick = if (isPermanentlyDenied) {
71                     {
72                         context.startActivity(
73                             Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS).apply {
74                                 data = Uri.fromParts("package", context.packageName, null)
75                             }
76                         )
77                     }
78                 } else {
79                     onRequestPermission
80                 },
81                 shape = MaterialTheme.shapes.extraLarge,
82                 color = MaterialTheme.colorScheme.primary,
83             ) {

```

```
84         Text(  
85             text = if (isPermanentlyDenied) "설정으로 이동" else "권한 허용",  
86             style = MaterialTheme.typography.labelLarge,  
87             color = MaterialTheme.colorScheme.onPrimary,  
88             modifier = Modifier.padding(horizontal = 24.dp, vertical = 12.dp),  
89         )  
90     }  
91 }  
92 }  
93 }
```

```

1  package com.runway.android.ui.components
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.layout.Arrangement
5  import androidx.compose.foundation.layout.Column
6  import androidx.compose.foundation.layout.Row
7  import androidx.compose.foundation.layout.Spacer
8  import androidx.compose.foundation.layout.fillMaxWidth
9  import androidx.compose.foundation.layout.height
10 import androidx.compose.foundation.layout.padding
11 import androidx.compose.foundation.layout.size
12 import androidx.compose.foundation.layout.width
13 import androidx.compose.material.icons.Icons
14 import androidx.compose.material.icons.filled.EmojiEvents
15 import androidx.compose.material.icons.filled.LocalFireDepartment
16 import androidx.compose.material.icons.filled.Speed
17 import androidx.compose.material.icons.filled.Straighten
18 import androidx.compose.material3.Icon
19 import androidx.compose.material3.MaterialTheme
20 import androidx.compose.material3.Surface
21 import androidx.compose.material3.Text
22 import androidx.compose.runtime.Composable
23 import androidx.compose.ui.Alignment
24 import androidx.compose.ui.Modifier
25 import androidx.compose.ui.graphics.vector.ImageVector
26 import androidx.compose.ui.unit.dp
27 import com.runway.android.data.running.model.PersonalRecordItem
28 import com.runway.android.data.running.model.PersonalRecordsResponse
29
30 @Composable
31 fun PersonalRecordsSection(
32     records: PersonalRecordsResponse?,
33     onRecordClick: (runId: String) -> Unit,
34     modifier: Modifier = Modifier,
35 ) {
36     Column(modifier = modifier) {
37         Row(
38             verticalAlignment = Alignment.CenterVertically,
39             horizontalArrangement = Arrangement.spacedBy(6.dp),
40         ) {
41             Icon(
42                 imageVector = Icons.Filled.EmojiEvents,
43                 contentDescription = null,
44                 tint = MaterialTheme.colorScheme.primary,
45                 modifier = Modifier.size(16.dp),
46             )
47             Text(
48                 text = "개인 최고 기록",
49                 style = MaterialTheme.typography.labelMedium,
50                 color = MaterialTheme.colorScheme.onSurfaceVariant,
51             )
52         }
53
54         Spacer(modifier = Modifier.height(8.dp))
55
56         if (records == null || records.totalCompletedRuns == 0L) {
57             Surface(
58                 modifier = Modifier.fillMaxWidth(),
59                 shape = MaterialTheme.shapes.extraLarge,
60                 color = MaterialTheme.colorScheme.surface,
61                 border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
62             ) {
63                 Text(
64                     text = "완료된 러닝 기록이 쌓이면 개인 최고 기록이 표시됩니다.",
65                     style = MaterialTheme.typography.bodySmall,
66                     color = MaterialTheme.colorScheme.onSurfaceVariant,
67                     modifier = Modifier.padding(horizontal = 16.dp, vertical = 14.dp),
68                 )
69             }
70             return
71         }
72
73         Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
74             records.longestRun?.let {
75                 PersonalRecordCard(
76                     icon = Icons.Filled.Straighten,
77                     label = "가장 긴 러닝",
78                     value = formatDistance(it.distanceMeters),
79                     sub = formatDuration(it.durationSeconds),
80                     item = it,
81                     onClick = onRecordClick,
82                 )
83             }

```

```

84         records.fastestPace?.let {
85             PersonalRecordCard(
86                 icon = Icons.Filled.Speed,
87                 label = "최고 페이스",
88                 value = formatPace(it.avgPaceSecondsPerKm),
89                 sub = formatDistance(it.distanceMeters),
90                 item = it,
91                 onClick = onRecordClick,
92             )
93         }
94         records.mostCalories?.let {
95             PersonalRecordCard(
96                 icon = Icons.Filled.LocalFireDepartment,
97                 label = "최고 칼로리",
98                 value = "${it.caloriesBurned ?: 0} kcal",
99                 sub = formatDistance(it.distanceMeters),
100                 item = it,
101                 onClick = onRecordClick,
102             )
103         }
104     }
105 }
106 }
107
108 @Composable
109 private fun PersonalRecordCard(
110     icon: ImageVector,
111     label: String,
112     value: String,
113     sub: String,
114     item: PersonalRecordItem,
115     onClick: (runId: String) -> Unit,
116 ) {
117     Surface(
118         modifier = Modifier.fillMaxWidth(),
119         shape = MaterialTheme.shapes.extraLarge,
120         color = MaterialTheme.colorScheme.surface,
121         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
122         onClick = { onClick(item.runId) },
123     ) {
124         Row(
125             modifier = Modifier
126                 .fillMaxWidth()
127                 .padding(horizontal = 16.dp, vertical = 12.dp),
128             verticalAlignment = Alignment.CenterVertically,
129         ) {
130             Icon(
131                 imageVector = icon,
132                 contentDescription = null,
133                 tint = MaterialTheme.colorScheme.primary,
134                 modifier = Modifier.size(18.dp),
135             )
136             Spacer(modifier = Modifier.width(12.dp))
137             Column(modifier = Modifier.weight(1f)) {
138                 Text(
139                     text = label,
140                     style = MaterialTheme.typography.labelSmall,
141                     color = MaterialTheme.colorScheme.onSurfaceVariant,
142                 )
143                 Text(
144                     text = value,
145                     style = MaterialTheme.typography.titleMedium,
146                     color = MaterialTheme.colorScheme.onSurface,
147                 )
148             }
149             Text(
150                 text = sub,
151                 style = MaterialTheme.typography.bodySmall,
152                 color = MaterialTheme.colorScheme.onSurfaceVariant,
153             )
154         }
155     }
156 }
157
158 private fun formatDistance(meters: Double?): String {
159     if (meters == null) return "--"
160     return if (meters >= 1000) "%.1f km".format(meters / 1000) else "${meters.toInt()} m"
161 }
162
163 private fun formatDuration(seconds: Int?): String {
164     if (seconds == null) return "--"
165     return "%d:%02d".format(seconds / 60, seconds % 60)
166 }
167
168 private fun formatPace(secsPerKm: Int?): String {
169     if (secsPerKm == null || secsPerKm <= 0) return "--'--\""

```

```
170     return "%d'%02d\"/km".format(secsPerKm / 60, secsPerKm % 60)
171 }
```

```

1  package com.runway.android.ui.components
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.background
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.layout.Box
7  import androidx.compose.foundation.layout.Column
8  import androidx.compose.foundation.layout.Row
9  import androidx.compose.foundation.layout.Spacer
10 import androidx.compose.foundation.layout.fillMaxWidth
11 import androidx.compose.foundation.layout.padding
12 import androidx.compose.foundation.layout.size
13 import androidx.compose.foundation.layout.width
14 import androidx.compose.material.icons.Icons
15 import androidx.compose.material.icons.automirrored.filled.DirectionsRun
16 import androidx.compose.material.icons.automirrored.filled.KeyboardArrowRight
17 import androidx.compose.material3.Icon
18 import androidx.compose.material3.MaterialTheme
19 import androidx.compose.material3.Surface
20 import androidx.compose.material3.Text
21 import androidx.compose.runtime.Composable
22 import androidx.compose.ui.Alignment
23 import androidx.compose.ui.Modifier
24 import androidx.compose.ui.unit.dp
25
26 data class RecentRun(
27     val runId: String,
28     val day: String,
29     val distanceKm: String,
30     val pace: String,
31     val duration: String,
32 )
33
34 @Composable
35 fun RecentRunCard(
36     run: RecentRun,
37     modifier: Modifier = Modifier,
38     onClick: (() -> Unit)? = null,
39 ) {
40     Surface(
41         modifier = modifier
42             .fillMaxWidth()
43             .then(if (onClick != null) Modifier.clickable { onClick() } else Modifier),
44         shape = MaterialTheme.shapes.extraLarge,
45         color = MaterialTheme.colorScheme.surface,
46         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
47     ) {
48         Row(
49             modifier = Modifier.padding(12.dp),
50             verticalAlignment = Alignment.CenterVertically,
51         ) {
52             Box(
53                 modifier = Modifier
54                     .size(44.dp)
55                     .background(
56                         color = MaterialTheme.colorScheme.surfaceVariant,
57                         shape = MaterialTheme.shapes.medium,
58                     ),
59                 contentAlignment = Alignment.Center,
60             ) {
61                 Icon(
62                     imageVector = Icons.AutoMirrored.Filled.DirectionsRun,
63                     contentDescription = null,
64                     tint = MaterialTheme.colorScheme.primary,
65                     modifier = Modifier.size(20.dp),
66                 )
67             }
68
69             Spacer(modifier = Modifier.width(12.dp))
70
71             Column(modifier = Modifier.weight(1f)) {
72                 Text(
73                     text = "${run.distanceKm} km · ${run.duration}",
74                     style = MaterialTheme.typography.titleMedium,
75                     color = MaterialTheme.colorScheme.onSurface,
76                 )
77                 Text(
78                     text = "${run.day} · Pace ${run.pace}/km",
79                     style = MaterialTheme.typography.bodyMedium,
80                     color = MaterialTheme.colorScheme.onSurfaceVariant,
81                 )
82             }
83         }
84     }

```



```
84         Icon(  
85             imageVector = Icons.AutoMirrored.Filled.KeyboardArrowRight,  
86             contentDescription = null,  
87             tint = MaterialTheme.colorScheme.onSurfaceVariant,  
88             modifier = Modifier.size(20.dp),  
89         )  
90     }  
91 }  
92 }
```

```

1  package com.runway.android.ui.components
2
3  import android.content.Context
4  import android.graphics.Bitmap
5  import android.graphics.Canvas as AndroidCanvas
6  import android.graphics.Paint as AndroidPaint
7  import android.location.Location
8  import androidx.compose.foundation.Canvas
9  import androidx.compose.foundation.background
10 import androidx.compose.runtime.LaunchedEffect
11 import androidx.compose.runtime.getValue
12 import androidx.compose.runtime.mutableStateOf
13 import androidx.compose.runtime.setValue
14 import kotlinx.coroutines.delay
15 import androidx.compose.ui.graphics.toArgb
16 import com.runway.android.ui.theme.LocalIsDarkTheme
17 import androidx.compose.material3.MaterialTheme
18 import androidx.compose.runtime.Composable
19 import androidx.compose.runtime.remember
20 import androidx.compose.ui.Modifier
21 import androidx.compose.ui.geometry.Offset
22 import androidx.compose.ui.graphics.Color
23 import androidx.compose.ui.graphics.Path
24 import androidx.compose.ui.graphics.StrokeCap
25 import androidx.compose.ui.graphics.StrokeJoin
26 import androidx.compose.ui.graphics.drawscope.Stroke
27 import androidx.compose.ui.platform.LocalContext
28 import androidx.compose.ui.unit.dp
29 import com.google.android.gms.maps.CameraUpdateFactory
30 import com.google.android.gms.maps.model.BitmapDescriptorFactory
31 import com.google.android.gms.maps.model.JointType
32 import com.google.android.gms.maps.model.LatLng
33 import com.google.android.gms.maps.model.MapStyleOptions
34 import com.google.android.gms.maps.model.RoundCap
35 import com.google.maps.android.compose.GoogleMap
36 import com.google.maps.android.compose.MapProperties
37 import com.google.maps.android.compose.MapUiSettings
38 import com.google.maps.android.compose.Marker
39 import com.google.maps.android.compose.MarkerState
40 import com.google.maps.android.compose.Polyline
41 import com.google.maps.android.compose.rememberCameraPositionState
42 import com.runway.android.R
43 import com.runway.android.core.map.MapPoint
44 import com.runway.android.core.map.toLatLngBounds
45
46 /**
47  * Shows a GoogleMap with the route polyline when [points] has ≥ 2 entries.
48  * Falls back to a Canvas route drawing when < 2 points.
49  * [currentLocation] overlays an azure marker for live GPS position when non-null.
50  */
51 @Composable
52 fun RouteMapView(
53     points: List<MapPoint>,
54     modifier: Modifier = Modifier,
55     currentLocation: MapPoint? = null,
56     gesturesEnabled: Boolean = false,
57     showKilometerMarkers: Boolean = false,
58     /** true 이면 GPS 위치 변경마다 현재 위치 중심 zoom 17로 카메라를 따라감 */
59     followCurrentLocation: Boolean = false,
60     onClick: (() -> Unit)? = null,
61 ) {
62     if (points.size >= 2) {
63         RouteGoogleMap(
64             points = points,
65             currentLocation = currentLocation,
66             modifier = modifier,
67             gesturesEnabled = gesturesEnabled,
68             showKilometerMarkers = showKilometerMarkers,
69             followCurrentLocation = followCurrentLocation,
70             onClick = onClick,
71         )
72     } else {
73         RouteCanvasFallback(modifier = modifier)
74     }
75 }
76
77 @Composable
78 private fun RouteGoogleMap(
79     points: List<MapPoint>,
80     currentLocation: MapPoint?,
81     modifier: Modifier,
82     gesturesEnabled: Boolean,
83     showKilometerMarkers: Boolean,

```

```

84     followCurrentLocation: Boolean,
85     onClick: (() -> Unit)? = null,
86 ) {
87     val context = LocalContext.current
88     val latLngs = remember(points) { points.map { LatLng(it.latitude, it.longitude) } }
89     val bounds = remember(points) { points.toLatLngBounds() }
90     val cameraPositionState = rememberCameraPositionState()
91     val primaryColor = MaterialTheme.colorScheme.primary
92     val kilometerMarkers = remember(points, showKilometerMarkers) {
93         if (showKilometerMarkers) points.calculateKilometerMarkers() else emptyList()
94     }
95     val primaryColorArgb = primaryColor.toArgb()
96     val currentLatLng = remember(currentLocation) {
97         currentLocation?.let { LatLng(it.latitude, it.longitude) }
98     }
99     val isDark = LocalIsDarkTheme.current
100    val mapStyleOptions = remember(isDark) {
101        val styleRes = if (isDark) R.raw.map_style_discover_dark else R.raw.map_style_light
102        runCatching { MapStyleOptions.loadRawResourceStyle(context, styleRes) }.getOrNull()
103    }
104
105    // 현재 위치 추적 카메라 - GPS 위치 변경마다 zoom 17로 따라감
106    var hasSetInitialLocation by remember { mutableStateOf(false) }
107    LaunchedEffect(currentLatLng) {
108        if (!followCurrentLocation || currentLatLng == null) return@LaunchedEffect
109        if (!hasSetInitialLocation) {
110            cameraPositionState.move(CameraUpdateFactory.newLatLngZoom(currentLatLng, 17f))
111            hasSetInitialLocation = true
112        } else {
113            cameraPositionState.animate(CameraUpdateFactory.newLatLngZoom(currentLatLng, 17f), 800)
114        }
115    }
116
117    // 최초 1회만 bounds 카메라 이동 (onMapLoaded는 타일 재로드 시마다 재호출되므로 상태로 보호)
118    var mapLoaded by remember { mutableStateOf(false) }
119    LaunchedEffect(mapLoaded) {
120        if (!mapLoaded) return@LaunchedEffect
121        val needBounds = !followCurrentLocation || currentLatLng == null
122        if (!needBounds) return@LaunchedEffect
123        delay(50)
124        runCatching {
125            cameraPositionState.animate(CameraUpdateFactory.newLatLngBounds(bounds, 48), 300)
126        }
127    }
128
129    GoogleMap(
130        modifier = modifier,
131        cameraPositionState = cameraPositionState,
132        properties = MapProperties(isMyLocationEnabled = false, mapStyleOptions = mapStyleOptions),
133        uiSettings = MapUiSettings(
134            zoomControlsEnabled = gesturesEnabled,
135            scrollGesturesEnabled = gesturesEnabled,
136            zoomGesturesEnabled = gesturesEnabled,
137            rotationGesturesEnabled = gesturesEnabled,
138            tiltGesturesEnabled = gesturesEnabled,
139            compassEnabled = gesturesEnabled,
140            mapToolbarEnabled = false,
141        ),
142        onMapClick = { onClick?.invoke() },
143        onMapLoaded = { if (!mapLoaded) mapLoaded = true },
144    ) {
145        Polyline(
146            points = latLngs,
147            color = primaryColor,
148            width = 8f,
149            startCap = RoundCap(),
150            endCap = RoundCap(),
151            jointType = JointType.ROUND,
152        )
153        kilometerMarkers.forEach { marker ->
154            val icon = remember(marker.km, primaryColorArgb) {
155                BitmapDescriptorFactory.fromBitmap(
156                    createKmMarkerBitmap(context, marker.km, primaryColorArgb)
157                )
158            }
159            Marker(
160                state = MarkerState(position = marker.position),
161                icon = icon,
162                title = "${marker.km}km",
163                zIndex = 1f,
164                anchor = Offset(0.5f, 0.5f),
165            )
166        }
167        val startIcon = remember(context) {
168            BitmapDescriptorFactory.fromBitmap(createStartMarkerBitmap(context))
169        }

```

```

170     val endIcon = remember(context) {
171         BitmapDescriptorFactory.fromBitmap(createEndMarkerBitmap(context))
172     }
173     Marker(
174         state = MarkerState(position = latLngs.first()),
175         icon = startIcon,
176         title = "출발",
177         anchor = Offset(0.5f, 0.5f),
178         zIndex = 2f,
179     )
180     Marker(
181         state = MarkerState(position = latLngs.last()),
182         icon = endIcon,
183         title = "도착",
184         anchor = Offset(0.5f, 0.5f),
185         zIndex = 2f,
186     )
187     if (currentLatLng != null) {
188         Marker(
189             state = MarkerState(position = currentLatLng),
190             title = "현재 위치",
191             icon = BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_AZURE),
192         )
193     }
194 }
195 }
196
197 private data class KilometerMarker(
198     val km: Int,
199     val position: LatLng,
200 )
201
202 private fun List<MapPoint>.calculateKilometerMarkers(): List<KilometerMarker> {
203     if (size < 2) return emptyList()
204
205     val markers = mutableListOf<KilometerMarker>()
206     var cumulativeMeters = 0.0
207     var nextKilometer = 1
208
209     for (i in 1 until size) {
210         val start = this[i - 1]
211         val end = this[i]
212         val segmentMeters = distanceMeters(start, end)
213         if (segmentMeters <= 0.0) continue
214
215         while (cumulativeMeters + segmentMeters >= nextKilometer * 1000.0) {
216             val targetIntoSegment = nextKilometer * 1000.0 - cumulativeMeters
217             val fraction = (targetIntoSegment / segmentMeters).coerceIn(0.0, 1.0)
218             markers.add(
219                 KilometerMarker(
220                     km = nextKilometer,
221                     position = LatLng(
222                         start.latitude + (end.latitude - start.latitude) * fraction,
223                         start.longitude + (end.longitude - start.longitude) * fraction,
224                     ),
225                 )
226             )
227             nextKilometer++
228         }
229
230         cumulativeMeters += segmentMeters
231     }
232
233     return markers
234 }
235
236 private fun createKmMarkerBitmap(context: Context, km: Int, fillColor: Int): Bitmap {
237     val density = context.resources.displayMetrics.density
238     val sizePx = (28 * density).toInt().coerceAtLeast(28)
239     val bitmap = Bitmap.createBitmap(sizePx, sizePx, Bitmap.Config.ARGB_8888)
240     val canvas = AndroidCanvas(bitmap)
241     val paint = AndroidPaint(AndroidPaint.ANTI_ALIAS_FLAG)
242
243     // Filled circle
244     paint.color = fillColor
245     paint.style = AndroidPaint.Style.FILL
246     canvas.drawCircle(sizePx / 2f, sizePx / 2f, sizePx / 2f, paint)
247
248     // White border
249     paint.color = android.graphics.Color.WHITE
250     paint.style = AndroidPaint.Style.STROKE
251     paint.strokeWidth = density * 2f
252     canvas.drawCircle(sizePx / 2f, sizePx / 2f, sizePx / 2f - density, paint)
253
254     // km number
255     paint.style = AndroidPaint.Style.FILL

```

```

256     paint.color = android.graphics.Color.WHITE
257     paint.textSize = sizePx * 0.42f
258     paint.textAlign = AndroidPaint.Align.CENTER
259     val textY = sizePx / 2f - (paint.descent() + paint.ascent()) / 2f
260     canvas.drawText(km.toString(), sizePx / 2f, textY, paint)
261
262     return bitmap
263 }
264
265 private fun createRoutePillMarker(context: Context, label: String, colorHex: String): Bitmap {
266     val density = context.resources.displayMetrics.density
267     val textPaint = AndroidPaint(AndroidPaint.ANTI_ALIAS_FLAG).apply {
268         textSize = 11f * density
269         isFakeBoldText = true
270         textAlign = AndroidPaint.Align.CENTER
271     }
272     val textW = textPaint.measureText(label)
273     val padH = 10f * density
274     val padV = 6f * density
275     val w = (textW + padH * 2).toInt().coerceAtLeast(1)
276     val h = (textPaint.textSize + padV * 2).toInt().coerceAtLeast(1)
277     val r = h / 2f
278
279     val bitmap = Bitmap.createBitmap(w, h, Bitmap.Config.ARGB_8888)
280     val canvas = AndroidCanvas(bitmap)
281     val bgPaint = AndroidPaint(AndroidPaint.ANTI_ALIAS_FLAG)
282
283     // Filled pill
284     bgPaint.color = android.graphics.Color.parseColor(colorHex)
285     bgPaint.style = AndroidPaint.Style.FILL
286     canvas.drawRoundRect(0f, 0f, w.toFloat(), h.toFloat(), r, r, bgPaint)
287
288     // White border
289     bgPaint.color = android.graphics.Color.WHITE
290     bgPaint.style = AndroidPaint.Style.STROKE
291     bgPaint.strokeWidth = 1.5f * density
292     canvas.drawRoundRect(
293         bgPaint.strokeWidth / 2, bgPaint.strokeWidth / 2,
294         w - bgPaint.strokeWidth / 2, h - bgPaint.strokeWidth / 2,
295         r, r, bgPaint,
296     )
297
298     // Label
299     textPaint.color = android.graphics.Color.WHITE
300     val textY = h / 2f - (textPaint.descent() + textPaint.ascent()) / 2f
301     canvas.drawText(label, w / 2f, textY, textPaint)
302
303     return bitmap
304 }
305
306 private fun createStartMarkerBitmap(context: Context): Bitmap =
307     createRoutePillMarker(context, "Start", "#22C55E")
308
309 private fun createEndMarkerBitmap(context: Context): Bitmap =
310     createRoutePillMarker(context, "Finish", "#EF4444")
311
312 private fun distanceMeters(start: MapPoint, end: MapPoint): Double {
313     if (start.latitude == end.latitude && start.longitude == end.longitude) return 0.0
314     val result = FloatArray(1)
315     Location.distanceBetween(start.latitude, start.longitude, end.latitude, end.longitude, result)
316     return result[0].toDouble()
317 }
318
319 @Composable
320 private fun RouteCanvasFallback(modifier: Modifier) {
321     val primaryColor = MaterialTheme.colorScheme.primary
322     val bgColor = MaterialTheme.colorScheme.surfaceVariant
323
324     Canvas(modifier = modifier.background(bgColor)) {
325         val w = size.width
326         val h = size.height
327         val strokePx = 2.5.dp.toPx()
328         val dotPx = 5.dp.toPx()
329
330         val gridColor = primaryColor.copy(alpha = 0.10f)
331         val gridStep = 24.dp.toPx()
332         var xi = 0f
333         while (xi <= w) { drawLine(gridColor, Offset(xi, 0f), Offset(xi, h), 1f); xi += gridStep }
334         var yi = 0f
335         while (yi <= h) { drawLine(gridColor, Offset(0f, yi), Offset(w, yi), 1f); yi += gridStep }
336
337         val startPt = Offset(w * 0.10f, h * 0.72f)
338         val endPt = Offset(w * 0.88f, h * 0.28f)
339         val path = Path().apply {
340             moveTo(startPt.x, startPt.y)
341             cubicTo(w * 0.30f, h * 0.15f, w * 0.62f, h * 0.92f, endPt.x, endPt.y)

```

```
342     }
343     drawPath(path, primaryColor.copy(alpha = 0.4f), style = Stroke(strokePx, cap = StrokeCap.Round, join = StrokeJoin.Round))
344     drawCircle(primaryColor.copy(alpha = 0.4f), dotPx, startPt)
345     drawCircle(primaryColor.copy(alpha = 0.4f), dotPx, endPt, style = Stroke(2.dp.toPx()))
346 }
347 }
```

```

1  package com.runway.android.ui.components
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.layout.Arrangement
5  import androidx.compose.foundation.layout.Column
6  import androidx.compose.foundation.layout.Row
7  import androidx.compose.foundation.layout.Spacer
8  import androidx.compose.foundation.layout.fillMaxWidth
9  import androidx.compose.foundation.layout.height
10 import androidx.compose.foundation.layout.padding
11 import androidx.compose.material3.HorizontalDivider
12 import androidx.compose.material3.MaterialTheme
13 import androidx.compose.material3.Surface
14 import androidx.compose.material3.Text
15 import androidx.compose.runtime.Composable
16 import androidx.compose.ui.Alignment
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.unit.dp
19 import com.runway.android.core.util.formatDistance
20 import com.runway.android.core.util.formatDuration
21 import com.runway.android.core.util.formatPace
22
23 @Composable
24 fun RunDetailMetricGrid(
25     distanceMeters: Double?,
26     durationSeconds: Int?,
27     avgPaceSecondsPerKm: Int?,
28     caloriesBurned: Int?,
29     avgHeartRateBpm: Int?,
30     modifier: Modifier = Modifier,
31 ) {
32     Surface(
33         modifier = modifier.fillMaxWidth(),
34         shape = MaterialTheme.shapes.extraLarge,
35         color = MaterialTheme.colorScheme.surface,
36         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
37     ) {
38         Column(modifier = Modifier.padding(20.dp)) {
39             Text(
40                 text = "STATS",
41                 style = MaterialTheme.typography.labelMedium,
42                 color = MaterialTheme.colorScheme.onSurfaceVariant,
43             )
44             Spacer(modifier = Modifier.height(12.dp))
45
46             MetricRow(label = "거리", value = formatDistance(distanceMeters))
47             MetricDivider()
48             MetricRow(label = "소요 시간", value = formatDuration(durationSeconds))
49             MetricDivider()
50             MetricRow(label = "평균 페이스", value = formatPace(avgPaceSecondsPerKm))
51
52             caloriesBurned?.let { cal ->
53                 MetricDivider()
54                 MetricRow(label = "칼로리", value = "$cal kcal")
55             }
56             avgHeartRateBpm?.let { hr ->
57                 MetricDivider()
58                 MetricRow(label = "평균 심박수", value = "$hr bpm")
59             }
60         }
61     }
62 }
63
64 @Composable
65 private fun MetricRow(label: String, value: String) {
66     Row(
67         modifier = Modifier.fillMaxWidth(),
68         horizontalArrangement = Arrangement.SpaceBetween,
69         verticalAlignment = Alignment.CenterVertically,
70     ) {
71         Text(
72             text = label,
73             style = MaterialTheme.typography.bodyMedium,
74             color = MaterialTheme.colorScheme.onSurfaceVariant,
75         )
76         Text(
77             text = value,
78             style = MaterialTheme.typography.titleMedium,
79             color = MaterialTheme.colorScheme.onSurface,
80         )
81     }
82 }
83

```

```
84 @Composable
85 private fun MetricDivider() {
86     HorizontalDivider(
87         modifier = Modifier.padding(vertical = 8.dp),
88         color = MaterialTheme.colorScheme.outlineVariant,
89     )
90 }
```



```

1  package com.runway.android.ui.components
2
3  import androidx.compose.animation.core.LinearEasing
4  import androidx.compose.animation.core.RepeatMode
5  import androidx.compose.animation.core.Spring
6  import androidx.compose.animation.core.animateFloat
7  import androidx.compose.animation.core.animateFloatAsState
8  import androidx.compose.animation.core.infiniteRepeatable
9  import androidx.compose.animation.core.rememberInfiniteTransition
10 import androidx.compose.animation.core.spring
11 import androidx.compose.animation.core.tween
12 import androidx.compose.foundation.background
13 import androidx.compose.foundation.interaction.MutableInteractionSource
14 import androidx.compose.foundation.interaction.collectIsPressedAsState
15 import androidx.compose.foundation.layout.Arrangement
16 import androidx.compose.foundation.layout.Box
17 import androidx.compose.foundation.layout.BoxWithConstraints
18 import androidx.compose.foundation.layout.Column
19 import androidx.compose.foundation.layout.Row
20 import androidx.compose.foundation.layout.Spacer
21 import androidx.compose.foundation.layout.fillMaxHeight
22 import androidx.compose.foundation.layout.fillMaxSize
23 import androidx.compose.foundation.layout.fillMaxWidth
24 import androidx.compose.foundation.layout.height
25 import androidx.compose.foundation.layout.offset
26 import androidx.compose.foundation.layout.padding
27 import androidx.compose.foundation.layout.size
28 import androidx.compose.foundation.layout.width
29 import androidx.compose.foundation.layout.widthIn
30 import androidx.compose.foundation.layout.wrapContentWidth
31 import androidx.compose.foundation.shape.CircleShape
32 import androidx.compose.foundation.shape.RoundedCornerShape
33 import androidx.compose.material.icons.Icons
34 import androidx.compose.material.icons.filled.AcUnit
35 import androidx.compose.material.icons.filled.Air
36 import androidx.compose.material.icons.filled.Cloud
37 import androidx.compose.material.icons.filled.Grain
38 import androidx.compose.material.icons.filled.KeyboardArrowDown
39 import androidx.compose.material.icons.filled.Thunderstorm
40 import androidx.compose.material.icons.filled.Thermostat
41 import androidx.compose.material.icons.filled.Umbrella
42 import androidx.compose.material.icons.filled.WaterDrop
43 import androidx.compose.material.icons.filled.WbSunny
44 import androidx.compose.material3.Icon
45 import androidx.compose.material3.MaterialTheme
46 import androidx.compose.material3.Surface
47 import androidx.compose.material3.Text
48 import androidx.compose.runtime.Composable
49 import androidx.compose.runtime.getValue
50 import androidx.compose.runtime.remember
51 import androidx.compose.ui.Alignment
52 import androidx.compose.ui.draw.scale
53 import androidx.compose.ui.Modifier
54 import androidx.compose.ui.draw.alpha
55 import androidx.compose.ui.draw.clip
56 import androidx.compose.ui.draw.drawWithContent
57 import androidx.compose.ui.graphics.Brush
58 import androidx.compose.ui.graphics.Color
59 import androidx.compose.ui.text.font.FontWeight
60 import androidx.compose.ui.text.style.TextAlign
61 import androidx.compose.ui.unit.dp
62 import androidx.compose.ui.unit.sp
63 import com.runway.android.BuildConfig
64 import com.runway.android.core.map.MapPoint
65 import com.runway.android.ui.home.RunGoal
66 import com.runway.android.ui.theme.LocalIsDarkTheme
67
68 private val HeroScrimDark = Color(0xFF0A0B10)
69 private val HeroScrimLight = Color(0xFFF5F5FA)
70
71 @Composable
72 fun RunHeroSection(
73     onStartRun: () -> Unit,
74     onSetGoal: () -> Unit = {},
75     selectedGoal: RunGoal? = null,
76     weatherInfo: WeatherInfo? = null,
77     currentLocation: MapPoint? = null,
78     hasLocationPermission: Boolean = false,
79     modifier: Modifier = Modifier,
80 ) {
81     val isDark = LocalIsDarkTheme.current
82     val heroScrim = if (isDark) HeroScrimDark else HeroScrimLight
83     val onHero = if (isDark) Color.White else Color(0xFF111119)

```

```

84 val pillBg = if (isDark) Color(0x1EFFFFFF) else Color(0x14000000)
85
86 Box(modifier = modifier) {
87
88     // — Real Google Map background —————
89     HomeMapView(
90         currentLocation = currentLocation,
91         hasLocationPermission = hasLocationPermission,
92         modifier = Modifier.fillMaxSize(),
93     )
94
95     // — Bottom scrim — starts at 80% height for a more gradual fade —
96     Box(
97         modifier = Modifier
98             .fillMaxWidth()
99             .fillMaxHeight(0.72f)
100             .align(Alignment.BottomCenter)
101             .background(
102                 Brush.verticalGradient(
103                     0.00f to Color.Transparent,
104                     0.28f to heroScrim.copy(alpha = 0.55f),
105                     1.00f to heroScrim,
106                 )
107             ),
108     )
109
110     // — Header tagline —————
111     Column(
112         modifier = Modifier
113             .align(Alignment.TopStart)
114             .padding(horizontal = 20.dp, vertical = 14.dp),
115     ) {
116         Text(
117             text = BuildConfig.APP_NAME.uppercase(),
118             fontSize = 12.sp,
119             fontWeight = FontWeight.SemiBold,
120             letterSpacing = 2.8.sp,
121             color = MaterialTheme.colorScheme.primary,
122         )
123         Spacer(Modifier.height(4.dp))
124         Text(
125             text = "Run your way.",
126             fontSize = 28.sp,
127             fontWeight = FontWeight.ExtraBold,
128             letterSpacing = (-0.5).sp,
129             color = onHero,
130         )
131     }
132
133     // — GPS status + weather summary —————
134     GpsStatusPill(
135         pillBg = pillBg,
136         onPill = onHero,
137         modifier = Modifier
138             .align(Alignment.TopCenter)
139             .padding(top = 102.dp)
140             .padding(horizontal = 20.dp),
141     )
142     WeatherSummaryPill(
143         weatherInfo = weatherInfo,
144         pillBg = pillBg,
145         onPill = onHero,
146         modifier = Modifier
147             .align(Alignment.TopCenter)
148             .padding(top = 142.dp)
149             .padding(horizontal = 20.dp),
150     )
151
152     // — 스크롤 유도 애니메이션 (좌/우) —————
153     ScrollHintIndicator(
154         modifier = Modifier
155             .align(Alignment.BottomStart)
156             .padding(start = 28.dp, bottom = 88.dp),
157     )
158     ScrollHintIndicator(
159         modifier = Modifier
160             .align(Alignment.BottomEnd)
161             .padding(end = 28.dp, bottom = 88.dp),
162     )
163
164     // — 시작 button + 목표 설정 —————
165     val startButtonInteraction = remember { MutableInteractionSource() }
166     val isStartPressed by startButtonInteraction.collectIsPressedAsState()
167     val startButtonScale by animateFloatAsState(
168         targetValue = if (isStartPressed) 0.93f else 1f,
169         animationSpec = spring(dampingRatio = Spring.DampingRatioMediumBouncy),

```

```

170         label = "startButtonScale",
171     )
172
173     Column(
174         modifier = Modifier
175             .align(Alignment.BottomCenter)
176             .padding(bottom = 80.dp),
177         horizontalAlignment = Alignment.CenterHorizontally,
178     ) {
179         Surface(
180             onClick = onStartRun,
181             modifier = Modifier.size(96.dp).scale(startButtonScale),
182             interactionSource = startButtonInteraction,
183             shape = CircleShape,
184             color = MaterialTheme.colorScheme.primary,
185             shadowElevation = 14.dp,
186         ) {
187             Box(contentAlignment = Alignment.Center) {
188                 Text(
189                     text = "시작",
190                     fontSize = 20.sp,
191                     fontWeight = FontWeight.ExtraBold,
192                     color = Color(0xFF0A0B10),
193                 )
194             }
195         }
196
197         Spacer(Modifier.height(20.dp))
198
199         Surface(
200             onClick = onSetGoal,
201             shape = RoundedCornerShape(50),
202             color = if (selectedGoal != null) {
203                 MaterialTheme.colorScheme.primary.copy(alpha = 0.18f)
204             } else {
205                 pillBg
206             },
207         ) {
208             Text(
209                 text = when (selectedGoal) {
210                     is RunGoal.TimeGoal -> "목표설정: ${selectedGoal.label()}"
211                     is RunGoal.DistanceGoal -> "목표설정: ${selectedGoal.label()}"
212                     is RunGoal.IntervalGoal -> "목표설정: ${selectedGoal.label()}"
213                     null -> "목표 설정"
214                 },
215                 style = MaterialTheme.typography.bodyLarge,
216                 fontWeight = FontWeight.SemiBold,
217                 color = if (selectedGoal != null) {
218                     MaterialTheme.colorScheme.primary
219                 } else {
220                     onHero.copy(alpha = 0.65f)
221                 },
222                 modifier = Modifier.padding(horizontal = 24.dp, vertical = 10.dp),
223             )
224         }
225     }
226 }
227
228
229 @Composable
230 private fun ScrollHintIndicator(modifier: Modifier = Modifier) {
231     val infiniteTransition = rememberInfiniteTransition(label = "scroll_hint")
232     val offsetY by infiniteTransition.animateFloat(
233         initialValue = 0f,
234         targetValue = 8f,
235         animationSpec = infiniteRepeatable(
236             animation = tween(durationMillis = 700),
237             repeatMode = RepeatMode.Reverse,
238         ),
239         label = "bounce",
240     )
241     val alpha by infiniteTransition.animateFloat(
242         initialValue = 0.45f,
243         targetValue = 0.85f,
244         animationSpec = infiniteRepeatable(
245             animation = tween(durationMillis = 700),
246             repeatMode = RepeatMode.Reverse,
247         ),
248         label = "alpha",
249     )
250
251     Column(
252         modifier = modifier.offset(y = offsetY.dp),
253         horizontalAlignment = Alignment.CenterHorizontally,
254     ) {
255         Icon(

```



```

342         tint = MaterialTheme.colorScheme.primary,
343         modifier = Modifier.size(18.dp),
344     )
345     Text(
346         text = weatherInfo.weatherLabel(),
347         style = MaterialTheme.typography.labelLarge,
348         fontWeight = FontWeight.Bold,
349         color = onFill,
350         maxLines = 1,
351         softWrap = false,
352     )
353 }
354 Spacer(Modifier.width(if (compact) 8.dp else 14.dp))
355 Row(horizontalArrangement = Arrangement.spacedBy(if (compact) 6.dp else 9.dp)) {
356     WeatherChip(
357         text = "${weatherInfo.tempCelsius}°C",
358         onFill = onFill,
359         emphasized = true,
360         icon = Icons.Filled.Thermostat,
361     )
362     WeatherChip(
363         text = "습도 ${weatherInfo.humidity}%",
364         onFill = onFill,
365         emphasized = true,
366         icon = Icons.Filled.WaterDrop,
367     )
368 }
369 }
370
371 Row(
372     modifier = Modifier.wrapContentWidth(),
373     verticalAlignment = Alignment.CenterVertically,
374     horizontalArrangement = Arrangement.Center,
375 ) {
376     DustChip(
377         label = if (compact) "미세" else "미세먼지",
378         value = weatherInfo.pm10.takeIf { it > 0 },
379         quality = if (weatherInfo.pm10 > 0) pm10Quality(weatherInfo.pm10) else "-",
380         color = if (weatherInfo.pm10 > 0) pm10Color(weatherInfo.pm10) else onFill.copy(alpha = 0.55f),
381         onFill = onFill,
382         compact = compact,
383     )
384     Spacer(Modifier.width(if (compact) 8.dp else 14.dp))
385     DustChip(
386         label = if (compact) "초미세" else "초미세먼지",
387         value = weatherInfo.pm25.takeIf { it > 0 },
388         quality = if (weatherInfo.pm25 > 0) pm25Quality(weatherInfo.pm25) else "-",
389         color = if (weatherInfo.pm25 > 0) pm25Color(weatherInfo.pm25) else onFill.copy(alpha = 0.55f),
390         onFill = onFill,
391         compact = compact,
392     )
393 }
394 } else {
395     Box(
396         modifier = Modifier
397             .width(240.dp)
398             .height(34.dp)
399             .clip(RoundedCornerShape(5.dp))
400             .background(onFill.copy(alpha = 0.07f))
401             .drawWithContent {
402                 drawContent()
403                 val center = size.width * shimmerOffset
404                 val half = size.width * 0.5f
405                 drawRect(
406                     brush = Brush.horizontalGradient(
407                         listOf(Color.Transparent, onFill.copy(0.22f), Color.Transparent),
408                         startX = center - half,
409                         endX = center + half,
410                     ),
411                 )
412             },
413     )
414 }
415 }
416 }
417 }
418
419 private fun WeatherInfo.weatherLabel(): String = when {
420     conditionId in 200..232 -> "천둥번개"
421     conditionId in 300..321 -> "이슬비"
422     conditionId in 500..531 -> "비"
423     conditionId in 600..622 -> "눈"
424     conditionId in 700..781 -> "안개"
425     conditionId == 800 -> "맑음"
426     conditionId in 801..802 -> "구름 조금"
427     conditionId in 803..804 -> "흐림"

```

```

428     else -> description.ifBlank { condition.ifBlank { "현재 날씨" } }
429 }
430
431 private fun WeatherInfo.weatherIcon() = when {
432     conditionId in 200..232 -> Icons.Filled.Thunderstorm
433     conditionId in 300..321 -> Icons.Filled.Grain
434     conditionId in 500..531 -> Icons.Filled.Umbrella
435     conditionId in 600..622 -> Icons.Filled.AcUnit
436     conditionId in 700..781 -> Icons.Filled.Air
437     conditionId == 800 -> Icons.Filled.WbSunny
438     else -> Icons.Filled.Cloud
439 }
440
441 @Composable
442 private fun WeatherChip(
443     text: String,
444     onFill: Color,
445     emphasized: Boolean = false,
446     icon: androidx.compose.ui.graphics.vector.ImageVector? = null,
447 ) {
448     Row(
449         verticalAlignment = Alignment.CenterVertically,
450         horizontalArrangement = Arrangement.spacedBy(3.dp),
451     ) {
452         icon?.let {
453             Icon(
454                 imageVector = it,
455                 contentDescription = null,
456                 tint = MaterialTheme.colorScheme.primary,
457                 modifier = Modifier.size(16.dp),
458             )
459         }
460         Text(
461             text = text,
462             style = if (emphasized) {
463                 MaterialTheme.typography.labelLarge
464             } else {
465                 MaterialTheme.typography.bodySmall
466             },
467             fontWeight = if (emphasized) FontWeight.Bold else FontWeight.Normal,
468             color = if (emphasized) onFill else onFill.copy(alpha = 0.75f),
469             maxLines = 1,
470             softWrap = false,
471         )
472     }
473 }
474
475 @Composable
476 private fun DustChip(
477     label: String,
478     value: Int?,
479     quality: String,
480     color: Color,
481     onFill: Color,
482     compact: Boolean,
483 ) {
484     Row(
485         verticalAlignment = Alignment.CenterVertically,
486         horizontalArrangement = Arrangement.spacedBy(3.dp),
487     ) {
488         Text(
489             text = label,
490             fontSize = if (compact) 11.sp else 12.sp,
491             color = onFill.copy(alpha = 0.55f),
492             maxLines = 1,
493             softWrap = false,
494             textAlign = TextAlign.Center,
495         )
496         Box(
497             modifier = Modifier
498                 .size(5.dp)
499                 .background(color, CircleShape),
500         )
501         Text(
502             text = value?.let { "$itμg/m³ $quality" } ?: quality,
503             fontSize = if (compact) 10.sp else 11.sp,
504             color = color,
505             maxLines = 1,
506             softWrap = false,
507             textAlign = TextAlign.Center,
508         )
509     }
510 }

```

```

1  package com.runway.android.ui.components
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.background
5  import androidx.compose.foundation.layout.Arrangement
6  import androidx.compose.foundation.layout.Box
7  import androidx.compose.foundation.layout.Column
8  import androidx.compose.foundation.layout.Row
9  import androidx.compose.foundation.layout.Spacer
10 import androidx.compose.foundation.layout.fillMaxWidth
11 import androidx.compose.foundation.layout.padding
12 import androidx.compose.foundation.layout.size
13 import androidx.compose.foundation.layout.width
14 import androidx.compose.material.icons.Icons
15 import androidx.compose.material.icons.automirrored.filled.DirectionsRun
16 import androidx.compose.material.icons.automirrored.filled.KeyboardArrowRight
17 import androidx.compose.material3.Icon
18 import androidx.compose.material3.MaterialTheme
19 import androidx.compose.material3.Surface
20 import androidx.compose.material3.Text
21 import androidx.compose.runtime.Composable
22 import androidx.compose.ui.Alignment
23 import androidx.compose.ui.Modifier
24 import androidx.compose.ui.unit.dp
25
26 data class RunHistoryItem(
27     val runId: String,
28     val dateLabel: String,
29     val distanceFormatted: String,
30     val duration: String,
31     val pace: String,
32     val status: String,
33     val calories: Int?,
34     val localDate: java.time.LocalDate? = null,
35     val rawDistanceMeters: Double? = null,
36     val rawDurationSeconds: Int? = null,
37 )
38
39 @Composable
40 fun RunHistoryCard(
41     item: RunHistoryItem,
42     onClick: () -> Unit,
43     modifier: Modifier = Modifier,
44 ) {
45     Surface(
46         modifier = modifier.fillMaxWidth(),
47         shape = MaterialTheme.shapes.extraLarge,
48         color = MaterialTheme.colorScheme.surface,
49         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
50         onClick = onClick,
51     ) {
52         Row(
53             modifier = Modifier.padding(12.dp),
54             verticalAlignment = Alignment.CenterVertically,
55         ) {
56             Box(
57                 modifier = Modifier
58                     .size(44.dp)
59                     .background(
60                         color = MaterialTheme.colorScheme.surfaceVariant,
61                         shape = MaterialTheme.shapes.medium,
62                     ),
63                 contentAlignment = Alignment.Center,
64             ) {
65                 Icon(
66                     imageVector = Icons.AutoMirrored.Filled.DirectionsRun,
67                     contentDescription = null,
68                     tint = MaterialTheme.colorScheme.primary,
69                     modifier = Modifier.size(20.dp),
70                 )
71             }
72
73             Spacer(modifier = Modifier.width(12.dp))
74
75             Column(modifier = Modifier.weight(1f)) {
76                 Text(
77                     text = "${item.distanceFormatted} · ${item.duration}",
78                     style = MaterialTheme.typography.titleMedium,
79                     color = MaterialTheme.colorScheme.onSurface,
80                 )
81                 Row(
82                     horizontalArrangement = Arrangement.spacedBy(4.dp),
83                     verticalAlignment = Alignment.CenterVertically,

```

```

84     ) {
85         Text(
86             text = item.dateLabel,
87             style = MaterialTheme.typography.bodyMedium,
88             color = MaterialTheme.colorScheme.onSurfaceVariant,
89         )
90         Text(
91             text = ".",
92             style = MaterialTheme.typography.bodyMedium,
93             color = MaterialTheme.colorScheme.onSurfaceVariant,
94         )
95         Text(
96             text = item.pace,
97             style = MaterialTheme.typography.bodyMedium,
98             color = MaterialTheme.colorScheme.onSurfaceVariant,
99         )
100         item.calories?.let { cal ->
101             Text(
102                 text = ".",
103                 style = MaterialTheme.typography.bodyMedium,
104                 color = MaterialTheme.colorScheme.onSurfaceVariant,
105             )
106             Text(
107                 text = "$cal kcal",
108                 style = MaterialTheme.typography.bodyMedium,
109                 color = MaterialTheme.colorScheme.onSurfaceVariant,
110             )
111         }
112     }
113 }
114
115 Icon(
116     imageVector = Icons.AutoMirrored.Filled.KeyboardArrowRight,
117     contentDescription = null,
118     tint = MaterialTheme.colorScheme.onSurfaceVariant,
119     modifier = Modifier.size(20.dp),
120 )
121 }
122 }
123 }

```



```
1 package com.runway.android.ui.components
2
3 import androidx.compose.foundation.layout.Column
4 import androidx.compose.foundation.layout.Spacer
5 import androidx.compose.foundation.layout.height
6 import androidx.compose.foundation.layout.padding
7 import androidx.compose.material3.MaterialTheme
8 import androidx.compose.material3.Text
9 import androidx.compose.runtime.Composable
10 import androidx.compose.ui.Alignment
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.unit.dp
13
14 @Composable
15 fun RunMetricCard(
16     label: String,
17     value: String,
18     unit: String,
19     modifier: Modifier = Modifier,
20 ) {
21     Column(
22         modifier = modifier.padding(vertical = 16.dp, horizontal = 8.dp),
23         horizontalAlignment = Alignment.CenterHorizontally,
24     ) {
25         Text(
26             text = label,
27             style = MaterialTheme.typography.labelMedium,
28             color = MaterialTheme.colorScheme.onSurfaceVariant,
29         )
30         Spacer(modifier = Modifier.height(4.dp))
31         Text(
32             text = value,
33             style = MaterialTheme.typography.titleLarge,
34             color = MaterialTheme.colorScheme.onSurface,
35         )
36         Text(
37             text = unit,
38             style = MaterialTheme.typography.labelSmall,
39             color = MaterialTheme.colorScheme.onSurfaceVariant,
40         )
41     }
42 }
```

```

1  package com.runway.android.ui.components
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.Canvas
5  import androidx.compose.foundation.layout.Arrangement
6  import androidx.compose.foundation.layout.Column
7  import androidx.compose.foundation.layout.Row
8  import androidx.compose.foundation.layout.Spacer
9  import androidx.compose.foundation.layout.fillMaxWidth
10 import androidx.compose.foundation.layout.height
11 import androidx.compose.foundation.layout.padding
12 import androidx.compose.material3.MaterialTheme
13 import androidx.compose.material3.Surface
14 import androidx.compose.material3.Text
15 import androidx.compose.runtime.Composable
16 import androidx.compose.runtime.remember
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.geometry.Offset
19 import androidx.compose.ui.graphics.Path
20 import androidx.compose.ui.graphics.drawscope.Stroke
21 import androidx.compose.ui.text.TextMeasurer
22 import androidx.compose.ui.text.drawText
23 import androidx.compose.ui.text.rememberTextMeasurer
24 import androidx.compose.ui.unit.dp
25 import com.runway.android.core.running.RunChartPoint
26
27 @Composable
28 fun RunPaceChart(
29     points: List<RunChartPoint>,
30     modifier: Modifier = Modifier,
31     kilometerElapsedSeconds: List<Int> = emptyList(),
32     averagePaceSecondsPerKm: Int? = null,
33     bestPaceSecondsPerKm: Int? = null,
34 ) {
35     if (points.isEmpty()) return
36
37     val lineColor = MaterialTheme.colorScheme.primary
38     val gridColor = MaterialTheme.colorScheme.outlineVariant
39     val labelStyle = MaterialTheme.typography.labelSmall
40     val labelColor = MaterialTheme.colorScheme.onSurfaceVariant
41
42     val textMeasurer = rememberTextMeasurer()
43
44     val minPace = remember(points) { points.minOf { it.paceSecondsPerKm } }
45     val maxPace = remember(points) { points.maxOf { it.paceSecondsPerKm } }
46     val paceRange = remember(minPace, maxPace) { (maxPace - minPace).coerceAtLeast(60) }
47
48     Surface(
49         modifier = modifier.fillMaxWidth(),
50         shape = MaterialTheme.shapes.extraLarge,
51         color = MaterialTheme.colorScheme.surface,
52         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
53     ) {
54         Column(modifier = Modifier.padding(16.dp)) {
55             Text(
56                 text = "페이스 변화",
57                 style = MaterialTheme.typography.labelMedium,
58                 color = MaterialTheme.colorScheme.onSurfaceVariant,
59             )
60             Spacer(modifier = Modifier.height(8.dp))
61             Canvas(
62                 modifier = Modifier
63                     .fillMaxWidth()
64                     .height(148.dp),
65             ) {
66                 val w = size.width
67                 val h = size.height
68                 val paddingLeft = 48.dp.toPx()
69                 val paddingBottom = 34.dp.toPx()
70                 val chartW = w - paddingLeft
71                 val chartH = h - paddingBottom
72                 val maxElapsed = points.last().elapsedSeconds.toFloat().coerceAtLeast(1f)
73
74                 // Grid lines (3 horizontal)
75                 val gridSteps = 3
76                 for (step in 0..gridSteps) {
77                     val y = chartH * step / gridSteps
78                     drawLine(
79                         color = gridColor,
80                         start = Offset(paddingLeft, y),
81                         end = Offset(w, y),
82                         strokeWidth = 0.5.dp.toPx(),
83                     )

```

```

84         // Y-axis label: pace increases downward (slower = higher y)
85         val paceAtStep = maxPace - (paceRange * step / gridSteps)
86         val label = formatPaceLabel(paceAtStep)
87         val measured = textMeasurer.measure(label, style = labelStyle)
88         drawText(
89             textMeasurer = textMeasurer,
90             text = label,
91             style = labelStyle.copy(color = labelColor),
92             topLeft = Offset(0f, y - measured.size.height / 2f),
93         )
94     }
95
96     kilometerElapsedSeconds.forEach { elapsedSeconds ->
97         val x = paddingLeft + (elapsedSeconds / maxElapsed) * chartW
98         drawLine(
99             color = gridColor,
100            start = Offset(x, 0f),
101            end = Offset(x, chartH),
102            strokeWidth = 0.5.dp.toPx(),
103        )
104        val label = formatElapsedTimeLabel(elapsedSeconds)
105        val measured = textMeasurer.measure(label, style = labelStyle)
106        drawText(
107            textMeasurer = textMeasurer,
108            text = label,
109            style = labelStyle.copy(color = labelColor),
110            topLeft = Offset(
111                x = (x - measured.size.width / 2f).coerceIn(paddingLeft, w - measured.size.width),
112                y = chartH + 8.dp.toPx(),
113            ),
114        )
115    }
116
117    // Pace line
118    val path = Path()
119    points.forEachIndexed { i, pt ->
120        val x = paddingLeft + (pt.elapsedSeconds / maxElapsed) * chartW
121        // higher pace (slower) → higher y
122        val normalized = (pt.paceSecondsPerKm - minPace).toFloat() / paceRange
123        val y = chartH * normalized
124        if (i == 0) path.moveTo(x, y) else path.lineTo(x, y)
125    }
126    drawPath(
127        path = path,
128        color = lineColor,
129        style = Stroke(width = 2.dp.toPx()),
130    )
131 }
132
133 if (averagePaceSecondsPerKm != null || bestPaceSecondsPerKm != null) {
134     Spacer(modifier = Modifier.height(12.dp))
135     Row(
136         modifier = Modifier.fillMaxWidth(),
137         horizontalArrangement = Arrangement.spacedBy(12.dp),
138     ) {
139         PaceSummary(
140             label = "평균 페이스",
141             value = formatPaceLabel(averagePaceSecondsPerKm),
142             modifier = Modifier.weight(1f),
143         )
144         PaceSummary(
145             label = "최고 페이스",
146             value = formatPaceLabel(bestPaceSecondsPerKm),
147             modifier = Modifier.weight(1f),
148         )
149     }
150 }
151 }
152 }
153
154 @Composable
155 private fun PaceSummary(
156     label: String,
157     value: String,
158     modifier: Modifier = Modifier,
159 ) {
160     Column(modifier = modifier) {
161         Text(
162             text = label,
163             style = MaterialTheme.typography.labelSmall,
164             color = MaterialTheme.colorScheme.onSurfaceVariant,
165         )
166         Spacer(modifier = Modifier.height(2.dp))
167         Text(
168             text = value,
169             style = MaterialTheme.typography.titleMedium,

```

```
170         color = MaterialTheme.colorScheme.onSurface,
171     )
172 }
173 }
174
175 private fun formatPaceLabel(secsPerKm: Int?): String {
176     if (secsPerKm == null || secsPerKm <= 0) return "---\\"
177     val m = secsPerKm / 60
178     val s = secsPerKm % 60
179     return "%d'%02d\\".format(m, s)
180 }
181
182 private fun formatElapsedTimeLabel(seconds: Int): String {
183     val h = seconds / 3600
184     val m = (seconds % 3600) / 60
185     val s = seconds % 60
186     return if (h > 0) "%d:%02d:%02d".format(h, m, s) else "%d:%02d".format(m, s)
187 }
```

```

1  package com.runway.android.ui.components
2
3  import androidx.compose.foundation.Canvas
4  import androidx.compose.foundation.background
5  import androidx.compose.foundation.layout.Box
6  import androidx.compose.foundation.layout.fillMaxSize
7  import androidx.compose.material3.MaterialTheme
8  import androidx.compose.material3.Text
9  import androidx.compose.runtime.Composable
10 import androidx.compose.ui.Alignment
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.geometry.Offset
13 import androidx.compose.ui.graphics.Color
14 import androidx.compose.ui.graphics.Path
15 import androidx.compose.ui.graphics.StrokeCap
16 import androidx.compose.ui.graphics.StrokeJoin
17 import androidx.compose.ui.graphics.drawscope.Stroke
18 import androidx.compose.ui.unit.dp
19 import com.runway.android.data.running.model.RunPointResponse
20
21 /**
22  * 실제 GPS 포인트를 Canvas에 렌더링하는 경로 미리보기.
23  * 포인트가 2개 미만이면 플레이스홀더를 표시한다.
24  */
25 @Composable
26 fun RunRoutePreview(
27     points: List<RunPointResponse>,
28     modifier: Modifier = Modifier,
29 ) {
30     val bgColor = MaterialTheme.colorScheme.surfaceVariant
31     val primaryColor = MaterialTheme.colorScheme.primary
32     val errorColor = MaterialTheme.colorScheme.error
33
34     if (points.size < 2) {
35         Box(
36             modifier = modifier.background(bgColor),
37             contentAlignment = Alignment.Center,
38         ) {
39             Text(
40                 text = "경로 데이터 없음",
41                 style = MaterialTheme.typography.bodyMedium,
42                 color = MaterialTheme.colorScheme.onSurfaceVariant,
43             )
44         }
45         return
46     }
47
48     val lats = points.map { it.latitude }
49     val lngs = points.map { it.longitude }
50     val minLat = lats.min()
51     val maxLat = lats.max()
52     val minLng = lngs.min()
53     val maxLng = lngs.max()
54
55     Box(modifier = modifier.background(bgColor)) {
56         Canvas(modifier = Modifier.fillMaxSize()) {
57             val pad = 28.dp.toPx()
58             val drawW = size.width - pad * 2
59             val drawH = size.height - pad * 2
60
61             val latRange = (maxLat - minLat).coerceAtLeast(0.0001)
62             val lngRange = (maxLng - minLng).coerceAtLeast(0.0001)
63
64             fun norm(lat: Double, lng: Double): Offset {
65                 val x = ((lng - minLng) / lngRange * drawW + pad).toFloat()
66                 val y = (((maxLat - lat) / latRange * drawH) + pad).toFloat()
67                 return Offset(x, y)
68             }
69
70             // 경로 선
71             val path = Path().apply {
72                 val first = norm(points[0].latitude, points[0].longitude)
73                 moveTo(first.x, first.y)
74                 for (i in 1 until points.size) {
75                     val pt = norm(points[i].latitude, points[i].longitude)
76                     lineTo(pt.x, pt.y)
77                 }
78             }
79             drawPath(
80                 path = path,
81                 color = primaryColor,
82                 style = Stroke(
83                     width = 3.dp.toPx(),

```

```
84         cap = StrokeCap.Round,
85         join = StrokeJoin.Round,
86     ),
87 )
88
89 // 시작점 (primary 색상 점)
90 val startPt = norm(points.first().latitude, points.first().longitude)
91 drawCircle(color = primaryColor, radius = 6.dp.toPx(), center = startPt)
92 drawCircle(color = Color.White, radius = 3.dp.toPx(), center = startPt)
93
94 // 종료점 (error 색상 점)
95 val endPt = norm(points.last().latitude, points.last().longitude)
96 drawCircle(color = errorColor, radius = 6.dp.toPx(), center = endPt)
97 drawCircle(color = Color.White, radius = 3.dp.toPx(), center = endPt)
98     }
99 }
100 }
```

```

1  package com.runway.android.ui.components
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.layout.Arrangement
5  import androidx.compose.foundation.layout.Column
6  import androidx.compose.foundation.layout.Row
7  import androidx.compose.foundation.layout.Spacer
8  import androidx.compose.foundation.layout.fillMaxWidth
9  import androidx.compose.foundation.layout.height
10 import androidx.compose.foundation.layout.padding
11 import androidx.compose.foundation.layout.width
12 import androidx.compose.material3.HorizontalDivider
13 import androidx.compose.material3.MaterialTheme
14 import androidx.compose.material3.Surface
15 import androidx.compose.material3.Text
16 import androidx.compose.runtime.Composable
17 import androidx.compose.ui.Alignment
18 import androidx.compose.ui.Modifier
19 import androidx.compose.ui.unit.dp
20 import com.runway.android.core.running.RunSplit
21 import com.runway.android.core.util.formatDuration
22 import com.runway.android.core.util.formatPace
23
24 @Composable
25 fun RunSplitsCard(
26     splits: List<RunSplit>,
27     modifier: Modifier = Modifier,
28 ) {
29     Surface(
30         modifier = modifier.fillMaxWidth(),
31         shape = MaterialTheme.shapes.extraLarge,
32         color = MaterialTheme.colorScheme.surface,
33         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
34     ) {
35         Column(modifier = Modifier.padding(20.dp)) {
36             Text(
37                 text = "SPLITS",
38                 style = MaterialTheme.typography.labelMedium,
39                 color = MaterialTheme.colorScheme.onSurfaceVariant,
40             )
41             Spacer(modifier = Modifier.height(12.dp))
42
43             if (splits.isEmpty()) {
44                 Text(
45                     text = "구간 기록을 계산할 수 있는 GPS 포인트가 부족합니다.",
46                     style = MaterialTheme.typography.bodyMedium,
47                     color = MaterialTheme.colorScheme.onSurfaceVariant,
48                 )
49             } else {
50                 splits.forEachIndexed { index, split ->
51                     SplitRow(split = split)
52                     if (index < splits.size - 1) {
53                         HorizontalDivider(
54                             modifier = Modifier.padding(vertical = 8.dp),
55                             color = MaterialTheme.colorScheme.outlineVariant,
56                         )
57                     }
58                 }
59             }
60         }
61     }
62 }
63
64 @Composable
65 private fun SplitRow(split: RunSplit) {
66     val isPartial = split.distanceKm < 0.95
67     Row(
68         modifier = Modifier.fillMaxWidth(),
69         horizontalArrangement = Arrangement.SpaceBetween,
70         verticalAlignment = Alignment.CenterVertically,
71     ) {
72         // Label: "km 1" for full, "0.42 km" for partial
73         Text(
74             text = if (isPartial) "%.2f km".format(split.distanceKm) else "km ${split.splitNumber}",
75             style = MaterialTheme.typography.bodyMedium,
76             color = MaterialTheme.colorScheme.onSurfaceVariant,
77             modifier = Modifier.width(64.dp),
78         )
79
80         Spacer(modifier = Modifier.weight(1f))
81
82         // Pace
83         Text(

```

```

84         text = formatPace(split.paceSecondsPerKm),
85         style = MaterialTheme.typography.titleMedium,
86         color = MaterialTheme.colorScheme.onSurface,
87     )
88
89     Spacer(modifier = Modifier.width(16.dp))
90
91     // Duration
92     Text(
93         text = formatDuration(split.durationSeconds),
94         style = MaterialTheme.typography.bodyMedium,
95         color = MaterialTheme.colorScheme.onSurfaceVariant,
96     )
97 }
98 }

```

android/app/src/main/java/com/runway/android/ui/components/RunningControlButton.kt

Kotlin | 47줄

```

1  package com.runway.android.ui.components
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.layout.Box
5  import androidx.compose.foundation.layout.fillMaxSize
6  import androidx.compose.foundation.layout.size
7  import androidx.compose.foundation.shape.CircleShape
8  import androidx.compose.material3.Icon
9  import androidx.compose.material3.Surface
10 import androidx.compose.runtime.Composable
11 import androidx.compose.ui.Alignment
12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.graphics.Color
14 import androidx.compose.ui.graphics.vector.ImageVector
15 import androidx.compose.ui.unit.Dp
16 import androidx.compose.ui.unit.dp
17
18 @Composable
19 fun RunningControlButton(
20     icon: ImageVector,
21     onClick: () -> Unit,
22     size: Dp,
23     containerColor: Color,
24     contentColor: Color,
25     modifier: Modifier = Modifier,
26     border: BorderStroke? = null,
27 ) {
28     Surface(
29         onClick = onClick,
30         modifier = modifier.size(size),
31         shape = CircleShape,
32         color = containerColor,
33         border = border,
34     ) {
35         Box(
36             contentAlignment = Alignment.Center,
37             modifier = Modifier.fillMaxSize(),
38         ) {
39             Icon(
40                 imageVector = icon,
41                 contentDescription = null,
42                 tint = contentColor,
43                 modifier = Modifier.size(size * 0.40f),
44             )
45         }
46     }
47 }

```


android/app/src/main/java/com/runway/android/ui/components/RunwayBottomNav.kt

Kotlin | 52줄

```
1 package com.runway.android.ui.components
2
3 import androidx.compose.foundation.layout.size
4 import androidx.compose.material3.Icon
5 import androidx.compose.material3.MaterialTheme
6 import androidx.compose.material3.NavigationBar
7 import androidx.compose.material3.NavigationBarItem
8 import androidx.compose.material3.NavigationBarItemDefaults
9 import androidx.compose.material3.Text
10 import androidx.compose.runtime.Composable
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.unit.dp
13 import com.runway.android.ui.navigation.MainTab
14
15 @Composable
16 fun RunwayBottomNav(
17     currentTab: MainTab,
18     onTabSelected: (MainTab) -> Unit,
19 ) {
20     NavigationBar(
21         containerColor = MaterialTheme.colorScheme.surface,
22         tonalElevation = 0.dp,
23     ) {
24         MainTab.entries.forEach { tab ->
25             val selected = tab == currentTab
26             NavigationBarItem(
27                 selected = selected,
28                 onClick = { onTabSelected(tab) },
29                 icon = {
30                     Icon(
31                         imageVector = tab.icon,
32                         contentDescription = tab.label,
33                         modifier = Modifier.size(22.dp),
34                     )
35                 },
36                 label = {
37                     Text(
38                         text = tab.label,
39                         style = MaterialTheme.typography.labelMedium,
40                     )
41                 },
42                 colors = NavigationBarItemDefaults.colors(
43                     selectedIconColor = MaterialTheme.colorScheme.primary,
44                     selectedTextColor = MaterialTheme.colorScheme.primary,
45                     unselectedIconColor = MaterialTheme.colorScheme.onSurfaceVariant,
46                     unselectedTextColor = MaterialTheme.colorScheme.onSurfaceVariant,
47                     indicatorColor = MaterialTheme.colorScheme.primary.copy(alpha = 0.12f),
48                 ),
49             )
50         }
51     }
52 }
```

android/app/src/main/java/com/runway/android/ui/components/RunwayErrorText.kt

Kotlin | 22줄

```
1 package com.runway.android.ui.components
2
3 import androidx.compose.foundation.layout.fillMaxWidth
4 import androidx.compose.material3.MaterialTheme
5 import androidx.compose.material3.Text
6 import androidx.compose.runtime.Composable
7 import androidx.compose.ui.Modifier
8 import androidx.compose.ui.text.style.TextAlign
9
10 @Composable
11 fun RunwayErrorText(
12     message: String,
13     modifier: Modifier = Modifier,
14 ) {
15     Text(
16         text = message,
17         style = MaterialTheme.typography.bodySmall,
18         color = MaterialTheme.colorScheme.error,
19         modifier = modifier.fillMaxWidth(),
20         textAlign = TextAlign.Center,
21     )
22 }
```

```
1 package com.runway.android.ui.components
2
3 import androidx.compose.foundation.layout.fillMaxWidth
4 import androidx.compose.foundation.layout.height
5 import androidx.compose.foundation.layout.size
6 import androidx.compose.material3.Button
7 import androidx.compose.material3.ButtonDefaults
8 import androidx.compose.material3.CircularProgressIndicator
9 import androidx.compose.material3.MaterialTheme
10 import androidx.compose.material3.Text
11 import androidx.compose.runtime.Composable
12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.unit.dp
14
15 @Composable
16 fun RunwayLoadingButton(
17     text: String,
18     onClick: () -> Unit,
19     isLoading: Boolean,
20     modifier: Modifier = Modifier,
21 ) {
22     Button(
23         onClick = onClick,
24         enabled = !isLoading,
25         modifier = modifier
26             .fillMaxWidth()
27             .height(56.dp),
28         shape = MaterialTheme.shapes.large,
29         colors = ButtonDefaults.buttonColors(
30             containerColor = MaterialTheme.colorScheme.primary,
31             contentColor = MaterialTheme.colorScheme.onPrimary,
32             disabledContainerColor = MaterialTheme.colorScheme.primary.copy(alpha = 0.7f),
33             disabledContentColor = MaterialTheme.colorScheme.onPrimary,
34         ),
35     ) {
36         if (isLoading) {
37             CircularProgressIndicator(
38                 modifier = Modifier.size(20.dp),
39                 color = MaterialTheme.colorScheme.onPrimary,
40                 strokeWidth = 2.dp,
41             )
42         } else {
43             Text(
44                 text = text,
45                 style = MaterialTheme.typography.bodyLarge,
46             )
47         }
48     }
49 }
```

```
1 package com.runway.android.ui.components
2
3 import androidx.compose.foundation.layout.fillMaxWidth
4 import androidx.compose.foundation.layout.height
5 import androidx.compose.material3.Button
6 import androidx.compose.material3.ButtonDefaults
7 import androidx.compose.material3.MaterialTheme
8 import androidx.compose.material3.Text
9 import androidx.compose.runtime.Composable
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.unit.dp
12
13 @Composable
14 fun RunwayPrimaryButton(
15     text: String,
16     onClick: () -> Unit,
17     modifier: Modifier = Modifier,
18     enabled: Boolean = true,
19 ) {
20     Button(
21         onClick = onClick,
22         enabled = enabled,
23         modifier = modifier
24         .fillMaxWidth()
25         .height(56.dp),
26         shape = MaterialTheme.shapes.large,
27         colors = ButtonDefaults.buttonColors(
28             containerColor = MaterialTheme.colorScheme.primary,
29             contentColor = MaterialTheme.colorScheme.onPrimary,
30             disabledContainerColor = MaterialTheme.colorScheme.primary.copy(alpha = 0.4f),
31             disabledContentColor = MaterialTheme.colorScheme.onPrimary.copy(alpha = 0.6f),
32         ),
33     ) {
34         Text(
35             text = text,
36             style = MaterialTheme.typography.bodyLarge,
37         )
38     }
39 }
```

```

1  package com.runway.android.ui.components
2
3  import androidx.compose.foundation.layout.Column
4  import androidx.compose.foundation.layout.Spacer
5  import androidx.compose.foundation.layout.fillMaxWidth
6  import androidx.compose.foundation.layout.height
7  import androidx.compose.foundation.text.KeyboardOptions
8  import androidx.compose.material3.MaterialTheme
9  import androidx.compose.material3.OutlinedTextField
10 import androidx.compose.material3.OutlinedTextFieldDefaults
11 import androidx.compose.material3.Text
12 import androidx.compose.runtime.Composable
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.text.input.VisualTransformation
15 import androidx.compose.ui.unit.dp
16
17 @Composable
18 fun RunwayTextField(
19     value: String,
20     onChange: (String) -> Unit,
21     label: String,
22     modifier: Modifier = Modifier,
23     placeholder: String = "",
24     visualTransformation: VisualTransformation = VisualTransformation.None,
25     keyboardOptions: KeyboardOptions = KeyboardOptions.Default,
26     isError: Boolean = false,
27 ) {
28     Column(modifier = modifier) {
29         Text(
30             text = label,
31             style = MaterialTheme.typography.labelSmall,
32             color = MaterialTheme.colorScheme.onSurfaceVariant,
33         )
34         Spacer(modifier = Modifier.height(6.dp))
35         OutlinedTextField(
36             value = value,
37             onChange = onChange,
38             modifier = Modifier.fillMaxWidth(),
39             placeholder = {
40                 Text(
41                     text = placeholder,
42                     style = MaterialTheme.typography.bodyMedium,
43                     color = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.5f),
44                 )
45             },
46             visualTransformation = visualTransformation,
47             keyboardOptions = keyboardOptions,
48             isError = isError,
49             singleLine = true,
50             shape = MaterialTheme.shapes.medium,
51             textStyle = MaterialTheme.typography.bodyMedium.copy(
52                 color = MaterialTheme.colorScheme.onSurface,
53             ),
54             colors = OutlinedTextFieldDefaults.colors(
55                 focusedContainerColor = MaterialTheme.colorScheme.surface,
56                 unfocusedContainerColor = MaterialTheme.colorScheme.surface,
57                 errorContainerColor = MaterialTheme.colorScheme.surface,
58                 focusedBorderColor = MaterialTheme.colorScheme.primary,
59                 unfocusedBorderColor = MaterialTheme.colorScheme.outline,
60                 errorBorderColor = MaterialTheme.colorScheme.error,
61                 cursorColor = MaterialTheme.colorScheme.primary,
62             ),
63         )
64     }
65 }

```

```
1 package com.runway.android.ui.components
2
3 import androidx.compose.foundation.clickable
4 import androidx.compose.foundation.layout.Arrangement
5 import androidx.compose.foundation.layout.Row
6 import androidx.compose.foundation.layout.fillMaxWidth
7 import androidx.compose.foundation.layout.padding
8 import androidx.compose.material3.MaterialTheme
9 import androidx.compose.material3.Text
10 import androidx.compose.runtime.Composable
11 import androidx.compose.ui.Alignment
12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.unit.dp
14
15 @Composable
16 fun SectionHeader(
17     title: String,
18     modifier: Modifier = Modifier,
19     cta: String? = null,
20     onCtaClick: () -> Unit = {},
21 ) {
22     Row(
23         modifier = modifier
24         .fillMaxWidth()
25         .padding(horizontal = 20.dp)
26         .padding(top = 20.dp, bottom = 8.dp),
27         horizontalArrangement = Arrangement.SpaceBetween,
28         verticalAlignment = Alignment.CenterVertically,
29     ) {
30         Text(
31             text = title,
32             style = MaterialTheme.typography.titleMedium,
33             color = MaterialTheme.colorScheme.onBackground,
34         )
35         if (cta != null) {
36             Text(
37                 text = cta,
38                 style = MaterialTheme.typography.labelLarge,
39                 color = MaterialTheme.colorScheme.primary,
40                 modifier = Modifier.clickable(onClick = onCtaClick),
41             )
42         }
43     }
44 }
```

```

1  package com.runway.android.ui.components
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.background
5  import androidx.compose.foundation.layout.Box
6  import androidx.compose.foundation.layout.Column
7  import androidx.compose.foundation.layout.Row
8  import androidx.compose.foundation.layout.Spacer
9  import androidx.compose.foundation.layout.fillMaxWidth
10 import androidx.compose.foundation.layout.padding
11 import androidx.compose.foundation.layout.size
12 import androidx.compose.foundation.layout.width
13 import androidx.compose.foundation.shape.CircleShape
14 import androidx.compose.material.icons.Icons
15 import androidx.compose.material.icons.automirrored.filled.KeyboardArrowRight
16 import androidx.compose.material.icons.filled.PlayArrow
17 import androidx.compose.material3.Icon
18 import androidx.compose.material3.MaterialTheme
19 import androidx.compose.material3.Surface
20 import androidx.compose.material3.Text
21 import androidx.compose.runtime.Composable
22 import androidx.compose.ui.Alignment
23 import androidx.compose.ui.Modifier
24 import androidx.compose.ui.unit.dp
25
26 @Composable
27 fun StartRunCard(
28     onClick: () -> Unit,
29     modifier: Modifier = Modifier,
30 ) {
31     Surface(
32         onClick = onClick,
33         modifier = modifier.fillMaxWidth(),
34         shape = MaterialTheme.shapes.extraLarge,
35         color = MaterialTheme.colorScheme.surface,
36         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
37     ) {
38         Row(
39             modifier = Modifier.padding(horizontal = 20.dp, vertical = 14.dp),
40             verticalAlignment = Alignment.CenterVertically,
41         ) {
42             Box(
43                 modifier = Modifier
44                     .size(44.dp)
45                     .background(MaterialTheme.colorScheme.primary, CircleShape),
46                 contentAlignment = Alignment.Center,
47             ) {
48                 Icon(
49                     imageVector = Icons.Filled.PlayArrow,
50                     contentDescription = null,
51                     tint = MaterialTheme.colorScheme.onPrimary,
52                     modifier = Modifier.size(22.dp),
53                 )
54             }
55
56             Spacer(modifier = Modifier.width(14.dp))
57
58             Column(modifier = Modifier.weight(1f)) {
59                 Text(
60                     text = "러닝 시작",
61                     style = MaterialTheme.typography.titleMedium,
62                     color = MaterialTheme.colorScheme.onSurface,
63                 )
64                 Text(
65                     text = "자유 러닝 · GPS 준비",
66                     style = MaterialTheme.typography.bodyMedium,
67                     color = MaterialTheme.colorScheme.onSurfaceVariant,
68                 )
69             }
70
71             Icon(
72                 imageVector = Icons.AutoMirrored.Filled.KeyboardArrowRight,
73                 contentDescription = null,
74                 tint = MaterialTheme.colorScheme.onSurfaceVariant,
75                 modifier = Modifier.size(20.dp),
76             )
77         }
78     }
79 }

```

```
1 package com.runway.android.ui.components
2
3 import androidx.compose.foundation.BorderStroke
4 import androidx.compose.foundation.layout.Arrangement
5 import androidx.compose.foundation.layout.Row
6 import androidx.compose.foundation.layout.fillMaxWidth
7 import androidx.compose.foundation.layout.padding
8 import androidx.compose.material3.MaterialTheme
9 import androidx.compose.material3.Surface
10 import androidx.compose.material3.Text
11 import androidx.compose.runtime.Composable
12 import androidx.compose.ui.Alignment
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.unit.dp
15
16 @Composable
17 fun StatsSummaryCard(
18     label: String,
19     value: String,
20     modifier: Modifier = Modifier.fillMaxWidth(),
21 ) {
22     Surface(
23         modifier = modifier,
24         shape = MaterialTheme.shapes.large,
25         color = MaterialTheme.colorScheme.surface,
26         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
27     ) {
28         Row(
29             modifier = Modifier
30                 .fillMaxWidth()
31                 .padding(horizontal = 20.dp, vertical = 16.dp),
32             horizontalArrangement = Arrangement.SpaceBetween,
33             verticalAlignment = Alignment.CenterVertically,
34         ) {
35             Text(
36                 text = label,
37                 style = MaterialTheme.typography.bodyMedium,
38                 color = MaterialTheme.colorScheme.onSurfaceVariant,
39             )
40             Text(
41                 text = value,
42                 style = MaterialTheme.typography.titleMedium,
43                 color = MaterialTheme.colorScheme.onSurface,
44             )
45         }
46     }
47 }
```

```

1  package com.runway.android.ui.components
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.layout.Arrangement
5  import androidx.compose.foundation.layout.Box
6  import androidx.compose.foundation.layout.Column
7  import androidx.compose.foundation.layout.Row
8  import androidx.compose.foundation.layout.Spacer
9  import androidx.compose.foundation.layout.fillMaxWidth
10 import androidx.compose.foundation.layout.offset
11 import androidx.compose.foundation.layout.padding
12 import androidx.compose.foundation.layout.size
13 import androidx.compose.foundation.layout.width
14 import androidx.compose.foundation.shape.CircleShape
15 import androidx.compose.material3.MaterialTheme
16 import androidx.compose.material3.Text
17 import androidx.compose.runtime.Composable
18 import androidx.compose.ui.Alignment
19 import androidx.compose.ui.Modifier
20 import androidx.compose.ui.draw.clip
21 import androidx.compose.ui.graphics.Color
22 import androidx.compose.ui.unit.dp
23
24 @Composable
25 fun StreakCard(
26     currentStreak: Int,
27     longestStreak: Int,
28     modifier: Modifier = Modifier,
29 ) {
30     Box(
31         modifier = modifier
32             .fillMaxWidth()
33             .clip(MaterialTheme.shapes.extraLarge)
34             .background(MaterialTheme.colorScheme.secondary),
35     ) {
36         Box(
37             modifier = Modifier
38                 .align(Alignment.TopEnd)
39                 .size(96.dp)
40                 .offset(x = 24.dp, y = (-24).dp)
41                 .background(Color.Black.copy(alpha = 0.1f), CircleShape),
42         )
43
44         Row(
45             modifier = Modifier
46                 .fillMaxWidth()
47                 .padding(20.dp),
48             horizontalArrangement = Arrangement.SpaceBetween,
49             verticalAlignment = Alignment.CenterVertically,
50         ) {
51             Column {
52                 Text(
53                     text = "STREAK",
54                     style = MaterialTheme.typography.labelMedium,
55                     color = MaterialTheme.colorScheme.onSecondary.copy(alpha = 0.7f),
56                 )
57                 Row(verticalAlignment = Alignment.Bottom) {
58                     Text(
59                         text = "$currentStreak",
60                         style = MaterialTheme.typography.displaySmall,
61                         color = MaterialTheme.colorScheme.onSecondary,
62                     )
63                     Spacer(modifier = Modifier.width(4.dp))
64                     Text(
65                         text = "days",
66                         style = MaterialTheme.typography.bodyLarge,
67                         color = MaterialTheme.colorScheme.onSecondary.copy(alpha = 0.8f),
68                         modifier = Modifier.padding(bottom = 6.dp),
69                     )
70                 }
71             }
72
73             Column(horizontalAlignment = Alignment.End) {
74                 Text(
75                     text = "BEST",
76                     style = MaterialTheme.typography.labelMedium,
77                     color = MaterialTheme.colorScheme.onSecondary.copy(alpha = 0.7f),
78                 )
79                 Text(
80                     text = "$longestStreak days",
81                     style = MaterialTheme.typography.titleLarge,
82                     color = MaterialTheme.colorScheme.onSecondary,
83                 )

```



```
84         }
85     }
86 }
87 }
```

android/app/src/main/java/com/runway/android/ui/components/WeatherInfo.kt

Kotlin | 41줄

```
1 package com.runway.android.ui.components
2
3 import androidx.compose.ui.graphics.Color
4
5 data class WeatherInfo(
6     val tempCelsius: Int,
7     val humidity: Int,
8     val pm10: Int,
9     val pm25: Int,
10    val conditionId: Int,
11    val condition: String,
12    val description: String,
13 )
14
15 fun pm10Quality(pm10: Int): String = when {
16     pm10 <= 30 -> "좋음"
17     pm10 <= 80 -> "보통"
18     pm10 <= 150 -> "나쁨"
19     else -> "매우나쁨"
20 }
21
22 fun pm25Quality(pm25: Int): String = when {
23     pm25 <= 15 -> "좋음"
24     pm25 <= 35 -> "보통"
25     pm25 <= 75 -> "나쁨"
26     else -> "매우나쁨"
27 }
28
29 fun pm10Color(pm10: Int): Color = when {
30     pm10 <= 30 -> Color(0xFF7EC850)
31     pm10 <= 80 -> Color(0xFFE8C44B)
32     pm10 <= 150 -> Color(0xFFE8904B)
33     else -> Color(0xFFE84B4B)
34 }
35
36 fun pm25Color(pm25: Int): Color = when {
37     pm25 <= 15 -> Color(0xFF7EC850)
38     pm25 <= 35 -> Color(0xFFE8C44B)
39     pm25 <= 75 -> Color(0xFFE8904B)
40     else -> Color(0xFFE84B4B)
41 }
```

```

1 package com.runway.android.ui.components
2
3 import androidx.compose.animation.core.animateFloatAsState
4 import androidx.compose.animation.core.tween
5 import androidx.compose.foundation.background
6 import androidx.compose.foundation.layout.Box
7 import androidx.compose.foundation.layout.Column
8 import androidx.compose.foundation.layout.Row
9 import androidx.compose.foundation.layout.Spacer
10 import androidx.compose.foundation.layout.fillMaxWidth
11 import androidx.compose.foundation.layout.height
12 import androidx.compose.foundation.layout.offset
13 import androidx.compose.foundation.layout.padding
14 import androidx.compose.foundation.layout.size
15 import androidx.compose.foundation.layout.width
16 import androidx.compose.foundation.shape.CircleShape
17 import androidx.compose.material3.MaterialTheme
18 import androidx.compose.material3.Text
19 import androidx.compose.runtime.Composable
20 import androidx.compose.runtime.LaunchedEffect
21 import androidx.compose.runtime.getValue
22 import androidx.compose.runtime.mutableStateOf
23 import androidx.compose.runtime.remember
24 import androidx.compose.runtime.setValue
25 import androidx.compose.ui.Alignment
26 import androidx.compose.ui.Modifier
27 import androidx.compose.ui.draw.clip
28 import androidx.compose.ui.graphics.Color
29 import androidx.compose.ui.unit.dp
30
31 data class WeeklyStats(
32     val distanceKm: String = "24.6",
33     val runs: String = "4",
34     val avgPace: String = "5'12\"",
35     val calories: String = "1,820",
36 )
37
38 @Composable
39 fun WeeklyStatsCard(
40     stats: WeeklyStats,
41     modifier: Modifier = Modifier,
42 ) {
43     var triggered by remember { mutableStateOf(false) }
44     val distTarget = stats.distanceKm.toFloatOrNull() ?: 0f
45     LaunchedEffect(stats.distanceKm) { triggered = true }
46     val animatedDist by animateFloatAsState(
47         targetValue = if (triggered) distTarget else 0f,
48         animationSpec = tween(900),
49         label = "weeklyDist",
50     )
51
52     Box(
53         modifier = modifier
54             .fillMaxWidth()
55             .clip(MaterialTheme.shapes.extraLarge)
56             .background(MaterialTheme.colorScheme.primary),
57     ) {
58         // Decorative circle - partially bleeds off the top-right corner
59         Box(
60             modifier = Modifier
61                 .align(Alignment.TopEnd)
62                 .size(128.dp)
63                 .offset(x = 32.dp, y = (-32).dp)
64                 .background(Color.Black.copy(alpha = 0.1f), CircleShape),
65         )
66
67         Column(modifier = Modifier.padding(20.dp)) {
68             Text(
69                 text = "THIS WEEK",
70                 style = MaterialTheme.typography.labelMedium,
71                 color = MaterialTheme.colorScheme.onPrimary.copy(alpha = 0.7f),
72             )
73
74             Spacer(modifier = Modifier.height(4.dp))
75
76             Row(verticalAlignment = Alignment.Bottom) {
77                 Text(
78                     text = "%.1f".format(animatedDist),
79                     style = MaterialTheme.typography.displayMedium,
80                     color = MaterialTheme.colorScheme.onPrimary,
81                 )
82                 Spacer(modifier = Modifier.width(6.dp))
83                 Text(

```

```

84         text = "km",
85         style = MaterialTheme.typography.bodyLarge,
86         color = MaterialTheme.colorScheme.onPrimary.copy(alpha = 0.8f),
87         modifier = Modifier.padding(bottom = 8.dp),
88     )
89 }
90
91 Spacer(modifier = Modifier.height(16.dp))
92
93 Row(horizontalArrangement = androidx.compose.foundation.layout.Arrangement.spacedBy(20.dp)) {
94     WeeklyStatItem(label = "RUNS", value = stats.runs)
95     WeeklyStatItem(label = "PACE", value = stats.avgPace)
96     WeeklyStatItem(label = "CAL", value = stats.calories)
97 }
98 }
99 }
100 }
101
102 @Composable
103 private fun WeeklyStatItem(label: String, value: String) {
104     Column {
105         Text(
106             text = label,
107             style = MaterialTheme.typography.labelMedium,
108             color = MaterialTheme.colorScheme.onPrimary.copy(alpha = 0.7f),
109         )
110         Spacer(modifier = Modifier.height(2.dp))
111         Text(
112             text = value,
113             style = MaterialTheme.typography.titleLarge,
114             color = MaterialTheme.colorScheme.onPrimary,
115         )
116     }
117 }

```

```

1  package com.runway.android.ui.course
2
3  import androidx.compose.foundation.layout.Arrangement
4  import androidx.compose.foundation.layout.Column
5  import androidx.compose.foundation.layout.Row
6  import androidx.compose.foundation.layout.Spacer
7  import androidx.compose.foundation.layout.fillMaxWidth
8  import androidx.compose.foundation.layout.height
9  import androidx.compose.foundation.layout.padding
10 import androidx.compose.material3.AlertDialog
11 import androidx.compose.material3.CircularProgressIndicator
12 import androidx.compose.material3.HorizontalDivider
13 import androidx.compose.material3.MaterialTheme
14 import androidx.compose.material3.Switch
15 import androidx.compose.material3.Text
16 import androidx.compose.material3.TextButton
17 import androidx.compose.runtime.Composable
18 import androidx.compose.ui.Alignment
19 import androidx.compose.ui.Modifier
20 import androidx.compose.ui.unit.dp
21 import com.runway.android.ui.components.RunwayTextField
22
23 @Composable
24 fun CreateCourseDialog(
25     courseName: String,
26     onCourseNameChange: (String) -> Unit,
27     isLoop: Boolean,
28     onIsLoopChange: (Boolean) -> Unit,
29     isCreating: Boolean,
30     errorMessage: String?,
31     onConfirm: () -> Unit,
32     onDismiss: () -> Unit,
33 ) {
34     AlertDialog(
35         onDismissRequest = onDismiss,
36         containerColor = MaterialTheme.colorScheme.surface,
37         title = {
38             Text(
39                 text = "코스 저장",
40                 style = MaterialTheme.typography.titleLarge,
41                 color = MaterialTheme.colorScheme.onSurface,
42             )
43         },
44         text = {
45             Column(verticalArrangement = Arrangement.spacedBy(16.dp)) {
46                 Text(
47                     text = "내 코스로 저장합니다. 코스를 10회 완주하면 공개 코스로 등록할 수 있습니다.",
48                     style = MaterialTheme.typography.bodySmall,
49                     color = MaterialTheme.colorScheme.onSurfaceVariant,
50                 )
51
52                 RunwayTextField(
53                     value = courseName,
54                     onValueChange = onCourseNameChange,
55                     label = "코스 이름 *",
56                     placeholder = "아침 러닝 코스",
57                     isError = errorMessage != null && courseName.isBlank(),
58                 )
59
60                 HorizontalDivider(color = MaterialTheme.colorScheme.outlineVariant)
61
62                 ToggleRow(
63                     label = "루프 코스",
64                     description = "시작점과 종료점이 같은 코스",
65                     checked = isLoop,
66                     onCheckedChange = onIsLoopChange,
67                 )
68
69                 if (errorMessage != null) {
70                     Text(
71                         text = errorMessage,
72                         style = MaterialTheme.typography.bodySmall,
73                         color = MaterialTheme.colorScheme.error,
74                     )
75                 }
76             }
77         },
78         confirmButton = {
79             if (isCreating) {
80                 CircularProgressIndicator(
81                     modifier = Modifier.padding(horizontal = 16.dp, vertical = 8.dp),
82                     strokeWidth = 2.dp,
83                 )
84             }
85         },
86     )
87 }

```

```

84         } else {
85             TextButton(
86                 onClick = onConfirm,
87                 enabled = courseName.isNotBlank(),
88             ) {
89                 Text(
90                     text = "만들기",
91                     color = if (courseName.isNotBlank())
92                         MaterialTheme.colorScheme.primary
93                     else
94                         MaterialTheme.colorScheme.onSurfaceVariant,
95                 )
96             }
97         },
98     ),
99     dismissButton = {
100         TextButton(
101             onClick = onDismiss,
102             enabled = !isCreating,
103         ) {
104             Text(
105                 text = "취소",
106                 color = MaterialTheme.colorScheme.onSurfaceVariant,
107             )
108         }
109     },
110 )
111 }
112
113 @Composable
114 private fun ToggleRow(
115     label: String,
116     description: String,
117     checked: Boolean,
118     onCheckedChange: (Boolean) -> Unit,
119 ) {
120     Row(
121         modifier = Modifier.fillMaxWidth(),
122         horizontalArrangement = Arrangement.SpaceBetween,
123         verticalAlignment = Alignment.CenterVertically,
124     ) {
125         Column(modifier = Modifier.weight(1f)) {
126             Text(
127                 text = label,
128                 style = MaterialTheme.typography.bodyMedium,
129                 color = MaterialTheme.colorScheme.onSurface,
130             )
131             Spacer(modifier = Modifier.height(2.dp))
132             Text(
133                 text = description,
134                 style = MaterialTheme.typography.bodySmall,
135                 color = MaterialTheme.colorScheme.onSurfaceVariant,
136             )
137         }
138         Switch(
139             checked = checked,
140             onCheckedChange = onCheckedChange,
141             modifier = Modifier.padding(start = 8.dp),
142         )
143     }
144 }

```

```

1 package com.runway.android.ui.course.detail
2
3 import androidx.compose.foundation.BorderStroke
4 import androidx.compose.foundation.background
5 import androidx.compose.foundation.clickable
6 import androidx.compose.foundation.layout.Arrangement
7 import androidx.compose.foundation.layout.Box
8 import androidx.compose.foundation.layout.Column
9 import androidx.compose.foundation.layout.ExperimentalLayoutApi
10 import androidx.compose.foundation.layout.FlowRow
11 import androidx.compose.foundation.layout.Row
12 import androidx.compose.foundation.layout.Spacer
13 import androidx.compose.foundation.layout.fillMaxSize
14 import androidx.compose.foundation.layout.fillMaxWidth
15 import androidx.compose.foundation.layout.height
16 import androidx.compose.foundation.layout.padding
17 import androidx.compose.foundation.layout.size
18 import androidx.compose.foundation.layout.width
19 import androidx.compose.foundation.rememberScrollState
20 import androidx.compose.foundation.verticalScroll
21 import androidx.compose.material.icons.Icons
22 import androidx.compose.material.icons.automirrored.filled.ArrowBack
23 import androidx.compose.material.icons.filled.EmojiEvents
24 import androidx.compose.material.icons.filled.Flag
25 import androidx.compose.material.icons.filled.Loop
26 import androidx.compose.material.icons.filled.Person
27 import androidx.compose.material.icons.filled.PlayArrow
28 import androidx.compose.material.icons.filled.Share
29 import androidx.compose.material.icons.filled.Star
30 import androidx.compose.material.icons.filled.StarBorder
31 import androidx.compose.material3.AlertDialog
32 import androidx.compose.material3.Card
33 import androidx.compose.material3.CardDefaults
34 import androidx.compose.material3.CircularProgressIndicator
35 import androidx.compose.material3.ExperimentalMaterial3Api
36 import androidx.compose.material3.HorizontalDivider
37 import androidx.compose.material3.Icon
38 import androidx.compose.material3.IconButton
39 import androidx.compose.material3.MaterialTheme
40 import androidx.compose.material3.Scaffold
41 import androidx.compose.material3.SuggestionChip
42 import androidx.compose.material3.Surface
43 import androidx.compose.material3.Text
44 import androidx.compose.material3.TextButton
45 import androidx.compose.material3.TopAppBar
46 import androidx.compose.material3.TopAppBarDefaults
47 import androidx.compose.material3.pulltorefresh.PullToRefreshBox
48 import androidx.compose.animation.core.animateFloatAsState
49 import androidx.compose.animation.core.animateIntAsState
50 import androidx.compose.animation.core.tween
51 import androidx.compose.runtime.Composable
52 import androidx.compose.runtime.DisposableEffect
53 import androidx.compose.runtime.LaunchedEffect
54 import androidx.compose.runtime.getValue
55 import androidx.compose.runtime.mutableStateOf
56 import androidx.compose.runtime.remember
57 import androidx.compose.runtime.setValue
58 import androidx.compose.ui.Alignment
59 import androidx.compose.ui.Lifecycle
60 import androidx.compose.ui.LifecycleEventObserver
61 import androidx.compose.ui.lifecycle.LocalLifecycleOwner
62 import androidx.compose.ui.Modifier
63 import androidx.compose.ui.draw.clip
64 import androidx.compose.ui.text.font.FontWeight
65 import androidx.compose.ui.text.style.TextAlign
66 import androidx.compose.ui.unit.dp
67 import androidx.compose.ui.platform.LocalContext
68 import androidx.hilt.navigation.compose.hiltViewModel
69 import com.runway.android.core.map.MapPoint
70 import com.runway.android.core.share.ShareUtils
71 import com.runway.android.data.attempt.model.LeadershipItem
72 import com.runway.android.data.attempt.model.MyBestAttemptResponse
73 import com.runway.android.ui.components.RouteMapView
74
75 @OptIn(ExperimentalMaterial3Api::class)
76 @Composable
77 fun CourseDetailScreen(
78     onBack: () -> Unit,
79     onNavigateToAttempt: (courseId: String, courseAttemptId: String, runningRecordId: String) -> Unit,
80     onNavigateToLeaderboard: (courseId: String) -> Unit,
81     onNavigateToMap: (courseId: String) -> Unit = {},
82     viewModel: CourseDetailViewModel = hiltViewModel(),
83 ) {

```

```

84     val context = LocalContext.current
85
86     LaunchedEffect(Unit) {
87         viewModel.navigateToAttempt.collect { event ->
88             onNavigateToAttempt(event.courseId, event.courseAttemptId, event.runningRecordId)
89         }
90     }
91
92     // 화면으로 돌아올 때 데이터 갱신 (완주 후 내 기록 업데이트 등)
93     // 최초 진입 시 ON_RESUME은 init { load() }와 중복되므로 건너뛰다.
94     val lifecycleOwner = LocalLifecycleOwner.current
95     var isFirstResume by remember { mutableStateOf(true) }
96     DisposableEffect(lifecycleOwner) {
97         val observer = LifecycleEventObserver { _, event ->
98             if (event == Lifecycle.Event.ON_RESUME) {
99                 if (isFirstResume) {
100                     isFirstResume = false
101                 } else {
102                     viewModel.refresh()
103                 }
104             }
105         }
106         lifecycleOwner.lifecycle.addObserver(observer)
107         onDispose { lifecycleOwner.lifecycle.removeObserver(observer) }
108     }
109
110     // 즐겨찾기 에러 다이얼로그
111     if (viewModel.favoriteError != null) {
112         AlertDialog(
113             onDismissRequest = viewModel::clearFavoriteError,
114             title = { Text("즐거찾기 오류") },
115             text = { Text(viewModel.favoriteError!!) },
116             confirmButton = {
117                 TextButton(onClick = viewModel::clearFavoriteError) { Text("확인") }
118             },
119         )
120     }
121
122     // 도전 시작 실패 다이얼로그
123     if (viewModel.startAttemptError != null) {
124         AlertDialog(
125             onDismissRequest = viewModel::clearStartAttemptError,
126             title = { Text("도전 시작 실패") },
127             text = { Text(viewModel.startAttemptError!!) },
128             confirmButton = {
129                 TextButton(onClick = viewModel::clearStartAttemptError) { Text("확인") }
130             },
131         )
132     }
133
134     // 신고 성공 다이얼로그
135     if (viewModel.reportSuccess) {
136         AlertDialog(
137             onDismissRequest = viewModel::clearReportSuccess,
138             title = { Text("신고 접수 완료") },
139             text = { Text("신고가 접수되었습니다. 검토 후 조치하겠습니다.") },
140             confirmButton = {
141                 TextButton(onClick = viewModel::clearReportSuccess) { Text("확인") }
142             },
143         )
144     }
145
146     // 평가 성공 다이얼로그
147     if (viewModel.ratingSuccess) {
148         AlertDialog(
149             onDismissRequest = viewModel::clearRatingSuccess,
150             title = { Text("평가 완료") },
151             text = { Text("평가해 주셔서 감사합니다.") },
152             confirmButton = {
153                 TextButton(onClick = viewModel::clearRatingSuccess) { Text("확인") }
154             },
155         )
156     }
157
158     // 평가 다이얼로그
159     if (viewModel.showRateDialog) {
160         RateCourseDialog(
161             selectedRating = viewModel.ratingValue,
162             onRatingChange = viewModel::onRatingValueChange,
163             comment = viewModel.ratingComment,
164             onCommentChange = viewModel::onRatingCommentChange,
165             isSubmitting = viewModel.isSubmittingRating,
166             errorMessage = viewModel.ratingError,
167             onConfirm = viewModel::submitRating,
168             onDismiss = viewModel::dismissRateDialog,
169         )

```

```

170     }
171
172     // 공개 잠금 팝업 - 10회 완주 미달 시
173     if (viewModel.showPublishLockedDialog) {
174         val completions = viewModel.courseDetail?.completionCount ?: 0
175         val remaining = (CourseDetailViewModel.PUBLISH_MIN_COMPLETIONS - completions).coerceAtLeast(0)
176         AlertDialog(
177             onDismissRequest = viewModel::dismissPublishLockedDialog,
178             title = { Text("공개 코스 등록 조건 미달") },
179             text = {
180                 Text(
181                     "내가 만든 코스를 ${CourseDetailViewModel.PUBLISH_MIN_COMPLETIONS}회 완주해야 공개 코스로 등록할 수 있습니다.\n\n" +
182                     "현재 완주: ${completions}회 / ${CourseDetailViewModel.PUBLISH_MIN_COMPLETIONS}회\n" +
183                     "앞으로 ${remaining}회 더 완주하면 공개할 수 있어요.",
184                 )
185             },
186             confirmButton = {
187                 TextButton(onClick = viewModel::dismissPublishLockedDialog) { Text("확인") }
188             },
189         )
190     }
191
192     // 발행 성공 다이얼로그
193     if (viewModel.publishSuccess) {
194         AlertDialog(
195             onDismissRequest = viewModel::clearPublishSuccess,
196             title = { Text("코스 공개 완료") },
197             text = { Text("코스가 공개되었습니다. 다른 러너들이 이 코스에 도전할 수 있습니다.") },
198             confirmButton = {
199                 TextButton(onClick = viewModel::clearPublishSuccess) { Text("확인") }
200             },
201         )
202     }
203
204     // 보관 성공 다이얼로그
205     // 삭제 확인 다이얼로그
206     if (viewModel.showDeleteDialog) {
207         AlertDialog(
208             onDismissRequest = viewModel::dismissDeleteDialog,
209             title = { Text("코스를 삭제할까요?") },
210             text = { Text("삭제된 코스는 복구할 수 없습니다.") },
211             confirmButton = {
212                 TextButton(
213                     onClick = viewModel::deleteDraftCourse,
214                     colors = androidx.compose.material3.ButtonDefaults.textButtonColors(
215                         contentColor = MaterialTheme.colorScheme.error
216                     ),
217                 ) { Text("삭제") }
218             },
219             dismissButton = {
220                 TextButton(onClick = viewModel::dismissDeleteDialog) { Text("취소") }
221             },
222         )
223     }
224
225     // 삭제 성공 → 뒤로가기
226     LaunchedEffect(viewModel.deleteSuccess) {
227         if (viewModel.deleteSuccess) onBackPressed()
228     }
229
230     if (viewModel.archiveSuccess) {
231         AlertDialog(
232             onDismissRequest = viewModel::clearArchiveSuccess,
233             title = { Text("코스 보관 완료") },
234             text = { Text("코스가 보관되었습니다. 탐색 결과에서 더 이상 표시되지 않습니다.") },
235             confirmButton = {
236                 TextButton(onClick = viewModel::clearArchiveSuccess) { Text("확인") }
237             },
238         )
239     }
240
241     // 보관 확인 다이얼로그
242     if (viewModel.showArchiveDialog) {
243         AlertDialog(
244             onDismissRequest = viewModel::dismissArchiveDialog,
245             title = { Text("코스를 보관할까요?") },
246             text = {
247                 Column {
248                     Text("보관된 코스는 탐색 결과에 표시되지 않으며, 더 이상 도전을 시작할 수 없습니다.")
249                     if (viewModel.archiveError != null) {
250                         Spacer(modifier = Modifier.height(8.dp))
251                         Text(
252                             text = viewModel.archiveError!!,
253                             style = MaterialTheme.typography.bodySmall,
254                             color = MaterialTheme.colorScheme.error,
255                         )
256                     }
257                 }
258             },
259             confirmButton = {
260                 TextButton(onClick = viewModel::archiveCourse) { Text("확인") }
261             },
262             dismissButton = {
263                 TextButton(onClick = viewModel::dismissArchiveDialog) { Text("취소") }
264             },
265         )
266     }
267 }

```



```

256     }
257 }
258 },
259 confirmButton = {
260     TextButton(
261         onClick = viewModel::submitArchive,
262         enabled = !viewModel.isArchiving,
263     ) {
264         if (viewModel.isArchiving) {
265             CircularProgressIndicator(
266                 modifier = Modifier.size(16.dp),
267                 strokeWidth = 2.dp,
268                 color = MaterialTheme.colorScheme.error,
269             )
270         } else {
271             Text("보관하기", color = MaterialTheme.colorScheme.error)
272         }
273     }
274 },
275 dismissButton = {
276     TextButton(
277         onClick = viewModel::dismissArchiveDialog,
278         enabled = !viewModel.isArchiving,
279     ) { Text("취소") }
280 },
281 )
282 }
283
284 // 발행 다이얼로그
285 if (viewModel.showPublishDialog) {
286     PublishCourseDialog(
287         isPublishing = viewModel.isPublishing,
288         error = viewModel.publishError,
289         onDismiss = viewModel::dismissPublishDialog,
290         onPublish = viewModel::submitPublish,
291     )
292 }
293
294 // 신고 다이얼로그
295 if (viewModel.showReportDialog) {
296     ReportCourseDialog(
297         selectedReason = viewModel.reportReason,
298         onReasonChange = viewModel::onReportReasonChange,
299         description = viewModel.reportDescription,
300         onDescriptionChange = viewModel::onReportDescriptionChange,
301         isSubmitting = viewModel.isSubmittingReport,
302         errorMessage = viewModel.reportError,
303         onConfirm = viewModel::submitReport,
304         onDismiss = viewModel::dismissReportDialog,
305     )
306 }
307
308 Scaffold(
309     topBar = {
310         TopAppBar(
311             title = {
312                 Text(
313                     text = viewModel.courseDetail?.name ?: "코스 상세",
314                     style = MaterialTheme.typography.titleMedium,
315                     color = MaterialTheme.colorScheme.onBackground,
316                     maxLines = 1,
317                 )
318             },
319             navigationIcon = {
320                 IconButton(onClick = onBack) {
321                     Icon(
322                         imageVector = Icons.AutoMirrored.Filled.ArrowBack,
323                         contentDescription = "뒤로 가기",
324                         tint = MaterialTheme.colorScheme.onBackground,
325                     )
326                 }
327             },
328             actions = {
329                 IconButton(
330                     onClick = viewModel::toggleFavorite,
331                     enabled = !viewModel.isFavoriteToggling,
332                 ) {
333                     Icon(
334                         imageVector = if (viewModel.isFavorited) Icons.Filled.Star else Icons.Filled.StarBorder,
335                         contentDescription = if (viewModel.isFavorited) "즐거찾기 해제" else "즐거찾기 추가",
336                         tint = if (viewModel.isFavorited) MaterialTheme.colorScheme.primary
337                             else MaterialTheme.colorScheme.onSurfaceVariant,
338                     )
339                 }
340                 IconButton(
341                     onClick = {

```

```

342         val detail = viewModel.courseDetail
343         if (detail != null) {
344             val distKm = if (detail.distanceMeters >= 1000)
345                 "%.1f km".format(detail.distanceMeters / 1000)
346             else "${detail.distanceMeters.toInt()} m"
347             ShareUtils.shareCourse(
348                 context = context,
349                 name = detail.name,
350                 distanceKm = distKm,
351                 completionCount = detail.completionCount,
352                 isLoop = detail.isLoop,
353             )
354         }
355     },
356 ) {
357     Icon(
358         imageVector = Icons.Filled.Share,
359         contentDescription = "공유",
360         tint = MaterialTheme.colorScheme.onSurfaceVariant,
361     )
362 }
363 IconButton(onClick = viewModel::openReportDialog) {
364     Icon(
365         imageVector = Icons.Filled.Flag,
366         contentDescription = "신고",
367         tint = MaterialTheme.colorScheme.onSurfaceVariant,
368     )
369 }
370 },
371 colors = TopAppBarDefaults.topAppBarColors(
372     containerColor = MaterialTheme.colorScheme.background,
373 ),
374 )
375 },
376 containerColor = MaterialTheme.colorScheme.background,
377 ) { innerPadding ->
378     PullToRefreshBox(
379         isRefreshing = viewModel.isRefreshing,
380         onRefresh = viewModel::pullRefresh,
381         modifier = Modifier
382             .fillMaxSize()
383             .padding(innerPadding),
384     ) {
385         when {
386             viewModel.isLoading -> LoadingState()
387             viewModel.errorMessage != null && viewModel.courseDetail == null ->
388                 ErrorState(
389                     message = viewModel.errorMessage!!,
390                     onRetry = viewModel::refresh,
391                 )
392             viewModel.courseDetail != null ->
393                 CourseDetailContent(
394                     viewModel = viewModel,
395                     onNavigateToLeaderboard = onNavigateToLeaderboard,
396                     onNavigateToMap = onNavigateToMap,
397                 )
398         }
399     }
400 }
401 }
402
403 @Composable
404 private fun LoadingState() {
405     Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
406         CircularProgressIndicator(color = MaterialTheme.colorScheme.primary)
407     }
408 }
409
410 @Composable
411 private fun ErrorState(message: String, onRetry: () -> Unit) {
412     Column(
413         modifier = Modifier
414             .fillMaxSize()
415             .padding(32.dp),
416         horizontalAlignment = Alignment.CenterHorizontally,
417         verticalArrangement = Arrangement.Center,
418     ) {
419         Text(
420             text = "코스를 불러오지 못했습니다",
421             style = MaterialTheme.typography.titleMedium,
422             color = MaterialTheme.colorScheme.onSurface,
423             textAlign = TextAlign.Center,
424         )
425         Spacer(modifier = Modifier.height(8.dp))
426         Text(
427             text = message,

```

```

428         style = MaterialTheme.typography.bodyMedium,
429         color = MaterialTheme.colorScheme.onSurfaceVariant,
430         textAlign = TextAlign.Center,
431     )
432     Spacer(modifier = Modifier.height(20.dp))
433     Surface(
434         onClick = onRetry,
435         shape = MaterialTheme.shapes.extraLarge,
436         color = MaterialTheme.colorScheme.primary,
437     ) {
438         Text(
439             text = "다시 시도",
440             style = MaterialTheme.typography.labelLarge,
441             color = MaterialTheme.colorScheme.onPrimary,
442             modifier = Modifier.padding(horizontal = 24.dp, vertical = 12.dp),
443         )
444     }
445 }
446 }
447
448 @Composable
449 private fun CourseDetailContent(
450     viewModel: CourseDetailViewModel,
451     onNavigateToLeaderboard: (courseId: String) -> Unit,
452     onNavigateToMap: (courseId: String) -> Unit,
453 ) {
454     val course = viewModel.courseDetail ?: return
455
456     Column(
457         modifier = Modifier
458             .fillMaxSize()
459             .verticalScroll(rememberScrollState()),
460     ) {
461         // — Route preview (클릭 → 전체 지도) —
462         Box(
463             modifier = Modifier
464                 .fillMaxWidth()
465                 .height(220.dp)
466                 .clip(MaterialTheme.shapes.medium)
467                 .clickable { onNavigateToMap(viewModel.courseId) },
468         ) {
469             RouteMapView(
470                 points = viewModel.coursePoints.map { MapPoint(it.latitude, it.longitude) },
471                 modifier = Modifier.fillMaxSize(),
472             )
473             // "자세히 보기" badge
474             Surface(
475                 modifier = Modifier
476                     .align(Alignment.BottomEnd)
477                     .padding(10.dp),
478                 shape = MaterialTheme.shapes.large,
479                 color = MaterialTheme.colorScheme.surface.copy(alpha = 0.88f),
480                 border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
481             ) {
482                 Text(
483                     text = "지도 자세히 보기",
484                     style = MaterialTheme.typography.labelMedium,
485                     color = MaterialTheme.colorScheme.primary,
486                     modifier = Modifier.padding(horizontal = 10.dp, vertical = 6.dp),
487                 )
488             }
489         }
490
491         Column(modifier = Modifier.padding(horizontal = 20.dp)) {
492             Spacer(modifier = Modifier.height(20.dp))
493
494             // — 제목 + 루프 배지 —
495             Row(
496                 modifier = Modifier.fillMaxWidth(),
497                 verticalAlignment = Alignment.CenterVertically,
498             ) {
499                 Text(
500                     text = course.name,
501                     style = MaterialTheme.typography.headlineSmall,
502                     color = MaterialTheme.colorScheme.onBackground,
503                     modifier = Modifier.weight(1f),
504                 )
505                 if (course.isLoop) {
506                     Spacer(modifier = Modifier.width(8.dp))
507                     Surface(
508                         shape = MaterialTheme.shapes.small,
509                         color = MaterialTheme.colorScheme.primaryContainer,
510                     ) {
511                         Row(
512                             modifier = Modifier.padding(horizontal = 10.dp, vertical = 4.dp),
513                             verticalAlignment = Alignment.CenterVertically,

```

```

514         ) {
515             Icon(
516                 imageVector = Icons.Filled.Loop,
517                 contentDescription = null,
518                 tint = MaterialTheme.colorScheme.onPrimaryContainer,
519                 modifier = Modifier.size(14.dp),
520             )
521             Spacer(modifier = Modifier.width(3.dp))
522             Text(
523                 text = "루프",
524                 style = MaterialTheme.typography.labelSmall,
525                 color = MaterialTheme.colorScheme.onPrimaryContainer,
526             )
527         }
528     }
529 }
530
531
532 Spacer(modifier = Modifier.height(6.dp))
533
534 // — 크리에이터 —
535 Row(verticalAlignment = Alignment.CenterVertically) {
536     Icon(
537         imageVector = Icons.Filled.Person,
538         contentDescription = null,
539         tint = MaterialTheme.colorScheme.onSurfaceVariant,
540         modifier = Modifier.size(15.dp),
541     )
542     Spacer(modifier = Modifier.width(4.dp))
543     Text(
544         text = course.creator.nickname,
545         style = MaterialTheme.typography.bodyMedium,
546         color = MaterialTheme.colorScheme.onSurfaceVariant,
547     )
548 }
549
550 Spacer(modifier = Modifier.height(20.dp))
551 HorizontalDivider(color = MaterialTheme.colorScheme.outline)
552 Spacer(modifier = Modifier.height(20.dp))
553
554 // — 통계 행 (count-up) —
555 var statsTriggered by remember { mutableStateOf(false) }
556 LaunchedEffect(Unit) { statsTriggered = true }
557 val animatedDistance by animateFloatAsState(
558     targetValue = if (statsTriggered) course.distanceMeters.toFloat() else 0f,
559     animationSpec = tween(1200),
560     label = "dist",
561 )
562 val animatedAttempts by animateIntAsState(
563     targetValue = if (statsTriggered) course.attemptCount else 0,
564     animationSpec = tween(1200),
565     label = "attempts",
566 )
567 val animatedCompletions by animateIntAsState(
568     targetValue = if (statsTriggered) course.completionCount else 0,
569     animationSpec = tween(1200),
570     label = "completions",
571 )
572 Row(
573     modifier = Modifier.fillMaxWidth(),
574     horizontalArrangement = Arrangement.SpaceEvenly,
575 ) {
576     StatItem(label = "거리", value = formatDistance(animatedDistance.toDouble()))
577     StatItem(label = "도전", value = "${animatedAttempts}회")
578     StatItem(label = "완주", value = "${animatedCompletions}회")
579 }
580
581 // — 평점 행 —
582 Spacer(modifier = Modifier.height(12.dp))
583 Row(
584     modifier = Modifier.fillMaxWidth(),
585     verticalAlignment = Alignment.CenterVertically,
586     horizontalArrangement = Arrangement.SpaceBetween,
587 ) {
588     Row(
589         verticalAlignment = Alignment.CenterVertically,
590         horizontalArrangement = Arrangement.spacedBy(4.dp),
591     ) {
592         Icon(
593             imageVector = Icons.Filled.Star,
594             contentDescription = null,
595             tint = MaterialTheme.colorScheme.primary,
596             modifier = Modifier.size(16.dp),
597         )
598         if (course.avgRating != null && (course.ratingCount ?: 0L) > 0) {
599             Text(

```

```

600         text = "%.1f".format(course.avgRating),
601         style = MaterialTheme.typography.bodyMedium,
602         color = MaterialTheme.colorScheme.onBackground,
603     )
604     Text(
605         text = " (${course.ratingCount}개)",
606         style = MaterialTheme.typography.bodySmall,
607         color = MaterialTheme.colorScheme.onSurfaceVariant,
608     )
609 } else {
610     Text(
611         text = "아직 평가 없음",
612         style = MaterialTheme.typography.bodySmall,
613         color = MaterialTheme.colorScheme.onSurfaceVariant,
614     )
615 }
616 }
617 if (!viewModel.hasRated) {
618     androidx.compose.material3.Surface(
619         onClick = viewModel::openRateDialog,
620         shape = MaterialTheme.shapes.extraLarge,
621         color = MaterialTheme.colorScheme.surface,
622         border = androidx.compose.foundation.BorderStroke(
623             1.dp, MaterialTheme.colorScheme.outline
624         ),
625     ) {
626         Text(
627             text = "평가하기",
628             style = MaterialTheme.typography.labelSmall,
629             color = MaterialTheme.colorScheme.onSurface,
630             modifier = Modifier.padding(horizontal = 12.dp, vertical = 6.dp),
631         )
632     }
633 } else {
634     Text(
635         text = "평가 완료",
636         style = MaterialTheme.typography.labelSmall,
637         color = MaterialTheme.colorScheme.primary,
638         modifier = Modifier.padding(horizontal = 12.dp, vertical = 6.dp),
639     )
640 }
641 }
642
643 // — 설명 —
644 if (!course.description.isNullOrBlank()) {
645     Spacer(modifier = Modifier.height(20.dp))
646     HorizontalDivider(color = MaterialTheme.colorScheme.outline)
647     Spacer(modifier = Modifier.height(20.dp))
648     Text(
649         text = "코스 설명",
650         style = MaterialTheme.typography.titleSmall,
651         color = MaterialTheme.colorScheme.onSurfaceVariant,
652     )
653     Spacer(modifier = Modifier.height(6.dp))
654     Text(
655         text = course.description,
656         style = MaterialTheme.typography.bodyMedium,
657         color = MaterialTheme.colorScheme.onBackground,
658     )
659 }
660
661 // — 코스 메타데이터 —
662 val hasMetadata = course.difficulty != null || course.riskLevel != null ||
663     course.slopeLevel != null || course.surfaceType != null ||
664     course.recommendedTime != null || !course.warnings.isNullOrBlank()
665 if (hasMetadata) {
666     Spacer(modifier = Modifier.height(20.dp))
667     HorizontalDivider(color = MaterialTheme.colorScheme.outline)
668     Spacer(modifier = Modifier.height(20.dp))
669     CourseMetadataSection(course = course)
670 }
671
672 // — 초안 상태 + 소유자 → 공개하기 + 삭제 버튼 —
673 if (course.status == "draft" && course.isOwner) {
674     Spacer(modifier = Modifier.height(20.dp))
675     HorizontalDivider(color = MaterialTheme.colorScheme.outline)
676     Spacer(modifier = Modifier.height(16.dp))
677     Row(
678         modifier = Modifier.fillMaxWidth(),
679         horizontalArrangement = Arrangement.spacedBy(10.dp),
680     ) {
681         Surface(
682             modifier = Modifier.weight(1f),
683             shape = MaterialTheme.shapes.extraLarge,
684             color = MaterialTheme.colorScheme.secondaryContainer,
685             onClick = viewModel::openPublishDialog,

```

```

686         ) {
687             Row(
688                 modifier = Modifier.padding(vertical = 14.dp),
689                 horizontalArrangement = Arrangement.Center,
690             ) {
691                 Text(
692                     text = "공개하기",
693                     style = MaterialTheme.typography.labelLarge,
694                     color = MaterialTheme.colorScheme.onSecondaryContainer,
695                 )
696             }
697         }
698         Surface(
699             modifier = Modifier.weight(1f),
700             shape = MaterialTheme.shapes.extraLarge,
701             color = MaterialTheme.colorScheme.errorContainer,
702             onClick = viewModel::openDeleteDialog,
703         ) {
704             Row(
705                 modifier = Modifier.padding(vertical = 14.dp),
706                 horizontalArrangement = Arrangement.Center,
707             ) {
708                 Text(
709                     text = "삭제",
710                     style = MaterialTheme.typography.labelLarge,
711                     color = MaterialTheme.colorScheme.onErrorContainer,
712                 )
713             }
714         }
715     }
716 }
717
718 Spacer(modifier = Modifier.height(20.dp))
719 HorizontalDivider(color = MaterialTheme.colorScheme.outline)
720 Spacer(modifier = Modifier.height(20.dp))
721
722 // — 리더보드 섹션 —
723 Row(
724     modifier = Modifier.fillMaxWidth(),
725     verticalAlignment = Alignment.CenterVertically,
726     horizontalArrangement = Arrangement.SpaceBetween,
727 ) {
728     Row(verticalAlignment = Alignment.CenterVertically) {
729         Icon(
730             imageVector = Icons.Filled.EmojiEvents,
731             contentDescription = null,
732             tint = MaterialTheme.colorScheme.primary,
733             modifier = Modifier.size(18.dp),
734         )
735         Spacer(modifier = Modifier.width(6.dp))
736         Text(
737             text = "리더보드",
738             style = MaterialTheme.typography.titleSmall,
739             color = MaterialTheme.colorScheme.onBackground,
740         )
741     }
742     Surface(
743         onClick = { onNavigateToLeaderboard(viewModel.courseId) },
744         shape = MaterialTheme.shapes.extraLarge,
745         color = MaterialTheme.colorScheme.surface,
746         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
747     ) {
748         Text(
749             text = "순위 보기",
750             style = MaterialTheme.typography.labelSmall,
751             color = MaterialTheme.colorScheme.onSurface,
752             modifier = Modifier.padding(horizontal = 12.dp, vertical = 6.dp),
753         )
754     }
755 }
756
757 Spacer(modifier = Modifier.height(12.dp))
758
759 // — 인라인 Top 5 미리보기 —
760 when {
761     viewModel.isLoadingLeaderboard -> {
762         Box(
763             modifier = Modifier
764                 .fillMaxWidth()
765                 .height(48.dp),
766             contentAlignment = Alignment.Center,
767         ) {
768             CircularProgressIndicator(
769                 modifier = Modifier.size(20.dp),
770                 strokeWidth = 2.dp,
771                 color = MaterialTheme.colorScheme.primary,

```

```

772         )
773     }
774 }
775 viewModel.previewLeaderboard.isEmpty() -> {
776     Text(
777         text = "아직 완주 기록이 없습니다.",
778         style = MaterialTheme.typography.bodySmall,
779         color = MaterialTheme.colorScheme.onSurfaceVariant,
780         modifier = Modifier.padding(vertical = 8.dp),
781     )
782 }
783 else -> {
784     Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
785         viewModel.previewLeaderboard.forEach { item ->
786             LeaderboardPreviewRow(item = item)
787         }
788     }
789 }
790 }
791
792 // — 내 기록 섹션 —
793 Spacer(modifier = Modifier.height(20.dp))
794 HorizontalDivider(color = MaterialTheme.colorScheme.outline)
795 Spacer(modifier = Modifier.height(20.dp))
796 if (viewModel.myBestAttempt != null) {
797     MyBestAttemptSection(best = viewModel.myBestAttempt!!)
798 } else if (!viewModel.isLoading) {
799     Row(verticalAlignment = Alignment.CenterVertically) {
800         Icon(
801             imageVector = Icons.Filled.Person,
802             contentDescription = null,
803             tint = MaterialTheme.colorScheme.primary,
804             modifier = Modifier.size(18.dp),
805         )
806         Spacer(modifier = Modifier.width(6.dp))
807         Text(
808             text = "내 기록",
809             style = MaterialTheme.typography.titleSmall,
810             color = MaterialTheme.colorScheme.onBackground,
811         )
812     }
813     Spacer(modifier = Modifier.height(10.dp))
814     Text(
815         text = "아직 이 코스를 완주한 기록이 없어요. 도전해 보세요!",
816         style = MaterialTheme.typography.bodyMedium,
817         color = MaterialTheme.colorScheme.onSurfaceVariant,
818     )
819 }
820
821 Spacer(modifier = Modifier.height(24.dp))
822
823 // — 도전 CTA —
824 val hasCompletions = (viewModel.myBestAttempt?.completionCount ?: 0) > 0
825 Surface(
826     modifier = Modifier.fillMaxWidth(),
827     shape = MaterialTheme.shapes.extraLarge,
828     color = if (viewModel.isStartingAttempt)
829         MaterialTheme.colorScheme.primary.copy(alpha = 0.6f)
830     else
831         MaterialTheme.colorScheme.primary,
832     onClick = { if (!viewModel.isStartingAttempt) viewModel.startAttempt() },
833 ) {
834     Row(
835         modifier = Modifier.padding(vertical = 16.dp),
836         horizontalArrangement = Arrangement.Center,
837         verticalAlignment = Alignment.CenterVertically,
838     ) {
839         if (viewModel.isStartingAttempt) {
840             CircularProgressIndicator(
841                 modifier = Modifier.size(18.dp),
842                 color = MaterialTheme.colorScheme.onPrimary,
843                 strokeWidth = 2.dp,
844             )
845             Spacer(modifier = Modifier.width(8.dp))
846             Text(
847                 text = "도전 시작 중...",
848                 style = MaterialTheme.typography.labelLarge,
849                 color = MaterialTheme.colorScheme.onPrimary,
850             )
851         } else {
852             Icon(
853                 imageVector = Icons.Filled.PlayArrow,
854                 contentDescription = null,
855                 tint = MaterialTheme.colorScheme.onPrimary,
856                 modifier = Modifier.size(18.dp),
857             )

```

```

858         Spacer(modifier = Modifier.width(6.dp))
859         Text(
860             text = if (hasCompletions) "다시 도전하기" else "코스 도전하기",
861             style = MaterialTheme.typography.labelLarge,
862             color = MaterialTheme.colorScheme.onPrimary,
863         )
864     }
865 }
866 }
867
868     Spacer(modifier = Modifier.height(32.dp))
869 }
870 }
871 }
872
873 @Composable
874 private fun StatItem(label: String, value: String) {
875     Column(horizontalAlignment = Alignment.CenterHorizontally) {
876         Text(
877             text = value,
878             style = MaterialTheme.typography.titleLarge,
879             color = MaterialTheme.colorScheme.onBackground,
880         )
881         Spacer(modifier = Modifier.height(2.dp))
882         Text(
883             text = label,
884             style = MaterialTheme.typography.bodySmall,
885             color = MaterialTheme.colorScheme.onSurfaceVariant,
886         )
887     }
888 }
889
890 @Composable
891 private fun LeaderboardPreviewRow(item: LeaderboardItem) {
892     Row(
893         modifier = Modifier.fillMaxWidth(),
894         verticalAlignment = Alignment.CenterVertically,
895         horizontalArrangement = Arrangement.SpaceBetween,
896     ) {
897         Row(
898             verticalAlignment = Alignment.CenterVertically,
899             horizontalArrangement = Arrangement.spacedBy(10.dp),
900         ) {
901             Text(
902                 text = "#${item.rank}",
903                 style = MaterialTheme.typography.labelMedium,
904                 fontWeight = FontWeight.Bold,
905                 color = if (item.rank <= 3)
906                     MaterialTheme.colorScheme.primary
907                 else
908                     MaterialTheme.colorScheme.onSurfaceVariant,
909                 modifier = Modifier.width(30.dp),
910             )
911             Text(
912                 text = item.nickname,
913                 style = MaterialTheme.typography.bodyMedium,
914                 color = MaterialTheme.colorScheme.onBackground,
915             )
916         }
917         Text(
918             text = formatSeconds(item.bestTimeSeconds),
919             style = MaterialTheme.typography.labelMedium,
920             color = MaterialTheme.colorScheme.onSurfaceVariant,
921         )
922     }
923 }
924
925 private fun formatSeconds(seconds: Int): String {
926     val h = seconds / 3600
927     val m = (seconds % 3600) / 60
928     val s = seconds % 60
929     return if (h > 0) "%d:%02d:%02d".format(h, m, s)
930     else "%d:%02d".format(m, s)
931 }
932
933 private fun formatDistance(meters: Double): String = when {
934     meters >= 1000 -> "%.1f km".format(meters / 1000)
935     else -> "${meters.toInt()} m"
936 }
937
938 @OptIn(ExperimentalLayoutApi::class)
939 @Composable
940 private fun CourseMetadataSection(course: com.runway.android.data.course.model.CourseDetailResponse) {
941     Text(
942         text = "코스 정보",
943         style = MaterialTheme.typography.titleSmall,

```



```

944         color = MaterialTheme.colorScheme.onSurfaceVariant,
945     )
946     Spacer(modifier = Modifier.height(10.dp))
947     FlowRow(
948         horizontalArrangement = Arrangement.spacedBy(8.dp),
949         verticalArrangement = Arrangement.spacedBy(8.dp),
950     ) {
951         course.difficulty?.let { MetadataChip(label = difficultyLabel(it)) }
952         course.slopeLevel?.let { MetadataChip(label = slopeLabel(it)) }
953         course.riskLevel?.let { MetadataChip(label = riskLabel(it)) }
954         course.surfaceType?.let { MetadataChip(label = surfaceLabel(it)) }
955         course.recommendedTime?.let { MetadataChip(label = timeLabel(it)) }
956     }
957     if (!course.warnings.isNullOrBlank()) {
958         Spacer(modifier = Modifier.height(10.dp))
959         Card(
960             colors = CardDefaults.cardColors(
961                 containerColor = MaterialTheme.colorScheme.errorContainer.copy(alpha = 0.4f),
962             ),
963             modifier = Modifier.fillMaxWidth(),
964         ) {
965             Row(
966                 modifier = Modifier.padding(12.dp),
967                 horizontalArrangement = Arrangement.spacedBy(8.dp),
968             ) {
969                 Icon(
970                     imageVector = Icons.Filled.Flag,
971                     contentDescription = null,
972                     tint = MaterialTheme.colorScheme.error,
973                     modifier = Modifier.size(16.dp),
974                 )
975                 Text(
976                     text = course.warnings,
977                     style = MaterialTheme.typography.bodySmall,
978                     color = MaterialTheme.colorScheme.onErrorContainer,
979                 )
980             }
981         }
982     }
983 }
984
985 @Composable
986 private fun MetadataChip(label: String) {
987     SuggestionChip(
988         onClick = {},
989         label = { Text(label, style = MaterialTheme.typography.labelSmall) },
990     )
991 }
992
993 @Composable
994 private fun MyBestAttemptSection(best: MyBestAttemptResponse) {
995     Row(
996         modifier = Modifier.fillMaxWidth(),
997         verticalAlignment = Alignment.CenterVertically,
998     ) {
999         Icon(
1000             imageVector = Icons.Filled.Person,
1001             contentDescription = null,
1002             tint = MaterialTheme.colorScheme.primary,
1003             modifier = Modifier.size(18.dp),
1004         )
1005         Spacer(modifier = Modifier.width(6.dp))
1006         Text(
1007             text = "내 기록",
1008             style = MaterialTheme.typography.titleSmall,
1009             color = MaterialTheme.colorScheme.onBackground,
1010         )
1011     }
1012     Spacer(modifier = Modifier.height(12.dp))
1013     Surface(
1014         modifier = Modifier.fillMaxWidth(),
1015         shape = MaterialTheme.shapes.large,
1016         color = MaterialTheme.colorScheme.surfaceVariant,
1017     ) {
1018         Row(
1019             modifier = Modifier.padding(horizontal = 20.dp, vertical = 14.dp),
1020             horizontalArrangement = Arrangement.SpaceEvenly,
1021         ) {
1022             Column(horizontalAlignment = Alignment.CenterHorizontally) {
1023                 Text(
1024                     text = if (best.bestTimeSeconds != null) formatSeconds(best.bestTimeSeconds) else "--",
1025                     style = MaterialTheme.typography.titleMedium,
1026                     fontWeight = FontWeight.Bold,
1027                     color = MaterialTheme.colorScheme.primary,
1028                 )
1029                 Text(

```

```

1030         text = "베스트 기록",
1031         style = MaterialTheme.typography.bodySmall,
1032         color = MaterialTheme.colorScheme.onSurfaceVariant,
1033     )
1034 }
1035 Column(horizontalAlignment = Alignment.CenterHorizontally) {
1036     Text(
1037         text = "${best.completionCount}회",
1038         style = MaterialTheme.typography.titleMedium,
1039         fontWeight = FontWeight.Bold,
1040         color = MaterialTheme.colorScheme.onSurface,
1041     )
1042     Text(
1043         text = "완주 횟수",
1044         style = MaterialTheme.typography.bodySmall,
1045         color = MaterialTheme.colorScheme.onSurfaceVariant,
1046     )
1047 }
1048 }
1049 }
1050 }
1051
1052 private fun difficultyLabel(v: String) = when (v) { "easy" -> "난이도: 쉬움"; "hard" -> "난이도: 어려움"; else -> "난이도: 보통" }
1053 private fun slopeLabel(v: String) = when (v) { "flat" -> "경사: 평탄"; "steep" -> "경사: 가파름"; else -> "경사: 완만" }
1054 private fun riskLabel(v: String) = when (v) { "low" -> "위험도: 낮음"; "high" -> "위험도: 높음"; else -> "위험도: 보통" }
1055 private fun surfaceLabel(v: String) = when (v) { "road" -> "도로"; "park" -> "공원"; "trail" -> "트레일"; else -> "혼합" }
1056 private fun timeLabel(v: String) = when (v) { "morning" -> "추천: 아침"; "day" -> "추천: 낮"; "night" -> "추천: 저녁"; else -> "추천: 무관" }

```

```

1  package com.runway.android.ui.course.detail
2
3  import androidx.compose.runtime.getValue
4  import androidx.compose.runtime.mutableStateOf
5  import androidx.compose.runtime.setValue
6  import androidx.lifecycle.SavedStateHandle
7  import androidx.lifecycle.ViewModelScope
8  import com.runway.android.core.result.NetworkResult
9  import com.runway.android.data.attempt.model.LeaderboardItem
10 import com.runway.android.data.attempt.model.MyBestAttemptResponse
11 import com.runway.android.data.attempt.model.StartAttemptRequest
12 import com.runway.android.data.course.model.CourseDetailResponse
13 import com.runway.android.data.course.model.CoursePointResponse
14 import com.runway.android.data.course.model.CourseRatingRequest
15 import com.runway.android.data.course.model.CourseReportRequest
16 import android.app.Application
17 import android.content.Context
18 import com.runway.android.data.course.model.PublishCourseRequest
19 import com.runway.android.domain.attempt.CourseAttemptRepository
20 import com.runway.android.domain.course.CourseRepository
21 import dagger.hilt.android.lifecycle.HiltViewModel
22 import kotlinx.coroutines.async
23 import kotlinx.coroutines.flow.MutableSharedFlow
24 import kotlinx.coroutines.flow.asSharedFlow
25 import kotlinx.coroutines.launch
26 import java.time.Instant
27 import javax.inject.Inject
28
29 data class AttemptStartedEvent(
30     val courseId: String,
31     val courseAttemptId: String,
32     val runningRecordId: String,
33 )
34
35 @HiltViewModel
36 class CourseDetailViewModel @Inject constructor(
37     application: Application,
38     savedStateHandle: SavedStateHandle,
39     private val courseRepository: CourseRepository,
40     private val courseAttemptRepository: CourseAttemptRepository,
41 ) : androidx.lifecycle.AndroidViewModel(application) {
42
43     private val ratingPrefs = application.getSharedPreferences("rated_courses", Context.MODE_PRIVATE)
44
45     companion object {
46         const val PUBLISH_MIN_COMPLETIONS = 10
47     }
48
49     val courseId: String = checkNotNull(savedStateHandle["courseId"])
50
51     var isLoading by mutableStateOf(false)
52     private set
53     var isRefreshing by mutableStateOf(false)
54     private set
55     var errorMessage by mutableStateOf<String?>(null)
56     private set
57     var courseDetail by mutableStateOf<CourseDetailResponse?>(null)
58     private set
59     var coursePoints by mutableStateOf<List<CoursePointResponse>>(emptyList())
60     private set
61     var previewLeaderboard by mutableStateOf<List<LeaderboardItem>>(emptyList())
62     private set
63     var isLoadingLeaderboard by mutableStateOf(false)
64     private set
65     var myBestAttempt by mutableStateOf<MyBestAttemptResponse?>(null)
66     private set
67
68     var isFavorited by mutableStateOf(false)
69     private set
70     var isFavoriteToggling by mutableStateOf(false)
71     private set
72     var favoriteError by mutableStateOf<String?>(null)
73     private set
74
75     var isStartingAttempt by mutableStateOf(false)
76     private set
77     var startAttemptError by mutableStateOf<String?>(null)
78     private set
79
80     var showReportDialog by mutableStateOf(false)
81     private set
82     var reportReason by mutableStateOf("")
83     private set

```

```

84     var reportDescription by mutableStateOf("")
85         private set
86     var isSubmittingReport by mutableStateOf(false)
87         private set
88     var reportError by mutableStateOf<String?>(null)
89         private set
90     var reportSuccess by mutableStateOf(false)
91         private set
92
93     var showPublishDialog by mutableStateOf(false)
94         private set
95     var isPublishing by mutableStateOf(false)
96         private set
97     var publishError by mutableStateOf<String?>(null)
98         private set
99     var publishSuccess by mutableStateOf(false)
100         private set
101     var showPublishLockedDialog by mutableStateOf(false)
102         private set
103
104     var showArchiveDialog by mutableStateOf(false)
105         private set
106     var isArchiving by mutableStateOf(false)
107         private set
108     var archiveError by mutableStateOf<String?>(null)
109         private set
110     var archiveSuccess by mutableStateOf(false)
111         private set
112
113     var showRateDialog by mutableStateOf(false)
114         private set
115     var ratingValue by mutableStateOf(0)
116         private set
117     var ratingComment by mutableStateOf("")
118         private set
119     var isSubmittingRating by mutableStateOf(false)
120         private set
121     var ratingError by mutableStateOf<String?>(null)
122         private set
123     var ratingSuccess by mutableStateOf(false)
124         private set
125     val hasRated: Boolean get() = ratingPrefs.getBoolean("rated_${courseId}", false)
126
127     var showDeleteDialog by mutableStateOf(false)
128         private set
129     var isDeleting by mutableStateOf(false)
130         private set
131     var deleteSuccess by mutableStateOf(false)
132         private set
133
134     private val _navigateToAttempt = MutableSharedFlow<AttemptStartedEvent>()
135     val navigateToAttempt = _navigateToAttempt.asSharedFlow()
136
137     init {
138         load()
139     }
140
141     fun refresh() = load()
142
143     fun pullRefresh() {
144         viewModelScope.launch {
145             isRefreshing = true
146             load()
147             isRefreshing = false
148         }
149     }
150
151     fun clearStartAttemptError() {
152         startAttemptError = null
153     }
154
155     fun clearReportSuccess() {
156         reportSuccess = false
157     }
158
159     fun openReportDialog() {
160         reportReason = ""
161         reportDescription = ""
162         reportError = null
163         reportSuccess = false
164         showReportDialog = true
165     }
166
167     fun dismissReportDialog() {
168         if (!isSubmittingReport) showReportDialog = false
169     }

```

```

170
171 fun onReportReasonChange(value: String) {
172     reportReason = value
173     reportError = null
174 }
175
176 fun onReportDescriptionChange(value: String) {
177     reportDescription = value
178 }
179
180 fun submitReport() {
181     if (isSubmittingReport || reportReason.isEmpty()) return
182     isSubmittingReport = true
183     reportError = null
184
185     viewModelScope.launch {
186         when (val result = courseRepository.reportCourse(
187             courseId = courseId,
188             request = CourseReportRequest(
189                 reason = reportReason,
190                 description = reportDescription.trim().ifBlank { null },
191             ),
192         )) {
193             is NetworkResult.Success -> {
194                 reportSuccess = true
195                 showReportDialog = false
196             }
197             is NetworkResult.ApiError -> reportError = when (result.errorCode) {
198                 "ALREADY_REPORTED" -> "이미 신고한 코스입니다."
199                 else -> result.message
200             }
201             is NetworkResult.NetworkError -> reportError = "네트워크 연결을 확인해 주세요."
202         }
203         isSubmittingReport = false
204     }
205 }
206
207 fun openRateDialog() {
208     ratingValue = 0
209     ratingComment = ""
210     ratingError = null
211     showRateDialog = true
212 }
213
214 fun dismissRateDialog() {
215     if (!isSubmittingRating) showRateDialog = false
216 }
217
218 fun onRatingValueChange(value: Int) {
219     ratingValue = value
220     ratingError = null
221 }
222
223 fun onRatingCommentChange(value: String) {
224     ratingComment = value
225 }
226
227 fun clearRatingSuccess() {
228     ratingSuccess = false
229 }
230
231 fun submitRating() {
232     if (isSubmittingRating || ratingValue == 0) return
233     isSubmittingRating = true
234     ratingError = null
235
236     viewModelScope.launch {
237         when (val result = courseRepository.rateCourse(
238             courseId = courseId,
239             request = CourseRatingRequest(
240                 rating = ratingValue,
241                 comment = ratingComment.trim().ifBlank { null },
242             ),
243         )) {
244             is NetworkResult.Success -> {
245                 ratingSuccess = true
246                 showRateDialog = false
247                 ratingPrefs.edit().putBoolean("rated_${courseId}", true).apply()
248                 // 평균 평점 반영을 위해 상세 정보 새로고침
249                 load()
250             }
251             is NetworkResult.ApiError -> ratingError = result.message
252             is NetworkResult.NetworkError -> ratingError = "네트워크 연결을 확인해 주세요."
253         }
254         isSubmittingRating = false
255     }

```

```

256     }
257
258     fun openDeleteDialog() { showDeleteDialog = true }
259     fun dismissDeleteDialog() { showDeleteDialog = false }
260
261     fun deleteDraftCourse() {
262         if (isDeleting) return
263         isDeleting = true
264         showDeleteDialog = false
265         viewModelScope.launch {
266             when (courseRepository.deleteDraftCourse(courseId)) {
267                 is NetworkResult.Success -> deleteSuccess = true
268                 is NetworkResult.ApiError, is NetworkResult.NetworkError -> isDeleting = false
269             }
270         }
271     }
272
273     fun openPublishDialog() {
274         val completions = courseDetail?.completionCount ?: 0
275         if (completions < PUBLISH_MIN_COMPLETIONS) {
276             showPublishLockedDialog = true
277         } else {
278             publishError = null
279             publishSuccess = false
280             showPublishDialog = true
281         }
282     }
283
284     fun dismissPublishLockedDialog() {
285         showPublishLockedDialog = false
286     }
287
288     fun dismissPublishDialog() {
289         if (!isPublishing) showPublishDialog = false
290     }
291
292     fun clearPublishSuccess() {
293         publishSuccess = false
294     }
295
296     fun submitPublish(request: PublishCourseRequest) {
297         if (isPublishing) return
298         isPublishing = true
299         publishError = null
300
301         viewModelScope.launch {
302             when (val result = courseRepository.publishCourse(courseId, request)) {
303                 is NetworkResult.Success -> {
304                     publishSuccess = true
305                     showPublishDialog = false
306                     load()
307                 }
308                 is NetworkResult.ApiError -> publishError = when (result.errorCode) {
309                     "COURSE_PUBLISH_METADATA_REQUIRED" -> "공개에 필요한 정보를 모두 입력해주세요."
310                     "COURSE_PUBLISH_NOT_ENOUGH_COMPLETIONS" -> "공개하려면 이 코스를 10회 이상 완주해야 합니다."
311                     "INVALID_COURSE_STATUS" -> "이미 공개된 코스입니다."
312                     else -> result.message
313                 }
314                 is NetworkResult.NetworkError -> publishError = "네트워크 연결을 확인해 주세요."
315             }
316             isPublishing = false
317         }
318     }
319
320     fun openArchiveDialog() {
321         archiveError = null
322         archiveSuccess = false
323         showArchiveDialog = true
324     }
325
326     fun dismissArchiveDialog() {
327         if (!isArchiving) showArchiveDialog = false
328     }
329
330     fun clearArchiveSuccess() {
331         archiveSuccess = false
332     }
333
334     fun submitArchive() {
335         if (isArchiving) return
336         isArchiving = true
337         archiveError = null
338
339         viewModelScope.launch {
340             when (val result = courseRepository.archiveCourse(courseId)) {
341                 is NetworkResult.Success -> {

```

```

342         archiveSuccess = true
343         showArchiveDialog = false
344         load()
345     }
346     is NetworkResult.ApiError -> archiveError = when (result.errorCode) {
347         "INVALID_COURSE_STATUS" -> "이미 보관된 코스입니다."
348         else -> result.message
349     }
350     is NetworkResult.NetworkError -> archiveError = "네트워크 연결을 확인해 주세요."
351 }
352 isArchiving = false
353 }
354 }
355
356 fun clearFavoriteError() {
357     favoriteError = null
358 }
359
360 fun toggleFavorite() {
361     if (isFavoriteToggling) return
362     isFavoriteToggling = true
363     favoriteError = null
364
365     val previousState = isFavorited
366     isFavorited = !previousState // 낙관적 업데이트
367
368     viewModelScope.launch {
369         val result = if (!previousState) {
370             courseRepository.addFavorite(courseId)
371         } else {
372             courseRepository.removeFavorite(courseId)
373         }
374         when (result) {
375             is NetworkResult.Success -> { /* 성공 - 낙관적 상태 유지 */ }
376             else -> {
377                 isFavorited = previousState // 롤백
378                 favoriteError = if (!previousState) "즐거찾기 추가에 실패했습니다." else "즐거찾기 해제에 실패했습니다."
379             }
380         }
381         isFavoriteToggling = false
382     }
383 }
384
385 fun startAttempt() {
386     if (isStartingAttempt) return
387     isStartingAttempt = true
388     startAttemptError = null
389
390     viewModelScope.launch {
391         when (val result = courseAttemptRepository.startAttempt(
392             courseId = courseId,
393             request = StartAttemptRequest(startedAt = Instant.now().toString()),
394         )) {
395             is NetworkResult.Success -> {
396                 _navigateToAttempt.emit(
397                     AttemptStartedEvent(
398                         courseId = courseId,
399                         courseAttemptId = result.data.courseAttemptId,
400                         runningRecordId = result.data.runningRecordId,
401                     )
402                 )
403             }
404             is NetworkResult.ApiError -> startAttemptError = result.message
405             is NetworkResult.NetworkError -> startAttemptError = "네트워크 연결을 확인해 주세요."
406         }
407         isStartingAttempt = false
408     }
409 }
410
411 private fun load() {
412     viewModelScope.launch {
413         isLoading = true
414         errorMessage = null
415
416         val detailDeferred = async { courseRepository.getCourseDetail(courseId) }
417         val pointsDeferred = async { courseRepository.getCoursePoints(courseId) }
418         val leaderboardDeferred = async {
419             isLoadingLeaderboard = true
420             courseAttemptRepository.getLeaderboard(courseId, page = 0, size = 5)
421         }
422         val myBestDeferred = async { courseAttemptRepository.getMyBestAttempt(courseId) }
423
424         when (val result = detailDeferred.await()) {
425             is NetworkResult.Success -> {
426                 courseDetail = result.data
427                 isFavorited = result.data.isFavorited

```

```

428     }
429     is NetworkResult.ApiError -> errorMessage = result.message
430     is NetworkResult.NetworkError -> errorMessage = "네트워크 연결을 확인해 주세요."
431 }
432
433 when (val result = pointsDeferred.await()) {
434     is NetworkResult.Success -> coursePoints = result.data.points
435     else -> { /* points 로드 실패는 detail 표시에 영향 없음 */ }
436 }
437
438 when (val result = leaderboardDeferred.await()) {
439     is NetworkResult.Success -> previewLeaderboard = result.data.items.take(5)
440     else -> { /* 리더보드 로드 실패 시 조용히 빈 상태 유지 */ }
441 }
442 isLoadingLeaderboard = false
443
444 when (val result = myBestDeferred.await()) {
445     is NetworkResult.Success -> myBestAttempt = result.data.takeIf { it.completionCount > 0 }
446     else -> { /* 내 기록 없으면 null 유지 */ }
447 }
448
449 isLoading = false
450 }
451 }
452 }

```



```

1  package com.runway.android.ui.course.detail
2
3  import android.content.Context
4  import android.graphics.Bitmap
5  import android.graphics.Paint
6  import android.graphics.Typeface
7  import android.location.Location
8  import androidx.compose.foundation.BorderStroke
9  import androidx.compose.foundation.background
10 import com.runway.android.ui.theme.LocalIsDarkTheme
11 import androidx.compose.foundation.layout.Box
12 import androidx.compose.foundation.layout.fillMaxSize
13 import androidx.compose.foundation.layout.padding
14 import androidx.compose.foundation.layout.size
15 import androidx.compose.foundation.layout.statusBarsPadding
16 import androidx.compose.foundation.shape.CircleShape
17 import androidx.compose.material.icons.Icons
18 import androidx.compose.material.icons.automirrored.filled.ArrowBack
19 import androidx.compose.material3.CircularProgressIndicator
20 import androidx.compose.material3.Icon
21 import androidx.compose.material3.MaterialTheme
22 import androidx.compose.material3.Surface
23 import androidx.compose.material3.Text
24 import androidx.compose.runtime.Composable
25 import androidx.compose.runtime.LaunchedEffect
26 import androidx.compose.runtime.getValue
27 import androidx.compose.runtime.mutableStateOf
28 import androidx.compose.runtime.remember
29 import androidx.compose.runtime.setValue
30 import kotlinx.coroutines.delay
31 import androidx.compose.ui.Alignment
32 import androidx.compose.ui.Modifier
33 import androidx.compose.ui.geometry.Offset
34 import androidx.compose.ui.graphics.toArgb
35 import androidx.compose.ui.platform.LocalContext
36 import androidx.compose.ui.text.font.FontWeight
37 import androidx.compose.ui.text.style.TextAlign
38 import androidx.compose.ui.unit.dp
39 import androidx.hilt.navigation.compose.hiltViewModel
40 import com.google.android.gms.maps.CameraUpdateFactory
41 import com.google.android.gms.maps.model.BitmapDescriptor
42 import com.google.android.gms.maps.model.BitmapDescriptorFactory
43 import com.google.android.gms.maps.model.JointType
44 import com.google.android.gms.maps.model.LatLng
45 import com.google.android.gms.maps.model.MapStyleOptions
46 import com.google.android.gms.maps.model.RoundCap
47 import com.google.maps.android.compose.GoogleMap
48 import com.google.maps.android.compose.MapProperties
49 import com.google.maps.android.compose.MapUiSettings
50 import com.google.maps.android.compose.Marker
51 import com.google.maps.android.compose.MarkerState
52 import com.google.maps.android.compose.Polyline
53 import com.google.maps.android.compose.rememberCameraPositionState
54 import com.runway.android.R
55 import com.runway.android.core.map.MapPoint
56 import com.runway.android.core.map.toLatLngBounds
57
58 @Composable
59 fun CourseMapDetailScreen(
60     onBack: () -> Unit,
61     viewModel: CourseMapDetailViewModel = hiltViewModel(),
62 ) {
63     val context = LocalContext.current
64
65     Box(modifier = Modifier.fillMaxSize()) {
66         when {
67             viewModel.isLoading -> {
68                 Box(
69                     modifier = Modifier
70                         .fillMaxSize()
71                         .background(MaterialTheme.colorScheme.background),
72                     contentAlignment = Alignment.Center,
73                 ) {
74                     CircularProgressIndicator(color = MaterialTheme.colorScheme.primary)
75                 }
76             }
77             viewModel.points.size < 2 -> {
78                 Box(
79                     modifier = Modifier
80                         .fillMaxSize()
81                         .background(MaterialTheme.colorScheme.background),
82                     contentAlignment = Alignment.Center,
83                 ) {

```

```

84         Text(
85             text = "경로 데이터가 없습니다.",
86             style = MaterialTheme.typography.bodyMedium,
87             color = MaterialTheme.colorScheme.onSurfaceVariant,
88             textAlign = TextAlign.Center,
89         )
90     }
91 }
92 else -> {
93     CourseFullMap(
94         points = viewModel.points,
95         isLoop = viewModel.isLoop,
96         context = context,
97     )
98 }
99 }
100
101 // Floating back button
102 Surface(
103     onClick = onBack,
104     modifier = Modifier
105         .align(Alignment.TopStart)
106         .statusBarsPadding()
107         .padding(12.dp),
108     shape = CircleShape,
109     color = MaterialTheme.colorScheme.surface.copy(alpha = 0.92f),
110     border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
111     shadowElevation = 4.dp,
112 ) {
113     Icon(
114         imageVector = Icons.AutoMirrored.Filled.ArrowBack,
115         contentDescription = "뒤로",
116         tint = MaterialTheme.colorScheme.onSurface,
117         modifier = Modifier
118             .padding(10.dp)
119             .size(22.dp),
120     )
121 }
122
123 // Course name pill
124 if (viewModel.courseName.isNotBlank()) {
125     Surface(
126         modifier = Modifier
127             .align(Alignment.TopCenter)
128             .statusBarsPadding()
129             .padding(top = 12.dp),
130         shape = MaterialTheme.shapes.extraLarge,
131         color = MaterialTheme.colorScheme.surface.copy(alpha = 0.92f),
132         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
133         shadowElevation = 4.dp,
134     ) {
135         Text(
136             text = viewModel.courseName,
137             style = MaterialTheme.typography.labelLarge,
138             fontWeight = FontWeight.SemiBold,
139             color = MaterialTheme.colorScheme.onSurface,
140             modifier = Modifier.padding(horizontal = 16.dp, vertical = 8.dp),
141         )
142     }
143 }
144 }
145 }
146
147 @Composable
148 private fun CourseFullMap(
149     points: List<MapPoint>,
150     isLoop: Boolean,
151     context: Context,
152 ) {
153     val latLngs = remember(points) { points.map { LatLng(it.latitude, it.longitude) } }
154     val bounds = remember(points) { points.toLatLngBounds() }
155     val cameraPositionState = rememberCameraPositionState()
156     val primaryColor = MaterialTheme.colorScheme.primary
157     val primaryArgb = primaryColor.toArgb()
158     val isDark = LocalIsDarkTheme.current
159     val mapStyleOptions = remember(isDark) {
160         val styleRes = if (isDark) R.raw.map_style_dark else R.raw.map_style_light
161         runCatching { MapStyleOptions.loadRawResourceStyle(context, styleRes) }.getOrNull()
162     }
163
164     val kmMarkers = remember(points) { calculateKmMarkers(points) }
165     val kmIcons = remember(primaryArgb, kmMarkers) {
166         kmMarkers.associate { (km, _) -> km to createKmMarkerIcon(context, km, primaryArgb) }
167     }
168     val startIcon = remember(context) {
169         BitmapDescriptorFactory.fromBitmap(createPillMarker(context, "Start", "#22C55E"))

```

```

170     }
171     val finishIcon = remember(context) {
172         BitmapDescriptorFactory.fromBitmap(createPillMarker(context, "Finish", "#EF4444"))
173     }
174
175     var mapLoaded by remember { mutableStateOf(false) }
176     LaunchedEffect(mapLoaded) {
177         if (!mapLoaded) return@LaunchedEffect
178         delay(50)
179         runCatching {
180             cameraPositionState.animate(CameraUpdateFactory.newLatLngBounds(bounds, 80), 300)
181         }
182     }
183
184     GoogleMap(
185         modifier = Modifier.fillMaxSize(),
186         cameraPositionState = cameraPositionState,
187         properties = MapProperties(isMyLocationEnabled = false, mapStyleOptions = mapStyleOptions),
188         uiSettings = MapUiSettings(
189             zoomControlsEnabled = true,
190             scrollGesturesEnabled = true,
191             zoomGesturesEnabled = true,
192             rotationGesturesEnabled = true,
193             tiltGesturesEnabled = false,
194             compassEnabled = true,
195             mapToolbarEnabled = false,
196         ),
197         onMapLoaded = { mapLoaded = true },
198     ) {
199         // Route polyline
200         Polyline(
201             points = latLngs,
202             color = primaryColor,
203             width = 14f,
204             startCap = RoundCap(),
205             endCap = RoundCap(),
206             jointType = JointType.ROUND,
207         )
208
209         // Start marker
210         Marker(
211             state = MarkerState(position = latLngs.first()),
212             title = "Start",
213             icon = startIcon,
214             anchor = Offset(0.5f, 0.5f),
215             zIndex = 3f,
216         )
217
218         // Finish marker - last point (loop 코스는 start 위치와 거의 같음)
219         Marker(
220             state = MarkerState(position = latLngs.last()),
221             title = "Finish",
222             icon = finishIcon,
223             anchor = Offset(0.5f, 0.5f),
224             zIndex = 3f,
225         )
226
227         // Km markers
228         kmMarkers.forEach { (km, point) ->
229             val icon = kmIcons[km] ?: return@forEach
230             Marker(
231                 state = MarkerState(position = LatLng(point.latitude, point.longitude)),
232                 title = "${km}km",
233                 icon = icon,
234                 zIndex = 1f,
235             )
236         }
237     }
238 }
239
240 // --- Utilities ---
241
242 /** Every 1km along the polyline, returns (kmNumber, interpolated MapPoint). */
243 private fun calculateKmMarkers(points: List<MapPoint>): List<Pair<Int, MapPoint>> {
244     if (points.size < 2) return emptyList()
245     val distResult = FloatArray(1)
246     var totalMeters = 0.0
247     var nextKm = 1
248     val markers = mutableListOf<Pair<Int, MapPoint>>()
249
250     for (i in 1 until points.size) {
251         val prev = points[i - 1]
252         val curr = points[i]
253         Location.distanceBetween(prev.latitude, prev.longitude, curr.latitude, curr.longitude, distResult)
254         val segLen = distResult[0].toDouble()
255         val segStart = totalMeters

```

```

256         totalMeters += segLen
257
258         while (nextKm * 1000.0 <= totalMeters) {
259             val fraction = if (segLen > 0.0) (nextKm * 1000.0 - segStart) / segLen else 0.0
260             val lat = prev.latitude + fraction * (curr.latitude - prev.latitude)
261             val lon = prev.longitude + fraction * (curr.longitude - prev.longitude)
262             markers.add(Pair(nextKm, MapPoint(lat, lon)))
263             nextKm++
264         }
265     }
266     return markers
267 }
268
269 /** Circle bitmap with km number - e.g. "1K", "2K". */
270 private fun createKmMarkerIcon(context: Context, km: Int, primaryArgb: Int): BitmapDescriptor {
271     val density = context.resources.displayMetrics.density
272     val size = (30 * density).toInt().coerceAtLeast(30)
273     val bmp = Bitmap.createBitmap(size, size, Bitmap.Config.ARGB_8888)
274     val canvas = android.graphics.Canvas(bmp)
275
276     val paint = Paint(Paint.ANTI_ALIAS_FLAG)
277
278     // White border ring
279     paint.color = android.graphics.Color.WHITE
280     canvas.drawCircle(size / 2f, size / 2f, size / 2f, paint)
281
282     // Filled circle
283     paint.color = primaryArgb
284     canvas.drawCircle(size / 2f, size / 2f, size / 2f - (1.5f * density), paint)
285
286     // Text
287     paint.color = android.graphics.Color.WHITE
288     paint.textSize = size * 0.40f
289     paint.textAlign = Paint.Align.CENTER
290     paint.typeface = Typeface.create(Typeface.DEFAULT, Typeface.BOLD)
291     val metrics = paint.fontMetrics
292     val textY = size / 2f - (metrics.ascent + metrics.descent) / 2f
293     canvas.drawText("${km}K", size / 2f, textY, paint)
294
295     return BitmapDescriptorFactory.fromBitmap(bmp)
296 }
297
298 /** Rounded-pill bitmap matching RouteMapView's Start/Finish style. */
299 private fun createPillMarker(context: Context, label: String, colorHex: String): Bitmap {
300     val density = context.resources.displayMetrics.density
301     val textPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
302         textSize = 11f * density
303         isFakeBoldText = true
304         textAlign = Paint.Align.CENTER
305     }
306     val textW = textPaint.measureText(label)
307     val padH = 10f * density
308     val padV = 6f * density
309     val w = (textW + padH * 2).toInt().coerceAtLeast(1)
310     val h = (textPaint.textSize + padV * 2).toInt().coerceAtLeast(1)
311     val r = h / 2f
312
313     val bmp = Bitmap.createBitmap(w, h, Bitmap.Config.ARGB_8888)
314     val canvas = android.graphics.Canvas(bmp)
315     val bg = Paint(Paint.ANTI_ALIAS_FLAG)
316
317     bg.color = android.graphics.Color.parseColor(colorHex)
318     bg.style = Paint.Style.FILL
319     canvas.drawRoundRect(0f, 0f, w.toFloat(), h.toFloat(), r, r, bg)
320
321     bg.color = android.graphics.Color.WHITE
322     bg.style = Paint.Style.STROKE
323     bg.strokeWidth = 1.5f * density
324     val sw = bg.strokeWidth / 2
325     canvas.drawRoundRect(sw, sw, w - sw, h - sw, r, r, bg)
326
327     textPaint.color = android.graphics.Color.WHITE
328     val textY = h / 2f - (textPaint.descent() + textPaint.ascent()) / 2f
329     canvas.drawText(label, w / 2f, textY, textPaint)
330
331     return bmp
332 }

```

```
1 package com.runway.android.ui.course.detail
2
3 import androidx.compose.runtime.getValue
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.compose.runtime.setValue
6 import androidx.lifecycle.SavedStateHandle
7 import androidx.lifecycle.ViewModel
8 import androidx.lifecycle.viewModelScope
9 import com.runway.android.core.map.MapPoint
10 import com.runway.android.core.result.NetworkResult
11 import com.runway.android.domain.course.CourseRepository
12 import dagger.hilt.android.lifecycle.HiltViewModel
13 import kotlinx.coroutines.async
14 import kotlinx.coroutines.launch
15 import javax.inject.Inject
16
17 @HiltViewModel
18 class CourseMapDetailViewModel @Inject constructor(
19     savedStateHandle: SavedStateHandle,
20     private val courseRepository: CourseRepository,
21 ) : ViewModel() {
22
23     val courseId: String = checkNotNull(savedStateHandle["courseId"])
24
25     var points by mutableStateOf<List<MapPoint>>(emptyList())
26     private set
27     var isLoop by mutableStateOf(false)
28     private set
29     var courseName by mutableStateOf("")
30     private set
31     var isLoading by mutableStateOf(true)
32     private set
33
34     init {
35         viewModelScope.launch {
36             val pointsDeferred = async { courseRepository.getCoursePoints(courseId) }
37             val detailDeferred = async { courseRepository.getCourseDetail(courseId) }
38
39             when (val r = pointsDeferred.await()) {
40                 is NetworkResult.Success -> points = r.data.points.map { MapPoint(it.latitude, it.longitude) }
41                 else -> {}
42             }
43             when (val r = detailDeferred.await()) {
44                 is NetworkResult.Success -> {
45                     isLoop = r.data.isLoop
46                     courseName = r.data.name
47                 }
48                 else -> {}
49             }
50             isLoading = false
51         }
52     }
53 }
```

```

1  package com.runway.android.ui.course.detail
2
3  import androidx.compose.foundation.layout.Arrangement
4  import androidx.compose.foundation.layout.Column
5  import androidx.compose.foundation.layout.Row
6  import androidx.compose.foundation.layout.Spacer
7  import androidx.compose.foundation.layout.fillMaxWidth
8  import androidx.compose.foundation.layout.height
9  import androidx.compose.foundation.layout.heightIn
10 import androidx.compose.foundation.layout.padding
11 import androidx.compose.foundation.rememberScrollState
12 import androidx.compose.foundation.verticalScroll
13 import androidx.compose.material3.AlertDialog
14 import androidx.compose.material3.ButtonDefaults
15 import androidx.compose.material3.CircularProgressIndicator
16 import androidx.compose.material3.FilterChip
17 import androidx.compose.material3.MaterialTheme
18 import androidx.compose.material3.OutlinedTextField
19 import androidx.compose.material3.Text
20 import androidx.compose.material3.TextButton
21 import androidx.compose.runtime.Composable
22 import androidx.compose.runtime.getValue
23 import androidx.compose.runtime.mutableStateOf
24 import androidx.compose.runtime.remember
25 import androidx.compose.runtime.setValue
26 import androidx.compose.ui.Modifier
27 import androidx.compose.ui.text.font.FontWeight
28 import androidx.compose.ui.unit.dp
29 import com.runway.android.data.course.model.PublishCourseRequest
30
31 private val DIFFICULTY_OPTIONS = listOf("EASY" to "쉬움", "NORMAL" to "보통", "HARD" to "어려움")
32 private val SLOPE_OPTIONS = listOf("FLAT" to "평탄", "MODERATE" to "완만", "STEEP" to "가파름")
33 private val RISK_OPTIONS = listOf("LOW" to "낮음", "MEDIUM" to "보통", "HIGH" to "높음")
34 private val SURFACE_OPTIONS = listOf("ROAD" to "도로", "PARK" to "공원", "TRAIL" to "트레일", "MIXED" to "혼합")
35 private val TIME_OPTIONS = listOf("MORNING" to "아침", "DAY" to "낮", "NIGHT" to "저녁", "ANY" to "무관")
36
37 @Composable
38 fun PublishCourseDialog(
39     isPublishing: Boolean,
40     error: String?,
41     onDismiss: () -> Unit,
42     onPublish: (PublishCourseRequest) -> Unit,
43 ) {
44     var difficulty by remember { mutableStateOf("") }
45     var slopeLevel by remember { mutableStateOf("") }
46     var riskLevel by remember { mutableStateOf("") }
47     var surfaceType by remember { mutableStateOf("") }
48     var recommendedTime by remember { mutableStateOf("") }
49     var warnings by remember { mutableStateOf("") }
50     var description by remember { mutableStateOf("") }
51
52     val allSelected = difficulty.isNotEmpty() && slopeLevel.isNotEmpty() &&
53         riskLevel.isNotEmpty() && surfaceType.isNotEmpty() && recommendedTime.isNotEmpty()
54
55     AlertDialog(
56         onDismissRequest = { if (!isPublishing) onDismiss() },
57         title = { Text("코스 공개", fontWeight = FontWeight.Bold) },
58         text = {
59             Column(
60                 modifier = Modifier
61                     .heightIn(max = 480.dp)
62                     .verticalScroll(rememberScrollState()),
63                 verticalArrangement = Arrangement.spacedBy(12.dp),
64             ) {
65                 Text(
66                     text = "공개 코스는 다른 러너가 따라 달릴 수 있으므로 난이도, 위험도, 주의사항을 자세히 작성해주세요.",
67                     style = MaterialTheme.typography.bodySmall,
68                     color = MaterialTheme.colorScheme.onSurfaceVariant,
69                 )
70
71                 ChipGroup("난이도 *", DIFFICULTY_OPTIONS, difficulty) { difficulty = it }
72                 ChipGroup("경사도 *", SLOPE_OPTIONS, slopeLevel) { slopeLevel = it }
73                 ChipGroup("위험도 *", RISK_OPTIONS, riskLevel) { riskLevel = it }
74                 ChipGroup("노면 유형 *", SURFACE_OPTIONS, surfaceType) { surfaceType = it }
75                 ChipGroup("추천 시간대 *", TIME_OPTIONS, recommendedTime) { recommendedTime = it }
76
77                 OutlinedTextField(
78                     value = warnings,
79                     onValueChange = { warnings = it },
80                     label = { Text("주의사항") },
81                     placeholder = { Text("야간에는 조명이 부족합니다.") },
82                     modifier = Modifier.fillMaxWidth(),
83                     minLines = 2,

```

```

84         maxLines = 4,
85     )
86     OutlinedTextField(
87         value = description,
88         onValueChange = { description = it },
89         label = { Text("코스 설명") },
90         placeholder = { Text("강변을 따라 달리는 평탄한 코스입니다.") },
91         modifier = Modifier.fillMaxWidth(),
92         minLines = 2,
93         maxLines = 4,
94     )
95
96     if (error != null) {
97         Text(
98             text = error,
99             style = MaterialTheme.typography.bodySmall,
100            color = MaterialTheme.colorScheme.error,
101        )
102    }
103
104    Spacer(Modifier.height(4.dp))
105 }
106 },
107 confirmButton = {
108     TextButton(
109         onClick = {
110             if (allSelected && !isPublishing) {
111                 onPublish(
112                     PublishCourseRequest(
113                         difficulty = difficulty,
114                         slopeLevel = slopeLevel,
115                         riskLevel = riskLevel,
116                         surfaceType = surfaceType,
117                         recommendedTime = recommendedTime,
118                         warnings = warnings.trim().ifBlank { null },
119                         description = description.trim().ifBlank { null },
120                     )
121                 )
122             }
123         },
124         enabled = allSelected && !isPublishing,
125         colors = ButtonDefaults.textButtonColors(contentColor = MaterialTheme.colorScheme.primary),
126     ) {
127         if (isPublishing) {
128             CircularProgressIndicator(
129                 modifier = Modifier.padding(horizontal = 8.dp),
130                 strokeWidth = 2.dp,
131             )
132         } else {
133             Text("공개하기")
134         }
135     }
136 },
137 dismissButton = {
138     TextButton(onClick = { if (!isPublishing) onDismiss() }) {
139         Text("취소")
140     }
141 },
142 )
143 }
144
145 @Composable
146 private fun ChipGroup(
147     label: String,
148     options: List<Pair<String, String>>,
149     selected: String,
150     onSelect: (String) -> Unit,
151 ) {
152     Column {
153         Text(
154             text = label,
155             style = MaterialTheme.typography.labelMedium,
156             color = MaterialTheme.colorScheme.onSurface,
157             modifier = Modifier.padding(bottom = 4.dp),
158         )
159         Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
160             options.forEach { (value, display) ->
161                 FilterChip(
162                     selected = selected == value,
163                     onClick = { onSelect(value) },
164                     label = { Text(display, style = MaterialTheme.typography.labelSmall) },
165                 )
166             }
167         }
168     }
169 }

```

```

1  package com.runway.android.ui.course.detail
2
3  import androidx.compose.foundation.layout.Arrangement
4  import androidx.compose.foundation.layout.Column
5  import androidx.compose.foundation.layout.Row
6  import androidx.compose.foundation.layout.Spacer
7  import androidx.compose.foundation.layout.fillMaxWidth
8  import androidx.compose.foundation.layout.height
9  import androidx.compose.foundation.layout.size
10 import androidx.compose.material.icons.Icons
11 import androidx.compose.material.icons.filled.Star
12 import androidx.compose.material.icons.filled.StarOutline
13 import androidx.compose.material3.AlertDialog
14 import androidx.compose.material3.CircularProgressIndicator
15 import androidx.compose.material3.Icon
16 import androidx.compose.material3.IconButton
17 import androidx.compose.material3.MaterialTheme
18 import androidx.compose.material3.OutlinedTextField
19 import androidx.compose.material3.Text
20 import androidx.compose.material3.TextButton
21 import androidx.compose.runtime.Composable
22 import androidx.compose.ui.Alignment
23 import androidx.compose.ui.Modifier
24 import androidx.compose.ui.unit.dp
25
26 @Composable
27 fun RateCourseDialog(
28     selectedRating: Int,
29     onRatingChange: (Int) -> Unit,
30     comment: String,
31     onCommentChange: (String) -> Unit,
32     isSubmitting: Boolean,
33     errorMessage: String?,
34     onConfirm: () -> Unit,
35     onDismiss: () -> Unit,
36 ) {
37     AlertDialog(
38         onDismissRequest = { if (!isSubmitting) onDismiss() },
39         title = { Text("코스 평가") },
40         text = {
41             Column {
42                 Text(
43                     text = "이 코스는 어떠셨나요?",
44                     style = MaterialTheme.typography.bodyMedium,
45                     color = MaterialTheme.colorScheme.onSurfaceVariant,
46                 )
47                 Spacer(modifier = Modifier.height(12.dp))
48
49                 // Star rating
50                 Row(
51                     modifier = Modifier.fillMaxWidth(),
52                     horizontalArrangement = Arrangement.Center,
53                     verticalAlignment = Alignment.CenterVertically,
54                 ) {
55                     (1..5).forEach { star ->
56                         IconButton(onClick = { onRatingChange(star) }) {
57                             Icon(
58                                 imageVector = if (star <= selectedRating) Icons.Filled.Star
59                                     else Icons.Filled.StarOutline,
60                                 contentDescription = "$star 점",
61                                 tint = if (star <= selectedRating) MaterialTheme.colorScheme.primary
62                                     else MaterialTheme.colorScheme.onSurfaceVariant,
63                                 modifier = Modifier.size(32.dp),
64                             )
65                         }
66                     }
67                 }
68
69                 Spacer(modifier = Modifier.height(12.dp))
70
71                 OutlinedTextField(
72                     value = comment,
73                     onValueChange = onCommentChange,
74                     modifier = Modifier.fillMaxWidth(),
75                     label = { Text("한 줄 코멘트 (선택)") },
76                     singleLine = true,
77                     enabled = !isSubmitting,
78                 )
79
80                 if (errorMessage != null) {
81                     Spacer(modifier = Modifier.height(8.dp))
82                     Text(
83                         text = errorMessage,

```



```
84         style = MaterialTheme.typography.bodySmall,
85         color = MaterialTheme.colorScheme.error,
86     )
87     }
88 }
89 },
90 confirmButton = {
91     if (isSubmitting) {
92         CircularProgressIndicator(modifier = Modifier.size(24.dp))
93     } else {
94         TextButton(
95             onClick = onConfirm,
96             enabled = selectedRating > 0,
97         ) {
98             Text("제출")
99         }
100     }
101 },
102 dismissButton = {
103     TextButton(onClick = { if (!isSubmitting) onDismiss() }) {
104         Text("취소")
105     }
106 },
107 )
108 }
```

```

1  package com.runway.android.ui.course.detail
2
3  import androidx.compose.foundation.layout.Column
4  import androidx.compose.foundation.layout.Row
5  import androidx.compose.foundation.layout.Spacer
6  import androidx.compose.foundation.layout.fillMaxWidth
7  import androidx.compose.foundation.layout.height
8  import androidx.compose.foundation.layout.padding
9  import androidx.compose.foundation.layout.size
10 import androidx.compose.foundation.selection.selectable
11 import androidx.compose.foundation.selection.selectableGroup
12 import androidx.compose.material3.AlertDialog
13 import androidx.compose.material3.CircularProgressIndicator
14 import androidx.compose.material3.MaterialTheme
15 import androidx.compose.material3.OutlinedTextField
16 import androidx.compose.material3.RadioButton
17 import androidx.compose.material3.Text
18 import androidx.compose.material3.TextButton
19 import androidx.compose.runtime.Composable
20 import androidx.compose.ui.Alignment
21 import androidx.compose.ui.Modifier
22 import androidx.compose.ui.semantics.Role
23 import androidx.compose.ui.unit.dp
24
25 private val REASONS = listOf(
26     "inappropriate_content" to "부적절한 콘텐츠",
27     "wrong_location" to "잘못된 위치 정보",
28     "spam" to "스팸 또는 광고",
29     "other" to "기타",
30 )
31
32 @Composable
33 fun ReportCourseDialog(
34     selectedReason: String,
35     onReasonChange: (String) -> Unit,
36     description: String,
37     onDescriptionChange: (String) -> Unit,
38     isSubmitting: Boolean,
39     errorMessage: String?,
40     onConfirm: () -> Unit,
41     onDismiss: () -> Unit,
42 ) {
43     AlertDialog(
44         onDismissRequest = { if (!isSubmitting) onDismiss() },
45         title = { Text("코스 신고") },
46         text = {
47             Column {
48                 Text(
49                     text = "신고 사유를 선택해 주세요.",
50                     style = MaterialTheme.typography.bodyMedium,
51                     color = MaterialTheme.colorScheme.onSurfaceVariant,
52                 )
53                 Spacer(modifier = Modifier.height(8.dp))
54                 Column(modifier = Modifier.selectableGroup()) {
55                     REASONS.forEach { (value, label) ->
56                         Row(
57                             modifier = Modifier
58                                 .fillMaxWidth()
59                                 .selectable(
60                                     selected = selectedReason == value,
61                                     onClick = { onReasonChange(value) },
62                                     role = Role.RadioButton,
63                                 )
64                             .padding(vertical = 4.dp),
65                             verticalAlignment = Alignment.CenterVertically,
66                         ) {
67                             RadioButton(
68                                 selected = selectedReason == value,
69                                 onClick = null,
70                             )
71                             Text(
72                                 text = label,
73                                 style = MaterialTheme.typography.bodyMedium,
74                                 color = MaterialTheme.colorScheme.onSurface,
75                                 modifier = Modifier.padding(start = 8.dp),
76                             )
77                         }
78                     }
79                 }
80                 Spacer(modifier = Modifier.height(8.dp))
81                 OutlinedTextField(
82                     value = description,
83                     onChange = onDescriptionChange,

```

```

84         label = { Text("추가 설명 (선택)") },
85         modifier = Modifier.fillMaxWidth(),
86         maxLines = 3,
87         enabled = !isSubmitting,
88     )
89     if (errorMessage != null) {
90         Spacer(modifier = Modifier.height(6.dp))
91         Text(
92             text = errorMessage,
93             style = MaterialTheme.typography.bodySmall,
94             color = MaterialTheme.colorScheme.error,
95         )
96     }
97 }
98 },
99 confirmButton = {
100     if (isSubmitting) {
101         CircularProgressIndicator(modifier = Modifier.size(24.dp))
102     } else {
103         TextButton(
104             onClick = onConfirm,
105             enabled = selectedReason.isNotEmpty(),
106         ) {
107             Text("신고하기", color = MaterialTheme.colorScheme.error)
108         }
109     }
110 },
111 dismissButton = {
112     TextButton(onClick = onDismiss, enabled = !isSubmitting) {
113         Text("취소")
114     }
115 },
116 )
117 }

```

```

1 package com.runway.android.ui.course.library
2
3 import androidx.compose.foundation.background
4 import androidx.compose.foundation.layout.Arrangement
5 import androidx.compose.foundation.layout.Box
6 import androidx.compose.foundation.layout.Column
7 import androidx.compose.foundation.layout.Row
8 import androidx.compose.foundation.layout.Spacer
9 import androidx.compose.foundation.layout.fillMaxSize
10 import androidx.compose.foundation.layout.fillMaxWidth
11 import androidx.compose.foundation.layout.height
12 import androidx.compose.foundation.layout.padding
13 import androidx.compose.foundation.layout.size
14 import androidx.compose.foundation.layout.width
15 import androidx.compose.foundation.lazy.LazyColumn
16 import androidx.compose.foundation.lazy.items
17 import androidx.compose.foundation.pager.HorizontalPager
18 import androidx.compose.foundation.pager.rememberPagerState
19 import androidx.compose.material.icons.Icons
20 import androidx.compose.material.icons.automirrored.filled.DirectionsRun
21 import androidx.compose.material.icons.filled.BookmarkBorder
22 import androidx.compose.material.icons.filled.Loop
23 import androidx.compose.material.icons.filled.Timer
24 import androidx.compose.material.icons.filled.TrendingUp
25 import androidx.compose.material3.CircularProgressIndicator
26 import androidx.compose.material3.ExperimentalMaterial3Api
27 import androidx.compose.material3.Icon
28 import androidx.compose.material3.MaterialTheme
29 import androidx.compose.material3.Surface
30 import androidx.compose.material3.Tab
31 import androidx.compose.material3.TabRow
32 import androidx.compose.material3.Text
33 import androidx.compose.material3.pulltorefresh.PullToRefreshBox
34 import androidx.compose.runtime.Composable
35 import androidx.compose.runtime.DisposableEffect
36 import androidx.compose.runtime.getValue
37 import androidx.compose.runtime.rememberCoroutineScope
38 import androidx.compose.ui.Alignment
39 import androidx.compose.ui.Modifier
40 import androidx.compose.ui.text.font.FontWeight
41 import androidx.compose.ui.text.style.TextOverflow
42 import androidx.compose.ui.unit.dp
43 import androidx.hilt.navigation.compose.hiltViewModel
44 import androidx.lifecycle.Lifecycle
45 import androidx.lifecycle.LifecycleEventObserver
46 import androidx.lifecycle.compose.LocalLifecycleOwner
47 import com.runway.android.data.course.model.CourseResponse
48 import com.runway.android.data.course.model.ParticipatedCourseItem
49 import kotlinx.coroutines.launch
50
51 @OptIn(ExperimentalMaterial3Api::class)
52 @Composable
53 fun CoursesLibraryScreen(
54     onNavigateToCourseDetail: (String) -> Unit = {},
55     viewModel: CoursesLibraryViewModel = hiltViewModel(),
56 ) {
57     val tabs = listOf("만든 코스", "즐거찾기", "참여한 코스")
58     val pagerState = rememberPagerState(pageCount = { tabs.size })
59     val scope = rememberCoroutineScope()
60
61     // ON_RESUME 마다 항상 재로드:
62     // - 탭 전환 시 컴포지블이 새로 생성되면 라이프사이클이 이미 RESUMED이므로
63     // observer 등록 직후 ON_RESUME이 즉시 발생 → reload ✓
64     // - 코스 상세에서 돌아올 때도 ON_RESUME 발생 → reload ✓
65     val lifecycleOwner = LocalLifecycleOwner.current
66     DisposableEffect(lifecycleOwner) {
67         val observer = LifecycleEventObserver { _, event ->
68             if (event == Lifecycle.Event.ON_RESUME) {
69                 viewModel.onResume()
70             }
71         }
72         lifecycleOwner.lifecycle.addObserver(observer)
73         onDispose { lifecycleOwner.lifecycle.removeObserver(observer) }
74     }
75
76     Column(
77         modifier = Modifier
78             .fillMaxSize()
79             .background(MaterialTheme.colorScheme.background),
80     ) {
81         Text(
82             text = "내 코스",
83             style = MaterialTheme.typography.headlineLarge,

```

```

84         fontWeight = FontWeight.Bold,
85         color = MaterialTheme.colorScheme.onBackground,
86         modifier = Modifier.padding(horizontal = 20.dp, vertical = 16.dp),
87     )
88
89     TabRow(
90         selectedTabIndex = pagerState.currentPage,
91         containerColor = MaterialTheme.colorScheme.background,
92         contentColor = MaterialTheme.colorScheme.primary,
93     ) {
94         tabs.forEachIndexed { index, title ->
95             Tab(
96                 selected = pagerState.currentPage == index,
97                 onClick = { scope.launch { pagerState.animateScrollToPage(index) } },
98                 text = {
99                     Text(
100                         text = title,
101                         style = MaterialTheme.typography.labelMedium,
102                         fontWeight = if (pagerState.currentPage == index) FontWeight.SemiBold
103                             else FontWeight.Normal,
104                     )
105                 },
106             )
107         }
108     }
109
110     PullToRefreshBox(
111         isRefreshing = viewModel.isRefreshing,
112         onRefresh = viewModel::refresh,
113         modifier = Modifier.fillMaxWidth().weight(1f),
114     ) {
115         HorizontalPager(state = pagerState, modifier = Modifier.fillMaxSize()) { page ->
116             when (page) {
117                 0 -> CreatedCoursesTab(
118                     courses = viewModel.myCourses,
119                     isLoading = viewModel.isLoadingMyCourses,
120                     error = viewModel.myCoursesError,
121                     onRetry = viewModel::loadMyCourses,
122                     onCourseClick = onNavigateToCourseDetail,
123                 )
124                 1 -> FavoriteCoursesTab(
125                     courses = viewModel.favoriteCourses,
126                     isLoading = viewModel.isLoadingFavorites,
127                     error = viewModel.favoritesError,
128                     onRetry = viewModel::loadFavoriteCourses,
129                     onCourseClick = onNavigateToCourseDetail,
130                 )
131                 2 -> ParticipatedCoursesTab(
132                     courses = viewModel.participatedCourses,
133                     isLoading = viewModel.isLoadingParticipated,
134                     error = viewModel.participatedError,
135                     onRetry = viewModel::loadParticipatedCourses,
136                     onCourseClick = onNavigateToCourseDetail,
137                 )
138             }
139         }
140     }
141 }
142
143
144 // — 만든 코스 탭 —————
145
146 @Composable
147 private fun CreatedCoursesTab(
148     courses: List<CourseResponse>,
149     isLoading: Boolean,
150     error: String?,
151     onRetry: () -> Unit,
152     onCourseClick: (String) -> Unit,
153 ) {
154     TabContent(isLoading = isLoading, error = error, onRetry = onRetry) {
155         if (courses.isEmpty()) {
156             EmptyState(
157                 icon = { Icon(Icons.AutoMirrored.Filled.DirectionsRun, null,
158                     modifier = Modifier.size(40.dp), tint = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.4f)) },
159                 message = "아직 만든 코스가 없습니다.",
160                 sub = "러닝을 완료한 후 코스를 등록해 보세요.",
161             )
162         } else {
163             LazyColumn(modifier = Modifier.fillMaxSize()) {
164                 items(courses, key = { it.courseId }) { course ->
165                     CreatedCourseCard(course = course, onClick = { onCourseClick(course.courseId) })
166                 }
167             }
168         }
169     }
170 }

```

```

170 }
171
172 @Composable
173 private fun CreatedCourseCard(course: CourseResponse, onClick: () -> Unit) {
174     Surface(
175         modifier = Modifier
176             .fillMaxWidth()
177             .padding(horizontal = 16.dp, vertical = 6.dp),
178         shape = MaterialTheme.shapes.large,
179         color = MaterialTheme.colorScheme.surface,
180         tonalElevation = 1.dp,
181         onClick = onClick,
182     ) {
183         Row(
184             modifier = Modifier.padding(16.dp),
185             verticalAlignment = Alignment.CenterVertically,
186             horizontalArrangement = Arrangement.spacedBy(14.dp),
187         ) {
188             // 거리 원형 배지
189             DistanceBadge(meters = course.distanceMeters)
190
191             Column(modifier = Modifier.weight(1f)) {
192                 Row(
193                     verticalAlignment = Alignment.CenterVertically,
194                     horizontalArrangement = Arrangement.spacedBy(8.dp),
195                 ) {
196                     Text(
197                         text = course.name,
198                         style = MaterialTheme.typography.titleSmall,
199                         fontWeight = FontWeight.SemiBold,
200                         color = MaterialTheme.colorScheme.onSurface,
201                         maxLines = 1,
202                         overflow = TextOverflow.Ellipsis,
203                         modifier = Modifier.weight(1f, fill = false),
204                     )
205                     StatusBadge(status = course.status)
206                 }
207                 if (!course.description.isNullOrBlank()) {
208                     Spacer(Modifier.height(2.dp))
209                     Text(
210                         text = course.description,
211                         style = MaterialTheme.typography.bodySmall,
212                         color = MaterialTheme.colorScheme.onSurfaceVariant,
213                         maxLines = 1,
214                         overflow = TextOverflow.Ellipsis,
215                     )
216                 }
217                 Spacer(Modifier.height(6.dp))
218                 Row(
219                     horizontalArrangement = Arrangement.spacedBy(12.dp),
220                     verticalAlignment = Alignment.CenterVertically,
221                 ) {
222                     if (course.isLoop) {
223                         StatChip(
224                             icon = { Icon(Icons.Filled.Loop, null, Modifier.size(13.dp),
225                                 tint = MaterialTheme.colorScheme.primary) },
226                             text = "루프",
227                         )
228                     }
229                     StatChip(
230                         icon = { Icon(Icons.Filled.TrendingUp, null, Modifier.size(13.dp),
231                             tint = MaterialTheme.colorScheme.onSurfaceVariant) },
232                         text = "도전 ${course.attemptCount}회",
233                     )
234                     StatChip(
235                         icon = { Icon(Icons.AutoMirrored.Filled.DirectionsRun, null, Modifier.size(13.dp),
236                             tint = MaterialTheme.colorScheme.onSurfaceVariant) },
237                         text = "완주 ${course.completionCount}회",
238                     )
239                 }
240             }
241         }
242     }
243 }
244
245 // — 즐겨찾기 탭 —————
246
247 @Composable
248 private fun FavoriteCoursesTab(
249     courses: List<CourseResponse>,
250     isLoading: Boolean,
251     error: String?,
252     onRetry: () -> Unit,
253     onCourseClick: (String) -> Unit,
254 ) {
255     TabContent(isLoading = isLoading, error = error, onRetry = onRetry) {

```

```

256     if (courses.isEmpty()) {
257         EmptyState(
258             icon = { Icon(Icons.Filled.BookmarkBorder, null,
259                 modifier = Modifier.size(40.dp), tint = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.4f)) },
260             message = "즐거찾기한 코스가 없습니다.",
261             sub = "코스 상세 화면에서 즐겨찾기를 추가해 보세요.",
262         )
263     } else {
264         LazyColumn(modifier = Modifier.fillMaxSize()) {
265             items(courses, key = { it.courseId }) { course ->
266                 FavoriteCourseCard(course = course, onClick = { onCourseClick(course.courseId) })
267             }
268         }
269     }
270 }
271 }
272
273 @Composable
274 private fun FavoriteCourseCard(course: CourseResponse, onClick: () -> Unit) {
275     Surface(
276         modifier = Modifier
277             .fillMaxWidth()
278             .padding(horizontal = 16.dp, vertical = 6.dp),
279         shape = MaterialTheme.shapes.large,
280         color = MaterialTheme.colorScheme.surface,
281         tonalElevation = 1.dp,
282         onClick = onClick,
283     ) {
284         Row(
285             modifier = Modifier.padding(16.dp),
286             verticalAlignment = Alignment.CenterVertically,
287             horizontalArrangement = Arrangement.spacedBy(14.dp),
288         ) {
289             DistanceBadge(meters = course.distanceMeters)
290
291             Column(modifier = Modifier.weight(1f)) {
292                 Text(
293                     text = course.name,
294                     style = MaterialTheme.typography.titleSmall,
295                     fontWeight = FontWeight.SemiBold,
296                     color = MaterialTheme.colorScheme.onSurface,
297                     maxLines = 1,
298                     overflow = TextOverflow.Ellipsis,
299                 )
300                 if (!course.description.isNullOrBlank()) {
301                     Spacer(Modifier.height(2.dp))
302                     Text(
303                         text = course.description,
304                         style = MaterialTheme.typography.bodySmall,
305                         color = MaterialTheme.colorScheme.onSurfaceVariant,
306                         maxLines = 1,
307                         overflow = TextOverflow.Ellipsis,
308                     )
309                 }
310                 Spacer(Modifier.height(6.dp))
311                 Row(
312                     horizontalArrangement = Arrangement.spacedBy(12.dp),
313                     verticalAlignment = Alignment.CenterVertically,
314                 ) {
315                     if (course.isLoop) {
316                         StatChip(
317                             icon = { Icon(Icons.Filled.Loop, null, Modifier.size(13.dp),
318                                 tint = MaterialTheme.colorScheme.primary) },
319                             text = "루프",
320                         )
321                     }
322                     StatChip(
323                         icon = { Icon(Icons.Filled.TrendingUp, null, Modifier.size(13.dp),
324                             tint = MaterialTheme.colorScheme.onSurfaceVariant) },
325                         text = "완주 ${course.completionCount}회",
326                     )
327                 }
328             }
329         }
330     }
331 }
332
333 // — 참여한 코스 탭 —————
334
335 @Composable
336 private fun ParticipatedCoursesTab(
337     courses: List<ParticipatedCourseItem>,
338     isLoading: Boolean,
339     error: String?,
340     onRetry: () -> Unit,
341     onCourseClick: (String) -> Unit,

```

```

342 ) {
343     TabContent(isLoading = isLoading, error = error, onRetry = onRetry) {
344         if (courses.isEmpty()) {
345             EmptyState(
346                 icon = { Icon(Icons.AutoMirrored.Filled.DirectionsRun, null,
347                     modifier = Modifier.size(40.dp), tint = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.4f)) },
348                 message = "아직 참여한 코스가 없습니다.",
349                 sub = "코스 탐색에서 도전할 코스를 찾아보세요.",
350             )
351         } else {
352             LazyColumn(modifier = Modifier.fillMaxSize()) {
353                 items(courses, key = { it.courseId }) { course ->
354                     ParticipatedCourseCard(course = course, onClick = { onCourseClick(course.courseId) })
355                 }
356             }
357         }
358     }
359 }
360
361 @Composable
362 private fun ParticipatedCourseCard(course: ParticipatedCourseItem, onClick: () -> Unit) {
363     Surface(
364         modifier = Modifier
365             .fillMaxWidth()
366             .padding(horizontal = 16.dp, vertical = 6.dp),
367         shape = MaterialTheme.shapes.large,
368         color = MaterialTheme.colorScheme.surface,
369         tonalElevation = 1.dp,
370         onClick = onClick,
371     ) {
372         Row(
373             modifier = Modifier.padding(16.dp),
374             verticalAlignment = Alignment.CenterVertically,
375             horizontalArrangement = Arrangement.spacedBy(14.dp),
376         ) {
377             DistanceBadge(meters = course.distanceMeters)
378
379             Column(modifier = Modifier.weight(1f)) {
380                 Text(
381                     text = course.name,
382                     style = MaterialTheme.typography.titleSmall,
383                     fontWeight = FontWeight.SemiBold,
384                     color = MaterialTheme.colorScheme.onSurface,
385                     maxLines = 1,
386                     overflow = TextOverflow.Ellipsis,
387                 )
388                 if (!course.description.isNullOrBlank()) {
389                     Spacer(Modifier.height(2.dp))
390                     Text(
391                         text = course.description,
392                         style = MaterialTheme.typography.bodySmall,
393                         color = MaterialTheme.colorScheme.onSurfaceVariant,
394                         maxLines = 1,
395                         overflow = TextOverflow.Ellipsis,
396                     )
397                 }
398                 Spacer(Modifier.height(6.dp))
399                 Row(
400                     horizontalArrangement = Arrangement.spacedBy(12.dp),
401                     verticalAlignment = Alignment.CenterVertically,
402                     modifier = Modifier.fillMaxWidth(),
403                 ) {
404                     if (course.isLoop) {
405                         StatChip(
406                             icon = { Icon(Icons.Filled.Loop, null, Modifier.size(13.dp),
407                                 tint = MaterialTheme.colorScheme.primary) },
408                             text = "루프",
409                         )
410                     }
411                     StatChip(
412                         icon = { Icon(Icons.AutoMirrored.Filled.DirectionsRun, null, Modifier.size(13.dp),
413                             tint = MaterialTheme.colorScheme.onSurfaceVariant) },
414                         text = "완주 ${course.completionCountByMe}/${course.attemptCountByMe}회",
415                     )
416                     course.bestTimeSecondsByMe?.let { secs ->
417                         StatChip(
418                             icon = { Icon(Icons.Filled.Timer, null, Modifier.size(13.dp),
419                                 tint = MaterialTheme.colorScheme.primary) },
420                             text = formatDuration(secs),
421                         )
422                     }
423                 }
424             }
425         }
426     }
427 }

```



```

428
429 // — 공통 컴포넌트 —————
430
431 @Composable
432 private fun DistanceBadge(meters: Double) {
433     Box(
434         modifier = Modifier
435             .size(52.dp)
436             .background(
437                 MaterialTheme.colorScheme.primaryContainer,
438                 shape = MaterialTheme.shapes.medium,
439             ),
440         contentAlignment = Alignment.Center,
441     ) {
442         Column(horizontalAlignment = Alignment.CenterHorizontally) {
443             val (value, unit) = if (meters >= 1000)
444                 "%.1f".format(meters / 1000.0) to "km"
445             else
446                 "${meters.toInt()}" to "m"
447             Text(
448                 text = value,
449                 style = MaterialTheme.typography.labelLarge,
450                 fontWeight = FontWeight.Bold,
451                 color = MaterialTheme.colorScheme.onPrimaryContainer,
452             )
453             Text(
454                 text = unit,
455                 style = MaterialTheme.typography.labelSmall,
456                 color = MaterialTheme.colorScheme.onPrimaryContainer.copy(alpha = 0.7f),
457             )
458         }
459     }
460 }
461
462 @Composable
463 private fun StatChip(icon: @Composable () -> Unit, text: String) {
464     Row(
465         verticalAlignment = Alignment.CenterVertically,
466         horizontalArrangement = Arrangement.spacedBy(3.dp),
467     ) {
468         icon()
469         Text(
470             text = text,
471             style = MaterialTheme.typography.bodySmall,
472             color = MaterialTheme.colorScheme.onSurfaceVariant,
473         )
474     }
475 }
476
477 @Composable
478 private fun StatusBadge(status: String) {
479     val (label, color) = when (status) {
480         "published" -> "공개" to MaterialTheme.colorScheme.primary
481         "draft" -> "초안" to MaterialTheme.colorScheme.onSurfaceVariant
482         "archived" -> "보관" to MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.6f)
483         else -> status to MaterialTheme.colorScheme.onSurfaceVariant
484     }
485     Box(
486         modifier = Modifier
487             .background(color.copy(alpha = 0.12f), shape = MaterialTheme.shapes.extraSmall)
488             .padding(horizontal = 7.dp, vertical = 2.dp),
489     ) {
490         Text(text = label, style = MaterialTheme.typography.labelSmall, color = color)
491     }
492 }
493
494 @Composable
495 private fun TabContent(
496     isLoading: Boolean,
497     error: String?,
498     onRetry: () -> Unit,
499     content: @Composable () -> Unit,
500 ) {
501     Box(modifier = Modifier.fillMaxSize()) {
502         when {
503             isLoading -> Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
504                 CircularProgressIndicator(color = MaterialTheme.colorScheme.primary)
505             }
506             error != null -> ErrorState(message = error, onRetry = onRetry)
507             else -> content()
508         }
509     }
510 }
511
512 @Composable
513 private fun EmptyState(

```

```

514     icon: @Composable () -> Unit,
515     message: String,
516     sub: String,
517 ) {
518     Column(
519         modifier = Modifier.fillMaxSize().padding(32.dp),
520         horizontalAlignment = Alignment.CenterHorizontally,
521         verticalArrangement = Arrangement.Center,
522     ) {
523         icon()
524         Spacer(Modifier.height(16.dp))
525         Text(text = message, style = MaterialTheme.typography.titleSmall,
526             color = MaterialTheme.colorScheme.onSurfaceVariant, fontWeight = FontWeight.Medium)
527         Spacer(Modifier.height(4.dp))
528         Text(text = sub, style = MaterialTheme.typography.bodySmall,
529             color = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.7f))
530     }
531 }
532
533 @Composable
534 private fun ErrorState(message: String, onRetry: () -> Unit) {
535     Column(
536         modifier = Modifier.fillMaxSize().padding(32.dp),
537         horizontalAlignment = Alignment.CenterHorizontally,
538         verticalArrangement = Arrangement.Center,
539     ) {
540         Text(text = message, style = MaterialTheme.typography.bodyMedium,
541             color = MaterialTheme.colorScheme.onSurfaceVariant)
542         Spacer(Modifier.height(16.dp))
543         Surface(onClick = onRetry, shape = MaterialTheme.shapes.extraLarge,
544             color = MaterialTheme.colorScheme.primary) {
545             Text(text = "다시 시도", style = MaterialTheme.typography.labelLarge,
546                 color = MaterialTheme.colorScheme.onPrimary,
547                 modifier = Modifier.padding(horizontal = 20.dp, vertical = 10.dp))
548         }
549     }
550 }
551
552 private fun formatDistance(meters: Double): String =
553     if (meters >= 1000) "%.1f km".format(meters / 1000.0) else "${meters.toInt()} m"
554
555 private fun formatDuration(seconds: Int): String {
556     val h = seconds / 3600
557     val m = (seconds % 3600) / 60
558     val s = seconds % 60
559     return if (h > 0) "%d:%02d:%02d".format(h, m, s) else "%d:%02d".format(m, s)
560 }

```

```

1  package com.runway.android.ui.course.library
2
3  import androidx.compose.runtime.getValue
4  import androidx.compose.runtime.mutableStateOf
5  import androidx.compose.runtime.setValue
6  import androidx.lifecycle.ViewModel
7  import androidx.lifecycle.viewModelScope
8  import com.runway.android.core.result.NetworkResult
9  import com.runway.android.data.course.model.CourseResponse
10 import com.runway.android.data.course.model.ParticipatedCourseItem
11 import com.runway.android.domain.course.CourseRepository
12 import dagger.hilt.android.lifecycle.HiltViewModel
13 import kotlinx.coroutines.async
14 import kotlinx.coroutines.coroutineScope
15 import kotlinx.coroutines.launch
16 import javax.inject.Inject
17
18 @HiltViewModel
19 class CoursesLibraryViewModel @Inject constructor(
20     private val courseRepository: CourseRepository,
21 ) : ViewModel() {
22
23     // — 만든 코스 —
24     var myCourses by mutableStateOf<List<CourseResponse>>(emptyList())
25     private set
26     var isLoadingMyCourses by mutableStateOf(false)
27     private set
28     var myCoursesError by mutableStateOf<String?>(null)
29     private set
30
31     // — 즐겨찾기 —
32     var favoriteCourses by mutableStateOf<List<CourseResponse>>(emptyList())
33     private set
34     var isLoadingFavorites by mutableStateOf(false)
35     private set
36     var favoritesError by mutableStateOf<String?>(null)
37     private set
38
39     // — 참여한 코스 —
40     var participatedCourses by mutableStateOf<List<ParticipatedCourseItem>>(emptyList())
41     private set
42     var isLoadingParticipated by mutableStateOf(false)
43     private set
44     var participatedError by mutableStateOf<String?>(null)
45     private set
46
47     var isRefreshing by mutableStateOf(false)
48     private set
49
50     init {
51         loadMyCourses()
52         loadFavoriteCourses()
53         loadParticipatedCourses()
54     }
55
56     fun refresh() {
57         viewModelScope.launch {
58             isRefreshing = true
59             coroutineScope {
60                 val a = async { doLoadMyCourses() }
61                 val b = async { doLoadFavorites() }
62                 val c = async { doLoadParticipated() }
63                 a.await(); b.await(); c.await()
64             }
65             isRefreshing = false
66         }
67     }
68
69     fun onResume() {
70         viewModelScope.launch {
71             coroutineScope {
72                 val a = async { doLoadMyCourses() }
73                 val b = async { doLoadFavorites() }
74                 val c = async { doLoadParticipated() }
75                 a.await(); b.await(); c.await()
76             }
77         }
78     }
79
80     fun loadMyCourses() {
81         viewModelScope.launch { doLoadMyCourses() }
82     }
83

```

```

84 fun loadFavoriteCourses() {
85     viewModelScope.launch { doLoadFavorites() }
86 }
87
88 fun loadParticipatedCourses() {
89     viewModelScope.launch { doLoadParticipated() }
90 }
91
92 private suspend fun doLoadMyCourses() {
93     isLoadingMyCourses = true
94     myCoursesError = null
95     when (val result = courseRepository.getMyCourses()) {
96         is NetworkResult.Success -> myCourses = result.data.content
97         is NetworkResult.ApiError -> myCoursesError = result.message
98         is NetworkResult.NetworkError -> myCoursesError = "네트워크 오류가 발생했습니다."
99     }
100     isLoadingMyCourses = false
101 }
102
103 private suspend fun doLoadFavorites() {
104     isLoadingFavorites = true
105     favoritesError = null
106     when (val result = courseRepository.getFavoriteCourses()) {
107         is NetworkResult.Success -> favoriteCourses = result.data.content
108         is NetworkResult.ApiError -> favoritesError = result.message
109         is NetworkResult.NetworkError -> favoritesError = "네트워크 오류가 발생했습니다."
110     }
111     isLoadingFavorites = false
112 }
113
114 private suspend fun doLoadParticipated() {
115     isLoadingParticipated = true
116     participatedError = null
117     when (val result = courseRepository.getParticipatedCourses()) {
118         is NetworkResult.Success -> participatedCourses = result.data.content
119         is NetworkResult.ApiError -> participatedError = result.message
120         is NetworkResult.NetworkError -> participatedError = "네트워크 오류가 발생했습니다."
121     }
122     isLoadingParticipated = false
123 }
124 }

```

```

1  package com.runway.android.ui.course.my
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.background
5  import androidx.compose.foundation.layout.Arrangement
6  import androidx.compose.foundation.layout.Box
7  import androidx.compose.foundation.layout.Column
8  import androidx.compose.foundation.layout.PaddingValues
9  import androidx.compose.foundation.layout.Row
10 import androidx.compose.foundation.layout.Spacer
11 import androidx.compose.foundation.layout.fillMaxSize
12 import androidx.compose.foundation.layout.fillMaxWidth
13 import androidx.compose.foundation.layout.height
14 import androidx.compose.foundation.layout.padding
15 import androidx.compose.foundation.layout.size
16 import androidx.compose.foundation.layout.statusBarsPadding
17 import androidx.compose.foundation.layout.width
18 import androidx.compose.foundation.lazy.LazyColumn
19 import androidx.compose.foundation.lazy.items
20 import androidx.compose.material.icons.Icons
21 import androidx.compose.material.icons.automirrored.filled.ArrowBack
22 import androidx.compose.material.icons.filled.Route
23 import androidx.compose.material3.Button
24 import androidx.compose.material3.CircularProgressIndicator
25 import androidx.compose.material3.ExperimentalMaterial3Api
26 import androidx.compose.material3.Icon
27 import androidx.compose.material3.IconButton
28 import androidx.compose.material3.MaterialTheme
29 import androidx.compose.material3.Surface
30 import androidx.compose.material3.Text
31 import androidx.compose.material3.pulltorefresh.PullToRefreshBox
32 import androidx.compose.runtime.Composable
33 import androidx.compose.ui.Alignment
34 import androidx.compose.ui.Modifier
35 import androidx.compose.ui.text.style.TextAlign
36 import androidx.compose.ui.text.style.TextOverflow
37 import androidx.compose.ui.unit.dp
38 import androidx.hilt.navigation.compose.hiltViewModel
39 import com.runway.android.data.course.model.CourseResponse
40
41 @OptIn(ExperimentalMaterial3Api::class)
42 @Composable
43 fun MyCoursesScreen(
44     onBack: () -> Unit,
45     onNavigateToCourseDetail: (String) -> Unit,
46     viewModel: MyCoursesViewModel = hiltViewModel(),
47 ) {
48     Column(
49         modifier = Modifier
50             .fillMaxSize()
51             .background(MaterialTheme.colorScheme.background)
52             .statusBarsPadding(),
53     ) {
54         Row(
55             modifier = Modifier
56                 .fillMaxWidth()
57                 .padding(horizontal = 8.dp, vertical = 4.dp),
58             verticalAlignment = Alignment.CenterVertically,
59         ) {
60             IconButton(onClick = onBack) {
61                 Icon(
62                     imageVector = Icons.AutoMirrored.Filled.ArrowBack,
63                     contentDescription = "뒤로 가기",
64                     tint = MaterialTheme.colorScheme.onBackground,
65                 )
66             }
67             Spacer(modifier = Modifier.width(4.dp))
68             Text(
69                 text = "내 코스",
70                 style = MaterialTheme.typography.headlineMedium,
71                 color = MaterialTheme.colorScheme.onBackground,
72             )
73         }
74
75         PullToRefreshBox(
76             isRefreshing = viewModel.isRefreshing,
77             onRefresh = viewModel::refresh,
78             modifier = Modifier
79                 .fillMaxSize()
80                 .weight(1f),
81         ) {
82             when {
83                 viewModel.isLoading -> {

```

```

84         Box(
85             modifier = Modifier.fillMaxSize(),
86             contentAlignment = Alignment.Center,
87         ) {
88             CircularProgressIndicator(
89                 modifier = Modifier.size(40.dp),
90                 color = MaterialTheme.colorScheme.primary,
91             )
92         }
93     }
94     viewModel.hasError -> {
95         Box(
96             modifier = Modifier.fillMaxSize(),
97             contentAlignment = Alignment.Center,
98         ) {
99             Column(horizontalAlignment = Alignment.CenterHorizontally) {
100                 Text(
101                     text = "코스 목록을 불러오지 못했습니다.",
102                     style = MaterialTheme.typography.bodyLarge,
103                     color = MaterialTheme.colorScheme.onSurfaceVariant,
104                 )
105                 Spacer(modifier = Modifier.height(16.dp))
106                 Button(onClick = viewModel::retry) { Text("다시 시도") }
107             }
108         }
109     }
110     viewModel.courses.isEmpty() -> {
111         Box(
112             modifier = Modifier.fillMaxSize(),
113             contentAlignment = Alignment.Center,
114         ) {
115             Column(
116                 horizontalAlignment = Alignment.CenterHorizontally,
117                 modifier = Modifier.padding(32.dp),
118             ) {
119                 Icon(
120                     imageVector = Icons.Filled.Route,
121                     contentDescription = null,
122                     tint = MaterialTheme.colorScheme.onSurfaceVariant,
123                     modifier = Modifier.size(48.dp),
124                 )
125                 Spacer(modifier = Modifier.height(12.dp))
126                 Text(
127                     text = "아직 만든 코스가 없어요",
128                     style = MaterialTheme.typography.titleMedium,
129                     color = MaterialTheme.colorScheme.onSurface,
130                     textAlign = TextAlign.Center,
131                 )
132                 Spacer(modifier = Modifier.height(6.dp))
133                 Text(
134                     text = "러닝을 완료한 후 경로를 코스로 등록해보세요.",
135                     style = MaterialTheme.typography.bodyMedium,
136                     color = MaterialTheme.colorScheme.onSurfaceVariant,
137                     textAlign = TextAlign.Center,
138                 )
139             }
140         }
141     }
142     else -> {
143         LazyColumn(
144             contentPadding = PaddingValues(horizontal = 20.dp, vertical = 12.dp),
145             verticalArrangement = Arrangement.spacedBy(12.dp),
146         ) {
147             items(viewModel.courses) { course ->
148                 MyCourseCard(
149                     course = course,
150                     onClick = { onNavigateToCourseDetail(course.courseId) },
151                 )
152             }
153         }
154     }
155 }
156 }
157 }
158 }
159
160 @Composable
161 private fun MyCourseCard(
162     course: CourseResponse,
163     onClick: () -> Unit,
164 ) {
165     Surface(
166         modifier = Modifier.fillMaxWidth(),
167         shape = MaterialTheme.shapes.extraLarge,
168         color = MaterialTheme.colorScheme.surface,
169         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),

```

```

170     onClick = onClick,
171 ) {
172     Column(modifier = Modifier.padding(16.dp)) {
173         Row(
174             modifier = Modifier.fillMaxWidth(),
175             horizontalArrangement = Arrangement.SpaceBetween,
176             verticalAlignment = Alignment.CenterVertically,
177         ) {
178             Text(
179                 text = course.name,
180                 style = MaterialTheme.typography.titleMedium,
181                 color = MaterialTheme.colorScheme.onSurface,
182                 modifier = Modifier.weight(1f),
183                 maxLines = 1,
184                 overflow = TextOverflow.Ellipsis,
185             )
186             Spacer(modifier = Modifier.width(8.dp))
187             CourseStatusChip(status = course.status)
188         }
189         if (!course.description.isNullOrBlank()) {
190             Spacer(modifier = Modifier.height(4.dp))
191             Text(
192                 text = course.description,
193                 style = MaterialTheme.typography.bodySmall,
194                 color = MaterialTheme.colorScheme.onSurfaceVariant,
195                 maxLines = 2,
196             )
197         }
198         Spacer(modifier = Modifier.height(10.dp))
199         Row(horizontalArrangement = Arrangement.spacedBy(16.dp)) {
200             MetaChip(label = formatDistance(course.distanceMeters))
201             MetaChip(label = "도전 ${course.attemptCount}회")
202             MetaChip(label = "완주 ${course.completionCount}회")
203         }
204     }
205 }
206
207
208 @Composable
209 private fun CourseStatusChip(status: String) {
210     val (label, color) = when (status) {
211         "published" -> "공개" to MaterialTheme.colorScheme.primary
212         "draft" -> "초안" to MaterialTheme.colorScheme.onSurfaceVariant
213         "archived" -> "보관" to MaterialTheme.colorScheme.onSurfaceVariant
214         else -> status to MaterialTheme.colorScheme.onSurfaceVariant
215     }
216     Surface(
217         shape = MaterialTheme.shapes.small,
218         color = color.copy(alpha = 0.12f),
219         border = BorderStroke(1.dp, color.copy(alpha = 0.3f)),
220     ) {
221         Text(
222             text = label,
223             style = MaterialTheme.typography.labelSmall,
224             color = color,
225             modifier = Modifier.padding(horizontal = 8.dp, vertical = 3.dp),
226         )
227     }
228 }
229
230 @Composable
231 private fun MetaChip(label: String) {
232     Text(
233         text = label,
234         style = MaterialTheme.typography.labelSmall,
235         color = MaterialTheme.colorScheme.onSurfaceVariant,
236     )
237 }
238
239 private fun formatDistance(meters: Double): String = when {
240     meters >= 1000 -> "%.1f km".format(meters / 1000)
241     else -> "${meters.toInt()} m"
242 }

```

```
1 package com.runway.android.ui.course.my
2
3 import androidx.compose.runtime.getValue
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.compose.runtime.setValue
6 import androidx.lifecycle.ViewModel
7 import androidx.lifecycle.ViewModelScope
8 import com.runway.android.core.result.NetworkResult
9 import com.runway.android.data.course.model.CourseResponse
10 import com.runway.android.domain.course.CourseRepository
11 import dagger.hilt.android.lifecycle.HiltViewModel
12 import kotlinx.coroutines.launch
13 import javax.inject.Inject
14
15 @HiltViewModel
16 class MyCoursesViewModel @Inject constructor(
17     private val courseRepository: CourseRepository,
18 ) : ViewModel() {
19
20     var isLoading by mutableStateOf(true)
21     private set
22     var isRefreshing by mutableStateOf(false)
23     private set
24     var hasError by mutableStateOf(false)
25     private set
26     var courses by mutableStateOf<List<CourseResponse>>(emptyList())
27     private set
28
29     init {
30         load()
31     }
32
33     fun retry() {
34         isLoading = true
35         hasError = false
36         courses = emptyList()
37         load()
38     }
39
40     fun refresh() {
41         viewModelScope.launch {
42             isRefreshing = true
43             doLoad()
44             isRefreshing = false
45         }
46     }
47
48     private fun load() {
49         viewModelScope.launch { doLoad() }
50     }
51
52     private suspend fun doLoad() {
53         hasError = false
54         when (val result = courseRepository.getMyCourses()) {
55             is NetworkResult.Success -> courses = result.data.content
56             else -> hasError = true
57         }
58         isLoading = false
59     }
60 }
```



```
1 package com.runway.android.ui.discover
2
3 import android.annotation.SuppressLint
4 import android.content.Context
5 import android.graphics.Bitmap
6 import android.graphics.BitmapFactory
7 import android.graphics.Canvas as AndroidCanvas
8 import android.graphics.Paint
9 import android.graphics.Path
10 import android.graphics.Rect
11 import android.graphics.RectF
12 import android.graphics.Typeface
13 import androidx.compose.animation.AnimatedVisibility
14 import androidx.compose.animation.slideInVertically
15 import androidx.compose.animation.slideOutVertically
16 import androidx.compose.foundation.BorderStroke
17 import androidx.compose.foundation.Canvas
18 import androidx.compose.foundation.background
19 import androidx.compose.foundation.clickable
20 import androidx.compose.foundation.layout.Arrangement
21 import androidx.compose.foundation.layout.Box
22 import androidx.compose.foundation.layout.Column
23 import androidx.compose.foundation.layout.PaddingValues
24 import androidx.compose.foundation.layout.Row
25 import androidx.compose.foundation.layout.Spacer
26 import androidx.compose.foundation.layout.fillMaxSize
27 import androidx.compose.foundation.layout.fillMaxWidth
28 import androidx.compose.foundation.layout.height
29 import androidx.compose.foundation.layout.padding
30 import androidx.compose.foundation.layout.size
31 import androidx.compose.foundation.layout.widthIn
32 import androidx.compose.foundation.lazy.LazyRow
33 import androidx.compose.foundation.lazy.items
34 import androidx.compose.foundation.shape.CircleShape
35 import androidx.compose.material.icons.Icons
36 import androidx.compose.material.icons.filled.Close
37 import androidx.compose.material.icons.filled.LocationOn
38 import androidx.compose.material.icons.filled.MyLocation
39 import androidx.compose.material.icons.filled.Route
40 import androidx.compose.material3.CircularProgressIndicator
41 import androidx.compose.material3.Icon
42 import androidx.compose.material3.IconButton
43 import androidx.compose.material3.MaterialTheme
44 import androidx.compose.material3.Surface
45 import androidx.compose.material3.Text
46 import androidx.compose.runtime.Composable
47 import androidx.compose.runtime.LaunchedEffect
48 import androidx.compose.runtime.derivedStateOf
49 import androidx.compose.runtime.getValue
50 import androidx.compose.runtime.mutableStateOf
51 import androidx.compose.runtime.remember
52 import androidx.compose.runtime.rememberCoroutineScope
53 import androidx.compose.runtime.setValue
54 import androidx.compose.ui.Alignment
55 import androidx.compose.ui.Modifier
56 import androidx.compose.ui.geometry.Offset
57 import androidx.compose.ui.graphics.Color
58 import androidx.compose.ui.graphics.toArgb
59 import androidx.compose.ui.platform.LocalContext
60 import androidx.compose.ui.text.font.FontWeight
61 import androidx.compose.ui.text.style.TextOverflow
62 import androidx.compose.ui.unit.dp
63 import androidx.compose.ui.unit.sp
64 import com.google.android.gms.maps.CameraUpdateFactory
65 import com.google.android.gms.maps.model.BitmapDescriptor
66 import com.google.android.gms.maps.model.BitmapDescriptorFactory
67 import com.google.android.gms.maps.model.CameraPosition
68 import com.google.android.gms.maps.model.LatLng
69 import com.google.android.gms.maps.model.MapStyleOptions
70 import com.google.maps.android.compose.GoogleMap
71 import com.google.maps.android.compose.MapProperties
72 import com.google.maps.android.compose.MapUiSettings
73 import com.google.maps.android.compose.Marker
74 import com.google.maps.android.compose.MarkerInfoWindow
75 import com.google.maps.android.compose.MarkerState
76 import com.google.maps.android.compose.rememberCameraPositionState
77 import com.runway.android.R
78 import com.runway.android.core.map.MapPoint
79 import com.runway.android.data.course.model.NearbyCourseItem
80 import com.runway.android.ui.theme.LocalIsDarkTheme
81 import kotlinx.coroutines.launch
82 import kotlin.math.floor
83 import kotlin.math.pow
```

```

84
85 // 화면에서 이 픽셀 이내의 마커를 하나의 클러스터로 묶는다
86 private const val CLUSTER_THRESHOLD_PX = 72.0
87
88 private data class CourseCluster(
89     val id: String,
90     val courses: List<NearbyCourseItem>,
91     val center: LatLng,
92 )
93
94 private fun clusterCourses(courses: List<NearbyCourseItem>, zoom: Float): List<CourseCluster> {
95     if (courses.isEmpty()) return emptyList()
96     val pixelsPerDeg = 256.0 * 2.0.pow(zoom.toDouble()) / 360.0
97     val cellDeg = CLUSTER_THRESHOLD_PX / pixelsPerDeg
98     val grid = LinkedHashMap<String, MutableList<NearbyCourseItem>>()
99     courses.forEach { course ->
100         val key = "${floor(course.startPoint.latitude / cellDeg).toLong()}, " +
101             "${floor(course.startPoint.longitude / cellDeg).toLong()}"
102         grid.getOrPut(key) { mutableListOf() }.add(course)
103     }
104     return grid.entries.map { (key, list) ->
105         CourseCluster(
106             id = key,
107             courses = list,
108             center = LatLng(
109                 list.sumOf { it.startPoint.latitude } / list.size,
110                 list.sumOf { it.startPoint.longitude } / list.size,
111             ),
112         )
113     }
114 }
115
116 @SuppressLint("MissingPermission")
117 @Composable
118 fun DiscoverCourseMap(
119     courses: List<NearbyCourseItem>,
120     isLoading: Boolean,
121     errorMessage: String?,
122     currentLocation: MapPoint?,
123     onCourseClick: (String) -> Unit,
124     modifier: Modifier = Modifier,
125 ) {
126     val context = LocalContext.current
127     val isDark = LocalIsDarkTheme.current
128     val mapStyleOptions = remember(isDark) {
129         val styleRes = if (isDark) R.raw.map_style_discover_dark else R.raw.map_style_light
130         runCatching { MapStyleOptions.loadRawResourceStyle(context, styleRes) }.getOrNull()
131     }
132     val cameraPositionState = rememberCameraPositionState {
133         position = CameraPosition.fromLatLngZoom(KOREA_CENTER, 6.55f)
134     }
135     val coroutineScope = rememberCoroutineScope()
136
137     // 단일 코스 선택 상태 (기존 말풍선)
138     var selectedSingle by remember { mutableStateOf<NearbyCourseItem?>(null) }
139     // 클러스터 선택 상태 → 하단 패널
140     var selectedCluster by remember { mutableStateOf<CourseCluster?>(null) }
141
142     val primaryArgb = MaterialTheme.colorScheme.primary.toArgb()
143     val surfaceArgb = MaterialTheme.colorScheme.surface.toArgb()
144     val primaryColor = MaterialTheme.colorScheme.primary
145
146     val markerIcons = remember(primaryArgb, surfaceArgb) {
147         SportMarkerIcons(
148             normal = createSportMarkerIcon(context, primaryArgb, surfaceArgb, selected = false),
149             selected = createSportMarkerIcon(context, primaryArgb, surfaceArgb, selected = true),
150         )
151     }
152
153     // 줌 레벨 변화 감지
154     val zoom by remember { derivedStateOf { cameraPositionState.position.zoom } }
155
156     // 줌 레벨에 따라 클러스터 재계산
157     val clusters by remember(courses, zoom) {
158         derivedStateOf { clusterCourses(courses, zoom) }
159     }
160
161     // 카메라 이동·줌 시 클러스터 패널 닫기
162     LaunchedEffect(zoom) {
163         selectedCluster = null
164         selectedSingle = null
165     }
166
167     LaunchedEffect(Unit) {
168         cameraPositionState.move(CameraUpdateFactory.newLatLngZoom(KOREA_CENTER, 6.55f))
169     }

```

```

170
171     Box(
172         modifier = modifier
173         .fillMaxSize()
174         .background(MaterialTheme.colorScheme.background),
175     ) {
176         GoogleMap(
177             modifier = Modifier.fillMaxSize(),
178             cameraPositionState = cameraPositionState,
179             properties = MapProperties(
180                 isMyLocationEnabled = currentLocation != null,
181                 mapStyleOptions = mapStyleOptions,
182             ),
183             uiSettings = MapUiSettings(
184                 zoomControlsEnabled = false,
185                 scrollGesturesEnabled = true,
186                 zoomGesturesEnabled = true,
187                 rotationGesturesEnabled = true,
188                 tiltGesturesEnabled = false,
189                 compassEnabled = true,
190                 mapToolbarEnabled = false,
191                 myLocationButtonEnabled = false,
192             ),
193             onMapClick = {
194                 selectedSingle = null
195                 selectedCluster = null
196             },
197         ) {
198             clusters.forEach { cluster ->
199                 if (cluster.courses.size == 1) {
200                     // — 단일 마커: 기존 맵풍선 —————
201                     val course = cluster.courses.first()
202                     val selected = course.courseId == selectedSingle?.courseId
203                     val state = remember(course.courseId) {
204                         MarkerState(position = cluster.center)
205                     }
206                     MarkerInfoWindow(
207                         state = state,
208                         anchor = Offset(0.5f, 0.92f),
209                         infoWindowAnchor = Offset(0.5f, 0.05f),
210                         icon = if (selected) markerIcons.selected else markerIcons.normal,
211                         zIndex = if (selected) 2f else 1f,
212                         onClick = { marker ->
213                             selectedSingle = course
214                             selectedCluster = null
215                             marker.showInfoWindow()
216                             true
217                         },
218                         onInfoWindowClick = { onCourseClick(course.courseId) },
219                     ) {
220                         CourseInfoBubble(course = course)
221                     }
222                 } else {
223                     // — 클러스터 마커: 숫자 뱃지 —————
224                     val isSelected = cluster.id == selectedCluster?.id
225                     val icon = remember(primaryArgb, surfaceArgb, cluster.courses.size, isSelected) {
226                         createClusterIcon(context, primaryArgb, surfaceArgb, cluster.courses.size, isSelected)
227                     }
228                     val state = remember(cluster.id) {
229                         MarkerState(position = cluster.center)
230                     }
231                     Marker(
232                         state = state,
233                         anchor = Offset(0.5f, 0.5f),
234                         icon = icon,
235                         zIndex = if (isSelected) 3f else 2f,
236                         onClick = {
237                             selectedCluster = if (isSelected) null else cluster
238                             selectedSingle = null
239                             true
240                         },
241                     )
242                 }
243             }
244         }
245
246         // — 코스 개수 칩 —————
247         Surface(
248             modifier = Modifier
249                 .align(Alignment.TopStart)
250                 .padding(12.dp),
251             shape = CircleShape,
252             color = MaterialTheme.colorScheme.surface.copy(alpha = 0.94f),
253             border = BorderStroke(1.dp, MaterialTheme.colorScheme.outlineVariant),
254             shadowElevation = 3.dp,
255         ) {

```

```

256         Row(
257             modifier = Modifier.padding(horizontal = 12.dp, vertical = 8.dp),
258             verticalAlignment = Alignment.CenterVertically,
259             horizontalArrangement = Arrangement.spacedBy(6.dp),
260         ) {
261             Icon(
262                 imageVector = Icons.Filled.LocationOn,
263                 contentDescription = null,
264                 tint = primaryColor,
265             )
266             Text(
267                 text = "전국 ${courses.size}개 코스",
268                 style = MaterialTheme.typography.labelLarge,
269                 color = MaterialTheme.colorScheme.onSurface,
270             )
271         }
272     }
273
274     // — 내 위치 버튼 —————
275     if (currentLocation != null) {
276         Surface(
277             onClick = {
278                 coroutineScope.launch {
279                     cameraPositionState.animate(
280                         CameraUpdateFactory.newLatLngZoom(
281                             LatLng(currentLocation.latitude, currentLocation.longitude),
282                             15f,
283                         ),
284                         durationMs = 700,
285                     )
286                 }
287             },
288             modifier = Modifier
289                 .align(Alignment.TopEnd)
290                 .padding(12.dp),
291             shape = CircleShape,
292             color = MaterialTheme.colorScheme.surface.copy(alpha = 0.96f),
293             border = BorderStroke(1.dp, MaterialTheme.colorScheme.outlineVariant),
294             shadowElevation = 4.dp,
295         ) {
296             Icon(
297                 imageVector = Icons.Filled.MyLocation,
298                 contentDescription = "내 위치로 이동",
299                 tint = primaryColor,
300                 modifier = Modifier.padding(12.dp),
301             )
302         }
303     }
304
305     // — 로딩 / 에러 —————
306     when {
307         isLoading -> {
308             Surface(
309                 modifier = Modifier.align(Alignment.Center),
310                 shape = CircleShape,
311                 color = MaterialTheme.colorScheme.surface.copy(alpha = 0.94f),
312                 shadowElevation = 4.dp,
313             ) {
314                 CircularProgressIndicator(
315                     modifier = Modifier.padding(18.dp),
316                     color = primaryColor,
317                 )
318             }
319         }
320         errorMessage != null -> {
321             Surface(
322                 modifier = Modifier
323                     .align(Alignment.Center)
324                     .padding(horizontal = 32.dp),
325                 shape = MaterialTheme.shapes.medium,
326                 color = MaterialTheme.colorScheme.surface.copy(alpha = 0.96f),
327                 border = BorderStroke(1.dp, MaterialTheme.colorScheme.outlineVariant),
328             ) {
329                 Text(
330                     text = errorMessage,
331                     style = MaterialTheme.typography.bodyMedium,
332                     color = MaterialTheme.colorScheme.onSurface,
333                     modifier = Modifier.padding(horizontal = 18.dp, vertical = 14.dp),
334                 )
335             }
336         }
337     }
338
339     // — 클러스터 하단 패널 —————
340     AnimatedVisibility(
341         visible = selectedCluster != null,

```

```

342         modifier = Modifier.align(Alignment.BottomCenter),
343         enter = slideInVertically(initialOffsetY = { it } ),
344         exit = slideOutVertically(targetOffsetY = { it } ),
345     ) {
346         selectedCluster?.let { cluster ->
347             ClusterPanel(
348                 cluster = cluster,
349                 onCourseClick = onCourseClick,
350                 onDismiss = { selectedCluster = null },
351             )
352         }
353     }
354 }
355 }
356
357 //
358 // 클러스터 하단 패널
359 //
360
361 @Composable
362 private fun ClusterPanel(
363     cluster: CourseCluster,
364     onCourseClick: (String) -> Unit,
365     onDismiss: () -> Unit,
366 ) {
367     Surface(
368         modifier = Modifier
369             .fillMaxWidth()
370             .padding(horizontal = 8.dp, vertical = 8.dp),
371         shape = MaterialTheme.shapes.large,
372         color = MaterialTheme.colorScheme.surface.copy(alpha = 0.97f),
373         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outlineVariant),
374         shadowElevation = 8.dp,
375     ) {
376         Column(modifier = Modifier.padding(top = 12.dp, bottom = 16.dp)) {
377             // 헤더
378             Row(
379                 modifier = Modifier
380                     .fillMaxWidth()
381                     .padding(horizontal = 16.dp),
382                 verticalAlignment = Alignment.CenterVertically,
383             ) {
384                 Text(
385                     text = "이 위치에 ${cluster.courses.size}개 코스",
386                     style = MaterialTheme.typography.titleSmall,
387                     fontWeight = FontWeight.Bold,
388                     color = MaterialTheme.colorScheme.onSurface,
389                     modifier = Modifier.weight(1f),
390                 )
391                 IconButton(
392                     onClick = onDismiss,
393                     modifier = Modifier.size(28.dp),
394                 ) {
395                     Icon(
396                         imageVector = Icons.Filled.Close,
397                         contentDescription = "닫기",
398                         tint = MaterialTheme.colorScheme.onSurfaceVariant,
399                         modifier = Modifier.size(18.dp),
400                     )
401                 }
402             }
403
404             Spacer(Modifier.height(8.dp))
405
406             // 가로 스크롤 코스 카드 목록
407             LazyRow(
408                 contentPadding = PaddingValues(horizontal = 12.dp),
409                 horizontalArrangement = Arrangement.spacedBy(10.dp),
410             ) {
411                 items(cluster.courses, key = { it.courseId }) { course ->
412                     ClusterCourseCard(
413                         course = course,
414                         onClick = { onCourseClick(course.courseId) },
415                     )
416                 }
417             }
418         }
419     }
420 }
421
422 @Composable
423 private fun ClusterCourseCard(
424     course: NearbyCourseItem,
425     onClick: () -> Unit,
426 ) {
427     Surface(

```

```

428         onClick = onClick,
429         modifier = Modifier.widthIn(min = 190.dp, max = 240.dp),
430         shape = MaterialTheme.shapes.medium,
431         color = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.6f),
432         border = BorderStroke(1.dp, MaterialTheme.colorScheme.primary.copy(alpha = 0.45f)),
433         shadowElevation = 2.dp,
434     ) {
435         Column(
436             modifier = Modifier.padding(horizontal = 14.dp, vertical = 12.dp),
437             verticalArrangement = Arrangement.spacedBy(5.dp),
438         ) {
439             Row(
440                 verticalAlignment = Alignment.CenterVertically,
441                 horizontalArrangement = Arrangement.spacedBy(6.dp),
442             ) {
443                 Icon(
444                     imageVector = Icons.Filled.Route,
445                     contentDescription = null,
446                     tint = MaterialTheme.colorScheme.primary,
447                     modifier = Modifier.size(16.dp),
448                 )
449                 Text(
450                     text = course.name,
451                     style = MaterialTheme.typography.titleSmall,
452                     fontWeight = FontWeight.Bold,
453                     color = MaterialTheme.colorScheme.onSurface,
454                     maxLines = 1,
455                     overflow = TextOverflow.Ellipsis,
456                 )
457             }
458             Text(
459                 text = "${formatCourseDistance(course.distanceMeters)} · " +
460                     "${if (course.isLoop) "루프" else "일반"} · 완주 ${course.completionCount}회",
461                 style = MaterialTheme.typography.bodySmall,
462                 color = MaterialTheme.colorScheme.onSurfaceVariant,
463                 maxLines = 1,
464                 overflow = TextOverflow.Ellipsis,
465             )
466             Text(
467                 text = "탭하여 상세 보기 →",
468                 style = MaterialTheme.typography.labelSmall,
469                 color = MaterialTheme.colorScheme.primary,
470             )
471         }
472     }
473 }
474
475 //
476 // 단일 마커 말풍선
477 //
478
479 @Composable
480 private fun CourseInfoBubble(course: NearbyCourseItem) {
481     val bubbleColor = MaterialTheme.colorScheme.surface
482     Column(horizontalAlignment = Alignment.CenterHorizontally) {
483         Surface(
484             modifier = Modifier.widthIn(min = 180.dp, max = 250.dp),
485             shape = MaterialTheme.shapes.small,
486             color = MaterialTheme.colorScheme.surface,
487             border = BorderStroke(1.dp, MaterialTheme.colorScheme.primary.copy(alpha = 0.55f)),
488             shadowElevation = 6.dp,
489         ) {
490             Column(
491                 modifier = Modifier.padding(horizontal = 14.dp, vertical = 12.dp),
492                 verticalArrangement = Arrangement.spacedBy(5.dp),
493             ) {
494                 Row(
495                     verticalAlignment = Alignment.CenterVertically,
496                     horizontalArrangement = Arrangement.spacedBy(7.dp),
497                 ) {
498                     Icon(
499                         imageVector = Icons.Filled.Route,
500                         contentDescription = null,
501                         tint = MaterialTheme.colorScheme.primary,
502                     )
503                     Text(
504                         text = course.name,
505                         style = MaterialTheme.typography.titleSmall,
506                         fontWeight = FontWeight.Bold,
507                         color = MaterialTheme.colorScheme.onSurface,
508                         maxLines = 1,
509                         overflow = TextOverflow.Ellipsis,
510                     )
511                 }
512                 Text(
513                     text = "${formatCourseDistance(course.distanceMeters)} · " +

```

```

514         "${if (course.isLoop) "루프 코스" else "일반 코스"} · 완주 ${course.completionCount}",
515         style = MaterialTheme.typography.bodySmall,
516         color = MaterialTheme.colorScheme.onSurfaceVariant,
517         maxLines = 1,
518         overflow = TextOverflow.Ellipsis,
519     )
520     Text(
521         text = "둘러서 코스 상세 보기",
522         style = MaterialTheme.typography.labelSmall,
523         color = MaterialTheme.colorScheme.primary,
524     )
525 }
526 }
527 Canvas(
528     modifier = Modifier
529         .widthIn(min = 18.dp, max = 18.dp)
530         .height(9.dp),
531 ) {
532     val path = androidx.compose.ui.graphics.Path().apply {
533         moveTo(0f, 0f)
534         lineTo(size.width, 0f)
535         lineTo(size.width / 2f, size.height)
536         close()
537     }
538     drawPath(path, color = bubbleColor)
539 }
540 }
541 }
542
543 // -----
544 // 아이콘 생성
545 // -----
546
547 private data class SportMarkerIcons(
548     val normal: BitmapDescriptor,
549     val selected: BitmapDescriptor,
550 )
551
552 /** 클러스터 마커: 기존 핀 + 우하단 숫자 뱃지 */
553 private fun createClusterIcon(
554     context: Context,
555     primaryColor: Int,
556     surfaceColor: Int,
557     count: Int,
558     selected: Boolean,
559 ): BitmapDescriptor {
560     val density = context.resources.displayMetrics.density
561
562     // 핀 영역은 기존 single 마커와 동일한 크기
563     val pinW = ((if (selected) 33 else 29) * density).toInt()
564     val pinH = ((if (selected) 37 else 33) * density).toInt()
565
566     // 뱃지 크기 (핀 우하단에 겹쳐 붙음)
567     val badgeR = 12f * density // 뱃지 반지름
568     val badgePad = 1.5f * density // 핀 경계 바깥쪽 여백
569
570     // 전체 비트맵: 핀 + 뱃지가 우하단으로 빠져나오는 만큼 확장
571     val totalW = pinW + (badgeR + badgePad).toInt()
572     val totalH = pinH + (badgeR + badgePad).toInt()
573
574     val bitmap = Bitmap.createBitmap(totalW, totalH, Bitmap.Config.ARGB_8888)
575     val canvas = AndroidCanvas(bitmap)
576
577     // — 핀 그리기 (기존 createSportMarkerIcon 로직 그대로) —
578     val logo = BitmapFactory.decodeResource(context.resources, R.mipmap.ic_launcher_foreground)
579     val shadowPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = 0x59000000 }
580     val borderPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
581         color = if (selected) surfaceColor else 0xE6FFFFFF.toInt()
582         style = Paint.Style.STROKE
583         strokeWidth = (if (selected) 2.5f else 1.3f) * density
584     }
585     val tailPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = primaryColor }
586
587     val circleDiameter = pinW * 0.88f
588     val circleLeft = (pinW - circleDiameter) / 2f
589     val circleTop = density
590     val circleRect = RectF(circleLeft, circleTop, circleLeft + circleDiameter, circleTop + circleDiameter)
591
592     val tail = Path().apply {
593         moveTo(pinW * 0.35f, circleRect.bottom - density)
594         lineTo(pinW * 0.65f, circleRect.bottom - density)
595         lineTo(pinW * 0.5f, pinH - density)
596         close()
597     }
598     canvas.drawPath(tail, shadowPaint)
599     canvas.drawPath(tail, tailPaint)

```

```

600 canvas.drawCircle(
601     circleRect.centerX() + density,
602     circleRect.centerY() + 1.8f * density,
603     circleDiameter / 2f,
604     shadowPaint,
605 )
606 val clipPath = Path().apply { addOval(circleRect, Path.Direction.CW) }
607 canvas.save()
608 canvas.clipPath(clipPath)
609 canvas.drawBitmap(
610     logo,
611     Rect(0, 0, logo.width, logo.height),
612     circleRect,
613     Paint(Paint.ANTI_ALIAS_FLAG or Paint.FILTER_BITMAP_FLAG),
614 )
615 canvas.restore()
616 canvas.drawOval(circleRect, borderPaint)
617 if (selected) {
618     val accentPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
619         color = primaryColor
620         style = Paint.Style.STROKE
621         strokeWidth = 1.2f * density
622     }
623     canvas.drawOval(
624         RectF(
625             circleRect.left - density,
626             circleRect.top - density,
627             circleRect.right + density,
628             circleRect.bottom + density,
629         ),
630         accentPaint,
631     )
632 }
633
634 // — 숫자 뱃지 (핀 원의 우하단 모서리에 겹쳐서) —————
635 // 원 중심 기준 45° 방향 끝 지점
636 val circleCx = circleRect.centerX()
637 val circleCy = circleRect.centerY()
638 val edgeFraction = 0.707f // cos(45°)
639 val badgeCx = circleCx + circleDiameter / 2f * edgeFraction
640 val badgeCy = circleCy + circleDiameter / 2f * edgeFraction
641
642 // 흰 테두리 (겹쳐 보이는 부분 가리개)
643 val badgeBorder = Paint(Paint.ANTI_ALIAS_FLAG).apply {
644     color = 0xFFFFFFFF.toInt()
645     style = Paint.Style.FILL
646 }
647 canvas.drawCircle(badgeCx, badgeCy, badgeR + 1.8f * density, badgeBorder)
648
649 // 뱃지 배경
650 val badgeFill = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = primaryColor }
651 canvas.drawCircle(badgeCx, badgeCy, badgeR, badgeFill)
652
653 // 숫자
654 val textPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
655     color = 0xFFFFFFFF.toInt()
656     textSize = (if (count >= 10) 11f else 13f) * density
657     textAlign = Paint.Align.CENTER
658     typeface = Typeface.create(Typeface.DEFAULT, Typeface.BOLD)
659 }
660 val textY = badgeCy - (textPaint.descent() + textPaint.ascent()) / 2f
661 canvas.drawText(count.toString(), badgeCx, textY, textPaint)
662
663 return BitmapDescriptorFactory.fromBitmap(bitmap)
664 }
665
666 /** 단일 코스 핀 마커 */
667 private fun createSportMarkerIcon(
668     context: Context,
669     primaryColor: Int,
670     surfaceColor: Int,
671     selected: Boolean,
672 ): BitmapDescriptor {
673     val density = context.resources.displayMetrics.density
674     val width = ((if (selected) 33 else 29) * density).toInt()
675     val height = ((if (selected) 37 else 33) * density).toInt()
676     val bitmap = Bitmap.createBitmap(width, height, Bitmap.Config.ARGB_8888)
677     val canvas = AndroidCanvas(bitmap)
678     val logo = BitmapFactory.decodeResource(context.resources, R.mipmap.ic_launcher_foreground)
679
680     val shadowPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = 0x59000000 }
681     val borderPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
682         color = if (selected) surfaceColor else 0xE6FFFFFF.toInt()
683         style = Paint.Style.STROKE
684         strokeWidth = (if (selected) 2.5f else 1.3f) * density
685     }

```



```

686     val tailPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = primaryColor }
687
688     val circleDiameter = width * 0.88f
689     val circleLeft = (width - circleDiameter) / 2f
690     val circleTop = density
691     val circleRect = RectF(circleLeft, circleTop, circleLeft + circleDiameter, circleTop + circleDiameter)
692
693     val tail = Path().apply {
694         moveTo(width * 0.35f, circleRect.bottom - density)
695         lineTo(width * 0.65f, circleRect.bottom - density)
696         lineTo(width * 0.5f, height - density)
697         close()
698     }
699     canvas.drawPath(tail, shadowPaint)
700     canvas.drawPath(tail, tailPaint)
701
702     canvas.drawCircle(
703         circleRect.centerX() + density,
704         circleRect.centerY() + 1.8f * density,
705         circleDiameter / 2f,
706         shadowPaint,
707     )
708
709     val clipPath = Path().apply { addOval(circleRect, Path.Direction.CW) }
710     canvas.save()
711     canvas.clipPath(clipPath)
712     canvas.drawBitmap(
713         logo,
714         Rect(0, 0, logo.width, logo.height),
715         circleRect,
716         Paint(Paint.ANTI_ALIAS_FLAG or Paint.FILTER_BITMAP_FLAG),
717     )
718     canvas.restore()
719
720     canvas.drawOval(circleRect, borderPaint)
721     if (selected) {
722         val accentPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
723             color = primaryColor
724             style = Paint.Style.STROKE
725             strokeWidth = 1.2f * density
726         }
727         canvas.drawOval(
728             RectF(
729                 circleRect.left - density,
730                 circleRect.top - density,
731                 circleRect.right + density,
732                 circleRect.bottom + density,
733             ),
734             accentPaint,
735         )
736     }
737
738     return BitmapDescriptorFactory.fromBitmap(bitmap)
739 }
740
741 private fun formatCourseDistance(distanceMeters: Double): String =
742     if (distanceMeters < 1000) "${distanceMeters.toInt()}m"
743     else "%.1fkm".format(distanceMeters / 1000.0)
744
745 private val KOREA_CENTER = LatLng(36.35, 127.8)

```

```

1 package com.runway.android.ui.discover
2
3 import android.Manifest
4 import android.content.pm.PackageManager
5 import androidx.activity.compose.rememberLauncherForActivityResult
6 import androidx.activity.result.contract.ActivityResultContracts
7 import androidx.compose.animation.AnimatedVisibility
8 import androidx.compose.animation.core.tween
9 import androidx.compose.animation.fadeIn
10 import androidx.compose.animation.slideInVertically
11 import androidx.compose.foundation.background
12 import androidx.compose.foundation.layout.Arrangement
13 import androidx.compose.foundation.layout.Box
14 import androidx.compose.foundation.layout.Column
15 import androidx.compose.foundation.layout.PaddingValues
16 import androidx.compose.foundation.layout.Row
17 import androidx.compose.foundation.layout.Spacer
18 import androidx.compose.foundation.layout.fillMaxSize
19 import androidx.compose.foundation.layout.fillMaxWidth
20 import androidx.compose.foundation.layout.height
21 import androidx.compose.foundation.layout.navigationBarsPadding
22 import androidx.compose.foundation.layout.padding
23 import androidx.compose.foundation.layout.size
24 import androidx.compose.foundation.layout.width
25 import androidx.compose.foundation.lazy.LazyColumn
26 import androidx.compose.foundation.lazy.LazyRow
27 import androidx.compose.foundation.lazy.items
28 import androidx.compose.foundation.lazy.itemsIndexed
29 import androidx.compose.foundation.text.KeyboardActions
30 import androidx.compose.foundation.text.KeyboardOptions
31 import androidx.compose.material.icons.Icons
32 import androidx.compose.material.icons.automirrored.filled.List
33 import androidx.compose.material.icons.filled.Close
34 import androidx.compose.material.icons.filled.FilterList
35 import androidx.compose.material.icons.filled.Map
36 import androidx.compose.material.icons.filled.Refresh
37 import androidx.compose.material.icons.filled.Search
38 import androidx.compose.material3.Badge
39 import androidx.compose.material3.BadgedBox
40 import androidx.compose.material3.CircularProgressIndicator
41 import androidx.compose.material3.ExperimentalMaterial3Api
42 import androidx.compose.material3.HorizontalDivider
43 import androidx.compose.material3.Icon
44 import androidx.compose.material3.IconButton
45 import androidx.compose.material3.MaterialTheme
46 import androidx.compose.material3.ModalBottomSheet
47 import androidx.compose.material3.Surface
48 import androidx.compose.material3.Text
49 import androidx.compose.material3.TextField
50 import androidx.compose.material3.TextFieldDefaults
51 import androidx.compose.material3.pulltorefresh.PullToRefreshBox
52 import androidx.compose.material3.rememberModalBottomSheetState
53 import androidx.compose.runtime.Composable
54 import androidx.compose.runtime.LaunchedEffect
55 import androidx.compose.runtime.getValue
56 import androidx.compose.runtime.mutableStateOf
57 import androidx.compose.runtime.remember
58 import androidx.compose.runtime.setValue
59 import androidx.compose.ui.Alignment
60 import androidx.compose.ui.Modifier
61 import androidx.compose.ui.graphics.Color
62 import androidx.compose.ui.platform.LocalContext
63 import androidx.compose.ui.platform.LocalSoftwareKeyboardController
64 import androidx.compose.ui.text.font.FontWeight
65 import androidx.compose.ui.text.input.ImeAction
66 import androidx.compose.ui.text.style.TextAlign
67 import androidx.compose.ui.unit.dp
68 import androidx.core.content.ContextCompat
69 import androidx.hilt.navigation.compose.hiltViewModel
70 import com.runway.android.ui.components.DiscoverCourseCard
71 import kotlinx.coroutines.delay
72
73 @OptIn(ExperimentalMaterial3Api::class)
74 @Composable
75 fun DiscoverScreen(
76     onNavigateToCourseDetail: (String) -> Unit = {},
77     viewModel: DiscoverViewModel = hiltViewModel(),
78 ) {
79     val context = LocalContext.current
80     val keyboardController = LocalSoftwareKeyboardController.current
81     var showFilterSheet by remember { mutableStateOf(false) }
82
83     val permissionLauncher = rememberLauncherForActivityResult(

```

```

84     ActivityResultContracts.RequestMultiplePermissions()
85 ) { permissions ->
86     val granted = permissions[Manifest.permission.ACCESS_FINE_LOCATION] == true ||
87         permissions[Manifest.permission.ACCESS_COARSE_LOCATION] == true
88     if (granted) viewModel.onLocationPermissionGranted() else viewModel.onLocationPermissionDenied()
89 }
90
91 LaunchedEffect(Unit) {
92     val hasFine = ContextCompat.checkSelfPermission(
93         context, Manifest.permission.ACCESS_FINE_LOCATION
94     ) == PackageManager.PERMISSION_GRANTED
95     val hasCoarse = ContextCompat.checkSelfPermission(
96         context, Manifest.permission.ACCESS_COARSE_LOCATION
97     ) == PackageManager.PERMISSION_GRANTED
98     if (hasFine || hasCoarse) viewModel.onLocationPermissionGranted()
99     else permissionLauncher.launch(
100         arrayOf(Manifest.permission.ACCESS_FINE_LOCATION, Manifest.permission.ACCESS_COARSE_LOCATION)
101     )
102 }
103
104 val onRefresh: () -> Unit = if (
105     viewModel.viewMode == DiscoverViewMode.LIST && viewModel.isLocationRequired
106 ) {
107     { permissionLauncher.launch(arrayOf(Manifest.permission.ACCESS_FINE_LOCATION, Manifest.permission.ACCESS_COARSE_LOCATION)) }
108 } else viewModel::refresh
109
110 val radiusLabel = when (viewModel.radiusMeters) {
111     1000 -> "1km"; 5000 -> "5km"; else -> "3km"
112 }
113
114 // Keep the map outside LazyColumn so map gestures never scroll the whole screen.
115 if (viewModel.viewMode == DiscoverViewMode.MAP) {
116     Column(
117         modifier = Modifier
118             .fillMaxSize()
119             .background(MaterialTheme.colorScheme.background),
120     ) {
121         Row(
122             modifier = Modifier
123                 .fillMaxWidth()
124                 .padding(horizontal = 20.dp)
125                 .padding(top = 16.dp, bottom = 4.dp),
126             horizontalArrangement = Arrangement.SpaceBetween,
127             verticalAlignment = Alignment.CenterVertically,
128         ) {
129             Column(modifier = Modifier.weight(1f)) {
130                 Text(
131                     text = "코스 탐색",
132                     style = MaterialTheme.typography.headlineLarge,
133                     color = MaterialTheme.colorScheme.onBackground,
134                 )
135                 Text(
136                     text = "전국 공개 코스를 지도에서 확인하세요",
137                     style = MaterialTheme.typography.bodyMedium,
138                     color = MaterialTheme.colorScheme.onSurfaceVariant,
139                 )
140             }
141             IconButton(onClick = onRefresh) {
142                 Icon(Icons.Filled.Refresh, "새로고침", tint = MaterialTheme.colorScheme.onSurfaceVariant)
143             }
144             BadgedBox(
145                 badge = {
146                     if (viewModel.activeFilterCount > 0) Badge { Text("${viewModel.activeFilterCount}") }
147                 }
148             ) {
149                 IconButton(onClick = { showFilterSheet = true }) {
150                     Icon(
151                         imageVector = Icons.Filled.FilterList,
152                         contentDescription = "필터",
153                         tint = if (viewModel.activeFilterCount > 0)
154                             MaterialTheme.colorScheme.primary
155                         else MaterialTheme.colorScheme.onSurfaceVariant,
156                     )
157                 }
158             }
159         }
160     }
161     Surface(
162         modifier = Modifier
163             .fillMaxWidth()
164             .padding(horizontal = 20.dp),
165         shape = MaterialTheme.shapes.extraLarge,
166         color = MaterialTheme.colorScheme.surface,
167         tonalElevation = 1.dp,
168     ) {
169         TextField(

```

```

170         value = viewModel.keyword,
171         onValueChange = viewModel::onKeywordChange,
172         modifier = Modifier.fillMaxWidth(),
173         placeholder = {
174             Text("코스 이름으로 검색", style = MaterialTheme.typography.bodyLarge,
175                 color = MaterialTheme.colorScheme.onSurfaceVariant)
176         },
177         leadingIcon = {
178             Icon(Icons.Filled.Search, null,
179                 tint = MaterialTheme.colorScheme.onSurfaceVariant,
180                 modifier = Modifier.size(18.dp))
181         },
182         trailingIcon = if (viewModel.keyword.isNotEmpty()) {
183             { IconButton(onClick = viewModel::clearKeyword) {
184                 Icon(Icons.Filled.Close, "검색어 지우기", tint = MaterialTheme.colorScheme.onSurfaceVariant)
185             }}
186         } else null,
187         keyboardOptions = KeyboardOptions(imeAction = ImeAction.Search),
188         keyboardActions = KeyboardActions(onSearch = { keyboardController?.hide(); viewModel.onSearch() }),
189         singleLine = true,
190         textStyle = MaterialTheme.typography.bodyLarge,
191         colors = TextFieldDefaults.colors(
192             focusedContainerColor = Color.Transparent,
193             unfocusedContainerColor = Color.Transparent,
194             focusedIndicatorColor = Color.Transparent,
195             unfocusedIndicatorColor = Color.Transparent,
196         ),
197     )
198 }
199
200 DiscoverViewModelSelector(
201     selectedMode = viewModel.viewMode,
202     onModeSelected = viewModel::onViewModeChange,
203     modifier = Modifier
204         .fillMaxWidth()
205         .padding(horizontal = 20.dp)
206         .padding(top = 12.dp),
207 )
208
209 Spacer(Modifier.height(12.dp))
210
211 DiscoverCourseMap(
212     courses = viewModel.mapCourses,
213     isLoading = viewModel.isMapLoading,
214     errorMessage = viewModel.mapErrorMessage,
215     currentLocation = viewModel.currentLocation,
216     onCourseClick = onNavigateToCourseDetail,
217     modifier = Modifier
218         .fillMaxWidth()
219         .weight(1f),
220 )
221 }
222
223 if (showFilterSheet) {
224     val sheetState = rememberModalBottomSheetState(skipPartiallyExpanded = true)
225     ModalBottomSheet(
226         onDismissRequest = { showFilterSheet = false },
227         sheetState = sheetState,
228         containerColor = MaterialTheme.colorScheme.surface,
229     ) {
230         FilterSheetContent(viewModel = viewModel, onDismiss = { showFilterSheet = false })
231     }
232 }
233 return
234 }
235
236 PullToRefreshBox(
237     isRefreshing = viewModel.isRefreshing,
238     onRefresh = onRefresh,
239     modifier = Modifier.fillMaxSize(),
240 ) {
241     LazyColumn(
242         modifier = Modifier
243             .fillMaxSize()
244             .background(MaterialTheme.colorScheme.background),
245         contentPadding = PaddingValues(bottom = 24.dp),
246     ) {
247         // — 헤더 —
248         item {
249             Row(
250                 modifier = Modifier
251                     .fillMaxWidth()
252                     .padding(horizontal = 20.dp)
253                     .padding(top = 16.dp, bottom = 4.dp),
254                 horizontalArrangement = Arrangement.SpaceBetween,
255                 verticalAlignment = Alignment.CenterVertically,
256             ) {

```

```

256         Column(modifier = Modifier.weight(1f)) {
257             Text(
258                 text = "코스 탐색",
259                 style = MaterialTheme.typography.headlineLarge,
260                 color = MaterialTheme.colorScheme.onBackground,
261             )
262             Text(
263                 text = if (viewModel.isLoading) "불러오는 중.."
264                     else "${viewModel.courses.size}개 코스 · 반경 $radiusLabel",
265                 style = MaterialTheme.typography.bodyMedium,
266                 color = MaterialTheme.colorScheme.onSurfaceVariant,
267             )
268         }
269         IconButton(onClick = onRefresh) {
270             Icon(Icons.Filled.Refresh, "새로고침", tint = MaterialTheme.colorScheme.onSurfaceVariant)
271         }
272         BadgedBox(
273             badge = {
274                 if (viewModel.activeFilterCount > 0) {
275                     Badge { Text("${viewModel.activeFilterCount}") }
276                 }
277             }
278         ) {
279             IconButton(onClick = { showFilterSheet = true }) {
280                 Icon(
281                     imageVector = Icons.Filled.FilterList,
282                     contentDescription = "필터",
283                     tint = if (viewModel.activeFilterCount > 0)
284                         MaterialTheme.colorScheme.primary
285                     else MaterialTheme.colorScheme.onSurfaceVariant,
286                 )
287             }
288         }
289     }
290 }
291
292 // — 검색바 —
293 item {
294     Surface(
295         modifier = Modifier
296             .fillMaxWidth()
297             .padding(horizontal = 20.dp),
298         shape = MaterialTheme.shapes.extraLarge,
299         color = MaterialTheme.colorScheme.surface,
300         tonalElevation = 1.dp,
301     ) {
302         TextField(
303             value = viewModel.keyword,
304             onValueChange = viewModel::onKeywordChange,
305             modifier = Modifier.fillMaxWidth(),
306             placeholder = {
307                 Text("코스 이름으로 검색", style = MaterialTheme.typography.bodyLarge,
308                     color = MaterialTheme.colorScheme.onSurfaceVariant)
309             },
310             leadingIcon = {
311                 Icon(Icons.Filled.Search, null,
312                     tint = MaterialTheme.colorScheme.onSurfaceVariant,
313                     modifier = Modifier.size(18.dp))
314             },
315             trailingIcon = if (viewModel.keyword.isNotEmpty()) {
316                 {
317                     IconButton(onClick = viewModel::clearKeyword) {
318                         Icon(Icons.Filled.Close, "검색어 지우기",
319                             tint = MaterialTheme.colorScheme.onSurfaceVariant)
320                     }
321                 }
322             } else null,
323             keyboardOptions = KeyboardOptions(imeAction = ImeAction.Search),
324             keyboardActions = KeyboardActions(onSearch = {
325                 keyboardController?.hide()
326                 viewModel.onSearch()
327             }),
328             singleLine = true,
329             textStyle = MaterialTheme.typography.bodyLarge,
330             colors = TextFieldDefaults.colors(
331                 focusedContainerColor = Color.Transparent,
332                 unfocusedContainerColor = Color.Transparent,
333                 focusedIndicatorColor = Color.Transparent,
334                 unfocusedIndicatorColor = Color.Transparent,
335             ),
336         )
337     }
338 }
339
340 item {
341     DiscoverViewModeSelector(

```

```

342         selectedMode = viewModel.viewMode,
343         onModeSelected = viewModel::onViewModeChange,
344         modifier = Modifier
345             .fillMaxWidth()
346             .padding(horizontal = 20.dp)
347             .padding(top = 12.dp),
348     )
349 }
350
351 // — 활성 필터 요약 칩 —
352 if (viewModel.activeFilterCount > 0) {
353     item {
354         Spacer(modifier = Modifier.height(10.dp))
355         LazyRow(
356             contentPadding = PaddingValues(horizontal = 20.dp),
357             horizontalArrangement = Arrangement.spacedBy(8.dp),
358         ) {
359             if (viewModel.radiusMeters != 3000) {
360                 item {
361                     ActiveChip(label = "반경 $radiusLabel") {
362                         viewModel.onRadiusChange(3000)
363                     }
364                 }
365             }
366             viewModel.isLoopFilter?.let {
367                 item {
368                     ActiveChip(label = "루프코스") {
369                         viewModel.onIsLoopFilterChange(null)
370                     }
371                 }
372             }
373             if (viewModel.distanceFilter != DistanceFilterOption.ALL) {
374                 item {
375                     ActiveChip(label = viewModel.distanceFilter.label) {
376                         viewModel.onDistanceFilterChange(DistanceFilterOption.ALL)
377                     }
378                 }
379             }
380             if (viewModel.sortOption != CourseSortOption.NEAREST) {
381                 item {
382                     ActiveChip(label = viewModel.sortOption.label) {
383                         viewModel.onSortChange(CourseSortOption.NEAREST)
384                     }
385                 }
386             }
387             item {
388                 Surface(
389                     shape = MaterialTheme.shapes.extraLarge,
390                     color = Color.Transparent,
391                     onClick = viewModel::resetFilters,
392                 ) {
393                     Text(
394                         text = "초기화",
395                         style = MaterialTheme.typography.labelLarge,
396                         color = MaterialTheme.colorScheme.onSurfaceVariant,
397                         modifier = Modifier.padding(horizontal = 12.dp, vertical = 8.dp),
398                     )
399                 }
400             }
401         }
402     }
403 }
404
405 item { Spacer(modifier = Modifier.height(16.dp)) }
406
407 // — 코스 목록 —
408 when {
409     viewModel.isLoading -> item {
410         Box(
411             modifier = Modifier.fillMaxWidth().padding(top = 80.dp),
412             contentAlignment = Alignment.Center,
413         ) {
414             CircularProgressIndicator(color = MaterialTheme.colorScheme.primary)
415         }
416     }
417
418     viewModel.errorMessage != null -> item {
419         Column(
420             modifier = Modifier.fillMaxWidth().padding(top = 80.dp, start = 32.dp, end = 32.dp),
421             horizontalAlignment = Alignment.CenterHorizontally,
422             verticalArrangement = Arrangement.spacedBy(12.dp),
423         ) {
424             Text("코스를 불러오지 못했습니다", style = MaterialTheme.typography.titleMedium,
425                 color = MaterialTheme.colorScheme.onSurface, textAlign = TextAlign.Center)
426             Text(viewModel.errorMessage!!, style = MaterialTheme.typography.bodyMedium,
427                 color = MaterialTheme.colorScheme.onSurfaceVariant, textAlign = TextAlign.Center)

```

```

428         Surface(onClick = onRefresh, shape = MaterialTheme.shapes.extraLarge,
429             color = MaterialTheme.colorScheme.primary) {
430             Text(
431                 text = if (viewModel.isLocationRequired) "위치 권한 허용" else "다시 시도",
432                 style = MaterialTheme.typography.labelLarge,
433                 color = MaterialTheme.colorScheme.onPrimary,
434                 modifier = Modifier.padding(horizontal = 20.dp, vertical = 10.dp),
435             )
436         }
437     }
438 }
439
440 viewModel.courses.isEmpty() -> item {
441     val hasActiveFilter = viewModel.activeFilterCount > 0
442     Column(
443         modifier = Modifier.fillMaxWidth().padding(top = 80.dp),
444         horizontalAlignment = Alignment.CenterHorizontally,
445         verticalArrangement = Arrangement.spacedBy(8.dp),
446     ) {
447         Text(
448             text = if (viewModel.keyword.isNotBlank()) "검색 결과가 없습니다" else "주변에 코스가 없습니다",
449             style = MaterialTheme.typography.titleMedium,
450             color = MaterialTheme.colorScheme.onSurface,
451         )
452         Text(
453             text = when {
454                 viewModel.keyword.isNotBlank() -> "다른 검색어나 더 넓은 반경으로 시도해 보세요."
455                 hasActiveFilter -> "필터를 해제하거나 반경을 늘려 보세요."
456                 else -> "반경을 늘리거나 직접 코스를 만들어 보세요."
457             },
458             style = MaterialTheme.typography.bodyMedium,
459             color = MaterialTheme.colorScheme.onSurfaceVariant,
460         )
461     }
462 }
463
464 else -> itemsIndexed(viewModel.courses, key = { _, c -> c.courseId }) { index, course ->
465     var visible by remember { mutableStateOf(false) }
466     LaunchedEffect(course.courseId) {
467         delay(index.coerceAtMost(8) * 50L)
468         visible = true
469     }
470     AnimatedVisibility(
471         visible = visible,
472         enter = fadeIn(tween(250)) + slideInVertically(tween(250)) { it / 3 },
473     ) {
474         DiscoverCourseCard(
475             course = course,
476             modifier = Modifier
477                 .padding(horizontal = 20.dp)
478                 .padding(bottom = 12.dp),
479             onClick = { onNavigateToCourseDetail(course.courseId) },
480         )
481     }
482 }
483 }
484 }
485 }
486
487 // — 필터 바텀시트 —
488 if (showFilterSheet) {
489     val sheetState = rememberModalBottomSheetState(skipPartiallyExpanded = true)
490     ModalBottomSheet(
491         onDismissRequest = { showFilterSheet = false },
492         sheetState = sheetState,
493         containerColor = MaterialTheme.colorScheme.surface,
494     ) {
495         FilterSheetContent(
496             viewModel = viewModel,
497             onDismiss = { showFilterSheet = false },
498         )
499     }
500 }
501 }
502
503 @Composable
504 private fun DiscoverViewModelSelector(
505     selectedMode: DiscoverViewModel,
506     onModeSelected: (DiscoverViewModel) -> Unit,
507     modifier: Modifier = Modifier,
508 ) {
509     Surface(
510         modifier = modifier,
511         shape = MaterialTheme.shapes.medium,
512         color = MaterialTheme.colorScheme.surfaceVariant,
513     ) {

```

```

514     Row(modifier = Modifier.padding(4.dp)) {
515         listOf(
516             Triple(DiscoverViewMode.LIST, Icons.AutoMirrored.Filled.List, "목록"),
517             Triple(DiscoverViewMode.MAP, Icons.Filled.Map, "지도"),
518         ).forEach { (mode, icon, label) ->
519             val selected = selectedMode == mode
520             Surface(
521                 onClick = { onModeSelected(mode) },
522                 modifier = Modifier.weight(1f),
523                 shape = MaterialTheme.shapes.small,
524                 color = if (selected) {
525                     MaterialTheme.colorScheme.surface
526                 } else {
527                     Color.Transparent
528                 },
529                 shadowElevation = if (selected) 1.dp else 0.dp,
530             ) {
531                 Row(
532                     modifier = Modifier.padding(vertical = 10.dp),
533                     horizontalArrangement = Arrangement.Center,
534                     verticalAlignment = Alignment.CenterVertically,
535                 ) {
536                     Icon(
537                         imageVector = icon,
538                         contentDescription = null,
539                         tint = if (selected) {
540                             MaterialTheme.colorScheme.primary
541                         } else {
542                             MaterialTheme.colorScheme.onSurfaceVariant
543                         },
544                         modifier = Modifier.size(18.dp),
545                     )
546                     Spacer(modifier = Modifier.width(6.dp))
547                     Text(
548                         text = label,
549                         style = MaterialTheme.typography.labelLarge,
550                         fontWeight = if (selected) FontWeight.SemiBold else FontWeight.Normal,
551                         color = if (selected) {
552                             MaterialTheme.colorScheme.onSurface
553                         } else {
554                             MaterialTheme.colorScheme.onSurfaceVariant
555                         },
556                     )
557                 }
558             }
559         }
560     }
561 }
562
563 // — 필터 바텀시트 내용 —————
564
565 @Composable
566 private fun FilterSheetContent(
567     viewModel: DiscoverViewModel,
568     onDismiss: () -> Unit,
569 ) {
570     Column(
571         modifier = Modifier
572             .fillMaxWidth()
573             .navigationBarsPadding()
574             .padding(horizontal = 20.dp)
575             .padding(bottom = 24.dp),
576     ) {
577         // 헤더
578         Row(
579             modifier = Modifier.fillMaxWidth(),
580             horizontalArrangement = Arrangement.SpaceBetween,
581             verticalAlignment = Alignment.CenterVertically,
582         ) {
583             Text(
584                 text = "필터 & 정렬",
585                 style = MaterialTheme.typography.headlineSmall,
586                 fontWeight = FontWeight.Bold,
587                 color = MaterialTheme.colorScheme.onSurface,
588             )
589             Surface(
590                 onClick = {
591                     viewModel.resetFilters()
592                     onDismiss()
593                 },
594                 color = Color.Transparent,
595                 shape = MaterialTheme.shapes.extraLarge,
596             ) {
597                 Text(
598                     text = "초기화",

```



```

600         style = MaterialTheme.typography.labelLarge,
601         color = MaterialTheme.colorScheme.onSurfaceVariant,
602         modifier = Modifier.padding(horizontal = 12.dp, vertical = 8.dp),
603     )
604 }
605 }
606
607 Spacer(modifier = Modifier.height(20.dp))
608 HorizontalDivider(color = MaterialTheme.colorScheme.outlineVariant.copy(alpha = 0.5f))
609 Spacer(modifier = Modifier.height(20.dp))
610
611 if (viewModel.viewMode == DiscoverViewMode.LIST) {
612     // — 반경 —
613     FilterSection(title = "반경") {
614         Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
615             listOf(1000 to "1km", 3000 to "3km", 5000 to "5km").forEach { (meters, label) ->
616                 SheetChip(
617                     label = label,
618                     selected = viewModel.radiusMeters == meters,
619                     onClick = { viewModel.onRadiusChange(meters) },
620                 )
621             }
622         }
623     }
624
625     Spacer(modifier = Modifier.height(20.dp))
626 }
627
628 // — 코스 유형 —
629 FilterSection(title = "코스 유형") {
630     Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
631         SheetChip(
632             label = "전체",
633             selected = viewModel.isLoopFilter == null,
634             onClick = { viewModel.onIsLoopFilterChange(null) },
635         )
636         SheetChip(
637             label = "루프코스",
638             selected = viewModel.isLoopFilter == true,
639             onClick = { viewModel.onIsLoopFilterChange(true) },
640         )
641         SheetChip(
642             label = "일반코스",
643             selected = viewModel.isLoopFilter == false,
644             onClick = { viewModel.onIsLoopFilterChange(false) },
645         )
646     }
647 }
648
649 Spacer(modifier = Modifier.height(20.dp))
650
651 // — 코스 거리 —
652 FilterSection(title = "코스 거리") {
653     Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
654         DistanceFilterOption.entries.forEach { option ->
655             SheetChip(
656                 label = option.label,
657                 selected = viewModel.distanceFilter == option,
658                 onClick = { viewModel.onDistanceFilterChange(option) },
659             )
660         }
661     }
662 }
663
664 Spacer(modifier = Modifier.height(20.dp))
665
666 // — 정렬 —
667 FilterSection(title = "정렬") {
668     Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
669         CourseSortOption.entries.forEach { option ->
670             SheetChip(
671                 label = option.label,
672                 selected = viewModel.sortOption == option,
673                 onClick = { viewModel.onSortChange(option) },
674             )
675         }
676     }
677 }
678
679 Spacer(modifier = Modifier.height(28.dp))
680
681 // 적용 버튼
682 Surface(
683     modifier = Modifier.fillMaxWidth(),
684     onClick = onDismiss,
685     shape = MaterialTheme.shapes.extraLarge,

```

```

686         color = MaterialTheme.colorScheme.primary,
687     ) {
688         Text(
689             text = "적용",
690             style = MaterialTheme.typography.titleSmall,
691             fontWeight = FontWeight.Bold,
692             color = MaterialTheme.colorScheme.onPrimary,
693             textAlign = TextAlign.Center,
694             modifier = Modifier.padding(vertical = 14.dp),
695         )
696     }
697 }
698 }
699
700 @Composable
701 private fun FilterSection(
702     title: String,
703     content: @Composable () -> Unit,
704 ) {
705     Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
706         Text(
707             text = title,
708             style = MaterialTheme.typography.labelLarge,
709             fontWeight = FontWeight.SemiBold,
710             color = MaterialTheme.colorScheme.onSurfaceVariant,
711         )
712         content()
713     }
714 }
715
716 @Composable
717 private fun SheetChip(
718     label: String,
719     selected: Boolean,
720     onClick: () -> Unit,
721 ) {
722     Surface(
723         shape = MaterialTheme.shapes.extraLarge,
724         color = if (selected) MaterialTheme.colorScheme.primary
725             else MaterialTheme.colorScheme.surfaceVariant,
726         onClick = onClick,
727     ) {
728         Text(
729             text = label,
730             style = MaterialTheme.typography.labelLarge,
731             fontWeight = if (selected) FontWeight.SemiBold else FontWeight.Normal,
732             color = if (selected) MaterialTheme.colorScheme.onPrimary
733                 else MaterialTheme.colorScheme.onSurfaceVariant,
734             modifier = Modifier.padding(horizontal = 16.dp, vertical = 9.dp),
735         )
736     }
737 }
738
739 // ——— 활성 필터 요약 칩 —————
740
741 @Composable
742 private fun ActiveChip(label: String, onDismiss: () -> Unit) {
743     Surface(
744         shape = MaterialTheme.shapes.extraLarge,
745         color = MaterialTheme.colorScheme.primaryContainer,
746         onClick = onDismiss,
747     ) {
748         Row(
749             modifier = Modifier.padding(start = 12.dp, end = 8.dp, top = 7.dp, bottom = 7.dp),
750             verticalAlignment = Alignment.CenterVertically,
751             horizontalArrangement = Arrangement.spacedBy(4.dp),
752         ) {
753             Text(
754                 text = label,
755                 style = MaterialTheme.typography.labelMedium,
756                 fontWeight = FontWeight.Medium,
757                 color = MaterialTheme.colorScheme.onPrimaryContainer,
758             )
759             Icon(
760                 imageVector = Icons.Filled.Close,
761                 contentDescription = "필터 해제",
762                 tint = MaterialTheme.colorScheme.onPrimaryContainer,
763                 modifier = Modifier.size(14.dp),
764             )
765         }
766     }
767 }

```

```

1 package com.runway.android.ui.discover
2
3 import android.annotation.SuppressLint
4 import androidx.compose.runtime.getValue
5 import androidx.compose.runtime.mutableStateOf
6 import androidx.compose.runtime.setValue
7 import androidx.lifecycle.ViewModel
8 import androidx.lifecycle.viewModelScope
9 import com.google.android.gms.location.FusedLocationProviderClient
10 import com.google.android.gms.location.Priority
11 import com.runway.android.core.map.MapPoint
12 import com.runway.android.core.result.NetworkResult
13 import com.runway.android.data.course.model.NearbyCourseItem
14 import com.runway.android.domain.course.CourseRepository
15 import dagger.hilt.android.lifecycle.HiltViewModel
16 import kotlinx.coroutines.Job
17 import kotlinx.coroutines.launch
18 import kotlinx.coroutines.tasks.await
19 import javax.inject.Inject
20
21 enum class DistanceFilterOption(val label: String, val minMeters: Double?, val maxMeters: Double?) {
22     ALL("전체 거리", null, null),
23     SHORT("5km 이하", null, 5000.0),
24     MEDIUM("5~10km", 5000.0, 10000.0),
25     LONG("10km 이상", 10000.0, null),
26 }
27
28 enum class CourseSortOption(val label: String) {
29     NEAREST("가까운 순"),
30     POPULAR("인기 순"),
31     COMPLETION_RATE("완주율 순"),
32 }
33
34 enum class DiscoverViewMode {
35     LIST,
36     MAP,
37 }
38
39 @HiltViewModel
40 class DiscoverViewModel @Inject constructor(
41     private val courseRepository: CourseRepository,
42     private val fusedLocationClient: FusedLocationProviderClient,
43 ) : ViewModel() {
44
45     var isLoading by mutableStateOf(false)
46     private set
47     var isRefreshing by mutableStateOf(false)
48     private set
49     var errorMessage by mutableStateOf<String?>(null)
50     private set
51     var radiusMeters by mutableStateOf(3000)
52     private set
53     var isLoopFilter by mutableStateOf<Boolean?>(null)
54     private set
55     var distanceFilter by mutableStateOf<DistanceFilterOption>(DistanceFilterOption.ALL)
56     private set
57     var isLocationRequired by mutableStateOf(false)
58     private set
59     var keyword by mutableStateOf("")
60     private set
61     var sortOption by mutableStateOf(CourseSortOption.NEAREST)
62     private set
63     var viewMode by mutableStateOf(DiscoverViewMode.LIST)
64     private set
65     var isMapLoading by mutableStateOf(false)
66     private set
67     var mapErrorMessage by mutableStateOf<String?>(null)
68     private set
69     var currentLocation by mutableStateOf<MapPoint?>(null)
70     private set
71
72     private var rawCourses = listOf<NearbyCourseItem>()
73     private var rawMapCourses = listOf<NearbyCourseItem>()
74     private var currentLatitude = 0.0
75     private var currentLongitude = 0.0
76     private var hasLocation = false
77     private var loadJob: Job? = null
78
79     val courses: List<NearbyCourseItem>
80         get() = when (sortOption) {
81             CourseSortOption.NEAREST -> rawCourses.sortedBy { it.distanceFromMeMeters }
82             CourseSortOption.POPULAR -> rawCourses.sortedByDescending { it.completionCount }
83             CourseSortOption.COMPLETION_RATE -> rawCourses.sortedByDescending { completionRate(it) }

```

```

84     }
85
86     val mapCourses: List<NearbyCourseItem>
87     get() = when (sortOption) {
88         CourseSortOption.NEAREST -> rawMapCourses
89         CourseSortOption.POPULAR -> rawMapCourses.sortedByDescending { it.completionCount }
90         CourseSortOption.COMPLETION_RATE -> rawMapCourses.sortedByDescending { completionRate(it) }
91     }
92
93     fun onViewModeChange(mode: DiscoverViewMode) {
94         if (viewMode == mode) return
95         viewMode = mode
96         if (mode == DiscoverViewMode.MAP && rawMapCourses.isEmpty()) {
97             loadMapCourses()
98         }
99     }
100
101     fun onSortChange(option: CourseSortOption) {
102         sortOption = option
103     }
104
105     fun onRadiusChange(meters: Int) {
106         if (radiusMeters == meters) return
107         radiusMeters = meters
108         if (hasLocation) loadCourses()
109     }
110
111     fun onIsLoopFilterChange(value: Boolean?) {
112         if (isLoopFilter == value) return
113         isLoopFilter = value
114         reloadVisibleMode()
115     }
116
117     fun onDistanceFilterChange(option: DistanceFilterOption) {
118         if (distanceFilter == option) return
119         distanceFilter = option
120         reloadVisibleMode()
121     }
122
123     fun onKeywordChange(value: String) {
124         keyword = value
125     }
126
127     fun onSearch() {
128         reloadVisibleMode()
129     }
130
131     fun clearKeyword() {
132         keyword = ""
133         reloadVisibleMode()
134     }
135
136     fun resetFilters() {
137         radiusMeters = 3000
138         isLoopFilter = null
139         distanceFilter = DistanceFilterOption.ALL
140         sortOption = CourseSortOption.NEAREST
141         reloadVisibleMode()
142     }
143
144     val activeFilterCount: Int
145     get() = listOfNotNull(
146         if (radiusMeters != 3000) "r" else null,
147         isLoopFilter?.let { "l" },
148         if (distanceFilter != DistanceFilterOption.ALL) "d" else null,
149         if (sortOption != CourseSortOption.NEAREST) "s" else null,
150     ).size
151
152     fun refresh() {
153         if (viewMode == DiscoverViewMode.MAP) {
154             loadMapCourses()
155             return
156         }
157         if (isLocationRequired || !hasLocation) return
158         viewModelScope.launch {
159             isRefreshing = true
160             doLoadCourses()
161             isRefreshing = false
162         }
163     }
164
165     @SuppressLint("MissingPermission")
166     fun onLocationPermissionGranted() {
167         isLocationRequired = false
168         viewModelScope.launch {
169             isLoading = true

```

```

170     errorMessage = null
171     try {
172         val location = fusedLocationClient.lastLocation.await()
173         ?: fusedLocationClient.getCurrentLocation(
174             Priority.PRIORITY_BALANCED_POWER_ACCURACY, null
175         ).await()
176         if (location != null) {
177             currentLatitude = location.latitude
178             currentLongitude = location.longitude
179             currentLocation = MapPoint(location.latitude, location.longitude)
180             hasLocation = true
181             loadCourses()
182         } else {
183             isLoading = false
184             errorMessage = "위치를 가져올 수 없습니다. 잠시 후 다시 시도해 주세요."
185         }
186     } catch (_: Exception) {
187         isLoading = false
188         errorMessage = "위치를 가져올 수 없습니다."
189     }
190 }
191 }
192
193 fun onLocationPermissionDenied() {
194     isLocationRequired = true
195     isLoading = false
196     errorMessage = "주변 코스 탐색에 위치 권한이 필요합니다."
197 }
198
199 private fun loadCourses() {
200     loadJob?.cancel()
201     loadJob = viewModelScope.launch { doLoadCourses() }
202 }
203
204 private fun loadMapCourses() {
205     loadJob?.cancel()
206     loadJob = viewModelScope.launch { doLoadMapCourses() }
207 }
208
209 private fun reloadVisibleMode() {
210     if (viewModel == DiscoverViewModel.MAP) {
211         loadMapCourses()
212     } else if (hasLocation) {
213         loadCourses()
214     }
215 }
216
217 private suspend fun doLoadCourses() {
218     isLoading = true
219     errorMessage = null
220     when (val result = courseRepository.getNearbyCourses(
221         latitude = currentLatitude,
222         longitude = currentLongitude,
223         radiusMeters = radiusMeters,
224         minDistanceMeters = distanceFilter.minMeters,
225         maxDistanceMeters = distanceFilter.maxMeters,
226         isLoop = isLoopFilter,
227         keyword = keyword.trim().takeIf { it.isNotBlank() },
228     )) {
229         is NetworkResult.Success -> rawCourses = result.data.content
230         is NetworkResult.ApiError -> rawCourses = listOf()
231         is NetworkResult.NetworkError -> errorMessage = "네트워크 오류가 발생했습니다."
232     }
233     isLoading = false
234 }
235
236 private suspend fun doLoadMapCourses() {
237     isMapLoading = true
238     mapErrorMessage = null
239     when (val result = courseRepository.getNearbyCourses(
240         latitude = KOREA_CENTER_LATITUDE,
241         longitude = KOREA_CENTER_LONGITUDE,
242         radiusMeters = NATIONWIDE_RADIUS_METERS,
243         minDistanceMeters = distanceFilter.minMeters,
244         maxDistanceMeters = distanceFilter.maxMeters,
245         isLoop = isLoopFilter,
246         keyword = keyword.trim().takeIf { it.isNotBlank() },
247         includeRoutePoints = false,
248         includeExactStartPoint = true,
249         size = MAP_COURSE_LIMIT,
250     )) {
251         is NetworkResult.Success -> rawMapCourses = result.data.content
252         is NetworkResult.ApiError -> {
253             rawMapCourses = emptyList()
254             mapErrorMessage = "전국 코스를 불러오지 못했습니다."
255         }
256     }

```

```
256         is NetworkResult.NetworkError -> mapErrorMessage = "네트워크 오류가 발생했습니다."
257     }
258     isMapLoading = false
259 }
260
261 companion object {
262     private const val KOREA_CENTER_LATITUDE = 36.35
263     private const val KOREA_CENTER_LONGITUDE = 127.8
264     private const val NATIONWIDE_RADIUS_METERS = 600_000
265     private const val MAP_COURSE_LIMIT = 500
266
267     fun completionRate(course: NearbyCourseItem): Float =
268         if (course.attemptCount > 0) course.completionCount.toFloat() / course.attemptCount else 0f
269
270     fun isPopular(course: NearbyCourseItem): Boolean =
271         course.completionCount >= 10 && completionRate(course) >= 0.6f
272
273     fun isNew(course: NearbyCourseItem): Boolean =
274         course.attemptCount == 0
275 }
276 }
```

```

1 package com.runway.android.ui.home
2
3 import com.google.gson.Gson
4 import okhttp3.HttpUrl.Companion.toHttpUrl
5 import okhttp3.OkHttpClient
6 import okhttp3.Request
7 import org.locationtech.proj4j.CRSFactory
8 import org.locationtech.proj4j.CoordinateTransformFactory
9 import org.locationtech.proj4j.ProjCoordinate
10 import kotlin.math.roundToInt
11
12 internal class AirKoreaClient(
13     private val httpClient: OkHttpClient,
14     private val gson: Gson = Gson(),
15 ) {
16     fun getAirQuality(
17         latitude: Double,
18         longitude: Double,
19         serviceKey: String,
20         stationCandidates: List<String> = emptyList(),
21     ): AirKoreaReading? {
22         if (serviceKey.isBlank()) return null
23
24         return runCatching {
25             val tmCoordinate = AirKoreaCoordinateConverter.toTm(latitude, longitude)
26             val nearestStation = findNearestStation(tmCoordinate, serviceKey)
27             buildList {
28                 nearestStation?.let(::add)
29                 addAll(stationCandidates)
30             }.distinct().firstNotNullOfOrNull { stationName ->
31                 getLatestReading(stationName, serviceKey)
32             }
33         }.getOrNull()
34     }
35
36     private fun findNearestStation(
37         coordinate: TmCoordinate,
38         serviceKey: String,
39     ): String? {
40         val url = NEARBY_STATION_URL.toHttpUrl().newBuilder()
41             .addQueryParameter("serviceKey", serviceKey)
42             .addQueryParameter("returnType", "json")
43             .addQueryParameter("tmX", coordinate.x.toString())
44             .addQueryParameter("tmY", coordinate.y.toString())
45             .addQueryParameter("ver", "1.1")
46             .build()
47
48         val response = execute(url.toString(), NearbyStationResponse::class.java)
49         return response?.response?.body?.items
50             ?.minByOrNull { it.tm?.toDoubleOrNull() ?: Double.MAX_VALUE }
51             ?.stationName
52             ?.takeIf(String::isNotBlank)
53     }
54
55     private fun getLatestReading(
56         stationName: String,
57         serviceKey: String,
58     ): AirKoreaReading? {
59         val url = STATION_READING_URL.toHttpUrl().newBuilder()
60             .addQueryParameter("serviceKey", serviceKey)
61             .addQueryParameter("returnType", "json")
62             .addQueryParameter("numOfRows", "1")
63             .addQueryParameter("pageNo", "1")
64             .addQueryParameter("stationName", stationName)
65             .addQueryParameter("dataTerm", "DAILY")
66             .addQueryParameter("ver", "1.3")
67             .build()
68
69         val item = execute(url.toString(), StationReadingResponse::class.java)
70             ?.response?.body?.items?.firstOrNull()
71             ?: return null
72         val pm10 = item.pm10Value.toMeasurement()
73         val pm25 = item.pm25Value.toMeasurement()
74         if (pm10 == null && pm25 == null) return null
75
76         return AirKoreaReading(
77             pm10 = pm10,
78             pm25 = pm25,
79             stationName = stationName,
80             measuredAt = item.dateTime.orEmpty(),
81         )
82     }
83 }

```

```

84     private fun <T> execute(url: String, responseType: Class<T>): T? {
85         val request = Request.Builder().url(url).get().build()
86         return httpClient.newCall(request).execute().use { response ->
87             if (!response.isSuccessful) return null
88             response.body?.charStream()?.use { gson.fromJson(it, responseType) }
89         }
90     }
91
92     private fun String?.toMeasurement(): Int? {
93         return this?.takeUnless { it == "-" }?.toDoubleOrNull()?.roundToInt()
94     }
95
96     private data class NearbyStationResponse(val response: NearbyResponse?)
97     private data class NearbyResponse(val body: NearbyBody?)
98     private data class NearbyBody(val items: List<NearbyStation> = emptyList())
99     private data class NearbyStation(
100         val stationName: String = "",
101         val tm: String? = null,
102     )
103
104     private data class StationReadingResponse(val response: ReadingResponse?)
105     private data class ReadingResponse(val body: ReadingBody?)
106     private data class ReadingBody(val items: List<ReadingItem> = emptyList())
107     private data class ReadingItem(
108         val pm10Value: String? = null,
109         val pm25Value: String? = null,
110         val dateTime: String? = null,
111     )
112
113     private companion object {
114         const val NEARBY_STATION_URL =
115             "https://apis.data.go.kr/B552584/MsrstnInfoInquireSvc/getNearbyMsrstnList"
116         const val STATION_READING_URL =
117             "https://apis.data.go.kr/B552584/ArpltnInforInquireSvc/getMsrstnAcctoRltmMesureDnsty"
118     }
119 }
120
121 internal data class AirKoreaReading(
122     val pm10: Int?,
123     val pm25: Int?,
124     val stationName: String,
125     val measuredAt: String,
126 )
127
128 internal data class TmCoordinate(val x: Double, val y: Double)
129
130 internal object AirKoreaCoordinateConverter {
131     private val transform = CoordinateTransformFactory().createTransform(
132         CRSFactory().createFromParameters(
133             "WGS84",
134             "+proj=longlat +datum=WGS84 +no_defs",
135         ),
136         CRSFactory().createFromParameters(
137             "AIR_KOREA_TM",
138             "+proj=tmerc +lat_0=38 +lon_0=127.0028902777778 +k=1 " +
139                 "+x_0=200000 +y_0=500000 +ellps=bessel " +
140                 "+towgs84=-146.43,507.89,681.46 +units=m +no_defs",
141         ),
142     )
143
144     fun toTm(latitude: Double, longitude: Double): TmCoordinate {
145         val output = ProjCoordinate()
146         transform.transform(ProjCoordinate(longitude, latitude), output)
147         return TmCoordinate(output.x, output.y)
148     }
149 }

```



```

1  package com.runway.android.ui.home
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.border
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.layout.Arrangement
7  import androidx.compose.foundation.layout.Box
8  import androidx.compose.foundation.layout.Column
9  import androidx.compose.foundation.layout.PaddingValues
10 import androidx.compose.foundation.layout.Row
11 import androidx.compose.foundation.layout.RowScope
12 import androidx.compose.foundation.layout.Spacer
13 import androidx.compose.foundation.layout.fillMaxWidth
14 import androidx.compose.foundation.layout.height
15 import androidx.compose.foundation.layout.padding
16 import androidx.compose.foundation.layout.width
17 import androidx.compose.foundation.lazy.LazyColumn
18 import androidx.compose.foundation.lazy.rememberLazyListState
19 import androidx.compose.foundation.rememberScrollState
20 import androidx.compose.foundation.shape.RoundedCornerShape
21 import androidx.compose.foundation.verticalScroll
22 import androidx.compose.material3.ExperimentalMaterial3Api
23 import androidx.compose.material3.HorizontalDivider
24 import androidx.compose.material3.MaterialTheme
25 import androidx.compose.material3.ModalBottomSheet
26 import androidx.compose.material3.Text
27 import androidx.compose.material3.TextButton
28 import androidx.compose.material3.rememberModalBottomSheetState
29 import androidx.compose.runtime.Composable
30 import androidx.compose.runtime.LaunchedEffect
31 import androidx.compose.runtime.derivedStateOf
32 import androidx.compose.runtime.getValue
33 import androidx.compose.runtime.mutableIntStateOf
34 import androidx.compose.runtime.mutableStateOf
35 import androidx.compose.runtime.remember
36 import androidx.compose.runtime.rememberCoroutineScope
37 import androidx.compose.runtime.setValue
38 import androidx.compose.runtime.snapshotFlow
39 import androidx.compose.ui.Alignment
40 import androidx.compose.ui.Modifier
41 import androidx.compose.ui.draw.clip
42 import androidx.compose.ui.graphics.Brush
43 import androidx.compose.ui.graphics.Color
44 import androidx.compose.ui.platform.LocalDensity
45 import androidx.compose.ui.text.font.FontWeight
46 import androidx.compose.ui.text.style.TextAlign
47 import androidx.compose.ui.unit.Dp
48 import androidx.compose.ui.unit.dp
49 import androidx.compose.ui.unit.sp
50 import com.runway.android.ui.components.RunwayPrimaryButton
51 import kotlinx.coroutines.flow.distinctUntilChanged
52 import kotlinx.coroutines.launch
53 import kotlin.math.roundToInt
54
55 private val SheetBg = Color(0xFF16171F)
56 private val ChipBg = Color(0xFF1E1F2A)
57 private val DividerColor = Color(0xFF2A2B38)
58
59 @OptIn(ExperimentalMaterial3Api::class)
60 @Composable
61 fun GoalSetupSheet(
62     onDismiss: () -> Unit,
63     onConfirm: (RunGoal) -> Unit,
64 ) {
65     val sheetState = rememberModalBottomSheetState(skipPartiallyExpanded = true)
66
67     ModalBottomSheet(
68         onDismissRequest = onDismiss,
69         sheetState = sheetState,
70         containerColor = SheetBg,
71         dragHandle = {
72             Box(
73                 modifier = Modifier
74                     .padding(top = 12.dp, bottom = 4.dp)
75                     .width(36.dp)
76                     .height(4.dp)
77                     .background(Color.White.copy(alpha = 0.15f), RoundedCornerShape(2.dp)),
78             )
79         },
80     ) {
81         GoalSetupContent(
82             onDismiss = onDismiss,
83             onConfirm = onConfirm,

```



```

170                .background(if (active) MaterialTheme.colorScheme.primary else ChipBg)
171                .clickable { selectedTab = tab }
172                .padding(horizontal = 18.dp, vertical = 9.dp),
173            )
174        }
175    }
176
177    HorizontalDivider(color = DividerColor)
178
179    when (selectedTab) {
180        GoalTab.TIME -> TimeGoalContent(
181            hours = targetHours,
182            minutes = targetMinutes,
183            onHoursChange = {
184                targetHours = it
185                if (it == 0 && targetMinutes == 0) targetMinutes = 1
186            },
187            onMinutesChange = {
188                targetMinutes = it.coerceAtLeast(if (targetHours == 0) 1 else 0)
189            },
190        )
191        GoalTab.DISTANCE -> DistanceGoalContent(
192            km = targetKm,
193            decimal = targetDecimal,
194            onKmChange = { targetKm = it },
195            onDecimalChange = { targetDecimal = it },
196        )
197        GoalTab.INTERVAL -> IntervalGoalContent(
198            warmupMode = warmupMode,
199            onWarmupModeChange = { warmupMode = it },
200            warmupMinutes = warmupMinutes,
201            onWarmupMinutesChange = { warmupMinutes = it },
202            warmupKm = warmupKm,
203            onWarmupKmChange = { warmupKm = it },
204            warmupDecimal = warmupDecimal,
205            onWarmupDecimalChange = { warmupDecimal = it },
206            workMode = workMode,
207            onWorkModeChange = { workMode = it },
208            workMinutes = workMinutes,
209            onWorkMinutesChange = { workMinutes = it },
210            workKm = workKm,
211            onWorkKmChange = { workKm = it },
212            workDecimal = workDecimal,
213            onWorkDecimalChange = { workDecimal = it },
214            workPace = workPace,
215            onWorkPaceChange = { workPace = it },
216            workHasPace = workHasPace,
217            onWorkHasPaceChange = { workHasPace = it },
218            recoveryMode = recoveryMode,
219            onRecoveryModeChange = { recoveryMode = it },
220            recoveryMinutes = recoveryMinutes,
221            onRecoveryMinutesChange = { recoveryMinutes = it },
222            recoveryKm = recoveryKm,
223            onRecoveryKmChange = { recoveryKm = it },
224            recoveryDecimal = recoveryDecimal,
225            onRecoveryDecimalChange = { recoveryDecimal = it },
226            recoveryPace = recoveryPace,
227            onRecoveryPaceChange = { recoveryPace = it },
228            recoveryHasPace = recoveryHasPace,
229            onRecoveryHasPaceChange = { recoveryHasPace = it },
230            cooldownMode = cooldownMode,
231            onCooldownModeChange = { cooldownMode = it },
232            cooldownMinutes = cooldownMinutes,
233            onCooldownMinutesChange = { cooldownMinutes = it },
234            cooldownKm = cooldownKm,
235            onCooldownKmChange = { cooldownKm = it },
236            cooldownDecimal = cooldownDecimal,
237            onCooldownDecimalChange = { cooldownDecimal = it },
238            sets = sets,
239            onSetsChange = { sets = it },
240        )
241    }
242
243    Spacer(Modifier.height(14.dp))
244    HorizontalDivider(color = DividerColor)
245    Spacer(Modifier.height(12.dp))
246
247    RunwayPrimaryButton(
248        text = "시작하기",
249        onClick = {
250            val goal = when (selectedTab) {
251                GoalTab.TIME -> RunGoal.TimeGoal((targetHours * 60 + targetMinutes).coerceAtLeast(1))
252                GoalTab.DISTANCE -> RunGoal.DistanceGoal(distanceKm(targetKm, targetDecimal))
253                GoalTab.INTERVAL -> RunGoal.IntervalGoal(
254                    warmup = IntervalSegment(
255                        duration = segmentDuration(warmupMode, warmupMinutes, warmupKm, warmupDecimal),

```

```

256         ),
257         work = IntervalSegment(
258             duration = segmentDuration(workMode, workMinutes, workKm, workDecimal),
259             paceTargetSecPerKm = if (workHasPace) workPace else null,
260         ),
261         recovery = IntervalSegment(
262             duration = segmentDuration(recoveryMode, recoveryMinutes, recoveryKm, recoveryDecimal),
263             paceTargetSecPerKm = if (recoveryHasPace) recoveryPace else null,
264         ),
265         cooldown = IntervalSegment(
266             duration = segmentDuration(cooldownMode, cooldownMinutes, cooldownKm, cooldownDecimal),
267         ),
268         sets = sets,
269     )
270 }
271 onConfirm(goal)
272 },
273 modifier = Modifier
274     .fillMaxWidth()
275     .padding(horizontal = 20.dp),
276 )
277
278 Spacer(Modifier.height(32.dp))
279 }
280 }
281
282 @Composable
283 private fun TimeGoalContent(
284     hours: Int,
285     minutes: Int,
286     onHoursChange: (Int) -> Unit,
287     onMinutesChange: (Int) -> Unit,
288 ) {
289     val totalMinutes = (hours * 60 + minutes).coerceAtLeast(1)
290
291     Column(modifier = Modifier.padding(horizontal = 20.dp, vertical = 18.dp)) {
292         GoalSectionLabel("목표 시간")
293         Spacer(Modifier.height(14.dp))
294         TimeWheelPicker(
295             hours = hours,
296             minutes = minutes,
297             onHoursChange = onHoursChange,
298             onMinutesChange = onMinutesChange,
299         )
300         Spacer(Modifier.height(18.dp))
301         GoalSummaryBox(text = "${timeLabel(totalMinutes)} 동안 달리기")
302     }
303 }
304
305 @Composable
306 private fun DistanceGoalContent(
307     km: Int,
308     decimal: Int,
309     onKmChange: (Int) -> Unit,
310     onDecimalChange: (Int) -> Unit,
311 ) {
312     Column(modifier = Modifier.padding(horizontal = 20.dp, vertical = 18.dp)) {
313         GoalSectionLabel("목표 거리")
314         Spacer(Modifier.height(14.dp))
315         DistanceWheelPicker(
316             km = km,
317             decimal = decimal,
318             onKmChange = onKmChange,
319             onDecimalChange = onDecimalChange,
320         )
321         Spacer(Modifier.height(18.dp))
322         GoalSummaryBox(text = "${distanceLabel(distanceMeters(km, decimal))} 달리기")
323     }
324 }
325
326 @Composable
327 private fun IntervalGoalContent(
328     warmupMode: SegmentMode, onWarmupModeChange: (SegmentMode) -> Unit,
329     warmupMinutes: Int, onWarmupMinutesChange: (Int) -> Unit,
330     warmupKm: Int, onWarmupKmChange: (Int) -> Unit,
331     warmupDecimal: Int, onWarmupDecimalChange: (Int) -> Unit,
332     workMode: SegmentMode, onWorkModeChange: (SegmentMode) -> Unit,
333     workMinutes: Int, onWorkMinutesChange: (Int) -> Unit,
334     workKm: Int, onWorkKmChange: (Int) -> Unit,
335     workDecimal: Int, onWorkDecimalChange: (Int) -> Unit,
336     workPace: Int, onWorkPaceChange: (Int) -> Unit,
337     workHasPace: Boolean, onWorkHasPaceChange: (Boolean) -> Unit,
338     recoveryMode: SegmentMode, onRecoveryModeChange: (SegmentMode) -> Unit,
339     recoveryMinutes: Int, onRecoveryMinutesChange: (Int) -> Unit,
340     recoveryKm: Int, onRecoveryKmChange: (Int) -> Unit,
341     recoveryDecimal: Int, onRecoveryDecimalChange: (Int) -> Unit,

```

```

342     recoveryPace: Int, onRecoveryPaceChange: (Int) -> Unit,
343     recoveryHasPace: Boolean, onRecoveryHasPaceChange: (Boolean) -> Unit,
344     cooldownMode: SegmentMode, onCooldownModeChange: (SegmentMode) -> Unit,
345     cooldownMinutes: Int, onCooldownMinutesChange: (Int) -> Unit,
346     cooldownKm: Int, onCooldownKmChange: (Int) -> Unit,
347     cooldownDecimal: Int, onCooldownDecimalChange: (Int) -> Unit,
348     sets: Int, onSetsChange: (Int) -> Unit,
349 ) {
350     Column(modifier = Modifier.padding(horizontal = 20.dp, vertical = 12.dp)) {
351         IntervalSection(title = "준비운동") {
352             SegmentDurationEditor(
353                 mode = warmupMode,
354                 onModeChange = onWarmupModeChange,
355                 minutes = warmupMinutes,
356                 onMinutesChange = onWarmupMinutesChange,
357                 km = warmupKm,
358                 onKmChange = onWarmupKmChange,
359                 decimal = warmupDecimal,
360                 onDecimalChange = onWarmupDecimalChange,
361             )
362         }
363
364         SectionDivider()
365
366         IntervalSection(title = "운동") {
367             SegmentDurationEditor(
368                 mode = workMode,
369                 onModeChange = onWorkModeChange,
370                 minutes = workMinutes,
371                 onMinutesChange = onWorkMinutesChange,
372                 km = workKm,
373                 onKmChange = onWorkKmChange,
374                 decimal = workDecimal,
375                 onDecimalChange = onWorkDecimalChange,
376             )
377             PaceEditor(
378                 title = "운동 페이스",
379                 enabled = workHasPace,
380                 onEnabledChange = onWorkHasPaceChange,
381                 paceSeconds = workPace,
382                 onPaceChange = onWorkPaceChange,
383             )
384         }
385
386         SectionDivider()
387
388         IntervalSection(title = "회복") {
389             SegmentDurationEditor(
390                 mode = recoveryMode,
391                 onModeChange = onRecoveryModeChange,
392                 minutes = recoveryMinutes,
393                 onMinutesChange = onRecoveryMinutesChange,
394                 km = recoveryKm,
395                 onKmChange = onRecoveryKmChange,
396                 decimal = recoveryDecimal,
397                 onDecimalChange = onRecoveryDecimalChange,
398             )
399             PaceEditor(
400                 title = "회복 페이스",
401                 enabled = recoveryHasPace,
402                 onEnabledChange = onRecoveryHasPaceChange,
403                 paceSeconds = recoveryPace,
404                 onPaceChange = onRecoveryPaceChange,
405             )
406         }
407
408         SectionDivider()
409
410         IntervalSection(title = "쿨다운") {
411             SegmentDurationEditor(
412                 mode = cooldownMode,
413                 onModeChange = onCooldownModeChange,
414                 minutes = cooldownMinutes,
415                 onMinutesChange = onCooldownMinutesChange,
416                 km = cooldownKm,
417                 onKmChange = onCooldownKmChange,
418                 decimal = cooldownDecimal,
419                 onDecimalChange = onCooldownDecimalChange,
420             )
421         }
422
423         SectionDivider()
424
425         IntervalSection(title = "총 반복 횟수") {
426             val setValues = remember { (1..20).toList() }
427             Row(verticalAlignment = Alignment.CenterVertically) {

```

```

428         WheelPicker(
429             values = setValues,
430             selectedValue = sets,
431             onValueChange = onSetsChange,
432             label = { it.toString() },
433             modifier = Modifier.weight(1f),
434         )
435         UnitLabel("회")
436     }
437 }
438
439 Spacer(Modifier.height(16.dp))
440 GoalSummaryBox(
441     text = "준비 ${segmentLabel(warmupMode, warmupMinutes, warmupKm, warmupDecimal)} → " +
442         "[운동 ${segmentLabel(workMode, workMinutes, workKm, workDecimal)} + " +
443         "회복 ${segmentLabel(recoveryMode, recoveryMinutes, recoveryKm, recoveryDecimal)}] × ${sets}회 → " +
444         "쿨다운 ${segmentLabel(cooldownMode, cooldownMinutes, cooldownKm, cooldownDecimal)}",
445 )
446 }
447 }
448
449 @Composable
450 private fun SegmentDurationEditor(
451     mode: SegmentMode,
452     onModeChange: (SegmentMode) -> Unit,
453     minutes: Int,
454     onMinutesChange: (Int) -> Unit,
455     km: Int,
456     onKmChange: (Int) -> Unit,
457     decimal: Int,
458     onDecimalChange: (Int) -> Unit,
459 ) {
460     ToggleRow(
461         options = SegmentMode.entries.map { it.label },
462         selectedIndex = if (mode == SegmentMode.TIME) 0 else 1,
463         onSelect = { onModeChange(if (it == 0) SegmentMode.TIME else SegmentMode.DISTANCE) },
464     )
465     Spacer(Modifier.height(12.dp))
466     if (mode == SegmentMode.TIME) {
467         MinuteWheelPicker(
468             minutes = minutes,
469             onMinutesChange = onMinutesChange,
470             maxMinutes = 120,
471         )
472     } else {
473         DistanceWheelPicker(
474             km = km,
475             decimal = decimal,
476             onKmChange = onKmChange,
477             onDecimalChange = onDecimalChange,
478             maxKm = 20,
479         )
480     }
481 }
482
483 @Composable
484 private fun PaceEditor(
485     title: String,
486     enabled: Boolean,
487     onEnabledChange: (Boolean) -> Unit,
488     paceSeconds: Int,
489     onPaceChange: (Int) -> Unit,
490 ) {
491     Spacer(Modifier.height(14.dp))
492     Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(8.dp)) {
493         GoalSectionLabel(title)
494         Spacer(Modifier.weight(1f))
495         ToggleRow(
496             options = listOf("설정", "없음"),
497             selectedIndex = if (enabled) 0 else 1,
498             onSelect = { onEnabledChange(it == 0) },
499         )
500     }
501     if (enabled) {
502         Spacer(Modifier.height(10.dp))
503         PaceWheelPicker(
504             seconds = paceSeconds,
505             onSecondsChange = onPaceChange,
506         )
507     }
508 }
509
510 @Composable
511 private fun TimeWheelPicker(
512     hours: Int,
513     minutes: Int,

```

```

514     onHoursChange: (Int) -> Unit,
515     onMinutesChange: (Int) -> Unit,
516 ) {
517     val hourValues = remember { (0..5).toList() }
518     val minuteValues = remember { (0..59).toList() }
519     WheelFrame {
520         WheelPicker(
521             values = hourValues,
522             selectedValue = hours,
523             onValueChange = onHoursChange,
524             label = { it.toString() },
525             modifier = Modifier.weight(1f),
526         )
527         UnitLabel("시간")
528         WheelPicker(
529             values = minuteValues,
530             selectedValue = minutes,
531             onValueChange = { onMinutesChange(it.coerceAtLeast(if (hours == 0) 1 else 0)) },
532             label = { it.toString().padStart(2, '0') },
533             modifier = Modifier.weight(1f),
534         )
535         UnitLabel("분")
536     }
537 }
538
539 @Composable
540 private fun MinuteWheelPicker(
541     minutes: Int,
542     onMinutesChange: (Int) -> Unit,
543     maxMinutes: Int,
544 ) {
545     val minuteValues = remember(maxMinutes) { (1..maxMinutes).toList() }
546     WheelFrame {
547         WheelPicker(
548             values = minuteValues,
549             selectedValue = minutes.coerceIn(1, maxMinutes),
550             onValueChange = onMinutesChange,
551             label = { it.toString() },
552             modifier = Modifier.weight(1f),
553         )
554         UnitLabel("분")
555     }
556 }
557
558 @Composable
559 private fun DistanceWheelPicker(
560     km: Int,
561     decimal: Int,
562     onKmChange: (Int) -> Unit,
563     onDecimalChange: (Int) -> Unit,
564     maxKm: Int = 50,
565 ) {
566     val kmValues = remember(maxKm) { (0..maxKm).toList() }
567     val decimalValues = remember { (0..99).toList() }
568     WheelFrame {
569         WheelPicker(
570             values = kmValues,
571             selectedValue = km.coerceIn(0, maxKm),
572             onValueChange = { value ->
573                 onKmChange(value)
574                 if (value == 0 && decimal == 0) onDecimalChange(10)
575             },
576             label = { it.toString() },
577             modifier = Modifier.weight(1f),
578         )
579         UnitLabel("km")
580         Text(
581             text = ".",
582             fontSize = 34.sp,
583             fontWeight = FontWeight.ExtraBold,
584             color = Color.White.copy(alpha = 0.85f),
585             modifier = Modifier.padding(horizontal = 2.dp),
586         )
587         WheelPicker(
588             values = decimalValues,
589             selectedValue = decimal.coerceIn(0, 99),
590             onValueChange = { value ->
591                 onDecimalChange(value)
592                 if (km == 0 && value == 0) onDecimalChange(10)
593             },
594             label = { it.toString().padStart(2, '0') },
595             modifier = Modifier.weight(1f),
596         )
597         UnitLabel("km")
598     }
599 }

```

```

600
601 @Composable
602 private fun PaceWheelPicker(
603     seconds: Int,
604     onSecondsChange: (Int) -> Unit,
605 ) {
606     val minutes = (seconds / 60).coerceIn(3, 12)
607     val sec = ((seconds % 60) / 5 * 5).coerceIn(0, 55)
608     val minuteValues = remember { (3..12).toList() }
609     val secondValues = remember { (0..55 step 5).toList() }
610     WheelFrame {
611         WheelPicker(
612             values = minuteValues,
613             selectedValue = minutes,
614             onValueChange = { onSecondsChange(it * 60 + sec) },
615             label = { it.toString() },
616             modifier = Modifier.weight(1f),
617         )
618         UnitLabel("분")
619         WheelPicker(
620             values = secondValues,
621             selectedValue = sec,
622             onValueChange = { onSecondsChange(minutes * 60 + it) },
623             label = { it.toString().padStart(2, '0') },
624             modifier = Modifier.weight(1f),
625         )
626         UnitLabel("/km")
627     }
628 }
629
630 @Composable
631 private fun <T> WheelPicker(
632     values: List<T>,
633     selectedValue: T,
634     onValueChange: (T) -> Unit,
635     label: (T) -> String,
636     modifier: Modifier = Modifier,
637     itemHeight: Dp = 52.dp,
638 ) {
639     val selectedIndex = values.indexOf(selectedValue).coerceAtLeast(0)
640     val listState = rememberLazyListState(initialFirstVisibleItemIndex = selectedIndex)
641     val scope = rememberCoroutineScope()
642     val density = LocalDensity.current
643     val itemHeightPx = with(density) { itemHeight.toPx() }
644     val centerIndex by remember {
645         derivedStateOf {
646             val offsetItems = (listState.firstVisibleItemScrollOffset / itemHeightPx).roundToInt()
647             (listState.firstVisibleItemIndex + offsetItems).coerceIn(values.indices)
648         }
649     }
650
651     LaunchedEffect(listState, values) {
652         snapshotFlow { centerIndex }
653             .distinctUntilChanged()
654             .collect { index -> onValueChange(values[index]) }
655     }
656
657     LaunchedEffect(listState, values) {
658         snapshotFlow { listState.isScrollInProgress }
659             .distinctUntilChanged()
660             .collect { scrolling ->
661                 if (!scrolling && values.isNotEmpty()) {
662                     listState.animateScrollToItem(centerIndex)
663                 }
664             }
665     }
666
667     LazyColumn(
668         state = listState,
669         modifier = modifier.height(itemHeight * 3),
670         contentPadding = PaddingValues(vertical = itemHeight),
671         horizontalAlignment = Alignment.CenterHorizontally,
672     ) {
673         items(values.size) { index ->
674             val selected = index == centerIndex
675             Box(
676                 modifier = Modifier
677                     .fillMaxWidth()
678                     .height(itemHeight)
679                     .clickable {
680                         scope.launch { listState.animateScrollToItem(index) }
681                     },
682                 contentAlignment = Alignment.Center,
683             ) {
684                 Text(
685                     text = label(values[index]),

```



```

686         fontSize = if (selected) 34.sp else 20.sp,
687         lineHeight = if (selected) 38.sp else 24.sp,
688         fontWeight = if (selected) FontWeight.ExtraBold else FontWeight.Medium,
689         color = if (selected) Color.White else Color.White.copy(alpha = 0.28f),
690         textAlign = TextAlign.Center,
691     )
692 }
693 }
694 }
695 }
696
697 @Composable
698 private fun WheelFrame(content: @Composable RowScope.() -> Unit) {
699     Box(
700         modifier = Modifier
701             .fillMaxWidth()
702             .height(166.dp)
703             .clip(RoundedCornerShape(18.dp))
704             .background(Color.White.copy(alpha = 0.035f))
705             .border(1.dp, Color.White.copy(alpha = 0.07f), RoundedCornerShape(18.dp)),
706         contentAlignment = Alignment.Center,
707     ) {
708         Box(
709             modifier = Modifier
710                 .fillMaxWidth()
711                 .height(54.dp)
712                 .align(Alignment.Center)
713                 .background(
714                     Brush.horizontalGradient(
715                         colors = listOf(
716                             MaterialTheme.colorScheme.primary.copy(alpha = 0.03f),
717                             MaterialTheme.colorScheme.primary.copy(alpha = 0.13f),
718                             MaterialTheme.colorScheme.primary.copy(alpha = 0.03f),
719                         ),
720                     ),
721                 )
722                 .border(
723                     width = 1.dp,
724                     color = MaterialTheme.colorScheme.primary.copy(alpha = 0.16f),
725                     shape = RoundedCornerShape(10.dp),
726                 ),
727             )
728         Row(
729             modifier = Modifier
730                 .fillMaxWidth()
731                 .padding(horizontal = 18.dp),
732             verticalAlignment = Alignment.CenterVertically,
733             horizontalArrangement = Arrangement.Center,
734             content = content,
735         )
736     }
737 }
738
739 @Composable
740 private fun UnitLabel(text: String) {
741     Text(
742         text = text,
743         fontSize = 15.sp,
744         fontWeight = FontWeight.SemiBold,
745         color = MaterialTheme.colorScheme.primary.copy(alpha = 0.88f),
746         modifier = Modifier.padding(horizontal = 6.dp),
747     )
748 }
749
750 @Composable
751 private fun SectionDivider() {
752     Spacer(Modifier.height(6.dp))
753     HorizontalDivider(color = DividerColor)
754     Spacer(Modifier.height(6.dp))
755 }
756
757 @Composable
758 private fun IntervalSection(title: String, content: @Composable () -> Unit) {
759     Column(modifier = Modifier.padding(vertical = 10.dp)) {
760         Text(
761             text = title,
762             style = MaterialTheme.typography.titleMedium,
763             fontWeight = FontWeight.Bold,
764             color = Color.White,
765         )
766         Spacer(Modifier.height(10.dp))
767         content()
768     }
769 }
770
771 @Composable

```

```

772 private fun GoalSectionLabel(text: String) {
773     Text(
774         text = text,
775         style = MaterialTheme.typography.labelMedium,
776         fontWeight = FontWeight.SemiBold,
777         color = Color.White.copy(alpha = 0.58f),
778     )
779 }
780
781 @Composable
782 private fun ToggleRow(options: List<String>, selectedIndex: Int, onSelect: (Int) -> Unit) {
783     Row(
784         modifier = Modifier
785             .clip(RoundedCornerShape(10.dp))
786             .background(ChipBg),
787     ) {
788         options.forEachIndexed { idx, label ->
789             val active = idx == selectedIndex
790             Text(
791                 text = label,
792                 fontSize = 13.sp,
793                 fontWeight = if (active) FontWeight.Bold else FontWeight.Medium,
794                 color = if (active) Color(0xFF0A0B10) else Color.White.copy(alpha = 0.58f),
795                 modifier = Modifier
796                     .clip(RoundedCornerShape(10.dp))
797                     .background(if (active) MaterialTheme.colorScheme.primary else Color.Transparent)
798                     .clickable { onSelect(idx) }
799                     .padding(horizontal = 15.dp, vertical = 8.dp),
800             )
801         }
802     }
803 }
804
805 @Composable
806 private fun GoalSummaryBox(text: String) {
807     Box(
808         modifier = Modifier
809             .fillMaxWidth()
810             .clip(RoundedCornerShape(14.dp))
811             .background(Color.White.copy(alpha = 0.05f))
812             .border(
813                 1.dp,
814                 MaterialTheme.colorScheme.primary.copy(alpha = 0.30f),
815                 RoundedCornerShape(14.dp),
816             )
817             .padding(horizontal = 16.dp, vertical = 13.dp),
818     ) {
819         Text(
820             text = text,
821             style = MaterialTheme.typography.bodyMedium,
822             color = MaterialTheme.colorScheme.primary.copy(alpha = 0.88f),
823         )
824     }
825 }
826
827 private fun segmentDuration(
828     mode: SegmentMode,
829     minutes: Int,
830     km: Int,
831     decimal: Int,
832 ): IntervalDuration = when (mode) {
833     SegmentMode.TIME -> IntervalDuration.ByTime(minutes.coerceAtLeast(1))
834     SegmentMode.DISTANCE -> IntervalDuration.ByDistance(distanceMeters(km, decimal))
835 }
836
837 private fun segmentLabel(mode: SegmentMode, minutes: Int, km: Int, decimal: Int): String =
838     when (mode) {
839         SegmentMode.TIME -> timeLabel(minutes.coerceAtLeast(1))
840         SegmentMode.DISTANCE -> distanceLabel(distanceMeters(km, decimal))
841     }
842
843 private fun distanceKm(km: Int, decimal: Int): Float =
844     (distanceMeters(km, decimal) / 1000f)
845
846 private fun distanceMeters(km: Int, decimal: Int): Int =
847     ((km.coerceAtLeast(0) * 1000) + (decimal.coerceIn(0, 99) * 10)).coerceAtLeast(100)
848
849 private fun distanceLabel(meters: Int): String {
850     val km = meters / 1000
851     val decimal = (meters % 1000) / 10
852     return if (decimal == 0) "${km}km" else "%d.%02dkm".format(km, decimal)
853 }
854
855 private fun timeLabel(totalMinutes: Int): String {
856     val hours = totalMinutes / 60
857     val minutes = totalMinutes % 60

```

```
858     return when {
859         hours > 0 && minutes > 0 -> "${hours}시간 ${minutes}분"
860         hours > 0 -> "${hours}시간"
861         else -> "${minutes}분"
862     }
863 }
```

```

1 package com.runway.android.ui.home
2
3 import androidx.compose.foundation.background
4 import androidx.compose.foundation.layout.Arrangement
5 import androidx.compose.foundation.layout.Box
6 import androidx.compose.foundation.layout.PaddingValues
7 import androidx.compose.foundation.layout.Spacer
8 import androidx.compose.foundation.layout.fillMaxSize
9 import androidx.compose.foundation.layout.fillMaxWidth
10 import androidx.compose.foundation.layout.height
11 import androidx.compose.foundation.layout.padding
12 import androidx.compose.foundation.layout.size
13 import androidx.compose.foundation.layout.width
14 import androidx.compose.animation.AnimatedVisibility
15 import androidx.compose.animation.core.Tween
16 import androidx.compose.animation.fadeIn
17 import androidx.compose.animation.slideInVertically
18 import androidx.compose.foundation.lazy.LazyColumn
19 import androidx.compose.foundation.lazy.LazyRow
20 import androidx.compose.foundation.lazy.items
21 import androidx.compose.foundation.lazy.itemsIndexed
22 import androidx.compose.foundation.lazy.rememberLazyListState
23 import androidx.compose.material3.CircularProgressIndicator
24 import androidx.compose.material3.ExperimentalMaterial3Api
25 import androidx.compose.material3.MaterialTheme
26 import androidx.compose.material3.Text
27 import androidx.compose.material3.pulltorefresh.PullToRefreshBox
28 import androidx.compose.runtime.Composable
29 import androidx.compose.runtime.LaunchedEffect
30 import androidx.compose.runtime.derivedStateOf
31 import androidx.compose.runtime.getValue
32 import androidx.compose.runtime.mutableStateOf
33 import androidx.compose.runtime.remember
34 import androidx.compose.runtime.setValue
35 import kotlinx.coroutines.delay
36 import androidx.compose.ui.Alignment
37 import androidx.compose.ui.Modifier
38 import androidx.compose.ui.graphics.graphicsLayer
39 import androidx.compose.ui.platform.LocalDensity
40 import androidx.compose.ui.unit.dp
41 import androidx.hilt.navigation.compose.hiltViewModel
42 import com.runway.android.ui.components.DiscoverCourseCard
43 import com.runway.android.ui.components.RecentRunCard
44 import com.runway.android.ui.components.RunHeroSection
45 import com.runway.android.ui.components.SectionHeader
46 import com.runway.android.ui.components.WeeklyStatsCard
47
48 @OptIn(ExperimentalMaterial3Api::class)
49 @Composable
50 fun HomeScreen(
51     onStartRun: () -> Unit = {},
52     onSetGoal: () -> Unit = {},
53     onSeeAllRuns: () -> Unit = {},
54     onNavigateToCourseDetail: (String) -> Unit = {},
55     onNavigateToDiscover: () -> Unit = {},
56     onNavigateToRunDetail: (String) -> Unit = {},
57     viewModel: HomeViewModel = hiltViewModel(),
58 ) {
59     LaunchedEffect(Unit) {
60         viewModel.tryLoadNearbyCourses()
61         viewModel.tryLoadWeather()
62     }
63
64     if (viewModel.showGoalSheet) {
65         GoalSetupSheet(
66             onDismiss = viewModel::closeGoalSheet,
67             onConfirm = { goal ->
68                 viewModel.setGoal(goal)
69                 onStartRun()
70             },
71         )
72     }
73
74     val listState = rememberLazyListState()
75     val density = LocalDensity.current
76
77     // smoothstep easing: 0f = hero fully visible, 1f = hero scrolled / content revealed
78     // fade distance = 380dp for a gradual, cinematic transition
79     val eased by remember {
80         derivedStateOf {
81             val raw = if (listState.firstVisibleItemIndex > 0) 1f
82             else {
83                 val fadePx = with(density) { 380.dp.toPx() }

```

```

84         (listState.firstVisibleItemScrollOffset / fadePx).coerceIn(0f, 1f)
85     }
86     // smoothstep:  $3t^2 - 2t^3$  - slow start, smooth middle, slow end
87     raw * raw * (3f - 2f * raw)
88 }
89 }
90
91 // Content slide-up distance (pixels) - items translate from below as they fade in
92 val slideDistPx = with(density) { 28.dp.toPx() }
93
94 PullToRefreshBox(
95     isRefreshing = viewModel.isRefreshing,
96     onRefresh = viewModel::refresh,
97     modifier = Modifier.fillMaxSize(),
98 ) {
99     LazyColumn(
100         state = listState,
101         modifier = Modifier
102             .fillMaxSize()
103             .background(MaterialTheme.colorScheme.background),
104         contentPadding = PaddingValues(bottom = 24.dp),
105     ) {
106
107         // — Run Hero: fades out as user scrolls —
108         item {
109             RunHeroSection(
110                 onStartRun = onStartRun,
111                 onSetGoal = viewModel::openGoalSheet,
112                 selectedGoal = viewModel.selectedGoal,
113                 weatherInfo = viewModel.weatherInfo,
114                 currentLocation = viewModel.currentLocation,
115                 hasLocationPermission = viewModel.locationPermissionGranted,
116                 modifier = Modifier
117                     .fillMaxWidth()
118                     .fillParentMaxHeight(1.08f)
119                     .graphicsLayer { alpha = 1f - eased },
120             )
121         }
122
123         // — Weekly stats: fades in + slides up —
124         item {
125             SectionHeader(
126                 title = "이번 주",
127                 modifier = Modifier.graphicsLayer {
128                     alpha = eased
129                     translationY = (1f - eased) * slideDistPx
130                 },
131             )
132         }
133         item {
134             WeeklyStatsCard(
135                 stats = viewModel.weeklyStats,
136                 modifier = Modifier
137                     .padding(horizontal = 20.dp)
138                     .graphicsLayer {
139                         alpha = eased
140                         translationY = (1f - eased) * slideDistPx
141                     },
142             )
143         }
144
145         // — Nearby courses: fades in + slides up —
146         item {
147             SectionHeader(
148                 title = "주변 코스",
149                 cta = "전체 보기",
150                 onCtaClick = onNavigateToDiscover,
151                 modifier = Modifier.graphicsLayer {
152                     alpha = eased
153                     translationY = (1f - eased) * slideDistPx
154                 },
155             )
156         }
157         item {
158             when {
159                 viewModel.isLoadingNearbyCourses -> {
160                     Box(
161                         modifier = Modifier
162                             .fillMaxWidth()
163                             .height(160.dp)
164                             .graphicsLayer {
165                                 alpha = eased
166                                 translationY = (1f - eased) * slideDistPx
167                             },
168                         contentAlignment = Alignment.Center,
169                     ) {

```

```

170         CircularProgressIndicator(color = MaterialTheme.colorScheme.primary)
171     }
172 }
173 viewModel.nearbyCourses.isEmpty() -> {
174     Box(
175         modifier = Modifier
176             .fillMaxWidth()
177             .height(80.dp)
178             .graphicsLayer {
179                 alpha = eased
180                 translationY = (1f - eased) * slideDistPx
181             },
182         contentAlignment = Alignment.Center,
183     ) {
184         Text(
185             text = if (viewModel.locationPermissionGranted)
186                 "주변 3km 내에 코스가 없어요. 직접 만들어보세요!"
187             else
188                 "위치 권한이 필요해요",
189             style = MaterialTheme.typography.bodyMedium,
190             color = MaterialTheme.colorScheme.onSurfaceVariant,
191         )
192     }
193 }
194 else -> {
195     LazyRow(
196         contentPadding = PaddingValues(horizontal = 20.dp),
197         horizontalArrangement = Arrangement.spacedBy(12.dp),
198         modifier = Modifier.graphicsLayer {
199             alpha = eased
200             translationY = (1f - eased) * slideDistPx
201         },
202     ) {
203         items(viewModel.nearbyCourses) { course ->
204             DiscoverCourseCard(
205                 course = course,
206                 modifier = Modifier.width(280.dp),
207                 onClick = { onNavigateToCourseDetail(course.courseId) },
208             )
209         }
210     }
211 }
212 }
213 }
214
215 // — Recent runs: eased is ~1.0 by this point, no graphicsLayer needed —
216 item {
217     SectionHeader(
218         title = "최근 러닝",
219         cta = if (!viewModel.isLoadingRuns && viewModel.recentRuns.isNotEmpty()) "전체 보기" else null,
220         onCtaClick = onSeeAllRuns,
221     )
222 }
223
224 when {
225     viewModel.isLoadingRuns -> item {
226         Box(
227             modifier = Modifier
228                 .fillMaxWidth()
229                 .height(80.dp),
230             contentAlignment = Alignment.Center,
231         ) {
232             CircularProgressIndicator(
233                 modifier = Modifier.size(28.dp),
234                 color = MaterialTheme.colorScheme.primary,
235                 strokeWidth = 2.5.dp,
236             )
237         }
238     }
239     viewModel.recentRuns.isEmpty() -> item {
240         Box(
241             modifier = Modifier
242                 .fillMaxWidth()
243                 .height(80.dp),
244             contentAlignment = Alignment.Center,
245         ) {
246             Text(
247                 text = "아직 러닝 기록이 없어요. 첫 러닝을 시작해보세요!",
248                 style = MaterialTheme.typography.bodyMedium,
249                 color = MaterialTheme.colorScheme.onSurfaceVariant,
250             )
251         }
252     }
253     else -> itemsIndexed(viewModel.recentRuns, key = { _, run -> run.runId }) { index, run ->
254         var visible by remember { mutableStateOf(false) }
255         LaunchedEffect(run.runId) {

```

```
256         delay(index * 60L)
257         visible = true
258     }
259     AnimatedVisibility(
260         visible = visible,
261         enter = slideInVertically(tween(300)) { it / 2 } + fadeIn(tween(300)),
262     ) {
263         RecentRunCard(
264             run = run,
265             modifier = Modifier
266                 .padding(horizontal = 20.dp)
267                 .padding(bottom = 8.dp),
268             onClick = { onNavigateToRunDetail(run.runId) },
269         )
270     }
271 }
272 }
273
274     item { Spacer(Modifier.height(8.dp)) }
275 }
276 } // PullToRefreshBox
277 }
```

```

1  package com.runway.android.ui.home
2
3  import android.Manifest
4  import android.annotation.SuppressLint
5  import android.content.Context
6  import android.content.pm.PackageManager
7  import android.location.Geocoder
8  import androidx.compose.runtime.getValue
9  import androidx.compose.runtime.mutableStateOf
10 import androidx.compose.runtime.setValue
11 import androidx.core.content.ContextCompat
12 import androidx.lifecycle.ViewModel
13 import androidx.lifecycle.viewModelScope
14 import com.google.android.gms.location.FusedLocationProviderClient
15 import com.google.android.gms.location.CurrentLocationRequest
16 import com.google.android.gms.location.Priority
17 import com.google.android.gms.tasks.CancellationTokenSource
18 import com.google.gson.Gson
19 import com.google.gson.annotations.SerializedName
20 import com.runway.android.BuildConfig
21 import com.runway.android.core.result.NetworkResult
22 import com.runway.android.data.course.model.NearbyCourseItem
23 import com.runway.android.data.running.model.RunSummaryResponse
24 import com.runway.android.data.running.model.RunningStatsResponse
25 import com.runway.android.domain.course.CourseRepository
26 import com.runway.android.domain.running.RunningRepository
27 import com.runway.android.domain.user.UserRepository
28 import com.runway.android.core.map.MapPoint
29 import com.runway.android.ui.components.RecentRun
30 import com.runway.android.ui.components.WeatherInfo
31 import com.runway.android.ui.components.WeeklyStats
32 import com.runway.android.core.util.formatDuration
33 import dagger.hilt.android.lifecycle.HiltViewModel
34 import dagger.hilt.android.qualifiers.ApplicationContext
35 import kotlinx.coroutines.Dispatchers
36 import kotlinx.coroutines.launch
37 import kotlinx.coroutines.tasks.await
38 import kotlinx.coroutines.withContext
39 import okhttp3.OkHttpClient
40 import okhttp3.Request
41 import java.time.DayOfWeek
42 import java.time.Instant
43 import java.time.LocalDate
44 import java.time.ZoneId
45 import java.time.format.TextStyle
46 import java.time.temporal.TemporalAdjusters
47 import java.util.Calendar
48 import java.util.Locale
49 import kotlin.math.roundToInt
50 import javax.inject.Inject
51 import javax.inject.Named
52
53 @HiltViewModel
54 class HomeViewModel @Inject constructor(
55     @ApplicationContext private val context: Context,
56     private val runningRepository: RunningRepository,
57     private val userRepository: UserRepository,
58     private val courseRepository: CourseRepository,
59     private val fusedLocationClient: FusedLocationProviderClient,
60     @Named("noAuth") private val okHttpClient: OkHttpClient,
61 ) : ViewModel() {
62     private val airKoreaClient = AirKoreaClient(okHttpClient)
63
64     val greeting: String = buildGreeting()
65
66     var nickname by mutableStateOf("")
67     private set
68     var weeklyStats by mutableStateOf(WeeklyStats("---", "--", "---'---'", "--"))
69     private set
70     var recentRuns by mutableStateOf<List<RecentRun>>(emptyList())
71     private set
72     var isLoadingRuns by mutableStateOf(true)
73     private set
74     var nearbyCourses by mutableStateOf<List<NearbyCourseItem>>(emptyList())
75     private set
76     var isLoadingNearbyCourses by mutableStateOf(false)
77     private set
78     var weatherInfo by mutableStateOf<WeatherInfo?>(null)
79     private set
80     var currentLocation by mutableStateOf<MapPoint?>(null)
81     private set
82     var selectedGoal by mutableStateOf<RunGoal?>(null)
83     private set

```



```

84     var showGoalSheet by mutableStateOf(false)
85
86     val locationPermissionGranted: Boolean
87         get() = hasLocationPermission()
88
89     fun openGoalSheet() { showGoalSheet = true }
90     fun closeGoalSheet() { showGoalSheet = false }
91
92     fun removeRecentRun(runId: String) {
93         recentRuns = recentRuns.filterNot { it.runId == runId }
94     }
95
96     var isRefreshing by mutableStateOf(false)
97     private set
98
99     fun reloadRecentRuns() {
100         viewModelScope.launch {
101             val result = runningRepository.getMyRuns(page = 0, size = 20)
102             if (result is NetworkResult.Success) {
103                 recentRuns = result.data.content.map { it.toRecentRun() }
104             }
105         }
106     }
107
108     fun refresh() {
109         viewModelScope.launch {
110             isRefreshing = true
111             nearbyCourses = emptyList()
112             weatherInfo = null
113             // Inline the essential awaits to correctly track completion
114             val runsResult = runningRepository.getMyRuns(page = 0, size = HOME_RUN_FETCH_SIZE)
115             if (runsResult is NetworkResult.Success) {
116                 recentRuns = runsResult.data.content.take(RECENT_RUN_COUNT).map { it.toRecentRun() }
117             }
118             val statsResult = runningRepository.getRunningStats("weekly")
119             weeklyStats = resolveWeeklyStats(runsResult, statsResult)
120             loadNearbyCoursesIfPermitted()
121             loadWeatherIfPermitted()
122             isRefreshing = false
123         }
124     }
125
126     fun setGoal(goal: RunGoal) {
127         selectedGoal = goal
128         showGoalSheet = false
129     }
130
131     fun clearGoal() { selectedGoal = null }
132
133     init {
134         loadData()
135     }
136
137     fun tryLoadNearbyCourses() {
138         if (isLoadingNearbyCourses || nearbyCourses.isNotEmpty()) return
139         loadNearbyCoursesIfPermitted()
140     }
141
142     // Called separately from HomeScreen to trigger weather load independently
143     fun tryLoadWeather() {
144         if (weatherInfo != null) return
145         loadWeatherIfPermitted()
146     }
147
148     @SuppressLint("MissingPermission")
149     private fun loadWeatherIfPermitted() {
150         if (!hasLocationPermission()) return
151         viewModelScope.launch {
152             try {
153                 val cts = CancellationTokenSource()
154                 val request = CurrentLocationRequest.Builder()
155                     .setPriority(Priority.PRIORITY_HIGH_ACCURACY)
156                     .setMaxUpdateAgeMillis(0)
157                     .setDurationMillis(12_000)
158                     .build()
159                 val location = fusedLocationClient
160                     .getCurrentLocation(request, cts.token)
161                     .await() ?: return@launch
162                 currentLocation = MapPoint(location.latitude, location.longitude)
163                 fetchWeatherData(location.latitude, location.longitude)
164             } catch (_: Exception) { }
165         }
166     }
167
168     @SuppressLint("MissingPermission")
169     private fun loadNearbyCoursesIfPermitted() {
170         if (!hasLocationPermission()) return
171     }

```

```

170     viewModelScope.launch {
171         isLoadingNearbyCourses = true
172         try {
173             val cts = CancellationTokenSource()
174             val location = fusedLocationClient
175                 .getCurrentLocation(Priority.PRIORITY_BALANCED_POWER_ACCURACY, cts.token)
176                 .await() ?: return@launch
177             if (currentLocation == null) {
178                 currentLocation = MapPoint(location.latitude, location.longitude)
179             }
180
181             when (val result = courseRepository.getNearbyCourses(
182                 latitude = location.latitude,
183                 longitude = location.longitude,
184                 radiusMeters = 3000,
185                 size = 5,
186             )) {
187                 is NetworkResult.Success -> nearbyCourses = result.data.content
188                 else -> {}
189             }
190         } catch (_: Exception) {
191             // silent fail on home screen
192         } finally {
193             isLoadingNearbyCourses = false
194         }
195     }
196 }
197
198 private suspend fun fetchWeatherData(lat: Double, lon: Double) {
199     val apiKey = BuildConfig.OPENWEATHER_API_KEY
200     if (apiKey.isBlank()) return
201
202     try {
203         val weatherResp = withContext(Dispatchers.IO) {
204             val req = Request.Builder()
205                 .url("https://api.openweathermap.org/data/2.5/weather?lat=$lat&lon=$lon&units=metric&lang=kr&appid=$apiKey")
206                 .build()
207             okhttpClient.newCall(req).execute().use { resp ->
208                 if (resp.isSuccessful) Gson().fromJson(resp.body?.string(), OWMWeatherResponse::class.java)
209                 else null
210             }
211         }
212
213         val airResp = withContext(Dispatchers.IO) {
214             val req = Request.Builder()
215                 .url("https://api.openweathermap.org/data/2.5/air_pollution?lat=$lat&lon=$lon&appid=$apiKey")
216                 .build()
217             okhttpClient.newCall(req).execute().use { resp ->
218                 if (resp.isSuccessful) Gson().fromJson(resp.body?.string(), OWMAirPollutionResponse::class.java)
219                 else null
220             }
221         }
222
223         val airKoreaResp = withContext(Dispatchers.IO) {
224             airKoreaClient.getAirQuality(
225                 latitude = lat,
226                 longitude = lon,
227                 serviceKey = BuildConfig.AIRKOREA_API_KEY,
228                 stationCandidates = resolveAirKoreaStationCandidates(lat, lon),
229             )
230         }
231
232         if (weatherResp != null) {
233             val condition = weatherResp.weather.firstOrNull()
234             val fallbackAir = airResp?.list?.firstOrNull()?.components
235             weatherInfo = WeatherInfo(
236                 tempCelsius = weatherResp.main.temp.toInt(),
237                 humidity = weatherResp.main.humidity,
238                 pm10 = airKoreaResp?.pm10 ?: fallbackAir?.pm10?.roundToInt() ?: 0,
239                 pm25 = airKoreaResp?.pm25 ?: fallbackAir?.pm25?.roundToInt() ?: 0,
240                 conditionId = condition?.id ?: 800,
241                 condition = condition?.main.orEmpty(),
242                 description = condition?.description.orEmpty(),
243             )
244         }
245     } catch (_: Exception) {
246         // silent fail - weather is non-critical
247     }
248 }
249
250 @Suppress("DEPRECATION")
251 private fun resolveAirKoreaStationCandidates(
252     latitude: Double,
253     longitude: Double,
254 ): List<String> {
255     if (!Geocoder.isPresent()) return emptyList()

```

```

256     val address = runCatching {
257         Geocoder(context, Locale.KOREA)
258             .getFromLocation(latitude, longitude, 1)
259             ?.firstOrNull()
260     }.getOrNull() ?: return emptyList()
261
262     return listOfNotNull(
263         address.subLocality,
264         address.subAdminArea,
265         address.locality,
266     ).map(String::trim).filter(String::isNotBlank).distinct()
267 }
268
269 private fun loadData() {
270     viewModelScope.launch {
271         launch {
272             val result = userRepository.getMe()
273             if (result is NetworkResult.Success) {
274                 nickname = result.data.nickname
275             }
276         }
277
278         val runsResult = runningRepository.getMyRuns(page = 0, size = HOME_RUN_FETCH_SIZE)
279         val statsResult = runningRepository.getRunningStats("weekly")
280
281         if (runsResult is NetworkResult.Success) {
282             recentRuns = runsResult.data.content.take(RECENT_RUN_COUNT).map { it.toRecentRun() }
283         }
284
285         weeklyStats = resolveWeeklyStats(runsResult, statsResult)
286         isLoadingRuns = false
287     }
288 }
289
290 private fun resolveWeeklyStats(
291     runsResult: NetworkResult<com.runway.android.core.model.PageResponse<RunSummaryResponse>>,
292     statsResult: NetworkResult<RunningStatsResponse>,
293 ): WeeklyStats {
294     val runs = (runsResult as? NetworkResult.Success)?.data?.content
295     val localStats = runs?.let(::calculateWeeklyStats)
296     val localRunCount = runs?.count(::isCurrentWeekCompletedRun) ?: 0
297     val serverStats = (statsResult as? NetworkResult.Success)?.data
298
299     return when {
300         localRunCount > 0 -> localStats!!
301         serverStats != null -> serverStats.toWeeklyStats()
302         localStats != null -> localStats
303         else -> WeeklyStats("0.0", "0", "---\\", "0")
304     }
305 }
306
307 private fun RunningStatsResponse.toWeeklyStats(): WeeklyStats {
308     val pace = if (averagePaceSecondsPerKm > 0) {
309         "%d'%02d\\".format(averagePaceSecondsPerKm / 60, averagePaceSecondsPerKm % 60)
310     } else {
311         "---\\\\"
312     }
313     val streakSuffix = if (currentStreakDays > 0) " · ${currentStreakDays}day streak" else ""
314     return WeeklyStats(
315         distanceKm = if (totalDistanceMeters < 1000) {
316             "%.2f".format(totalDistanceMeters / 1000.0)
317         } else {
318             "%.1f".format(totalDistanceMeters / 1000.0)
319         },
320         runs = "$totalRuns$streakSuffix",
321         avgPace = pace,
322         calories = totalCaloriesBurned.toString(),
323     )
324 }
325
326 private fun calculateWeeklyStats(runs: List<RunSummaryResponse>): WeeklyStats {
327     val weekRuns = runs.filter(::isCurrentWeekCompletedRun)
328
329     val totalDistanceKm = weekRuns.sumOf { it.distanceMeters ?: 0.0 } / 1000.0
330     val totalSeconds = weekRuns.sumOf { it.durationSeconds ?: 0 }
331     val avgPaceSecsPerKm = if (totalDistanceKm > 0.001) (totalSeconds / totalDistanceKm).toInt() else 0
332     val recordedCalories = weekRuns.mapNotNull { it.caloriesBurned }
333     val calories = if (recordedCalories.isNotEmpty()) {
334         recordedCalories.sum()
335     } else {
336         (totalDistanceKm * 72).toInt()
337     }
338
339     return WeeklyStats(
340         distanceKm = "%.1f".format(totalDistanceKm),
341         runs = weekRuns.size.toString(),

```

```

342         avgPace = if (avgPaceSecsPerKm > 0) "%d'%02d\".format(avgPaceSecsPerKm / 60, avgPaceSecsPerKm % 60) else "---'--\\",
343         calories = calories.toString(),
344     )
345 }
346
347 private fun isCurrentWeekCompletedRun(run: RunSummaryResponse): Boolean {
348     if (!run.status.equals("completed", ignoreCase = true)) return false
349
350     val zone = ZoneId.systemDefault()
351     val weekStart = LocalDate.now(zone)
352         .with(TemporalAdjusters.previousOrSame(DayOfWeek.MONDAY))
353         .atStartOfDay(zone)
354         .toInstant()
355     val nextWeekStart = weekStart.plusSeconds(7 * 24 * 60 * 60)
356     val startedAt = runCatching { Instant.parse(run.startedAt) }.getOrNull() ?: return false
357     return startedAt >= weekStart && startedAt < nextWeekStart
358 }
359
360 private fun RunSummaryResponse.toRecentRun(): RecentRun {
361     val zone = ZoneId.systemDefault()
362     val today = LocalDate.now(zone)
363     val dateLabel = runCatching {
364         val dt = Instant.parse(startedAt).atZone(zone).toLocalDate()
365         when {
366             dt == today -> "Today"
367             dt == today.minusDays(1) -> "Yesterday"
368             else -> dt.dayOfWeek.getDisplayName(TextStyle.SHORT, Locale.ENGLISH)
369         }
370     }.getOrDefault("")
371
372     val distKm = "%.2f".format((distanceMeters ?: 0.0) / 1000.0)
373     val dur = formatDuration(durationSeconds)
374     val pace = avgPaceSecondsPerKm?.let { secs ->
375         "%d'%02d\".format(secs / 60, secs % 60)
376     } ?: "---'--\\\"
377
378     return RecentRun(runId = runId, day = dateLabel, distanceKm = distKm, pace = pace, duration = dur)
379 }
380
381 private fun hasLocationPermission(): Boolean {
382     val fine = ContextCompat.checkSelfPermission(context, Manifest.permission.ACCESS_FINE_LOCATION) == PackageManager.PERMISSION_GRANTED
383     val coarse = ContextCompat.checkSelfPermission(context, Manifest.permission.ACCESS_COARSE_LOCATION) == PackageManager.PERMISSION_GRANTED
384     return fine || coarse
385 }
386
387 private fun buildGreeting(): String {
388     return when (Calendar.getInstance().get(Calendar.HOUR_OF_DAY)) {
389         in 5..11 -> "Good morning,"
390         in 12..17 -> "Good afternoon,"
391         else -> "Good evening,"
392     }
393 }
394
395 // — OpenWeatherMap response models —————
396
397 private data class OWMWeatherResponse(
398     val main: OWMMain,
399     val weather: List<OWMWeatherCondition> = emptyList(),
400 )
401 private data class OWMMain(val temp: Double, val humidity: Int)
402 private data class OWMWeatherCondition(
403     val id: Int,
404     val main: String,
405     val description: String,
406 )
407
408 private data class OWMAirPollutionResponse(val list: List<OWMAirEntry>)
409 private data class OWMAirEntry(val components: OWMComponents)
410 private data class OWMComponents(
411     val pm10: Double,
412     @SerializedName("pm2_5") val pm25: Double,
413 )
414
415 private companion object {
416     const val HOME_RUN_FETCH_SIZE = 200
417     const val RECENT_RUN_COUNT = 20
418 }
419 }

```

```

1  package com.runway.android.ui.home
2
3  sealed class RunGoal {
4
5      data class TimeGoal(val targetMinutes: Int) : RunGoal() {
6          fun label() = "${targetMinutes}분"
7      }
8
9      data class DistanceGoal(val targetKm: Float) : RunGoal() {
10         fun label() = if (targetKm == targetKm.toInt().toFloat()) "${targetKm.toInt()}km"
11             else "${targetKm}km"
12     }
13
14     data class IntervalGoal(
15         val warmup: IntervalSegment,
16         val work: IntervalSegment,
17         val recovery: IntervalSegment,
18         val cooldown: IntervalSegment? = null,
19         val sets: Int,
20     ) : RunGoal() {
21         fun label() = "인터벌 ${sets}회"
22     }
23 }
24
25 // — Interval building blocks —————
26
27 sealed class IntervalDuration {
28     data class ByTime(val minutes: Int) : IntervalDuration() {
29         fun label() = "${minutes}분"
30     }
31     data class ByDistance(val meters: Int) : IntervalDuration() {
32         fun label() = if (meters >= 1000) "${"%1f".format(meters / 1000f)}km" else "${meters}m"
33     }
34     object None : IntervalDuration() {
35         fun label() = "없음"
36     }
37 }
38
39 data class IntervalSegment(
40     val duration: IntervalDuration,
41     val paceTargetSecPerKm: Int? = null,    // null = no pace target
42 ) {
43     fun paceLabel() = paceTargetSecPerKm?.let {
44         val m = it / 60; val s = it % 60
45         "%d'%02d\"/km".format(m, s)
46     }
47 }
48
49 // — Preset values —————
50
51 object GoalPresets {
52     val timeMinutes = listOf(10, 15, 20, 30, 45, 60, 90)
53     val distanceKm = listOf(1f, 3f, 5f, 10f, 15f, 21.1f, 42.2f)
54
55     val warmupTimes = listOf(5, 10, 15, 20)           // minutes
56     val warmupDistances = listOf(500, 1000, 1500, 2000) // metres
57
58     val workDistances = listOf(200, 400, 800, 1000, 2000, 3000, 5000) // metres
59     val workPaces = listOf(240, 270, 300, 330, 360, 390, 420) // sec/km
60
61     val recoveryDistances = listOf(100, 200, 400, 800) // metres
62     val recoveryPaces = listOf(360, 390, 420, 450) // sec/km
63
64     val sets = listOf(1, 2, 3, 4, 5, 6, 8, 10)
65
66     fun paceLabel(secPerKm: Int): String {
67         val m = secPerKm / 60; val s = secPerKm % 60
68         return "%d'%02d\".format(m, s)
69     }
70 }

```

```

1 package com.runway.android.ui.leaderboard
2
3 import androidx.compose.animation.AnimatedVisibility
4 import androidx.compose.animation.core.animateFloatAsState
5 import androidx.compose.animation.core.tween
6 import androidx.compose.animation.fadeIn
7 import androidx.compose.animation.slideInVertically
8 import androidx.compose.foundation.BorderStroke
9 import androidx.compose.foundation.background
10 import androidx.compose.foundation.layout.Arrangement
11 import androidx.compose.foundation.layout.Box
12 import androidx.compose.foundation.layout.Column
13 import androidx.compose.foundation.layout.PaddingValues
14 import androidx.compose.foundation.layout.Row
15 import androidx.compose.foundation.layout.Spacer
16 import androidx.compose.foundation.layout.fillMaxSize
17 import androidx.compose.foundation.layout.fillMaxWidth
18 import androidx.compose.foundation.layout.height
19 import androidx.compose.foundation.layout.padding
20 import androidx.compose.foundation.layout.size
21 import androidx.compose.foundation.layout.width
22 import androidx.compose.foundation.lazy.LazyColumn
23 import androidx.compose.foundation.lazy.items
24 import androidx.compose.foundation.lazy.itemsIndexed
25 import kotlin.coroutines.delay
26 import androidx.compose.foundation.shape.CircleShape
27 import androidx.compose.material.icons.Icons
28 import androidx.compose.material.icons.automirrored.filled.ArrowBack
29 import androidx.compose.material.icons.filled.EmojiEvents
30 import androidx.compose.material.icons.filled.Refresh
31 import androidx.compose.material3.CircularProgressIndicator
32 import androidx.compose.material3.ExperimentalMaterial3Api
33 import androidx.compose.material3.FilterChip
34 import androidx.compose.material3.pulltorefresh.PullToRefreshBox
35 import androidx.compose.material3.HorizontalDivider
36 import androidx.compose.material3.Icon
37 import androidx.compose.material3.IconButton
38 import androidx.compose.material3.MaterialTheme
39 import androidx.compose.material3.Scaffold
40 import androidx.compose.material3.Surface
41 import androidx.compose.material3.Tab
42 import androidx.compose.material3.TabRow
43 import androidx.compose.material3.Text
44 import androidx.compose.material3.TopAppBar
45 import androidx.compose.material3.TopAppBarDefaults
46 import androidx.compose.runtime.Composable
47 import androidx.compose.runtime.LaunchedEffect
48 import androidx.compose.runtime.getValue
49 import androidx.compose.runtime.mutableStateOf
50 import androidx.compose.runtime.remember
51 import androidx.compose.runtime.setValue
52 import androidx.compose.ui.Alignment
53 import androidx.compose.ui.draw.drawWithContent
54 import androidx.compose.ui.geometry.Offset
55 import androidx.compose.ui.graphics.Brush
56 import androidx.compose.ui.graphics.Color
57 import androidx.compose.ui.Modifier
58 import androidx.compose.ui.text.font.FontWeight
59 import androidx.compose.ui.text.style.TextAlign
60 import androidx.compose.ui.unit.Dp
61 import androidx.compose.ui.unit.dp
62 import androidx.hilt.navigation.compose.hiltViewModel
63 import com.runway.android.data.attempt.model.LeaderboardItem
64 import com.runway.android.data.attempt.model.LeaderboardResponse
65 import com.runway.android.ui.course.detail.RateCourseDialog
66
67 @OptIn(ExperimentalMaterial3Api::class)
68 @Composable
69 fun CourseLeaderboardScreen(
70     onBack: () -> Unit,
71     viewModel: CourseLeaderboardViewModel = hiltViewModel(),
72 ) {
73     if (viewModel.showRateDialog) {
74         RateCourseDialog(
75             selectedRating = viewModel.ratingValue,
76             onRatingChange = viewModel::onRatingValueChange,
77             comment = viewModel.ratingComment,
78             onCommentChange = viewModel::onRatingCommentChange,
79             isSubmitting = viewModel.isSubmittingRating,
80             errorMessage = viewModel.ratingError,
81             onConfirm = viewModel::submitRating,
82             onDismiss = viewModel::dismissRateDialog,
83         )

```

```

84     }
85
86     Scaffold(
87         topBar = {
88             TopAppBar(
89                 title = {
90                     Text(
91                         text = "리더보드",
92                         style = MaterialTheme.typography.titleMedium,
93                         color = MaterialTheme.colorScheme.onBackground,
94                     )
95                 },
96                 navigationIcon = {
97                     IconButton(onClick = onBackPressed) {
98                         Icon(
99                             imageVector = Icons.AutoMirrored.Filled.ArrowBack,
100                             contentDescription = "뒤로 가기",
101                             tint = MaterialTheme.colorScheme.onBackground,
102                         )
103                     }
104                 },
105                 actions = {
106                     IconButton(onClick = viewModel::refresh) {
107                         Icon(
108                             imageVector = Icons.Filled.Refresh,
109                             contentDescription = "새로고침",
110                             tint = MaterialTheme.colorScheme.onSurfaceVariant,
111                         )
112                     }
113                 },
114                 colors = TopAppBarDefaults.topAppBarColors(
115                     containerColor = MaterialTheme.colorScheme.background,
116                 ),
117             )
118         },
119         containerColor = MaterialTheme.colorScheme.background,
120     ) { innerPadding ->
121         PullToRefreshBox(
122             isRefreshing = viewModel.isRefreshing,
123             onRefresh = viewModel::refresh,
124             modifier = Modifier
125                 .fillMaxSize()
126                 .padding(innerPadding),
127         ) {
128             when {
129                 viewModel.isLoading -> Box(
130                     modifier = Modifier.fillMaxSize(),
131                     contentAlignment = Alignment.Center,
132                 ) {
133                     CircularProgressIndicator(color = MaterialTheme.colorScheme.primary)
134                 }
135
136                 viewModel.errorMessage != null -> Column(
137                     modifier = Modifier
138                         .fillMaxSize()
139                         .padding(32.dp),
140                     horizontalAlignment = Alignment.CenterHorizontally,
141                     verticalArrangement = Arrangement.Center,
142                 ) {
143                     Text(
144                         text = "리더보드를 불러오지 못했습니다",
145                         style = MaterialTheme.typography.titleMedium,
146                         color = MaterialTheme.colorScheme.onSurface,
147                         textAlign = TextAlign.Center,
148                     )
149                     Spacer(modifier = Modifier.height(8.dp))
150                     Text(
151                         text = viewModel.errorMessage!!,
152                         style = MaterialTheme.typography.bodyMedium,
153                         color = MaterialTheme.colorScheme.onSurfaceVariant,
154                         textAlign = TextAlign.Center,
155                     )
156                     Spacer(modifier = Modifier.height(20.dp))
157                     Surface(
158                         onClick = viewModel::refresh,
159                         shape = MaterialTheme.shapes.extraLarge,
160                         color = MaterialTheme.colorScheme.primary,
161                     ) {
162                         Text(
163                             text = "다시 시도",
164                             style = MaterialTheme.typography.labelLarge,
165                             color = MaterialTheme.colorScheme.onPrimary,
166                             modifier = Modifier.padding(horizontal = 24.dp, vertical = 12.dp),
167                         )
168                     }
169                 }
170             }
171         }
172     }

```

```

170
171         viewModel.leaderboard != null -> LeaderboardContent(
172             leaderboard = viewModel.leaderboard!!,
173             sortBy = viewModel.sortBy,
174             onSortChange = viewModel::updateSortBy,
175             isPR = viewModel.isPR,
176             previousBestSeconds = viewModel.previousBestSeconds,
177             improvementSeconds = viewModel.improvementSeconds,
178         )
179     }
180 }
181 }
182 }
183
184 @Composable
185 private fun LeaderboardContent(
186     leaderboard: LeaderboardResponse,
187     sortBy: String,
188     onSortChange: (String) -> Unit,
189     isPR: Boolean = false,
190     previousBestSeconds: Int? = null,
191     improvementSeconds: Int? = null,
192 ) {
193     val items = leaderboard.items
194
195     // 배너 슬라이드인: 첫 composition 후 visible = true로 전환
196     val isFirstCompletion = isPR && previousBestSeconds == null
197     val showBanner = isFirstCompletion ||
198         (isPR && previousBestSeconds != null) ||
199         (!isPR && improvementSeconds != null && previousBestSeconds != null && improvementSeconds <= 0)
200     var bannerVisible by remember { mutableStateOf(false) }
201     LaunchedEffect(Unit) { bannerVisible = true }
202
203     // PR/첫 완주 배너에 1회 shimmer
204     var shimmerTriggered by remember { mutableStateOf(false) }
205     LaunchedEffect(isPR) {
206         if (isPR) {
207             kotlinx.coroutines.delay(400)
208             shimmerTriggered = true
209         }
210     }
211     val shimmerProgress by animateFloatAsState(
212         targetValue = if (shimmerTriggered) 1f else 0f,
213         animationSpec = tween(800),
214         label = "prShimmer",
215     )
216
217     Column {
218         // 완주 결과 배너 (첫 완주 / 신기록 / 회귀)
219         AnimatedVisibility(
220             visible = bannerVisible && showBanner,
221             enter = slideInVertically(
222                 initialOffsetY = { it },
223                 animationSpec = tween(350),
224             ) + fadeIn(tween(350)),
225         ) {
226             when {
227                 isFirstCompletion -> {
228                     Box(
229                         modifier = Modifier
230                             .fillMaxWidth()
231                             .drawWithContent {
232                                 drawContent()
233                                 val center = size.width * (shimmerProgress * 2.5f - 0.5f)
234                                 val halfWidth = size.width * 0.35f
235                                 drawRect(
236                                     brush = Brush.horizontalGradient(
237                                         colors = listOf(
238                                             Color.Transparent,
239                                             Color.White.copy(alpha = 0.28f),
240                                             Color.Transparent,
241                                         ),
242                                     startX = center - halfWidth,
243                                     endX = center + halfWidth,
244                                 ),
245                                     topLeft = Offset.Zero,
246                                     size = size,
247                                 )
248                             },
249                 ) {
250                     Surface(
251                         color = MaterialTheme.colorScheme.primary,
252                         modifier = Modifier.fillMaxWidth(),
253                     ) {
254                         Row(
255                             modifier = Modifier.padding(horizontal = 20.dp, vertical = 14.dp),

```



```

256         horizontalArrangement = Arrangement.spacedBy(10.dp),
257         verticalAlignment = Alignment.CenterVertically,
258     ) {
259         Icon(
260             imageVector = Icons.Filled.EmojiEvents,
261             contentDescription = null,
262             tint = MaterialTheme.colorScheme.onPrimary,
263             modifier = Modifier.size(22.dp),
264         )
265         Column {
266             Text(
267                 text = "첫 완주!",
268                 style = MaterialTheme.typography.titleSmall,
269                 fontWeight = FontWeight.Bold,
270                 color = MaterialTheme.colorScheme.onPrimary,
271             )
272             Text(
273                 text = "이 코스를 처음 완주했습니다. 대단해요!",
274                 style = MaterialTheme.typography.bodySmall,
275                 color = MaterialTheme.colorScheme.onPrimary.copy(alpha = 0.85f),
276             )
277         }
278     }
279 }
280 }
281 }
282 isPR && previousBestSeconds != null -> {
283     Box(
284         modifier = Modifier
285             .fillMaxWidth()
286             .drawWithContent {
287                 drawContent()
288                 val center = size.width * (shimmerProgress * 2.5f - 0.5f)
289                 val halfWidth = size.width * 0.35f
290                 drawRect(
291                     brush = Brush.horizontalGradient(
292                         colors = listOf(
293                             Color.Transparent,
294                             Color.White.copy(alpha = 0.28f),
295                             Color.Transparent,
296                         ),
297                     startX = center - halfWidth,
298                     endX = center + halfWidth,
299                 ),
300                 topLeft = Offset.Zero,
301                 size = size,
302             )
303     ),
304 ) {
305     Surface(
306         color = MaterialTheme.colorScheme.primary,
307         modifier = Modifier.fillMaxWidth(),
308     ) {
309         Row(
310             modifier = Modifier.padding(horizontal = 20.dp, vertical = 14.dp),
311             horizontalArrangement = Arrangement.spacedBy(10.dp),
312             verticalAlignment = Alignment.CenterVertically,
313         ) {
314             Icon(
315                 imageVector = Icons.Filled.EmojiEvents,
316                 contentDescription = null,
317                 tint = MaterialTheme.colorScheme.onPrimary,
318                 modifier = Modifier.size(22.dp),
319             )
320             Column {
321                 Text(
322                     text = "코스 신기록!",
323                     style = MaterialTheme.typography.titleSmall,
324                     fontWeight = FontWeight.Bold,
325                     color = MaterialTheme.colorScheme.onPrimary,
326                 )
327                 if (improvementSeconds != null && improvementSeconds > 0) {
328                     Text(
329                         text = "이전 기록보다 ${formatTime(improvementSeconds)} 빠른 (이전: ${formatTime(previousBestSeco
330 nds)})",
331                         style = MaterialTheme.typography.bodySmall,
332                         color = MaterialTheme.colorScheme.onPrimary.copy(alpha = 0.85f),
333                     )
334                 }
335             }
336         }
337     }
338 }
339 !isPR && improvementSeconds != null && previousBestSeconds != null && improvementSeconds <= 0 -> {
340     Surface(

```

```

341         color = MaterialTheme.colorScheme.surfaceVariant,
342         modifier = Modifier.fillMaxWidth(),
343     ) {
344         Row(
345             modifier = Modifier.padding(horizontal = 20.dp, vertical = 14.dp),
346             horizontalArrangement = Arrangement.spacedBy(10.dp),
347             verticalAlignment = Alignment.CenterVertically,
348         ) {
349             Column {
350                 if (improvementSeconds == 0) {
351                     Text(
352                         text = "지난번과 동일한 기록이에요!",
353                         style = MaterialTheme.typography.titleSmall,
354                         fontWeight = FontWeight.SemiBold,
355                         color = MaterialTheme.colorScheme.onSurfaceVariant,
356                     )
357                     Text(
358                         text = "최고 기록: ${formatTime(previousBestSeconds)} · 다음엔 더 빠르게 달려보세요!",
359                         style = MaterialTheme.typography.bodySmall,
360                         color = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.8f),
361                     )
362                 } else {
363                     val regressionSeconds = -improvementSeconds
364                     Text(
365                         text = "지난번보다 ${formatTime(regressionSeconds)} 느림",
366                         style = MaterialTheme.typography.titleSmall,
367                         fontWeight = FontWeight.SemiBold,
368                         color = MaterialTheme.colorScheme.onSurfaceVariant,
369                     )
370                     Text(
371                         text = "최고 기록: ${formatTime(previousBestSeconds)} · 다음엔 더 잘 달릴 수 있어요!",
372                         style = MaterialTheme.typography.bodySmall,
373                         color = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.8f),
374                     )
375                 }
376             }
377         }
378     }
379 }
380 }
381 }
382
383 // 정렬 필터 탭
384 val tabs = listOf("fastest_time" to "빠른 시간 순", "most_completions" to "완주 횟수 순")
385 val selectedIndex = tabs.indexOfFirst { it.first == sortBy }.coerceAtLeast(0)
386 TabRow(
387     selectedTabIndex = selectedIndex,
388     containerColor = MaterialTheme.colorScheme.background,
389     contentColor = MaterialTheme.colorScheme.primary,
390 ) {
391     tabs.forEachIndexed { index, (key, label) ->
392         Tab(
393             selected = selectedIndex == index,
394             onClick = { onSortChange(key) },
395             text = { Text(label, style = MaterialTheme.typography.labelMedium) },
396         )
397     }
398 }
399
400 // 내 순위 배너
401 leaderboard.myRank?.let { rank ->
402     Surface(
403         color = MaterialTheme.colorScheme.primaryContainer,
404         modifier = Modifier.fillMaxWidth(),
405     ) {
406         Row(
407             modifier = Modifier.padding(horizontal = 20.dp, vertical = 10.dp),
408             horizontalArrangement = Arrangement.SpaceBetween,
409             verticalAlignment = Alignment.CenterVertically,
410         ) {
411             Text(
412                 text = "내 순위",
413                 style = MaterialTheme.typography.labelMedium,
414                 color = MaterialTheme.colorScheme.onPrimaryContainer,
415             )
416             Text(
417                 text = "${rank}위",
418                 style = MaterialTheme.typography.titleMedium,
419                 fontWeight = FontWeight.Bold,
420                 color = MaterialTheme.colorScheme.primary,
421             )
422         }
423     }
424     HorizontalDivider(color = MaterialTheme.colorScheme.outline.copy(alpha = 0.3f))
425 }
426

```

```

427         if (items.isEmpty()) {
428             Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
429                 Column(horizontalAlignment = Alignment.CenterHorizontally) {
430                     Icon(
431                         imageVector = Icons.Filled.EmojiEvents,
432                         contentDescription = null,
433                         tint = MaterialTheme.colorScheme.onSurfaceVariant,
434                         modifier = Modifier.size(48.dp),
435                     )
436                     Spacer(modifier = Modifier.height(12.dp))
437                     Text(
438                         text = "아직 완주 기록이 없습니다",
439                         style = MaterialTheme.typography.titleMedium,
440                         color = MaterialTheme.colorScheme.onSurface,
441                     )
442                     Spacer(modifier = Modifier.height(4.dp))
443                     Text(
444                         text = "이 코스의 첫 번째 완주자가 되어보세요!",
445                         style = MaterialTheme.typography.bodyMedium,
446                         color = MaterialTheme.colorScheme.onSurfaceVariant,
447                     )
448                 }
449             }
450         } else {
451             LazyColumn(
452                 contentPadding = PaddingValues(bottom = 24.dp),
453             ) {
454                 // ——— 시상대 (top 3) ———
455                 if (items.size >= 2) {
456                     item {
457                         Spacer(modifier = Modifier.height(16.dp))
458                         Podium(
459                             items = items.take(3),
460                             modifier = Modifier.padding(horizontal = 20.dp),
461                         )
462                         Spacer(modifier = Modifier.height(20.dp))
463                     }
464                 }
465
466                 // ——— 나머지 순위 ———
467                 itemsIndexed(if (items.size >= 2) items.drop(3) else items) { index, item ->
468                     var visible by remember { mutableStateOf(false) }
469                     LaunchedEffect(item.userId) {
470                         delay(index.coerceAtMost(6) * 60L)
471                         visible = true
472                     }
473                     AnimatedVisibility(
474                         visible = visible,
475                         enter = fadeIn(tween(300)) + slideInVertically(tween(300)) { it / 2 },
476                     ) {
477                         RankRow(
478                             item = item,
479                             modifier = Modifier
480                                 .padding(horizontal = 20.dp)
481                                 .padding(bottom = 8.dp),
482                         )
483                     }
484                 }
485             }
486         }
487     } // Column 닫기
488 }
489
490 @Composable
491 private fun Podium(items: List<LeaderboardItem>, modifier: Modifier = Modifier) {
492     val ordered = listOf(
493         items.getOrNull(1), // 2위 (왼쪽)
494         items.getOrNull(0), // 1위 (가운데, 높음)
495         items.getOrNull(2), // 3위 (오른쪽)
496     )
497     val heights = listOf(80.dp, 112.dp, 64.dp)
498
499     Row(
500         modifier = modifier.fillMaxWidth(),
501         horizontalArrangement = Arrangement.SpaceEvenly,
502         verticalAlignment = Alignment.Bottom,
503     ) {
504         ordered.forEachIndexed { idx, item ->
505             if (item != null) {
506                 PodiumSlot(item = item, barHeight = heights[idx], isFirst = idx == 1)
507             }
508         }
509     }
510 }
511
512 @Composable

```

```

513 private fun PodiumSlot(item: LeaderboardItem, barHeight: Dp, isFirst: Boolean) {
514     Column(
515         horizontalAlignment = Alignment.CenterHorizontally,
516         verticalArrangement = Arrangement.Bottom,
517         modifier = Modifier.width(88.dp),
518     ) {
519         Box(
520             modifier = Modifier
521                 .size(44.dp)
522                 .background(
523                     color = if (isFirst) MaterialTheme.colorScheme.primary
524                     else MaterialTheme.colorScheme.surfaceVariant,
525                     shape = CircleShape,
526                 ),
527             contentAlignment = Alignment.Center,
528         ) {
529             Text(
530                 text = item.nickname.firstOrNull()?.uppercaseChar()?.toString() ?: "R",
531                 style = MaterialTheme.typography.titleMedium,
532                 color = if (isFirst) MaterialTheme.colorScheme.onPrimary
533                 else MaterialTheme.colorScheme.onSurfaceVariant,
534             )
535         }
536         Spacer(modifier = Modifier.height(4.dp))
537         Text(
538             text = item.nickname,
539             style = MaterialTheme.typography.bodyMedium,
540             color = MaterialTheme.colorScheme.onBackground,
541             maxLines = 1,
542         )
543         Text(
544             text = formatTime(item.bestTimeSeconds),
545             style = MaterialTheme.typography.labelLarge,
546             color = MaterialTheme.colorScheme.primary,
547         )
548         Spacer(modifier = Modifier.height(4.dp))
549         Surface(
550             modifier = Modifier
551                 .fillMaxWidth()
552                 .height(barHeight),
553             shape = MaterialTheme.shapes.medium,
554             color = if (isFirst) MaterialTheme.colorScheme.primary.copy(alpha = 0.2f)
555             else MaterialTheme.colorScheme.surface,
556             border = if (isFirst)
557                 BorderStroke(1.dp, MaterialTheme.colorScheme.primary.copy(alpha = 0.4f))
558             else
559                 BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
560         ) {
561             Box(contentAlignment = Alignment.TopCenter) {
562                 Row(
563                     modifier = Modifier.padding(top = 8.dp),
564                     verticalAlignment = Alignment.CenterVertically,
565                 ) {
566                     Icon(
567                         imageVector = Icons.Filled.EmojiEvents,
568                         contentDescription = null,
569                         tint = if (isFirst) MaterialTheme.colorScheme.primary
570                         else MaterialTheme.colorScheme.onSurfaceVariant,
571                         modifier = Modifier.size(14.dp),
572                     )
573                     Spacer(modifier = Modifier.width(2.dp))
574                     Text(
575                         text = item.rank.toString(),
576                         style = MaterialTheme.typography.labelLarge,
577                         fontWeight = FontWeight.Bold,
578                         color = if (isFirst) MaterialTheme.colorScheme.primary
579                         else MaterialTheme.colorScheme.onSurfaceVariant,
580                     )
581                 }
582             }
583         }
584     }
585 }
586
587 @Composable
588 private fun RankRow(item: LeaderboardItem, modifier: Modifier = Modifier) {
589     Surface(
590         modifier = modifier.fillMaxWidth(),
591         shape = MaterialTheme.shapes.extraLarge,
592         color = MaterialTheme.colorScheme.surface,
593         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
594     ) {
595         Row(
596             modifier = Modifier.padding(horizontal = 16.dp, vertical = 12.dp),
597             verticalAlignment = Alignment.CenterVertically,
598         ) {

```

```

599     Text(
600         text = item.rank.toString(),
601         style = MaterialTheme.typography.labelLarge,
602         color = MaterialTheme.colorScheme.onSurfaceVariant,
603         modifier = Modifier.width(24.dp),
604     )
605     Spacer(modifier = Modifier.width(12.dp))
606     Box(
607         modifier = Modifier
608             .size(32.dp)
609             .background(MaterialTheme.colorScheme.surfaceVariant, CircleShape),
610         contentAlignment = Alignment.Center,
611     ) {
612         Text(
613             text = item.nickname.firstOrNull()?.uppercaseChar()?.toString() ?: "R",
614             style = MaterialTheme.typography.bodyMedium,
615             fontWeight = FontWeight.SemiBold,
616             color = MaterialTheme.colorScheme.onSurfaceVariant,
617         )
618     }
619     Spacer(modifier = Modifier.width(12.dp))
620     Column(modifier = Modifier.weight(1f)) {
621         Text(
622             text = item.nickname,
623             style = MaterialTheme.typography.bodyLarge,
624             color = MaterialTheme.colorScheme.onSurface,
625         )
626         Text(
627             text = "원주 ${item.completionCount}회",
628             style = MaterialTheme.typography.bodySmall,
629             color = MaterialTheme.colorScheme.onSurfaceVariant,
630         )
631     }
632     Text(
633         text = formatTime(item.bestTimeSeconds),
634         style = MaterialTheme.typography.labelLarge,
635         color = MaterialTheme.colorScheme.primary,
636     )
637 }
638 }
639 }
640
641 private fun formatTime(seconds: Int): String {
642     val h = seconds / 3600
643     val m = (seconds % 3600) / 60
644     val s = seconds % 60
645     return if (h > 0) "%d:%02d:%02d".format(h, m, s)
646     else "%d:%02d".format(m, s)
647 }

```

```

1 package com.runway.android.ui.leaderboard
2
3 import androidx.compose.runtime.getValue
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.compose.runtime.setValue
6 import androidx.lifecycle.SavedStateHandle
7 import androidx.lifecycle.ViewModel
8 import androidx.lifecycle.viewModelScope
9 import com.runway.android.core.result.NetworkResult
10 import com.runway.android.data.attempt.model.LeaderboardResponse
11 import com.runway.android.data.course.model.CourseRatingRequest
12 import com.runway.android.domain.attempt.CourseAttemptRepository
13 import com.runway.android.domain.course.CourseRepository
14 import dagger.hilt.android.lifecycle.HiltViewModel
15 import kotlinx.coroutines.launch
16 import javax.inject.Inject
17
18 @HiltViewModel
19 class CourseLeaderboardViewModel @Inject constructor(
20     savedStateHandle: SavedStateHandle,
21     private val courseAttemptRepository: CourseAttemptRepository,
22     private val courseRepository: CourseRepository,
23 ) : ViewModel() {
24
25     val courseId: String = checkNotNull(savedStateHandle["courseId"])
26
27     val isPR: Boolean = savedStateHandle["isPR"] ?: false
28     val previousBestSeconds: Int? = (savedStateHandle["previousBestSeconds"] as? Int)?.takeIf { it != Int.MIN_VALUE }
29     val improvementSeconds: Int? = (savedStateHandle["improvementSeconds"] as? Int)?.takeIf { it != Int.MIN_VALUE }
30     val justCompleted: Boolean = savedStateHandle["justCompleted"] ?: false
31
32     var isLoading by mutableStateOf(false)
33     private set
34     var isRefreshing by mutableStateOf(false)
35     private set
36     var errorMessage by mutableStateOf<String?>(null)
37     private set
38     var leaderboard by mutableStateOf<LeaderboardResponse?>(null)
39     private set
40     var sortBy by mutableStateOf("fastest_time")
41     private set
42
43     var showRateDialog by mutableStateOf(justCompleted)
44     private set
45     var ratingValue by mutableStateOf(0)
46     private set
47     var ratingComment by mutableStateOf("")
48     private set
49     var isSubmittingRating by mutableStateOf(false)
50     private set
51     var ratingError by mutableStateOf<String?>(null)
52     private set
53     var ratingSubmitted by mutableStateOf(false)
54     private set
55
56     init {
57         load()
58     }
59
60     fun refresh() {
61         viewModelScope.launch {
62             isRefreshing = true
63             doLoad()
64             isRefreshing = false
65         }
66     }
67
68     fun dismissRateDialog() {
69         if (!isSubmittingRating) showRateDialog = false
70     }
71
72     fun onRatingValueChange(value: Int) {
73         ratingValue = value
74         ratingError = null
75     }
76
77     fun onRatingCommentChange(value: String) {
78         ratingComment = value
79     }
80
81     fun submitRating() {
82         if (isSubmittingRating || ratingValue == 0) return
83         isSubmittingRating = true

```

```

84     ratingError = null
85     viewModelScope.launch {
86         when (val result = courseRepository.rateCourse(
87             courseId = courseId,
88             request = CourseRatingRequest(
89                 rating = ratingValue,
90                 comment = ratingComment.trim().ifBlank { null },
91             ),
92         )) {
93             is NetworkResult.Success -> {
94                 ratingSubmitted = true
95                 showRateDialog = false
96             }
97             is NetworkResult.ApiError -> ratingError = when (result.errorCode) {
98                 "ALREADY_RATED" -> "이미 평가한 코스입니다."
99                 else -> result.message
100             }
101             is NetworkResult.NetworkError -> ratingError = "네트워크 연결을 확인해 주세요."
102         }
103         isSubmittingRating = false
104     }
105 }
106
107 fun updateSortBy(value: String) {
108     if (sortBy == value) return
109     sortBy = value
110     load()
111 }
112
113 private fun load() {
114     viewModelScope.launch { doLoad() }
115 }
116
117 private suspend fun doLoad() {
118     isLoading = true
119     errorMessage = null
120     when (val result = courseAttemptRepository.getLeaderboard(courseId, sortBy = sortBy)) {
121         is NetworkResult.Success -> leaderboard = result.data
122         is NetworkResult.ApiError -> errorMessage = result.message
123         is NetworkResult.NetworkError -> errorMessage = "네트워크 연결을 확인해 주세요."
124     }
125     isLoading = false
126 }
127 }

```

```
1 package com.runway.android.ui.leaderboard
2
3 import androidx.compose.foundation.layout.Arrangement
4 import androidx.compose.foundation.layout.Box
5 import androidx.compose.foundation.layout.Column
6 import androidx.compose.foundation.layout.Spacer
7 import androidx.compose.foundation.layout.fillMaxSize
8 import androidx.compose.foundation.layout.height
9 import androidx.compose.foundation.layout.padding
10 import androidx.compose.foundation.layout.size
11 import androidx.compose.material.icons.Icons
12 import androidx.compose.material.icons.filled.EmojiEvents
13 import androidx.compose.material3.Icon
14 import androidx.compose.material3.MaterialTheme
15 import androidx.compose.material3.Text
16 import androidx.compose.runtime.Composable
17 import androidx.compose.ui.Alignment
18 import androidx.compose.ui.Modifier
19 import androidx.compose.ui.text.style.TextAlign
20 import androidx.compose.ui.unit.dp
21
22 @Composable
23 fun LeaderboardScreen(
24     onNavigateToDiscover: () -> Unit = {},
25 ) {
26     Box(
27         modifier = Modifier
28             .fillMaxSize()
29             .padding(24.dp),
30         contentAlignment = Alignment.Center,
31     ) {
32         Column(
33             horizontalAlignment = Alignment.CenterHorizontally,
34             verticalArrangement = Arrangement.Center,
35         ) {
36             Icon(
37                 imageVector = Icons.Filled.EmojiEvents,
38                 contentDescription = null,
39                 tint = MaterialTheme.colorScheme.primary,
40                 modifier = Modifier.size(64.dp),
41             )
42             Spacer(modifier = Modifier.height(20.dp))
43             Text(
44                 text = "리더보드",
45                 style = MaterialTheme.typography.headlineMedium,
46                 color = MaterialTheme.colorScheme.onBackground,
47             )
48             Spacer(modifier = Modifier.height(8.dp))
49             Text(
50                 text = "코스별 리더보드는 코스 상세 화면에서 확인할 수 있어요.\n탐색 탭에서 코스를 찾아 도전해보세요!",
51                 style = MaterialTheme.typography.bodyMedium,
52                 color = MaterialTheme.colorScheme.onSurfaceVariant,
53                 textAlign = TextAlign.Center,
54             )
55         }
56     }
57 }
```



```

1  package com.runway.android.ui.navigation
2
3  import android.widget.Toast
4  import androidx.activity.compose.BackHandler
5  import androidx.compose.foundation.layout.Box
6  import androidx.compose.foundation.layout.fillMaxSize
7  import androidx.compose.foundation.layout.padding
8  import androidx.compose.material3.Scaffold
9  import androidx.compose.runtime.Composable
10 import androidx.compose.runtime.LaunchedEffect
11 import androidx.compose.runtime.collectAsState
12 import androidx.compose.runtime.getValue
13 import androidx.compose.runtime.mutableIntStateOf
14 import androidx.compose.runtime.mutableStateOf
15 import androidx.compose.runtime.remember
16 import androidx.compose.runtime.saveable.rememberSaveable
17 import androidx.compose.runtime.setValue
18 import androidx.compose.ui.Modifier
19 import androidx.compose.ui.platform.LocalContext
20 import androidx.hilt.navigation.compose.hiltViewModel
21 import com.runway.android.ui.components.RunwayBottomNav
22 import com.runway.android.ui.discover.DiscoverScreen
23 import com.runway.android.ui.home.HomeScreen
24 import com.runway.android.ui.home.HomeViewModel
25 import com.runway.android.ui.course.library.CoursesLibraryScreen
26 import com.runway.android.ui.posture.PostureAnalysisState
27 import com.runway.android.core.posture.local.MAX_POSTURE_ANALYSIS_HISTORY
28 import com.runway.android.ui.posture.PostureAnalysisViewModel
29 import com.runway.android.ui.posture.PostureAnalyzingScreen
30 import com.runway.android.ui.posture.PostureHomeScreen
31 import com.runway.android.ui.posture.PostureResultScreen
32 import com.runway.android.ui.profile.ProfileScreen
33 import com.runway.android.ui.tracking.TrackingRecoveryDialog
34 import com.runway.android.ui.tracking.TrackingRecoveryViewModel
35
36 @Composable
37 fun MainScaffold(
38     onStartRun: () -> Unit = {},
39     onLogout: () -> Unit = {},
40     onNavigateToCourseDetail: (String) -> Unit = {},
41     onNavigateToMyRuns: () -> Unit = {},
42     onNavigateToCourses: () -> Unit = {},
43     onNavigateToRunDetail: (String) -> Unit = {},
44     onNavigateToStats: () -> Unit = {},
45     onNavigateToAchievements: () -> Unit = {},
46     onNavigateToReminder: () -> Unit = {},
47     onNavigateToSettings: () -> Unit = {},
48     homeViewModel: HomeViewModel = hiltViewModel(),
49 ) {
50     val context = LocalContext.current
51     val recoveryViewModel: TrackingRecoveryViewModel = hiltViewModel()
52     val postureViewModel: PostureAnalysisViewModel = hiltViewModel()
53
54     var currentTabOrdinal by rememberSaveable { mutableIntStateOf(MainTab.HOME.ordinal) }
55     val currentTab = MainTab.entries[currentTabOrdinal]
56     var showPostureCapture by remember { mutableStateOf(false) }
57     var postureResultId by remember { mutableStateOf<String?>(null) }
58     val postureState by postureViewModel.analysisState.collectAsState()
59     val postureHistory by postureViewModel.analysisHistory.collectAsState()
60     val canCreatePostureLimitAnalysis = postureHistory.size < MAX_POSTURE_ANALYSIS_HISTORY
61     val showPostureLimitMessage = {
62         Toast.makeText(
63             context,
64             "분석 이력은 최대 ${MAX_POSTURE_ANALYSIS_HISTORY}개입니다. 기존 이력을 삭제해주세요.",
65             Toast.LENGTH_LONG,
66         ).show()
67     }
68
69     // Handle analysis errors without mutating state during composition
70     LaunchedEffect(postureState) {
71         if (postureState is PostureAnalysisState.Error) {
72             val msg = (postureState as PostureAnalysisState.Error).message
73             Toast.makeText(context, msg, Toast.LENGTH_LONG).show()
74             postureViewModel.resetState()
75         }
76     }
77
78     BackHandler(enabled = currentTab != MainTab.HOME) {
79         when {
80             currentTab == MainTab.POSTURE && showPostureCapture -> showPostureCapture = false
81             currentTab == MainTab.POSTURE && postureResultId != null -> postureResultId = null
82             currentTab == MainTab.POSTURE && postureState is PostureAnalysisState.Success -> postureViewModel.resetState()
83             else -> currentTabOrdinal = MainTab.HOME.ordinal

```

```

84     }
85 }
86
87 Scaffold(
88     bottomBar = {
89         RunwayBottomNav(
90             currentTab = currentTab,
91             onTabSelected = { currentTabOrdinal = it.ordinal },
92         )
93     },
94 ) { innerPadding ->
95     Box(
96         modifier = Modifier
97             .fillMaxSize()
98             .padding(innerPadding),
99     ) {
100         when (currentTab) {
101             MainTab.HOME -> HomeScreen(
102                 onStartRun = onStartRun,
103                 onSeeAllRuns = onNavigateToMyRuns,
104                 onNavigateToCourseDetail = onNavigateToCourseDetail,
105                 onNavigateToDiscover = { currentTabOrdinal = MainTab.DISCOVER.ordinal },
106                 onNavigateToRunDetail = onNavigateToRunDetail,
107                 viewModel = homeViewModel,
108             )
109             MainTab.DISCOVER -> DiscoverScreen(
110                 onNavigateToCourseDetail = onNavigateToCourseDetail,
111             )
112             MainTab.COURSES -> CoursesLibraryScreen(
113                 onNavigateToCourseDetail = onNavigateToCourseDetail,
114             )
115             MainTab.PROFILE -> ProfileScreen(
116                 onLogout = onLogout,
117                 onNavigateToMyRuns = onNavigateToMyRuns,
118                 onNavigateToCourses = onNavigateToCourses,
119                 onNavigateToRunDetail = onNavigateToRunDetail,
120                 onNavigateToStats = onNavigateToStats,
121                 onNavigateToAchievements = onNavigateToAchievements,
122                 onNavigateToReminder = onNavigateToReminder,
123                 onNavigateToSettings = onNavigateToSettings,
124             )
125             MainTab.POSTURE -> when {
126                 showPostureCapture -> {
127                     com.runway.android.ui.posture.PostureCaptureScreen(
128                         onVideoReady = { uri ->
129                             showPostureCapture = false
130                             postureViewModel.analyze(uri)
131                         },
132                         onBack = { showPostureCapture = false },
133                     )
134                 }
135                 postureState is PostureAnalysisState.Analyzing -> {
136                     PostureAnalyzingScreen()
137                 }
138                 postureState is PostureAnalysisState.Success -> {
139                     val success = postureState as PostureAnalysisState.Success
140                     PostureResultScreen(
141                         analysisId = success.id,
142                         result = success.result,
143                         onBack = { postureViewModel.resetState() },
144                         onRetake = {
145                             if (canCreatePostureAnalysis) {
146                                 postureViewModel.resetState()
147                                 showPostureCapture = true
148                             } else {
149                                 showPostureLimitMessage()
150                             }
151                         },
152                         viewModel = postureViewModel,
153                     )
154                 }
155                 postureResultId != null -> {
156                     PostureResultScreen(
157                         analysisId = postureResultId,
158                         result = null,
159                         onBack = { postureResultId = null },
160                         onRetake = {
161                             if (canCreatePostureAnalysis) {
162                                 postureResultId = null
163                                 showPostureCapture = true
164                             } else {
165                                 showPostureLimitMessage()
166                             }
167                         },
168                         viewModel = postureViewModel,
169                     )

```

```

170         }
171         else -> {
172             PostureHomeScreen(
173                 onStartCapture = { showPostureCapture = true },
174                 onSelectVideo = { uri -> postureViewModel.analyze(uri) },
175                 onOpenResult = { postureResultId = it },
176                 viewModel = postureViewModel,
177             )
178         }
179     }
180 }
181
182 if (recoveryViewModel.isVisible) {
183     TrackingRecoveryDialog(viewModel = recoveryViewModel)
184 }
185 }
186 }
187 }

```

android/app/src/main/java/com/runway/android/ui/navigation/MainTab.kt

Kotlin | 20줄

```

1 package com.runway.android.ui.navigation
2
3 import androidx.compose.material.icons.Icons
4 import androidx.compose.material.icons.automirrored.filled.DirectionsRun
5 import androidx.compose.material.icons.filled.Explore
6 import androidx.compose.material.icons.filled.Home
7 import androidx.compose.material.icons.filled.Person
8 import androidx.compose.material.icons.filled.Route
9 import androidx.compose.ui.graphics.vector.ImageVector
10
11 enum class MainTab(
12     val icon: ImageVector,
13     val label: String,
14 ) {
15     HOME(Icons.Filled.Home, "홈"),
16     DISCOVER(Icons.Filled.Explore, "탐색"),
17     COURSES(Icons.Filled.Route, "코스"),
18     POSTURE(Icons.AutoMirrored.Filled.DirectionsRun, "자세"),
19     PROFILE(Icons.Filled.Person, "내 정보"),
20 }

```

```

1  package com.runway.android.ui.navigation
2
3  import androidx.compose.animation.core.Tween
4  import androidx.compose.animation.fadeIn
5  import androidx.compose.animation.fadeOut
6  import androidx.compose.animation.scaleIn
7  import androidx.compose.animation.scaleOut
8  import androidx.compose.runtime.Composable
9  import androidx.compose.runtime.LaunchedEffect
10 import androidx.compose.runtime.collectAsState
11 import androidx.compose.runtime.getValue
12 import androidx.hilt.navigation.compose.hiltViewModel
13 import androidx.navigation.NavType
14 import androidx.navigation.compose.NavHost
15 import androidx.navigation.compose.composable
16 import androidx.navigation.compose.rememberNavController
17 import androidx.navigation.navArgument
18 import androidx.navigation.navDeepLink
19 import com.runway.android.ui.MainViewModel
20 import com.runway.android.ui.achievements.AchievementsScreen
21 import com.runway.android.ui.home.HomeViewModel
22 import com.runway.android.ui.running.history.MyRunsViewModel
23 import com.runway.android.ui.permission.PermissionScreen
24 import com.runway.android.ui.attempt.CourseAttemptTrackingScreen
25 import com.runway.android.ui.auth.login.LoginScreen
26 import com.runway.android.ui.share.RunShareImageScreen
27 import com.runway.android.ui.auth.signup.SignupScreen
28 import com.runway.android.ui.course.detail.CourseDetailScreen
29 import com.runway.android.ui.course.detail.CourseMapDetailScreen
30 import com.runway.android.ui.course.my.MyCoursesScreen
31 import com.runway.android.ui.leaderboard.CourseLeaderboardScreen
32 import com.runway.android.ui.onboarding.OnboardingScreen
33 import com.runway.android.ui.running.RunResultScreen
34 import com.runway.android.ui.running.RunningTrackingScreen
35 import com.runway.android.ui.reminder.ReminderScreen
36 import com.runway.android.ui.settings.SettingsScreen
37 import com.runway.android.ui.running.history.MyRunsScreen
38 import com.runway.android.ui.running.history.RunDetailScreen
39 import com.runway.android.ui.splash.RunwaySplashScreen
40 import com.runway.android.ui.stats.StatsScreen
41
42 @Composable
43 fun RunwayNavGraph() {
44     val mainViewModel: MainViewModel = hiltViewModel()
45     val isLoggedIn by mainViewModel.isLoggedIn.collectAsState()
46     val isOnboardingCompleted by mainViewModel.isOnboardingCompleted.collectAsState()
47     val watchPendingCourseId by mainViewModel.watchPendingCourseId.collectAsState()
48     val watchPendingRunId by mainViewModel.watchPendingRunId.collectAsState()
49     val navController = rememberNavController()
50
51     LaunchedEffect(isLoggedIn) {
52         if (isLoggedIn == false) {
53             val currentRoute = navController.currentDestination?.route
54             if (currentRoute != null &&
55                 currentRoute != RunwayRoutes.SPLASH &&
56                 currentRoute != RunwayRoutes.LOGIN &&
57                 currentRoute != RunwayRoutes.SIGNUP
58             ) {
59                 navController.navigate(RunwayRoutes.LOGIN) {
60                     popUpTo(0) { inclusive = true }
61                 }
62             }
63         }
64     }
65
66     NavHost(
67         navController = navController,
68         startDestination = RunwayRoutes.SPLASH,
69     ) {
70         composable(RunwayRoutes.SPLASH) {
71             RunwaySplashScreen(
72                 onAnimationFinished = {
73                     val destination = when {
74                         isLoggedIn == false -> RunwayRoutes.LOGIN
75                         isLoggedIn == true && isOnboardingCompleted == false -> RunwayRoutes.ONBOARDING
76                         isLoggedIn == true && isOnboardingCompleted == true -> RunwayRoutes.MAIN
77                         else -> RunwayRoutes.LOGIN
78                     }
79                     navController.navigate(destination) {
80                         popUpTo(RunwayRoutes.SPLASH) { inclusive = true }
81                         launchSingleTop = true
82                     }
83                 },

```

```

84     )
85 }
86
87 // — Auth —
88
89 composable(
90     route = RunwayRoutes.LOGIN,
91     deepLinks = listOf(navDeepLink { uriPattern = "pathfinder://auth/login" }),
92 ) {
93     LoginScreen(
94         onNavigateToSignup = { navController.navigate(RunwayRoutes.SIGNUP) },
95         onLoginSuccess = {
96             val dest = if (mainViewModel.isOnboardingCompleted.value == false)
97                 RunwayRoutes.ONBOARDING else RunwayRoutes.MAIN
98             navController.navigate(dest) {
99                 popUpTo(0) { inclusive = true }
100             }
101         },
102     )
103 }
104
105 composable(RunwayRoutes.SIGNUP) {
106     SignupScreen(
107         onNavigateBack = { navController.popBackStack() },
108         onSignupSuccess = {
109             navController.navigate(RunwayRoutes.LOGIN) {
110                 popUpTo(RunwayRoutes.SIGNUP) { inclusive = true }
111             }
112         },
113     )
114 }
115
116 // — Onboarding —
117
118 composable(RunwayRoutes.ONBOARDING) {
119     OnboardingScreen(
120         onComplete = {
121             navController.navigate(RunwayRoutes.PERMISSION) {
122                 popUpTo(RunwayRoutes.ONBOARDING) { inclusive = true }
123             }
124         },
125     )
126 }
127
128 // — 권한 설명 —
129
130 composable(RunwayRoutes.PERMISSION) {
131     PermissionScreen(
132         onContinue = {
133             navController.navigate(RunwayRoutes.MAIN) {
134                 popUpTo(0) { inclusive = true }
135             }
136         },
137     )
138 }
139
140 // — Main shell (BottomNav 포함) —
141
142 composable(RunwayRoutes.MAIN) { backStackEntry ->
143     LaunchedEffect(watchPendingCourseId) {
144         val courseId = watchPendingCourseId ?: return@LaunchedEffect
145         mainViewModel.consumeWatchPendingCourse()
146         navController.navigate(RunwayRoutes.courseDetail(courseId))
147     }
148     LaunchedEffect(watchPendingRunId) {
149         val runId = watchPendingRunId ?: return@LaunchedEffect
150         mainViewModel.consumeWatchPendingRun()
151         navController.navigate(RunwayRoutes.runDetail(runId))
152     }
153     val homeViewModel: HomeViewModel = hiltViewModel()
154     val deletedRunId by backStackEntry.savedStateHandle
155         .getStateFlow<String?>("deletedRunId", null)
156         .collectAsState()
157     LaunchedEffect(deletedRunId) {
158         deletedRunId?.let {
159             homeViewModel.removeRecentRun(it)
160             backStackEntry.savedStateHandle.remove<String>("deletedRunId")
161         }
162     }
163     val newRunCompleted by backStackEntry.savedStateHandle
164         .getStateFlow("newRunCompleted", false)
165         .collectAsState()
166     LaunchedEffect(newRunCompleted) {
167         if (newRunCompleted) {
168             homeViewModel.reloadRecentRuns()
169             backStackEntry.savedStateHandle["newRunCompleted"] = false
170         }
171     }
172 }

```

```

170     }
171 }
172 MainScaffold(
173     onStartRun = { navController.navigate(RunwayRoutes.RUNNING) },
174     onLogout = {
175         navController.navigate(RunwayRoutes.LOGIN) {
176             popUpTo(0) { inclusive = true }
177         }
178     },
179     onNavigateToCourseDetail = { courseId ->
180         navController.navigate(RunwayRoutes.courseDetail(courseId))
181     },
182     onNavigateToMyRuns = { navController.navigate(RunwayRoutes.MY_RUNS) },
183     onNavigateToCourses = { navController.navigate(RunwayRoutes.MY_COURSES) },
184     onNavigateToRunDetail = { runId ->
185         navController.navigate(RunwayRoutes.runDetail(runId))
186     },
187     onNavigateToStats = { navController.navigate(RunwayRoutes.STATS) },
188     onNavigateToAchievements = { navController.navigate(RunwayRoutes.ACHIEVEMENTS) },
189     onNavigateToReminder = { navController.navigate(RunwayRoutes.REMINDER) },
190     onNavigateToSettings = { navController.navigate(RunwayRoutes.SETTINGS) },
191     homeViewModel = homeViewModel,
192 )
193 }
194
195 // — 자유 러닝 (BottomNav 없음) —
196
197 composable(RunwayRoutes.RUNNING) {
198     RunningTrackingScreen(
199         onFinish = { runId, elapsedSeconds, distanceKm ->
200             navController.navigate(
201                 RunwayRoutes.runResult(runId ?: "none", elapsedSeconds, distanceKm),
202             ) {
203                 popUpTo(RunwayRoutes.RUNNING) { inclusive = true }
204             }
205         },
206         onBack = { navController.popBackStack() },
207     )
208 }
209
210 composable(
211     route = "${RunwayRoutes.RUN_RESULT}/{runId}/{elapsedSeconds}/{distanceKm}",
212     arguments = listOf(
213         navArgument("runId") { type = NavType.StringType },
214         navArgument("elapsedSeconds") { type = NavType.IntType },
215         navArgument("distanceKm") { type = NavType.FloatType },
216     ),
217 ) {
218     RunResultScreen(
219         onBackToHome = {
220             runCatching {
221                 navController.getBackStackEntry(RunwayRoutes.MAIN)
222                     .savedStateHandle["newRunCompleted"] = true
223             }
224             navController.popBackStack(RunwayRoutes.MAIN, inclusive = false)
225         },
226         onShareImage = { runId ->
227             navController.navigate(RunwayRoutes.runShare(runId))
228         },
229         onOpenRunDetail = { runId ->
230             navController.navigate(RunwayRoutes.runDetail(runId))
231         },
232         onNavigateToCourseDetail = { courseId ->
233             runCatching {
234                 navController.getBackStackEntry(RunwayRoutes.MAIN)
235                     .savedStateHandle["newRunCompleted"] = true
236             }
237             navController.navigate(RunwayRoutes.courseDetail(courseId)) {
238                 popUpTo(RunwayRoutes.MAIN) { inclusive = false }
239             }
240         },
241     )
242 }
243
244 // — 코스 상세 (BottomNav 없음) —
245
246 composable(
247     route = RunwayRoutes.COURSE_DETAIL,
248     arguments = listOf(navArgument("courseId") { type = NavType.StringType }),
249     enterTransition = { scaleIn(initialScale = 0.92f, animationSpec = tween(350)) + fadeIn(tween(350)) },
250     popExitTransition = { scaleOut(targetScale = 0.92f, animationSpec = tween(300)) + fadeOut(tween(300)) },
251 ) {
252     CourseDetailScreen(
253         onBack = { navController.popBackStack() },
254         onNavigateToAttempt = { courseId, courseAttemptId, runningRecordId ->
255             navController.navigate(

```

```

256         RunwayRoutes.courseAttempt(courseId, courseAttemptId, runningRecordId)
257     )
258 },
259     onNavigateToLeaderboard = { courseId ->
260         navController.navigate(RunwayRoutes.courseLeaderboard(courseId))
261     },
262     onNavigateToMap = { courseId ->
263         navController.navigate(RunwayRoutes.courseMapDetail(courseId))
264     },
265 )
266 }
267
268 // — 코스 전체 지도 (BottomNav 없음) —
269
270 composable(
271     route = RunwayRoutes.COURSE_MAP_DETAIL,
272     arguments = listOf(navArgument("courseId") { type = NavType.StringType }),
273 ) {
274     CourseMapDetailScreen(
275         onBack = { navController.popBackStack() },
276     )
277 }
278
279 // — 코스 도전 트래킹 (BottomNav 없음) —
280
281 composable(
282     route = RunwayRoutes.COURSE_ATTEMPT,
283     arguments = listOf(
284         navArgument("courseId") { type = NavType.StringType },
285         navArgument("courseAttemptId") { type = NavType.StringType },
286         navArgument("runningRecordId") { type = NavType.StringType },
287     ),
288 ) {
289     CourseAttemptTrackingScreen(
290         onNavigateToLeaderboard = { courseId, isPR, previousBestSeconds, improvementSeconds ->
291             navController.navigate(
292                 RunwayRoutes.courseLeaderboard(courseId, isPR, previousBestSeconds, improvementSeconds, justCompleted = true)
293             ) {
294                 popUpTo(RunwayRoutes.COURSE_ATTEMPT) { inclusive = true }
295             }
296         },
297         onNavigateBack = { navController.popBackStack() },
298     )
299 }
300
301 // — 내 러닝 기록 목록 (BottomNav 없음) —
302
303 composable(RunwayRoutes.MY_RUNS) { backStackEntry ->
304     val myRunsViewModel: MyRunsViewModel = hiltViewModel()
305     val deletedRunId by backStackEntry.savedStateHandle
306         .getStateFlow<String?>("deletedRunId", null)
307         .collectAsState()
308     LaunchedEffect(deletedRunId) {
309         deletedRunId?.let {
310             myRunsViewModel.removeRunLocally(it)
311             backStackEntry.savedStateHandle.remove<String>("deletedRunId")
312         }
313     }
314     MyRunsScreen(
315         onBack = { navController.popBackStack() },
316         onNavigateToDetail = { runId ->
317             navController.navigate(RunwayRoutes.runDetail(runId))
318         },
319         viewModel = myRunsViewModel,
320     )
321 }
322
323 // — 러닝 기록 상세 (BottomNav 없음) —
324
325 composable(
326     route = RunwayRoutes.RUN_DETAIL,
327     arguments = listOf(navArgument("runId") { type = NavType.StringType }),
328 ) {
329     RunDetailScreen(
330         onBack = { navController.popBackStack() },
331         onDelete = { deletedRunId ->
332             navController.previousBackStackEntry
333                 ?.savedStateHandle
334                 ?.set("deletedRunId", deletedRunId)
335             navController.popBackStack()
336         },
337         onShareImage = { runId ->
338             navController.navigate(RunwayRoutes.runShare(runId))
339         },
340         onNavigateToCourseDetail = { courseId ->
341             navController.navigate(RunwayRoutes.courseDetail(courseId))

```

```

342         },
343     )
344 }
345
346 // — 내 코스 목록 (BottomNav 없음) —
347
348 composable(RunwayRoutes.MY_COURSES) {
349     MyCoursesScreen(
350         onBack = { navController.popBackStack() },
351         onNavigateToCourseDetail = { courseId ->
352             navController.navigate(RunwayRoutes.courseDetail(courseId))
353         },
354     )
355 }
356
357 // — 코스 리더보드 (BottomNav 없음) —
358
359 composable(
360     route = RunwayRoutes.COURSE_LEADERBOARD,
361     arguments = listOf(
362         navArgument("courseId") { type = NavType.StringType },
363         navArgument("isPR") { type = NavType.BoolType; defaultValue = false },
364         navArgument("previousBestSeconds") { type = NavType.IntType; defaultValue = Int.MIN_VALUE },
365         navArgument("improvementSeconds") { type = NavType.IntType; defaultValue = Int.MIN_VALUE },
366         navArgument("justCompleted") { type = NavType.BoolType; defaultValue = false },
367     ),
368 ) {
369     CourseLeaderboardScreen(
370         onBack = { navController.popBackStack() },
371     )
372 }
373
374 // — 공유 이미지 생성 (BottomNav 없음) —
375
376 composable(
377     route = RunwayRoutes.RUN_SHARE,
378     arguments = listOf(navArgument("runId") { type = NavType.StringType }),
379 ) {
380     RunShareImageScreen(
381         onBack = { navController.popBackStack() },
382     )
383 }
384
385 // — 통계 —
386
387 composable(RunwayRoutes.STATS) {
388     StatsScreen(onBack = { navController.popBackStack() })
389 }
390
391 // — 업적 —
392
393 composable(RunwayRoutes.ACHIEVEMENTS) {
394     AchievementsScreen(onBack = { navController.popBackStack() })
395 }
396
397 // — 리마인더 —
398
399 composable(RunwayRoutes.REMINDER) {
400     ReminderScreen(onBack = { navController.popBackStack() })
401 }
402
403 // — 설정 —
404
405 composable(RunwayRoutes.SETTINGS) {
406     SettingsScreen(onBack = { navController.popBackStack() })
407 }
408 }
409 }

```



```

1 package com.runway.android.ui.navigation
2
3 import android.net.Uri
4
5 object RunwayRoutes {
6     const val SPLASH = "splash"
7     const val LOGIN = "login"
8     const val SIGNUP = "signup"
9     const val ONBOARDING = "onboarding"
10    const val PERMISSION = "permission"
11    const val MAIN = "main"
12    const val RUNNING = "running"
13    const val RUN_RESULT = "run_result"
14    const val MY_RUNS = "my_runs"
15    const val MY_COURSES = "my_courses"
16    const val RUN_DETAIL = "run_detail/{runId}"
17    const val COURSE_DETAIL = "course_detail/{courseId}"
18    const val COURSE_ATTEMPT = "course_attempt/{courseId}/{courseAttemptId}/{runningRecordId}"
19    const val COURSE_LEADERBOARD = "course_leaderboard/{courseId}?isPR={isPR}&previousBestSeconds={previousBestSeconds}&improvementSeco
nds={improvementSeconds}&justCompleted={justCompleted}"
20    const val COURSE_MAP_DETAIL = "course_map_detail/{courseId}"
21    const val RUN_SHARE = "run_share/{runId}"
22    const val STATS = "stats"
23    const val ACHIEVEMENTS = "achievements"
24    const val REMINDER = "reminder"
25    const val SETTINGS = "settings"
26    const val POSTURE_CAPTURE = "posture_capture"
27    const val POSTURE_RESULT = "posture_result/{analysisId}"
28
29    fun postureResult(analysisId: String) = "posture_result/${Uri.encode(analysisId)}"
30
31    fun courseDetail(courseId: String) = "course_detail/${Uri.encode(courseId)}"
32    fun runShare(runId: String) = "run_share/${Uri.encode(runId)}"
33
34    fun runResult(runId: String, elapsedSeconds: Int, distanceKm: Float) =
35        "$RUN_RESULT/${Uri.encode(runId)}/${elapsedSeconds}/${distanceKm}"
36
37    fun runDetail(runId: String) = "run_detail/${Uri.encode(runId)}"
38
39    fun courseAttempt(
40        courseId: String,
41        courseAttemptId: String,
42        runningRecordId: String,
43    ) = "course_attempt/${Uri.encode(courseId)}/${Uri.encode(courseAttemptId)}/${Uri.encode(runningRecordId)}"
44
45    fun courseLeaderboard(
46        courseId: String,
47        isPR: Boolean = false,
48        previousBestSeconds: Int? = null,
49        improvementSeconds: Int? = null,
50        justCompleted: Boolean = false,
51    ) = buildString {
52        append("course_leaderboard/${Uri.encode(courseId)}")
53        append("?isPR=$isPR")
54        append("&justCompleted=$justCompleted")
55        if (previousBestSeconds != null) append("&previousBestSeconds=$previousBestSeconds")
56        if (improvementSeconds != null) append("&improvementSeconds=$improvementSeconds")
57    }
58    fun courseMapDetail(courseId: String) = "course_map_detail/${Uri.encode(courseId)}"
59 }

```

```

1  package com.runway.android.ui.onboarding
2
3  import androidx.compose.animation.AnimatedContent
4  import androidx.compose.animation.core.animateDpAsState
5  import androidx.compose.animation.core.tween
6  import androidx.compose.animation.fadeIn
7  import androidx.compose.animation.fadeOut
8  import androidx.compose.animation.slideInVertically
9  import androidx.compose.animation.slideOutVertically
10 import androidx.compose.animation.togetherWith
11 import androidx.compose.foundation.background
12 import androidx.compose.foundation.layout.Arrangement
13 import androidx.compose.foundation.layout.Box
14 import androidx.compose.foundation.layout.Column
15 import androidx.compose.foundation.layout.Row
16 import androidx.compose.foundation.layout.Spacer
17 import androidx.compose.foundation.layout.fillMaxSize
18 import androidx.compose.foundation.layout.fillMaxWidth
19 import androidx.compose.foundation.layout.height
20 import androidx.compose.foundation.layout.navigationBarsPadding
21 import androidx.compose.foundation.layout.padding
22 import androidx.compose.foundation.layout.size
23 import androidx.compose.foundation.layout.statusBarsPadding
24 import androidx.compose.foundation.pager.HorizontalPager
25 import androidx.compose.foundation.pager.rememberPagerState
26 import androidx.compose.foundation.shape.CircleShape
27 import androidx.compose.material.icons.Icons
28 import androidx.compose.material.icons.filled.EmojiEvents
29 import androidx.compose.material.icons.filled.Map
30 import androidx.compose.material.icons.filled.Route
31 import androidx.compose.material3.Icon
32 import androidx.compose.material3.MaterialTheme
33 import androidx.compose.material3.Surface
34 import androidx.compose.material3.Text
35 import androidx.compose.material3.TextButton
36 import androidx.compose.runtime.Composable
37 import androidx.compose.runtime.LaunchedEffect
38 import androidx.compose.runtime.getValue
39 import androidx.compose.runtime.rememberCoroutineScope
40 import androidx.compose.ui.Alignment
41 import androidx.compose.ui.Modifier
42 import androidx.compose.ui.graphics.vector.ImageVector
43 import androidx.compose.ui.text.style.TextAlign
44 import androidx.compose.ui.unit.dp
45 import androidx.hilt.navigation.compose.hiltViewModel
46 import kotlinx.coroutines.launch
47
48 private data class OnboardingPage(
49     val icon: ImageVector,
50     val title: String,
51     val description: String,
52 )
53
54 private val PAGES = listOf(
55     OnboardingPage(
56         icon = Icons.Filled.Route,
57         title = "달린 경로가 코스가 됩니다",
58         description = "GPS로 기록한 러닝 경로를 다른 사람도 도전할 수 있는 코스로 등록해보세요.",
59     ),
60     OnboardingPage(
61         icon = Icons.Filled.Map,
62         title = "주변 코스를 찾아 도전할 수 있습니다",
63         description = "내 주변에 있는 코스를 탐색하고, 마음에 드는 코스에 도전해보세요.",
64     ),
65     OnboardingPage(
66         icon = Icons.Filled.EmojiEvents,
67         title = "리더보드에서 기록을 비교하세요",
68         description = "같은 코스를 달린 러너들과 완주 기록을 비교하고 순위를 확인해보세요.",
69     ),
70 )
71
72 @Composable
73 fun OnboardingScreen(
74     onComplete: () -> Unit,
75     viewModel: OnboardingViewModel = hiltViewModel(),
76 ) {
77     LaunchedEffect(Unit) {
78         viewModel.onComplete.collect { onComplete() }
79     }
80
81     val pagerState = rememberPagerState { PAGES.size }
82     val coroutineScope = rememberCoroutineScope()
83     val isLastPage = pagerState.currentPage == PAGES.size - 1

```

```

84
85     Box(
86         modifier = Modifier
87             .fillMaxSize()
88             .background(MaterialTheme.colorScheme.background),
89     ) {
90         // — Page content —————
91         HorizontalPager(
92             state = pagerState,
93             modifier = Modifier.fillMaxSize(),
94         ) { pageIndex ->
95             val page = PAGES[pageIndex]
96             Column(
97                 modifier = Modifier
98                     .fillMaxSize()
99                     .padding(horizontal = 32.dp)
100                     .padding(bottom = 160.dp),
101                 horizontalAlignment = Alignment.CenterHorizontally,
102                 verticalArrangement = Arrangement.Center,
103             ) {
104                 Box(
105                     modifier = Modifier
106                         .size(96.dp)
107                         .background(MaterialTheme.colorScheme.primaryContainer, CircleShape),
108                     contentAlignment = Alignment.Center,
109                 ) {
110                     Icon(
111                         imageVector = page.icon,
112                         contentDescription = null,
113                         tint = MaterialTheme.colorScheme.onPrimaryContainer,
114                         modifier = Modifier.size(48.dp),
115                     )
116                 }
117
118                 Spacer(modifier = Modifier.height(32.dp))
119
120                 Text(
121                     text = page.title,
122                     style = MaterialTheme.typography.headlineSmall,
123                     color = MaterialTheme.colorScheme.onBackground,
124                     textAlign = TextAlign.Center,
125                 )
126
127                 Spacer(modifier = Modifier.height(12.dp))
128
129                 Text(
130                     text = page.description,
131                     style = MaterialTheme.typography.bodyLarge,
132                     color = MaterialTheme.colorScheme.onSurfaceVariant,
133                     textAlign = TextAlign.Center,
134                 )
135             }
136         }
137
138         // — Skip button —————
139         TextButton(
140             onClick = viewModel::complete,
141             modifier = Modifier
142                 .align(Alignment.TopEnd)
143                 .statusBarsPadding()
144                 .padding(end = 8.dp),
145         ) {
146             Text(
147                 text = "건너뛰기",
148                 style = MaterialTheme.typography.bodyMedium,
149                 color = MaterialTheme.colorScheme.onSurfaceVariant,
150             )
151         }
152
153         // — Bottom controls —————
154         Column(
155             modifier = Modifier
156                 .align(Alignment.BottomCenter)
157                 .fillMaxWidth()
158                 .navigationBarsPadding()
159                 .padding(horizontal = 32.dp, vertical = 32.dp),
160             horizontalAlignment = Alignment.CenterHorizontally,
161         ) {
162             Row(
163                 horizontalArrangement = Arrangement.spacedBy(6.dp),
164                 verticalAlignment = Alignment.CenterVertically,
165             ) {
166                 PAGES.indices.forEach { index ->
167                     val dotWidth by animateDpAsState(
168                         targetValue = if (index == pagerState.currentPage) 20.dp else 6.dp,
169                         animationSpec = tween(300),

```

```

170         label = "dot_$index",
171     )
172     Box(
173         modifier = Modifier
174             .size(dotWidth, 6.dp)
175             .background(
176                 color = if (index == pagerState.currentPage)
177                     MaterialTheme.colorScheme.primary
178                 else
179                     MaterialTheme.colorScheme.outline,
180                 shape = CircleShape,
181             ),
182     )
183 }
184 }
185
186 Spacer(modifier = Modifier.height(24.dp))
187
188 Surface(
189     modifier = Modifier.fillMaxWidth(),
190     shape = MaterialTheme.shapes.extraLarge,
191     color = MaterialTheme.colorScheme.primary,
192     onClick = {
193         if (isLastPage) {
194             viewModel.complete()
195         } else {
196             coroutineScope.launch {
197                 pagerState.animateScrollToPage(pagerState.currentPage + 1)
198             }
199         }
200     },
201 ) {
202     AnimatedContent(
203         targetState = isLastPage,
204         transitionSpec = {
205             (slideInVertically { it } + fadeIn(tween(200))) togetherWith
206             (slideOutVertically { -it } + fadeOut(tween(200)))
207         },
208         label = "ctaLabel",
209     ) { last ->
210         Text(
211             text = if (last) "시작하기" else "다음",
212             style = MaterialTheme.typography.labelLarge,
213             color = MaterialTheme.colorScheme.onPrimary,
214             textAlign = TextAlign.Center,
215             modifier = Modifier
216                 .fillMaxWidth()
217                 .padding(vertical = 16.dp),
218         )
219     }
220 }
221 }
222 }
223 }

```

```
1 package com.runway.android.ui.onboarding
2
3 import androidx.compose.runtime.getValue
4 import androidx.compose.runtime.mutableIntStateOf
5 import androidx.compose.runtime.setValue
6 import androidx.lifecycle.ViewModel
7 import androidx.lifecycle.ViewModelScope
8 import com.runway.android.core.datastore.OnboardingDataStore
9 import dagger.hilt.android.lifecycle.HiltViewModel
10 import kotlinx.coroutines.flow.MutableSharedFlow
11 import kotlinx.coroutines.flow.asSharedFlow
12 import kotlinx.coroutines.launch
13 import javax.inject.Inject
14
15 @HiltViewModel
16 class OnboardingViewModel @Inject constructor(
17     private val onboardingDataStore: OnboardingDataStore,
18 ) : ViewModel() {
19
20     var currentPage by mutableIntStateOf(0)
21     private set
22
23     private val _onComplete = MutableSharedFlow<Unit>()
24     val onComplete = _onComplete.asSharedFlow()
25
26     fun nextPage(totalPages: Int) {
27         if (currentPage < totalPages - 1) {
28             currentPage++
29         } else {
30             complete()
31         }
32     }
33
34     fun complete() {
35         viewModelScope.launch {
36             onboardingDataStore.markCompleted()
37             _onComplete.emit(Unit)
38         }
39     }
40 }
```

```

1  package com.runway.android.ui.permission
2
3  import android.Manifest
4  import android.os.Build
5  import androidx.activity.compose.rememberLauncherForActivityResult
6  import androidx.activity.result.contract.ActivityResultContracts
7  import androidx.compose.foundation.background
8  import androidx.compose.foundation.layout.Arrangement
9  import androidx.compose.foundation.layout.Box
10 import androidx.compose.foundation.layout.Column
11 import androidx.compose.foundation.layout.Row
12 import androidx.compose.foundation.layout.Spacer
13 import androidx.compose.foundation.layout.fillMaxSize
14 import androidx.compose.foundation.layout.fillMaxWidth
15 import androidx.compose.foundation.layout.height
16 import androidx.compose.foundation.layout.navigationBarsPadding
17 import androidx.compose.foundation.layout.padding
18 import androidx.compose.foundation.layout.size
19 import androidx.compose.foundation.layout.statusBarsPadding
20 import androidx.compose.foundation.layout.width
21 import androidx.compose.material.icons.Icons
22 import androidx.compose.material.icons.filled.LocationOn
23 import androidx.compose.material.icons.filled.Notifications
24 import androidx.compose.material3.Icon
25 import androidx.compose.material3.MaterialTheme
26 import androidx.compose.material3.Surface
27 import androidx.compose.material3.Text
28 import androidx.compose.material3.TextButton
29 import androidx.compose.runtime.Composable
30 import androidx.compose.ui.Alignment
31 import androidx.compose.ui.Modifier
32 import androidx.compose.ui.graphics.vector.ImageVector
33 import androidx.compose.ui.text.style.TextAlign
34 import androidx.compose.ui.unit.dp
35 import com.runway.android.BuildConfig
36
37 @Composable
38 fun PermissionScreen(
39     onContinue: () -> Unit,
40 ) {
41     val permissionsToRequest = buildList {
42         add(Manifest.permission.ACCESS_FINE_LOCATION)
43         add(Manifest.permission.ACCESS_COARSE_LOCATION)
44         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
45             add(Manifest.permission.POST_NOTIFICATIONS)
46         }
47     }
48
49     val launcher = rememberLauncherForActivityResult(
50         contract = ActivityResultContracts.RequestMultiplePermissions(),
51     ) {
52         // 위치 권한 허용 여부와 무관하게 계속 진행 (거부된 경우 홈 화면에서 재요청)
53         onContinue()
54     }
55
56     Box(
57         modifier = Modifier
58             .fillMaxSize()
59             .background(MaterialTheme.colorScheme.background),
60     ) {
61         Column(
62             modifier = Modifier
63                 .fillMaxSize()
64                 .statusBarsPadding()
65                 .padding(horizontal = 32.dp),
66             horizontalAlignment = Alignment.CenterHorizontally,
67         ) {
68             Spacer(modifier = Modifier.height(48.dp))
69
70             Text(
71                 text = "앱 권한 설정",
72                 style = MaterialTheme.typography.headlineSmall,
73                 color = MaterialTheme.colorScheme.onBackground,
74                 textAlign = TextAlign.Center,
75             )
76
77             Spacer(modifier = Modifier.height(8.dp))
78
79             Text(
80                 text = "${BuildConfig.APP_NAME}가 제대로 동작하려면 아래 권한이 필요합니다.",
81                 style = MaterialTheme.typography.bodyMedium,
82                 color = MaterialTheme.colorScheme.onSurfaceVariant,
83                 textAlign = TextAlign.Center,

```

```

84     )
85
86     Spacer(modifier = Modifier.height(40.dp))
87
88     PermissionItem(
89         icon = Icons.Filled.LocationOn,
90         title = "위치 (필수)",
91         description = "GPS로 러닝 경로를 기록하고 주변 코스를 탐색하는 데 사용됩니다. 정말 위치 권한이 필요합니다.",
92     )
93
94     Spacer(modifier = Modifier.height(16.dp))
95
96     PermissionItem(
97         icon = Icons.Filled.Notifications,
98         title = "알림 (선택)",
99         description = "러닝 리마인더와 코스 업데이트 알림을 보내는 데 사용됩니다. 거부해도 앱을 이용할 수 있습니다.",
100    )
101 }
102
103 Column(
104     modifier = Modifier
105         .align(Alignment.BottomCenter)
106         .fillMaxWidth()
107         .navigationBarsPadding()
108         .padding(horizontal = 32.dp, vertical = 32.dp),
109     horizontalAlignment = Alignment.CenterHorizontally,
110 ) {
111     Surface(
112         modifier = Modifier.fillMaxWidth(),
113         shape = MaterialTheme.shapes.extraLarge,
114         color = MaterialTheme.colorScheme.primary,
115         onClick = { launcher.launch(permissionsToRequest.toTypedArray()) },
116     ) {
117         Text(
118             text = "권한 설정하기",
119             style = MaterialTheme.typography.labelLarge,
120             color = MaterialTheme.colorScheme.onPrimary,
121             textAlign = TextAlign.Center,
122             modifier = Modifier.padding(vertical = 16.dp),
123         )
124     }
125
126     Spacer(modifier = Modifier.height(8.dp))
127
128     TextButton(onClick = onContinue) {
129         Text(
130             text = "나중에 하기",
131             style = MaterialTheme.typography.bodyMedium,
132             color = MaterialTheme.colorScheme.onSurfaceVariant,
133         )
134     }
135 }
136 }
137 }
138
139 @Composable
140 private fun PermissionItem(
141     icon: ImageVector,
142     title: String,
143     description: String,
144 ) {
145     Surface(
146         modifier = Modifier.fillMaxWidth(),
147         shape = MaterialTheme.shapes.extraLarge,
148         color = MaterialTheme.colorScheme.surface,
149     ) {
150         Row(
151             modifier = Modifier.padding(20.dp),
152             verticalAlignment = Alignment.Top,
153             horizontalArrangement = Arrangement.spacedBy(16.dp),
154         ) {
155             Box(
156                 modifier = Modifier
157                     .size(44.dp)
158                     .background(
159                         MaterialTheme.colorScheme.primaryContainer,
160                         MaterialTheme.shapes.medium,
161                     ),
162                 contentAlignment = Alignment.Center,
163             ) {
164                 Icon(
165                     imageVector = icon,
166                     contentDescription = null,
167                     tint = MaterialTheme.colorScheme.onPrimaryContainer,
168                     modifier = Modifier.size(24.dp),
169                 )

```

```
170     }
171     Column(modifier = Modifier.weight(1f)) {
172         Text(
173             text = title,
174             style = MaterialTheme.typography.titleSmall,
175             color = MaterialTheme.colorScheme.onSurface,
176         )
177         Spacer(modifier = Modifier.height(4.dp))
178         Text(
179             text = description,
180             style = MaterialTheme.typography.bodySmall,
181             color = MaterialTheme.colorScheme.onSurfaceVariant,
182         )
183     }
184 }
185 }
186 }
```



```

1  package com.runway.android.ui.posture
2
3  import android.content.Context
4  import android.net.Uri
5  import androidx.lifecycle.ViewModel
6  import androidx.lifecycle.ViewModelScope
7  import com.google.gson.Gson
8  import com.google.gson.reflect.TypeToken
9  import com.runway.android.core.datastore.TokenDataStore
10 import com.runway.android.core.posture.PostureEvaluator
11 import com.runway.android.core.posture.PosturePoseAnalyzer
12 import com.runway.android.core.posture.PostureResult
13 import com.runway.android.core.posture.PostureVideoFrame
14 import com.runway.android.core.posture.local.PostureAnalysisDao
15 import com.runway.android.core.posture.local.PostureAnalysisEntity
16 import com.runway.android.core.posture.local.MAX_POSTURE_ANALYSIS_HISTORY
17 import dagger.hilt.android.lifecycle.HiltViewModel
18 import dagger.hilt.android.qualifiers.ApplicationContext
19 import kotlinx.coroutines.flow.MutableStateFlow
20 import kotlinx.coroutines.flow.SharingStarted
21 import kotlinx.coroutines.flow.StateFlow
22 import kotlinx.coroutines.flow.asStateFlow
23 import kotlinx.coroutines.flow.filterNotNull
24 import kotlinx.coroutines.flow.flatMapLatest
25 import kotlinx.coroutines.flow.stateIn
26 import kotlinx.coroutines.launch
27 import java.io.File
28 import java.util.UUID
29 import javax.inject.Inject
30
31 sealed interface PostureAnalysisState {
32     data object Idle : PostureAnalysisState
33     data object Analyzing : PostureAnalysisState
34     data class Success(val result: PostureResult, val id: String) : PostureAnalysisState
35     data class Error(val message: String) : PostureAnalysisState
36 }
37
38 @HiltViewModel
39 @OptIn(kotlinx.coroutines.ExperimentalCoroutinesApi::class)
40 class PostureAnalysisViewModel @Inject constructor(
41     @ApplicationContext private val context: Context,
42     private val poseAnalyzer: PosturePoseAnalyzer,
43     private val evaluator: PostureEvaluator,
44     private val dao: PostureAnalysisDao,
45     private val tokenDataStore: TokenDataStore,
46 ) : ViewModel() {
47
48     private val gson = Gson()
49     private val videoFramesType = object : TypeToken<List<PostureVideoFrame>>() {}.type
50
51     private val _analysisState = MutableStateFlow<PostureAnalysisState>(PostureAnalysisState.Idle)
52     val analysisState: StateFlow<PostureAnalysisState> = _analysisState.asStateFlow()
53
54     val analysisHistory = tokenDataStore.userIdFlow
55         .filterNotNull()
56         .flatMapLatest(dao::observeAll)
57         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), emptyList())
58
59     init {
60         viewModelScope.launch {
61             tokenDataStore.getUserId()?.let { ownerId ->
62                 dao.assignUnowned(ownerId)
63             }
64         }
65     }
66
67     fun analyze(videoUri: Uri) {
68         if (_analysisState.value is PostureAnalysisState.Analyzing) return
69         viewModelScope.launch {
70             val ownerId = tokenDataStore.getUserId()
71             if (ownerId == null) {
72                 deleteTemporaryCapture(videoUri)
73                 _analysisState.value = PostureAnalysisState.Error("로그인 정보를 확인할 수 없습니다.")
74                 return@launch
75             }
76             if (dao.count(ownerId) >= MAX_POSTURE_ANALYSIS_HISTORY) {
77                 deleteTemporaryCapture(videoUri)
78                 _analysisState.value = PostureAnalysisState.Error(
79                     "분석 이력은 최대 ${MAX_POSTURE_ANALYSIS_HISTORY}개까지 저장할 수 있습니다. 기존 이력을 삭제한 후 다시 시도해주세요."
80                 )
81                 return@launch
82             }
83             _analysisState.value = PostureAnalysisState.Analyzing

```

```

84     val id = UUID.randomUUID().toString()
85     var savedVideoPath: String? = null
86
87     runCatching {
88         val output = poseAnalyzer.extractAnalysis(videoUri, id)
89         // Capture early so onFailure can always clean up the copied video,
90         // even if analysis finds no poses. Using return@launch here would
91         // bypass onFailure/also and leak the file.
92         savedVideoPath = output.videoPath
93
94         check(output.frames.isNotEmpty()) {
95             "영상에서 자세를 감지하지 못했습니다. 전신이 잘 보이는 측면 러닝 영상을 선택하거나 다시 촬영해주세요."
96         }
97
98         val result = evaluator.evaluate(output.frames)
99         val framesJson = gson.toJson(output.videoFrames)
100        dao.insert(result.toEntity(ownerId, id, output.videoPath, framesJson, output.videoWidth, output.videoHeight))
101        _analysisState.value = PostureAnalysisState.Success(result, id)
102    }.onFailure { e ->
103        savedVideoPath?.let { File(it).delete() }
104        _analysisState.value = PostureAnalysisState.Error(
105            e.message ?: "분석 중 오류가 발생했습니다."
106        )
107    }.also {
108        // Always delete the original cache file (privacy).
109        // The video was already copied to internal storage by the analyzer.
110        runCatching {
111            deleteTemporaryCapture(videoUri)
112        }
113    }
114 }
115 }
116
117 fun resetState() {
118     _analysisState.value = PostureAnalysisState.Idle
119 }
120
121 suspend fun loadById(id: String): PostureAnalysisEntity? {
122     val ownerId = tokenDataStore.getUserId() ?: return null
123     return dao.findById(id, ownerId)
124 }
125
126 fun parseVideoFrames(json: String?): List<PostureVideoFrame> {
127     if (json.isNullOrEmpty()) return emptyList()
128     return runCatching { gson.fromJson<List<PostureVideoFrame>>(json, videoFramesType) }
129         .getOrElse(emptyList())
130 }
131
132 fun deleteAnalysis(id: String) {
133     viewModelScope.launch {
134         val ownerId = tokenDataStore.getUserId() ?: return@launch
135         val entity = dao.findById(id, ownerId)
136         entity?.videoPath?.let { path -> File(path).delete() }
137         dao.deleteById(id, ownerId)
138     }
139 }
140
141 private fun deleteTemporaryCapture(videoUri: Uri) {
142     if (videoUri.scheme == "file") {
143         videoUri.path?.let { path -> File(path).delete() }
144     }
145 }
146
147 private fun PostureResult.toEntity(
148     ownerId: String,
149     id: String,
150     videoPath: String?,
151     videoFramesJson: String?,
152     videoWidth: Int,
153     videoHeight: Int,
154 ) = PostureAnalysisEntity(
155     id = id,
156     ownerId = ownerId,
157     createdAt = System.currentTimeMillis(),
158     overallScore = overallScore,
159     grade = grade,
160     overallFeedback = overallFeedback,
161     kneeScore = knee.score,
162     kneeMeasuredAngle = knee.measuredValue,
163     kneeFeedback = knee.feedback,
164     kneeTip = knee.tip,
165     trunkScore = trunk.score,
166     trunkMeasuredAngle = trunk.measuredValue,
167     trunkFeedback = trunk.feedback,
168     trunkTip = trunk.tip,
169     elbowScore = elbow.score,

```

```

170     elbowMeasuredAngle = elbow.measuredValue,
171     elbowFeedback = elbow.feedback,
172     elbowTip = elbow.tip,
173     hipScore = hip.score,
174     hipMeasuredAngle = hip.measuredValue,
175     hipFeedback = hip.feedback,
176     hipTip = hip.tip,
177     overstrideScore = overstride.score,
178     overstrideRatio = overstride.measuredValue,
179     overstrideFeedback = overstride.feedback,
180     overstrideTip = overstride.tip,
181     cadenceScore = cadence.score,
182     cadenceSpm = cadence.measuredValue,
183     cadenceFeedback = cadence.feedback,
184     cadenceTip = cadence.tip,
185     verticalOscScore = verticalOscillation.score,
186     verticalOscPercent = verticalOscillation.measuredValue,
187     verticalOscFeedback = verticalOscillation.feedback,
188     verticalOscTip = verticalOscillation.tip,
189     videoPath = videoPath,
190     videoFramesJson = videoFramesJson,
191     videoWidth = videoWidth,
192     videoHeight = videoHeight,
193 )
194 }

```

android/app/src/main/java/com/runway/android/ui/posture/PostureAnalyzingScreen.kt

Kotlin | 34줄

```

1  package com.runway.android.ui.posture
2
3  import androidx.compose.foundation.layout.Arrangement
4  import androidx.compose.foundation.layout.Column
5  import androidx.compose.foundation.layout.Spacer
6  import androidx.compose.foundation.layout.fillMaxSize
7  import androidx.compose.foundation.layout.height
8  import androidx.compose.foundation.layout.size
9  import androidx.compose.material3.CircularProgressIndicator
10 import androidx.compose.material3.MaterialTheme
11 import androidx.compose.material3.Text
12 import androidx.compose.runtime.Composable
13 import androidx.compose.ui.Alignment
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.unit.dp
16
17 @Composable
18 fun PostureAnalyzingScreen(modifier: Modifier = Modifier) {
19     Column(
20         modifier = modifier.fillMaxSize(),
21         verticalArrangement = Arrangement.Center,
22         horizontalAlignment = Alignment.CenterHorizontally,
23     ) {
24         CircularProgressIndicator(modifier = Modifier.size(64.dp), strokeWidth = 4.dp)
25         Spacer(modifier.height(24.dp))
26         Text("자세를 분석하고 있어요...", style = MaterialTheme.typography.titleMedium)
27         Spacer(modifier.height(8.dp))
28         Text(
29             "분석에는 약 1분 정도 소요됩니다",
30             style = MaterialTheme.typography.bodySmall,
31             color = MaterialTheme.colorScheme.onSurfaceVariant,
32         )
33     }
34 }

```

```
1 package com.runway.android.ui.posture
2
3 import android.Manifest
4 import android.media.AudioManager
5 import android.media.ToneGenerator
6 import android.net.Uri
7 import androidx.activity.compose.rememberLauncherForActivityResult
8 import androidx.activity.result.contract.ActivityResultContracts
9 import androidx.camera.core.CameraSelector
10 import androidx.camera.video.FileOutputOptions
11 import androidx.camera.video.Quality
12 import androidx.camera.video.QualitySelector
13 import androidx.camera.video.Recorder
14 import androidx.camera.video.Recording
15 import androidx.camera.video.VideoCapture
16 import androidx.camera.video.VideoRecordEvent
17 import androidx.camera.view.PreviewView
18 import androidx.compose.animation.AnimatedContent
19 import androidx.compose.animation.core.tween
20 import androidx.compose.animation.fadeIn
21 import androidx.compose.animation.fadeOut
22 import androidx.compose.animation.scaleIn
23 import androidx.compose.animation.scaleOut
24 import androidx.compose.animation.togetherWith
25 import androidx.compose.foundation.Image
26 import androidx.compose.foundation.background
27 import androidx.compose.foundation.layout.Box
28 import androidx.compose.foundation.layout.Column
29 import androidx.compose.foundation.layout.Spacer
30 import androidx.compose.foundation.layout.fillMaxSize
31 import androidx.compose.foundation.layout.fillMaxWidth
32 import androidx.compose.foundation.layout.height
33 import androidx.compose.foundation.layout.padding
34 import androidx.compose.foundation.layout.size
35 import androidx.compose.foundation.shape.CircleShape
36 import androidx.compose.material.icons.Icons
37 import androidx.compose.material.icons.automirrored.filled.ArrowBack
38 import androidx.compose.material.icons.filled.Cameraswitch
39 import androidx.compose.material.icons.filled.FiberManualRecord
40 import androidx.compose.material.icons.filled.Stop
41 import androidx.compose.material3.Icon
42 import androidx.compose.material3.IconButton
43 import androidx.compose.material3.LinearProgressIndicator
44 import androidx.compose.material3.MaterialTheme
45 import androidx.compose.material3.Text
46 import androidx.compose.material3.TextButton
47 import androidx.compose.runtime.Composable
48 import androidx.compose.runtime.DisposableEffect
49 import androidx.compose.runtime.LaunchedEffect
50 import androidx.compose.runtime.getValue
51 import androidx.compose.runtime.mutableIntStateOf
52 import androidx.compose.runtime.mutableStateOf
53 import androidx.compose.runtime.remember
54 import androidx.compose.runtime.rememberCoroutineScope
55 import androidx.compose.runtime.setValue
56 import androidx.compose.ui.Alignment
57 import androidx.compose.ui.Modifier
58 import androidx.compose.ui.graphics.Color
59 import androidx.compose.ui.layout.ContentScale
60 import androidx.compose.ui.platform.LocalContext
61 import androidx.compose.ui.res.painterResource
62 import androidx.compose.ui.text.font.FontWeight
63 import androidx.compose.ui.text.style.TextAlign
64 import androidx.compose.ui.unit.dp
65 import androidx.compose.ui.unit.sp
66 import androidx.compose.ui.viewinterop.AndroidView
67 import androidx.core.content.ContextCompat
68 import androidx.lifecycle.compose.LocalLifecycleOwner
69 import com.runway.android.R
70 import kotlinx.coroutines.delay
71 import kotlinx.coroutines.launch
72
73 private const val MAX_RECORDING_SECONDS = 15
74 private const val MIN_RECORDING_SECONDS = 10
75
76 @Composable
77 fun PostureCaptureScreen(
78     onVideoReady: (Uri) -> Unit,
79     onBack: () -> Unit,
80 ) {
81     val context = LocalContext.current
82     val lifecycleOwner = LocalLifecycleOwner.current
83     val scope = rememberCoroutineScope()
```

```

84
85     var hasCameraPermission by remember { mutableStateOf(false) }
86     var isRecording by remember { mutableStateOf(false) }
87     var elapsedSeconds by remember { mutableIntStateOf(0) }
88     var showGuide by remember { mutableStateOf(true) }
89     var countdownValue by remember { mutableStateOf<Int?>(null) }
90     var activeRecording by remember { mutableStateOf<Recording?>(null) }
91     var lensFacing by remember { mutableIntStateOf(CameraSelector.LENS_FACING_BACK) }
92     var hasFrontCamera by remember { mutableStateOf(false) }
93     var isCameraReady by remember { mutableStateOf(false) }
94
95     val toneGen = remember { ToneGenerator(AudioManager.STREAM_NOTIFICATION, 90) }
96     DisposableEffect(Unit) { onDispose { toneGen.release() } }
97
98     val cameraPermissionLauncher = rememberLauncherForActivityResult(
99         ActivityResultContracts.RequestPermission()
100     ) { granted -> hasCameraPermission = granted }
101
102     LaunchedEffect(Unit) {
103         cameraPermissionLauncher.launch(Manifest.permission.CAMERA)
104     }
105
106     // Auto-stop timer when recording
107     LaunchedEffect(isRecording) {
108         if (isRecording) {
109             elapsedSeconds = 0
110             while (isRecording && elapsedSeconds < MAX_RECORDING_SECONDS) {
111                 delay(1000)
112                 elapsedSeconds++
113             }
114             if (isRecording) {
115                 toneGen.startTone(ToneGenerator.TONE_PROP_BEEP2, 500)
116                 activeRecording?.stop()
117             }
118         }
119     }
120
121     val previewView = remember { PreviewView(context) }
122     var videoCapture by remember { mutableStateOf<VideoCapture<Recorder>?>(null) }
123
124     if (hasCameraPermission) {
125         DisposableEffect(lifecycleOwner, lensFacing) {
126             isCameraReady = false
127             videoCapture = null
128             val cameraProviderFuture = androidx.camera.lifecycle.ProcessCameraProvider.getInstance(context)
129             cameraProviderFuture.addListener({
130                 val cameraProvider = cameraProviderFuture.get()
131                 hasFrontCamera = runCatching {
132                     cameraProvider.hasCamera(CameraSelector.DEFAULT_FRONT_CAMERA)
133                 }.getOrElse(false)
134                 val selector = CameraSelector.Builder()
135                     .requireLensFacing(lensFacing)
136                     .build()
137                 val preview = androidx.camera.core.Preview.Builder().build()
138                     .also { it.surfaceProvider = previewView.surfaceProvider }
139                 val recorder = Recorder.Builder()
140                     .setQualitySelector(QualitySelector.from(Quality.HD))
141                     .build()
142                 val capture = VideoCapture.withOutput(recorder)
143                 runCatching {
144                     cameraProvider.unbindAll()
145                     cameraProvider.bindToLifecycle(
146                         lifecycleOwner,
147                         selector,
148                         preview,
149                         capture,
150                     )
151                 }.onSuccess {
152                     videoCapture = capture
153                     isCameraReady = true
154                 }.onFailure {
155                     if (lensFacing == CameraSelector.LENS_FACING_FRONT) {
156                         lensFacing = CameraSelector.LENS_FACING_BACK
157                     }
158                 }
159             }, ContextCompat.getMainExecutor(context))
160             onDispose {
161                 runCatching {
162                     androidx.camera.lifecycle.ProcessCameraProvider.getInstance(context).get().unbindAll()
163                 }
164             }
165         }
166     }
167
168     // Starts countdown then recording
169     fun startCountdownAndRecord() {

```

```

170     val vc = videoCapture ?: return
171     scope.launch {
172         showGuide = false
173         for (i in 5 downTo 1) {
174             countdownValue = i
175             delay(1000)
176         }
177         countdownValue = null
178
179         // Start beep
180         toneGen.startTone(ToneGenerator.TONE_PROP_BEEP, 250)
181
182         val videoFile = java.io.File(
183             context.cacheDir,
184             "posture_${System.currentTimeMillis()}.mp4",
185         )
186         val outputOptions = FileOutputOptions.Builder(videoFile).build()
187         activeRecording = vc.output.prepareRecording(context, outputOptions)
188             .start(ContextCompat.getMainExecutor(context)) { event ->
189                 when (event) {
190                     is VideoRecordEvent.Start -> isRecording = true
191                     is VideoRecordEvent.Finalize -> {
192                         toneGen.startTone(ToneGenerator.TONE_PROP_BEEP2, 500)
193                         isRecording = false
194                         if (!event.hasError()) onVideoReady(Uri.fromFile(videoFile))
195                         else videoFile.delete()
196                     }
197                     else -> Unit
198                 }
199             }
200     }
201 }
202
203 Box(modifier = Modifier.fillMaxSize().background(Color.Black)) {
204     if (hasCameraPermission) {
205         AndroidView(factory = { previewView }, modifier = Modifier.fillMaxSize())
206     }
207
208     // Silhouette guide (hide while recording)
209     if (showGuide && countdownValue == null && !isRecording) {
210         PostureSilhouetteGuide(modifier = Modifier.fillMaxSize())
211     }
212
213     // Back button
214     IconButton(
215         onClick = onBack,
216         modifier = Modifier.align(Alignment.TopStart).padding(16.dp),
217     ) {
218         Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "뒤로", tint = Color.White)
219     }
220
221     if (hasCameraPermission && hasFrontCamera) {
222         val canSwitchCamera = !isRecording && countdownValue == null
223         IconButton(
224             onClick = {
225                 lensFacing = if (lensFacing == CameraSelector.LENS_FACING_BACK) {
226                     CameraSelector.LENS_FACING_FRONT
227                 } else {
228                     CameraSelector.LENS_FACING_BACK
229                 }
230             },
231             enabled = canSwitchCamera,
232             modifier = Modifier
233                 .align(Alignment.TopEnd)
234                 .padding(16.dp)
235                 .background(Color.Black.copy(alpha = 0.35f), CircleShape),
236         ) {
237             Icon(
238                 imageVector = Icons.Filled.Cameraswitch,
239                 contentDescription = if (lensFacing == CameraSelector.LENS_FACING_FRONT) {
240                     "후면 카메라로 전환"
241                 } else {
242                     "셀카 카메라로 전환"
243                 },
244                 tint = Color.White.copy(alpha = if (canSwitchCamera) 1f else 0.45f),
245             )
246         }
247     }
248
249     // Countdown overlay
250     AnimatedContent(
251         targetState = countdownValue,
252         modifier = Modifier.align(Alignment.Center),
253         transitionSpec = {
254             (scaleIn(initialScale = 2.2f, animationSpec = tween(200)) +
255                 fadeIn(animationSpec = tween(150))) togetherWith

```

```

256         (scaleOut(targetScale = 0.4f, animationSpec = tween(300)) +
257             fadeOut(animationSpec = tween(200)))
258     },
259     label = "countdown",
260 ) { value ->
261     if (value != null) {
262         Text(
263             text = value.toString(),
264             fontSize = 120.sp,
265             fontWeight = FontWeight.Bold,
266             color = Color.White,
267             textAlign = TextAlign.Center,
268         )
269     }
270 }
271
272 // Bottom controls
273 Column(
274     modifier = Modifier
275         .align(Alignment.BottomCenter)
276         .padding(bottom = 48.dp, start = 24.dp, end = 24.dp),
277     horizontalAlignment = Alignment.CenterHorizontally,
278 ) {
279     // Guide phase - show instructions + 분석 시작 button
280     if (showGuide && countdownValue == null && !isRecording) {
281         Text(
282             "카메라를 옆면 허리 높이에 두고\n전신이 화면에 들어오도록 맞추세요",
283             style = MaterialTheme.typography.bodyMedium,
284             color = Color.White,
285             textAlign = TextAlign.Center,
286         )
287         Spacer(Modifier.height(16.dp))
288         TextButton(
289             onClick = { startCountdownAndRecord() },
290             enabled = isCameraReady,
291             modifier = Modifier
292                 .background(
293                     MaterialTheme.colorScheme.primary.copy(
294                         alpha = if (isCameraReady) 0.85f else 0.35f,
295                     ),
296                     MaterialTheme.shapes.extraLarge,
297                 )
298                 .padding(horizontal = 8.dp),
299             ) {
300                 Text("분석 시작", color = Color.Black, fontWeight = FontWeight.Bold)
301             }
302     }
303
304     // Recording phase
305     if (isRecording) {
306         val progress = elapsedSeconds / MAX_RECORDING_SECONDS.toFloat()
307         val isSufficient = elapsedSeconds >= MIN_RECORDING_SECONDS
308         Text(
309             "${elapsedSeconds}초 / ${MAX_RECORDING_SECONDS}초",
310             style = MaterialTheme.typography.bodyMedium,
311             color = if (isSufficient) Color(0xFF22C55E) else Color.White,
312         )
313         Spacer(Modifier.height(8.dp))
314         LinearProgressIndicator(
315             progress = { progress },
316             modifier = Modifier.fillMaxWidth().height(4.dp),
317             color = if (isSufficient) Color(0xFF22C55E) else MaterialTheme.colorScheme.primary,
318             trackColor = Color.White.copy(alpha = 0.3f),
319         )
320         if (!isSufficient) {
321             Spacer(Modifier.height(6.dp))
322             Text(
323                 "10초 이상 녹화하면 더 정확합니다",
324                 style = MaterialTheme.typography.labelSmall,
325                 color = Color.White.copy(alpha = 0.8f),
326             )
327         }
328         Spacer(Modifier.height(16.dp))
329         RecordButton(
330             onClick = {
331                 toneGen.startTone(ToneGenerator.TONE_PROP_BEEP2, 500)
332                 activeRecording?.stop()
333             },
334             isRecording = true,
335         )
336     }
337 }
338 }
339 }
340
341 @Composable

```

```

342 private fun RecordButton(isRecording: Boolean, onClick: () -> Unit) {
343     IconButton(
344         onClick = onClick,
345         modifier = Modifier
346             .size(72.dp)
347             .background(
348                 if (isRecording) MaterialTheme.colorScheme.error else Color.White,
349                 CircleShape,
350             ),
351     ) {
352         Icon(
353             imageVector = if (isRecording) Icons.Filled.Stop else Icons.Filled.FiberManualRecord,
354             contentDescription = if (isRecording) "녹화 중지" else "녹화 시작",
355             tint = if (isRecording) Color.White else MaterialTheme.colorScheme.error,
356             modifier = Modifier.size(36.dp),
357         )
358     }
359 }
360
361 @Composable
362 private fun PostureSilhouetteGuide(modifier: Modifier = Modifier) {
363     Box(modifier = modifier.background(Color.Black.copy(alpha = 0.28f))) {
364         Image(
365             painter = painterResource(R.drawable.runner_silhouette_guide),
366             contentDescription = null,
367             contentScale = ContentScale.Fit,
368             modifier = Modifier
369                 .fillMaxSize()
370                 .padding(horizontal = 32.dp, vertical = 24.dp)
371         )
372     }
373 }

```



```

1  package com.runway.android.ui.posture
2
3  import android.net.Uri
4  import androidx.activity.compose.rememberLauncherForActivityResult
5  import androidx.activity.result.PickVisualMediaRequest
6  import androidx.activity.result.contract.ActivityResultContracts
7  import androidx.compose.foundation.BorderStroke
8  import androidx.compose.foundation.background
9  import androidx.compose.foundation.layout.Arrangement
10 import androidx.compose.foundation.layout.Box
11 import androidx.compose.foundation.layout.BoxWithConstraints
12 import androidx.compose.foundation.layout.Column
13 import androidx.compose.foundation.layout.PaddingValues
14 import androidx.compose.foundation.layout.Row
15 import androidx.compose.foundation.layout.Spacer
16 import androidx.compose.foundation.layout.fillMaxSize
17 import androidx.compose.foundation.layout.fillMaxWidth
18 import androidx.compose.foundation.layout.height
19 import androidx.compose.foundation.layout.padding
20 import androidx.compose.foundation.layout.size
21 import androidx.compose.foundation.layout.width
22 import androidx.compose.foundation.lazy.LazyColumn
23 import androidx.compose.foundation.lazy.items
24 import androidx.compose.material.icons.Icons
25 import androidx.compose.material.icons.automirrored.filled.DirectionsRun
26 import androidx.compose.material.icons.automirrored.filled.KeyboardArrowRight
27 import androidx.compose.material.icons.filled.Delete
28 import androidx.compose.material.icons.filled.Info
29 import androidx.compose.material.icons.filled.VideoLibrary
30 import androidx.compose.material.icons.filled.VideoCameraBack
31 import androidx.compose.material3.AlertDialog
32 import androidx.compose.material3.Button
33 import androidx.compose.material3.ButtonDefaults
34 import androidx.compose.material3.ExperimentalMaterial3Api
35 import androidx.compose.material3.Icon
36 import androidx.compose.material3.IconButton
37 import androidx.compose.material3.MaterialTheme
38 import androidx.compose.material3.Scaffold
39 import androidx.compose.material3.Surface
40 import androidx.compose.material3.Text
41 import androidx.compose.material3.TextButton
42 import androidx.compose.material3.TopAppBar
43 import androidx.compose.material3.TopAppBarDefaults
44 import androidx.compose.runtime.Composable
45 import androidx.compose.runtime.collectAsState
46 import androidx.compose.runtime.getValue
47 import androidx.compose.runtime.mutableStateOf
48 import androidx.compose.runtime.remember
49 import androidx.compose.runtime.setValue
50 import androidx.compose.ui.Alignment
51 import androidx.compose.ui.Modifier
52 import androidx.compose.ui.graphics.Color
53 import androidx.compose.ui.text.font.FontWeight
54 import androidx.compose.ui.unit.sp
55 import androidx.compose.ui.text.style.TextOverflow
56 import androidx.compose.ui.tooling.preview.Preview
57 import androidx.compose.ui.unit.dp
58 import androidx.hilt.navigation.compose.hiltViewModel
59 import com.runway.android.core.posture.local.PostureAnalysisEntity
60 import com.runway.android.core.posture.local.MAX_POSTURE_ANALYSIS_HISTORY
61 import com.runway.android.ui.theme.RunwayTheme
62 import java.text.SimpleDateFormat
63 import java.util.Date
64 import java.util.Locale
65
66 @OptIn(ExperimentalMaterial3Api::class)
67 @Composable
68 fun PostureHomeScreen(
69     onStartCapture: () -> Unit,
70     onSelectVideo: (Uri) -> Unit,
71     onOpenResult: (String) -> Unit,
72     viewModel: PostureAnalysisViewModel = hiltViewModel(),
73 ) {
74     val history by viewModel.analysisHistory.collectAsState()
75     var showHelp by remember { mutableStateOf(false) }
76     var showLimitDialog by remember { mutableStateOf(false) }
77     var pendingDelete by remember { mutableStateOf<PostureAnalysisEntity?>(null) }
78     val isAtLimit = history.size >= MAX_POSTURE_ANALYSIS_HISTORY
79     val videoPicker = rememberLauncherForActivityResult(
80         contract = ActivityResultContracts.PickVisualMedia(),
81     ) { uri ->
82         uri?.let(onSelectVideo)
83     }

```

```

84
85     if (showHelp) {
86         PostureHelpDialog(onDismiss = { showHelp = false })
87     }
88     if (showLimitDialog) {
89         AlertDialog(
90             onDismissRequest = { showLimitDialog = false },
91             title = { Text("분석 이력 저장 한도") },
92             text = {
93                 Text("분석 이력은 최대 ${MAX_POSTURE_ANALYSIS_HISTORY}개까지 저장됩니다. 기존 이력을 하나 삭제하면 새 영상을 분석할 수 있습니다.")
94             },
95             confirmButton = {
96                 TextButton(onClick = { showLimitDialog = false }) { Text("확인") }
97             },
98         )
99     }
100     pendingDelete?.let { entity ->
101         AlertDialog(
102             onDismissRequest = { pendingDelete = null },
103             title = { Text("분석 이력 삭제") },
104             text = { Text("이 자세 분석 피드백을 삭제할까요? 삭제한 이력은 복구할 수 없습니다.") },
105             confirmButton = {
106                 TextButton(
107                     onClick = {
108                         viewModel.deleteAnalysis(entity.id)
109                         pendingDelete = null
110                     },
111                 ) {
112                     Text("삭제", color = MaterialTheme.colorScheme.error)
113                 }
114             },
115             dismissButton = {
116                 TextButton(onClick = { pendingDelete = null }) { Text("취소") }
117             },
118         )
119     }
120
121     Scaffold(
122         topBar = {
123             TopAppBar(
124                 title = { Text("자세 분석") },
125                 colors = TopAppBarDefaults.topAppBarColors(
126                     containerColor = MaterialTheme.colorScheme.background,
127                 ),
128                 actions = {
129                     IconButton(onClick = { showHelp = true }) {
130                         Icon(Icons.Filled.Info, contentDescription = "이용 방법")
131                     }
132                 },
133             )
134         },
135         containerColor = MaterialTheme.colorScheme.background,
136     ) { padding ->
137         LazyColumn(
138             modifier = Modifier
139                 .fillMaxSize()
140                 .padding(padding)
141                 .background(MaterialTheme.colorScheme.background),
142             contentPadding = PaddingValues(start = 20.dp, top = 12.dp, end = 20.dp, bottom = 24.dp),
143             verticalArrangement = Arrangement.spacedBy(14.dp),
144         ) {
145             item {
146                 PostureHeroCard(
147                     historyCount = history.size,
148                     isAtLimit = isAtLimit,
149                     onStartCapture = {
150                         if (isAtLimit) showLimitDialog = true else onStartCapture()
151                     },
152                     onSelectVideo = {
153                         if (isAtLimit) {
154                             showLimitDialog = true
155                         } else {
156                             videoPicker.launch(
157                                 PickVisualMediaRequest(ActivityResultContracts.PickVisualMedia.VideoOnly),
158                             )
159                         }
160                     },
161                 )
162             }
163             item {
164                 PostureDisclaimerBanner()
165             }
166             item {
167                 Row(
168                     modifier = Modifier
169                         .fillMaxWidth()

```

```

170         .padding(top = 4.dp),
171         horizontalArrangement = Arrangement.SpaceBetween,
172         verticalAlignment = Alignment.CenterVertically,
173     ) {
174         Text(
175             text = "분석 이력",
176             style = MaterialTheme.typography.titleMedium,
177             color = MaterialTheme.colorScheme.onBackground,
178         )
179         Text(
180             text = "${history.size}개",
181             style = MaterialTheme.typography.labelLarge,
182             color = MaterialTheme.colorScheme.onSurfaceVariant,
183         )
184     }
185 }
186 if (history.isEmpty()) {
187     item {
188         PostureEmptyState()
189     }
190 } else {
191     items(history) { entity ->
192         PostureHistoryCard(
193             entity = entity,
194             onClick = { onOpenResult(entity.id) },
195             onDelete = { pendingDelete = entity },
196         )
197     }
198 }
199 }
200 }
201 }
202
203 @Composable
204 private fun PostureEmptyState() {
205     Surface(
206         modifier = Modifier.fillMaxWidth(),
207         shape = MaterialTheme.shapes.extraLarge,
208         color = MaterialTheme.colorScheme.surface,
209         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
210     ) {
211         Column(
212             modifier = Modifier.padding(24.dp),
213             horizontalAlignment = Alignment.CenterHorizontally,
214         ) {
215             Icon(
216                 imageVector = Icons.AutoMirrored.Filled.DirectionsRun,
217                 contentDescription = null,
218                 modifier = Modifier.size(52.dp),
219                 tint = MaterialTheme.colorScheme.primary,
220             )
221             Spacer(Modifier.height(16.dp))
222             Text(
223                 "아직 분석 기록이 없습니다",
224                 style = MaterialTheme.typography.titleMedium,
225                 color = MaterialTheme.colorScheme.onSurface,
226             )
227             Spacer(Modifier.height(8.dp))
228             Text(
229                 "측면 러닝 영상을 촬영하거나 갤러리에서 선택해 무릎, 상체, 팔 움직임을 분석할 수 있습니다.",
230                 style = MaterialTheme.typography.bodyMedium,
231                 color = MaterialTheme.colorScheme.onSurfaceVariant,
232             )
233         }
234     }
235 }
236
237 @Composable
238 private fun PostureHeroCard(
239     historyCount: Int,
240     isAtLimit: Boolean,
241     onStartCapture: () -> Unit,
242     onSelectVideo: () -> Unit,
243 ) {
244     Surface(
245         modifier = Modifier.fillMaxWidth(),
246         shape = MaterialTheme.shapes.extraLarge,
247         color = MaterialTheme.colorScheme.surface,
248         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
249     ) {
250         Column(modifier = Modifier.padding(18.dp)) {
251             Row(
252                 verticalAlignment = Alignment.CenterVertically,
253                 horizontalArrangement = Arrangement.spacedBy(14.dp),
254             ) {
255                 Box(

```

```

256         modifier = Modifier
257             .size(48.dp)
258             .background(MaterialTheme.colorScheme.primary.copy(alpha = 0.14f), MaterialTheme.shapes.large),
259         contentAlignment = Alignment.Center,
260     ) {
261         Icon(
262             imageVector = Icons.AutoMirrored.Filled.DirectionsRun,
263             contentDescription = null,
264             tint = MaterialTheme.colorScheme.primary,
265         )
266     }
267     Column(modifier = Modifier.weight(1f)) {
268         Text(
269             text = "러닝 자세 피드백",
270             style = MaterialTheme.typography.titleMedium,
271             color = MaterialTheme.colorScheme.onSurface,
272         )
273         Spacer(Modifier.height(3.dp))
274         Text(
275             text = if (isAtLimit) {
276                 "${historyCount}/${MAX_POSTURE_ANALYSIS_HISTORY}개 저장됨 · 삭제 후 새 분석 가능"
277             } else {
278                 "${historyCount}/${MAX_POSTURE_ANALYSIS_HISTORY}개 저장됨"
279             },
280             style = MaterialTheme.typography.bodySmall,
281             color = if (isAtLimit) MaterialTheme.colorScheme.error
282                 else MaterialTheme.colorScheme.onSurfaceVariant,
283         )
284     }
285 }
286 Spacer(Modifier.height(16.dp))
287 Row(
288     modifier = Modifier.fillMaxWidth(),
289     horizontalArrangement = Arrangement.spacedBy(10.dp),
290 ) {
291     Button(
292         onClick = onStartCapture,
293         modifier = Modifier.weight(1f),
294         colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.primary),
295     ) {
296         Icon(Icons.Filled.VideoCameraBack, contentDescription = null, modifier = Modifier.size(18.dp))
297         Spacer(Modifier.width(6.dp))
298         Text("촬영")
299     }
300     Button(
301         onClick = onSelectVideo,
302         modifier = Modifier.weight(1f),
303         colors = ButtonDefaults.buttonColors(
304             containerColor = MaterialTheme.colorScheme.secondaryContainer,
305             contentColor = MaterialTheme.colorScheme.onSecondaryContainer,
306         ),
307     ) {
308         Icon(Icons.Filled.VideoLibrary, contentDescription = null, modifier = Modifier.size(18.dp))
309         Spacer(Modifier.width(6.dp))
310         Text("영상 선택")
311     }
312 }
313 }
314 }
315 }
316
317 @OptIn(ExperimentalMaterial3Api::class)
318 @Composable
319 private fun PostureHistoryCard(
320     entity: PostureAnalysisEntity,
321     onClick: () -> Unit,
322     onDelete: () -> Unit,
323 ) {
324     val dateStr = SimpleDateFormat("M월 d일 HH:mm", Locale.KOREAN).format(Date(entity.createdAt))
325     val metricSummary = listOf(
326         "무릎" to entity.kneeScore,
327         "상체" to entity.trunkScore,
328         "팔" to entity.elbowScore,
329         "고관절" to entity.hipScore,
330     ).joinToString(" · ") { (label, score) -> "$label $score" }
331
332     Surface(
333         onClick = onClick,
334         modifier = Modifier.fillMaxWidth(),
335         shape = MaterialTheme.shapes.extraLarge,
336         color = MaterialTheme.colorScheme.surface,
337         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
338     ) {
339         Row(
340             modifier = Modifier
341                 .fillMaxWidth()

```

[illegible]

```

428         color = MaterialTheme.colorScheme.primary,
429     )
430     Text(step, style = MaterialTheme.typography.bodyMedium)
431 }
432 }
433 }
434 },
435 confirmButton = {
436     TextButton(onClick = onDismiss) { Text("확인") }
437 },
438 )
439 }
440
441 @Composable
442 private fun PostureDisclaimerBanner() {
443     BoxWithConstraints(modifier = Modifier.fillMaxWidth()) {
444         // 화면 너비 기준으로 글자 크기 결정 (360dp 기준 9sp, 이하면 더 작게)
445         val fontSize = (maxWidth.value * 0.025f).coerceIn(8f, 10f).sp
446         Surface(
447             modifier = Modifier.fillMaxWidth(),
448             shape = MaterialTheme.shapes.large,
449             color = Color(0x33F59E0B),
450         ) {
451             Row(
452                 modifier = Modifier.padding(horizontal = 14.dp, vertical = 10.dp),
453                 horizontalArrangement = Arrangement.spacedBy(8.dp),
454                 verticalAlignment = Alignment.CenterVertically,
455             ) {
456                 Icon(
457                     imageVector = Icons.Filled.Info,
458                     contentDescription = null,
459                     tint = Color(0xFFD97706),
460                     modifier = Modifier.size(16.dp),
461                 )
462                 Text(
463                     text = "본 분석은 러닝 자세를 이해하기 위한 참고 자료입니다.\n" +
464                         "촬영 환경, 카메라 각도, 의류, 조명 등에 따라 정확도가 달라질 수 있으며,\n" +
465                         "의료적 판단의 근거로 사용해서는 안 됩니다.",
466                     style = MaterialTheme.typography.labelSmall.copy(fontSize = fontSize),
467                     color = MaterialTheme.colorScheme.onSurfaceVariant,
468                 )
469             }
470         }
471     }
472 }
473
474 @Preview(showBackground = true)
475 @Composable
476 private fun PostureEmptyStatePreview() {
477     RunwayTheme {
478         PostureEmptyState()
479     }
480 }

```

```

1  package com.runway.android.ui.posture
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.background
5  import androidx.compose.foundation.layout.Arrangement
6  import androidx.compose.foundation.layout.Box
7  import androidx.compose.foundation.layout.Column
8  import androidx.compose.foundation.layout.Row
9  import androidx.compose.foundation.layout.Spacer
10 import androidx.compose.foundation.layout.fillMaxSize
11 import androidx.compose.foundation.layout.fillMaxWidth
12 import androidx.compose.foundation.layout.height
13 import androidx.compose.foundation.layout.padding
14 import androidx.compose.foundation.layout.size
15 import androidx.compose.foundation.layout.width
16 import androidx.compose.foundation.rememberScrollState
17 import androidx.compose.foundation.verticalScroll
18 import androidx.compose.material.icons.Icons
19 import androidx.compose.material.icons.automirrored.filled.ArrowBack
20 import androidx.compose.material.icons.filled.Delete
21 import androidx.compose.material.icons.filled.Info
22 import androidx.compose.material3.AlertDialog
23 import androidx.compose.material3.ButtonDefaults
24 import androidx.compose.material3.Card
25 import androidx.compose.material3.CardDefaults
26 import androidx.compose.material3.CircularProgressIndicator
27 import androidx.compose.material3.ExperimentalMaterial3Api
28 import androidx.compose.material3.Icon
29 import androidx.compose.material3.IconButton
30 import androidx.compose.material3.LinearProgressIndicator
31 import androidx.compose.material3.MaterialTheme
32 import androidx.compose.material3.OutlinedButton
33 import androidx.compose.material3.Scaffold
34 import androidx.compose.material3.Surface
35 import androidx.compose.material3.Text
36 import androidx.compose.material3.TextButton
37 import androidx.compose.material3.TopAppBar
38 import androidx.compose.material3.TopAppBarDefaults
39 import androidx.compose.runtime.Composable
40 import androidx.compose.runtime.LaunchedEffect
41 import androidx.compose.runtime.getValue
42 import androidx.compose.runtime.mutableStateOf
43 import androidx.compose.runtime.remember
44 import androidx.compose.runtime.setValue
45 import androidx.compose.ui.Alignment
46 import androidx.compose.ui.Modifier
47 import androidx.compose.ui.graphics.Color
48 import androidx.compose.ui.text.font.FontWeight
49 import androidx.compose.ui.text.style.TextAlign
50 import androidx.compose.ui.tooling.preview.Preview
51 import androidx.compose.ui.unit.dp
52 import androidx.hilt.navigation.compose.hiltViewModel
53 import com.runway.android.core.posture.PostureCategoryResult
54 import com.runway.android.core.posture.PostureResult
55 import com.runway.android.core.posture.local.PostureAnalysisEntity
56 import com.runway.android.ui.theme.RunwayTheme
57 import java.util.Locale
58 import kotlin.math.roundToInt
59
60 @OptIn(ExperimentalMaterial3Api::class)
61 @Composable
62 fun PostureResultScreen(
63     analysisId: String?,
64     result: PostureResult?,
65     onBack: () -> Unit,
66     onRetake: () -> Unit,
67     viewModel: PostureAnalysisViewModel = hiltViewModel(),
68 ) {
69     var entity by remember { mutableStateOf<PostureAnalysisEntity?>(null) }
70     var loaded by remember { mutableStateOf(result != null && analysisId == null) }
71     var showDeleteDialog by remember { mutableStateOf(false) }
72
73     // Always load entity from DB when we have an ID - entity carries the video path + frames
74     LaunchedEffect(analysisId) {
75         if (analysisId != null) {
76             loaded = false
77             entity = viewModel.loadById(analysisId)
78             loaded = true
79         }
80     }
81
82     val displayResult: PostureResult? = entity?.toPostureResult() ?: result
83     val canDelete = analysisId != null || entity != null

```

```

84
85     val videoFrames = remember(entity?.videoFramesJson) {
86         viewModel.parseVideoFrames(entity?.videoFramesJson)
87     }
88
89     if (showDeleteDialog) {
90         AlertDialog(
91             onDismissRequest = { showDeleteDialog = false },
92             title = { Text("분석 이력 삭제") },
93             text = { Text("이 자세 분석 피드백을 삭제할까요? 삭제한 이력은 복구할 수 없습니다.") },
94             confirmButton = {
95                 TextButton(
96                     onClick = {
97                         (analysisId ?: entity?.id)?.let(viewModel::deleteAnalysis)
98                         showDeleteDialog = false
99                         onBackPressed()
100                     },
101                 ) {
102                     Text("삭제", color = MaterialTheme.colorScheme.error)
103                 }
104             },
105             dismissButton = {
106                 TextButton(onClick = { showDeleteDialog = false }) { Text("취소") }
107             },
108         )
109     }
110
111     Scaffold(
112         topBar = {
113             TopAppBar(
114                 title = { Text("자세 분석 결과") },
115                 navigationIcon = {
116                     IconButton(onClick = onBackPressed) {
117                         Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "뒤로")
118                     }
119                 },
120                 actions = {
121                     if (canDelete) {
122                         IconButton(onClick = { showDeleteDialog = true }) {
123                             Icon(
124                                 imageVector = Icons.Filled.Delete,
125                                 contentDescription = "분석 이력 삭제",
126                                 tint = MaterialTheme.colorScheme.error,
127                             )
128                         }
129                     }
130                 },
131                 colors = TopAppBarDefaults.topAppBarColors(
132                     containerColor = MaterialTheme.colorScheme.background,
133                 ),
134             )
135         },
136         containerColor = MaterialTheme.colorScheme.background,
137     ) { padding ->
138         when {
139             !loaded -> Box(Modifier.fillMaxSize().padding(padding), contentAlignment = Alignment.Center) {
140                 CircularProgressIndicator()
141             }
142             displayResult == null -> Box(Modifier.fillMaxSize().padding(padding), contentAlignment = Alignment.Center) {
143                 Text("결과를 불러올 수 없습니다.", color = MaterialTheme.colorScheme.onSurfaceVariant)
144             }
145             else -> PostureResultContent(
146                 result = displayResult,
147                 videoPath = entity?.videoPath,
148                 videoFrames = videoFrames,
149                 videoWidth = entity?.videoWidth ?: 0,
150                 videoHeight = entity?.videoHeight ?: 0,
151                 modifier = Modifier.padding(padding),
152                 onRetake = onRetake,
153                 onBackPressed = onBackPressed,
154             )
155         }
156     }
157 }
158
159 @Composable
160 private fun PostureResultContent(
161     result: PostureResult,
162     videoPath: String?,
163     videoFrames: List<com.runway.android.core.posture.PostureVideoFrame>,
164     videoWidth: Int,
165     videoHeight: Int,
166     modifier: Modifier = Modifier,
167     onRetake: () -> Unit,
168     onBackPressed: () -> Unit,
169 ) {

```



```

170     Column(
171         modifier = modifier
172         .fillMaxSize()
173         .background(MaterialTheme.colorScheme.background)
174         .verticalScroll(rememberScrollState())
175         .padding(horizontal = 20.dp, vertical = 14.dp),
176         horizontalAlignment = Alignment.CenterHorizontally,
177     ) {
178         // Video replay section (only when video file exists)
179         if (videoPath != null && videoFrames.isNotEmpty()) {
180             PostureVideoPlayerCard(
181                 videoPath = videoPath,
182                 videoFrames = videoFrames,
183                 videoWidth = videoWidth,
184                 videoHeight = videoHeight,
185             )
186             Spacer(Modifier.height(18.dp))
187         }
188
189         // Overall score card
190         Surface(
191             modifier = Modifier.fillMaxWidth(),
192             shape = MaterialTheme.shapes.extraLarge,
193             color = MaterialTheme.colorScheme.surface,
194             border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
195         ) {
196             Column(
197                 modifier = Modifier.padding(20.dp),
198                 horizontalAlignment = Alignment.CenterHorizontally,
199             ) {
200                 Box(
201                     modifier = Modifier
202                         .size(112.dp)
203                         .background(gradeColor(result.grade).copy(alpha = 0.12f), MaterialTheme.shapes.extraLarge),
204                     contentAlignment = Alignment.Center,
205                 ) {
206                     Column(horizontalAlignment = Alignment.CenterHorizontally) {
207                         Text(
208                             result.grade,
209                             style = MaterialTheme.typography.displaySmall.copy(fontWeight = FontWeight.Bold),
210                             color = gradeColor(result.grade),
211                         )
212                         Text(
213                             "${result.overallScore}점",
214                             style = MaterialTheme.typography.bodyLarge,
215                             color = MaterialTheme.colorScheme.onSurface,
216                         )
217                     }
218                 }
219                 Spacer(Modifier.height(16.dp))
220                 Text(
221                     result.overallFeedback,
222                     style = MaterialTheme.typography.bodyMedium,
223                     textAlign = TextAlign.Center,
224                     color = MaterialTheme.colorScheme.onSurfaceVariant,
225                 )
226             }
227         }
228         Spacer(Modifier.height(18.dp))
229
230         // Scored category cards
231         val scoredCategories = listOf(
232             "무릎 굴곡" to result.knee,
233             "상체 기울기" to result.trunk,
234             "팔꿈치 각도" to result.elbow,
235             "고관절 스윙" to result.hip,
236         )
237         scoredCategories.forEach { (label, cat) ->
238             PostureCategoryCard(label = label, category = cat)
239             Spacer(Modifier.height(10.dp))
240         }
241
242         // Reference metrics (bilateral signals - not included in overall score)
243         val hasReferenceData = result.cadence.measuredValue > 0f ||
244             result.verticalOscillation.measuredValue > 0f
245         if (hasReferenceData) {
246             Spacer(Modifier.height(6.dp))
247             Text(
248                 "참고 지표",
249                 style = MaterialTheme.typography.labelMedium,
250                 color = MaterialTheme.colorScheme.onSurfaceVariant,
251                 modifier = Modifier.fillMaxWidth(),
252             )
253             Text(
254                 "양쪽 다리 움직임으로 측정 - 점수에 포함되지 않습니다",
255                 style = MaterialTheme.typography.labelSmall,

```

```

256         color = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.6f),
257         modifier = Modifier.fillMaxWidth(),
258     )
259     Spacer(Modifier.height(8.dp))
260     if (result.cadence.measuredValue > 0f) {
261         PostureCategoryCard(label = "케이던스", category = result.cadence, isReference = true)
262         Spacer(Modifier.height(10.dp))
263     }
264     if (result.verticalOscillation.measuredValue > 0f) {
265         PostureCategoryCard(label = "수직진폭", category = result.verticalOscillation, isReference = true)
266         Spacer(Modifier.height(10.dp))
267     }
268 }
269
270 Spacer(Modifier.height(4.dp))
271 OutlinedButton(
272     onClick = onRetake,
273     modifier = Modifier.fillMaxWidth(),
274     border = BorderStroke(1.dp, MaterialTheme.colorScheme.primary),
275     colors = ButtonDefaults.outlinedButtonColors(contentColor = MaterialTheme.colorScheme.primary),
276 ) {
277     Text("다시 분석하기")
278 }
279 Spacer(Modifier.height(8.dp))
280 OutlinedButton(
281     onClick = onBack,
282     modifier = Modifier.fillMaxWidth(),
283 ) {
284     Text("이력 보기")
285 }
286
287 Spacer(Modifier.height(20.dp))
288 PostureDisclaimerCard()
289 Spacer(Modifier.height(16.dp))
290 }
291 }
292
293 @Composable
294 private fun PostureCategoryCard(
295     label: String,
296     category: PostureCategoryResult,
297     isReference: Boolean = false,
298 ) {
299     val borderColor = if (isReference)
300         MaterialTheme.colorScheme.outline.copy(alpha = 0.5f)
301     else
302         MaterialTheme.colorScheme.outline
303
304     Card(
305         modifier = Modifier.fillMaxWidth(),
306         colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
307         border = BorderStroke(1.dp, borderColor),
308         elevation = CardDefaults.cardElevation(0.dp),
309         shape = MaterialTheme.shapes.extraLarge,
310     ) {
311         Column(modifier = Modifier.padding(16.dp)) {
312             Row(
313                 modifier = Modifier.fillMaxWidth(),
314                 horizontalArrangement = Arrangement.SpaceBetween,
315                 verticalAlignment = Alignment.CenterVertically,
316             ) {
317                 Text(
318                     label,
319                     style = MaterialTheme.typography.titleSmall,
320                     fontWeight = FontWeight.SemiBold,
321                     color = if (isReference) MaterialTheme.colorScheme.onSurface.copy(alpha = 0.75f)
322                     else MaterialTheme.colorScheme.onSurface,
323                 )
324                 Text(
325                     "${category.score}점",
326                     style = MaterialTheme.typography.titleSmall.copy(fontWeight = FontWeight.Bold),
327                     color = scoreColor(category.score),
328                 )
329             }
330             Spacer(Modifier.height(8.dp))
331             LinearProgressIndicator(
332                 progress = { category.score / 100f },
333                 modifier = Modifier.fillMaxWidth().height(6.dp),
334                 color = scoreColor(category.score),
335                 trackColor = MaterialTheme.colorScheme.surfaceVariant,
336             )
337             Spacer(Modifier.height(10.dp))
338
339             val displayValue = when {
340                 category.unit == "spm" ->
341                     "측정값: ${category.measuredValue.roundToInt()} spm " +

```

```

342         "이상범위: ${category.idealMin.roundToInt()}~${category.idealMax.roundToInt()} spm"
343     else ->
344         "측정값: ${String.format(Locale.US, "%.1f", category.measuredValue)}${category.unit}   " +
345         "이상범위: ${String.format(Locale.US, "%.0f", category.idealMin)}~${String.format(Locale.US, "%.0f", category.
idealMax)}${category.unit}"
346     }
347     Text(displayValue, style = MaterialTheme.typography.labelSmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
348     Spacer(Modifier.height(6.dp))
349     Text(category.feedback, style = MaterialTheme.typography.bodySmall)
350     if (category.tip.isNotEmpty()) {
351         Spacer(Modifier.height(4.dp))
352         Text(
353             "💡 ${category.tip}",
354             style = MaterialTheme.typography.labelSmall,
355             color = MaterialTheme.colorScheme.primary,
356         )
357     }
358 }
359 }
360 }
361
362 @Composable
363 private fun scoreColor(score: Int) = when {
364     score >= 80 -> Color(0xFF22C55E)
365     score >= 60 -> Color(0xFF3B82F6)
366     score >= 45 -> Color(0xFFFF59E0B)
367     else -> MaterialTheme.colorScheme.error
368 }
369
370 private fun PostureAnalysisEntity.toPostureResult(): PostureResult = PostureResult(
371     overallScore = overallScore,
372     grade = grade,
373     overallFeedback = overallFeedback,
374     knee = PostureCategoryResult(kneeScore, kneeMeasuredAngle, 135f, 180f, "", kneeFeedback, kneeTip),
375     trunk = PostureCategoryResult(trunkScore, trunkMeasuredAngle, 5f, 15f, "", trunkFeedback, trunkTip),
376     elbow = PostureCategoryResult(elbowScore, elbowMeasuredAngle, 60f, 90f, "", elbowFeedback, elbowTip),
377     hip = PostureCategoryResult(hipScore, hipMeasuredAngle, 29f, 41f, "", hipFeedback, hipTip),
378     overstride = PostureCategoryResult(overstrideScore, overstrideRatio, 0f, 15f, "", overstrideFeedback, overstrideTip),
379     cadence = PostureCategoryResult(cadenceScore, cadenceSpm, 170f, 180f, "spm", cadenceFeedback, cadenceTip),
380     verticalOscillation = PostureCategoryResult(verticalOscScore, verticalOscPercent, 0f, 6.5f, "%", verticalOscFeedback, verticalOscTip),
381 )
382
383 @Preview(showBackground = true)
384 @Composable
385 private fun PostureResultPreview() {
386     RunwayTheme {
387         PostureResultContent(
388             result = PostureResult(
389                 overallScore = 74,
390                 grade = "B",
391                 overallFeedback = "전반적으로 좋은 자세입니다. 상체 기울기 부분을 보완하면 더 좋아집니다.",
392                 knee = PostureCategoryResult(82, 148f, 135f, 165f, "", "척지 시 무릎 각도가 이상적입니다.", ""),
393                 trunk = PostureCategoryResult(55, 2.3f, 5f, 10f, "", "상체를 약 5도 앞으로 기울여 보세요.", "전방 기울기는 추진력과 효율을 높여줍니다."),
394                 elbow = PostureCategoryResult(90, 91f, 85f, 95f, "", "팔꿈치 각도가 이상적입니다.", ""),
395                 hip = PostureCategoryResult(78, 35f, 29f, 41f, "", "고관절 스윙 범위가 좋습니다.", ""),
396                 overstride = PostureCategoryResult(70, 17f, 0f, 15f, "", "착지 위치가 약간 앞쪽입니다.", "발이 엉덩이 아래에 가깝게 착지하면 제동력을 줄일 수 있습니다."),
397                 cadence = PostureCategoryResult(85, 174f, 170f, 180f, "spm", "케이던스 174spm으로 이상적입니다.", ""),
398                 verticalOscillation = PostureCategoryResult(72, 6.8f, 4f, 8f, "%", "수직진폭이 이상적입니다.", ""),
399             ),
400             videoPath = null,
401             videoFrames = emptyList(),
402             videoWidth = 0,
403             videoHeight = 0,
404             onRetake = {},
405             onBack = {},
406         )
407     }
408 }
409
410 @Composable
411 private fun PostureDisclaimerCard() {
412     val warningColor = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.55f)
413     Surface(
414         modifier = Modifier.fillMaxWidth(),
415         shape = MaterialTheme.shapes.large,
416         color = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.45f),
417         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outlineVariant.copy(alpha = 0.5f)),
418     ) {
419         Row(
420             modifier = Modifier.padding(horizontal = 14.dp, vertical = 12.dp),
421             horizontalArrangement = Arrangement.spacedBy(10.dp),
422         ) {
423             Icon(

```

```
424         imageVector = Icons.Filled.Info,
425         contentDescription = null,
426         tint = warningColor,
427         modifier = Modifier
428             .size(16.dp)
429             .padding(top = 1.dp),
430     )
431     Text(
432         text = "본 분석은 러닝 자세를 이해하기 위한 참고 자료입니다. " +
433             "촬영 환경, 카메라 각도, 의류, 조명 등에 따라 정확도가 달라질 수 있으며, " +
434             "의료적 판단의 근거로 사용해서는 안 됩니다.",
435         style = MaterialTheme.typography.labelSmall,
436         color = warningColor,
437         lineHeight = MaterialTheme.typography.labelSmall.lineHeight,
438     )
439 }
440 }
441 }
```

```
1 package com.runway.android.ui.posture
2
3 import android.net.Uri
4 import androidx.compose.foundation.Canvas
5 import androidx.compose.foundation.background
6 import androidx.compose.foundation.layout.Arrangement
7 import androidx.compose.foundation.layout.Box
8 import androidx.compose.foundation.layout.BoxWithConstraints
9 import androidx.compose.foundation.layout.Column
10 import androidx.compose.foundation.layout.Row
11 import androidx.compose.foundation.layout.fillMaxSize
12 import androidx.compose.foundation.layout.fillMaxWidth
13 import androidx.compose.foundation.layout.height
14 import androidx.compose.foundation.layout.heightIn
15 import androidx.compose.foundation.layout.navigationBarsPadding
16 import androidx.compose.foundation.layout.padding
17 import androidx.compose.foundation.layout.size
18 import androidx.compose.foundation.shape.CircleShape
19 import androidx.compose.material.icons.Icons
20 import androidx.compose.material.icons.filled.Fullscreen
21 import androidx.compose.material.icons.filled.FullscreenExit
22 import androidx.compose.material.icons.filled.Pause
23 import androidx.compose.material.icons.filled.PlayArrow
24 import androidx.compose.material.icons.filled.Replay
25 import androidx.compose.material3.Icon
26 import androidx.compose.material3.IconButton
27 import androidx.compose.material3.MaterialTheme
28 import androidx.compose.material3.Slider
29 import androidx.compose.material3.SliderDefaults
30 import androidx.compose.material3.Surface
31 import androidx.compose.material3.Text
32 import androidx.compose.runtime.Composable
33 import androidx.compose.runtime.DisposableEffect
34 import androidx.compose.runtime.LaunchedEffect
35 import androidx.compose.runtime.getValue
36 import androidx.compose.runtime.mutableFloatStateOf
37 import androidx.compose.runtime.mutableIntStateOf
38 import androidx.compose.runtime.mutableLongStateOf
39 import androidx.compose.runtime.mutableStateOf
40 import androidx.compose.runtime.remember
41 import androidx.compose.runtime.setValue
42 import androidx.compose.ui.Alignment
43 import androidx.compose.ui.Modifier
44 import androidx.compose.ui.geometry.Offset
45 import androidx.compose.ui.geometry.Size
46 import androidx.compose.ui.graphics.Color
47 import androidx.compose.ui.graphics.Path
48 import androidx.compose.ui.graphics.StrokeCap
49 import androidx.compose.ui.graphics.drawscope.DrawScope
50 import androidx.compose.ui.graphics.drawscope.Stroke
51 import androidx.compose.ui.platform.LocalContext
52 import androidx.compose.ui.text.TextStyle
53 import androidx.compose.ui.text.drawText
54 import androidx.compose.ui.text.font.FontWeight
55 import androidx.compose.ui.text.rememberTextMeasurer
56 import androidx.compose.ui.unit.dp
57 import androidx.compose.ui.unit.sp
58 import androidx.compose.ui.viewinterop.AndroidView
59 import androidx.compose.ui.window.Dialog
60 import androidx.compose.ui.window.DialogProperties
61 import androidx.media3.common.MediaItem
62 import androidx.media3.exoplayer.ExoPlayer
63 import androidx.media3.ui.PlayerView
64 import com.runway.android.core.posture.RunningFormStrikeDetector
65 import com.runway.android.core.posture.SKEL_L_ANKLE
66 import com.runway.android.core.posture.SKEL_L_ELBOW
67 import com.runway.android.core.posture.SKEL_L_HIP
68 import com.runway.android.core.posture.SKEL_L_KNEE
69 import com.runway.android.core.posture.SKEL_L_SHOULDER
70 import com.runway.android.core.posture.SKEL_L_WRIST
71 import com.runway.android.core.posture.SKEL_NOSE
72 import com.runway.android.core.posture.SKEL_R_ANKLE
73 import com.runway.android.core.posture.SKEL_R_ELBOW
74 import com.runway.android.core.posture.SKEL_R_HIP
75 import com.runway.android.core.posture.SKEL_R_KNEE
76 import com.runway.android.core.posture.SKEL_R_SHOULDER
77 import com.runway.android.core.posture.SKEL_R_WRIST
78 import com.runway.android.core.posture.PostureVideoFrame
79 import com.runway.android.core.posture.SkeletonPoint
80 import com.runway.android.core.posture.interpolateFrame
81 import android.util.Log
82 import kotlinx.coroutines.delay
83 import java.io.File
```

```

84 import kotlin.math.abs
85 import kotlin.math.acos
86 import kotlin.math.sqrt
87
88 private const val TAG_OVERLAY = "PostureOverlay"
89
90 // — GaitKeeper colors —————
91 private val C_TORSO = Color(0xFF3B82F6) // blue
92 private val C_ARM = Color(0xFF10B981) // green
93 private val C_LEFT_LEG = Color(0xFFFF59E0B) // amber
94 private val C_RIGHT_LEG = Color(0xFFFFE44444) // red
95 private val C_JOINT_BG = Color(0xFF111827) // dark fill
96 private const val SCORE_THRESH = 0.3f
97
98 @Composable
99 fun PostureVideoPlayerCard(
100     videoPath: String,
101     videoFrames: List<PostureVideoFrame>,
102     videoWidth: Int,
103     videoHeight: Int,
104     modifier: Modifier = Modifier,
105 ) {
106     val context = LocalContext.current
107     val videoFile = remember(videoPath) { File(videoPath) }
108     if (!videoFile.exists()) return
109
110     val exoPlayer = remember {
111         ExoPlayer.Builder(context).build().apply {
112             setMediaItem(MediaItem.fromUri(Uri.fromFile(videoFile)))
113             prepare()
114             playWhenReady = false
115         }
116     }
117     DisposableEffect(exoPlayer) { onDispose { exoPlayer.release() } }
118
119     val sortedFrames = remember(videoFrames) { videoFrames.sortedBy { it.t } }
120
121     // running-form-analyzer FootStrikeDetector — one per ankle
122     val strikeDetectorL = remember(videoFrames) { RunningFormStrikeDetector() }
123     val strikeDetectorR = remember(videoFrames) { RunningFormStrikeDetector() }
124
125     var currentPosition by remember { mutableLongStateOf(0L) }
126     var duration by remember { mutableStateOf(1L) }
127     var isPlaying by remember { mutableStateOf(false) }
128     var isEnded by remember { mutableStateOf(false) }
129     var playbackSpeed by remember { mutableFloatStateOf(1f) }
130     var isFullscreen by remember { mutableStateOf(false) }
131
132     var currentFrame by remember { mutableStateOf<PostureVideoFrame?>(null) }
133     var overlayAngles by remember { mutableStateOf<OverlayAngles?>(null) }
134     var strikeAgeL by remember { mutableIntStateOf(Int.MAX_VALUE) }
135     var strikeAgeR by remember { mutableIntStateOf(Int.MAX_VALUE) }
136
137     LaunchedEffect(exoPlayer) {
138         while (true) {
139             val pos = exoPlayer.currentPosition.coerceAtLeast(0L)
140             val dur = exoPlayer.duration
141             if (dur > 0) duration = dur
142             isPlaying = exoPlayer.isPlaying
143             isEnded = !exoPlayer.isPlaying && pos >= duration - 100
144             currentPosition = pos
145             delay(80)
146         }
147     }
148
149     LaunchedEffect(currentPosition) {
150         val interpolated = interpolateFrame(sortedFrames, currentPosition)
151         currentFrame = interpolated?.let {
152             // 분석 시 저장한 Pose Landmarker VIDEO 모드의 추적 좌표를 그대로 사용한다.
153             // 재생 시점에는 추가 post-processing 을 하지 않는다(이중 처리 시 0.25x 에서 큰 지연).
154             val pts = it.pts
155
156             val lAnkle = pts.getOrNull(SKEL_L_ANKLE)
157             val rAnkle = pts.getOrNull(SKEL_R_ANKLE)
158             if (lAnkle != null && strikeDetectorL.update(lAnkle, it.t)) strikeAgeL = 0
159             if (rAnkle != null && strikeDetectorR.update(rAnkle, it.t)) strikeAgeR = 0
160             strikeAgeL++; strikeAgeR++
161
162             overlayAngles = computeOverlayAngles(pts, videoWidth, videoHeight)
163
164             it
165         }
166     }
167
168     LaunchedEffect(playbackSpeed) { exoPlayer.setPlaybackSpeed(playbackSpeed) }
169

```

```

170     val textMeasurer = rememberTextMeasurer()
171
172     val resetAfterSeek = {
173         strikeDetectorL.reset()
174         strikeDetectorR.reset()
175         strikeAgeL = Int.MAX_VALUE
176         strikeAgeR = Int.MAX_VALUE
177         isEnded = false
178     }
179     val togglePlayback = {
180         when {
181             isEnded -> {
182                 exoPlayer.seekTo(0)
183                 resetAfterSeek()
184                 exoPlayer.play()
185             }
186             isPlaying -> exoPlayer.pause()
187             else -> exoPlayer.play()
188         }
189     }
190
191     if (!isFullscreen) {
192         PostureVideoPlayerSurface(
193             exoPlayer = exoPlayer,
194             currentFrame = currentFrame,
195             videoWidth = videoWidth,
196             videoHeight = videoHeight,
197             overlayAngles = overlayAngles,
198             strikeAgeL = strikeAgeL,
199             strikeAgeR = strikeAgeR,
200             currentPosition = currentPosition,
201             duration = duration,
202             isPlaying = isPlaying,
203             isEnded = isEnded,
204             playbackSpeed = playbackSpeed,
205             isFullscreen = false,
206             onSeek = { fraction ->
207                 exoPlayer.seekTo((fraction * duration).toLong())
208                 resetAfterSeek()
209             },
210             onTogglePlayback = togglePlayback,
211             onSpeedChange = { playbackSpeed = it },
212             onFullscreenChange = { isFullscreen = it },
213             textMeasurer = textMeasurer,
214             modifier = modifier.fillMaxWidth(),
215         )
216     } else {
217         Dialog(
218             onDismissRequest = { isFullscreen = false },
219             properties = DialogProperties(
220                 usePlatformDefaultWidth = false,
221                 decorFitsSystemWindows = false,
222             ),
223         ) {
224             PostureVideoPlayerSurface(
225                 exoPlayer = exoPlayer,
226                 currentFrame = currentFrame,
227                 videoWidth = videoWidth,
228                 videoHeight = videoHeight,
229                 overlayAngles = overlayAngles,
230                 strikeAgeL = strikeAgeL,
231                 strikeAgeR = strikeAgeR,
232                 currentPosition = currentPosition,
233                 duration = duration,
234                 isPlaying = isPlaying,
235                 isEnded = isEnded,
236                 playbackSpeed = playbackSpeed,
237                 isFullscreen = true,
238                 onSeek = { fraction ->
239                     exoPlayer.seekTo((fraction * duration).toLong())
240                     resetAfterSeek()
241                 },
242                 onTogglePlayback = togglePlayback,
243                 onSpeedChange = { playbackSpeed = it },
244                 onFullscreenChange = { isFullscreen = it },
245                 textMeasurer = textMeasurer,
246                 modifier = Modifier.fillMaxSize(),
247             )
248         }
249     }
250 }
251
252 @Composable
253 private fun PostureVideoPlayerSurface(
254     exoPlayer: ExoPlayer,
255     currentFrame: PostureVideoFrame?,

```

```

256     videoWidth: Int,
257     videoHeight: Int,
258     overlayAngles: OverlayAngles?,
259     strikeAgeL: Int,
260     strikeAgeR: Int,
261     currentPosition: Long,
262     duration: Long,
263     isPlaying: Boolean,
264     isEnded: Boolean,
265     playbackSpeed: Float,
266     isFullscreen: Boolean,
267     onSeek: (Float) -> Unit,
268     onTogglePlayback: () -> Unit,
269     onSpeedChange: (Float) -> Unit,
270     onFullscreenChange: (Boolean) -> Unit,
271     textMeasurer: androidx.compose.ui.text.TextMeasurer,
272     modifier: Modifier = Modifier,
273 ) {
274     Surface(
275         modifier = modifier,
276         shape = if (isFullscreen) MaterialTheme.shapes.extraSmall else MaterialTheme.shapes.extraLarge,
277         color = Color.Black,
278     ) {
279         BoxWithConstraints {
280             val fullscreenControlsHeight = maxHeight * 0.22f
281             Column(modifier = if (isFullscreen) Modifier.fillMaxSize() else Modifier) {
282                 Box(
283                     modifier = if (isFullscreen) {
284                         Modifier.fillMaxWidth().weight(1f)
285                     } else {
286                         Modifier.fillMaxWidth().height(260.dp)
287                     },
288                 ) {
289                     AndroidView(
290                         factory = { ctx ->
291                             PlayerView(ctx).apply {
292                                 player = exoPlayer
293                                 useController = false
294                             }
295                         },
296                         modifier = Modifier.matchParentSize(),
297                     )
298
299                     if (currentFrame != null && videoWidth > 0 && videoHeight > 0) {
300                         SkeletonOverlay(
301                             frame = currentFrame,
302                             videoWidth = videoWidth,
303                             videoHeight = videoHeight,
304                             overlayAngles = overlayAngles,
305                             strikeAgeL = strikeAgeL,
306                             strikeAgeR = strikeAgeR,
307                             textMeasurer = textMeasurer,
308                             modifier = Modifier.matchParentSize(),
309                         )
310                     }
311
312                     IconButton(
313                         onClick = { onFullscreenChange(!isFullscreen) },
314                         modifier = Modifier
315                             .align(Alignment.TopEnd)
316                             .padding(8.dp)
317                             .size(44.dp)
318                             .background(Color.Black.copy(alpha = 0.55f), CircleShape),
319                     ) {
320                         Icon(
321                             imageVector = if (isFullscreen) Icons.Filled.FullscreenExit else Icons.Filled.Fullscreen,
322                             contentDescription = if (isFullscreen) "전체화면 종료" else "전체화면",
323                             tint = Color.White,
324                         )
325                     }
326                 }
327
328                 Column(
329                     modifier = Modifier
330                         .fillMaxWidth()
331                         .then(
332                             if (isFullscreen) {
333                                 Modifier
334                                     .heightIn(min = 112.dp)
335                                     .height(fullscreenControlsHeight)
336                             } else {
337                                 Modifier
338                                     .height(260.dp)
339                             }
340                         )
341                     .background(Color(0xFF111111))
342                     .then(if (isFullscreen) Modifier.navigationBarsPadding() else Modifier)

```



```

342         .padding(horizontal = 16.dp, vertical = 12.dp),
343         verticalArrangement = Arrangement.Center,
344     ) {
345         Slider(
346             value = if (duration > 0) currentPosition.toFloat() / duration else 0f,
347             onChange = onSeek,
348             modifier = Modifier.fillMaxWidth().height(24.dp),
349             colors = SliderDefaults.colors(
350                 thumbColor = MaterialTheme.colorScheme.primary,
351                 activeTrackColor = MaterialTheme.colorScheme.primary,
352                 inactiveTrackColor = Color.White.copy(alpha = 0.25f),
353             ),
354         )
355
356         Row(
357             modifier = Modifier.fillMaxWidth(),
358             verticalAlignment = Alignment.CenterVertically,
359             horizontalArrangement = Arrangement.SpaceBetween,
360         ) {
361             IconButton(
362                 onClick = onTogglePlayback,
363                 modifier = Modifier
364                     .size(44.dp)
365                     .background(Color.White.copy(alpha = 0.12f), CircleShape),
366             ) {
367                 Icon(
368                     imageVector = when {
369                         isEnded -> Icons.Filled.Replay
370                         isPlaying -> Icons.Filled.Pause
371                         else -> Icons.Filled.PlayArrow
372                     },
373                     contentDescription = if (isPlaying) "일시정지" else "재생",
374                     tint = Color.White,
375                 )
376             }
377
378             Text(
379                 text = "${formatMs(currentPosition)} / ${formatMs(duration)}",
380                 style = MaterialTheme.typography.labelSmall,
381                 color = Color.White.copy(alpha = 0.7f),
382             )
383
384             Row(horizontalArrangement = Arrangement.spacedBy(6.dp)) {
385                 listOf(0.25f to "0.25x", 0.5f to "0.5x", 1f to "1x").forEach { (speed, label) ->
386                     SpeedChip(
387                         label = label,
388                         selected = playbackSpeed == speed,
389                         onClick = { onSpeedChange(speed) },
390                     )
391                 }
392             }
393         }
394     }
395 }
396
397 }
398
399 // — Overlay angles computed from smoothed skeleton (running-form-analyzer port) —
400
401 private data class OverlayAngles(
402     val trunkAngle: Float?,
403     val kneeAngleL: Float?,
404     val kneeAngleR: Float?,
405     val shankAngleL: Float?,
406     val shankAngleR: Float?,
407     val elbowAngle: Float?,
408 )
409
410 /**
411  * 오버레이 각도 계산. [videoWidth]/[videoHeight] 로 aspect ratio 를 보장해
412  * normalized x·y 좌표의 물리 비율 불일치를 제거한다.
413  */
414 private fun computeOverlayAngles(
415     pts: List<SkeletonPoint>,
416     videoWidth: Int = 1,
417     videoHeight: Int = 1,
418 ): OverlayAngles {
419     val ar = if (videoHeight > 0) videoWidth.toFloat() / videoHeight else 1f
420     fun ok(idx: Int) = idx < pts.size && pts[idx].v >= SCORE_THRESH
421
422     fun angle3(aIdx: Int, bIdx: Int, cIdx: Int): Float? {
423         if (!ok(aIdx) || !ok(bIdx) || !ok(cIdx)) return null
424         val a = pts[aIdx]; val b = pts[bIdx]; val c = pts[cIdx]
425         val ax = (a.x - b.x) * ar; val ay = a.y - b.y
426         val cx = (c.x - b.x) * ar; val cy = c.y - b.y

```

```

428     val dot = ax * cx + ay * cy
429     val mag = sqrt((ax*ax+ay*ay) * (cx*cx+cy*cy))
430     if (mag < 1e-6f) return null
431     return Math.toDegrees(acos((dot/mag).coerceIn(-1f, 1f)).toDouble()).toFloat()
432 }
433
434 fun shank(kneeIdx: Int, ankleIdx: Int): Float? {
435     if (!ok(kneeIdx) || !ok(ankleIdx)) return null
436     val k = pts[kneeIdx]; val a = pts[ankleIdx]
437     val shinX = (a.x - k.x) * ar; val shinY = a.y - k.y
438     val mag = sqrt(shinX*shinX + shinY*shinY)
439     if (mag < 1e-6f) return null
440     return Math.toDegrees(acos((shinY/mag).coerceIn(-1f, 1f)).toDouble()).toFloat()
441 }
442
443 val trunkAngle = run {
444     if (!ok(SKELE_L_SHOULDER) || !ok(SKELE_R_SHOULDER) || !ok(SKELE_L_HIP) || !ok(SKELE_R_HIP)) null
445     else {
446         val dx = ((pts[SKELE_L_SHOULDER].x + pts[SKELE_R_SHOULDER].x)/2f -
447             (pts[SKELE_L_HIP].x + pts[SKELE_R_HIP].x) /2f) * ar
448         val dy = (pts[SKELE_L_HIP].y + pts[SKELE_R_HIP].y) /2f -
449             (pts[SKELE_L_SHOULDER].y + pts[SKELE_R_SHOULDER].y)/2f
450         if (sqrt(dx*dx+dy*dy) < 1e-6f) null
451         else Math.toDegrees(Math.atan2(dx.toDouble(), dy.toDouble()).toFloat()
452     )
453     }
454 }
455
456 return OverlayAngles(
457     trunkAngle = trunkAngle,
458     kneeAngleL = angle3(SKELE_L_HIP, SKELE_L_KNEE, SKELE_L_ANKLE),
459     kneeAngleR = angle3(SKELE_R_HIP, SKELE_R_KNEE, SKELE_R_ANKLE),
460     shankAngleL = shank(SKELE_L_KNEE, SKELE_L_ANKLE),
461     shankAngleR = shank(SKELE_R_KNEE, SKELE_R_ANKLE),
462     elbowAngle = run {
463         val lv = if (pts.size > SKELE_L_ELBOW) pts[SKELE_L_ELBOW].v else 0f
464         val rv = if (pts.size > SKELE_R_ELBOW) pts[SKELE_R_ELBOW].v else 0f
465         if (lv >= rv) angle3(SKELE_L_SHOULDER, SKELE_L_ELBOW, SKELE_L_WRIST)
466         else angle3(SKELE_R_SHOULDER, SKELE_R_ELBOW, SKELE_R_WRIST)
467     },
468 )
469
470 // — Skeleton overlay (GaitKeeper visual + running-form-analyzer angles) —————
471
472 @Composable
473 private fun SkeletonOverlay(
474     frame: PostureVideoFrame,
475     videoWidth: Int,
476     videoHeight: Int,
477     overlayAngles: OverlayAngles?,
478     strikeAgeL: Int,
479     strikeAgeR: Int,
480     textMeasurer: androidx.compose.ui.text.TextMeasurer,
481     modifier: Modifier = Modifier,
482 ) {
483     Canvas(modifier = modifier) {
484         val vAspect = videoWidth.toFloat() / videoHeight
485         val cAspect = size.width / size.height
486         val scale: Float; val dx: Float; val dy: Float
487         if (vAspect > cAspect) {
488             scale = size.width / videoWidth; dx = 0f
489             dy = (size.height - videoHeight * scale) / 2f
490             Log.d(TAG_OVERLAY, "overlay(letterbox-h): canvas=${size.width.toInt()}*${size.height.toInt()} " +
491                 "video=${videoWidth}*${videoHeight} scale=${"%0.4f".format(scale)} " +
492                 "displayedRect=[0,${dy.toInt()},${size.width.toInt()},${(size.height-dy).toInt()}]")
493         } else {
494             scale = size.height / videoHeight; dy = 0f
495             dx = (size.width - videoWidth * scale) / 2f
496             Log.d(TAG_OVERLAY, "overlay(letterbox-v): canvas=${size.width.toInt()}*${size.height.toInt()} " +
497                 "video=${videoWidth}*${videoHeight} scale=${"%0.4f".format(scale)} " +
498                 "displayedRect=[${dx.toInt()},0,${(size.width-dx).toInt()},${size.height.toInt()}]")
499         }
500
501         val pts = frame.pts
502         if (pts.isEmpty()) return@Canvas
503
504         fun toOff(idx: Int): Offset {
505             val p = pts[idx]
506             return Offset(p.x * videoWidth * scale + dx, p.y * videoHeight * scale + dy)
507         }
508         fun vis(idx: Int) = idx < pts.size && pts[idx].v >= SCORE_THRESH
509
510         // — Skeleton edges (GaitKeeper colors) —————
511
512         fun edge(a: Int, b: Int, color: Color, thick: Float = 3f) {
513             if (!vis(a) || !vis(b)) return

```

```

514         val oA = toOff(a); val oB = toOff(b)
515         drawLine(Color.Black, oA, oB, thick + 3f, cap = StrokeCap.Round)
516         drawLine(color, oA, oB, thick, cap = StrokeCap.Round)
517     }
518
519     fun joint(idx: Int, color: Color, r: Float = 6f) {
520         if (!vis(idx)) return
521         drawCircle(C_JOINT_BG, r, toOff(idx))
522         drawCircle(Color.White, r, toOff(idx), style = Stroke(1.5f))
523         drawCircle(color, r-2f, toOff(idx))
524     }
525
526     // Torso (blue)
527     edge(SKEL_L_SHOULDER, SKEL_R_SHOULDER, C_TORSO)
528     edge(SKEL_L_SHOULDER, SKEL_L_HIP, C_TORSO)
529     edge(SKEL_R_SHOULDER, SKEL_R_HIP, C_TORSO)
530     edge(SKEL_L_HIP, SKEL_R_HIP, C_TORSO)
531
532     // Arms (green)
533     edge(SKEL_L_SHOULDER, SKEL_L_ELBOW, C_ARM)
534     edge(SKEL_L_ELBOW, SKEL_L_WRIST, C_ARM)
535     edge(SKEL_R_SHOULDER, SKEL_R_ELBOW, C_ARM)
536     edge(SKEL_R_ELBOW, SKEL_R_WRIST, C_ARM)
537
538     // Left leg (amber)
539     edge(SKEL_L_HIP, SKEL_L_KNEE, C_LEFT_LEG, 4f)
540     edge(SKEL_L_KNEE, SKEL_L_ANKLE, C_LEFT_LEG, 4f)
541
542     // Right leg (red)
543     edge(SKEL_R_HIP, SKEL_R_KNEE, C_RIGHT_LEG, 4f)
544     edge(SKEL_R_KNEE, SKEL_R_ANKLE, C_RIGHT_LEG, 4f)
545
546     // Joints
547     listOf(SKEL_NOSE, SKEL_L_SHOULDER, SKEL_R_SHOULDER).forEach { joint(it, C_TORSO) }
548     listOf(SKEL_L_ELBOW, SKEL_L_WRIST, SKEL_R_ELBOW, SKEL_R_WRIST).forEach { joint(it, C_ARM) }
549     listOf(SKEL_L_HIP, SKEL_L_KNEE, SKEL_L_ANKLE).forEach { joint(it, C_LEFT_LEG) }
550     listOf(SKEL_R_HIP, SKEL_R_KNEE, SKEL_R_ANKLE).forEach { joint(it, C_RIGHT_LEG) }
551
552     // — GaitKeeper-style strike arrows (running-form-analyzer triggers) —
553     val maxAge = 8
554     if (vis(SKEL_L_ANKLE) && strikeAgeL < maxAge) {
555         drawStrikeArrow(toOff(SKEL_L_ANKLE), C_LEFT_LEG, strikeAgeL.toFloat() / maxAge)
556     }
557     if (vis(SKEL_R_ANKLE) && strikeAgeR < maxAge) {
558         drawStrikeArrow(toOff(SKEL_R_ANKLE), C_RIGHT_LEG, strikeAgeR.toFloat() / maxAge)
559     }
560
561     // — running-form-analyzer angle overlay panel —————
562     val angles = overlayAngles ?: return@Canvas
563     drawAnglePanel(angles, textMeasurer)
564 }
565 }
566
567 private fun DrawScope.drawStrikeArrow(ankle: Offset, color: Color, ageFraction: Float) {
568     val alpha = (1f - ageFraction).coerceIn(0f, 1f)
569     val arrowH = 36f; val arrowW = 18f
570     val tipY = ankle.y - 8f
571     // Shaft
572     drawLine(
573         color.copy(alpha = alpha),
574         start = Offset(ankle.x, tipY - arrowH),
575         end = Offset(ankle.x, tipY - 10f),
576         strokeWidth = 4f, cap = StrokeCap.Round,
577     )
578     // Downward triangle
579     val path = Path().apply {
580         moveTo(ankle.x, tipY)
581        .lineTo(ankle.x - arrowW / 2f, tipY - 14f)
582        .lineTo(ankle.x + arrowW / 2f, tipY - 14f)
583         close()
584     }
585     drawPath(path, color.copy(alpha = alpha))
586 }
587
588 private fun DrawScope.drawAnglePanel(
589     angles: OverlayAngles,
590     textMeasurer: androidx.compose.ui.text.TextMeasurer,
591 ) {
592     val lines = buildList {
593         angles.trunkAngle?.let { add("Trunk ${it.fmt()}°") }
594         angles.kneeAngleL?.let { add("Knee L ${it.fmt()}°") }
595         angles.kneeAngleR?.let { add("Knee R ${it.fmt()}°") }
596         angles.shankAngleL?.let { add("Shank L ${it.fmt()}°") }
597         angles.shankAngleR?.let { add("Shank R ${it.fmt()}°") }
598         angles.elbowAngle?.let { add("Elbow ${it.fmt()}°") }
599     }

```

```

600     if (lines.isEmpty()) return
601
602     val style = TextStyle(fontSize = 11.sp, color = Color.White, fontWeight = FontWeight.Medium)
603
604     // Measure actual text dimensions so layout never overlaps
605     val measured = lines.map { textMeasurer.measure(it, style) }
606     val textH = measured.first().size.height.toFloat()
607     val lineH = textH * 1.25f
608     val padX = 10f; val padY = 8f
609     val panelW = (measured.maxOf { it.size.width.toFloat() }) + padX * 2
610     val panelH = lines.size * lineH + padY * 2
611
612     drawRect(
613         Color.Black.copy(alpha = 0.6f),
614         topLeft = Offset(6f, 6f),
615         size = Size(panelW, panelH),
616     )
617
618     measured.forEachIndexed { i, m ->
619         drawText(m, topLeft = Offset(6f + padX, 6f + padY + i * lineH))
620     }
621 }
622
623 private fun Float.fmt() = "%.0f".format(this)
624
625 // — Speed chip —————
626
627 @Composable
628 private fun SpeedChip(label: String, selected: Boolean, onClick: () -> Unit) {
629     Surface(
630         onClick = onClick,
631         shape = MaterialTheme.shapes.small,
632         color = if (selected) MaterialTheme.colorScheme.primary else Color.White.copy(alpha = 0.12f),
633     ) {
634         Text(
635             text = label,
636             modifier = Modifier.padding(horizontal = 8.dp, vertical = 4.dp),
637             style = MaterialTheme.typography.labelSmall.copy(
638                 fontWeight = if (selected) FontWeight.Bold else FontWeight.Normal,
639             ),
640             color = if (selected) Color.Black else Color.White,
641             fontSize = 11.sp,
642         )
643     }
644 }
645
646 private fun formatMs(ms: Long): String {
647     val s = (ms / 1000).coerceAtLeast(0)
648     return "%d:%02d".format(s / 60, s % 60)
649 }

```

```

1  package com.runway.android.ui.profile
2
3  import androidx.activity.compose.rememberLauncherForActivityResult
4  import androidx.activity.result.PickVisualMediaRequest
5  import androidx.activity.result.contract.ActivityResultContracts
6  import androidx.compose.foundation.BorderStroke
7  import androidx.compose.foundation.background
8  import androidx.compose.foundation.clickable
9  import androidx.compose.foundation.layout.Arrangement
10 import androidx.compose.foundation.layout.Box
11 import androidx.compose.foundation.layout.Column
12 import androidx.compose.foundation.layout.Row
13 import androidx.compose.foundation.layout.Spacer
14 import androidx.compose.foundation.layout.fillMaxSize
15 import androidx.compose.foundation.layout.fillMaxWidth
16 import androidx.compose.foundation.layout.height
17 import androidx.compose.foundation.layout.padding
18 import androidx.compose.foundation.layout.size
19 import androidx.compose.foundation.rememberScrollState
20 import androidx.compose.foundation.shape.CircleShape
21 import androidx.compose.foundation.verticalScroll
22 import androidx.compose.material.icons.Icons
23 import androidx.compose.material.icons.automirrored.filled.DirectionsRun
24 import androidx.compose.material.icons.automirrored.filled.KeyboardArrowRight
25 import androidx.compose.animation.core.RepeatMode
26 import androidx.compose.animation.core.animateFloat
27 import androidx.compose.animation.core.animateFloatAsState
28 import androidx.compose.animation.core.animateIntAsState
29 import androidx.compose.animation.core.infiniteRepeatable
30 import androidx.compose.animation.core.rememberInfiniteTransition
31 import androidx.compose.animation.core.tween
32 import androidx.compose.material.icons.filled.BarChart
33 import androidx.compose.material.icons.filled.CameraAlt
34 import androidx.compose.material.icons.filled.EmojiEvents
35 import androidx.compose.material.icons.filled.NotificationsActive
36 import androidx.compose.material.icons.filled.Route
37 import androidx.compose.material.icons.filled.Settings
38 import androidx.compose.material.icons.filled.Whatsshot
39 import androidx.compose.ui.graphics.vector.ImageVector
40 import androidx.compose.material3.Button
41 import androidx.compose.material3.CircularProgressIndicator
42 import androidx.compose.material3.ExperimentalMaterial3Api
43 import androidx.compose.material3.pulltorefresh.PullToRefreshBox
44 import androidx.compose.material3.Icon
45 import androidx.compose.material3.MaterialTheme
46 import androidx.compose.material3.OutlinedTextField
47 import androidx.compose.material3.Surface
48 import androidx.compose.material3.Text
49 import androidx.compose.material3.TextButton
50 import androidx.compose.runtime.Composable
51 import androidx.compose.runtime.LaunchedEffect
52 import androidx.compose.runtime.getValue
53 import androidx.compose.runtime.mutableStateOf
54 import androidx.compose.runtime.remember
55 import androidx.compose.runtime.setValue
56 import androidx.compose.ui.Alignment
57 import androidx.compose.ui.draw.scale
58 import androidx.compose.ui.Modifier
59 import androidx.compose.ui.draw.clip
60 import androidx.compose.ui.layout.ContentScale
61 import androidx.compose.ui.text.style.TextAlign
62 import androidx.compose.ui.unit.dp
63 import androidx.hilt.navigation.compose.hiltViewModel
64 import coil.compose.AsyncImage
65 import com.runway.android.ui.components.PersonalRecordsSection
66
67 @OptIn(ExperimentalMaterial3Api::class)
68 @Composable
69 fun ProfileScreen(
70     onLogout: () -> Unit,
71     onNavigateToMyRuns: () -> Unit = {},
72     onNavigateToCourses: () -> Unit = {},
73     onNavigateToRunDetail: (String) -> Unit = {},
74     onNavigateToStats: () -> Unit = {},
75     onNavigateToAchievements: () -> Unit = {},
76     onNavigateToReminder: () -> Unit = {},
77     onNavigateToSettings: () -> Unit = {},
78     viewModel: ProfileViewModel = hiltViewModel(),
79 ) {
80     val imagePicker = rememberLauncherForActivityResult(
81         contract = ActivityResultContracts.PickVisualMedia(),
82     ) { uri -> viewModel.onImageSelected(uri) }
83

```

```

84 PullToRefreshBox(
85     isRefreshing = viewModel.isRefreshing,
86     onRefresh = viewModel::refresh,
87     modifier = Modifier.fillMaxSize(),
88 ) {
89     Column(
90         modifier = Modifier
91             .fillMaxSize()
92             .background(MaterialTheme.colorScheme.background)
93             .verticalScroll(rememberScrollState())
94             .padding(horizontal = 20.dp),
95         horizontalAlignment = Alignment.CenterHorizontally,
96     ) {
97         Spacer(modifier = Modifier.height(32.dp))
98
99         Text(
100             text = "프로필",
101             style = MaterialTheme.typography.headlineLarge,
102             color = MaterialTheme.colorScheme.onBackground,
103             modifier = Modifier.align(Alignment.Start),
104         )
105
106         Spacer(modifier = Modifier.height(32.dp))
107
108         if (viewModel.isLoading) {
109             Box(
110                 modifier = Modifier.fillMaxWidth(),
111                 contentAlignment = Alignment.Center,
112             ) {
113                 CircularProgressIndicator(
114                     modifier = Modifier.size(40.dp),
115                     color = MaterialTheme.colorScheme.primary,
116                 )
117             }
118         } else if (viewModel.profileError) {
119             Text(
120                 text = "프로필을 불러오지 못했습니다.",
121                 style = MaterialTheme.typography.bodyMedium,
122                 color = MaterialTheme.colorScheme.onSurfaceVariant,
123             )
124         } else {
125             // — 아바타 —
126             Box(
127                 modifier = Modifier.size(80.dp),
128                 contentAlignment = Alignment.BottomEnd,
129             ) {
130                 val avatarModifier = Modifier
131                     .size(80.dp)
132                     .clip(CircleShape)
133                     .then(
134                         if (viewModel.isEditing)
135                             Modifier.clickable {
136                                 imagePicker.launch(
137                                     PickVisualMediaRequest(ActivityResultContracts.PickVisualMedia.ImageOnly)
138                                 )
139                             }
140                         else Modifier
141                     )
142
143                 val localUri = viewModel.selectedImageUri
144                 val remoteUrl = viewModel.profileImageUrl
145
146                 if (localUri != null) {
147                     AsyncImage(
148                         model = localUri,
149                         contentDescription = "프로필 이미지",
150                         contentScale = ContentScale.Crop,
151                         modifier = avatarModifier,
152                     )
153                 } else if (remoteUrl != null) {
154                     AsyncImage(
155                         model = remoteUrl,
156                         contentDescription = "프로필 이미지",
157                         contentScale = ContentScale.Crop,
158                         modifier = avatarModifier,
159                     )
160                 } else {
161                     Box(
162                         modifier = avatarModifier.background(
163                             MaterialTheme.colorScheme.primary,
164                             CircleShape,
165                         ),
166                         contentAlignment = Alignment.Center,
167                     ) {
168                         Text(
169                             text = viewModel.nickname.firstOrNull()?.uppercaseChar()?.toString() ?: "R",

```

```

170         style = MaterialTheme.typography.headlineLarge,
171         color = MaterialTheme.colorScheme.onPrimary,
172     )
173     }
174 }
175
176 if (viewModel.isEditing) {
177     Box(
178         modifier = Modifier
179             .size(24.dp)
180             .background(MaterialTheme.colorScheme.primary, CircleShape),
181         contentAlignment = Alignment.Center,
182     ) {
183         Icon(
184             imageVector = Icons.Filled.CameraAlt,
185             contentDescription = "사진 변경",
186             tint = MaterialTheme.colorScheme.onPrimary,
187             modifier = Modifier.size(14.dp),
188         )
189     }
190 }
191 }
192
193 Spacer(modifier = Modifier.height(12.dp))
194
195 Text(
196     text = viewModel.nickname,
197     style = MaterialTheme.typography.headlineMedium,
198     color = MaterialTheme.colorScheme.onBackground,
199 )
200
201 Text(
202     text = viewModel.email,
203     style = MaterialTheme.typography.bodyMedium,
204     color = MaterialTheme.colorScheme.onSurfaceVariant,
205 )
206
207 Spacer(modifier = Modifier.height(12.dp))
208
209 if (viewModel.isEditing) {
210     // — 편집 모드 —
211     OutlinedTextField(
212         value = viewModel.editNickname,
213         onValueChange = viewModel::updateEditNickname,
214         label = { Text("닉네임") },
215         singleLine = true,
216         modifier = Modifier.fillMaxWidth(),
217     )
218     Spacer(modifier = Modifier.height(8.dp))
219     OutlinedTextField(
220         value = viewModel.editBio,
221         onValueChange = viewModel::updateEditBio,
222         label = { Text("소개") },
223         minLines = 2,
224         maxLines = 4,
225         modifier = Modifier.fillMaxWidth(),
226     )
227     if (viewModel.saveError != null) {
228         Spacer(modifier = Modifier.height(4.dp))
229         Text(
230             text = viewModel.saveError!!,
231             style = MaterialTheme.typography.bodySmall,
232             color = MaterialTheme.colorScheme.error,
233         )
234     }
235     Spacer(modifier = Modifier.height(12.dp))
236     Row(
237         modifier = Modifier.fillMaxWidth(),
238         horizontalArrangement = Arrangement.spacedBy(8.dp),
239     ) {
240         TextButton(
241             onClick = viewModel::cancelEditing,
242             modifier = Modifier.weight(1f),
243         ) {
244             Text("취소")
245         }
246         Button(
247             onClick = viewModel::saveProfile,
248             enabled = !viewModel.isSaving,
249             modifier = Modifier.weight(1f),
250         ) {
251             if (viewModel.isSaving) {
252                 CircularProgressIndicator(
253                     modifier = Modifier.size(16.dp),
254                     strokeWidth = 2.dp,
255                     color = MaterialTheme.colorScheme.onPrimary,

```

```

256         )
257     } else {
258         Text("저장")
259     }
260 }
261 }
262 } else {
263     // — 표시 모드 —
264     if (viewModel.bio.isNotEmpty()) {
265         Text(
266             text = viewModel.bio,
267             style = MaterialTheme.typography.bodyMedium,
268             color = MaterialTheme.colorScheme.onSurface,
269             textAlign = TextAlign.Center,
270         )
271         Spacer(modifier = Modifier.height(8.dp))
272     }
273     TextButton(onClick = viewModel::startEditing) {
274         Text("편집", style = MaterialTheme.typography.labelMedium)
275     }
276 }
277
278 Spacer(modifier = Modifier.height(20.dp))
279
280 viewModel.stats?.let { s ->
281     var statsTriggered by remember { mutableStateOf(false) }
282     LaunchedEffect(Unit) { statsTriggered = true }
283
284     val animatedRuns by animateIntAsState(
285         targetValue = if (statsTriggered) s.totalRuns.toInt() else 0,
286         animationSpec = tween(1200), label = "runs",
287     )
288     val distanceTarget = s.totalDistanceKm.toFloatOrNull() ?: 0f
289     val animatedDistance by animateFloatAsState(
290         targetValue = if (statsTriggered) distanceTarget else 0f,
291         animationSpec = tween(1200), label = "dist",
292     )
293     val animatedStreak by animateIntAsState(
294         targetValue = if (statsTriggered) s.currentStreakDays else 0,
295         animationSpec = tween(1200), label = "streak",
296     )
297     val flameTransition = rememberInfiniteTransition(label = "flame")
298     val flameScale by flameTransition.animateFloat(
299         initialValue = 1f, targetValue = 1.15f,
300         animationSpec = infiniteRepeatable(tween(800), RepeatMode.Reverse),
301         label = "flameScale",
302     )
303
304     Surface(
305         modifier = Modifier.fillMaxWidth(),
306         shape = MaterialTheme.shapes.extraLarge,
307         color = MaterialTheme.colorScheme.surface,
308         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
309     ) {
310         Column(modifier = Modifier.padding(20.dp)) {
311             Text(
312                 text = "STATS",
313                 style = MaterialTheme.typography.labelMedium,
314                 color = MaterialTheme.colorScheme.onSurfaceVariant,
315             )
316             Spacer(modifier = Modifier.height(12.dp))
317             Row(
318                 modifier = Modifier.fillMaxWidth(),
319                 horizontalArrangement = Arrangement.SpaceBetween,
320             ) {
321                 Column {
322                     Text(
323                         text = "총 러닝",
324                         style = MaterialTheme.typography.labelSmall,
325                         color = MaterialTheme.colorScheme.onSurfaceVariant,
326                     )
327                     Text(
328                         text = "${animatedRuns}회",
329                         style = MaterialTheme.typography.bodyLarge,
330                         color = MaterialTheme.colorScheme.onSurface,
331                     )
332                 }
333                 Column {
334                     Text(
335                         text = "총 거리",
336                         style = MaterialTheme.typography.labelSmall,
337                         color = MaterialTheme.colorScheme.onSurfaceVariant,
338                     )
339                     Text(
340                         text = "${"%2f".format(animatedDistance)} km",
341                         style = MaterialTheme.typography.bodyLarge,

```



```

342         color = MaterialTheme.colorScheme.onSurface,
343     )
344 }
345 if (s.currentStreakDays > 0) {
346     Column {
347         Text(
348             text = "현재 streak",
349             style = MaterialTheme.typography.labelSmall,
350             color = MaterialTheme.colorScheme.onSurfaceVariant,
351         )
352         Row(
353             verticalAlignment = Alignment.CenterVertically,
354             horizontalArrangement = Arrangement.spacedBy(4.dp),
355         ) {
356             Icon(
357                 imageVector = Icons.Filled.Whatshot,
358                 contentDescription = null,
359                 tint = MaterialTheme.colorScheme.primary,
360                 modifier = Modifier.size(18.dp).scale(flameScale),
361             )
362             Text(
363                 text = "${animatedStreak}일 연속",
364                 style = MaterialTheme.typography.bodyLarge,
365                 color = MaterialTheme.colorScheme.primary,
366             )
367         }
368     }
369 }
370 }
371 }
372 }
373
374 Spacer(modifier = Modifier.height(20.dp))
375
376 PersonalRecordsSection(
377     records = viewModel.personalRecords,
378     onRecordClick = onNavigateToRunDetail,
379     modifier = Modifier.fillMaxWidth(),
380 )
381 }
382 }
383
384 Spacer(modifier = Modifier.height(24.dp))
385
386 // — Stats 메뉴 —
387 ProfileMenuItem(
388     icon = Icons.Filled.BarChart,
389     label = "통계",
390     onClick = onNavigateToStats,
391 )
392
393 Spacer(modifier = Modifier.height(8.dp))
394
395 // — Achievements 메뉴 —
396 ProfileMenuItem(
397     icon = Icons.Filled.EmojiEvents,
398     label = "업적",
399     onClick = onNavigateToAchievements,
400 )
401
402 Spacer(modifier = Modifier.height(8.dp))
403
404 // — My Runs 메뉴 —
405 ProfileMenuItem(
406     icon = Icons.AutoMirrored.Filled.DirectionsRun,
407     label = "내 러닝",
408     onClick = onNavigateToMyRuns,
409 )
410
411 Spacer(modifier = Modifier.height(8.dp))
412
413 // — My Courses 메뉴 —
414 ProfileMenuItem(
415     icon = Icons.Filled.Route,
416     label = "내 코스",
417     onClick = onNavigateToCourses,
418 )
419
420 Spacer(modifier = Modifier.height(8.dp))
421
422 // — Reminder 메뉴 —
423 ProfileMenuItem(
424     icon = Icons.Filled.NotificationsActive,
425     label = "러닝 리마인더",
426     onClick = onNavigateToReminder,
427 )

```

```

428
429     Spacer(modifier = Modifier.height(8.dp))
430
431     // — Settings 메뉴 —
432     ProfileMenuItem(
433         icon = Icons.Filled.Settings,
434         label = "설정",
435         onClick = onNavigateToSettings,
436     )
437
438     Spacer(modifier = Modifier.height(12.dp))
439
440     // — 로그아웃 —
441     Surface(
442         modifier = Modifier.fillMaxWidth(),
443         shape = MaterialTheme.shapes.extraLarge,
444         color = MaterialTheme.colorScheme.error.copy(alpha = 0.1f),
445         border = BorderStroke(1.dp, MaterialTheme.colorScheme.error.copy(alpha = 0.4f)),
446         onClick = { viewModel.logout(onLogout) },
447     ) {
448         Text(
449             text = "로그아웃",
450             style = MaterialTheme.typography.bodyLarge,
451             color = MaterialTheme.colorScheme.error,
452             modifier = Modifier
453                 .fillMaxWidth()
454                 .padding(vertical = 14.dp),
455             textAlign = TextAlign.Center,
456         )
457     }
458
459     Spacer(modifier = Modifier.height(24.dp))
460 }
461 } // PullToRefreshBox
462 }
463
464 @Composable
465 private fun ProfileMenuItem(
466     icon: ImageVector,
467     label: String,
468     onClick: () -> Unit,
469 ) {
470     Surface(
471         modifier = Modifier.fillMaxWidth(),
472         shape = MaterialTheme.shapes.extraLarge,
473         color = MaterialTheme.colorScheme.surface,
474         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
475         onClick = onClick,
476     ) {
477         Row(
478             modifier = Modifier
479                 .fillMaxWidth()
480                 .padding(horizontal = 20.dp, vertical = 14.dp),
481             horizontalArrangement = Arrangement.SpaceBetween,
482             verticalAlignment = Alignment.CenterVertically,
483         ) {
484             Row(
485                 verticalAlignment = Alignment.CenterVertically,
486                 horizontalArrangement = Arrangement.spacedBy(12.dp),
487             ) {
488                 Icon(
489                     imageVector = icon,
490                     contentDescription = null,
491                     tint = MaterialTheme.colorScheme.primary,
492                     modifier = Modifier.size(20.dp),
493                 )
494                 Text(
495                     text = label,
496                     style = MaterialTheme.typography.bodyLarge,
497                     color = MaterialTheme.colorScheme.onSurface,
498                 )
499             }
500             Icon(
501                 imageVector = Icons.AutoMirrored.Filled.KeyboardArrowRight,
502                 contentDescription = null,
503                 tint = MaterialTheme.colorScheme.onSurfaceVariant,
504             )
505         }
506     }
507 }

```

```

1  package com.runway.android.ui.profile
2
3  import android.net.Uri
4  import androidx.compose.runtime.getValue
5  import androidx.compose.runtime.mutableStateOf
6  import androidx.compose.runtime.setValue
7  import androidx.lifecycle.ViewModel
8  import androidx.lifecycle.viewModelScope
9  import com.runway.android.core.result.NetworkResult
10 import com.runway.android.data.running.model.PersonalRecordsResponse
11 import com.runway.android.data.user.model.UpdateProfileRequest
12 import com.runway.android.domain.auth.AuthRepository
13 import com.runway.android.domain.running.RunningRepository
14 import com.runway.android.domain.user.UserRepository
15 import dagger.hilt.android.lifecycle.HiltViewModel
16 import kotlinx.coroutines.async
17 import kotlinx.coroutines.launch
18 import javax.inject.Inject
19
20 data class ProfileStats(
21     val totalRuns: Long,
22     val totalDistanceKm: String,
23     val currentStreakDays: Int = 0,
24 )
25
26 @HiltViewModel
27 class ProfileViewModel @Inject constructor(
28     private val authRepository: AuthRepository,
29     private val userRepository: UserRepository,
30     private val runningRepository: RunningRepository,
31 ) : ViewModel() {
32
33     var nickname by mutableStateOf("")
34     private set
35     var email by mutableStateOf("")
36     private set
37     var bio by mutableStateOf("")
38     private set
39     var profileImageUrl by mutableStateOf<String?>(null)
40     private set
41     var stats by mutableStateOf<ProfileStats?>(null)
42     private set
43     var personalRecords by mutableStateOf<PersonalRecordsResponse?>(null)
44     private set
45     var isLoading by mutableStateOf(true)
46     private set
47     var isRefreshing by mutableStateOf(false)
48     private set
49     var profileError by mutableStateOf(false)
50     private set
51
52     // 편집 상태
53     var isEditing by mutableStateOf(false)
54     private set
55     var editNickname by mutableStateOf("")
56     private set
57     var editBio by mutableStateOf("")
58     private set
59     var selectedImageUri by mutableStateOf<Uri?>(null)
60     private set
61     var isSaving by mutableStateOf(false)
62     private set
63     var saveError by mutableStateOf<String?>(null)
64     private set
65
66     init {
67         loadProfile()
68     }
69
70     fun refresh() {
71         viewModelScope.launch {
72             isRefreshing = true
73             doLoad()
74             isRefreshing = false
75         }
76     }
77
78     private fun loadProfile() {
79         viewModelScope.launch { doLoad() }
80     }
81
82     private suspend fun doLoad() = kotlinx.coroutines.coroutineScope {
83         val profileDeferred = async { userRepository.getMe() }

```

```

84     val recordsDeferred = async { runningRepository.getPersonalRecords() }
85     val statsDeferred = async { runningRepository.getRunningStats("all") }
86
87     val profileResult = profileDeferred.await()
88     val recordsResult = recordsDeferred.await()
89     val statsResult = statsDeferred.await()
90
91     if (profileResult is NetworkResult.Success) {
92         nickname = profileResult.data.nickname
93         email = profileResult.data.email
94         bio = profileResult.data.bio ?: ""
95         profileImageUrl = profileResult.data.profileImageUrl
96     } else {
97         profileError = true
98     }
99
100    val currentStreak = if (statsResult is NetworkResult.Success) statsResult.data.currentStreakDays else 0
101
102    if (recordsResult is NetworkResult.Success) {
103        val r = recordsResult.data
104        stats = ProfileStats(
105            totalRuns = r.totalCompletedRuns,
106            totalDistanceKm = "%.1f".format(r.totalDistanceMeters / 1000.0),
107            currentStreakDays = currentStreak,
108        )
109        personalRecords = r
110    } else {
111        stats = ProfileStats(totalRuns = 0L, totalDistanceKm = "0.0", currentStreakDays = currentStreak)
112    }
113
114    isLoading = false
115 }
116
117 fun startEditing() {
118     editNickname = nickname
119     editBio = bio
120     selectedImageUri = null
121     saveError = null
122     isEditing = true
123 }
124
125 fun cancelEditing() {
126     isEditing = false
127     selectedImageUri = null
128     saveError = null
129 }
130
131 fun updateEditNickname(value: String) { editNickname = value }
132 fun updateEditBio(value: String) { editBio = value }
133 fun onImageSelected(uri: Uri?) { selectedImageUri = uri }
134
135 fun saveProfile() {
136     val trimmedNickname = editNickname.trim()
137     if (trimmedNickname.isBlank()) {
138         saveError = "닉네임을 입력해 주세요."
139         return
140     }
141     viewModelScope.launch {
142         isSaving = true
143         saveError = null
144
145         // 이미지가 선택됐으면 먼저 업로드
146         val newImageUrl: String? = selectedImageUri?.let { uri ->
147             when (val uploadResult = userRepository.uploadProfileImage(uri)) {
148                 is NetworkResult.Success -> uploadResult.data
149                 else -> {
150                     saveError = "이미지 업로드에 실패했습니다."
151                     isSaving = false
152                     return@launch
153                 }
154             }
155         }
156
157         when (val result = userRepository.updateMe(
158             UpdateProfileRequest(
159                 nickname = trimmedNickname,
160                 profileImageUrl = newImageUrl ?: profileImageUrl,
161                 bio = editBio.trim().ifEmpty { null },
162             )
163         )) {
164             is NetworkResult.Success -> {
165                 nickname = result.data.nickname
166                 bio = result.data.bio ?: ""
167                 profileImageUrl = result.data.profileImageUrl
168                 selectedImageUri = null
169                 isEditing = false

```

```
170         }
171         is NetworkResult.ApiError -> saveError = when (result.errorCode) {
172             "DUPLICATED_NICKNAME" -> "이미 사용 중인 닉네임입니다."
173             else -> "저장에 실패했습니다."
174         }
175         is NetworkResult.NetworkError -> saveError = "네트워크 연결을 확인해 주세요."
176     }
177     isSaving = false
178 }
179 }
180
181 fun logout(onComplete: () -> Unit) {
182     viewModelScope.launch {
183         authRepository.logout()
184         onComplete()
185     }
186 }
187 }
```

```

1  package com.runway.android.ui.reminder
2
3  import android.app.TimePickerDialog
4  import androidx.compose.foundation.background
5  import androidx.compose.foundation.layout.Arrangement
6  import androidx.compose.foundation.layout.Column
7  import androidx.compose.foundation.layout.Row
8  import androidx.compose.foundation.layout.Spacer
9  import androidx.compose.foundation.layout.fillMaxSize
10 import androidx.compose.foundation.layout.fillMaxWidth
11 import androidx.compose.foundation.layout.height
12 import androidx.compose.foundation.layout.padding
13 import androidx.compose.foundation.layout.size
14 import androidx.compose.foundation.layout.statusBarsPadding
15 import androidx.compose.foundation.layout.width
16 import androidx.compose.foundation.rememberScrollState
17 import androidx.compose.foundation.verticalScroll
18 import androidx.compose.material.icons.Icons
19 import androidx.compose.material.icons.automirrored.filled.ArrowBack
20 import androidx.compose.material.icons.filled.AccessTime
21 import androidx.compose.material.icons.filled.NotificationsActive
22 import androidx.compose.material.icons.filled.NotificationsOff
23 import androidx.compose.material3.FilterChip
24 import androidx.compose.material3.Icon
25 import androidx.compose.material3.IconButton
26 import androidx.compose.material3.MaterialTheme
27 import androidx.compose.material3.Surface
28 import androidx.compose.material3.Switch
29 import androidx.compose.material3.Text
30 import androidx.compose.runtime.Composable
31 import androidx.compose.ui.Alignment
32 import androidx.compose.ui.Modifier
33 import androidx.compose.ui.platform.LocalContext
34 import androidx.compose.ui.text.font.FontWeight
35 import androidx.compose.ui.unit.dp
36 import androidx.hilt.navigation.compose.hiltViewModel
37 import java.util.Calendar
38
39 @Composable
40 fun ReminderScreen(
41     onBack: () -> Unit,
42     viewModel: ReminderViewModel = hiltViewModel(),
43 ) {
44     val context = LocalContext.current
45
46     Column(
47         modifier = Modifier
48             .fillMaxSize()
49             .background(MaterialTheme.colorScheme.background)
50             .statusBarsPadding()
51             .verticalScroll(rememberScrollState()),
52     ) {
53         // 상단 바
54         Row(
55             modifier = Modifier
56                 .fillMaxWidth()
57                 .padding(horizontal = 8.dp, vertical = 4.dp),
58             verticalAlignment = Alignment.CenterVertically,
59         ) {
60             IconButton(onClick = onBack) {
61                 Icon(
62                     imageVector = Icons.AutoMirrored.Filled.ArrowBack,
63                     contentDescription = "뒤로 가기",
64                     tint = MaterialTheme.colorScheme.onBackground,
65                 )
66             }
67             Spacer(modifier = Modifier.width(4.dp))
68             Text(
69                 text = "러닝 리마인더",
70                 style = MaterialTheme.typography.headlineMedium,
71                 color = MaterialTheme.colorScheme.onBackground,
72             )
73         }
74
75         Spacer(modifier = Modifier.height(16.dp))
76
77         // 알림 활성화 토글
78         Surface(
79             modifier = Modifier
80                 .fillMaxWidth()
81                 .padding(horizontal = 20.dp),
82             shape = MaterialTheme.shapes.extraLarge,
83             color = MaterialTheme.colorScheme.surface,

```

```

84         tonalElevation = 1.dp,
85     ) {
86         Row(
87             modifier = Modifier
88                 .fillMaxWidth()
89                 .padding(horizontal = 20.dp, vertical = 16.dp),
90             horizontalArrangement = Arrangement.SpaceBetween,
91             verticalAlignment = Alignment.CenterVertically,
92         ) {
93             Row(
94                 verticalAlignment = Alignment.CenterVertically,
95                 horizontalArrangement = Arrangement.spacedBy(12.dp),
96             ) {
97                 Icon(
98                     imageVector = if (viewModel.enabled) Icons.Filled.NotificationsActive
99                     else Icons.Filled.NotificationsOff,
100                    contentDescription = null,
101                    tint = if (viewModel.enabled) MaterialTheme.colorScheme.primary
102                    else MaterialTheme.colorScheme.onSurfaceVariant,
103                    modifier = Modifier.size(24.dp),
104                )
105                Column {
106                    Text(
107                        text = "리마인더 알림",
108                        style = MaterialTheme.typography.bodyLarge,
109                        fontWeight = FontWeight.Medium,
110                        color = MaterialTheme.colorScheme.onSurface,
111                    )
112                    Text(
113                        text = if (viewModel.enabled) "활성화됨" else "비활성화됨",
114                        style = MaterialTheme.typography.bodySmall,
115                        color = MaterialTheme.colorScheme.onSurfaceVariant,
116                    )
117                }
118            }
119            Switch(
120                checked = viewModel.enabled,
121                onCheckedChange = viewModel::toggleEnabled,
122            )
123        }
124    }
125
126    Spacer(modifier = Modifier.height(16.dp))
127
128    // 시간 설정
129    Surface(
130        modifier = Modifier
131            .fillMaxWidth()
132            .padding(horizontal = 20.dp),
133        shape = MaterialTheme.shapes.extraLarge,
134        color = MaterialTheme.colorScheme.surface,
135        tonalElevation = 1.dp,
136        onClick = {
137            TimePickerDialog(
138                context,
139                { _, h, m -> viewModel.setTime(h, m) },
140                viewModel.hour,
141                viewModel.minute,
142                true,
143            ).show()
144        },
145        enabled = viewModel.enabled,
146    ) {
147        Row(
148            modifier = Modifier
149                .fillMaxWidth()
150                .padding(horizontal = 20.dp, vertical = 20.dp),
151            horizontalArrangement = Arrangement.SpaceBetween,
152            verticalAlignment = Alignment.CenterVertically,
153        ) {
154            Row(
155                verticalAlignment = Alignment.CenterVertically,
156                horizontalArrangement = Arrangement.spacedBy(12.dp),
157            ) {
158                Icon(
159                    imageVector = Icons.Filled.AccessTime,
160                    contentDescription = null,
161                    tint = if (viewModel.enabled) MaterialTheme.colorScheme.primary
162                    else MaterialTheme.colorScheme.onSurface.copy(alpha = 0.38f),
163                    modifier = Modifier.size(24.dp),
164                )
165                Text(
166                    text = "알림 시간",
167                    style = MaterialTheme.typography.bodyLarge,
168                    color = if (viewModel.enabled) MaterialTheme.colorScheme.onSurface
169                    else MaterialTheme.colorScheme.onSurface.copy(alpha = 0.38f),

```

```

170         )
171     }
172     Text(
173         text = "%02d:%02d".format(viewModel.hour, viewModel.minute),
174         style = MaterialTheme.typography.titleLarge,
175         fontWeight = FontWeight.Bold,
176         color = if (viewModel.enabled) MaterialTheme.colorScheme.primary
177         else MaterialTheme.colorScheme.onSurface.copy(alpha = 0.38f),
178     )
179 }
180 }
181
182 Spacer(modifier = Modifier.height(16.dp))
183
184 // 요일 선택
185 Column(
186     modifier = Modifier
187         .fillMaxWidth()
188         .padding(horizontal = 20.dp),
189 ) {
190     Text(
191         text = "반복 요일",
192         style = MaterialTheme.typography.titleSmall,
193         color = if (viewModel.enabled) MaterialTheme.colorScheme.onSurface
194         else MaterialTheme.colorScheme.onSurface.copy(alpha = 0.38f),
195         modifier = Modifier.padding(bottom = 8.dp),
196     )
197     val days = listOf(
198         Calendar.MONDAY to "월",
199         Calendar.TUESDAY to "화",
200         Calendar.WEDNESDAY to "수",
201         Calendar.THURSDAY to "목",
202         Calendar.FRIDAY to "금",
203         Calendar.SATURDAY to "토",
204         Calendar.SUNDAY to "일",
205     )
206     Row(
207         horizontalArrangement = Arrangement.spacedBy(8.dp),
208     ) {
209         days.forEach { (dayConst, label) ->
210             FilterChip(
211                 selected = dayConst in viewModel.enabledDays,
212                 onClick = { if (viewModel.enabled) viewModel.toggleDay(dayConst) },
213                 label = { Text(label) },
214                 enabled = viewModel.enabled,
215             )
216         }
217     }
218 }
219
220 Spacer(modifier = Modifier.height(24.dp))
221
222 // 안내 문구
223 Text(
224     text = "설정된 요일과 시간에 러닝 알림이 전송됩니다.\n앱이 재시작된 경우에도 알림이 자동으로 복원됩니다.",
225     style = MaterialTheme.typography.bodySmall,
226     color = MaterialTheme.colorScheme.onSurfaceVariant,
227     modifier = Modifier.padding(horizontal = 24.dp),
228 )
229 }
230 }

```



```
1 package com.runway.android.ui.reminder
2
3 import androidx.compose.runtime.getValue
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.compose.runtime.setValue
6 import androidx.lifecycle.ViewModel
7 import androidx.lifecycle.viewModelScope
8 import com.runway.android.core.notification.ReminderScheduler
9 import com.runway.android.data.reminder.ReminderPrefs
10 import com.runway.android.data.reminder.ReminderPrefsRepository
11 import dagger.hilt.android.lifecycle.HiltViewModel
12 import kotlinx.coroutines.launch
13 import java.util.Calendar
14 import javax.inject.Inject
15
16 @HiltViewModel
17 class ReminderViewModel @Inject constructor(
18     private val reminderPrefs: ReminderPrefsRepository,
19     private val scheduler: ReminderScheduler,
20 ) : ViewModel() {
21
22     var enabled by mutableStateOf(false)
23     private set
24     var hour by mutableStateOf(7)
25     private set
26     var minute by mutableStateOf(0)
27     private set
28     var enabledDays by mutableStateOf(setOf(Calendar.MONDAY, Calendar.WEDNESDAY, Calendar.FRIDAY))
29     private set
30
31     init {
32         viewModelScope.launch {
33             reminderPrefs.getPrefs().collect { prefs ->
34                 enabled = prefs.enabled
35                 hour = prefs.hour
36                 minute = prefs.minute
37                 enabledDays = prefs.enabledDays
38             }
39         }
40     }
41
42     fun toggleEnabled(value: Boolean) {
43         enabled = value
44         save()
45     }
46
47     fun setTime(h: Int, m: Int) {
48         hour = h
49         minute = m
50         save()
51     }
52
53     fun toggleDay(dayOfWeek: Int) {
54         enabledDays = if (dayOfWeek in enabledDays) {
55             enabledDays - dayOfWeek
56         } else {
57             enabledDays + dayOfWeek
58         }
59         save()
60     }
61
62     private fun save() {
63         val prefs = ReminderPrefs(
64             enabled = enabled,
65             hour = hour,
66             minute = minute,
67             enabledDays = enabledDays,
68         )
69         viewModelScope.launch {
70             reminderPrefs.save(prefs)
71             scheduler.scheduleAll(prefs)
72         }
73     }
74 }
```

```

1  package com.runway.android.ui.running
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.background
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.layout.Arrangement
7  import androidx.compose.foundation.layout.Box
8  import androidx.compose.foundation.layout.Column
9  import androidx.compose.foundation.layout.Row
10 import androidx.compose.foundation.layout.Spacer
11 import androidx.compose.foundation.layout.fillMaxSize
12 import androidx.compose.foundation.layout.fillMaxWidth
13 import androidx.compose.foundation.layout.height
14 import androidx.compose.foundation.layout.padding
15 import androidx.compose.foundation.layout.size
16 import androidx.compose.foundation.layout.statusBarsPadding
17 import androidx.compose.foundation.layout.width
18 import androidx.compose.foundation.rememberScrollState
19 import androidx.compose.foundation.verticalScroll
20 import androidx.compose.material.icons.Icons
21 import androidx.compose.material.icons.automirrored.filled.DirectionsRun
22 import androidx.compose.material.icons.automirrored.filled.DirectionsWalk
23 import androidx.compose.material.icons.filled.LocalFireDepartment
24 import androidx.compose.material.icons.filled.Timer
25 import androidx.compose.material.icons.filled.CheckCircle
26 import androidx.compose.animation.AnimatedVisibility
27 import androidx.compose.animation.core.animateFloatAsState
28 import androidx.compose.animation.core.tween
29 import androidx.compose.animation.fadeIn
30 import androidx.compose.animation.slideInVertically
31 import androidx.compose.material3.CircularProgressIndicator
32 import androidx.compose.material3.Icon
33 import androidx.compose.material3.MaterialTheme
34 import androidx.compose.material3.Surface
35 import androidx.compose.material3.Text
36 import androidx.compose.material3.TextButton
37 import androidx.compose.runtime.Composable
38 import androidx.compose.runtime.LaunchedEffect
39 import androidx.compose.runtime.getValue
40 import androidx.compose.runtime.mutableStateOf
41 import androidx.compose.runtime.remember
42 import androidx.compose.runtime.setValue
43 import androidx.compose.ui.Alignment
44 import kotlinx.coroutines.delay
45 import androidx.compose.ui.Modifier
46 import androidx.compose.ui.draw.clip
47 import androidx.compose.ui.graphics.vector.ImageVector
48 import androidx.compose.ui.text.style.TextAlign
49 import androidx.compose.ui.unit.dp
50 import androidx.hilt.navigation.compose.hiltViewModel
51 import com.runway.android.ui.components.RouteMapView
52 import com.runway.android.ui.components.RunwayPrimaryButton
53 import com.runway.android.ui.course.CreateCourseDialog
54
55 @Composable
56 fun RunResultScreen(
57     onBackToHome: () -> Unit,
58     onShareImage: (runId: String) -> Unit = {},
59     onOpenRunDetail: (String) -> Unit = {},
60     onNavigateToCourseDetail: (String) -> Unit = {},
61     viewModel: RunResultViewModel = hiltViewModel(),
62 ) {
63     LaunchedEffect(Unit) {
64         viewModel.courseCreated.collect { courseId ->
65             onNavigateToCourseDetail(courseId)
66         }
67     }
68
69     var distTriggered by remember { mutableStateOf(false) }
70     var cardsVisible by remember { mutableStateOf(false) }
71     LaunchedEffect(Unit) {
72         distTriggered = true
73         delay(250)
74         cardsVisible = true
75     }
76     val distTarget = viewModel.distanceText.toFloatOrNull() ?: 0f
77     val animatedDist by animateFloatAsState(
78         targetValue = if (distTriggered) distTarget else 0f,
79         animationSpec = tween(900),
80         label = "heroDistance",
81     )
82
83     Box(

```

```

84     modifier = Modifier
85         .fillMaxSize()
86         .background(MaterialTheme.colorScheme.background),
87     ) {
88         // — Scrollable content —
89         Column(
90             modifier = Modifier
91                 .fillMaxSize()
92                 .verticalScroll(rememberScrollState())
93                 .statusBarsPadding()
94                 .padding(bottom = 160.dp),
95         ) {
96             // Top bar
97             Box(
98                 modifier = Modifier
99                     .fillMaxWidth()
100                     .padding(horizontal = 20.dp, vertical = 12.dp),
101             ) {
102                 Text(
103                     text = "러닝 완료",
104                     style = MaterialTheme.typography.titleMedium,
105                     color = MaterialTheme.colorScheme.onBackground,
106                     modifier = Modifier.align(Alignment.Center),
107                 )
108             }
109
110             Spacer(modifier = Modifier.height(16.dp))
111
112             // Distance summary
113             Column(
114                 modifier = Modifier.fillMaxWidth(),
115                 horizontalAlignment = Alignment.CenterHorizontally,
116             ) {
117                 Text(
118                     text = "RUN COMPLETE",
119                     style = MaterialTheme.typography.labelLarge,
120                     color = MaterialTheme.colorScheme.primary,
121                 )
122                 Spacer(modifier = Modifier.height(8.dp))
123                 Row(verticalAlignment = Alignment.Bottom) {
124                     Text(
125                         text = "%.2f".format(animatedDist),
126                         style = MaterialTheme.typography.displayMedium,
127                         color = MaterialTheme.colorScheme.onBackground,
128                     )
129                     Spacer(modifier = Modifier.width(6.dp))
130                     Text(
131                         text = "km",
132                         style = MaterialTheme.typography.headlineMedium,
133                         color = MaterialTheme.colorScheme.onSurfaceVariant,
134                         modifier = Modifier.padding(bottom = 8.dp),
135                     )
136                 }
137                 Text(
138                     text = "자유 런",
139                     style = MaterialTheme.typography.bodyMedium,
140                     color = MaterialTheme.colorScheme.onSurfaceVariant,
141                 )
142             }
143
144             Spacer(modifier = Modifier.height(24.dp))
145
146             // — Result cards 2x2 —
147             AnimatedVisibility(
148                 visible = cardsVisible,
149                 enter = slideInVertically(tween(350)) { it / 2 } + fadeIn(tween(350)),
150             ) {
151                 Column(
152                     modifier = Modifier.padding(horizontal = 20.dp),
153                     verticalArrangement = Arrangement.spacedBy(12.dp),
154                 ) {
155                     Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
156                         ResultCard(
157                             icon = Icons.Filled.Timer,
158                             label = "TIME",
159                             value = viewModel.timerText,
160                             modifier = Modifier.weight(1f),
161                         )
162                         ResultCard(
163                             icon = Icons.AutoMirrored.Filled.DirectionsRun,
164                             label = "PACE",
165                             value = viewModel.paceText,
166                             modifier = Modifier.weight(1f),
167                         )
168                     }
169                     Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {

```

```

170         ResultCard(
171             icon = Icons.Filled.LocalFireDepartment,
172             label = "CALORIES",
173             value = viewModel.caloriesText,
174             modifier = Modifier.weight(1f),
175         )
176         ResultCard(
177             icon = Icons.AutoMirrored.Filled.DirectionsWalk,
178             label = "STEPS",
179             value = viewModel.stepsText,
180             modifier = Modifier.weight(1f),
181         )
182     }
183 }
184 }
185
186 Spacer(modifier = Modifier.height(16.dp))
187
188 // — Route map preview —
189 Box(
190     modifier = Modifier
191         .fillMaxWidth()
192         .height(140.dp)
193         .padding(horizontal = 20.dp)
194         .clip(MaterialTheme.shapes.extraLarge)
195         .clickable(enabled = viewModel.runId != null && !viewModel.isLoadingRoute) {
196             viewModel.runId?.let(onOpenRunDetail)
197         },
198 ) {
199     if (viewModel.isLoadingRoute) {
200         Box(
201             modifier = Modifier
202                 .fillMaxSize()
203                 .background(MaterialTheme.colorScheme.surfaceVariant),
204             contentAlignment = Alignment.Center,
205         ) {
206             CircularProgressIndicator(modifier = Modifier.size(28.dp))
207         }
208     } else {
209         RouteMapView(
210             points = viewModel.routePoints,
211             modifier = Modifier.fillMaxSize(),
212         )
213         Box(
214             modifier = Modifier
215                 .fillMaxSize()
216                 .clickable(enabled = viewModel.runId != null) {
217                     viewModel.runId?.let(onOpenRunDetail)
218                 },
219         )
220         Surface(
221             modifier = Modifier
222                 .align(Alignment.BottomEnd)
223                 .padding(10.dp),
224             shape = MaterialTheme.shapes.large,
225             color = MaterialTheme.colorScheme.surface.copy(alpha = 0.88f),
226             border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
227         ) {
228             Text(
229                 text = "지도 자세히 보기",
230                 style = MaterialTheme.typography.labelMedium,
231                 color = MaterialTheme.colorScheme.primary,
232                 modifier = Modifier.padding(horizontal = 10.dp, vertical = 6.dp),
233             )
234         }
235     }
236 }
237 }
238
239 // — Sticky bottom buttons —
240 Column(
241     modifier = Modifier
242         .align(Alignment.BottomCenter)
243         .fillMaxWidth()
244         .padding(horizontal = 20.dp, vertical = 24.dp),
245     verticalArrangement = Arrangement.spacedBy(8.dp),
246 ) {
247     if (viewModel.courseCreatedSuccess) {
248         Surface(
249             modifier = Modifier.fillMaxWidth(),
250             shape = MaterialTheme.shapes.extraLarge,
251             color = MaterialTheme.colorScheme.primaryContainer,
252         ) {
253             Row(
254                 modifier = Modifier.padding(vertical = 14.dp),
255                 horizontalArrangement = Arrangement.Center,

```

```

256         verticalAlignment = Alignment.CenterVertically,
257     ) {
258         Icon(
259             imageVector = Icons.Filled.CheckCircle,
260             contentDescription = null,
261             tint = MaterialTheme.colorScheme.onPrimaryContainer,
262             modifier = Modifier.size(18.dp),
263         )
264         Spacer(modifier = Modifier.width(8.dp))
265         Text(
266             text = "코스가 생성되었습니다!",
267             style = MaterialTheme.typography.labelLarge,
268             color = MaterialTheme.colorScheme.onPrimaryContainer,
269         )
270     }
271 }
272 } else {
273     Text(
274         text = "방금 달린 경로를 다른 사람이 도전할 수 있는 코스로 만들어보세요.",
275         style = MaterialTheme.typography.bodySmall,
276         color = MaterialTheme.colorScheme.onSurfaceVariant,
277         textAlign = TextAlign.Center,
278         modifier = Modifier.fillMaxWidth(),
279     )
280     RunwayPrimaryButton(
281         text = "이 러닝으로 코스 만들기",
282         onClick = { viewModel.onShowCreateDialog() },
283         enabled = viewModel.runId != null,
284     )
285 }
286 if (viewModel.runId != null) {
287     TextButton(
288         onClick = { onShareImage(viewModel.runId) },
289         modifier = Modifier.fillMaxWidth(),
290     ) {
291         Text(
292             text = "공유 이미지 만들기",
293             style = MaterialTheme.typography.bodyLarge,
294             color = MaterialTheme.colorScheme.primary,
295             textAlign = TextAlign.Center,
296         )
297     }
298 }
299 TextButton(
300     onClick = onBackToHome,
301     modifier = Modifier.fillMaxWidth(),
302 ) {
303     Text(
304         text = "홈으로",
305         style = MaterialTheme.typography.bodyLarge,
306         color = MaterialTheme.colorScheme.onSurfaceVariant,
307         textAlign = TextAlign.Center,
308     )
309 }
310 }
311 }
312
313 // — 코스 저장 다이얼로그 (항상 draft로 저장) —
314 if (viewModel.showCreateDialog) {
315     CreateCourseDialog(
316         courseName = viewModel.courseName,
317         onCourseNameChange = viewModel::onCourseNameChange,
318         isLoop = viewModel.isLoop,
319         onIsLoopChange = viewModel::onIsLoopChange,
320         isCreating = viewModel.isCreating,
321         errorMessage = viewModel.createError,
322         onConfirm = viewModel::createCourse,
323         onDismiss = viewModel::onDismissCreateDialog,
324     )
325 }
326 }
327
328 @Composable
329 private fun ResultCard(
330     icon: ImageVector,
331     label: String,
332     value: String,
333     modifier: Modifier = Modifier,
334 ) {
335     Surface(
336         modifier = modifier,
337         shape = MaterialTheme.shapes.extraLarge,
338         color = MaterialTheme.colorScheme.surface,
339         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
340     ) {
341         Column(modifier = Modifier.padding(16.dp)) {

```

```
342         Row(verticalAlignment = Alignment.CenterVertically) {
343             Icon(
344                 imageVector = icon,
345                 contentDescription = null,
346                 tint = MaterialTheme.colorScheme.onSurfaceVariant,
347                 modifier = Modifier.size(14.dp),
348             )
349             Spacer(modifier = Modifier.width(4.dp))
350             Text(
351                 text = label,
352                 style = MaterialTheme.typography.labelMedium,
353                 color = MaterialTheme.colorScheme.onSurfaceVariant,
354             )
355         }
356         Spacer(modifier = Modifier.height(6.dp))
357         Text(
358             text = value,
359             style = MaterialTheme.typography.titleLarge,
360             color = MaterialTheme.colorScheme.onSurface,
361         )
362     }
363 }
364 }
```

```

1  package com.runway.android.ui.running
2
3  import androidx.compose.runtime.getValue
4  import androidx.compose.runtime.mutableStateOf
5  import androidx.compose.runtime.setValue
6  import androidx.lifecycle.SavedStateHandle
7  import androidx.lifecycle.ViewModel
8  import androidx.lifecycle.viewModelScope
9  import com.runway.android.core.map.MapPoint
10 import com.runway.android.core.result.NetworkResult
11 import com.runway.android.data.course.model.CreateCourseFromRunRequest
12 import com.runway.android.domain.course.CourseRepository
13 import com.runway.android.domain.running.RunningRepository
14 import dagger.hilt.android.lifecycle.HiltViewModel
15 import kotlinx.coroutines.flow.MutableSharedFlow
16 import kotlinx.coroutines.flow.asSharedFlow
17 import kotlinx.coroutines.launch
18 import java.util.Locale
19 import javax.inject.Inject
20
21 @HiltViewModel
22 class RunResultViewModel @Inject constructor(
23     savedStateHandle: SavedStateHandle,
24     private val courseRepository: CourseRepository,
25     private val runningRepository: RunningRepository,
26 ) : ViewModel() {
27
28     val runId: String? = savedStateHandle.get<String>("runId")?.takeIf { it != "none" }
29
30     var routePoints by mutableStateOf<List<MapPoint>>(emptyList())
31     private set
32
33     var isLoadingRoute by mutableStateOf(runId != null)
34     private set
35
36     private val elapsedSeconds: Int = savedStateHandle.get<Int>("elapsedSeconds") ?: 0
37     private val distanceKm: Float = savedStateHandle.get<Float>("distanceKm") ?: 0f
38
39     init {
40         runId?.let { id ->
41             viewModelScope.launch {
42                 val result = runningRepository.getRunDetail(id)
43                 if (result is NetworkResult.Success) {
44                     routePoints = result.data.points.map { MapPoint(it.latitude, it.longitude) }
45                 }
46                 isLoadingRoute = false
47             }
48         }
49     }
50
51     val timerText: String = run {
52         val h = elapsedSeconds / 3600
53         val m = (elapsedSeconds % 3600) / 60
54         val s = elapsedSeconds % 60
55         if (h > 0) "%d:%02d:%02d".format(h, m, s) else "%02d:%02d".format(m, s)
56     }
57
58     val distanceText: String = "%.2f".format(distanceKm)
59     val paceText: String = if (distanceKm > 0.001f) {
60         val secsPerKm = (elapsedSeconds / distanceKm).toInt()
61         "%d'%02d\"/km".format(secsPerKm / 60, secsPerKm % 60)
62     } else {
63         "--'--\"/km"
64     }
65
66     val caloriesText: String = "${(distanceKm * 72).toInt()} kcal"
67     val stepsText: String = String.format(Locale.US, "%d", (distanceKm * 1320).toInt())
68
69     // — 코스 생성 다이얼로그 상태 —
70
71     var showCreateDialog by mutableStateOf(false)
72     private set
73     var courseName by mutableStateOf("")
74     private set
75     var courseDescription by mutableStateOf("")
76     private set
77     var isLoop by mutableStateOf(false)
78     private set
79     var isCreating by mutableStateOf(false)
80     private set
81     var createError by mutableStateOf<String?>(null)
82     private set
83     var courseCreatedSuccess by mutableStateOf(false)
84     private set

```

```

84
85 private val _courseCreated = MutableSharedFlow<String>()
86 val courseCreated = _courseCreated.asSharedFlow()
87
88 fun onShowCreateDialog() {
89     createError = null
90     showCreateDialog = true
91 }
92
93 fun onDismissCreateDialog() {
94     if (!isCreating) showCreateDialog = false
95 }
96
97 fun onCourseNameChange(value: String) {
98     courseName = value
99     createError = null
100 }
101
102 fun onDescriptionChange(value: String) {
103     courseDescription = value
104 }
105
106 fun onIsLoopChange(value: Boolean) {
107     isLoop = value
108 }
109
110 fun createCourse() {
111     val id = runId ?: return
112     if (courseName.isBlank()) {
113         createError = "코스 이름을 입력해 주세요."
114         return
115     }
116     viewModelScope.launch {
117         isCreating = true
118         createError = null
119         val request = CreateCourseFromRunRequest(
120             name = courseName.trim(),
121             description = null,
122             isLoop = isLoop,
123             publish = false,
124         )
125         when (val result = courseRepository.createCourseFromRun(id, request)) {
126             is NetworkResult.Success -> {
127                 showCreateDialog = false
128                 courseCreatedSuccess = true
129                 kotlinx.coroutines.delay(1_500)
130                 _courseCreated.emit(result.data.courseId)
131             }
132             is NetworkResult.ApiError -> {
133                 createError = when {
134                     result.errorCode == "NOT_COMPLETED_RUN" ->
135                         "완료된 러닝만 코스로 만들 수 있습니다."
136                     result.statusCode == 401 ->
137                         "로그인이 만료되었습니다. 다시 로그인해주세요."
138                     else -> result.message
139                 }
140             }
141             is NetworkResult.NetworkError ->
142                 createError = "네트워크 오류가 발생했습니다."
143         }
144         isCreating = false
145     }
146 }
147 }

```



```

1  package com.runway.android.ui.running
2
3  import android.content.Context
4  import android.os.Build
5  import android.os.VibrationEffect
6  import android.os.Vibrator
7  import android.os.VibratorManager
8  import androidx.compose.animation.AnimatedContent
9  import androidx.compose.animation.core.Animatable
10 import androidx.compose.animation.core.LinearEasing
11 import androidx.compose.animation.core.tween
12 import androidx.compose.animation.fadeIn
13 import androidx.compose.animation.fadeOut
14 import androidx.compose.animation.scaleIn
15 import androidx.compose.animation.scaleOut
16 import androidx.compose.animation.togetherWith
17 import androidx.compose.foundation.Canvas
18 import androidx.compose.foundation.background
19 import androidx.compose.foundation.gestures.detectTapGestures
20 import androidx.compose.foundation.layout.Box
21 import androidx.compose.foundation.layout.fillMaxSize
22 import androidx.compose.foundation.layout.padding
23 import androidx.compose.foundation.layout.size
24 import androidx.compose.material3.MaterialTheme
25 import androidx.compose.material3.Text
26 import androidx.compose.runtime.Composable
27 import androidx.compose.runtime.LaunchedEffect
28 import androidx.compose.runtime.getValue
29 import androidx.compose.runtime.mutableIntStateOf
30 import androidx.compose.runtime.mutableStateOf
31 import androidx.compose.runtime.remember
32 import androidx.compose.runtime.setValue
33 import androidx.compose.ui.Alignment
34 import androidx.compose.ui.Modifier
35 import androidx.compose.ui.graphics.Color
36 import androidx.compose.ui.input.pointer.pointerInput
37 import androidx.compose.ui.graphics.StrokeCap
38 import androidx.compose.ui.graphics.drawscope.Stroke
39 import androidx.compose.ui.platform.LocalContext
40 import androidx.compose.ui.text.font.FontWeight
41 import androidx.compose.ui.unit.dp
42 import androidx.compose.ui.unit.sp
43 import com.runway.android.BuildConfig
44 import kotlinx.coroutines.delay
45
46 private val CountdownBackground = Color(0xFF07070E)
47
48 @Composable
49 fun RunningCountdownOverlay(
50     onFinish: () -> Unit,
51     modifier: Modifier = Modifier,
52 ) {
53     var count by remember { mutableIntStateOf(3) }
54     var skipped by remember { mutableStateOf(false) }
55     val arcProgress = remember { Animatable(0f) }
56     val context = LocalContext.current
57
58     LaunchedEffect(count) {
59         arcProgress.snapTo(0f)
60         if (count > 0) {
61             arcProgress.animateTo(1f, animationSpec = tween(900, easing = LinearEasing))
62         }
63     }
64
65     LaunchedEffect(Unit) {
66         vibrateCountdownTick(context)
67         delay(1_000); if (skipped) return@LaunchedEffect
68         count = 2
69         vibrateCountdownTick(context)
70         delay(1_000); if (skipped) return@LaunchedEffect
71         count = 1
72         vibrateCountdownTick(context)
73         delay(1_000); if (skipped) return@LaunchedEffect
74         count = 0
75         vibrateCountdownGo(context)
76         delay(500)
77         onFinish()
78     }
79
80     val primary = MaterialTheme.colorScheme.primary
81
82     Box(
83         modifier = modifier

```

```

84         .fillMaxSize()
85         .background(CountdownBackground)
86         .pointerInput(Unit) {
87             detectTapGestures {
88                 skipped = true
89                 vibrateCountdownGo(context)
90                 onFinish()
91             }
92         },
93         contentAlignment = Alignment.Center,
94     ) {
95         // Subtle brand mark at top
96         Text(
97             text = BuildConfig.APP_NAME.uppercase(),
98             fontSize = 13.sp,
99             fontWeight = FontWeight.ExtraBold,
100            letterSpacing = 5.sp,
101            color = Color.White.copy(alpha = 0.10f),
102            modifier = Modifier
103                .align(Alignment.TopCenter)
104                .padding(top = 60.dp),
105        )
106
107        // Tap hint
108        Text(
109            text = "화면을 탭하면 바로 시작",
110            fontSize = 13.sp,
111            color = Color.White.copy(alpha = 0.35f),
112            modifier = Modifier
113                .align(Alignment.BottomCenter)
114                .padding(bottom = 72.dp),
115        )
116
117        // Ring + number
118        Box(
119            contentAlignment = Alignment.Center,
120            modifier = Modifier.size(220.dp),
121        ) {
122            Canvas(modifier = Modifier.fillMaxSize()) {
123                val stroke = Stroke(width = 6.dp.toPx(), cap = StrokeCap.Round)
124                // Dim background ring
125                drawArc(
126                    color = primary.copy(alpha = 0.15f),
127                    startAngle = -90f,
128                    sweepAngle = 360f,
129                    useCenter = false,
130                    style = stroke,
131                )
132                // Bright sweeping ring
133                drawArc(
134                    color = primary,
135                    startAngle = -90f,
136                    sweepAngle = 360f * arcProgress.value,
137                    useCenter = false,
138                    style = stroke,
139                )
140            }
141
142            AnimatedContent(
143                targetState = count,
144                transitionSpec = {
145                    // New number slams in from large scale; old shrinks out
146                    scaleIn(
147                        initialScale = 1.5f,
148                        animationSpec = tween(260),
149                    ) + fadeIn(tween(180)) togetherWith
150                    scaleOut(
151                        targetScale = 0.7f,
152                        animationSpec = tween(220),
153                    ) + fadeOut(tween(160))
154                },
155                label = "countdown_number",
156            ) { c ->
157                Text(
158                    text = if (c == 0) "GO!" else c.toString(),
159                    fontSize = if (c == 0) 72.sp else 116.sp,
160                    fontWeight = FontWeight.Black,
161                    color = if (c == 0) primary else Color.White,
162                )
163            }
164        }
165    }
166 }
167 }
168
169 @Suppress("DEPRECATION")

```

```
170 private fun vibrateCountdownTick(context: Context) {
171     getVibrator(context).vibrate(
172         VibrationEffect.createOneShot(38, VibrationEffect.DEFAULT_AMPLITUDE)
173     )
174 }
175
176 @Suppress("DEPRECATION")
177 private fun vibrateCountdownGo(context: Context) {
178     getVibrator(context).vibrate(
179         VibrationEffect.createWaveform(longArrayOf(0, 80, 45, 130), -1)
180     )
181 }
182
183 @Suppress("DEPRECATION")
184 private fun getVibrator(context: Context): Vibrator =
185     if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
186         (context.getSystemService(Context.VIBRATOR_MANAGER_SERVICE) as VibratorManager).defaultVibrator
187     } else {
188         context.getSystemService(Context.VIBRATOR_SERVICE) as Vibrator
189     }
```

```

1  package com.runway.android.ui.running
2
3  import android.Manifest
4  import android.app.Activity
5  import android.content.pm.PackageManager
6  import android.os.Build
7  import androidx.activity.compose.BackHandler
8  import androidx.activity.compose.rememberLauncherForActivityResult
9  import androidx.activity.result.contract.ActivityResultContracts
10 import androidx.compose.animation.AnimatedContent
11 import androidx.compose.animation.AnimatedVisibility
12 import androidx.compose.animation.EnterTransition
13 import androidx.compose.animation.core.InfiniteRepeatableSpec
14 import androidx.compose.animation.core.RepeatMode
15 import androidx.compose.animation.core.animateFloat
16 import androidx.compose.animation.core.rememberInfiniteTransition
17 import androidx.compose.animation.core.tween
18 import androidx.compose.animation.fadeIn
19 import androidx.compose.animation.fadeOut
20 import androidx.compose.animation.slideInVertically
21 import androidx.compose.animation.slideOutVertically
22 import androidx.compose.animation.togetherWith
23 import androidx.compose.foundation.BorderStroke
24 import androidx.compose.foundation.Canvas
25 import androidx.compose.foundation.background
26 import androidx.compose.foundation.layout.Arrangement
27 import androidx.compose.foundation.layout.Box
28 import androidx.compose.foundation.layout.Column
29 import androidx.compose.foundation.layout.Row
30 import androidx.compose.foundation.layout.Spacer
31 import androidx.compose.foundation.layout.fillMaxSize
32 import androidx.compose.foundation.layout.fillMaxWidth
33 import androidx.compose.foundation.layout.height
34 import androidx.compose.foundation.layout.padding
35 import androidx.compose.foundation.layout.size
36 import androidx.compose.foundation.layout.statusBarsPadding
37 import androidx.compose.foundation.layout.width
38 import androidx.compose.foundation.shape.CircleShape
39 import androidx.compose.foundation.shape.RoundedCornerShape
40 import androidx.compose.material.icons.Icons
41 import androidx.compose.material.icons.filled.Pause
42 import androidx.compose.material.icons.filled.PlayArrow
43 import androidx.compose.material.icons.filled.Stop
44 import androidx.compose.material3.AlertDialog
45 import androidx.compose.material3.CircularProgressIndicator
46 import androidx.compose.material3.MaterialTheme
47 import androidx.compose.material3.Surface
48 import androidx.compose.material3.Text
49 import androidx.compose.material3.TextButton
50 import androidx.compose.runtime.Composable
51 import androidx.compose.runtime.LaunchedEffect
52 import androidx.compose.runtime.getValue
53 import androidx.compose.runtime.mutableStateOf
54 import androidx.compose.runtime.remember
55 import androidx.compose.runtime.setValue
56 import androidx.compose.ui.Alignment
57 import androidx.compose.ui.Modifier
58 import androidx.compose.ui.draw.clip
59 import androidx.compose.ui.graphics.Color
60 import androidx.compose.ui.platform.LocalContext
61 import androidx.compose.ui.text.font.FontWeight
62 import androidx.compose.ui.text.style.TextAlign
63 import androidx.compose.ui.unit.dp
64 import androidx.compose.ui.unit.sp
65 import androidx.core.content.ContextCompat
66 import androidx.hilt.navigation.compose.hiltViewModel
67 import com.runway.android.core.location.GpsStatus
68 import com.runway.android.ui.components.BatteryOptimizationCard
69 import com.runway.android.ui.components.LocationPermissionCard
70 import com.runway.android.ui.components.RunningControlButton
71 import com.runway.android.ui.components.rememberBatteryOptimizationIgnored
72 import com.runway.android.ui.theme.WarningYellow
73
74 @Composable
75 fun RunningTrackingScreen(
76     onFinish: (runId: String?, elapsedSeconds: Int, distanceKm: Float) -> Unit,
77     onBack: () -> Unit,
78     viewModel: RunningTrackingViewModel = hiltViewModel(),
79 ) {
80     LaunchedEffect(Unit) {
81         viewModel.navigateToResult.collect { result ->
82             onFinish(result.runId, result.elapsedSeconds, result.distanceKm)
83         }
84     }

```

```

84     }
85
86     val context = LocalContext.current
87     var permissionDeniedPermanently by remember { mutableStateOf(false) }
88     var showBatteryCard by remember { mutableStateOf(true) }
89     val isBatteryOptimizationIgnored = rememberBatteryOptimizationIgnored()
90
91     val permissionLauncher = rememberLauncherForActivityResult(
92         ActivityResultContracts.RequestMultiplePermissions()
93     ) { permissions ->
94         val locationGranted = permissions[Manifest.permission.ACCESS_FINE_LOCATION] == true ||
95             permissions[Manifest.permission.ACCESS_COARSE_LOCATION] == true
96         if (locationGranted) {
97             viewModel.startTracking()
98         } else {
99             val activity = context as? Activity
100             val shouldShow = activity?.shouldShowRequestPermissionRationale(
101                 Manifest.permission.ACCESS_FINE_LOCATION
102             ) ?: false
103             permissionDeniedPermanently = !shouldShow
104         }
105     }
106
107     var countdownDone by remember { mutableStateOf(false) }
108
109     LaunchedEffect(countdownDone) {
110         if (!countdownDone) return@LaunchedEffect
111         val hasFine = ContextCompat.checkSelfPermission(
112             context, Manifest.permission.ACCESS_FINE_LOCATION
113         ) == PackageManager.PERMISSION_GRANTED
114         val hasCoarse = ContextCompat.checkSelfPermission(
115             context, Manifest.permission.ACCESS_COARSE_LOCATION
116         ) == PackageManager.PERMISSION_GRANTED
117         val hasActivityRecognition = Build.VERSION.SDK_INT < Build.VERSION_CODES.Q ||
118             ContextCompat.checkSelfPermission(
119                 context,
120                 Manifest.permission.ACTIVITY_RECOGNITION,
121             ) == PackageManager.PERMISSION_GRANTED
122
123         if ((hasFine || hasCoarse) && hasActivityRecognition) {
124             viewModel.startTracking()
125         } else {
126             val permissions = buildList {
127                 add(Manifest.permission.ACCESS_FINE_LOCATION)
128                 add(Manifest.permission.ACCESS_COARSE_LOCATION)
129                 if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
130                     add(Manifest.permission.ACTIVITY_RECOGNITION)
131                 }
132                 if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
133                     add(Manifest.permission.POST_NOTIFICATIONS)
134                 }
135             }.toTypedArray()
136             permissionLauncher.launch(permissions)
137         }
138     }
139
140     val isRunning = viewModel.runningState == RunningState.RUNNING
141
142     var showExitDialog by remember { mutableStateOf(false) }
143     BackHandler(enabled = !viewModel.isFinishing) {
144         if (!countdownDone) onBack() else showExitDialog = true
145     }
146
147     if (showExitDialog) {
148         AlertDialog(
149             onDismissRequest = { showExitDialog = false },
150             title = { Text("러닝 화면을 나가시겠어요?") },
151             text = { Text("러닝은 백그라운드에서 계속됩니다. 홈으로 돌아가도 기록은 유지됩니다.") },
152             confirmButton = {
153                 TextButton(onClick = { showExitDialog = false; onBack() }) {
154                     Text("나가기")
155                 }
156             },
157             dismissButton = {
158                 TextButton(onClick = { showExitDialog = false }) {
159                     Text("계속 달리기")
160                 }
161             },
162         )
163     }
164
165     Box(modifier = Modifier.fillMaxSize()) {
166         Column(
167             modifier = Modifier
168                 .fillMaxSize()
169                 .background(MaterialTheme.colorScheme.background)

```

```

170         .statusBarsPadding(),
171     ) {
172         // — Status row —
173         Row(
174             modifier = Modifier
175                 .fillMaxWidth()
176                 .padding(horizontal = 20.dp, vertical = 12.dp),
177             horizontalArrangement = Arrangement.SpaceBetween,
178             verticalAlignment = Alignment.CenterVertically,
179         ) {
180             RecordingPill(state = viewModel.runningState)
181             Text(
182                 text = when {
183                     viewModel.isConnected -> "GPS 연결 중..."
184                     viewModel.gpsStatus == GpsStatus.PERMISSION_REQUIRED -> "위치 권한 필요"
185                     viewModel.gpsStatus == GpsStatus.WAITING_FOR_FIX -> "GPS 신호 수신 중..."
186                     viewModel.runningState == RunningState.AUTO_PAUSED -> "자동 일시정지 중"
187                     !isRunning -> "일시정지"
188                     else -> "GPS · Active"
189                 },
190                 style = MaterialTheme.typography.labelLarge,
191                 color = MaterialTheme.colorScheme.onSurfaceVariant,
192             )
193         }
194
195         Spacer(modifier = Modifier.height(12.dp))
196
197         // — Milestone banner —
198         viewModel.milestoneMessage?.let { msg ->
199             Surface(
200                 modifier = Modifier
201                     .fillMaxWidth()
202                     .padding(horizontal = 20.dp),
203                 shape = MaterialTheme.shapes.extraLarge,
204                 color = MaterialTheme.colorScheme.primaryContainer,
205                 border = BorderStroke(1.dp, MaterialTheme.colorScheme.primary.copy(alpha = 0.4f)),
206             ) {
207                 Text(
208                     text = msg,
209                     style = MaterialTheme.typography.bodyMedium,
210                     color = MaterialTheme.colorScheme.onPrimaryContainer,
211                     textAlign = TextAlign.Center,
212                     modifier = Modifier.padding(horizontal = 16.dp, vertical = 10.dp),
213                 )
214             }
215             Spacer(modifier = Modifier.height(8.dp))
216         }
217
218         // — Battery optimization card —
219         if (showBatteryCard && !isBatteryOptimizationIgnored && viewModel.gpsStatus == GpsStatus.ACTIVE) {
220             BatteryOptimizationCard(
221                 onDismiss = { showBatteryCard = false },
222                 modifier = Modifier.padding(horizontal = 20.dp),
223             )
224             Spacer(modifier = Modifier.height(8.dp))
225         }
226
227         if (viewModel.gpsStatus == GpsStatus.PERMISSION_REQUIRED) {
228             Box(
229                 modifier = Modifier
230                     .weight(1f)
231                     .fillMaxWidth(),
232                 contentAlignment = Alignment.Center,
233             ) {
234                 LocationPermissionCard(
235                     isPermanentlyDenied = permissionDeniedPermanently,
236                     onRequestPermission = {
237                         permissionLauncher.launch(
238                             arrayOf(
239                                 Manifest.permission.ACCESS_FINE_LOCATION,
240                                 Manifest.permission.ACCESS_COARSE_LOCATION,
241                             )
242                         )
243                     },
244                 )
245             }
246         } else {
247             FreeRunDataPanel(
248                 timerText = viewModel.timerText,
249                 distanceText = viewModel.distanceText,
250                 paceText = viewModel.paceText,
251                 speedText = viewModel.speedText,
252                 cadenceText = viewModel.cadenceText,
253                 isRunning = isRunning,
254                 modifier = Modifier
255                     .weight(1f)

```

```

256         .fillMaxWidth()
257         .padding(horizontal = 20.dp),
258     )
259 }
260
261 Spacer(modifier = Modifier.height(24.dp))
262
263 // — Controls —
264 Row(
265     modifier = Modifier.fillMaxWidth(),
266     horizontalArrangement = Arrangement.Center,
267     verticalAlignment = Alignment.CenterVertically,
268 ) {
269     if (viewModel.isFinishing) {
270         CircularProgressIndicator(
271             modifier = Modifier.size(60.dp),
272             color = MaterialTheme.colorScheme.error,
273             strokeWidth = 3.dp,
274         )
275     } else {
276         RunningControlButton(
277             icon = Icons.Filled.Stop,
278             onClick = viewModel::finish,
279             size = 60.dp,
280             containerColor = MaterialTheme.colorScheme.surface,
281             contentColor = MaterialTheme.colorScheme.error,
282             border = BorderStroke(1.dp, MaterialTheme.colorScheme.error.copy(alpha = 0.5f)),
283         )
284     }
285
286     Spacer(modifier = Modifier.width(24.dp))
287
288     if (isRunning) {
289         RunningControlButton(
290             icon = Icons.Filled.Pause,
291             onClick = viewModel::pause,
292             size = 84.dp,
293             containerColor = MaterialTheme.colorScheme.primary,
294             contentColor = MaterialTheme.colorScheme.onPrimary,
295         )
296     } else {
297         // Shows resume for both PAUSED and AUTO_PAUSED
298         RunningControlButton(
299             icon = Icons.Filled.PlayArrow,
300             onClick = viewModel::resume,
301             size = 84.dp,
302             containerColor = MaterialTheme.colorScheme.primary,
303             contentColor = MaterialTheme.colorScheme.onPrimary,
304         )
305     }
306 }
307
308 if (viewModel.finishError != null) {
309     Spacer(modifier = Modifier.height(12.dp))
310     Text(
311         text = viewModel.finishError!!,
312         style = MaterialTheme.typography.bodySmall,
313         color = MaterialTheme.colorScheme.error,
314         textAlign = TextAlign.Center,
315         modifier = Modifier
316             .fillMaxWidth()
317             .padding(horizontal = 32.dp),
318     )
319 }
320
321 Spacer(modifier = Modifier.height(44.dp))
322 }
323
324 AnimatedVisibility(
325     visible = !countdownDone,
326     enter = EnterTransition.None,
327     exit = fadeOut(tween(350)),
328 ) {
329     RunningCountdownOverlay(onFinished = { countdownDone = true })
330 }
331 } // Box
332 }
333
334 @Composable
335 private fun FreeRunDataPanel(
336     timerText: String,
337     distanceText: String,
338     paceText: String,
339     speedText: String,
340     cadenceText: String,
341     isRunning: Boolean,

```

```

342     modifier: Modifier = Modifier,
343 ) {
344     Surface(
345         modifier = modifier,
346         shape = RoundedCornerShape(28.dp),
347         color = MaterialTheme.colorScheme.surface,
348         border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline),
349     ) {
350         Column(
351             modifier = Modifier
352                 .fillMaxSize()
353                 .padding(horizontal = 24.dp, vertical = 24.dp),
354             horizontalAlignment = Alignment.CenterHorizontally,
355             verticalArrangement = Arrangement.Center,
356         ) {
357             Text(
358                 text = if (isRunning) "자유 러닝" else "일시정지 중",
359                 style = MaterialTheme.typography.labelLarge,
360                 color = MaterialTheme.colorScheme.onSurfaceVariant,
361             )
362             Spacer(modifier = Modifier.height(12.dp))
363             AnimatedContent(
364                 targetState = distanceText,
365                 transitionSpec = {
366                     (slideInVertically(tween(250)) { it } + fadeIn(tween(250))) togetherWith
367                     (slideOutVertically(tween(200)) { -it } + fadeOut(tween(200)))
368                 },
369                 label = "distanceSlot",
370             ) { text ->
371                 Text(
372                     text = text,
373                     fontSize = 84.sp,
374                     lineHeight = 88.sp,
375                     fontWeight = FontWeight.ExtraBold,
376                     color = MaterialTheme.colorScheme.onSurface,
377                     textAlign = TextAlign.Center,
378                 )
379             }
380             Text(
381                 text = "km",
382                 style = MaterialTheme.typography.titleMedium,
383                 color = MaterialTheme.colorScheme.primary,
384                 fontWeight = FontWeight.Bold,
385             )
386             Spacer(modifier = Modifier.height(32.dp))
387             Row(
388                 modifier = Modifier.fillMaxWidth(),
389                 horizontalArrangement = Arrangement.spacedBy(12.dp),
390             ) {
391                 CompactMetricBlock(
392                     label = "TIME",
393                     value = timerText,
394                     unit = "",
395                     modifier = Modifier.weight(1f),
396                 )
397                 CompactMetricBlock(
398                     label = "AVG PACE",
399                     value = paceText,
400                     unit = "/km",
401                     modifier = Modifier.weight(1f),
402                 )
403             }
404             Spacer(modifier = Modifier.height(12.dp))
405             Row(
406                 modifier = Modifier.fillMaxWidth(),
407                 horizontalArrangement = Arrangement.spacedBy(12.dp),
408             ) {
409                 CompactMetricBlock(
410                     label = "CADENCE",
411                     value = cadenceText,
412                     unit = "spm",
413                     modifier = Modifier.weight(1f),
414                 )
415                 CompactMetricBlock(
416                     label = "SPEED",
417                     value = speedText,
418                     unit = "km/h",
419                     modifier = Modifier.weight(1f),
420                 )
421             }
422         }
423     }
424 }
425
426 @Composable
427 private fun CompactMetricBlock(

```



```

428     label: String,
429     value: String,
430     unit: String,
431     modifier: Modifier = Modifier,
432 ) {
433     Column(
434         modifier = modifier
435         .clip(RoundedCornerShape(18.dp))
436         .background(MaterialTheme.colorScheme.surfaceVariant)
437         .padding(vertical = 16.dp, horizontal = 10.dp),
438         horizontalAlignment = Alignment.CenterHorizontally,
439     ) {
440         Text(
441             text = label,
442             style = MaterialTheme.typography.labelMedium,
443             color = MaterialTheme.colorScheme.onSurfaceVariant,
444         )
445         Spacer(modifier = Modifier.height(6.dp))
446         Text(
447             text = value,
448             style = MaterialTheme.typography.headlineSmall,
449             fontWeight = FontWeight.ExtraBold,
450             color = MaterialTheme.colorScheme.onSurface,
451             textAlign = TextAlign.Center,
452         )
453         Text(
454             text = unit,
455             style = MaterialTheme.typography.labelSmall,
456             color = MaterialTheme.colorScheme.onSurfaceVariant,
457         )
458     }
459 }
460
461 @Composable
462 private fun RecordingPill(state: RunningState) {
463     val infiniteTransition = rememberInfiniteTransition(label = "dot")
464     val dotAlpha by infiniteTransition.animateFloat(
465         initialValue = 1f,
466         targetValue = 0.15f,
467         animationSpec = InfiniteRepeatableSpec(tween(700), repeatMode = RepeatMode.Reverse),
468         label = "dot_alpha",
469     )
470
471     val pillColor: Color = when (state) {
472         RunningState.RUNNING -> MaterialTheme.colorScheme.primary
473         RunningState.PAUSED -> MaterialTheme.colorScheme.onSurfaceVariant
474         RunningState.AUTO_PAUSED -> WarningYellow
475     }
476
477     val label: String = when (state) {
478         RunningState.RUNNING -> "RECORDING"
479         RunningState.PAUSED -> "PAUSED"
480         RunningState.AUTO_PAUSED -> "AUTO PAUSED"
481     }
482
483     Surface(
484         shape = CircleShape,
485         color = MaterialTheme.colorScheme.surface,
486         border = BorderStroke(1.dp, pillColor.copy(alpha = 0.5f)),
487     ) {
488         Row(
489             modifier = Modifier.padding(horizontal = 12.dp, vertical = 6.dp),
490             verticalAlignment = Alignment.CenterVertically,
491         ) {
492             Canvas(modifier = Modifier.size(6.dp)) {
493                 drawCircle(
494                     color = pillColor.copy(alpha = if (state == RunningState.RUNNING) dotAlpha else 0.5f),
495                 )
496             }
497             Spacer(modifier = Modifier.width(6.dp))
498             Text(
499                 text = label,
500                 style = MaterialTheme.typography.labelLarge,
501                 color = pillColor,
502             )
503         }
504     }
505 }

```

```

1  package com.runway.android.ui.running
2
3  import android.content.Context
4  import android.content.Intent
5  import android.os.Build
6  import android.os.VibrationEffect
7  import android.os.Vibrator
8  import android.os.VibratorManager
9  import androidx.compose.runtime.getValue
10 import androidx.compose.runtime.mutableStateOf
11 import androidx.compose.runtime.setValue
12 import androidx.core.content.ContextCompat
13 import androidx.lifecycle.ViewModel
14 import androidx.lifecycle.viewModelScope
15 import com.runway.android.core.location.GpsStatus
16 import com.runway.android.core.result.NetworkResult
17 import com.runway.android.core.tracking.PauseReason
18 import com.runway.android.core.tracking.PendingPointQueue
19 import com.runway.android.core.tracking.RunTrackingAction
20 import com.runway.android.core.tracking.RunTrackingManager
21 import com.runway.android.core.tracking.RunTrackingMode
22 import com.runway.android.core.tracking.RunTrackingService
23 import com.runway.android.core.tracking.TrackingSessionSnapshot
24 import com.runway.android.core.tracking.TrackingSessionStore
25 import com.runway.android.core.tracking.toRunPointRequest
26 import com.runway.android.data.running.model.FinishRunRequest
27 import com.runway.android.data.running.model.SavePointsRequest
28 import com.runway.android.data.running.model.StartRunRequest
29 import com.runway.android.domain.running.RunningRepository
30 import com.runway.android.core.util.formatDuration
31 import com.runway.android.core.voice.RunningVoiceGuide
32 import dagger.hilt.android.lifecycle.HiltViewModel
33 import dagger.hilt.android.qualifiers.ApplicationContext
34 import kotlinx.coroutines.CoroutineScope
35 import kotlinx.coroutines.Job
36 import kotlinx.coroutines.delay
37 import kotlinx.coroutines.flow.MutableSharedFlow
38 import kotlinx.coroutines.flow.asSharedFlow
39 import kotlinx.coroutines.launch
40 import java.time.Instant
41 import javax.inject.Inject
42 import javax.inject.Named
43
44 data class RunResult(
45     val runId: String?,
46     val elapsedSeconds: Int,
47     val distanceKm: Float,
48 )
49
50 enum class RunningState { RUNNING, PAUSED, AUTO_PAUSED }
51
52 @HiltViewModel
53 class RunningTrackingViewModel @Inject constructor(
54     @ApplicationContext private val context: Context,
55     private val runningRepository: RunningRepository,
56     private val manager: RunTrackingManager,
57     private val pendingPointQueue: PendingPointQueue,
58     private val sessionStore: TrackingSessionStore,
59     private val voiceGuide: RunningVoiceGuide,
60     @Named("appScope") private val appScope: CoroutineScope,
61 ) : ViewModel() {
62
63     var gpsStatus by mutableStateOf(GpsStatus.PERMISSION_REQUIRED)
64     private set
65     var runningState by mutableStateOf(RunningState.RUNNING)
66     private set
67     var elapsedSeconds by mutableStateOf(0)
68     private set
69     var distanceKm by mutableStateOf(0.0)
70     private set
71     var isConnecting by mutableStateOf(true)
72     private set
73     var isFinishing by mutableStateOf(false)
74     private set
75     var finishError by mutableStateOf<String?>(null)
76     private set
77     var milestoneMessage by mutableStateOf<String?>(null)
78     private set
79
80     private var lastSpeedMps by mutableStateOf<Float?>(null)
81     private var cadenceSpm by mutableStateOf<Int?>(null)
82
83     val timerText: String

```

```

84     get() = formatDuration(elapsedSeconds)
85
86     val distanceText: String
87     get() = "%.2f".format(distanceKm)
88
89     val paceText: String
90     get() {
91         if (distanceKm < 0.001) return "--'--\\""
92         val secsPerKm = (elapsedSeconds / distanceKm).toInt()
93         return "%d'%02d\".format(secsPerKm / 60, secsPerKm % 60)
94     }
95
96     val speedText: String
97     get() {
98         val gpsSpeed = lastSpeedMps
99         return if (gpsSpeed != null && gpsSpeed >= 0.1f) {
100             "%.1f".format(gpsSpeed * 3.6f)
101         } else if (elapsedSeconds < 1) {
102             "0.0"
103         } else {
104             "%.1f".format(distanceKm / (elapsedSeconds / 3600.0))
105         }
106     }
107
108     val cadenceText: String
109     get() = cadenceSpm?.takeIf { it > 0 }?.toString() ?: "--"
110
111     private val _navigateToResult = MutableSharedFlow<RunResult>()
112     val navigateToResult = _navigateToResult.asSharedFlow()
113
114     private var runId: String? = null
115     private var isFinished = false
116     private var serviceStarted = false
117     private var prevAutoPaused = false
118
119     private var stateObserveJob: Job? = null
120     private var batchJob: Job? = null
121     private var milestoneCollectJob: Job? = null
122     private var milestoneDisplayJob: Job? = null
123
124     fun startTracking() {
125         if (serviceStarted) return
126         serviceStarted = true
127         gpsStatus = GpsStatus.WAITING_FOR_FIX
128         voiceGuide.start()
129         startRun()
130
131         ContextCompat.startForegroundService(
132             context,
133             Intent(context, RunTrackingService::class.java).apply {
134                 action = RunTrackingAction.ACTION_START
135                 putExtra(RunTrackingAction.EXTRA_MODE, RunTrackingMode.FREE_RUN.name)
136             }
137         )
138
139         stateObserveJob = viewModelScope.launch {
140             manager.state.collect { state ->
141                 if (state.hasFirstFix && gpsStatus != GpsStatus.ACTIVE) {
142                     gpsStatus = GpsStatus.ACTIVE
143                 }
144                 elapsedSeconds = state.elapsedSeconds
145                 distanceKm = state.distanceMeters / 1000.0
146                 lastSpeedMps = state.currentSpeedMps
147                 cadenceSpm = state.cadenceSpm
148
149                 // 자동 일시정지 ⇄ 재개 전환 시 음성 안내
150                 if (state.isAutoPaused != prevAutoPaused) {
151                     if (state.isAutoPaused) voiceGuide.autoPaused()
152                     else voiceGuide.autoResumed()
153                     prevAutoPaused = state.isAutoPaused
154                 }
155
156                 runningState = when {
157                     state.isAutoPaused -> RunningState.AUTO_PAUSED
158                     state.isPaused -> RunningState.PAUSED
159                     else -> RunningState.RUNNING
160                 }
161             }
162         }
163
164         milestoneCollectJob = viewModelScope.launch {
165             manager.milestoneFlow.collect { km -> triggerMilestone(km) }
166         }
167     }
168
169     private fun triggerMilestone(km: Int) {

```

```

170     vibrate()
171     voiceGuide.kilometer(km, elapsedSeconds)
172     milestoneDisplayJob?.cancel()
173     milestoneMessage = "${km}km 완료 · 현재 페이스 $spaceText /km"
174     milestoneDisplayJob = viewModelScope.launch {
175         delay(4_000)
176         milestoneMessage = null
177     }
178 }
179
180 @Suppress("DEPRECATION")
181 private fun vibrate() {
182     val vibrator = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
183         (context.getSystemService(Context.VIBRATOR_MANAGER_SERVICE) as VibratorManager).defaultVibrator
184     } else {
185         context.getSystemService(Context.VIBRATOR_SERVICE) as Vibrator
186     }
187     vibrator.vibrate(
188         VibrationEffect.createWaveform(longArrayOf(0, 200, 100, 200), -1)
189     )
190 }
191
192 private fun startRun() {
193     viewModelScope.launch {
194         val result = runningRepository.startRun(StartRunRequest(startedAt = Instant.now().toString()))
195         when (result) {
196             is NetworkResult.Success -> {
197                 val rid = result.data.runId
198                 runId = rid
199                 sessionStore.saveSnapshot(
200                     TrackingSessionSnapshot(
201                         runningRecordId = rid,
202                         courseAttemptId = null,
203                         elapsedSeconds = 0,
204                         distanceMeters = 0.0,
205                     )
206                 )
207                 isConnecting = false
208                 startBatchSaving()
209             }
210             else -> isConnecting = false
211         }
212     }
213 }
214
215 private fun startBatchSaving() {
216     batchJob = viewModelScope.launch {
217         while (true) {
218             delay(5_000L)
219             flushPendingPoints()
220             updateSnapshot()
221         }
222     }
223 }
224
225 private suspend fun flushPendingPoints() {
226     val rid = runId ?: return
227     val inMemory = manager.consumePoints()
228     pendingPointQueue.add(rid, inMemory)
229
230     val batch = pendingPointQueue.dequeue(rid, 50)
231     if (batch.isEmpty()) return
232
233     val result = runningRepository.savePoints(rid, SavePointsRequest(batch.map { it.toRunPointRequest() } ))
234     if (result is NetworkResult.Success) {
235         pendingPointQueue.deleteByIds(batch.map { it.id })
236     }
237 }
238
239 private suspend fun updateSnapshot() {
240     val rid = runId ?: return
241     sessionStore.saveSnapshot(
242         TrackingSessionSnapshot(
243             runningRecordId = rid,
244             courseAttemptId = null,
245             elapsedSeconds = elapsedSeconds,
246             distanceMeters = distanceKm * 1000.0,
247         )
248     )
249 }
250
251 fun pause() {
252     manager.pause()
253     val rid = runId ?: return
254     viewModelScope.launch {
255         if (runningRepository.pauseRun(rid) !is NetworkResult.Success) {

```

```

256         manager.resume()
257     }
258 }
259 }
260
261 fun resume() {
262     val wasAutoPaused = manager.state.value.isAutoPaused
263     manager.resume()
264     if (!wasAutoPaused) {
265         val rid = runId ?: return
266         viewModelScope.launch {
267             if (runningRepository.resumeRun(rid) !is NetworkResult.Success) {
268                 manager.pause()
269             }
270         }
271     }
272 }
273
274 fun finish() {
275     if (isFinishing) return
276     isFinishing = true
277     finishError = null
278
279     val currentRunId = runId
280     val seconds = elapsedSeconds
281     val distance = distanceKm
282
283     viewModelScope.launch {
284         if (currentRunId != null) {
285             pendingPointQueue.add(currentRunId, manager.consumePoints())
286             // 큐가 빌 때까지 업로드. 연속 실패 10회 시 중단 (서버/네트워크 문제).
287             var failures = 0
288             while (failures < 10) {
289                 val batch = pendingPointQueue.dequeue(currentRunId, 50)
290                 if (batch.isEmpty()) break
291                 val r = runningRepository.savePoints(
292                     currentRunId,
293                     SavePointsRequest(batch.map { it.toRunPointRequest() })),
294                 )
295                 if (r is NetworkResult.Success) {
296                     pendingPointQueue.deleteByIds(batch.map { it.id })
297                     failures = 0
298                 } else {
299                     failures++
300                 }
301             }
302
303             when (runningRepository.finishRun(
304                 currentRunId,
305                 FinishRunRequest(
306                     endedAt = Instant.now().toString(),
307                     distanceMeters = distance * 1000.0,
308                     durationSeconds = seconds,
309                     avgPaceSecondsPerKm = if (distance > 0.001) (seconds / distance).toInt() else 0,
310                     caloriesBurned = (distance * 72).toInt(),
311                 ),
312             )) {
313                 is NetworkResult.ApiError,
314                 is NetworkResult.NetworkError -> {
315                     isFinishing = false
316                     finishError = "런 완료에 실패했습니다. 다시 시도해 주세요."
317                     return@launch
318                 }
319                 is NetworkResult.Success -> {
320                     voiceGuide.finish()
321                     sessionStore.clearSnapshot()
322                     pendingPointQueue.deleteByRunningRecordId(currentRunId)
323                 }
324             }
325         }
326
327         stopServiceAndJobs()
328         isFinished = true
329         _navigateToResult.emit(
330             RunResult(
331                 runId = currentRunId,
332                 elapsedSeconds = seconds,
333                 distanceKm = distance.toFloat(),
334             )
335         )
336     }
337 }
338
339 private fun stopServiceAndJobs() {
340     stateObserveJob?.cancel()
341     batchJob?.cancel()

```

```
342     milestoneCollectJob?.cancel()
343     milestoneDisplayJob?.cancel()
344     manager.stop()
345     context.startService(
346         Intent(context, RunTrackingService::class.java).apply {
347             action = RunTrackingAction.ACTION_STOP
348         }
349     )
350 }
351
352 override fun onCleared() {
353     super.onCleared()
354     if (!isFinished) {
355         stopServiceAndJobs()
356         val rid = runId
357         if (rid != null) {
358             appScope.launch {
359                 runningRepository.abandonRun(rid)
360                 sessionStore.clearSnapshot()
361                 pendingPointQueue.deleteByRunningRecordId(rid)
362             }
363         }
364     }
365 }
366 }
```

```

1  package com.runway.android.ui.running.history
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.layout.Arrangement
5  import androidx.compose.foundation.layout.Box
6  import androidx.compose.foundation.layout.Column
7  import androidx.compose.foundation.layout.PaddingValues
8  import androidx.compose.foundation.layout.Row
9  import androidx.compose.foundation.layout.Spacer
10 import androidx.compose.foundation.layout.fillMaxSize
11 import androidx.compose.foundation.layout.fillMaxWidth
12 import androidx.compose.foundation.layout.height
13 import androidx.compose.foundation.layout.padding
14 import androidx.compose.foundation.layout.size
15 import androidx.compose.foundation.layout.statusBarsPadding
16 import androidx.compose.foundation.layout.width
17 import androidx.compose.foundation.lazy.LazyColumn
18 import androidx.compose.foundation.lazy.items
19 import androidx.compose.material.icons.Icons
20 import androidx.compose.material.icons.automirrored.filled.ArrowBack
21 import androidx.compose.material.icons.filled.Delete
22 import androidx.compose.material3.Button
23 import androidx.compose.material3.CircularProgressIndicator
24 import androidx.compose.material3.ExperimentalMaterial3Api
25 import androidx.compose.material3.HorizontalDivider
26 import androidx.compose.material3.Icon
27 import androidx.compose.material3.IconButton
28 import androidx.compose.material3.MaterialTheme
29 import androidx.compose.material3.Surface
30 import androidx.compose.material3.SwipeToDismissBox
31 import androidx.compose.material3.SwipeToDismissBoxValue
32 import androidx.compose.material3.Text
33 import androidx.compose.material3.rememberSwipeToDismissBoxState
34 import androidx.compose.runtime.Composable
35 import androidx.compose.runtime.LaunchedEffect
36 import androidx.compose.ui.Alignment
37 import androidx.compose.ui.Modifier
38 import androidx.compose.ui.graphics.Color
39 import androidx.compose.ui.text.font.FontWeight
40 import androidx.compose.ui.unit.dp
41 import androidx.hilt.navigation.compose.hiltViewModel
42 import com.runway.android.core.util.formatDistance
43 import com.runway.android.core.util.formatDuration
44 import com.runway.android.ui.components.RunHistoryCard
45
46 @OptIn(ExperimentalMaterial3Api::class)
47 @Composable
48 fun MyRunsScreen(
49     onBack: () -> Unit,
50     onNavigateToDetail: (String) -> Unit,
51     viewModel: MyRunsViewModel = hiltViewModel(),
52 ) {
53     Column(
54         modifier = Modifier
55             .fillMaxSize()
56             .background(MaterialTheme.colorScheme.background)
57             .statusBarsPadding(),
58     ) {
59         // 상단 바
60         Row(
61             modifier = Modifier
62                 .fillMaxWidth()
63                 .padding(horizontal = 8.dp, vertical = 4.dp),
64             verticalAlignment = Alignment.CenterVertically,
65         ) {
66             IconButton(onClick = onBack) {
67                 Icon(
68                     imageVector = Icons.AutoMirrored.Filled.ArrowBack,
69                     contentDescription = "뒤로 가기",
70                     tint = MaterialTheme.colorScheme.onBackground,
71                 )
72             }
73             Spacer(modifier = Modifier.width(4.dp))
74             Text(
75                 text = "내 러닝 기록",
76                 style = MaterialTheme.typography.headlineMedium,
77                 color = MaterialTheme.colorScheme.onBackground,
78             )
79         }
80
81         when {
82             viewModel.isLoading -> {
83                 Box(

```

```

84         modifier = Modifier.fillMaxSize(),
85         contentAlignment = Alignment.Center,
86     ) {
87         CircularProgressIndicator(
88             modifier = Modifier.size(40.dp),
89             color = MaterialTheme.colorScheme.primary,
90         )
91     }
92 }
93 viewModel.hasError -> {
94     Box(
95         modifier = Modifier.fillMaxSize(),
96         contentAlignment = Alignment.Center,
97     ) {
98         Column(horizontalAlignment = Alignment.CenterHorizontally) {
99             Text(
100                 text = "기록을 불러오지 못했습니다.",
101                 style = MaterialTheme.typography.bodyLarge,
102                 color = MaterialTheme.colorScheme.onSurfaceVariant,
103             )
104             Spacer(modifier = Modifier.height(16.dp))
105             Button(onClick = viewModel::retry) {
106                 Text("다시 시도")
107             }
108         }
109     }
110 }
111 else -> {
112     androidx.compose.material3.pulltorefresh.PullToRefreshBox(
113         isRefreshing = viewModel.isRefreshing,
114         onRefresh = viewModel::refresh,
115         modifier = Modifier.fillMaxSize(),
116     ) {
117         LazyColumn(
118             modifier = Modifier.fillMaxSize(),
119             contentPadding = PaddingValues(bottom = 24.dp),
120         ) {
121             // 캘린더
122             item {
123                 RunCalendar(
124                     selectedMonth = viewModel.selectedMonth,
125                     selectedDate = viewModel.selectedDate,
126                     datesWithRuns = viewModel.datesWithRuns,
127                     onPreviousMonth = viewModel::previousMonth,
128                     onNextMonth = viewModel::nextMonth,
129                     onDateSelected = viewModel::selectDate,
130                 )
131             }
132             // 월 요약
133             item {
134                 MonthlySummaryBar(summary = viewModel.monthlySummary)
135                 HorizontalDivider(color = MaterialTheme.colorScheme.outlineVariant)
136             }
137             // 런 목록
138             if (viewModel.runs.isEmpty()) {
139                 item {
140                     Box(
141                         modifier = Modifier
142                             .fillMaxWidth()
143                             .padding(vertical = 40.dp),
144                         contentAlignment = Alignment.Center,
145                     ) {
146                         Text(
147                             text = if (viewModel.selectedDate != null) "선택한 날짜에 러닝 기록이 없습니다."
148                             else "이 달의 러닝 기록이 없습니다.",
149                             style = MaterialTheme.typography.bodyLarge,
150                             color = MaterialTheme.colorScheme.onSurfaceVariant,
151                         )
152                     }
153                 }
154             } else {
155                 items(viewModel.runs, key = { it.runId }) { item ->
156                     SwipeToDeleteCard(
157                         onDelete = { viewModel.deleteRun(item.runId) },
158                     ) {
159                         Box(modifier = Modifier.padding(horizontal = 20.dp, vertical = 4.dp)) {
160                             RunHistoryCard(
161                                 item = item,
162                                 onClick = { onNavigateToDetail(item.runId) },
163                             )
164                         }
165                     }
166                 }
167             }
168         }
169     }

```



```

170         }
171     } // PullToRefreshBox
172 }
173 }
174 }
175 }
176
177 @OptIn(ExperimentalMaterial3Api::class)
178 @Composable
179 private fun SwipeToDeleteCard(
180     onDelete: () -> Unit,
181     content: @Composable () -> Unit,
182 ) {
183     val state = rememberSwipeToDismissBoxState(
184         confirmValueChange = { value ->
185             if (value == SwipeToDismissBoxValue.EndToStart) {
186                 onDelete()
187                 true
188             } else false
189         },
190         positionalThreshold = { it * 0.4f },
191     )
192     LaunchedEffect(state.currentValue) {
193         if (state.currentValue == SwipeToDismissBoxValue.Settled) return@LaunchedEffect
194     }
195     SwipeToDismissBox(
196         state = state,
197         backgroundColor = {
198             if (state.dismissDirection == SwipeToDismissBoxValue.EndToStart) {
199                 Box(
200                     modifier = Modifier
201                         .fillMaxSize()
202                         .padding(horizontal = 20.dp, vertical = 4.dp)
203                         .background(
204                             MaterialTheme.colorScheme.errorContainer,
205                             MaterialTheme.shapes.extraLarge,
206                         ),
207                     contentAlignment = Alignment.CenterEnd,
208                 ) {
209                     Icon(
210                         imageVector = Icons.Filled.Delete,
211                         contentDescription = "삭제",
212                         tint = MaterialTheme.colorScheme.onErrorContainer,
213                         modifier = Modifier.padding(end = 24.dp),
214                     )
215                 }
216             }
217         },
218         enableDismissFromStartToEnd = false,
219         enableDismissFromEndToStart = true,
220     ) {
221         content()
222     }
223 }
224
225 @Composable
226 private fun MonthlySummaryBar(
227     summary: MonthlySummary,
228     modifier: Modifier = Modifier,
229 ) {
230     Surface(
231         modifier = modifier.fillMaxWidth(),
232         color = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f),
233     ) {
234         Row(
235             modifier = Modifier
236                 .fillMaxWidth()
237                 .padding(horizontal = 20.dp, vertical = 12.dp),
238             horizontalArrangement = Arrangement.SpaceEvenly,
239         ) {
240             SummaryStatItem(
241                 label = "러닝",
242                 value = "${summary.runCount}회",
243             )
244             SummaryStatItem(
245                 label = "총 거리",
246                 value = formatDistance(summary.totalDistanceMeters),
247             )
248             SummaryStatItem(
249                 label = "총 시간",
250                 value = formatDuration(summary.totalDurationSeconds),
251             )
252         }
253     }
254 }
255

```

```
256 @Composable
257 private fun SummaryStatItem(
258     label: String,
259     value: String,
260 ) {
261     Column(horizontalAlignment = Alignment.CenterHorizontally) {
262         Text(
263             text = value,
264             style = MaterialTheme.typography.titleSmall,
265             fontWeight = FontWeight.Bold,
266             color = MaterialTheme.colorScheme.onSurface,
267         )
268         Text(
269             text = label,
270             style = MaterialTheme.typography.labelSmall,
271             color = MaterialTheme.colorScheme.onSurfaceVariant,
272         )
273     }
274 }
```

```

1 package com.runway.android.ui.running.history
2
3 import androidx.compose.runtime.getValue
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.compose.runtime.setValue
6 import androidx.lifecycle.ViewModel
7 import androidx.lifecycle.viewModelScope
8 import com.runway.android.core.result.NetworkResult
9 import com.runway.android.core.util.formatDistance
10 import com.runway.android.core.util.formatDuration
11 import com.runway.android.core.util.formatPace
12 import com.runway.android.core.util.formatRunDateShort
13 import com.runway.android.data.running.model.RunSummaryResponse
14 import com.runway.android.domain.running.RunningRepository
15 import com.runway.android.ui.components.RunHistoryItem
16 import dagger.hilt.android.lifecycle.HiltViewModel
17 import kotlinx.coroutines.launch
18 import java.time.Instant
19 import java.time.LocalDate
20 import java.time.YearMonth
21 import java.time.ZoneId
22 import javax.inject.Inject
23
24 @HiltViewModel
25 class MyRunsViewModel @Inject constructor(
26     private val runningRepository: RunningRepository,
27 ) : ViewModel() {
28
29     var allRuns by mutableStateOf<List<RunHistoryItem>>(emptyList())
30     private set
31     var isLoading by mutableStateOf(true)
32     private set
33     var isRefreshing by mutableStateOf(false)
34     private set
35     var hasError by mutableStateOf(false)
36     private set
37     var totalCount by mutableStateOf(0L)
38     private set
39
40     var selectedMonth by mutableStateOf(YearMonth.now())
41     private set
42     var selectedDate by mutableStateOf<LocalDate?>(null)
43     private set
44
45     /** 현재 월에서 런이 있는 날짜 집합 */
46     val datesWithRuns: Set<LocalDate>
47     get() = allRuns
48         .filter { it.localDate?.let { d -> YearMonth.from(d) == selectedMonth } ?: false }
49         .mapNotNull { it.localDate }
50         .toSet()
51
52     /** 화면에 표시할 런 목록 - 날짜 선택 시 해당 날짜만, 아니면 현재 월 전체 */
53     val runs: List<RunHistoryItem>
54     get() {
55         val date = selectedDate
56         return if (date != null) {
57             allRuns.filter { it.localDate == date }
58         } else {
59             allRuns.filter { it.localDate?.let { d -> YearMonth.from(d) == selectedMonth } ?: false }
60         }
61     }
62
63     /** 현재 월 요약: 완주 횟수, 총 거리(m), 총 시간(초) */
64     val monthlySummary: MonthlySummary
65     get() {
66         val monthRuns = allRuns.filter {
67             it.localDate?.let { d -> YearMonth.from(d) == selectedMonth } ?: false
68         }
69         val count = monthRuns.size
70         val totalDistanceM = monthRuns.sumOf { it.rawDistanceMeters ?: 0.0 }
71         val totalDurationSec = monthRuns.sumOf { it.rawDurationSeconds ?: 0 }
72         return MonthlySummary(count, totalDistanceM, totalDurationSec)
73     }
74
75     init {
76         loadRuns()
77     }
78
79     fun retry() {
80         isLoading = true
81         hasError = false
82         allRuns = emptyList()
83         viewModelScope.launch { doLoad() }

```

```

84     }
85
86     fun refresh() {
87         viewModelScope.launch {
88             isRefreshing = true
89             hasError = false
90             doLoad()
91             isRefreshing = false
92         }
93     }
94
95     fun deleteRun(runId: String) {
96         viewModelScope.launch {
97             runningRepository.deleteRun(runId)
98             allRuns = allRuns.filterNot { it.runId == runId }
99         }
100     }
101
102     fun removeRunLocally(runId: String) {
103         allRuns = allRuns.filterNot { it.runId == runId }
104     }
105
106     fun previousMonth() {
107         selectedMonth = selectedMonth.minusMonths(1)
108         selectedDate = null
109     }
110
111     fun nextMonth() {
112         val next = selectedMonth.plusMonths(1)
113         if (!next.isAfter(YearMonth.now())) {
114             selectedMonth = next
115             selectedDate = null
116         }
117     }
118
119     fun selectDate(date: LocalDate) {
120         selectedDate = if (selectedDate == date) null else date
121     }
122
123     private fun loadRuns() {
124         viewModelScope.launch { doLoad() }
125     }
126
127     private suspend fun doLoad() {
128         val result = runningRepository.getMyRuns(page = 0, size = 200)
129         when (result) {
130             is NetworkResult.Success -> {
131                 totalCount = result.data.totalElements
132                 allRuns = result.data.content.map { it.toHistoryItem() }
133             }
134             else -> hasError = true
135         }
136         isLoading = false
137     }
138
139     private fun RunSummaryResponse.toHistoryItem(): RunHistoryItem {
140         val date = startedAt.toLocalDate()
141         return RunHistoryItem(
142             runId = runId,
143             dateLabel = formatRunDateShort(startedAt),
144             distanceFormatted = formatDistance(distanceMeters),
145             duration = formatDuration(durationSeconds),
146             pace = formatPace(avgPaceSecondsPerKm),
147             status = status,
148             calories = caloriesBurned,
149             localDate = date,
150             rawDistanceMeters = distanceMeters,
151             rawDurationSeconds = durationSeconds,
152         )
153     }
154 }
155
156 data class MonthlySummary(
157     val runCount: Int,
158     val totalDistanceMeters: Double,
159     val totalDurationSeconds: Int,
160 )
161
162 private fun String?.toLocalDate(): LocalDate? {
163     if (this == null) return null
164     return runCatching {
165         Instant.parse(this).atZone(ZoneId.systemDefault()).toLocalDate()
166     }.getOrNull()
167 }

```

```

1  package com.runway.android.ui.running.history
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.clickable
5  import androidx.compose.foundation.layout.Arrangement
6  import androidx.compose.foundation.layout.Box
7  import androidx.compose.foundation.layout.Column
8  import androidx.compose.foundation.layout.Row
9  import androidx.compose.foundation.layout.Spacer
10 import androidx.compose.foundation.layout.aspectRatio
11 import androidx.compose.foundation.layout.fillMaxWidth
12 import androidx.compose.foundation.layout.height
13 import androidx.compose.foundation.layout.padding
14 import androidx.compose.foundation.layout.size
15 import androidx.compose.foundation.shape.CircleShape
16 import androidx.compose.material.icons.Icons
17 import androidx.compose.material.icons.automirrored.filled.KeyboardArrowLeft
18 import androidx.compose.material.icons.automirrored.filled.KeyboardArrowRight
19 import androidx.compose.material3.Icon
20 import androidx.compose.material3.IconButton
21 import androidx.compose.material3.MaterialTheme
22 import androidx.compose.material3.Text
23 import androidx.compose.runtime.Composable
24 import androidx.compose.ui.Alignment
25 import androidx.compose.ui.Modifier
26 import androidx.compose.ui.draw.clip
27 import androidx.compose.ui.graphics.Color
28 import androidx.compose.ui.text.font.FontWeight
29 import androidx.compose.ui.text.style.TextAlign
30 import androidx.compose.ui.unit.dp
31 import androidx.compose.ui.unit.sp
32 import java.time.DayOfWeek
33 import java.time.LocalDate
34 import java.time.YearMonth
35 import java.time.format.TextStyle
36 import java.util.Locale
37
38 @Composable
39 fun RunCalendar(
40     selectedMonth: YearMonth,
41     selectedDate: LocalDate?,
42     datesWithRuns: Set<LocalDate>,
43     onPreviousMonth: () -> Unit,
44     onNextMonth: () -> Unit,
45     onDateSelected: (LocalDate) -> Unit,
46     modifier: Modifier = Modifier,
47 ) {
48     val today = LocalDate.now()
49     val canGoNext = selectedMonth.plusMonths(1) <= YearMonth.from(today)
50
51     Column(
52         modifier = modifier
53         .fillMaxWidth()
54         .background(MaterialTheme.colorScheme.surface)
55         .padding(horizontal = 16.dp, vertical = 8.dp),
56     ) {
57         // 월 헤더 + 이전/다음 버튼
58         Row(
59             modifier = Modifier.fillMaxWidth(),
60             verticalAlignment = Alignment.CenterVertically,
61             horizontalArrangement = Arrangement.SpaceBetween,
62         ) {
63             IconButton(onClick = onPreviousMonth) {
64                 Icon(
65                     imageVector = Icons.AutoMirrored.Filled.KeyboardArrowLeft,
66                     contentDescription = "이전 달",
67                     tint = MaterialTheme.colorScheme.onSurface,
68                 )
69             }
70             Text(
71                 text = "${selectedMonth.year}년 ${selectedMonth.monthValue}월",
72                 style = MaterialTheme.typography.titleMedium,
73                 fontWeight = FontWeight.Bold,
74                 color = MaterialTheme.colorScheme.onSurface,
75             )
76             IconButton(
77                 onClick = onNextMonth,
78                 enabled = canGoNext,
79             ) {
80                 Icon(
81                     imageVector = Icons.AutoMirrored.Filled.KeyboardArrowRight,
82                     contentDescription = "다음 달",
83                     tint = if (canGoNext) MaterialTheme.colorScheme.onSurface

```

```

84         else MaterialTheme.colorScheme.onSurface.copy(alpha = 0.3f),
85     )
86 }
87 }
88
89 Spacer(modifier = Modifier.height(4.dp))
90
91 // 요일 헤더
92 val weekDays = listOf("일", "월", "화", "수", "목", "금", "토")
93 Row(modifier = Modifier.fillMaxWidth()) {
94     weekDays.forEach { day ->
95         Text(
96             text = day,
97             modifier = Modifier.weight(1f),
98             textAlign = TextAlign.Center,
99             style = MaterialTheme.typography.labelSmall,
100             color = MaterialTheme.colorScheme.onSurfaceVariant,
101         )
102     }
103 }
104
105 Spacer(modifier = Modifier.height(4.dp))
106
107 // 날짜 그리드
108 val firstDay = selectedMonth.atDay(1)
109 // 일요일(7→0) 기준 offset
110 val startOffset = (firstDay.dayOfWeek.value % 7)
111 val daysInMonth = selectedMonth.lengthOfMonth()
112 val totalCells = startOffset + daysInMonth
113 val rows = (totalCells + 6) / 7
114
115 for (row in 0 until rows) {
116     Row(modifier = Modifier.fillMaxWidth()) {
117         for (col in 0 until 7) {
118             val cell = row * 7 + col
119             val dayNum = cell - startOffset + 1
120             if (dayNum < 1 || dayNum > daysInMonth) {
121                 Box(modifier = Modifier.weight(1f).aspectRatio(1f))
122             } else {
123                 val date = selectedMonth.atDay(dayNum)
124                 val hasRun = date in datesWithRuns
125                 val isSelected = date == selectedDate
126                 val isToday = date == today
127                 val isFuture = date.isAfter(today)
128
129                 CalendarDay(
130                     dayNum = dayNum,
131                     hasRun = hasRun,
132                     isSelected = isSelected,
133                     isToday = isToday,
134                     isFuture = isFuture,
135                     onClick = { if (!isFuture) onDateSelected(date) },
136                     modifier = Modifier.weight(1f),
137                 )
138             }
139         }
140     }
141 }
142 }
143 }
144
145 @Composable
146 private fun CalendarDay(
147     dayNum: Int,
148     hasRun: Boolean,
149     isSelected: Boolean,
150     isToday: Boolean,
151     isFuture: Boolean,
152     onClick: () -> Unit,
153     modifier: Modifier = Modifier,
154 ) {
155     val primaryColor = MaterialTheme.colorScheme.primary
156     val bgColor = when {
157         isSelected -> primaryColor
158         else -> Color.Transparent
159     }
160     val textColor = when {
161         isFuture -> MaterialTheme.colorScheme.onSurface.copy(alpha = 0.3f)
162         isSelected -> MaterialTheme.colorScheme.onPrimary
163         isToday -> primaryColor
164         else -> MaterialTheme.colorScheme.onSurface
165     }
166
167     Box(
168         modifier = modifier
169             .aspectRatio(1f)

```

```
170         .padding(2.dp)
171         .clip(CircleShape)
172         .background(bgColor)
173         .clickable(enabled = !isFuture, onClick = onClick),
174     contentAlignment = Alignment.Center,
175 ) {
176     Column(horizontalAlignment = Alignment.CenterHorizontally) {
177         Text(
178             text = dayNum.toString(),
179             style = MaterialTheme.typography.bodySmall,
180             fontSize = 14.sp,
181             fontWeight = if (isToday && !isSelected) FontWeight.Bold else FontWeight.Normal,
182             color = textColor,
183         )
184         if (hasRun) {
185             Box(
186                 modifier = Modifier
187                     .size(4.dp)
188                     .clip(CircleShape)
189                     .background(
190                         if (isSelected) MaterialTheme.colorScheme.onPrimary
191                         else primaryColor,
192                     ),
193             )
194         }
195     }
196 }
197 }
```

```

1  package com.runway.android.ui.running.history
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.layout.Arrangement
5  import androidx.compose.foundation.layout.Box
6  import androidx.compose.foundation.layout.Column
7  import androidx.compose.foundation.layout.PaddingValues
8  import androidx.compose.foundation.layout.Row
9  import androidx.compose.foundation.layout.Spacer
10 import androidx.compose.foundation.layout.fillMaxSize
11 import androidx.compose.foundation.layout.fillMaxWidth
12 import androidx.compose.foundation.layout.height
13 import androidx.compose.foundation.layout.navigationBarsPadding
14 import androidx.compose.foundation.layout.padding
15 import androidx.compose.foundation.layout.size
16 import androidx.compose.foundation.layout.statusBarsPadding
17 import androidx.compose.foundation.layout.width
18 import androidx.compose.foundation.lazy.LazyColumn
19 import androidx.compose.foundation.shape.CircleShape
20 import androidx.compose.material.icons.Icons
21 import androidx.compose.material.icons.automirrored.filled.ArrowBack
22 import androidx.compose.material.icons.automirrored.filled.TextSnippet
23 import androidx.compose.material.icons.filled.AddLocation
24 import androidx.compose.material.icons.filled.Close
25 import androidx.compose.material.icons.filled.ContentCut
26 import androidx.compose.material.icons.filled.Delete
27 import androidx.compose.material.icons.filled.Fullscreen
28 import androidx.compose.material.icons.filled.MoreVert
29 import androidx.compose.material.icons.filled.Share
30 import androidx.compose.material3.AlertDialog
31 import androidx.compose.material3.ButtonDefaults
32 import androidx.compose.material3.Button
33 import androidx.compose.material3.CircularProgressIndicator
34 import androidx.compose.material3.DropdownMenu
35 import androidx.compose.material3.DropdownMenuItem
36 import androidx.compose.material3.Icon
37 import androidx.compose.material3.IconButton
38 import androidx.compose.material3.MaterialTheme
39 import androidx.compose.material3.Slider
40 import androidx.compose.material3.Text
41 import androidx.compose.material3.TextButton
42 import androidx.compose.runtime.Composable
43 import androidx.compose.runtime.LaunchedEffect
44 import androidx.compose.runtime.getValue
45 import androidx.compose.runtime.mutableStateOf
46 import androidx.compose.runtime.remember
47 import androidx.compose.runtime.setValue
48 import androidx.compose.ui.Alignment
49 import androidx.compose.ui.Modifier
50 import androidx.compose.ui.draw.clip
51 import androidx.compose.ui.graphics.Color
52 import androidx.compose.ui.text.font.FontWeight
53 import androidx.compose.ui.unit.dp
54 import androidx.compose.ui.unit.sp
55 import androidx.compose.ui.platform.LocalContext
56 import androidx.compose.ui.window.Dialog
57 import androidx.compose.ui.window.DialogProperties
58 import androidx.hilt.navigation.compose.hiltViewModel
59 import com.runway.android.core.map.MapPoint
60 import com.runway.android.ui.course.CreateCourseDialog
61 import com.runway.android.core.share.ShareUtils
62 import com.runway.android.core.util.formatDuration
63 import com.runway.android.core.util.formatPace
64 import com.runway.android.core.util.formatRunDateFull
65 import com.runway.android.core.util.formatTime
66 import com.runway.android.ui.components.RouteMapView
67 import com.runway.android.ui.components.RunDetailMetricGrid
68 import com.runway.android.ui.components.RunPaceChart
69 import com.runway.android.ui.components.RunSplitsCard
70
71 @Composable
72 fun RunDetailScreen(
73     onBackPressed: () -> Unit,
74     onDelete: (runId: String) -> Unit = {},
75     onShareImage: (runId: String) -> Unit = {},
76     onNavigateToCourseDetail: (String) -> Unit = {},
77     viewModel: RunDetailViewModel = hiltViewModel(),
78 ) {
79     val context = LocalContext.current
80
81     // 삭제 완료 시 onDelete 콜백 호출 (NavGraph에서 pop + savedInstanceState 처리)
82     LaunchedEffect(viewModel.isDeleted) {
83         if (viewModel.isDeleted) {

```



```

84         onDelete(viewModel.runId)
85     }
86 }
87
88 // 코스 생성 완료 시 코스 상세로 이동
89 LaunchedEffect(Unit) {
90     viewModel.courseCreated.collect { courseId ->
91         onNavigateToCourseDetail(courseId)
92     }
93 }
94
95 // 삭제 확인 다이얼로그
96 if (viewModel.showDeleteDialog) {
97     AlertDialog(
98         onDismissRequest = viewModel::dismissDeleteDialog,
99         title = { Text("기록 삭제") },
100        text = { Text("이 러닝 기록을 영구적으로 삭제할까요? 되돌릴 수 없습니다.") },
101        confirmButton = {
102            TextButton(onClick = viewModel::confirmDelete) {
103                Text("삭제", color = MaterialTheme.colorScheme.error)
104            }
105        },
106        dismissButton = {
107            TextButton(onClick = viewModel::dismissDeleteDialog) { Text("취소") }
108        },
109    )
110 }
111
112 // 거리 수정 다이얼로그
113 if (viewModel.showTrimDialog) {
114     val maxKm = viewModel.maxTrimKm
115     AlertDialog(
116         onDismissRequest = viewModel::dismissTrimDialog,
117         title = { Text("거리 수정") },
118         text = {
119             Column {
120                 Text(
121                     text = "현재 기록: ${"%%.2f".format(maxKm)} km",
122                     style = MaterialTheme.typography.bodySmall,
123                     color = MaterialTheme.colorScheme.onSurfaceVariant,
124                 )
125                 Spacer(modifier = Modifier.height(12.dp))
126                 Text(
127                     text = "${"%%.2f".format(viewModel.trimTargetKm)} km",
128                     style = MaterialTheme.typography.headlineMedium.copy(
129                         fontWeight = FontWeight.Bold,
130                         fontSize = 32.sp,
131                     ),
132                     color = MaterialTheme.colorScheme.primary,
133                 )
134                 Slider(
135                     value = viewModel.trimTargetKm,
136                     onValueChange = { viewModel.trimTargetKm = it },
137                     valueRange = 0.1f..maxKm,
138                     steps = ((maxKm - 0.1f) / 0.1f).toInt().coerceAtLeast(0),
139                     modifier = Modifier.fillMaxWidth(),
140                 )
141                 Text(
142                     text = "줄이기만 가능 · 시간/페이스는 비례 조정",
143                     style = MaterialTheme.typography.labelSmall,
144                     color = MaterialTheme.colorScheme.onSurfaceVariant,
145                 )
146             }
147         },
148         confirmButton = {
149             TextButton(
150                 onClick = viewModel::confirmTrim,
151                 enabled = viewModel.trimTargetKm < maxKm,
152             ) {
153                 Text("수정")
154             }
155         },
156         dismissButton = {
157             TextButton(onClick = viewModel::dismissTrimDialog) { Text("취소") }
158         },
159     )
160 }
161
162 // 코스 저장 다이얼로그 (항상 draft로 저장)
163 if (viewModel.showCreateCourseDialog) {
164     CreateCourseDialog(
165         courseName = viewModel.courseName,
166         onCourseNameChange = { viewModel.courseName = it },
167         isLoop = viewModel.isLoop,
168         onIsLoopChange = { viewModel.isLoop = it },
169         isCreating = viewModel.isCreatingCourse,

```

```

170         errorMessage = viewModel.createCourseError,
171         onConfirm = viewModel::createCourse,
172         onDismiss = viewModel::dismissCreateCourseDialog,
173     )
174 }
175
176 Column(
177     modifier = Modifier
178         .fillMaxSize()
179         .background(MaterialTheme.colorScheme.background)
180         .statusBarsPadding(),
181 ) {
182     // ─ 상단 바 ─
183     Row(
184         modifier = Modifier
185             .fillMaxWidth()
186             .padding(horizontal = 8.dp, vertical = 4.dp),
187         verticalAlignment = Alignment.CenterVertically,
188     ) {
189         IconButton(onClick = onBackPressed) {
190             Icon(
191                 imageVector = Icons.AutoMirrored.Filled.ArrowBack,
192                 contentDescription = "뒤로 가기",
193                 tint = MaterialTheme.colorScheme.onBackground,
194             )
195         }
196         Spacer(modifier = Modifier.width(4.dp))
197         Text(
198             text = "러닝 상세",
199             style = MaterialTheme.typography.headlineMedium,
200             color = MaterialTheme.colorScheme.onBackground,
201             modifier = Modifier.weight(1f),
202         )
203         IconButton(
204             onClick = {
205                 val detail = viewModel.detail
206                 if (detail != null) {
207                     ShareUtils.shareRunSummary(
208                         context = context,
209                         distanceFormatted = if ((detail.distanceMeters ?: 0.0) >= 1000)
210                             "%.2f km".format((detail.distanceMeters ?: 0.0) / 1000)
211                         else "${(detail.distanceMeters ?: 0).toInt()} m",
212                         duration = formatDuration(detail.durationSeconds),
213                         pace = formatPace(detail.avgPaceSecondsPerKm),
214                         date = formatRunDateFull(detail.startedAt),
215                     )
216                 }
217             },
218         ) {
219             Icon(
220                 imageVector = Icons.AutoMirrored.Filled.TextSnippet,
221                 contentDescription = "텍스트 공유",
222                 tint = MaterialTheme.colorScheme.onSurfaceVariant,
223             )
224         }
225         IconButton(onClick = { onShareImage(viewModel.runId) }) {
226             Icon(
227                 imageVector = Icons.Filled.Share,
228                 contentDescription = "공유 이미지 만들기",
229                 tint = MaterialTheme.colorScheme.onSurfaceVariant,
230             )
231         }
232         // : 더보기 메뉴
233         var menuExpanded by remember { mutableStateOf(false) }
234         Box {
235             IconButton(onClick = { menuExpanded = true }) {
236                 Icon(
237                     imageVector = Icons.Filled.MoreVert,
238                     contentDescription = "더보기",
239                     tint = MaterialTheme.colorScheme.onSurfaceVariant,
240                 )
241             }
242             DropdownMenu(
243                 expanded = menuExpanded,
244                 onDismissRequest = { menuExpanded = false },
245             ) {
246                 if (viewModel.canCreateCourse) {
247                     DropdownMenuItem(
248                         text = { Text("코스 만들기") },
249                         leadingIcon = {
250                             Icon(
251                                 Icons.Filled.AddLocation,
252                                 contentDescription = null,
253                                 tint = MaterialTheme.colorScheme.primary,
254                             )
255                         },
256                     )
257                 }
258             }
259         }
260     }
261 }

```

```

256         onClick = {
257             menuExpanded = false
258             viewModel.openCreateCourseDialog()
259         },
260     )
261 }
262 DropdownMenuItem(
263     text = { Text("거리 수정") },
264     leadingIcon = {
265         Icon(Icons.Filled.ContentCut, contentDescription = null)
266     },
267     onClick = {
268         menuExpanded = false
269         viewModel.openTrimDialog()
270     },
271     enabled = viewModel.detail != null,
272 )
273 DropdownMenuItem(
274     text = { Text("삭제", color = MaterialTheme.colorScheme.error) },
275     leadingIcon = {
276         Icon(
277             Icons.Filled.Delete,
278             contentDescription = null,
279             tint = MaterialTheme.colorScheme.error,
280         )
281     },
282     onClick = {
283         menuExpanded = false
284         viewModel.openDeleteDialog()
285     },
286 )
287 }
288 }
289 }
290
291 // ——— 콘텍스트 ———
292 when {
293     viewModel.isLoading -> {
294         Box(
295             modifier = Modifier.fillMaxSize(),
296             contentAlignment = Alignment.Center,
297         ) {
298             CircularProgressIndicator(
299                 modifier = Modifier.size(40.dp),
300                 color = MaterialTheme.colorScheme.primary,
301             )
302         }
303     }
304     viewModel.hasError -> {
305         Box(
306             modifier = Modifier.fillMaxSize(),
307             contentAlignment = Alignment.Center,
308         ) {
309             Column(horizontalAlignment = Alignment.CenterHorizontally) {
310                 Text(
311                     text = "기록을 불러오지 못했습니다.",
312                     style = MaterialTheme.typography.bodyLarge,
313                     color = MaterialTheme.colorScheme.onSurfaceVariant,
314                 )
315                 Spacer(modifier = Modifier.height(16.dp))
316                 Button(onClick = viewModel::retry) {
317                     Text("다시 시도")
318                 }
319             }
320         }
321     }
322     viewModel.detail != null -> {
323         val detail = viewModel.detail!!
324         val kilometerElapsedSeconds = remember(viewModel.splits) {
325             viewModel.splits.runningFold(0) { elapsed, split ->
326                 elapsed + split.durationSeconds
327             }.drop(1).take(viewModel.splits.count { it.distanceKm >= 0.99 })
328         }
329         val bestPaceSecondsPerKm = remember(viewModel.splits, viewModel.chartPoints) {
330             viewModel.splits.minOfOrNull { it.paceSecondsPerKm }
331             ?: viewModel.chartPoints.minOfOrNull { it.paceSecondsPerKm }
332         }
333
334         var showFullScreenMap by remember { mutableStateOf(false) }
335         val fullScreenPoints = remember(detail.points) {
336             detail.points.map { MapPoint(it.latitude, it.longitude) }
337         }
338
339         if (showFullScreenMap) {
340             Dialog(
341                 onDismissRequest = { showFullScreenMap = false },

```

```

342         properties = DialogProperties(
343             usePlatformDefaultWidth = false,
344             dismissOnClickOutside = true,
345         ),
346     ) {
347         Box(
348             modifier = Modifier
349                 .fillMaxSize()
350                 .background(Color.Black),
351         ) {
352             RouteMapView(
353                 points = fullScreenPoints,
354                 modifier = Modifier.fillMaxSize(),
355                 gesturesEnabled = true,
356                 showKilometerMarkers = true,
357             )
358             IconButton(
359                 onClick = { showFullScreenMap = false },
360                 modifier = Modifier
361                     .align(Alignment.TopEnd)
362                     .statusBarsPadding()
363                     .padding(12.dp)
364                     .background(
365                         color = Color.Black.copy(alpha = 0.5f),
366                         shape = CircleShape,
367                     ),
368             ) {
369                 Icon(
370                     imageVector = Icons.Filled.Close,
371                     contentDescription = "닫기",
372                     tint = Color.White,
373                 )
374             }
375         }
376     }
377 }
378
379 LazyColumn(
380     contentPadding = PaddingValues(
381         start = 20.dp,
382         end = 20.dp,
383         top = 12.dp,
384         bottom = 32.dp,
385     ),
386     verticalArrangement = Arrangement.spacedBy(16.dp),
387 ) {
388     // — 날짜 + 시각 헤더 —
389     item {
390         Column {
391             Text(
392                 text = formatRunDateFull(detail.startedAt),
393                 style = MaterialTheme.typography.titleLarge,
394                 color = MaterialTheme.colorScheme.onBackground,
395             )
396             Spacer(modifier = Modifier.height(2.dp))
397             val startTime = formatTime(detail.startedAt)
398             val endTime = formatTime(detail.endedAt)
399             val timeRange = if (endTime.isNotEmpty()) "$startTime → $endTime"
400                 else startTime
401             Text(
402                 text = timeRange,
403                 style = MaterialTheme.typography.bodyMedium,
404                 color = MaterialTheme.colorScheme.onSurfaceVariant,
405             )
406         }
407     }
408
409     // — 지표 그리드 —
410     item {
411         RunDetailMetricGrid(
412             distanceMeters = detail.distanceMeters,
413             durationSeconds = detail.durationSeconds,
414             avgPaceSecondsPerKm = detail.avgPaceSecondsPerKm,
415             caloriesBurned = detail.caloriesBurned,
416             avgHeartRateBpm = detail.avgHeartRateBpm,
417         )
418     }
419
420     // — 경로 미리보기 —
421     item {
422         Text(
423             text = "ROUTE",
424             style = MaterialTheme.typography.labelMedium,
425             color = MaterialTheme.colorScheme.onSurfaceVariant,
426         )
427         Spacer(modifier = Modifier.height(8.dp))

```

```

428         Box(
429             modifier = Modifier
430                 .fillMaxWidth()
431                 .height(220.dp)
432                 .clip(MaterialTheme.shapes.extraLarge),
433         ) {
434             RouteMapView(
435                 points = detail.points.map { MapPoint(it.latitude, it.longitude) },
436                 modifier = Modifier.fillMaxSize(),
437                 gesturesEnabled = false,
438                 showKilometerMarkers = true,
439                 onClick = { showFullScreenMap = true },
440             )
441             Icon(
442                 imageVector = Icons.Filled.Fullscreen,
443                 contentDescription = "전체 화면으로 보기",
444                 tint = Color.White,
445                 modifier = Modifier
446                     .align(Alignment.TopEnd)
447                     .padding(10.dp)
448                     .background(
449                         color = Color.Black.copy(alpha = 0.4f),
450                         shape = CircleShape,
451                     )
452                     .padding(4.dp),
453             )
454         }
455     }
456
457     // — 페이스 차트 —
458     if (viewModel.chartPoints.isNotEmpty()) {
459         item {
460             RunFaceChart(
461                 points = viewModel.chartPoints,
462                 kilometerElapsedSeconds = kilometerElapsedSeconds,
463                 averagePaceSecondsPerKm = detail.avgPaceSecondsPerKm,
464                 bestPaceSecondsPerKm = bestPaceSecondsPerKm,
465             )
466         }
467     }
468
469     // — 1km 구간 기록 —
470     item {
471         RunSplitsCard(splits = viewModel.splits)
472     }
473
474     // — 코스 만들기 —
475     if (viewModel.canCreateCourse) {
476         item {
477             Button(
478                 onClick = viewModel::openCreateCourseDialog,
479                 modifier = Modifier
480                     .fillMaxWidth()
481                     .navigationBarsPadding(),
482                 colors = ButtonDefaults.buttonColors(
483                     containerColor = MaterialTheme.colorScheme.primary,
484                 ),
485             ) {
486                 Icon(
487                     imageVector = Icons.Filled.AddLocation,
488                     contentDescription = null,
489                     modifier = Modifier.size(18.dp),
490                 )
491                 Spacer(Modifier.width(8.dp))
492                 Text("코스 만들기", fontWeight = FontWeight.Bold)
493             }
494         }
495     }
496 }
497
498 }
499
500 }

```

```

1 package com.runway.android.ui.running.history
2
3 import androidx.compose.runtime.getValue
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.compose.runtime.setValue
6 import androidx.lifecycle.SavedStateHandle
7 import androidx.lifecycle.ViewModel
8 import androidx.lifecycle.viewModelScope
9 import com.runway.android.core.result.NetworkResult
10 import com.runway.android.core.running.RunChartCalculator
11 import com.runway.android.core.running.RunChartPoint
12 import com.runway.android.core.running.RunSplit
13 import com.runway.android.core.running.SplitCalculator
14 import com.runway.android.data.course.model.CreateCourseFromRunRequest
15 import com.runway.android.data.running.model.RunDetailResponse
16 import com.runway.android.domain.course.CourseRepository
17 import com.runway.android.domain.running.RunningRepository
18 import dagger.hilt.android.lifecycle.HiltViewModel
19 import kotlinx.coroutines.channels.Channel
20 import kotlinx.coroutines.flow.receiveAsFlow
21 import kotlinx.coroutines.launch
22 import javax.inject.Inject
23
24 @HiltViewModel
25 class RunDetailViewModel @Inject constructor(
26     savedStateHandle: SavedStateHandle,
27     private val runningRepository: RunningRepository,
28     private val courseRepository: CourseRepository,
29 ) : ViewModel() {
30
31     val runId: String = checkNotNull(savedStateHandle["runId"])
32
33     var detail by mutableStateOf<RunDetailResponse?>(null)
34     private set
35     var splits by mutableStateOf<List<RunSplit>>(emptyList())
36     private set
37     var chartPoints by mutableStateOf<List<RunChartPoint>>(emptyList())
38     private set
39     var isLoading by mutableStateOf(true)
40     private set
41     var hasError by mutableStateOf(false)
42     private set
43     var showDeleteDialog by mutableStateOf(false)
44     var showTrimDialog by mutableStateOf(false)
45     var trimTargetKm by mutableStateOf(0f)
46     var isActionLoading by mutableStateOf(false)
47     private set
48     var isDeleted by mutableStateOf(false)
49     private set
50     var actionError by mutableStateOf<String?>(null)
51
52     val maxTrimKm: Float get() = ((detail?.distanceMeters ?: 0.0) / 1000.0).toFloat()
53
54     init {
55         loadDetail()
56     }
57
58     fun openDeleteDialog() { showDeleteDialog = true }
59     fun dismissDeleteDialog() { showDeleteDialog = false }
60     fun openTrimDialog() {
61         trimTargetKm = maxTrimKm
62         showTrimDialog = true
63     }
64     fun dismissTrimDialog() { showTrimDialog = false }
65
66     fun confirmDelete() {
67         viewModelScope.launch {
68             isActionLoading = true
69             showDeleteDialog = false
70             val result = runningRepository.deleteRun(runId)
71             if (result is com.runway.android.core.result.NetworkResult.Success) {
72                 isDeleted = true
73             } else {
74                 actionError = "삭제에 실패했습니다."
75             }
76             isActionLoading = false
77         }
78     }
79
80     fun confirmTrim() {
81         val targetMeters = (trimTargetKm * 1000.0).toDouble()
82         viewModelScope.launch {
83             isActionLoading = true

```

```

84         showTrimDialog = false
85         when (runningRepository.trimRun(runId, targetMeters)) {
86             is com.runway.android.core.result.NetworkResult.Success -> {
87                 isLoading = true
88                 detail = null
89                 loadDetail()
90             }
91             else -> actionError = "수정에 실패했습니다."
92         }
93         isActionLoading = false
94     }
95 }
96
97 fun clearActionError() { actionError = null }
98
99 // — 코스 생성 —
100
101 var showCreateCourseDialog by mutableStateOf(false)
102 var courseName by mutableStateOf("")
103 var isLoop by mutableStateOf(false)
104 var isCreatingCourse by mutableStateOf(false)
105 var createCourseError by mutableStateOf<String?>(null)
106
107 private val _courseCreated = Channel<String>(Channel.BUFFERED)
108 val courseCreated = _courseCreated.receiveAsFlow()
109
110 val canCreateCourse: Boolean get() = detail?.status == "completed"
111
112 fun openCreateCourseDialog() {
113     courseName = ""
114     isLoop = false
115     createCourseError = null
116     showCreateCourseDialog = true
117 }
118
119 fun dismissCreateCourseDialog() {
120     if (!isCreatingCourse) showCreateCourseDialog = false
121 }
122
123 fun createCourse() {
124     if (courseName.isBlank()) {
125         createCourseError = "코스 이름을 입력해 주세요."
126         return
127     }
128     viewModelScope.launch {
129         isCreatingCourse = true
130         createCourseError = null
131         val request = CreateCourseFromRunRequest(
132             name = courseName.trim(),
133             description = null,
134             isLoop = isLoop,
135             publish = false,
136         )
137         when (val result = courseRepository.createCourseFromRun(runId, request)) {
138             is NetworkResult.Success -> {
139                 showCreateCourseDialog = false
140                 _courseCreated.send(result.data.courseId)
141             }
142             is NetworkResult.ApiError -> createCourseError = when (result.errorCode) {
143                 "NOT_COMPLETED_RUN" -> "완료된 러닝만 코스로 만들 수 있습니다."
144                 else -> result.message
145             }
146             is NetworkResult.NetworkError -> createCourseError = "네트워크 오류가 발생했습니다."
147         }
148         isCreatingCourse = false
149     }
150 }
151
152 fun retry() {
153     isLoading = true
154     hasError = false
155     detail = null
156     splits = emptyList()
157     chartPoints = emptyList()
158     loadDetail()
159 }
160
161 private fun loadDetail() {
162     viewModelScope.launch {
163         when (val result = runningRepository.getRunDetail(runId)) {
164             is NetworkResult.Success -> {
165                 detail = result.data
166                 splits = SplitCalculator.calculate(result.data.points)
167                 chartPoints = RunChartCalculator.calculate(result.data.points)
168             }
169             else -> hasError = true

```

```
170         }
171         isLoading = false
172     }
173 }
174 }
```



```

1  package com.runway.android.ui.settings
2
3  import androidx.compose.foundation.BorderStroke
4  import androidx.compose.foundation.background
5  import androidx.compose.foundation.border
6  import androidx.compose.foundation.clickable
7  import androidx.compose.foundation.layout.Arrangement
8  import androidx.compose.foundation.layout.Box
9  import androidx.compose.foundation.layout.Column
10 import androidx.compose.foundation.layout.Row
11 import androidx.compose.foundation.layout.Spacer
12 import androidx.compose.foundation.layout.fillMaxSize
13 import androidx.compose.foundation.layout.fillMaxWidth
14 import androidx.compose.foundation.layout.height
15 import androidx.compose.foundation.layout.padding
16 import androidx.compose.foundation.layout.size
17 import androidx.compose.foundation.layout.statusBarsPadding
18 import androidx.compose.foundation.layout.width
19 import androidx.compose.foundation.shape.CircleShape
20 import androidx.compose.foundation.rememberScrollState
21 import androidx.compose.foundation.verticalScroll
22 import androidx.compose.material.icons.Icons
23 import androidx.compose.material.icons.automirrored.filled.ArrowBack
24 import androidx.compose.material.icons.filled.BrightnessMedium
25 import androidx.compose.material.icons.filled.Check
26 import androidx.compose.material.icons.filled.DarkMode
27 import androidx.compose.material.icons.filled.LightMode
28 import androidx.compose.material.icons.filled.CheckCircle
29 import androidx.compose.material3.Icon
30 import androidx.compose.material3.IconButton
31 import androidx.compose.material3.MaterialTheme
32 import androidx.compose.material3.Surface
33 import androidx.compose.material3.Text
34 import androidx.compose.runtime.Composable
35 import androidx.compose.runtime.collectAsState
36 import androidx.compose.runtime.getValue
37 import androidx.compose.ui.Alignment
38 import androidx.compose.ui.Modifier
39 import androidx.compose.ui.graphics.Color
40 import androidx.compose.ui.graphics.vector.ImageVector
41 import androidx.compose.ui.unit.dp
42 import androidx.hilt.navigation.compose.hiltViewModel
43 import com.runway.android.ui.theme.AccentColor
44 import com.runway.android.ui.theme.ThemeMode
45
46 @Composable
47 fun SettingsScreen(
48     onBack: () -> Unit,
49     viewModel: SettingsViewModel = hiltViewModel(),
50 ) {
51     val currentMode by viewModel.themeMode.collectAsState()
52     val currentAccent by viewModel.accentColor.collectAsState()
53
54     Column(
55         modifier = Modifier
56             .fillMaxSize()
57             .background(MaterialTheme.colorScheme.background)
58             .statusBarsPadding(),
59     ) {
60         Row(
61             modifier = Modifier
62                 .fillMaxWidth()
63                 .padding(horizontal = 8.dp, vertical = 4.dp),
64             verticalAlignment = Alignment.CenterVertically,
65         ) {
66             IconButton(onClick = onBack) {
67                 Icon(
68                     imageVector = Icons.AutoMirrored.Filled.ArrowBack,
69                     contentDescription = "뒤로 가기",
70                     tint = MaterialTheme.colorScheme.onBackground,
71                 )
72             }
73             Spacer(modifier = Modifier.width(4.dp))
74             Text(
75                 text = "설정",
76                 style = MaterialTheme.typography.headlineMedium,
77                 color = MaterialTheme.colorScheme.onBackground,
78             )
79         }
80
81         Column(
82             modifier = Modifier
83                 .fillMaxSize()

```

```

84         .verticalScroll(rememberScrollState())
85         .padding(horizontal = 20.dp),
86     ) {
87         Spacer(modifier = Modifier.height(20.dp))
88
89         Text(
90             text = "테마",
91             style = MaterialTheme.typography.labelMedium,
92             color = MaterialTheme.colorScheme.onSurfaceVariant,
93         )
94
95         Spacer(modifier = Modifier.height(12.dp))
96
97         Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
98             ThemeOptionTile(
99                 icon = Icons.Filled.DarkMode,
100                 title = "다크",
101                 description = "항상 어두운 테마를 사용합니다",
102                 selected = currentMode == ThemeMode.DARK,
103                 onClick = { viewModel.setThemeMode(ThemeMode.DARK) },
104             )
105             ThemeOptionTile(
106                 icon = Icons.Filled.LightMode,
107                 title = "라이트",
108                 description = "항상 밝은 테마를 사용합니다",
109                 selected = currentMode == ThemeMode.LIGHT,
110                 onClick = { viewModel.setThemeMode(ThemeMode.LIGHT) },
111             )
112             ThemeOptionTile(
113                 icon = Icons.Filled.BrightnessMedium,
114                 title = "시스템 설정",
115                 description = "기기의 다크 모드 설정을 따릅니다",
116                 selected = currentMode == ThemeMode.SYSTEM,
117                 onClick = { viewModel.setThemeMode(ThemeMode.SYSTEM) },
118             )
119         }
120
121         Spacer(modifier = Modifier.height(28.dp))
122         Text(
123             text = "컨셉 색상",
124             style = MaterialTheme.typography.labelMedium,
125             color = MaterialTheme.colorScheme.onSurfaceVariant,
126         )
127         Spacer(modifier = Modifier.height(6.dp))
128         Text(
129             text = "아이콘, 버튼과 강조 텍스트에 적용됩니다",
130             style = MaterialTheme.typography.bodySmall,
131             color = MaterialTheme.colorScheme.onSurfaceVariant,
132         )
133         Spacer(modifier = Modifier.height(14.dp))
134         AccentColorPicker(
135             selected = currentAccent,
136             onSelect = viewModel::setAccentColor,
137         )
138
139         Spacer(modifier = Modifier.height(28.dp))
140     }
141 }
142
143
144 @Composable
145 private fun AccentColorPicker(
146     selected: AccentColor,
147     onSelect: (AccentColor) -> Unit,
148 ) {
149     Row(
150         modifier = Modifier.fillMaxWidth(),
151         horizontalArrangement = Arrangement.SpaceBetween,
152     ) {
153         AccentColor.entries.forEach { accent ->
154             Column(
155                 horizontalAlignment = Alignment.CenterHorizontally,
156                 verticalArrangement = Arrangement.spacedBy(7.dp),
157             ) {
158                 Box(
159                     modifier = Modifier
160                         .size(42.dp)
161                         .background(accent.color, CircleShape)
162                         .border(
163                             width = if (accent == selected) 3.dp else 1.dp,
164                             color = if (accent == selected) {
165                                 MaterialTheme.colorScheme.onBackground
166                             } else {
167                                 MaterialTheme.colorScheme.outline
168                             },
169                             shape = CircleShape,

```

```

170         )
171         .clickable { onSelect(accent) },
172         contentAlignment = Alignment.Center,
173     ) {
174         if (accent == selected) {
175             Icon(
176                 imageVector = Icons.Filled.Check,
177                 contentDescription = "${accent.displayName} 선택됨",
178                 tint = Color(0xFF111119),
179                 modifier = Modifier.size(22.dp),
180             )
181         }
182     }
183     Text(
184         text = accent.displayName,
185         style = MaterialTheme.typography.labelSmall,
186         color = if (accent == selected) {
187             MaterialTheme.colorScheme.primary
188         } else {
189             MaterialTheme.colorScheme.onSurfaceVariant
190         },
191     )
192 }
193 }
194 }
195 }
196
197 @Composable
198 private fun ThemeOptionTile(
199     icon: ImageVector,
200     title: String,
201     description: String,
202     selected: Boolean,
203     onClick: () -> Unit,
204 ) {
205     Surface(
206         onClick = onClick,
207         modifier = Modifier.fillMaxWidth(),
208         shape = MaterialTheme.shapes.extraLarge,
209         color = if (selected) MaterialTheme.colorScheme.primaryContainer
210             else MaterialTheme.colorScheme.surface,
211         border = BorderStroke(
212             width = if (selected) 2.dp else 1.dp,
213             color = if (selected) MaterialTheme.colorScheme.primary
214                 else MaterialTheme.colorScheme.outline,
215         ),
216     ) {
217         Row(
218             modifier = Modifier
219                 .fillMaxWidth()
220                 .padding(horizontal = 20.dp, vertical = 16.dp),
221             verticalAlignment = Alignment.CenterVertically,
222             horizontalArrangement = Arrangement.spacedBy(16.dp),
223         ) {
224             Box(
225                 modifier = Modifier
226                     .size(44.dp)
227                     .background(
228                         color = if (selected) MaterialTheme.colorScheme.primary.copy(alpha = 0.15f)
229                             else MaterialTheme.colorScheme.surfaceVariant,
230                         shape = MaterialTheme.shapes.large,
231                     ),
232                 contentAlignment = Alignment.Center,
233             ) {
234                 Icon(
235                     imageVector = icon,
236                     contentDescription = null,
237                     tint = if (selected) MaterialTheme.colorScheme.primary
238                         else MaterialTheme.colorScheme.onSurfaceVariant,
239                     modifier = Modifier.size(22.dp),
240                 )
241             }
242
243             Column(modifier = Modifier.weight(1f)) {
244                 Text(
245                     text = title,
246                     style = MaterialTheme.typography.bodyLarge,
247                     color = if (selected) MaterialTheme.colorScheme.onPrimaryContainer
248                         else MaterialTheme.colorScheme.onSurface,
249                 )
250                 Text(
251                     text = description,
252                     style = MaterialTheme.typography.bodySmall,
253                     color = MaterialTheme.colorScheme.onSurfaceVariant,
254                 )
255             }

```

```

256
257         if (selected) {
258             Icon(
259                 imageVector = Icons.Filled.CheckCircle,
260                 contentDescription = null,
261                 tint = MaterialTheme.colorScheme.primary,
262                 modifier = Modifier.size(22.dp),
263             )
264         }
265     }
266 }
267 }

```

android/app/src/main/java/com/runway/android/ui/settings/SettingsViewModel.kt

Kotlin | 33줄

```

1  package com.runway.android.ui.settings
2
3  import androidx.lifecycle.ViewModel
4  import androidx.lifecycle.viewModelScope
5  import com.runway.android.core.datastore.ThemeDataStore
6  import com.runway.android.ui.theme.AccentColor
7  import com.runway.android.ui.theme.ThemeMode
8  import dagger.hilt.android.lifecycle.HiltViewModel
9  import kotlinx.coroutines.flow.SharingStarted
10 import kotlinx.coroutines.flow.StateFlow
11 import kotlinx.coroutines.flow.stateIn
12 import kotlinx.coroutines.launch
13 import javax.inject.Inject
14
15 @HiltViewModel
16 class SettingsViewModel @Inject constructor(
17     private val themeDataStore: ThemeDataStore,
18 ) : ViewModel() {
19
20     val themeMode: StateFlow<ThemeMode> = themeDataStore.themeModeFlow
21         .stateIn(viewModelScope, SharingStarted.Eagerly, ThemeMode.DARK)
22
23     val accentColor: StateFlow<AccentColor> = themeDataStore.accentColorFlow
24         .stateIn(viewModelScope, SharingStarted.Eagerly, AccentColor.GREEN)
25
26     fun setThemeMode(mode: ThemeMode) {
27         viewModelScope.launch { themeDataStore.setThemeMode(mode) }
28     }
29
30     fun setAccentColor(color: AccentColor) {
31         viewModelScope.launch { themeDataStore.setAccentColor(color) }
32     }
33 }

```

```

1  package com.runway.android.ui.share
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.border
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.layout.Arrangement
7  import androidx.compose.foundation.layout.Box
8  import androidx.compose.foundation.layout.Column
9  import androidx.compose.foundation.layout.PaddingValues
10 import androidx.compose.foundation.layout.Row
11 import androidx.compose.foundation.layout.Spacer
12 import androidx.compose.foundation.layout.aspectRatio
13 import androidx.compose.foundation.layout.fillMaxSize
14 import androidx.compose.foundation.layout.fillMaxWidth
15 import androidx.compose.foundation.layout.height
16 import androidx.compose.foundation.layout.padding
17 import androidx.compose.foundation.layout.size
18 import androidx.compose.foundation.layout.statusBarsPadding
19 import androidx.compose.foundation.layout.width
20 import androidx.compose.foundation.lazy.LazyRow
21 import androidx.compose.foundation.lazy.items
22 import androidx.compose.material3.FilterChip
23 import androidx.compose.foundation.rememberScrollState
24 import androidx.compose.foundation.shape.RoundedCornerShape
25 import androidx.compose.foundation.verticalScroll
26 import androidx.compose.material.icons.Icons
27 import androidx.compose.material.icons.automirrored.filled.ArrowBack
28 import androidx.compose.material.icons.filled.Share
29 import androidx.compose.material3.Button
30 import androidx.compose.material3.CircularProgressIndicator
31 import androidx.compose.material3.Icon
32 import androidx.compose.material3.IconButton
33 import androidx.compose.material3.MaterialTheme
34 import androidx.compose.material3.Text
35 import androidx.compose.animation.Crossfade
36 import androidx.compose.animation.core.Spring
37 import androidx.compose.animation.core.animateDpAsState
38 import androidx.compose.animation.core.animateFloatAsState
39 import androidx.compose.animation.core.spring
40 import androidx.compose.animation.core.tween
41 import androidx.compose.runtime.Composable
42 import androidx.compose.runtime.LaunchedEffect
43 import androidx.compose.runtime.getValue
44 import androidx.compose.ui.Alignment
45 import androidx.compose.ui.Modifier
46 import androidx.compose.ui.draw.clip
47 import androidx.compose.ui.draw.scale
48 import androidx.compose.ui.graphics.Color
49 import androidx.compose.ui.platform.LocalContext
50 import androidx.compose.ui.text.font.FontWeight
51 import androidx.compose.ui.unit.dp
52 import androidx.compose.ui.unit.sp
53 import com.runway.android.BuildConfig
54 import androidx.hilt.navigation.compose.hiltViewModel
55 import com.runway.android.core.share.MetricPreset
56 import com.runway.android.core.share.ShareTemplate
57 import com.runway.android.core.util.formatDuration
58 import com.runway.android.data.running.model.RunDetailResponse
59 import java.time.Instant
60 import java.time.ZoneId
61 import java.time.format.DateTimeFormatter
62
63 @Composable
64 fun RunShareImageScreen(
65     onBack: () -> Unit,
66     viewModel: RunShareImageViewModel = hiltViewModel(),
67 ) {
68     val context = LocalContext.current
69
70     LaunchedEffect(Unit) {
71         viewModel.shareEvent.collect { uri ->
72             context.startActivity(com.runway.android.core.share.ImageShareUtil.buildShareIntent(uri))
73         }
74     }
75
76     Column(
77         modifier = Modifier
78             .fillMaxSize()
79             .background(MaterialTheme.colorScheme.background)
80             .statusBarsPadding(),
81     ) {
82         // — TopBar —
83         Row(

```

```

84         modifier = Modifier
85             .fillMaxWidth()
86             .padding(horizontal = 8.dp, vertical = 4.dp),
87         verticalAlignment = Alignment.CenterVertically,
88     ) {
89         IconButton(onClick = onBack) {
90             Icon(
91                 imageVector = Icons.AutoMirrored.Filled.ArrowBack,
92                 contentDescription = "뒤로 가기",
93                 tint = MaterialTheme.colorScheme.onBackground,
94             )
95         }
96         Spacer(modifier = Modifier.width(4.dp))
97         Text(
98             text = "공유 이미지",
99             style = MaterialTheme.typography.headlineMedium,
100             color = MaterialTheme.colorScheme.onBackground,
101         )
102     }
103
104     when {
105         viewModel.isLoading -> Box(
106             modifier = Modifier.fillMaxSize(),
107             contentAlignment = Alignment.Center,
108         ) {
109             CircularProgressIndicator(color = MaterialTheme.colorScheme.primary)
110         }
111
112         viewModel.hasError -> Box(
113             modifier = Modifier.fillMaxSize(),
114             contentAlignment = Alignment.Center,
115         ) {
116             Text(
117                 text = "러닝 데이터를 불러오지 못했습니다.",
118                 style = MaterialTheme.typography.bodyLarge,
119                 color = MaterialTheme.colorScheme.onSurfaceVariant,
120             )
121         }
122
123         viewModel.detail != null -> {
124             val detail = viewModel.detail!!
125
126             Column(
127                 modifier = Modifier
128                     .weight(1f)
129                     .verticalScroll(rememberScrollState())
130                     .padding(horizontal = 20.dp),
131             ) {
132                 Spacer(modifier = Modifier.height(12.dp))
133
134                 // — 카드 미리보기 (crossfade on template change) —
135                 Crossfade(
136                     targetState = viewModel.selectedTemplate,
137                     animationSpec = tween(300),
138                     label = "templateCrossfade",
139                 ) { template ->
140                     ShareCardPreview(
141                         detail = detail,
142                         template = template,
143                         preset = viewModel.selectedPreset,
144                         modifier = Modifier
145                             .fillMaxWidth()
146                             .aspectRatio(1f)
147                             .clip(MaterialTheme.shapes.extraLarge),
148                     )
149                 }
150
151                 Spacer(modifier = Modifier.height(20.dp))
152
153                 // — 템플릿 선택 —
154                 Text(
155                     text = "테마 선택",
156                     style = MaterialTheme.typography.labelMedium,
157                     color = MaterialTheme.colorScheme.onSurfaceVariant,
158                 )
159                 Spacer(modifier = Modifier.height(8.dp))
160                 LazyRow(
161                     horizontalArrangement = Arrangement.spacedBy(10.dp),
162                     contentPadding = PaddingValues(horizontal = 2.dp),
163                 ) {
164                     items(ShareTemplate.entries) { template ->
165                         TemplateChip(
166                             template = template,
167                             selected = viewModel.selectedTemplate == template,
168                             onClick = { viewModel.onTemplateChange(template) },
169                         )

```

```

170         }
171     }
172
173     Spacer(modifier = Modifier.height(20.dp))
174
175     // ——— 지표 프리셋 선택 ———
176     Text(
177         text = "지표 선택",
178         style = MaterialTheme.typography.labelMedium,
179         color = MaterialTheme.colorScheme.onSurfaceVariant,
180     )
181     Spacer(modifier = Modifier.height(8.dp))
182     Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
183         MetricPreset.entries.forEach { preset ->
184             FilterChip(
185                 selected = viewModel.selectedPreset == preset,
186                 onClick = { viewModel.onPresetChange(preset) },
187                 label = { Text(preset.displayName, style = MaterialTheme.typography.labelMedium) },
188             )
189         }
190     }
191
192     Spacer(modifier = Modifier.height(24.dp))
193
194     // ——— 예러 메시지 ———
195     if (viewModel.shareError != null) {
196         Text(
197             text = viewModel.shareError!!,
198             style = MaterialTheme.typography.bodySmall,
199             color = MaterialTheme.colorScheme.error,
200             modifier = Modifier.padding(bottom = 8.dp),
201         )
202     }
203 }
204
205 // ——— 공유 버튼 ———
206 Button(
207     onClick = { viewModel.captureAndShare(context) },
208     enabled = !viewModel.isSharing,
209     modifier = Modifier
210         .fillMaxWidth()
211         .padding(horizontal = 20.dp, vertical = 16.dp),
212 ) {
213     if (viewModel.isSharing) {
214         CircularProgressIndicator(
215             modifier = Modifier.size(18.dp),
216             strokeWidth = 2.dp,
217             color = MaterialTheme.colorScheme.onPrimary,
218         )
219         Spacer(modifier = Modifier.width(8.dp))
220         Text("이미지 생성 중...")
221     } else {
222         Icon(
223             imageVector = Icons.Filled.Share,
224             contentDescription = null,
225             modifier = Modifier.size(18.dp),
226         )
227         Spacer(modifier = Modifier.width(8.dp))
228         Text("공유하기")
229     }
230 }
231 }
232 }
233 }
234 }
235
236 @Composable
237 private fun ShareCardPreview(
238     detail: RunDetailResponse,
239     template: ShareTemplate,
240     preset: MetricPreset,
241     modifier: Modifier = Modifier,
242 ) {
243     val bgColor = Color(template.bgColor)
244     val accentColor = Color(template.accentColor)
245     val textPrimary = Color(template.textPrimary)
246     val textSecondary = Color(template.textSecondary)
247
248     val distKm = (detail.distanceMeters ?: 0.0) / 1000.0
249
250     Box(
251         modifier = modifier.background(bgColor),
252     ) {
253         // Top accent line
254         Box(
255             modifier = Modifier

```

```

256         .fillMaxWidth()
257         .height(4.dp)
258         .background(accentColor),
259     )
260
261     Column(
262         modifier = Modifier
263             .fillMaxSize()
264             .padding(20.dp),
265     ) {
266         Spacer(modifier = Modifier.height(8.dp))
267
268         // Brand
269         Text(
270             text = BuildConfig.APP_NAME.uppercase(),
271             color = textPrimary,
272             fontSize = 22.sp,
273             fontWeight = FontWeight.Black,
274             letterSpacing = 3.sp,
275         )
276         Text(
277             text = formatPreviewDate(detail.startedAt),
278             color = textSecondary,
279             fontSize = 12.sp,
280         )
281
282         Spacer(modifier = Modifier.height(12.dp))
283
284         // Distance
285         Text(
286             text = "%.2f".format(distKm),
287             color = textPrimary,
288             fontSize = 64.sp,
289             fontWeight = FontWeight.Black,
290             lineHeight = 64.sp,
291         )
292         Text(text = "km", color = textSecondary, fontSize = 16.sp)
293
294         Spacer(modifier = Modifier.height(12.dp))
295
296         // Metrics - preset-dependent
297         when (preset) {
298             MetricPreset.FULL_STATS -> {
299                 Row(horizontalArrangement = Arrangement.spacedBy(24.dp)) {
300                     Column {
301                         Text(text = "시간", color = textSecondary, fontSize = 11.sp)
302                         Text(
303                             text = formatPreviewDuration(detail.durationSeconds),
304                             color = textPrimary,
305                             fontSize = 16.sp,
306                             fontWeight = FontWeight.Bold,
307                         )
308                     }
309                     Column {
310                         Text(text = "페이스", color = textSecondary, fontSize = 11.sp)
311                         Text(
312                             text = formatPreviewPace(detail.avgPaceSecondsPerKm),
313                             color = textPrimary,
314                             fontSize = 16.sp,
315                             fontWeight = FontWeight.Bold,
316                         )
317                     }
318                     if (detail.caloriesBurned != null) {
319                         Column {
320                             Text(text = "칼로리", color = textSecondary, fontSize = 11.sp)
321                             Text(
322                                 text = "${detail.caloriesBurned} kcal",
323                                 color = textPrimary,
324                                 fontSize = 16.sp,
325                                 fontWeight = FontWeight.Bold,
326                             )
327                         }
328                     }
329                 }
330             }
331             MetricPreset.DISTANCE_FOCUS -> {
332                 Column {
333                     Text(text = "소요 시간", color = textSecondary, fontSize = 11.sp)
334                     Text(
335                         text = formatPreviewDuration(detail.durationSeconds),
336                         color = textPrimary,
337                         fontSize = 16.sp,
338                         fontWeight = FontWeight.Bold,
339                     )
340                 }
341             }
342         }
343     }

```



```

342         MetricPreset.PACE_FOCUS -> {
343             Column {
344                 Text(text = "평균 페이스", color = textSecondary, fontSize = 11.sp)
345                 Text(
346                     text = formatPreviewPace(detail.avgPaceSecondsPerKm),
347                     color = accentColor,
348                     fontSize = 28.sp,
349                     fontWeight = FontWeight.Black,
350                 )
351                 Spacer(modifier = Modifier.height(8.dp))
352                 Text(text = "소요 시간", color = textSecondary, fontSize = 11.sp)
353                 Text(
354                     text = formatPreviewDuration(detail.durationSeconds),
355                     color = textPrimary,
356                     fontSize = 16.sp,
357                     fontWeight = FontWeight.Bold,
358                 )
359             }
360         }
361     }
362
363     Spacer(modifier = Modifier.weight(1f))
364
365     Text(
366         text = "Run your way.",
367         color = accentColor,
368         fontSize = 12.sp,
369         letterSpacing = 1.sp,
370     )
371 }
372 }
373 }
374
375 @Composable
376 private fun TemplateChip(
377     template: ShareTemplate,
378     selected: Boolean,
379     onClick: () -> Unit,
380 ) {
381     val bgColor = Color(template.bgColor)
382     val accentColor = Color(template.accentColor)
383     val borderWidth by animateDpAsState(
384         targetValue = if (selected) 2.dp else 0.dp,
385         animationSpec = tween(200),
386         label = "chipBorder",
387     )
388     val chipScale by animateFloatAsState(
389         targetValue = if (selected) 1.06f else 1f,
390         animationSpec = spring(dampingRatio = Spring.DampingRatioMediumBouncy),
391         label = "chipScale",
392     )
393
394     Box(
395         modifier = Modifier
396             .size(width = 80.dp, height = 48.dp)
397             .scale(chipScale)
398             .clip(RoundedCornerShape(10.dp))
399             .background(bgColor)
400             .border(borderWidth, accentColor, RoundedCornerShape(10.dp))
401             .clickable(onClick = onClick),
402         contentAlignment = Alignment.BottomStart,
403     ) {
404         Box(
405             modifier = Modifier
406                 .fillMaxWidth()
407                 .height(3.dp)
408                 .background(accentColor),
409         )
410         Text(
411             text = template.displayName,
412             color = Color(template.textPrimary),
413             fontSize = 9.sp,
414             fontWeight = FontWeight.Bold,
415             modifier = Modifier.padding(start = 6.dp, bottom = 8.dp),
416         )
417     }
418 }
419
420 private fun formatPreviewDate(isoDate: String?): String {
421     if (isoDate == null) return ""
422     return runCatching {
423         val instant = Instant.parse(isoDate)
424         DateTimeFormatter.ofPattern("yyyy.MM.dd")
425             .withZone(ZoneId.systemDefault())
426             .format(instant)
427     }.getOrDefault("")

```

```
428 }
429
430 private fun formatPreviewDuration(seconds: Int?): String {
431     return formatDuration(seconds)
432 }
433
434 private fun formatPreviewPace(secsPerKm: Int?): String {
435     if (secsPerKm == null || secsPerKm <= 0) return "---\\"
436     return "%d'%02d\\".format(secsPerKm / 60, secsPerKm % 60)
437 }
```

```

1  package com.runway.android.ui.share
2
3  import android.content.Context
4  import android.net.Uri
5  import androidx.compose.runtime.getValue
6  import androidx.compose.runtime.mutableStateOf
7  import androidx.compose.runtime.setValue
8  import androidx.lifecycle.SavedStateHandle
9  import androidx.lifecycle.ViewModel
10 import androidx.lifecycle.viewModelScope
11 import com.runway.android.core.result.NetworkResult
12 import com.runway.android.core.share.ImageCaptureUtil
13 import com.runway.android.core.share.ImageShareUtil
14 import com.runway.android.core.share.MetricPreset
15 import com.runway.android.core.share.ShareTemplate
16 import com.runway.android.data.running.model.RunDetailResponse
17 import com.runway.android.domain.running.RunningRepository
18 import dagger.hilt.android.lifecycle.HiltViewModel
19 import kotlinx.coroutines.Dispatchers
20 import kotlinx.coroutines.flow.MutableSharedFlow
21 import kotlinx.coroutines.flow.asSharedFlow
22 import kotlinx.coroutines.launch
23 import kotlinx.coroutines.withContext
24 import javax.inject.Inject
25
26 @HiltViewModel
27 class RunShareImageViewModel @Inject constructor(
28     savedStateHandle: SavedStateHandle,
29     private val runningRepository: RunningRepository,
30 ) : ViewModel() {
31
32     val runId: String = checkNotNull(savedStateHandle["runId"])
33
34     var detail by mutableStateOf<RunDetailResponse?>(null)
35     private set
36     var isLoading by mutableStateOf(true)
37     private set
38     var hasError by mutableStateOf(false)
39     private set
40     var selectedTemplate by mutableStateOf(ShareTemplate.DARK_SPORT)
41     private set
42     var selectedPreset by mutableStateOf(MetricPreset.FULL_STATS)
43     private set
44     var isSharing by mutableStateOf(false)
45     private set
46     var shareError by mutableStateOf<String?>(null)
47     private set
48
49     private val _shareEvent = MutableSharedFlow<Uri>()
50     val shareEvent = _shareEvent.asSharedFlow()
51
52     init {
53         loadDetail()
54     }
55
56     fun onTemplateChange(template: ShareTemplate) {
57         selectedTemplate = template
58     }
59
60     fun onPresetChange(preset: MetricPreset) {
61         selectedPreset = preset
62     }
63
64     fun captureAndShare(context: Context) {
65         if (isSharing) return
66         val d = detail ?: return
67         isSharing = true
68         shareError = null
69
70         viewModelScope.launch(Dispatchers.IO) {
71             runCatching {
72                 val bitmap = ImageCaptureUtil.draw(d, selectedTemplate, selectedPreset)
73                 ImageShareUtil.saveToCache(context, bitmap)
74                 ?: error("이미지를 캐시에 저장하지 못했습니다.")
75             }.onSuccess { uri ->
76                 _shareEvent.emit(uri)
77             }.onFailure {
78                 withContext(Dispatchers.Main) {
79                     shareError = "이미지를 생성하지 못했습니다."
80                 }
81             }
82             withContext(Dispatchers.Main) { isSharing = false }
83         }

```

```
84     }
85
86     fun clearShareError() {
87         shareError = null
88     }
89
90     private fun loadDetail() {
91         viewModelScope.launch {
92             when (val result = runningRepository.getRunDetail(runId)) {
93                 is NetworkResult.Success -> detail = result.data
94                 else -> hasError = true
95             }
96             isLoading = false
97         }
98     }
99 }
```

```

1  package com.runway.android.ui.splash
2
3  import androidx.compose.foundation.Canvas
4  import androidx.compose.foundation.Image
5  import androidx.compose.foundation.layout.Box
6  import androidx.compose.foundation.layout.fillMaxSize
7  import androidx.compose.foundation.layout.size
8  import androidx.compose.material3.MaterialTheme
9  import androidx.compose.runtime.Composable
10 import androidx.compose.ui.Alignment
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.geometry.Offset
13 import androidx.compose.ui.graphics.Color
14 import androidx.compose.ui.graphics.Path
15 import androidx.compose.ui.graphics.PathMeasure
16 import androidx.compose.ui.graphics.StrokeCap
17 import androidx.compose.ui.graphics.drawscope.Stroke
18 import androidx.compose.ui.graphics.graphicsLayer
19 import androidx.compose.ui.layout.ContentScale
20 import androidx.compose.ui.res.painterResource
21 import androidx.compose.ui.tooling.preview.Preview
22 import androidx.compose.ui.unit.dp
23 import com.runway.android.R
24 import com.runway.android.ui.theme.RunwayTheme
25
26 @Composable
27 fun RunwayLogoAnimation(
28     progress: Float,
29     modifier: Modifier = Modifier,
30 ) {
31     val clampedProgress = progress.coerceIn(0f, 1f)
32     Box(
33         modifier = modifier.size(240.dp),
34         contentAlignment = Alignment.Center,
35     ) {
36         PathFinderMarkDrawAnimation(
37             progress = clampedProgress,
38             modifier = Modifier.fillMaxSize(),
39         )
40     }
41 }
42
43 @Composable
44 private fun PathFinderMarkDrawAnimation(
45     progress: Float,
46     modifier: Modifier = Modifier,
47 ) {
48     val primaryColor = MaterialTheme.colorScheme.primary
49     val drawProgress = (easeInOut(progress) / 0.78f).coerceIn(0f, 1f)
50     val revealProgress = ((progress - 0.70f) / 0.30f).coerceIn(0f, 1f)
51
52     Box(modifier = modifier, contentAlignment = Alignment.Center) {
53         Canvas(modifier = Modifier.fillMaxSize()) {
54             val diameter = size.minDimension * 0.68f
55             val left = (size.width - diameter) / 2f
56             val top = (size.height - diameter) / 2f
57             val scale = diameter / 48f
58
59             drawCircle(
60                 color = Color(0xFF111318),
61                 radius = diameter / 2f,
62                 center = Offset(size.width / 2f, size.height / 2f),
63             )
64
65             val mark = Path().apply {
66                 moveTo(left + 13f * scale, top + 30f * scale)
67                 lineTo(left + 20f * scale, top + 18f * scale)
68                 lineTo(left + 25f * scale, top + 24f * scale)
69                 lineTo(left + 31f * scale, top + 13f * scale)
70                 lineTo(left + 35f * scale, top + 16f * scale)
71                 lineTo(left + 26f * scale, top + 35f * scale)
72                 lineTo(left + 20f * scale, top + 28f * scale)
73                 lineTo(left + 17f * scale, top + 33f * scale)
74                 close()
75             }
76             val measure = PathMeasure().apply { setPath(mark, false) }
77             val segment = Path()
78             measure.getSegment(0f, measure.length * drawProgress, segment, true)
79             drawPath(
80                 path = segment,
81                 color = primaryColor,
82                 style = Stroke(width = 2.4f * scale, cap = StrokeCap.Round),
83             )

```

```

84     }
85
86     Image(
87         painter = painterResource(id = R.drawable.app_logo_mark),
88         contentDescription = "PathFinder Logo",
89         contentScale = ContentScale.Fit,
90         modifier = Modifier
91             .fillMaxSize(0.68f)
92             .graphicsLayer {
93                 alpha = easeOut(revealProgress)
94                 val revealScale = 0.94f + 0.06f * revealProgress
95                 scaleX = revealScale
96                 scaleY = revealScale
97             },
98     )
99 }
100 }
101
102 private fun easeOut(value: Float): Float {
103     val t = value.coerceIn(0f, 1f)
104     return 1f - (1f - t) * (1f - t)
105 }
106
107 private fun easeInOut(value: Float): Float {
108     val t = value.coerceIn(0f, 1f)
109     return if (t < 0.5f) 2f * t * t else -1f + (4f - 2f * t) * t
110 }
111
112 @Preview(showBackground = true, backgroundColor = 0xFF000000)
113 @Composable
114 private fun PathFinderLogoAnimationPreview() {
115     RunwayTheme {
116         Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
117             RunwayLogoAnimation(progress = 0.6f)
118         }
119     }
120 }

```

```

1  package com.runway.android.ui.splash
2
3  import androidx.compose.animation.core.Animatable
4  import androidx.compose.animation.core.LinearEasing
5  import androidx.compose.animation.core.Tween
6  import androidx.compose.foundation.background
7  import androidx.compose.foundation.layout.Box
8  import androidx.compose.foundation.layout.Column
9  import androidx.compose.foundation.layout.Spacer
10 import androidx.compose.foundation.layout.fillMaxSize
11 import androidx.compose.foundation.layout.height
12 import androidx.compose.foundation.layout.navigationBarsPadding
13 import androidx.compose.material3.MaterialTheme
14 import androidx.compose.material3.Text
15 import androidx.compose.runtime.Composable
16 import androidx.compose.runtime.LaunchedEffect
17 import androidx.compose.runtime.remember
18 import androidx.compose.ui.Alignment
19 import androidx.compose.ui.Modifier
20 import androidx.compose.ui.draw.alpha
21 import androidx.compose.ui.graphics.graphicsLayer
22 import androidx.compose.ui.text.font.FontWeight
23 import androidx.compose.ui.text.style.TextAlign
24 import androidx.compose.ui.tooling.preview.Preview
25 import androidx.compose.ui.unit.dp
26 import com.runway.android.BuildConfig
27 import com.runway.android.ui.theme.RunwayTheme
28
29 private const val SplashAnimationDurationMillis = 2_000
30
31 @Composable
32 fun RunwaySplashScreen(
33     onAnimationFinished: () -> Unit,
34     modifier: Modifier = Modifier,
35 ) {
36     val progress = remember { Animatable(0f) }
37
38     LaunchedEffect(Unit) {
39         try {
40             progress.animateTo(
41                 targetValue = 1f,
42                 animationSpec = tween(
43                     durationMillis = SplashAnimationDurationMillis,
44                     easing = LinearEasing,
45                 ),
46             )
47         } finally {
48             onAnimationFinished()
49         }
50     }
51
52     val taglineAlpha = ((progress.value - 0.85f) / 0.15f).coerceIn(0f, 1f)
53     val brandProgress = ((progress.value - 0.68f) / 0.24f).coerceIn(0f, 1f)
54
55     Box(
56         modifier = modifier
57             .fillMaxSize()
58             .background(MaterialTheme.colorScheme.background),
59         contentAlignment = Alignment.Center,
60     ) {
61         Column(horizontalAlignment = Alignment.CenterHorizontally) {
62             RunwayLogoAnimation(progress = progress.value)
63             Text(
64                 text = BuildConfig.APP_NAME,
65                 style = MaterialTheme.typography.headlineLarge,
66                 fontWeight = FontWeight.ExtraBold,
67                 color = MaterialTheme.colorScheme.onBackground,
68                 modifier = Modifier.graphicsLayer {
69                     alpha = brandProgress
70                     translationY = (1f - brandProgress) * 18.dp.toPx()
71                 },
72             )
73         }
74         Text(
75             text = "Run your way.",
76             style = MaterialTheme.typography.bodySmall,
77             color = MaterialTheme.colorScheme.onSurfaceVariant,
78             textAlign = TextAlign.Center,
79             modifier = Modifier
80                 .align(Alignment.BottomCenter)
81                 .navigationBarsPadding()
82                 .alpha(taglineAlpha)
83                 .height(48.dp),

```

```
84     )
85   }
86 }
87
88 @Preview(showBackground = true, backgroundColor = 0xFF111119)
89 @Composable
90 private fun RunwaySplashScreenPreview() {
91     RunwayTheme {
92         RunwaySplashScreen(onAnimationFinished = {})
93     }
94 }
```



```

1  package com.runway.android.ui.stats
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.layout.Arrangement
5  import androidx.compose.foundation.layout.Box
6  import androidx.compose.foundation.layout.Column
7  import androidx.compose.foundation.layout.Row
8  import androidx.compose.foundation.layout.Spacer
9  import androidx.compose.foundation.layout.fillMaxSize
10 import androidx.compose.foundation.layout.fillMaxWidth
11 import androidx.compose.foundation.layout.height
12 import androidx.compose.foundation.layout.padding
13 import androidx.compose.foundation.layout.size
14 import androidx.compose.foundation.rememberScrollState
15 import androidx.compose.foundation.verticalScroll
16 import androidx.compose.material.icons.Icons
17 import androidx.compose.material.icons.automirrored.filled.ArrowBack
18 import androidx.compose.material3.CircularProgressIndicator
19 import androidx.compose.material3.ExperimentalMaterial3Api
20 import androidx.compose.material3.FilterChip
21 import androidx.compose.material3.pulltorefresh.PullToRefreshBox
22 import androidx.compose.material3.FilterChipDefaults
23 import androidx.compose.material3.Icon
24 import androidx.compose.material3.IconButton
25 import androidx.compose.material3.MaterialTheme
26 import androidx.compose.material3.Text
27 import androidx.compose.material3.TextButton
28 import androidx.compose.material3.TopAppBar
29 import androidx.compose.material3.TopAppBarDefaults
30 import androidx.compose.runtime.Composable
31 import androidx.compose.ui.Alignment
32 import androidx.compose.ui.Modifier
33 import androidx.compose.ui.unit.dp
34 import androidx.hilt.navigation.compose.hiltViewModel
35 import com.runway.android.data.running.model.RunningStatsResponse
36 import com.runway.android.ui.components.StatsSummaryCard
37 import com.runway.android.ui.components.StreakCard
38
39 @OptIn(ExperimentalMaterial3Api::class)
40 @Composable
41 fun StatsScreen(
42     onBackPressed: () -> Unit,
43     viewModel: StatsViewModel = hiltViewModel(),
44 ) {
45     Column(
46         modifier = Modifier
47             .fillMaxSize()
48             .background(MaterialTheme.colorScheme.background),
49     ) {
50         TopAppBar(
51             title = {
52                 Text(
53                     text = "통계",
54                     style = MaterialTheme.typography.titleLarge,
55                     color = MaterialTheme.colorScheme.onBackground,
56                 )
57             },
58             navigationIcon = {
59                 IconButton(onClick = onBackPressed) {
60                     Icon(
61                         imageVector = Icons.AutoMirrored.Filled.ArrowBack,
62                         contentDescription = "뒤로",
63                         tint = MaterialTheme.colorScheme.onBackground,
64                     )
65                 }
66             },
67             colors = TopAppBarDefaults.topAppBarColors(
68                 containerColor = MaterialTheme.colorScheme.background,
69             ),
70         )
71
72         // 기간 선택 칩
73         Row(
74             modifier = Modifier
75                 .fillMaxWidth()
76                 .padding(horizontal = 16.dp),
77             horizontalArrangement = Arrangement.spacedBy(8.dp),
78         ) {
79             viewModel.periodLabels.forEachIndexed { index, label ->
80                 FilterChip(
81                     selected = viewModel.selectedPeriodIndex == index,
82                     onClick = { viewModel.selectPeriod(index) },
83                     label = {

```

```

84         Text(
85             text = label,
86             style = MaterialTheme.typography.labelMedium,
87         )
88     },
89     colors = FilterChipDefaults.filterChipColors(
90         selectedContainerColor = MaterialTheme.colorScheme.primary,
91         selectedLabelColor = MaterialTheme.colorScheme.onPrimary,
92         containerColor = MaterialTheme.colorScheme.surface,
93         labelColor = MaterialTheme.colorScheme.onSurfaceVariant,
94     ),
95 )
96 }
97 }
98
99 Spacer(modifier = Modifier.height(8.dp))
100
101 PullToRefreshBox(
102     isRefreshing = viewModel.isRefreshing,
103     onRefresh = viewModel::refresh,
104     modifier = Modifier.fillMaxSize(),
105 ) {
106     when {
107         viewModel.isLoading -> {
108             Box(
109                 modifier = Modifier.fillMaxSize(),
110                 contentAlignment = Alignment.Center,
111             ) {
112                 CircularProgressIndicator(
113                     modifier = Modifier.size(40.dp),
114                     color = MaterialTheme.colorsScheme.primary,
115                 )
116             }
117         }
118
119         viewModel.errorMessage != null -> {
120             Box(
121                 modifier = Modifier.fillMaxSize(),
122                 contentAlignment = Alignment.Center,
123             ) {
124                 Column(horizontalAlignment = Alignment.CenterHorizontally) {
125                     Text(
126                         text = viewModel.errorMessage!!,
127                         style = MaterialTheme.typography.bodyMedium,
128                         color = MaterialTheme.colorsScheme.onSurfaceVariant,
129                     )
130                     Spacer(modifier = Modifier.height(8.dp))
131                     TextButton(onClick = viewModel::retry) {
132                         Text("다시 시도")
133                     }
134                 }
135             }
136         }
137
138         viewModel.stats != null -> {
139             StatsContent(
140                 stats = viewModel.stats!!,
141                 periodLabel = viewModel.periodLabels[viewModel.selectedPeriodIndex],
142             )
143         }
144     }
145 } // PullToRefreshBox
146 }
147 }
148
149 @Composable
150 private fun StatsContent(
151     stats: RunningStatsResponse,
152     periodLabel: String,
153 ) {
154     Column(
155         modifier = Modifier
156             .fillMaxSize()
157             .verticalScroll(rememberScrollState())
158             .padding(horizontal = 16.dp),
159         verticalArrangement = Arrangement.spacedBy(12.dp),
160     ) {
161         // streak 카드
162         StreakCard(
163             currentStreak = stats.currentStreakDays,
164             longestStreak = stats.longestStreakDays,
165             modifier = Modifier.fillMaxWidth(),
166         )
167
168         // 핵심 지표 2열 그리드
169         StatsRow(

```

```

170         left = Pair("총 러닝", "${stats.totalRuns}회"),
171         right = Pair("활동일", "${stats.activeDays}일"),
172     )
173     StatsRow(
174         left = Pair("총 거리", formatDistance(stats.totalDistanceMeters)),
175         right = Pair("최장 러닝", formatDistance(stats.longestRunMeters)),
176     )
177     StatsRow(
178         left = Pair("총 시간", formatDuration(stats.totalDurationSeconds)),
179         right = Pair("평균 페이스", formatPace(stats.averagePaceSecondsPerKm)),
180     )
181     StatsSummaryCard(
182         label = "총 칼로리",
183         value = "${stats.totalCaloriesBurned} kcal",
184     )
185
186     Spacer(modifier = Modifier.height(16.dp))
187 }
188 }
189
190 @Composable
191 private fun StatsRow(left: Pair<String, String>, right: Pair<String, String>) {
192     Row(
193         modifier = Modifier.fillMaxWidth(),
194         horizontalArrangement = Arrangement.spacedBy(12.dp),
195     ) {
196         StatsSummaryCard(
197             label = left.first,
198             value = left.second,
199             modifier = Modifier.weight(1f),
200         )
201         StatsSummaryCard(
202             label = right.first,
203             value = right.second,
204             modifier = Modifier.weight(1f),
205         )
206     }
207 }
208
209 private fun formatDistance(meters: Double): String {
210     return if (meters < 1000) "${meters.toInt()} m"
211     else "${("%.1f".format(meters / 1000.0))} km"
212 }
213
214 private fun formatDuration(seconds: Long): String {
215     val h = seconds / 3600
216     val m = (seconds % 3600) / 60
217     return if (h > 0) "${h}h ${m}m" else "${m}m"
218 }
219
220 private fun formatPace(secsPerKm: Int): String {
221     if (secsPerKm <= 0) return "--"
222     return "%d'%02d\".format(secsPerKm / 60, secsPerKm % 60)
223 }

```

```

1  package com.runway.android.ui.stats
2
3  import androidx.compose.runtime.getValue
4  import androidx.compose.runtime.mutableStateOf
5  import androidx.compose.runtime.setValue
6  import androidx.lifecycle.ViewModel
7  import androidx.lifecycle.viewModelScope
8  import com.runway.android.core.result.NetworkResult
9  import com.runway.android.data.running.model.RunningStatsResponse
10 import com.runway.android.domain.running.RunningRepository
11 import dagger.hilt.android.lifecycle.HiltViewModel
12 import kotlinx.coroutines.launch
13 import javax.inject.Inject
14
15 @HiltViewModel
16 class StatsViewModel @Inject constructor(
17     private val runningRepository: RunningRepository,
18 ) : ViewModel() {
19
20     val periods = listOf("weekly", "monthly", "yearly", "all")
21     val periodLabels = listOf("이번 주", "이번 달", "올해", "전체")
22
23     var selectedPeriodIndex by mutableStateOf(1) // monthly default
24     private set
25     var stats by mutableStateOf<RunningStatsResponse?>(null)
26     private set
27     var isLoading by mutableStateOf(false)
28     private set
29     var isRefreshing by mutableStateOf(false)
30     private set
31     var errorMessage by mutableStateOf<String?>(null)
32     private set
33
34     init {
35         loadStats()
36     }
37
38     fun selectPeriod(index: Int) {
39         if (selectedPeriodIndex == index) return
40         selectedPeriodIndex = index
41         loadStats()
42     }
43
44     fun retry() = viewModelScope.launch { doLoadStats() }
45
46     fun refresh() {
47         viewModelScope.launch {
48             isRefreshing = true
49             doLoadStats()
50             isRefreshing = false
51         }
52     }
53
54     private fun loadStats() = viewModelScope.launch { doLoadStats() }
55
56     private suspend fun doLoadStats() {
57         isLoading = true
58         errorMessage = null
59         when (val result = runningRepository.getRunningStats(periods[selectedPeriodIndex])) {
60             is NetworkResult.Success -> stats = result.data
61             is NetworkResult.ApiError -> errorMessage = result.message
62             is NetworkResult.NetworkError -> errorMessage = "네트워크 연결을 확인해 주세요."
63         }
64         isLoading = false
65     }
66 }

```

android/app/src/main/java/com/runway/android/ui/theme/Color.kt

Kotlin | 43줄

```
1 package com.runway.android.ui.theme
2
3 import androidx.compose.ui.graphics.Color
4
5 // — Dark theme backgrounds —
6 val BackgroundDark = Color(0xFF111119)
7 val SurfaceDark = Color(0xFF1B1B2B)
8 val MutedDark = Color(0xFF222232)
9 val SecondaryContainerDark = Color(0xFF262638)
10
11 // — Light theme backgrounds —
12 val BackgroundLight = Color(0xFFFF5F5FA)
13 val SurfaceLight = Color(0xFFFFFFFF)
14 val MutedLight = Color(0xFFEEEEEF4)
15 val SecondaryContainerLight = Color(0xFFE8E8F0)
16
17 // — Text (dark theme) —
18 val OnSurfaceWhite = Color(0xFFF8F8FC)
19 val OnSurfaceMuted = Color(0xFF888AA8)
20
21 // — Text (light theme) —
22 val OnSurfaceDark = Color(0xFF111119)
23 val OnSurfaceMutedLight = Color(0xFF666680)
24
25 // — Border / Outline —
26 val BorderColorDark = Color(0xFF2B2B3D)
27 val BorderColorLight = Color(0xFFD8D8E8)
28
29 // — Primary - Neon lime green —
30 val RunwayGreen = Color(0xFFA4E168)
31 val OnRunwayGreen = Color(0xFF111119)
32
33 // — User-selectable accent colors —
34 val RunwayRed = Color(0xFFFF7474)
35 val RunwayOrange = Color(0xFFFFA45B)
36 val RunwayYellow = Color(0xFFFF3D45C)
37 val RunwayBlue = Color(0xFF65B7FF)
38 val RunwayPurple = Color(0xFFC58BFF)
39
40 // — Semantic states —
41 val DestructiveRed = Color(0xFFDC5040)
42 val SuccessGreen = Color(0xFF7FD98A)
43 val WarningYellow = Color(0xFFCCBE50)
```

android/app/src/main/java/com/runway/android/ui/theme/Shape.kt

Kotlin | 14줄

```
1 package com.runway.android.ui.theme
2
3 import androidx.compose.foundation.shape.RoundedCornerShape
4 import androidx.compose.material3.Shapes
5 import androidx.compose.ui.unit.dp
6
7 // Corner radii mirroring Lovable's --radius scale (base = 1rem ≈ 16dp)
8 val RunwayShapes = Shapes(
9     extraSmall = RoundedCornerShape(8.dp), // Filter chips, small badges
10    small = RoundedCornerShape(12.dp), // Icon containers, avatar fallbacks
11    medium = RoundedCornerShape(16.dp), // Default cards, list items
12    large = RoundedCornerShape(20.dp), // Input fields, primary buttons
13    extraLarge = RoundedCornerShape(24.dp), // Stats card, map container, metric grid
14 )
```

```

1  package com.runway.android.ui.theme
2
3  import androidx.compose.foundation.isSystemInDarkTheme
4  import androidx.compose.material3.MaterialTheme
5  import androidx.compose.material3.darkColorScheme
6  import androidx.compose.material3.lightColorScheme
7  import androidx.compose.runtime.Composable
8  import androidx.compose.runtime.CompositionLocalProvider
9  import androidx.compose.runtime.compositionLocalOf
10 import androidx.compose.ui.graphics.Color
11
12 /** True when the active theme is dark. Use instead of isSystemInDarkTheme() inside the app. */
13 val LocalIsDarkTheme = compositionLocalOf { true }
14
15 enum class ThemeMode { DARK, LIGHT, SYSTEM }
16
17 enum class AccentColor(
18     val displayName: String,
19     val color: Color,
20     val darkContainer: Color,
21     val lightContainer: Color,
22 ) {
23     RED("빨강", RunwayRed, Color(0xFF3D1B20), Color(0xFFFFDADA)),
24     ORANGE("주황", RunwayOrange, Color(0xFF3B2414), Color(0xFFFFDCC2)),
25     YELLOW("노랑", RunwayYellow, Color(0xFF342D12), Color(0xFFFFEFA8)),
26     GREEN("초록", RunwayGreen, Color(0xFF1C2E14), Color(0xFFD6F5B0)),
27     BLUE("파랑", RunwayBlue, Color(0xFF142B3D), Color(0xFFCDE8FF)),
28     PURPLE("보라", RunwayPurple, Color(0xFF2D1D3D), Color(0xFFE8D8FF)),
29 }
30
31 private fun runwayDarkColorScheme(accent: AccentColor) = darkColorScheme(
32     background = BackgroundDark,
33     surface = SurfaceDark,
34     surfaceVariant = MutedDark,
35
36     primary = accent.color,
37     onPrimary = OnRunwayGreen,
38     primaryContainer = accent.darkContainer,
39     onPrimaryContainer = accent.color,
40
41     secondary = SecondaryContainerDark,
42     onSecondary = OnSurfaceWhite,
43     secondaryContainer = SecondaryContainerDark,
44     onSecondaryContainer = OnSurfaceWhite,
45
46     error = DestructiveRed,
47     onError = OnSurfaceWhite,
48     errorContainer = Color(0xFF3B1410),
49     onErrorContainer = DestructiveRed,
50
51     onBackground = OnSurfaceWhite,
52     onSurface = OnSurfaceWhite,
53     onSurfaceVariant = OnSurfaceMuted,
54
55     outline = BorderColorDark,
56     outlineVariant = BorderColorDark,
57
58     inverseSurface = OnSurfaceWhite,
59     inverseOnSurface = BackgroundDark,
60     inversePrimary = accent.color,
61 )
62
63 private fun runwayLightColorScheme(accent: AccentColor) = lightColorScheme(
64     background = BackgroundLight,
65     surface = SurfaceLight,
66     surfaceVariant = MutedLight,
67
68     primary = accent.color,
69     onPrimary = OnRunwayGreen,
70     primaryContainer = accent.lightContainer,
71     onPrimaryContainer = OnSurfaceDark,
72
73     secondary = SecondaryContainerLight,
74     onSecondary = OnSurfaceDark,
75     secondaryContainer = SecondaryContainerLight,
76     onSecondaryContainer = OnSurfaceDark,
77
78     error = DestructiveRed,
79     onError = OnSurfaceWhite,
80     errorContainer = Color(0xFFFFE8EB),
81     onErrorContainer = DestructiveRed,
82
83     onBackground = OnSurfaceDark,

```

```

84     onSurface = OnSurfaceDark,
85     onSurfaceVariant = OnSurfaceMutedLight,
86
87     outline = BorderColorLight,
88     outlineVariant = BorderColorLight,
89
90     inverseSurface = OnSurfaceDark,
91     inverseOnSurface = BackgroundLight,
92     inversePrimary = accent.color,
93 )
94
95 @Composable
96 fun RunwayTheme(
97     themeMode: ThemeMode = ThemeMode.DARK,
98     accentColor: AccentColor = AccentColor.GREEN,
99     content: @Composable () -> Unit,
100 ) {
101     val isDark = when (themeMode) {
102         ThemeMode.DARK -> true
103         ThemeMode.LIGHT -> false
104         ThemeMode.SYSTEM -> isSystemInDarkTheme()
105     }
106     CompositionLocalProvider(LocalIsDarkTheme provides isDark) {
107         MaterialTheme(
108             colorScheme = if (isDark) {
109                 runwayDarkColorScheme(accentColor)
110             } else {
111                 runwayLightColorScheme(accentColor)
112             },
113             typography = RunwayTypography,
114             shapes = RunwayShapes,
115             content = content,
116         )
117     }
118 }

```

```

1  package com.runway.android.ui.theme
2
3  import androidx.compose.material3.Typography
4  import androidx.compose.ui.text.TextStyle
5  import androidx.compose.ui.text.font.FontFamily
6  import androidx.compose.ui.text.font.FontWeight
7  import androidx.compose.ui.text.googlefonts.Font
8  import androidx.compose.ui.text.googlefonts.GoogleFont
9  import androidx.compose.ui.unit.sp
10 import com.runway.android.R
11
12 private val fontProvider = GoogleFont.Provider(
13     providerAuthority = "com.google.android.gms.fonts",
14     providerPackage = "com.google.android.gms",
15     certificates = R.array.com_google_android_gms_fonts_certs,
16 )
17
18 private val notoSansKr = GoogleFont("Noto Sans KR")
19 private val jetbrainsMono = GoogleFont("JetBrains Mono")
20
21 val DisplayFontFamily = FontFamily(
22     Font(googleFont = notoSansKr, fontProvider = fontProvider, weight = FontWeight.Normal),
23     Font(googleFont = notoSansKr, fontProvider = fontProvider, weight = FontWeight.Medium),
24     Font(googleFont = notoSansKr, fontProvider = fontProvider, weight = FontWeight.SemiBold),
25     Font(googleFont = notoSansKr, fontProvider = fontProvider, weight = FontWeight.Bold),
26     Font(googleFont = notoSansKr, fontProvider = fontProvider, weight = FontWeight.ExtraBold),
27 )
28
29 val MonoFontFamily = FontFamily(
30     Font(googleFont = jetbrainsMono, fontProvider = fontProvider, weight = FontWeight.Normal),
31     Font(googleFont = jetbrainsMono, fontProvider = fontProvider, weight = FontWeight.Medium),
32     Font(googleFont = jetbrainsMono, fontProvider = fontProvider, weight = FontWeight.Bold),
33 )
34
35 val RunwayTypography = Typography(
36     // — 대형 숫자 — 러닝 타이머 (28:14), 영웅 거리 (5.42 km)
37     displayLarge = TextStyle(
38         fontFamily = MonoFontFamily,
39         fontWeight = FontWeight.Bold,
40         fontSize = 64.sp,
41         letterSpacing = (-2).sp,
42         lineHeight = 70.sp,
43     ),
44     // — 주요 통계 숫자 — 주간 거리 (24.6 km)
45     displayMedium = TextStyle(
46         fontFamily = MonoFontFamily,
47         fontWeight = FontWeight.Bold,
48         fontSize = 48.sp,
49         letterSpacing = (-1).sp,
50         lineHeight = 54.sp,
51     ),
52     // — 화면 타이틀 — "코스 탐색", "리더보드"
53     headlineLarge = TextStyle(
54         fontFamily = DisplayFontFamily,
55         fontWeight = FontWeight.ExtraBold,
56         fontSize = 30.sp,
57         letterSpacing = (-0.3).sp,
58         lineHeight = 38.sp,
59     ),
60     // — 섹션 헤더 / 카드 타이틀 — "Riverside Loop"
61     headlineMedium = TextStyle(
62         fontFamily = DisplayFontFamily,
63         fontWeight = FontWeight.Bold,
64         fontSize = 24.sp,
65         letterSpacing = (-0.2).sp,
66         lineHeight = 32.sp,
67     ),
68     // — 소형 화면 헤더 — 설정, 내 기록 서브헤더
69     headlineSmall = TextStyle(
70         fontFamily = DisplayFontFamily,
71         fontWeight = FontWeight.Bold,
72         fontSize = 21.sp,
73         letterSpacing = 0.sp,
74         lineHeight = 28.sp,
75     ),
76     // — 추적 그리드 수치 — "5'12\"", "11.5"
77     titleLarge = TextStyle(
78         fontFamily = DisplayFontFamily,
79         fontWeight = FontWeight.Bold,
80         fontSize = 22.sp,
81         letterSpacing = 0.sp,
82         lineHeight = 30.sp,
83     ),

```



```

84 // — 리스트 항목 제목 / 카드 이름
85 titleMedium = TextStyle(
86     fontFamily = DisplayFontFamily,
87     fontWeight = FontWeight.SemiBold,
88     fontSize = 17.sp,
89     letterSpacing = 0.sp,
90     lineHeight = 25.sp,
91 ),
92 // — 서브 레이블 / 작은 제목
93 titleSmall = TextStyle(
94     fontFamily = DisplayFontFamily,
95     fontWeight = FontWeight.Medium,
96     fontSize = 15.sp,
97     letterSpacing = 0.sp,
98     lineHeight = 22.sp,
99 ),
100 // — 본문 / 설명 — 기본 읽기 텍스트
101 bodyLarge = TextStyle(
102     fontFamily = DisplayFontFamily,
103     fontWeight = FontWeight.Normal,
104     fontSize = 16.sp,
105     letterSpacing = 0.sp,
106     lineHeight = 26.sp,
107 ),
108 // — 보조 본문 — 메타데이터, 거리·위치
109 bodyMedium = TextStyle(
110     fontFamily = DisplayFontFamily,
111     fontWeight = FontWeight.Normal,
112     fontSize = 15.sp,
113     letterSpacing = 0.sp,
114     lineHeight = 23.sp,
115 ),
116 // — 캡션 / 설명 서브텍스트
117 bodySmall = TextStyle(
118     fontFamily = DisplayFontFamily,
119     fontWeight = FontWeight.Normal,
120     fontSize = 14.sp,
121     letterSpacing = 0.sp,
122     lineHeight = 21.sp,
123 ),
124 // — 칩 / 배지 / 탭 레이블
125 labelLarge = TextStyle(
126     fontFamily = DisplayFontFamily,
127     fontWeight = FontWeight.Medium,
128     fontSize = 14.sp,
129     letterSpacing = 0.sp,
130     lineHeight = 20.sp,
131 ),
132 // — 소형 캡션 — "거리", "페이스"
133 labelMedium = TextStyle(
134     fontFamily = DisplayFontFamily,
135     fontWeight = FontWeight.Normal,
136     fontSize = 13.sp,
137     letterSpacing = 0.sp,
138     lineHeight = 18.sp,
139 ),
140 // — 최소 레이블 — 단위 (km, /km)
141 labelSmall = TextStyle(
142     fontFamily = DisplayFontFamily,
143     fontWeight = FontWeight.Normal,
144     fontSize = 12.sp,
145     letterSpacing = 0.sp,
146     lineHeight = 16.sp,
147 ),
148 )

```

```

1  package com.runway.android.ui.tracking
2
3  import androidx.compose.foundation.layout.Column
4  import androidx.compose.foundation.layout.Spacer
5  import androidx.compose.foundation.layout.height
6  import androidx.compose.material3.AlertDialog
7  import androidx.compose.material3.CircularProgressIndicator
8  import androidx.compose.material3.MaterialTheme
9  import androidx.compose.material3.Text
10 import androidx.compose.material3.TextButton
11 import androidx.compose.runtime.Composable
12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.unit.dp
14
15 @Composable
16 fun TrackingRecoveryDialog(viewModel: TrackingRecoveryViewModel) {
17     val snap = viewModel.snapshot ?: return
18
19     AlertDialog(
20         onDismissRequest = {},
21         title = { Text("이전 런 세션 발견") },
22         text = {
23             Column {
24                 Text("앱이 종료되었지만 완료되지 않은 런 세션이 있습니다.")
25                 Spacer(Modifier.height(8.dp))
26                 Text("거리: %.2f km".format(snap.distanceMeters / 1000.0))
27                 Text("시간: %02d:%02d".format(snap.elapsedSeconds / 60, snap.elapsedSeconds % 60))
28                 if (viewModel.isRecovering) {
29                     Spacer(Modifier.height(8.dp))
30                     CircularProgressIndicator()
31                 }
32                 if (viewModel.recoveryError != null) {
33                     Spacer(Modifier.height(8.dp))
34                     Text(
35                         text = viewModel.recoveryError!!,
36                         color = MaterialTheme.colorScheme.error,
37                         style = MaterialTheme.typography.bodySmall,
38                     )
39                 }
40             }
41         },
42         confirmButton = {
43             TextButton(
44                 onClick = viewModel::finish,
45                 enabled = !viewModel.isRecovering,
46             ) {
47                 Text("완료 처리")
48             }
49         },
50         dismissButton = {
51             if (viewModel.recoveryError != null) {
52                 // 오류 상태에서는 포기 대신 세션 강제 닫기 버튼 제공
53                 TextButton(onClick = viewModel::forceClose) {
54                     Text("세션 닫기", color = MaterialTheme.colorScheme.error)
55                 }
56             } else {
57                 TextButton(
58                     onClick = viewModel::abandon,
59                     enabled = !viewModel.isRecovering,
60                 ) {
61                     Text("포기")
62                 }
63             }
64         },
65     )
66 }

```

```

1 package com.runway.android.ui.tracking
2
3 import androidx.compose.runtime.getValue
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.compose.runtime.setValue
6 import androidx.lifecycle.ViewModel
7 import androidx.lifecycle.ViewModelScope
8 import com.runway.android.core.result.NetworkResult
9 import com.runway.android.core.tracking.PendingPointQueue
10 import com.runway.android.core.tracking.RunTrackingManager
11 import com.runway.android.core.tracking.TrackingSessionSnapshot
12 import com.runway.android.core.tracking.TrackingSessionStore
13 import com.runway.android.core.tracking.toRunPointRequest
14 import com.runway.android.data.attempt.model.AbandonAttemptRequest
15 import com.runway.android.data.attempt.model.FinishAttemptRequest
16 import com.runway.android.data.running.model.FinishRunRequest
17 import com.runway.android.data.running.model.SavePointsRequest
18 import com.runway.android.domain.attempt.CourseAttemptRepository
19 import com.runway.android.domain.running.RunningRepository
20 import dagger.hilt.android.lifecycle.HiltViewModel
21 import kotlinx.coroutines.launch
22 import java.time.Instant
23 import javax.inject.Inject
24
25 @HiltViewModel
26 class TrackingRecoveryViewModel @Inject constructor(
27     private val sessionStore: TrackingSessionStore,
28     private val pendingPointQueue: PendingPointQueue,
29     private val runningRepository: RunningRepository,
30     private val courseAttemptRepository: CourseAttemptRepository,
31     private val manager: RunTrackingManager,
32 ) : ViewModel() {
33
34     var snapshot by mutableStateOf<TrackingSessionSnapshot?>(null)
35     private set
36     var isRecovering by mutableStateOf(false)
37     private set
38     var isVisible by mutableStateOf(false)
39     private set
40     var recoveryError by mutableStateOf<String?>(null)
41     private set
42
43     init {
44         viewModelScope.launch {
45             sessionStore.snapshot.collect { snap ->
46                 snapshot = snap
47                 isVisible = snap != null && !manager.state.value.isTracking
48             }
49         }
50     }
51
52     fun finish() {
53         val snap = snapshot ?: return
54         if (isRecovering) return
55         isRecovering = true
56         recoveryError = null
57
58         viewModelScope.launch {
59             // Best-effort upload persisted points
60             repeat(3) {
61                 val batch = pendingPointQueue.dequeue(snap.runningRecordId, 20)
62                 if (batch.isNotEmpty()) {
63                     val r = runningRepository.savePoints(
64                         snap.runningRecordId,
65                         SavePointsRequest(batch.map { it.toRunPointRequest() }),
66                     )
67                     if (r is NetworkResult.Success) {
68                         pendingPointQueue.deleteByIds(batch.map { it.id })
69                     }
70                 }
71             }
72
73             // elapsedSeconds == 0인데 distanceMeters > 0이면 IMPOSSIBLE_SPEED 에러 방지
74             val safeDistance = if (snap.elapsedSeconds == 0) 0.0 else snap.distanceMeters
75             val safeDistKm = safeDistance / 1000.0
76
77             val result = if (snap.courseAttemptId != null) {
78                 courseAttemptRepository.finishAttempt(
79                     attemptId = snap.courseAttemptId,
80                     request = FinishAttemptRequest(
81                         endedAt = Instant.now().toString(),
82                         distanceMeters = safeDistance,
83                         durationSeconds = snap.elapsedSeconds,

```

```

84         avgPaceSecondsPerKm = if (safeDistKm > 0.001) (snap.elapsedSeconds / safeDistKm).toInt() else 0,
85         caloriesBurned = (safeDistKm * 72).toInt(),
86     ),
87 )
88 } else {
89     runningRepository.finishRun(
90         runId = snap.runningRecordId,
91         request = FinishRunRequest(
92             endedAt = Instant.now().toString(),
93             distanceMeters = safeDistance,
94             durationSeconds = snap.elapsedSeconds,
95             avgPaceSecondsPerKm = if (safeDistKm > 0.001) (snap.elapsedSeconds / safeDistKm).toInt() else 0,
96             caloriesBurned = (safeDistKm * 72).toInt(),
97         ),
98     )
99 }
100
101 when {
102     result is NetworkResult.Success -> cleanUp(snap.runningRecordId)
103     // 서버에서 이미 완료/포기 처리됐거나 레코드가 없는 경우:
104     // 네트워크 응답을 못 받은 상태에서 재시도한 케이스이므로 로컬 세션만 정리
105     result is NetworkResult.ApiError && result.errorCode in ALREADY_TERMINAL_ERRORS ->
106         cleanUp(snap.runningRecordId)
107     else -> {
108         recoveryError = "완료 처리에 실패했습니다. 다시 시도하거나 세션을 닫아주세요."
109         isRecovering = false
110     }
111 }
112 }
113 }
114
115 fun abandon() {
116     val snap = snapshot ?: return
117     if (isRecovering) return
118     isRecovering = true
119     recoveryError = null
120
121     viewModelScope.launch {
122         val result = if (snap.courseAttemptId != null) {
123             courseAttemptRepository.abandonAttempt(
124                 attemptId = snap.courseAttemptId,
125                 request = AbandonAttemptRequest(endedAt = Instant.now().toString()),
126             )
127         } else {
128             runningRepository.abandonRun(snap.runningRecordId)
129         }
130
131         when {
132             result is NetworkResult.Success -> cleanUp(snap.runningRecordId)
133             result is NetworkResult.ApiError && result.errorCode in ALREADY_TERMINAL_ERRORS ->
134                 cleanUp(snap.runningRecordId)
135             else -> {
136                 recoveryError = "포기 처리에 실패했습니다. 다시 시도하거나 세션을 닫아주세요."
137                 isRecovering = false
138             }
139         }
140     }
141 }
142
143 /** 서버 오류가 지속될 때 사용자가 로컬 세션을 강제로 폐기하는 탈출구 */
144 fun forceClose() {
145     val snap = snapshot ?: return
146     viewModelScope.launch {
147         cleanUp(snap.runningRecordId)
148     }
149 }
150
151 private suspend fun cleanUp(runningRecordId: String) {
152     sessionStore.clearSnapshot()
153     pendingPointQueue.deleteByRunningRecordId(runningRecordId)
154     isRecovering = false
155     isVisible = false
156 }
157
158 companion object {
159     private val ALREADY_TERMINAL_ERRORS = setOf(
160         "INVALID_ATTEMPT_STATUS", // 이미 완료/포기된 코스 시도
161         "INVALID_RUN_STATUS", // 이미 완료/포기된 런 기록
162         "COURSE_ATTEMPT_NOT_FOUND", // 서버에서 삭제된 코스 시도
163         "RUN_NOT_FOUND", // 서버에서 삭제된 런 기록
164     )
165 }
166 }

```

```
1 package com.runway.android.core.posture
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertNotNull
5 import org.junit.Test
6
7 class PostureAngleCalculatorTest {
8
9     @Test
10     fun `calculates bilateral knee angles in pixel aspect space`() {
11         val result = PostureAngleCalculator.compute(landmarks(), aspectRatio = 16f / 9f)
12
13         assertNotNull(result)
14         assertEquals(180f, result!!.leftKneeAngle!!, 0.1f)
15         assertEquals(180f, result.rightKneeAngle!!, 0.1f)
16     }
17
18     @Test
19     fun `drops individual angle below original confidence threshold`() {
20         val points = landmarks().toMutableList()
21         points[SKELETON_L_KNEE] = points[SKELETON_L_KNEE].copy(v = 0.2f)
22
23         val result = PostureAngleCalculator.compute(points)
24
25         assertEquals(null, result?.leftKneeAngle)
26         assertNotNull(result?.rightKneeAngle)
27     }
28
29     private fun landmarks(): List<SkeletonPoint> = listOf(
30         point(0.50f, 0.10f),
31         point(0.45f, 0.25f),
32         point(0.55f, 0.25f),
33         point(0.45f, 0.40f),
34         point(0.55f, 0.40f),
35         point(0.45f, 0.55f),
36         point(0.55f, 0.55f),
37         point(0.45f, 0.50f),
38         point(0.55f, 0.50f),
39         point(0.45f, 0.70f),
40         point(0.55f, 0.70f),
41         point(0.45f, 0.90f),
42         point(0.55f, 0.90f),
43     )
44
45     private fun point(x: Float, y: Float) = SkeletonPoint(x, y, 0.95f)
46 }
```

```

1 package com.runway.android.core.posture
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertNull
5 import org.junit.Assert.assertSame
6 import org.junit.Test
7
8 class PostureFrameInterpolatorTest {
9
10     // — bisectRight —
11
12     @Test
13     fun `bisectRight returns 0 for empty list`() {
14         assertEquals(0, bisectRight(emptyList(), 500L))
15     }
16
17     @Test
18     fun `bisectRight returns size when all frames are before posMs`() {
19         val frames = listOf(frame(100), frame(200), frame(300))
20         assertEquals(3, bisectRight(frames, 400L))
21     }
22
23     @Test
24     fun `bisectRight returns 0 when all frames are after posMs`() {
25         val frames = listOf(frame(100), frame(200), frame(300))
26         assertEquals(0, bisectRight(frames, 50L))
27     }
28
29     @Test
30     fun `bisectRight returns correct index for exact match`() {
31         // bisectRight: first index with t > posMs, so exact match goes to right side
32         val frames = listOf(frame(100), frame(200), frame(300))
33         assertEquals(2, bisectRight(frames, 200L))
34     }
35
36     @Test
37     fun `bisectRight returns correct index between frames`() {
38         val frames = listOf(frame(100), frame(200), frame(300))
39         // 150 is between index 0 (t=100) and index 1 (t=200)
40         assertEquals(1, bisectRight(frames, 150L))
41     }
42
43     // — interpolateFrame —
44
45     @Test
46     fun `interpolateFrame returns null for empty list`() {
47         assertNull(interpolateFrame(emptyList(), 100L))
48     }
49
50     @Test
51     fun `interpolateFrame returns only frame for single-element list`() {
52         val f = frame(100, 0.2f, 0.3f)
53         assertSame(f, interpolateFrame(listOf(f), 50L))
54     }
55
56     @Test
57     fun `interpolateFrame returns first frame when posMs before all frames`() {
58         val frames = listOf(frame(100, 0.1f, 0.1f), frame(200, 0.9f, 0.9f))
59         val result = interpolateFrame(frames, 50L)!!
60         assertEquals(0.1f, result.pts[0].x, 0.001f)
61     }
62
63     @Test
64     fun `interpolateFrame returns last frame when posMs after all frames`() {
65         val frames = listOf(frame(100, 0.1f, 0.1f), frame(200, 0.9f, 0.9f))
66         val result = interpolateFrame(frames, 300L)!!
67         assertEquals(0.9f, result.pts[0].x, 0.001f)
68     }
69
70     @Test
71     fun `interpolateFrame at exact midpoint gives 0.5 lerp`() {
72         val frames = listOf(frame(0, 0.0f, 0.0f), frame(200, 1.0f, 1.0f))
73         val result = interpolateFrame(frames, 100L)!!
74         assertEquals(0.5f, result.pts[0].x, 0.001f)
75         assertEquals(0.5f, result.pts[0].y, 0.001f)
76     }
77
78     @Test
79     fun `interpolateFrame at t=0.25 gives correct lerp`() {
80         val frames = listOf(frame(0, 0.0f, 0.0f), frame(400, 1.0f, 1.0f))
81         val result = interpolateFrame(frames, 100L)!!
82         assertEquals(0.25f, result.pts[0].x, 0.001f)
83     }

```

```

84
85     @Test
86     fun `interpolateFrame at frame boundary returns that frame x`() {
87         val a = frame(100, 0.2f, 0.3f)
88         val b = frame(200, 0.8f, 0.7f)
89         // posMs == a.t → t=0.0 → lerp returns a's values
90         val result = interpolateFrame(listOf(a, b), 100L)!!
91         assertEquals(0.2f, result.pts[0].x, 0.001f)
92     }
93
94     @Test
95     fun `interpolateFrame returns before when pts sizes differ`() {
96         val a = PostureVideoFrame(0L, listOf(SkeletonPoint(0f, 0f, 1f), SkeletonPoint(0f, 0f, 1f)))
97         val b = PostureVideoFrame(100L, listOf(SkeletonPoint(1f, 1f, 1f))) // different size
98         val result = interpolateFrame(listOf(a, b), 50L)!!
99         assertEquals(a, result)
100     }
101
102     @Test
103     fun `interpolateFrame result has posMs as timestamp`() {
104         val frames = listOf(frame(0, 0f, 0f), frame(200, 1f, 1f))
105         val result = interpolateFrame(frames, 75L)!!
106         assertEquals(75L, result.t)
107     }
108
109     // — duplicate timestamps —
110
111     @Test
112     fun `no duplicate timestamps when all frames have unique t`() {
113         val frames = listOf(frame(0), frame(100), frame(200), frame(300))
114         val timestamps = frames.map { it.t }
115         assertEquals(timestamps.size, timestamps.toSet().size)
116     }
117
118     // — helper —
119
120     private fun frame(t: Long, x: Float = 0f, y: Float = 0f) =
121         PostureVideoFrame(t, listOf(SkeletonPoint(x, y, 1f)))
122 }

```

```
1 package com.runway.android.core.posture
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertTrue
5 import org.junit.Test
6
7 class PostureLandmarkCorrectorTest {
8
9     @Test
10     fun `moves side-profile knees behind facing direction`() {
11         val rightFacing = landmarks(noseX = 0.62f)
12         val correctedRight = PostureLandmarkCorrector().correct(rightFacing, 0L)
13         assertTrue(correctedRight[SKEL_L_KNEE].x < rightFacing[SKEL_L_KNEE].x)
14         assertTrue(correctedRight[SKEL_R_KNEE].x < rightFacing[SKEL_R_KNEE].x)
15
16         val leftFacing = landmarks(noseX = 0.38f)
17         val correctedLeft = PostureLandmarkCorrector().correct(leftFacing, 0L)
18         assertTrue(correctedLeft[SKEL_L_KNEE].x > leftFacing[SKEL_L_KNEE].x)
19         assertTrue(correctedLeft[SKEL_R_KNEE].x > leftFacing[SKEL_R_KNEE].x)
20     }
21
22     @Test
23     fun `does not directionally shift knees in frontal view`() {
24         val frontal = landmarks(noseX = 0.5f)
25         val corrected = PostureLandmarkCorrector().correct(frontal, 0L)
26
27         assertEquals(frontal[SKEL_L_KNEE].x, corrected[SKEL_L_KNEE].x, 0.0001f)
28         assertEquals(frontal[SKEL_R_KNEE].x, corrected[SKEL_R_KNEE].x, 0.0001f)
29     }
30
31     private fun landmarks(noseX: Float): List<SkeletonPoint> = listOf(
32         point(noseX, 0.2f),
33         point(0.49f, 0.3f),
34         point(0.51f, 0.3f),
35         point(0.47f, 0.42f),
36         point(0.53f, 0.42f),
37         point(0.46f, 0.55f),
38         point(0.54f, 0.55f),
39         point(0.49f, 0.5f),
40         point(0.51f, 0.5f),
41         point(0.57f, 0.7f),
42         point(0.59f, 0.7f),
43         point(0.55f, 0.92f),
44         point(0.57f, 0.92f),
45     )
46
47     private fun point(x: Float, y: Float) = SkeletonPoint(x, y, 0.95f)
48 }
```



```
1 package com.runway.android.core.posture
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertTrue
5 import org.junit.Test
6
7 class PostureLandmarkValidatorTest {
8
9     @Test
10     fun `hides leg attached to a nearby background structure`() {
11         val points = normalPose().toMutableList().apply {
12             this[SKELETON_R_KNEE] = point(0.62f, 0.58f)
13             this[SKELETON_R_ANKLE] = point(0.74f, 0.62f)
14         }
15
16         val result = PostureLandmarkValidator().validate(points)
17
18         assertEquals(0f, result[SKELETON_R_KNEE].v, 0f)
19         assertEquals(0f, result[SKELETON_R_ANKLE].v, 0f)
20         assertTrue(result[SKELETON_L_KNEE].v > 0f)
21     }
22
23     @Test
24     fun `keeps balanced running legs visible`() {
25         val result = PostureLandmarkValidator().validate(normalPose())
26
27         assertTrue(result[SKELETON_L_KNEE].v > 0f)
28         assertTrue(result[SKELETON_R_KNEE].v > 0f)
29         assertTrue(result[SKELETON_L_ANKLE].v > 0f)
30         assertTrue(result[SKELETON_R_ANKLE].v > 0f)
31     }
32
33     @Test
34     fun `rejects a one frame ankle teleport`() {
35         val validator = PostureLandmarkValidator()
36         validator.validate(normalPose())
37         val jumped = normalPose().toMutableList().apply {
38             this[SKELETON_R_ANKLE] = point(0.95f, 0.20f)
39         }
40
41         val result = validator.validate(jumped)
42
43         assertEquals(0f, result[SKELETON_R_ANKLE].v, 0f)
44     }
45
46     @Test
47     fun `hides a high confidence leg that is too short for the torso`() {
48         val points = normalPose().toMutableList().apply {
49             this[SKELETON_R_KNEE] = point(0.55f, 0.55f)
50             this[SKELETON_R_ANKLE] = point(0.58f, 0.62f)
51         }
52
53         val result = PostureLandmarkValidator().validate(points)
54
55         assertEquals(0f, result[SKELETON_R_KNEE].v, 0f)
56         assertEquals(0f, result[SKELETON_R_ANKLE].v, 0f)
57     }
58
59     private fun normalPose(): List<SkeletonPoint> = listOf(
60         point(0.50f, 0.16f),
61         point(0.46f, 0.28f), point(0.54f, 0.28f),
62         point(0.43f, 0.40f), point(0.57f, 0.40f),
63         point(0.42f, 0.52f), point(0.58f, 0.52f),
64         point(0.47f, 0.48f), point(0.53f, 0.48f),
65         point(0.40f, 0.68f), point(0.60f, 0.68f),
66         point(0.34f, 0.88f), point(0.66f, 0.88f),
67     )
68
69     private fun point(x: Float, y: Float) = SkeletonPoint(x, y, 0.95f)
70 }
```

```

1  package com.runway.android.core.posture
2
3  import org.junit.Assert.assertEquals
4  import org.junit.Assert.assertTrue
5  import org.junit.Before
6  import org.junit.Test
7  import kotlin.math.PI
8  import kotlin.math.sin
9
10 class PostureRuleEngineTest {
11     private lateinit var engine: PostureRuleEngine
12
13     @Before
14     fun setup() {
15         engine = PostureRuleEngine()
16     }
17
18     @Test
19     fun `empty frames return score zero`() {
20         assertEquals(0, engine.evaluate(emptyList()).overallScore)
21     }
22
23     @Test
24     fun `original front knee threshold marks angle above 135 good`() {
25         val result = engine.evaluate(listOf(frame(knee = 140f)))
26         assertTrue(result.knee.score in 90..100)
27     }
28
29     @Test
30     fun `original front knee threshold marks 126 to 135 needs improvement`() {
31         val result = engine.evaluate(listOf(frame(knee = 130f)))
32         assertTrue(result.knee.score in 65..89)
33     }
34
35     @Test
36     fun `original front knee threshold marks below 126 bad`() {
37         val result = engine.evaluate(listOf(frame(knee = 120f)))
38         assertTrue(result.knee.score in 30..64)
39     }
40
41     @Test
42     fun `uses knee angle from detected foot strike`() {
43         val result = engine.evaluate(
44             listOf(
45                 frame(knee = 120f),
46                 frame(knee = 145f, leftStrike = true),
47             ),
48         )
49         assertEquals(145f, result.knee.measuredValue, 0.001f)
50         assertTrue(result.knee.score in 90..100)
51     }
52
53     @Test
54     fun `original torso threshold marks 5 to 15 good`() {
55         assertTrue(engine.evaluate(listOf(frame(trunk = 12f))).trunk.score in 90..100)
56         assertTrue(engine.evaluate(listOf(frame(trunk = 3f))).trunk.score in 65..89)
57         assertTrue(engine.evaluate(listOf(frame(trunk = 20f))).trunk.score in 30..64)
58     }
59
60     @Test
61     fun `original elbow threshold marks 60 to 90 good`() {
62         assertTrue(engine.evaluate(listOf(frame(elbow = 75f))).elbow.score in 90..100)
63         assertTrue(engine.evaluate(listOf(frame(elbow = 55f))).elbow.score in 65..89)
64         assertTrue(engine.evaluate(listOf(frame(elbow = 110f))).elbow.score in 30..64)
65     }
66
67     @Test
68     fun `original hip ankle and shank thresholds are evaluated at strike`() {
69         val good = engine.evaluate(
70             listOf(frame(hipAnkle = 10f, shank = 8f, leftStrike = true)),
71         )
72         val bad = engine.evaluate(
73             listOf(frame(hipAnkle = 24f, shank = 18f, leftStrike = true)),
74         )
75
76         assertTrue(good.overstride.score in 90..100)
77         assertTrue(bad.overstride.score in 30..64)
78     }
79
80     @Test
81     fun `hip mobility tracks left and right swing cycles independently`() {
82         val hips = listOf(
83             10f to -10f,

```

```

84         20f to -20f,
85         35f to -35f,
86         20f to -20f,
87     )
88     val frames = hips.map { (left, right) ->
89         frame().copy(leftHipAngle = left, rightHipAngle = right)
90     }
91
92     val hip = engine.evaluate(frames).hip
93
94     assertEquals(35f, hip.measuredValue, 0.001f)
95     assertTrue(hip.score in 90..100)
96 }
97
98 @Test
99 fun `cadence counts both ankle strike events`() {
100     val frames = listOf(
101         frame(timestamp = 0L, leftStrike = true),
102         frame(timestamp = 500L, rightStrike = true),
103         frame(timestamp = 1_000L, leftStrike = true),
104         frame(timestamp = 1_500L, rightStrike = true),
105         frame(timestamp = 2_000L),
106     )
107
108     val cadence = engine.evaluate(frames).cadence
109     assertEquals(120f, cadence.measuredValue, 0.001f)
110     assertTrue(cadence.score > 0)
111 }
112
113 @Test
114 fun `cadence uses alternating ankle phase instead of total video event count`() {
115     val frames = (0 until 90).map { index ->
116         val timestamp = index * (1_000L / 15L)
117         val seconds = timestamp / 1_000.0
118         val ankleDifference = (0.16 * sin(2.0 * PI * 1.5 * seconds)).toFloat()
119         frame(timestamp = timestamp).copy(
120             landmarks = ankleLandmarks(ankleDifference),
121         )
122     }
123
124     val cadence = engine.evaluate(frames).cadence
125
126     assertTrue(cadence.measuredValue in 175f..185f)
127     assertTrue(cadence.score > 0)
128 }
129
130 @Test
131 fun `cadence uses one reliable leg when the other leg is hidden`() {
132     val frames = (0 until 120).map { index ->
133         val timestamp = index * (1_000L / 15L)
134         val seconds = timestamp / 1_000.0
135         val ankleX = (0.20 * sin(2.0 * PI * 1.5 * seconds)).toFloat()
136         frame(timestamp = timestamp).copy(
137             landmarks = singleLegLandmarks(ankleX),
138         )
139     }
140
141     val cadence = engine.evaluate(frames).cadence
142
143     assertTrue(cadence.measuredValue in 175f..185f)
144 }
145
146 @Test
147 fun `legacy representative fields remain evaluable`() {
148     val legacy = PostureFrameAngles(
149         kneeFlexAngle = 140f,
150         trunkLeanAngle = 10f,
151         elbowAngle = 75f,
152         hipExtensionAngle = 35f,
153         overstrideRatio = 10f,
154         isLandingFrame = true,
155         visibility = 0.9f,
156         shankAngle = 8f,
157     )
158
159     val result = engine.evaluate(listOf(legacy))
160     assertTrue(result.knee.score in 90..100)
161     assertTrue(result.trunk.score in 90..100)
162     assertTrue(result.elbow.score in 90..100)
163     assertTrue(result.hip.score in 90..100)
164     assertTrue(result.overstride.score in 90..100)
165 }
166
167 private fun frame(
168     knee: Float = 140f,
169     trunk: Float = 10f,

```

```

170     elbow: Float = 75f,
171     hip: Float = 35f,
172     hipAnkle: Float = 10f,
173     shank: Float = 8f,
174     leftStrike: Boolean = false,
175     rightStrike: Boolean = false,
176     timestamp: Long = 0L,
177 ) = PostureFrameAngles(
178     kneeFlexAngle = knee,
179     trunkLeanAngle = trunk,
180     elbowAngle = elbow,
181     hipExtensionAngle = hip,
182     overstrideRatio = hipAnkle,
183     isLandingFrame = leftStrike || rightStrike,
184     visibility = 0.95f,
185     timestampMs = timestamp,
186     hipMidY = 0.5f,
187     nearAnkleY = 0.9f,
188     shankAngle = shank,
189     leftKneeAngle = knee,
190     leftElbowAngle = elbow,
191     leftHipAngle = hip,
192     leftHipAnkleAngle = hipAnkle,
193     leftShankAngle = shank,
194     leftFootStrike = leftStrike,
195     rightFootStrike = rightStrike,
196 )
197
198 private fun ankleLandmarks(difference: Float): List<SkeletonPoint> =
199     MutableList(13) { SkeletonPoint(0.5f, 0.5f, 0.95f) }.apply {
200         this[SKEL_L_ANKLE] = SkeletonPoint(0.45f, 0.75f + difference / 2f, 0.95f)
201         this[SKEL_R_ANKLE] = SkeletonPoint(0.55f, 0.75f - difference / 2f, 0.95f)
202     }
203
204 private fun singleLegLandmarks(ankleOffsetX: Float): List<SkeletonPoint> =
205     MutableList(13) { SkeletonPoint(0.5f, 0.5f, 0.95f) }.apply {
206         this[SKEL_L_HIP] = SkeletonPoint(0.5f, 0.5f, 0.95f)
207         this[SKEL_L_ANKLE] = SkeletonPoint(0.5f + ankleOffsetX, 0.85f, 0.95f)
208         this[SKEL_R_HIP] = SkeletonPoint(0.5f, 0.5f, 0f)
209         this[SKEL_R_ANKLE] = SkeletonPoint(0.5f, 0.5f, 0f)
210     }
211 }

```

```

1  package com.runway.android.core.posture
2
3  import org.junit.Assert.assertEquals
4  import org.junit.Assert.assertNotSame
5  import org.junit.Assert.assertTrue
6  import org.junit.Test
7
8  class PostureSkeletonSmootherTest {
9
10     @Test
11     fun `first smooth call preserves coordinates`() {
12         val raw = listOf(pt(0.5f, 0.5f, 0.9f), pt(0.1f, 0.2f, 0.8f))
13         val result = PostureSkeletonSmoother().smooth(raw, 0L)
14
15         assertEquals(raw[0].x, result[0].x, 0.0001f)
16         assertEquals(raw[0].y, result[0].y, 0.0001f)
17         assertEquals(raw[1].x, result[1].x, 0.0001f)
18         assertEquals(raw[1].y, result[1].y, 0.0001f)
19     }
20
21     @Test
22     fun `repeated visible samples converge toward input`() {
23         val smoother = PostureSkeletonSmoother()
24         smoother.smooth(listOf(pt(0f, 0f, 0.9f)), 0L)
25
26         var result = emptyList<SkeletonPoint>()
27         repeat(30) { index ->
28             result = smoother.smooth(listOf(pt(0.8f, 0.6f, 0.9f)), (index + 1) * 100L)
29         }
30
31         assertTrue(kotlin.math.abs(result[0].x - 0.8f) < 0.01f)
32         assertTrue(kotlin.math.abs(result[0].y - 0.6f) < 0.01f)
33     }
34
35     @Test
36     fun `low visibility sample stays at last reliable coordinate`() {
37         val smoother = PostureSkeletonSmoother(lowVisFreezeThreshold = 0.45f)
38         smoother.smooth(listOf(pt(0.2f, 0.3f, 0.9f)), 0L)
39         val result = smoother.smooth(listOf(pt(0.9f, 0.8f, 0.2f)), 100L)
40
41         assertEquals(0.2f, result[0].x, 0.0001f)
42         assertEquals(0.3f, result[0].y, 0.0001f)
43     }
44
45     @Test
46     fun `faster movement receives a more responsive cutoff`() {
47         val slow = PostureSkeletonSmoother(minCutoff = 1f, beta = 4f)
48         val fast = PostureSkeletonSmoother(minCutoff = 1f, beta = 4f)
49         slow.smooth(listOf(pt(0f, 0f, 0.9f)), 0L)
50         fast.smooth(listOf(pt(0f, 0f, 0.9f)), 0L)
51
52         val slowTarget = 0.02f
53         val fastTarget = 0.2f
54         val slowResult = slow.smooth(listOf(pt(slowTarget, 0f, 0.9f)), 100L)
55         val fastResult = fast.smooth(listOf(pt(fastTarget, 0f, 0.9f)), 100L)
56
57         assertTrue(fastResult[0].x / fastTarget > slowResult[0].x / slowTarget)
58     }
59
60     @Test
61     fun `reset causes next coordinates to pass through`() {
62         val smoother = PostureSkeletonSmoother()
63         smoother.smooth(listOf(pt(0f, 0f, 0.9f)), 0L)
64         smoother.reset()
65
66         val raw = pt(0.7f, 0.3f, 0.9f)
67         val result = smoother.smooth(listOf(raw), 100L)
68         assertEquals(raw.x, result[0].x, 0.0001f)
69         assertEquals(raw.y, result[0].y, 0.0001f)
70     }
71
72     @Test
73     fun `landmark count change reinitializes filters`() {
74         val smoother = PostureSkeletonSmoother()
75         smoother.smooth(listOf(pt(0f, 0f, 1f), pt(0f, 0f, 1f)), 0L)
76
77         val raw = pt(0.5f, 0.5f, 0.8f)
78         val result = smoother.smooth(listOf(raw), 100L)
79         assertEquals(raw.x, result[0].x, 0.0001f)
80         assertEquals(raw.y, result[0].y, 0.0001f)
81     }
82
83     @Test

```

```

84     fun `smooth returns a new list`() {
85         val smoother = PostureSkeletonSmoother()
86         val raw = listOf(pt(0f, 0f, 1f))
87         val result = smoother.smooth(raw, 0L)
88
89         assertNotSame(raw, result)
90     }
91
92     private fun pt(x: Float, y: Float, v: Float) = SkeletonPoint(x, y, v)
93 }

```

android/app/src/test/java/com/runway/android/core/posture/RunningFormStrikeDetectorTest.kt

Kotlin | 30줄

```

1  package com.runway.android.core.posture
2
3  import org.junit.Assert.assertFalse
4  import org.junit.Assert.assertTrue
5  import org.junit.Test
6
7  class RunningFormStrikeDetectorTest {
8
9      @Test
10     fun `detects original y-axis local minimum after upward swing`() {
11         val detector = RunningFormStrikeDetector(windowSize = 3)
12
13         assertFalse(detector.update(point(0.70f), 0L))
14         assertFalse(detector.update(point(0.65f), 500L))
15         assertTrue(detector.update(point(0.69f), 1_000L))
16     }
17
18     @Test
19     fun `suppresses strikes closer than 400 milliseconds`() {
20         val detector = RunningFormStrikeDetector(windowSize = 3)
21         detector.update(point(0.70f), 0L)
22         detector.update(point(0.65f), 100L)
23         assertTrue(detector.update(point(0.69f), 200L))
24
25         detector.update(point(0.64f), 300L)
26         assertFalse(detector.update(point(0.69f), 400L))
27     }
28
29     private fun point(y: Float) = SkeletonPoint(x = 0.5f, y = y, v = 0.9f)
30 }

```

android/app/src/test/java/com/runway/android/core/voice/RunningVoiceTextTest.kt

Kotlin | 14줄

```

1  package com.runway.android.core.voice
2
3  import org.junit.Assert.assertEquals
4  import org.junit.Test
5
6  class RunningVoiceTextTest {
7      @Test
8      fun `formats kilometer time and average pace naturally`() {
9          assertEquals(
10             "2킬로미터 완료. 시간 12분 20초. 평균 페이스 6분 10초.",
11             RunningVoiceText.kilometer(kilometers = 2, elapsedSeconds = 740),
12             )
13     }
14 }

```

```

1 package com.runway.android.core.wear
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertNull
5 import org.junit.Test
6
7 class WatchCommandParserTest {
8     @Test
9     fun `parses distance goal command`() {
10         val command = WatchCommandParser.parse(
11             """{"type":"START_DISTANCE_GOAL_RUN","targetMeters":5000}""".encodeToByteArray(),
12         )
13
14         assertEquals(
15             WatchCommand.StartDistanceGoalRun(
16                 targetMeters = 5000,
17                 completionAction = WatchGoalCompletionAction.CONTINUE,
18             ),
19             command,
20         )
21     }
22
23     @Test
24     fun `clamps interval values to supported range`() {
25         val command = WatchCommandParser.parse(
26             """{"type":"START_INTERVAL_RUN","workSeconds":10,"restSeconds":10,"sets":99}""".encodeToByteArray(),
27         )
28
29         assertEquals(
30             WatchCommand.StartIntervalRun(
31                 work = WatchIntervalTarget.Time(10),
32                 recovery = WatchIntervalTarget.Time(10),
33                 sets = 20,
34                 completionAction = WatchGoalCompletionAction.CONTINUE,
35             ),
36             command,
37         )
38     }
39
40     @Test
41     fun `parses distance interval and pause completion action`() {
42         val command = WatchCommandParser.parse(
43             """
44             {
45                 "type":"START_INTERVAL_RUN",
46                 "workType":"DISTANCE",
47                 "workMeters":400,
48                 "recoveryType":"DISTANCE",
49                 "recoveryMeters":200,
50                 "sets":6,
51                 "completionAction":"PAUSE"
52             }
53             """.trimIndent().encodeToByteArray(),
54         )
55
56         assertEquals(
57             WatchCommand.StartIntervalRun(
58                 work = WatchIntervalTarget.Distance(400),
59                 recovery = WatchIntervalTarget.Distance(200),
60                 sets = 6,
61                 completionAction = WatchGoalCompletionAction.PAUSE,
62             ),
63             command,
64         )
65     }
66
67     @Test
68     fun `rejects unknown or malformed command`() {
69         assertNull(WatchCommandParser.parse("""{"type":"UNKNOWN}""".encodeToByteArray()))
70         assertNull(WatchCommandParser.parse("not-json".encodeToByteArray()))
71     }
72 }
73

```

android/app/src/test/java/com/runway/android/ui/home/AirKoreaCoordinateConverterTest.kt

Kotlin | 17줄

```
1 package com.runway.android.ui.home
2
3 import org.junit.Assert.assertTrue
4 import org.junit.Test
5
6 class AirKoreaCoordinateConverterTest {
7     @Test
8     fun `converts Seoul City Hall coordinates to AirKorea TM`() {
9         val result = AirKoreaCoordinateConverter.toTm(
10             latitude = 37.5665,
11             longitude = 126.9780,
12         )
13
14         assertTrue("Unexpected TM x: ${result.x}", result.x in 197_000.0..199_000.0)
15         assertTrue("Unexpected TM y: ${result.y}", result.y in 449_000.0..453_000.0)
16     }
17 }
```

android/build.gradle.kts

Kotlin (Gradle) | 7줄

```
1 plugins {
2     id("com.android.application") version "8.13.2" apply false
3     id("org.jetbrains.kotlin.android") version "2.0.21" apply false
4     id("org.jetbrains.kotlin.plugin.compose") version "2.0.21" apply false
5     id("com.google.dagger.hilt.android") version "2.52" apply false
6     id("com.google.devtools.ksp") version "2.0.21-1.0.28" apply false
7 }
```

android/settings.gradle.kts

Kotlin (Gradle) | 24줄

```
1 pluginManagement {
2     repositories {
3         google {
4             content {
5                 includeGroupByRegex("com\\.\\.android\\.")
6                 includeGroupByRegex("com\\.\\.google\\.")
7                 includeGroupByRegex("androidx\\.")
8             }
9         }
10         mavenCentral()
11         gradlePluginPortal()
12     }
13 }
14
15 dependencyResolutionManagement {
16     repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
17     repositories {
18         google()
19         mavenCentral()
20     }
21 }
22
23 rootProject.name = "RunWay"
24 include(":app")
```



```
1  plugins {
2      id("com.android.application")
3      id("org.jetbrains.kotlin.android")
4      id("org.jetbrains.kotlin.plugin.compose")
5  }
6
7  android {
8      namespace = "com.runway.wear"
9      compileSdk = 35
10
11     defaultConfig {
12         // Wear Data Layer only connects apps with the same application ID and signing key.
13         applicationId = "com.runway.android"
14         minSdk = 30
15         targetSdk = 35
16         versionCode = 1
17         versionName = "1.0"
18     }
19
20     buildTypes {
21         release {
22             isMinifyEnabled = false
23             proguardFiles(
24                 getDefaultProguardFile("proguard-android-optimize.txt"),
25                 "proguard-rules.pro",
26             )
27         }
28     }
29
30     compileOptions {
31         sourceCompatibility = JavaVersion.VERSION_17
32         targetCompatibility = JavaVersion.VERSION_17
33     }
34
35     buildFeatures {
36         compose = true
37     }
38 }
39
40 kotlin {
41     compilerOptions {
42         jvmTarget.set(org.jetbrains.kotlin.gradle.dsl.JvmTarget.JVM_17)
43     }
44 }
45
46 dependencies {
47     val composeBom = platform("androidx.compose:compose-bom:2024.10.01")
48     implementation(composeBom)
49     implementation("androidx.compose.ui:ui")
50     implementation("androidx.compose.ui:ui-tooling-preview")
51     implementation("androidx.compose.foundation:foundation")
52     implementation("androidx.compose.material3:material3")
53     implementation("androidx.compose.material:material-icons-extended")
54
55     implementation("androidx.activity:activity-compose:1.9.3")
56     implementation("androidx.concurrent:concurrent-futures-ktx:1.2.0")
57     implementation("androidx.core:core-ktx:1.15.0")
58     implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.8.7")
59     implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.7")
60     implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.7")
61
62     implementation("androidx.wear:wear:1.3.0")
63     implementation("androidx.wear:wear-remote-interactions:1.1.0")
64     implementation("androidx.wear.compose:compose-material3:1.6.2")
65     implementation("androidx.health:health-services-client:1.1.0-rc01")
66     implementation("com.google.android.gms:play-services-wearable:19.0.0")
67     implementation("com.google.android.gms:play-services-location:21.3.0")
68     implementation("com.google.guava:guava:33.3.1-android")
69     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.9.0")
70     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-play-services:1.9.0")
71
72     testImplementation("junit:junit:4.13.2")
73     debugImplementation("androidx.compose.ui:ui-tooling")
74 }
```

```
1 package com.runway.wear
2
3 import android.Manifest
4 import android.content.Intent
5 import android.os.Bundle
6 import android.view.WindowManager
7 import androidx.activity.ComponentActivity
8 import androidx.activity.compose.rememberLauncherForActivityResult
9 import androidx.activity.compose.setContent
10 import androidx.activity.result.contract.ActivityResultContracts
11 import androidx.compose.runtime.LaunchedEffect
12 import androidx.compose.runtime.getValue
13 import androidx.compose.runtime.mutableIntStateOf
14 import androidx.compose.runtime.mutableStateOf
15 import androidx.compose.runtime.setValue
16 import androidx.lifecycle.Lifecycle
17 import androidx.lifecycle.compose.LocalLifecycleOwner
18 import androidx.lifecycle.compose.currentStateAsState
19 import androidx.lifecycle.viewmodel.compose.viewModel
20 import androidx.wear.ambient.AmbientLifecycleObserver
21 import com.runway.wear.ui.PathFinderWearApp
22
23 class MainActivity : ComponentActivity() {
24     private var launchToken by mutableIntStateOf(0)
25     private var isAmbient by mutableStateOf(false)
26
27     private val ambientObserver = AmbientLifecycleObserver(
28         this,
29         object : AmbientLifecycleObserver.AmbientLifecycleCallback {
30             override fun onEnterAmbient(ambientDetails: AmbientLifecycleObserver.AmbientDetails) {
31                 isAmbient = true
32             }
33             override fun onExitAmbient() {
34                 isAmbient = false
35             }
36             override fun onUpdateAmbient() {}
37         }
38     )
39
40     override fun onCreate(savedInstanceState: Bundle?) {
41         super.onCreate(savedInstanceState)
42         lifecycle.addObserver(ambientObserver)
43         // 앱이 포그라운드에서 있는 동안 항상 화면 유지
44         window.addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)
45         if (savedInstanceState == null) launchToken++
46         setContent {
47             val watchViewModel: WatchViewModel = viewModel()
48             val lifecycleState = LocalLifecycleOwner.current.lifecycle.currentStateAsState()
49             val permissionLauncher = rememberLauncherForActivityResult(
50                 ActivityResultContracts.RequestMultiplePermissions(),
51             ) { }
52             LaunchedEffect(Unit) {
53                 permissionLauncher.launch(
54                     arrayOf(
55                         Manifest.permission.ACCESS_FINE_LOCATION,
56                         Manifest.permission.ACTIVITY_RECOGNITION,
57                         Manifest.permission.BODY_SENSORS,
58                     ),
59                 )
60             }
61             LaunchedEffect(lifecycleState.value) {
62                 if (lifecycleState.value == Lifecycle.State.RESUMED) {
63                     watchViewModel.refreshConnection()
64                 }
65             }
66             PathFinderWearApp(watchViewModel, launchToken, isAmbient)
67         }
68     }
69
70     override fun onNewIntent(intent: Intent) {
71         super.onNewIntent(intent)
72         launchToken++
73     }
74
75
76 }
```

```
1 package com.runway.wear
2
3 import android.app.NotificationChannel
4 import android.app.NotificationManager
5 import android.app.Service
6 import android.content.Context
7 import android.content.Intent
8 import android.os.Build
9 import android.os.IBinder
10 import androidx.core.app.NotificationCompat
11
12 class RunForegroundService : Service() {
13
14     override fun onBind(intent: Intent?): IBinder? = null
15
16     override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
17         if (intent?.action == ACTION_STOP) {
18             stopForeground(STOP_FOREGROUND_REMOVE)
19             stopSelf()
20             return START_NOT_STICKY
21         }
22         createChannel()
23         val notification = NotificationCompat.Builder(this, CHANNEL_ID)
24             .setContentTitle("러닝 진행 중")
25             .setContentText("PathFinder가 러닝을 기록하고 있습니다")
26             .setSmallIcon(R.drawable.app_logo_mark)
27             .setOngoing(true)
28             .setSilent(true)
29             .build()
30         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
31             startForeground(NOTIF_ID, notification, android.content.pm.ServiceInfo.FOREGROUND_SERVICE_TYPE_LOCATION)
32         } else {
33             startForeground(NOTIF_ID, notification)
34         }
35         return START_STICKY
36     }
37
38     private fun createChannel() {
39         val manager = getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
40         if (manager.getNotificationChannel(CHANNEL_ID) == null) {
41             manager.createNotificationChannel(
42                 NotificationChannel(CHANNEL_ID, "러닝 기록", NotificationManager.IMPORTANCE_LOW)
43             )
44         }
45     }
46
47     companion object {
48         const val ACTION_STOP = "STOP"
49         const val CHANNEL_ID = "run_foreground"
50         const val NOTIF_ID = 1001
51
52         fun start(context: Context) {
53             val intent = Intent(context, RunForegroundService::class.java)
54             context.startForegroundService(intent)
55         }
56
57         fun stop(context: Context) {
58             val intent = Intent(context, RunForegroundService::class.java).apply {
59                 action = ACTION_STOP
60             }
61             context.startService(intent)
62         }
63     }
64 }
```

```

1  package com.runway.wear
2
3  import android.app.Application
4  import android.annotation.SuppressLint
5  import android.os.HandlerThread
6  import android.os.Looper
7  import androidx.lifecycle.AndroidViewModel
8  import androidx.lifecycle.ViewModelScope
9  import com.runway.wear.data.WatchDataLayerClient
10 import com.runway.wear.data.PendingWatchRun
11 import com.runway.wear.data.PendingWatchRunStore
12 import com.runway.wear.data.WatchRunPoint
13 import com.runway.wear.data.WatchRunSyncRepository
14 import com.runway.wear.data.OfflineCourse
15 import com.runway.wear.data.OfflineCourseRepository
16 import com.runway.wear.data.OfflineCourseStore
17 import com.google.android.gms.location.LocationServices
18 import com.google.android.gms.location.Priority
19 import com.google.android.gms.location.LocationCallback
20 import com.google.android.gms.location.LocationRequest
21 import com.google.android.gms.location.LocationResult
22 import com.runway.wear.data.PhoneAuthStateRepository
23 import com.runway.wear.data.PhoneRunStateRepository
24 import com.runway.wear.data.WatchSettings
25 import com.runway.wear.data.WatchSettingsStore
26 import com.runway.wear.health.HealthServicesManager
27 import com.runway.wear.model.GoalCompletionAction
28 import com.runway.wear.model.IntervalTarget
29 import com.runway.wear.model.RunGoal
30 import com.runway.wear.model.WatchRunState
31 import com.runway.wear.model.WatchScreen
32 import com.runway.wear.voice.RunningVoiceGuide
33 import kotlinx.coroutines.Dispatchers
34 import kotlinx.coroutines.Job
35 import kotlinx.coroutines.delay
36 import kotlinx.coroutines.flow.MutableStateFlow
37 import kotlinx.coroutines.flow.StateFlow
38 import kotlinx.coroutines.flow.asStateFlow
39 import kotlinx.coroutines.flow.update
40 import kotlinx.coroutines.launch
41 import kotlinx.coroutines.withContext
42 import kotlinx.coroutines.tasks.await as awaitTask
43 import java.time.Instant
44
45 class WatchViewModel(application: Application) : AndroidViewModel(application) {
46     private val dataLayer = WatchDataLayerClient(application)
47     private val healthServices = HealthServicesManager(application)
48     private val voiceGuide = RunningVoiceGuide(application)
49     private val settingsStore = WatchSettingsStore(application)
50     private val pendingRunStore = PendingWatchRunStore(application)
51     private val offlineCourseStore = OfflineCourseStore(application)
52     private val fusedLocationClient = LocationServices.getFusedLocationProviderClient(application)
53
54     private val initialSettings = settingsStore.load()
55     private val _state = MutableStateFlow(
56         WatchRunState(
57             voiceGuidanceEnabled = initialSettings.voiceGuidanceEnabled,
58             autoPauseEnabled = initialSettings.autoPauseEnabled,
59             goalCompletionAction = initialSettings.goalCompletionAction,
60         ),
61     )
62     val state: StateFlow<WatchRunState> = _state.asStateFlow()
63
64     private var timerJob: Job? = null
65     private var metricsJob: Job? = null
66     private var gpsAutoPauseJob: Job? = null
67     private var intervalSegmentStartSeconds = 0L
68     private var intervalSegmentStartMeters = 0.0
69     private var lastAnnouncedKilometer = 0
70     private var settingsReturnScreen = WatchScreen.HOME
71     private var lastAutoPaused = false
72     private var lastGpsStatus = "SEARCHING"
73     private var runStartedAt: Instant? = null
74     private val runPoints = mutableListOf<WatchRunPoint>()
75     private var activeCourse: OfflineCourse? = null
76     private var lastCourseWarning = false
77     private var pausedByCourseDeviation = false
78     private var pausedByGpsInactivity = false
79     private var lowSpeedTicks = 0
80     private var highSpeedTicks = 0
81     private var gpsLocationCallback: LocationCallback? = null
82     private var homeGpsCallback: LocationCallback? = null
83     private var deviationGpsCallback: LocationCallback? = null

```

```

84 private val gpsHandlerThread = HandlerThread("gps-auto-pause").also { it.start() }
85 private var lastReadyLocation: android.location.Location? = null
86
87 init {
88     viewModelScope.launch(Dispatchers.IO) {
89         OfflineCourseRepository.update(offlineCourseStore.load())
90     }
91     refreshConnection()
92     observePhoneState()
93     observeRunSync()
94     observeOfflineCourses()
95     observePhoneAuthState()
96     startHomeGpsMonitor()
97 }
98
99 fun refreshConnection() {
100     viewModelScope.launch {
101         val connected = dataLayer.isPhoneConnected()
102         _state.update { it.copy(isPhoneConnected = connected) }
103         if (connected) {
104             dataLayer.requestAuthState()
105             dataLayer.syncPendingRuns(pendingRunStore)
106         }
107     }
108 }
109
110 fun navigate(screen: WatchScreen) {
111     _state.update { it.copy(screen = screen) }
112 }
113
114 fun navigateBack() {
115     when (_state.value.screen) {
116         WatchScreen.HOME -> Unit
117         WatchScreen.COURSE_LIST -> navigate(WatchScreen.HOME)
118         WatchScreen.COURSE_DETAIL -> navigate(WatchScreen.COURSE_LIST)
119         WatchScreen.GOAL_TYPE -> navigate(WatchScreen.HOME)
120         WatchScreen.TIME_GOAL,
121         WatchScreen.DISTANCE_GOAL,
122         WatchScreen.INTERVAL_GOAL,
123         -> navigate(WatchScreen.GOAL_TYPE)
124         WatchScreen.SETTINGS -> navigate(settingsReturnScreen)
125         WatchScreen.PREPARING -> cancelPreparing()
126         WatchScreen.COUNTDOWN -> cancelCountdown()
127         WatchScreen.TRACKING -> pause()
128         WatchScreen.PAUSED -> navigate(WatchScreen.TRACKING)
129         WatchScreen.SUMMARY -> returnHome()
130     }
131 }
132
133 fun openSettings() {
134     settingsReturnScreen = _state.value.screen
135     navigate(WatchScreen.SETTINGS)
136 }
137
138 fun setVoiceGuidanceEnabled(enabled: Boolean) {
139     _state.update { it.copy(voiceGuidanceEnabled = enabled) }
140     if (!enabled) voiceGuide.stopGuidance()
141     persistSettings()
142 }
143
144 fun setAutoPauseEnabled(enabled: Boolean) {
145     _state.update { it.copy(autoPauseEnabled = enabled) }
146     persistSettings()
147 }
148
149 fun setGoalCompletionAction(action: GoalCompletionAction) {
150     _state.update { it.copy(goalCompletionAction = action) }
151     persistSettings()
152 }
153
154 fun start(goal: RunGoal) {
155     activeCourse = null
156     _state.update { it.copy(screen = WatchScreen.PREPARING, pendingGoal = goal) }
157 }
158
159 fun confirmPreparing() {
160     _state.update { it.copy(screen = WatchScreen.COUNTDOWN) }
161 }
162
163 fun cancelPreparing() {
164     _state.update { it.copy(screen = WatchScreen.HOME, pendingGoal = null) }
165 }
166
167 fun commitCountdown() {
168     val goal = _state.value.pendingGoal ?: RunGoal.Free
169     _state.update { it.copy(pendingGoal = null) }

```

```

170     startTracking(goal)
171 }
172
173 fun cancelCountdown() {
174     _state.update { it.copy(screen = WatchScreen.HOME, pendingGoal = null) }
175 }
176
177 fun selectCourse(courseId: String) {
178     val course = OfflineCourseRepository.courses.value.firstOrNull { it.courseId == courseId }
179     ?: return
180     _state.update {
181         it.copy(
182             screen = WatchScreen.COURSE_DETAIL,
183             selectedCourseId = course.courseId,
184             courseName = course.name,
185             courseDistanceMeters = course.distanceMeters,
186         )
187     }
188 }
189
190 fun startSelectedCourse() {
191     val courseId = _state.value.selectedCourseId ?: return
192     activeCourse = OfflineCourseRepository.courses.value.firstOrNull { it.courseId == courseId }
193     ?: return
194     lastCourseWarning = false
195     pausedByCourseDeviation = false
196     _state.update { it.copy(screen = WatchScreen.PREPARING, pendingGoal = RunGoal.Free) }
197 }
198
199 @SuppressLint("MissingPermission")
200 fun syncNearbyCourses() {
201     if (!_state.value.isPhoneConnected) {
202         _state.update { it.copy(phoneStatusMessage = "주변 코스 저장은 폰 연결이 필요합니다") }
203         return
204     }
205     _state.update { it.copy(isSyncingCourses = true, phoneStatusMessage = "현재 위치 확인 중") }
206     viewModelScope.launch {
207         val location = runCatching {
208             fusedLocationClient.getCurrentLocation(Priority.PRIORITY_HIGH_ACCURACY, null).awaitTask()
209         }.getOrNull()
210         if (location == null) {
211             _state.update {
212                 it.copy(isSyncingCourses = false, phoneStatusMessage = "위치 위치를 확인하지 못했습니다")
213             }
214             return@launch
215         }
216         val sent = dataLayer.requestNearbyCourses(location.latitude, location.longitude)
217         _state.update {
218             it.copy(
219                 isSyncingCourses = sent,
220                 phoneStatusMessage = if (sent) "주변 코스를 불러오는 중" else "폰 연결을 확인해 주세요",
221             )
222         }
223     }
224 }
225
226 private fun startTracking(goal: RunGoal) {
227     runStartedAt = Instant.now()
228     runPoints.clear()
229     intervalSegmentStartSeconds = 0L
230     intervalSegmentStartMeters = 0.0
231     lastAnnouncedKilometer = 0
232     lastAutoPaused = false
233     lastGpsStatus = "SEARCHING"
234     nearFinishAnnounced = false
235     lastCourseWarning = false
236     pausedByCourseDeviation = false
237     maxCourseProgressMeters = 0.0
238     resetGpsAutoPause()
239     _state.update {
240         WatchRunState(
241             screen = WatchScreen.TRACKING,
242             goal = goal,
243             isPhoneConnected = it.isPhoneConnected,
244             voiceGuidanceEnabled = it.voiceGuidanceEnabled,
245             autoPauseEnabled = it.autoPauseEnabled,
246             goalCompletionAction = goal.completionAction,
247             isRunning = true,
248             selectedCourseId = activeCourse?.courseId,
249             courseName = activeCourse?.name,
250             courseDistanceMeters = activeCourse?.distanceMeters ?: 0.0,
251         )
252     }
253     // GPS 준비된 위치를 첫 번째 포인트로 즉시 기록
254     lastReadyLocation?.let { loc ->
255         runPoints.add(

```

```

256         WatchRunPoint(
257             sequence = 0,
258             latitude = loc.latitude,
259             longitude = loc.longitude,
260             altitudeMeters = loc.altitude,
261             speedMps = 0.0,
262             recordedAt = Instant.now().toString(),
263         )
264     )
265 }
266 observeMetrics()
267 viewModelScope.launch {
268     runCatching { healthServices.start(_state.value.autoPauseEnabled) }
269     // 코스 도전 시: 출발 위치가 경로 밖이면 exercise 시작 직후 즉시 일시정지
270     val course = activeCourse
271     val loc = lastReadyLocation
272     if (course != null && loc != null) {
273         val proximity = courseProximity(loc.latitude, loc.longitude, course)
274         if (proximity != null && proximity.nearestMeters > COURSE_OFF_THRESHOLD_METERS) {
275             delay(300) // exercise 초기화 대기
276             lastCourseWarning = true
277             pausedByCourseDeviation = true
278             _state.update {
279                 it.copy(
280                     screen = WatchScreen.PAUSED,
281                     isPaused = true,
282                     isOffCourse = true,
283                     distanceToCourseMeters = proximity.nearestMeters,
284                 )
285             }
286             speakIfEnabled { offCourse() }
287             runCatching { healthServices.pause() }
288             startDeviationMonitor()
289         }
290     }
291 }
292 speakIfEnabled { start() }
293 startTimer()
294 RunForegroundService.start(getApplication())
295 }
296
297 fun pause() {
298     pausedByGpsInactivity = false
299     _state.update { it.copy(screen = WatchScreen.PAUSED, isPaused = true) }
300     viewModelScope.launch {
301         runCatching { healthServices.pause() }
302     }
303 }
304
305 fun resume() {
306     // 경로 이탈로 정지된 경우 수동 재개 불가 - deviation monitor가 자동으로 재개
307     if (pausedByCourseDeviation) return
308     pausedByGpsInactivity = false
309     resetGpsAutoPause()
310     _state.update { it.copy(screen = WatchScreen.TRACKING, isPaused = false) }
311     viewModelScope.launch {
312         runCatching { healthServices.resume() }
313     }
314 }
315
316 fun finish() {
317     timerJob?.cancel()
318     stopGpsAutoPauseMonitor()
319     stopDeviationMonitor()
320     speakIfEnabled { finish() }
321     val completed = _state.value
322     _state.update {
323         it.copy(
324             screen = WatchScreen.SUMMARY,
325             isRunning = false,
326             isPaused = false,
327             phoneStatusMessage = "폰 연결 시 자동으로 동기화됩니다",
328         )
329     }
330     RunForegroundService.stop(getApplication())
331     viewModelScope.launch {
332         runCatching { healthServices.finish() }
333         val durationSeconds = completed.elapsedSeconds.coerceAtMost(Int.MAX_VALUE.toLong()).toInt()
334         val distanceKm = completed.distanceMeters / 1000.0
335         pendingRunStore.add(
336             PendingWatchRun.create(
337                 startedAt = (runStartedAt ?: Instant.now()).toString(),
338                 courseId = activeCourse?.courseId,
339                 endedAt = Instant.now().toString(),
340                 distanceMeters = completed.distanceMeters,
341                 durationSeconds = durationSeconds,

```

```

342         avgPaceSecondsPerKm = if (distanceKm > 0.001) {
343             (durationSeconds / distanceKm).toInt()
344         } else {
345             0
346         },
347         caloriesBurned = (distanceKm * 72).toInt(),
348         avgHeartRateBpm = completed.heartRateBpm,
349         points = runPoints.toList(),
350     ),
351 )
352 val sent = dataLayer.syncPendingRuns(pendingRunStore)
353 if (sent > 0) {
354     _state.update { it.copy(phoneStatusMessage = "폰으로 기록을 전송했습니다") }
355 }
356 }
357 }
358
359 fun abandon() {
360     timerJob?.cancel()
361     metricsJob?.cancel()
362     stopGpsAutoPauseMonitor()
363     stopDeviationMonitor()
364     speakIfEnabled { cancel() }
365     RunForegroundService.stop(getApplication())
366     viewModelScope.launch {
367         runCatching { healthServices.finish() }
368     }
369     _state.value = freshAuthenticatedState()
370 }
371
372 fun returnHome() {
373     metricsJob?.cancel()
374     stopGpsAutoPauseMonitor()
375     stopDeviationMonitor()
376     RunForegroundService.stop(getApplication())
377     _state.value = freshAuthenticatedState()
378 }
379
380 private fun startTimer() {
381     timerJob?.cancel()
382     timerJob = viewModelScope.launch {
383         while (true) {
384             delay(1_000)
385             _state.update { current ->
386                 if (current.isRunning && !current.isPaused) {
387                     current.copy(elapsedSeconds = current.elapsedSeconds + 1)
388                 } else {
389                     current
390                 }
391             }
392             updateGoalProgress()
393         }
394     }
395 }
396
397 @SuppressLint("MissingPermission")
398 private fun startGpsAutoPauseMonitor() {
399     stopGpsAutoPauseMonitor()
400     val request = LocationRequest.Builder(
401         Priority.PRIORITY_HIGH_ACCURACY,
402         GPS_SAMPLE_INTERVAL_MILLIS,
403     )
404     .setMinUpdateIntervalMillis(GPS_SAMPLE_INTERVAL_MILLIS)
405     .build()
406     val callback = object : LocationCallback() {
407         override fun onLocationResult(result: LocationResult) {
408             result.lastLocation?.let { location ->
409                 handleSpeedTick(location.speed.toDouble())
410             }
411         }
412     }
413     gpsLocationCallback = callback
414     fusedLocationClient.requestLocationUpdates(request, callback, gpsHandlerThread.looper)
415 }
416
417 private fun handleSpeedTick(speedMps: Double) {
418     val current = _state.value
419     if (!current.autoPauseEnabled || !current.isRunning) return
420
421     if (!current.isPaused) {
422         if (speedMps < AUTO_PAUSE_SPEED_MPS) {
423             lowSpeedTicks++
424             highSpeedTicks = 0
425         } else {
426             lowSpeedTicks = 0
427             highSpeedTicks = 0

```



```

428     }
429     if (lowSpeedTicks >= AUTO_PAUSE_TICKS) {
430         lowSpeedTicks = 0
431         pausedByGpsInactivity = true
432         _state.update { it.copy(screen = WatchScreen.TRACKING, isPaused = true) }
433         viewModelScope.launch { runCatching { healthServices.pause() } }
434         speakIfEnabled { autoPaused() }
435     }
436 } else if (pausedByGpsInactivity && !pausedByCourseDeviation) {
437     if (speedMps > AUTO_RESUME_SPEED_MPS) {
438         highSpeedTicks++
439         if (highSpeedTicks >= AUTO_RESUME_TICKS) {
440             highSpeedTicks = 0
441             lowSpeedTicks = 0
442             pausedByGpsInactivity = false
443             _state.update { it.copy(screen = WatchScreen.TRACKING, isPaused = false) }
444             viewModelScope.launch { runCatching { healthServices.resume() } }
445             speakIfEnabled { autoResumed() }
446         }
447     } else {
448         highSpeedTicks = 0
449     }
450 }
451 }
452
453 private fun resetGpsAutoPause() {
454     pausedByGpsInactivity = false
455     lowSpeedTicks = 0
456     highSpeedTicks = 0
457 }
458
459 @SuppressLint("MissingPermission")
460 private fun startHomeGpsMonitor() {
461     // 마지막으로 알려진 위치가 있으면 즉시 표시
462     runCatching {
463         fusedLocationClient.lastLocation.addOnSuccessListener { location ->
464             if (location != null && location.accuracy <= GPS_READY_ACCURACY_METERS) {
465                 _state.update { it.copy(gpsReady = true) }
466             }
467         }
468     }
469
470     val request = LocationRequest.Builder(Priority.PRIORITY_HIGH_ACCURACY, 1_000L)
471         .setMinUpdateIntervalMillis(500L)
472         .setWaitForAccurateLocation(false)
473         .build()
474     val callback = object : LocationCallback() {
475         override fun onLocationResult(result: LocationResult) {
476             val location = result.lastLocation ?: return
477             if (location.accuracy <= GPS_READY_ACCURACY_METERS) {
478                 lastReadyLocation = location
479                 _state.update { it.copy(gpsReady = true) }
480             }
481         }
482     }
483     homeGpsCallback = callback
484     runCatching {
485         fusedLocationClient.requestLocationUpdates(request, callback, gpsHandlerThread.looper)
486     }
487 }
488
489 private fun stopHomeGpsMonitor() {
490     homeGpsCallback?.let { fusedLocationClient.removeLocationUpdates(it) }
491     homeGpsCallback = null
492 }
493
494 @SuppressLint("MissingPermission")
495 private fun startDeviationMonitor() {
496     stopDeviationMonitor()
497     val course = activeCourse ?: return
498     val request = LocationRequest.Builder(Priority.PRIORITY_HIGH_ACCURACY, 2_000L)
499         .setMinUpdateIntervalMillis(1_000L)
500         .build()
501     val callback = object : LocationCallback() {
502         override fun onLocationResult(result: LocationResult) {
503             val loc = result.lastLocation ?: return
504             val proximity = courseProximity(loc.latitude, loc.longitude, course) ?: return
505             val backOnCourse = proximity.nearestMeters <= COURSE_OFF_THRESHOLD_METERS
506             _state.update { it.copy(distanceToCourseMeters = proximity.nearestMeters, isOffCourse = !backOnCourse) }
507             if (backOnCourse && pausedByCourseDeviation) {
508                 stopDeviationMonitor()
509                 lastCourseWarning = false
510                 pausedByCourseDeviation = false
511                 speakIfEnabled { backOnCourse() }
512                 resume()
513             }
514         }
515     }

```

```

514     }
515 }
516 deviationGpsCallback = callback
517 runCatching {
518     fusedLocationClient.requestLocationUpdates(request, callback, gpsHandlerThread.looper)
519 }
520 }
521
522 private fun stopDeviationMonitor() {
523     deviationGpsCallback?.let { fusedLocationClient.removeLocationUpdates(it) }
524     deviationGpsCallback = null
525 }
526
527 private fun stopGpsAutoPauseMonitor() {
528     gpsAutoPauseJob?.cancel()
529     gpsAutoPauseJob = null
530     gpsLocationCallback?.let { fusedLocationClient.removeLocationUpdates(it) }
531     gpsLocationCallback = null
532     resetGpsAutoPause()
533 }
534
535 private fun observeMetrics() {
536     metricsJob?.cancel()
537     metricsJob = viewModelScope.launch {
538         healthServices.updates.collect { update ->
539             update.locations.forEach { location ->
540                 runPoints += WatchRunPoint(
541                     sequence = runPoints.size,
542                     latitude = location.latitude,
543                     longitude = location.longitude,
544                     altitudeMeters = location.altitudeMeters,
545                     speedMps = location.speedMps,
546                     recordedAt = location.recordedAt,
547                 )
548                 withContext(Dispatchers.Default) {
549                     updateCourseProgress(location.latitude, location.longitude)
550                 }
551             }
552             _state.update { current ->
553                 val distance = update.distanceMeters ?: current.distanceMeters
554                 val elapsedMinutes = current.elapsedSeconds / 60f
555                 val pace = if (distance >= 20 && elapsedMinutes > 0) {
556                     (elapsedMinutes / (distance / 1000f)).toFloat()
557                 } else {
558                     current.paceMinPerKm
559                 }
560                 current.copy(
561                     distanceMeters = distance,
562                     paceMinPerKm = pace,
563                     heartRateBpm = update.heartRateBpm ?: current.heartRateBpm,
564                     cadenceSpm = update.cadenceSpm ?: current.cadenceSpm,
565                     gpsStatus = update.gpsStatus ?: current.gpsStatus,
566                 )
567             }
568             updateAutoPause(update.isAutoPaused)
569             updateGpsVoice(update.gpsStatus)
570             announceCompletedKilometer()
571             updateGoalProgress()
572         }
573     }
574 }
575
576 private fun updateGoalProgress() {
577     val current = _state.value
578     if (!current.isRunning || current.isPaused || current.goalCompleted) return
579
580     when (val goal = current.goal) {
581         RunGoal.Free -> Unit
582         is RunGoal.Time -> {
583             if (current.elapsedSeconds >= goal.minutes * 60L) completeGoal(goal.completionAction)
584         }
585         is RunGoal.Distance -> {
586             if (current.distanceMeters >= goal.meters) completeGoal(goal.completionAction)
587         }
588         is RunGoal.Interval -> updateIntervalProgress(current, goal)
589     }
590 }
591
592 private fun updateIntervalProgress(current: WatchRunState, goal: RunGoal.Interval) {
593     val target = if (current.intervalIsWork) goal.work else goal.recovery
594     val progress = when (target) {
595         is IntervalTarget.Time -> {
596             val elapsed = current.elapsedSeconds - intervalSegmentStartSeconds
597             ((elapsed * 100) / target.seconds.coerceAtLeast(1)).toInt()
598         }
599         is IntervalTarget.Distance -> {

```

```

600         val distance = current.distanceMeters - intervalSegmentStartMeters
601         ((distance * 100) / target.meters.coerceAtLeast(1)).toInt()
602     }
603     }.coerceIn(0, 100)
604
605     val remaining = when (target) {
606     is IntervalTarget.Time -> {
607         val elapsed = current.elapsedSeconds - intervalSegmentStartSeconds
608         "${(target.seconds - elapsed).coerceAtLeast(0)}초 남음"
609     }
610     is IntervalTarget.Distance -> {
611         val distance = current.distanceMeters - intervalSegmentStartMeters
612         "${(target.meters - distance).coerceAtLeast(0.0).toInt()}m 남음"
613     }
614     }
615
616     _state.update {
617         it.copy(
618             intervalSegmentProgress = progress,
619             intervalRemainingLabel = remaining,
620         )
621     }
622
623     if (progress < 100) return
624     if (current.intervalIsWork) {
625         startNextIntervalSegment(isWork = false, step = current.intervalStep)
626     } else if (current.intervalStep < goal.sets) {
627         startNextIntervalSegment(isWork = true, step = current.intervalStep + 1)
628     } else {
629         completeGoal(goal.completionAction)
630     }
631 }
632
633 private fun startNextIntervalSegment(isWork: Boolean, step: Int) {
634     val current = _state.value
635     intervalSegmentStartSeconds = current.elapsedSeconds
636     intervalSegmentStartMeters = current.distanceMeters
637     _state.update {
638         it.copy(
639             intervalIsWork = isWork,
640             intervalStep = step,
641             intervalSegmentProgress = 0,
642         )
643     }
644     speakIfEnabled { intervalChanged(isWork, step) }
645 }
646
647 private fun completeGoal(@Suppress("UNUSED_PARAMETER") action: GoalCompletionAction) {
648     _state.update { it.copy(goalCompleted = true) }
649     val selectedAction = _state.value.goalCompletionAction
650     speakIfEnabled {
651         goalCompleted(selectedAction == GoalCompletionAction.CONTINUE)
652     }
653     if (selectedAction == GoalCompletionAction.PAUSE) pause()
654 }
655
656 private fun observePhoneState() {
657     viewModelScope.launch {
658         PhoneRunStateRepository.state.collect { phoneState ->
659             phoneState ?: return@collect
660             _state.update { current ->
661                 current.copy(
662                     isPhoneConnected = true,
663                     phoneStatusMessage = when (phoneState.status) {
664                         "STARTING" -> "폰에서 기록을 준비하고 있습니다"
665                         "RUNNING" -> null
666                         "PAUSED" -> "폰 기록 일시정지"
667                         "FINISHED" -> "폰에 기록이 저장되었습니다"
668                         "ABANDONED" -> "폰 기록이 취소되었습니다"
669                         "ERROR" -> phoneErrorMessage(phoneState.error)
670                         else -> current.phoneStatusMessage
671                     },
672                 )
673             }
674         }
675     }
676 }
677
678 private fun observeRunSync() {
679     viewModelScope.launch {
680         WatchRunSyncRepository.lastSyncedId.collect { localId ->
681             if (localId != null) {
682                 _state.update { it.copy(phoneStatusMessage = "폰에 기록이 저장되었습니다") }
683             }
684         }
685     }

```

```

686     viewModelScope.launch {
687         WatchRunSyncRepository.lastSyncedRunId.collect { runId ->
688             if (runId != null) {
689                 _state.update { it.copy(syncedRunId = runId) }
690             }
691         }
692     }
693 }
694
695 fun openRunOnPhone() {
696     val runId = _state.value.syncedRunId ?: return
697     if (!_state.value.isPhoneConnected) return
698     viewModelScope.launch { dataLayer.requestOpenRunOnPhone(runId) }
699 }
700
701 fun openPhoneApp() {
702     viewModelScope.launch { dataLayer.requestOpenPhoneApp() }
703 }
704
705 private fun observePhoneAuthState() {
706     viewModelScope.launch {
707         PhoneAuthStateRepository.isLoggedIn.collect { loggedIn ->
708             _state.update { it.copy(phoneLoggedIn = loggedIn) }
709         }
710     }
711 }
712
713 private fun observeOfflineCourses() {
714     viewModelScope.launch {
715         OfflineCourseRepository.courses.collect { courses ->
716             _state.update {
717                 it.copy(
718                     isSyncingCourses = false,
719                     phoneStatusMessage = if (courses.isNotEmpty()) {
720                         "주변 코스 ${courses.size}개를 저장했습니다"
721                     } else {
722                         it.phoneStatusMessage
723                     },
724                 )
725             }
726         }
727     }
728 }
729
730 private var nearFinishAnnounced = false
731
732 fun openCourseOnPhone() {
733     val courseId = _state.value.selectedCourseId ?: return
734     if (!_state.value.isPhoneConnected) {
735         _state.update { it.copy(phoneStatusMessage = "폰 연결 후 이용 가능합니다") }
736         return
737     }
738     viewModelScope.launch { dataLayer.requestOpenCourseOnPhone(courseId) }
739 }
740
741 private var maxCourseProgressMeters = 0.0
742
743 private fun updateCourseProgress(latitude: Double, longitude: Double) {
744     val course = activeCourse ?: return
745     val proximity = courseProximity(latitude, longitude, course) ?: return
746     val nearest = proximity.nearestMeters
747     val offCourse = nearest > COURSE_OFF_THRESHOLD_METERS
748
749     // 코스 위에 있을 때만 진행도 업데이트 (되돌아가도 감소 안 함)
750     if (!offCourse) {
751         maxCourseProgressMeters = maxOf(maxCourseProgressMeters, proximity.projectedMeters)
752     }
753
754     val rawProgress = if (course.distanceMeters > 1.0) {
755         (maxCourseProgressMeters * 100.0 / course.distanceMeters).toInt().coerceIn(0, 100)
756     } else {
757         0
758     }
759     // 완주 조건: 코스 진행도 100% AND 종착점 30m 이내
760     val progress = if (rawProgress >= 100) {
761         val endPoint = if (course.isLoop) course.points.firstOrNull() else course.points.lastOrNull()
762         val nearEnd = endPoint?.let {
763             distanceMeters(latitude, longitude, it.latitude, it.longitude) <= COURSE_FINISH_RADIUS_METERS
764         } ?: true
765         if (nearEnd) 100 else 99
766     } else rawProgress
767     _state.update {
768         it.copy(
769             courseProgressPercent = progress,
770             distanceToCourseMeters = nearest,
771             isOffCourse = offCourse,

```

```

772     )
773 }
774
775 // 이탈 상태 전환 시 음성 + 일시정지 + FLP 복귀 모니터 시작
776 if (offCourse != lastCourseWarning) {
777     lastCourseWarning = offCourse
778     speakIfEnabled {
779         if (offCourse) offCourse() else backOnCourse()
780     }
781     if (offCourse && !_state.value.isPaused) {
782         pausedByCourseDeviation = true
783         pause()
784         startDeviationMonitor()
785     } else if (!offCourse && pausedByCourseDeviation) {
786         stopDeviationMonitor()
787         pausedByCourseDeviation = false
788         resume()
789     }
790 } else if (offCourse && !_state.value.isPaused) {
791     pausedByCourseDeviation = true
792     pause()
793     startDeviationMonitor()
794 }
795
796 // 90% 도달 시 1회 거의 다 왔습니다
797 if (!nearFinishAnnounced && progress in 90..99) {
798     nearFinishAnnounced = true
799     speakIfEnabled { nearCourseFinish() }
800 }
801 }
802
803 override fun onCleared() {
804     stopHomeGpsMonitor()
805     stopGpsAutoPauseMonitor()
806     gpsHandlerThread.quitSafely()
807     voiceGuide.shutdown()
808     super.onCleared()
809 }
810
811 private fun freshAuthenticatedState(): WatchRunState = WatchRunState(
812     isPhoneConnected = _state.value.isPhoneConnected,
813     phoneLoggedIn = _state.value.phoneLoggedIn,
814     voiceGuidanceEnabled = _state.value.voiceGuidanceEnabled,
815     autoPauseEnabled = _state.value.autoPauseEnabled,
816     goalCompletionAction = _state.value.goalCompletionAction,
817     gpsReady = lastReadyLocation != null, // 이미 잡은 GPS는 유지
818 )
819
820 private fun distanceMeters(
821     latitude1: Double,
822     longitude1: Double,
823     latitude2: Double,
824     longitude2: Double,
825 ): Double {
826     val earthRadius = 6_371_000.0
827     val lat1 = Math.toRadians(latitude1)
828     val lat2 = Math.toRadians(latitude2)
829     val deltaLat = Math.toRadians(latitude2 - latitude1)
830     val deltaLon = Math.toRadians(longitude2 - longitude1)
831     val a = kotlin.math.sin(deltaLat / 2).let { it * it } +
832         kotlin.math.cos(lat1) * kotlin.math.cos(lat2) *
833         kotlin.math.sin(deltaLon / 2).let { it * it }
834     return earthRadius * 2 * kotlin.math.atan2(kotlin.math.sqrt(a), kotlin.math.sqrt(1 - a))
835 }
836
837 // 코스 경로까지의 최단 거리 AND 코스 시작점으로부터의 투영 거리(arclength) 반환
838 private data class CourseProximity(val nearestMeters: Double, val projectedMeters: Double)
839
840 private fun courseProximity(latitude: Double, longitude: Double, course: OfflineCourse): CourseProximity? {
841     if (course.points.isEmpty()) return null
842     if (course.points.size == 1) {
843         val p = course.points.first()
844         return CourseProximity(distanceMeters(latitude, longitude, p.latitude, p.longitude), 0.0)
845     }
846     var bestNearest = Double.MAX_VALUE
847     var bestProjected = 0.0
848     var cumulative = 0.0
849     course.points.zipWithNext().forEach { (start, end) ->
850         val segLen = distanceMeters(start.latitude, start.longitude, end.latitude, end.longitude)
851         val dLat = end.latitude - start.latitude
852         val dLon = end.longitude - start.longitude
853         val lenSq = dLat * dLat + dLon * dLon
854         val t = if (lenSq < 1e-12) 0.0 else
855             ((latitude - start.latitude) * dLat + (longitude - start.longitude) * dLon) / lenSq.coerceIn(0.0, 1.0)
856         val projLat = start.latitude + t * dLat
857         val projLon = start.longitude + t * dLon

```

```

858         val nearDist = distanceMeters(latitude, longitude, projLat, projLon)
859         if (nearDist < bestNearest) {
860             bestNearest = nearDist
861             bestProjected = cumulative + t * segLen
862         }
863         cumulative += segLen
864     }
865     return CourseProximity(bestNearest, bestProjected)
866 }
867
868 private fun announceCompletedKilometer() {
869     val current = _state.value
870     val completedKilometer = (current.distanceMeters / 1000.0).toInt()
871     if (completedKilometer <= lastAnnouncedKilometer) return
872     lastAnnouncedKilometer = completedKilometer
873     speakIfEnabled {
874         kilometer(
875             kilometers = completedKilometer,
876             elapsedSeconds = current.elapsedSeconds,
877             heartRateBpm = current.heartRateBpm,
878         )
879     }
880 }
881
882 private fun updateAutoPause(isAutoPaused: Boolean?) {
883     if (isAutoPaused == null || isAutoPaused == lastAutoPaused) return
884     lastAutoPaused = isAutoPaused
885     _state.update {
886         it.copy(
887             isPaused = isAutoPaused,
888             screen = WatchScreen.TRACKING,
889         )
890     }
891     speakIfEnabled {
892         if (isAutoPaused) autoPaused() else autoResumed()
893     }
894 }
895
896 private fun updateGpsVoice(gpsStatus: String?) {
897     gpsStatus ?: return
898     if (gpsStatus == lastGpsStatus) return
899     if (gpsStatus == "POOR") {
900         speakIfEnabled { gpsWeak() }
901     } else if (lastGpsStatus == "POOR" && gpsStatus == "GOOD") {
902         speakIfEnabled { gpsRecovered() }
903     }
904     lastGpsStatus = gpsStatus
905 }
906
907 private fun persistSettings() {
908     val current = _state.value
909     viewModelScope.launch(Dispatchers.IO) {
910         settingsStore.save(
911             WatchSettings(
912                 voiceGuidanceEnabled = current.voiceGuidanceEnabled,
913                 autoPauseEnabled = current.autoPauseEnabled,
914                 goalCompletionAction = current.goalCompletionAction,
915             ),
916         )
917     }
918 }
919
920 private inline fun speakIfEnabled(block: RunningVoiceGuide.() -> Unit) {
921     if (_state.value.voiceGuidanceEnabled) voiceGuide.block()
922 }
923
924 private fun phoneErrorMessage(error: String?): String = when (error) {
925     "RUN_ALREADY_ACTIVE" -> "폰에서 이미 러닝 중입니다"
926     "PHONE_PERMISSION_REQUIRED" -> "폰 앱에서 위치 권한을 허용하세요"
927     "TRACKING_SERVICE_START_FAILED" -> "폰의 위치 권한을 확인하세요"
928     "RUN_CREATE_FAILED" -> "폰에서 기록을 만들지 못했습니다"
929     "PAUSE_FAILED" -> "폰 기록을 일시정지하지 못했습니다"
930     "RESUME_FAILED" -> "폰 기록을 재개하지 못했습니다"
931     "FINISH_FAILED" -> "폰 기록 저장을 다시 시도해 주세요"
932     "NO_ACTIVE_RUN" -> "폰에 진행 중인 기록이 없습니다"
933     else -> "폰 동기화에 실패했습니다"
934 }
935
936 private companion object {
937     const val GPS_SAMPLE_INTERVAL_MILLIS = 1_000L
938     const val AUTO_PAUSE_SPEED_MPS = 0.5
939     const val AUTO_RESUME_SPEED_MPS = 1.0
940     const val AUTO_PAUSE_TICKS = 5
941     const val AUTO_RESUME_TICKS = 3
942     const val GPS_READY_ACCURACY_METERS = 30f
943     const val COURSE_OFF_THRESHOLD_METERS = 20.0

```

```
944         const val COURSE_FINISH_RADIUS_METERS = 30.0
945     }
946 }
```

```

1  package com.runway.wear.data
2
3  import android.content.Context
4  import kotlinx.coroutines.flow.MutableStateFlow
5  import kotlinx.coroutines.flow.asStateFlow
6  import org.json.JSONArray
7  import org.json.JSONObject
8
9  data class OfflineCoursePoint(
10     val latitude: Double,
11     val longitude: Double,
12     val altitudeMeters: Double,
13 ) {
14     fun toJson() = JSONObject()
15         .put("latitude", latitude)
16         .put("longitude", longitude)
17         .put("altitudeMeters", altitudeMeters)
18 }
19
20 data class OfflineCourse(
21     val courseId: String,
22     val name: String,
23     val description: String?,
24     val distanceMeters: Double,
25     val distanceFromMeMeters: Double,
26     val isLoop: Boolean,
27     val isPublic: Boolean,
28     val points: List<OfflineCoursePoint>,
29 ) {
30     fun toJson() = JSONObject()
31         .put("courseId", courseId)
32         .put("name", name)
33         .put("description", description)
34         .put("distanceMeters", distanceMeters)
35         .put("distanceFromMeMeters", distanceFromMeMeters)
36         .put("isLoop", isLoop)
37         .put("isPublic", isPublic)
38         .put("points", JSONArray().apply { points.forEach { put(it.toJson()) } })
39
40     companion object {
41         fun fromJson(json: JSONObject) = OfflineCourse(
42             courseId = json.getString("courseId"),
43             name = json.getString("name"),
44             description = json.optString("description").takeUnless { it.isBlank() || it == "null" },
45             distanceMeters = json.getDouble("distanceMeters"),
46             distanceFromMeMeters = json.optDouble("distanceFromMeMeters", 0.0),
47             isLoop = json.optBoolean("isLoop"),
48             isPublic = json.optBoolean("isPublic", true),
49             points = json.getJSONArray("points").let { points ->
50                 buildList {
51                     for (index in 0 until points.length()) {
52                         val point = points.getJSONObject(index)
53                         add(
54                             OfflineCoursePoint(
55                                 latitude = point.getDouble("latitude"),
56                                 longitude = point.getDouble("longitude"),
57                                 altitudeMeters = point.optDouble("altitudeMeters", 0.0),
58                             ),
59                         )
60                     }
61                 }
62             },
63         )
64     }
65 }
66
67 object OfflineCourseRepository {
68     private val _courses = MutableStateFlow<List<OfflineCourse>>(emptyList())
69     val courses = _courses.asStateFlow()
70
71     fun update(courses: List<OfflineCourse>) {
72         _courses.value = courses
73     }
74 }
75
76 class OfflineCourseStore(context: Context) {
77     private val preferences = context.applicationContext.getSharedPreferences(
78         "pathfinder_offline_courses",
79         Context.MODE_PRIVATE,
80     )
81
82     fun load(): List<OfflineCourse> {
83         val array = runCatching { JSONArray(preferences.getString(KEY_COURSES, "[]")) }

```



```
84         .getOrDefault(JSONArray())
85     return buildList {
86         for (index in 0 until array.length()) {
87             runCatching { OfflineCourse.fromJson(array.getJSONObject(index)) }
88                 .getOrNull()
89                 ?.let(::add)
90         }
91     }
92 }
93
94 fun replace(courses: List<OfflineCourse>) {
95     val array = JSONArray().apply { courses.forEach { put(it.toJson()) } }
96     preferences.edit().putString(KEY_COURSES, array.toString()).commit()
97     OfflineCourseRepository.update(courses)
98 }
99
100 private companion object {
101     const val KEY_COURSES = "courses"
102 }
103 }
```

```

1  package com.runway.wear.data
2
3  import android.content.Context
4  import org.json.JSONArray
5  import org.json.JSONObject
6  import java.util.UUID
7
8  data class PendingWatchRun(
9      val localId: String,
10     val courseId: String?,
11     val startedAt: String,
12     val endedAt: String,
13     val distanceMeters: Double,
14     val durationSeconds: Int,
15     val avgPaceSecondsPerKm: Int,
16     val caloriesBurned: Int,
17     val avgHeartRateBpm: Int?,
18     val points: List<WatchRunPoint>,
19 ) {
20     fun toJson(): JSONObject = JSONObject()
21         .put("localId", localId)
22         .put("courseId", courseId)
23         .put("startedAt", startedAt)
24         .put("endedAt", endedAt)
25         .put("distanceMeters", distanceMeters)
26         .put("durationSeconds", durationSeconds)
27         .put("avgPaceSecondsPerKm", avgPaceSecondsPerKm)
28         .put("caloriesBurned", caloriesBurned)
29         .put("avgHeartRateBpm", avgHeartRateBpm)
30         .put("points", JSONArray().apply { points.forEach { put(it.toJson()) } })
31
32     companion object {
33         fun create(
34             startedAt: String,
35             courseId: String? = null,
36             endedAt: String,
37             distanceMeters: Double,
38             durationSeconds: Int,
39             avgPaceSecondsPerKm: Int,
40             caloriesBurned: Int,
41             avgHeartRateBpm: Int?,
42             points: List<WatchRunPoint>,
43         ) = PendingWatchRun(
44             localId = UUID.randomUUID().toString(),
45             courseId = courseId,
46             startedAt = startedAt,
47             endedAt = endedAt,
48             distanceMeters = distanceMeters,
49             durationSeconds = durationSeconds,
50             avgPaceSecondsPerKm = avgPaceSecondsPerKm,
51             caloriesBurned = caloriesBurned,
52             avgHeartRateBpm = avgHeartRateBpm,
53             points = points,
54         )
55
56         fun fromJson(json: JSONObject) = PendingWatchRun(
57             localId = json.getString("localId"),
58             courseId = json.optString("courseId").takeUnless { it.isBlank() || it == "null" },
59             startedAt = json.getString("startedAt"),
60             endedAt = json.getString("endedAt"),
61             distanceMeters = json.getDouble("distanceMeters"),
62             durationSeconds = json.getInt("durationSeconds"),
63             avgPaceSecondsPerKm = json.getInt("avgPaceSecondsPerKm"),
64             caloriesBurned = json.getInt("caloriesBurned"),
65             avgHeartRateBpm = json.optInt("avgHeartRateBpm").takeIf {
66                 json.has("avgHeartRateBpm") && !json.isNull("avgHeartRateBpm")
67             },
68             points = json.optJSONArray("points")?.let { pointsJson ->
69                 buildList {
70                     for (index in 0 until pointsJson.length()) {
71                         add(WatchRunPoint.fromJson(pointsJson.getJSONObject(index)))
72                     }
73                 }
74             }.orEmpty(),
75         )
76     }
77 }
78
79 data class WatchRunPoint(
80     val sequence: Int,
81     val latitude: Double,
82     val longitude: Double,
83     val altitudeMeters: Double,

```

```

84     val speedMps: Double,
85     val recordedAt: String,
86 ) {
87     fun toJson(): JSONObject = JSONObject()
88         .put("sequence", sequence)
89         .put("latitude", latitude)
90         .put("longitude", longitude)
91         .put("altitudeMeters", altitudeMeters)
92         .put("speedMps", speedMps)
93         .put("recordedAt", recordedAt)
94
95     companion object {
96         fun fromJson(json: JSONObject) = WatchRunPoint(
97             sequence = json.getInt("sequence"),
98             latitude = json.getDouble("latitude"),
99             longitude = json.getDouble("longitude"),
100            altitudeMeters = json.getDouble("altitudeMeters"),
101            speedMps = json.getDouble("speedMps"),
102            recordedAt = json.getString("recordedAt"),
103        )
104    }
105 }
106
107 class PendingWatchRunStore(context: Context) {
108     private val preferences = context.applicationContext.getSharedPreferences(
109         "pathfinder_pending_watch_runs",
110         Context.MODE_PRIVATE,
111     )
112
113     @Synchronized
114     fun all(): List<PendingWatchRun> {
115         val json = runCatching { JSONArray(preferences.getString(KEY_RUNS, "[")) }
116         .getOrDefault(JSONArray())
117         return buildList {
118             for (index in 0 until json.length()) {
119                 runCatching { PendingWatchRun.fromJson(json.getJSONObject(index)) }
120                 .getOrNull()
121                 ?.let(::add)
122             }
123         }
124     }
125
126     @Synchronized
127     fun add(run: PendingWatchRun) {
128         save(all().filterNot { it.localId == run.localId } + run)
129     }
130
131     @Synchronized
132     fun remove(localId: String) {
133         save(all().filterNot { it.localId == localId })
134     }
135
136     private fun save(runs: List<PendingWatchRun>) {
137         val json = JSONArray().apply { runs.forEach { put(it.toJson()) } }
138         preferences.edit().putString(KEY_RUNS, json.toString()).commit()
139     }
140
141     private companion object {
142         const val KEY_RUNS = "runs"
143     }
144 }

```

```

1  package com.runway.wear.data
2
3  import com.google.android.gms.wearable.DataEvent
4  import com.google.android.gms.wearable.DataEventBuffer
5  import com.google.android.gms.wearable.DataMapItem
6  import com.google.android.gms.wearable.MessageEvent
7  import com.google.android.gms.wearable.Node
8  import com.google.android.gms.wearable.Wearable
9  import com.google.android.gms.wearable.WearableListenerService
10 import kotlinx.coroutines.CoroutineScope
11 import kotlinx.coroutines.Dispatchers
12 import kotlinx.coroutines.SupervisorJob
13 import kotlinx.coroutines.launch
14 import kotlinx.coroutines.flow.MutableStateFlow
15 import kotlinx.coroutines.flow.asStateFlow
16 import kotlinx.coroutines.tasks.await
17 import org.json.JSONArray
18 import org.json.JSONObject
19
20 data class PhoneRunState(
21     val status: String,
22     val error: String? = null,
23 )
24
25 object PhoneRunStateRepository {
26     private val _state = MutableStateFlow<PhoneRunState?>(null)
27     val state = _state.asStateFlow()
28
29     fun update(payload: ByteArray) {
30         val parsed = runCatching {
31             val json = JSONObject(payload.decodeToString())
32             PhoneRunState(
33                 status = json.getString("status"),
34                 error = json.optString("error").takeUnless { it.isBlank() || it == "null" },
35             )
36         }.getOrNull() ?: return
37         _state.value = parsed
38     }
39 }
40
41 object PhoneAuthStateRepository {
42     private val _isLoggedIn = MutableStateFlow<Boolean?>(null)
43     val isLoggedIn = _isLoggedIn.asStateFlow()
44
45     fun update(isLoggedIn: Boolean) {
46         _isLoggedIn.value = isLoggedIn
47     }
48 }
49
50 data class SyncAck(val localId: String, val runId: String?)
51
52 object WatchRunSyncRepository {
53     private val _lastSyncedId = MutableStateFlow<String?>(null)
54     val lastSyncedId = _lastSyncedId.asStateFlow()
55
56     private val _lastSyncedRunId = MutableStateFlow<String?>(null)
57     val lastSyncedRunId = _lastSyncedRunId.asStateFlow()
58
59     fun acknowledge(payload: ByteArray): String? {
60         val ack = runCatching {
61             val json = JSONObject(payload.decodeToString())
62             SyncAck(
63                 localId = json.getString("localId"),
64                 runId = json.optString("runId").takeUnless { it.isBlank() || it == "null" },
65             )
66         }.getOrNull() ?: return null
67         _lastSyncedId.value = ack.localId
68         ack.runId?.let { _lastSyncedRunId.value = it }
69         return ack.localId
70     }
71 }
72
73 class PhoneStateListenerService : WearableListenerService() {
74     private val serviceScope = CoroutineScope(SupervisorJob() + Dispatchers.IO)
75
76     override fun onMessageReceived(messageEvent: MessageEvent) {
77         when (messageEvent.path) {
78             WatchPaths.STATE -> PhoneRunStateRepository.update(messageEvent.data)
79             WatchPaths.RUN_UPLOAD_ACK -> {
80                 WatchRunSyncRepository.acknowledge(messageEvent.data)?.let { localId ->
81                     PendingWatchRunStore(this).remove(localId)
82                 }
83             }
84         }
85     }
86 }

```

```

84         WatchPaths.AUTH_STATE -> {
85             runCatching {
86                 val json = JSONObject(messageEvent.data.decodeToString())
87                 PhoneAuthStateRepository.update(json.getBoolean("isLoggedIn"))
88             }
89         }
90     }
91 }
92
93 override fun onDataChange(dataEvents: DataEventBuffer) {
94     dataEvents
95         .filter { it.type == DataEvent.TYPE_CHANGED }
96         .filter { it.dataItem.uri.path == WatchPaths.COURSE_CATALOG }
97         .forEach { event ->
98             val asset = DataMapItem.fromDataItem(event.dataItem).dataMap.getAsset("courses")
99             ?: return@forEach
100             serviceScope.launch {
101                 val descriptor = runCatching {
102                     Wearable.getDataClient(this@PhoneStateListenerService)
103                         .getFdForAsset(asset)
104                         .await()
105                 }.getOrNull() ?: return@launch
106                 val courses = descriptor.inputStream.use { input ->
107                     val array = runCatching { JSONArray(input.readBytes().decodeToString()) }
108                     .getOrNull() ?: return@launch
109                     buildList {
110                         for (index in 0 until array.length()) {
111                             runCatching { OfflineCourse.fromJson(array.getJSONObject(index)) }
112                                 .getOrNull()
113                                 ?.let(::add)
114                         }
115                     }
116                 }
117                 OfflineCourseStore(this@PhoneStateListenerService).replace(courses)
118             }
119         }
120 }
121
122 override fun onPeerConnected(peer: Node) {
123     serviceScope.launch {
124         WatchDataLayerClient(this@PhoneStateListenerService)
125             .syncPendingRuns(PendingWatchRunStore(this@PhoneStateListenerService))
126     }
127 }
128 }

```

```

1 package com.runway.wear.data
2
3 import android.content.Context
4 import com.google.android.gms.wearable.Wearable
5 import com.google.android.gms.wearable.Asset
6 import com.google.android.gms.wearable.PutDataMapRequest
7 import com.runway.wear.model.GoalCompletionAction
8 import com.runway.wear.model.IntervalTarget
9 import com.runway.wear.model.RunGoal
10 import kotlinx.coroutines.tasks.await
11 import org.json.JSONObject
12
13 object WatchPaths {
14     const val COMMAND = "/runway/watch/command"
15     const val STATE = "/runway/watch/state"
16     const val COURSE_START = "/runway/watch/course-start"
17     const val RUN_UPLOAD = "/runway/watch/run-upload"
18     const val RUN_UPLOAD_ACK = "/runway/watch/run-upload-ack"
19     const val COURSE_REQUEST = "/runway/watch/course-request"
20     const val COURSE_CATALOG = "/runway/watch/course-catalog"
21     const val OPEN_COURSE = "/runway/watch/open-course"
22     const val OPEN_RUN = "/runway/watch/open-run"
23     const val AUTH_REQUEST = "/runway/watch/auth/request"
24     const val AUTH_STATE = "/runway/watch/auth/state"
25     const val OPEN_APP = "/runway/watch/open-app"
26 }
27
28 class WatchDataLayerClient(context: Context) {
29     private val applicationContext = context.applicationContext
30     private val nodeClient = Wearable.getNodeClient(applicationContext)
31     private val messageClient = Wearable.getMessageClient(applicationContext)
32     private val dataClient = Wearable.getDataClient(applicationContext)
33
34     suspend fun isPhoneConnected(): Boolean = runCatching {
35         nodeClient.connectedNodes.await().isEmpty()
36     }.getOrElse(false)
37
38     suspend fun sendStart(goal: RunGoal): Boolean = sendCommand(
39         when (goal) {
40             RunGoal.Free -> JSONObject().put("type", "START_FREE_RUN")
41             is RunGoal.Time -> JSONObject()
42                 .put("type", "START_TIME_GOAL_RUN")
43                 .put("targetMinutes", goal.minutes)
44                 .putCompletionAction(goal.completionAction)
45             is RunGoal.Distance -> JSONObject()
46                 .put("type", "START_DISTANCE_GOAL_RUN")
47                 .put("targetMeters", goal.meters)
48                 .putCompletionAction(goal.completionAction)
49             is RunGoal.Interval -> JSONObject()
50                 .put("type", "START_INTERVAL_RUN")
51                 .put("sets", goal.sets)
52                 .putTarget("work", goal.work)
53                 .putTarget("recovery", goal.recovery)
54                 .putCompletionAction(goal.completionAction)
55         },
56     )
57
58     suspend fun pause(): Boolean = sendCommand(JSONObject().put("type", "PAUSE_RUN"))
59     suspend fun resume(): Boolean = sendCommand(JSONObject().put("type", "RESUME_RUN"))
60     suspend fun finish(): Boolean = sendCommand(JSONObject().put("type", "FINISH_RUN"))
61     suspend fun abandon(): Boolean = sendCommand(JSONObject().put("type", "ABANDON_RUN"))
62
63     suspend fun requestOpenCourseOnPhone(courseId: String): Boolean =
64         sendMessage(WatchPaths.OPEN_COURSE, courseId.encodeToByteArray())
65
66     suspend fun requestOpenRunOnPhone(runId: String): Boolean =
67         sendMessage(WatchPaths.OPEN_RUN, runId.encodeToByteArray())
68
69     suspend fun requestAuthState(): Boolean = sendMessage(WatchPaths.AUTH_REQUEST, ByteArray(0))
70     suspend fun requestOpenPhoneApp(): Boolean = sendMessage(WatchPaths.OPEN_APP, ByteArray(0))
71
72     suspend fun requestNearbyCourses(latitude: Double, longitude: Double): Boolean = sendMessage(
73         WatchPaths.COURSE_REQUEST,
74         JSONObject()
75             .put("latitude", latitude)
76             .put("longitude", longitude)
77             .toString()
78             .encodeToByteArray(),
79     )
80
81     suspend fun syncPendingRuns(store: PendingWatchRunStore): Int {
82         if (!isPhoneConnected()) return 0
83         var sent = 0

```

```

84     store.all().forEach { run ->
85         val request = PutDataMapRequest.create("${WatchPaths.RUN_UPLOAD}/${run.localId}").apply {
86             dataMap.putString("localId", run.localId)
87             dataMap.putAsset("run", Asset.createFromBytes(run.toJson().toString().encodeToByteArray()))
88         }.asPutDataRequest().setUrgent()
89         if (runCatching { dataClient.putDataItem(request).await() }.isSuccess) {
90             sent++
91         }
92     }
93     return sent
94 }
95
96 private suspend fun sendCommand(payload: JSONObject): Boolean =
97     sendMessage(WatchPaths.COMMAND, payload.toString().encodeToByteArray())
98
99 private suspend fun sendMessage(path: String, payload: ByteArray): Boolean = runCatching {
100     val nodes = nodeClient.connectedNodes.await()
101     if (nodes.isEmpty()) return false
102     nodes.forEach { node ->
103         messageClient.sendMessage(
104             node.id,
105             path,
106             payload,
107         ).await()
108     }
109     true
110 }.getOrElse(false)
111
112 private fun JSONObject.putCompletionAction(action: GoalCompletionAction): JSONObject =
113     put("completionAction", action.name)
114
115 private fun JSONObject.putTarget(prefix: String, target: IntervalTarget): JSONObject {
116     return when (target) {
117         is IntervalTarget.Time -> put("${prefix}Type", "TIME")
118             .put("${prefix}Seconds", target.seconds)
119         is IntervalTarget.Distance -> put("${prefix}Type", "DISTANCE")
120             .put("${prefix}Meters", target.meters)
121     }
122 }
123 }

```

wear-os/app/src/main/java/com/runway/wear/data/WatchSettingsStore.kt

Kotlin | 33줄

```

1 package com.runway.wear.data
2
3 import android.content.Context
4 import com.runway.wear.model.GoalCompletionAction
5
6 data class WatchSettings(
7     val voiceGuidanceEnabled: Boolean = true,
8     val autoPauseEnabled: Boolean = true,
9     val goalCompletionAction: GoalCompletionAction = GoalCompletionAction.PAUSE,
10 )
11
12 class WatchSettingsStore(context: Context) {
13     private val preferences = context.applicationContext.getSharedPreferences(
14         "pathfinder_watch_settings",
15         Context.MODE_PRIVATE,
16     )
17
18     fun load(): WatchSettings = WatchSettings(
19         voiceGuidanceEnabled = preferences.getBoolean("voice_guidance", true),
20         autoPauseEnabled = preferences.getBoolean("auto_pause", true),
21         goalCompletionAction = preferences.getString("goal_completion", null)
22             ?.let { runCatching { GoalCompletionAction.valueOf(it) }.getOrNull() }
23             ?: GoalCompletionAction.PAUSE,
24     )
25
26     fun save(settings: WatchSettings) {
27         preferences.edit()
28             .putBoolean("voice_guidance", settings.voiceGuidanceEnabled)
29             .putBoolean("auto_pause", settings.autoPauseEnabled)
30             .putString("goal_completion", settings.goalCompletionAction.name)
31             .apply()
32     }
33 }

```

```

1  package com.runway.wear.health
2
3  import android.annotation.SuppressLint
4  import android.content.Context
5  import androidx.health.services.client.ExerciseUpdateCallback
6  import androidx.health.services.client.HealthServices
7  import androidx.health.services.client.data.Availability
8  import androidx.health.services.client.data.DataType
9  import androidx.health.services.client.data.ExerciseConfig
10 import androidx.health.services.client.data.ExerciseLapSummary
11 import androidx.health.services.client.data.ExerciseState
12 import androidx.health.services.client.data.ExerciseType
13 import androidx.health.services.client.data.ExerciseUpdate
14 import androidx.health.services.client.data.LocationAvailability
15 import android.os.SystemClock
16 import java.time.Instant
17 import androidx.health.services.client.endExercise
18 import androidx.health.services.client.getCapabilities
19 import androidx.health.services.client.pauseExercise
20 import androidx.health.services.client.resumeExercise
21 import androidx.health.services.client.startExercise
22 import kotlinx.coroutines.channels.awaitClose
23 import kotlinx.coroutines.channels.trySendBlocking
24 import kotlinx.coroutines.flow.callbackFlow
25
26 data class HealthMetricUpdate(
27     val distanceMeters: Double? = null,
28     val heartRateBpm: Int? = null,
29     val cadenceSpm: Int? = null,
30     val gpsStatus: String? = null,
31     val isAutoPaused: Boolean? = null,
32     val locations: List<HealthLocationSample> = emptyList(),
33 )
34
35 data class HealthLocationSample(
36     val latitude: Double,
37     val longitude: Double,
38     val altitudeMeters: Double,
39     val speedMps: Double,
40     val recordedAt: String,
41 )
42
43 @SuppressLint("RestrictedApi")
44 class HealthServicesManager(context: Context) {
45     private val exerciseClient = HealthServices.getClient(context.applicationContext).exerciseClient
46
47     suspend fun start(autoPauseEnabled: Boolean) {
48         val capabilities = exerciseClient.getCapabilities()
49             .getExerciseTypeCapabilities(ExerciseType.RUNNING)
50         val requested = setOf(
51             DataType.DISTANCE_TOTAL,
52             DataType.HEART_RATE_BPM,
53             DataType.STEPS_PER_MINUTE,
54             DataType.LOCATION,
55             DataType.SPEED,
56         ).intersect(capabilities.supportedDataTypes)
57
58         exerciseClient.startExercise(
59             ExerciseConfig(
60                 exerciseType = ExerciseType.RUNNING,
61                 dataTypes = requested,
62                 isAutoPauseAndResumeEnabled = autoPauseEnabled,
63                 isGpsEnabled = true,
64             ),
65         )
66     }
67
68     suspend fun pause() = exerciseClient.pauseExercise()
69     suspend fun resume() = exerciseClient.resumeExercise()
70     suspend fun finish() = exerciseClient.endExercise()
71
72     val updates = callbackFlow {
73         val callback = object : ExerciseUpdateCallback {
74             override fun onExerciseUpdateReceived(update: ExerciseUpdate) {
75                 val metrics = update.latestMetrics
76                 val bootInstant = Instant.now().minusMillis(SystemClock.elapsedRealtime())
77                 val speed = metrics.getData(DataType.SPEED).lastOrNull()?.value ?: 0.0
78                 trySendBlocking(
79                     HealthMetricUpdate(
80                         distanceMeters = metrics.getData(DataType.DISTANCE_TOTAL)?.total,
81                         heartRateBpm = metrics.getData(DataType.HEART_RATE_BPM)
82                             .lastOrNull()?.value?.toInt(),
83                         cadenceSpm = metrics.getData(DataType.STEPS_PER_MINUTE)

```



```

84         .lastOrNull()?.value?.toInt(),
85         isAutoPaused = when (update.exerciseStateInfo.state) {
86             ExerciseState.AUTO_PAUSING,
87             ExerciseState.AUTO_PAUSED,
88             -> true
89             ExerciseState.AUTO_RESUMING,
90             ExerciseState.ACTIVE,
91             -> false
92             else -> null
93         },
94         locations = metrics.getData(DataType.LOCATION).map { point ->
95             HealthLocationSample(
96                 latitude = point.value.latitude,
97                 longitude = point.value.longitude,
98                 altitudeMeters = point.value.altitude
99                     .takeUnless { it == Double.MIN_VALUE || !it.isFinite() }
100                     ?: 0.0,
101                 speedMps = speed,
102                 recordedAt = point.getTimeInstant(bootInstant).toString(),
103             )
104         },
105     ),
106 )
107 }
108
109 override fun onLapSummaryReceived(lapSummary: ExerciseLapSummary) = Unit
110 override fun onRegistered() = Unit
111 override fun onRegistrationFailed(throwable: Throwable) {
112     close(throwable)
113 }
114
115 override fun onAvailabilityChanged(
116     dataType: DataType<*, *>,
117     availability: Availability,
118 ) {
119     if (availability is LocationAvailability) {
120         trySendBlocking(
121             HealthMetricUpdate(
122                 gpsStatus = when (availability) {
123                     LocationAvailability.ACQUIRED_TETHERED,
124                     LocationAvailability.ACQUIRED_UNTETHERED -> "GOOD"
125                     LocationAvailability.NO_GNSS -> "POOR"
126                     else -> "SEARCHING"
127                 },
128             ),
129         )
130     }
131 }
132 }
133 exerciseClient.setUpdateCallback(callback)
134 awaitClose { exerciseClient.clearUpdateCallbackAsync(callback) }
135 }
136 }

```

```
1 package com.runway.wear.model
2
3 enum class GoalCompletionAction {
4     PAUSE,
5     CONTINUE,
6 }
7
8 sealed interface IntervalTarget {
9     data class Time(val seconds: Int) : IntervalTarget
10    data class Distance(val meters: Int) : IntervalTarget
11 }
12
13 sealed interface RunGoal {
14     val completionAction: GoalCompletionAction
15
16     data object Free : RunGoal {
17         override val completionAction = GoalCompletionAction.CONTINUE
18     }
19
20     data class Time(
21         val minutes: Int,
22         override val completionAction: GoalCompletionAction,
23     ) : RunGoal
24
25     data class Distance(
26         val meters: Int,
27         override val completionAction: GoalCompletionAction,
28     ) : RunGoal
29
30     data class Interval(
31         val work: IntervalTarget,
32         val recovery: IntervalTarget,
33         val sets: Int,
34         override val completionAction: GoalCompletionAction,
35     ) : RunGoal
36 }
37
38 enum class WatchScreen {
39     HOME,
40     COURSE_LIST,
41     COURSE_DETAIL,
42     GOAL_TYPE,
43     TIME_GOAL,
44     DISTANCE_GOAL,
45     INTERVAL_GOAL,
46     SETTINGS,
47     PREPARING,
48     COUNTDOWN,
49     TRACKING,
50     PAUSED,
51     SUMMARY,
52 }
53
54 data class WatchRunState(
55     val screen: WatchScreen = WatchScreen.HOME,
56     val goal: RunGoal = RunGoal.Free,
57     val pendingGoal: RunGoal? = null,
58     val isPhoneConnected: Boolean = false,
59     val gpsReady: Boolean = false,
60     val syncedRunId: String? = null,
61     val voiceGuidanceEnabled: Boolean = true,
62     val autoPauseEnabled: Boolean = true,
63     val goalCompletionAction: GoalCompletionAction = GoalCompletionAction.PAUSE,
64     val isRunning: Boolean = false,
65     val isPaused: Boolean = false,
66     val elapsedSeconds: Long = 0,
67     val distanceMeters: Double = 0.0,
68     val paceMinPerKm: Float = 0f,
69     val heartRateBpm: Int? = null,
70     val cadenceSpm: Int? = null,
71     val gpsStatus: String = "SEARCHING",
72     val intervalStep: Int = 1,
73     val intervalIsWork: Boolean = true,
74     val intervalSegmentProgress: Int = 0,
75     val intervalRemainingLabel: String? = null,
76     val goalCompleted: Boolean = false,
77     val phoneStatusMessage: String? = null,
78     val selectedCourseId: String? = null,
79     val courseName: String? = null,
80     val courseDistanceMeters: Double = 0.0,
81     val courseProgressPercent: Int = 0,
82     val distanceToCourseMeters: Double? = null,
83     val isOffCourse: Boolean = false,
```

```

84     val isSyncingCourses: Boolean = false,
85     val phoneLoggedIn: Boolean? = null,
86 ) {
87     val progressPercent: Int?
88         get() = GoalProgress.percent(this)
89
90     val remainingLabel: String?
91         get() = GoalProgress.remaining(this)
92 }
93
94 object GoalProgress {
95     fun percent(state: WatchRunState): Int? = when (val goal = state.goal) {
96         RunGoal.Free -> null
97         is RunGoal.Time -> (state.elapsedSeconds * 100 / (goal.minutes * 60L))
98             .toInt().coerceIn(0, 100)
99         is RunGoal.Distance -> (state.distanceMeters * 100 / goal.meters)
100             .toInt().coerceIn(0, 100)
101         is RunGoal.Interval -> {
102             val completedSegments = (state.intervalStep - 1) * 2 +
103                 if (state.intervalIsWork) 0 else 1
104             ((completedSegments * 100 + state.intervalSegmentProgress) / (goal.sets * 2))
105                 .coerceIn(0, 100)
106         }
107     }
108
109     fun remaining(state: WatchRunState): String? {
110         if (state.goalCompleted) return "목표 달성 · 계속 기록 중"
111         return when (val goal = state.goal) {
112             RunGoal.Free -> null
113             is RunGoal.Time ->
114                 "${formatDuration((goal.minutes * 60L - state.elapsedSeconds).coerceAtLeast(0))} 남음"
115             is RunGoal.Distance -> {
116                 val remainingKm = (goal.meters - state.distanceMeters).coerceAtLeast(0.0) / 1000.0
117                 "%.1fkm 남음".format(remainingKm)
118             }
119             is RunGoal.Interval -> {
120                 val phase = if (state.intervalIsWork) "운동" else "회복"
121                 "$phase ${state.intervalStep}/${state.sets} · ${state.intervalRemainingLabel.orEmpty()}"
122             }
123         }
124     }
125 }
126
127 fun formatDuration(seconds: Long): String =
128     "%02d:%02d".format(seconds / 60, seconds % 60)
129
130 fun formatPace(paceMinPerKm: Float): String {
131     if (paceMinPerKm <= 0f || !paceMinPerKm.isFinite()) return "--:--\"
132     val minutes = paceMinPerKm.toInt()
133     val seconds = ((paceMinPerKm - minutes) * 60).toInt().coerceIn(0, 59)
134     return "%d'%02d\".format(minutes, seconds)
135 }

```

```
1 package com.runway.wear.ui
2
3 import androidx.activity.compose.BackHandler
4 import androidx.compose.foundation.Image
5 import androidx.compose.foundation.background
6 import androidx.compose.foundation.border
7 import androidx.compose.foundation.clickable
8 import androidx.compose.foundation.gestures.detectTapGestures
9 import androidx.compose.foundation.gestures.rememberScrollableState
10 import androidx.compose.foundation.gestures.snapping.rememberSnapFlingBehavior
11 import androidx.compose.foundation.layout.Arrangement
12 import androidx.compose.foundation.layout.Box
13 import androidx.compose.foundation.layout.BoxWithConstraints
14 import androidx.compose.foundation.layout.Column
15 import androidx.compose.foundation.layout.ColumnScope
16 import androidx.compose.foundation.layout.Row
17 import androidx.compose.foundation.layout.Spacer
18 import androidx.compose.foundation.layout.fillMaxSize
19 import androidx.compose.foundation.layout.fillMaxWidth
20 import androidx.compose.foundation.layout.height
21 import androidx.compose.foundation.layout.heightIn
22 import androidx.compose.foundation.layout.padding
23 import androidx.compose.foundation.layout.size
24 import androidx.compose.foundation.layout.width
25 import androidx.compose.foundation.lazy.LazyColumn
26 import androidx.compose.foundation.lazy.items
27 import androidx.compose.foundation.lazy.rememberLazyListState
28 import androidx.compose.foundation.pager.HorizontalPager
29 import androidx.compose.foundation.pager.rememberPagerState
30 import androidx.compose.foundation.rememberScrollState
31 import androidx.compose.foundation.shape.CircleShape
32 import androidx.compose.foundation.shape.RoundedCornerShape
33 import androidx.compose.foundation.verticalScroll
34 import androidx.compose.animation.AnimatedContent
35 import androidx.compose.animation.fadeIn
36 import androidx.compose.animation.fadeOut
37 import androidx.compose.animation.scaleIn
38 import androidx.compose.animation.scaleOut
39 import androidx.compose.animation.togetherWith
40 import androidx.compose.animation.core.Animatable
41 import androidx.compose.animation.core.LinearEasing
42 import androidx.compose.animation.core.RepeatMode
43 import androidx.compose.animation.core.animateFloat
44 import androidx.compose.animation.core.animateFloatAsState
45 import androidx.compose.animation.core.infiniteRepeatable
46 import androidx.compose.animation.core.rememberInfiniteTransition
47 import androidx.compose.animation.core.tween
48 import androidx.compose.material.icons.Icons
49 import androidx.compose.material.icons.automirrored.filled.DirectionsRun
50 import androidx.compose.material.icons.automirrored.filled.KeyboardArrowLeft
51 import androidx.compose.material.icons.automirrored.filled.VolumeUp
52 import androidx.compose.foundation.Canvas
53 import androidx.compose.material.icons.filled.Check
54 import androidx.compose.material.icons.filled.Close
55 import androidx.compose.material.icons.filled.PhoneAndroid
56 import androidx.compose.material.icons.filled.Flag
57 import androidx.compose.material.icons.filled.Pause
58 import androidx.compose.material.icons.filled.PlayArrow
59 import androidx.compose.material.icons.filled.Route
60 import androidx.compose.material.icons.filled.Settings
61 import androidx.compose.material.icons.filled.Stop
62 import androidx.compose.material.icons.filled.Timer
63 import androidx.compose.material3.Button
64 import androidx.compose.material3.ButtonDefaults
65 import androidx.compose.material3.Icon
66 import androidx.compose.material3.IconButton
67 import androidx.compose.material3.LinearProgressIndicator
68 import androidx.compose.material3.MaterialTheme
69 import androidx.compose.material3.Surface
70 import androidx.compose.material3.Switch
71 import androidx.compose.material3.Text
72 import androidx.compose.runtime.Composable
73 import androidx.compose.runtime.LaunchedEffect
74 import androidx.compose.runtime.getValue
75 import androidx.compose.runtime.mutableFloatStateOf
76 import androidx.compose.runtime.mutableIntStateOf
77 import androidx.compose.runtime.mutableLongStateOf
78 import androidx.compose.runtime.mutableStateOf
79 import androidx.compose.runtime.remember
80 import androidx.compose.runtime.rememberCoroutineScope
81 import androidx.compose.runtime.snapshotFlow
82 import androidx.compose.runtime.setValue
83 import androidx.compose.ui.Alignment
```

```

84 import androidx.compose.ui.Modifier
85 import androidx.compose.ui.focus.FocusRequester
86 import androidx.compose.ui.graphics.Color
87 import androidx.compose.ui.graphics.StrokeCap
88 import androidx.compose.ui.graphics.drawscope.Stroke
89 import androidx.compose.ui.graphics.graphicsLayer
90 import androidx.compose.ui.input.pointer.PointerInput
91 import androidx.compose.ui.res.painterResource
92 import androidx.compose.ui.text.font.FontWeight
93 import androidx.compose.ui.text.style.TextAlign
94 import androidx.compose.ui.unit.dp
95 import androidx.compose.ui.unit.sp
96 import androidx.lifecycle.compose.collectAsStateWithLifecycle
97 import androidx.lifecycle.viewmodel.compose.viewModel
98 import androidx.wear.compose.foundation.requestFocusOnHierarchyActive
99 import androidx.wear.compose.foundation.rotary.RotaryScrollableDefaults
100 import androidx.wear.compose.foundation.rotary.rotaryScrollable
101 import com.runway.wear.R
102 import com.runway.wear.WatchViewModel
103 import com.runway.wear.model.GoalCompletionAction
104 import com.runway.wear.model.IntervalTarget
105 import com.runway.wear.model.RunGoal
106 import com.runway.wear.model.WatchRunState
107 import com.runway.wear.model.WatchScreen
108 import com.runway.wear.model.formatDuration
109 import com.runway.wear.model.formatPace
110 import com.runway.wear.data.OfflineCourse
111 import com.runway.wear.data.OfflineCourseRepository
112 import kotlinx.coroutines.launch
113 import kotlinx.coroutines.delay
114 import kotlinx.coroutines.flow.distinctUntilChanged
115
116 private val Background = Color(0xFF090B0F)
117 private val SurfaceColor = Color(0xFF171A20)
118 private val Accent = Color(0xFFB8FF00)
119 private val Muted = Color(0xFF9AA3AF)
120 private val Danger = Color(0xFFFF665E)
121
122 @Composable
123 fun PathfinderWearApp(
124     viewModel: WatchViewModel = viewModel(),
125     launchToken: Int = 0,
126     isAmbient: Boolean = false,
127 ) {
128     val state by viewModel.state.collectAsStateWithLifecycle()
129     var showLaunchAnimation by remember(launchToken) { mutableStateOf(true) }
130     LaunchedEffect(launchToken) {
131         delay(700)
132         showLaunchAnimation = false
133     }
134     BackHandler(
135         enabled = state.screen != WatchScreen.HOME,
136         onBack = viewModel::navigateBack,
137     )
138     MaterialTheme(
139         colorScheme = MaterialTheme.colorScheme.copy(
140             primary = Accent,
141             surface = SurfaceColor,
142             background = Background,
143             onBackground = Color.White,
144             onSurface = Color.White,
145         ),
146     ) {
147         Surface(modifier = Modifier.fillMaxSize(), color = Background) {
148             if (showLaunchAnimation) {
149                 LaunchAnimationScreen()
150             } else if (isAmbient && (state.screen == WatchScreen.TRACKING || state.screen == WatchScreen.PAUSED)) {
151                 AmbientTrackingScreen(state)
152             } else when (state.screen) {
153                 WatchScreen.HOME -> HomeScreen(
154                     state = state,
155                     onStart = viewModel::start,
156                     onGoal = { viewModel.navigate(WatchScreen.GOAL_TYPE) },
157                     onCourses = { viewModel.navigate(WatchScreen.COURSE_LIST) },
158                     onLogin = viewModel::openPhoneApp,
159                 )
160                 WatchScreen.COURSE_LIST -> CourseListScreen(state, viewModel)
161                 WatchScreen.COURSE_DETAIL -> CourseDetailScreen(state, viewModel)
162                 WatchScreen.GOAL_TYPE -> GoalTypeScreen(viewModel)
163                 WatchScreen.TIME_GOAL -> TimeGoalScreen(viewModel)
164                 WatchScreen.DISTANCE_GOAL -> DistanceGoalScreen(viewModel)
165                 WatchScreen.INTERVAL_GOAL -> IntervalGoalScreen(viewModel)
166                 WatchScreen.SETTINGS -> RunningSettingsScreen(state, viewModel)
167                 WatchScreen.PREPARING -> PreparingScreen(
168                     state = state,
169                     onConfirm = viewModel::confirmPreparing,

```

```

170         onCancel = viewModel::cancelPreparing,
171     )
172     WatchScreen.COUNTDOWN -> CountdownScreen(
173         onFinish = viewModel::commitCountdown,
174         onCancel = viewModel::cancelCountdown,
175     )
176     WatchScreen.TRACKING -> TrackingScreen(state, viewModel)
177     WatchScreen.PAUSED -> PausedScreen(state, viewModel)
178     WatchScreen.SUMMARY -> SummaryScreen(state, viewModel::returnHome, viewModel::openRunOnPhone)
179 }
180 }
181 }
182 }
183
184 @Composable
185 private fun LaunchAnimationScreen() {
186     var started by remember { mutableStateOf(false) }
187     LaunchedEffect(Unit) { started = true }
188     val scale by animateFloatAsState(
189         targetValue = if (started) 1f else 0.72f,
190         animationSpec = tween(420),
191         label = "launchScale",
192     )
193     val alpha by animateFloatAsState(
194         targetValue = if (started) 1f else 0f,
195         animationSpec = tween(320),
196         label = "launchAlpha",
197     )
198     Column(
199         modifier = Modifier.fillMaxSize(),
200         horizontalAlignment = Alignment.CenterHorizontally,
201         verticalArrangement = Arrangement.Center,
202     ) {
203         Image(
204             painter = painterResource(R.drawable.app_logo_mark),
205             contentDescription = "PathFinder",
206             modifier = Modifier
207                 .size(72.dp)
208                 .graphicsLayer {
209                     scaleX = scale
210                     scaleY = scale
211                     this.alpha = alpha
212                 },
213         )
214         Spacer(Modifier.height(10.dp))
215         Text(
216             "PathFinder",
217             color = Color.White.copy(alpha = alpha),
218             fontWeight = FontWeight.Bold,
219             fontSize = 18.sp,
220         )
221     }
222 }
223
224 @Composable
225 private fun HomeScreen(
226     state: WatchRunState,
227     onStart: (RunGoal) -> Unit,
228     onGoal: () -> Unit,
229     onCourses: () -> Unit,
230     onLogin: () -> Unit = {},
231 ) {
232     WatchPage(scrollable = true) {
233         Image(
234             painter = painterResource(R.drawable.app_logo_mark),
235             contentDescription = "PathFinder",
236             modifier = Modifier.size(if (compact) 34.dp else 52.dp),
237         )
238         Text("PathFinder", fontWeight = FontWeight.Bold, fontSize = if (compact) 16.sp else 18.sp)
239         GpsStatusIndicator(state.gpsReady)
240         ConnectionLabel(state.isPhoneConnected)
241         PrimaryAction(
242             label = "바로 시작",
243             icon = Icons.Filled.PlayArrow,
244             onClick = { onStart(RunGoal.Free) },
245             compact = compact,
246         )
247         SecondaryAction("목표 설정", Icons.Filled.Flag, onClick = onGoal, compact = compact)
248         SecondaryAction("주변 코스 도전", Icons.Filled.Route, onClick = onCourses, compact = compact)
249         if (state.isPhoneConnected && state.phoneLoggedIn == false) {
250             SecondaryAction(
251                 label = "핸드폰에서 로그인하기",
252                 icon = Icons.Filled.PhoneAndroid,
253                 onClick = onLogin,
254                 color = Danger,
255             )

```

```

256     }
257     Text(
258         if (state.isPhoneConnected) "폰 연결됨 · 독립 GPS 준비" else "독립 GPS 모드 · 동기화 대기",
259         color = Muted,
260         fontSize = 10.sp,
261         textAlign = TextAlign.Center,
262     )
263     state.phoneStatusMessage?.let { StatusText(it) }
264 }
265 }
266
267 @Composable
268 private fun CourseListScreen(state: WatchRunState, viewModel: WatchViewModel) {
269     val courses by OfflineCourseRepository.courses.collectAsStateWithLifecycle()
270     WatchPage(scrollable = true) {
271         BackTitle("저장된 코스", viewModel::navigateBack)
272         SecondaryAction(
273             label = if (state.isSyncingCourses) "불러오는 중..." else "주변 코스 불러오기",
274             icon = Icons.Filled.Route,
275             onClick = viewModel::syncNearbyCourses,
276             color = if (state.isPhoneConnected) Accent else Muted,
277         )
278         if (!state.isPhoneConnected) {
279             Text("새 코스 저장은 폰 연결 시 가능합니다", color = Muted, fontSize = 9.sp)
280         }
281         if (courses.isEmpty()) {
282             Text(
283                 "저장된 코스가 없습니다\n폰 연결 후 주변 코스를 저장하세요",
284                 color = Muted,
285                 fontSize = 10.sp,
286                 textAlign = TextAlign.Center,
287             )
288         } else {
289             courses.forEach { course ->
290                 CourseCard(course) { viewModel.selectCourse(course.courseId) }
291             }
292         }
293         state.phoneStatusMessage?.let { Text(it, color = Muted, fontSize = 9.sp) }
294     }
295 }
296
297 @Composable
298 private fun CourseCard(course: OfflineCourse, onClick: () -> Unit) {
299     Column(
300         modifier = Modifier
301             .fillMaxWidth()
302             .background(SurfaceColor, RoundedCornerShape(10.dp))
303             .clickable(onClick = onClick)
304             .padding(horizontal = 12.dp, vertical = 10.dp),
305     ) {
306         Row(
307             verticalAlignment = Alignment.CenterVertically,
308             horizontalArrangement = Arrangement.spacedBy(5.dp),
309         ) {
310             Text(course.name, color = Color.White, fontWeight = FontWeight.Bold, fontSize = 12.sp, modifier = Modifier.weight(1f, fill
= false))
311             Box(
312                 modifier = Modifier
313                     .background(
314                         if (course.isPublic) Accent.copy(alpha = 0.18f) else Color.White.copy(alpha = 0.1f),
315                         RoundedCornerShape(4.dp),
316                     )
317                     .padding(horizontal = 4.dp, vertical = 1.dp),
318             ) {
319                 Text(
320                     if (course.isPublic) "공개" else "개인",
321                     color = if (course.isPublic) Accent else Muted,
322                     fontSize = 8.sp,
323                     fontWeight = FontWeight.Bold,
324                 )
325             }
326         }
327         Text(
328             "%1fkM · 현재 위치에서 %0fm".format(
329                 course.distanceMeters / 1000.0,
330                 course.distanceFromMeMeters,
331             ),
332             color = Muted,
333             fontSize = 9.sp,
334         )
335     }
336 }
337
338 @Composable
339 private fun CourseDetailScreen(state: WatchRunState, viewModel: WatchViewModel) {
340     val course = OfflineCourseRepository.courses.value

```

```

341         .firstOrNull { it.courseId == state.selectedCourseId }
342     WatchPage(scrollable = true) {
343         BackTitle("코스 도전", viewModel::navigateBack)
344         Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(5.dp)) {
345             Text(course?.name ?: state.courseName.orEmpty(), fontWeight = FontWeight.Bold, fontSize = 17.sp, modifier = Modifier.weight
(1f, fill = false))
346             if (course != null) {
347                 Box(
348                     modifier = Modifier
349                         .background(
350                             if (course.isPublic) Accent.copy(alpha = 0.18f) else Color.White.copy(alpha = 0.1f),
351                             RoundedCornerShape(4.dp),
352                         )
353                     .padding(horizontal = 5.dp, vertical = 2.dp),
354                 ) {
355                     Text(
356                         if (course.isPublic) "공개" else "개인",
357                         color = if (course.isPublic) Accent else Muted,
358                         fontSize = 9.sp,
359                         fontWeight = FontWeight.Bold,
360                     )
361                 }
362             }
363         }
364         Text(
365             "%.2f km".format((course?.distanceMeters ?: state.courseDistanceMeters) / 1000.0),
366             color = Accent,
367             fontSize = 22.sp,
368             fontWeight = FontWeight.Bold,
369         )
370         course?.description?.let {
371             Text(it, color = Muted, fontSize = 9.sp, textAlign = TextAlign.Center)
372         }
373         Text("경로 ${course?.points?.size ?: 0}개 지점 오프라인 저장됨", color = Muted, fontSize = 9.sp)
374         PrimaryAction("코스 도전 시작", Icons.Filled.PlayArrow, onClick = viewModel::startSelectedCourse)
375         SecondaryAction(
376             label = "에서 경로살펴보기",
377             icon = Icons.Filled.PhoneAndroid,
378             onClick = viewModel::openCourseOnPhone,
379             color = if (state.isPhoneConnected) Color.White else Muted,
380         )
381     }
382 }
383
384 @Composable
385 private fun GoalTypeScreen(viewModel: WatchViewModel) {
386     WatchPage(scrollable = true) {
387         BackTitle("목표 선택", viewModel::navigateBack)
388         SecondaryAction("시간", Icons.Filled.Timer, onClick = {
389             viewModel.navigate(WatchScreen.TIME_GOAL)
390         })
391         SecondaryAction("거리", Icons.Filled.Route, onClick = {
392             viewModel.navigate(WatchScreen.DISTANCE_GOAL)
393         })
394         SecondaryAction("인터벌", Icons.AutoMirrored.Filled.DirectionsRun, onClick = {
395             viewModel.navigate(WatchScreen.INTERVAL_GOAL)
396         })
397     }
398 }
399
400 @Composable
401 private fun TimeGoalScreen(viewModel: WatchViewModel) {
402     var hours by remember { mutableIntStateOf(0) }
403     var minutes by remember { mutableIntStateOf(30) }
404     var completion by remember {
405         mutableStateOf(viewModel.state.value.goalCompletionAction)
406     }
407     var selected by remember { mutableStateOf<String?>(null) }
408
409     RotarySettingPage(
410         title = "시간 목표",
411         selected = selected,
412         onBack = viewModel::navigateBack,
413         onRotate = { delta ->
414             when (selected) {
415                 "hours" -> hours = wrap(hours, delta, 0, 5)
416                 "minutes" -> minutes = wrap(minutes, delta, 0, 59)
417             }
418         },
419         picker = when (selected) {
420             "hours" -> NumberPickerSpec("시간", hours, 0, 5, onValueChange = { hours = it })
421             "minutes" -> NumberPickerSpec("분", minutes, 0, 59, "%02d", onValueChange = { minutes = it })
422             else -> null
423         },
424         onDismissPicker = { selected = null },
425     ) {

```



```

426     SettingLabel("목표 시간")
427     Row(verticalAlignment = Alignment.CenterVertically) {
428         DialValue("$hours", "시간", selected == "hours") {
429             selected = toggleSelection(selected, "hours")
430         }
431         Text(":", color = Muted, fontSize = 24.sp, modifier = Modifier.padding(horizontal = 4.dp))
432         DialValue("%02d".format(minutes), "분", selected == "minutes") {
433             selected = toggleSelection(selected, "minutes")
434         }
435     }
436     Text("${hours * 60 + minutes}분 동안 달리기", color = Muted, fontSize = 11.sp)
437     CompletionChoice(completion) { completion = it }
438     PrimaryAction("시작하기", Icons.Filled.PlayArrow) {
439         val totalMinutes = (hours * 60 + minutes).coerceAtLeast(1)
440         viewModel.start(RunGoal.Time(totalMinutes, completion))
441     }
442 }
443 }
444
445 @Composable
446 private fun DistanceGoalScreen(viewModel: WatchViewModel) {
447     var km by remember { mutableIntStateOf(5) }
448     var decimal by remember { mutableIntStateOf(0) }
449     var completion by remember {
450         mutableStateOf(viewModel.state.value.goalCompletionAction)
451     }
452     var selected by remember { mutableStateOf<String?>(null) }
453
454     RotarySettingPage(
455         title = "거리 목표",
456         selected = selected,
457         onBack = viewModel::navigateBack,
458         onRotate = { delta ->
459             when (selected) {
460                 "km" -> km = wrap(km, delta, 0, 50)
461                 "decimal" -> decimal = wrap(decimal, delta, 0, 9)
462             }
463         },
464         picker = when (selected) {
465             "km" -> NumberPickerSpec("킬로미터", km, 0, 50, onValueChange = { km = it })
466             "decimal" -> NumberPickerSpec("100미터 단위", decimal, 0, 9, onValueChange = { decimal = it })
467             else -> null
468         },
469         onDismissPicker = { selected = null },
470     ) {
471         SettingLabel("목표 거리")
472         Row(verticalAlignment = Alignment.CenterVertically) {
473             DialValue("$km", "km", selected == "km") {
474                 selected = toggleSelection(selected, "km")
475             }
476             Text(".", color = Muted, fontSize = 26.sp)
477             DialValue("$decimal", "100m", selected == "decimal") {
478                 selected = toggleSelection(selected, "decimal")
479             }
480         }
481         Text("%.1fkm 달리기".format(km + decimal / 10f), color = Muted, fontSize = 11.sp)
482         CompletionChoice(completion) { completion = it }
483         PrimaryAction("시작하기", Icons.Filled.PlayArrow) {
484             val meters = (km * 1000 + decimal * 100).coerceAtLeast(100)
485             viewModel.start(RunGoal.Distance(meters, completion))
486         }
487     }
488 }
489
490 private enum class SegmentMode { TIME, DISTANCE }
491
492 @Composable
493 private fun IntervalGoalScreen(viewModel: WatchViewModel) {
494     var workMode by remember { mutableStateOf(SegmentMode.TIME) }
495     var workMinutes by remember { mutableIntStateOf(3) }
496     var workSeconds by remember { mutableIntStateOf(0) }
497     var workKm by remember { mutableIntStateOf(0) }
498     var workDecimal by remember { mutableIntStateOf(4) }
499     var recoveryMode by remember { mutableStateOf(SegmentMode.TIME) }
500     var recoveryMinutes by remember { mutableIntStateOf(1) }
501     var recoverySeconds by remember { mutableIntStateOf(0) }
502     var recoveryKm by remember { mutableIntStateOf(0) }
503     var recoveryDecimal by remember { mutableIntStateOf(2) }
504     var sets by remember { mutableIntStateOf(4) }
505     var completion by remember {
506         mutableStateOf(viewModel.state.value.goalCompletionAction)
507     }
508     var selected by remember { mutableStateOf<String?>(null) }
509
510     RotarySettingPage(
511         title = "인터벌",

```

```

512     selected = selected,
513     onBack = viewModel::navigateBack,
514     onRotate = { delta ->
515         when (selected) {
516             "workMinutes" -> workMinutes = wrap(workMinutes, delta, 0, 60)
517             "workSeconds" -> workSeconds = wrap(workSeconds, delta, 0, 59)
518             "workKm" -> workKm = wrap(workKm, delta, 0, 20)
519             "workDecimal" -> workDecimal = wrap(workDecimal, delta, 0, 9)
520             "recoveryMinutes" -> recoveryMinutes = wrap(recoveryMinutes, delta, 0, 30)
521             "recoverySeconds" -> recoverySeconds = wrap(recoverySeconds, delta, 0, 59)
522             "recoveryKm" -> recoveryKm = wrap(recoveryKm, delta, 0, 10)
523             "recoveryDecimal" -> recoveryDecimal = wrap(recoveryDecimal, delta, 0, 9)
524             "sets" -> sets = wrap(sets, delta, 1, 20)
525         }
526     },
527     picker = intervalPickerSpec(
528         selected = selected,
529         workMinutes = workMinutes,
530         workSeconds = workSeconds,
531         workKm = workKm,
532         workDecimal = workDecimal,
533         recoveryMinutes = recoveryMinutes,
534         recoverySeconds = recoverySeconds,
535         recoveryKm = recoveryKm,
536         recoveryDecimal = recoveryDecimal,
537         sets = sets,
538         onWorkMinutes = { workMinutes = it },
539         onWorkSeconds = { workSeconds = it },
540         onWorkKm = { workKm = it },
541         onWorkDecimal = { workDecimal = it },
542         onRecoveryMinutes = { recoveryMinutes = it },
543         onRecoverySeconds = { recoverySeconds = it },
544         onRecoveryKm = { recoveryKm = it },
545         onRecoveryDecimal = { recoveryDecimal = it },
546         onSets = { sets = it },
547     ),
548     onDismissPicker = { selected = null },
549 ) {
550     IntervalSegmentEditor(
551         title = "운동",
552         mode = workMode,
553         onModeChange = { workMode = it; selected = null },
554         minutes = workMinutes,
555         seconds = workSeconds,
556         km = workKm,
557         decimal = workDecimal,
558         selected = selected,
559         prefix = "work",
560         onSelect = { selected = toggleSelection(selected, it) },
561     )
562     IntervalSegmentEditor(
563         title = "회복",
564         mode = recoveryMode,
565         onModeChange = { recoveryMode = it; selected = null },
566         minutes = recoveryMinutes,
567         seconds = recoverySeconds,
568         km = recoveryKm,
569         decimal = recoveryDecimal,
570         selected = selected,
571         prefix = "recovery",
572         onSelect = { selected = toggleSelection(selected, it) },
573     )
574     SettingLabel("반복 횟수")
575     DialValue("$sets", "회", selected == "sets") {
576         selected = toggleSelection(selected, "sets")
577     }
578     CompletionChoice(completion) { completion = it }
579     PrimaryAction("시작하기", Icons.Filled.PlayArrow) {
580         viewModel.start(
581             RunGoal.Interval(
582                 work = intervalTarget(workMode, workMinutes, workSeconds, workKm, workDecimal),
583                 recovery = intervalTarget(
584                     recoveryMode,
585                     recoveryMinutes,
586                     recoverySeconds,
587                     recoveryKm,
588                     recoveryDecimal,
589                 ),
590                 sets = sets,
591                 completionAction = completion,
592             ),
593         )
594     }
595 }
596 }
597

```

```

598 @Composable
599 private fun IntervalSegmentEditor(
600     title: String,
601     mode: SegmentMode,
602     onModeChange: (SegmentMode) -> Unit,
603     minutes: Int,
604     seconds: Int,
605     km: Int,
606     decimal: Int,
607     selected: String?,
608     prefix: String,
609     onSelect: (String) -> Unit,
610 ) {
611     SettingLabel(title)
612     Row(horizontalArrangement = Arrangement.spacedBy(6.dp)) {
613         ModeChip("시간", mode == SegmentMode.TIME) { onModeChange(SegmentMode.TIME) }
614         ModeChip("거리", mode == SegmentMode.DISTANCE) { onModeChange(SegmentMode.DISTANCE) }
615     }
616     Row(verticalAlignment = Alignment.CenterVertically) {
617         if (mode == SegmentMode.TIME) {
618             DialValue("$minutes", "분", selected == "${prefix}Minutes") {
619                 onSelect("${prefix}Minutes")
620             }
621             Spacer(Modifier.width(5.dp))
622             DialValue("%02d".format(seconds), "초", selected == "${prefix}Seconds") {
623                 onSelect("${prefix}Seconds")
624             }
625         } else {
626             DialValue("$km", "km", selected == "${prefix}Km") {
627                 onSelect("${prefix}Km")
628             }
629             Text(".", color = Muted, fontSize = 24.sp)
630             DialValue("$decimal", "100m", selected == "${prefix}Decimal") {
631                 onSelect("${prefix}Decimal")
632             }
633         }
634     }
635 }
636
637 @Composable
638 private fun TrackingScreen(state: WatchRunState, viewModel: WatchViewModel) {
639     val pagerState = rememberPagerState(initialPage = 1, pageCount = { 2 })
640     Box(modifier = Modifier.fillMaxSize()) {
641         HorizontalPager(state = pagerState, modifier = Modifier.fillMaxSize()) { page ->
642             if (page == 0) {
643                 TrackingControls(state, viewModel)
644             } else {
645                 TrackingMetrics(state)
646             }
647         }
648         Row {
649             modifier = Modifier.align(Alignment.BottomCenter).padding(bottom = 8.dp),
650             horizontalArrangement = Arrangement.spacedBy(5.dp),
651         ) {
652             repeat(2) { index ->
653                 Box(
654                     Modifier
655                         .size(if (pagerState.currentPage == index) 7.dp else 5.dp)
656                         .background(
657                             if (pagerState.currentPage == index) Accent else Muted.copy(alpha = 0.45f),
658                             CircleShape,
659                         ),
660                 )
661             }
662         }
663     }
664 }
665
666 @Composable
667 private fun TrackingMetrics(state: WatchRunState) {
668     WatchPage {
669         Text(
670             when {
671                 state.isOffCourse -> "코스 이탈 · 경로로 돌아가세요"
672                 state.courseName != null -> "${state.courseName} ${state.courseProgressPercent}%"
673                 else -> state.remainingLabel ?: "자유 러닝"
674             },
675             color = if (state.gpsStatus == "POOR" || state.isOffCourse) Danger else Accent,
676             fontSize = 11.sp,
677             textAlign = TextAlign.Center,
678         )
679         Text(formatDuration(state.elapsedSeconds), fontSize = if (compact) 32.sp else 38.sp, fontWeight = FontWeight.Bold)
680         Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceEvenly) {
681             Metric("페이스", formatPace(state.paceMinPerKm))
682             Metric("거리", "%.2fkm".format(state.distanceMeters / 1000.0))
683         }

```

```

684         state.progressPercent?.let { progress ->
685             LinearProgressIndicator(
686                 progress = { progress / 100f },
687                 modifier = Modifier.fillMaxWidth().height(7.dp),
688                 color = Accent,
689                 trackColor = Color.White.copy(alpha = 0.12f),
690             )
691         }
692         if (state.courseName != null) {
693             LinearProgressIndicator(
694                 progress = { state.courseProgressPercent / 100f },
695                 modifier = Modifier.fillMaxWidth().height(7.dp),
696                 color = if (state.isOffCourse) Danger else Accent,
697                 trackColor = Color.White.copy(alpha = 0.12f),
698             )
699             state.distanceToCourseMeters?.let {
700                 Text("코스까지 ${it.toInt()}m", color = if (state.isOffCourse) Danger else Muted, fontSize = 9.sp)
701             }
702         }
703         Row(horizontalArrangement = Arrangement.spacedBy(14.dp)) {
704             Metric("심박", state.heartRateBpm?.toString() ?: "--")
705             Metric("케이던스", state.cadenceSpm?.toString() ?: "--")
706         }
707         Text("오른쪽으로 밀어 제어", color = Muted, fontSize = 9.sp)
708     }
709 }
710
711 @Composable
712 private fun TrackingControls(state: WatchRunState, viewModel: WatchViewModel) {
713     var confirmAction by remember { mutableStateOf<ConfirmAction?>(null) }
714
715     if (confirmAction != null) {
716         ConfirmDialog(
717             message = if (confirmAction == ConfirmAction.FINISH) "러닝을 종료하시겠습니까?" else "러닝을 취소하시겠습니까?",
718             confirmLabel = if (confirmAction == ConfirmAction.FINISH) "종료" else "취소",
719             confirmColor = if (confirmAction == ConfirmAction.FINISH) Accent else Danger,
720             onConfirm = {
721                 val action = confirmAction
722                 confirmAction = null
723                 if (action == ConfirmAction.FINISH) viewModel.finish() else viewModel.abandon()
724             },
725             onDismiss = { confirmAction = null },
726         )
727         return
728     }
729
730     BoxWithConstraints(Modifier.fillMaxSize()) {
731         val btnSize = if (maxHeight <= 200.dp) 50.dp else 58.dp
732         val iconSize = if (maxHeight <= 200.dp) 22.dp else 26.dp
733         val gap = if (maxHeight <= 200.dp) 6.dp else 8.dp
734         Column(
735             modifier = Modifier
736                 .fillMaxSize()
737                 .padding(horizontal = 16.dp, vertical = if (maxHeight <= 200.dp) 8.dp else 12.dp),
738             horizontalAlignment = Alignment.CenterHorizontally,
739             verticalArrangement = Arrangement.spacedBy(4.dp, Alignment.CenterVertically),
740         ) {
741             if (state.isOffCourse && state.isPaused) {
742                 Text(
743                     "경로 복귀 시 자동 재개",
744                     color = Danger,
745                     fontSize = 10.sp,
746                     fontWeight = FontWeight.Bold,
747                     textAlign = TextAlign.Center,
748                 )
749             } else {
750                 Text(
751                     "${formatDuration(state.elapsedSeconds)} · %.2fkm".format(state.distanceMeters / 1000.0),
752                     color = if (state.isPaused) Accent else Muted,
753                     fontSize = 11.sp,
754                     fontWeight = if (state.isPaused) FontWeight.Bold else FontWeight.Normal,
755                 )
756             }
757             Row(horizontalArrangement = Arrangement.spacedBy(gap)) {
758                 CompactControlButton(
759                     label = if (state.isPaused) "재생" else "일시정지",
760                     icon = if (state.isPaused) Icons.Filled.PlayArrow else Icons.Filled.Pause,
761                     color = if (state.isOffCourse && state.isPaused) Muted else Accent,
762                     size = btnSize,
763                     iconSize = iconSize,
764                     onClick = if (state.isPaused) viewModel::resume else viewModel::pause,
765                 )
766                 CompactControlButton(
767                     label = "종료",
768                     icon = Icons.Filled.Stop,
769                     color = Color.White,

```

```

770         size = btnSize,
771         iconSize = iconSize,
772         onClick = { confirmAction = ConfirmAction.FINISH },
773     )
774 }
775 Row(horizontalArrangement = Arrangement.spacedBy(gap)) {
776     CompactControlButton(
777         label = "취소",
778         icon = Icons.Filled.Close,
779         color = Danger,
780         size = btnSize,
781         iconSize = iconSize,
782         onClick = { confirmAction = ConfirmAction.ABANDON },
783     )
784     CompactControlButton(
785         label = "설정",
786         icon = Icons.Filled.Settings,
787         color = Color.White,
788         size = btnSize,
789         iconSize = iconSize,
790         onClick = viewModel::openSettings,
791     )
792 }
793 }
794 }
795 }
796
797 private enum class ConfirmAction { FINISH, ABANDON }
798
799 @Composable
800 private fun RunningSettingsScreen(
801     state: WatchRunState,
802     viewModel: WatchViewModel,
803 ) {
804     WatchPage {
805         BackTitle("러닝 설정", viewModel::navigateBack)
806         SettingToggleRow(
807             label = "음성 안내",
808             icon = Icons.AutoMirrored.Filled.VolumeUp,
809             checked = state.voiceGuidanceEnabled,
810             onCheckedChange = viewModel::setVoiceGuidanceEnabled,
811         )
812         SettingToggleRow(
813             label = "자동 일시정지",
814             icon = Icons.Filled.Pause,
815             checked = state.autoPauseEnabled,
816             onCheckedChange = viewModel::setAutoPauseEnabled,
817         )
818         SettingLabel("목표 달성 후")
819         Row(horizontalArrangement = Arrangement.spacedBy(6.dp)) {
820             ModeChip(
821                 "자동 일시정지",
822                 state.goalCompletionAction == GoalCompletionAction.PAUSE,
823             ) {
824                 viewModel.setGoalCompletionAction(GoalCompletionAction.PAUSE)
825             }
826             ModeChip(
827                 "계속 기록",
828                 state.goalCompletionAction == GoalCompletionAction.CONTINUE,
829             ) {
830                 viewModel.setGoalCompletionAction(GoalCompletionAction.CONTINUE)
831             }
832         }
833         Text(
834             "자동 일시정지는 다음 러닝부터 적용됩니다",
835             color = Muted,
836             fontSize = 9.sp,
837             textAlign = TextAlign.Center,
838         )
839     }
840 }
841
842 @Composable
843 private fun PausedScreen(state: WatchRunState, viewModel: WatchViewModel) {
844     TrackingControls(state, viewModel)
845 }
846
847 @Composable
848 private fun SummaryScreen(
849     state: WatchRunState,
850     onDone: () -> Unit,
851     onOpenRunOnPhone: () -> Unit = {},
852 ) {
853     WatchPage(scrollable = true) {
854         Text("런 완료", color = Accent, fontWeight = FontWeight.Bold, fontSize = 20.sp)
855         Text("%.2f km".format(state.distanceMeters / 1000.0), fontWeight = FontWeight.Bold, fontSize = 32.sp)

```

```

856     Text(formatDuration(state.elapsedSeconds), fontSize = 18.sp)
857     Text("평균 페이스 ${formatPace(state.paceMinPerKm)}/km", color = Muted, fontSize = 11.sp)
858     Text(
859         state.phoneStatusMessage ?: "폰 연결 시 기록과 GPS 경로를 자동 동기화합니다",
860         color = Muted,
861         fontSize = 10.sp,
862         textAlign = TextAlign.Center,
863     )
864     if (state.syncedRunId != null && state.isPhoneConnected) {
865         SecondaryAction(
866             label = "에서 코스로 등록",
867             icon = Icons.Filled.PhoneAndroid,
868             onClick = onOpenRunOnPhone,
869         )
870     }
871     PrimaryAction("완료", Icons.Filled.Check, onClick = onDone)
872 }
873 }
874
875 @Composable
876 private fun RotarySettingPage(
877     title: String,
878     selected: String?,
879     onBack: () -> Unit,
880     onRotate: (Int) -> Unit,
881     picker: NumberPickerSpec?,
882     onDismissPicker: () -> Unit,
883     content: @Composable ColumnScope.() -> Unit,
884 ) {
885     if (picker != null) {
886         FullScreenNumberPicker(picker, onDismissPicker)
887         return
888     }
889
890     val scrollState = rememberScrollState()
891     val focusRequester = remember { FocusRequester() }
892     var rotaryAccumulator by remember { mutableFloatStateOf(0f) }
893     var lastStepMs by remember { mutableStateOf(0L) }
894     val valueState = rememberScrollableState { delta ->
895         rotaryAccumulator += delta
896         if (kotlin.math.abs(rotaryAccumulator) >= ROTARY_STEP_THRESHOLD) {
897             val now = System.currentTimeMillis()
898             val direction = if (rotaryAccumulator > 0f) 1 else -1
899             rotaryAccumulator = 0f
900             if (now - lastStepMs >= ROTARY_DEBOUNCE_MS) {
901                 lastStepMs = now
902                 onRotate(direction)
903             }
904         }
905         delta
906     }
907     val rotaryBehavior = RotaryScrollableDefaults.behavior(
908         scrollableState = if (selected == null) scrollState else valueState,
909     )
910
911     BoxWithConstraints(Modifier.fillMaxSize()) {
912         val compact = maxHeight <= 260.dp
913         Column(
914             modifier = Modifier
915                 .fillMaxSize()
916                 .requestFocusOnHierarchyActive()
917                 .rotaryScrollable(
918                     behavior = rotaryBehavior,
919                     focusRequester = focusRequester,
920                 )
921                 .verticalScroll(scrollState)
922                 .padding(horizontal = if (compact) 18.dp else 24.dp)
923                 .padding(top = if (compact) 8.dp else 14.dp, bottom = 26.dp),
924             horizontalAlignment = Alignment.CenterHorizontally,
925             verticalArrangement = Arrangement.spacedBy(if (compact) 7.dp else 10.dp),
926         ) {
927             BackTitle(title, onBack)
928             content()
929             Text(
930                 "숫자를 누르면 확대 선택 화면이 열립니다",
931                 color = Muted,
932                 fontSize = 9.sp,
933                 textAlign = TextAlign.Center,
934             )
935         }
936     }
937 }
938
939 private data class NumberPickerSpec(
940     val title: String,
941     val value: Int,

```

```

942     val min: Int,
943     val max: Int,
944     val format: String = "%d",
945     val onValueChange: (Int) -> Unit,
946 )
947
948 @Composable
949 private fun FullScreenNumberPicker(
950     spec: NumberPickerSpec,
951     onDone: () -> Unit,
952 ) {
953     val values = remember(spec.min, spec.max) { (spec.min..spec.max).toList() }
954     var localValue by remember(spec.title) { mutableIntStateOf(spec.value) }
955     val listState = rememberLazyListState(
956         initialFirstVisibleItemIndex = (spec.value - spec.min).coerceIn(0, values.lastIndex),
957     )
958     val scope = rememberCoroutineScope()
959     var rotaryAccumulator by remember { mutableFloatStateOf(0f) }
960     var lastStepMs by remember { mutableStateOf(0L) }
961     val rotaryState = rememberScrollableState { delta ->
962         rotaryAccumulator += delta
963         if (kotlin.math.abs(rotaryAccumulator) >= ROTARY_STEP_THRESHOLD) {
964             val now = System.currentTimeMillis()
965             val direction = if (rotaryAccumulator > 0f) 1 else -1
966             rotaryAccumulator = 0f
967             val next = (localValue + direction).coerceIn(spec.min, spec.max)
968             if (next != localValue && now - lastStepMs >= ROTARY_DEBOUNCE_MS) {
969                 lastStepMs = now
970                 localValue = next
971                 scope.launch { listState.scrollToItem(next - spec.min) }
972             }
973         }
974         delta
975     }
976     val focusRequester = remember { FocusRequester() }
977     val rotaryBehavior = RotaryScrollableDefaults.behavior(rotaryState)
978
979     val commitAndDone = {
980         spec.onValueChange(localValue)
981         onDone()
982     }
983
984     BackHandler(onBack = commitAndDone)
985     LaunchedEffect(listState) {
986         snapshotFlow { listState.isScrollInProgress }
987             .distinctUntilChanged()
988             .collect { scrolling ->
989                 if (!scrolling) {
990                     val layout = listState.layoutInfo
991                     val center = (layout.viewportStartOffset + layout.viewportEndOffset) / 2
992                     val centered = layout.visibleItemsInfo.minByOrNull {
993                         kotlin.math.abs((it.offset + it.size / 2) - center)
994                     } ?: return@collect
995                     localValue = values[centered.index]
996                 }
997             }
998     }
999
1000     Box(
1001         modifier = Modifier
1002             .fillMaxSize()
1003             .background(Background)
1004             .requestFocusOnHierarchyActive()
1005             .rotaryScrollable(rotaryBehavior, focusRequester),
1006     ) {
1007         LazyColumn(
1008             state = listState,
1009             modifier = Modifier.fillMaxSize(),
1010             contentPadding = androidx.compose.foundation.layout.PaddingValues(vertical = 72.dp),
1011             flingBehavior = rememberSnapFlingBehavior(listState),
1012             horizontalAlignment = Alignment.CenterHorizontally,
1013         ) {
1014             items(values, key = { it }) { value ->
1015                 val selected = value == localValue
1016                 Text(
1017                     text = spec.format.format(value),
1018                     modifier = Modifier
1019                         .fillMaxWidth()
1020                         .height(52.dp)
1021                         .clickable {
1022                             localValue = value
1023                             commitAndDone()
1024                         }
1025                     .padding(vertical = 8.dp),
1026                     color = if (selected) Accent else Muted,
1027                     fontSize = 28.sp,

```

```

1028         fontWeight = FontWeight.Bold,
1029         textAlign = TextAlign.Center,
1030     )
1031     }
1032 }
1033 Text(
1034     spec.title,
1035     modifier = Modifier.align(Alignment.TopCenter).padding(top = 14.dp),
1036     color = Color.White,
1037     fontSize = 13.sp,
1038     fontWeight = FontWeight.Bold,
1039 )
1040 Button(
1041     onClick = commitAndDone,
1042     modifier = Modifier.align(Alignment.BottomCenter).padding(bottom = 10.dp).height(38.dp),
1043     colors = ButtonDefaults.buttonColors(containerColor = Accent, contentColor = Color.Black),
1044 ) {
1045     Text("확인", fontWeight = FontWeight.Bold)
1046 }
1047 }
1048 }
1049
1050 private const val ROTARY_STEP_THRESHOLD = 96f
1051 private const val ROTARY_DEBOUNCE_MS = 150L
1052
1053 private data class WatchPageScope(val compact: Boolean)
1054
1055 @Composable
1056 private fun WatchPage(
1057     scrollable: Boolean = true,
1058     content: @Composable WatchPageScope.() -> Unit,
1059 ) {
1060     BoxWithConstraints(Modifier.fillMaxSize()) {
1061         val compact = maxHeight <= 260.dp
1062         val scrollState = rememberScrollState()
1063         val focusRequester = remember { FocusRequester() }
1064         val rotaryBehavior = RotaryScrollableDefaults.behavior(scrollableState = scrollableState)
1065         Column(
1066             modifier = Modifier
1067                 .fillMaxSize()
1068                 .then(
1069                     if (scrollable) {
1070                         Modifier
1071                             .requestFocusOnHierarchyActive()
1072                             .rotaryScrollable(
1073                                 behavior = rotaryBehavior,
1074                                 focusRequester = focusRequester,
1075                             )
1076                             .verticalScroll(scrollState)
1077                     } else {
1078                         Modifier
1079                     },
1080                 )
1081             .padding(horizontal = if (compact) 18.dp else 24.dp)
1082             .padding(
1083                 top = if (compact) 10.dp else 16.dp,
1084                 bottom = if (scrollable) {
1085                     if (compact) 28.dp else 36.dp
1086                 } else {
1087                     if (compact) 10.dp else 16.dp
1088                 },
1089             ),
1090             horizontalAlignment = Alignment.CenterHorizontally,
1091             verticalArrangement = Arrangement.spacedBy(
1092                 if (compact) 7.dp else 10.dp,
1093                 Alignment.CenterVertically,
1094             ),
1095         ) {
1096             WatchPageScope(compact).content()
1097         }
1098     }
1099 }
1100
1101 @Composable
1102 private fun DialValue(
1103     value: String,
1104     unit: String,
1105     selected: Boolean,
1106     onClick: () -> Unit,
1107 ) {
1108     Column(
1109         modifier = Modifier
1110             .heightIn(min = 54.dp)
1111             .background(if (selected) Accent.copy(alpha = 0.12f) else SurfaceColor, RoundedCornerShape(8.dp))
1112             .border(1.dp, if (selected) Accent else Color.Transparent, RoundedCornerShape(8.dp))
1113             .clickable(onClick = onClick)

```



```

1114         .padding(horizontal = 12.dp, vertical = 6.dp),
1115         horizontalAlignment = Alignment.CenterHorizontally,
1116     ) {
1117         Text(value, fontWeight = FontWeight.Bold, fontSize = 24.sp, color = if (selected) Accent else Color.White)
1118         Text(unit, color = Muted, fontSize = 9.sp)
1119     }
1120 }
1121
1122 @Composable
1123 private fun CompletionChoice(
1124     selected: GoalCompletionAction,
1125     onSelected: (GoalCompletionAction) -> Unit,
1126 ) {
1127     SettingLabel("목표 달성 후")
1128     Row(horizontalArrangement = Arrangement.spacedBy(6.dp)) {
1129         ModeChip("자동 일시정지", selected == GoalCompletionAction.PAUSE) {
1130             onSelected(GoalCompletionAction.PAUSE)
1131         }
1132         ModeChip("계속 기록", selected == GoalCompletionAction.CONTINUE) {
1133             onSelected(GoalCompletionAction.CONTINUE)
1134         }
1135     }
1136 }
1137
1138 @Composable
1139 private fun ModeChip(label: String, selected: Boolean, onClick: () -> Unit) {
1140     Box(
1141         modifier = Modifier
1142             .background(if (selected) Accent else SurfaceColor, RoundedCornerShape(16.dp))
1143             .clickable(onClick = onClick)
1144             .padding(horizontal = 10.dp, vertical = 7.dp),
1145         contentAlignment = Alignment.Center,
1146     ) {
1147         Text(label, color = if (selected) Color.Black else Color.White, fontSize = 10.sp, fontWeight = FontWeight.Bold)
1148     }
1149 }
1150
1151 @Composable
1152 private fun BackTitle(title: String, onBack: () -> Unit) {
1153     Row(
1154         modifier = Modifier
1155             .fillMaxWidth()
1156             .padding(horizontal = 10.dp, vertical = 6.dp),
1157         verticalAlignment = Alignment.CenterVertically,
1158     ) {
1159         IconButton(
1160             onClick = onBack,
1161             modifier = Modifier
1162                 .size(40.dp)
1163                 .background(SurfaceColor, CircleShape),
1164         ) {
1165             Icon(
1166                 Icons.AutoMirrored.Filled.KeyboardArrowLeft,
1167                 contentDescription = "뒤로가기",
1168                 modifier = Modifier.size(24.dp),
1169             )
1170         }
1171         Spacer(Modifier.width(8.dp))
1172         Text(
1173             text = title,
1174             modifier = Modifier.weight(1f),
1175             fontWeight = FontWeight.Bold,
1176             fontSize = 16.sp,
1177             maxLines = 1,
1178         )
1179     }
1180 }
1181
1182 @Composable
1183 private fun GpsStatusIndicator(gpsReady: Boolean) {
1184     val infiniteTransition = rememberInfiniteTransition(label = "gps_blink")
1185     val blinkAlpha by infiniteTransition.animateFloat(
1186         initialValue = 0.2f,
1187         targetValue = 1f,
1188         animationSpec = infiniteRepeatable(
1189             animation = tween(700, easing = LinearEasing),
1190             repeatMode = RepeatMode.Reverse,
1191         ),
1192         label = "blink",
1193     )
1194     Row(verticalAlignment = Alignment.CenterVertically) {
1195         Box(
1196             Modifier.size(8.dp).background(
1197                 color = if (gpsReady) Accent else Accent.copy(alpha = blinkAlpha),
1198                 shape = CircleShape,
1199             ),
1200     }

```

```

1200     )
1201     Spacer(Modifier.width(5.dp))
1202     Text(
1203         text = if (gpsReady) "GPS 준비됨" else "GPS 탐색 중",
1204         color = if (gpsReady) Accent else Muted,
1205         fontSize = 10.sp,
1206     )
1207 }
1208 }
1209
1210 @Composable
1211 private fun ConnectionLabel(connected: Boolean) {
1212     Row(verticalAlignment = Alignment.CenterVertically) {
1213         Box(Modifier.size(7.dp).background(if (connected) Accent else Muted, CircleShape))
1214         Spacer(Modifier.width(5.dp))
1215         Text(if (connected) "폰 연결됨" else "오프라인 가능", color = Muted, fontSize = 10.sp)
1216     }
1217 }
1218
1219 @Composable
1220 private fun PrimaryAction(
1221     label: String,
1222     icon: androidx.compose.ui.graphics.vector.ImageVector,
1223     compact: Boolean = false,
1224     onClick: () -> Unit,
1225 ) {
1226     Button(
1227         onClick = onClick,
1228         modifier = Modifier.fillMaxWidth().height(if (compact) 36.dp else 44.dp),
1229         colors = ButtonDefaults.buttonColors(containerColor = Accent, contentColor = Color.Black),
1230     ) {
1231         Icon(icon, contentDescription = null, modifier = Modifier.size(17.dp))
1232         Spacer(Modifier.width(5.dp))
1233         Text(label, fontWeight = FontWeight.Bold, fontSize = 12.sp)
1234     }
1235 }
1236
1237 @Composable
1238 private fun SecondaryAction(
1239     label: String,
1240     icon: androidx.compose.ui.graphics.vector.ImageVector,
1241     onClick: () -> Unit,
1242     color: Color = Color.White,
1243     compact: Boolean = false,
1244 ) {
1245     Button(
1246         onClick = onClick,
1247         modifier = Modifier.fillMaxWidth().height(if (compact) 36.dp else 42.dp),
1248         colors = ButtonDefaults.buttonColors(containerColor = SurfaceColor, contentColor = color),
1249     ) {
1250         Icon(icon, contentDescription = null, modifier = Modifier.size(16.dp))
1251         Spacer(Modifier.width(5.dp))
1252         Text(label, fontSize = 12.sp)
1253     }
1254 }
1255
1256 @Composable
1257 private fun CompactControlButton(
1258     label: String,
1259     icon: androidx.compose.ui.graphics.vector.ImageVector,
1260     color: Color,
1261     size: androidx.compose.ui.unit.Dp,
1262     iconSize: androidx.compose.ui.unit.Dp,
1263     onClick: () -> Unit,
1264 ) {
1265     Column(
1266         horizontalAlignment = Alignment.CenterHorizontally,
1267         verticalArrangement = Arrangement.spacedBy(3.dp),
1268     ) {
1269         IconButton(
1270             onClick = onClick,
1271             modifier = Modifier
1272                 .size(size)
1273                 .background(
1274                     color = if (color == Accent) Accent else SurfaceColor,
1275                     shape = RoundedCornerShape(10.dp),
1276                 )
1277                 .border(
1278                     width = 1.dp,
1279                     color = if (color == Accent) Color.Transparent else color.copy(alpha = 0.35f),
1280                     shape = RoundedCornerShape(10.dp),
1281                 ),
1282         ) {
1283             Icon(
1284                 imageVector = icon,
1285                 contentDescription = label,

```

```

1286         tint = if (color == Accent) Color.Black else color,
1287         modifier = Modifier.size(iconSize),
1288     )
1289 }
1290 Text(label, color = if (color == Danger) Danger else Muted, fontSize = 10.sp, maxLines = 1)
1291 }
1292 }
1293
1294 @Composable
1295 private fun RunControlButton(
1296     label: String,
1297     icon: androidx.compose.ui.graphics.vector.ImageVector,
1298     color: Color,
1299     onClick: () -> Unit,
1300     compact: Boolean,
1301 ) {
1302     Column(
1303         horizontalAlignment = Alignment.CenterHorizontally,
1304         verticalArrangement = Arrangement.spacedBy(5.dp),
1305     ) {
1306         IconButton(
1307             onClick = onClick,
1308             modifier = Modifier
1309                 .size(if (compact) 72.dp else 82.dp)
1310                 .background(
1311                     color = if (color == Accent) Accent else SurfaceColor,
1312                     shape = RoundedCornerShape(12.dp),
1313                 )
1314                 .border(
1315                     width = 1.dp,
1316                     color = if (color == Accent) Color.Transparent else color.copy(alpha = 0.35f),
1317                     shape = RoundedCornerShape(12.dp),
1318                 ),
1319         ) {
1320             Icon(
1321                 imageVector = icon,
1322                 contentDescription = label,
1323                 tint = if (color == Accent) Color.Black else color,
1324                 modifier = Modifier.size(if (compact) 28.dp else 32.dp),
1325             )
1326         }
1327         Text(
1328             text = label,
1329             color = if (color == Danger) Danger else Muted,
1330             fontSize = 9.sp,
1331             maxLines = 1,
1332         )
1333     }
1334 }
1335
1336 @Composable
1337 private fun SettingToggleRow(
1338     label: String,
1339     icon: androidx.compose.ui.graphics.vector.ImageVector,
1340     checked: Boolean,
1341     onCheckedChange: (Boolean) -> Unit,
1342 ) {
1343     Row(
1344         modifier = Modifier
1345             .fillMaxWidth()
1346             .background(SurfaceColor, RoundedCornerShape(8.dp))
1347             .clickable { onCheckedChange(!checked) }
1348             .padding(horizontal = 12.dp, vertical = 8.dp),
1349         verticalAlignment = Alignment.CenterVertically,
1350     ) {
1351         Icon(
1352             imageVector = icon,
1353             contentDescription = null,
1354             tint = if (checked) Accent else Muted,
1355             modifier = Modifier.size(18.dp),
1356         )
1357         Spacer(Modifier.width(8.dp))
1358         Text(
1359             text = label,
1360             modifier = Modifier.weight(1f),
1361             fontSize = 11.sp,
1362             fontWeight = FontWeight.SemiBold,
1363         )
1364         Switch(
1365             checked = checked,
1366             onCheckedChange = onCheckedChange,
1367         )
1368     }
1369 }
1370
1371 @Composable

```

```

1372 private fun Metric(label: String, value: String) {
1373     Column(horizontalAlignment = Alignment.CenterHorizontally) {
1374         Text(label, color = Muted, fontSize = 9.sp)
1375         Text(value, fontWeight = FontWeight.Bold, fontSize = 15.sp)
1376     }
1377 }
1378
1379 @Composable
1380 private fun SettingLabel(text: String) {
1381     Text(text, color = Muted, fontSize = 10.sp, fontWeight = FontWeight.Bold)
1382 }
1383
1384 @Composable
1385 private fun StatusText(text: String) {
1386     Text(text, color = Danger, fontSize = 9.sp, textAlign = TextAlign.Center)
1387 }
1388
1389 private fun toggleSelection(current: String?, target: String): String? =
1390     if (current == target) null else target
1391
1392 private fun wrap(value: Int, delta: Int, min: Int, max: Int, step: Int = 1): Int {
1393     val next = value + delta
1394     return when {
1395         next > max -> min
1396         next < min -> max
1397         else -> (next / step) * step
1398     }
1399 }
1400
1401 private fun intervalPickerSpec(
1402     selected: String?,
1403     workMinutes: Int,
1404     workSeconds: Int,
1405     workKm: Int,
1406     workDecimal: Int,
1407     recoveryMinutes: Int,
1408     recoverySeconds: Int,
1409     recoveryKm: Int,
1410     recoveryDecimal: Int,
1411     sets: Int,
1412     onWorkMinutes: (Int) -> Unit,
1413     onWorkSeconds: (Int) -> Unit,
1414     onWorkKm: (Int) -> Unit,
1415     onWorkDecimal: (Int) -> Unit,
1416     onRecoveryMinutes: (Int) -> Unit,
1417     onRecoverySeconds: (Int) -> Unit,
1418     onRecoveryKm: (Int) -> Unit,
1419     onRecoveryDecimal: (Int) -> Unit,
1420     onSets: (Int) -> Unit,
1421 ): NumberPickerSpec? = when (selected) {
1422     "workMinutes" -> NumberPickerSpec("운동 분", workMinutes, 0, 60, onValueChange = onWorkMinutes)
1423     "workSeconds" -> NumberPickerSpec(
1424         "운동 초",
1425         workSeconds,
1426         0,
1427         59,
1428         "%02d",
1429         onWorkSeconds,
1430     )
1431     "workKm" -> NumberPickerSpec("운동 km", workKm, 0, 20, onValueChange = onWorkKm)
1432     "workDecimal" -> NumberPickerSpec("운동 100m", workDecimal, 0, 9, onValueChange = onWorkDecimal)
1433     "recoveryMinutes" -> NumberPickerSpec(
1434         "회복 분",
1435         recoveryMinutes,
1436         0,
1437         30,
1438         onValueChange = onRecoveryMinutes,
1439     )
1440     "recoverySeconds" -> NumberPickerSpec(
1441         "회복 초",
1442         recoverySeconds,
1443         0,
1444         59,
1445         "%02d",
1446         onRecoverySeconds,
1447     )
1448     "recoveryKm" -> NumberPickerSpec("회복 km", recoveryKm, 0, 10, onValueChange = onRecoveryKm)
1449     "recoveryDecimal" -> NumberPickerSpec(
1450         "회복 100m",
1451         recoveryDecimal,
1452         0,
1453         9,
1454         onValueChange = onRecoveryDecimal,
1455     )
1456     "sets" -> NumberPickerSpec("반복 횟수", sets, 1, 20, onValueChange = onSets)
1457     else -> null

```

```

1458 }
1459
1460 private fun intervalTarget(
1461     mode: SegmentMode,
1462     minutes: Int,
1463     seconds: Int,
1464     km: Int,
1465     decimal: Int,
1466 ): IntervalTarget = when (mode) {
1467     SegmentMode.TIME -> IntervalTarget.Time((minutes * 60 + seconds).coerceAtLeast(10))
1468     SegmentMode.DISTANCE -> IntervalTarget.Distance((km * 1000 + decimal * 100).coerceAtLeast(100))
1469 }
1470
1471 @Composable
1472 private fun PreparingScreen(
1473     state: WatchRunState,
1474     onConfirm: () -> Unit,
1475     onCancel: () -> Unit,
1476 ) {
1477     val infiniteTransition = rememberInfiniteTransition(label = "gps_blink")
1478     val blinkAlpha by infiniteTransition.animateFloat(
1479         initialValue = 0.2f,
1480         targetValue = 1f,
1481         animationSpec = infiniteRepeatable(
1482             animation = tween(700),
1483             repeatMode = RepeatMode.Reverse,
1484         ),
1485         label = "blink",
1486     )
1487
1488     WatchPage {
1489         Text("준비 중", fontWeight = FontWeight.Bold, fontSize = 18.sp)
1490         Spacer(Modifier.height(4.dp))
1491
1492         // GPS 상태
1493         Row(
1494             verticalAlignment = Alignment.CenterVertically,
1495             horizontalArrangement = Arrangement.spacedBy(8.dp),
1496         ) {
1497             Box(
1498                 modifier = Modifier
1499                     .size(10.dp)
1500                     .background(
1501                         color = if (state.gpsReady) Accent
1502                             else Accent.copy(alpha = blinkAlpha),
1503                         shape = CircleShape,
1504                     ),
1505             )
1506             Text(
1507                 text = if (state.gpsReady) "GPS 준비됨" else "GPS 탐색 중...",
1508                 color = if (state.gpsReady) Accent else Muted,
1509                 fontSize = 13.sp,
1510             )
1511         }
1512
1513         if (state.gpsReady) {
1514             Text("✓ 준비 완료", color = Accent, fontSize = 11.sp, fontWeight = FontWeight.Bold)
1515         } else {
1516             Text("GPS 신호를 기다리는 중입니다", color = Muted, fontSize = 9.sp, textAlign = TextAlign.Center)
1517         }
1518
1519         Button(
1520             onClick = onConfirm,
1521             enabled = state.gpsReady,
1522             modifier = Modifier.fillMaxWidth().height(36.dp),
1523             colors = ButtonDefaults.buttonColors(
1524                 containerColor = Accent,
1525                 contentColor = Color.Black,
1526                 disabledContainerColor = SurfaceColor,
1527                 disabledContentColor = Muted,
1528             ),
1529         ) {
1530             Icon(Icons.AutoMirrored.Filled.DirectionsRun, null, modifier = Modifier.size(16.dp))
1531             Spacer(Modifier.width(4.dp))
1532             Text("시작하기", fontWeight = FontWeight.Bold, fontSize = 12.sp)
1533         }
1534         SecondaryAction(
1535             label = "취소",
1536             icon = Icons.Filled.Close,
1537             onClick = onCancel,
1538         )
1539     }
1540 }
1541
1542 @Composable
1543 private fun CountdownScreen(

```

```

1544     onFinish: () -> Unit,
1545     onCancel: () -> Unit,
1546 ) {
1547     var count by remember { mutableIntStateOf(3) }
1548     val arcProgress = remember { Animatable(0f) }
1549
1550     LaunchedEffect(count) {
1551         arcProgress.snapTo(0f)
1552         if (count > 0) arcProgress.animateTo(1f, tween(900, easing = LinearEasing))
1553     }
1554
1555     LaunchedEffect(Unit) {
1556         delay(1_000); count = 2
1557         delay(1_000); count = 1
1558         delay(1_000); count = 0
1559         delay(400); onFinish()
1560     }
1561
1562     BackHandler(onBack = onCancel)
1563
1564     Box(
1565         modifier = Modifier
1566             .fillMaxSize()
1567             .background(Background)
1568             .pointerInput(Unit) { detectTapGestures { onFinish() } },
1569         contentAlignment = Alignment.Center,
1570     ) {
1571         Box(contentAlignment = Alignment.Center, modifier = Modifier.size(160.dp)) {
1572             Canvas(modifier = Modifier.fillMaxSize()) {
1573                 val stroke = Stroke(width = 6.dp.toPx(), cap = StrokeCap.Round)
1574                 drawArc(Accent.copy(alpha = 0.15f), -90f, 360f, false, style = stroke)
1575                 drawArc(Accent, -90f, 360f * arcProgress.value, false, style = stroke)
1576             }
1577             AnimatedContent(
1578                 targetState = count,
1579                 transitionSpec = {
1580                     (scaleIn(initialScale = 1.4f, animationSpec = tween(220)) +
1581                         fadeIn(tween(160))) togetherWith
1582                     (scaleOut(targetScale = 0.7f, animationSpec = tween(180)) +
1583                         fadeOut(tween(140)))
1584                 },
1585                 label = "countdown",
1586             ) { c ->
1587                 Text(
1588                     text = if (c == 0) "GO!" else c.toString(),
1589                     fontSize = if (c == 0) 42.sp else 72.sp,
1590                     fontWeight = FontWeight.Black,
1591                     color = if (c == 0) Accent else Color.White,
1592                 )
1593             }
1594         }
1595         Text(
1596             "화면을 탭하면 바로 시작",
1597             modifier = Modifier.align(Alignment.BottomCenter).padding(bottom = 24.dp),
1598             color = Muted,
1599             fontSize = 10.sp,
1600         )
1601     }
1602 }
1603
1604 @Composable
1605 private fun ConfirmDialog(
1606     message: String,
1607     confirmLabel: String,
1608     confirmColor: Color,
1609     onConfirm: () -> Unit,
1610     onDismiss: () -> Unit,
1611 ) {
1612     Box(
1613         modifier = Modifier
1614             .fillMaxSize()
1615             .background(Background),
1616         contentAlignment = Alignment.Center,
1617     ) {
1618         Column(
1619             horizontalAlignment = Alignment.CenterHorizontally,
1620             verticalArrangement = Arrangement.spacedBy(12.dp),
1621             modifier = Modifier.padding(horizontal = 20.dp),
1622         ) {
1623             Text(message, fontWeight = FontWeight.Bold, fontSize = 14.sp, textAlign = TextAlign.Center)
1624             Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
1625                 Button(
1626                     onClick = onDismiss,
1627                     modifier = Modifier.weight(1f).height(40.dp),
1628                     colors = ButtonDefaults.buttonColors(containerColor = SurfaceColor, contentColor = Color.White),
1629                 ) { Text("아니요", fontSize = 11.sp) }

```

```

1630         Button(
1631             onClick = onConfirm,
1632             modifier = Modifier.weight(1f).height(40.dp),
1633             colors = ButtonDefaults.buttonColors(containerColor = confirmColor, contentColor = Color.Black),
1634             ) { Text(confirmLabel, fontSize = 11.sp, fontWeight = FontWeight.Bold) }
1635     }
1636 }
1637 }
1638 }
1639
1640 @Composable
1641 private fun AmbientTrackingScreen(state: WatchRunState) {
1642     Box(
1643         modifier = Modifier
1644             .fillMaxSize()
1645             .background(Color.Black),
1646         contentAlignment = Alignment.Center,
1647     ) {
1648         Column(
1649             horizontalAlignment = Alignment.CenterHorizontally,
1650             verticalArrangement = Arrangement.spacedBy(6.dp),
1651         ) {
1652             Text(
1653                 text = formatDuration(state.elapsedSeconds),
1654                 fontSize = 38.sp,
1655                 fontWeight = FontWeight.Bold,
1656                 color = Color.White,
1657             )
1658             Text(
1659                 text = "%.2f km".format(state.distanceMeters / 1000.0),
1660                 fontSize = 20.sp,
1661                 color = Color.White.copy(alpha = 0.7f),
1662             )
1663             Text(
1664                 text = formatPace(state.paceMinPerKm) + "/km",
1665                 fontSize = 14.sp,
1666                 color = Color.White.copy(alpha = 0.5f),
1667             )
1668         }
1669         if (state.isPaused) {
1670             Text(
1671                 text = "일시정지",
1672                 modifier = Modifier.align(Alignment.BottomCenter).padding(bottom = 16.dp),
1673                 fontSize = 11.sp,
1674                 color = Color.White.copy(alpha = 0.4f),
1675             )
1676         }
1677     }
1678 }

```

```

1  package com.runway.wear.voice
2
3  import android.content.Context
4  import android.speech.tts.TextToSpeech
5  import android.speech.tts.Voice
6  import java.util.Locale
7  import java.util.concurrent.ConcurrentLinkedQueue
8
9  class RunningVoiceGuide(context: Context) : TextToSpeech.OnInitListener {
10     private val pending = ConcurrentLinkedQueue<String>()
11     private val textToSpeech = TextToSpeech(
12         context.applicationContext,
13         this,
14         GOOGLE_TTS_PACKAGE,
15     )
16     @Volatile private var ready = false
17
18     override fun onInit(status: Int) {
19         if (status != TextToSpeech.SUCCESS) return
20         textToSpeech.language = Locale.KOREA
21         textToSpeech.setSpeechRate(1.08f)
22         textToSpeech.setPitch(1.08f)
23         selectKoreanFemaleVoice(textToSpeech.voices)?.let { textToSpeech.voice = it }
24         ready = true
25         while (true) {
26             val message = pending.poll() ?: break
27             speak(message)
28         }
29     }
30
31     fun start() = speak("러닝을 시작합니다. 오늘도 힘차게 달려볼까요?")
32
33     fun finish() = speak("러닝을 종료합니다. 수고하셨습니다.")
34
35     fun cancel() = speak("러닝을 취소합니다.")
36
37     fun autoPaused() = speak("움직임이 멈춰 자동으로 일시정지합니다.")
38
39     fun autoResumed() = speak("움직임이 감지되어 러닝을 다시 시작합니다.")
40
41     fun gpsWeak() = speak("GPS 신호가 약합니다. 시야가 트인 곳으로 이동해 주세요.")
42
43     fun gpsRecovered() = speak("GPS 신호가 안정되었습니다.")
44
45     fun offCourse() = speak("코스를 이탈했습니다.")
46
47     fun backOnCourse() = speak("코스로 복귀했습니다.")
48
49     fun nearCourseFinish() = speak("거의 다 왔습니다.")
50
51     fun goalCompleted(continueRecording: Boolean) = speak(
52         if (continueRecording) {
53             "목표를 달성했습니다. 기록을 계속합니다."
54         } else {
55             "목표를 달성했습니다. 러닝을 일시정지합니다."
56         },
57     )
58
59     fun intervalChanged(isWork: Boolean, step: Int) = speak(
60         if (isWork) "${step}세트 운동을 시작합니다." else "회복 구간을 시작합니다.",
61     )
62
63     fun kilometer(
64         kilometers: Int,
65         elapsedSeconds: Long,
66         heartRateBpm: Int?,
67     ) = speak(RunningVoiceText.kilometer(kilometers, elapsedSeconds, heartRateBpm))
68
69     fun shutdown() {
70         pending.clear()
71         textToSpeech.stop()
72         textToSpeech.shutdown()
73     }
74
75     fun stopGuidance() {
76         pending.clear()
77         textToSpeech.stop()
78     }
79
80     private fun speak(message: String) {
81         if (!ready) {
82             pending.offer(message)
83             return

```



```

84     }
85     textToSpeech.speak(
86         message,
87         TextToSpeech.QUEUE_FLUSH,
88         null,
89         "pathfinder-watch- $\{System.nanoTime()\}$ ",
90     )
91 }
92
93 private fun selectKoreanFemaleVoice(voices: Set<Voice>?): Voice? {
94     val korean = voices.orEmpty().filter { it.locale.language == Locale.KOREAN.language }
95     val offline = korean.filter { !it.isNetworkConnectionRequired }
96     // Google TTS: "ko-kr-x-kof-local", Samsung TTS: "ko-KR-SMTf00" 등 f 패턴 우선
97     return offline.firstOrNull { isFemale(it.name) }
98         ?: offline.firstOrNull { !isMale(it.name) }
99         ?: offline.firstOrNull()
100        ?: korean.firstOrNull { isFemale(it.name) }
101        ?: korean.firstOrNull()
102    }
103
104    private fun isFemale(name: String) =
105        name.contains("female", ignoreCase = true) ||
106        name.contains("SMTf", ignoreCase = false) ||
107        FEMALE_PATTERN.containsMatchIn(name)
108
109    private fun isMale(name: String) =
110        name.contains("male", ignoreCase = true) ||
111        name.contains("SMTm", ignoreCase = false) ||
112        MALE_PATTERN.containsMatchIn(name)
113
114    private companion object {
115        const val GOOGLE_TTS_PACKAGE = "com.google.android.tts"
116        val FEMALE_PATTERN = Regex("[^a-zA-Z]f\\d")
117        val MALE_PATTERN = Regex("[^a-zA-Z]m\\d")
118    }
119 }
120
121 internal object RunningVoiceText {
122     fun kilometer(
123         kilometers: Int,
124         elapsedSeconds: Long,
125         heartRateBpm: Int?,
126     ): String {
127         val averagePace = if (kilometers > 0) elapsedSeconds / kilometers else 0
128         return buildString {
129             append("${kilometers}킬로미터 완료. ")
130             append("시간 ${duration(elapsedSeconds)}. ")
131             append("평균 페이스 ${pace(averagePace)}. ")
132             if (heartRateBpm != null && heartRateBpm > 0) {
133                 append("현재 심박수 ${heartRateBpm}.")
134             } else {
135                 append("심박수는 측정 중입니다.")
136             }
137         }
138     }
139
140     private fun duration(totalSeconds: Long): String {
141         val hours = totalSeconds / 3600
142         val minutes = totalSeconds % 3600 / 60
143         val seconds = totalSeconds % 60
144         return buildList {
145             if (hours > 0) add("${hours}시간")
146             if (minutes > 0) add("${minutes}분")
147             if (seconds > 0 || isEmpty()) add("${seconds}초")
148         }.joinToString(" ")
149     }
150
151     private fun pace(secondsPerKilometer: Long): String {
152         return "${secondsPerKilometer / 60}분 ${secondsPerKilometer % 60}초"
153     }
154 }

```

wear-os/app/src/test/java/com/runway/wear/model/GoalProgressTest.kt

Kotlin | 32줄

```
1 package com.runway.wear.model
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Test
5
6 class GoalProgressTest {
7     @Test
8     fun `time goal progress is clamped`() {
9         val goal = RunGoal.Time(10, GoalCompletionAction.PAUSE)
10        assertEquals(50, GoalProgress.percent(WatchRunState(goal = goal, elapsedSeconds = 300)))
11        assertEquals(100, GoalProgress.percent(WatchRunState(goal = goal, elapsedSeconds = 900)))
12    }
13
14    @Test
15    fun `distance goal reports remaining kilometers`() {
16        assertEquals(
17            "2.5km 남음",
18            GoalProgress.remaining(
19                WatchRunState(
20                    goal = RunGoal.Distance(5_000, GoalCompletionAction.CONTINUE),
21                    distanceMeters = 2_500.0,
22                ),
23            ),
24        )
25    }
26
27    @Test
28    fun `pace formatter handles missing data`() {
29        assertEquals("--'--\\\"", formatPace(0f))
30        assertEquals("5'30\\\"", formatPace(5.5f))
31    }
32 }
```

wear-os/app/src/test/java/com/runway/wear/voice/RunningVoiceTextTest.kt

Kotlin | 14줄

```
1 package com.runway.wear.voice
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Test
5
6 class RunningVoiceTextTest {
7     @Test
8     fun `formats kilometer guide with heart rate`() {
9         assertEquals(
10            "2킬로미터 완료. 시간 12분 20초. 평균 페이스 6분 10초. 현재 심박수 152.",
11            RunningVoiceText.kilometer(2, 740, 152),
12        )
13    }
14 }
```

wear-os/build.gradle.kts

Kotlin (Gradle) | 5줄

```
1 plugins {
2     id("com.android.application") version "8.13.2" apply false
3     id("org.jetbrains.kotlin.android") version "2.0.21" apply false
4     id("org.jetbrains.kotlin.plugin.compose") version "2.0.21" apply false
5 }
```

```
1  pluginManagement {
2      repositories {
3          google {
4              content {
5                  includeGroupByRegex("com\\.\\.android.*")
6                  includeGroupByRegex("com\\.\\.google.*")
7                  includeGroupByRegex("androidx.*")
8              }
9          }
10         mavenCentral()
11         gradlePluginPortal()
12     }
13 }
14
15 dependencyResolutionManagement {
16     repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
17     repositories {
18         google()
19         mavenCentral()
20     }
21 }
22
23 rootProject.name = "RunWayWear"
24 include(":app")
```